PHP — Funções essenciais

Manual oficial do PHP (php.net) — Português do Brasil

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

Strings

Introdução

Todas estas funções manipulam strings de várias formas. Algumas seções mais especializadas podem ser encontradas nas seções de expressões regulares e manipulação de URLs.

Para informações sobre como as strings funcionam, especialmente sobre o uso de aspas simples, aspas duplas e sequências de escape veja o verbete sobre Strings na seção Tipos do manual.

Instalação/Configuração

Índice

Instalação

Nenhuma instalação é necessária para usar estas funções; elas fazem parte do núcleo do PHP.

A partir do PHP 8.1.0, crypt() pode ser compilada contra uma bilbioteca de criptografia de sistema especificando-se a opção de configuração **--with-external-libcrypt**.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

CRYPT_SALT_LENGTH (int)

CRYPT_STD_DES int

Indica se hashes baseados em DES padrão são suportados em crypt(). Sempre 1.

CRYPT_EXT_DES int

Indica se hashes baseados em DES estendido são suportados em crypt(). Sempre 1.

CRYPT_MD5 int

Indica se hashes MD5 são suportados em crypt(). Sempre 1.

CRYPT_BLOWFISH int

Indica se hashes Blowfish são suportados em crypt(). Sempre 1.

CRYPT_SHA256 (int)

Indica se hashes SHA-256 hashes são suportados em crypt(). Sempre 1.

CRYPT_SHA512 (int)

Indica se hashes SHA-512 hashes são suportados em crypt(). Sempre 1.

HTML_SPECIALCHARS (int)

HTML_ENTITIES (int)

ENT_COMPAT (int)

ENT_QUOTES (int)

ENT_NOQUOTES (int)

ENT_IGNORE (int)

ENT_SUBSTITUTE (int)

`ENT_DISALLOWED` (int)

`ENT_HTML401` (int)

`ENT_XML1` (int)

`ENT_XHTML` (int)

`ENT_HTML5` (int)

`CHAR_MAX` (int)

`LC_CTYPE` (int)

Classificação de caracteres e conversões afetadas por configuração de localidade.

`LC_NUMERIC` (int)

Separador de decimais afetado por configuração de localidade.

`LC_TIME` (int)

Formatação de data e horário afetada por configuração de localidade.

`LC_COLLATE` (int)

Comparação de strings afetada por configuração de localidade.

`LC_MONETARY` (int)

Formatação de moeda afetada por configuração de localidade.

`LC_ALL` (int)

Afeta todas as funções que qualquer das outras constantes `LC_*` afeta.

`LC_MESSAGES` (int)

Respostas do sistema afetadas por configuração de localidade. Disponível se o PHP foi compilado com `libintl`.

`STR_PAD_LEFT` (int)

`STR_PAD_RIGHT` (int)

`STR_PAD_BOTH` (int)

Constantes da Categoria `LC_TIME` da função nl_langinfo()

Constantes	Descrição
`ABDAY_1` (int)	Nome abreviado do primeiro dia da semana.
`ABDAY_2` (int)	Nome abreviado do segundo dia da semana.
`ABDAY_3` (int)	Nome abreviado do terceiro dia da semana.
`ABDAY_4` (int)	Nome abreviado do quarto dia da semana.
`ABDAY_5` (int)	Nome abreviado do quinto dia da semana.
`ABDAY_6` (int)	Nome abreviado do sexto dia da semana.
`ABDAY_7` (int)	Nome abreviado do sétimo dia da semana.

DAY_1 (int)	Nome do primeiro dia da semana.
DAY_2 (int)	Nome do segundo dia da semana.
DAY_3 (int)	Nome do terceiro dia da semana.
DAY_4 (int)	Nome do quarto dia da semana.
DAY_5 (int)	Nome do quinto dia da semana.
DAY_6 (int)	Nome do sexto dia da semana.
DAY_7 (int)	Nome do sétimo dia da semana.
ABMON_1 (int)	Nome abreviado do primeiro mês do ano.
ABMON_2 (int)	Nome abreviado do segundo mês do ano.
ABMON_3 (int)	Nome abreviado do terceiro mês do ano.
ABMON_4 (int)	Nome abreviado do quarto mês do ano.
ABMON_5 (int)	Nome abreviado do quinto mês do ano.
ABMON_6 (int)	Nome abreviado do sexto mês do ano.
ABMON_7 (int)	Nome abreviado do sétimo mês do ano.
ABMON_8 (int)	Nome abreviado do oitavo mês do ano.
ABMON_9 (int)	Nome abreviado do nono mês do ano.
ABMON_10 (int)	Nome abreviado do décimo mês do ano.
ABMON_11 (int)	Nome abreviado do décimo primeiro mês do ano.
ABMON_12 (int)	Nome abreviado do décimo segundo mês do ano.
MON_1 (int)	Nome do primeiro mês do ano.
MON_2 (int)	Nome do segundo mês do ano.
MON_3 (int)	Nome do terceiro mês do ano.

`MON_4` (int)	Nome do quarto mês do ano.
`MON_5` (int)	Nome do quinto mês do ano.
`MON_6` (int)	Nome do sexto mês do ano.
`MON_7` (int)	Nome do sétimo mês do ano.
`MON_8` (int)	Nome do oitavo mês do ano.
`MON_9` (int)	Nome do nono mês do ano.
`MON_10` (int)	Nome do décimo mês do ano.
`MON_11` (int)	Nome do décimo primeiro mês do ano.
`MON_12` (int)	Nome do décimo segundo mês do ano.
`AM_STR` (int)	String para horários antes do meio-dia.
`PM_STR` (int)	String para horários após meio-dia.
`D_T_FMT` (int)	String que pode ser usada como formatação para strftime() para representar horário e data.
`D_FMT` (int)	String que pode ser usada como formatação para strftime() para representar data.
`T_FMT` (int)	String que pode ser usada como formatação para strftime() para representar horário.
`T_FMT_AMPM` (int)	String que pode ser usada como formatação para strftime() para representar horário no formato de 12-horas com informação de antes ou após meio-dia.
`ERA` (int)	Era alternativa.
`ERA_YEAR` (int)	Ano no formato da era alternativa.
`ERA_D_T_FMT` (int)	Data e horário no formato da era alternativa (string pode ser usada em strftime()).
`ERA_D_FMT` (int)	Data em formato da era alternativa (string pode ser usada em strftime()).
`ERA_T_FMT` (int)	Horário no formato da era alternativa (string pode ser usada em strftime()).
`ALT_DIGITS` (int)	Símbolos alternativos para dígitos.

nl_langinfo() `LC_MONETARY` Category Constants

Constantes	Descrição

`INT_CURR_SYMBOL` (int)	Símbolo da moeda internacional.
`CURRENCY_SYMBOL` (int)	Símbolo da moeda local.
`CRNCYSTR` (int)	O mesmo que `CURRENCY_SYMBOL`.
`MON_DECIMAL_POINT` (int)	Caractere separador de decimais.
`MON_THOUSANDS_SEP` (int)	Separador de milhares (grupos de três dígitos).
`MON_GROUPING` (int)	Similar ao elemento `"grouping"`.
`POSITIVE_SIGN` (int)	Sinal para valores positivos.
`NEGATIVE_SIGN` (int)	Sinal para valores negativos.
`INT_FRAC_DIGITS` (int)	Dígitos fracionários internacionais.
`FRAC_DIGITS` (int)	Dígitos fracionários locais.
`P_CS_PRECEDES` (int)	Retorna 1 se `CURRENCY_SYMBOL` precede um valor positivo.
`P_SEP_BY_SPACE` (int)	Retorna 1 se um espaço separa `CURRENCY_SYMBOL` de um valor positivo.
`N_CS_PRECEDES` (int)	Retorna 1 se `CURRENCY_SYMBOL` precede um valor negativo.
`N_SEP_BY_SPACE` (int)	Retorna 1 se um espaço separa `CURRENCY_SYMBOL` de um valor negativo.
`P_SIGN_POSN` (int)	• Retorna 0 se parênteses envolvem a quandidade e o `CURRENCY_SYMBOL`. • Retorna 1 se a string de sinal precede a quantidade e o `CURRENCY_SYMBOL`. • Retorna 2 se a string de sinal segue a quantidade e o `CURRENCY_SYMBOL`. • Retorna 3 se a string de sinal precede imediatamente o `CURRENCY_SYMBOL`. • Retorna 4 se a string de sinal segue imediatamente o `CURRENCY_SYMBOL`.
`N_SIGN_POSN` (int)	

Constantes da Categoria `LC_NUMERIC` da função nl_langinfo()

Constantes	Descrição
`DECIMAL_POINT` (int)	Caractere separador de decimais.
`RADIXCHAR` (int)	O mesmo que `DECIMAL_POINT`.
`THOUSANDS_SEP` (int)	Caractere separador de milhares (grupos de três dígitos).

THOUSEP (int)	O mesmo que THOUSANDS_SEP.
GROUPING (int)	

Constantes da Categoria LC_MESSAGES da função nl_langinfo()

Constantes	Descrição
YESEXPR (int)	String de expressão regular para corresponder à entrada "sim".
NOEXPR (int)	String de expressão regular para corresponder à entrada "não".
YESSTR (int)	String de saída para "sim".
NOSTR (int)	String de saída para "não".

Constantes da Categoria LC_CTYPE da função nl_langinfo()

Constantes	Descrição
CODESET (int)	Retorna uma string com o nome da codificação de caracteres.

Funções de String

Veja também

Para funções de manipulação de string mais poderosas, veja as funções de expressões regulares estilo Perl. Para trabalhar com strings com codificação multibyte, veja as funções Multibyte String.

Índice

addcslashes

addcslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

addcslashes — Escapa string com barras invertidas no estilo C

Descrição

function **addcslashes**(string `$string`, string `$characters`): string

Retorna uma string com barras invertidas antes dos caracteres que estão listados no parâmetro `characters`.

Parâmetros

`string`

A string a ser escapada.

`characters`

Uma lista de caracteres a serem escapados. Se `characters` contiver caracteres `\n`, `\r` etc., eles serão convertidos no estilo da linguagem C, enquanto que outros caracteres não alfanuméricos com códigos ASCII menores que 32 e maiores que 126 serão convertidos para a representação octal.

Ao definir uma sequência de caracteres no argumento `characters` certifique-se que são conhecidos todos os caracteres que aparecem entre aqueles definidos como o início e o final do intervalo.

Exemplo #1 addcslashes() com intervalos

```php
<?php
echo addcslashes('foo[ ]', 'A..z');
// saída:  \f\o\o\[ \]
// Todas as letras maiúsculas e minúsculas serão escapadas
// ... mas também os caracteres [\]^_`
?>
```

Adicionamente, se o primeiro caractere em uma sequência possuir um código ASCII maior que o segundo caractere na sequência, na verdade nenhum intervalo será construído. Apenas os caracteres do início, do final e o caractere "." serão escapados. Use a função ord() para encontrar o código ASCII de um caractere.

Exemplo #2 addcslashes() com caracteres na ordem incorreta

```php
<?php
echo addcslashes("zoo['.']", 'z..A');
// saída:  \zoo['\.']
?>
```

Cuidado ao escapar os caracteres 0, a, b, f, n, r, t e v. Eles serão convertidos para \0, \a, \b, \f, \n, \r, \t e \v, que são sequências de escape predefinidas em C. Muitas dessas sequências são também definidas em outras linguagens derivadas do C, incluindo o PHP, o que significa que pode não ser obtido o resultado desejado se a saída de **addcslashes()** for usada para gerar código nessas linguagens com estes caracteres definidos no parâmetro `characters`.

Valor Retornado

Retorna a string escapada.

Examples

Parâmetro `characters` definido para algo como "\0..\37", que escapa todos os caracteres com código entre 0 e 31 (octal \37).

Exemplo #3 Exemplo de addcslashes()

```php
<?php
$not_escaped = "PHP isThirty\nYears Old!\tYay to the Elephant!\n";
$escaped = addcslashes($not_escaped, "\0..\37!@\177..\377");
echo $escaped;
?>
```

Veja também

- stripcslashes() - Remove o escape de strings escapadas com addcslashes
- stripslashes() - Desfaz os escapes de uma string escapada
- addslashes() - Adiciona barras invertidas a uma string
- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- quotemeta() - Escapa meta caracteres

addslashes

addslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

addslashes — Adiciona barras invertidas a uma string

Descrição

function **addslashes**(string `$string`): string

Retorna uma string com barras invertidas adicionadas antes de caracteres que precisam ser escapados. Estes caracteres são:

- aspas simples (')
- aspas duplas (")
- barra invertida (\)
- NUL (o byte NUL)

Um caso de uso de **addslashes()** é o escape dos caracteres mencionados em uma string que deve ser avaliada pelo PHP:

Exemplo #1 Escapando caracteres

```php
<?php
$str = "O'Reilly?";
eval("echo '" . addslashes($str) . "';");
?>
```

A função **addslashes()** às vezes é usada incorretamente para tentar prevenir Injeção de SQL. Em vez disso, funções de escape específicas do banco de dados e/ou instruções preparadas devem ser usadas.

Parâmetros

`string`

A string a ser escapada.

Valor Retornado

Retorna a string com escapes.

Examples

Exemplo #2 Exemplo com addslashes()

```php
<?php
$str = "Seu nome é O'reilly?";

// Mostra: Seu nome é O\'reilly?
echo addslashes($str);
?>
```

Veja também

- stripcslashes() - Remove o escape de strings escapadas com addcslashes
- stripslashes() - Desfaz os escapes de uma string escapada
- addcslashes() - Escapa string com barras invertidas no estilo C
- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- quotemeta() - Escapa meta caracteres
- get_magic_quotes_gpc() - Obtém a configuração atual de magic_quotes_gpc

bin2hex

bin2hex

(PHP 4, PHP 5, PHP 7, PHP 8)

bin2hex — Converte um dado binário em representação hexadecimal

Descrição

function **bin2hex**(string `$string`): string

Retorna uma string ASCII uma representação hexadecimal de `string`. A conversão é feita byte-wise com o high-nibble primeiro.

Parâmetros

`string`

Uma string.

Valor Retornado

Retorna a representação hexadecimal de uma dada string.

Examples

Exemplo #1 Exemplo de bin2hex()

```php
<?php

$hex = bin2hex('Hello world!');
```

```
var_dump($hex);
var_dump(hex2bin($hex));
?>
```

O exemplo acima produzirá:

```
string(24) "48656c6c6f20776f726c6421"
string(12) "Hello world!"
```

Veja também

- hex2bin() - Decodifica uma string binária codificada em hexadecimal
- pack() - Empacota dados em uma string binária

chop

chop

(PHP 4, PHP 5, PHP 7, PHP 8)

chop — Sinônimo de rtrim()

Descrição

Esta função é um apelido para: rtrim().

Notas

> **Nota** :
>
> **chop()** é diferente da função do Perl `chop()`, que remove o último caractere da string.

chr

chr

(PHP 4, PHP 5, PHP 7, PHP 8)

chr — Gera uma string de um byte a partir de um número

Descrição

function **chr**(int `$codepoint`): string

Retorna uma string de um caractere contendo o caractere especificado pela interpretação do parâmetro `codepoint` como um inteiro sem sinal.

Esta função pode ser usada para criar uma string de um caracteres em uma codificação de 1-byte como ASCII, ISO-8859, ou Windows 1252, passando-se a posição do caractere desejado na tabela de mapeamento da codificação. Entretanto, observe que esta função não tem ciência de codificação de strings e, em particular, não pode receber um valor de ponto de código Unicode para gerar uma string em uma codificação multi-byte como UTF-8 ou UTF-16.

Esta função é complementar à função ord().

Parâmetros

`codepoint`

Um inteiro entre 0 e 255.

Valores fora do intervalo válido (0..255) sofrerão operação binária AND com o valor 255, o que é equivalente ao algoritmo a seguir:

```
while ($bytevalue < 0) {
    $bytevalue += 256;
}
$bytevalue %= 256;
```

Valor Retornado

Uma string de um caractere contendo o byte especificado.

Registro de Alterações

Versão	Descrição
8.5.0	Passar inteiros fora do intervalo `[0, 255]` foi descontinuado.
7.4.0	A função não mais aceita silenciosamente códigos não suportados no parâmetro `codepoint`, convertendo-os para `0`.

Examples

Exemplo #1 Exemplo de chr()

```php
<?php
// Assume que a string será usada como codificação ASCII ou compatível

$str = "A string termina com ESC: ";
$str .= chr(27); /* adiciona um caractere ESC ao final de $str */
echo $str, PHP_EOL;

/* Frequentemente isto é mais útil: */
$str = sprintf("A string termina com ESC: %c", 27);
echo $str, PHP_EOL;
?>
```

Exemplo #2 Comportamento com parâmetro fora dos limites

```php
<?php
echo chr(-159), chr(833), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
aA
```

Exemplo #3 Construindo uma string UTF-8 a partir de bytes individuais

```php
<?php
$str = chr(240) . chr(159) . chr(144) . chr(152);
echo $str, PHP_EOL;
?>
```

O exemplo acima produzirá:

Veja também

- sprintf() - Retona uma string formatada com string de formatação %c
- ord() - Converte o primeiro byte de uma string para um valor entre 0 e 255
- Uma » tabela ASCII

- mb_chr() - Retorna um caractere pelo valor do ponto de código Unicode
- IntlChar::chr() - Retorna um caractere Unicode por valor de ponto de código

chunk_split

chunk_split

(PHP 4, PHP 5, PHP 7, PHP 8)

chunk_split — Divide uma string em pedaços menores

Descrição

function **chunk_split**(string `$string`, int `$length` = 76, string `$separator` = "\r\n"): string

Pode ser usada para dividir uma string em pedaços menores, o que é útil para por exemplo converter a saída de base64_encode() para adequação às semânticas RFC 2045 . Ela insere um separador definido em `separator` a cada número de caracteres definido em`length`.

Parâmetros

`string`

A string a ser dividida.

`length`

O tamanho do pedaço.

`separator`

A sequência de terminação de linha.

Valor Retornado

Retorna a string dividida.

Examples

Exemplo #1 Exemplo de chunk_split()

```php
<?php
$data = 'Esta é uma string bem longa que será quebrada em partes porque a linha será muito comprida depois de ter sido codificada em base64.';

// formata $data usando semântica RFC 2045
$new_string = chunk_split(base64_encode($data));
echo $new_string, PHP_EOL;
?>
```

Veja também

- str_split() - Converte uma string em um array
- explode() - Divide uma string com base em outra string
- wordwrap() - Quebra uma string em um dado número de caracteres
- » RFC 2045

convert_cyr_string

convert_cyr_string

(PHP 4, PHP 5, PHP 7)

convert_cyr_string — Converte de um conjunto de caracteres cirílico para outro

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **convert_cyr_string**(string `$str`, string `$from`, string `$to`): string

Converte de um conjunto de caracteres cirílico para outro.

Parâmetros

`str`

A string a ser convertida.

`from`

A fonte do conjunto de caracteres cirílico, como um único caractere.

`to`

O conjunto de caracteres cirílico alvo, como um único caractere.

Os caracteres suportados são:

- k - koi8-r
- w - windows-1251
- i - iso8859-5
- a - x-cp866
- d - x-cp866
- m - x-mac-cyrillic

Valor Retornado

Retorna a string convertida.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- mb_convert_encoding() - Converte uma string de uma codificação de caracteres para outra
- iconv() - Converte uma string de uma codificação de caracteres para outra

convert_uudecode

convert_uudecode

(PHP 5, PHP 7, PHP 8)

convert_uudecode — Decodifica uma string codificada com uuencode

Descrição

function **convert_uudecode**(string `$string`): string|false

convert_uudecode() decodifica uma string codificada com uuencode.

> **Nota** :
>
> **convert_uudecode()** não aceita a linha `begin` nem a linha `end`, que são parte dos *arquivos* codificados com uuencode.

Parâmetros

`string`

Os dados codificados com uuencode.

Valor Retornado

Retorna os dados decodificados como uma string ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo da convert_uudecode()

```php
<?php
echo convert_uudecode("+22!L;W9E(%!(4\"$`\n`");
?>
```

O exemplo acima produzirá:

```
I love PHP!
```

Veja também

- convert_uuencode() - Codifica uma string com uuencode

convert_uuencode

convert_uuencode

(PHP 5, PHP 7, PHP 8)

convert_uuencode — Codifica uma string com uuencode

Descrição

function **convert_uuencode**(string `$string`): string

convert_uuencode() codifica uma string usando o algoritmo uuencode.

Uuencode traduz toda string (incluindo dados binários) em caracteres imprimíveis, fazendo eles seguro para transmissão em rede. Dados uuencoded fica 35% maior que o original.

> **Nota** :
>
> **convert_uuencode()** não produz a linha `begin` nem a linha `end`, que são parte dos *arquivos* codificados com uuencode.

Parâmetros

`string`

A informação a ser codificada.

Valor Retornado

Retorna os dados convertidos com uuencode.

Registro de Alterações

Versão	Descrição
8.0.0	Antes desta versão, tentar converter uma string vazia retornava `false` sem nenhuma razão particular.

Examples

Exemplo #1 Exemplo da convert_uuencode()

```php
<?php
$some_string = "test\ntext text\r\n";

echo convert_uuencode($some_string);
?>
```

O exemplo acima produzirá:

```
0=&5S=`IT97AT('1E>'0-"@``
`
```

Veja também

- convert_uudecode() - Decodifica uma string codificada com uuencode
- base64_encode() - Codifica dados com MIME base64

count_chars

count_chars

(PHP 4, PHP 5, PHP 7, PHP 8)

count_chars — Retorna informação sobre caracteres usados em uma string

Descrição

function **count_chars**(string `$string`, int `$mode` = 0): array|string

Conta o número de ocorrências de cada valor de byte (0..255) em `string` e retorna de diversas formas.

Parâmetros

`string`

A string examinada.

`mode`

Veja em valores de retorno.

Valor Retornado

Dependendo do parâmetro `mode` **count_chars()** retorna um dos valores a seguir:

- 0 - um array com valores de byte como chaves e a frequência de cada byte como valor.
- 1 - o mesmo que 0 mas apenas valores de byte com uma frequência maior que zero são listados.
- 2 - o mesmo que 0 mas apenas valores de byte com uma frequência igual a zero são listados.
- 3 - uma string contendo todos os caracteres únicos.
- 4 - uma string contendo todos os caracteres não utilizados.

Registro de Alterações

Versão	Descrição
8.0.0	Antes desta versão, a função retornava `false` em caso de falha.

Examples

Exemplo #1 Exemplo de count_chars()

```
<?php
$data = "Dois Ps e um H.";

foreach (count_chars($data, 1) as $i => $val) {
   echo "Há $val instância(s) de \"" , chr($i) , "\" na string.\n";
}
?>
```

O exemplo acima produzirá:

```
Há 4 instância(s) de " " na string.
Há 1 instância(s) de "." na string.
Há 1 instância(s) de "D" na string.
Há 1 instância(s) de "H" na string.
Há 1 instância(s) de "P" na string.
Há 1 instância(s) de "e" na string.
Há 1 instância(s) de "i" na string.
Há 1 instância(s) de "m" na string.
Há 1 instância(s) de "o" na string.
Há 2 instância(s) de "s" na string.
Há 1 instância(s) de "u" na string.
```

Veja também

- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- substr_count() - Conta o número de ocorrências de uma substring

crc32

crc32

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

crc32 — Calcula polinômio crc32 de uma string

Descrição

function **crc32**(string `$string`): int

Gera a soma de verificação de redundância cíclica de 32-bits de comprimento da `string`. É usado geralmente para validar a integridade de um dado sendo transferido.

> **Aviso**
>
> Pelo fato do tipo inteiro do PHP ser com sinal, muitas somas de verificação crc32 resultarão em inteiros negativos em plataformas de 32 bits. Em instalações de 64 bits todos os resultados de **crc32()** serão inteiros positivos.
>
> Por isso, é necessário usar o formatador "%u" da função sprintf() ou printf() para obter a representação em string da soma de verificação sem sinal da função **crc32()** em formato decimal.
>
> Para uma representação hexadecimal da soma de verificação, pode-se usar tanto o formatador "%x" de sprintf() ou printf() quanto as funções de conversão dechex(), ambas destas soluções também tomam o cuidado de converter o resultado de **crc32()** para um inteiro sem sinal.
>
> Foi considerado que instalações de 64 bits também retornassem inteiros negativos para valores mais altos de resultados, porém isto iria quebrar a conversão para hexadecimal, já que os negativos teriam um deslocamento extra de 0xFFFFFFFF######## neste caso.

> Como a representação hexadecimal parece ser o uso de caso mais comum, decidiu-se não quebrar esta conversão mesmo que ela quebre comparações diretas em decimal em aproximadamente 50% dos casos ao mover de 32 para 64 bits.
>
> Em retrospecto, fazer com que a função retornasse um inteiro talvez não tenha sido a melhor ideia, e retornar uma representação hexadecimal em string diretamente (como, por exemplo, a função md5() faz) poderia ter sido um plano melhor para se começar.
>
> Para uma solução mais portável, pode-se considerar a função genérica hash(). `hash("crc32b", $str)` retornará a mesma string que `str_pad(dechex(crc32($str)), 8, '0', STR_PAD_LEFT)`.

Parâmetros

`string`

Os dados.

Valor Retornado

Retorna a soma de verificação crc32 da `string` como um inteiro.

Examples

Exemplo #1 Mostrando uma soma de verificação crc32

Este exemplo mostra como exibir uma soma de verificação convertida com a função printf():

```php
<?php
$checksum = crc32("The quick brown fox jumped over the lazy dog.");
printf("%u\n", $checksum);
?>
```

Veja também

- hash() - Gera um valor hash (resumo da mensagem)
- md5() - Calcula o hash md5 de uma string
- sha1() - Calcula o hash SHA1 de uma string

crypt

crypt

(PHP 4, PHP 5, PHP 7, PHP 8)

crypt — Hash unidirecional de string

> **Aviso**
>
> Esta função (ainda) não é compatível com dados binários!

Descrição

function **crypt**(#[\SensitiveParameter] string `$string`, string `$salt`): string

crypt() retorna uma string com hash usando o algorimo padrão do Unix baseado em DES ou algortimos alternativos. password_verify() é compatível com **crypt()**. Portanto, hashes de senhas criados por **crypt()** podem ser usados com password_verify().

Antes do PHP 8.0.0, o parâmetro `salt` era opcional. Entretanto, **crypt()** criava um hash fraco sem `salt` e emitia um erro do tipo **`E_NOTICE`**. Certifique-se de especificar um sal forte o suficiente para uma melhor segurança de dados.

password_hash() usa um hash forte, gera um sal forte e aplica arredondamentos apropriados automaticamente. password_hash() é um empacotador simples de **crypt()** e compatível com hashes de senhas existentes. O uso de password_hash() é encorajado.

O tipo de hash é disparado pelo argumento salt (sal). Se nenhum sal for fornecido, o PHP irá gerar automaticamente um sal padrão de 2 caracteres (DES) ou de 12 caracteres (MD5), dependendo da disponibilidade de crypt() MD5. O PHP define uma constante de nome **`CRYPT_SALT_LENGTH`** que indica o comprimento máximo de sal permitido pelos hashes disponíveis.

A função **crypt()** padrão baseada em DES retorna o sal como os 2 primeiros caracteres da saída. Ela também somente usa os primeiros 8 caracteres da `string`, portanto strings mais longas que começam com os mesmos 8 caracteres irão gerar o mesmo resultado (quando o

mesmo sal é usado).

Os seguintes tipos de hash são suportados:

- `CRYPT_STD_DES` - Hash padrão baseado em DES com sal de 2 caracteres do conjunto "./0-9A-Za-z". Usar caracteres inválidos no sal irá causar falha na função crypt().
- `CRYPT_EXT_DES` - Hash estendido baseado em DEX. O "sal" é uma string de 9 caracteres que consiste de um "_" seguido por 4 caracteres de contagem de iteração e 4 caracteres de sal. Cada string de 4 caracteres é codificada em 24 bits, os caracteres menos significativos primeiro. Os valores `0` a `63` são codificados como `./0-9A-Za-z`. Usar caracteres inválidos no sal irá causar falha na função crypt().
- `CRYPT_MD5` - Hash MD5 com sal de 12 caracteres iniciando com 1
- `CRYPT_BLOWFISH` - Hash Blowfish com o sal desta forma: "$2a$", "$2x$" ou "$2y$", um parâmetro de custo de 2 dígitos, "$", e 22 caracteres do conjunto "./0-9A-Za-z". Usar caracteres fora deste conjunto no sal fará com que crypt() retorne uma string de comprimento zero. O parâmetro de custo de 2 dígitos é o logaritmo base 2 da contagem de iteração para o algortimo de hash baseado em Blowfish subjacente e precisa estar no intervalo 04-31. Valores fora deste intervalo causarão falha na função crypt(). Hashes "$2x$" são potencialmente fracas; hashes "$2a$" são compatíveis e mitigam esta fraqueza. Para novas hashes, "$2y$" deve ser usada.
- `CRYPT_SHA256` - Hash SHA-256 com sal de 16 caracteres prefixados com 5. Se a string do sal iniciar com 'rounds=<N>$', o valor numérico de N é usado para indicar quantas rodadas do hash devem ser executadas, bem parecido com o parâmetro de custo do Blowfish. O número padrão de rodadas é 5000, existe um mínimo de 1.000 e um máximo de 999.999.999. Qualquer seleção de N fora deste intervalo será truncado ao limite mais próximo.
- `CRYPT_SHA512` - Hash SHA-512 com sal de 16 caracteres prefixados com 6. Se a string do sal iniciar com 'rounds=<N>$', o valor numérico de N é usado para indicar quantas rodadas do hash devem ser executadas, bem parecido com o parâmetro de custo do Blowfish. O número padrão de rodadas é 5000, existe um mínimo de 1.000 e um máximo de 999.999.999. Qualquer seleção de N fora deste intervalo será truncado ao limite mais próximo.

Parâmetros

`string`

A string cujo hash será calculado.

> **Cuidado**
>
> Usar o algortimo `CRYPT_BLOWFISH` fará com que o parâmetro `string` seja truncado para um comprimento máximo de 72 bytes.

`salt`

Uma string de sal para servir como base para a operação de hash. Se não fornecido, o comportamento é definido pela implementação do algoritmo e pode levar a resultados inesperados.

Valor Retornado

Retorna o hash da string ou uma string mais curta que 13 caracteres que seja garantidamente diferente do sal em caso de falha.

> **Aviso**
>
> Ao validar senhas, uma função de comparação de strings que não seja vulnerável a ataques de temporização deve ser usada para comparar a saída de **crypt()** ao hash conhecido anteriormente. O PHP fornece a função hash_equals() para este propósito.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `salt` não é mais opcional.

Examples

Exemplo #1 Exemplos de crypt()

```
<?php
$user_input = 'rasmuslerdorf';
```

```php
$hashed_password =
'$6$rounds=1000000$NJy4rIPjpOaU$0ACEYGg/aKCY3v8O8AfyiO7CTfZQ8/W231Qfh2tRLmfdvFD6XfHk12u6hMr9cYIA4hnpjLNSTRtUwYr9km9Ij/';

// Valida um hash existente criado por crypt() de forma compatível com software não-PHP.
if (hash_equals($hashed_password, crypt($user_input, $hashed_password))) {
   echo "Senha verificada!";
}
?>
```

Notas

> **Nota** :
>
> Não existe função de descriptografia, já que **crypt()** usa um algoritmo unidirecional.

Veja também

- hash_equals() - Comparação de strings segura para ataque de temporização
- password_hash() - Cria um hash de senha
- A página do manual Unix para a função crypt() para mais informação

echo

echo

(PHP 4, PHP 5, PHP 7, PHP 8)

echo — Envia uma ou mais strings para a saída

Descrição

function **echo**(string `...$expressions`): void

Envia uma ou mais expressões para a saída, sem novas linhas ou espaços adicionais.

`echo` não é uma função mas sim uma construção de linguagem. Seus argumentos são uma lista de expressões logo após a palavra-chave `echo`, separadas por vírgulas e não delimitadas por parênteses. Diferentemente de outras construções de linguagem, `echo` não tem nenhum valor de retorno, portando não pode ser usada no contexto de uma expressão.

`echo` também tem uma sintaxe de atalho em HTML, onde pode-se seguir imediatamente a etiqueta de abertura com um sinal de igual. Esta sintaxe está disponível mesmo que a configuração short_open_tag esteja desabilitada.

```php
I have <?=$foo?> foo.
```

As principais diferenças para a função print são que `echo` aceita múltiplos argumentos e não tem um valor de retorno.

Parâmetros

`expressions`

> Uma ou mais expressões a serem enviadas para a saída, separadas por vírgula. Valores não-string serão convertidas forçadamente para strings, mesmo quando a diretiva `strict_types` estiver habilitada.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplos de `echo`

```php
<?php
echo "echo não requer parênteses.";

// Strings podem ser passadas individualmente como múltiplos argumentos ou
// concatenadas e passadas como um único argumento
echo 'Esta ', 'string ', 'foi ', 'criada ', 'com múltiplos parâmetros.', "\n";
echo 'Esta ' . 'string ' . 'foi ' . 'criada ' . 'com concatenação.' . "\n";
```

```php
// Nenhuma nova linha ou espaço são adicionados; o código abaixo mostra "olámundo" em apenas uma linha
echo "olá";
echo "mundo";

// O mesmo que o exemplo acima
echo "olá", "mundo";

echo "Esta string ocupa
múltiplas linhas. As novas linhas
também estarão na saída.";

echo "Esta string ocupa\nmúltiplas linhas. As novas linhas\ntambém estarão na saída.";

// O argumento pode ser qualquer expressão que produza uma string
$foo = "exemplo";
echo "foo é um $foo"; // foo é um exemplo

$fruits = ["limão", "laranja", "banana"];
echo implode(" e ", $fruits); // limão e laranja e banana

// Expressões não-string não convertidas à força para string, mesmo se declare(strict_types=1) for usada
echo 6 * 7; // 42

// Entretanto os exemplos a seguir irão funcionar:
($some_var) ? print 'true' : print 'false'; // print também é uma contrução de linguagem mas
                                            // é uma expressão válida, retornando 1,
                                            // portanto pode ser usada neste contexto.

echo $some_var ? 'true': 'false'; // primeiro avalia a expressão e depois passa para echo
?>
```

Exemplo #2 `echo` não é uma expressão

```php
<?php
// Como echo não se comporta como uma expressão, o código a seguir é inválido.
($some_var) ? echo 'true' : echo 'false';
?>
```

Notas

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

> **Nota** : **Usando com parênteses**
>
> Delimitar um argumento de `echo` com parênteses não irá causar um erro de sintaxe, e produzirá uma sintaxe que se parece com uma chamade normal de uma função. Entretanto, isto pode gerar confusão, porque os parênteses, na verdade, são parte da expressão que será enviada para a saída, e não parte da sintaxe de `echo` em si.
>
> **Exemplo #3 Usando parênteses**
>
> ```php
> <?php
> echo "olá", PHP_EOL;
> // mostra "olá"
>
> echo("olá"), PHP_EOL;
> // também mostra "olá", porque ("olá") é uma expressão válida
>
> echo(1 + 2) * 3, PHP_EOL;
> // mostra "9"; os parênteses fazem com que 1+2 seja avaliada primeiro, e depois 3*3
> // a instrução echo enxerga toda a expressão como um só argumento
>
> echo "olá", " mundo", PHP_EOL;
> // mostra "olá mundo"
> ```

```php
echo("olá"), (" mundo"), PHP_EOL;
// mostra "olá mundo"; os parênteses são parte de cada expressão
?>
```

Exemplo #4 Expressão inválida

```php
<?php
echo("olá", " mundo"), PHP_EOL;
// Lança um erro de interpretação porque ("olá", " mundo") não é uma expressão válida
?>
```

> **Dica**
>
> Passar múltiplos argumentos para `echo` pode evitar complicações que surgem com a precedência do operador de concatenação no PHP. Por exemplo, o operador de concatenação tem precedência maior que o operador ternário, e antes do PHP 8.0.0 tinha a mesma precedência que adição e subtração:
>
> ```php
> <?php
> // Abaixo, a expressão 'Olá ' . isset($name) é avaliada primeiro,
> // e é sempre verdadeira, portanto o argumento para echo será sempre $name
> echo 'Olá ' . isset($name) ? $name : 'Maria Rodrigues' . '!';
>
> // O comportamento pretendido requer parênteses adicionais
> echo 'Olá ' . (isset($name) ? $name : 'Maria Rodrigues') . '!';
>
> // Nas versões do PHP anteriores à 8.0.0, o código abaixo mostra "2" ao invés de "Soma: 3"
> echo 'Soma: ' . 1 + 2;
>
> // Da mesma forma, adicionar parênteses assegura a ordem pretendida de avaliação
> echo 'Soma: ' . (1 + 2);
> ```
>
> Se múltiplos argumentos forem passados, os parênteses não serão necessário para reforçar precedência, pois cada expressão é independente:
>
> ```php
> <?php
> echo "Olá ", isset($name) ? $name : "Maria Rodrigues", "!";
>
> echo "Soma: ", 1 + 2;
> ```

Veja também

- print - Exibe uma string
- printf() - Envia uma string formatada para a saída
- flush() - Descarrega o buffer na saída do sistema
- Formas de especificar strings literais

explode

explode

(PHP 4, PHP 5, PHP 7, PHP 8)

explode — Divide uma string com base em outra string

Descrição

function **explode**(string `$separator`, string `$string`, int `$limit` = **`PHP_INT_MAX`**): array

Retorna um array de strings, sendo que cada uma é uma sub-string da `string`, formado pela separação dela nos limites formados pela string informada em `separator`.

Parâmetros

`separator`

A string que define o separador.

`string`

A string de entrada.

`limit`

Se `limit` estiver definido e for positivo, o array retornado conterá um máximo de `limit` elementos com o último elemento contendo todo o restante da `string`.

Se `limit` for negativo, todos os componentes exceto os (último - `limit`) são retornados.

Se `limit` for zero, é tratado como 1.

> **Nota** :
>
> Antes do PHP 8.0, implode() aceitava seus dois primeiros parâmetros em qualquer ordem. A função **explode()** nunca suportou isto: deve-se garantir que o parâmetro `separator` vem antes do parâmetro `string`.

Valor Retornado

Retorna um array de strings criado pela separação da `string` nos limites formados pelo parâmetro `separator`.

Se `separator` for uma string vazia (""), **explode()** lança um ValueError. Se `separator` contiver um valor não contido em `string` e um valor negativo for usado em `limit`, um array vazio será retornado, caso contrário um array contendo `string` será retornado. Se valores de `separator` aparecerem no início ou no final da `string`, said values elementos de array vazios serão adicionados na posição inicial ou final do array retornado, respectivamente.

Registro de Alterações

Versão	Descrição
8.0.0	**explode()** agora lança um ValueError quando `separator` é informado com uma string vazia (""). Anteriormente, **explode()** retornava `false` nesse caso.

Examples

Exemplo #1 Exemplos de **explode()**

```php
<?php
// Exemplo 1
$pizza  = "fatia1 fatia2 fatia3 fatia4 fatia5 fatia6";
$fatias = explode(" ", $pizza);
echo $fatias[0], PHP_EOL; // fatia1
echo $fatias[1], PHP_EOL; // fatia2

// Exemplo 2
$data = "foo:*:1023:1000::/home/foo:/bin/sh";
list($user, $pass, $uid, $gid, $gecos, $home, $shell) = explode(":", $data);
echo $user, PHP_EOL; // foo
echo $pass, PHP_EOL; // *
?>
```

Exemplo #2 Exemplo de retorno de **explode()**

```php
<?php
/*
   Uma string que não contém o separador simplesmente
   irá retornar um array de 1 elemento com a string original.
*/
$input1 = "olá";
$input2 = "olá,mundo";
$input3 = ',';
var_dump( explode( ',', $input1 ) );
var_dump( explode( ',', $input2 ) );
var_dump( explode( ',', $input3 ) );

?>
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  string(5) "hello"
}
array(2) {
  [0]=>
  string(5) "hello"
  [1]=>
  string(5) "there"
}
array(2) {
  [0]=>
  string(0) ""
  [1]=>
  string(0) ""
}
```

Exemplo #3 Exemplos do parâmetro `limit`

```php
<?php
$str = 'um|dois|três|quatro';

// limite positivo
print_r(explode('|', $str, 2));

// limite negativo
print_r(explode('|', $str, -1));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => um
    [1] => dois|três|quatro
)
Array
(
    [0] => um
    [1] => dois
    [2] => três
)
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- preg_split() - Divide uma string com base em expressão regular
- str_split() - Converte uma string em um array
- mb_split() - Divide uma string multibyte usando expressão regular
- str_word_count() - Retorna informação sobre palavras usadas em uma string
- strtok() - Divide uma string em tokens

- implode() - Aglutina elementos de um array com uma string

fprintf

fprintf

(PHP 5, PHP 7, PHP 8)

fprintf — Escreve uma string formatada para um fluxo

Descrição

function **fprintf**(resource `$stream`, string `$format`, mixed `...$values`): int

Escreve uma string produzida de acordo com o formato em `format` para o recurso de fluxo especificado por `stream`.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
-	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.
+	Prefixa números positivos com um sinal de mais `+`; Por padrão, apenas números negativos são prefixados com um sinal negativo.
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
`'`(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :
>
> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
`G`	Como o especificador `g`, mas usa `E` e `f`.
`h`	Como o especificador `g`, mas usa `F`. Disponível a partir do PHP 8.0.0.
`H`	Como o especificador `g`, mas usa `E` e `F`. Disponível a partir do PHP 8.0.0.
`o`	O argumento é tratado como um número inteiro e apresentado como um número octal.
`s`	O argumento é tratado e apresentado como uma string.
`u`	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
`x`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
`X`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**
>
> O especificador de tipo `c` ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna o tamanho da string escrita.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ArgumentCountError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, `false` era retornado e um `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não retorna mais `false` em caso de falha.
8.0.0	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um `E_WARNING`.
8.0.0	Lança um erro ValueError se `[width]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente, esta função emitia um `E_WARNING`.
8.0.0	Lança um erro ValueError se `[precision]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente, esta função emitia um `E_WARNING`.
8.0.0	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um `E_WARNING`.

Examples

Exemplo #1 fprintf(): inteiros com zero à esquerda

```php
<?php
if (!($fp = fopen('date.txt', 'w'))) {
    return;
}

$year = 2005;
$month = 5;
$day = 6;

fprintf($fp, "%04d-%02d-%02d", $year, $month, $day);
// escreverá a data em formato ISO no arquivo date.txt
?>
```

Exemplo #2 fprintf(): formatando moeda

```php
<?php
if (!($fp = fopen('currency.txt', 'w'))) {
    return;
}

$money1 = 68.75;
$money2 = 54.35;
$money = $money1 + $money2;
// echo $money irá mostrar "123.1";
$len = fprintf($fp, '%01.2f', $money);
// escreverá "123.10" em currency.txt

echo "foram escritos $len bytes em currency.txt";
// usa o valor de retorno de fprintf para determinar quantos bytes foram escritos
?>
```

Veja também

- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada
- vprintf() - Mostra uma string formatada
- vsprintf() - Retorna uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

get_html_translation_table

get_html_translation_table

(PHP 4, PHP 5, PHP 7, PHP 8)

get_html_translation_table — Retorna a tabela de tradução usada por htmlspecialchars() e htmlentities()

Descrição

function **get_html_translation_table**(int `$table` = `HTML_SPECIALCHARS`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, string `$encoding` = "UTF-8"): array

get_html_translation_table() retorna a tabela de tradução usada internamente para htmlspecialchars() e htmlentities().

> **Nota** :
>
> Caracteres especiais podem ser codificados de diversas maneiras. Por exemplo, " pode ser codificado como `"`, `"` ou `"`. **get_html_translation_table()** retorna apenas a forma usada por htmlspecialchars() e htmlentities().

Parâmetros

`table`

Informa que tabela deve ser retornada. Pode ser `HTML_ENTITIES` ou `HTML_SPECIALCHARS`.

`flags`

Uma máscara de bits de uma ou mais das opções abaixo, que especificam que tipo de aspas a tabela conterá, assim como para que tipo de documento a tabela serve. O padrão é `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Constantes de `flags` disponíveis

Nome da Constante	Descrição
`ENT_COMPAT`	A tabela conterá entidades para aspas duplas, mas não para aspas simples.
`ENT_QUOTES`	A tabela conterá entitades tanto para aspas duplas quanto para aspas simples.
`ENT_NOQUOTES`	A tabela não conterá entidades nem para aspas simples nem para aspas duplas.
`ENT_SUBSTITUTE`	Substitui sequências de unidade de código inválidas com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de retornar uma string vazia.
`ENT_HTML401`	Tabela para HTML 4.01.
`ENT_XML1`	Tabela para XML 1.
`ENT_XHTML`	Tabela para XHTML.
`ENT_HTML5`	Tabela para HTML 5.

`encoding`

A codificação a ser usada. Se omitida, o valor padrão para este argumento é UTF-8.

Os seguintes conjuntos de caracteres são suportados:

Conjuntos de caracteres suportados

Conjunto de caracteres	Apelidos	Descrição
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Conjunto de caracteres cirílicos, pouco usado (latim/cirílico).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adiciona o símbolo do Euro, letras francesas e finlandesas faltantes no Latin-1 (ISO-8859-1).
UTF-8		Código de multi-byte 8-bits Unicode compatível com ASCII.
cp866	ibm866, 866	Conjunto de caracteres do DOS específico para o russo.
cp1251	Windows-1251, win-1251, 1251	Conjunto de caracteres do Windows específico para o russo.
cp1252	Windows-1252, 1252	Conjunto de caracteres do Windows específico para a Europa Ocidental.

Conjunto de caracteres	Apelidos	Descrição
KOI8-R	koi8-ru, koi8r	Russo.
BIG5	950	Chinês tradicional, usado principalmente em Taiwan.
GB2312	936	Chinês simplificado, conjunto de caracteres padrão nacional.
BIG5-HKSCS		Big5 com extensões de Hong Kong, chinês tradicional.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japonês
EUC-JP	EUCJP, eucJP-win	Japonês
MacRoman		Conjunto de caracteres que era usado pelo Mac OS.
`''`		Uma string vazia ativa a detecção a partir de codificação de script (multibyte Zend), conjunto padrão de caracteres e localidade atual (consulte nl_langinfo() e setlocale()), nesta ordem. Não recomendado.

> **Nota** :
>
> Nenhum outro conjunto de caracteres é reconhecido. A codificação padrão será usada no lugar e um alerta será emitido.

Valor Retornado

Retorna a tabela de tradução como um array, com os caracteres originais como chaves e as entidades como valores.

Registro de Alterações

Versão	Descrição
8.1.0	`flags` mudou de `ENT_COMPAT` para `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.

Examples

Exemplo #1 Exemplo de Tabela de Tradução

```php
<?php
var_dump(get_html_translation_table(HTML_ENTITIES, ENT_QUOTES | ENT_HTML5));
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1510) {
  ["
"]=>
  string(9) "&NewLine;"
  ["!"]=>
  string(6) "&excl;"
  [""""]=>
  string(6) "&quot;"
  ["#"]=>
  string(5) "&num;"
  ["$"]=>
  string(8) "&dollar;"
  ["%"]=>
  string(8) "&percnt;"
  ["&"]=>
  string(5) "&amp;"
  ["'"]=>
  string(6) "&apos;"
  // ...
}
```

Veja também

- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- html_entity_decode() - Converte entidades HTML aos seus caracteres correspondentes

hebrev

hebrev

(PHP 4, PHP 5, PHP 7, PHP 8)

hebrev — Converte texto hebraico lógico para texto visual

Descrição

function **hebrev**(string `$string`, int `$max_chars_per_line = 0`): string

Converte texto hebraico lógico para texto visual.

A função tenta evitar quebra de palavras.

Parâmetros

`string`

Uma string de entrada em hebraico.

`max_chars_per_line`

Este parâmetro opcional indica o número máximo de caracteres por linha que serão retornados.

Valor Retornado

Retorna a string visual.

Veja também

- hebrevc() - Converte texto hebraico lógico para texto visual com conversão de novas linhas

hebrevc

hebrevc

(PHP 4, PHP 5, PHP 7)

hebrevc — Converte texto hebraico lógico para texto visual com conversão de novas linhas

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **hebrevc**(string `$hebrew_text`, int `$max_chars_per_line = 0`): string

Esta função é similar a hebrev() com a diferença que ela converte novas linhas (\n) para "
\n".

A função tenta evitar quebra de palavras.

Parâmetros

`hebrew_text`

Ums string de entrada em hebraico.

`max_chars_per_line`

Este parâmetro opcional indica o número máximo de caracteres por linha que serão retornados.

Valor Retornado

Retorna a string visual.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Veja também

- hebrev() - Converte texto hebraico lógico para texto visual

hex2bin

hex2bin

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

hex2bin — Decodifica uma string binária codificada em hexadecimal

Descrição

function **hex2bin**(string `$string`): string|false

Decodifica uma string binária codificada em hexadecimal.

> **Cuidado**
>
> Esta função *NÃO* converte um número hexadecimal para um número binário. Isso pode ser feito através da função base_convert().

Parâmetros

`string`

Representação hexadecimal dos dados.

Valor Retornado

Retorna a representação binária dos dados fornecidos ou `false` em caso de falha.

Erros/Exceções

Se a string fornecida em hexadecimal tiver tamanho ímpar ou for um hexadecimal inválido, um erro de nível `E_WARNING` será lançado.

Examples

Exemplo #1 Exemplo de hex2bin()

```php
<?php
$hex = hex2bin("6578616d706c65206865782064617461");
var_dump($hex);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(16) "example hex data"
```

Veja também

- bin2hex() - Converte um dado binário em representação hexadecimal

- unpack() - Desempacota dados de uma string binária

html_entity_decode

html_entity_decode

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

html_entity_decode — Converte entidades HTML aos seus caracteres correspondentes

Descrição

function **html_entity_decode**(string `$string`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, ?string `$encoding` = `null`): string

html_entity_decode() é oposta a htmlentities() porque converte entidades HTML na `string` aos seus caracteres correspondentes.

Mais precisamente, esta função decodifica todas as entidades (incluindo as numéricas): a) que são necessariamente válidas para o tipo de documento escolhido, isto é, para XML esta função não decodifica entidades nomeadas que possam ser definidas em algum DTD; e b) cujos caracteres estejam no conjunto de caracteres da codificação associados com a codificação escolhida e sejam permitidos no tipo de documento escolhido. Todas as outras entidades ficarão inalteradas.

Parâmetros

`string`

A string de entrada.

`flags`

Uma máscara de bits de uma ou mais das opções a seguir, que especificam como lidar com aspas e que tipos de documentos são usados. O padrão é `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Constantes disponíveis para `flags`

Nome da Constante	Descrição
`ENT_COMPAT`	Converte aspas duplas e não converte aspas simples.
`ENT_QUOTES`	Converte tanto aspas duplas quanto simples.
`ENT_NOQUOTES`	Não converte aspas duplas ou simples.
`ENT_SUBSTITUTE`	Substitui sequências de unidade de código inválidas com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de retornar uma string vazia.
`ENT_HTML401`	Lida com o código como HTML 4.01.
`ENT_XML1`	Lida com o código como XML 1.
`ENT_XHTML`	Lida com o código como XHTML.
`ENT_HTML5`	Lida com o código como HTML 5.

`encoding`

Um argumento opcional que define a codificação usada na conversão de caracteres.

Se omitido, `encoding` assume como padrão o valor da opção de configuração default_charset .

Embora este argumento seja tecnicamente opcional, especificar o valor correto para o código é altamente recomendável se a opção de configuração default_charset puder ser definida incorretamente para a entrada fornecida.

Os seguintes conjuntos de caracteres são suportados:

Conjuntos de caracteres suportados

Conjunto de caracteres	Apelidos	Descrição
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Conjunto de caracteres cirílicos, pouco usado (latim/cirílico).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adiciona o símbolo do Euro, letras francesas e finlandesas faltantes no Latin-1 (ISO-8859-1).

Conjunto de caracteres	Apelidos	Descrição
UTF-8		Código de multi-byte 8-bits Unicode compatível com ASCII.
cp866	ibm866, 866	Conjunto de caracteres do DOS específico para o russo.
cp1251	Windows-1251, win-1251, 1251	Conjunto de caracteres do Windows específico para o russo.
cp1252	Windows-1252, 1252	Conjunto de caracteres do Windows específico para a Europa Ocidental.
KOI8-R	koi8-ru, koi8r	Russo.
BIG5	950	Chinês tradicional, usado principalmente em Taiwan.
GB2312	936	Chinês simplificado, conjunto de caracteres padrão nacional.
BIG5-HKSCS		Big5 com extensões de Hong Kong, chinês tradicional.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japonês
EUC-JP	EUCJP, eucJP-win	Japonês
MacRoman		Conjunto de caracteres que era usado pelo Mac OS.
''		Uma string vazia ativa a detecção a partir de codificação de script (multibyte Zend), conjunto padrão de caracteres e localidade atual (consulte nl_langinfo() e setlocale()), nesta ordem. Não recomendado.

> **Nota** :
>
> Nenhum outro conjunto de caracteres é reconhecido. A codificação padrão será usada no lugar e um alerta será emitido.

Valor Retornado

Retorna a string decodificada.

Registro de Alterações

Versão	Descrição
8.1.0	O padrão de `flags` mudou de `ENT_COMPAT` para `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.
8.0.0	`encoding` agora pode ser nulo.

Examples

Exemplo #1 Decodificando entidades HTML

```
<?php
$orig = "I'll \"walk\" the <b>dog</b> now";

$a = htmlentities($orig);

$b = html_entity_decode($a);

echo $a, PHP_EOL; // I'll &quot;walk&quot; the &lt;b&gt;dog&lt;/b&gt; now

echo $b, PHP_EOL; // I'll "walk" the <b>dog</b> now
?>
```

Notas

> **Nota** :
>
> Pode-se perguntar por que trim(html_entity_decode(' ')); não reduz a string a uma string vazia, e isso acontece porque a entidades ' ' não tem código ASCII 32 (que é o código removido por trim()) mas sim 160 (0xa0) na codificação padrão ISO 8859-1.

Veja também

- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- get_html_translation_table() - Retorna a tabela de tradução usada por htmlspecialchars e htmlentities
- urldecode() - Decodifica uma URL codificada

htmlentities

htmlentities

(PHP 4, PHP 5, PHP 7, PHP 8)

htmlentities — Converte todos os caracteres aplicáveis em entidades HTML

Descrição

```
function htmlentities(
    string $string,
    int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401,
    ?string $encoding = null,
    bool $double_encode = true
): string
```

Esta função é idêntica à htmlspecialchars() em todas as maneiras, exceto que com **htmlentities()**, todos os caracteres que têm equivalentes em HTML são traduzidos para essas entidades. A função get_html_translation_table() pode ser usada para retornar a tabela de tradução usada dependendo das constantes fornecidas em `flags`.

Se for desejada a decodificação (operação reversa) pode-se usar html_entity_decode().

Parâmetros

`string`

A string de entrada.

`flags`

Uma máscara de bits de uma ou mais das opções a seguir, que especificam como lidar com aspas, sequências inválidas de unidades de código e tipos de documentos usados. O padrão é `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Constantes disponíveis para `flags`

Nome da Constante	Descrição
`ENT_COMPAT`	Converte aspas duplas e não converte aspas simples.
`ENT_QUOTES`	Converte tanto aspas duplas quanto simples.
`ENT_NOQUOTES`	Não converte aspas duplas ou simples.
`ENT_IGNORE`	Descarta silenciosamente sequências de unidades de código inválidas ao invés de retornar uma string vazia. O uso desta opção é desencorajado pois » pode ter implicações de segurança.
`ENT_SUBSTITUTE`	Substitui sequências de unidade de código inválidas com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de retornar uma string vazia.
`ENT_DISALLOWED`	Substitui pontos de código inválidos para o tipo de documento informado por um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#FFFD; ao invés de não converter. Isto pode ser útil, por exemplo, para garantir boa formatação de documentos XML com conteúdo externo embutido.
`ENT_HTML401`	Lida com o código como HTML 4.01.
`ENT_XML1`	Lida com o código como XML 1.
`ENT_XHTML`	Lida com o código como XHTML.
`ENT_HTML5`	Lida com o código como HTML 5.

`encoding`

Um argumento opcional que define a codificação usada na conversão de caracteres.

Se omitido, `encoding` assume como padrão o valor da opção de configuração default_charset .

Embora este argumento seja tecnicamente opcional, especificar o valor correto para o código é altamente recomendável se a opção de configuração default_charset puder ser definida incorretamente para a entrada fornecida.

Os seguintes conjuntos de caracteres são suportados:

Conjuntos de caracteres suportados

Conjunto de caracteres	Apelidos	Descrição
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Conjunto de caracteres cirílicos, pouco usado (latim/cirílico).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adiciona o símbolo do Euro, letras francesas e finlandesas faltantes no Latin-1 (ISO-8859-1).
UTF-8		Código de multi-byte 8-bits Unicode compatível com ASCII.
cp866	ibm866, 866	Conjunto de caracteres do DOS específico para o russo.
cp1251	Windows-1251, win-1251, 1251	Conjunto de caracteres do Windows específico para o russo.
cp1252	Windows-1252, 1252	Conjunto de caracteres do Windows específico para a Europa Ocidental.
KOI8-R	koi8-ru, koi8r	Russo.
BIG5	950	Chinês tradicional, usado principalmente em Taiwan.
GB2312	936	Chinês simplificado, conjunto de caracteres padrão nacional.
BIG5-HKSCS		Big5 com extensões de Hong Kong, chinês tradicional.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japonês
EUC-JP	EUCJP, eucJP-win	Japonês
MacRoman		Conjunto de caracteres que era usado pelo Mac OS.
`''`		Uma string vazia ativa a detecção a partir de codificação de script (multibyte Zend), conjunto padrão de caracteres e localidade atual (consulte nl_langinfo() e setlocale()), nesta ordem. Não recomendado.

> **Nota** :
>
> Nenhum outro conjunto de caracteres é reconhecido. A codificação padrão será usada no lugar e um alerta será emitido.

`double_encode`

Quando `double_encode` estiver desligado, o PHP não irá codificar entidades HTML existentes. O padrão é converter tudo.

Valor Retornado

Retorna a string codificada.

Se a `string` de entrada contiver uma sequência de unidade de código inválida na condificação informada em `encoding`, uma string vazia será retornada, a menos que uma das opções `ENT_IGNORE` ou `ENT_SUBSTITUTE` esteja definida.

Registro de Alterações

Versão	Descrição
8.1.0	O padrão de `flags` mudou de `ENT_COMPAT` para `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.
8.0.0	`encoding` agora pode ser nulo.

Examples

Exemplo #1 Um exemplo de htmlentities()

```
<?php
$str = "Uma 'palavra' em <b>negrito</b>";
```

```php
echo htmlentities($str);
echo "\n\n";
echo htmlentities($str, ENT_COMPAT);
?>
```

O exemplo acima produzirá:

```
Uma &#039;palavra&#039; em &lt;b&gt;negrito&lt;/b&gt;

Uma 'palavra' em &lt;b&gt;negrito&lt;/b&gt;
```

Exemplo #2 Uso de `ENT_IGNORE`

```php
<?php
$str = "\x8F!!!";

// Saída: uma string vazia
echo htmlentities($str, ENT_QUOTES, "UTF-8");

// Saída: "!!!"
echo htmlentities($str, ENT_QUOTES | ENT_IGNORE, "UTF-8");
?>
```

Veja também

- html_entity_decode() - Converte entidades HTML aos seus caracteres correspondentes
- get_html_translation_table() - Retorna a tabela de tradução usada por htmlspecialchars e htmlentities
- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- nl2br() - Insere quebras de linha HTML antes de todos os caracteres de nova linha em um string
- urlencode() - Codifica uma string no formato URL

htmlspecialchars

htmlspecialchars

(PHP 4, PHP 5, PHP 7, PHP 8)

htmlspecialchars — Converte caracteres especiais para entidades HTML

Descrição

function **htmlspecialchars**(
  string $string,
  int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401,
  ?string $encoding = null,
  bool $double_encode = true
): string

Certos caracteres têm significado especial em HTML, e devem ser representados por entidades HTML se for necessário preservar seus significados. Esta função retorna uma string com estas conversões realizadas. Se for necessário traduzir todas as substrings de entrada que têm entidades nomeadas associadas, use htmlentities() no lugar desta função.

Se a string de entrada passada para esta função e o documento final compartilham o mesmo conjunto de caracteres, esta função é suficiente para preparar a entrada para inclusão na maioria dos contextos de um documento HTML. Se, por outro lado, a entrada puder representar caracteres que não são codificados no conjunto de caracteres do documento final, e for necessário reter esses caracteres (como entidades numéricas ou nomeadas), tanto esta funçã quanto htmlentities() (que apenas codifica substrings que têm entidades nomeadas equivalentes) podem ser insuficientes. Pode ser necessário usar mb_encode_numericentity() no lugar desta função.

Traduções realizadas

Caractere	Substituição

Caractere	Substituição
& (e comercial)	`&`
" (asplas duplas)	`"`, a menos que **ENT_NOQUOTES** esteja definida
' (aspas simples)	`'` (para **ENT_HTML401**) ou `'` (para **ENT_XML1**, **ENT_XHTML** ou **ENT_HTML5**), mas apenas quando **ENT_QUOTES** estiver definida
< (menor que)	`<`
> (maior que)	`>`

Parâmetros

`string`

A string a ser convertida.

`flags`

Uma máscara de bits de uma ou mais das opções a seguir, que especificam como lidar com aspas, com sequências de unidade de código inválidas e o tipo de documento usado. O padrão é `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Constantes disponíveis para `flags`

Nome da Constante	Descrição
ENT_COMPAT	Converte aspas duplas e não converte aspas simples.
ENT_QUOTES	Converte tanto aspas duplas quanto simples.
ENT_NOQUOTES	Não converte aspas duplas ou simples.
ENT_IGNORE	Descarta silenciosamente as sequências de unidade de código inválidas ao invés de retornar uma string vazia. É desencorajado o uso desta já que » podem haver implicações em segurança.
ENT_SUBSTITUTE	Substitui sequências de unidade de código inválidas com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de retornar uma string vazia.
ENT_DISALLOWED	Substitui pontos de código inválidos para o tipo de documento fornecido com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de deixá-lo inalterado. Isto pode ser útil, por exemplo, para garantir a boa formatação de documentos XML com conteúdo externo incluído.
ENT_HTML401	Lida com o código como HTML 4.01.
ENT_XML1	Lida com o código como XML 1.
ENT_XHTML	Lida com o código como XHTML.
ENT_HTML5	Lida com o código como HTML 5.

`encoding`

Um argumento opcional que define a codificação usada na conversão de caracteres.

Se omitido, `encoding` assume como padrão o valor da opção de configuração default_charset .

Embora este argumento seja tecnicamente opcional, especificar o valor correto para o código é altamente recomendável se a opção de configuração default_charset puder ser definida incorretamente para a entrada fornecida.

Para os propósitos desta função, as codificações `ISO-8859-1`, `ISO-8859-15`, `UTF-8`, `cp866`, `cp1251`, `cp1252` e `KOI8-R` são efetivamente equivalentes, desde que o parâmetro `string` em si seja válido para a codificação, já que os caracteres afetados por **htmlspecialchars()** ocupam as mesmas posições em todas estas codificações.

Os seguintes conjuntos de caracteres são suportados:

Conjuntos de caracteres suportados

Conjunto de caracteres	Apelidos	Descrição
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Conjunto de caracteres cirílicos, pouco usado (latim/cirílico).

Conjunto de caracteres	Apelidos	Descrição
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adiciona o símbolo do Euro, letras francesas e finlandesas faltantes no Latin-1 (ISO-8859-1).
UTF-8		Código de multi-byte 8-bits Unicode compatível com ASCII.
cp866	ibm866, 866	Conjunto de caracteres do DOS específico para o russo.
cp1251	Windows-1251, win-1251, 1251	Conjunto de caracteres do Windows específico para o russo.
cp1252	Windows-1252, 1252	Conjunto de caracteres do Windows específico para a Europa Ocidental.
KOI8-R	koi8-ru, koi8r	Russo.
BIG5	950	Chinês tradicional, usado principalmente em Taiwan.
GB2312	936	Chinês simplificado, conjunto de caracteres padrão nacional.
BIG5-HKSCS		Big5 com extensões de Hong Kong, chinês tradicional.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japonês
EUC-JP	EUCJP, eucJP-win	Japonês
MacRoman		Conjunto de caracteres que era usado pelo Mac OS.
''		Uma string vazia ativa a detecção a partir de codificação de script (multibyte Zend), conjunto padrão de caracteres e localidade atual (consulte nl_langinfo() e setlocale()), nesta ordem. Não recomendado.

> **Nota** :
>
> Nenhum outro conjunto de caracteres é reconhecido. A codificação padrão será usada no lugar e um alerta será emitido.

`double_encode`

Quando `double_encode` estiver desligada, o PHP não codificará entidades HTML existentes, o padrão é converter tudo.

Valor Retornado

A string convertida.

Se a `string` de entrada contiver uma sequência de unidade de código inválida na codificação informada em `encoding`, uma string vazia será retornada, a menos que **`ENT_IGNORE`** ou **`ENT_SUBSTITUTE`** esteja definida.

Registro de Alterações

Versão	Descrição
8.1.0	O padrão para `flags` mudou de **`ENT_COMPAT`** para **`ENT_QUOTES`** | **`ENT_SUBSTITUTE`** | **`ENT_HTML401`**.

Examples

Exemplo #1 Exemplo de **htmlspecialchars()**

```
<?php
$new = htmlspecialchars("<a href='test'>Test</a>", ENT_QUOTES);
echo $new; // &lt;a href=&#039;test&#039;&gt;Test&lt;/a&gt;
?>
```

Notas

> **Nota** :
>
> Observe que esta função não traduz nada além do que está listado acima. Para tradução completa de entidade, consulte htmlentities().

> **Nota** :

> Em caso de um valor ambíguo no parâmetro `flags`, as seguintes regras se aplicam:
>
> - Quando nenhuma das constantes **ENT_COMPAT**, **ENT_QUOTES** ou **ENT_NOQUOTES** estiver presente, o padrão é **ENT_NOQUOTES**.
> - Quando mais que uma das constantes **ENT_COMPAT**, **ENT_QUOTES** ou **ENT_NOQUOTES** estiverem presentes, **ENT_QUOTES** tem a maior precedência, seguida por **ENT_COMPAT**.
> - Quando nenhuma das constantes **ENT_HTML401**, **ENT_HTML5**, **ENT_XHTML** ou **ENT_XML1** estiver presente, o padrão é **ENT_HTML401**.
> - Quando mais que uma das constantes **ENT_HTML401**, **ENT_HTML5**, **ENT_XHTML** ou **ENT_XML1** estiverem presentes, **ENT_HTML5** tem a maior precedência, seguida por **ENT_XHTML**, **ENT_XML1** e **ENT_HTML401**.
> - Quando mais que uma das constantes **ENT_DISALLOWED**, **ENT_IGNORE** ou **ENT_SUBSTITUTE** estiverem presentes, **ENT_IGNORE** tem a maior precedência, seguida por **ENT_SUBSTITUTE**.

Veja também

- get_html_translation_table() - Retorna a tabela de tradução usada por htmlspecialchars e htmlentities
- htmlspecialchars_decode() - Converte entidades especiais HTML de volta para caracteres
- strip_tags() - Retira as tags HTML e PHP de uma string
- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- nl2br() - Insere quebras de linha HTML antes de todos os caracteres de nova linha em um string

htmlspecialchars_decode

htmlspecialchars_decode

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

htmlspecialchars_decode — Converte entidades especiais HTML de volta para caracteres

Descrição

function **htmlspecialchars_decode**(string `$string`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401): string

Esta função é oposta a htmlspecialchars(). Ela converte de volta entidade especiais HTML em caracteres.

As entidades convertidas são: `&`, `"` (quando **ENT_NOQUOTES** não estiver definida), `'` (quando **ENT_QUOTES** estiver definida), `<` e `>`.

Parâmetros

`string`

A string a ser decodificada.

`flags`

Uma máscara de bits de uma ou mais das opções a seguir, que especificam como lidar com aspas e que tipos de documentos são usados. O padrão é `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Constantes disponíveis para `flags`

Nome da Constante	Descrição
ENT_COMPAT	Converte aspas duplas e não converte aspas simples.
ENT_QUOTES	Converte tanto aspas duplas quanto simples.
ENT_NOQUOTES	Não converte aspas duplas ou simples.
ENT_SUBSTITUTE	Substitui sequências de unidade de código inválidas com um Caractere de Substituição Unicode U+FFFD (UTF-8) ou &#xFFFD; ao invés de retornar uma string vazia.
ENT_HTML401	Lida com o código como HTML 4.01.
ENT_XML1	Lida com o código como XML 1.
ENT_XHTML	Lida com o código como XHTML.
ENT_HTML5	Lida com o código como HTML 5.

Valor Retornado

Retorna a string decodificada.

Registro de Alterações

Versão	Descrição
8.1.0	O padrão de `flags` mudou de `ENT_COMPAT` para `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.

Examples

Exemplo #1 Um exemplo de htmlspecialchars_decode()

```
<?php
$str = "<p>this -&gt; &quot;</p>\n";

echo htmlspecialchars_decode($str);

// observe que aqui as aspas não são convertidas
echo htmlspecialchars_decode($str, ENT_NOQUOTES);
?>
```

O exemplo acima produzirá:

```
<p>this -> "</p>
<p>this -> &quot;</p>
```

Veja também

- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- html_entity_decode() - Converte entidades HTML aos seus caracteres correspondentes
- get_html_translation_table() - Retorna a tabela de tradução usada por htmlspecialchars e htmlentities

implode

implode

(PHP 4, PHP 5, PHP 7, PHP 8)

implode — Aglutina elementos de um array com uma string

Descrição

function **implode**(string `$separator`, array `$array`): string

Assinatura alternativa (não suportada com argumentos nomeados):

function **implode**(array `$array`): string

Assinatura legada (descontinuada a partir do PHP 7.4.0, removida a partir do PHP 8.0.0):

function **implode**(array `$array`, string `$separator`): string

Combina elementos de um array com um string de separação definida por `separator`.

Parâmetros

`separator`

Opcional. O padrão é uma string vazia.

`array`

O array de strings a implodir.

Valor Retornado

Retorna uma string contendo um representação de todos os elementos do array na mesma ordem, com a string separadora entre eles.

Registro de Alterações

Versão	Descrição
8.0.0	Passar `separator` depois de `array` não é mais suportado.
7.4.0	Passar `separator` depois de `array` (isto é, usando a assinatura legada) foi descontinuado.

Examples

Exemplo #1 Exemplo de implode()

```php
<?php

$array = ['sobrenome', 'email', 'telefone'];
var_dump(implode(",", $array)); // string(24) "sobrenome,email,telefone"

// String vazia ao usar um array vazio:
var_dump(implode('olá', [])); // string(0) ""

// O separador é opcional:
var_dump(implode(['a', 'b', 'c'])); // string(3) "abc"

?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- explode() - Divide uma string com base em outra string
- preg_split() - Divide uma string com base em expressão regular
- http_build_query() - Gera a string de consulta em formato URL

join

join

(PHP 4, PHP 5, PHP 7, PHP 8)

join — Sinônimo de implode()

Descrição

Esta função é um apelido para: implode().

lcfirst

lcfirst

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

lcfirst — Torna minúsculo o primeiro caractere de uma string

Descrição

function **lcfirst**(string `$string`): string

Retorna uma string com o primeiro caractere de `string` em letra minúscula se o caractere for um ASCII na faixa de `"A"` (0x41) a `"Z"` (0x5a).

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string resultante.

Registro de Alterações

Versão	Descrição
8.2.0	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale(). Somente caracteres ASCII serão convertidos.

Examples

Exemplo #1 Exemplo de lcfirst()

```php
<?php
$foo = 'OlaMundo';
echo lcfirst($foo), PHP_EOL;      // olaMundo

$bar = 'OLA MUNDO!';
echo lcfirst($bar);               // oLA MUNDO!
echo lcfirst(strtoupper($bar));   // oLA MUNDO!
?>
```

Veja também

- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo
- strtolower() - Converte uma string para minúsculas
- strtoupper() - Converte uma string em maiúsculas
- ucwords() - Converte para maiúsculas o primeiro caractere de cada palavra

levenshtein

levenshtein

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

levenshtein — Calcula a distância Levenshtein entre duas strings

Descrição

```
function levenshtein(
    string $string1,
    string $string2,
    int $insertion_cost = 1,
    int $replacement_cost = 1,
    int $deletion_cost = 1
): int
```

A distância Levenshtein é definida como o número mínimo de caracteres que devem ser substituídos, inseridos ou removidos para transformar a `string1` na `string2`. A complexidade do algoritmo é `O(m*n)`, onde `n` e `m` são os comprimentos da `string1` e da `string2` (muito bom quando comparado com similar_text(), que é `O(max(n,m)**3)`, mas mesmo assim é custoso).

Se `insertion_cost`, `replacement_cost` e/ou `deletion_cost` não são iguais a `1`, o algoritmo se adapta para escolher a transformação menos custosa. Por exemplo, se `$insertion_cost + $deletion_cost < $replacement_cost`, nenhuma substituição será feita, mas apenas inserções e remoções.

Parâmetros

`string1`

Uma das strings a ser avaliada para a distância Levenshtein.

`string2`

Uma das strings a ser avaliada para a distância Levenshtein.

`insertion_cost`

Define o custo da inserção.

`replacement_cost`

Define o custo da substituição.

`deletion_cost`

Define o custo da remoção.

Valor Retornado

Eta função retorna a distância Levenshtein entre as duas strings.

Registro de Alterações

Versão	Descrição
8.0.0	Antes desta versão, **levenshtein()** tinha de ser chamada com dois ou com cinco argumentos.
8.0.0	Antes desta versão, **levenshtein()** retornaria `-1` se uma das strings fosse maior que 255 caracteres.

Examples

Exemplo #1 Exemplo de **levenshtein()**

```php
<?php
// palavra com erro ortográfico
$input = 'abacachi';

// array com palavras para comparação
$words  = array('maçã','abacaxi','banana','laranja',
                'rabanete','cenoura','ervilha','feijão','batata');

// nenhuma distância mais curta encontrada, ainda
$shortest = -1;

// itera pelas palavras para encontrar a mais próxima
foreach ($words as $word) {

    // calcula a distância entre a palavra de entrada
    // e a palavra atual
    $lev = levenshtein($input, $word);

    // verifica se há correspondência exata
    if ($lev == 0) {

        // a mais próxima é essa (correspondência exata)
        $closest = $word;
        $shortest = 0;

        // sai do loop; já foi encontrada a correspondência exata
        break;
    }

    // se esta distância for menor que a próxima
    // OU se se uma próxima palavra mais curta ainda não foi encontrada
    if ($lev <= $shortest || $shortest < 0) {
        // define a corrspondência mais próxima e a distância mais curta
        $closest  = $word;
        $shortest = $lev;
    }
}

echo "Palavra de entrada: $input\n";
if ($shortest == 0) {
    echo "Correspondência exata encontrada: $closest\n";
} else {
    echo "Você quis dizer: $closest?\n";
```

```
}

?>
```

O exemplo acima produzirá:

```
Palavra de entrada: abacachi
Você quis dizer: abacaxi?
```

Veja também

- soundex() - Calcula a chave soundex de uma string
- similar_text() - Calcula a similaridade entre duas strings
- metaphone() - Calcula a chave de Metaphone de uma string

localeconv

localeconv

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

localeconv — Obtém informação de formatação numérica

Descrição

function **localeconv**(): array

Retorna um array associativo contendo informação de formatação numérica e monetária de acordo com a localidade.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

localeconv() retorna dados com base na localidade atual definida por setlocale(). O array associativo retornado contém os seguintes campos:

Elemento do array	Descrição
decimal_point	Caractere de separação decimal
thousands_sep	Caractere de separação de milhares
grouping	Array contendo agrupamentos numéricos
int_curr_symbol	Símbolo internacional da moeda (ex.: BRL)
currency_symbol	Símbolo local da moeda (ex.: R$)
mon_decimal_point	Caractere de separação de decimais da moeda
mon_thousands_sep	Caractere de separação de milhares da moeda
mon_grouping	Array contendo agrupamentos monetários
positive_sign	Sinal para valores positivos
negative_sign	Sinal para valores negativos
int_frac_digits	Dígitos fracionários internacionais
frac_digits	Dígitos fracionários locais
p_cs_precedes	1 se currency_symbol precede um valor positivo, 0 se ele o sucede
p_sep_by_space	1 se um espaço separa currency_symbol de um valor positivo, 0 caso contrário
n_cs_precedes	1 se currency_symbol precede um valor negativo, 0 se ele o sucede
n_sep_by_space	1 se um espaço separa currency_symbol de um valor negativo, 0 caso contrário

Elemento do array	Descrição
p_sign_posn	• 0 - Parênteses envolvem a quantidade e o currency_symbol • 1 - A string de sinal precede a quantidade e o currency_symbol • 2 - A string de sinal sucede a quantidade e o currency_symbol • 3 - A string de sinal precede imediatamente o currency_symbol • 4 - A string de sinal sucede imediatamente o currency_symbol
n_sign_posn	• 0 - Parênteses envolvem a quantidade e o currency_symbol • 1 - A string de sinal precede a quantidade e o currency_symbol • 2 - A string de sinal sucede a quantidade e o currency_symbol • 3 - A string de sinal precede imediatamente o currency_symbol • 4 - A string de sinal sucede imediatamente o currency_symbol

O `p_sign_posn` e o `n_sign_posn` contêm uma string de opções de formatação. Cada número representa uma das condições listadas acima.

Os campos de agrupamento contêm arrays que definem a maneira em que os números devem ser agrupados. Por exemplo, o campo de agrupamento monetário para a localidade nl_NL (em modo UTF-8 com o sinal do Euro), contêm um array de 2 itens com os valores 3 e 3. Quanto maior o índice no array, mais a esquerda fica o agrupamento. Se um elemento do array for igual a `CHAR_MAX`, não são feitos mais agrupamentos. Se um elemento do array for igual a 0, o elemento anterior deve ser usado.

Examples

Exemplo #1 Exemplo de localeconv()

```php
<?php
if (false !== setlocale(LC_ALL, 'nl_NL.UTF-8@euro')) {
    $locale_info = localeconv();
    print_r($locale_info);
}
?>
```

O exemplo acima produzirá:

```
Array
(
    [decimal_point] => .
    [thousands_sep] =>
    [int_curr_symbol] => EUR
    [currency_symbol] => €
    [mon_decimal_point] => ,
    [mon_thousands_sep] =>
    [positive_sign] =>
    [negative_sign] => -
    [int_frac_digits] => 2
    [frac_digits] => 2
    [p_cs_precedes] => 1
    [p_sep_by_space] => 1
    [n_cs_precedes] => 1
    [n_sep_by_space] => 1
    [p_sign_posn] => 1
    [n_sign_posn] => 2
    [grouping] => Array
        (
        )

    [mon_grouping] => Array
        (
            [0] => 3
            [1] => 3
        )

)
```

Veja também

- setlocale() - Define informação de localidade

ltrim

ltrim

(PHP 4, PHP 5, PHP 7, PHP 8)

ltrim — Retira espaços em branco (ou outros caracteres) do início de uma string

Descrição

function **ltrim**(string `$string`, string `$characters = " \n\r\t\v\x00"`): string

Retira espaços em branco (ou outros caracteres) do início de uma string.

Sem o segundo parâmetro, **ltrim()** removerá os seguintes caracteres:

- `" "`: caractere ASCII SP `0x20`, um espaço normal.
- `"\t"`: caractere ASCII HT `0x09`, uma tabulação.
- `"\n"`: caractere ASCII LF `0x0A`, uma nova linha (alimentação de linha).
- `"\r"`: caractere ASCII CR `0x0D`, um retorno de carro.
- `"\0"`: caractere ASCII NUL `0x00`, o byte nulo.
- `"\v"`: caractere ASCII VT `0x0B`, uma tabulação vertical.

Parâmetros

`string`

A string de entrada.

`characters`

Opcionalmente, os caracteres removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos. Com `..` é possível especificar um intervalo incremental de caracteres.

Valor Retornado

Esta função retorna uma string com os espaços em branco removidos do ínicio da `string`.

Examples

Exemplo #1 Exemplo de uso de ltrim()

```php
<?php

$text = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);

print "\n";


$trimmed = ltrim($text);
var_dump($trimmed);

$trimmed = ltrim($text, " \t.");
var_dump($trimmed);

$trimmed = ltrim($hello, "Hdle");
```

```php
var_dump($trimmed);

// tira os caracteres de controle ASCII do início de $binary
// (de 0 a 31 inclusive)
$clean = ltrim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

O exemplo acima produzirá:

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"

string(30) "These are a few words :) ...  "
string(30) "These are a few words :) ...  "
string(7) "o World"
string(15) "Example string
"
```

Veja também

- trim() - Retira espaços (ou outros caracteres) do início e do final de uma string
- rtrim() - Retira espaços em branco (ou outros caracteres) do final de uma string

md5

md5

(PHP 4, PHP 5, PHP 7, PHP 8)

md5 — Calcula o hash md5 de uma string

> **Aviso**
>
> Não é recomendado usar esta função para proteger senhas, devido à natureza rápida deste algoritmo de hash. Consulte as Perguntas Frequentes sobre Hashing de Senha para obter detalhes e melhores práticas.

Descrição

function **md5**(string `$string`, bool `$binary` = `false`): string

Calcula o hash MD5 hash da `string` usando o » Algoritmo de Sintetização de Mensagem MD5 da RSA Data Security, Inc., e retorna esse hash.

Parâmetros

`string`

A string.

`binary`

Se o parâmetro opcional `binary` for definido para `true`, a sintetização MD5 é retornada em formato bruto binário com um comprimento de 16 bytes.

Valor Retornado

Retorna o hash como um número hexadecimal de 32 caracteres.

Examples

Exemplo #1 Um exemplo de **md5()**

```php
<?php
$str = 'apple';

if (md5($str) === '1f3870be274f6c49b3e31a0c6728957f') {
    echo "Would you like a green or red apple?";
}
?>
```

Veja também

- hash() - Gera um valor hash (resumo da mensagem)
- password_hash() - Cria um hash de senha

md5_file

md5_file

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

md5_file — Calcula o hash md5 de um arquivo

Descrição

function **md5_file**(string `$filename`, bool `$binary` = **`false`**): string|false

Calcula o hash MD5 do arquivo especificado pelo parâmetro `filename` usando o » Algoritmo de Sintetização de Mensagem da RSA Data Security, Inc., e retorna esse hash. O hash é um número hexadecimal de 32 caracteres.

Parâmetros

`filename`

O nome do arquivo.

`binary`

Quando **`true`**, retorna a sintetização MD5 em formato binário bruto com um comprimento de 16 bytes.

Valor Retornado

Retorna uma string em caso de sucesso, **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de uso de **md5_file()**

```php
<?php
$file = '/examples/book.xml';

echo 'Hash MD5 do arquivo ' . $file . ': ' . md5_file($file);
?>
```

Veja também

- hash_file() - Gera um valor hash usando o conteúdo de um arquivo fornecido
- hash_init() - Inicializa um contexto de hash incremental
- md5() - Calcula o hash md5 de uma string

metaphone

metaphone

(PHP 4, PHP 5, PHP 7, PHP 8)

metaphone — Calcula a chave de Metaphone de uma string

Descrição

function **metaphone**(string `$string`, int `$max_phonemes` = 0): string

Calcula a chave de Metaphone da `string`.

Parecida com a função soundex(), metaphone cria a mesma chave para palavras que soam de forma similar. É mais precisa que soundex() já que ela conhece as regras básicas de pronúncia em inglês. As chaves geradas têm comprimento variável.

Metaphone foi desenvolvido por Lawrence Philips <lphilips at verity dot com>. É descrita em ["Practical Algorithms for Programmers", Binstock & Rex, Addison Wesley, 1995].

Parâmetros

`string`

A string de entrada.

`max_phonemes`

Este parâmetro restringe o comprimento da chave retornada ao número máximo de *characters* especificado em `max_phonemes`. Entretanto, os fonemas resultantes são sempre transcritos completamente, portanto o comprimento resultante da string pode ser um pouco maior que `max_phonemes`. O valor padrão de `0` significa sem restrição.

Valor Retornado

Retorna a chave de Metaphone como uma string.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função retornava `false` em caso de falha.

Examples

Exemplo #1 Exemplo básico de metaphone()

```php
<?php
var_dump(metaphone('programming'));
var_dump(metaphone('programmer'));
?>
```

O exemplo acima produzirá:

```
string(7) "PRKRMNK"
string(6) "PRKRMR"
```

Exemplo #2 Usando o parâmetro `max_phonemes`

```php
<?php
var_dump(metaphone('programming', 5));
var_dump(metaphone('programmer', 5));
?>
```

O exemplo acima produzirá:

```
string(5) "PRKRM"
string(5) "PRKRM"
```

Exemplo #3 Usando o parâmetro `max_phonemes`

Neste exemplo, **metaphone()** é instruída a produzir uma string de cinco caracteres, mas isso iria separar o fonema final (`'x'` deve ser transcrito para `'KS'`), portanto a função retorna uma string com seis caracteres.

```
<?php
var_dump(metaphone('Asterix', 5));
?>
```

O exemplo acima produzirá:

```
string(6) "ASTRKS"
```

Veja também

- levenshtein() - Calcula a distância Levenshtein entre duas strings
- similar_text() - Calcula a similaridade entre duas strings
- soundex() - Calcula a chave soundex de uma string

money_format

money_format

(PHP 4 >= 4.3.0, PHP 5, PHP 7)

money_format — Formata um número como uma string de moeda

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **money_format**(string `$format`, float `$number`): string

money_format() retorna uma versão formatada do número informado no parâmetro `number`. Esta função engloba a biblioteca de função do C **strfmon()**, com a diferença que esta implementação converte apenas um número por vez.

Parâmetros

`format`

A especificação do formato consiste na seguinte sequência:

- um caractere `%`
- opções facultativas
- largura de campo, opcional
- precisão à esquerda, opcional
- precisão à direita, opcional
- conversão de caractere, requerida

Opções

Uma ou mais das opções abaixo podem ser usadas:

=f

O caracter = seguido por um caractere (único byte) f para ser usado como o caractere númerico de preenchimento. O caractere de preenchimento padrão é o espaço.

^

Desabilita o uso de agrupamento de caractere (como definido pela localidade atual).

+ ou (

Especifica o estilo de formatação para números positivos e negativos. Se + for usado, o equivalente da localidade para + e - será usado. Se (for usado, quantidades negativas são fechadas entre parênteses. se nenhuma especificação é passada, o padrão será +.

!

Suprime o símbolo atual da string de saída.

-

Se presente, isto fará todos os campos justificados à esquerda (arrumados para a direita), em oposição ao padrão que é para campos serem justificados à direita (arrumados para a esquerda).

Largura de campo

w

Uma string de dígito decimal especificando uma largura de campo mínima. O campo será justificado à direita a menos que a opção - seja usada. Valor padrão é 0 (zero) por definição.

Precisão à esquerda

#n

O número máximo de dígitos (n) esperado à esquerda do caracatere separador de decimais (ex.: vírgula). Isto é usado normalmente para manter a saída formatada alinhada nas mesmas colunas, usando o caractere de preenchimento se o número de dígitos for menor que n. Se o número de dígitos atual for maior que n, esta especificação é ignorada.

Se o agrupamento foi suprimido usando a opção ^, separadores de agrupamento serão inseridos antes dos caracteres de preenchimento (se houver), mesmo se o caractere de preenchimento for um dígito.

Para garantir alinhamento, quaisquer caracteres que ocorrerem antes ou depois do número na saída formatada, como moeda ou símbolos de sinal, são arrumados conforme necessário, com caracteres de espaço para fazer seus formatos positivos e negativos em um comprimento igual.

Precisão à direita

.p

Um ponto seguido por um número de dígitos (p) depois do caractere separador de decimais. Se o valor de p for 0 (zero), o caractere separador de decimais e os digitos à sua direita serão omitidos. Se nenhuma precisão à direita for incluida, o padrão será ditado pela localidade atual em uso. A quantidade sendo formatada é arredondada para o número de dígitos especificados a priori para ser formatado.

Caracteres de conversão

i

O número é formatado de acordo com o formato de moeda internacional da localidade (ex. para a localidade en_US: USD 1,234.56).

n

O número é formatado de acordo com o formato de moeda nacional da localidade (ex. para a localidade de_DE: DM1.234,56).

%

Retorna o caractere %.

number

O número a ser formatado.

Valor Retornado

Retorna a string formatada. Os caracteres antes e depois da string de formatação serão retornados sem modificação. O parâmetro `number` com dígitos inválidos causa o retorno de `null` e o lançamento de um `E_WARNING`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada. Utilize NumberFormatter::formatCurrency() em seu lugar.

Examples

Exemplo #1 Exemplo de money_format()

Diferentes localidades e especificações de formatos são usados no exemplo abaixo para ilustrar o uso desta função.

```
<?php

$number = 1234.56;

// Exibindo o formato internacional para a localidade en_US
setlocale(LC_MONETARY, 'en_US');
echo money_format('%i', $number) . "\n";
// USD 1,234.56

// Formato italiano com duas casas decimais
setlocale(LC_MONETARY, 'it_IT');
echo money_format('%.2n', $number) . "\n";
// Eu 1.234,56

// Usando número negativo
$number = -1234.5672;

// Formato nacional americano, usando () para números negativos
// e 10 dígitos de precisão à esquerda
setlocale(LC_MONETARY, 'en_US');
echo money_format('%(#10n', $number) . "\n";
// ($        1,234.57)

// Formato similar ao anterior, adicionando o uso de 2 dígitos para precisão
// à direita e '*' como caractere de preenchimento
echo money_format('%=*(#10.2n', $number) . "\n";
// ($********1,234.57)

// Valor alinhado à esquerda, com 14 posições de largura, 8 dígitos de
// precisão à esquerda, 2 à direita, sem caractere de agrupamento
// e usando o formato internacional da localidade de_DE.
setlocale(LC_MONETARY, 'de_DE');
echo money_format('%=*^-14#8.2i', 1234.56) . "\n";
// Eu 1234,56****

// Adicionando texto antes e depois da especificação de conversão
setlocale(LC_MONETARY, 'en_GB');
$fmt = 'O valor final é %i (depois de 10%% de desconto)';
echo money_format($fmt, 1234.56) . "\n";
// O valor final é  GBP 1,234.56 (depois de 10%% de desconto)

?>
```

Notas

> **Nota** :
>
> A função **money_format()** é apenas definida se o sistema tem capacidades de strfmon. Por exemplo, o Windows não tem esta definição, portanto, **money_format()** está indefinida no Windows.

> **Nota** :
>
> A categoria `LC_MONETARY` de configurações de localidade, afeta o comportamento desta função. Utilize setlocale() para definir a localidade padrão adequadamente antes de usar esta função.

Veja também

- setlocale() - Define informação de localidade
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- sprintf() - Retona uma string formatada
- printf() - Envia uma string formatada para a saída
- number_format() - Formata um número com milhares agrupados

nl_langinfo

nl_langinfo

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

nl_langinfo — Consulta informação de língua e localidade

Descrição

function **nl_langinfo**(int `$item`): string|false

nl_langinfo() é usada para acessar elementos individuais de categorias de localidade. Diferentemente de localeconv(), que retorna todos os elementos, **nl_langinfo()** permite selecionar qualquer elemento específico.

Parâmetros

`item`

`item` pode ser um valor inteiro do elemento ou o nome da constante do elemento. A lista de nomes de constantes que podem ser usadas para o parâmetro `item` e suas descrições está apresentada abaixo. Algumas destas constantes podem não estar definidas ou podem não conter valor para certas localidades.

nl_langinfo Constants

Constante	Descrição
Constantes da Categoria `LC_TIME`	
`ABDAY_(1-7)`	Nome abreviado no n-ésimo dia da semana.
`DAY_(1-7)`	Nome do n-ésimo dia da semana (DAY_1 = domingo).
`ABMON_(1-12)`	Nome abreviado do n-ésimo mês do ano.
`MON_(1-12)`	Nome do n-ésimo mês do ano.
`AM_STR`	String para horários antes do meio-dia (ante meridiem).
`PM_STR`	String para horário após o meio-dia (post meridiem).
`D_T_FMT`	String que pode ser usada como formatação para strftime() para representar horário e data.
`D_FMT`	String que pode ser usada como formatação para strftime() para representar data.
`T_FMT`	String que pode ser usada como formatação para strftime() para representar horário.
`T_FMT_AMPM`	String que pode ser usada como formatação para strftime() para representar horário em formato de 12 horas com ante/post meridiem.
`ERA`	Era alternativa.
`ERA_YEAR`	Ano no formato da era alternativa.
`ERA_D_T_FMT`	Data e horário em formato da era alternativa (string pode ser usada em strftime()).
`ERA_D_FMT`	Data em formato da era alternativa (string pode ser usada em strftime()).
`ERA_T_FMT`	Horário em formato da era alternativa (string pode ser usada em strftime()).
Constantes da Categoria `LC_MONETARY`	
`INT_CURR_SYMBOL`	Símbolo internacional da moeda.

Constante	Descrição
CURRENCY_SYMBOL	Símbolo local da moeda.
CRNCYSTR	O mesmo que CURRENCY_SYMBOL.
MON_DECIMAL_POINT	Caractere de seperação de decimais.
MON_THOUSANDS_SEP	Separador de milhares (grupos de três dígitos).
MON_GROUPING	O mesmo que o elemento "grouping".
POSITIVE_SIGN	Sinal para valores positivos.
NEGATIVE_SIGN	Sinal para valores negativos.
INT_FRAC_DIGITS	Dígitos fracionários internacionais.
FRAC_DIGITS	Dígitos fracionários locais.
P_CS_PRECEDES	Retorna 1 se CURRENCY_SYMBOL precede um valor positivo.
P_SEP_BY_SPACE	Retorna 1 se um espaço separa CURRENCY_SYMBOL de um valor positivo.
N_CS_PRECEDES	Retorna 1 se CURRENCY_SYMBOL precede um valor negativo.
N_SEP_BY_SPACE	Retorna 1 se um espaço separa CURRENCY_SYMBOL de um valor negativo.
P_SIGN_POSN	• Retorna 0 se parênteses envolvem a quantidade e o CURRENCY_SYMBOL. • Retorna 1 se a string de sinal precede a quantidade e o CURRENCY_SYMBOL. • Retorna 2 se a string de sinal sucede a quantidade e o CURRENCY_SYMBOL. • Retorna 3 se a string de sinal precede imediatamente o CURRENCY_SYMBOL. • Retorna 4 se a string de sinal imediatamente sucede o CURRENCY_SYMBOL.
N_SIGN_POSN	
Constantes da Categoria LC_NUMERIC	
DECIMAL_POINT	Caractere separador de decimais.
RADIXCHAR	O mesmo valor de DECIMAL_POINT.
THOUSANDS_SEP	Caractere separador de milhares (grupos de três dígitos).
THOUSEP	O mesmo valor de THOUSANDS_SEP.
GROUPING	
Constantes da Categoria LC_MESSAGES	
YESEXPR	String de expressão regular para corresponder à entrada "yes" (sim).
NOEXPR	String de expressão regular para corresponder à entrada "no" (não).
YESSTR	String de saída para "yes" (sim).
NOSTR	String de saída para "no" (não).
Constantes da Categoria LC_CTYPE	
CODESET	Retorna uma string com o nome da codificação de caracteres.

Valor Retornado

Retorna o elemento como uma string ou `false` se `item` não for válido.

Examples

Exemplo #1 Exemplo de **nl_langinfo()**

```
<?php

var_dump(nl_langinfo(CODESET));
var_dump(nl_langinfo(YESEXPR));
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(14) "ANSI_X3.4-1968"
string(5) "^[yY]"
```

Notas

> **Nota** :
>
> Esta função não foi implementada em plataformas Windows.

Veja também

- setlocale() - Define informação de localidade
- localeconv() - Obtém informação de formatação numérica

nl2br

nl2br

(PHP 4, PHP 5, PHP 7, PHP 8)

nl2br — Insere quebras de linha HTML antes de todos os caracteres de nova linha em um string

Descrição

function **nl2br**(string `$string`, bool `$use_xhtml = true`): string

Retorna a `string` com `<br />` ou `<br>` inseridos antes de todas os caracteres de nova linha (`\r\n`, `\n\r`, `\n` e `\r`).

Parâmetros

`string`

A string de entrada.

`use_xhtml`

Informa se devem ser usadas quebras de linha compatíveis com XHTML ou não.

Valor Retornado

Retorna a string modificada.

Examples

Exemplo #1 Usando nl2br()

```php
<?php
echo nl2br("isto não é\n aquilo");
?>
```

O exemplo acima produzirá:

```
isto não é<br />
 aquilo
```

Exemplo #2 Grando marcação HTML válida usando o parâmetro `use_xhtml`

```php
<?php
echo nl2br("Bem-vindo\r\nEste é meu documento HTML", false);
?>
```

O exemplo acima produzirá:

```
Bem-vindo<br>
Este é meu documento HTML
```

Exemplo #3 Vários separadores de linha

```php
<?php
$string = "Isto\r\né\n\ruma\nstring\r";
echo nl2br($string);
?>
```

O exemplo acima produzirá:

```
Isto<br />
é<br />
uma<br />
string<br />
```

Veja também

- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- wordwrap() - Quebra uma string em um dado número de caracteres
- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição

number_format

number_format

(PHP 4, PHP 5, PHP 7, PHP 8)

number_format — Formata um número com milhares agrupados

Descrição

function **number_format**(
    float $num,
    int $decimals = 0,
    ?string $decimal_separator = ".",
    ?string $thousands_separator = ","
): string

Formata um número com milhares agrupados e opcionalmente dígitos decimais usando a regra de arredondamento de meia unidade para cima.

Parâmetros

`num`

O número a ser formatado.

`decimals`

Define o número de dígitos decimais. Se for igual a `0`, o separador de decimais definido por `decimal_separator` é omitido do valor de retorno. A partir do PHP 8.3.0, quando o valor é negativo, `num` é arredondado para o número de dígitos significativos informado em `decimals` antes do separador de decimais. Antes do PHP 8.3.0, valores negativos eram ignorados e manipulados como se fossem `0`.

`decimal_separator`

Define a string separadora de decimais.

`thousands_separator`

Define a string separadora de milhares.

Valor Retornado

Uma versão formatada de `num`.

Registro de Alterações

Versão	Descrição
8.3.0	Adicionada a manipulação de valores negativos para `decimals`.
8.0.0	Antes desta versão, **number_format()** aceitava um, dois ou quatro parâmetros (mas não três).
7.2.0	**number_format()** foi modificado para não ser capaz de retornar `-0`, anteriormente `-0` poderia ser retornado para casos onde `num` fosse `-0.01`.

Examples

Exemplo #1 Exemplo de **number_format()**

Por exemplo, a notação francesa normalmente usa duas casas decimais, vírgula (',') como separador de decimais e espaço (' ') como separador de milhares. O exemplo a seguir demonstra várias formas de formatar um número:

```php
<?php

$number = 1234.56;

// notação inglesa (padrão)
echo number_format($number), PHP_EOL;
// 1,235

// notação francesa
echo number_format($number, 2, ',', ' '), PHP_EOL;
// 1 234,56

$number = 1234.5678;

// notação inglesa sem separador de milhares
echo number_format($number, 2, '.', ''), PHP_EOL;
// 1234.57

?>
```

Exemplo #2 Um valor negativo para `decimals`

A partir do PHP 8.3.0, um valor negativo para `decimals` é usado para arredondar o número de dígitos significativos antes do separador de decimais.

```php
<?php
$number = "1234.5678";
var_dump(number_format($number, -1));
var_dump(number_format($number, -2));
var_dump(number_format($number, -3));
?>
```

O exemplo acima produzirá:

```
```

```
string(5) "1,230"
string(5) "1,200"
string(5) "1,000"
```

Veja também

- money_format() - Formata um número como uma string de moeda
- sprintf() - Retona uma string formatada
- printf() - Envia uma string formatada para a saída
- sscanf() - Interpreta a entrada de uma string de acordo com um formato

ord

ord

(PHP 4, PHP 5, PHP 7, PHP 8)

ord — Converte o primeiro byte de uma string para um valor entre 0 e 255

Descrição

function **ord**(string `$character`): int

Interpreta o valor binário do primeiro byte de `character` como um inteiro sem sinal entre 0 e 255.

Se a string estiver em codificação de um byte, como ASCII, ISO-8859 ou Windows 1252, é equivalente a retornar a posição de um caractere na tabela de mapeamento do conjunto de caracteres. Entretanto, note que esta função não tem ciência da codificação da string, e em particular nunca irá identificar um ponto de código Unicode em uma codificação multi-byte como UTF-8 ou UTF-16.

Esta função complementa a função chr().

Parâmetros

`character`

A string que terá o primeiro caractere avaliado.

Valor Retornado

Um inteiro entre 0 e 255.

Registro de Alterações

Versão	Descrição
8.5.0	Passar uma string que não seja um byte único foi descontinuado.

Examples

Exemplo #1 Exemplo de **ord()**

```php
<?php
$str = "\n";
if (ord($str) == 10) {
    echo "O primeiro caractere de \$str é uma nova linha.\n";
}
?>
```

Exemplo #2 Examinando os bytes individuais de uma string UTF-8

```php
<?php
$str = "🐘";
for ( $pos=0; $pos < strlen($str); $pos ++ ) {
 $byte = substr($str, $pos);
 echo 'Byte ' . $pos . ' de $str tem valor ' . ord($byte) . PHP_EOL;
```

```
}
?>
```

O exemplo acima produzirá:

```
Byte 0 de $str tem valor 240

Byte 1 de $str tem valor 159

Byte 2 de $str tem valor 144

Byte 3 de $str tem valor 152
```

Veja também

- chr() - Gera uma string de um byte a partir de um número
- » Tabela ASCII
- mb_ord() - Obtém o ponto de código Unicode do caractere
- IntlChar::ord() - Retorna o valor do ponto de código Unicode do caractere

parse_str

parse_str

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_str — Interpreta uma string como uma requisição URL

Descrição

function **parse_str**(string `$string`, array `&$result`): void

Interpreta `string` como se ela fosse uma string de consulta passada via URL e define chaves no array informado em `result`. Se nenhum parâmetro `result` for passado, os valores são definidos como variáveis no escopo atual.

Parâmetros

`string`

A string de entrada.

`result`

Uma variável passada por referência, que será definida como um array contendo os pares chave-valor extraídos de `string`. Se o parâmetro `result` não for passado, uma variável separada será definida no escopo local para cada chave.

> **Aviso**
>
> Utilizar esta função sem o parâmetro `result` é altamente *DESENCORAJADO* e foi *DESCONTINUADO* a partir do PHP 7.2. Desde o 8.0.0, o parâmetro `result` é *obrigatório*.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`result` não é mais opcional.
7.2.0	Utilização de **parse_str()** sem o segundo parâmetro agora emite um aviso de `E_DEPRECATED`.

Examples

Exemplo #1 Usando parse_str()

```php
<?php
$str = "primeiro=valor&arr[]=foo+bar&arr[]=baz";

// Recomendado
parse_str($str, $output);
echo $output['primeiro'], PHP_EOL;  // valor
echo $output['arr'][0], PHP_EOL; // foo bar
echo $output['arr'][1], PHP_EOL; // baz
?>
```

Quaisquer espaços e pontos em nomes de parâmetros são convertidos em sublinhados ao criar chaves de array ou variáveis locais. Isso ocorre porque nomes de variáveis em PHP não podem conter espaços ou pontos, e se aplica mesmo ao usar esta função com o parâmetro `result` recomendado.

Exemplo #2 Codificação de nomes em parse_str()

```php
<?php
parse_str("Meu Valor=Algo", $output);
echo $output['Meu_Valor']; // Algo
?>
```

Notas

> **Nota** :
>
> **parse_str()** é afetada pela diretiva max_input_vars. Exceder esse limite emite um `E_WARNING`, e quaisquer variáveis além do limite não são adicionadas ao array de resultados. O padrão é 1000; ajuste max_input_vars conforme necessário.

> **Nota** :
>
> Todos os valores preenchidos no array `result` (ou variáveis criadas se o segundo parâmetro não estiver definido) já são decodificados por URL usando as mesmas regras de urldecode().

> **Nota** :
>
> Para obter a string de consulta da solicitação atual, pode ser usada a variável *$_SERVER['QUERY_STRING']*. Além disso, leia a seção sobre variáveis de fontes externas.

Veja também

- parse_url() - Analisa uma URL e retorna seus componentes
- pathinfo() - Retorna informações sobre um caminho de arquivo
- http_build_query() - Gera a string de consulta em formato URL
- urldecode() - Decodifica uma URL codificada

print

print

(PHP 4, PHP 5, PHP 7, PHP 8)

print — Exibe uma string

Descrição

function **print**(string `$expression`): int

Exibe o conteúdo de `expression`.

`print` não é uma função, é uma construção de linguagem. Seu argumento é a expressão logo após a palavra-chave `print`, não delimitada por parênteses.

As principais diferenças para echo são que `print` aceita apenas um único argumento e sempre retorna `1`.

Parâmetros

`expression`

A expressão a ser mostrada. Valores não-string serão convertidos coercitivamente para strings, mesmo quando a diretiva `strict_types` estiver habilitada.

Valor Retornado

Retorna `1`, sempre.

Examples

Exemplo #1 Exemplos de `print`

```php
<?php
print "print não requer parênteses.";
print PHP_EOL;

// Não é adicionado espaço ou quebra de linha; o exemplo abaixo mostra "olámundo" em uma só linha
print "olá";
print "mundo";
print PHP_EOL;

print "Esta string ocupa
múltiplas linhas. As quebras de linha
também serão mostradas.";
print PHP_EOL;

print "Esta string ocupa\nmúltiplas linhas. As quebras de linha\ntambém serão mostradas.";
print PHP_EOL;

// O argumento pode ser qualquer expressão que produza uma string
$foo = "exemplo";
print "foo é um $foo"; // foo é um exemplo
print PHP_EOL;

$frutas = ["limão", "laranja", "banana"];
print implode(" e ", $frutas); // limão e laranja e banana
print PHP_EOL;

// Expressões não-string são convertidas coercitivamente para strings, mesmo que declare(strict_types=1) seja usada
print 6 * 7; // 42
print PHP_EOL;

// Como print tem um valor de retorno, pode ser usada em expressões
// O código a seguir mostra "olá mundo"
if ( print "olá" ) {
    echo " mundo";
}
print PHP_EOL;

// O código a seguir mostra "verdadeiro"
( 1 === 1 ) ? print 'verdadeiro' : print 'falso';
print PHP_EOL;
?>
```

Notas

> **Nota** : **Usando com parênteses**
>
> Envolver o argumento de `print` com parênteses não irá causar um erro de sintaxe, e produz uma sintaxe que se parece com uma chamada normal de função. Entretanto, isso pode causar confusão, porque os parênteses são na verdade parte da expressão sendo mostrada, e não parte da sintaxe de `print` propriamente.

```php
<?php
print "olá";
// exibe "olá"
```

```php
print("hello");
// também exibe "olá", porque ("olá") é uma expressão válida

print(1 + 2) * 3;
// exibe "9"; os parênteses fazem com que 1+2 seja avaliado primeiro, e depois 3*3
// a declaração print enxerga toda a expressão como um único argumento

if ( print("olá") && false ) {
    print " - dentro do if";
}
else {
    print " - dentro do else";
}
// Exibe " - dentro do if"
// A expressão ("olá") && false é avaliada primeiro, produzindo resultado falso.
// Isto é convertido à força para uma string vazia "" e exibido.
// A declaração print a partir daí retorna 1, então o código dentro do bloco if é executado
?>
```

Ao usar `print` com uma expressão maior, colocar tanto a palavra-chave quanto seu argumento entre parênteses pode ser necessário para gerar o resultado desejado:

```php
<?php
if ( (print "olá") && false ) {
    print " - dentro do if";
}
else {
    print " - dentro do else";
}
// Exibe "olá - dentro do else"
// Diferentemente do exemplo anterior, a expressão (print "olá") é avaliada primeiro.
// Depois de exibir "olá", print retorna 1.
// Já que 1 && false é falso, o código dentro do bloco else é executado.

print "olá " && print "mundo";
// Exibe "mundo1"; print "mundo" é avaliado primeiro,
// e depois a expressão "olá " && 1 é passada ao print da esquerda

(print "olá ") && (print "mundo");
// Exibe "olá mundo"; os parênteses forçam a avaliação das expressões print
// antes da operação &&
?>
```

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

Veja também

- echo - Envia uma ou mais strings para a saída
- printf() - Envia uma string formatada para a saída
- flush() - Descarrega o buffer na saída do sistema
- Maneiras de especificar strings literais

printf

printf

(PHP 4, PHP 5, PHP 7, PHP 8)

printf — Envia uma string formatada para a saída

Descrição

function **printf**(string `$format`, mixed `...$values`): int

Produz uma saída de acordo com o formato definido no parâmetro `format`.

Parâmetros

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
`-`	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.
`+`	Prefixa números positivos com um sinal de mais `+`; Por padrão, apenas números negativos são prefixados com um sinal negativo.
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
`'`(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :
>
> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e

Especificador	Descrição
	precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
G	Como o especificador g, mas usa E e F.
h	Como o especificador g, mas usa F. Disponível a partir do PHP 8.0.0.
H	Como o especificador g, mas usa E e F. Disponível a partir do PHP 8.0.0.
o	O argumento é tratado como um número inteiro e apresentado como um número octal.
s	O argumento é tratado e apresentado como uma string.
u	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
x	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
X	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**
>
> O especificador de tipo c ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna o comprimento da string enviada para a saída.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ArgumentCountError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, **`false`** era retornado e um **`E_WARNING`** era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não retorna mais **`false`** em caso de falha.
8.0.0	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um **`E_WARNING`**.
8.0.0	Lança um erro ValueError se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente, esta função emitia um **`E_WARNING`**.
8.0.0	Lança um erro ValueError se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente, esta função emitia um **`E_WARNING`**.

Versão	Descrição
8.0.0	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um `E_WARNING`.

Examples

Exemplo #1 printf(): vários exemplos

```php
<?php
$n =  43951789;
$u = -43951789;
$c = 65; // ASCII 65 é 'A'

// observe o duplo %%, isto mostra um caractere '%' literal
printf("%%b = '%b'\n", $n); // representação binária
printf("%%c = '%c'\n", $c); // mostra o caractere ASCII, o mesmo que a função chr() faz
printf("%%d = '%d'\n", $n); // representação padrão de número inteiro
printf("%%e = '%e'\n", $n); // notação científica
printf("%%u = '%u'\n", $n); // representação de inteiro sem sinal de um inteiro positivo
printf("%%u = '%u'\n", $u); // representação de inteiro sem sinal de um inteiro negativo
printf("%%f = '%f'\n", $n); // representação em ponto flutuante
printf("%%o = '%o'\n", $n); // representação em octal
printf("%%s = '%s'\n", $n); // representação em string
printf("%%x = '%x'\n", $n); // representação em hexadecimal (minúsculas)
printf("%%X = '%X'\n", $n); // representação em hexadecimal (maiúsculas)

printf("%%+d = '%+d'\n", $n); // especificador de sinal em um inteiro positivo
printf("%%+d = '%+d'\n", $u); // especificador de sinal em um inteiro negativo
?>
```

O exemplo acima produzirá:

```
%b = '10100111101010011010101101'
%c = 'A'
%d = '43951789'
%e = '4.39518e+7'
%u = '43951789'
%u = '4251015507'
%f = '43951789.000000'
%o = '247523255'
%s = '43951789'
%x = '29ea6ad'
%X = '29EA6AD'
%+d = '+43951789'
%+d = '-43951789'
```

Exemplo #2 printf(): especificadores de string

```php
<?php
$s = 'monkey';
$t = 'many monkeys';

printf("[%s]\n",        $s); // saída padrão de string
printf("[%10s]\n",      $s); // alinhamento à direita com espaços
printf("[%-10s]\n",     $s); // alinhamento à esquerda com espaços
printf("[%010s]\n",     $s); // preenchimento com zeros funciona em strings também
printf("[%'#10s]\n",    $s); // usa o caractere customizado de preenchimento '#'
printf("[%'#*s]\n", 10, $s); // fornece a largura de preenchimento como um argumento adicional
printf("[%10.9s]\n",    $t); // alinhamento à direita com corte de 9 caracteres
printf("[%-10.9s]\n",   $t); // alinhamento à esquerda com corte de 9 caracteres
?>
```

O exemplo acima produzirá:

```
[monkey]
[    monkey]
[monkey    ]
[0000monkey]
[####monkey]
[####monkey]
[ many monk]
[many monk ]
```

Veja também

- print - Exibe uma string
- sprintf() - Retona uma string formatada
- fprintf() - Escreve uma string formatada para um fluxo
- vprintf() - Mostra uma string formatada
- vsprintf() - Retorna uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix
- flush() - Descarrega o buffer na saída do sistema

quoted_printable_decode

quoted_printable_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

quoted_printable_decode — Converte uma string Quoted-Printable para uma string de 8 bits

Descrição

function **quoted_printable_decode**(string `$string`): string

Esta função retorna uma string binária de 8 bits correspondendo à string Quoted-Printable decodificada (de acordo com a » RFC2045 seção 6.7, e não à » RFC2821 seção 4.5.2, de forma que pontos adicionais não são removidos do início de uma linha).

Esta função é similar à imap_qprint(), exceto que essa não requer o módulo IMAP para funcionar.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna uma string binária de 8 bits.

Examples

Exemplo #1 Exemplo de quoted_printable_decode()

```php
<?php

$encoded = quoted_printable_encode('Möchten Sie ein paar Äpfel?');

var_dump($encoded);
var_dump(quoted_printable_decode($encoded));
?>
```

O exemplo acima produzirá:

```
string(37) "M=C3=B6chten Sie ein paar =C3=84pfel?"
string(29) "Möchten Sie ein paar Äpfel?"
```

Veja também

- quoted_printable_encode() - Converte uma string de 8 bits para uma string Quoted-Printable

quoted_printable_encode

quoted_printable_encode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

quoted_printable_encode — Converte uma string de 8 bits para uma string Quoted-Printable

Descrição

function **quoted_printable_encode**(string `$string`): string

Retorna uma string Quoted-Printable de acordo com a » RFC2045 seção 6.7.

Esta função é similar à imap_8bit(), exceto que essa não requer o módulo IMAP para funcionar.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string codificada.

Examples

Exemplo #1 Exemplo de quoted_printable_encode()

```php
<?php

$encoded = quoted_printable_encode('Möchten Sie ein paar Äpfel?');

var_dump($encoded);
var_dump(quoted_printable_decode($encoded));
?>
```

O exemplo acima produzirá:

```
string(37) "M=C3=B6chten Sie ein paar =C3=84pfel?"
string(29) "Möchten Sie ein paar Äpfel?"
```

Veja também

- quoted_printable_decode() - Converte uma string Quoted-Printable para uma string de 8 bits
- iconv_mime_encode() - Compõe um campo de cabeçalho MIME

quotemeta

quotemeta

(PHP 4, PHP 5, PHP 7, PHP 8)

quotemeta — Escapa meta caracteres

Descrição

function **quotemeta**(string `$string`): string

Retorna uma versão de da string com uma barra invertida (\) antes de cada caractere que esteja neste conjunto:

```
. \ + * ? [ ^ ] ( $ )
```

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string com os meta caracteres escapados ou `false` se uma string vazia for fornecida no parâmetro `string`.

Examples

Exemplo #1 Exemplo de quotemeta()

```php
<?php

var_dump(quotemeta('PHP é uma linguagem de script popular. Rápida, flexível e pragmática.'));
?>
```

O exemplo acima produzirá:

```
string(71) "PHP é uma linguagem de script popular\. Rápida, flexível e pragmática\."
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- addslashes() - Adiciona barras invertidas a uma string
- addcslashes() - Escapa string com barras invertidas no estilo C
- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- htmlspecialchars() - Converte caracteres especiais para entidades HTML
- nl2br() - Insere quebras de linha HTML antes de todos os caracteres de nova linha em um string
- stripslashes() - Desfaz os escapes de uma string escapada
- stripcslashes() - Remove o escape de strings escapadas com addcslashes
- preg_quote() - Escapa caracteres de expressão regular

rtrim

rtrim

(PHP 4, PHP 5, PHP 7, PHP 8)

rtrim — Retira espaços em branco (ou outros caracteres) do final de uma string

Descrição

function **rtrim**(string `$string`, string `$characters` = " \n\r\t\v\x00"): string

Esta função retorna uma string com os espaços (ou outros caracteres) retirados do final de `string`.

Sem o segundo parâmetro, **rtrim()** irá retirar os seguintes caracteres:

- `" "`: caractere ASCII SP `0x20`, um espaço normal.
- `"\t"`: caractere ASCII HT `0x09`, uma tabulação.
- `"\n"`: caractere ASCII LF `0x0A`, uma nova linha (alimentação de linha).
- `"\r"`: caractere ASCII CR `0x0D`, um retorno de carro.
- `"\0"`: caractere ASCII NUL `0x00`, o byte nulo.
- `"\v"`: caractere ASCII VT `0x0B`, uma tabulação vertical.

Parâmetros

`string`

A string de entrada.

`characters`

Opcionalmente, os caracteres removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos. Com `..` é possível especificar um intervalo incremental de caracteres.

Valor Retornado

Retorna a string modificada.

Examples

Exemplo #1 Exemplo de uso da função rtrim()

```php
<?php

$text = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);

print "\n";

$trimmed = rtrim($text);
var_dump($trimmed);

$trimmed = rtrim($text, " \t.");
var_dump($trimmed);

$trimmed = rtrim($hello, "Hdle");
var_dump($trimmed);

// Retira os caracteres de controle ASCII no final de $binary
// (de 0 a 31 inclusive)
$clean = rtrim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

O exemplo acima produzirá:

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"
```

```
string(30) "            These are a few words :) ..."
string(26) "            These are a few words :)"
string(9) "Hello Wor"
string(15) "    Example string"
```

Veja também

- trim() - Retira espaços (ou outros caracteres) do início e do final de uma string
- ltrim() - Retira espaços em branco (ou outros caracteres) do início de uma string

setlocale

setlocale

(PHP 4, PHP 5, PHP 7, PHP 8)

setlocale — Define informação de localidade

Descrição

function **setlocale**(int `$category`, ?string `$locales`, string `...$rest`): string|false

Assinatura alternativa (não suportada com argumentos nomeados):

function **setlocale**(int `$category`, array `$locale_array`): string|false

Define informação de localidade.

> **Aviso**
>
> As informações de localidade são mantidas por processo, não por thread. Se o PHP estiver sendo executado em uma API de servidor multithread, mudanças repentinas poderão ocorrer nas configurações de localidade enquanto um script estiver em execução, embora o script em si nunca tenha chamado **setlocale()**. Isso acontece devido a outros scripts sendo executados em diferentes threads do mesmo processo ao mesmo tempo, alterando a localidade de todo o processo usando **setlocale()**. No Windows, as informações de localidade são mantidas por thread a partir do PHP 7.0.5.

Parâmetros

`category`

`category` é uma constante nomeada que especifica a categoria das funções afetadas pela configuração de localidade:

- **`LC_ALL`** para todas as opções abaixo
- **`LC_COLLATE`** para comparação de strings, consulte strcoll()
- **`LC_CTYPE`** para classificação e conversão de caracteres, por exemplo ctype_alpha()
- **`LC_MONETARY`** para localeconv()
- **`LC_NUMERIC`** para separador de decimais (Consulte também localeconv())
- **`LC_TIME`** para formatação de data e horário com strftime()
- **`LC_MESSAGES`** para respostas de sistema (disponível se o PHP foi compilado com a `libintl`)

`locales`

Se `locales` for uma string vazia `""` ou **`null`**, os nomes de localidade serão definidos a partir dos valores das variáveis de ambiente com os mesmos nomes das categorias acima, ou de "LANG".

Se `locales` for `"0"`, a configuração de localidade não será afetada, apenas a configuração atual será retornada.

Se `locales` for seguido por parâmetros adicionais, a função tentará definir cada parâmetro como um novo locale até obter sucesso. Isto é útil se uma localidade for conhecida por nomes diferentes em sistemas diferentes ou para fornecer um substituto para uma localidade possivelmente não disponível.

`rest`

Parâmetros de string opcionais para tentar como configurações de localidade até obter sucesso.

`locale_array`

A função tenta definir cada elemento do array como um novo local até obter sucesso. Isto é útil se uma localidade for conhecida por nomes diferentes em sistemas diferentes ou para fornecer um substituto para uma localidade possivelmente não disponível.

> **Nota** :
>
> No Windows, setlocale(LC_ALL, '') define os nomes de localidade a partir dados configurações regionais e de idioma do sistema (acessíveis através do Painel de Controle).

Valor Retornado

Retorna a nova localidade atual ou `false` se a funcionalidade de localidade não estiver implementada em sua plataforma, a localidade especificada não existir ou o nome da categoria for inválido.

Um nome de categoria inválido também gera uma mensagem de alerta. Os nomes de categoria/localidade podem ser encontrados na » RFC 1766 e na » ISO 639. Sistemas diferentes possuem esquemas de nomenclatura diferentes para localidades.

> **Nota** :
>
> O valor de retorno de **setlocale()** depende do sistema que o PHP está sendo executado. Ele retorna exatamente o que a função de sistema `setlocale` retorna.

Registro de Alterações

Versão	Descrição
8.5.0	Passar um inteiro `0` como o argumento de `locales` não é mais suportado e agora lança um TypeError.

Examples

Exemplo #1 Exemplos de **setlocale()**

```php
<?php
/* Define a localidade para holandês */
setlocale(LC_ALL, 'nl_NL');

/* Saída: vrijdag 22 december 1978 */
echo strftime("%A %e %B %Y", mktime(0, 0, 0, 12, 22, 1978));

/* tenta nomes de localidades diferentes possíves para alemão */
$loc_de = setlocale(LC_ALL, 'de_DE@euro', 'de_DE', 'de', 'ge');
echo "A localidade preferida para alemão neste sistema é '$loc_de'";
?>
```

Exemplo #2 Recuperando a configuração atual de **setlocale()**

```php
<?php
/* Recuperando a configuração atual */
$current = setlocale(LC_ALL, null);

echo "Localidade atual: '$current'";
?>
```

Exemplo #3 Exemplos de **setlocale()** para Windows

```php
<?php
/* Define a localidade para holandês */
setlocale(LC_ALL, 'nld_nld');

/* Saída: vrijdag 22 december 1978 */
echo strftime("%A %d %B %Y", mktime(0, 0, 0, 12, 22, 1978));
```

```php
/* tenta nomes de localidades diferentes possíves para alemão */
$loc_de = setlocale(LC_ALL, 'de_DE@euro', 'de_DE', 'deu_deu');
echo "A localidade preferida para alemão neste sistema é '$loc_de'";
?>
```

Notas

> **Dica**
>
> Os usuários do Windows encontrarão informações úteis sobre strings de `locales` no site MSDN da Microsoft. As strings de idioma suportadas podem ser encontradas na » documentação de strings de linguagem e as strings de país/região suportadas podem ser encontradas na » documentação de strings de país/região.

sha1

sha1

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

sha1 — Calcula o hash SHA1 de uma string

> **Aviso**
>
> Não é recomendado usar esta função para proteger senhas, devido à natureza rápida deste algoritmo de hash. Consulte as Perguntas Frequentes sobre Hashing de Senha para obter detalhes e melhores práticas.

Descrição

function **sha1**(string `$string`, bool `$binary` = `false`): string

Calcula o hash SHA1 da `string` usando o algoritmo » US Secure Hash Algorithm 1.

Parâmetros

`string`

A string de entrada.

`binary`

Se o parâmetro opcional `binary` for definido para `true`, o hash SHA1 é retornado em formato binário bruto com um comprimento de 20 bytes, caso contrário o valor retornado é um número hexadecimal de 40 caracteres.

Valor Retornado

Retorna o hash SHA1 como uma string.

Examples

Exemplo #1 Um exemplo de sha1()

```php
<?php
$str = 'banana';

if (sha1($str) === '250e77f12a5ab6972a0895d290c4792f0a326ea8') {
    echo "Gostaria de uma banana prata ou uma banana da terra?";
}
?>
```

Veja também

- hash() - Gera um valor hash (resumo da mensagem)
- password_hash() - Cria um hash de senha

sha1_file

sha1_file

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

sha1_file — Calcula a hash sha1 de um arquivo

Descrição

function **sha1_file**(string `$filename`, bool `$binary = false`): string|false

Calcula a hash sha1 do arquivo especificado por `filename` usando o » US Secure Hash Algorithm 1, e retorna esta hash. A hash é um número hexadecimal de 40 caracteres.

Parâmetros

`filename`

O nome do arquivo de onde a hash será obtida.

`binary`

Quando `true`, retorna a hash no formato binário bruto com um tamanho de 20 caracteres.

Valor Retornado

Retorna uma string em caso de sucesso, `false` caso contrário.

Examples

Exemplo #1 Exemplo de sha1_file()

```php
<?php
foreach(glob('/examples/*.xml') as $ent)
{
    if (is_dir($ent)) {
        continue;
    }

    echo $ent . ' (SHA1: ' . sha1_file($ent) . ')', PHP_EOL;
}
?>
```

Veja também

- hash_file() - Gera um valor hash usando o conteúdo de um arquivo fornecido
- hash_init() - Inicializa um contexto de hash incremental
- sha1() - Calcula o hash SHA1 de uma string

similar_text

similar_text

(PHP 4, PHP 5, PHP 7, PHP 8)

similar_text — Calcula a similaridade entre duas strings

Descrição

function **similar_text**(string `$string1`, string `$string2`, float `&$percent = null`): int

Calcula a similaridade entre duas strings como descrito em Programming Classics: Implementing the World's Best Algorithms by Oliver (ISBN 0-131-00413-1). Note que esta implementação não usa uma pilha como no pseudo-código de Oliver, mas chamadas recursivas que podem ou não acelerar todo o processo. Note também que a complexidade deste algoritmo é O(N**3) onde N é o tamanho da string mais longa.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

> **Nota** :
>
> Trocar a `string1` de lugar com a `string2` pode gerar um resultado diferente; veja o exemplo abaixo.

`percent`

Ao passar referência como terceiro argumento, **similar_text()** irá calcular a similaridade em percentual, dividindo o resultado de **similar_text()** percentual média de comprimentos das strings fornecidas vezes `100`.

Valor Retornado

Retorna o número de caracteres correspondentes em ambas as strings.

O número de caracteres correspondentes é calculado encontrando-se a primeira substring comum mais longa, e depois fazendo o mesmo para os prefixos e sufixos, recursivamente. Os comprimentos de todas as substrings comuns encontradas são somados.

Examples

Exemplo #1 Exemplo de similar_text() com inversão de ordem de argumentos

Este exemplo mostra que a inversão da ordem de `string1` e `string2` pode gerar resultados diferentes.

```php
<?php
$sim = similar_text('bafoobar', 'barfoo', $perc);
echo "similarity: $sim ($perc %)\n";
$sim = similar_text('barfoo', 'bafoobar', $perc);
echo "similarity: $sim ($perc %)\n";
```

O exemplo acima produzirá algo semelhante a:

```
similarity: 5 (71.428571428571 %)
similarity: 3 (42.857142857143 %)
```

Veja também

- levenshtein() - Calcula a distância Levenshtein entre duas strings
- metaphone() - Calcula a chave de Metaphone de uma string
- soundex() - Calcula a chave soundex de uma string

soundex

soundex

(PHP 4, PHP 5, PHP 7, PHP 8)

soundex — Calcula a chave soundex de uma string

Descrição

function **soundex**(string `$string`): string

Calcula a chave soundex de `string`.

Chaves soundex têm a propriedade de que palavras pronunciadas de forma semelhante produzem a mesma chave soundex e podem, portanto, ser usadas para simplificar pesquisas em bancos de dados onde a pronúncia é conhecida mas não a soletração.

Esta função soundex em particular é a descrita por Donald Knuth em "The Art Of Computer Programming, vol. 3: Sorting And Searching", Addison-Wesley (1973), pp. 391-392.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a chave soundex como uma string com quatro caracteres. Se pelo menos uma letra estiver contida na `string`, a string retornada começa com uma letra. Caso contrário, `"0000"` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	Antes desta versão, chamar a função com uma string vazia retornava `false` por nenhuma razão em particular.

Examples

Exemplo #1 Exemplos de Soundex

```php
<?php
echo soundex("Euler"), PHP_EOL, soundex("Ellery"), PHP_EOL;

soundex("Euler")       == soundex("Ellery");    // E460
soundex("Gauss")       == soundex("Ghosh");     // G200
soundex("Hilbert")     == soundex("Heilbronn"); // H416
soundex("Knuth")       == soundex("Kant");      // K530
soundex("Lloyd")       == soundex("Ladd");      // L300
soundex("Lukasiewicz") == soundex("Lissajous"); // L222
?>
```

Veja também

- levenshtein() - Calcula a distância Levenshtein entre duas strings
- metaphone() - Calcula a chave de Metaphone de uma string
- similar_text() - Calcula a similaridade entre duas strings

sprintf

sprintf

(PHP 4, PHP 5, PHP 7, PHP 8)

sprintf — Retona uma string formatada

Descrição

function **sprintf**(string `$format`, mixed `...$values`): string

Retorna uma string produzida de acordo com a string de formatação informada no parâmetro `format`.

Parâmetros

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
-	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.
+	Prefixa números positivos com um sinal de mais +; Por padrão, apenas números negativos são prefixados com um sinal negativo.

Sinalizador	Descrição
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
`'`(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :
>
> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
`G`	Como o especificador `g`, mas usa `E` e `f`.
`h`	Como o especificador `g`, mas usa `F`. Disponível a partir do PHP 8.0.0.
`H`	Como o especificador `g`, mas usa `E` e `F`. Disponível a partir do PHP 8.0.0.
`o`	O argumento é tratado como um número inteiro e apresentado como um número octal.
`s`	O argumento é tratado e apresentado como uma string.
`u`	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
`x`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
`X`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**
>
> O especificador de tipo `c` ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna uma string produzida de acordo com a string de formatação informada no parâmetro `format`.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ArgumentCountError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, **`false`** era retornado e um **`E_WARNING`** era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não retorna mais **`false`** em caso de falha.
8.0.0	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um **`E_WARNING`**.
8.0.0	Lança um erro ValueError se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente, esta função emitia um **`E_WARNING`**.
8.0.0	Lança um erro ValueError se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente, esta função emitia um **`E_WARNING`**.
8.0.0	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um **`E_WARNING`**.

Examples

Exemplo #1 Troca de ordem dos argumentos

A string de formatação suporta troca/numeração de argumentos.

```php
<?php
$num = 5;
$location = 'árvore';

$format = 'Há %d macacos na %s';
echo sprintf($format, $num, $location);
?>
```

O exemplo acima produzirá:

```
Há 5 macacos na árvore
```

Entretanto imagine que uma string de formatação está sendo criada em um arquivo separado, normalmente porque ele poderia ser internacionalizado e seria re-escrito como:

Exemplo #2 Ordem incorreta de argumentos

A string de formato suporta numeração/troca de argumentos.

```
<?php
$num = 5;
$location = 'árvore';

$format = 'A %s contém %d macacos';
echo sprintf($format, $num, $location);
?>
```

Agora existe um problema. A ordem dos marcadores na string de formatação não corresponde à ordem dos argumentos no código. O código poderia ser deixado inalterado e na string de formatação poderiam ser indicados a que argumentos os marcadores se referem. A string de formatação seria então escrita assim:

Exemplo #3 Usando marcador de ordenação

```
<?php
$num = 5;
$location = 'árvore';

$format = 'A %2$s contém %1$d macacos';
echo sprintf($format, $num, $location);
?>
```

Uma vantagem adicional é que os marcadores podem ser repetidos sem adicionar mais argumentos no código.

Exemplo #4 Marcador repetido

```
<?php
$num = 5;
$location = 'árvore';

$format = 'A %2$s contém %1$d macacos.
           É uma bela %2$s cheia de %1$d macacos.';
echo sprintf($format, $num, $location);
?>
```

Ao usar troca ordem de argumentos, o *especificador de posição* `n$` precisa vir imediatamente depois do símbolo de porcentagem (`%`) e antes de qualquer outro especificador, como mostrado abaixo.

Exemplo #5 Especificando caractere de preenchimento

```
<?php
echo sprintf("%'.9d\n", 123);
echo sprintf("%'.09d\n", 123);
?>
```

O exemplo acima produzirá:

```
......123
000000123
```

Exemplo #6 Especificador de posição com outros especificadores

```php
<?php
$num = 5;
$location = 'árvore';

$format = 'A %2$s contém %1$04d macacos';
echo sprintf($format, $num, $location);
?>
```

O exemplo acima produzirá:

```
A árvore contém 0005 macacos
```

Exemplo #7 sprintf(): inteiros preenchidos com zero à esquerda

```php
<?php
$year = 2005;
$month = 5;
$day = 6;

$isodate = sprintf("%04d-%02d-%02d", $year, $month, $day);
echo $isodate, PHP_EOL;
?>
```

Exemplo #8 sprintf(): formatando moeda

```php
<?php
$money1 = 68.75;
$money2 = 54.35;
$money = $money1 + $money2;
echo $money, PHP_EOL;

$formatted = sprintf("%01.2f", $money);
echo $formatted, PHP_EOL;
?>
```

O exemplo acima produzirá:

```
123.1
123.10
```

Exemplo #9 sprintf(): notação científica

```php
<?php
$number = 362525200;

echo sprintf("%.3e", $number), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
3.625e+8
```

Veja também

- printf() - Envia uma string formatada para a saída
- fprintf() - Escreve uma string formatada para um fluxo
- vprintf() - Mostra uma string formatada
- vsprintf() - Retorna uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

sscanf

sscanf

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

sscanf — Interpreta a entrada de uma string de acordo com um formato

Descrição

function **sscanf**(string `$string`, string `$format`, mixed `&...$vars`): array|int|null

A função **sscanf()** é a análoga de entrada a printf(). **sscanf()** lê da string `string` e interpreta-a de acordo com o especificado em `format`.

Qualquer espaço em branco na string de formato corresponde a qualquer espaço em branco na string de entrada. Isto significa que até mesmo uma tabulação (`\t`) na string de formato pode corresponder a um caractere de espaço simples na string de entrada.

Parâmetros

`string`

A string de entrada a ser analisada.

`format`

O formato interpretado para `string`, que é descrito na documentação para sprintf() com as seguintes diferenças:

- A função não reconhece a localidade.
- `F`, `g`, `G` e `b` não são suportados.
- `D` significa número decimal.
- `i` significa inteiro com detecção de base.
- `n` representa o número de caracteres processados até o momento.
- `s` pára de ler em qualquer caractere de espaço em branco.
- `*` em vez de `argnum$` suprime a atribuição desta especificação de conversão.

`vars`

Variáveis opcionais passadas por referência que conterão os valores analisados.

Valor Retornado

Se apenas dois parâmetros forem passados para esta função, os valores interpretados serão retornados como um array. Caso contrário, se parâmetros opcionais forem passados, a função retornará o número de valores atribuídos. Os parâmetros opcionais precisam ser passados por referência.

Se houver mais substrings esperadas em `format` do que estiverem disponíveis na `string`, `null` será retornado.

Quando parâmetros opcionais são usados e o fim da `string` de entrada é alcançado antes que qualquer valor tenha sido interpretado, `-1` é retornado.

Examples

Exemplo #1 Exemplo de sscanf()

```php
<?php
// Pegando o número serial
list($serial) = sscanf("SN/2350001", "SN/%d");
// e a data de produção
```

```php
$mandate = "January 01 2000";
list($month, $day, $year) = sscanf($mandate, "%s %d %d");
echo "O item $serial foi produzido em: $year-" . substr($month, 0, 3) . "-$day\n";
?>
```

Se parâmetros opcionais forem passados, a função retornará o número de valores atribuídos.

Exemplo #2 sscanf() - usando parâmetros opcionais

```php
<?php
// pega informação do autor e gera uma entrada de DocBook
$auth = "24\tLewis Carroll";
$n = sscanf($auth, "%d\t%s %s", $id, $first, $last);
echo "<author id='$id'>
    <firstname>$first</firstname>
    <surname>$last</surname>
</author>\n";
?>
```

Veja também

- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada
- fprintf() - Escreve uma string formatada para um fluxo
- vprintf() - Mostra uma string formatada
- vsprintf() - Retorna uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

str_contains

str_contains

(PHP 8)

str_contains — Determina se uma string contém uma substring fornecida

Descrição

function **str_contains**(string `$haystack`, string `$needle`): bool

Realiza uma busca, sensível a maiúsculas/minúsculas, indicando se `needle` existe em `haystack`.

Parâmetros

`haystack`

A string onde será feita a busca.

`needle`

A substring para ser buscada em `haystack`.

Valor Retornado

Retorna `true` se `needle` está em `haystack`, `false` caso contrário.

Examples

Exemplo #1 Usando a string vazia `''`

```php
<?php
if (str_contains('abc', '')) {
    echo "Verificar a existência de uma string vazia sempre retornará true";
}
?>
```

O exemplo acima produzirá:

```
Verificar a existência de uma string vazia sempre retornará true
```

Exemplo #2 Mostrando a sensibilidade a maiúsculas/minúsculas

```php
<?php
$string = 'A raposa preguiçosa pulou a cerca';

if (str_contains($string, 'raposa')) {
    echo "A string 'raposa' foi encontrada na string\n";
}

if (str_contains($string, 'Raposa')) {
    echo 'A string "Raposa" foi encontrada na string';
} else {
    echo '"Raposa" não foi encontrada porque a letra maiúscula não corresponde';
}

?>
```

O exemplo acima produzirá:

```
A string 'raposa' foi encontrada na string
"Raposa" não foi encontrada porque a letra maiúscula não corresponde
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- str_ends_with() - Verifica se uma string termina com uma substring fornecida
- str_starts_with() - Verifica se uma string começa com uma substring fornecida
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string
- strpbrk() - Procura na string por um dos caracteres de um conjunto
- substr() - Retorna parte de uma string
- preg_match() - Realiza uma correspondência com expressão regular

str_decrement

str_decrement

(PHP 8 >= 8.3.0)

str_decrement — Decrementa uma string alfanumérica

Descrição

function **str_decrement**(string `$string`): string

Retorna a `string` ASCII alfanumérica decrementada.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string ASCII alfanumérica decrementada.

Erros/Exceções

Uma exceção ValueError é lançada se `string` estiver vazia.

Uma exceção ValueError é lançada se `string` não for uma string ASCII alfanumérica.

Uma exceção ValueError é lançada se `string` não puder ser decrementada. Por exemplo, `"A"` ou `"0"`.

Examples

Exemplo #1 Exemplo básico de str_decrement()

```php
<?php
$str = 'ABC';
var_dump(str_decrement($str));
?>
```

O exemplo acima produzirá:

```
string(3) "ABB"
```

Exemplo #2 Exemplo de str_decrement() "pegando emprestado"

```php
<?php
$str = 'ZA';
var_dump(str_decrement($str));

$str = 'AA';
var_dump(str_decrement($str));
?>
```

O exemplo acima produzirá:

```
string(2) "YZ"
string(1) "Z"
```

Veja também

- str_increment() - Incrementa uma string alfanumérica

str_ends_with

str_ends_with

(PHP 8)

str_ends_with — Verifica se uma string termina com uma substring fornecida

Descrição

function **str_ends_with**(string `$haystack`, string `$needle`): bool

Realiza uma verificação sensível a maiúsculas/minúsculas indicando se `haystack` termina com `needle`.

Parâmetros

`haystack`

A string onde será feita a busca.

`needle`

A substring para procurar em `haystack`.

Valor Retornado

Retorna **`true`** se `haystack` terminar com `needle`, **`false`** caso contrário.

Examples

Exemplo #1 Usando a string vazia `''`

```php
<?php
if (str_ends_with('abc', '')) {
    echo "Toda string termina com uma string vazia";
}
?>
```

O exemplo acima produzirá:

```
Toda string termina com uma string vazia
```

Exemplo #2 Mostrando a sensibilidade a maiúsculas/minúsculas

```php
<?php
$string = 'A raposa preguiçosa pulou a cerca';

if (str_ends_with($string, 'cerca')) {
    echo "A string termina com 'cerca'\n";
}

if (str_ends_with($string, 'Cerca')) {
    echo 'A string termina com "Cerca"';
} else {
    echo '"Cerca" não foi encontrada porque a letra maiúscula não corresponde';
}

?>
```

O exemplo acima produzirá:

```
A string termina com 'cerca'
"Cerca" não foi encontrada porque a letra maiúscula não corresponde
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- str_contains() - Determina se uma string contém uma substring fornecida
- str_starts_with() - Verifica se uma string começa com uma substring fornecida
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string
- strpbrk() - Procura na string por um dos caracteres de um conjunto
- substr() - Retorna parte de uma string
- preg_match() - Realiza uma correspondência com expressão regular

str_getcsv

str_getcsv

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

str_getcsv — Analisa uma string CSV e retorna os dados em um array

Descrição

```
function str_getcsv(
    string $string,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\"
): array
```

Interpreta uma string de entrada procurando por campos no formato CSV e retorna um array contendo os campos encontrados.

> **Nota** :
>
> As configurações de localidade são consideradas por esta função. Por exemplo, dados em certas codificações de um byte podem ser interpretados incorretamente se `LC_CTYPE` for `en_US.UTF-8`.

Parâmetros

`string`

A string a ser analisada.

`separator`

O parâmetro `separator` define o delimitador de campo. Precisa ser um caractere de um byte.

`enclosure`

O parâmetro `enclosure` define o caractere que cerca um campo. Precisa ser um caractere de um byte.

`escape`

O parâmetro `escape` define o caractere de escape. Precisa ser um caractere de um byte ou uma string vazia. Uma string vazia (`""`) desabilita o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado.

> Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados.

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

Valor Retornado

Retorna um array indexado contendo os campos lidos.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
8.4.0	Agora lança um ValueError se `separator`, `enclosure`, ou `escape` for inválido. Isto imita o comportamento de fgetcsv() e de fputcsv().
8.3.0	Uma string vazia é retornada ao invés de uma string com um único byte nulo para o último campo, se ele contiver apenas um demarcador não terminado.
7.4.0	O parâmetro `escape` agora interpreta uma string vazia como o sinal para desabilitar o mecanismo de escape. Antes, uma string vazia era tratada como valor padrão do parâmetro.

Examples

Exemplo #1 Exemplo de str_getcsv()

```php
<?php

$string = 'PHP,Java,Python,Kotlin,Swift';
$data = str_getcsv($string, escape: '\\');

var_dump($data);
?>
```

O exemplo acima produzirá:

```
array(5) {
  [0]=>
  string(3) "PHP"
  [1]=>
  string(4) "Java"
  [2]=>
  string(6) "Python"
  [3]=>
  string(6) "Kotlin"
  [4]=>
```

```
  string(5) "Swift"
}
```

Exemplo #2 Exemplo de str_getcsv()com uma string vazia

> **Cuidado**
>
> Com uma string vazia esta função retorna o valor `[null]` em vez de um array vazio.

```php
<?php

$string = '';
$data = str_getcsv($string, escape: '\\');

var_dump($data);
?>
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  NULL
}
```

Veja também

- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV

str_increment

str_increment

(PHP 8 >= 8.3.0)

str_increment — Incrementa uma string alfanumérica

Descrição

function **str_increment**(string `$string`): string

Retorna a `string` ASCII alfanumérica incrementada.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string ASCII alfanumérica incrementada.

Erros/Exceções

Uma exceção ValueError é lançada se `string` estiver vazia.

Uma exceção ValueError é lançada se `string` não for uma string ASCII alfanumérica.

Examples

Exemplo #1 Exemplo básico de str_increment()

```php
<?php
$str = 'ABC';
var_dump(str_increment($str));
?>
```

O exemplo acima produzirá:

```
string(3) "ABD"
```

Exemplo #2 Exemplo de str_increment() com "vai um"

```php
<?php
$str = 'DZ';
var_dump(str_increment($str));

$str = 'ZZ';
var_dump(str_increment($str));
?>
```

O exemplo acima produzirá:

```
string(2) "EA"
string(3) "AAA"
```

Veja também

- str_decrement() - Decrementa uma string alfanumérica

str_ireplace

str_ireplace

(PHP 5, PHP 7, PHP 8)

str_ireplace — Versão insensível a maiúsculas/minúsculas de str_replace()

Descrição

```
function str_ireplace(
    array|string $search,
    array|string $replace,
    string|array $subject,
    int &$count = null
): string|array
```

Esta função retorna uma string ou um array com todas as ocorrências de `search` em `subject` (ignorando maiúsculas e minúsculas) substituídas pelo valor `replace` fornecido.

Para substituir texto baseado em um padrão em vez de uma string fixa, use preg_replace() com o modificador de expressões regulares `i`.

Parâmetros

Se `search` e `replace` forem arrays, **str_ireplace()** pega um valor de cada array e os usa para pesquisar e substituir em `subject`. Se `replace` tiver menos valores que `search`, uma string vazia será usada para o restante dos valores de substituição. Se `search` for um array e `replace` for uma string, esta string de substituição será usada para cada valor de `search`. O inverso não faria sentido, no entanto.

Se `search` ou `replace` forem arrays, seus elementos serão processados do primeiro ao último.

`search`

O valor que está sendo pesquisado, também conhecido como *agulha*. Um array pode ser usado para designar múltiplas agulhas.

`replace`

O valor de substituição que substitui os valores de `search` encontrados. Um array pode ser usado para designar múltiplas substituições.

`subject`

A string ou array sendo pesquisado e substituído, também conhecido como o *palheiro*.

Se `subject` for um array, a pesquisa e a substituição são realizadas com cada entrada de `subject`, e o valor de retorno é também um array.

`count`

Se informado, será definido como o número de substituições realizadas.

Valor Retornado

Retorna uma string ou um array de substituições.

Registro de Alterações

Versão	Descrição
8.2.0	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale(). Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.

Examples

Exemplo #1 Exemplo de str_ireplace()

```php
<?php
$bodytag = str_ireplace("%body%", "black", "<body text=%BODY%>");
echo $bodytag; // <body text=black>
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

> **Cuidado**
>
> #### Armadilha na ordem de substituição
>
> Como **str_ireplace()** substitui da esquerda para a direita, ela pode substituir um valor inserido anteriormente ao realizar múltiplas substituições. O exemplo #2 na documentação da função str_replace() demonstra o efeito disto na prática.

Veja também

- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- strtr() - Traduz caracteres ou substitui substrings

str_pad

str_pad

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

str_pad — Preenche uma string até um determinado comprimento com outra string

Descrição

```
function str_pad(
    string $string,
    int $length,
    string $pad_string = " ",
    int $pad_type = STR_PAD_RIGHT
): string
```

Esta função retorna a string `string` preenchida à esquerda, à direita ou em ambos os lados até o comprimento de preenchimento especificado. Se o argumento opcional `pad_string` não for fornecido, a `string` será preenchida com espaços, caso contrário será preenchida com caracteres de `pad_string` até o limite.

Parâmetros

`string`

A string de entrada.

`length`

O comprimento desejado da sequência preenchida final. Se o valor de `length` for negativo, menor ou igual ao comprimento da string de entrada, nenhum preenchimento ocorrerá a `string` original será retornada.

`pad_string`

> **Nota** :
>
> O `pad_string` pode ser truncado se o número necessário de caracteres de preenchimento não puder ser dividido igualmente pelo comprimento do `pad_string`.

`pad_type`

O argumento opcional `pad_type` pode ser `STR_PAD_RIGHT`, `STR_PAD_LEFT` ou `STR_PAD_BOTH`. Se `pad_type` não for especificado, será assumido como `STR_PAD_RIGHT`.

Valor Retornado

Retorna a string preenchida.

Examples

Exemplo #1 Exemplo de str_pad()

```php
<?php
$input = "Alien";
echo str_pad($input, 10), PHP_EOL;                      // produz "Alien     "
echo str_pad($input, 10, "-=", STR_PAD_LEFT), PHP_EOL;  // produz "-=-=-Alien"
echo str_pad($input, 10, "_", STR_PAD_BOTH), PHP_EOL;   // produz "__Alien___"
echo str_pad($input,  6, "___"), PHP_EOL;               // produz "Alien_"
echo str_pad($input,  3, "*"), PHP_EOL;                 // produz "Alien"
?>
```

Veja também

- mb_str_pad() - Preenche uma string multibyte para um comprimento específico com outra string multibyte

str_repeat

str_repeat

(PHP 4, PHP 5, PHP 7, PHP 8)

str_repeat — Repete uma string

Descrição

function **str_repeat**(string `$string`, int `$times`): string

Retorna a `string` repetida pelo número de vezes definido em `times`.

Parâmetros

`string`

A string a ser repetida.

`times`

Número de vezes que a `string` será repetida.

`times` deve ser maior ou igual a 0. Se o parâmetro `times` for 0, a função irá retornar uma string vazia.

Valor Retornado

Retorna a string repetida.

Examples

Exemplo #1 Exemplo de str_repeat()

```php
<?php
echo str_repeat("-=", 10);
?>
```

O exemplo acima produzirá:

```
-=-=-=-=-=-=-=-=-=-=
```

Veja também

- for
- str_pad() - Preenche uma string até um determinado comprimento com outra string
- substr_count() - Conta o número de ocorrências de uma substring

str_replace

str_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

str_replace — Substitui todas as ocorrências da string de pesquisa com a string de substituição

Descrição

```
function str_replace(
    array|string $search,
    array|string $replace,
    string|array $subject,
    int &$count = null
): string|array
```

Esta função retorna uma string ou um array com todas as ocorrências de `search` em `subject` substituídas com o valor informado em `replace`.

Para substituir texto baseado em um padrão que não seja uma string fixa, use preg_replace().

Parâmetros

Se `search` e `replace` forem arrays, então **str_replace()** pega um valor de cada array e os usa para pesquisar e substituir em `subject`. Se `replace` tiver menos valores do que `search`, então uma string vazia é usada para o restante dos valores de substituição. Se `search` for um array e `replace` for uma string, então esta string de substituição é usada para cada valor de `search`. O contrário não faria sentido, contudo.

Se `search` ou `replace` forem arrays, seus elementos são processados do primeiro ao último.

`search`

O valor sendo pesquisado, também conhecido como *agulha*. Um array pode ser usado para designar múltiplas agulhas.

`replace`

O valor de substituição que será usado no lugar dos valores de `search` encontrados. Um array pode ser usado para deignar múltiplas substituições.

`subject`

A string ou array sendo pesquisado ou substituído, também conhecido como o *palheiro*.

Se `subject` for um array, então a pesquisa e a substituição são executadas com cada entrada de `subject`, e o valor de retorno é um array também.

`count`

Se passado, será definido para o número de substituições realizadas.

Valor Retornado

Esta função retorna uma string ou um array com os valores substituídos.

Examples

Exemplo #1 Exemplos básicos de str_replace()

```php
<?php
// Produz: <body text='black'>
$bodytag = str_replace("%body%", "black", "<body text='%body%'>");
echo $bodytag, PHP_EOL;

// Produz: Hll Wrld f PHP
$vowels = array("a", "e", "i", "o", "u", "A", "E", "I", "O", "U");
$onlyconsonants = str_replace($vowels, "", "Hello World of PHP");
echo $onlyconsonants, PHP_EOL;

// Produz: Você deveria comer pizza, cerveja e sorvete todos os dias.
$frase    = "Você deveria comer frutas, vegetais, e fibra todos os dias.";
$saudavel = array("frutas", "vegetais", "fibra");
$saboroso = array("pizza", "cerveja", "sorvete");

$novafrase = str_replace($saudavel, $saboroso, $frase);
echo $novafrase, PHP_EOL;

// Produz: 2
$str = str_replace("ll", "", "good golly miss molly!", $count);
echo $count, PHP_EOL;
?>
```

Exemplo #2 Exemplos de armadilhas potenciais de str_replace()

```php
<?php
// Ordem da substituição
$str     = "Linha 1\nLinha 2\rLinha 3\r\nLinha 4\n";
$order   = array("\r\n", "\n", "\r");
$replace = '<br />';

// Processa \r\n primeiro para que não sejam substituídos duas vezes.
$newstr = str_replace($order, $replace, $str);
echo $newstr, PHP_EOL;
```

```php
// Mostra F porque A é substituído por B, depois B é substituído por C, e assim por diante...
// Finalmente E é substituído por F, porque a substituição é feita da esquerda para a direita.
$search  = array('A', 'B', 'C', 'D', 'E');
$replace = array('B', 'C', 'D', 'E', 'F');
$subject = 'A';
echo str_replace($search, $replace, $subject), PHP_EOL;

// Mostra: apearpearle pear
// Pelo memos motivo mencionado acima
$letters = array('a', 'p');
$fruit   = array('apple', 'pear');
$text    = 'a p';
$output  = str_replace($letters, $fruit, $text);
echo $output, PHP_EOL;
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

> **Cuidado**
>
> #### Armadilha da ordem de substituição
>
> Como **str_replace()** substitui da esquerda para a direita, ela pode substituir um valor previamente inserido quando estiver fazendo múltiplas substituições. Veja também os exemplos neste documento.

> **Nota** :
>
> Esta função é sensível a maiúsculas/minúsculas. Use str_ireplace() para substituição insensível a maiúsculas/minúsculas.

Veja também

- str_ireplace() - Versão insensível a maiúsculas/minúsculas de str_replace
- substr_replace() - Substitui o texto dentro de uma parte de uma string
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- strtr() - Traduz caracteres ou substitui substrings

str_rot13

str_rot13

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

str_rot13 — Executa a transformação rot13 em uma string

Descrição

function **str_rot13**(string `$string`): string

Executa a codificação ROT13 no argumento `string` e retorna a string resultante.

A codificação ROT13 simplesmente troca cada letra por 13 lugares no alfabeto, enquanto não muda os caracteres não alfabéticos. Codificação e decodificação são feitos pela mesma função, passando uma string codificada como argumento retornará a string com a versão original.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a versão ROT13 de uma dada string.

Examples

Exemplo #1 Exemplo de str_rot13()

```php
<?php

echo str_rot13('PHP 8.3.0'); // CUC 8.3.0

?>
```

str_shuffle

str_shuffle

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

str_shuffle — Embaralha uma string aleatoriamente

Descrição

function **str_shuffle**(string `$string`): string

str_shuffle() embaralha uma string. Uma permutação de todas as possíveis é criada.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**
>
> Esta função usa a instância global do Mt19937 (“Mersenne Twister”) como a fonte de aleatoriedade a por isso compartilha seu estado com todas as outras funções que usam o Mt19937 global. Usar qualquer uma destas funções avança a sequência para *todas* as outras funções, independentemente do escopo.
>
> Gerar sequências repetíveis ao semear mt_srand() ou srand() com um valor conhecido também gerará saída repetível desta função.
>
> Prefira usar métodos de Random\Randomizer em todo código novo.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string embaralhada.

Registro de Alterações

Versão	Descrição
7.1.0	O algoritmo interno de randomização mudou e agora usa o Gerador de Números Aleatórios » Mersenne Twister em vez da função rand da biblioteca libc.

Examples

Exemplo #1 Exemplo de str_shuffle()

```php
<?php
$str = 'abcdef';
```

```php
$shuffled = str_shuffle($str);

// Isto exibirá algo como: bfdaec
echo $shuffled;
?>
```

Veja também

- Random\Randomizer::shuffleBytes() - Obtém uma permutação byte a byte de uma string
- Random\Randomizer::shuffleArray() - Obtém uma permutação de um array

str_split

str_split

(PHP 5, PHP 7, PHP 8)

str_split — Converte uma string em um array

Descrição

function **str_split**(string `$string`, int `$length` = 1): array

Converte uma string em um array.

Parâmetros

`string`

A string de entrada.

`length`

Comprimento máximo do pedaço.

Valor Retornado

Se o parâmetro opcional `length` for especificado, o array retornado será dividido em pedaços, sendo cada um deles com `length` de comprimento, exceto o pedaço final que pode ser mais curto se a string não for divisível uniformemente. O `length` padrão é `1`, o que significa que cada pedaço terá um byte de tamanho.

Erros/Exceções

Se `length` for menor que `1`, uma exceção ValueError será lançada.

Registro de Alterações

Versão	Descrição
8.2.0	Se `string` estiver vazio, um array agora é retornado. Anteriormente, um array contendo uma única string vazia foi retornada.
8.0.0	Se `length` for menor que `1`, uma exceção ValueError será lançada agora; anteriormente, um erro de nível `E_WARNING` era gerado e a função retornava `false`.

Examples

Exemplo #1 Exemplos de uso de str_split()

```php
<?php

$str = "Tchau Amigos";

$arr1 = str_split($str);
$arr2 = str_split($str, 3);

print_r($arr1);
print_r($arr2);

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => T
    [1] => c
    [2] => h
    [3] => a
    [4] => u
    [5] =>
    [6] => A
    [7] => m
    [8] => i
    [9] => g
    [10] => o
    [11] => s
)

Array
(
    [0] => Tch
    [1] => au
    [2] => Ami
    [3] => gos
)
```

Notas

> **Nota** :
>
> **str_split()** dividirá em bytes, em vez de caracteres, ao lidar com uma string codificada em múltiplos bytes. mb_str_split() pode ser usada para dividir a string em pontos de código. grapheme_str_split() pode ser usada para dividir a string em grupos de grafema.

Veja também

- mb_str_split() - Dada uma string multibyte, retorna um array de seus caracteres
- grapheme_str_split() - Separa uma string em um array
- chunk_split() - Divide uma string em pedaços menores
- preg_split() - Divide uma string com base em expressão regular
- explode() - Divide uma string com base em outra string
- count_chars() - Retorna informação sobre caracteres usados em uma string
- str_word_count() - Retorna informação sobre palavras usadas em uma string
- for

str_starts_with

str_starts_with

(PHP 8)

str_starts_with — Verifica se uma string começa com uma substring fornecida

Descrição

function **str_starts_with**(string `$haystack`, string `$needle`): bool

Realiza uma verificação sensível a maiúsculas/minúsculas indicando se `haystack` começa com `needle`.

Parâmetros

`haystack`

A string onde será feita a busca.

`needle`

A substring para procurar em `haystack`.

Valor Retornado

Retorna **`true`** se `haystack` começa com `needle`, **`false`** caso contrário.

Examples

Exemplo #1 Usando uma string vazia `''`

```php
<?php
if (str_starts_with('abc', '')) {
    echo "Toda string começa com uma string vazia";
}
?>
```

O exemplo acima produzirá:

```
Toda string começa com uma string vazia
```

Exemplo #2 Mostrando a sensibilidade a maiúsculas/minúsculas

```php
<?php
$string = 'Uma raposa preguiçosa pulou a cerca';

if (str_starts_with($string, 'Uma')) {
    echo "A string começa com 'Uma'\n";
}

if (str_starts_with($string, 'uma')) {
    echo 'A string começa com "uma"';
} else {
    echo '"uma" não foi encontrada porque a letra maiúscula não corresponde';
}

?>
```

O exemplo acima produzirá:

```
A string começa com 'Uma'
"uma" não foi encontrada porque a letra maiúscula não corresponde
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- str_contains() - Determina se uma string contém uma substring fornecida
- str_ends_with() - Verifica se uma string termina com uma substring fornecida
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string

- strpbrk() - Procura na string por um dos caracteres de um conjunto
- substr() - Retorna parte de uma string
- preg_match() - Realiza uma correspondência com expressão regular

str_word_count

str_word_count

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

str_word_count — Retorna informação sobre palavras usadas em uma string

Descrição

function **str_word_count**(string `$string`, int `$format` = 0, ?string `$characters` = `null`): array|int

Conta o número de palavras dentro de `string`. Se o parâmetro opcional `format` não for especificado, então o valor de retorno será um inteiro representando o número de palavras encontradas. Caso `format` seja especificado, o valor de retorno será um array, cujo conteúdo é dependente de `format`. Os valores possíveis para `format` e as saídas resultantes estão listadas abaixo.

Para o propósito desta função, 'palavra' é definida como uma string dependente da localidade contendo caracteres alfabéticos, que também podem conter, mas não iniciar com os caracteres "'" e "-". Note que localidades que utilizam caracteres multibyte não são suportadas.

Parâmetros

`string`

A string

`format`

Especifica o valor de retorno desta função. Os valores atualmente suportados são:

- 0 - retorna o número de palavras encontradas
- 1 - retorna um array contendo todas as palavras encontradas dentro da `string`
- 2 - retorna um array associativo, onde a chave é a posição numérica da palavra dentro da `string` e o valor é a própria palavra

`characters`

Uma lista de caracteres adicionais que serão considerados como 'palavra'.

Valor Retornado

Retorna um array ou um inteiro, dependendo do parâmetro `format`.

Registro de Alterações

Versão	Descrição
8.0.0	`characters` agora pode ser nulo.

Examples

Exemplo #1 Um exemplo de str_word_count()

```php
<?php

$str = "Hello fri3nd, you're
       looking          good today!";

print_r(str_word_count($str, 1));
print_r(str_word_count($str, 2));
print_r(str_word_count($str, 1, 'àáãç3'));

echo str_word_count($str);

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Hello
    [1] => fri
    [2] => nd
    [3] => you're
    [4] => looking
    [5] => good
    [6] => today
)
Array
(
    [0] => Hello
    [6] => fri
    [10] => nd
    [14] => you're
    [28] => looking
    [45] => good
    [50] => today
)
Array
(
    [0] => Hello
    [1] => fri3nd
    [2] => you're
    [3] => looking
    [4] => good
    [5] => today
)
7
```

Veja também

- explode() - Divide uma string com base em outra string
- preg_split() - Divide uma string com base em expressão regular
- count_chars() - Retorna informação sobre caracteres usados em uma string
- substr_count() - Conta o número de ocorrências de uma substring

strcasecmp

strcasecmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strcasecmp — Comparação binária segura de strings insensível a maiúsculas/minúsculas

Descrição

function **strcasecmp**(string `$string1`, string `$string2`): int

Comparação binária segura de strings insensível a maiúsculas/minúsculas. A comparação não leva em conta a localidade; apenas letras ASCII são comparadas de forma insensível a maiúsculas/minúsculas.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Exemplo #1 Exemplo de strcasecmp()

```php
<?php
$var1 = "Olá";
$var2 = "olá";
if (strcasecmp($var1, $var2) == 0) {
    echo '$var1 é igual a $var2 em uma comparação de strings insensível a maiúsculas/minúsculas';
}
?>
```

Veja também

- strcmp() - Comparação binária segura de strings
- preg_match() - Realiza uma correspondência com expressão regular
- substr_compare() - Comparação binária de duas strings a partir de uma posição até n caracteres
- strncasecmp() - Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres
- stristr() - strstr insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string

strchr

strchr

(PHP 4, PHP 5, PHP 7, PHP 8)

strchr — Sinônimo de strstr()

Descrição

Esta função é um apelido para: strstr().

strcmp

strcmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strcmp — Comparação binária segura de strings

Descrição

function **strcmp**(string `$string1`, string `$string2`): int

Observe que esta comparação é sensível a maiúsculas/minúsculas. Para comparação insensível a maiúsculas/minúsculas, consulte strcasecmp().

Observe que esta comparação não considera a localidade. Para comparação que considera a localidade, consulte strcoll() ou Collator::compare().

Parâmetros

`string1`

A primeira string

`string2`

A segunda string

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Exemplo #1 Exemplo de **strcmp()**

```
<?php
$var1 = "Olá";
$var2 = "olá";
if (strcmp($var1, $var2) !== 0) {
    echo '$var1 não é igual a $var2 em uma comparação sensível a maiúsculas/minúsculas';
}
?>
```

Veja também

- Comparações de string completa
  - strcasecmp() - Comparação binária segura de strings insensível a maiúsculas/minúsculas
  - Collator::compare() - Compara duas strings Unicode
  - strcoll() - Comparação de strings baseada em localidade
- Comparações de string parcial
  - substr_compare() - Comparação binária de duas strings a partir de uma posição até n caracteres
  - strncmp() - Comparação de strings segura para binários dos primeiros n caracteres
  - strstr() - Encontra a primeira ocorrência de uma string
- Outras comparações de string / similares
  - preg_match() - Realiza uma correspondência com expressão regular
  - levenshtein() - Calcula a distância Levenshtein entre duas strings
  - metaphone() - Calcula a chave de Metaphone de uma string
  - similar_text() - Calcula a similaridade entre duas strings
  - soundex() - Calcula a chave soundex de uma string

strcoll

strcoll

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

strcoll — Comparação de strings baseada em localidade

Descrição

function **strcoll**(string `$string1`, string `$string2`): int

Observe que esta comparação é sensível a maiúsculas/minúsculas, e diferentemente de strcmp(), esta função não é segura para binários.

strcoll() usa a localidade atual para fazer as comparações. Se a localidade atual é C ou POSIX, esta função é equivalente a strcmp().

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

Valor Retornado

Retorna < 0 se `string1` for abaixo de `string2`; > 0 se `string1` for acima de `string2`, e 0 se forem iguais.

Veja também

- preg_match() - Realiza uma correspondência com expressão regular
- strcmp() - Comparação binária segura de strings
- strcasecmp() - Comparação binária segura de strings insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string
- stristr() - strstr insensível a maiúsculas/minúsculas
- strncasecmp() - Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres
- strncmp() - Comparação de strings segura para binários dos primeiros n caracteres
- strstr() - Encontra a primeira ocorrência de uma string
- setlocale() - Define informação de localidade

strcspn

strcspn

(PHP 4, PHP 5, PHP 7, PHP 8)

strcspn — Encontra o tamanho do segmento inicial que não corresponde à máscara

Descrição

```
function strcspn(
    string $string,
    string $characters,
    int $offset = 0,
    ?int $length = null
): int
```

Retorna o tamanho do segmento inicial de `string` que *não* contiver nenhum dos caracteres definidos no parâmetro `characters`.

Se `offset` e `length` forem omitidos, então tudo da `string` será examinado. Se forem incluídos, então o efeito será o mesmo que chamar `strcspn(substr($string, $offset, $length), $characters)` (consulte substr para mais informações).

Parâmetros

`string`

O string a ser examinado.

`characters`

A string contendo os caracteres não permitidos.

`offset`

A posição na `string` para iniciar a pesquisa.

Se `offset` for informado e não negativo, a função **strcspn()** começará a examinar a `string` na posição `offset`. Por exemplo, na string '`abcdef`', o caractere na posição `0` é '`a`', na posição `2` é '`c`', e assim por diante.

Se `offset` for informado e negativo, a função **strcspn()** começará a examinar a `string` na posição `offset` a partir do final da `string`.

`length`

O tamanho do segmento da `string` a ser examinado.

Se `length` for informado e não negativo, a `string` será examinada pelo número de caracteres em `length` a partir da posição inicial.

Se `length` for informado e negativo, a `string` será examinada a partir da posição inicial até o número de caracteres informado em `length` contados a partir do final da `string`.

Valor Retornado

Retorna o tamanho do segmento inicial da `string` que consiste inteiramente de caracteres que *não* estejam em `characters`.

> **Nota** :
>
> Quando `offset` estiver definido, o tamanho retornado é contado a partir deste posição, e não a partir do início da `string`.

Registro de Alterações

Versão	Descrição
8.4.0	Antes do PHP 8.4.0, quando `characters` fosse uma string vazia, a pesquisa incorretamente encerraria no primeiro byte nulo em `string`.
8.0.0	`length` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de strcspn()

```php
<?php
$a = strcspn('banana', 'a');
$b = strcspn('banana', 'abcd');
$c = strcspn('banana', 'z');
$d = strcspn('abcdhelloabcd', 'a', -9);
$e = strcspn('abcdhelloabcd', 'a', -9, -5);

var_dump($a);
var_dump($b);
var_dump($c);
var_dump($d);
var_dump($e);
?>
```

O exemplo acima produzirá:

```
int(1)
int(0)
int(6)
int(5)
int(4)
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strspn() - Encontra o comprimento do segmento inicial de uma string composta totalmente de caracteres contidos em uma máscara informada

strip_tags

strip_tags

(PHP 4, PHP 5, PHP 7, PHP 8)

strip_tags — Retira as tags HTML e PHP de uma string

Descrição

function **strip_tags**(string `$string`, array|string|null `$allowed_tags` = `null`): string

Esta função tenta retornar uma string com todos os bytes nulos e tags HTML e PHP removidas da `string` informada. Usa a mesma máquina de estados de remoção de tags que a função fgetss().

Parâmetros

`string`

A string de entrada.

`allowed_tags`

Pode-se utilizar o segundo parâmetro, que é opcional, para indicar tags que não devam ser retiradas. Podem ser informadas como string, ou a partir do PHP 7.4.0, como array. Refira-se ao exemplo abaixo sobre o formato deste parâmetro.

> **Nota** :
>
> Comentários HTML e tags PHP também são retirados. Isto não pode ser modificado com `allowed_tags`.

> **Nota** :
>
> Tags XHTML com auto-fechamento são ignoradas e apenas tags sem auto-fechamento devem ser usadas em `allowed_tags`. Por exemplo, para permitir tanto `<br>` quanto `<br/>`, deve-se usar:
>
> ```php
> <?php
> strip_tags($input, '<br>');
> ?>
> ```

Valor Retornado

Retorna a string modificada.

Registro de Alterações

Versão	Descrição
8.0.0	`allowed_tags` agora pode ser nulo.
7.4.0	O parâmetro `allowed_tags` agora alternativamente aceita um array.

Examples

Exemplo #1 Exemplo de **strip_tags()**

```php
<?php
$text = '<p>Test paragraph.</p><!-- Comment --> <a href="#fragment">Other text</a>';
echo strip_tags($text);
echo "\n";

// Permite <p> e <a>
echo strip_tags($text, '<p><a>');

// A partir do PHP 7.4.0 a linha acima pode ser escrita como:
// echo strip_tags($text, ['p', 'a']);
?>
```

O exemplo acima produzirá:

```
Test paragraph. Other text
<p>Test paragraph.</p> <a href="#fragment">Other text</a>
```

Notas

> **Aviso**
>
> Esta função não deve ser usada para evitar ataques XSS. Use funções mais apropriadas como htmlspecialchars() ou outros meios dependendo do contexto da saída.

> **Aviso**
>
> Como **strip_tags()** não valida realmente o HTML, tags parciais ou quebradas podem resultar na remoção de mais texto e dados que o esperado.

> **Aviso**
>
> Esta função não modifica nenhum dos atributos das tags que forem permitidas usando `allowed_tags`, incluindo os atributos `style` e `onmouseover` que um usuário malicioso pode abusar ao enviar texto que será mostrado a outros usuários.

> **Nota** :
>
> Nomes de tags no HTML de entrada que forem maiores que 1023 bytes em comprimento serão tratados como se fossem inválidos, independente, do parâmetro `allowed_tags`.

Veja também

- htmlspecialchars() - Converte caracteres especiais para entidades HTML

stripcslashes

stripcslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

stripcslashes — Remove o escape de strings escapadas com addcslashes()

Descrição

function **stripcslashes**(string `$string`): string

Retorna uma string com as barras invertidas retiradas. Reconhece representações no estilo C `\n`, `\r` ..., octais e hexadecimais.

Parâmetros

`string`

A string para remover os escapes.

Valor Retornado

Retorna a string com escapes retirados.

Examples

Exemplo #1 Exemplo de stripcslashes()

```php
<?php

var_dump(stripcslashes('I\'d have a coffee.\nNot a problem.') === "I'd have a coffee.
Not a problem."); // verdadeiro
?>
```

Veja também

- addcslashes() - Escapa string com barras invertidas no estilo C

stripos

stripos

(PHP 5, PHP 7, PHP 8)

stripos — Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas

Descrição

function **stripos**(string `$haystack`, string `$needle`, int `$offset = 0`): int|false

Encontra a posição da primeira ocorrência de `needle` na string `haystack`.

Diferentemente de strpos(), **stripos()** é insensível a maiúsculas/minúsculas.

Parâmetros

`haystack`

A string onde a substring será procurada.

`needle`

A substring sendo pesquisada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`offset`

Se especificado, a pesquisa iniciará nesse número de caracteres contados a partir do início da string. Se o deslocamento for negativo, a busca iniciará nesse número de caracteres contados a partir do final da string.

Valor Retornado

Retorna a posição onde a substring pesquisada existe em relação ao início da string `haystack` (independente do deslocamento). Observe também que as posições das strings começam em 0 e não em 1.

Retorna `false` se a substring não for encontrada.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.2.0	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale(). Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar um int em `needle` não é mais suportado.
7.3.0	Passar um int em `needle` foi descontinuado.
7.1.0	Suporte para valores negativos em `offset` foi adicionado.

Examples

Exemplo #1 Exemplos de stripos()

```php
<?php
$findme    = 'a';
$mystring1 = 'xyz';
$mystring2 = 'ABC';

$pos1 = stripos($mystring1, $findme);
$pos2 = stripos($mystring2, $findme);

// Não, 'a' certamente não existe em 'xyz'
if ($pos1 === false) {
    echo "A string '$findme' não foi encontrada na string '$mystring1'", PHP_EOL;
}

// Observe o uso de !==.  Simplesmente != não iria funcionar como esperado
// porque a posição de 'a' é na posição 0 (primeiro) caractere.
if ($pos2 !== false) {
    echo "Encontrado '$findme' em '$mystring2' na posição $pos2", PHP_EOL;
}
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- mb_stripos() - Encontra a posição da primeira ocorrência de uma string dentro de outra, sem diferenciar maiúsculas de minúsculas
- str_contains() - Determina se uma string contém uma substring fornecida
- str_ends_with() - Verifica se uma string termina com uma substring fornecida
- str_starts_with() - Verifica se uma string começa com uma substring fornecida
- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- stristr() - strstr insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string
- str_ireplace() - Versão insensível a maiúsculas/minúsculas de str_replace

stripslashes

stripslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

stripslashes — Desfaz os escapes de uma string escapada

Descrição

function **stripslashes**(string `$string`): string

Remove barras invertidas de uma string.

stripslashes() pode ser utilizada quando você não está inserindo estes dados em um lugar (como um banco de dados) que precise de escapar. Por exemplo, se você esta usando estes dados para gerar um formulário HTML.

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna uma string com as barras invertidas retiradas. (`\'` se torna `'` e assim por diante.) Duas barras invertidas (`\\`) se tornam uma (`\`).

Examples

Exemplo #1 Exemplo stripslashes()

```php
<?php
$str = "Seu nome é O\'reilly?";

// Mostra: Seu nome é O'reilly?
echo stripslashes($str);
?>
```

> **Nota** :
>
> **stripslashes()** não é recursiva. Se você quer aplicar esta função a um array multi-dimensional, você precisa usar uma função recursiva.

Exemplo #2 Usando stripslashes() em um array

```php
<?php
function stripslashes_deep($value)
{
    $value = is_array($value) ?
                array_map('stripslashes_deep', $value) :
                stripslashes($value);

    return $value;
}

// Example
$array = array("f\\'oo", "b\\'ar", array("fo\\'o", "b\\'ar"));
$array = stripslashes_deep($array);

// Output
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => f'oo
    [1] => b'ar
    [2] => Array
        (
            [0] => fo'o
            [1] => b'ar
        )

)
```

Veja também

- addslashes() - Adiciona barras invertidas a uma string
- get_magic_quotes_gpc() - Obtém a configuração atual de magic_quotes_gpc

stristr

stristr

(PHP 4, PHP 5, PHP 7, PHP 8)

stristr — strstr() insensível a maiúsculas/minúsculas

Descrição

function **stristr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

Retorna todo o `haystack` iniciando a partir de (e incluindo) a primeira ocorrência de `needle` até o final.

Parâmetros

`haystack`

A string onde será feita a pesquisa.

`needle`

A string que está sendo procurada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`before_needle`

Se igual a `true`, **stristr()** retorna a parte de `haystack` antes da primeira ocorrência de `needle` (excluindo-o).

`needle` e `haystack` são examinados de maneira insensível a maiúsculas/minúsculas.

Valor Retornado

Retorna a substring correspondente. Se `needle` não for encontrada, retorna `false`.

Registro de Alterações

Versão	Descrição
8.2.0	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale(). Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar um int em `needle` não é mais suportado.
7.3.0	Passar um int em `needle` foi descontinuado.

Examples

Exemplo #1 Exemplo de stristr()

```
<?php
  $email = 'USER@EXAMPLE.com';
  echo stristr($email, 'e'), PHP_EOL; // mostra ER@EXAMPLE.com
  echo stristr($email, 'e', true), PHP_EOL; // mostra US
?>
```

Exemplo #2 Testando se uma string é encontrada ou não

```
<?php
  $string = 'Olá Mundo!';
  if (stristr($string, 'Terra') === FALSE) {
    echo '"Terra" não encontrado na string';
  }
// mostra: "Terra" não encontrado na string
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strstr() - Encontra a primeira ocorrência de uma string
- strrchr() - Encontra a última ocorrência de um caractere em uma string
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strpbrk() - Procura na string por um dos caracteres de um conjunto
- preg_match() - Realiza uma correspondência com expressão regular

strlen

strlen

(PHP 4, PHP 5, PHP 7, PHP 8)

strlen — Retorna o tamanho de uma string

Descrição

function **strlen**(string `$string`): int

Retorna o tamanho da `string` informada.

Parâmetros

`string`

A string a ter seu tamanho medido.

Valor Retornado

O tamanho da `string` em bytes.

Examples

Exemplo #1 Exemplo de strlen()

```php
<?php
$str = 'abcdef';
echo strlen($str), PHP_EOL; // 6

$str = ' ab cd ';
echo strlen($str), PHP_EOL; // 7
?>
```

Notas

> **Nota** :
>
> **strlen()** retorna o número de bytes em vez do número de caracteres de uma string.

Veja também

- count() - Conta todos os elementos de um array ou de um objeto Countable
- grapheme_strlen() - Obtém o comprimento de uma string em unidade de grafema
- iconv_strlen() - Retorna a contagem de caracteres da string
- mb_strlen() - Obtém o comprimento da string

strnatcasecmp

strnatcasecmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strnatcasecmp — Comparação de strings insensível a maiúsculas/minúsculas usando o algoritmo de "ordem natural"

Descrição

function **strnatcasecmp**(string `$string1`, string `$string2`): int

Esta função implementa um algoritmo de comparação que ordena strings alfanuméricos da maneira que um ser humano faria. O comportamento desta função é similar a strnatcmp(), exceto que a comparação não é sensível a maiúsculas/minúsculas. Para maiores informações consulte: » Comparação de Strings em Ordem Natural, de Martin Pool.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Exemplo #1 Exemplo de **strnatcasecmp()**

```
<?php

var_dump(strnatcasecmp('Apple', 'Banana'));
var_dump(strnatcasecmp('Banana', 'Apple'));
var_dump(strnatcasecmp('apple', 'Apple'));
?>
```

O exemplo acima produzirá:

```
int(-1)
int(1)
int(0)
```

Veja também

- preg_match() - Realiza uma correspondência com expressão regular
- strcmp() - Comparação binária segura de strings
- strcasecmp() - Comparação binária segura de strings insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string
- stristr() - strstr insensível a maiúsculas/minúsculas
- strncasecmp() - Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres
- strncmp() - Comparação de strings segura para binários dos primeiros n caracteres
- strstr() - Encontra a primeira ocorrência de uma string
- setlocale() - Define informação de localidade

strnatcmp

strnatcmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strnatcmp — Comparações de strings usando um algoritmo de "ordem natural"

Descrição

function **strnatcmp**(string `$string1`, string `$string2`): int

Esta função implementa um algoritmo de comparação que ordena strings alfanuméricas da mesma forma que um ser humano faria, o que é descrito como "ordenação natural". Observe que esta comparação é sensível a maiúsculas/minúsculas.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Um exemplo da diferença entre este algoritmo e os algoritmos de ordenação de strings comuns (usados em strcmp()) pode ser visto abaixo:

Exemplo #1 strcmp()

```php
<?php
$arr1 = $arr2 = array("img12.png", "img10.png", "img2.png", "img1.png");
echo "Comparação padrão de strings\n";
usort($arr1, "strcmp");
print_r($arr1);
echo "\nComparação de strings em ordem natural\n";
usort($arr2, "strnatcmp");
print_r($arr2);
?>
```

O exemplo acima produzirá:

```
Comparação padrão de strings
Array
(
    [0] => img1.png
    [1] => img10.png
    [2] => img12.png
    [3] => img2.png
)

Comparação de strings em ordem natural
Array
(
    [0] => img1.png
    [1] => img2.png
    [2] => img10.png
    [3] => img12.png
)
```

Para mais informações, consulte a página » Comparação de String em Ordem Natural, de Martin Pool.

Veja também

- preg_match() - Realiza uma correspondência com expressão regular
- strcasecmp() - Comparação binária segura de strings insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string
- stristr() - strstr insensível a maiúsculas/minúsculas
- strcmp() - Comparação binária segura de strings
- strncmp() - Comparação de strings segura para binários dos primeiros n caracteres
- strncasecmp() - Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres
- strnatcasecmp() - Comparação de strings insensível a maiúsculas/minúsculas usando o algoritmo de "ordem natural"
- strstr() - Encontra a primeira ocorrência de uma string
- natsort() - Ordena um array utilizando o algoritmo de "ordem natural"
- natcasesort() - Ordena um array utilizando o algoritmo de "ordem natural" sem diferenciar maiúsculas e minúsculas

strncasecmp

strncasecmp

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

strncasecmp — Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres

Descrição

function **strncasecmp**(string `$string1`, string `$string2`, int `$length`): int

Esta função é similar a strcasecmp(), com a diferença que pode ser especificado o número (limite superior) de caracteres de cada string usada na comparação.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

`length`

O tamanho das strings, que será usado na comparação.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Exemplo #1 Exemplo de **strncasecmp()**

```php
<?php

$var1 = 'Hello John';
$var2 = 'hello Doe';
if (strncasecmp($var1, $var2, 5) === 0) {
    echo 'Primeiros 5 caracteres de $var1 e $var2 são iguais em uma comparação insensível a maiúsculas/minúsculas';
}
?>
```

Veja também

- strncmp() - Comparação de strings segura para binários dos primeiros n caracteres
- preg_match() - Realiza uma correspondência com expressão regular
- substr_compare() - Comparação binária de duas strings a partir de uma posição até n caracteres
- strcasecmp() - Comparação binária segura de strings insensível a maiúsculas/minúsculas
- stristr() - strstr insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string

strncmp

strncmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strncmp — Comparação de strings segura para binários dos primeiros n caracteres

Descrição

function **strncmp**(string `$string1`, string `$string2`, int `$length`): int

Esta função é parecida com strcmp(), com a diferença que pode-se especificar o (limite máximo do) número de caracteres de cada string que serão usados na comparação.

Observe que esta comparação é sensível a maiúsculas/minúsculas.

Parâmetros

`string1`

A primeira string.

`string2`

A segunda string.

`length`

Número de caracteres a usar na comparação.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.

Examples

Exemplo #1 Exemplo de **strncmp()**

```php
<?php

$var1 = 'Olá, João';
$var2 = 'Olá, Pedro';
if (strncmp($var1, $var2, 3) === 0) {
    echo 'Primeiros 3 caracteres de $var1 e $var2 são iguais em uma comparação de strings sensível a maiúsculas/minúsculas.';
}
?>
```

Veja também

- strncasecmp() - Comparação binária de strings, insensível a maiúsculas/minúsculas, dos primeiros n caracteres
- preg_match() - Realiza uma correspondência com expressão regular
- substr_compare() - Comparação binária de duas strings a partir de uma posição até n caracteres

- strcmp() - Comparação binária segura de strings
- strstr() - Encontra a primeira ocorrência de uma string
- substr() - Retorna parte de uma string

strpbrk

strpbrk

(PHP 5, PHP 7, PHP 8)

strpbrk — Procura na string por um dos caracteres de um conjunto

Descrição

function **strpbrk**(string `$string`, string `$characters`): string|false

strpbrk() procura na `string` por qualquer dos caracteres contidos em `characters`.

Parâmetros

`string`

A string onde `characters` é procurado.

`characters`

Este parâmetro é sensível a maiúsculas/minúsculas.

Valor Retornado

Retorna uma string iniciando do caractere encontrado, ou `false` se nada for encontrado.

Examples

Exemplo #1 Exemplo de strpbrk()

```php
<?php

$text = 'This is a Simple text.';

// isto mostrará "is is a Simple text." porque 'i' aparece primeiro
echo strpbrk($text, 'mi'), PHP_EOL;

// isto mostrará "Simple text." porque a função é sensível a maiúsculas/minúsculas
echo strpbrk($text, 'S'), PHP_EOL;
?>
```

Veja também

- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- strstr() - Encontra a primeira ocorrência de uma string
- preg_match() - Realiza uma correspondência com expressão regular

strpos

strpos

(PHP 4, PHP 5, PHP 7, PHP 8)

strpos — Encontra a posição da primeira ocorrência de uma substring em uma string

Descrição

function **strpos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Encontra a posição numérica da primeira ocorrência de `needle` na string `haystack`.

Parâmetros

`haystack`

A string onde a substring será pesquisada.

`needle`

A substring a ser procurada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`offset`

Se especificado, a pesquisa iniciará nessa posição de caractere contada a partir do início da string. Se for negativo, a pesquisa iniciará nessa posição de caractere contada a partir do final da string.

Valor Retornado

Retorna a posição em que a substring existe relativa ao início da string `haystack` (independente do `offset`). Observe também que as posições de string iniciam em `0`, e não em `1`.

Retorna **`false`** se a substring não for encontrada.

> **Aviso**
>
> Esta função pode retornar o valor booleano **`false`**, mas também pode retornar um valor não booleano que pode ser avaliado como **`false`**. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar um int em `needle` não é mais suportado.
7.3.0	Passar um int em `needle` foi descontinuado.
7.1.0	Suporte a `offset` negativo foi adicionado.

Examples

Exemplo #1 Usando `===`

```php
<?php
$mystring = 'abc';
$findme   = 'a';
$pos = strpos($mystring, $findme);

// Observe o uso de ===. Simplesmente == não funcionaria como esperado
// porque o caractere 'a' fica na posição 0 (primeira) da string.
if ($pos === false) {
    echo "A string '$findme' não foi encontrada em '$mystring'";
} else {
    echo "A string '$findme' foi encontrada em '$mystring'";
    echo " e existe na posição $pos";
}
?>
```

Exemplo #2 Usando `!==`

```php
<?php
$mystring = 'abc';
$findme   = 'a';
$pos = strpos($mystring, $findme);

// O operador !== também pode ser usado. Usar != não funcionaria como esperado
```

```php
// porque a posição de 'a' é 0. A comparação (0 != false) é avaliada
// como falsa.
if ($pos !== false) {
     echo "A string '$findme' foi encontrada em '$mystring'";
         echo " e existe na posição $pos";
} else {
     echo "A string '$findme' não foi encontrada em '$mystring'";
}
?>
```

Exemplo #3 Usando um deslocamento

```php
<?php
// Pode-se procurar um caractere, ignorando tudo antes do deslocamento
$newstring = 'abcdef abcdef';
$pos = strpos($newstring, 'a', 1); // $pos = 7, e não 0

echo $pos, PHP_EOL;
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- str_contains() - Determina se uma string contém uma substring fornecida
- str_ends_with() - Verifica se uma string termina com uma substring fornecida
- str_starts_with() - Verifica se uma string começa com uma substring fornecida
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string
- strpbrk() - Procura na string por um dos caracteres de um conjunto
- substr() - Retorna parte de uma string
- preg_match() - Realiza uma correspondência com expressão regular

strrchr

strrchr

(PHP 4, PHP 5, PHP 7, PHP 8)

strrchr — Encontra a última ocorrência de um caractere em uma string

Descrição

function **strrchr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

Esta função retorna a porção de `haystack` que inicia na última ocorrência de `needle` e continua até o final de `haystack`.

Parâmetros

`haystack`

A string na qual a pesquisa será realizada.

`needle`

Se `needle` contiver mais de um caractere, apenas o primeiro será usado. Este comportamento é diferente quando comparado com strstr().

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo

do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`before_needle`

Se for `true`, **strrchr()** retorna a parte de `haystack` antes da última ocorrência do caractere `needle` (excluindo o caractere).

Valor Retornado

Esta função retorna a porção de uma string, ou `false` se `needle` não for encontrado.

Registro de Alterações

Versão	Descrição
8.3.0	O parâmetro `before_needle` foi adicionado.
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar um int em `needle` não é mais suportado.
7.3.0	Passar um int em `needle` foi descontinuado.

Examples

Exemplo #1 Exemplo de strrchr()

```php
<?php
$ext = strrchr('algum_arquivo.txt', '.');
echo "extensão do arquivo: $ext \n";
$ext = $ext ? strtolower(substr($ext, 1)) : '';
echo "extensão do arquivo: $ext";
?>
```

O exemplo acima produzirá algo semelhante a:

```
extensão do arquivo: .txt
extensão do arquivo: txt
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strstr() - Encontra a primeira ocorrência de uma string
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string

strrev

strrev

(PHP 4, PHP 5, PHP 7, PHP 8)

strrev — Reverte uma string

Descrição

function **strrev**(string `$string`): string

Retorna a `string` revertida.

Parâmetros

`string`

A string a ser revertida.

Valor Retornado

Retorna a string revertida.

Examples

Exemplo #1 Revertendo uma string strrev()

```php
<?php
echo strrev("Hello world!"); // mostra "!dlrow olleH"
?>
```

strripos

strripos

(PHP 5, PHP 7, PHP 8)

strripos — Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas

Descrição

function **strripos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Encontra a posição numérica da última ocorrência de `needle` na string `haystack`.

Diferentemente de strrpos(), **strripos()** é insensível a maiúsculas/minúsculas.

Parâmetros

`haystack`

A string onde a substring será procurada.

`needle`

A substring procurada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`offset`

Se zero ou positivo, a pesquisa é realizada da esquerda para a direita pulando os primeiros `offset` bytes de `haystack`.

Se negativo, a pesquisa é realizada da direita para a esquerda pulando os últimos `offset` bytes de `haystack` e pesquisando pela primeira ocorrência de `needle`.

> **Nota** :
>
> Efetivamente, é o mesmo que procurar pela última ocorrência de `needle` antes dos últimos `offset` bytes.

Valor Retornado

Retorna a posição onde a string procurada `needle` existe relativamente ao início da string `haystack` (independente da direção de busca ou de `offset`).

> **Nota** :
>
> A posições na string iniciam em 0, e não em 1.

Retorna `false` se a string em `needle` não for encontrada.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.2.0	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale(). Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar int no parâmetro `needle` não é mais suportado.
7.3.0	Passar um int no parâmetro `needle` foi descontinuado.

Examples

Exemplo #1 Um exemplo simples de strripos()

```php
<?php

$haystack = 'ababcd';
$needle   = 'aB';

$pos      = strripos($haystack, $needle);

if ($pos === false) {
    echo "Desculpe, não encontramos `$needle` em `$haystack`";
} else {
    echo "Parabéns!\n";
    echo "Encontramos a última ocorrência de `$needle` em `$haystack` na posição `$pos`";
}

?>
```

O exemplo acima produzirá:

```
Parabéns!
Encontramos a última ocorrência de `aB` em `ababcd` na posição `2`
```

Veja também

- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- strrchr() - Encontra a última ocorrência de um caractere em uma string
- stristr() - strstr insensível a maiúsculas/minúsculas
- substr() - Retorna parte de uma string

strrpos

strrpos

(PHP 4, PHP 5, PHP 7, PHP 8)

strrpos — Encontra a posição da última ocorrência de uma substring em uma string

Descrição

function **strrpos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Encontra a posição numérica da última ocorrência de `needle` na string `haystack`.

Parâmetros

`haystack`

A string onde a substring será procurada.

`needle`

A substring procurada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`offset`

Se zero ou positivo, a pesquisa é realizada da esquerda para a direita pulando os primeiros `offset` bytes de `haystack`.

Se negativo, a pesquisa inicia a `offset` bytes da direita ao invés do início de `haystack`. A pesquisa é realizada da direita para a esquerda, pesquisando pela primeira ocorrência de `needle` a partir do byte selecionado.

> **Nota** :
>
> Efetivamente, é o mesmo que procurar pela última ocorrência de `needle` antes dos últimos `offset` bytes.

Valor Retornado

Retorna a posição onde a string procurada existe relativamente ao início da string `haystack` (independente da direção de busca ou do deslocamento).

> **Nota** :
>
> A posições na string iniciam em 0, e não em 1.

Retorna `false` se a string não for encontrada.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar int no parâmetro `needle` não é mais suportado.
7.3.0	Passar um int no parâmetro `needle` foi descontinuado.

Examples

Exemplo #1 Verificando se uma substring existe em uma string

É fácil se enganar nos valores de retorno para "caracteres encontrados na posição 0" e "caractere não encontrado". Veja como detectar a diferença:

```php
<?php
$mystring = 'Elephpant';

$pos = strrpos($mystring, "b");
if ($pos === false) { // observe: três sinais de igualdade
    // não encontrada
}

?>
```

Exemplo #2 Pesquisa com deslocamentos

```php
<?php
$foo = "0123456789a123456789b123456789c";

// Procurando por '0' a partir do byte 0 (a partir do início)
var_dump(strrpos($foo, '0', 0));

// Procurando por '0' a partir do byte 1 (depois do byte 0)
var_dump(strrpos($foo, '0', 1));

// Procurando por '7' a partir do byte 20
var_dump(strrpos($foo, '7', 20));

// Procurando por '7' a partir do byte 28
var_dump(strrpos($foo, '7', 28));

// Procurando por '7' da direita para a esquerda a partir do 5º byte contando do final
var_dump(strrpos($foo, '7', -5));

// Procurando por 'c' da direita para a esquerda a partir do 2º byte contando do final
var_dump(strrpos($foo, 'c', -2));

// Procurando por '9c' da direita para a esquerda a partir do 2º byte condando do final
var_dump(strrpos($foo, '9c', -2));
?>
```

O exemplo acima produzirá:

```
int(0)
bool(false)
int(27)
bool(false)
int(17)
bool(false)
int(29)
```

Veja também

- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strrchr() - Encontra a última ocorrência de um caractere em uma string
- substr() - Retorna parte de uma string

strspn

strspn

(PHP 4, PHP 5, PHP 7, PHP 8)

strspn — Encontra o comprimento do segmento inicial de uma string composta totalmente de caracteres contidos em uma máscara informada

Descrição

```
function strspn(
    string $string,
    string $characters,
    int $offset = 0,
    ?int $length = null
): int
```

Encontra o comprimento do segmento inicial da `string` que contenha *somente* caracteres contidos em `characters`.

Se `offset` e `length` forem omitidos, então toda a `string` será examinada. Se forem incluídos, o efeito será o mesmo que chamar `strspn(substr($string, $offset, $length), $characters)` (consulte substr para mais informações).

A linha de código:

```php
<?php
$var = strspn("42 is the answer to the 128th question.", "1234567890");
?>
```

atribuirá `2` para *$var*, porque a string "42" é o segmento inicial de `string` que consiste somente de caracteres contidos em "1234567890".

Parâmetros

`string`

A primeira string a ser examinada.

`characters`

A lista de caracteres permitidos.

`offset`

A posição na `string` para iniciar a pesquisa.

Se `offset` for informado e não negativo, a função **strspn()** começará a examinar a `string` na posição `offset`. Por exemplo, na string '`abcdef`', o caractere na posição `0` é '`a`', na posição `2` é '`c`', e assim por diante.

Se `offset` for informado e negativo, a função **strspn()** começará a examinar a `string` na posição `offset` a partir do final da `string`.

`length`

O tamanho do segmento da `string` a ser examinado.

Se `length` for informado e não negativo, a `string` será examinada pelo número de caracteres em `length` a partir da posição inicial.

Se `length` for informado e negativo, a `string` será examinada a partir da posição inicial até o número de caracteres informado em `length` contados a partir do final da `string`.

Valor Retornado

Retorna o comprimento do segmento inicial de `string` que consistir inteiramente dos caracteres definidos em `characters`.

> **Nota** :
>
> Quando o parâmetro `offset` estiver definido, o comprimento retornado é contado iniciando-se desta posição, não do início da `string`.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de strspn()

```php
<?php
// string não começa com nenhum dos caracteres da máscara
var_dump(strspn("foo", "o"));

// examina dois caracteres da string iniciando na posição 1
var_dump(strspn("foo", "o", 1, 2));

// examina um caractere da string iniciando na posição 1
var_dump(strspn("foo", "o", 1, 1));
?>
```

O exemplo acima produzirá:

```
int(0)
int(2)
int(1)
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strcspn() - Encontra o tamanho do segmento inicial que não corresponde à máscara

strstr

strstr

(PHP 4, PHP 5, PHP 7, PHP 8)

strstr — Encontra a primeira ocorrência de uma string

Descrição

function **strstr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

Retorna parte da string `haystack` iniciando a partir da primeira ocorrência de `needle` (inclusive) até o final de `haystack`.

> **Nota** :
>
> Este função é sensível a maiúsculas/minúsculas. Para pesquisas insensíveis a maiúsculas/minúsculas, use stristr().

> **Nota** :
>
> Se for necessário apenas determinar se um `needle` em particular ocorre dentro de `haystack`, a função mais rápida e que consome menos memória, str_contains(), deve ser usada em seu lugar.

Parâmetros

`haystack`

A string de entrada.

`needle`

A string sendo procurada.

Antes do PHP 8.0.0, se `needle` não for uma string, ela será convertida para um número inteiro e aplicada como o valor ordinal de um caractere. Este comportamento foi descontinuado a partir do PHP 7.3.0 e depender dele é altamente desaconselhado. Dependendo do comportamento pretendido, o parâmetro `needle` deve ser explicitamente convertido em string ou uma chamada explícita para chr() deve ser realizada.

`before_needle`

Se for igual a `true`, **strstr()** retorna a parte de `haystack` antes da primeira ocorrência de `needle` (excluindo este último).

Valor Retornado

Retorna a parte da string ou `false` se `needle` não for encontrada.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	Passar um int em `needle` não é mais suportado.
7.3.0	Passar um int em `needle` foi descontinuado.

Examples

Exemplo #1 Exemplo de strstr()

```php
<?php
$email  = 'nome@exemplo.com';
$domain = strstr($email, '@');
echo $domain, PHP_EOL; // mostra @exemplo.com

$user = strstr($email, '@', true);
echo $user, PHP_EOL; // mostra nome
?>
```

Veja também

- stristr() - strstr insensível a maiúsculas/minúsculas
- strrchr() - Encontra a última ocorrência de um caractere em uma string
- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- strpbrk() - Procura na string por um dos caracteres de um conjunto
- preg_match() - Realiza uma correspondência com expressão regular

strtok

strtok

(PHP 4, PHP 5, PHP 7, PHP 8)

strtok — Divide uma string em tokens

Descrição

function **strtok**(string `$string`, string `$token`): string|false

Assinatura alternativa (não suportada com argumentos nomeados):

function **strtok**(string `$token`): string|false

strtok() divide a `string` em partes menores (tokens), com cada token sendo delimitado pelo caractere definido em `token`. Isto é, em uma string como "Este é um texto de exemplo", pode-se dividir esta string em palavras individuais usando-se o caractere de espaço como o `token`.

Observe que somente a primeira chamada a strtok usa o argumento `string`. Cada chamada subsequente a strtok precisará apenas do parâmetro `token`, já que ela guarda a posição atual na string. Para reiniciar, ou para dividir uma nova string em tokens, deve-se simplesmente chamar strtok com o argumento `string` novamente. Observe que pode-se incluir múltiplos tokens no parâmetro `token`. A string será dividida sempre que qualquer dos caracteres presentes no argumento `token` for encontrado.

> **Nota** :
>
> Esta função comporta-se de maneira um pouco diferente do que o esperado quando se está familiar com explode(). Primeiramente uma sequência de dois ou mais caracteres contíguos de `token` na string é considerado como um único delimitador. Além disso, um `token` situado no início ou no final da string é ignorado. Por exemplo, se uma string `";aaa;;bbb;"` for usada, chamadas sucessivas a **strtok()** com `";"` como `token` retornaria as strings "aaa" e "bbb", e depois `false`. Como resultado, a string seria dividida em apenas dois elementos, enquanto que `explode(";", $string)` retornaria um array com 5 elementos.

Parâmetros

`string`

A string que será dividida em strings menores (tokens).

`token`

O delimitador usado ao dividir a `string`.

Valor Retornado

Um token de string ou `false` se nenhum token adicional estiver disponível.

Registro de Alterações

Versão	Descrição
8.3.0	Agora emite um alerta `E_WARNING` quando `token` não for informado.

Examples

Exemplo #1 Exemplo de strtok()

```php
<?php
$string = "Este é\tum texto de\nexemplo";
/* Pode-se usar também tabulação e nova linha como caracteres de divisão */
$tok = strtok($string, " \n\t");

while ($tok !== false) {
    echo "Palavra={$tok}\n";
    $tok = strtok(" \n\t");
}
?>
```

Exemplo #2 Comportamento de strtok() com uma parte vazia encontrada

```php
<?php
$first_token  = strtok('/palavra', '/');
$second_token = strtok('/');
var_dump($first_token, $second_token);
?>
```

O exemplo acima produzirá:

```
string(7) "palavra"
bool(false)
```

Exemplo #3 A diferença entre strtok() e explode()

```php
<?php
$string = ";aaa;;bbb;";

$parts = [];
```

```php
$tok = strtok($string, ";");
while ($tok !== false) {
    $parts[] = $tok;
    $tok = strtok(";");
}
echo json_encode($parts),"\n";

$parts = explode(";", $string);
echo json_encode($parts),"\n";
```

O exemplo acima produzirá:

```
["aaa","bbb"]
["","aaa","","bbb",""]
```

Notas

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Veja também

- explode() - Divide uma string com base em outra string

strtolower

strtolower

(PHP 4, PHP 5, PHP 7, PHP 8)

strtolower — Converte uma string para minúsculas

Descrição

function **strtolower**(string `$string`): string

Retorna uma `string` com todos os caracteres alfabéticos ASCII convertidos para minúsculas.

Bytes na faixa `"A"` (0x41) a `"Z"` (0x5a) serão convertidos para a letra minúscula correspondente pela adição de 32 a cada valor de byte.

Esta função pode ser usada para converter caracteres ASCII em string codificadas com UTF-8, já que os caracteres multibyte UTF-8 serão ignorados. Para converter caracteres multibyte não-ASCII, use mb_strtolower().

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string minúscula.

Registro de Alterações

Versão	Descrição
8.2.0	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale(). Somente caracteres ASCII serão convertidos.

Examples

Exemplo #1 Exemplo de strtolower()

```php
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = strtolower($str);
echo $str; // Prints mary had a little lamb and she loved it so
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strtoupper() - Converte uma string em maiúsculas
- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo
- ucwords() - Converte para maiúsculas o primeiro caractere de cada palavra
- mb_strtolower() - Torna uma string minúscula

strtoupper

strtoupper

(PHP 4, PHP 5, PHP 7, PHP 8)

strtoupper — Converte uma string em maiúsculas

Descrição

function **strtoupper**(string `$string`): string

Retorna `string` com todos os caracteres ASCII alfabéticos convertidos em maiúsculas.

Bytes no intervalo `"a"` (0x61) a `"z"` (0x7a) serão convertidos para a letra maiúscula correspondente pela subtração de 32 de cada valor de byte.

Esta função pode ser usada para converter caracteres ASCII em strings codificadas com UTF-8, já que caracteres UTF-8 multi-byte serão ignorados. Para converter caracteres multi-byte não-ASCII, use mb_strtoupper().

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string em maiúsculas.

Registro de Alterações

Versão	Descrição
8.2.0	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale(). Somente caracteres ASCII serão convertidos.

Examples

Exemplo #1 Exemplo de strtoupper()

```php
<?php
$str = "Maria Tinha Um Pequeno Porco e Ela AMAVA-O Tanto!";
$str = strtoupper($str);
echo $str; // Prints MARIA TINHA UM PEQUENO PORCO E ELA AMAVA-O TANTO!
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strtolower() - Converte uma string para minúsculas
- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo
- ucwords() - Converte para maiúsculas o primeiro caractere de cada palavra
- mb_strtoupper() - Faz uma string ficar em maiúsculas

strtr

strtr

(PHP 4, PHP 5, PHP 7, PHP 8)

strtr — Traduz caracteres ou substitui substrings

Descrição

function **strtr**(string `$string`, string `$from`, string `$to`): string

Assinatura alternativa (não suportada com argumentos nomeados):

function **strtr**(string `$string`, array `$replace_pairs`): string

Se for chamada com três argumentos, esta função retorna uma cópia de `string` onde todas as ocorrências de cada caractere (de um byte) em `from` tenham sido traduzidas para o caractere correspondente em `to`, isto é, cada ocorrência de `$from[$n]` tenha sido substituída por `$to[$n]`, onde `$n` é um deslocamento válido em ambos os argumentos.

Se `from` e `to` tiverem comprimentos diferentes, os caracteres adicionais no argumento mais longo são ignorados. O comprimento de `string` será o mesmo do valor de retorno.

Se for chamada com dois argumentos, o segundo deve ser um array na forma `array('from' => 'to', ...)`. O valor de retorno é uma string onde todas as ocorrências das chaves do array tenham sido substituídas pelos valores correspondentes. As chaves mais longas são substituídas primeiro. Assim que ums substring é substituída, seu novo valor não será pesquisado novamente.

Neste caso, as chaves e os valores podem ter qualquer comprimento, desde que não haja chave vazia; adicionalmente, o comprimento do valor de retorno pode ser diferente do comprimento de `string`. Entretanto, esta função será mais eficiente quando todas as chaves têm o mesmo tamanho.

Parâmetros

`string`

A string a ser traduzida.

`from`

A string que deverá ser traduzida para `to`.

`to`

A string que substituirá `from`.

`replace_pairs`

O parâmetro `replace_pairs` pode ser usado no lugar de `to` e `from`, como um array na forma `array('from' => 'to', ...)`.

Se `replace_pairs` contiver uma chave que seja uma string vazia (`""`), o elemento será ignorado; a partir do PHP 8.0.0, um alerta `E_WARNING` é emitido neste caso.

Valor Retornado

Retorna a string traduzida.

Examples

Exemplo #1 Exemplo de strtr()

```php
<?php

$original = "He1Lo, w0rld!";

// Com 3 argumentos, strtr() faz uma tradução byte a byte.
$result = strtr($original, "1L0", "llo");
echo $result;
```

O exemplo acima produzirá:

```
Hello, world!
```

O exemplo a seguir mostra o comportamento de **strtr()** quando chamada com apenas dois argumentos. Observe a preferência das substituições (`"h"` não é capturado pois existem correspondências mais longas) e como o texto já substituído não é pesquisado novamente.

Exemplo #2 Exemplo de strtr() com dois argumentos

```php
<?php

$replace_pairs = ["h" => "-", "hello" => "hi", "hi" => "hello"];
echo strtr("hi all, I said hello", $replace_pairs);
```

O exemplo acima produzirá:

```
hello all, I said hi
```

Os dois modos de comportamento são substancialmente diferentes. Com três argumentos, **strtr()** irá substituir bytes; com dois, ela pode substituir substrings mais longas.

Exemplo #3 Comparação de comportamentos de strtr()

```php
<?php

// Caracteres de um só byte
$singlebyte_string = "baab";
echo strtr($singlebyte_string, "ab", "01"), "\n"; // Tradução byte por byte

$replace_pairs = ["ab" => "01"];
echo strtr($singlebyte_string, $replace_pairs), "\n"; // Tradução de substring

// Caracteres Multi-byte
$multibyte_string = "Ä"; // Em UTF-8, codificado como dois bytes: "\xC3\x84"

$result = strtr($multibyte_string, 'Ä', 'A');
echo bin2hex($result), "\n"; // "4184", onde "c3" se torna "41" ("A"), enquanto que "84" é mantido e quebra o UTF-8

$replace_pairs = ['Ä' => 'A'];
echo strtr($multibyte_string, $replace_pairs); // Uma tradução correta de substring multi-byte
```

O exemplo acima produzirá:

```
1001
ba01
4184
A
```

Veja também

- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui

substr

substr

(PHP 4, PHP 5, PHP 7, PHP 8)

substr — Retorna parte de uma string

Descrição

function **substr**(string `$string`, int `$offset`, ?int `$length = null`): string

Retorna a parte de `string` especificada pelo parâmetro `offset` e `length`.

Parâmetros

`string`

A string de entrada

`offset`

Se `offset` for não-negativo, a string retornada iniciará na posição `offset` em `string`, começando em zero. Por exemplo, na string '`abcdef`', o caractere na posição `0` é '`a`', na posição `2` é '`c`', e assim por diante.

Se `offset` for negativo, a string retornada irá começar no caractere `offset` contado a partir do fim da `string`.

Se a `string` for mais curta que o parâmetro `offset`, uma string vazia será retornada.

Exemplo #1 Usando um `offset` negativo

```php
<?php
echo substr("abcdef", -1), PHP_EOL;    // retorna "f"
echo substr("abcdef", -2), PHP_EOL;    // retorna "ef"
echo substr("abcdef", -3, 1), PHP_EOL; // retorna "d"
?>
```

`length`

Se `length` for fornecido e for positivo, a string retornada irá conter, no máximo, `length` caracteres começando em `offset` (dependendo do tamanho da `string`).

Se `length` for informado e for negativo, então esta quantidade de caracteres será omitida do final da `string`. Se `offset` denota a posição dessa truncagem ou além dela, uma string vazia será retornada.

Se `length` for informado e for igual a `0`, uma string vazia será retornada.

Se `length` for omitido ou `null`, a substring iniciando em `offset` será retornada.

Exemplo #2 Usando um `length` negativo

```php
<?php
echo substr("abcdef", 0, -1), PHP_EOL;  // retorna "abcde"
echo substr("abcdef", 2, -1), PHP_EOL;  // retorna "cde"
echo substr("abcdef", 4, -4), PHP_EOL;  // retorna ""; antes do PHP 8.0.0, falso era retornado
echo substr("abcdef", -3, -1), PHP_EOL; // retorna "de"
?>
```

Valor Retornado

Retorna a parte extraída da `string`, ou uma string vazia.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora pode ser nulo. Quando `length` for especificamente definido para `null`, a função retorna uma substring terminando no final da string, quando anteriormente retornava uma string vazia.
8.0.0	A função retorna uma string vazia quando antes retornava `false`.

Examples

Exemplo #3 Uso básico de substr()

```php
<?php
echo substr('abcdef', 1), PHP_EOL;     // bcdef
echo substr("abcdef", 1, null), PHP_EOL; // bcdef; antes do PHP 8.0.0, string vazia era retornada
echo substr('abcdef', 1, 3), PHP_EOL;  // bcd
echo substr('abcdef', 0, 4), PHP_EOL;  // abcd
echo substr('abcdef', 0, 8), PHP_EOL;  // abcdef
echo substr('abcdef', -1, 1), PHP_EOL; // f

// O acesso a caracteres individuais pode
// ser feito através de indexação
$string = 'abcdef';
echo $string[0], PHP_EOL;                // a
echo $string[3], PHP_EOL;                // d
echo $string[strlen($string)-1], PHP_EOL; // f

?>
```

Exemplo #4 Comportamento de conversão no substr()

```php
<?php
class apple {
    public function __toString() {
        return "green";
    }
}

echo "1) ".var_export(substr("pear", 0, 2), true), PHP_EOL;
echo "2) ".var_export(substr(54321, 0, 2), true), PHP_EOL;
echo "3) ".var_export(substr(new apple(), 0, 2), true), PHP_EOL;
echo "4) ".var_export(substr(true, 0, 1), true), PHP_EOL;
echo "5) ".var_export(substr(false, 0, 1), true), PHP_EOL;
echo "6) ".var_export(substr("", 0, 1), true), PHP_EOL;
echo "7) ".var_export(substr(1.2e3, 0, 4), true), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
1) 'pe'
2) '54'
3) 'gr'
4) '1'
5) ''
6) ''
7) '1200'
```

Exemplo #5 Faixa Inválida de Caracteres

Se uma faixa inválida de caracteres for requisitada, **substr()** retorna uma string vazia a partir do PHP 8.0.0; anteriormente, `false` era retornado.

```php
<?php
var_dump(substr('a', 2));
?>
```

Saída do exemplo acima no PHP 8:

```
string(0) ""
```

Saída do exemplo acima no PHP 7:

```
bool(false)
```

Veja também

- strrchr() - Encontra a última ocorrência de um caractere em uma string
- substr_replace() - Substitui o texto dentro de uma parte de uma string
- preg_match() - Realiza uma correspondência com expressão regular
- trim() - Retira espaços (ou outros caracteres) do início e do final de uma string
- mb_substr() - Obtém parte da string
- wordwrap() - Quebra uma string em um dado número de caracteres
- Acesso e modificação de strings por caractere

substr_compare

substr_compare

(PHP 5, PHP 7, PHP 8)

substr_compare — Comparação binária de duas strings a partir de uma posição até n caracteres

Descrição

```
function substr_compare(
    string $haystack,
    string $needle,
    int $offset,
    ?int $length = null,
    bool $case_insensitive = false
): int
```

substr_compare() compara `haystack` a partir da posição `offset` com `needle` até `length` caracteres.

Parâmetros

`haystack`

A string principal a ser comparada.

`needle`

A string secundária a ser comparada.

`offset`

A posição inicial para a comparação. Se negativa, inicia contando da final da string.

`length`

O comprimento da comparação. O valor padrão é o maior comprimento entre o comprimento de `needle` comparado com o comprimento de `haystack` menos o `offset`.

`case_insensitive`

Se `case_insensitive` for `true`, a comparação é insensível a maiúsculas/minúsculas.

Valor Retornado

Retorna um valor menor que 0 se `string1` for abaixo de `string2`; um valor maior que 0 se `string1` for acima de `string2` e `0` se elas forem iguais. Nenhum significado em particular pode ser inferido com confiança desse valor a não ser o seu sinal.

Se `offset` for igual a (antes do PHP 7.2.18, 7.3.5) ou maior que o comprimento de `haystack`, ou `length` estiver definido e menor que 0, **substr_compare()** mostra um alerta e retorna `false`.

Registro de Alterações

Versão	Descrição
8.2.0	Não é mais garantido que esta função retornará `strlen($string1) - strlen($string2)` quando os comprimentos das strings não forem iguais, mas ppodem agora retornar `-1` ou `1` no lugar.
8.0.0	`length` agora pode ser nulo.
7.2.18, 7.3.5	`offset` agora pode ser igual ao comprimento de `haystack`.

Examples

Exemplo #1 Exemplo de substr_compare()

```php
<?php
echo substr_compare("abcde", "bc", 1, 2), PHP_EOL; // 0
echo substr_compare("abcde", "de", -2, 2), PHP_EOL; // 0
echo substr_compare("abcde", "bcg", 1, 2), PHP_EOL; // 0
echo substr_compare("abcde", "BC", 1, 2, true), PHP_EOL; // 0
echo substr_compare("abcde", "bc", 1, 3), PHP_EOL; // 1
echo substr_compare("abcde", "cd", 1, 2), PHP_EOL; // -1
echo substr_compare("abcde", "abc", 5, 1), PHP_EOL; // -1
?>
```

Veja também

- strncmp() - Comparação de strings segura para binários dos primeiros n caracteres

substr_count

substr_count

(PHP 4, PHP 5, PHP 7, PHP 8)

substr_count — Conta o número de ocorrências de uma substring

Descrição

```
function substr_count(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?int $length = null
): int
```

substr_count() retorna o número de vezes que a substring `needle` ocorre na string `haystack`. Note que `needle` faz distinção de maiúscula e minúscula.

> **Nota** :
>
> Esta função não conta substrings sobrepostas. Veja o exemplo abaixo!

Parâmetros

`haystack`

A string onde será feita a busca

`needle`

A substring a ser procurada

`offset`

A posição onde deve-se iniciar a contagem. Se a posição for negativa, a contagem começa do final da string.

`length`

O comprimento máximo depois da posição especificada para buscar pela substring. Mostra um alerta se a posição mais o comprimento for maior que o comprimento de `haystack`. Um comprimento negativo conta a partir do final de `haystack`.

Valor Retornado

Esta função retorna um int.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora pode ser nulo.
7.1.0	Suporte a valores negativos para `offset` e `length` foi adicionado. `length` agora também pode ser `0`.

Examples

Exemplo #1 Exemplo de substr_count()

```php
<?php
$text = 'This is a test';
echo strlen($text), PHP_EOL; // 14

echo substr_count($text, 'is'), PHP_EOL; // 2

// a string é reduzida a 's is a test', então mostra 1
echo substr_count($text, 'is', 3), PHP_EOL;

// o texto é reduzido a 's i', então mostra 0
echo substr_count($text, 'is', 3, 3), PHP_EOL;

// mostra apenas 1, porque não conta substrings sobrepostos
$text2 = 'gcdgcdgcd';
echo substr_count($text2, 'gcdgcd'), PHP_EOL;

// lança uma exceção porque 5+10 > 14
echo substr_count($text, 'is', 5, 10), PHP_EOL;
?>
```

Veja também

- count_chars() - Retorna informação sobre caracteres usados em uma string
- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- substr() - Retorna parte de uma string
- strstr() - Encontra a primeira ocorrência de uma string

substr_replace

substr_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

substr_replace — Substitui o texto dentro de uma parte de uma string

Descrição

```
function substr_replace(
    array|string $string,
    array|string $replace,
    array|int $offset,
    array|int|null $length = null
): string|array
```

substr_replace() substitui uma cópia de `string` delimitada pelos parâmetros `offset` e (opcionalmente) `length` com a string informada em `replace`.

Parâmetros

`string`

A string de entrada.

Um array de strings pode ser fornecido, e neste caso as substituições irão ocorrer em um string de cada vez. Neste caso, os parâmetros `replace`, `offset` e `length` podem ser fornecidos tanto como valores escalares a ser aplicados a um string de entrada de cada vez, quanto como arrays, neste caso o elemento correspondente do array será usado por cada string de entrada.

`replace`

A string substituta.

`offset`

Se `offset` for não-negativo, a substituição começará na posição `offset` da `string`.

Se `offset` for negativo, a substituição começará a partir da posição `offset` contada a partir do final da `string`.

`length`

Se fornecido e positivo, ele representa o comprimento da porção de `string` que deve ser substituída. Se negativo, ele representa o número de caracteres do final de `string` onde parar a substituição. Se ele não for fornecido, o padrão será até strlen(`string`); isto é, parar a substituição no final de `string`. Obviamente, se `length` for zero então esta função irá ter efeito de inserir `replace` em `string` na posição fornecida em `offset`.

Valor Retornado

A string de resultado é retornada. Se `string` for um array, então um array será retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de **substr_replace()**

```php
<?php
$var = 'ABCDEFGH:/MNRPQR/';
echo "Original: $var<hr />\n";

/* Estes dois exemplos substituem tudo de $var com 'bob'. */
echo substr_replace($var, 'bob', 0) . "<br />\n";
echo substr_replace($var, 'bob', 0, strlen($var)) . "<br />\n";

/* Insere 'bob' direto no começo de $var. */
echo substr_replace($var, 'bob', 0, 0) . "<br />\n";

/* Estes dois exemplos substituem 'MNRPQR' em $var com 'bob'. */
echo substr_replace($var, 'bob', 10, -1) . "<br />\n";
```

```
echo substr_replace($var, 'bob', -7, -1) . "<br />\n";

/* Deleta 'MNRPQR' de $var. */
echo substr_replace($var, '', 10, -1) . "<br />\n";
?>
```

Exemplo #2 Usando substr_replace() para substituir múltiplas strings de uma só vez

```
<?php
$input = array('A: XXX', 'B: XXX', 'C: XXX');

// Um caso simples: substituir XXX em cada string com YYY.
echo implode('; ', substr_replace($input, 'YYY', 3, 3))."\n";

// Um caso mais complicado onde cada substituição é diferente.
$replace = array('AAA', 'BBB', 'CCC');
echo implode('; ', substr_replace($input, $replace, 3, 3))."\n";

// Substituir um número diferente de caracteres a cada vez.
$length = array(1, 2, 3);
echo implode('; ', substr_replace($input, $replace, 3, $length))."\n";
?>
```

O exemplo acima produzirá:

```
A: YYY; B: YYY; C: YYY
A: AAA; B: BBB; C: CCC
A: AAAXX; B: BBBX; C: CCC
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição
- substr() - Retorna parte de uma string
- Acesso e modificação de strings por caractere

trim

trim

(PHP 4, PHP 5, PHP 7, PHP 8)

trim — Retira espaços (ou outros caracteres) do início e do final de uma string

Descrição

function **trim**(string `$string`, string `$characters` = " \n\r\t\v\x00"): string

Esta função retorna uma string com os espaços retirados do ínicio e do final de `string`. Sem o segundo parâmetro, **trim()** irá retirar estes caracteres

- `" "`: caractere ASCII SP `0x20`, um espaço normal.
- `"\t"`: caractere ASCII HT `0x09`, uma tabulação.
- `"\n"`: caractere ASCII LF `0x0A`, uma nova linha (alimentação de linha).
- `"\r"`: caractere ASCII CR `0x0D`, um retorno de carro.

- `"\0"`: caractere ASCII NUL `0x00`, o byte nulo.
- `"\v"`: caractere ASCII VT `0x0B`, uma tabulação vertical.

Parâmetros

`string`

A string que será modificada.

`characters`

Opcionalmente, os caracteres removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos. Com `..` é possível especificar um intervalo incremental de caracteres.

Valor Retornado

A string com caracteres removidos.

Examples

Exemplo #1 Exemplo de uso de trim()

```php
<?php

$text   = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);

print "\n";

$trimmed = trim($text);
var_dump($trimmed);

$trimmed = trim($text, " \t.");
var_dump($trimmed);

$trimmed = trim($hello, "Hdle");
var_dump($trimmed);

$trimmed = trim($hello, 'HdWr');
var_dump($trimmed);

// retira os caracteres ASCII de controle no início e fim de $binary
// (de 0 a 31 inclusive)
$clean = trim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

O exemplo acima produzirá:

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"

string(28) "These are a few words :) ..."
string(24) "These are a few words :)"
string(5) "o Wor"
string(9) "ello Worl"
string(14) "Example string"
```

Exemplo #2 Usando trim() em valores de array

```php
<?php
function trim_value(&$value)
{
    $value = trim($value);
}

$fruit = array('apple','banana ', ' cranberry ');
var_dump($fruit);

array_walk($fruit, 'trim_value');
var_dump($fruit);

?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  string(5) "apple"
  [1]=>
  string(7) "banana "
  [2]=>
  string(11) " cranberry "
}
array(3) {
  [0]=>
  string(5) "apple"
  [1]=>
  string(6) "banana"
  [2]=>
  string(9) "cranberry"
}
```

Notas

> **Nota** : **Possível armadilha: removendo caracteres no meio da string**
>
> Como **trim()** retira caracteres do início e do fim de uma string, pode ser confuso quando caracteres são (ou não) removidos do meio. `trim('abc', 'bad')` remove tanto 'a' quanto 'b' porque ele retira 'a', portanto move 'b' para o início e também será retirado. Portanto, este é o motivo pelo qual isso "funciona" enquanto que `trim('abc', 'b')` aparentemente não funciona.

Veja também

- ltrim() - Retira espaços em branco (ou outros caracteres) do início de uma string
- rtrim() - Retira espaços em branco (ou outros caracteres) do final de uma string
- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição

ucfirst

ucfirst

(PHP 4, PHP 5, PHP 7, PHP 8)

ucfirst — Transforma o primeiro caractere de uma string em maiúsculo

Descrição

function **ucfirst**(string `$string`): string

Retorna uma string com o primeiro caractere de `string` maiúsculo, se esse caractere for ASCII no intervalo de `"a"` (0x61) a `"z"` (0x7a).

Parâmetros

`string`

A string de entrada.

Valor Retornado

Retorna a string resultante.

Registro de Alterações

Versão	Descrição
8.2.0	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale(). Somente caracteres ASCII serão convertidos.

Examples

Exemplo #1 Exemplo de ucfirst()

```php
<?php
$foo = 'olá, mundo!';
echo ucfirst($foo), PHP_EOL;             // Olá, mundo!

$bar = 'OLÁ MUNDO!';
echo ucfirst($bar), PHP_EOL;             // OLÁ, MUNDO!
echo ucfirst(strtolower($bar)), PHP_EOL; // Olá, mundo!
?>
```

Veja também

- lcfirst() - Torna minúsculo o primeiro caractere de uma string
- strtolower() - Converte uma string para minúsculas
- strtoupper() - Converte uma string em maiúsculas
- ucwords() - Converte para maiúsculas o primeiro caractere de cada palavra
- mb_convert_case() - Realiza a conversão de caixa em uma string

ucwords

ucwords

(PHP 4, PHP 5, PHP 7, PHP 8)

ucwords — Converte para maiúsculas o primeiro caractere de cada palavra

Descrição

function **ucwords**(string `$string`, string `$separators` = " \t\r\n\f\v"): string

Retorna uma string com o primeiro caractere de cada palavra em `string` em maiúscula, se este caractere for uma letra ASCII entre `"a"` (0x61) e `"z"` (0x7a).

Nesta função, uma palavra é qualquer sequência de caracteres não listada no parâmetro `separators`. Por padrão, os separadores são: espaço, tab horizontal, retorno de carro, nova linha, form feed e tab vertical.

Para realizar uma conversão similar em strings multibyte, utilize mb_convert_case() com a opção `MB_CASE_TITLE`.

Parâmetros

`string`

A string de entrada.

`separators`

O parâmetro opcional `separators` contém os caracteres separadores das palavras.

Valor Retornado

Retorna a string modificada.

Registro de Alterações

Versão	Descrição
8.2.0	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale(). Somente caracteres ASCII serão convertidos.

Examples

Exemplo #1 Exemplo de ucwords()

```
<?php
$foo = 'hello world!';
echo ucwords($foo), PHP_EOL;              // Hello World!

$bar = 'HELLO WORLD!';
echo ucwords($bar), PHP_EOL;              // HELLO WORLD!
echo ucwords(strtolower($bar)), PHP_EOL;  // Hello World!
?>
```

Exemplo #2 Exemplo de ucwords() com delimitador

```
<?php
$foo = 'hello|world!';
echo ucwords($foo), PHP_EOL;             // Hello|world!

echo ucwords($foo, "|"), PHP_EOL;        // Hello|World!
?>
```

Exemplo #3 Exemplo de ucwords() com delimitadores adicionais

```
<?php
$foo = "mike o'hara";
echo ucwords($foo), PHP_EOL;                // Mike O'hara

echo ucwords($foo, " \t\r\n\f\v'"), PHP_EOL; // Mike O'Hara
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- strtoupper() - Converte uma string em maiúsculas
- strtolower() - Converte uma string para minúsculas
- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo
- mb_convert_case() - Realiza a conversão de caixa em uma string

utf8_decode

utf8_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

utf8_decode — Converte uma string de UTF-8 para ISO-8859-1, substituindo caracteres inválidos ou não representáveis

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.2.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **utf8_decode**(string `$string`): string

Esta função converte a string `string` da codificação `UTF-8` para a `ISO-8859-1`. Bytes na string que não sejam válidos em `UTF-8`, e caracteres `UTF-8` que não existam em `ISO-8859-1` (isto é, pontos de código acima de `U+00FF`) são substituídos por `?`.

> **Nota** :
>
> Muitas páginas da web que dizem usar a codificação `ISO-8859-1` na verdade usam a codificação similar `Windows-1252`, e os navegadores irão interpretar as páginas `ISO-8859-1` como `Windows-1252`. `Windows-1252` apresenta caracteres imprimíveis adicionais, como o símbolo do Euro (€) e aspas inglesas (“ ”), ao invés de certos caracteres de controle `ISO-8859-1`. Esta função não converterá esses caracteres `Windows-1252` corretamente. Use uma função diferente se a conversão de `Windows-1252` for requerida.

Parâmetros

`string`

Uma string em UTF-8.

Valor Retornado

Retorna a conversão em ISO-8859-1 da `string`.

Registro de Alterações

Versão	Descrição
8.2.0	Esta função foi descontinuada.
7.2.0	Esta função foi movida de extensão XML para o núcleo do PHP. Em versões anteriores, estava disponível somente se a extensão XML estivesse instalada.

Examples

Exemplo #1 Exemplos básicos

```php
<?php
// Converte a string 'Zoë' de UTF-8 para ISO 8859-1
$utf8_string = "\x5A\x6F\xC3\xAB";
$iso8859_1_string = utf8_decode($utf8_string);
echo bin2hex($iso8859_1_string), "\n";

// Sequências inválidas em UTF-8 são substituídas por '?'
$invalid_utf8_string = "\xC3";
$iso8859_1_string = utf8_decode($invalid_utf8_string);
var_dump($iso8859_1_string);

// Caracteres que não existem em ISO 8859-1, tal como
// '€' (símbolo do Euro) também são substituídas por '?'
$utf8_string = "\xE2\x82\xAC";
$iso8859_1_string = utf8_decode($utf8_string);
var_dump($iso8859_1_string);
?>
```

O exemplo acima produzirá:

```
5a6feb
string(1) "?"
string(1) "?"
```

Notas

> **Nota** : **Descontinuação e alternativas**

Esta função foi *descontinuada* a partir do PHP 8.2.0, e será removida em uma versão futura. Usos existentes devem ser verificados e substituídos com alternativas apropriadas.

Funcionalidade similar pode ser obtida com mb_convert_encoding(), que suporta ISO-8859-1 e muitas outras codificações de caracteres.

```php
<?php
$utf8_string = "\xC3\xAB"; // 'ë' (e com trema) em UTF-8
$iso8859_1_string = mb_convert_encoding($utf8_string, 'ISO-8859-1', 'UTF-8');
echo bin2hex($iso8859_1_string), "\n";

$utf8_string = "\xCE\xBB"; // 'λ' (letra grega lambda minúscula) em UTF-8
$iso8859_7_string = mb_convert_encoding($utf8_string, 'ISO-8859-7', 'UTF-8');
echo bin2hex($iso8859_7_string), "\n";

$utf8_string = "\xE2\x82\xAC"; // '€' (símbolo do Euro) em UTF-8 (ausente na ISO-8859-1)
$windows_1252_string = mb_convert_encoding($utf8_string, 'Windows-1252', 'UTF-8');
echo bin2hex($windows_1252_string), "\n";
?>
```

O exemplo acima produzirá:

```
eb
eb
80
```

Outros exemplos que podem estar disponíveis dependendo das extensões instaladas são UConverter::transcode() e iconv().

Todos os exemplos a seguir dão o mesmo resultado:

```php
<?php
$utf8_string = "\x5A\x6F\xC3\xAB"; // 'Zoë' em UTF-8
$iso8859_1_string = utf8_decode($utf8_string);
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = mb_convert_encoding($utf8_string, 'ISO-8859-1', 'UTF-8');
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = iconv('UTF-8', 'ISO-8859-1', $utf8_string);
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = UConverter::transcode($utf8_string, 'ISO-8859-1', 'UTF8');
echo bin2hex($iso8859_1_string), "\n";
?>
```

O exemplo acima produzirá:

```
5a6feb
5a6feb
5a6feb
5a6feb
```

Especificar `'?'` como a opção `'to_subst'` no método UConverter::transcode() dá o mesmo resultado que **utf8_decode()** para strings inválidas ou que não possam ser representadas em ISO 8859-1.

```php
<?php
$utf8_string = "\xE2\x82\xAC"; // € (símbolo do Euro) não existe na ISO 8859-1
$iso8859_1_string = UConverter::transcode(
    $utf8_string, 'ISO-8859-1', 'UTF-8', ['to_subst' => '?']
```

```
);
var_dump($iso8859_1_string);
?>
```

O exemplo acima produzirá:

```
string(1) "?"
```

Veja também

- utf8_encode() - Converte uma string ISO-8859-1 em UTF-8
- mb_convert_encoding() - Converte uma string de uma codificação de caracteres para outra
- UConverter::transcode() - Converte uma string de uma codificação de caracteres para outra
- iconv() - Converte uma string de uma codificação de caracteres para outra

utf8_encode

utf8_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

utf8_encode — Converte uma string ISO-8859-1 em UTF-8

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.2.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **utf8_encode**(string `$string`): string

Esta função converte a string `string` da codificação `ISO-8859-1` para a `UTF-8`.

> **Nota** :
>
> Esta função não tenta descobrir a codificação da string fornecida, ela assume que esteja codificada como ISO-8859-1 (também conhecida como "Latin 1") e converte-a para UTF-8. Como toda sequência de bytes é uma string ISO-8859-1 válida, isto nunca irá resultar em erro, mas não irá resultar em uma string útil se uma codificação diferente era a intenção.
>
> Muitas páginas da web que dizem usar a codificação `ISO-8859-1` na verdade usam a codificação similar `Windows-1252`, e os navegadores irão interpretar as páginas `ISO-8859-1` como `Windows-1252`. `Windows-1252` apresenta caracteres imprimíveis adicionais, como o símbolo do Euro (€) e aspas inglesas (“ ”), ao invés de certos caracteres de controle `ISO-8859-1`. Esta função não converterá esses caracteres `Windows-1252` corretamente. Use uma função diferente se a conversão de `Windows-1252` for requerida.

Parâmetros

`string`

Uma string em ISO-8859-1.

Valor Retornado

Retorna a conversão em UTF-8 da `string`.

Registro de Alterações

Versão	Descrição
8.2.0	Esta função foi descontinuada.

Versão	Descrição
7.2.0	Esta função foi movida de extensão XML para o núcleo do PHP. Em versões anteriores, estava disponível somente se a extensão XML estivesse instalada.

Examples

Exemplo #1 Exemplo básico

```php
<?php
// Converte a string 'Zoë' de ISO 8859-1 para UTF-8
$iso8859_1_string = "\x5A\x6F\xEB";
$utf8_string = utf8_encode($iso8859_1_string);
echo bin2hex($utf8_string), "\n";
?>
```

O exemplo acima produzirá:

```
5a6fc3ab
```

Notas

> **Nota** : **Descontinuação e alternativas**
>
> Esta função foi *descontinuada* a partir do PHP 8.2.0, e será removida em uma versão futura. Usos existentes devem ser verificados e substituídos com alternativas apropriadas.
>
> Funcionalidade similar pode ser obtida com mb_convert_encoding(), que suporta ISO-8859-1 e muitas outras codificações de caracteres.

```php
<?php
$iso8859_1_string = "\xEB"; // 'ë' (e com trema) em ISO-8859-1
$utf8_string = mb_convert_encoding($iso8859_1_string, 'UTF-8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$iso8859_7_string = "\xEB"; // a mesma string em ISO-8859-7 representa 'λ' (letra grega lambda minúscula)
$utf8_string = mb_convert_encoding($iso8859_7_string, 'UTF-8', 'ISO-8859-7');
echo bin2hex($utf8_string), "\n";

$windows_1252_string = "\x80"; // '€' (símbolo do Euro) em Windows-1252, mas não em ISO-8859-1
$utf8_string = mb_convert_encoding($windows_1252_string, 'UTF-8', 'Windows-1252');
echo bin2hex($utf8_string), "\n";
?>
```

O exemplo acima produzirá:

```
c3ab
cebb
e282ac
```

Outros exemplos que podem estar disponíveis dependendo das extensões instaladas são UConverter::transcode() e iconv().

Todos os exemplos a seguir dão o mesmo resultado:

```php
<?php
$iso8859_1_string = "\x5A\x6F\xEB"; // 'Zoë' em ISO-8859-1
```

```php
$utf8_string = utf8_encode($iso8859_1_string);
echo bin2hex($utf8_string), "\n";

$utf8_string = mb_convert_encoding($iso8859_1_string, 'UTF-8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$utf8_string = UConverter::transcode($iso8859_1_string, 'UTF8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$utf8_string = iconv('ISO-8859-1', 'UTF-8', $iso8859_1_string);
echo bin2hex($utf8_string), "\n";
?>
```

O exemplo acima produzirá:

```
5a6fc3ab
5a6fc3ab
5a6fc3ab
5a6fc3ab
```

Veja também

- utf8_decode() - Converte uma string de UTF-8 para ISO-8859-1, substituindo caracteres inválidos ou não representáveis
- mb_convert_encoding() - Converte uma string de uma codificação de caracteres para outra
- UConverter::transcode() - Converte uma string de uma codificação de caracteres para outra
- iconv() - Converte uma string de uma codificação de caracteres para outra

vfprintf

vfprintf

(PHP 5, PHP 7, PHP 8)

vfprintf — Escreve uma string formatada para um fluxo

Descrição

function **vfprintf**(resource `$stream`, string `$format`, array `$values`): int

Escreve uma string produzida de acordo com `format` para o recurso do fluxo especificado por `stream`.

Funciona como fprintf() mas aceita um array de argumentos, ao invés de um número váriavel de argumentos.

Parâmetros

`stream`

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
-	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.

Sinalizador	Descrição
`+`	Prefixa números positivos com um sinal de mais `+`; Por padrão, apenas números negativos são prefixados com um sinal negativo.
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
`'`(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :
>
> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
`G`	Como o especificador `g`, mas usa `E` e `f`.
`h`	Como o especificador `g`, mas usa `F`. Disponível a partir do PHP 8.0.0.
`H`	Como o especificador `g`, mas usa `E` e `F`. Disponível a partir do PHP 8.0.0.
`o`	O argumento é tratado como um número inteiro e apresentado como um número octal.
`s`	O argumento é tratado e apresentado como uma string.
`u`	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
`x`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
`X`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**

> O especificador de tipo `c` ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna o tamanho da string mostrada.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que `PHP_INT_MAX`. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que `PHP_INT_MAX`. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, `false` era retornado e um `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não mais retorna `false` em caso de falha.
8.0.0	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError se `[width]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError se `[precision]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um `E_WARNING`.

Examples

Exemplo #1 vfprintf(): inteiros precedidos por zero

```php
<?php
if (!($fp = fopen('date.txt', 'w')))
    return;

$year = 2025;
$month = 5;
$day = 6;
vfprintf($fp, "%04d-%02d-%02d", array($year, $month, $day));
// escreverá a data em formato ISP em date.txt
?>
```

Veja também

- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada
- fprintf() - Escreve uma string formatada para um fluxo
- vprintf() - Mostra uma string formatada
- vsprintf() - Retorna uma string formatada
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

vprintf

vprintf

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

vprintf — Mostra uma string formatada

Descrição

function **vprintf**(string `$format`, array `$values`): int

Motra valores de array como uma string formatada de acordo com `format` (que está descrito na documentação da função sprintf()).

Funciona como printf() mas aceita uma array de argumentos, ao invés de um número variável de argumentos.

Parâmetros

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
-	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.
+	Prefixa números positivos com um sinal de mais `+`; Por padrão, apenas números negativos são prefixados com um sinal negativo.
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
'(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :

> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
`G`	Como o especificador `g`, mas usa `E` e `f`.
`h`	Como o especificador `g`, mas usa `F`. Disponível a partir do PHP 8.0.0.
`H`	Como o especificador `g`, mas usa `E` e `F`. Disponível a partir do PHP 8.0.0.
`o`	O argumento é tratado como um número inteiro e apresentado como um número octal.
`s`	O argumento é tratado e apresentado como uma string.
`u`	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
`x`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
`X`	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**
>
> O especificador de tipo `c` ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna o comprimento da string mostrada.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que `PHP_INT_MAX`. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que `PHP_INT_MAX`. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, `false` era retornado e um `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não mais retorna `false` em caso de falha.
8.0.0	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError se `[width]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError se `[precision]` for menor que zero ou maior que `PHP_INT_MAX`; anteriormente esta função emitia um `E_WARNING`.
8.0.0	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um `E_WARNING`.

Examples

Exemplo #1 vprintf(): inteiros preenchidos com zero

```php
<?php
vprintf("%04d-%02d-%02d", explode('-', '1988-8-1'));
?>
```

O exemplo acima produzirá:

```
1988-08-01
```

Veja também

- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada
- fprintf() - Escreve uma string formatada para um fluxo
- vsprintf() - Retorna uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

vsprintf

vsprintf

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

vsprintf — Retorna uma string formatada

Descrição

function **vsprintf**(string `$format`, array `$values`): string

Funciona como sprintf() mas aceita um array de argumentos, ao invés de um número variável de argumentos.

Parâmetros

`format`

A string de formato é composta de zero ou mais diretivas: caracteres comuns (excluindo `%`) que são copiados diretamente para o resultado e *especificações de conversão*, cada uma das quais resulta na busca de seu próprio parâmetro.

Uma especificação de conversão segue este protótipo: `%[argnum$][flags][width][.precision]especificador`.

Argnum

Um número inteiro seguido por um cifrão `$`, para especificar qual o número de argumento a tratar na conversão.

Flags

Sinalizador	Descrição
`-`	Justificar à esquerda dentro da largura de campo especificada; A justificação à direita é o padrão.
`+`	Prefixa números positivos com um sinal de mais `+`; Por padrão, apenas números negativos são prefixados com um sinal negativo.
(espaço)	Preenche o resultado com espaços. Este é o padrão.
`0`	Preenche números com zeros apenas à esquerda. Com os especificadores `s`, também pode preencher com zeros à direita.
`'`(caractere)	Preenche o resultado com o caractere especificado.

Width

Um número inteiro que define em quantos caracteres (mínimo) esta conversão deve resultar, ou `*`. Se `*` for usado, a largura será fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Precision

Um ponto `.` opcionalmente seguido por um número inteiro ou `*`, cujo significado depende do especificador:

- Para os especificadores `e`, `E`, `f` e `F`: este é o número de dígitos a serem impressos após o ponto decimal (por padrão, é 6).
- Para os especificadores `g`, `G`, `h` e `H`: este é o número máximo de dígitos significativos a serem impressos.
- Para o especificador `s`: atua como um ponto de corte, definindo um limite máximo de caracteres para a string.

> **Nota** :
>
> Se o ponto for especificado sem um valor explícito para precisão, 0 é assumido. Se `*` for usado, a precisão é fornecida como um valor inteiro adicional precedendo aquele formatado pelo especificador.

Especificadores

Especificador	Descrição
`%`	Um caractere literal de porcentagem. Nenhum argumento é necessário.
`b`	O argumento é tratado como um número inteiro e apresentado como um número binário.
`c`	O argumento é tratado como um número inteiro e apresentado como o caractere com aquele código ASCII.
`d`	O argumento é tratado como um número inteiro e apresentado como um número decimal (com sinal).
`e`	O argumento é tratado como notação científica (por exemplo, 1.2e+2).
`E`	Como o especificador `e`, mas usa letra maiúscula (por exemplo, 1.2E+2).
`f`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (com reconhecimento da localidade).
`F`	O argumento é tratado como um float e apresentado como um número de ponto flutuante (sem reconhecimento da localidade).
`g`	Formato geral. Deixa P igual à precisão se for diferente de zero, 6 se a precisão for omitida, ou 1 se a precisão for zero. Então, se uma conversão com estilo E tivesse um expoente de X: Se P > X ≥ −4, a conversão é com estilo f e precisão P − (X + 1). Caso contrário, a conversão é com estilo e e precisão P − 1.
`G`	Como o especificador `g`, mas usa `E` e `f`.

Especificador	Descrição
h	Como o especificador g, mas usa F. Disponível a partir do PHP 8.0.0.
H	Como o especificador g, mas usa E e F. Disponível a partir do PHP 8.0.0.
o	O argumento é tratado como um número inteiro e apresentado como um número octal.
s	O argumento é tratado e apresentado como uma string.
u	O argumento é tratado como um número inteiro e apresentado como um número decimal sem sinal.
x	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras minúsculas).
X	O argumento é tratado como um número inteiro e apresentado como um número hexadecimal (com letras maiúsculas).

> **Aviso**
>
> O especificador de tipo c ignora preenchimento e largura.

> **Aviso**
>
> Tentar usar uma combinação dos especificadores de string e largura com conjuntos de caracteres que requerem mais de um byte por caractere pode resultar em resultados inesperados.

As variáveis serão forçadas a um tipo adequado para o especificador:

Tratamento de Tipos

Tipo	Especificadores
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Valor Retornado

Retorna valores de array como uma string formatada de acordo com `format`.

Erros/Exceções

A partir do PHP 8.0.0, um erro ValueError será lançado se o número de argumentos for zero. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

A partir do PHP 8.0.0, um erro ValueError será lançado quando menos argumentos do que o necessário forem fornecidos. Antes do PHP 8.0.0, **`false`** era retornado e um **`E_WARNING`** era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não mais retorna **`false`** em caso de falha.
8.0.0	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um **`E_WARNING`**.
8.0.0	Lança uma exceção ValueError se `[width]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente esta função emitia um **`E_WARNING`**.
8.0.0	Lança uma exceção ValueError se `[precision]` for menor que zero ou maior que **`PHP_INT_MAX`**; anteriormente esta função emitia um **`E_WARNING`**.
8.0.0	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um **`E_WARNING`**.

Examples

Exemplo #1 vsprintf(): inteiros preenchidos com zero

```php
<?php
print vsprintf("%04d-%02d-%02d", explode('-', '1988-8-1'));
?>
```

O exemplo acima produzirá:

```
1988-08-01
```

Veja também

- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada
- fprintf() - Escreve uma string formatada para um fluxo
- vprintf() - Mostra uma string formatada
- vfprintf() - Escreve uma string formatada para um fluxo
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- number_format() - Formata um número com milhares agrupados
- date() - Formata um timestamp Unix

wordwrap

wordwrap

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

wordwrap — Quebra uma string em um dado número de caracteres

Descrição

function **wordwrap**(
  string $string,
  int $width = 75,
  string $break = "\n",
  bool $cut_long_words = false
): string

Quebra uma string em um dado número de caracteres usando um caractere de quebra de string. As strings são quebradas após um caractere de espaço (U+0020) a menos que `cut_long_words` seja definido para `true`.

Parâmetros

`string`

A string de entrada.

`width`

O número de caracteres nos quais a string será quebrada.

`break`

A linha é quebrada inserindo o caractere informado no parâmetro opcional `break`. Não pode ser uma string vazia. O padrão é "\n".

`cut_long_words`

Se `cut_long_words` for definido para `true`, a string será sempre quebrada na largura especificada em `width` ou antes. Então se existir uma palavra mais longa que a largura informada, ela será quebrada (veja o segundo exemplo). Quando for definido para `false`, a

função não quebrará a palavra mesmo se `width` for menor que o comprimento dela.

Valor Retornado

Retorna a string informada quebrada na largura especificada.

Erros/Exceções

Se `break` for uma string vazia, uma exceção ValueError é lançada.

Registro de Alterações

Versão	Descrição
8.0.0	Se `break` for uma string vazia, uma exceção ValueError é lançada; anteriormente, neste caso, um `E_WARNING` era emitido e a função retornava `false`.

Examples

Exemplo #1 Exemplo de wordwrap()

```php
<?php
$text = "Um pequeno jabuti xereta viu dez cegonhas felizes.";
$newtext = wordwrap($text, 20, "<br />\n");

echo $newtext;
?>
```

O exemplo acima produzirá:

```
Um pequeno jabuti<br />
xereta viu dez<br />
cegonhas felizes.
```

Exemplo #2 Exemplo de wordwrap()

```php
<?php
$text = "Uma longa palaaaaaaaaavra.";
$newtext = wordwrap($text, 8, "\n", true);

echo "$newtext\n";
?>
```

O exemplo acima produzirá:

```
Uma
longa
palaaaaa
aaaavra.
```

Exemplo #3 Exemplo de wordwrap()

```php
<?php
$text = "Uma longa palaaaaaaaaaaaaaaavra e reticências";
$newtext = wordwrap($text, 8, "\n", false);

echo "$newtext\n";
?>
```

O exemplo acima produzirá:

```
Uma
longa
palaaaaaaaaaaaaaaavra
e
reticências
```

Veja também

- nl2br() - Insere quebras de linha HTML antes de todos os caracteres de nova linha em um string
- chunk_split() - Divide uma string em pedaços menores

Registro de Alterações

As alterações a seguir foram feitas em classes/funções/métodos desta extensão.

Version	Function	Description
8.5.0	chr	Passar inteiros fora do intervalo [0, 255] foi descontinuado.
	ord	Passar uma string que não seja um byte único foi descontinuado.
	setlocale	Passar um inteiro 0 como o argumento de locales não é mais suportado e agora lança um TypeError.
8.4.0	str_getcsv	Depender do valor padrão de escape agora está descontinuado.
	str_getcsv	Agora lança um ValueError se separator, enclosure, ou escape for inválido. Isto imita o comportamento de fgetcsv e de fputcsv.
	strcspn	Antes do PHP 8.4.0, quando characters fosse uma string vazia, a pesquisa incorretamente encerraria no primeiro byte nulo em string.
8.3.0	number_format	Adicionada a manipulação de valores negativos para decimals.
	str_getcsv	Uma string vazia é retornada ao invés de uma string com um único byte nulo para o último campo, se ele contiver apenas um demarcador não terminado.
	strrchr	O parâmetro before_needle foi adicionado.
	strtok	Agora emite um alerta E_WARNING quando token não for informado.
8.2.0	lcfirst	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale. Somente caracteres ASCII serão convertidos.
	str_ireplace	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale. Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
	str_split	Se string estiver vazio, um array agora é retornado. Anteriormente, um array contendo uma única string vazia foi retornada.
	strcasecmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	strcmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	stripos	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale. Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
	stristr	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale. Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
	strnatcasecmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.

Version	Function	Description
	strnatcmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	strncasecmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	strncmp	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	strripos	A redução de todas as letras a maiúsculas ou minúsculas não depende mais da localidade definida com setlocale. Somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita. Os bytes não ASCII serão comparados por seu valor de byte.
	strtolower	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale. Somente caracteres ASCII serão convertidos.
	strtoupper	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale. Somente caracteres ASCII serão convertidos.
	substr_compare	Não é mais garantido que esta função retornará strlen($string1) - strlen($string2) quando os comprimentos das strings não forem iguais, mas ppodem agora retornar -1 ou 1 no lugar.
	ucfirst	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale. Somente caracteres ASCII serão convertidos.
	ucwords	A conversão de maiúsculas e minúsculas não depende mais da localidade definida com setlocale. Somente caracteres ASCII serão convertidos.
	utf8_decode	Esta função foi descontinuada.
	utf8_encode	Esta função foi descontinuada.
8.1.0	get_html_translation_table	flags mudou de ENT_COMPAT para ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	html_entity_decode	O padrão de flags mudou de ENT_COMPAT para ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlentities	O padrão de flags mudou de ENT_COMPAT para ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlspecialchars	O padrão para flags mudou de ENT_COMPAT para ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlspecialchars_decode	O padrão de flags mudou de ENT_COMPAT para ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
8.0.0	convert_cyr_string	Esta função foi removida.
	convert_uuencode	Antes desta versão, tentar converter uma string vazia retornava false sem nenhuma razão particular.
	count_chars	Antes desta versão, a função retornava false em caso de falha.
	crypt	O parâmetro salt não é mais opcional.
	explode	explode agora lança um ValueError quando separator é informado com uma string vazia (""). Anteriormente, explode retornava false nesse caso.
	fprintf	Esta função não retorna mais false em caso de falha.
	fprintf	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um E_WARNING.
	fprintf	Lança um erro ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	fprintf	Lança um erro ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	fprintf	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um E_WARNING.
	hebrevc	Esta função foi removida.
	html_entity_decode	encoding agora pode ser nulo.
	htmlentities	encoding agora pode ser nulo.
	implode	Passar separator depois de array não é mais suportado.
	levenshtein	Antes desta versão, levenshtein tinha de ser chamada com dois ou com cinco argumentos.
	levenshtein	Antes desta versão, levenshtein retornaria -1 se uma das strings fosse maior que 255 caracteres.

Version	Function	Description
	metaphone	Esta função retornava false em caso de falha.
	money_format	Esta função foi removida.
	number_format	Antes desta versão, number_format aceitava um, dois ou quatro parâmetros (mas não três).
	parse_str	result não é mais opcional.
	printf	Esta função não retorna mais false em caso de falha.
	printf	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um E_WARNING.
	printf	Lança um erro ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	printf	Lança um erro ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	printf	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um E_WARNING.
	soundex	Antes desta versão, chamar a função com uma string vazia retornava false por nenhuma razão em particular.
	sprintf	Esta função não retorna mais false em caso de falha.
	sprintf	Lança um erro ValueError se o número de argumentos for zero; anteriormente, esta função emitia um E_WARNING.
	sprintf	Lança um erro ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	sprintf	Lança um erro ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente, esta função emitia um E_WARNING.
	sprintf	Lança um erro ArgumentCountError quando menos argumentos do que o necessário são fornecidos; anteriormente, esta função emitia um E_WARNING.
	str_split	Se length for menor que 1, uma exceção ValueError será lançada agora; anteriormente, um erro de nível E_WARNING era gerado e a função retornava false.
	str_word_count	characters agora pode ser nulo.
	strcspn	length agora pode ser nulo.
	strip_tags	allowed_tags agora pode ser nulo.
	stripos	O parâmetro needle agora aceita uma string vazia.
	stripos	Passar um int em needle não é mais suportado.
	stristr	O parâmetro needle agora aceita uma string vazia.
	stristr	Passar um int em needle não é mais suportado.
	strpos	O parâmetro needle agora aceita uma string vazia.
	strpos	Passar um int em needle não é mais suportado.
	strrchr	O parâmetro needle agora aceita uma string vazia.
	strrchr	Passar um int em needle não é mais suportado.
	strripos	O parâmetro needle agora aceita uma string vazia.
	strripos	Passar int no parâmetro needle não é mais suportado.
	strrpos	O parâmetro needle agora aceita uma string vazia.
	strrpos	Passar int no parâmetro needle não é mais suportado.
	strspn	length agora pode ser nulo.
	strstr	O parâmetro needle agora aceita uma string vazia.
	strstr	Passar um int em needle não é mais suportado.
	substr	length agora pode ser nulo. Quando length for especificamente definido para null, a função retorna uma substring terminando no final da string, quando anteriormente retornava uma string vazia.
	substr	A função retorna uma string vazia quando antes retornava false.

Version	Function	Description
	substr_compare	length agora pode ser nulo.
	substr_count	length agora pode ser nulo.
	substr_replace	length agora pode ser nulo.
	vfprintf	Esta função não mais retorna false em caso de falha.
	vfprintf	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um E_WARNING.
	vfprintf	Lança uma exceção ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vfprintf	Lança uma exceção ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vfprintf	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um E_WARNING.
	vprintf	Esta função não mais retorna false em caso de falha.
	vprintf	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um E_WARNING.
	vprintf	Lança uma exceção ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vprintf	Lança uma exceção ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vprintf	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um E_WARNING.
	vsprintf	Esta função não mais retorna false em caso de falha.
	vsprintf	Lança uma exceção ValueError se o número de argumentos for zero; anteriormente esta função emitia um E_WARNING.
	vsprintf	Lança uma exceção ValueError se [width] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vsprintf	Lança uma exceção ValueError se [precision] for menor que zero ou maior que PHP_INT_MAX; anteriormente esta função emitia um E_WARNING.
	vsprintf	Lança uma exceção ValueError quando houver menos parâmetros que os necessários; anteriormente esta função emitia um E_WARNING.
	wordwrap	Se break for uma string vazia, uma exceção ValueError é lançada; anteriormente, neste caso, um E_WARNING era emitido e a função retornava false.
7.4.0	chr	A função não mais aceita silenciosamente códigos não suportados no parâmetro codepoint, convertendo-os para 0.
	convert_cyr_string	Esta função foi descontinuada.
	hebrevc	Esta função foi descontinuada.
	implode	Passar separator depois de array (isto é, usando a assinatura legada) foi descontinuado.
	money_format	Esta função foi descontinuada. Utilize NumberFormatter::formatCurrency em seu lugar.
	str_getcsv	O parâmetro escape agora interpreta uma string vazia como o sinal para desabilitar o mecanismo de escape. Antes, uma string vazia era tratada como valor padrão do parâmetro.
	strip_tags	O parâmetro allowed_tags agora alternativamente aceita um array.
7.3.5	substr_compare	offset agora pode ser igual ao comprimento de haystack.
7.3.0	stripos	Passar um int em needle foi descontinuado.
	stristr	Passar um int em needle foi descontinuado.
	strpos	Passar um int em needle foi descontinuado.
	strrchr	Passar um int em needle foi descontinuado.
	strripos	Passar um int no parâmetro needle foi descontinuado.
	strrpos	Passar um int no parâmetro needle foi descontinuado.
	strstr	Passar um int em needle foi descontinuado.

Version	Function	Description
7.2.18	substr_compare	offset agora pode ser igual ao comprimento de haystack.
7.2.0	number_format	number_format foi modificado para não ser capaz de retornar -0, anteriormente -0 poderia ser retornado para casos onde num fosse -0.01.
	parse_str	Utilização de parse_str sem o segundo parâmetro agora emite um aviso de E_DEPRECATED.
	utf8_decode	Esta função foi movida de extensão XML para o núcleo do PHP. Em versões anteriores, estava disponível somente se a extensão XML estivesse instalada.
	utf8_encode	Esta função foi movida de extensão XML para o núcleo do PHP. Em versões anteriores, estava disponível somente se a extensão XML estivesse instalada.
7.1.0	str_shuffle	O algoritmo interno de randomização mudou e agora usa o Gerador de Números Aleatórios Mersenne Twister em vez da função rand da biblioteca libc.
	stripos	Suporte para valores negativos em offset foi adicionado.
	strpos	Suporte a offset negativo foi adicionado.
	substr_count	Suporte a valores negativos para offset e length foi adicionado. length agora também pode ser 0.

Arrays

Introdução

Estas funções permitem interagir e manipular arrays de várias formas. Arrays são essenciais para armazenar, gerenciar e operar sobre conjuntos de variáveis.

Arrays simples e multidimensionais são suportados e podem ser criados pelo usuário ou por outras funções. Existem funções específicas de manipulação de banco de dados para preencher arrays a partir de consultas a banco de dados, e diversas funções retornam arrays.

Consulte a seção Arrays do manual para uma explicação detalhada sobre como arrays são implementados e usados no PHP. Veja também Operadores de array para outras maneiras de manipular os arrays.

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

`CASE_LOWER` (int)

> `CASE_LOWER` é utilizada com a função array_change_key_case() para converter chaves de arrays para minúsculas. É o valor padrão utilizado na função array_change_key_case(). A partir do PHP 8.2.0, somente caracteres ASCII são convertidos.

`CASE_UPPER` (int)

> `CASE_UPPER` é utilizada com a função array_change_key_case() para converter chaves de arrays para maiúsculas. A partir do PHP 8.2.0, somente caracteres ASCII são convertidos.

Opções de ordenação:

`SORT_ASC` (int)

> `SORT_ASC` é utilizada com array_multisort() para classificar em ordem crescente.

`SORT_DESC` (int)

> `SORT_DESC` é utilizada com array_multisort() para classificar em ordem decrescente.

Opções de tipos de ordenação: utilizadas por várias funções de ordenação

`SORT_REGULAR` (int)

> `SORT_REGULAR` é utilizada para comparar itens normalmente.

`SORT_NUMERIC` (int)

> `SORT_NUMERIC` é utilizada para comparar itens numericamente.

`SORT_STRING` (int)

> `SORT_STRING` é utilizada para comparar itens como strings.

`SORT_LOCALE_STRING` (int)

> `SORT_LOCALE_STRING` é utilizada para comparar itens como strings, baseado na localidade atual.

`SORT_NATURAL` (int)

> `SORT_NATURAL` é utilizada para comparar itens como strings utilizando a "ordenação natural", como a função natsort().

`SORT_FLAG_CASE` (int)

> `SORT_FLAG_CASE` pode ser combinada (operador OR binário) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings de forma insensível a miúsculas/minúsculas. A partir do PHP 8.2.0, somente a redução de todas as letras ASCII a maiúsculas ou minúsculas será feita.

Opções de filtro:

`ARRAY_FILTER_USE_KEY` (int)

`ARRAY_FILTER_USE_KEY` é utilizada com a função array_filter() para passar cada chave como primeiro argumento de uma função de retorno informada.

`ARRAY_FILTER_USE_BOTH` (int)

`ARRAY_FILTER_USE_BOTH` é utilizada com a função array_filter() para passar chave e valor para a função de retorno informada.

`COUNT_NORMAL` (int)

`COUNT_RECURSIVE` (int)

`EXTR_OVERWRITE` (int)

`EXTR_SKIP` (int)

`EXTR_PREFIX_SAME` (int)

`EXTR_PREFIX_ALL` (int)

`EXTR_PREFIX_INVALID` (int)

`EXTR_PREFIX_IF_EXISTS` (int)

`EXTR_IF_EXISTS` (int)

`EXTR_REFS` (int)

Ordenando Arrays

O PHP tem muitas funções para lidar com ordenação de arrays, e esse documento existe para ajudar o desenvolvedor a lidar com elas.

As principais diferenças são:

- Algumas ordenam com base nas chaves do array, enquanto outras pelos valores: `$array['chave'] = 'valor';`
- A correlação entre as chaves e os valores do array não são mantidas depois da ordenação, o que pode fazer com que as chaves sejam resetadas numericamente (0, 1, 2, ...)
- A ordem da ordenação: alfabética, ascendente (menor para maior), descendente (maior para menor), natural, aleatório, ou definida pelo usuário
- Nota: Todas essas funções agem diretamente na própria variável do array, ao invés de retornar um novo array ordenado
- Se qualquer uma dessas funções avaliar dois membros como iguais então eles retém sua ordem original. Antes do PHP 8.2.0, sua ordem eram indefinida (a ordenação não é estável).

Atributos das funções de ordenação

Nome da função	Ordena por	Mantém a associação de chaves	Ordenação	Funções relacionadas
array_multisort()	valor	string chaves sim, int chaves não	primeiro array ou opções de ordenação	array_walk()
asort()	valor	sim	ascendente	arsort()
arsort()	valor	sim	descendente	asort()
krsort()	chave	sim	descendente	ksort()
ksort()	chave	sim	ascendente	krsort()
natcasesort()	valor	sim	natural, insensível a maiúsculas/minúsculas	natsort()
natsort()	valor	sim	natural	natcasesort()
rsort()	valor	não	descendente	sort()
shuffle()	valor	não	aleatória	array_rand()
sort()	valor	não	menor para maior	rsort()
uasort()	valor	sim	definida pelo usuário	uksort()
uksort()	chave	sim	definida pelo usuário	uasort()
usort()	valor	não	definida pelo usuário	uasort()

Funções para Array

Veja também

Veja também is_array(), explode(), implode(), preg_split(), e unset().

Índice

array

array

(PHP 4, PHP 5, PHP 7, PHP 8)

array — Cria um array

Descrição

function **array**(mixed `...$values`): array

Cria um array. Leia a seção sobre o tipo array para mais informação sobre o que é um array, incluindo detalhes sobre a sintaxe alternativa de colchetes (`[]`).

Parâmetros

`values`

> A sintaxe "índice => valores", separados por vírgulas, definem índice e valores respectivamente. O índice pode ser do tipo string ou inteiro. Quando o índice é omitido, um índice inteiro é automaticamente gerado, começando em 0. Se o índice é um inteiro, o próximo índice gerado será igual ao maior índice inteiro + 1. Note que quando dois índices idênticos são definidos, o último sobrescreve o primeiro.
>
> Inserir uma vírgula após a última entrada definida do array, mesmo sendo incomum, é uma sintaxe válida.

Valor Retornado

Retorna um array dos parâmetros. Os parâmetros podem ser fornecidos com um índice com o operador `=>`. Leia a seção sobre tipo de array para mais informação sobre o que é um array.

Examples

O exemplo a seguir demonstra como criar um array de duas dimensões, como especificar chaves para arrays associativos, e como pular e continuar índices numéricos em arrays normais.

Exemplo #1 Exemplo de array()

```php
<?php
$frutas = array (
    "frutas"  => array("a" => "laranja", "b" => "banana", "c" => "maçã"),
    "numeros" => array(1, 2, 3, 4, 5, 6),
    "buracos" => array("primeiro", 5 => "segundo", "terceiro")
);
print_r($frutas);
?>
```

Exemplo #2 Indexação automática com array()

```php
<?php
$array = array(1, 1, 1, 1,  1, 8 => 1, 4 => 1, 19, 3 => 13);
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 1
```

```
    [1] => 1
    [2] => 1
    [3] => 13
    [4] => 1
    [8] => 1
    [9] => 19
)
```

Note que o índice 3 é definido duas vezes, e retêm seu último valor 13. O índice 4 é definido depois do índice 8, e o índice gerado a seguir (valor 19) é 9, uma vez que o maior índice era 8.

Esse exemplo cria um array com o índices iniciando em 1.

Exemplo #3 Array com índices a partir do 1 com array()

```php
<?php
$firstQuarter = array(1 => 'Janeiro', 'Fevereiro', 'Março');
print_r($firstQuarter);
?>
```

O exemplo acima produzirá:

```
Array
(
    [1] => Janeiro
    [2] => Fevereiro
    [3] => Março
)
```

Como no Perl, pode-se acessar o valor de um array dentro de aspas. Entretanto, com o PHP é necessário envolver o array entre chaves {}.

Exemplo #4 Acessando um array dentro de chaves

```php
<?php
$foo = array('bar' => 'baz');
echo "Hello {$foo['bar']}!"; // Hello baz!
?>
```

Notas

> **Nota** :
>
> **array()** é um construtor de linguagem usado para representar arrays literais, e não uma função normal.

Veja também

- array_pad() - Expande um array para um certo comprimento utilizando um determinado valor
- list() - Cria variáveis como se fossem arrays
- count() - Conta todos os elementos de um array ou de um objeto Countable
- range() - Cria um array contendo uma faixa de elementos
- `foreach`
- O tipo array

array_all

array_all

(PHP 8 >= 8.4.0)

array_all — Verifica se todos os elementos de um array satisfazem uma função de retorno

Descrição

function **array_all**(array `$array`, callable `$callback`): bool

array_all() retorna **`true`**, se a função de retorno `callback` retornar **`true`** para todos os elementos. Caso contrário, retorna **`false`**.

Parâmetros

`array`

O array a ser pesquisado.

`callback`

A função de retorno a ser chamada para verificar cada elemento, que precisa ter a seguinte assinatura:

function callback(mixed `$value`, mixed `$key`): bool
Se esta função retornar **`false`**, **`false`** é retornado por **array_all()** e a função de retorno não será chamada para os elementos restantes.

Valor Retornado

A função retorna **`true`**, se `callback` retornar **`true`** para todos os elementos. Caso contrário retorna **`false`**.

Examples

Exemplo #1 Exemplo de array_all()

```php
<?php
$array = [
    'a' => 'cachorro',
    'b' => 'gato',
    'c' => 'vaca',
    'd' => 'pato',
    'e' => 'elefante',
    'f' => 'ganso'
];

// Verifica se todos os nomes de animais têm menos de 12 letras.
var_dump(array_all($array, function (string $value) {
    return strlen($value) < 12;
}));

// Verifica se todos os nomes de animais têm mais que 5 letras.
var_dump(array_all($array, function (string $value) {
    return strlen($value) > 5;
}));

// Verifica se todas as chaves são strings.
var_dump(array_all($array, function (string $value, $key) {
   return is_string($key);
}));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
bool(true)
```

Veja também

- array_any() - Verifica se pelo menos um elemento de um array satisfaz uma função de retorno
- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_find() - Retorna um primeiro elemento que satisfaz uma função de retorno
- array_find_key() - Retorna a chave do primeiro elemento que satisfaz uma função de retorno

array_any

array_any

(PHP 8 >= 8.4.0)

array_any — Verifica se pelo menos um elemento de um array satisfaz uma função de retorno

Descrição

function **array_any**(array `$array`, callable `$callback`): bool

array_any() retorna **`true`**, se a função de retorno `callback` retornar **`true`** para qualquer elemento. Caso contrário retorna **`false`**.

Parâmetros

`array`

O array a ser pesquisado.

`callback`

A função de retorno a ser chamada para verificar cada elemento, que precisa ter a seguinte assinatura:

function callback(mixed `$value`, mixed `$key`): bool
Se esta função retornar **`true`**, **`true`** é retornado por **array_any()** e a função de retorno não será chamada para os elementos restantes.

Valor Retornado

A função retorna **`true`**, se houver pelo menos um elemento para o qual `callback` retorna **`true`**. Caso contrário retorna **`false`**.

Examples

Exemplo #1 Exemplo de array_any()

```php
<?php
$array = [
    'a' => 'cachorro',
    'b' => 'gato',
    'c' => 'vaca',
    'd' => 'pato',
    'e' => 'elefante',
    'f' => 'ganso'
];

// Verifica se algum nome de animal tem mais de 5 letras.
var_dump(array_any($array, function (string $value) {
    return strlen($value) > 5;
}));

// Verifica se algum nome de animal tem menos de 3 letras.
var_dump(array_any($array, function (string $value) {
    return strlen($value) < 3;
}));

// Verifica se alguma chave do array não é uma string.
var_dump(array_any($array, function (string $value, $key) {
   return !is_string($key);
}));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
bool(false)
```

Veja também

- array_all() - Verifica se todos os elementos de um array satisfazem uma função de retorno
- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_find() - Retorna um primeiro elemento que satisfaz uma função de retorno
- array_find_key() - Retorna a chave do primeiro elemento que satisfaz uma função de retorno

array_change_key_case

array_change_key_case

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_change_key_case — Modifica a caixa de todas as chaves em um array

Descrição

function **array_change_key_case**(array `$array`, int `$case` = `CASE_LOWER`): array

Retorna um array com todas as chaves de `array` em minúsculo ou maiúsculo. Índices numéricos são mantidos como são.

Parâmetros

`array`

O array a ser modificado

`case`

Tanto `CASE_UPPER` quanto `CASE_LOWER` (padrão)

Valor Retornado

Retorna um array com as chaves em minúsculo ou maiúsculo.

Examples

Exemplo #1 Exemplo da função array_change_key_case()

```php
<?php
$input_array = array("primeiRo" => 1, "segunDo" => 4);
print_r(array_change_key_case($input_array, CASE_UPPER));
?>
```

O exemplo acima produzirá:

```
Array
(
    [PRIMEIRO] => 1
    [SEGUNDO] => 4
)
```

Notas

> **Nota** :

> Se um array tem índices que serão os mesmos após a utilização desta função (ex: "chaVE" e "CHave"), o valor que estiver ao final do array irá sobrescrever os outros índices.

array_chunk

array_chunk

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_chunk — Divide um array em pedaços

Descrição

function **array_chunk**(array `$array`, int `$length`, bool `$preserve_keys = false`): array

Divide um array em pedaços com tamanho informado pelo parâmetro `length`. O último pedaço pode conter menos elementos que o parâmetro `length`.

Parâmetros

`array`

O array.

`length`

O tamanho de cada pedaço.

`preserve_keys`

Quando definido para `true`, as chaves serão preservadas. O padrão é `false`, que reindexará os pedaços numericamente.

Valor Retornado

Retorna um array multidimensional numericamente indexado, iniciando em 0, com cada dimensão contendo `length` elementos.

Erros/Exceções

Se `length` for menor que `1`, será lançado um ValueError.

Registro de Alterações

Versão	Descrição
8.0.0	Se `length` for menor que `1`, agora um ValueError é lançado; antes, um erro de nível `E_WARNING` era emitido em vez de retornar `null`.

Examples

Exemplo #1 Exemplo de array_chunk()

```php
<?php
$input_array = array('a', 'b', 'c', 'd', 'e');
print_r(array_chunk($input_array, 2));
print_r(array_chunk($input_array, 2, true));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => a
            [1] => b
        )

    [1] => Array
```

```
        (
            [0] => c
            [1] => d
        )

    [2] => Array
        (
            [0] => e
        )

)
Array
(
    [0] => Array
        (
            [0] => a
            [1] => b
        )

    [1] => Array
        (
            [2] => c
            [3] => d
        )

    [2] => Array
        (
            [4] => e
        )

)
```

Veja também

- array_slice() - Extrai uma parcela de um array

array_column

array_column

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

array_column — Retorna os valores de uma coluna do array informado

Descrição

function **array_column**(array `$array`, int|string|null `$column_key`, int|string|null `$index_key` = `null`): array

A função **array_column()** retorna os valores de uma coluna do `array` informado, indentificada pelo `column_key`. Opcionalmente, uma `index_key` pode ser informada para indexar os valores do array retornado pelos valores da coluna `index_key` do array.

Parâmetros

`array`

> Um array multidimensional ou um array de objetos que se deseja extrair os valores da coluna. Se um array de objetos for fornecido, propriedades públicas podem ser extraídas diretamente. Para extrair propriedades protegidas e privadas, a classe deve implementar ambos os métodos mágicos **__get()** e **__isset()**.

`column_key`

> A coluna de valores a ser retornada. Este valor pode ser uma chave inteira da coluna que se deseja recuperar, ou uma uma string com o nome da chave de um array associativo ou nome de propriedade. Também pode ser `null` para retornar arrays completos ou objetos (isso é útil com o parâmetro `index_key`, para reindexar o array).

`index_key`

A coluna a ser utilizada como índices/chaves do array retornado. Este valor pode ser uma chave inteira da coluna, ou uma uma string com o nome da chave. O valor é convertido como de costume para chaves de array (no entanto, antes do PHP 8.0.0, objetos que suportam conversão para string também eram permitidos).

Valor Retornado

Retorna um array com os valores que representam uma coluna do array informado.

Registro de Alterações

Versão	Descrição
8.0.0	Objetos em colunas indicadas pelo parâmetro `index_key` não serão mais convertidos em string e agora irão lançar um TypeError em vez disso.

Examples

Exemplo #1 Obtém a coluna first_name dos registros

```php
<?php

// Array representando um possível resultado retornado de um banco de dados
$records = [
    [
        'id' => 2135,
        'first_name' => 'John',
        'last_name' => 'Doe',
    ],
    [
        'id' => 3245,
        'first_name' => 'Sally',
        'last_name' => 'Smith',
    ],
    [
        'id' => 5342,
        'first_name' => 'Jane',
        'last_name' => 'Jones',
    ],
    [
        'id' => 5623,
        'first_name' => 'Peter',
        'last_name' => 'Doe',
    ]
];

$first_names = array_column($records, 'first_name');
print_r($first_names);

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => John
    [1] => Sally
    [2] => Jane
    [3] => Peter
)
```

Exemplo #2 Obtém a coluna last_name dos registros, indexados pela coluna "id"

```php
<?php
```

```php
// Usando o array $records do Exemplo #1
$records = [
    [
        'id' => 2135,
        'first_name' => 'John',
        'last_name' => 'Doe',
    ],
    [
        'id' => 3245,
        'first_name' => 'Sally',
        'last_name' => 'Smith',
    ],
    [
        'id' => 5342,
        'first_name' => 'Jane',
        'last_name' => 'Jones',
    ],
    [
        'id' => 5623,
        'first_name' => 'Peter',
        'last_name' => 'Doe',
    ]
];

$last_names = array_column($records, 'last_name', 'id');
print_r($last_names);

?>
```

O exemplo acima produzirá:

```
Array
(
    [2135] => Doe
    [3245] => Smith
    [5342] => Jones
    [5623] => Doe
)
```

Exemplo #3 Obtém a coluna de nomes de usuários da propriedade pública "username" de um objeto

```php
<?php

class User
{
    public $username;

    public function __construct(string $username)
    {
        $this->username = $username;
    }
}

$users = [
    new User('user 1'),
    new User('user 2'),
    new User('user 3'),
];

print_r(array_column($users, 'username'));

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => user 1
    [1] => user 2
    [2] => user 3
)
```

Exemplo #4 Obtém a coluna de nomes da propriedade privada "name" de um objeto utilizando os métodos mágicos __isset() e __get()

```php
<?php

class Person
{
    private $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function __get($prop)
    {
        return $this->$prop;
    }

    public function __isset($prop) : bool
    {
        return isset($this->$prop);
    }
}

$people = [
    new Person('Fred'),
    new Person('Jane'),
    new Person('John'),
];

print_r(array_column($people, 'name'));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Fred
    [1] => Jane
    [2] => John
)
```

Se o método **__isset()** não for fornecido, um array vazio será retornado.

array_combine

array_combine

(PHP 5, PHP 7, PHP 8)

array_combine — Cria um array usando um array para chaves e outro para valores

Descrição

function **array_combine**(array `$keys`, array `$values`): array

Cria um array usando os valores do array `keys` como chaves e os valores do array `values` como valores correspondentes.

Parâmetros

`keys`

Array a ser usado como chaves. Valores ilegais para chave serão convertidos para string.

`values`

Array a ser usado como valores

Valor Retornado

Retorna o array combinado.

Erros/Exceções

A partir do PHP 8.0.0, a ValueError é disparado se o número de elementos em `keys` e `values` não correspondem. Antes do PHP 8.0.0, um `E_WARNING` será disparado no lugar.

Registro de Alterações

Versão	Descrição
8.0.0	**array_combine()** será agora dispara ValueError se o numero de elementos para cada array não é igual; antes esta função retorna `false` no lugar.

Examples

Exemplo #1 Um simples exemplo usando a função array_combine()

```php
<?php
$a = array('verde', 'vermelho', 'amarelo');
$b = array('abacate', 'maçã', 'banana');
$c = array_combine($a, $b);

print_r($c);
?>
```

O exemplo acima produzirá:

```
Array
(
    [verde] => abacate
    [vermelho] => maçã
    [amarelo] => banana
)
```

Veja também

- array_merge() - Combina um ou mais arrays
- array_walk() - Aplica uma determinada função em cada elemento de um array
- array_values() - Retorna todos os valores de um array
- array_map() - Aplica uma função em todos os elementos dos arrays dados

array_count_values

array_count_values

(PHP 4, PHP 5, PHP 7, PHP 8)

array_count_values — Conta as ocorrências de cada valor distinto em um array

Descrição

function **array_count_values**(array `$array`): array

A função **array_count_values()** retorna um array utilizando os valores do parâmetro `array` (que devem ser ints ou strings) como chaves e suas respectivas frequências em `array` como valores.

Parâmetros

`array`

O array de valores para contar

Valor Retornado

Retorna um array associativo de valores de `array` como chaves e suas quantias como valor.

Erros/Exceções

Dispara `E_WARNING` para cada elemento que não for string ou int.

Examples

Exemplo #1 Exemplo da array_count_values()

```php
<?php
$array = array(1, "ola", 1, "mundo", "ola");
print_r(array_count_values($array));
?>
```

O exemplo acima produzirá:

```
Array
(
    [1] => 2
    [ola] => 2
    [mundo] => 1
)
```

Veja também

- count() - Conta todos os elementos de um array ou de um objeto Countable
- array_unique() - Remove os valores duplicados de um array
- array_values() - Retorna todos os valores de um array
- count_chars() - Retorna informação sobre caracteres usados em uma string

array_diff

array_diff

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_diff — Computa as diferenças entre arrays

Descrição

function **array_diff**(array `$array`, array `...$arrays`): array

Compara `array` com um ou mais arrays e retorna os valores no `array` que não estão presentes em nenhum dos outros arrays.

Parâmetros

`array`

O array a ser comparado

`arrays`

Arrays para comparar

Valor Retornado

Retorna um array contendo todas as entradas de `array` que não estão presentes em nenhum dos outros arrays. Chaves no array `array` são preservadas.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo da função array_diff()

```php
<?php
$array1 = array("a" => "verde", "vermelho", "azul", "vermelho");
$array2 = array("b" => "verde", "amarelo", "vermelho");
$result = array_diff($array1, $array2);

print_r($result);
?>
```

Multiplas ocorrências de *$array1* são todas tratadas da mesma maneira. Isto irá mostrar:

```
Array
(
  [1] => azul
)
```

Dois elementos são considerados iguais se e somente se `(string) $elem1 === (string) $elem2`. Isto é, quando a representação em string é a mesma.

Exemplo #2 Exemplo de array_diff() com tipos não correspondentes

```php
<?php
// Isto irá gerar uma Notícia de que um array não pode ser convertido em uma string.
$fonte = [1, 2, 3, 4];
$filtro = [3, 4, [5], 6];
$resultado = array_diff($fonte, $filtro);

// Enquanto isto não é um problema, uma vez que o objeto pode ser convertido em string.
class S {
  private $v;

  public function __construct(string $v) {
    $this->v = $v;
  }

  public function __toString() {
    return $this->v;
  }
}

$fonte = [new S('a'), new S('b'), new S('c')];
```

```php
$filtro = [new S('b'), new S('c'), new S('d')];

$resultado = array_diff($fonte, $filtro);

// $resultado agora contém uma instância de S('a');
var_dump($resultado);
?>
```

Para usar uma função de comparação alternativa, veja array_udiff().

Notas

> **Nota** :
>
> Esta função verifica somente uma dimensão de um array n-dimensional. É claro que as dimensões mais profundas podem ser verificadas usando `array_diff($array1[0], $array2[0]);`.

Veja também

- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice

array_diff_assoc

array_diff_assoc

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

array_diff_assoc — Computa a diferença entre arrays com checagem adicional de índice

Descrição

function **array_diff_assoc**(array `$array`, array `...$arrays`): array

Compara `array` com `arrays` e retorna a diferença. Ao contrário de array_diff(), as chaves dos arrays também são usadas na comparação.

Parâmetros

`array`

O array a ser comparado

`arrays`

Arrays para comparar

Valor Retornado

Retorna um array contendo todos os valores do `array` que não estão presentes em qualquer um dos outros arrays.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo de array_diff_assoc()

Neste exemplo o par `"a" => "verde"` está presente em ambos os arrays e por isso não está na saída da função. Por outro lado, o par `0 => "vermelho"` está na saída porque a chave de `"vermelho"` é automaticamente atribuído para `0` no primeiro array, enquanto é atribuído para `1` no segundo array já que a chave `0` já foi tomada por `amarelo`.

```php
<?php
$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
```

```php
$array2 = array("a" => "verde", "amarelo", "vermelho");
$result = array_diff_assoc($array1, $array2);
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [b] => marrom
    [c] => azul
    [0] => vermelho
)
```

Exemplo #2 Exemplo da função array_diff_assoc()

Dois valores dos pares *chave* => *valor* são considerados iguais somente se `(string) $elem1 === (string) $elem2` . Em outras palavras, uma checagem estrita de string é feita para que representações de string sejam as mesmas.

```php
<?php
$array1 = array(0, 1, 2);
$array2 = array("00", "01", "2");
$result = array_diff_assoc($array1, $array2);
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 0
    [1] => 1
)
```

Notas

> **Nota** :
>
> Esta função checa somente uma dimensão de um array n-dimensional. É possível verificar dimensões mais profundas usando, por exemplo, `array_diff_assoc($array1[0], $array2[0]);`.

> **Nota** :
>
> Certifique-se de que os argumentos sejam passados na ordem correta ao comparar arrays semelhantes com mais chaves. O novo array deve ser o primeiro array da lista.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno

- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice

array_diff_key

array_diff_key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_diff_key — Computa a diferença entre arrays usando as chaves na comparação

Descrição

function **array_diff_key**(array `$array`, array `...$arrays`): array

Compara as chaves do `array` com as chaves do `arrays` e retorna a diferença. Esta função é similar a array_diff(), com exceção que a comparação é feita com as chaves, ao invés dos valores.

Parâmetros

`array`

O array a ser comparado

`arrays`

Arrays para comparar

Valor Retornado

Retorna um array contendo todas as entradas de `array` cujas chaves não estão presentes em qualquer dos outros arrays.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo da função array_diff_key()

As duas chaves dos pares `chave => valor` são consideradas iguais somente se `(string) $key1 === (string) $key2` . Em outras palavras, uma checagem de tipo estrita é feita para que representações de string sejam as mesmas.

```php
<?php
$array1 = array('azul' => 1, 'vermelho' => 2, 'verde' => 3, 'roxo' => 4);
$array2 = array('verde' => 5, 'amarelo' => 6, 'ciano' => 8);

var_dump(array_diff_key($array1, $array2));
?>
```

O exemplo acima produzirá:

```
array(3) {
  ["azul"]=>
  int(1)
  ["vermelho"]=>
  int(2)
  ["roxo"]=>
  int(4)
}
```

```php
<?php
$array1 = array('azul' => 1, 'vermelho'  => 2, 'verde' => 3, 'roxo' => 4);
$array2 = array('verde' => 5, 'amarelo' => 7, 'ciano' => 8);
$array3 = array('azul' => 6, 'amarelo' => 7, 'malva' => 8);

var_dump(array_diff_key($array1, $array2, $array3));
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["vermelho"]=>
  int(2)
  ["roxo"]=>
  int(4)
}
```

Notas

> **Nota** :
>
> Esta função verifica somente uma dimensão de um array n-dimensional. É claro que as dimensões mais profundas podem ser verificadas usando `array_diff_key($array1[0], $array2[0]);`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_diff_ukey() - Computa a diferença entre arrays usando uma função callback na comparação de chaves
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_intersect_key() - Calcula a interseção de arrays, comparando pelas chaves
- array_intersect_ukey() - Computa a interseção de arrays usando uma função de retorno nas chaves para comparação

array_diff_uassoc

array_diff_uassoc

(PHP 5, PHP 7, PHP 8)

array_diff_uassoc — Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário

Descrição

function **array_diff_uassoc**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

Compara `array` com `arrays` e retorna a diferença. Ao contrário de array_diff(), as chaves dos arrays são usadas na comparação.

Ao contrário de array_diff_assoc(), uma função de callback fornecida pelo usuário é usada na comparação dos índices, não uma função interna.

Parâmetros

`array`

O array a ser comparado

`arrays`

Arrays para comparar

`key_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Retorna um array contendo todas as entradas de `array` que não estão presentes em qualquer um dos outros arrays.

Examples

Exemplo #1 Exemplo de array_diff_uassoc()

Neste exemplo o `"a" => "verde"` está presente em ambos os arrays e assim não está na saída da função. Ao contrário disso, o par `0 => "vermelho"` está na saída porque a chave de `"vermelho"` é atribuído automaticamente como `0` no primeiro array, enquanto é atribuído em `1` no segundo array, já que a chave `0` já foi atribuída a `amarelo`.

```php
<?php
function key_compare_func($a, $b)
{
    return $a <=> $b;
}

$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
$array2 = array("a" => "verde", "amarelo", "vermelho");
$result = array_diff_uassoc($array1, $array2, "key_compare_func");
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [b] => marrom
    [c] => azul
    [0] => vermelho
)
```

A igualdade de 2 índices é checada pela função de callback fornecida pelo usuário.

Notas

> **Nota** :
>
> Esta função checa somente uma dimensão de um array n-dimensional. É possível verificar dimensões mais profundas usando, por exemplo, `array_diff_uassoc($array1[0], $array2[0], "key_compare_func");`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_diff_ukey

array_diff_ukey

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_diff_ukey — Computa a diferença entre arrays usando uma função callback na comparação de chaves

Descrição

function **array_diff_ukey**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

Compara as chaves de `array` com as chaves de `arrays` e retorna a diferença. Esta função é similar a array_diff(), com exceção que a comparação é feita nas chaves ao invés dos valores.

Diferente de array_diff_key() uma função callback é fornecida e usada para comparação de índices, não função interna.

Parâmetros

`array`

O array a ser comparado

`arrays`

Arrays para comparar

`key_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Retorna um array contendo todas as entradas de `array` que não estão presentes em nenhum outro array.

Examples

Exemplo #1 Exemplo da função array_diff_ukey()

```php
<?php
function key_compare_func($key1, $key2)
{
    if ($key1 == $key2)
        return 0;
    else if ($key1 > $key2)
        return 1;
    else
        return -1;
}

$array1 = array('blue'  => 1, 'red'  => 2, 'green'  => 3, 'purple' => 4);
$array2 = array('green' => 5, 'blue' => 6, 'yellow' => 7, 'cyan'   => 8);

var_dump(array_diff_ukey($array1, $array2, 'key_compare_func'));
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["red"]=>
  int(2)
  ["purple"]=>
  int(4)
}
```

Notas

> **Nota** :
>
> Esta função verifica somente uma dimensão de um array n-dimensional. É claro que as dimensões mais profundas podem ser verificadas usando `array_diff_ukey($array1[0], $array2[0], 'callback_func');`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_diff_key() - Computa a diferença entre arrays usando as chaves na comparação
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_intersect_key() - Calcula a interseção de arrays, comparando pelas chaves
- array_intersect_ukey() - Computa a interseção de arrays usando uma função de retorno nas chaves para comparação

array_fill

array_fill

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_fill — Preenche um array com valores

Descrição

function **array_fill**(int `$start_index`, int `$count`, mixed `$value`): array

Preenche um array com `count` elementos com o valor do parâmetro `value` e chaves começando a partir de `start_index`.

Parâmetros

`start_index`

O primeiro índice do array retornado

Se `start_index` for negativo, o primeiro índice do array retornado será `start_index`, e os índices seguintes começarão do zero antes do PHP 8.0.0. A partir do PHP 8.0.0, chaves negativas são incrementadas normalmente (veja o exemplo).

`count`

Número de elementos a inserir. Deve ser maior ou igual a zero, e menor ou igual a `2147483647`.

`value`

Valores a preencher

Valor Retornado

Retorna o array preenchido

Erros/Exceções

Lança um ValueError se `count` estiver fora do alcance.

Registro de Alterações

Versão	Descrição
8.0.0	**array_fill()** agora lança um ValueError se `count` estiver fora de alance; anteriormente um `E_WARNING` era disparado, e a função retornava `false`.

Examples

Exemplo #1 Exemplo da função array_fill()

```php
<?php
$a = array_fill(5, 6, 'banana');
print_r($a);
?>
```

O exemplo acima produzirá:

```
Array
(
    [5] => banana
    [6] => banana
    [7] => banana
    [8] => banana
    [9] => banana
    [10] => banana
)
```

Exemplo #2 Exemplo da array_fill() com um índice de início negativo

```php
<?php
$a = array_fill(-2, 4, 'pera');
print_r($a);
?>
```

Saída do exemplo acima no PHP 8:

```
Array
(
    [-2] => pera
    [-1] => pera
    [0] => pera
    [1] => pera
)
```

Saída do exemplo acima no PHP 7:

```
Array
(
    [-2] => pera
    [0] => pera
    [1] => pera
    [2] => pera
)
```

Note que o índice `-1` não está presente antes do PHP 8.0.0.

Notas

Veja também a seção Arrays do manual para a explicação detalhada de chaves negativas.

Veja também

- array_fill_keys() - Preenche um array com valores, especificando chaves
- str_repeat() - Repete uma string
- range() - Cria um array contendo uma faixa de elementos

array_fill_keys

array_fill_keys

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

array_fill_keys — Preenche um array com valores, especificando chaves

Descrição

function **array_fill_keys**(array `$keys`, mixed `$value`): array

Preenche um array com o valor do parâmetro `value`, usando os valores do array `keys` como chaves.

Parâmetros

`keys`

Array de valores que serão usados como chaves. Valores ilegais para chaves serão convertidos para string.

`value`

Valor para usar no preenchimento

Valor Retornado

Retorna o array preenchido

Examples

Exemplo #1 Exemplo da função array_fill_keys()

```php
<?php
$keys = array('foo', 5, 10, 'bar');
$a = array_fill_keys($keys, 'banana');
print_r($a);
?>
```

O exemplo acima produzirá:

```
Array
(
    [foo] => banana
    [5] => banana
    [10] => banana
    [bar] => banana
)
```

Veja também

- array_fill() - Preenche um array com valores
- array_combine() - Cria um array usando um array para chaves e outro para valores

array_filter

array_filter

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

array_filter — Filtra elementos de um array utilizando uma função callback

Descrição

function **array_filter**(array `$array`, ?callable `$callback` = **`null`**, int `$mode` = 0): array

Itera sobre cada valor de `array` passando-os para a função `callback`. Se a função `callback` retornar **`true`**, o valor atual de `array` é retornado no array resultante.

Chaves do array são preservadas,e podem resultar em lacunas se o `array` foi indexado. O array resultante pode ser reindexado usando a função array_values().

Parâmetros

`array`

O array a ser iterado

`callback`

A função callback a ser usada

Se nenhum `callback` é fornecido, todas entradas vazias do `array` serão removidas. Veja empty() para saber como o PHP define vazia nesse caso.

`mode`

Flag determinando quais argumentos são passados para `callback`:

- `ARRAY_FILTER_USE_KEY` - passa chaves como argumentos para `callback` em vez de valor
- `ARRAY_FILTER_USE_BOTH` - passa tanto valor quanto chave como argumento para `callback` em vez do valor

Padrão é `0` que em vez disso irá passar valor como o único argumento para `callback`.

Valor Retornado

Retorna o array filtrado.

Registro de Alterações

Versão	Descrição
8.0.0	`callback` agora é anulável.
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um `E_WARNING`.

Examples

Exemplo #1 Exemplo da função array_filter()

```php
<?php
function impar($var)
{
    // retorna se o inteiro informado é impar
    return $var & 1;
}

function par($var)
{
    // retorna se o inteiro informado é par
    return !($var & 1);
}

$array1 = ['a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5];
$array2 = [6, 7, 8, 9, 10, 11, 12];

echo "Ímpares: \n";
print_r(array_filter($array1, "impar"));
echo "Pares: \n";
print_r(array_filter($array2, "par"));
?>
```

O exemplo acima produzirá:

```
Ímpares:
Array
(
    [a] => 1
    [c] => 3
    [e] => 5
)
Pares:
Array
(
    [0] => 6
    [2] => 8
    [4] => 10
    [6] => 12
)
```

Exemplo #2 array_filter() sem `callback`

```php
<?php

$entry = [
    0 => 'foo',
    1 => false,
    2 => -1,
    3 => null,
    4 => '',
    5 => '0',
    6 => 0,
];

print_r(array_filter($entry));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => foo
    [2] => -1
)
```

Exemplo #3 array_filter() com `mode`

```php
<?php

$arr = ['a' => 1, 'b' => 2, 'c' => 3, 'd' => 4];

var_dump(array_filter($arr, function($k) {
    return $k == 'b';
}, ARRAY_FILTER_USE_KEY));

var_dump(array_filter($arr, function($v, $k) {
    return $k == 'b' || $v == 4;
}, ARRAY_FILTER_USE_BOTH));
?>
```

O exemplo acima produzirá:

```
array(1) {
  ["b"]=>
  int(2)
}
array(2) {
  ["b"]=>
  int(2)
  ["d"]=>
  int(4)
}
```

Notas

> **Cuidado**
>
> Se o array for modificado por uma função de retorno (ex.: elemento adicionado ou indefinido), o comportamento desta função é indefinido.

Veja também

- array_intersect() - Calcula a interseção entre arrays
- array_find() - Retorna um primeiro elemento que satisfaz uma função de retorno
- array_any() - Verifica se pelo menos um elemento de um array satisfaz uma função de retorno
- array_map() - Aplica uma função em todos os elementos dos arrays dados
- array_reduce() - Reduz um array para um único valor através de um processo iterativo via callback

array_find

array_find

(PHP 8 >= 8.4.0)

array_find — Retorna um primeiro elemento que satisfaz uma função de retorno

Descrição

function **array_find**(array `$array`, callable `$callback`): mixed

array_find() retorna o valor do primeiro elemento de um array para o qual a função `callback` retorna **`true`**. Se nenhum elemento correspondente for encontrado, a função retorna **`null`**.

Parâmetros

`array`

O array a ser pesquisado.

`callback`

A função de retorno a ser chamada para verificar cada elemento, que precisa ter a seguinte assinatura:

function callback(mixed `$value`, mixed `$key`): bool
Se esta função retornar **`true`**, o valor é retornado por **array_find()** e a função de retorno não será chamada para os elementos restantes.

Valor Retornado

A função retorna o valor do primeiro elemento para o qual `callback` retorna **`true`**. Se nenhum elemento correspondente for encontrado, a função retorna **`null`**.

Examples

Exemplo #1 Exemplo de array_find()

```php
<?php
$array = [
    'a' => 'cachorro',
    'b' => 'gato',
    'c' => 'vaca',
    'd' => 'pato',
    'e' => 'elefante',
    'f' => 'ganso'
];

// Encontra o primeiro animal com nome maior que 4 letras.
var_dump(array_find($array, function (string $value) {
    return strlen($value) > 4;
}));

// Encontra o primeiro animal cujo nome começa com f.
var_dump(array_find($array, function (string $value) {
    return str_starts_with($value, 'f');
}));

// Encontra o primeiro animal onde a chave do array é a primeira letra do nome.
var_dump(array_find($array, function (string $value, $key) {
   return $value[0] === $key;
}));
```

```php
// Encontra o primeiro animal onde a chave do array corresponde a uma expressão regular.
var_dump(array_find($array, function ($value, $key) {
   return preg_match('/^([a-f])$/', $key);
}));
?>
```

O exemplo acima produzirá:

```
string(8) "cachorro"
NULL
string(8) "elefante"
string(3) "cachorro"
```

Veja também

- array_find_key() - Retorna a chave do primeiro elemento que satisfaz uma função de retorno
- array_all() - Verifica se todos os elementos de um array satisfazem uma função de retorno
- array_any() - Verifica se pelo menos um elemento de um array satisfaz uma função de retorno
- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_reduce() - Reduz um array para um único valor através de um processo iterativo via callback

array_find_key

array_find_key

(PHP 8 >= 8.4.0)

array_find_key — Retorna a chave do primeiro elemento que satisfaz uma função de retorno

Descrição

function **array_find_key**(array `$array`, callable `$callback`): mixed

array_find_key() retorna a chave do primeiro elemento de um array para o qual a função `callback` retorna `true`. Se nenhum elemento correspondente for encontrado, a função retorna `null`.

Parâmetros

`array`

O array a ser pesquisado.

`callback`

A função de retorno a ser chamada para verificar cada elemento, que precisa ter a seguinte assinatura:

function callback(mixed `$value`, mixed `$key`): bool
Se esta função retornar `true`, a chave é retornada por **array_find_key()** e a função de retorno não será chamada para os elementos restantes.

Valor Retornado

A função retorna a chave do primeiro elemento para o qual `callback` retorna `true`. Se nenhum elemento correspondente for encontrado, a função retorna `null`.

Examples

Exemplo #1 Exemplo de array_find_key()

```php
<?php
$array = [
    'a' => 'cachorro',
    'b' => 'gato',
```

```php
    'c' => 'vaca',
    'd' => 'pato',
    'e' => 'elefante',
    'f' => 'ganso'
];

// Encontra o primeiro animal com nome mais longo que 4 caracteres.
var_dump(array_find_key($array, function (string $value) {
    return strlen($value) > 4;
}));

// Encontra o primeiro animal cujo nome começa com f.
var_dump(array_find_key($array, function (string $value) {
    return str_starts_with($value, 'f');
}));

// Encontra o primeiro animal onde a chave do array é a primeira letra do animal.
var_dump(array_find_key($array, function (string $value, $key) {
   return $value[0] === $key;
}));

// Encontra o primeiro animal onde o chave do array corresponde a uma expressão regular.
var_dump(array_find_key($array, function ($value, $key) {
   return preg_match('/^([a-f])$/', $key);
}));
?>
```

O exemplo acima produzirá:

```
string(1) "a"
NULL
string(1) "e"
string(1) "a"
```

Veja também

- array_find() - Retorna um primeiro elemento que satisfaz uma função de retorno
- array_all() - Verifica se todos os elementos de um array satisfazem uma função de retorno
- array_any() - Verifica se pelo menos um elemento de um array satisfaz uma função de retorno
- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_reduce() - Reduz um array para um único valor através de um processo iterativo via callback

array_first

array_first

(PHP 8 >= 8.5.0)

array_first — Obtém o primeiro valor de um array

Descrição

function **array_first**(array `$array`): mixed

Obtém o primeiro valor do `array` fornecido.

Parâmetros

`array`

Um array.

Valor Retornado

Retorna o primeiro valor de `array` se o array não estiver vazio; `null` caso contrário.

Examples

Exemplo #1 Uso básico de array_first()

```php
<?php
$array = [1 => 'a', 0 => 'b', 3 => 'c', 2 => 'd'];

$firstValue = array_first($array);

var_dump($firstValue);
?>
```

O exemplo acima produzirá:

```
string(1) "a"
```

Veja também

- array_key_first() - Obtém a primeira chave de um array
- array_last() - Obtém o último valor de um array

array_flip

array_flip

(PHP 4, PHP 5, PHP 7, PHP 8)

array_flip — Permuta todas as chaves e seus valores associados em um array

Descrição

function **array_flip**(array `$array`): array

array_flip() retorna um array com suas relações trocadas, ou seja, as chaves de `array` passam a ser os valores e os valores de `array` passam a ser as chaves.

Note que os valores de `array` devem ser chaves válidas, ou seja, eles precisam ser int ou string. Um aviso será mostrado se um valor é de um tipo inválido para chaves, e o par chave/valor em questão *não será invertido*.

Se um valor tem várias ocorrências, a última chave será usada como valor, e todos os outros serão perdidos.

Parâmetros

`array`

Um array de pares chave/valor para inverter.

Valor Retornado

Retorna o array trocado.

Examples

Exemplo #1 Exemplo da função array_flip()

```php
<?php
$input = array("oranges", "apples", "pears");
$flipped = array_flip($input);

print_r($flipped);
?>
```

O exemplo acima produzirá:

```
Array
(
    [oranges] => 0
    [apples] => 1
    [pears] => 2
)
```

Exemplo #2 Exemplo da função array_flip(): colisão

```php
<?php
$trans = array("a" => 1, "b" => 1, "c" => 2);
$trans = array_flip($trans);

print_r($trans);
?>
```

O exemplo acima produzirá:

```
Array
(
    [1] => b
    [2] => c
)
```

Veja também

- array_values() - Retorna todos os valores de um array
- array_keys() - Retorna todas as chaves ou uma parte das chaves de um array
- array_reverse() - Retorna um array com os elementos na ordem inversa

array_intersect

array_intersect

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_intersect — Calcula a interseção entre arrays

Descrição

function **array_intersect**(array `$array`, array `...$arrays`): array

array_intersect() retorna um array contendo todos os valores de `array` que estão presentes nos outros argumentos. Note que as chaves são preservadas.

Parâmetros

`array`

O array com os valores que serão verificados.

`arrays`

Arrays para comparar os valores.

Valor Retornado

Retorna um array contendo todos os valores em `array` cujo existem em todos os parâmetros.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo da array_intersect()

```php
<?php
$array1 = array("a" => "verde", "vermelho", "azul");
$array2 = array("b" => "verde", "amarelo", "vermelho");
$result = array_intersect($array1, $array2);
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => verde
    [0] => vermelho
)
```

Notas

> **Nota** :
>
> Dois elementos são considerados iguais se, e somente se, `(string) $elem1 === (string) $elem2`. Em palavras: quando a representação em string é a mesma.

Veja também

- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice

array_intersect_assoc

array_intersect_assoc

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

array_intersect_assoc — Computa a interseção de arrays com uma adicional verificação de índice

Descrição

function **array_intersect_assoc**(array `$array`, array `...$arrays`): array

array_intersect_assoc() retorna um array contendo todos os valores de `array` que estão presentes em todos os argumentos. Note que estas chaves também são usadas na comparação diferindo de array_intersect().

Parâmetros

`array`

O array com os valores a serem verificados.

`arrays`

Arrays para comparar os valores.

Valor Retornado

Retorna um array associativo contendo todos os valores em `array` que estão presentes em todos os argumentos.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo da array_intersect_assoc()

```php
<?php
$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
$array2 = array("a" => "verde", "b" => "amarelo", "azul", "vermelho");
$result_array = array_intersect_assoc($array1, $array2);
print_r($result_array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => verde
)
```

Em nosso exemplo, é possível ver que somente o par `"a" => "verde"` está presente em ambos array, então logo é retornado. O valor `"vermelho"` não é retornado porque no *$array1* a chave é `0` quando a chave de "vermelho" no *$array2* é `1`, e a chave `"b"` não será retornada por que seu valor é diferente em cada array.

Os dois valores dos pares `key => value` são considerados iguais somente se `(string) $elem1 === (string) $elem2` . Em outras palavras, uma verificação do tipo é executada, então a representação string precisa ser a mesma.

Veja também

- array_intersect() - Calcula a interseção entre arrays
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas
- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice

array_intersect_key

array_intersect_key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_intersect_key — Calcula a interseção de arrays, comparando pelas chaves

Descrição

function **array_intersect_key**(array `$array`, array `...$arrays`): array

array_intersect_key() retorna um array contendo todos os valores de `array` que tem as mesmas chaves presentes em todos os argumentos.

Parâmetros

`array`

O array com as chaves a serem verificadas.

`arrays`

Arrays para comparar as chaves.

Valor Retornado

Retorna um array associativo contendo todos os valores de `array` que estão presentes em todos os argumentos.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo da array_intersect_key()

```php
<?php
$array1 = array('azul'  => 1, 'vermelho' => 2, 'verde'   => 3, 'roxo'  => 4);
$array2 = array('verde' => 5, 'azul'     => 6, 'amarelo' => 7, 'ciano' => 8);

var_dump(array_intersect_key($array1, $array2));
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["azul"]=>
  int(1)
  ["verde"]=>
  int(3)
}
```

Em nosso exemplo, é possível pode ver que somente as chaves `'azul'` e `'verde'` estão presentes em ambos os array e assim são retornados. Observe Também que os valores das chaves `'azul'` e `'verde'` diferem nos dois arrays. A combinação ocorre porque somente as chaves são verificadas. Os valores retornados são do `array`.

As duas chaves do par `key => value` são considerados iguais somente se `(string) $key1 === (string) $key2` . Em outras palavras, um verificação do tipo é executada, então a representação string precisa ser a mesma.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_diff_key() - Computa a diferença entre arrays usando as chaves na comparação

- array_diff_ukey() - Computa a diferença entre arrays usando uma função callback na comparação de chaves
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_intersect_ukey() - Computa a interseção de arrays usando uma função de retorno nas chaves para comparação

array_intersect_uassoc

array_intersect_uassoc

(PHP 5, PHP 7, PHP 8)

array_intersect_uassoc — Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback

Descrição

function **array_intersect_uassoc**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

array_intersect_uassoc() retorna um array contendo todos os valores de `array` que estão presentes em todos os argumentos. Note que as chaves são usadas na comparação, ao contrário da função array_intersect().

Parâmetros

`array`

Array inicial para comparação dos arrays.

`arrays`

Arrays para comparar as chaves.

`key_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Retorna os valores de `array` cujos valores existem em todos os argumentos.

Examples

Exemplo #1 Exemplo da função array_intersect_uassoc()

```php
<?php
$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
$array2 = array("a" => "VERDE", "B" => "marrom", "amarelo", "vermelho");

print_r(array_intersect_uassoc($array1, $array2, "strcasecmp"));
?>
```

O exemplo acima produzirá:

```
Array
(
    [b] => marrom
)
```

Veja também

- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas
- array_intersect_key() - Calcula a interseção de arrays, comparando pelas chaves
- array_intersect_ukey() - Computa a interseção de arrays usando uma função de retorno nas chaves para comparação

array_intersect_ukey

array_intersect_ukey

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_intersect_ukey — Computa a interseção de arrays usando uma função de retorno nas chaves para comparação

Descrição

function **array_intersect_ukey**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

array_intersect_ukey() retorna um array contendo todos os valores de `array` que têm as mesmas chaves presentes em todos os argumentos.

Parâmetros

`array`

Array inicial para comparação dos arrays.

`arrays`

Arrays para comparar as chaves.

`key_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Retorna os valores de `array` cujas chaves existem em todos os argumentos.

Examples

Exemplo #1 Exemplo da função array_intersect_ukey()

```php
<?php
function key_compare_func($key1, $key2)
{
    if ($key1 == $key2)
        return 0;
    else if ($key1 > $key2)
        return 1;
    else
        return -1;
```

```php
}

$array1 = array('azul'  => 1, 'vermelho' => 2, 'verde'   => 3, 'roxo'  => 4);
$array2 = array('verde' => 5, 'azul'     => 6, 'amarelo' => 7, 'rosa'  => 8);

var_dump(array_intersect_ukey($array1, $array2, 'key_compare_func'));
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["azul"]=>
  int(1)
  ["verde"]=>
  int(3)
}
```

Em nosso exemplo, é possível ver que somente as chaves `'azul'` e `'verde'` estão presentes em ambos os arrays e assim retornadas. Observe também que os valores para as chaves `'azul'` e `'verde'` diferem entre os dois arrays. Uma combinação ainda ocorre porque somente as chaves são verificadas. Os valores retornados são os do `array`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_diff_key() - Computa a diferença entre arrays usando as chaves na comparação
- array_diff_ukey() - Computa a diferença entre arrays usando uma função callback na comparação de chaves
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_intersect_key() - Calcula a interseção de arrays, comparando pelas chaves

array_is_list

array_is_list

(PHP 8 >= 8.1.0)

array_is_list — Verifica se um `array` é uma lista

Descrição

function **array_is_list**(array `$array`): bool

Determina se o `array` fornecido é uma lista. Um array é considerado uma lista se suas chaves consistirem em números consecutivos de `0` a `count($array)-1`.

Parâmetros

`array`

O array que está sendo avaliado.

Valor Retornado

Retorna `true` se `array` for uma lista, `false` caso contrário.

Examples

Exemplo #1 Exemplo de array_is_list()

```php
<?php
var_dump(array_is_list([])); // true
var_dump(array_is_list(['maçã', 2, 3])); // true
var_dump(array_is_list([0 => 'maçã', 'laranja'])); // true

// O array não começa em 0
var_dump(array_is_list([1 => 'maçã', 'laranja'])); // false

// As chaves não estão na ordem correta
var_dump(array_is_list([1 => 'maçã', 0 => 'laranja'])); // false

// Chaves não inteiras
var_dump(array_is_list([0 => 'maçã', 'foo' => 'bar'])); // false

// Chaves não consecutivas
var_dump(array_is_list([0 => 'maçã', 2 => 'bar'])); // false
?>
```

Notas

> **Nota** :
>
> Esta função retorna `true` em arrays vazios.

Veja também

- array_values() - Retorna todos os valores de um array

array_key_exists

array_key_exists

(PHP 4 >= 4.0.7, PHP 5, PHP 7, PHP 8)

array_key_exists — Verifica se uma chave ou índice existe em um array

Descrição

function **array_key_exists**(string|int|float|bool|resource|null `$key`, array `$array`): bool

array_key_exists() retorna `true` se a chave `key` existe no array. `key` pode ser qualquer valor possível para um índice de um array.

Parâmetros

`key`

Valor para verificar.

`array`

Um array com chaves para verificar.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

> **Nota** :
>
> **array_key_exists()** procurará pelas chaves na primeira dimensão apenas. Chaves aninhadas em arrays multidimensionais não serão encontradas.

Registro de Alterações

Versão	Descrição
8.5.0	Usar null no parâmetro `key` foi descontinuado, use uma string vazia no lugar.
8.0.0	O parâmetro `key` agora aceita bool, float, int, null, resource e string como argumentos.
8.0.0	Passar um object para o parâmetro `array` não é mais suportado.
7.4.0	Passar um object para o parâmetro `array` foi descontinuado. Use property_exists() em seu lugar.

Examples

Exemplo #1 Exemplo da função array_key_exists()

```php
<?php
$searchArray = ['primeiro' => 1, 'segundo' => 4];
var_dump(array_key_exists('first', $searchArray));
?>
```

O exemplo acima produzirá:

```
bool(true)
```

Exemplo #2 array_key_exists() versus isset()

isset() não retorna `true` para valores de chave que correspondam a um valor `null`, enquanto que **array_key_exists()** faz isto.

```php
<?php
$searchArray = ['primeiro' => null, 'segundo' => 4];

var_dump(isset($searchArray['first']));
var_dump(array_key_exists('first', $searchArray));
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
```

Veja também

- isset() - Determina se uma variável está declarada e é diferente de null
- array_keys() - Retorna todas as chaves ou uma parte das chaves de um array
- in_array() - Verifica se um valor existe em um array
- property_exists() - Verifica se o objeto ou a classe tem uma propriedade

array_key_first

array_key_first

(PHP 7 >= 7.3.0, PHP 8)

array_key_first — Obtém a primeira chave de um array

Descrição

function **array_key_first**(array `$array`): int|string|null

Obtém a primeira chave do `array` fornecido sem afetar o ponteiro do array interno.

Parâmetros

`array`

Um array.

Valor Retornado

Retorna a primeira chave do `array` se o array não estiver vazio; `null` caso contrário.

Examples

Exemplo #1 Uso básico de array_key_first()

```php
<?php
$array = ['a' => 1, 'b' => 2, 'c' => 3];

$firstKey = array_key_first($array);

var_dump($firstKey);
?>
```

O exemplo acima produzirá:

```
string(1) "a"
```

Notas

> **Dica**
>
> Existem várias maneiras de fornecer essa funcionalidade para versões anteriores ao PHP 7.3.0. É possível usar array_keys(), mas isso pode ser bastante ineficiente. Também é possível usar reset() e key(), mas isso pode alterar o ponteiro do array interno. Uma solução eficiente, que não altera o ponteiro interno do array, escrito como polyfill:
>
> ```php
> <?php
> if (!function_exists('array_key_first')) {
>     function array_key_first(array $arr) {
>         foreach($arr as $key => $unused) {
>             return $key;
>         }
>         return NULL;
>     }
> }
> ?>
> ```

Veja também

- array_first() - Obtém o primeiro valor de um array
- array_key_last() - Obtém a última chave de um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento

array_key_last

array_key_last

(PHP 7 >= 7.3.0, PHP 8)

array_key_last — Obtém a última chave de um array

Descrição

function **array_key_last**(array `$array`): int|string|null

Obtém a última chave do `array` fornecido sem afetar o ponteiro do array interno.

Parâmetros

`array`

Um array.

Valor Retornado

Retorna a última chave do `array` se o array não estiver vazio; `null` caso contrário.

Veja também

- array_last() - Obtém o último valor de um array
- array_key_first() - Obtém a primeira chave de um array
- end() - Define o ponteiro interno de um array para seu último elemento

array_keys

array_keys

(PHP 4, PHP 5, PHP 7, PHP 8)

array_keys — Retorna todas as chaves ou uma parte das chaves de um array

Descrição

function **array_keys**(array `$array`): array
function **array_keys**(array `$array`, mixed `$filter_value`, bool `$strict = false`): array

array_keys() retorna as chaves, numéricas e string, do array `array`.

Se um parâmetro `filter_value` for especificado, então apenas as chaves para esse valor serão retornadas. Do contrário, todas as chaves de `array` serão retornadas.

Parâmetros

`array`

Um array contendo chaves a serem retornadas.

`filter_value`

Se especificado, somente chaves contendo este valor são retornadas.

`strict`

Determina se a comparação é rígida (===) deve ser utilizada durante a busca.

Valor Retornado

Retorna um array de todas as chaves em `array`.

Examples

Exemplo #1 Exemplo de array_keys()

```php
<?php
$array = array(0 => 100, "cor" => "vermelho");
print_r(array_keys($array));

$array = array("azul", "vermelho", "verde", "azul", "azul");
print_r(array_keys($array, "azul"));

$array = array("cor"     => array("azul", "vermelho", "verde"),
               "tamanho" => array("pequeno", "medio", "grande"));
print_r(array_keys($array));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 0
    [1] => cor
)
Array
(
    [0] => 0
    [1] => 3
    [2] => 4
)
Array
(
    [0] => cor
    [1] => tamanho
)
```

Veja também

- array_values() - Retorna todos os valores de um array
- array_combine() - Cria um array usando um array para chaves e outro para valores
- array_key_exists() - Verifica se uma chave ou índice existe em um array
- array_search() - Procura por um valor em um array e retorna sua chave correspondente caso seja encontrado

array_last

array_last

(PHP 8 >= 8.5.0)

array_last — Obtém o último valor de um array

Descrição

function **array_last**(array `$array`): mixed

Obtém o último valor do `array` fornecido.

Parâmetros

`array`

Um array.

Valor Retornado

Retorna o último valor de `array` se o array não estiver vazio; `null` caso contrário.

Examples

Exemplo #1 Uso básico de array_last()

```php
<?php
$array = [1 => 'a', 0 => 'b', 3 => 'c', 2 => 'd'];

$lastValue = array_last($array);

var_dump($lastValue);
?>
```

O exemplo acima produzirá:

```
string(1) "d"
```

Veja também

- array_key_last() - Obtém a última chave de um array
- array_first() - Obtém o primeiro valor de um array

array_map

array_map

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

array_map — Aplica uma função em todos os elementos dos arrays dados

Descrição

function **array_map**(?callable `$callback`, array `$array`, array `...$arrays`): array

array_map() retorna um array contendo os resultados da aplicação do `callback` ao valor correspondente do `array` (e `arrays` se mais arrays forem fornecidos) usados como argumentos para a função de callback. O número de parâmetros que a função `callback` aceita deve coincidir com o número de arrays passados para **array_map()**. Arrays de entrada excedentes são ignorados. Um ArgumentCountError é lançado se um número insuficiente de argumentos é fornecido.

Parâmetros

`callback`

Um callable para executar para cada elemento em cada array.

`null` pode ser passado como um valor para `callback` para realizar uma operação zip em múltiplos arrays e retornar um array onde cada elemento é um array contendo os elementos dos arrays de entrada na mesma posição do ponteiro interno do array (veja o exemplo abaixo). Se apenas `array` for fornecido, **array_map()** retornará o array de entrada.

`array`

Um array para percorrer chamando função `callback`.

`arrays`

Lista variável suplementar de argumentos array para passar pela função `callback`.

Valor Retornado

Retorna um array contendo os resultados da aplicação da função `callback` ao valor correspondente do `array` (e `arrays` se mais arrays forem fornecidos) usados como argumentos para o callback.

O array retornado irá preservar as chaves do argumento array se e somente se exatamente um array for passado. Se mais de um array for passado, o array retornado terá chaves inteiras sequenciais.

Registro de Alterações

Versão	Descrição
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um `E_WARNING`.

Examples

Exemplo #1 Exemplo da função array_map()

```php
<?php
function cube($n)
{
    return ($n * $n * $n);
}

$a = [1, 2, 3, 4, 5];
```

```php
$b = array_map('cube', $a);
print_r($b);
?>
```

Faz com que *$b* tenha:

```
Array
(
    [0] => 1
    [1] => 8
    [2] => 27
    [3] => 64
    [4] => 125
)
```

Exemplo #2 array_map() utilizando uma função lambda

```php
<?php
$func = function(int $value): int {
    return $value * 2;
};

print_r(array_map($func, range(1, 5)));

// Ou a partir do PHP 7.4.0:

print_r(array_map(fn($value): int => $value * 2, range(1, 5)));

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 2
    [1] => 4
    [2] => 6
    [3] => 8
    [4] => 10
)
Array
(
    [0] => 2
    [1] => 4
    [2] => 6
    [3] => 8
    [4] => 10
)
```

Exemplo #3 array_map() - usando mais de um array

```php
<?php
function mostrar_Espanhol(int $n, string $m): string
{
    return "O número {$n} é chamado de {$m} em espanhol";
}

function map_Espanhol(int $n, string $m): array
{
```

```
    return [$n => $m];
}

$a = [1, 2, 3, 4, 5];
$b = ['uno', 'dos', 'tres', 'cuatro', 'cinco'];

$c = array_map('mostrar_Espanhol', $a, $b);
print_r($c);

$d = array_map('map_Espanhol', $a , $b);
print_r($d);
?>
```

O exemplo acima produzirá:

```
// Saída de $c
Array
(
    [0] => O número 1 é chamado de uno em espanhol
    [1] => O número 2 é chamado de dos em espanhol
    [2] => O número 3 é chamado de tres em espanhol
    [3] => O número 4 é chamado de cuatro em espanhol
    [4] => O número 5 é chamado de cinco em espanhol
)

// Saída de $d
Array
(
    [0] => Array
        (
            [1] => uno
        )

    [1] => Array
        (
            [2] => dos
        )

    [2] => Array
        (
            [3] => tres
        )

    [3] => Array
        (
            [4] => cuatro
        )

    [4] => Array
        (
            [5] => cinco
        )

)
```

Normalmente quando se usa dois ou mais arrays, eles devem ter o mesmo tamanho porque a função de retorno é aplicada paralelamente nos elementos correpondentes. Se os arrays tem tamanhos diferentes, o array menor será estendido com elementos vazios.

Uma forma interessante de se usar esta função é na construção de um array de arrays, o que pode ser facilmente feito usando `null` como o nome da função de retorno.

Exemplo #4 Realizando uma operação zip de arrays

```
<?php
$a = [1, 2, 3, 4, 5];
```

```php
$b = ['one', 'two', 'three', 'four', 'five'];
$c = ['uno', 'dos', 'tres', 'cuatro', 'cinco'];

$d = array_map(null, $a, $b, $c);
print_r($d);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => 1
            [1] => one
            [2] => uno
        )

    [1] => Array
        (
            [0] => 2
            [1] => two
            [2] => dos
        )

    [2] => Array
        (
            [0] => 3
            [1] => three
            [2] => tres
        )

    [3] => Array
        (
            [0] => 4
            [1] => four
            [2] => cuatro
        )

    [4] => Array
        (
            [0] => 5
            [1] => five
            [2] => cinco
        )

)
```

Exemplo #5 `callback` `null` apenas com `array`

```php
<?php
$array = [1, 2, 3];
var_dump(array_map(null, $array));
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
```

```
  int(3)
}
```

Exemplo #6 array_map() - com chaves string

```php
<?php
$arr = ['stringkey' => 'value'];
function cb1($a) {
    return [$a];
}
function cb2($a, $b) {
    return [$a, $b];
}
var_dump(array_map('cb1', $arr));
var_dump(array_map('cb2', $arr, $arr));
var_dump(array_map(null,  $arr));
var_dump(array_map(null, $arr, $arr));
?>
```

O exemplo acima produzirá:

```
array(1) {
  ["stringkey"]=>
  array(1) {
    [0]=>
    string(5) "value"
  }
}
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(5) "value"
    [1]=>
    string(5) "value"
  }
}
array(1) {
  ["stringkey"]=>
  string(5) "value"
}
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(5) "value"
    [1]=>
    string(5) "value"
  }
}
```

Exemplo #7 array_map() - arrays associativos

Enquanto **array_map()** não suporta diretamente o uso da chave do array como entrada, isso pode ser simulado usando array_keys().

```php
<?php
$arr = [
    'v1' => 'Primeiro lançamento',
    'v2' => 'Segundo lançamento',
    'v3' => 'Terceiro lançamento',
];
```

```php
// Nota: Antes da 7.4.0, use a sintaxe mais longa para funções anônimas.
$callback = fn(string $k, string $v): string => "$k foi o $v";

$result = array_map($callback, array_keys($arr), array_values($arr));

var_dump($result);
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  string(24) "v1 foi o Primeiro lançamento"
  [1]=>
  string(25) "v2 foi o Segundo lançamento"
  [2]=>
  string(24) "v3 foi o Terceiro lançamento"
}
```

Veja também

- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_reduce() - Reduz um array para um único valor através de um processo iterativo via callback
- array_walk() - Aplica uma determinada função em cada elemento de um array

array_merge

array_merge

(PHP 4, PHP 5, PHP 7, PHP 8)

array_merge — Combina um ou mais arrays

Descrição

function **array_merge**(array `...$arrays`): array

Combina os elementos de dois ou mais arrays de forma que os elementos de um são colocados no final do array anterior. Retorna o array resultante da fusão.

Se os arrays dados têm as mesmas chaves string, então o último valor para uma chave irá sobrescrever o valor anterior. Se, no entanto, os arrays tem as mesmas chaves numéricas, o último valor para uma chave **não** sobrescreverá o valor original, e sim adicionado ao array resultante.

Se apenas um array é dado e o array é indexado numericamente, as chaves são reindexadas de uma maneira contínua.

Parâmetros

`arrays`

Uma lista de arrays a combinar.

Valor Retornado

Retorna o array resultante. Se chamado sem argumentos, retorna um array vazio.

Registro de Alterações

Versão	Descrição
7.4.0	A função agora pode ser chamada sem argumentos. Anteriormente, ao menos um parâmetro era necessário.

Examples

Exemplo #1 Exemplo de array_merge()

```php
<?php

$array1 = ["cor" => "vermelho", 2, 4];
$array2 = ["a", "b", "cor" => "verde", "forma" => "trapezoide", 4];

$result = array_merge($array1, $array2);

print_r($result);
```

O exemplo acima produzirá:

```
Array
(
    [cor] => verde
    [0] => 2
    [1] => 4
    [2] => a
    [3] => b
    [forma] => trapezoide
    [4] => 4
)
```

Exemplo #2 Exemplo simples de array_merge()

```php
<?php

$array1 = [];
$array2 = [1 => "data"];

$result = array_merge($array1, $array2);

print_r($result);
```

Não esqueça que as chaves numéricas serão reordenadas!

```
Array
(
    [0] => data
)
```

Se a intenção for apenas adicionar os elementos do segundo array no primeiro, sem sobrescrever os elementos e sem causar a reindexação, deve-se utilizar o operador +:

```php
<?php

$array1 = [0 => 'zero_a', 2 => 'two_a', 3 => 'three_a'];
$array2 = [1 => 'one_b', 3 => 'three_b', 4 => 'four_b'];

$result = $array1 + $array2;

var_dump($result);
```

As chaves do primeiro array são preservadas. Se uma chave existir em dois arrays, então o elemento do primeiro array será preservado e o elemento de mesma chave no segundo array será descartado.

```
array(5) {
  [0]=>
  string(6) "zero_a"
  [2]=>
  string(5) "two_a"
  [3]=>
  string(7) "three_a"
  [1]=>
  string(5) "one_b"
  [4]=>
  string(6) "four_b"
}
```

Veja também

- array_merge_recursive() - Mescla um ou mais arrays recursivamente
- array_replace() - Substitui elementos de arrays passados no primeiro array
- array_combine() - Cria um array usando um array para chaves e outro para valores
- operadores de array

array_merge_recursive

array_merge_recursive

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_merge_recursive — Mescla um ou mais arrays recursivamente

Descrição

function **array_merge_recursive**(array `...$arrays`): array

array_merge_recursive() mescla os elementos de um ou mais arrays de forma que os elementos de um são colocados no final do array anterior. Retorna o array resultante da fusão.

Se os arrays dados tem as mesmas chaves string, então os valores para uma chave são mesclados em um array, e isso é feito recursivamente, sendo que, se um dos valores for um array também, este função irá mesclá-lo com os valores correspondentes no array resultante também. Se, no entanto, os arrays tem as mesmas chaves numéricas, o último valor para uma chave não sobrescreverá o valor original, e sim adicionado ao array resultante.

Parâmetros

`arrays`

Lista variável de arrays para mesclar recursivamente.

Valor Retornado

Um array de valores resultados da mesclagem dos argumentos. Se chamada sem nenhum argumento, retorna um array vazio.

Registro de Alterações

Versão	Descrição
7.4.0	Essa função agora pode ser chamada sem nenhum parâmetro. Anteriormente, pelo menos um parâmetro era necessário.

Examples

Exemplo #1 Exemplo de **array_merge_recursive()**

```php
<?php
$ar1 = array("cor" => array ("favorita" => "vermelho"), 5);
$ar2 = array(10, "cor" => array ("favorita" => "verde", "azul"));
$result = array_merge_recursive($ar1, $ar2);
```

```
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [cor] => Array
        (
            [favorita] => Array
                (
                    [0] => vermelho
                    [1] => verde
                )

            [0] => azul
        )

    [0] => 5
    [1] => 10
)
```

Veja também

- array_merge() - Combina um ou mais arrays
- array_replace_recursive() - Substitui elementos de arrays passado no primeiro array recursivamente

array_multisort

array_multisort

(PHP 4, PHP 5, PHP 7, PHP 8)

array_multisort — Ordena múltiplos arrays ou arrays multidimensionais

Descrição

```
function array_multisort(
    array &$array1,
    mixed $array1_sort_order = SORT_ASC,
    mixed $array1_sort_flags = SORT_REGULAR,
    mixed ...$rest
): true
```

array_multisort() pode ser usada para ordenar vários arrays de uma vez, ou um array multidimensional por uma ou mais dimensões.

Chaves associativas (string) serão mantidas, mas chaves numéricas serão reindexadas.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array1`

Um array a ser ordenado.

`array1_sort_order`

A ordenação para ser utilizada no argumento array anterior. Pode ser **`SORT_ASC`** para ordenar na ascendente (crescente) ou **`SORT_DESC`** para ordernar na descendente (decrescente).

Este argumento pode ser trocado com `array1_sort_flags` ou ser omitido completamente, e nesse caso é utilizado **`SORT_ASC`**.

`array1_sort_flags`

Opções de ordenamento para o argumento array anterior:

Flags:

- **`SORT_REGULAR`** - comparar itens normalmente (não modidifica tipos)
- **`SORT_NUMERIC`** - compara itens numericamente
- **`SORT_STRING`** - compara itens como strings
- **`SORT_LOCALE_STRING`** - compara itens como strings, utilizando a localidade atual. Utiliza a localidade, que pode ser modificada com setlocale()
- **`SORT_NATURAL`** - compara itens como strings utilizando a "ordenação natural" de natsort()
- **`SORT_FLAG_CASE`** - pode ser combinada (operação OR binária) com **`SORT_STRING`** ou **`SORT_NATURAL`** para ordenar as strings sem considerar maiúsculas e minúsculas

Este argumento pode ser trocado com `array1_sort_order` ou completamente omitido, e nesse caso é utilizado **`SORT_REGULAR`**.

`rest`

Outros arrays, podendo informar também ordem e opções de ordenação. Somente elementos correspondentes a elementos equivalentes nos arrays anteriores são comparados. Em outras palavras, a ordem é lexigráfica.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.5.0	Agora o tipo de retorno é **`true`**, antes era bool.

Examples

Exemplo #1 Ordenando vários arrays

```php
<?php
$ar1 = array(10, 100, 100, 0);
$ar2 = array(1, 3, 2, 4);
array_multisort($ar1, $ar2);

var_dump($ar1);
var_dump($ar2);
?>
```

Nesse exemplo, após a ordenação, o primeiro array conterá 0, 10, 100, 100. O segundo array conterá 4, 1, 2, 3. Os itens no segundo array correspondentes ao itens idênticos do primeiro array (100 e 100) também são ordenados.

```
array(4) {
  [0]=> int(0)
  [1]=> int(10)
  [2]=> int(100)
  [3]=> int(100)
}
array(4) {
```

```
  [0]=> int(4)
  [1]=> int(1)
  [2]=> int(2)
  [3]=> int(3)
}
```

Exemplo #2 Ordenando arrays multidimensionais

```php
<?php
$ar = array(
       array("10", 11, 100, 100, "a"),
       array(   1,  2, "2",   3,   1)
      );
array_multisort($ar[0], SORT_ASC, SORT_STRING,
                $ar[1], SORT_NUMERIC, SORT_DESC);
var_dump($ar);
?>
```

Neste exemplo, após a ordenação, o primeiro array será transformado para "10", 100, 100, 11, "a" (ordenado os itens como strings em ordem crescente). O segundo conterá 1, 3, "2", 2, 1 (ordenado como números, em ordem decrescente).

```
array(2) {
  [0]=> array(5) {
    [0]=> string(2) "10"
    [1]=> int(100)
    [2]=> int(100)
    [3]=> int(11)
    [4]=> string(1) "a"
  }
  [1]=> array(5) {
    [0]=> int(1)
    [1]=> int(3)
    [2]=> string(1) "2"
    [3]=> int(2)
    [4]=> int(1)
  }
}
```

Exemplo #3 Ordenando resultados de bancos de dados

Neste exemplo, cada elemento no array *data* representa um iten em uma tabela. Esse tipo de conjunto de dados é típico de registros de base de dados.

Exemplo:

```
volume | edition
-------+--------
    67 |       2
    86 |       1
    85 |       6
    98 |       2
    86 |       6
    67 |       7
```

Os dados estão em um array, chamado *data*. Isso pode ser obtido, por exemplo, de rodar mysqli_fetch_assoc() em loop.

Vamos ordenar *volume* decrescente e *edition* crescente.

Aqui temos um array de registros, mas **array_multisort()** exige um array de colunas, então é preciso converter os dados utilizando o código abaixo para então executar a ordenação.

```php
<?php
// Os dados como foram criados percorrendo o resultado de mysqli_fetch_assoc:
$data[] = array('volume' => 67, 'edition' => 2);
$data[] = array('volume' => 86, 'edition' => 1);
$data[] = array('volume' => 85, 'edition' => 6);
$data[] = array('volume' => 98, 'edition' => 2);
$data[] = array('volume' => 86, 'edition' => 6);
$data[] = array('volume' => 67, 'edition' => 7);

// Obtém uma lista de colunas
foreach ($data as $key => $row) {
    $volume[$key]  = $row['volume'];
    $edition[$key] = $row['edition'];
}

// Pode ser usada a função array_column() no lugar do código acima
$volume  = array_column($data, 'volume');
$edition = array_column($data, 'edition');

// Ordena os dados por volume decrescente, edição crescente.
// Adiciona $data como último parâmetro, para ordenar por uma chave comum.
array_multisort($volume, SORT_DESC, $edition, SORT_ASC, $data);

// Percorre os dados e exibe os valores ordenados para cada coluna
echo 'volume | edition', PHP_EOL;
echo '-------+--------', PHP_EOL;
for ($i = 0; $i < count($data); $i++) {
     printf("%6d | %7d\n", $volume[$i], $edition[$i]);
}
?>
```

O conjunto de dados agora está ordenado, assim:

```
volume | edition
-------+--------
    98 |       2
    86 |       1
    86 |       6
    85 |       6
    67 |       2
    67 |       7
```

Exemplo #4 Ordenação sem diferenciar maiúsculas/minúsculas

`SORT_STRING` e `SORT_REGULAR` são sensíveis a maiúsculas e minúsculas, de forma que strings começando com maiúsculas aparecerão primeiro que strings começando com minúsculas.

Para realizar uma ordenação sem diferenciar maiúsculas/minúsculas é preciso forçar para que a ordenação seja executada com uma cópia do array original, transformado em minúsculas.

```php
<?php
$array = array('Alpha', 'atomic', 'Beta', 'bank');
$array_lowercase = array_map('strtolower', $array);

array_multisort($array_lowercase, SORT_ASC, SORT_STRING, $array);

print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Alpha
    [1] => atomic
    [2] => bank
    [3] => Beta
)
```

Veja também

- usort() - Ordena um array pelos valores utilizando uma função de comparação definida pelo usuário
- Comparação entre as funções de ordenação de array

array_pad

array_pad

(PHP 4, PHP 5, PHP 7, PHP 8)

array_pad — Expande um array para um certo comprimento utilizando um determinado valor

Descrição

function **array_pad**(array `$array`, int `$length`, mixed `$value`): array

array_pad() retorna uma cópia de `array` expandido para o comprimento especificado por `length` com o valor `value`. Se `length` for positivo então o array é expandido pela direita, se for negativo, pela esquerda. Se o valor absoluto de `length` for menor ou igual ao comprimento de `array`, então ele permanece inalterado.

Parâmetros

`array`

Array inicial de valores para ser preenchido.

`length`

Nova tamanho do array.

`value`

Valor para preencher se `array` é menor que `length`.

Valor Retornado

Retorna uma cópia de `array` preenchido pela quantidade especificada por `length` com o valor `value`. Se `length` é positivo, então o array é preenchido na direita, se ele é negativo, então da esquerda. Se o valor absoluto de `length` é menor que ou igual ao comprimento de `array` então nenhum preenchimento ocorre.

Registro de Alterações

Versão	Descrição
8.3.0	Antes do PHP 8.3.0, somente 1.048.576 elementos podiam ser adicionados de uma vez. Agora é limitado apenas pelo tamanho máximo de um array.

Examples

Exemplo #1 Exemplo de array_pad()

```php
<?php
$input = array(12, 10, 9);
```

```php
$result = array_pad($input, 5, 0);
// $result é array(12, 10, 9, 0, 0)
echo join(', ', $result), PHP_EOL;

$result = array_pad($input, -7, -1);
// $result é array(-1, -1, -1, -1, 12, 10, 9)
echo join(', ', $result), PHP_EOL;

$result = array_pad($input, 2, "noop");
// Não será expandido
echo join(', ', $result), PHP_EOL;
?>
```

Veja também

- array_fill() - Preenche um array com valores
- range() - Cria um array contendo uma faixa de elementos

array_pop

array_pop

(PHP 4, PHP 5, PHP 7, PHP 8)

array_pop — Extrai um elemento do final do array

Descrição

function **array_pop**(array `&$array`): mixed

array_pop() extrai e retorna o valor do último elemento de `array`, diminuindo o `array` em um elemento.

> **Nota** :
>
> Após o uso desta função, assim como na função reset(), o ponteiro interno do array fornecido apontará para o primeiro elemento.

Parâmetros

`array`

O array para obter os valores.

Valor Retornado

Retorna o valor do último elemento de `array`. Se `array` é vazio, `null` será retornado.

Examples

Exemplo #1 Exemplo da array_pop()

```php
<?php
$stack = array("orange", "banana", "apple", "raspberry");
$fruit = array_pop($stack);
print_r($stack);
?>
```

Depois disso, *$stack* irá conter apenas 3 elementos:

```
Array
(
    [0] => orange
    [1] => banana
    [2] => apple
)
```

e `raspberry` será passado para *$fruit*.

Veja também

- array_push() - Adiciona um ou mais elementos no final de um array
- array_shift() - Retira o primeiro elemento de um array
- array_unshift() - Adiciona um ou mais elementos no início de um array

array_product

array_product

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_product — Calcula o produto dos valores de um array

Descrição

function **array_product**(array `$array`): int|float

array_product() retorna o produto dos valores de um array.

Parâmetros

`array`

O array.

Valor Retornado

Retorna o produto com um inteiro ou float.

Registro de Alterações

Versão	Descrição
8.3.0	Agora emite um `E_WARNING` quando os valores do `array` não puderem ser convertidos para int ou float. Anteriormente, arrays e objects eram ignorados enquanto outros valores eram convertidos para int. Adicionalmente, objetos que definem uma conversão numérica (ex.: GMP) agora são convertidos ao invés de ignorados.

Examples

Exemplo #1 Exemplo da **array_product()**

```php
<?php

$a = array(2, 4, 6, 8);
echo "product(a) = " . array_product($a) . "\n";
echo "product(array()) = " . array_product(array()) . "\n";

?>
```

O exemplo acima produzirá:

```
product(a) = 384
product(array()) = 1
```

array_push

array_push

(PHP 4, PHP 5, PHP 7, PHP 8)

array_push — Adiciona um ou mais elementos no final de um array

Descrição

function **array_push**(array `&$array`, mixed `...$values`): int

array_push() trata `array` como uma pilha, e adiciona as variáveis passadas como argumentos no final de `array`. O tamanho do `array` aumenta de acordo com o número de variáveis adicionadas. Tem o mesmo efeito de:

```
<?php
$array[] = $var;
?>
```

repetido para cada argumento.

> **Nota** :
>
> Se a função **array_push()** for usada para adicionar um elemento ao array, é melhor usar `$array[] =` porque desta forma não há uma chamada a uma função.

> **Nota** :
>
> **array_push()** irá emitir um aviso se o primeiro argumento não for um array. Isto era diferente do funcionamento de `$var[]` onde um novo array era criado, antes do PHP 7.1.0.

Parâmetros

`array`

O array de entrada.

`values`

Os valores a serem colocados no final do `array`.

Valor Retornado

Retorna o novo número de elementos do array.

Registro de Alterações

Versão	Descrição
7.3.0	Essa função agora pode ser chamada com apenas um parâmetro. Anteriormente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo de array_push()

```
<?php
$cesta = array("laranja", "morango");
array_push($cesta, "melancia", "batata");
print_r($cesta);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => laranja
    [1] => morango
    [2] => melancia
```

```
    [3] => batata
)
```

Veja também

- array_pop() - Extrai um elemento do final do array
- array_shift() - Retira o primeiro elemento de um array
- array_unshift() - Adiciona um ou mais elementos no início de um array

array_rand

array_rand

(PHP 4, PHP 5, PHP 7, PHP 8)

array_rand — Escolhe uma ou mais chaves aleatórias de um array

Descrição

function **array_rand**(array `$array`, int `$num` = 1): int|string|array

Escolhe aleatóriamente um ou mais itens de um array, e retorna a chave (ou chaves) do itens selecionados.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**
>
> Esta função usa a instância global do Mt19937 (“Mersenne Twister”) como a fonte de aleatoriedade a por isso compartilha seu estado com todas as outras funções que usam o Mt19937 global. Usar qualquer uma destas funções avança a sequência para *todas* as outras funções, independentemente do escopo.
>
> Gerar sequências repetíveis ao semear mt_srand() ou srand() com um valor conhecido também gerará saída repetível desta função.
>
> Prefira usar métodos de Random\Randomizer em todo código novo.

Parâmetros

`array`

O array de entrada. Não pode ser vazio.

`num`

Especifica quantos elementos deseja obter. Deve ser maior que zero, e menor ou igual ao comprimento do `array`.

Valor Retornado

Se estiver selecionando apenas um elemento, **array_rand()** retorna a chave para este elemento aleatório. De outra forma, ele retorna um array com as chaves desses elementos selecionados. Assim é possível selecionar os elementos pelas chaves no array assim como valores aleatórios. Se múltiplas chaves forem retornadas, elas serão retornadas na ordem em que estavam presentes no array original.

Erros/Exceções

Lança uma exceção ValueError se `array` estiver vazio, ou se `num` estiver fora do intervalo permitido.

Registro de Alterações

Versão	Descrição
8.0.0	**array_rand()** agora lança uma exceção ValueError se `num` estiver fora do intervalo; anteriormente um `E_WARNING` era emitido, e a função retornava `null`.
8.0.0	**array_rand()** agora lança uma exceção ValueError se `array` estiver vazio; anteriormente um `E_WARNING` era emitido, e a função retornava `null`.
7.1.0	O algoritmo de embaralhamento interno foi alterado para usar o Gerador de Números Aleatórios » Mersenne Twister em vez da função rand da libc.

Examples

Exemplo #1 Exemplo de array_rand()

```php
<?php
$input = array("Neo", "Morpheus", "Trinity", "Cypher", "Tank");
$rand_keys = array_rand($input, 2);
echo $input[$rand_keys[0]] . "\n";
echo $input[$rand_keys[1]] . "\n";
?>
```

Veja também

- Random\Randomizer::pickArrayKeys() - Seleciona chaves aleatórias de um array
- Random\Randomizer::shuffleArray() - Obtém uma permutação de um array

array_reduce

array_reduce

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

array_reduce — Reduz um array para um único valor através de um processo iterativo via callback

Descrição

function **array_reduce**(array `$array`, callable `$callback`, mixed `$initial` = `null`): mixed

array_reduce() aplica iterativamente a função `callback` aos elementos de `array`, de forma a reduzir o array a um único valor.

Parâmetros

`array`

O array de entrada.

`callback`

function callback(mixed `$carry`, mixed `$item`): mixed

`carry`

Informa o valor da iteração anterior. No caso da primeira iteração ele terá o valor de `initial`.

`item`

Mantém o valor da iteração atual.

`initial`

Se o argumento opcional `initial` for passado, ele será utilizado no início do processo, ou como um resultado final se o array estiver vazio.

Valor Retornado

Retorna o valor resultante.

Se a array estiver vazio e `initial` não for passado, **array_reduce()** retornará `null`.

Registro de Alterações

Versão	Descrição
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um `E_WARNING`.

Examples

Exemplo #1 Exemplo de array_reduce()

```php
<?php
function sum($carry, $item)
{
    $carry += $item;
    return $carry;
}

function product($carry, $item)
{
    $carry *= $item;
    return $carry;
}

$a = array(1, 2, 3, 4, 5);
$x = array();

var_dump(array_reduce($a, "sum")); // int(15)
var_dump(array_reduce($a, "product", 10)); // int(1200), because: 10*1*2*3*4*5
var_dump(array_reduce($x, "sum", "No data to reduce")); // string(17) "No data to reduce"
?>
```

Veja também

- array_filter() - Filtra elementos de um array utilizando uma função callback
- array_map() - Aplica uma função em todos os elementos dos arrays dados
- array_unique() - Remove os valores duplicados de um array
- array_count_values() - Conta as ocorrências de cada valor distinto em um array

array_replace

array_replace

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

array_replace — Substitui elementos de arrays passados no primeiro array

Descrição

function **array_replace**(array `$array`, array `...$replacements`): array

array_replace() cria um novo array e atribui itens a ele para cada chave em cada um dos arrays fornecidos. Se uma chave aparecer em múltiplos arrays de entrada, o valor do array de entrada mais à direita será usado.

array_replace() não processa os itens dos elementos recursivamente, ela substitui o valor inteiro para cada chave quando faz uma substituição.

Parâmetros

`array`

O array no qual os elementos são substituídos.

`replacements`

Arrays dos quais os elementos serão extraídos. Os valores dos arrays posteriores substituem os valores anteriores.

Valor Retornado

Retorna um array.

Examples

Exemplo #1 Exemplo de array_replace()

```php
<?php
$base = array("laranja", "banana", "maçã", "framboesa");
$substituicoes = array(0 => "abacaxi", 4 => "cereja");
$substituicoes2 = array(0 => "uva");

$cesta = array_replace($base, $substituicoes, $substituicoes2);
var_dump($cesta);
?>
```

O exemplo acima produzirá:

```
array(5) {
  [0]=>
  string(3) "uva"
  [1]=>
  string(6) "banana"
  [2]=>
  string(4) "maçã"
  [3]=>
  string(9) "framboesa"
  [4]=>
  string(6) "cereja"
}
```

Exemplo #2 Exemplo de como arrays aninhados são manipulados

```php
<?php
$base = [ 'cítricas' => [ 'laranja', 'limão' ], 'pomóideas' => [ 'maçã' ] ];
$substituicoes = [ 'cítricas' => [ 'toranja' ] ];
$substituicoes2 = [ 'cítricas' => [ 'kumquat', 'cidra' ], 'pomóideas' => [ 'nêspera' ] ];

$cesta = array_replace($base, $substituicoes, $substituicoes2);
var_dump($cesta);
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["cítricas"]=>
  array(2) {
    [0]=>
    string(7) "kumquat"
    [1]=>
    string(5) "cidra"
  }
  ["pomóideas"]=>
  array(1) {
    [0]=>
    string(7) "nêspera"
  }
}
```

Veja também

- array_replace_recursive() - Substitui elementos de arrays passado no primeiro array recursivamente
- array_merge() - Combina um ou mais arrays

array_replace_recursive

array_replace_recursive

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

array_replace_recursive — Substitui elementos de arrays passado no primeiro array recursivamente

Descrição

function **array_replace_recursive**(array `$array`, array `...$replacements`): array

array_replace_recursive() substitui os valores de `array` pelos mesmos valores de todos os arrays seguintes. Se uma chave do primeiro array existir no segundo array, seu valor será substituído pelo valor do segundo array. Se a chave existir no segundo array, e não no primeiro, ela será criada no primeiro array. Se uma chave existir apenas no primeiro array, ela será deixada como está. Se vários arrays forem passados para substituição, eles serão processados em ordem, o array posterior sobrescrevendo os valores anteriores.

array_replace_recursive() é recursivo: ele será recursivo em arrays e aplicará o mesmo processo ao valor interno.

Quando o valor do primeiro array for escalar, ele será substituído pelo valor do segundo array, seja ele escalar ou array. Quando o valor no primeiro array e no segundo array são ambos arrays, **array_replace_recursive()** substituirá seus respectivos valores recursivamente.

Parâmetros

`array`

O array no qual os elementos são substituídos.

`replacements`

Arrays dos quais os elementos serão extraídos.

Valor Retornado

Returns an array.

Examples

Exemplo #1 Exemplo de array_replace_recursive()

```php
<?php
$base = array('frutas_citricas' => array("laranja") , 'frutas_vermelhas' => array("amora", "framboesa"));
$substituicoes = array('frutas_citricas' => array('abacaxi'), 'frutas_vermelhas' => array('mirtilo'));

$cesta = array_replace_recursive($base, $substituicoes);
print_r($cesta);

$cesta = array_replace($base, $substituicoes);
print_r($cesta);
?>
```

O exemplo acima produzirá:

```
Array
(
    [frutas_citricas] => Array
        (
            [0] => abacaxi
        )

    [frutas_vermelhas] => Array
        (
            [0] => mirtilo
            [1] => framboesa
        )

)
Array
(
```

```
    [frutas_citricas] => Array
        (
            [0] => abacaxi
        )

    [frutas_vermelhas] => Array
        (
            [0] => mirtilo
        )

)
```

Exemplo #2 array_replace_recursive() e recursive behavior

```php
<?php
$base = array('frutas_citricas' => array("laranja") , 'frutas_vermelhas' => array("amora", "framboesa"), 'outras' => 'banana'
);
$substituicoes = array('frutas_citricas' => 'abacaxi', 'frutas_vermelhas' => array('mirtilo'), 'outras' => array('lichia'));
$substituicoes2 = array('frutas_citricas' => array('abacaxi'), 'frutas_vermelhas' => array('mirtilo'), 'outras' => 'lichia');

$cesta = array_replace_recursive($base, $substituicoes, $substituicoes2);
print_r($cesta);

?>
```

O exemplo acima produzirá:

```
Array
(
    [frutas_citricas] => Array
        (
            [0] => abacaxi
        )

    [frutas_vermelhas] => Array
        (
            [0] => mirtilo
            [1] => framboesa
        )

    [outras] => lichia
)
```

Veja também

- array_replace() - Substitui elementos de arrays passados no primeiro array
- array_merge_recursive() - Mescla um ou mais arrays recursivamente

array_reverse

array_reverse

(PHP 4, PHP 5, PHP 7, PHP 8)

array_reverse — Retorna um array com os elementos na ordem inversa

Descrição

function **array_reverse**(array `$array`, bool `$preserve_keys = false`): array

Recebe como entrada `array` e retorna um novo array com a ordem dos elementos invertida.

Parâmetros

`array`

O array de entrada.

`preserve_keys`

Se definido para `true` as chaves serão preservadas. Chaves não numéricas não são afetadas por essa opção, e são sempre preservadas.

Valor Retornado

Retorna o array invertido.

Examples

Exemplo #1 Exemplo de array_reverse()

```php
<?php
$input  = array("php", 4.0, array("green", "red"));
$reversed = array_reverse($input);
$preserved = array_reverse($input, true);

print_r($input);
print_r($reversed);
print_r($preserved);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => php
    [1] => 4
    [2] => Array
        (
            [0] => green
            [1] => red
        )

)
Array
(
    [0] => Array
        (
            [0] => green
            [1] => red
        )

    [1] => 4
    [2] => php
)
Array
(
    [2] => Array
        (
            [0] => green
            [1] => red
        )

    [1] => 4
    [0] => php
)
```

Veja também

- array_flip() - Permuta todas as chaves e seus valores associados em um array

array_search

array_search

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

array_search — Procura por um valor em um array e retorna sua chave correspondente caso seja encontrado

Descrição

function **array_search**(mixed `$needle`, array `$haystack`, bool `$strict = false`): int|string|false

Procura a `needle` (agulha) em `haystack` (palheiro).

Parâmetros

`needle`

O valor procurado.

> **Nota** :
>
> Se `needle` for uma string, a comparação é feita de uma maneira que diferencia maiúsculas e minúsculas.

`haystack`

O array.

`strict`

Se o terceiro parâmetro opcional `strict` for informador como `true` então **array_search()** fará uma busca por elementos *idênticos* em `haystack`. Isto significa que haverá uma checagem adicional de tipos no parâmetro `needle` ao percorrer `haystack`, de forma que os objetos sejam a mesma instância.

Valor Retornado

Retorna a chave para `needle` se foi encontrada no array, `false` caso contrário.

Se `needle` for encontrado em `haystack` mais de uma vez, a primeira chave encontrada é retornada. Para retornar as chaves de todos os valores correspondentes, utilize a função array_keys() com o parâmetro opcional `filter_value`.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Examples

Exemplo #1 Exemplo de array_search()

```php
<?php
$array = array(0 => 'blue', 1 => 'red', 2 => 'green', 3 => 'red');

$key = array_search('green', $array); // $key = 2;
print_r($key);

$key = array_search('red', $array);   // $key = 1;
print_r($key);
?>
```

Veja também

- array_keys() - Retorna todas as chaves ou uma parte das chaves de um array
- array_values() - Retorna todos os valores de um array

- array_key_exists() - Verifica se uma chave ou índice existe em um array
- in_array() - Verifica se um valor existe em um array

array_shift

array_shift

(PHP 4, PHP 5, PHP 7, PHP 8)

array_shift — Retira o primeiro elemento de um array

Descrição

function **array_shift**(array `&$array`): mixed

array_shift() retira o primeiro elemento de `array` e o retorna, diminuindo `array` em um elemento e movendo todos os outros elementos para trás. Todas as chaves numéricas serão alteradas para começar a contar a partir de zero, enquanto chaves literais não serão afetadas.

> **Nota** :
>
> Após o uso desta função, assim como na função reset(), o ponteiro interno do array fornecido apontará para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Retorna o valor removido, ou `null` se `array` for vazio ou não é um array.

Examples

Exemplo #1 Exemplo da array_shift()

```php
<?php
$stack = array("orange", "banana", "apple", "raspberry");
$fruit = array_shift($stack);
print_r($stack);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => banana
    [1] => apple
    [2] => raspberry
)
```

e `orange` seria passado para *$fruit*.

Veja também

- array_unshift() - Adiciona um ou mais elementos no início de um array
- array_push() - Adiciona um ou mais elementos no final de um array
- array_pop() - Extrai um elemento do final do array

array_slice

array_slice

(PHP 4, PHP 5, PHP 7, PHP 8)

array_slice — Extrai uma parcela de um array

Descrição

```
function array_slice(
    array $array,
    int $offset,
    ?int $length = null,
    bool $preserve_keys = false
): array
```

array_slice() retorna a sequência de elementos de um `array` delimitado pelos parâmetros `offset` e `length`.

Parâmetros

`array`

O array de entrada.

`offset`

Se `offset` for não negativo, a sequência iniciará no início do `array`.

Se `offset` for negativo, a sequência iniciará a partir do fim do `array`.

> **Nota** :
>
> O parâmetro `offset` denota uma posição dentro do array, não das chaves.

`length`

Se `length` for positivo, então a sequência terá até essa quantidade de elementos.

Se o array for mais curto que `length`, então somente os elementos presentes serão retornados.

Se `length` for negativo então a sequência conterá até essa quantidade de elementos do final do array.

Se esse parâmetro for omitido, a sequência conterá todos os elementos a partir de `offset` até o final do `array`.

`preserve_keys`

> **Nota** :
>
> **array_slice()** irá reordenar e resetar os índices inteiros do array por padrão. Este comportamento pode ser alterado ao informar `preserve_keys` para `true`. Chaves string são sempre preservados, independente do parâmetro.

Valor Retornado

Retorna a parcela. Se o offset for maior que o tamanho do array, um array vazio é retornado.

Examples

Exemplo #1 Exemplos de array_slice()

```php
<?php
$input = array("a", "b", "c", "d", "e");

$output = array_slice($input, 2);      // retorna "c", "d", e "e"
$output = array_slice($input, -2, 1);  // retorna "d"
$output = array_slice($input, 0, 3);   // retorna "a", "b", e "c"

// Obverse as diferenças nas chaves retornadas
print_r(array_slice($input, 2, -1));
print_r(array_slice($input, 2, -1, true));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => c
    [1] => d
)
Array
(
    [2] => c
    [3] => d
)
```

Exemplo #2 array_slice() e arrays baseados em 1

```php
<?php
$input = array(1 => "a", "b", "c", "d", "e");
print_r(array_slice($input, 1, 2));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => b
    [1] => c
)
```

Exemplo #3 array_slice() com chaves mistas

```php
<?php
$ar = array('a'=>'apple', 'b'=>'banana', '42'=>'pear', 'd'=>'orange');
print_r(array_slice($ar, 0, 3));
print_r(array_slice($ar, 0, 3, true));
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => apple
    [b] => banana
    [0] => pear
)
Array
(
    [a] => apple
    [b] => banana
    [42] => pear
)
```

Veja também

- array_chunk() - Divide um array em pedaços
- array_splice() - Remove uma parte array e a substitui por outra coisa
- unset() - unset remove a definição de uma variável informada

array_splice

array_splice

(PHP 4, PHP 5, PHP 7, PHP 8)

array_splice — Remove uma parte array e a substitui por outra coisa

Descrição

```
function array_splice(
    array &$array,
    int $offset,
    ?int $length = null,
    mixed $replacement = []
): array
```

Remove os elementos especificados por `offset` e `length` do array `array`, e os substitui com os elementos do array `replacement`, se for especificado.

> **Nota** :
>
> Chaves numéricas em `array` não são preservadas.

> **Nota** :
>
> Se `replacement` não for um array, ele será convertido para um array (i.e. `(array) $replacement`). Isso pode resultar em um comportamento inesperado ao usar um objeto ou `replacement` `null`.

Parâmetros

`array`

O array de entrada.

`offset`

Se `offset` for positivo então o começo da região removida será nesse deslocamento a partir do início do array `array`.

Se `offset` for negativo então o começo da região removida será nesse deslocamento a partir do final do array `array`.

`length`

Se `length` for omitido, remove tudo a partir de `offset` até o final do array.

Se `length` for especificado e for positivo, então essa quantidade de elementos será removida.

Se `length` for especificado e for negativo, então o final da região removida será essa quantidade de elementos a partir do final do array.

Se `length` for especificado e for zero, nenhum elemento será removido.

> **Dica**
>
> Para remover tudo a partir de `offset` até o final do array quando `replacement` também é especificado, use `count($input)` para `length`.

`replacement`

Se o array `replacement` for especificado, então os elementos removidos serão substituídos pelo elementos desse array.

Se `offset` e `length` são tais que nada é removido, então os elementos do array `replacement` serão inseridos no lugar especificado pelo `offset`.

> **Nota** :
>
> Chaves em `replacement` não são preservadas.

Se `replacement` for de apenas um elemento então não é necessário colocar `array()` ou colchetes ao redor dele, a não ser que elementos seja um array em si, um objeto ou `null`.

Valor Retornado

Retorna um array que consiste nos elementos extraídos.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora é anulável.

Examples

Exemplo #1 Exemplos da array_splice()

```php
<?php
$input = array("vermelho", "verde", "azul", "amarelo");
array_splice($input, 2);
var_dump($input);

$input = array("vermelho", "verde", "azul", "amarelo");
array_splice($input, 1, -1);
var_dump($input);

$input = array("vermelho", "verde", "azul", "amarelo");
array_splice($input, 1, count($input), "laranja");
var_dump($input);

$input = array("vermelho", "verde", "azul", "amarelo");
array_splice($input, -1, 1, array("preto", "marrom"));
var_dump($input);
?>
```

O exemplo acima produzirá:

```
array(2) {
  [0]=>
  string(3) "vermelho"
  [1]=>
  string(5) "verde"
}
array(2) {
  [0]=>
  string(3) "vermelho"
  [1]=>
  string(6) "amarelo"
}
array(2) {
  [0]=>
  string(3) "vermelho"
  [1]=>
  string(6) "laranja"
}
array(5) {
  [0]=>
  string(3) "vermelho"
  [1]=>
  string(5) "verde"
```

```
  [2]=>
  string(4) "azul"
  [3]=>
  string(5) "preto"
  [4]=>
  string(6) "marrom"
}
```

Exemplo #2 Declarações equivalentes a vários exemplos da array_splice()

As seguintes declarações são equivalentes:

```php
<?php

// adiciona dois elementos a $input
array_push($input, $x, $y);
array_splice($input, count($input), 0, array($x, $y));

// remove o último elemento de $input
array_pop($input);
array_splice($input, -1);

// remove o primeiro elemento de $input
array_shift($input);
array_splice($input, 0, 1);

// insere um elemento no início de $input
array_unshift($input, $x, $y);
array_splice($input, 0, 0, array($x, $y));

// substitui o valor em $input no índice $x
$input[$x] = $y; // para arrays onde a chave é igual ao deslocamento
array_splice($input, $x, 1, $y);

?>
```

Veja também

- array_merge() - Combina um ou mais arrays
- array_slice() - Extrai uma parcela de um array
- unset() - unset remove a definição de uma variável informada

array_sum

array_sum

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

array_sum — Calcula a soma dos elementos de um array

Descrição

function **array_sum**(array `$array`): int|float

array_sum() retorna a soma dos valores de um array.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Retorna a soma de valores como um inteiro ou float; `0` se o `array` estiver vazio.

Registro de Alterações

Versão	Descrição
8.3.0	Agora emite um `E_WARNING` quando os valores do `array` não puderem ser convertidos para int ou float. Anteriormente, arrays e objects eram ignorados enquanto outros valores eram convertidos para int. Adicionalmente, objetos que definem uma conversão numérica (ex.: GMP) agora são convertidos ao invés de ignorados.

Examples

Exemplo #1 Exemplos da array_sum()

```php
<?php
$a = array(2, 4, 6, 8);
echo "soma(a) = ".array_sum($a)."\n";

$b = array("a" => 1.2, "b" => 2.3, "c" => 3.4);
echo "soma(b) = ".array_sum($b)."\n";
?>
```

O exemplo acima produzirá:

```
soma(a) = 20
soma(b) = 6.9
```

array_udiff

array_udiff

(PHP 5, PHP 7, PHP 8)

array_udiff — Computa a diferença de arrays usando uma função de callback para comparação dos dados

Descrição

function **array_udiff**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computa a diferença de arrays usando uma função de callback para comparação dos dados. Esta é contrária a array_diff() que usa uma função interna para comparar os dados.

Parâmetros

`array`

O primeiro array.

`arrays`

Arrays para comparar.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**

> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Valor Retornado

Retorna um array contendo todos os valores de `array` que não estão presentes em qualquer dos outros argumentos.

Examples

Exemplo #1 Exemplo de array_udiff() usando Objetos stdClass

```php
<?php
// Arrays para comprar
$array1 = array(new stdClass, new stdClass,
                new stdClass, new stdClass,
               );

$array2 = array(
                new stdClass, new stdClass,
               );

// Define algumas propriedades para cada objeto
$array1[0]->largura = 11; $array1[0]->altura = 3;
$array1[1]->largura = 7;  $array1[1]->altura = 1;
$array1[2]->largura = 2;  $array1[2]->altura = 9;
$array1[3]->largura = 5;  $array1[3]->altura = 7;

$array2[0]->largura = 7;  $array2[0]->altura = 5;
$array2[1]->largura = 9;  $array2[1]->altura = 2;

function comparar_por_area($a, $b) {
    $areaA = $a->largura * $a->altura;
    $areaB = $b->largura * $b->altura;

    if ($areaA < $areaB) {
        return -1;
    } elseif ($areaA > $areaB) {
        return 1;
    } else {
        return 0;
    }
}

print_r(array_udiff($array1, $array2, 'comparar_por_area'));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => stdClass Object
        (
            [largura] => 11
            [altura] => 3
        )
```

```
    [1] => stdClass Object
        (
            [largura] => 7
            [altura] => 1
        )

)
```

Exemplo #2 Exemplo de array_udiff() usando Objetos DateTime

```php
<?php
class MeuCalendario {
    public $livres = array();
    public $reservadas = array();

    public function __construct($semana = 'now') {
        $inicio = new DateTime($semana);
        $inicio->modify('Monday this week midnight');
        $fim = clone $inicio;
        $fim->modify('Friday this week midnight');
        $intervalo = new DateInterval('P1D');
        foreach (new DatePeriod($inicio, $intervalo, $fim) as $tempoLivre) {
            $this->livres[] = $tempoLivre;
        }
    }

    public function marcarCompromisso(DateTime $data, $nota) {
        $this->reservadas[] = array('data' => $data->modify('midnight'), 'nota' => $nota);
    }

    public function checarDisponibilidade() {
        return array_udiff($this->livres, $this->reservadas, array($this, 'compararPersonalizado'));
    }

    public function compararPersonalizado($livres, $reservadas) {
        if (is_array($livres)) $a = $livres['data'];
        else $a = $livres;
        if (is_array($reservadas)) $b = $reservadas['data'];
        else $b = $reservadas;
        if ($a == $b) {
            return 0;
        } elseif ($a > $b) {
            return 1;
        } else {
            return -1;
        }
    }
}

// Cria um calendário para compromissos semanais
$meuCalendario = new MeuCalendario;

// Marca alguns compromissos para esta semana
$meuCalendario->marcarCompromisso(new DateTime('Monday this week'), "Limpando apartamento do GoogleGuy.");
$meuCalendario->marcarCompromisso(new DateTime('Wednesday this week'), "Fazendo uma viagem de snowboarding.");
$meuCalendario->marcarCompromisso(new DateTime('Friday this week'), "Corrigindo código bugado.");

// Checar disponibilidade de dias comparando datas $reservadas com datas $livres
echo "Estou disponível nos seguintes dias desta semana...\n\n";
foreach ($meuCalendario->checarDisponibilidade() as $livre) {
    echo $livre->format('l'), "\n";
}
echo "\n\n";
echo "Estou ocupado(a) nos seguintes dias desta semana...\n\n";
foreach ($meuCalendario->reservadas as $reservada) {
    echo $reservada['data']->format('l'), ": ", $reservada['nota'], "\n";
}
?>
```

O exemplo acima produzirá:

```
Estou disponível nos seguintes dias desta semana...

Tuesday
Thursday


Estou ocupado(a) nos seguintes dias desta semana...

Monday: Limpando apartamento do GoogleGuy.
Wednesday: Fazendo uma viagem de snowboarding.
Friday: Corrigindo código bugado.
```

Notas

> **Nota** :
>
> Observe que esta função verifica somente uma dimensão de um array n-dimensional. É claro que dimensões mais profundas podem ser verificadas usando `array_udiff($array1[0], $array2[0], "func_compara_dados");`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_udiff_assoc

array_udiff_assoc

(PHP 5, PHP 7, PHP 8)

array_udiff_assoc — Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno

Descrição

function **array_udiff_assoc**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno.

> **Nota** :
>
> Observe que esta função somente verifica uma dimensão de um array n-dimensional. Obviamente dimensões mais profundas podem ser verificadas usando, por exemplo, `array_udiff_assoc($array1[0], $array2[0], "alguma_função_de_comparação");`.

Parâmetros

`array`

O primeiro array.

`arrays`

Arrays para comparar.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**
>
> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Valor Retornado

array_udiff_assoc() retorna um array contendo todos os valores de `array` que não estão presentes em qualquer dos outros argumentos. Observe que as chaves são usadas na comparação, ao contrário de array_diff() e array_udiff(). A comparação dos dados dos arrays é executada usando uma função de retorno fornecida pelo usuário. Neste aspecto, o comportamento é oposto ao de array_diff_assoc(), que usa uma função interna para comparação.

Examples

Exemplo #1 Exemplo de **array_udiff_assoc()**

```php
<?php
class cr {
    private $priv_member;
    function __construct($val)
    {
        $this->priv_member = $val;
    }

    static function comp_func_cr($a, $b)
    {
        if ($a->priv_member === $b->priv_member) return 0;
        return ($a->priv_member > $b->priv_member)? 1:-1;
    }
}

$a = array("0.1" => new cr(9), "0.5" => new cr(12), 0 => new cr(23), 1=> new cr(4), 2 => new cr(-15),);
$b = array("0.2" => new cr(9), "0.5" => new cr(22), 0 => new cr(3), 1=> new cr(4), 2 => new cr(-15),);

$result = array_udiff_assoc($a, $b, array("cr", "comp_func_cr"));
```

```
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0.1] => cr Object
        (
            [priv_member:cr:private] => 9
        )

    [0.5] => cr Object
        (
            [priv_member:cr:private] => 12
        )

    [0] => cr Object
        (
            [priv_member:cr:private] => 23
        )

)
```

Em nosso exemplo acima é possível ver que o par `"1" => new cr(4)` está presente em ambos os arrays e assim ele não está presente na saída da função.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_diff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice que feita por uma função de callback fornecida pelo usuário
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_udiff_uassoc() - Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_udiff_uassoc

array_udiff_uassoc

(PHP 5, PHP 7, PHP 8)

array_udiff_uassoc — Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback

Descrição

function **array_udiff_uassoc**(
  array `$array`,
  array `...$arrays`,
  callable `$value_compare_func`,
  callable `$key_compare_func`
): array

Computa a diferença entre arrays com checagem adicional de índice, compara dados e índices por uma função de callback.

Note que as chaves são usadas na comparação, ao contrário de array_diff() e array_udiff().

Parâmetros

`array`

O primeiro array.

`arrays`

Arrays para comparar.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**
>
> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

`key_compare_func`

A comparação das chaves (índices) é feita também por função de callback: `key_compare_func`. Este comportamento é contrário ao que array_udiff_assoc() faz, já que esta compara os índices usando uma função interna.

Valor Retornado

Retorna um array contendo todos os valores de `array` que não estão presentes em qualquer dos outros argumentos.

Examples

Exemplo #1 Exemplo de array_udiff_uassoc()

```php
<?php
class cr {
    private $priv_member;
    function __construct($val)
    {
        $this->priv_member = $val;
    }

    static function comp_func_cr($a, $b)
    {
        if ($a->priv_member === $b->priv_member) return 0;
        return ($a->priv_member > $b->priv_member)? 1:-1;
    }
```

```
    static function comp_func_key($a, $b)
    {
        if ($a === $b) return 0;
        return ($a > $b)? 1:-1;
    }
}
$a = array("0.1" => new cr(9), "0.5" => new cr(12), 0 => new cr(23), 1=> new cr(4), 2 => new cr(-15),);
$b = array("0.2" => new cr(9), "0.5" => new cr(22), 0 => new cr(3), 1=> new cr(4), 2 => new cr(-15),);

$result = array_udiff_uassoc($a, $b, array("cr", "comp_func_cr"), array("cr", "comp_func_key"));
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0.1] => cr Object
        (
            [priv_member:cr:private] => 9
        )

    [0.5] => cr Object
        (
            [priv_member:cr:private] => 12
        )

    [0] => cr Object
        (
            [priv_member:cr:private] => 23
        )
)
```

Em nosso exemplo acima é possível ver que o par `"1" => new cr(4)` está presente em ambos os arrays e assim não está na saída da função. Mantenha em mente que duas funções de retorno devem ser fornecidas.

Notas

> **Nota** :
>
> Observe que esta função somente verifica uma dimensão de um array n-dimensional. É claro que dimensões mais profundas podem ser verificadas usando, por exemplo, `array_udiff_uassoc($array1[0], $array2[0], "func_compara_dados", "func_compara_chave");`.

Veja também

- array_diff() - Computa as diferenças entre arrays
- array_diff_assoc() - Computa a diferença entre arrays com checagem adicional de índice
- array_udiff() - Computa a diferença de arrays usando uma função de callback para comparação dos dados
- array_udiff_assoc() - Computa a diferença entre arrays com verificação adicional de índice, comparando dados por uma função de retorno
- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_uintersect

array_uintersect

(PHP 5, PHP 7, PHP 8)

array_uintersect — Computa a interseção de array, comparando dados com uma função callback

Descrição

function **array_uintersect**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computa a interseção de arrays, compara as informações usando uma função callback.

Parâmetros

`array`

O primeiro array.

`arrays`

Arrays para comparar.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**
>
> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Valor Retornado

Retorna um array contendo todos os valores de `array` que estão presentes em todos os argumentos.

Examples

Exemplo #1 Exemplo da array_uintersect()

```php
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "GREEN", "B" => "brown", "yellow", "red");

print_r(array_uintersect($array1, $array2, "strcasecmp"));
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => green
    [b] => brown
    [0] => red
)
```

Veja também

- array_intersect() - Calcula a interseção entre arrays
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_uintersect_assoc

array_uintersect_assoc

(PHP 5, PHP 7, PHP 8)

array_uintersect_assoc — Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback

Descrição

function **array_uintersect_assoc**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback.

Note que as chaves são utilizadas na comparação, diferente de array_uintersect(). Os dados são comparados utilizando funções de callback.

Parâmetros

`array`

Primeiro array.

`arrays`

Arrays para comparar.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**
>
> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:

```php
<?php
$arrayA = ["string", 1];
$arrayB = [["value" => 1]];
// $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
$compareFunc = static function ($item1, $item2) {
    $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
    $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
    return $value1 <=> $value2;
};
?>
```

Valor Retornado

Retorna um array contendo todos os valores de `array` que estão presentes em todos os argumentos.

Examples

Exemplo #1 Exemplo de array_uintersect_assoc()

```php
<?php
$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
$array2 = array("a" => "VERDE", "B" => "marrom", "amarelo", "vermelho");

print_r(array_uintersect_assoc($array1, $array2, "strcasecmp"));
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => verde
)
```

Veja também

- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_uintersect_uassoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

array_uintersect_uassoc

array_uintersect_uassoc

(PHP 5, PHP 7, PHP 8)

array_uintersect_uassoc — Computa a interseção de arrays com checagem adicional de índice, compara os dados e os índices utilizando funções de callback separadas

Descrição

function **array_uintersect_uassoc**(
  array `$array1`,
  array `...$arrays`,
  callable `$value_compare_func`,
  callable `$key_compare_func`
): array

Computa a interseção de arrays com checagem adicional de índice, compara dados e índices utilizando funções de callback separadas.

Parâmetros

`array1`

Primeiro array.

`arrays`

Outros arrays.

`value_compare_func`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

> **Cuidado**
>
> A função chamada de ordenação deve lidar com qualquer valor de qualquer array em qualquer ordem, independentemente da ordem em que eles foram originalmente fornecidos. O motivo é que cada array individual é primeiramente ordenado antes de ser comparado com outros arrays. Por exemplo:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 e $item2 podem ser "string", 1 ou ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

`key_compare_func`

Função para comparação das chaves (índices).

Valor Retornado

Retorna um array contendo todos os valores de `array1` que estão presentes em todos os argumentos.

Examples

Exemplo #1 Exemplo de array_uintersect_uassoc()

```php
<?php
$array1 = array("a" => "verde", "b" => "marrom", "c" => "azul", "vermelho");
$array2 = array("a" => "VERDE", "B" => "marrom", "amarelo", "vermelho");

print_r(array_uintersect_uassoc($array1, $array2, "strcasecmp", "strcasecmp"));
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => verde
    [b] => marrom
)
```

Veja também

- array_uintersect() - Computa a interseção de array, comparando dados com uma função callback
- array_intersect_assoc() - Computa a interseção de arrays com uma adicional verificação de índice
- array_intersect_uassoc() - Computa a interseção de arrays com checagem de índice adicional, compara índices por uma função de callback
- array_uintersect_assoc() - Computa a interseção de arrays com checagem adicional de índice, compara os dados utilizando uma função de callback

array_unique

array_unique

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_unique — Remove os valores duplicados de um array

Descrição

function **array_unique**(array `$array`, int `$flags = SORT_STRING`): array

Recebe uma entrada `array` e retorna um novo array sem valores duplicados.

Note que as chaves são preservadas. Se vários elementos forem iguais sob as `flags` fornecidas, então a chave e o valor do primeiro elemento igual serão retidos.

> **Nota** :
>
> Dois elementos são considerados iguais se, e somente se, `(string) $elem1 === (string) $elem2`, ou seja, quando a representação em string é a mesma, o primeiro elemento será usado.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser utilizado para modificar o comportamento de comparação usando estes valores:

Flags de tipo de comparação:

- `SORT_REGULAR` - compara os itens normalmente (não altera tipos)
- `SORT_NUMERIC` - compara itens numericamente
- `SORT_STRING` - compara itens como strings
- `SORT_LOCALE_STRING` - compara itens como strings baseado na localidade atual.

Valor Retornado

Retorna o array filtrado.

Registro de Alterações

Versão	Descrição
7.2.0	Se `flags` for `SORT_STRING`, o `array` anterior foi copiado e os elementos não únicos foram removidos (sem empacotar o array em seguida), mas um novo array é criado ao adicionar os novos elementos. Isto pode resultar em índices numéricos diferentes.

Examples

Exemplo #1 Exemplo de array_unique()

```php
<?php

$input = ["a" => "verde", "vermelho", "b" => "verde", "azul", "vermelho"];
$result = array_unique($input);
print_r($result);

?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => verde
    [0] => vermelho
    [1] => azul
)
```

Exemplo #2 array_unique() e tipos

```php
<?php

$input = [4, "4", "3", 4, 3, "3"];
$result = array_unique($input);
var_dump($result);

?>
```

O exemplo acima produzirá:

```
array(2) {
  [0]=>
  int(4)
  [2]=>
  string(1) "3"
}
```

Notas

> **Nota** :
>
> Note que **array_unique()** não se destina a funcionar em arrays multidimensionais.

Veja também

- array_count_values() - Conta as ocorrências de cada valor distinto em um array

array_unshift

array_unshift

(PHP 4, PHP 5, PHP 7, PHP 8)

array_unshift — Adiciona um ou mais elementos no início de um array

Descrição

function **array_unshift**(array `&$array`, mixed `...$values`): int

array_unshift() adiciona os elementos passados como argumentos no início de `array`. Note que a lista de elementos é adicionada como um todo, de forma que eles ficam na mesma ordem. Todas as chaves numéricas serão modificadas para começar a contar de zero enquanto chaves literais permanecerão inalteradas.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`values`

Os valores a serem adicionados.

Valor Retornado

Retorna o novo número de elementos em `array`.

Registro de Alterações

Versão	Descrição
7.3.0	Essa função agora pode ser chamada com apenas um parâmetro. Anteriomente, pelo menos dois parâmetros eram necessários.

Examples

Exemplo #1 Exemplo de array_unshift()

```php
<?php

$cesta = [
    "laranja",
    "banana"
];

array_unshift($cesta, "melancia", "morango");

var_dump($cesta);

?>
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  string(8) "melancia"
  [1]=>
  string(7) "morango"
  [2]=>
  string(7) "laranja"
  [3]=>
  string(6) "banana"
}
```

Exemplo #2 Uso de arrays associativos

Se um array associativo é pre anexado a outro array associativo, o array anexado é numeralmente indexado no array anterior.

```php
<?php

$frutas = [
    'maca' => [
        'McIntosh' => 'vermelho',
        'Granny Smith' => 'verde',
    ],
    'laranja' => [
        'Navel' => 'laranja',
        'Valencia' => 'laranja',
    ],
];

$vegetais = [
    'alface' => [
        'Iceberg' => 'verde',
        'Butterhead' => 'verde',
    ],
    'cenoura' => [
        'Deep Purple Hybrid' => 'roxo',
        'Imperator' => 'laranja',
    ],
    'pepino' => [
        'Kirby' => 'verde',
        'Gherkin' => 'verde',
    ],
];

array_unshift($frutas, $vegetais);

var_dump($frutas);

?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  array(3) {
    ["alface"]=>
    array(2) {
      ["Iceberg"]=>
      string(5) "verde"
      ["Butterhead"]=>
      string(5) "verde"
    }
    ["cenoura"]=>
    array(2) {
      ["Deep Purple Hybrid"]=>
      string(4) "roxo"
      ["Imperator"]=>
      string(7) "laranja"
    }
    ["pepino"]=>
    array(2) {
      ["Kirby"]=>
      string(5) "verde"
      ["Gherkin"]=>
      string(5) "verde"
    }
  }
  ["maca"]=>
  array(2) {
    ["McIntosh"]=>
    string(8) "vermelho"
    ["Granny Smith"]=>
    string(5) "verde"
```

```
  }
  ["laranja"]=>
  array(2) {
    ["Navel"]=>
    string(7) "laranja"
    ["Valencia"]=>
    string(7) "laranja"
  }
}
```

Veja também

- array_merge() - Combina um ou mais arrays
- array_shift() - Retira o primeiro elemento de um array
- array_push() - Adiciona um ou mais elementos no final de um array
- array_pop() - Extrai um elemento do final do array

array_values

array_values

(PHP 4, PHP 5, PHP 7, PHP 8)

array_values — Retorna todos os valores de um array

Descrição

function **array_values**(array `$array`): array

array_values() retorna todos os valores do array `array` num array indexado numericamente.

Parâmetros

`array`

O array.

Valor Retornado

Retorna um array indexado de valores.

Examples

Exemplo #1 Exemplo de array_values()

```php
<?php
$array = array("tamanho" => "G", "cor" => "dourado");
print_r(array_values ($array));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => G
    [1] => dourado
)
```

Veja também

- array_keys() - Retorna todas as chaves ou uma parte das chaves de um array

- array_combine() - Cria um array usando um array para chaves e outro para valores

array_walk

array_walk

(PHP 4, PHP 5, PHP 7, PHP 8)

array_walk — Aplica uma determinada função em cada elemento de um array

Descrição

function **array_walk**(array|object `&$array`, callable `$callback`, mixed `$arg` = `null`): true

Aplica uma função definida pelo usuário passada pelo argumento `callback` em cada elemento do `array`.

array_walk() não é afetado pelo ponteiro interno do `array`. **array_walk()** percorrerá todo o array desconsiderando a posição do ponteiro.

Parâmetros

`array`

O array de entrada.

`callback`

Normalmente, `callback` recebe dois parâmetros. O valor do parâmetro `array` é o primeiro e a chave/índice é o segundo.

> **Nota** :
>
> Se `callback` realmente precisar alterar os valores do array, especifique que o primeiro parâmetro de `callback` deve ser passado por referência. Assim, qualquer mudança feita nesses elementos serão feitas no próprio array também.

> **Nota** :
>
> Muitas funções internas (por exemplo strtolower()) lançarão uma exceção se forem passados mais argumentos que o esperado e não forem utilizados diretamente na função `callback`.

Apenas os valores de um `array` podem potencialmente ser alterados; sua estrutura não pode ser alterada, por exemplo, o programador não pode adicionar, remover ou reordenar elementos. Se a função não respeitar esta regra, o comportamento desta função é indefinido e imprevisível.

`arg`

Se o parâmetro opcional `arg` for fornecido, será passado como o terceiro parâmetro para a função `callback`.

Valor Retornado

Sempre retorna `true`.

Erros/Exceções

A partir do PHP 7.1.0, uma exceção ArgumentCountError será lançada se a função `callback` requerer mais de 2 parâmetros (o valor e a chave do item de array), ou se mais de 3 argumentos no caso de `arg` ser informado. Anteriormente, nesses casos um erro de nível E_WARNING seria gerado em cada vez que **array_walk()** chamasse a função `callback`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.
8.0.0	Se `callback` espera que o segundo ou o terceiro parâmetro seja passado por referência, essa função agora emite um `E_WARNING`.

Examples

Exemplo #1 Exemplo de array_walk()

```php
<?php
$frutas = array("d" => "limão", "a" => "laranja", "b" => "banana", "c" => "melancia");

function test_alter(&$item1, $key, $prefix)
{
    $item1 = "$prefix: $item1";
}

function test_print($item2, $key)
{
    echo "$key. $item2\n";
}

echo "Antes ...:\n";
array_walk($frutas, 'test_print');

array_walk($frutas, 'test_alter', 'fruta');
echo "... e depois:\n";

array_walk($frutas, 'test_print');
?>
```

O exemplo acima produzirá:

```
Antes ...:
d. limão
a. laranja
b. banana
c. melancia
... e depois:
d. fruit: limão
a. fruit: laranja
b. fruit: banana
c. fruit: melancia
```

Exemplo #2 Exemplo de array_walk() e funções anônimas

```php
<?php
$elements = ['a', 'b', 'c'];

array_walk($elements, function ($value, $key) {
  echo "{$key} => {$value}\n";
});

?>
```

O exemplo acima produzirá:

```
0 => a
1 => b
2 => c
```

Veja também

- array_walk_recursive() - Aplica um função do usuário recursivamente para cada membro de um array
- iterator_apply() - Chama uma função para cada elemento em um iterador
- list() - Cria variáveis como se fossem arrays
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor

- call_user_func_array() - Chama uma função de retorno com um array de parâmetros
- array_map() - Aplica uma função em todos os elementos dos arrays dados
- `foreach`

array_walk_recursive

array_walk_recursive

(PHP 5, PHP 7, PHP 8)

array_walk_recursive — Aplica um função do usuário recursivamente para cada membro de um array

Descrição

function **array_walk_recursive**(array|object `&$array`, callable `$callback`, mixed `$arg` = `null`): true

Aplica a função `callback` definida pelo usuário para cada elemento do array `array`. Esta função irá recursar em arrays mais profundos.

Passar um objeto como o `array` é permitido, e será tratado como se tivesse sido convertido para um array primeiro. Independentemente de a função ter começado com um array ou um objeto, ela não irá recursar em objetos internos.

Parâmetros

`array`

O array de entrada.

`callback`

Tipicamente, `callback` usa dois parâmetros. O valor do parâmetro `array` sendo o primeiro, e a chave/índice o segundo.

> **Nota** :
>
> Se `callback` precisa trabalhar com atuais valores da array, especifique o primeiro parâmetro da `callback` com uma referência. Então, as modificações feitas para estes elementos serão feitas no próprio array original.

`arg`

Se o parâmetro opcional `arg` for fornecido, ele será passado como o terceiro parâmetro para o `callback`.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo da array_walk_recursive()

```php
<?php
$doce = array('a' => 'maçã', 'b' => 'banana');
$frutas = array('doce' => $doce, 'azedo' => 'limão');

function test_print($item, $chave)
{
    echo "$chave contém $item\n";
}

array_walk_recursive($frutas, 'test_print');
?>
```

O exemplo acima produzirá:

```
a contém maçã
b contém banana
azedo contém limão
```

Pode-se notar que a chave '`doce`' nunca é exibida. Qualquer chave que contém um array não será passada para a função.

Veja também

- array_walk() - Aplica uma determinada função em cada elemento de um array

arsort

arsort

(PHP 4, PHP 5, PHP 7, PHP 8)

arsort — Ordena um array em ordem descrescente e mantém a associação de índices

Descrição

function **arsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Ordena o `array` em ordem decrescente, de forma que as chaves mantenham suas correlações com os valores aos quais estão associadas.

É usada principalmente para ordenar arrays associativos onde a ordem dos elementos é um fator significativo.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()
- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de arsort()

```
<?php
$frutas = array("d" => "limão", "a" => "laranja", "b" => "banana", "c" => "maçã");
arsort($frutas);
foreach ($frutas as $chave => $valor) {
    echo "$chave = $valor\n";
}
?>
```

O exemplo acima produzirá:

```
c = maçã
d = limão
a = laranja
b = banana
```

As frutas foram ordenadas na ordem alfabética inversa, e os índices associados a cada elemento foram mantidos.

Veja também

- sort() - Ordena um array em ordem ascendente
- asort() - Ordena um array em ordem crescente e mantém a associação de índices
- Comparação entre as funções de ordenação de array

asort

asort

(PHP 4, PHP 5, PHP 7, PHP 8)

asort — Ordena um array em ordem crescente e mantém a associação de índices

Descrição

function **asort**(array `&$array`, int `$flags` = `SORT_REGULAR`): true

Ordena um `array` em ordem crescente, se forma que as chaves mantenham suas correlações com os valores aos quais estão associadas.

Isto é usado principalmente ao ordenar arrays associativos onde a ordem real dos elementos é significativa.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- **SORT_REGULAR** - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- **SORT_NUMERIC** - compara os elementos numericamente
- **SORT_STRING** - compara os elementos como strings
- **SORT_LOCALE_STRING** - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- **SORT_NATURAL** - compara os elementos como strings usando "ordenação natural" como natsort()
- **SORT_FLAG_CASE** - pode ser combinada (usando o operador bit a bit OR) com **SORT_STRING** ou **SORT_NATURAL** para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna **true**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **true**; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de asort()

```php
<?php
$frutas = array("d" => "limão", "a" => "laranja", "b" => "banana", "c" => "maçã");
asort($frutas);
foreach( $frutas as $chave => $valor ){
    echo "$chave = $valor\n";
}
?>
```

O exemplo acima produzirá:

```
b = banana
a = laranja
d = limão
c = maçã
```

As frutas foram ordenadas na ordem alfabética, e os índices associados a cada elemento foram mantidos.

Veja também

- sort() - Ordena um array em ordem ascendente
- arsort() - Ordena um array em ordem descrescente e mantém a associação de índices
- Comparação entre as funções de ordenação de array

compact

compact

(PHP 4, PHP 5, PHP 7, PHP 8)

compact — Cria um array contendo variáveis e seus valores

Descrição

function **compact**(array|string `$var_name`, array|string `...$var_names`): array

Cria um array contendo variáveis e seus valores.

Para cada um dos parâmetros passados, **compact()** procura uma variável com o nome especificado na tabela de símbolos atual e a adiciona no array de saída de forma que o nome da variável se torne a chave e o conteúdo da variável se torne o valor para essa chave. Em resumo, ela faz o oposto de extract().

> **Nota** :
>
> Antes do PHP 7.3, quaisquer strings que não estejam definidas seriam silenciosamente ignoradas.

Parâmetros

`var_name`
`var_names`

compact() recebe um número variável de parâmetros. Cada parâmetro pode ser ou uma string contendo o nome da variável, ou um array de nomes de variáveis. O array pode conter outros arrays de nomes de variáveis; **compact()** o manipula recursivamente.

Valor Retornado

Retorna a saída do array com todas as variáveis adicionadas a ele.

Erros/Exceções

compact() emite um erro nível `E_WARNING` se uma determinada string se refere a uma variável não definida.

Registro de Alterações

Versão	Descrição
8.0.0	Se uma determinada string fizer referência a uma variável não definida, um erro de nível `E_WARNING` será emitido.
7.3.0	**compact()** agora emite um erro nível `E_NOTICE` se uma determinada string se refere a uma variável não definida. Anteriomente, strings como essa seriam ignoradas silenciosamente.

Examples

Exemplo #1 Exemplo de compact()

```php
<?php

$cidade = "Sao Paulo";
$estado = "SP";
$evento = "SIGGRAPH";

$vars_localidade = array("cidade", "estado");

$result = compact("evento", $vars_localidade);
print_r($result);

?>
```

O exemplo acima produzirá:

```
Array
(
    [evento] => SIGGRAPH
    [cidade] => Sao Paulo
    [estado] => SP
)
```

Notas

> **Nota** : **Armadilha**
>
> Devido a variáveis variável não poderem ser usadas com arrays superglobais do PHP dentro de funções, os arrays superglobais não podem ser passados na **compact()**.

Veja também

- extract() - Importa variáveis para a tabela de símbolos a partir de um array

count

count

(PHP 4, PHP 5, PHP 7, PHP 8)

count — Conta todos os elementos de um array ou de um objeto Countable

Descrição

function **count**(Countable|array `$value`, int `$mode` = `COUNT_NORMAL`): int

Conta todos os elementos de um array quando usado com um array. Quando usado com um objeto que implementa a interface Countable, retorna o valor de retorno do método Countable::count().

Parâmetros

`value`

Um array ou objeto Countable.

`mode`

Se o parâmetro opcional `mode` for definido para `COUNT_RECURSIVE` (ou 1), a função **count()** irá recursivamente contar o array. Isto é particularmente útil para contar todos os elementos de um array multidimensional.

> **Cuidado**
>
> **count()** pode detectar recursividade para evitar um loop infinito, mas irá emitir um aviso `E_WARNING` cada vez que o fizer (caso o array contenha a si mesmo mais de uma vez) e retornará uma contagem maior que a esperada.

Valor Retornado

Retorna o número de elementos em `value`. Antes do PHP 8.0.0, se o parâmetro não fosse um array nem um object que implementa a interface Countable, `1` seria retornado, a menos que `value` fosse `null`, e neste caso `0` seria retornado.

Registro de Alterações

Versão	Descrição
8.0.0	**count()** irá agora disparar TypeError em tipos contáveis inválidos passados no parâmetro `value`.
7.2.0	**count()** irá agora emitir um aviso em tipos contáveis inválidos passados no parâmetro `value`.

Examples

Exemplo #1 Exemplo da função **count()**

```php
<?php
$a[0] = 1;
$a[1] = 3;
$a[2] = 5;
var_dump(count($a));
```

```php
$b[0]  = 7;
$b[5]  = 9;
$b[10] = 11;
var_dump(count($b));
?>
```

O exemplo acima produzirá:

```
int(3)
int(3)
```

Exemplo #2 Exemplo de count() em não-Countable|array (contra exemplo - não deve ser feito)

```php
<?php
$b[0]  = 7;
$b[5]  = 9;
$b[10] = 11;
var_dump(count($b));

var_dump(count(null));

var_dump(count(false));
?>
```

O exemplo acima produzirá:

```
int(3)

Fatal error: Uncaught TypeError: count(): Argument #1 ($var) must be of type Countable .. on line 12
```

Exemplo #3 Uso recursivo da função count()

```php
<?php
$comidas = array('frutas'   => array('laranja', 'banana', 'maçã'),
                 'vegetais' => array('cenoura', 'couve', 'ervilha'));

// contagem recursiva
var_dump(count($comidas, COUNT_RECURSIVE));

// contagem normal
var_dump(count($comidas));

?>
```

O exemplo acima produzirá:

```
int(8)
int(2)
```

Exemplo #4 Objeto Countable

```php
<?php
class CountOfMethods implements Countable
{
    private function someMethod()
    {
    }

    public function count(): int
    {
        return count(get_class_methods($this));
    }
}

$obj = new CountOfMethods();
var_dump(count($obj));
?>
```

O exemplo acima produzirá:

```
int(2)
```

Veja também

- is_array() - Verifica se a variável é um array
- isset() - Determina se uma variável está declarada e é diferente de null
- empty() - Determina se uma variável está vazia
- strlen() - Retorna o tamanho de uma string
- is_countable() - Verifica se o conteúdo de uma variável é um valor contável
- Arrays

current

current

(PHP 4, PHP 5, PHP 7, PHP 8)

current — Retorna o elemento atual em um array

Descrição

function **current**(array|object `$array`): mixed

Todo array tem um ponteiro interno para o elemento "atual", o qual é inicializado para apontar para o primeiro elemento inserido em um array.

Parâmetros

`array`

O array.

Valor Retornado

A função **current()** simplesmente retorna o valor do elemento do array que está sendo apontado pelo ponteiro interno. Ela não move o ponteiro de forma alguma. Se o ponteiro interno aponta para além do final da lista de elementos ou o array está vazio, **current()** retorna `false`.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo do uso de current() e amigos

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = current($transport), PHP_EOL; // $mode = 'bike';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'foot';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
echo $mode = current($transport), PHP_EOL; // $mode = 'plane';

$arr = array();
var_dump(current($arr)); // bool(false)

$arr = array(array());
var_dump(current($arr)); // array(0) { }
?>
```

Notas

> **Nota** :
>
> Os resultados de chamar **current()** em um array vazio e em um array, cujo ponteiro interno aponta para além do fim dos elementos, são indistinguíveis de um elemento bool `false`. Para percorrer adequadamente um array que pode conter elementos `false`, veja a estrutura de controle `foreach`.
>
> Para ainda usar **current()** e checar adequadamente se o valor é realmente um elemento do array, a key() do elemento **current()** deve ser verificada como estritamente diferente de `null`.

Veja também

- end() - Define o ponteiro interno de um array para seu último elemento
- key() - Retorna uma chave de um array
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor
- prev() - Retrocede o ponteiro interno de um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento
- next() - Avança o ponteiro interno de um array
- `foreach`

each

each

(PHP 4, PHP 5, PHP 7)

each — Retorna o par atual de chave e valor de um array e avança o seu cursor

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.2.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **each**(array|object `&$array`): array

Retorna o par atual de chave e valor de um array e avança o seu cursor.

Depois da execução de **each()**, o cursor do array irá apontar para o próximo elemento do array, ou após o último elemento se ele atingir o final do array. Deve-se utilizar reset() se for desejado percorrer o array novamente usando a função.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Retorna o par atual de chave e valor do `array`. Este par é retornado em um array de quatro elementos, com as chaves `0`, `1`, `key` e `value`. Elementos `0` e `key` contêm o nome da chave do elemento do array; e `1` e `value` contêm os dados.

Se o cursor interno do array estiver apontando para além do final do conteúdo do array, **each()** retorna `false`.

Examples

Exemplo #1 Exemplos de each()

```php
<?php
$foo = array("bob", "fred", "jussi", "jouni", "egon", "marliese");
$bar = each($foo);
print_r($bar);
?>
```

$bar agora contém os seguintes pares de chaves e valores:

```
Array
(
    [1] => bob
    [value] => bob
    [0] => 0
    [key] => 0
)
```

```php
<?php
$foo = array("Robert" => "Bob", "Seppo" => "Sepi");
$bar = each($foo);
print_r($bar);
?>
```

$bar agora contém os seguintes pares de chaves e valores:

```
Array
(
    [1] => Bob
    [value] => Bob
    [0] => Robert
    [key] => Robert
)
```

each() é tipicamente usada em conjunto com list() para percorrer um array, por exemplo:

Exemplo #2 Percorrendo um array com each()

```php
<?php
$fruta = array('a' => 'amora', 'b' => 'banana', 'c' => 'caju');

reset($fruta);
while (list($chave, $valor) = each($fruta)) {
      echo "$chave => $valor\n";
}
?>
```

O exemplo acima produzirá:

```
a => amora
b => banana
c => caju
```

> **Cuidado**
>
> Pelo fato da atribuição de um array a outra variável redefinir o ponteiro original do array, o exemplo acima deveria causar um loop sem fim se a variável *$fruta* fosse atribuída a outra variável dentro do loop.

> **Aviso**
>
> **each()** também aceitará objetos, porém pode retornar resultados inesperados. Portanto, não é recomendado iterar por propriedades de objeto com **each()**.

Veja também

- key() - Retorna uma chave de um array
- list() - Cria variáveis como se fossem arrays
- current() - Retorna o elemento atual em um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento
- next() - Avança o ponteiro interno de um array
- prev() - Retrocede o ponteiro interno de um array
- `foreach`
- Object Iteration

end

end

(PHP 4, PHP 5, PHP 7, PHP 8)

end — Define o ponteiro interno de um array para seu último elemento

Descrição

function **end**(array|object `&$array`): mixed

end() avança o ponteiro interno de `array` até o último elemento, e retorna seu valor.

Parâmetros

`array`

O array. Este array é passado por referência porque ele é modificado pela função. Isto significa que deve-se passar a esta função uma variável real, e não uma função retornando um array, porque somente variáveis reais podem ser passadas por referência.

Valor Retornado

Retorna o valor do último elemento ou `false` para array vazio.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo de **end()**

```php
<?php

  $frutas = array('melancia', 'banana', 'morango');
  echo end($frutas); // morango

?>
```

Veja também

- current() - Retorna o elemento atual em um array
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor
- prev() - Retrocede o ponteiro interno de um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento
- next() - Avança o ponteiro interno de um array
- array_key_last() - Obtém a última chave de um array

extract

extract

(PHP 4, PHP 5, PHP 7, PHP 8)

extract — Importa variáveis para a tabela de símbolos a partir de um array

Descrição

function **extract**(array `&$array`, int `$flags = EXTR_OVERWRITE`, string `$prefix = ""`): int

Importa variáveis a partir de um array para a tabela de símbolos atual.

Verifica se cada chave tem um nome válido de variável. Também verifica por colisões com variáveis já existentes na tabela de símbolos.

> **Aviso**
>
> Não use **extract()** em dados inseguros, como informações externas (ex.: *$_GET*, *$_FILES*).

Parâmetros

`array`

Um array associativo. Esta função trata chaves como nomes de variáveis e valores como os valores das variáveis. Para cada par chave/valor ela cria uma variável na tabela de símbolos atual, seguindo os parâmetros `flags` e `prefix`.

É necesário usar um array associativo, um array numericamente indexado não produzirá resultados até que seja usa a opção `EXTR_PREFIX_ALL` ou `EXTR_PREFIX_INVALID`.

`flags`

A forma como chaves inválidas/numéricas e colisões são tratadas é determinada pelas opções de extração definidas no parâmetro `flags`. Pode ser um dos seguintes valores:

`EXTR_OVERWRITE`

Se houver uma colisão, sobrescreve a variável existente.

`EXTR_SKIP`

Se houver uma colisão, não sobrescreve a variável existente.

EXTR_PREFIX_SAME

Se houver uma colisão, adiciona um prefixo ao nome da variável definido pelo argumento `prefix`.

EXTR_PREFIX_ALL

Adiciona um prefixo ao nome de todas as variáveis definido por `prefix`.

EXTR_PREFIX_INVALID

Adiciona um prefixo definido por `prefix` apenas para variáveis como nomes inválidos ou numéricos.

EXTR_IF_EXISTS

Só sobrescreve a variável se ela já existe na tabela de símbolos corrente, caso contrário, não faz nada. Isso é útil quando se quer definir uma lista de variáveis válidas e então extrair só as que foram definidas em *$_REQUEST*, por exemplo.

EXTR_PREFIX_IF_EXISTS

Só cria nomes de variáveis usando o prefixo se na tabela de símbolos já existe uma variável com o nome sem esse prefixo.

EXTR_REFS

Extrai variáveis como referências, ou seja, os valores das variáveis importadas ainda estarão referenciando os valores do parâmetro `array`. Essa opção pode ser usada sozinha ou em conjunto com as outras usando o operador 'ou' em `flags`.

Se `flags` não for especificado, é assumido o valor **EXTR_OVERWRITE**.

`prefix`

Note que `prefix` só é necessário se `flags` for **EXTR_PREFIX_SAME**, **EXTR_PREFIX_ALL**, ou **EXTR_PREFIX_INVALID** ou **EXTR_PREFIX_IF_EXISTS**. Se o nome com o prefixo não for um nome de variável válido, ela não será importada para a tabela de símbolos. Prefixos são automaticamente separados da chave do array pelo caractere de sublinhado (_).

Valor Retornado

Retorna o número de variáveis importadas com sucesso para a tabela de símbolos.

Examples

Exemplo #1 Exemplo da extract()

```php
<?php
$size = "large";
$var_array = array(
    "color" => "blue",
    "size"  => "medium",
    "shape" => "sphere"
);

extract($var_array, EXTR_PREFIX_SAME, "wddx");

echo "$color, $size, $shape, $wddx_size\n";

?>
```

O exemplo acima produzirá:

```
blue, large, sphere, medium
```

A variável *$size* não foi sobrescrita porque foi especificada a opção **EXTR_PREFIX_SAME**, o que resultou na criação da variável *$wddx_size*. Se **EXTR_SKIP** fosse utilizada, então *$wddx_size* não seria criada. **EXTR_OVERWRITE** teria feito com que *$size* tivesse o valor "medium", e

`EXTR_PREFIX_ALL` resultaria em novas variáveis com os nomes *$wddx_color*, *$wddx_size* e *$wddx_shape*.

Notas

> **Aviso**
>
> Não use **extract()** em dados não confiáveis, como entrada de usuário (por exemplo, *$_GET*, *$_FILES*, etc.). Se usar, certifique-se de que está usando um dos valores de `flags` que não fazem sobrescrita como `EXTR_SKIP` e tenha cuidado para que extraia na mesma ordem que é definida em variables_order no *php.ini*.

Veja também

- compact() - Cria um array contendo variáveis e seus valores
- list() - Cria variáveis como se fossem arrays

in_array

in_array

(PHP 4, PHP 5, PHP 7, PHP 8)

in_array — Verifica se um valor existe em um array

Descrição

function **in_array**(mixed `$needle`, array `$haystack`, bool `$strict = false`): bool

Procura por `needle` (agulha) no `haystack` (palheiro) usando comparação relaxada a não ser que `strict` seja definido.

Parâmetros

`needle`

O valor procurado.

> **Nota** :
>
> Se `needle` for uma string, a comparação é feita diferenciando caracteres maiúsculos e minúsculos.

`haystack`

O array.

`strict`

Se o terceiro parâmetro `strict` for `true` então **in_array()** também irá checar os tipos de `needle` em `haystack`.

> **Nota** :
>
> Antes do PHP 8.0.0, uma `needle` `string` irá corresponder a um valor de `0` no array em modo não estrito, e vice-versa. Isso pode levar a resultados indesejáveis. Casos extremos similares exitem para outros tipos, também. Se não houver certeza absoluta dos tipos dos valores envolvidos, a opção `strict` deve ser sempre usada para evitar comportamento inesperado.

Valor Retornado

Retorna `true` se `needle` for encontrado no array, `false` caso contrário.

Examples

Exemplo #1 Exemplo de in_array()

```php
<?php
$os = array("Mac", "NT", "Irix", "Linux");
if (in_array("Irix", $os)) {
    echo "Tem Irix";
}
```

```php
if (in_array("mac", $os)) {
    echo "Tem mac";
}
?>
```

A segunda condicional falha pois **in_array()** diferencia letras minúsculas e maiúsculas. Então, a saída seria:

```
Tem Irix
```

Exemplo #2 in_array() com verificação de tipos

```php
<?php
$a = array('1.10', 12.4, 1.13);

if (in_array('12.4', $a, true)) {
    echo "'12.4' encontrado com verificação de tipo\n";
}

if (in_array(1.13, $a, true)) {
    echo "1.13 encontrado com verificação de tipo\n";
}
?>
```

O exemplo acima produzirá:

```
1.13 encontrado com verificação de tipo
```

Exemplo #3 Exemplo de in_array() passando um array como agulha

```php
<?php
$a = array(array('p', 'h'), array('p', 'r'), 'o');

if (in_array(array('p', 'h'), $a)) {
    echo "'ph' foi encontrado\n";
}

if (in_array(array('f', 'i'), $a)) {
    echo "'fi' foi encontrado\n";
}

if (in_array('o', $a)) {
    echo "'o' foi encontrado\n";
}
?>
```

O exemplo acima produzirá:

```
'ph' foi encontrado
'o' foi encontrado
```

Veja também

- array_search() - Procura por um valor em um array e retorna sua chave correspondente caso seja encontrado
- isset() - Determina se uma variável está declarada e é diferente de null
- array_key_exists() - Verifica se uma chave ou índice existe em um array

key

key

(PHP 4, PHP 5, PHP 7, PHP 8)

key — Retorna uma chave de um array

Descrição

function **key**(array|object `$array`): int|string|null

key() retorna a chave da posição atual de um array.

Parâmetros

`array`

O array.

Valor Retornado

A função **key()** simplesmente retorna a chave do elemento do array que está atualmente sendo apontada pelo ponteiro interno. Isso não move o ponteiro de nenhuma forma. Se o ponteiro interno aponta além do fim de elementos da lista ou o array está vazio, **key()** retorna `null`.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo da key()

```php
<?php
$array = array(
    'fruit1' => 'apple',
    'fruit2' => 'orange',
    'fruit3' => 'grape',
    'fruit4' => 'apple',
    'fruit5' => 'apple');

// este ciclo exibirá toda a chave do array associativo
// onde o valor é igual a "apple"
while ($fruit_name = current($array)) {
    if ($fruit_name == 'apple') {
        echo key($array), "\n";
    }
    next($array);
}
?>
```

O exemplo acima produzirá:

```
fruit1
fruit4
fruit5
```

Veja também

- current() - Retorna o elemento atual em um array
- next() - Avança o ponteiro interno de um array
- array_key_first() - Obtém a primeira chave de um array
- foreach

key_exists

key_exists

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

key_exists — Sinônimo de array_key_exists()

Descrição

Esta função é um apelido para: array_key_exists().

krsort

krsort

(PHP 4, PHP 5, PHP 7, PHP 8)

krsort — Ordena um array pelas chaves em ordem descrescente

Descrição

function **krsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Ordena o `array` pelas chaves em ordem descrescente.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()

- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.
8.2.0	Esta função agora faz comparação numérica de string com a constante `SORT_REGULAR` usando as regras padrão do PHP 8.

Examples

Exemplo #1 Exemplo de krsort()

```php
<?php
$frutas = array("d"=>"limão", "a"=>"laranja", "b"=>"banana", "c"=>"maçã");
krsort($frutas);
foreach ($frutas as $chave => $valor) {
    echo "$chave = $valor\n";
}
?>
```

O exemplo acima produzirá:

```
d = limão
c = maçã
b = banana
a = laranja
```

Veja também

- sort() - Ordena um array em ordem ascendente
- ksort() - Ordena um array pelas chaves em ordem crescente
- Comparação entre as funções de ordenação de array

ksort

ksort

(PHP 4, PHP 5, PHP 7, PHP 8)

ksort — Ordena um array pelas chaves em ordem crescente

Descrição

function **ksort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Ordena o `array` pelas chaves em ordem crescente.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :

> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- **SORT_REGULAR** - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- **SORT_NUMERIC** - compara os elementos numericamente
- **SORT_STRING** - compara os elementos como strings
- **SORT_LOCALE_STRING** - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- **SORT_NATURAL** - compara os elementos como strings usando "ordenação natural" como natsort()
- **SORT_FLAG_CASE** - pode ser combinada (usando o operador bit a bit OR) com **SORT_STRING** ou **SORT_NATURAL** para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna **true**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **true**; anteriormente, era bool.
8.2.0	Esta função agora faz comparação numérica de string com a constante **SORT_REGULAR** usando as regras padrão do PHP 8.

Examples

Exemplo #1 Exemplo da ksort()

```php
<?php
$frutas = array("d"=>"limão", "a"=>"laranja", "b" =>"banana", "c"=>"maçã");
ksort($frutas);
foreach ($frutas as $chave => $valor) {
    echo "$chave = $valor\n";
}
?>
```

O exemplo acima produzirá:

```
a = laranja
b = banana
c = maçã
d = limão
```

Exemplo #2 ksort() com chaves int

```php
<?php
$a = [0 => 'Primeiro', 2 => 'Último', 1 => 'Intermediário'];
var_dump($a);
```

```php
ksort($a);
var_dump($a);
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  string(5) "Primeiro"
  [2]=>
  string(4) "Último"
  [1]=>
  string(6) "Intermediário"
}
array(3) {
  [0]=>
  string(5) "Primeiro"
  [1]=>
  string(6) "Intermediário"
  [2]=>
  string(4) "Último"
}
```

Veja também

- sort() - Ordena um array em ordem ascendente
- krsort() - Ordena um array pelas chaves em ordem descrescente
- Comparação entre as funções de ordenação de array

list

list

(PHP 4, PHP 5, PHP 7, PHP 8)

list — Cria variáveis como se fossem arrays

Descrição

function **list**(mixed `$var`, mixed `...$vars` = ?): array

Assim como array(), não é exatamente uma função, e sim uma construção da própria linguagem. **list()** é usada para criar uma lista de variáveis em apenas uma operação. Somente arrays e objetos que implementam ArrayAccess podem ser desconstruídos. Expressões **list()** não podem ser completamente vazias.

> **Nota** :
>
> Antes do PHP 7.1.0, **list()** funcionava somente em arrays numéricos e assumia que o índice numérico iniciava em 0.

A partir do PHP 7.1.0, **list()** também pode conter chaves explícitas, permitindo a desconstrução de arrays com chaves não inteiras ou não sequenciais. Para mais detalhes, consulte a seção sobre desconstrução de arrays.

> **Nota** :
>
> A tentativa de acessar uma chave de array que não foi definida é o mesmo que acessar qualquer outra variável indefinida: uma mensagem de erro de nível `E_WARNING` (nível `E_NOTICE` antes do PHP 8.0.0) será emitida e o resultado será `null`.
>
> A tentativa de desconstruir um valor escalar atribui `null` a todas as variáveis. A tentativa de desconstruir um objeto que não implementa ArrayAccess é um erro fatal.

Parâmetros

`var`

A variável.

`vars`

Outras variaveis.

Valor Retornado

Retorna o array atribuído.

Registro de Alterações

Versão	Descrição
7.3.0	Foi adicionado suporte para atribuições de referência na deconstrução de array.
7.1.0	Agora é possível especificar chaves em **list()**. Isso habilita desconstruir os arrays com chaves não-inteiras e não-sequenciais.

Examples

Exemplo #1 Exemplo de list()

```php
<?php

$info = array('Café', 'marrom', 'cafeína');

// Listando todas as variáveis
list($bebida, $cor, $substancia) = $info;
echo "$bebida é $cor e $substancia o faz especial.\n";

// Listando apenas alguns deles
list($bebida, , $substancia) = $info;
echo "$bebida tem $substancia.\n";

// Ou ignoramos os primeiros valores para conseguir apenas o último
list( , , $substancia) = $info;
echo "I need $substancia!\n";

// list() não funciona com strings
list($bar) = "abcde";
var_dump($bar); // NULL
?>
```

Exemplo #2 Exemplo de list()

```php
<?php
$resultado = $pdo->query("SELECT id, nome FROM empregados");
while (list($id, $nome) = $resultado->fetch(PDO::FETCH_NUM)) {
    echo "id: $id, nome: $nome\n";
}
?>
```

Exemplo #3 list() e ordem das definições de índice

```php
<?php

list($a, list($b, $c)) = array(1, array(2, 3));

var_dump($a, $b, $c);

?>
```

O exemplo acima produzirá:

```
```

```
int(1)
int(2)
int(3)
```

A ordem em que são definidos os índices do array para ser consumido por **list()** é irrelevante.

Exemplo #4 Usando list() com índices de array

```php
<?php
$foo = array(2 => 'a', 'foo' => 'b', 0 => 'c');
$foo[1] = 'd';
list($x, $y, $z) = $foo;
var_dump($foo, $x, $y, $z);
```

O retorno desse script é o seguinte (note a ordem dos elementos em comparação com a ordem que foram escritos com a sintaxe **list()**):

```
array(4) {
  [2]=>
  string(1) "a"
  ["foo"]=>
  string(1) "b"
  [0]=>
  string(1) "c"
  [1]=>
  string(1) "d"
}
string(1) "c"
string(1) "d"
string(1) "a"
```

Exemplo #5 list() com chaves

A partir do PHP 7.1.0, **list()** também pode conter chaves explícitas que podem ser informadas como expressões arbitrárias. A mistura de chaves inteiras e strings é permitida; no entanto, elementos com e sem chaves não podem ser misturados.

```php
<?php
$dados = [
    ["id" => 1, "nome" => 'Mario'],
    ["id" => 2, "nome" => 'Frederico'],
];
foreach ($dados as ["id" => $id, "nome" => $nome]) {
    echo "id: $id, nome: $nome\n";
}
echo PHP_EOL;
list(1 => $segundo, 3 => $quarto) = [1, 2, 3, 4];
echo "$segundo, $quarto\n";
```

O exemplo acima produzirá:

```
id: 1, name: Mario
id: 2, name: Frederico

2, 4
```

Veja também

- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor
- array() - Cria um array
- extract() - Importa variáveis para a tabela de símbolos a partir de um array

natcasesort

natcasesort

(PHP 4, PHP 5, PHP 7, PHP 8)

natcasesort — Ordena um array utilizando o algoritmo de "ordem natural" sem diferenciar maiúsculas e minúsculas

Descrição

function **natcasesort**(array `&$array`): true

natcasesort() é uma versão insensível a maiúsculas/minúsculas de natsort().

Essa função implementa o algoritmo de ordenação que classifica strings alfanuméricas da forma que humanos fariam mantendo associação de chave/valor. Essa forma de ordenação é denominada "ordenação natural".

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de natcasesort()

```php
<?php
$array1 = $array2 = array('IMG0.png', 'img12.png', 'img10.png', 'img2.png', 'img1.png', 'IMG3.png');

sort($array1);
echo "Ordenação padrão\n";
print_r($array1);

natcasesort($array2);
echo "\nOrdenação natural (insensível a maiúsculas/minúsculas)\n";
print_r($array2);
?>
```

O exemplo acima produzirá:

```
Ordenação padrão
Array
(
    [0] => IMG0.png
    [1] => IMG3.png
    [2] => img1.png
    [3] => img10.png
    [4] => img12.png
    [5] => img2.png
)

Ordenação natural (insensível a maiúsculas/minúsculas)
Array
(
    [0] => IMG0.png
    [4] => img1.png
    [3] => img2.png
    [5] => IMG3.png
    [2] => img10.png
    [1] => img12.png
)
```

Para mais informação, consulte a página » Natural Order String Comparison de Martin Pool.

Veja também

- natsort() - Ordena um array utilizando o algoritmo de "ordem natural"
- Comparação entre as funções de ordenação de array
- strnatcmp() - Comparações de strings usando um algoritmo de "ordem natural"
- strnatcasecmp() - Comparação de strings insensível a maiúsculas/minúsculas usando o algoritmo de "ordem natural"

natsort

natsort

(PHP 4, PHP 5, PHP 7, PHP 8)

natsort — Ordena um array utilizando o algoritmo de "ordem natural"

Descrição

function **natsort**(array `&$array`): true

Essa função implementa um algoritmo que ordena strings alfanuméricas da forma como um ser humano faria enquanto mantém associações de chave/valor. Isto é descrito como "ordenação natural". Um exemplo da diferença entre este algoritmo e o algoritmo de ordenação comum de strings (usado em sort()) pode ser visto no exemplo abaixo:

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de natsort() demonstrando seu uso básico

```php
<?php
$array1 = $array2 = array("img12.png", "img10.png", "img2.png", "img1.png");

asort($array1);
echo "Classificação padrão\n";
print_r($array1);

natsort($array2);
echo "\nClassificação em ordem natural\n";
print_r($array2);
?>
```

O exemplo acima produzirá:

```
Classificação padrão
Array
(
    [3] => img1.png
    [1] => img10.png
    [0] => img12.png
    [2] => img2.png
)

Classificação em ordem natural
Array
(
    [3] => img1.png
    [2] => img2.png
    [1] => img10.png
    [0] => img12.png
)
```

Para mais informações, consulte a página » Comparação de Strings em Ordem Natural do autor Martin Pool.

Exemplo #2 Exemplos de natsort() demonstrando potenciais ciladas

```php
<?php
echo "Números negativos\n";
$negativos = array('-5','3','-2','0','-1000','9','1');
print_r($negativos);
natsort($negativos);
print_r($negativos);

echo "Zero à esquerda\n";
$zeros = array('09', '8', '10', '009', '011', '0');
print_r($zeros);
natsort($zeros);
print_r($zeros);
?>
```

O exemplo acima produzirá:

```
Números negativos
Array
(
    [0] => -5
    [1] => 3
    [2] => -2
    [3] => 0
    [4] => -1000
    [5] => 9
    [6] => 1
)
Array
(
    [2] => -2
    [0] => -5
    [4] => -1000
    [3] => 0
    [6] => 1
    [1] => 3
    [5] => 9
)
Zero à esquerda
Array
(
    [0] => 09
    [1] => 8
    [2] => 10
    [3] => 009
    [4] => 011
    [5] => 0
)
Array
(
    [5] => 0
    [1] => 8
    [0] => 09
    [3] => 009
    [2] => 10
    [4] => 011
)
```

Veja também

- natcasesort() - Ordena um array utilizando o algoritmo de "ordem natural" sem diferenciar maiúsculas e minúsculas
- Comparação entre as funções de ordenação de array
- strnatcmp() - Comparações de strings usando um algoritmo de "ordem natural"
- strnatcasecmp() - Comparação de strings insensível a maiúsculas/minúsculas usando o algoritmo de "ordem natural"

next

next

(PHP 4, PHP 5, PHP 7, PHP 8)

next — Avança o ponteiro interno de um array

Descrição

function **next**(array|object `&$array`): mixed

next() funciona de forma semelhante a current(), com uma diferença. Ele avança o ponteiro interno do array em uma posição antes de retornar o valor do elemento. Isso significa que ela retorna o próximo valor do array avançando o ponteiro interno em uma posição.

Parâmetros

`array`

O array a ser afetado.

Valor Retornado

Retorna o elemento do array que está na próxima posição que é apontada pelo ponteiro interno do array, ou `false` caso não existam mais elementos.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo de uso de next() e funções relacionadas

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = next($transport), PHP_EOL;    // $mode = 'car';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
?>
```

Notas

> **Nota** :
>
> O final do array não é distinguível de um elemento bool `false`. Para atravessar um array que contenha elementos `false`, veja a função `foreach`.
>
> Para continuar utilizando **next()** e verificar se o fim do array foi atingido, verifique se key() é `null`.

Veja também

- current() - Retorna o elemento atual em um array
- end() - Define o ponteiro interno de um array para seu último elemento
- prev() - Retrocede o ponteiro interno de um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor

pos

pos

(PHP 4, PHP 5, PHP 7, PHP 8)

pos — Sinônimo de current()

Descrição

Esta função é um apelido para: current()

prev

prev

(PHP 4, PHP 5, PHP 7, PHP 8)

prev — Retrocede o ponteiro interno de um array

Descrição

function **prev**(array|object `&$array`): mixed

Retrocede o ponteiro interno do array.

prev() funciona como next(), exceto que ela retrocede o ponteiro interno do array ao invés de avançá-lo.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Retorna o valor do array que está na posição anterior à posição apontada pelo ponteiro interno, ou `false` se não houver mais elementos.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo de uso de prev() e funções relacionadas

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = next($transport), PHP_EOL;    // $mode = 'car';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
?>
```

Notas

> **Nota** :
>
> Não será possível distinguir entre o início de um array e um elemento bool `false`. Para que seja possível identificar o início do array, deve ser verificado se a key() do elemento **prev()** não é `null`.

Veja também

- current() - Retorna o elemento atual em um array
- end() - Define o ponteiro interno de um array para seu último elemento
- next() - Avança o ponteiro interno de um array
- reset() - Faz o ponteiro interno de um array apontar para o seu primeiro elemento
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor

range

range

(PHP 4, PHP 5, PHP 7, PHP 8)

range — Cria um array contendo uma faixa de elementos

Descrição

function **range**(string|int|float `$start`, string|int|float `$end`, int|float `$step` = 1): array

Cria um array contendo uma faixa de elementos.

Se ambos `start` e `end` forem strings, e `step` é int, o array produzido será uma sequência de bytes. Em outros casos, o array produzido será uma sequência de números.

A sequência é crescente se `start` é menor ou igual a `end`. Em outros casos, a sequência é decrescente.

Parâmetros

`start`

O primeiro valor da sequência.

`end`

O último valor possível da sequência.

`step`

`step` indica o tamanho do deslocamento entre os valores produzidos na sequência.

`step` pode ser negativo em sequências decrescentes.

Se `step` é um float sem uma parte fracional, ele é interpretado como int.

Valor Retornado

Retorna uma sequência de elementos como um array onde o primeiro elemento é `start` progredindo a até `end`, com cada valor da sequência separados por `step` valores.

O último elemento do array retornado pode ser `end` ou um valor anterior da sequência, a depender do valor de `step`.

Se ambos `start` e `end` são strings, e `step` é int, o array produzido será uma sequência de bytes, geralmente as letras latinas no padrão ASCII.

Se `start`, `end`, ou `step` forem float, o array produzido será uma sequência de floats.

Em outros casos, o array produzido será uma sequência de int.

Erros/Exceções

- Se `step` é `0`, um erro ValueError é lançado.
- se `start`, `end`, ou `step` não for is_finite(), um erro ValueError é lançado.
- Se `step` é negativo, mas o array produzido é crescente (`$start <= $end`), um erro ValueError é lançado.
- Se `start` ou `end` é um string vazia `''`, um aviso **`E_WARNING`** é emitido e a string vazia é interpretada como `0`.
- Se `start` ou `end` forem uma string não numérica com mais de um byte, um aviso **`E_WARNING`** é emitido.
- Se `start` ou `end` é uma string que pode ser implicitamente convertida para um int, um aviso **`E_WARNING`** é emitido.
- Se `step` é um float, e `start` e `end` forem uma string não numérica, um aviso **`E_WARNING`** é emitido.

Registro de Alterações

Versão	Descrição
8.3.0	Se ambos `start` e `end` forem strings então **range()** nem sempre produzirá um array de bytes. Anteriormente, se um dos valores limite for uma string numérica, então o outro valor string é implicitamente convertido para um int.
8.3.0	Um aviso `E_WARNING` agora é emitido se `start` ou `end` é uma string implicitamente conversível para int se quaisquer valores limite é interpretado como um número.
8.3.0	Um aviso `E_WARNING` agora é emitido se `start` ou `end` é uma string não numérica com mais de um byte.
8.3.0	Um aviso `E_WARNING` agora é emitido se `start` ou `end` é uma string vazia.
8.3.0	Se `step` é um float sem parte fracionária, ele será interpretado como um int.
8.3.0	Um erro ValueError agora é lançado se `step` é negativo quando da produção de um array crescente.
8.3.0	Um erro ValueError agora é lançado se `step` não for finito.
8.3.0	Um erro TypeError agora é lançado se `start` ou `end` é um array, object ou resource. Anteriormente eles eram implicitamente convertidos para int.

Examples

Exemplo #1 Exemplos de range()

```php
<?php
echo implode(', ', range(0, 12)), PHP_EOL;

echo implode(', ', range(0, 100, 10)), PHP_EOL;

echo implode(', ', range('a', 'i')), PHP_EOL;

echo implode(', ', range('c', 'a')), PHP_EOL;

echo implode(', ', range('A', 'z')), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
a, b, c, d, e, f, g, h, i
c, b, a
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, [, \, ], ^, _, `, a, b, c, d, e, f, g, h, i,
j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z
```

Veja também

- shuffle() - Embaralha um array
- array_fill() - Preenche um array com valores
- `foreach`

reset

reset

(PHP 4, PHP 5, PHP 7, PHP 8)

reset — Faz o ponteiro interno de um array apontar para o seu primeiro elemento

Descrição

function **reset**(array|object `&$array`): mixed

reset() retrocede o ponteiro interno de `array` para o primeiro elemento e retorna o valor do primeiro elemento do array.

Parâmetros

`array`

O array de entrada.

Valor Retornado

Retorna o valor do primeiro elemento do array, ou `false` se o array estiver vazio.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função em objects foi descontinuado. Converta o object para um array usando get_mangled_object_vars() primeiro ou, em vez disso, use os métodos fornecidos por uma classe que implementa Iterator, como ArrayIterator.
7.4.0	Instâncias de classes SPL agora são tratadas como objetos vazios que não possuem propriedades em vez de chamar o método da interface Iterator com o mesmo nome desta função.

Examples

Exemplo #1 Exemplo da reset()

```php
<?php

$array = array('primeiro passo', 'segundo passo', 'terceiro passo', 'quarto passo');

// por definição, o ponteiro está sobre o primeiro elemento
echo current($array)."<br />\n"; // "Primeiro passo"

// pula dois passos
next($array);
next($array);
echo current($array)."<br />\n"; // "passo três"

// reinicia o ponteiro, começa novamente o primeiro passo
reset($array);
echo current($array)."<br />\n"; // "primeiro passo"

?>
```

Notas

> **Nota** :
>
> O valor retornado em um array vazio não é diferente de o valor retornado no caso de um array contendo um primeiro elemento bool `false`. Para verificar corretamente o primeiro elemento de um array que possa conter elementos `false`, primeiro verifique se count() do array, ou verifique se key() não é `null`, depois de chamar **reset()**.

Veja também

- current() - Retorna o elemento atual em um array
- each() - Retorna o par atual de chave e valor de um array e avança o seu cursor
- end() - Define o ponteiro interno de um array para seu último elemento
- next() - Avança o ponteiro interno de um array
- prev() - Retrocede o ponteiro interno de um array
- array_key_first() - Obtém a primeira chave de um array

rsort

rsort

(PHP 4, PHP 5, PHP 7, PHP 8)

rsort — Ordena um array em ordem descrescente

Descrição

function **rsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Ordena `array` pelos valores em ordem descrescente.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Esta função atribui novas chaves aos elementos do `array`. Ela removerá todas as chaves existentes que possam ter sido atribuídas, em vez de apenas reordenar as chaves.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()
- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de rsort()

```php
<?php
$frutas = array ("limão", "laranja", "banana", "maçã");
rsort ($frutas);
foreach( $frutas as $chave => $valor ){
```

```
    echo "$chave = $valor\n";
}
?>
```

O exemplo acima produzirá:

```
0 = maçã
1 = limão
2 = laranja
3 = banana
```

As frutas foram ordenadas em ordem alfabética decrescente.

Veja também

- sort() - Ordena um array em ordem ascendente
- arsort() - Ordena um array em ordem descrescente e mantém a associação de índices
- krsort() - Ordena um array pelas chaves em ordem descrescente
- Comparação entre as funções de ordenação de array

shuffle

shuffle

(PHP 4, PHP 5, PHP 7, PHP 8)

shuffle — Embaralha um array

Descrição

function **shuffle**(array `&$array`): true

Essa função embaralha (mistura de forma aleatória os elementos de) um array.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**
>
> Esta função usa a instância global do Mt19937 (“Mersenne Twister”) como a fonte de aleatoriedade a por isso compartilha seu estado com todas as outras funções que usam o Mt19937 global. Usar qualquer uma destas funções avança a sequência para *todas* as outras funções, independentemente do escopo.
>
> Gerar sequências repetíveis ao semear mt_srand() ou srand() com um valor conhecido também gerará saída repetível desta função.
>
> Prefira usar métodos de Random\Randomizer em todo código novo.

Parâmetros

`array`

O array.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
7.1.0	O algoritmo interno de randomização mudou para o Gerado de Números Aleatórios » Mersenne Twister no lugar da função rand do libc.

Examples

Exemplo #1 Exemplo de shuffle()

```php
<?php
$numeros = range(1,20);
shuffle($numeros);
foreach ($numeros as $numero) {
    echo "$numero ";
}
?>
```

Notas

> **Nota** :
>
> Esta função atribui novas chaves aos elementos do `array`. Ela removerá todas as chaves existentes que possam ter sido atribuídas, em vez de apenas reordenar as chaves.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Veja também

- Random\Randomizer::shuffleArray() - Obtém uma permutação de um array
- Random\Randomizer::shuffleBytes() - Obtém uma permutação byte a byte de uma string
- Random\Randomizer::pickArrayKeys() - Seleciona chaves aleatórias de um array
- Comparação entre as funções de ordenação de array

sizeof

sizeof

(PHP 4, PHP 5, PHP 7, PHP 8)

sizeof — Sinônimo de count()

Descrição

Esta função é um apelido para: count().

sort

sort

(PHP 4, PHP 5, PHP 7, PHP 8)

sort — Ordena um array em ordem ascendente

Descrição

function **sort**(array `&$array`, int `$flags` = `SORT_REGULAR`): true

Ordena os valores do `array` em ordem ascendente, modificando o array.

> **Nota** :

> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Esta função atribui novas chaves aos elementos do `array`. Ela removerá todas as chaves existentes que possam ter sido atribuídas, em vez de apenas reordenar as chaves.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- **`SORT_REGULAR`** - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- **`SORT_NUMERIC`** - compara os elementos numericamente
- **`SORT_STRING`** - compara os elementos como strings
- **`SORT_LOCALE_STRING`** - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- **`SORT_NATURAL`** - compara os elementos como strings usando "ordenação natural" como natsort()
- **`SORT_FLAG_CASE`** - pode ser combinada (usando o operador bit a bit OR) com **`SORT_STRING`** ou **`SORT_NATURAL`** para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **`true`**; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de sort()

```php
<?php

$fruits = array("lemon", "orange", "banana", "apple");
sort($fruits);
foreach ($fruits as $key => $val) {
    echo "fruits[" . $key . "] = " . $val . "\n";
}

?>
```

O exemplo acima produzirá:

```
```

```
fruits[0] = apple
fruits[1] = banana
fruits[2] = lemon
fruits[3] = orange
```

As frutas foram classificadas em ordem alfabética.

Exemplo #2 Exemplo de sort() utilizando comparações naturais ignorando maiúsculas e minúculas

```php
<?php

$fruits = array(
    "Orange1", "orange2", "Orange3", "orange20"
);
sort($fruits, SORT_NATURAL | SORT_FLAG_CASE);
foreach ($fruits as $key => $val) {
    echo "fruits[" . $key . "] = " . $val . "\n";
}

?>
```

O exemplo acima produzirá:

```
fruits[0] = Orange1
fruits[1] = orange2
fruits[2] = Orange3
fruits[3] = orange20
```

A lista foi ordenada como se tivesse sido utilizada natcasesort().

Notas

> **Nota** :
>
> Como muitas funções de ordenação do PHP, **sort()** usa uma implementação da » Quicksort. O pivot é escolhido no meio da partição, o que resulta num processamento rápido para arrays já ordenados. Mas isso é um detalhe de implementação, que não pode ser garantido.

> **Aviso**
>
> Cuidado ao ordenar arrays com valores com tipos variados, dado que **sort()** pode produzir resultados inesperados, mesmo se `flags` for `SORT_REGULAR`.

Veja também

- rsort() - Ordena um array em ordem descrescente
- Comparação entre as funções de ordenação de array

uasort

uasort

(PHP 4, PHP 5, PHP 7, PHP 8)

uasort — Ordena um array com uma função de comparação definida pelo usuário e mantém associações de índices

Descrição

function **uasort**(array `&$array`, callable `$callback`): true

Ordena o `array` de forma que as chaves mantenham suas correlações com os valores aos quais estão associadas, usando uma função de comparação definida pelo usuário.

Esta função é usada principalmente para ordenar arrays associativos onde a ordem real dos elementos é significativa.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um `E_WARNING`.

Examples

Exemplo #1 Exemplo básico de uasort()

```php
<?php
// Função de comparação
function cmp($a, $b) {
    if ($a == $b) {
        return 0;
    }
    return ($a < $b) ? -1 : 1;
}

// Array a ser ordenado
$array = array('a' => 4, 'b' => 8, 'c' => -1, 'd' => -9, 'e' => 2, 'f' => 5, 'g' => 3, 'h' => -4);
print_r($array);

// Ordena e mostra o array resultante
uasort($array, 'cmp');
```

```
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [a] => 4
    [b] => 8
    [c] => -1
    [d] => -9
    [e] => 2
    [f] => 5
    [g] => 3
    [h] => -4
)
Array
(
    [d] => -9
    [h] => -4
    [c] => -1
    [e] => 2
    [g] => 3
    [a] => 4
    [f] => 5
    [b] => 8
)
```

Veja também

- usort() - Ordena um array pelos valores utilizando uma função de comparação definida pelo usuário
- uksort() - Ordena um array pelas chaves utilizando uma função de comparação definida pelo usuário.
- Comparação entre as funções de ordenação de array

uksort

uksort

(PHP 4, PHP 5, PHP 7, PHP 8)

uksort — Ordena um array pelas chaves utilizando uma função de comparação definida pelo usuário.

Descrição

function **uksort**(array `&$array`, callable `$callback`): true

Ordena o `array` pelas chaves usando uma função de comparação fornecida pelo usuário para determinar a ordem.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Redefine o ponteiro interno do array para o primeiro elemento.

Parâmetros

`array`

O array de entrada.

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um `E_WARNING`.

Examples

Exemplo #1 Exemplo de uksort()

```php
<?php
function cmp($a, $b)
{
    $a = preg_replace('@^(a|an|the) @', '', $a);
    $b = preg_replace('@^(a|an|the) @', '', $b);
    return strcasecmp($a, $b);
}

$a = array("John" => 1, "the Earth" => 2, "an apple" => 3, "a banana" => 4);

uksort($a, "cmp");

foreach ($a as $key => $value) {
    echo "$key: $value\n";
}
?>
```

O exemplo acima produzirá:

```
an apple: 3
a banana: 4
the Earth: 2
John: 1
```

Veja também

- usort() - Ordena um array pelos valores utilizando uma função de comparação definida pelo usuário
- uasort() - Ordena um array com uma função de comparação definida pelo usuário e mantém associações de índices
- Comparação entre as funções de ordenação de array

usort

usort

(PHP 4, PHP 5, PHP 7, PHP 8)

usort — Ordena um array pelos valores utilizando uma função de comparação definida pelo usuário

Descrição

function **usort**(array `&$array`, callable `$callback`): true

Ordenar o `array` pelos valores usando uma função de comparação definida pelo usuário para determinar a ordem.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

> **Nota** :
>
> Esta função atribui novas chaves aos elementos do `array`. Ela removerá todas as chaves existentes que possam ter sido atribuídas, em vez de apenas reordenar as chaves.

Parâmetros

`array`

The input array.

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **`true`**; anteriormente, era bool.
8.0.0	Se a função `callback` espera que um parâmetro seja passado por referência, esta função agora emitirá um **`E_WARNING`**.

Examples

Exemplo #1 Exemplo de usort()

```php
<?php
function cmp($a, $b)
{
    if ($a == $b) {
    return 0;
    }
    return ($a < $b) ? -1 : 1;
}

$a = array(3, 2, 5, 6, 1);

usort($a, "cmp");
```

```php
foreach ($a as $chave => $valor) {
    echo "$chave: $valor\n";
}
?>
```

O exemplo acima produzirá:

```
0: 1
1: 2
2: 3
3: 5
4: 6
```

O operador nave espacial (<=>) pode ser usado para simplificar ainda mais a comparação interna.

```php
<?php
function cmp($a, $b)
{
    return $a <=> $b;
}

$a = array(3, 2, 5, 6, 1);

usort($a, "cmp");

foreach ($a as $chave => $valor) {
    echo "$chave: $valor\n";
}
?>
```

> **Nota** :
>
> Obviamente que nesse caso trivial a função sort() seria mais apropriada.

Exemplo #2 Exemplo de usort() usando um array multidimensional

```php
<?php
function cmp($a, $b)
{
    return strcmp($a["fruta"], $b["fruta"]);
}

$frutas[0]["fruta"] = "limões";
$frutas[1]["fruta"] = "abacaxis";
$frutas[2]["fruta"] = "goiabas";

usort($frutas, "cmp");

foreach ($frutas as $chave => $valor) {
    echo "\$frutas[$chave]: " . $valor["fruta"] . "\n";
}
?>
```

O exemplo acima produzirá:

```
$frutas[0]: abacaxis
$frutas[1]: goiabas
$frutas[2]: limões
```

Na ordenação de um array multi-dimensional, *$a* e *$b* contêm referências ao primeiro índice do array.

Exemplo #3 Exemplo de usort() usando uma função membro de um objeto

```php
<?php
class TestObj {
    public string $name;

    function TestObj($name)
    {
        $this->name = $name;
    }

    /* Essa é a função estática de comparação */
    static function cmp_obj($a, $b)
    {
        return strtolower($a->name) <=> strtolower($b->name);
    }
}

$a[] = new TestObj("c");
$a[] = new TestObj("b");
$a[] = new TestObj("d");

usort($a, [TestObj::class, "cmp_obj"]);

foreach ($a as $item) {
    echo $item->name . "\n";
}
?>
```

O exemplo acima produzirá:

```
b
c
d
```

Exemplo #4 Exemplo de usort() usando uma closure para ordernar um array multidimensional

```php
<?php
$array[0] = array('key_a' => 'z', 'key_b' => 'c');
$array[1] = array('key_a' => 'x', 'key_b' => 'b');
$array[2] = array('key_a' => 'y', 'key_b' => 'a');

function build_sorter($key) {
    return function ($a, $b) use ($key) {
        return strnatcmp($a[$key], $b[$key]);
    };
}

usort($array, build_sorter('key_b'));

foreach ($array as $item) {
    echo $item['key_a'] . ', ' . $item['key_b'] . "\n";
}
?>
```

O exemplo acima produzirá:

```
y, a
x, b
z, c
```

Exemplo #5 Exemplo de usort() usando o operador nave espacial

O operador nave espacial permite comparação direta de valores compostos atráves de múltiplos eixos. O exemplo a seguir irá ordenar `$pessoas` pelo último nome, e depois pelo primeiro nome se o último nome for igual.

```php
<?php
$pessoas[0] = ['primeiro' => 'Adam', 'ultimo' => 'West'];
$pessoas[1] = ['primeiro' => 'Alec', 'ultimo' => 'Baldwin'];
$pessoas[2] = ['primeiro' => 'Adam', 'ultimo' => 'Baldwin'];

function sorter(array $a, array $b) {
    return [$a['ultimo'], $a['primeiro']] <=> [$b['ultimo'], $b['primeiro']];
}

usort($pessoas, 'sorter');

foreach ($pessoas as $pessoa) {
    print $pessoa['ultimo'] . ', ' . $pessoa['primeiro'] . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
Baldwin, Adam
Baldwin, Alec
West, Adam
```

Veja também

- uasort() - Ordena um array com uma função de comparação definida pelo usuário e mantém associações de índices
- uksort() - Ordena um array pelas chaves utilizando uma função de comparação definida pelo usuário.
- Comparação entre as funções de ordenação de array

Funções Matemáticas

Introdução

Estas funções matemáticas lidam apenas com valores dentro dos limites dos tipos int e float. Para manipular números maiores, consulte matemática com precisão arbitrária ou funções GNU de precisão múltipla.

Veja também

- Operadores aritméticos
- Funções matemáticas com precisão arbitrária
- Funções GNU de precisão múltipla

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

Constantes matemáticas

`M_PI` (float)

Aproximação do número π (pi). (`3.14159265358979323846`).

`M_E` (float)

Aproximação do número de Euler `e` (`2.7182818284590452354`).

`M_LOG2E` (float)

Aproximação de $\log_2(e)$ (`1.4426950408889634074`).

`M_LOG10E` (float)

Aproximação de $\log_{10}(e)$ (`0.43429448190325182765`).

`M_LN2` (float)

Aproximação de `ln(2)` (`0.69314718055994530942`).

`M_LN10` (float)

Aproximação de `ln(10)` (`2.30258509299404568402`).

`M_PI_2` (float)

Aproximação de `π/2` (`1.57079632679489661923`).

`M_PI_4` (float)

Aproximação de `π/4` (`0.78539816339744830962`).

`M_1_PI` (float)

Aproximação de `1/π` (`0.31830988618379067154`).

`M_2_PI` (float)

Aproximação de `2/π` (`0.63661977236758134308`).

`M_SQRTPI` (float)

Aproximação de `sqrt(π)` (`1.77245385090551602729`).

`M_2_SQRTPI` (float)

Aproximação de `2/sqrt(π)` (`1.12837916709551257390`).

`M_SQRT2` (float)

Aproximação de `sqrt(2)` (`1.41421356237309504880`).

`M_SQRT3` (float)

Aproximação de `sqrt(3)` (`1.73205080756887729352`).

`M_SQRT1_2` (float)

Aproximação de `1/sqrt(2)` (`0.70710678118654752440`).

`M_LNPI` (float)

Aproximação de `ln(π)` (`1.14472988584940017414`).

`M_EULER` (float)

Aproximação da constante de Euler γ (`0.57721566490153286061`).

Constantes de ponto flutuante IEEE 754

`NAN` (float)

Um não-número (Not A Number)

`INF` (float)

Infinito

Constantes de arredondamento

> **Nota** :
>
> A partir do PHP 8.4.0, é recomendado usar a enumeração RoundingMode no lugar destas constantes.

`PHP_ROUND_HALF_UP` (int)

Arredonda a metade na direção oposta ao zero.

`PHP_ROUND_HALF_DOWN` (int)

Arredonda a metade na direção do zero.

`PHP_ROUND_HALF_EVEN` (int)

Arredonda a metade para o número par.

`PHP_ROUND_HALF_ODD` (int)

Arredonda a metade para o número ímpar.

RoundingMode

(PHP 8 >= 8.4.0)

Introdução

A enumeração **RoundingMode** é usada para especificar como o arredondamento deve ser realizado por round(), bcround() e BCMath\Number::round().

Resumo da enumeração

enum **RoundingMode**
{
case HalfAwayFromZero ; // Arredonda para o inteiro mais próximo. Se a parte decimal for igual a 5, arredonda para o inteiro com maior valor absoluto.

case HalfTowardsZero ; // Arredonda para o inteiro mais próximo. Se a parte decimal for igual a 5, arredonda para o inteiro com menor valor absoluto.

case HalfEven ; // Arredonda para o inteiro mais próximo. Se a parte decimal for igual a 5, arredonda para o inteiro par.

case HalfOdd ; // Arredonda para o inteiro mais próximo. Se a parte decimal for igual a 5, arredonda para o inteiro ímpar.

case TowardsZero ; // Arredonda para o inteiro mais próximo com um valor absoluto menor ou igual.

case AwayFromZero ; // Arredonda para o inteiro mais próximo com um valor absoluto maior ou igual.

case NegativeInfinity ; // Arredonda para o maior inteiro que seja menor ou igual.

case PositiveInfinity ; // Arredonda para o menor inteiro que seja maior ou igual.

}

Funções de Math

Índice

abs

abs

(PHP 4, PHP 5, PHP 7, PHP 8)

abs — Valor absoluto

Descrição

function **abs**(int|float $num): int|float

Retorna o valor absoluto de num.

Parâmetros

num

O valor numérico a processar

Valor Retornado

O valor absoluto de num. Se o argumento num for do tipo float, o número retornado também será float, caso contrário será int (pois float normalmente tem uma faixa de valores maior que int).

Registro de Alterações

Versão	Descrição
8.0.0	num não aceita mais objetos internos que suportem conversão numérica.

Examples

Exemplo #1 Exemplo de abs()

```
<?php
var_dump(abs(-4.2));
var_dump(abs(5));
var_dump(abs(-5));
?>
```

O exemplo acima produzirá:

```
float(4.2)
int(5)
int(5)
```

Veja também

- gmp_abs() - Valor absoluto
- gmp_sign() - Sinal do número

acos

acos

(PHP 4, PHP 5, PHP 7, PHP 8)

acos — Arco cosseno

Descrição

function **acos**(float $num): float

Retorna o arco cosseno de num em radianos. **acos()** é a função inversa de cos(), o que significa que $num == cos(acos($num)) para qualquer valor de num que esteja no domínio de **acos()**.

Parâmetros

num

O argumento a processar

Valor Retornado

O arco cosseno de num em radianos.

Veja também

- cos() - Cosseno
- acosh() - Arco cosseno hiperbólico
- asin() - Arco seno
- atan() - Arco tangente

acosh

acosh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

acosh — Arco cosseno hiperbólico

Descrição

function **acosh**(float $num): float

Retorna o arco cosseno hiperbólico de num, isto é, o valor cujo cosseno hiperbólico é num.

Parâmetros

num

O valor a processar

Valor Retornado

O arco cosseno hiperbólico de num

Veja também

- cosh() - Cosseno hiperbólico
- acos() - Arco cosseno
- asinh() - Arco seno hiperbólico
- atanh() - Arco tangente hiperbólico

asin

asin

(PHP 4, PHP 5, PHP 7, PHP 8)

asin — Arco seno

Descrição

function **asin**(float `$num`): float

Retorna o arco seno de `num` em radianos. **asin()** é a função inversa da função sin(), o que significa que `$num == sin(asin($num))` para qualquer valor de `num` que esteja no domínio da função **asin()**.

Parâmetros

`num`

O argumento a processar

Valor Retornado

O arco seno de `num` em radianos

Veja também

- sin() - Seno
- asinh() - Arco seno hiperbólico
- acos() - Arco cosseno
- atan() - Arco tangente

asinh

asinh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

asinh — Arco seno hiperbólico

Descrição

function **asinh**(float `$num`): float

Retorna o arco seno hiperbólico de `num`, isto é, o valor cujo seno hiperbólico é `num`.

Parâmetros

`num`

O argumento a processar

Valor Retornado

O arco seno hiperbólico de `num`

Veja também

- sinh() - Seno hiperbólico
- asin() - Arco seno
- acosh() - Arco cosseno hiperbólico
- atanh() - Arco tangente hiperbólico

atan

atan

(PHP 4, PHP 5, PHP 7, PHP 8)

atan — Arco tangente

Descrição

function **atan**(float $num): float

Retorna o arco tangente de num em radianos. **atan()** é a função inversa da função tan(), o que significa que $num == tan(atan($num)) para qualquer valor de num que esteja no domínio de **atan()**.

Parâmetros

num

O argumento a processar

Valor Retornado

O arco tangente de num em radianos.

Veja também

- tan() - Tangente
- atanh() - Arco tangente hiperbólico
- asin() - Arco seno
- acos() - Arco cosseno

atan2

atan2

(PHP 4, PHP 5, PHP 7, PHP 8)

atan2 — Arco tangente de duas variáveis

Descrição

function **atan2**(float $y, float $x): float

Esta função calcula o arco tangente das duas variaveis x e y. É similar a calcular o arco tangente de y / x, exceto que os sinais de ambos os argumentos são usados para determinar o quadrante do resultado.

Esta função retorna o resultado em radianos entre -PI e PI (inclusive).

Parâmetros

y

Dividendo

x

Divisor

Valor Retornado

O arco tangente de y/x em radianos.

Veja também

- atan() - Arco tangente

atanh

atanh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

atanh — Arco tangente hiperbólico

Descrição

function **atanh**(float $num): float

Retorna o arco tangente hiperbólico de `num`, isto é, o valor cuja tangente hiperbólica é `num`.

Parâmetros

`num`

O argumento a processar

Valor Retornado

O arco tangente hiperbólico de `num`

Veja também

- tanh() - Tangente hiperbólica
- atan() - Arco tangente
- asinh() - Arco seno hiperbólico
- acosh() - Arco cosseno hiperbólico

base_convert

base_convert

(PHP 4, PHP 5, PHP 7, PHP 8)

base_convert — Converte um número entre bases arbitrárias

Descrição

function **base_convert**(string `$num`, int `$from_base`, int `$to_base`): string

Retorna uma string contendo `num` representado na base `to_base`. A base na qual `num` é informada está especificada em `from_base`. Tanto `from_base` quanto `to_base` devem estar entre 2 e 36, inclusive. Dígitos em números com uma base maior do que 10 serão representados com letras a-z, com 'a' significando 10, 'b' significando 11 e 'z' significando 35. A forma das letras não importa, ou seja `num` é interpretado sem sensibilidade a maiúsculas/minúsculas.

> **Aviso**
>
> **base_convert()** pode perder precisão com números grandes devido a propriedade relacionadaa ao o tipo interno de float usado. Consulte a seção Números de ponto flutuante no manual para mais informações e limitações específicas.

Parâmetros

`num`

O número a ser convertido. Quaisquer caracteres inválidos em `num` serão ignorados silenciosamente. A partir do PHP 7.4.0, informar caracteres inválidos foi descontinuado.

`from_base`

A base em que `num` está

`to_base`

A base para a qual deve-se converter `num`

Valor Retornado

`num` convertido para a base `to_base`

Registro de Alterações

Versão	Descrição
7.4.0	Passar caracteres inválidos agora gerará uma nota de descontinuação. O resultado ainda é calculado como se os caracteres inválidos não existissem.

Examples

Exemplo #1 Exemplo de base_convert()

```php
<?php
$hexadecimal = 'a37334';
echo base_convert($hexadecimal, 16, 2);
?>
```

O exemplo acima produzirá:

```
10100011011110011001110100
```

Veja também

- intval() - Obtém o valor em número inteiro de uma variável

bindec

bindec

(PHP 4, PHP 5, PHP 7, PHP 8)

bindec — Binário para decimal

Descrição

function **bindec**(string `$binary_string`): int|float

Retorna o equivalente decimal do número binário representado pelo argumento `binary_string`.

bindec() converte um número binário em um int ou, se necessário por questão de tamanho, um float.

bindec() interpreta todos os valores de `binary_string` como inteiros sem sinal. Isto é devido a **bindec()** enxergar o bit mais significativo como uma outra ordem de magnitude e não como um bit de sinal.

Parâmetros

`binary_string`

A string binária a converter. Quaisquer caracteres inválidos em `binary_string` são silenciosamente ignorados. A partir do PHP 7.4.0, fornecer qualquer caractere inválido foi descontinuado.

> **Aviso**
>
> O parâmetro deve ser uma string. Usar outros tipos de dados produzirá resultados inesperados.

Valor Retornado

O valor decimal de `binary_string`

Registro de Alterações

Versão	Descrição
7.4.0	Passar caracteres inválidos agora irá gerar uma nota de descontinuação. O resultado ainda será computado como se os caracteres inválidos não existissem.

Examples

Exemplo #1 Exemplo de bindec()

```php
<?php
echo bindec('110011') . "\n";
echo bindec('000110011') . "\n";
```

```php
echo bindec('111');
?>
```

O exemplo acima produzirá:

```
51
51
7
```

Exemplo #2 bindec() interpreta a entrada como inteiros sem sinal

```php
<?php
/*
 * A lição deste exemplo está na saída
 * e não no código do PHP em si.
 */

$magnitude_lower = pow(2, (PHP_INT_SIZE * 8) - 2);
p($magnitude_lower - 1);
p($magnitude_lower, 'Percebeu a rolagem?  Veja da próxima vez...');

p(PHP_INT_MAX, 'PHP_INT_MAX');
p(~PHP_INT_MAX, 'interpretado como um a mais que PHP_INT_MAX');

if (PHP_INT_SIZE == 4) {
    $note = 'interpretado como o maior inteiro sem sinal';
} else {
    $note = 'interpretado como o maior inteiro sem sinal
             (18446744073709551615) porém distorcido pela precisão de float';
}
p(-1, $note);


function p($input, $note = '') {
    echo "entrada:        $input\n";

    $format = '%0' . (PHP_INT_SIZE * 8) . 'b';
    $bin = sprintf($format, $input);
    echo "binário:       $bin\n";

    ini_set('precision', 20);  // Para legibilidade em máquinas 64 bits.
    $dec = bindec($bin);
    echo 'bindec():     ' . $dec . "\n";

    if ($note) {
        echo "NOTA:         $note\n";
    }

    echo "\n";
}
?>
```

Saída do exemplo acima em máquinas de 32 bits:

```
input:        1073741823
binary:       00111111111111111111111111111111
bindec():     1073741823

input:        1073741824
binary:       01000000000000000000000000000000
bindec():     1073741824
NOTE:         Percebeu a rolagem?  Veja da próxima vez...
```

```
input:        2147483647
binary:       01111111111111111111111111111111
bindec():     2147483647
NOTE:         PHP_INT_MAX

input:        -2147483648
binary:       10000000000000000000000000000000
bindec():     2147483648
NOTE:         interpretado como um a mais que PHP_INT_MAX

input:        -1
binary:       11111111111111111111111111111111
bindec():     4294967295
NOTE:         interpretado como o maior inteiro sem sinal
```

Saída do exemplo acima em máquinas de 64 bits:

```
input:        4611686018427387903
binary:       0011111111111111111111111111111111111111111111111111111111111111
bindec():     4611686018427387903

input:        4611686018427387904
binary:       0100000000000000000000000000000000000000000000000000000000000000
bindec():     4611686018427387904
NOTE:         Percebeu a rolagem?  Veja da próxima vez...

input:        9223372036854775807
binary:       0111111111111111111111111111111111111111111111111111111111111111
bindec():     9223372036854775807
NOTE:         PHP_INT_MAX

input:        -9223372036854775808
binary:       1000000000000000000000000000000000000000000000000000000000000000
bindec():     9223372036854775808
NOTE:         interpretado como um a mais que PHP_INT_MAX

input:        -1
binary:       1111111111111111111111111111111111111111111111111111111111111111
bindec():     18446744073709551616
NOTE:         interpretado como o maior inteiro sem sinal
              (18446744073709551615) porém distorcido pela precisão de float
```

Notas

> **Nota** :
>
> A função pode converter números que são grandes demais para caber no tipo int da plataforma, valores maiores são retornados como float neste caso.

Veja também

- decbin() - Decimal para binário
- octdec() - Octal para decimal
- hexdec() - Hexadecimal para decimal
- base_convert() - Converte um número entre bases arbitrárias

ceil

ceil

(PHP 4, PHP 5, PHP 7, PHP 8)

ceil — Arredonda frações para cima

Descrição

function **ceil**(int|float `$num`): float

Retorna o próximo maior valor inteiro arredondando-se `num` para cima, se necessário.

Parâmetros

`num`

O valor a ser arredondado

Valor Retornado

`num` arredondado pra o próximo inteiro. O valor retornado de **ceil()** ainda será do tipo float porque a faixa de valores suportados por float é normalmente maior do que o do tipo int.

Registro de Alterações

Versão	Descrição
8.0.0	`num` não mais aceita objetos internos que suportam conversão numérica.

Examples

Exemplo #1 Exemplo de ceil()

```php
<?php
echo ceil(4.3), PHP_EOL;    // 5
echo ceil(9.999), PHP_EOL;  // 10
echo ceil(-3.14), PHP_EOL; // -3
?>
```

Veja também

- floor() - Arredonda frações para baixo
- round() - Arredonda um float

clamp

clamp

(PHP 8 >= 8.6.0)

clamp — Return the given value if in range, else return the nearest bound

Descrição

function **clamp**(mixed `$value`, mixed `$min`, mixed `$max`): mixed

Return the given value if in range of `min` and `max`. Else if it's lower than `min`, return `min`. Else return `max`.

> **Nota** :
>
> Values of different types will be compared using the standard comparison rules. For instance, a non-numeric string will be compared to an int as though it were `0`, but multiple non-numeric string values will be compared alphanumerically. The actual value returned will be of the original type with no conversion applied.

> **Cuidado**
>
> Be careful when passing arguments of different types because **clamp()** can produce unpredictable results.

Parâmetros

`value`

Any comparable value to be clamped between `min` and `max`.

`min`

A comparable minimum to limit the `value` to.

`max`

A comparable maximum to limit the `value` to.

Valor Retornado

clamp() returns the parameter `value` if considered "between" `min` and `max` according to standard comparisons.

If `value` is **NAN**, then the returned value will also be **NAN**.

Erros/Exceções

If `min` is greater than `max`, **clamp()** throws a ValueError.

If `min` or `max` is **NAN**, **clamp()** throws a ValueError.

Examples

```php
<?php
echo clamp(-5, min: 0, max: 100), PHP_EOL;  // 0
echo clamp(55, min: 0, max: 100), PHP_EOL;  // 55
echo clamp(103, min: 0, max: 100), PHP_EOL;  // 100

echo clamp("J", min: "A", max: "F"), PHP_EOL;  // "F"

clamp(
    new \DateTimeImmutable('2025-08-01'),
    min: new \DateTimeImmutable('2025-08-15'),
    max: new \DateTimeImmutable('2025-09-15'),
)->format('Y-m-d'), PHP_EOL; // 2025-08-15
?>
```

Veja também

- min() - Encontra o menor valor
- max() - Localiza o maior valor

cos

cos

(PHP 4, PHP 5, PHP 7, PHP 8)

cos — Cosseno

Descrição

function **cos**(float $num): float

cos() retorna o cosseno do parâmetro `num`. O parâmetro `num` deve estar em radianos.

Parâmetros

`num`

Um ângulo em radianos

Valor Retornado

O cosseno de `num`

Examples

Exemplo #1 Exemplo de cos()

```php
<?php

echo cos(M_PI); // -1

?>
```

Veja também

- acos() - Arco cosseno
- sin() - Seno
- tan() - Tangente
- deg2rad() - Converte o número em graus para o equivalente em radianos

cosh

cosh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

cosh — Cosseno hiperbólico

Descrição

function **cosh**(float `$num`): float

Retorna o cosseno hiperbólico de `num`, definido por `(exp($num) + exp(-$num))/2`.

Parâmetros

`num`

O argumento a processar

Valor Retornado

O cosseno hiperbólico de `num`

Veja também

- cos() - Cosseno
- acosh() - Arco cosseno hiperbólico
- sinh() - Seno hiperbólico
- tanh() - Tangente hiperbólica

decbin

decbin

(PHP 4, PHP 5, PHP 7, PHP 8)

decbin — Decimal para binário

Descrição

function **decbin**(int `$num`): string

Retorna uma string contendo uma representação binária do argumento `num` informado.

Parâmetros

`num`

Valor decimal a ser convertido

Limites de entrada em máquinas de 32 bits

`num` positivo	`num` negativo	valor de retorno

num positivo	num negativo	valor de retorno
0		0
1		1
2		10
... progressão normal ...		
2147483646		11111111111111111111111111111110
2147483647 (maior inteiro com sinal)		11111111111111111111111111111111 (31 números 1)
2147483648	-2147483648	10000000000000000000000000000000
... progressão normal ...		
4294967294	-2	11111111111111111111111111111110
4294967295 (maior inteiro sem sinal)	-1	11111111111111111111111111111111 (32 números 1)

Limites de entrada em máquinas de 64 bits

num positivo	num negativo	valor de retorno
0		0
1		1
2		10
... progressão normal ...		
9223372036854775806		111111111111111111111111111111111111111111111111111111111111110
9223372036854775807 (maior inteiro com sinal)		111111111111111111111111111111111111111111111111111111111111111 (63 números 1)
	-9223372036854775808	1000000000000000000000000000000000000000000000000000000000000000
... progressão normal ...		
	-2	111111111111111111111111111111111111111111111111111111111111110
	-1	111111111111111111111111111111111111111111111111111111111111111 (64 números 1)

Valor Retornado

Representação de num em string binária.

Examples

Exemplo #1 Exemplo de decbin()

```php
<?php
echo decbin(12) . "\n";
echo decbin(26);
?>
```

O exemplo acima produzirá:

```
1100
11010
```

Veja também

- bindec() - Binário para decimal
- decoct() - Decimal para octal
- dechex() - Decimal para hexadecimal
- base_convert() - Converte um número entre bases arbitrárias

- printf() - Envia uma string formatada para a saída, usando `%b`, `%032b` ou `%064b` como formato
- sprintf() - Retona uma string formatada, usando `%b`, `%032b` ou `%064b` como formato

dechex

dechex

(PHP 4, PHP 5, PHP 7, PHP 8)

dechex — Decimal para hexadecimal

Descrição

function **dechex**(int `$num`): string

Retorna uma string contendo uma representação hexadecimal do argumento sem sinal `num` fornecido.

O maior número que pode ser convertido é `PHP_INT_MAX * 2 + 1` (ou `-1`): em plataformas de 32 bits, será `4294967295` em decimal, que resulta em **dechex()** retornando `ffffffff`.

Parâmetros

`num`

O valor decimal a ser convertido

Como o tipo int do PHP tem sinal, mas **dechex()** lida com inteiros sem sinal, inteiros negativos serão tratados como se fossem sem sinal.

Valor Retornado

Representação de `num` em string hexadecimal.

Examples

Exemplo #1 Exemplo de dechex()

```php
<?php
echo dechex(10) . "\n";
echo dechex(47);
?>
```

O exemplo acima produzirá:

```
a
2f
```

Exemplo #2 Exemplo de dechex() com inteiros grandes

```php
<?php
// A saída abaixo assume uma plataforma de 32 bits.
// Note que a saída é a mesma para todos os valores.
echo dechex(-1)."\n";
echo dechex(PHP_INT_MAX * 2 + 1)."\n";
echo dechex(pow(2, 32) - 1)."\n";
?>
```

O exemplo acima produzirá:

```
ffffffff
ffffffff
```

```
ffffffff
```

Veja também

- hexdec() - Hexadecimal para decimal
- decbin() - Decimal para binário
- decoct() - Decimal para octal
- base_convert() - Converte um número entre bases arbitrárias

decoct

decoct

(PHP 4, PHP 5, PHP 7, PHP 8)

decoct — Decimal para octal

Descrição

function **decoct**(int `$num`): string

Retorna uma string contendo uma representação octal do argumento `num` fornecido. O maior número que pode ser convertido depende da plataforma em uso. Para plataformas de 32 bits é normalmente `4294967295` em decimal, resultando em `37777777777`. Para plataformas de 64 bits é normalmente `9223372036854775807` em decimal resultando em `7777777777777777777`.

Parâmetros

`num`

Valor decimal a ser convertido

Valor Retornado

Representação de `num` em string octal.

Examples

Exemplo #1 Exemplo de decoct()

```php
<?php
echo decoct(15) . "\n";
echo decoct(264);
?>
```

O exemplo acima produzirá:

```
17
410
```

Veja também

- octdec() - Octal para decimal
- decbin() - Decimal para binário
- dechex() - Decimal para hexadecimal
- base_convert() - Converte um número entre bases arbitrárias

deg2rad

deg2rad

(PHP 4, PHP 5, PHP 7, PHP 8)

deg2rad — Converte o número em graus para o equivalente em radianos

Descrição

function **deg2rad**(float $num): float

Esta função converte num de graus para o equivalente em radianos.

Parâmetros

num

Valor do ângulo em graus

Valor Retornado

O radiano equivalente ao num

Examples

Exemplo #1 Exemplo de deg2rad()

```php
<?php

echo deg2rad(45), PHP_EOL; // 0.785398163397
var_dump(deg2rad(45) === M_PI_4); // bool(true)

?>
```

Veja também

- rad2deg() - Converte o número em radianos para o equivalente em graus

exp

exp

(PHP 4, PHP 5, PHP 7, PHP 8)

exp — Calcula o expoente de e

Descrição

function **exp**(float $num): float

Retorna e elevada à potência de num.

> **Nota** :
>
> 'e' é a base do sistema natural de logaritmos, ou aproximadamente 2.718282.

Parâmetros

num

O argumento a ser processado

Valor Retornado

'e' elevado à potência de num

Examples

Exemplo #1 Exemplo de exp()

```php
<?php
echo exp(12), PHP_EOL;
```

```
echo exp(5.7);
?>
```

O exemplo acima produzirá:

```
162754.791419
298.86740096706
```

Veja também

- log() - Logaritmo natural
- pow() - Expressão exponencial

expm1

expm1

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

expm1 — Retorna `exp($num) - 1`, computado de forma precisa mesmo quando o valor do número está próximo de zero.

Descrição

function **expm1**(float `$num`): float

expm1() retorna o equivalente a `exp($num) - 1` computado de forma precisa mesmo se o valor de `num` for próximo de zero, que é um caso onde `exp($num) - 1` poderia ser impreciso devido à subtração de dois números quase iguais.

Parâmetros

`num`

O argumento a ser processado

Valor Retornado

`e` elevado à potência de `num` menos um.

Veja também

- log1p() - Retorna log(1 + número), calculado de forma que seja preciso mesmo quando o valor do número seja próximo de zero.
- exp() - Calcula o expoente de e

fdiv

fdiv

(PHP 8)

fdiv — Divide dois números de acordo com a IEEE 754

Descrição

function **fdiv**(float `$num1`, float `$num2`): float

Retorna o resultado em ponto flutuante da divisão de `num1` por `num2`. Se `num2` for zero, um dos valores **`INF`**, **`-INF`** ou **`NAN`** será retornado.

Note que em comparações, **`NAN`** nunca será igual (==) ou idêntica (===) a nenhum valor, incluindo a si mesma.

Parâmetros

`num1`

O dividendo (numerador)

`num2`

O divisor

Valor Retornado

O resultado em ponto flutuante de `num1`/`num2`

Examples

Exemplo #1 Usando **fdiv()**

```
<?php
var_dump(fdiv(5.7, 1.3)); // float(4.3846153846153851)
var_dump(fdiv(4, 2)); // float(2)
var_dump(fdiv(1.0, 0.0)); // float(INF)
var_dump(fdiv(-1.0, 0.0)); // float(-INF)
var_dump(fdiv(0.0, 0.0)); // float(NAN)
?>
```

Veja também

- Operador de divisão `/`
- intdiv() - Divisão de inteiros - Divisão de inteiros
- fmod() - Retorna o resto (módulo) em ponto flutuante da divisão dos argumentos
- fpow() - Eleva um número à potência de outro, de acordo com a IEEE 754

floor

floor

(PHP 4, PHP 5, PHP 7, PHP 8)

floor — Arredonda frações para baixo

Descrição

function **floor**(int|float `$num`): float

Retorna o próximo menor valor inteiro (como float), arredondando `num` para baixo, se necessário.

Parâmetros

`num`

O valor numérico a ser arredondado

Valor Retornado

`num` arredondado para o inteiro menor mais próximo. O valor retornado por **floor()** ainda será do tipo float.

Registro de Alterações

Versão	Descrição
8.0.0	`num` não mais aceita tipos internos que suportam conversão numérica.

Examples

Exemplo #1 Exemplos de **floor()**

```
<?php
echo floor(4.3), PHP_EOL;   // 4
echo floor(9.999), PHP_EOL; // 9
echo floor(-3.14), PHP_EOL; // -4
?>
```

Veja também

- ceil() - Arredonda frações para cima

- round() - Arredonda um float

fmod

fmod

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

fmod — Retorna o resto (módulo) em ponto flutuante da divisão dos argumentos

Descrição

function **fmod**(float `$num1`, float `$num2`): float

Retorna o resto em ponto flutuante da divisão do dividendo (`num1`) pelo divisor (`num2`). O resto (*r*) é definido como: num1 = i * num2 + r, para algum inteiro *i*. Se o `num2` for diferente de zero, *r* terá o mesmo sinal de `num1` e uma magnitude menor que a magnitude de `num2`.

Parâmetros

`num1`

O dividendo

`num2`

O divisor

Valor Retornado

O resto em ponto flutuante de `num1`/`num2`. **`NAN`** (float) se o segundo argumento for 0.

Examples

Exemplo #1 Usando fmod()

```
<?php
$x = 5.7;
$y = 1.3;
$r = fmod($x, $y);
// $r é igual a 0.5, porque 4 * 1.3 + 0.5 = 5.7

var_dump($x, $y, $r);
?>
```

Veja também

- `/` - Divisão de ponto flutuante
- `%` - Módulo de inteiros
- intdiv() - Divisão de inteiros - Divisão de inteiros

fpow

fpow

(PHP 8 >= 8.4.0)

fpow — Eleva um número à potência de outro, de acordo com a IEEE 754

Descrição

function **fpow**(float `$num`, float `$exponent`): float

Retorna o resultado em ponto flutuante da elevação de `num` à potência de `exponent`. Se `num` for zero e `exponent` for menor que zero, retorna **`INF`**.

Parâmetros

`num`

A base a ser usada.

`exponent`

O expoente.

Valor Retornado

Retorna um float correspondente a $\texttt{\$num}^{\texttt{\$exponent}}$.

Examples

Exemplo #1 Exemplo de fpow()

```php
<?php
var_dump(fpow(10, 2));
var_dump(fpow(0, -3));
var_dump(fpow(-1, 5.5));
?>
```

O exemplo acima produzirá:

```
float(100)
float(INF)
float(NAN)
```

Veja também

- Operador de exponenciação **
- pow() - Expressão exponencial
- fdiv() - Divide dois números de acordo com a IEEE 754
- fmod() - Retorna o resto (módulo) em ponto flutuante da divisão dos argumentos

hexdec

hexdec

(PHP 4, PHP 5, PHP 7, PHP 8)

hexdec — Hexadecimal para decimal

Descrição

function **hexdec**(string `$hex_string`): int|float

Retorna o equivalente em decimal do número hexadecimal representado pelo argumento `hex_string`. **hexdec()** converte uma string hexadecimal para um número decimal.

hexdec() irá ignorar qualquer caractere não hexadecimal que encontrar. A partir do PHP 7.4.0 fornecer caracteres inválidos foi descontinuado.

Parâmetros

`hex_string`

A string hexadecimal a ser convertida

Valor Retornado

A representação decimal de `hex_string`

Registro de Alterações

Versão	Descrição
7.4.0	Passar caracteres inválidos agora irá gerar uma nota de descontinuação. O resultado ainda será computado como se os caracteres inválidos não existissem.

Examples

Exemplo #1 Exemplo de hexdec()

```
<?php
var_dump(hexdec("ee")); // exibe "int(238)"
var_dump(hexdec("a0")); // exibe "int(160)"
?>
```

Exemplo #2 hexdec() com caracteres inválidos

```
<?php
var_dump(hexdec("See"));  // print "int(238)"
var_dump(hexdec("that")); // mostra "int(10)"
?>
```

Notas

> **Nota** :
>
> A função pode converter números que são longos demais para caber nos tipos int das plataformas, valores maiores são retornados como float neste caso.

Veja também

- dechex() - Decimal para hexadecimal
- bindec() - Binário para decimal
- octdec() - Octal para decimal
- base_convert() - Converte um número entre bases arbitrárias

hypot

hypot

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

hypot — Calcula o comprimento da hipotenusa de triângulo retângulo

Descrição

function **hypot**(float $x, float $y): float

hypot() retorna o comprimento da hipotenusa de um triângulo retângulo com lados de tamanho `x` e `y`, ou a distância do ponto (`x`, `y`) a partir da origem. É o equivalente a `sqrt($x*$x + $y*$y)`.

Parâmetros

x

Comprimento do primeiro lado

y

Comprimento do segundo lado

Valor Retornado

Comprimento calculado da hipotenusa

intdiv

intdiv

(PHP 7, PHP 8)

intdiv — Divisão de inteiros

Descrição

function **intdiv**(int $num1, int $num2): int

Retorna o quociente inteiro da divisão de num1 por num2.

Parâmetros

num1

Número a ser dividido.

num2

Número que divide o num1.

Valor Retornado

O quociente inteiro da divisão de num1 por num2.

Erros/Exceções

Se num2 for 0, uma exceção DivisionByZeroError é lançada. Se num1 for **PHP_INT_MIN** e num2 for -1, então uma exceção ArithmeticError é lançada.

Examples

Exemplo #1 Exemplo de intdiv()

```
<?php
var_dump(intdiv(3, 2));
var_dump(intdiv(-3, 2));
var_dump(intdiv(3, -2));
var_dump(intdiv(-3, -2));
var_dump(intdiv(PHP_INT_MAX, PHP_INT_MAX));
var_dump(intdiv(PHP_INT_MIN, PHP_INT_MIN));
?>
```

O exemplo acima produzirá:

```
int(1)
int(-1)
int(-1)
int(1)
int(1)
int(1)
```

Exemplo #2 Exemplo de intdiv() com divisor inválido

```
<?php
try {
    intdiv(PHP_INT_MIN, -1);
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
}

try {
    intdiv(1, 0);
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
```

```
}
?>
```

O exemplo acima produzirá:

```
ArithmeticError: Division of PHP_INT_MIN by -1 is not an integer
DivisionByZeroError: Division by zero
```

Veja também

- `/` - Divisão de ponto flutuante
- `%` - Resto de divisão de inteiros
- fmod() - Retorna o resto (módulo) em ponto flutuante da divisão dos argumentos - Resto de divisão em ponto flutuante

is_finite

is_finite

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_finite — Verifica se um float é finito

Descrição

function **is_finite**(float `$num`): bool

Retorna informando se `num` é um float finito.

Um float finito não é **`NAN`** (is_nan()), nem um infinito (is_infinite()).

Parâmetros

`num`

O float a ser verificado

Valor Retornado

`true` se `num` não for **`NAN`**, **`INF`** nem **`-INF`**, caso contrário retorna **`false`**.

Examples

Exemplo #1 Exemplo de is_finite()

```
<?php
$float = 1.2345;
var_dump($float, is_finite($float));

$nan = sqrt(-1);
var_dump($nan, is_finite($nan));

$inf = 1e308 * 2;
var_dump($inf, is_finite($inf));
?>
```

O exemplo acima produzirá:

```
float(1.2345)
bool(true)
float(NAN)
bool(false)
```

```
float(INF)
bool(false)
```

Veja também

- is_infinite() - Verifica se um float é infinito
- is_nan() - Verifica se um float é NAN

is_infinite

is_infinite

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_infinite — Verifica se um float é infinito

Descrição

function **is_infinite**(float `$num`): bool

Retorna informando se `num` é `INF` ou `-INF`.

Parâmetros

`num`

O float a ser verificado

Valor Retornado

`true` se `num` for `INF` ou `-INF`, senão retorna `false`.

Examples

Exemplo #1 Exemplo de is_infinite()

```php
<?php
$inf = 1e308 * 2;

var_dump($inf, is_infinite($inf));

$negative_inf = -$inf;

var_dump($negative_inf, is_infinite($negative_inf));
?>
```

O exemplo acima produzirá:

```
float(INF)
bool(true)
float(-INF)
bool(true)
```

Veja também

- is_finite() - Verifica se um float é finito
- is_nan() - Verifica se um float é NAN

is_nan

is_nan

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_nan — Verifica se um float é NAN

Descrição

function **is_nan**(float `$num`): bool

Retorna indicando se o `num` é um `NAN` ("Not A Number").

`NAN` é retornado de operações matemáticas que são indefinidas, por exemplo ao passar parâmetros fora do domínio de entrada da função. A raiz quadrada (sqrt()) só é definida para números positivos, e informar um número negativo resultará em `NAN`. Outros exemplos de operações que retornam `NAN` são a divisão de `INF` por `INF` e operações que envolvam um valor `NAN` existente.

> **Nota** :
>
> Apesar do nome "Not A Number", `NAN` é um valor válido do tipo float.

> **Cuidado**
>
> `NAN` não se compara em igualdade a `NAN`. Para verificar se um float é `NAN`, **is_nan()** deve ser utilizado. Usar `$float === NAN` não funcionará.

Parâmetros

`num`

O float a ser verificado

Valor Retornado

`true` se `num` for `NAN`, senão retorna `false`.

Examples

Exemplo #1 Exemplo de is_nan()

```php
<?php
$nan = sqrt(-1);

var_dump($nan, is_nan($nan));
?>
```

O exemplo acima produzirá:

```
float(NAN)
bool(true)
```

Veja também

- is_finite() - Verifica se um float é finito
- is_infinite() - Verifica se um float é infinito

log

log

(PHP 4, PHP 5, PHP 7, PHP 8)

log — Logaritmo natural

Descrição

function **log**(float $num, float $base = M_E): float

Se a base opcional for informada, **log()** retorna $\log_{base}$ num, senão **log()** retorna o logaritmo natural de num.

Parâmetros

num

O valor para o qual o logaritmo será calculado

base

A base logaritmica opcional a ser utilizada (padrão é 'e' para logaritmo natural).

Valor Retornado

O logarimo de num na base, se fornecida, ou o logaritmo natural.

Veja também

- log10() - Logaritmo na base 10
- exp() - Calcula o expoente de e
- pow() - Expressão exponencial
- error_log() - Envia uma mensagem de erro para as rotinas de tratamento de erros definidas

log10

log10

(PHP 4, PHP 5, PHP 7, PHP 8)

log10 — Logaritmo na base 10

Descrição

function **log10**(float $num): float

Retorna o logaritmo na base 10 de num.

Parâmetros

num

O argumento a ser processado

Valor Retornado

O logaritmo na base 10 de num

Veja também

- log() - Logaritmo natural

log1p

log1p

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

log1p — Retorna log(1 + número), calculado de forma que seja preciso mesmo quando o valor do número seja próximo de zero.

Descrição

function **log1p**(float $num): float

log1p() retorna log(1 + num) computado de forma que seja preciso mesmo quando o valor de num esteja próximo de zero. log() poderia retornar apenas log(1) neste caso devido à falta de precisão.

Parâmetros

`num`

O argumento a ser processado

Valor Retornado

log(1 + `num`)

Veja também

- expm1() - Retorna exp($num) - 1, computado de forma precisa mesmo quando o valor do número está próximo de zero.
- log() - Logaritmo natural
- log10() - Logaritmo na base 10

max

max

(PHP 4, PHP 5, PHP 7, PHP 8)

max — Localiza o maior valor

Descrição

function **max**(mixed `$value`, mixed `...$values`): mixed

Forma alternativa (não suportada com argumentos nomeados):

function **max**(array `$value_array`): mixed

Se o primeiro e único parâmetro for um array, **max()** retorna o maior valor do array. Se no mínimo dois parâmetros forem fornecidos, **max()** retorna o maior desses valores.

> **Nota** :
>
> Valores de diferentes tipos serão comparados utilizando as regras padrões de comparação. Por exemplo, uma string não numérica será comparada a um int como se fosse `0`, mas várias strings não numéricas serão comparadas alfanumericamente. O valor real retornado terá o tipo do original, sem nenhuma conversão aplicada.

> **Cuidado**
>
> Cuidado ao passar argumentos de tipos diferentes, pois **max()** pode produzir resultados imprevisíveis.

Parâmetros

`value`

Qualquer valor comparável.

`values`

Quaisquer valores comparáveis.

`value_array`

Um array contendo os valores.

Valor Retornado

max() retorna o parâmetro considerado o "maior" de acordo com as comparações padrões. Se vários valores de tipos diferentes forem considerados iguais (por exemplo, `0` e `'abc'`) então o primeiro fornecido à função será retornado.

Erros/Exceções

Se um array vazio for informado, **max()** lança um ValueError.

Registro de Alterações

Versão	Descrição
8.0.0	**max()** lança um ValueError em caso de erros. Anteriormente, retornava `false` e emitia um alerta `E_WARNING`.
8.0.0	Como as comparações de strings com números foram alteradas, **max()** não mais retorna um valor diferente por conta da ordem dos argumentos informados.

Examples

Exemplo #1 Exemplos de max()

```php
<?php
echo max(2, 3, 1, 6, 7), PHP_EOL;  // 7
echo max(array(2, 4, 5)), PHP_EOL; // 5

// Aqui na prática compara-se -1 < 0, então 'hello' é o maior valor
echo max('hello', -1), PHP_EOL;    // hello

// Arrays de múltiplos valores e tamanhos diferentes, retora o mais longo
$val = max(array(2, 2, 2), array(1, 1, 1, 1)); // array(1, 1, 1, 1)
var_dump($val);

// Em arrays de mesmo tamanho compara-se da esquerda para direita
// e nesse exemplo: 2 == 2, mas 5 > 4
$val = max(array(2, 4, 8), array(2, 5, 1)); // array(2, 5, 1)
var_dump($val);

// Se um array e um não array são informados, o array será retornado
// pois é considerado maior que qualquer outro valor
$val = max('string', array(2, 5, 7), 42);   // array(2, 5, 7)
var_dump($val);

// Se um argumento é NULL ou booleano, ele será comparado contra
// outros valores utilizando a regra FALSE < TRUE independente dos tipos envolvidos.
// No exemplo abaixo -10 é tratado como TRUE na comparação
$val = max(-10, FALSE); // -10
var_dump($val);

// 0 por outro lado é considerado false, de forma que é "menor que" TRUE
$val = max(0, TRUE); // TRUE
var_dump($val);
?>
```

Veja também

- min() - Encontra o menor valor
- count() - Conta todos os elementos de um array ou de um objeto Countable

min

min

(PHP 4, PHP 5, PHP 7, PHP 8)

min — Encontra o menor valor

Descrição

function **min**(mixed `$value`, mixed `...$values`): mixed

Forma alternativa (não suportada com argumentos nomeados):

function **min**(array `$value_array`): mixed

Se o primeiro e único argumento é um array, **min()** retorna o menor valor no array. Se no mínimo dois parâmetros são fornecidos, **min()** retorna o menor deles.

> **Nota** :

> Valores de tipos diferentes serão comparados utilizando as regras padrões de comparação. Por exemplo, uma string não numérica será comparada com um inteiro int como se fosse `0`, mas várias strings não numéricos serão comparadas alfanumericamente. O valor real retornado terá o tipo do original, sem nenhuma conversão aplicada.

> **Cuidado**
>
> Cuidado ao passar argumentos de tipos diferentes, pois **min()** pode produzir resultados imprevisíveis.

Parâmetros

`value`

Qualquer valor comparável.

`values`

Quaisquer valores comparáveis.

`value_array`

Um array contendo os valores.

Valor Retornado

min() retorna o valor considerado o "menor" de acordo com as comparações padrão. Se valores de diferentes tipos forem avaliados como iguais (por exemplo `0` e `'abc'`) então o primeiro valor fornecido à função será retornado.

Erros/Exceções

Se um array vazio for informado, **min()** lança um ValueError.

Registro de Alterações

Versão	Descrição
8.0.0	**min()** lança um ValueError em caso de falhas; anteriormente retornava `false` e emitia um alerta `E_WARNING`.
8.0.0	Como as comparações de número e string foram alteradas, **min()** não mais retorna um valor diferente por conta da ordem dos argumentos informados.

Examples

Exemplo #1 Exemplos de min()

```php
<?php
echo min(2, 3, 1, 6, 7), PHP_EOL;  // 1
echo min(array(2, 4, 5)), PHP_EOL; // 2

// Aqui a comparação que ocorre é -1 < 0, e -1 é o menor valor
echo min('hello', -1), PHP_EOL;    // -1

// Quando comparando arrays de vários comprimentos, min retorna o mais curto
$val = min(array(2, 2, 2), array(1, 1, 1, 1)); // array(2, 2, 2)
var_dump($val);

// Vários arrays de mesmo tamanho, a comparação ocorre nos elementos, da esquerda para direita
// so in our example: 2 == 2, but 4 < 5
$val = min(array(2, 4, 8), array(2, 5, 1)); // array(2, 4, 8)
var_dump($val);

// Se for fornecido um array e um primitivo, o array nunca é retornado
// pois as comparações tratam os arrays como maiores que outros valores
$val = min('string', array(2, 5, 7), 42);   // string
var_dump($val);

// Se um argumento é null ou booleano, ele será comparado com outros valores
// usando a regra que FALSE < TRUE e NULL == FALSE,
// independente dos tipos envolvidos
// Nos exemplos abaixo -10 e 10 são tratados como TRUE nas comparações
$val = min(-10, FALSE, 10); // FALSE
```

```php
var_dump($val);

$val = min(-10, NULL, 10);  // NULL
var_dump($val);

// 0, por outro lado, é tratado como FALSE, então é "menor que" TRUE
$val = min(0, TRUE); // 0
var_dump($val);
?>
```

Veja também

- max() - Localiza o maior valor
- count() - Conta todos os elementos de um array ou de um objeto Countable

octdec

octdec

(PHP 4, PHP 5, PHP 7, PHP 8)

octdec — Octal para decimal

Descrição

function **octdec**(string `$octal_string`): int|float

Retorna o decimal equivalente do número octal representado pelo argumento `octal_string`.

Parâmetros

`octal_string`

A string octal a ser convertida. Quaisquer caracteres inválidos em `octal_string` são ignorados silenciosamente. A partir do PHP 7.4.0, informar caracteres inválidos foi descontinuado.

Valor Retornado

A representação decimal de `octal_string`

Registro de Alterações

Versão	Descrição
7.4.0	Passar caracteres inválidos gerará um aviso de descontinuação. O resultado será calculado como se os caracteres inválidos não existissem.

Examples

Exemplo #1 Exemplo da octdec()

```php
<?php
echo octdec('77') . "\n";
echo octdec(decoct(45));
?>
```

O exemplo acima produzirá:

```
63
45
```

Notas

> **Nota** :

> A função pode converter números que são muito grandes para caber no tipo int da plataforma, e nesse caso os valores são retornados como float.

Veja também

- decoct() - Decimal para octal
- bindec() - Binário para decimal
- hexdec() - Hexadecimal para decimal
- base_convert() - Converte um número entre bases arbitrárias

pi

pi

(PHP 4, PHP 5, PHP 7, PHP 8)

pi — Obtém o valor de pi

Descrição

function **pi**(): float

Retorna uma aproximação de pi. Alternativamente, pode-se usar a constante `M_PI` que fornece resultado idêntico a **pi()**.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor de pi como float.

Examples

Exemplo #1 Exemplo de pi()

```php
<?php
echo pi(), PHP_EOL; // 3.1415926535898
echo M_PI, PHP_EOL; // 3.1415926535898
?>
```

pow

pow

(PHP 4, PHP 5, PHP 7, PHP 8)

pow — Expressão exponencial

Descrição

function **pow**(mixed `$num`, mixed `$exponent`): int|float|object

Retorna `num` elevado à porência de `exponent`.

> **Nota** :
>
> É possível usar o operador ** no lugar desta função.

Parâmetros

`num`

A base a ser usada

`exponent`

O expoente

Valor Retornado

`num` elevado à potência de `exponent`. Se ambos os valores forem inteiros não negativos e o resultado puder ser representado como um inteiro, o resultado será retornado com o tipo int, caso contrário será retornado como um float.

Extensões do PHP podem substituir o comportamento desta operação e podem fazer com que ela retorne um objeto.

Registro de Alterações

Versão	Descrição
8.4.0	Elevar `0` a um `exponent` negativo agora foi descontinuado.

Examples

Exemplo #1 Alguns exemplos de pow()

```php
<?php

var_dump(pow(2, 8)); // int(256)
echo pow(-1, 20), PHP_EOL; // 1
echo pow(0, 0), PHP_EOL; // 1
echo pow(10, -1), PHP_EOL; // 0.1

echo pow(-1, 5.5), PHP_EOL; // NAN
?>
```

Exemplo #2 Exemplos de pow() com objeto da extensão GMP

```php
<?php
var_dump(pow(new GMP("3"), new GMP("2"))); // object(GMP)
?>
```

Notas

> **Nota** :
>
> Esta função converterá todas as entradas para números, mesmo valores não escalares, o que pode levar a resultados *estranhos*.

Veja também

- Operador de exponenciação **
- fpow() - Eleva um número à potência de outro, de acordo com a IEEE 754
- exp() - Calcula o expoente de e
- sqrt() - Raiz quadrada
- bcpow() - Eleva um número de precisão arbitrária a outro
- gmp_pow() - Elevar o número a uma potência

rad2deg

rad2deg

(PHP 4, PHP 5, PHP 7, PHP 8)

rad2deg — Converte o número em radianos para o equivalente em graus

Descrição

function **rad2deg**(float `$num`): float

Esta função converte `num` de radianos para graus.

Parâmetros

`num`

Um valor em radianos

Valor Retornado

O equivalente de `num` em graus

Examples

Exemplo #1 Exemplo de rad2deg()

```php
<?php

echo rad2deg(M_PI_4); // 45

?>
```

Veja também

- deg2rad() - Converte o número em graus para o equivalente em radianos

round

round

(PHP 4, PHP 5, PHP 7, PHP 8)

round — Arredonda um float

Descrição

function **round**(int|float `$num`, int `$precision` = 0, int|RoundingMode `$mode` = **`RoundingMode::HalfAwayFromZero`**): float

Retorna um valor arredondado de `num` com a precisão especificada em `precision` (número de dígitos depois do ponto decimal). `precision` também pode ser negativo ou zero (padrão).

Parâmetros

`num`

Valor a ser arredondado.

`precision`

O número opcional de dígitos decimais para arredondar.

Se `precision` for positivo, `num` será arredondado para `precision` de dígitos significativos depois do ponto decimal.

Se `precision` for negativo, `num` será arredondado para `precision` de dígitos significativos depois do ponto decimal, isto é: o múltiplo mais próximo de `pow(10, -$precision)`, por exemplo: para um `precision` de -1 `num` é arredondado para dezenas, para um `precision` de -2 para centenas, etc.

`mode`

Use RoundingMode ou uma das seguintes constantes a seguir para especificar o modo com o qual o arredondamento ocorre.

Constantes	Descrição
`PHP_ROUND_HALF_UP`	Arredonda `num` para longe de zero quando ele está na metade do caminho, tornando 1.5 em 2 e -1.5 em -2.
`PHP_ROUND_HALF_DOWN`	Arredonda `num` na direção de zero quando ele está na metade do caminho, tornando 1.5 em 1 e -1.5 em -1.
`PHP_ROUND_HALF_EVEN`	Arredonda `num` em direção ao valor par mais próximo quando ele está na metade do caminho, tornando ambos 1.5 e 2.5 em 2.
`PHP_ROUND_HALF_ODD`	Arredonda `num` em direção ao valor ímpar mais próximo quando ele está na metade do caminho, tornando 1.5 em 1 e 2.5 em 3.

No entanto, observe que alguns modos novos adicionados existem apenas em RoundingMode.

Valor Retornado

O valor arredondado para `precision` fornecido como um float.

Erros/Exceções

A função lança uma exceção ValueError se `mode` for inválido. Antes do PHP 8.4.0, um modo inválido seria considerado, silenciosamente, como o padrão `PHP_ROUND_HALF_UP`.

Registro de Alterações

Versão	Descrição
8.4.0	Quatro novos modos de arredondamento foram adicionados.
8.4.0	Agora lança uma exceção ValueError se `mode` for inválido.
8.0.0	`num` não mais aceita objetos internos que suportam conversões numéricas.

Examples

Exemplo #1 Exemplos de round()

```
<?php
var_dump(round(3.4));
var_dump(round(3.5));
var_dump(round(3.6));
var_dump(round(3.6, 0));
var_dump(round(5.045, 2));
var_dump(round(5.055, 2));
var_dump(round(345, -2));
var_dump(round(345, -3));
var_dump(round(678, -2));
var_dump(round(678, -3));
?>
```

O exemplo acima produzirá:

```
float(3)
float(4)
float(4)
float(4)
float(5.05)
float(5.06)
float(300)
float(0)
float(700)
float(1000)
```

Exemplo #2 Como `precision` afeta um float

```
<?php
$numero = 135.79;

var_dump(round($numero, 3));
var_dump(round($numero, 2));
var_dump(round($numero, 1));
var_dump(round($numero, 0));
var_dump(round($numero, -1));
var_dump(round($numero, -2));
var_dump(round($numero, -3));
?>
```

O exemplo acima produzirá:

```
float(135.79)
float(135.79)
float(135.8)
float(136)
```

```
float(140)
float(100)
float(0)
```

Exemplo #3 Exemplos de `mode`

```php
<?php
echo 'Modos de arredondamento com 9.5' . PHP_EOL;
var_dump(round(9.5, 0, PHP_ROUND_HALF_UP));
var_dump(round(9.5, 0, PHP_ROUND_HALF_DOWN));
var_dump(round(9.5, 0, PHP_ROUND_HALF_EVEN));
var_dump(round(9.5, 0, PHP_ROUND_HALF_ODD));

echo PHP_EOL;
echo 'Modos de arredondamento com 8.5' . PHP_EOL;
var_dump(round(8.5, 0, PHP_ROUND_HALF_UP));
var_dump(round(8.5, 0, PHP_ROUND_HALF_DOWN));
var_dump(round(8.5, 0, PHP_ROUND_HALF_EVEN));
var_dump(round(8.5, 0, PHP_ROUND_HALF_ODD));
?>
```

O exemplo acima produzirá:

```
Modos de arredondamento com 9.5
float(10)
float(9)
float(10)
float(9)

Modos de arredondamento com 8.5
float(9)
float(8)
float(8)
float(9)
```

Exemplo #4 Exemplos de `mode` com `precision`

```php
<?php
echo 'Usando PHP_ROUND_HALF_UP com precisão decimal de 1 dígito' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_UP));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_UP));

echo PHP_EOL;
echo 'Usando PHP_ROUND_HALF_DOWN com precisão decimal de 1 dígito' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_DOWN));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_DOWN));

echo PHP_EOL;
echo 'Usando PHP_ROUND_HALF_EVEN com precisão decimal de 1 dígito' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_EVEN));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_EVEN));

echo PHP_EOL;
echo 'Usando PHP_ROUND_HALF_ODD com precisão decimal de 1 dígito' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_ODD));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_ODD));
?>
```

O exemplo acima produzirá:

```
Usando PHP_ROUND_HALF_UP com precisão decimal de 1 digito
float(1.6)
float(-1.6)

Usando PHP_ROUND_HALF_DOWN com precisão decimal de 1 digito
float(1.5)
float(-1.5)

Usando PHP_ROUND_HALF_EVEN com precisão decimal de 1 digito
float(1.6)
float(-1.6)

Usando PHP_ROUND_HALF_ODD com precisão decimal de 1 digito
float(1.5)
float(-1.5)
```

Exemplo #5 Exemplo de uso de RoundingMode

```php
<?php
foreach (RoundingMode::cases() as $mode) {
    foreach ([
        8.5,
        9.5,
        -3.5,
    ] as $number) {
        printf("%-17s: %+.17g -> %+.17g\n", $mode->name, $number, round($number, 0, $mode));
    }
    echo "\n";
}
?>
```

O exemplo acima produzirá:

```
HalfAwayFromZero : +8.5 -> +9
HalfAwayFromZero : +9.5 -> +10
HalfAwayFromZero : -3.5 -> -4

HalfTowardsZero  : +8.5 -> +8
HalfTowardsZero  : +9.5 -> +9
HalfTowardsZero  : -3.5 -> -3

HalfEven         : +8.5 -> +8
HalfEven         : +9.5 -> +10
HalfEven         : -3.5 -> -4

HalfOdd          : +8.5 -> +9
HalfOdd          : +9.5 -> +9
HalfOdd          : -3.5 -> -3

TowardsZero      : +8.5 -> +8
TowardsZero      : +9.5 -> +9
TowardsZero      : -3.5 -> -3

AwayFromZero     : +8.5 -> +9
AwayFromZero     : +9.5 -> +10
AwayFromZero     : -3.5 -> -4

NegativeInfinity : +8.5 -> +8
NegativeInfinity : +9.5 -> +9
NegativeInfinity : -3.5 -> -4

PositiveInfinity : +8.5 -> +9
PositiveInfinity : +9.5 -> +10
PositiveInfinity : -3.5 -> -3
```

Veja também

- ceil() - Arredonda frações para cima
- floor() - Arredonda frações para baixo
- number_format() - Formata um número com milhares agrupados

sin

sin

(PHP 4, PHP 5, PHP 7, PHP 8)

sin — Seno

Descrição

function **sin**(float $num): float

sin() retorna o seno do parâmetro num. O argumento num deve estar em radianos.

Parâmetros

num

O valor em radianos

Valor Retornado

O seno do num

Examples

Exemplo #1 Exemplo da sin()

```
<?php
// Precisão depende da diretiva precision
echo sin(deg2rad(60)), PHP_EOL;  //  0.866025403 ...
echo sin(60), PHP_EOL;           // -0.304810621 ...
?>
```

Veja também

- asin() - Arco seno
- sinh() - Seno hiperbólico
- cos() - Cosseno
- tan() - Tangente
- deg2rad() - Converte o número em graus para o equivalente em radianos

sinh

sinh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

sinh — Seno hiperbólico

Descrição

function **sinh**(float $num): float

Retorna o seno hiperbólico de num, definido como (exp($num) - exp(-$num))/2.

Parâmetros

num

O argumento para processar

Valor Retornado

O seno hiperbólico de `num`

Veja também

- sin() - Seno
- asinh() - Arco seno hiperbólico
- cosh() - Cosseno hiperbólico
- tanh() - Tangente hiperbólica

sqrt

sqrt

(PHP 4, PHP 5, PHP 7, PHP 8)

sqrt — Raiz quadrada

Descrição

function **sqrt**(float `$num`): float

Retorna a raiz quadrada de `num`.

Parâmetros

`num`

O argumento para processar

Valor Retornado

A raiz quadrada de `num` ou o valor especial `NAN` para números negativos.

Examples

Exemplo #1 Exemplo da sqrt()

```php
<?php
// Precisão depende de sua diretiva precision
echo sqrt(9), PHP_EOL; // 3
echo sqrt(10), PHP_EOL; // 3.16227766 ...
?>
```

Veja também

- pow() - Expressão exponencial
- `M_SQRTPI` - `sqrt(pi)`
- `M_2_SQRTPI` - `2/sqrt(pi)`
- `M_SQRT2` - `sqrt(2)`
- `M_SQRT3` - `sqrt(3)`
- `M_SQRT1_2` - `1/sqrt(2)`

tan

tan

(PHP 4, PHP 5, PHP 7, PHP 8)

tan — Tangente

Descrição

function **tan**(float `$num`): float

tan() retorna a tangente do parâmetro `num`. O argumento `num` deve estar em radianos.

Parâmetros

`num`

O argumento para processar em radianos

Valor Retornado

A tangente de `num`

Examples

Exemplo #1 Exemplo da tan()

```php
<?php

echo tan(M_PI_4); // 1

?>
```

Veja também

- atan() - Arco tangente
- atan2() - Arco tangente de duas variáveis
- sin() - Seno
- cos() - Cosseno
- tanh() - Tangente hiperbólica
- deg2rad() - Converte o número em graus para o equivalente em radianos

tanh

tanh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

tanh — Tangente hiperbólica

Descrição

function **tanh**(float `$num`): float

Retorna a tangente hiperbólica de `num`, definida como `sinh($num)/cosh($num)`.

Parâmetros

`num`

O argumento para processar

Valor Retornado

A tangente hiperbólica de `num`

Veja também

- tan() - Tangente
- atanh() - Arco tangente hiperbólico
- sinh() - Seno hiperbólico
- cosh() - Cosseno hiperbólico

Data e Horário

Introdução

A classe DateTimeImmutable e classes relacionadas permitem representar informações de data/horário. Os objetos podem ser criados passando-se uma representação de informação de data/horário no formato string, ou a partir do horário atual do sistema.

Existe também um conjunto rico de métodos para modificar e formatar estas informações, incluindo manipulação de fusos horários e transições de horários de verão.

As características de data/horário no PHP implementam o calendário ISO 8601, que é um » calendário gregoriano proléptico que implementa as regras atuais de anos bissextos estendidas a períodos anteriores ao que o calendário gregoriano foi adotado, e também inclui o ano `0` como o número do ano entre `-1` AEC e `1` EC. Segundos intercalares não são suportados.

A informação de data e horário é armazenada internamente como um número de 64 bits, sendo assim, todas as datas úteis concebíveis (incluindo anos negativos) são suportadas. O intervalo vai de aproximadamente 292 bilhões de anos no passado até o mesmo valor no futuro.

> **Nota** :
>
> Os fusos horários referenciados nesta seção podem ser encontrados em Lista de Fusos Horários Suportados.

Instalação/Configuração

Índice

Instalação

Nenhuma instalação é necessária para usar estas funções; elas fazem parte do núcleo do PHP.

> **Nota** : **Recuperando o banco de dados atualizado de fusos horários**
>
> A versão mais recente do banco de dados de fusos horários pode ser instalada com o pacote PECL » timezonedb.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração de Data/Horário

Nome	Padrão	Modificável	Registro de Alterações
date.default_latitude	"31.7667"	`INI_ALL`	
date.default_longitude	"35.2333"	`INI_ALL`	
date.sunrise_zenith	"90.833333"	`INI_ALL`	Antes do PHP 8.0.0, o padrão era "90.583333"
date.sunset_zenith	"90.833333"	`INI_ALL`	Antes do PHP 8.0.0, o padrão era "90.583333"
date.timezone	"UTC"	`INI_ALL`	A partir do PHP 8.2, um aviso é emitido quando este parâmetro é configurado para um valor inválido ou para um string vazio.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`date.default_latitude` float

A latitude padrão variando de `0` no equador, até `+90` para o norte, e `-90` para o sul.

`date.default_longitude` float

A longitude padrão variando de `0` no meridiano primário até `+180` para o leste e `-180` para o oeste.

`date.sunrise_zenith` float

O ápice do nascer do sol padrão.

O valor padrão é 90°50'. Os 50' adicionais são devido a dois componentes: o raio aparente do sol, que é 16', e a refração atmosférica aparente, que é de 34'.

`date.sunset_zenith` float

O ápice do pôr-do-sol padrão.

`date.timezone` string

O fuso horário padrão usado por todas as funções de data/horário. A ordem de precedência para qual fuso horário será utilizado se nenhum for explicitamente mencionado é descrita na função date_default_timezone_get(). Acesse Lista de Fusos Horários Suportados para uma lista de todos os fusos horários suportados.

> **Nota** :
>
> As primeiras quatro opções de configuração são utilizadas somente pelas funções date_sunrise() e date_sunset().

Constantes predefinidas

As constantes `DATE_*` estão definidas e oferecem uma representação padrão de datas, que podem ser utilizadas em conjunto com funções de formatação de data (como a função date()).

Valores possíveis do parâmetro `returnFormat` para date_sunrise() e date_sunset()

> **Aviso**
>
> Estas constantes estão descontinuadas a partir do PHP 8.4.0. As funções correspondentes date_sunrise() e date_sunset() estão descontinuadas a partir do PHP 8.1.0.

`SUNFUNCS_RET_TIMESTAMP` (int)

Timestamp

`SUNFUNCS_RET_STRING` (int)

Horas:minutos (exemplo: `08:02`)

`SUNFUNCS_RET_DOUBLE` (int)

Horas como número em ponto flutuante (exemplo `8.75`)

Constantes `DATE_*`
`DATE_ATOM` (string)

Atômico (exemplo: `2005-08-15T15:52:01+00:00`); compatível com ISO-8601, RFC 3339 e esquema XML

`DATE_COOKIE` (string)

Cookies HTTP (exemplo: `Monday, 15-Aug-2005 15:52:01 UTC`)

`DATE_ISO8601` (string)

Tipo ISO-8601 (exemplo: `2005-08-15T15:52:01+0000`)

> **Nota** :
>
> Este formato não é compatível com ISO-8601, mas é deixado desta forma por motivo de compatibilidade. Use `DATE_ISO8601_EXPANDED`, `DATE_ATOM` para compatibilidade com ISO-8601 em seu lugar (ref. ISO8601:2004 seção 4.3.3 cláusula d).

`DATE_ISO8601_EXPANDED` (string)

ISO-8601 Expandida (exemplo: `+10191-07-26T08:59:52+01:00`)

> **Nota** :
>
> Este formato permite intervalos de anos fora dos limites padrão da ISO-8601 de `0000-9999` através da inclusão se um caractere de sinal. Ele também garante que a parte de fuso horário (`+01:00`) seja compatível com a ISO-8601.

DATE_RFC822 (string)

RFC 822 (exemplo: `Mon, 15 Aug 05 15:52:01 +0000`)

DATE_RFC850 (string)

RFC 850 (exemplo: `Monday, 15-Aug-05 15:52:01 UTC`)

DATE_RFC1036 (string)

RFC 1036 (exemplo: `Mon, 15 Aug 05 15:52:01 +0000`)

DATE_RFC1123 (string)

RFC 1123 (exemplo: `Mon, 15 Aug 2005 15:52:01 +0000`)

DATE_RFC7231 (string)

RFC 7231 (a partir do PHP 7.0.19 e 7.1.5) (exemplo: `Sat, 30 Apr 2016 17:52:13 GMT`)

DATE_RFC2822 (string)

RFC 2822 (exemplo: `Mon, 15 Aug 2005 15:52:01 +0000`)

DATE_RFC3339 (string)

O mesmo que **DATE_ATOM**.

DATE_RFC3339_EXTENDED (string)

Formato RFC 3339 ESTENDIDO (exemplo: `2005-08-15T15:52:01.000+00:00`)

DATE_RSS (string)

RSS (exemplo: `Mon, 15 Aug 2005 15:52:01 +0000`). Sinônimo de **DATE_RFC1123**.

DATE_W3C (string)

World Wide Web Consortium (exemplo: `2005-08-15T15:52:01+00:00`). Sinônimo de **DATE_RFC3339**.

Examples

Índice

Aritmética de Data e Horário

Os exemplos a seguir mostram algumas armadilhas da aritmética de data/horário em relação a transições de horário de verão e meses que contêm números diferentes de dias.

Exemplo #1 DateTimeImmutable::add/sub adicionando intervalos que tempo decorrido

Adicionar PT24H sobre uma transição de horário de verão irá aparentar uma adição de 23/25 horas (para a maioria dos fusos horários).

```
<?php
$dt = new DateTimeImmutable("2015-11-01 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->add(new DateInterval("PT3H"));
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

O exemplo acima produzirá:

```
```

```
Início: 2015-11-01 00:00:00 -04:00
Fim:    2015-11-01 02:00:00 -05:00
```

Exemplo #2 DateTimeImmutable::modify e strtotime incrementando ou decrementando valores de componentes individuais

Inserir "+24 hours" sobre uma transição de horário de verão irá adicionar exatamente 24 horas como visto na string de data/horário (a menos que o horário de início ou fim esteja em um ponto de transição).

```php
<?php
$dt = new DateTimeImmutable("2015-11-01 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+24 hours");
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

O exemplo acima produzirá:

```
Início: 2015-11-01 00:00:00 -04:00
Fim:    2015-11-02 00:00:00 -05:00
```

Exemplo #3 Adicionar ou substrair períodos pode causar transferência de datas

Como em 31 de janeiro + 1 mês, irá resultar em 2 de março (em ano bissexto) ou 3 de março (em ano normal).

```php
<?php
echo "Ano normal:\n"; // fevereiro tem 28 dias
$dt = new DateTimeImmutable("2015-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+1 month");
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;

echo "Ano bissexto:\n"; // fevereiro tem 29 dias
$dt = new DateTimeImmutable("2016-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+1 month");
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

O exemplo acima produzirá:

```
Ano normal:
Início: 2015-01-31 00:00:00 -05:00
Fim:    2015-03-03 00:00:00 -05:00
Ano bissexto:
Início: 2016-01-31 00:00:00 -05:00
Fim:    2016-03-02 00:00:00 -05:00
```

Para obter o último dia do mês seguinte (para evitar a transferência), o formato `last day of` está disponível.

```php
<?php
echo "Ano normal:\n"; // fevereiro tem 28 dias
$dt = new DateTimeImmutable("2015-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("last day of next month");
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;

echo "Ano bissexto:\n"; // fevereiro tem 29 dias
$dt = new DateTimeImmutable("2016-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Início: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

```php
$dt = $dt->modify("last day of next month");
echo "Fim:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

O exemplo acima produzirá:

```
Ano normal:
Início: 2015-01-31 00:00:00 -05:00
Fim:    2015-02-28 00:00:00 -05:00
Ano bissexto:
Início: 2016-01-31 00:00:00 -05:00
Fim:    2016-02-29 00:00:00 -05:00
```

DateTime

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introdução

Representação de data e hora.

Esta classe se comporta da mesma maneira que a classe DateTimeImmutable, exceto pelo fato de que os objetos se alteram quando métodos de modificação como DateTime::modify() são chamados.

> **Aviso**
>
> Chamar métodos em objetos da classe **DateTime** irá alterar a informação encapsulada nestes objetos. Se a intenção for evitar isso, deve ser usado o operador `clone` para criar um novo objeto. É recomendado usar DateTimeImmutable no lugar de **DateTime** para obter este comportamento por padrão.

Resumo da classe

class **DateTime** implements DateTimeInterface {
/* Constantes herdadas */
public const string *DateTimeInterface::ATOM* = "Y-m-d\\TH:i:sP";
public const string *DateTimeInterface::COOKIE* = "l, d-M-Y H:i:s T";
public const string *DateTimeInterface::ISO8601* = "Y-m-d\\TH:i:sO";
public const string *DateTimeInterface::ISO8601_EXPANDED* = "X-m-d\\TH:i:sP";
public const string *DateTimeInterface::RFC822* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC850* = "l, d-M-y H:i:s T";
public const string *DateTimeInterface::RFC1036* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC1123* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::RFC7231* = "D, d M Y H:i:s \\G\\M\\T";
public const string *DateTimeInterface::RFC2822* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::RFC3339* = "Y-m-d\\TH:i:sP";
public const string *DateTimeInterface::RFC3339_EXTENDED* = "Y-m-d\\TH:i:s.vP";
public const string *DateTimeInterface::RSS* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::W3C* = "Y-m-d\\TH:i:sP";
/* Métodos */
public function __construct(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`)
public function add(DateInterval `$interval`): DateTime
public static function createFromFormat(string `$format`, string `$datetime`, ?DateTimeZone `$timezone` = `null`): DateTime|false
public static function createFromImmutable(DateTimeImmutable `$object`): static
public static function createFromInterface(DateTimeInterface `$object`): DateTime
public static function createFromTimestamp(int|float `$timestamp`): static
public function modify(string `$modifier`): DateTime
public static function __set_state(array `$array`): DateTime
public function setDate(int `$year`, int `$month`, int `$day`): DateTime
public function setISODate(int `$year`, int `$week`, int `$dayOfWeek` = 1): DateTime
public function setMicrosecond(int `$microsecond`): static

```
public function setTime(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
public function setTimestamp(int $timestamp): DateTime
public function setTimezone(DateTimeZone $timezone): DateTime
public function sub(DateInterval $interval): DateTime
public function diff(DateTimeInterface $targetObject, bool $absolute = false): DateInterval
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated]
public function __wakeup(): void
}
```

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.
7.2.0	As constantes da classe **DateTime** são agora definidas na classe DateTimeInterface.
7.1.0	O construtor da classe **DateTime** agora inclui os microssegundos atuais no valor construído. Antes disso, ele sempre inicializava os microssegundos em `0`.

Índice

DateTime::add

DateTime::add

date_add

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::add -- date_add — Modifica um objeto DateTime, com adição de dias, meses, anos, horas, minutos e segundos.

Descrição

Estilo orientado a objetos

public function **DateTime::add**(DateInterval `$interval`): DateTime

Estilo procedural

function date_add(DateTime `$object`, DateInterval `$interval`): DateTime

Adiciona o objeto DateInterval informado a um objeto DateTime especificado.

Equivalente a DateTimeImmutable::add() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`interval`

Um objeto DateInterval

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Veja também

- DateTimeImmutable::add() - Retorna um novo objeto, com adição de dias, meses, anos, horas, minutos e segundos

DateTime::__construct

DateTime::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::__construct — Retorna um novo objeto DateTime

Descrição

public function **DateTime::__construct**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`)

Equivalente a DateTimeImmutable::__construct() mas funciona com DateTime. Considere usar DateTimeImmutable e suas caracterísitcas.

Retorna um novo objeto DateTime.

Parâmetros

`datetime`

Uma string de data/hora. Os formatos válidos são explicados em Formatos de Data e Hora.

Coloque `"now"` neste parâmetro para obter o horário atual quando utilizar o parâmetro `timezone`.

`timezone`

Um objeto DateTimeZone, representando o fuso horário do parâmetro `datetime`.

Se `timezone` for `null` ou omitido, o fuso horário atual será utilizado.

> **Nota** :
>
> O parâmetro `timezone` e o fuso horário atual são ignorados quando o parâmetro `datetime` é um timestamp UNIX (exemplo, `@946684800`) ou quando o fuso horário é informado (exemplo. `2010-01-28T15:00:00+02:00`).

Valor Retornado

Retorna uma nova instância de DateTime.

Erros/Exceções

Se uma string inválida de data/horário for passada, uma exceção DateMalformedStringException é disparada. Antes do PHP 8.3, era uma Exception.

Registro de Alterações

Versão	Descrição
8.3.0	Agora dispara DateMalformedStringException se uma string inválida é passada, em vez de Exception.

Veja também

- DateTimeImmutable::__construct() - Retorna um novo objeto DateTimeImmutable

DateTime::createFromFormat

DateTime::createFromFormat

date_create_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::createFromFormat -- date_create_from_format — Interpreta um string de data/horário de acordo com um formato especificado.

Descrição

Estilo orientado a objetos

public static function **DateTime::createFromFormat**(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTime|false

Estilo procedural

function date_create_from_format(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTime|false

Retorna um novo objeto DateTime representando a data e o horário especificados pela string `datetime`, que foi formatada de acordo com o parâmetro `format` fornecido.

Equivalente a DateTimeImmutable::createFromFormat() e date_create_immutable_from_format(), respectivamente, mas cria um objeto DateTime.

Este método, incluindo parâmetros, exemplos e considerações estão documentados na página DateTimeImmutable::createFromFormat.

Parâmetros

Ver DateTimeImmutable::createFromFormat.

Valor Retornado

Retorna uma nova instância de DateTime ou `false` em caso de falha.

Erros/Exceções

Este método dispara uma exceção ValueError quando o parâmetro `datetime` contém bytes nulos.

Registro de Alterações

Versão	Descrição
8.0.21, 8.1.8, 8.2.0	Agora dispara ValueError quando bytes nulos são passados pelo parâmetro `datetime`, o que antes era silencioamente ignorado.

Examples

Para um conjunto extenso de exemplos, veja DateTimeImmutable::createFromFormat.

Veja também

- DateTimeImmutable::createFromFormat() - Interpreta um string de data/horário de acordo com o formato especificado

DateTime::createFromImmutable

DateTime::createFromImmutable

(PHP 7 >= 7.3.0, PHP 8)

DateTime::createFromImmutable — Retorna uma nova instância de DateTime encapsulando o objeto DateTimeImmutable informado

Descrição

public static function **DateTime::createFromImmutable**(DateTimeImmutable `$object`): static

Parâmetros

`object`

O objeto imutável DateTimeImmutable que necessita ser convertido para uma versão mutável. Este objeto não é modificado, em vez disso uma nova instância DateTime é criada contendo as mesmas informações de data, horário e fuso.

Valor Retornado

Retorna uma nova instância DateTime.

Registro de Alterações

Versão	Descrição
8.0.0	O método agora retorna uma instância da classe que está sendo chamada. Anteriormente, ele criava uma nova instância de DateTime.

Examples

Exemplo #1 Criando um objeto de data/horário mutável

```
<?php
$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$mutable = DateTime::createFromImmutable( $date );
```

DateTime::createFromInterface

DateTime::createFromInterface

(PHP 8)

DateTime::createFromInterface — Retorna um novo objeto DateTime encapsulando o objeto DateTimeInterface informado

Descrição

public static function **DateTime::createFromInterface**(DateTimeInterface `$object`): DateTime

Parâmetros

`object`

O objeto DateTimeInterface que necessita ser convertido para uma versão mutável. Este objeto não é modificado, em vez disso uma nova instância DateTime é criada contendo as mesmas informações de data, horário e fuso.

Valor Retornado

Retorna uma nova instância de DateTime.

Examples

Exemplo #1 Criando um objeto de data/horário mutável

```
<?php
$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$mutable = DateTime::createFromInterface($date);

$date = new DateTime("2014-06-20 11:45 Europe/London");
$also_mutable = DateTime::createFromInterface($date);
```

DateTime::createFromTimestamp

DateTime::createFromTimestamp

(PHP 8 >= 8.4.0)

DateTime::createFromTimestamp — Cria uma instância a partir de um timestamp Unix

Descrição

public static function **DateTime::createFromTimestamp**(int|float `$timestamp`): static

Cria uma instância a partir de um timestamp Unix.

Parâmetros

`timestamp`

Timestamp Unix representando a data/horário. Um valor float também é aceito, o que permite precisão de microssegundos.

Valor Retornado

Retorna uma nova instância de DateTime.

Erros/Exceções

Se `timestamp` estiver fora do intervalo [`PHP_INT_MIN`, `PHP_INT_MAX`], uma DateRangeError é lançada.

Examples

Exemplo #1 Exemplo de DateTime::createFromTimestamp()

```php
<?php
$date = DateTime::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u');
?>
```

O exemplo acima produzirá:

```
1970-01-01 00:02:03.456789
```

DateTime::getLastErrors

DateTime::getLastErrors

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::getLastErrors — Sinônimo de DateTimeImmutable::getLastErrors()

Descrição

Esta função é um apelido para: DateTimeImmutable::getLastErrors()

DateTime::modify

DateTime::modify

date_modify

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::modify -- date_modify — Altera o timestamp

Descrição

Estilo orientado a objetos

public function **DateTime::modify**(string `$modifier`): DateTime

Estilo procedural

function date_modify(DateTime `$object`, string `$modifier`): DateTime|false

Altera o timestamp de um objeto DateTime, incrementando-o ou decrementando-o, em um formato aceito pela função DateTimeImmutable::__construct().

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`modifier`

Uma string de data/hora. Os formatos válidos são explicados em Formatos de Data e Hora.

Valor Retornado

Retorna DateTime em caso de sucesso. Estilo procedural retorna `false` em caso de falha.

Erros/Exceções

Apenas para API orientada a objetos: Se uma string inválida de data/horário for passada, uma exceção DateMalformedStringException é lançada.

Registro de Alterações

Versão	Descrição
8.4.0	Agora tem o tipo de retorno provisório DateTime. Anteriormente era DateTime|false.
8.3.0	**DateTime::modify()** agora lança uma exceção DateMalformedStringException se uma string inválida for passada. Anteriormente, retornava `false` e um alerta era emitido. A função date_modify() não foi alterada.

Examples

Exemplo #1 Exemplo do método DateTime::modify()

Estilo orientado a objetos

```php
<?php
$date = new DateTime('2006-12-12');
$date->modify('+1 day');
echo $date->format('Y-m-d');
```

O exemplo acima produzirá:

```
2006-12-13
```

Estilo procedural

```php
<?php
$date = date_create('2006-12-12');
date_modify($date, '+1 day');
echo date_format($date, 'Y-m-d');
```

O exemplo acima produzirá:

```
2006-12-13
```

Exemplo #2 Cuidados na adição ou subtração de meses

```php
<?php
$date = new DateTime('2000-12-31');

$date->modify('+1 month');
echo $date->format('Y-m-d') . "\n";

$date->modify('+1 month');
echo $date->format('Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2001-01-31
2001-03-03
```

Exemplo #3 Todos os formatos de data e horário são suportados

```php
<?php
$date = new DateTime('2020-12-31');

$date->modify('July 1st, 2023');
echo $date->format('Y-m-d H:i') . "\n";

$date->modify('Monday next week');
echo $date->format('Y-m-d H:i') . "\n";

$date->modify('17:30');
echo $date->format('Y-m-d H:i') . "\n";
```

O exemplo acima produzirá:

```
2023-07-01 00:00
2023-07-03 00:00
2023-07-03 17:30
```

Veja também

- strtotime() - Interpreta qualquer descrição textual de data/horário em inglês para timestamp Unix
- DateTimeImmutable::modify() - Cria um novo objeto com seu timestamp modificado
- DateTime::add() - Modifica um objeto DateTime, com adição de dias, meses, anos, horas, minutos e segundos.
- DateTime::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos de um objeto DateTime
- DateTime::setDate() - Define a data
- DateTime::setISODate() - Define uma data ISO
- DateTime::setTime() - Define o horário
- DateTime::setTimestamp() - Define a data e hora baseada em um timestamp Unix

DateTime::__set_state

DateTime::__set_state

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::__set_state — O manipulador __set_state

Descrição

public static function **DateTime::__set_state**(array `$array`): DateTime

O manipulador __set_state().

Equivalente a DateTimeImmutable::__set_state() mas funciona com DateTime.

Parâmetros

`array`

Inicialização do array.

Valor Retornado

Retorna uma nova instância de um objeto DateTime.

DateTime::setDate

DateTime::setDate

date_date_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setDate -- date_date_set — Define a data

Descrição

Estilo orientado a objetos

public function **DateTime::setDate**(int `$year`, int `$month`, int `$day`): DateTime

Estilo procedural

```
function date_date_set(
    DateTime $object,
    int $year,
    int $month,
    int $day
): DateTime
```

Redefine a data atual do objeto Datetime para uma diferente.

Equivalente a DateTimeImmutable::setDate() mas funciona com DateTime, e modifica o objeto existente.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`year`

Ano da data.

`month`

Mês da data.

`day`

Dia da data.

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Veja também

- DateTimeImmutable::setDate() - Define a data

DateTime::setISODate

DateTime::setISODate

date_isodate_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setISODate -- date_isodate_set — Define uma data ISO

Descrição

Estilo orientado a objetos

public function **DateTime::setISODate**(int `$year`, int `$week`, int `$dayOfWeek` = 1): DateTime

Estilo procedural

```
function date_isodate_set(
    DateTime $object,
    int $year,
    int $week,
    int $dayOfWeek = 1
): DateTime
```

Define uma data, de acordo com o padrão ISO 8601 - utilizando a diferença de semanas e dias em vez de datas específicas.

Equivalente a DateTimeImmutable::setISODate() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`year`

Ano da data.

`week`

Semana da data.

`dayOfWeek`

Diferença do primeiro dia da semana.

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Veja também

- DateTimeImmutable::setISODate() - Define a data ISO

DateTime::setMicrosecond

DateTime::setMicrosecond

(PHP 8 >= 8.4.0)

DateTime::setMicrosecond — Define a parte de microssegundos do horário

Descrição

public function **DateTime::setMicrosecond**(int `$microsecond`): static

Define a parte de microssegundos do horário.

Funciona como em DateTimeImmutable::setMicrosecond(), porém com DateTime.

Parâmetros

`microsecond`

O valor de microssegundos a ser definido (`0` a `999999`).

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Erros/Exceções

Se `microsecond` estiver fora do intervalo [`0`, `999999`], uma DateRangeError é lançada.

Examples

Exemplo #1 Exemplo de DateTime::setMicrosecond()

```php
<?php
$date = DateTime::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
$date->setMicrosecond(987654);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
?>
```

O exemplo acima produzirá:

```
1970-01-01 00:02:03.456789
1970-01-01 00:02:03.987654
```

Veja também

- DateTimeInterface::getMicrosecond() - Obtém a parte de microssegundos do timestamp Unix

DateTime::setTime

DateTime::setTime

date_time_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setTime -- date_time_set — Define o horário

Descrição

Estilo orientado a objetos

```
public function DateTime::setTime(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
```

Estilo procedural

```
function date_time_set(
    DateTime $object,
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
```

Redefine o horário atual do objeto DateTime para um diferente.

Equivalente a DateTimeImmutable::setTime() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`hour`

Horas do horário.

`minute`

Minutos do horário.

`second`

Segundos do horário.

`microsecond`

Microssegundos do horário.

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Registro de Alterações

Versão	Descrição
8.1.0	O comportamento com duplicidade de horários existentes (durante um retorno de horário de verão) mudou. Anteriormente, o PHP pegaria a segunda ocorrência (depois do retorno do horário de verão), em vez da primeira ocorrência (antes do retorno do horário de verão).
7.1.0	O parâmetro `microsecond` foi adicionado.

Veja também

- DateTimeImmutable::setTime() - Define um horário

DateTime::setTimestamp

DateTime::setTimestamp

date_timestamp_set

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::setTimestamp -- date_timestamp_set — Define a data e hora baseada em um timestamp Unix

Descrição

Estilo orientado a objetos

public function **DateTime::setTimestamp**(int `$timestamp`): DateTime

Estilo procedural

function date_timestamp_set(DateTime `$object`, int `$timestamp`): DateTime

Define a data e hora baseada em um timestamp Unix.

Equivalente a DateTimeImmutable::setTimestamp() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`timestamp`

Timestam Unix representando a data. Configurar o timestamp fora dos limites de int é possível utilizando o método DateTimeImmutable::modify() com o formato `@`.

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Veja também

- DateTimeImmutable::setTimestamp() - Define a data e o horário com base em um timestamp Unix
- DateTime::setMicrosecond() - Define a parte de microssegundos do horário

DateTime::setTimezone

DateTime::setTimezone

date_timezone_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setTimezone -- date_timezone_set — Define o fuso horário de um objeto DateTime

Descrição

Estilo orientado a objetos

public function **DateTime::setTimezone**(DateTimeZone `$timezone`): DateTime

Estilo procedural

function date_timezone_set(DateTime `$object`, DateTimeZone `$timezone`): DateTime

Define um novo fuso horário a um objeto (object) DateTime.

Equivalente a DateTimeImmutable::setTimezone() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`timezone`

Um objeto DateTimeZone que representa o fuso horário desejado.

Valor Retornado

Retorna o objeto da classe DateTime para ligação de métodos. O ponto no tempo subjacente não é modificado quando este método é chamado.

Examples

Exemplo #1 Exemplo do método DateTime::setTimeZone()

Estilo orientado a objetos

```php
<?php
$date = new DateTime('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

$date->setTimezone(new DateTimeZone('Pacific/Chatham'));
echo $date->format('Y-m-d H:i:sP') . "\n";
```

O exemplo acima produzirá:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

Estilo procedural

```php
<?php
$date = date_create('2000-01-01', timezone_open('Pacific/Nauru'));
echo date_format($date, 'Y-m-d H:i:sP') . "\n";

date_timezone_set($date, timezone_open('Pacific/Chatham'));
echo date_format($date, 'Y-m-d H:i:sP') . "\n";
```

O exemplo acima produzirá:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

Veja também

- DateTimeImmutable::setTimezone() - Define o fuso horário
- DateTime::getTimezone() - Retorna o fuso horário correspondente a um DateTime informado
- DateTimeZone::__construct() - Cria um novo objeto DateTimeZone

DateTime::sub

DateTime::sub

date_sub

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::sub -- date_sub — Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos de um objeto DateTime

Descrição

Estilo orientado a objetos

public function **DateTime::sub**(DateInterval `$interval`): DateTime

Estilo procedural

function date_sub(DateTime `$object`, DateInterval `$interval`): DateTime

Modifica o objeto DateTime especificado, subtraindo o objeto DateInterval especificado.

Equivalente a DateTimeImmutable::sub() mas funciona com DateTime.

A versão procedural usa o objeto DateTime como seu primeiro argumento.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`interval`

Um objeto DateInterval

Valor Retornado

Retorna o objeto DateTime modificado para encadeamento de métodos.

Erros/Exceções

Apenas para API orientada a objetos: Se uma operação não suportada for realizada, como o uso de um objeto DateInterval representando especificações de tempo relativo tipo `next weekday`, uma exceção DateInvalidOperationException é lançada.

Registro de Alterações

Versão	Descrição
8.3.0	Agora lança uma exceção DateInvalidOperationException com **DateTime::sub()**, em vez de um alerta quando uma operação não suportada é realizada. A função date_sub() não foi alterada.

Veja também

- DateTimeImmutable::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos

DateTimeImmutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introdução

Representação de data e hora.

Esta classe se comporta como a classe DateTime exceto pelo fato de que novos objetos são retornados quando métodos de modificação como DateTime::modify() são chamados.

Resumo da classe

```
class DateTimeImmutable implements DateTimeInterface {
/* Constantes herdadas */
public const string DateTimeInterface::ATOM = "Y-m-d\\TH:i:sP";
public const string DateTimeInterface::COOKIE = "l, d-M-Y H:i:s T";
public const string DateTimeInterface::ISO8601 = "Y-m-d\\TH:i:sO";
public const string DateTimeInterface::ISO8601_EXPANDED = "X-m-d\\TH:i:sP";
public const string DateTimeInterface::RFC822 = "D, d M y H:i:s O";
public const string DateTimeInterface::RFC850 = "l, d-M-y H:i:s T";
public const string DateTimeInterface::RFC1036 = "D, d M y H:i:s O";
public const string DateTimeInterface::RFC1123 = "D, d M Y H:i:s O";
public const string DateTimeInterface::RFC7231 = "D, d M Y H:i:s \\G\\M\\T";
public const string DateTimeInterface::RFC2822 = "D, d M Y H:i:s O";
public const string DateTimeInterface::RFC3339 = "Y-m-d\\TH:i:sP";
public const string DateTimeInterface::RFC3339_EXTENDED = "Y-m-d\\TH:i:s.vP";
public const string DateTimeInterface::RSS = "D, d M Y H:i:s O";
public const string DateTimeInterface::W3C = "Y-m-d\\TH:i:sP";
/* Métodos */
public function __construct(string $datetime = "now", ?DateTimeZone $timezone = null)
#[\NoDiscard(message: "as DateTimeImmutable::add() does not modify the object itself")]
public function add(DateInterval $interval): DateTimeImmutable
public static function createFromFormat(string $format, string $datetime, ?DateTimeZone $timezone = null): DateTimeImmutable|false
public static function createFromInterface(DateTimeInterface $object): DateTimeImmutable
public static function createFromMutable(DateTime $object): static
public static function createFromTimestamp(int|float $timestamp): static
public static function getLastErrors(): array|false
#[\NoDiscard(message: "as DateTimeImmutable::modify() does not modify the object itself")]
public function modify(string $modifier): DateTimeImmutable
public static function __set_state(array $array): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setDate() does not modify the object itself")]
public function setDate(int $year, int $month, int $day): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setISODate() does not modify the object itself")]
public function setISODate(int $year, int $week, int $dayOfWeek = 1): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setMicrosecond() does not modify the object itself")]
public function setMicrosecond(int $microsecond): static
#[\NoDiscard(message: "as DateTimeImmutable::setTime() does not modify the object itself")]
public function setTime(
    int $hour,
    int $minute,
```

```
    int $second = 0,
    int $microsecond = 0
): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setTimestamp() does not modify the object itself")]
public function setTimestamp(int $timestamp): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setTimezone() does not modify the object itself")]
public function setTimezone(DateTimeZone $timezone): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::sub() does not modify the object itself")]
public function sub(DateInterval $interval): DateTimeImmutable
public function diff(DateTimeInterface $targetObject, bool $absolute = false): DateInterval
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated]
public function __wakeup(): void
}
```

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.
7.1.0	O construtor da classe **DateTimeImmutable** agora inclui os microssegundos atuais no valor construído. Antes disso, ele iria sempre inicializar os microssegundos em `0`.

Índice

DateTimeImmutable::add

DateTimeImmutable::add

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::add — Retorna um novo objeto, com adição de dias, meses, anos, horas, minutos e segundos

Descrição

```
#[\NoDiscard(message: "as DateTimeImmutable::add() does not modify the object itself")]
public function DateTimeImmutable::add(DateInterval $interval): DateTimeImmutable
```

Cria um novo objeto DateTimeImmutable, e adiciona o objeto DateInterval a ele, para representar o novo valor.

Parâmetros

`interval`

Um objeto DateInterval

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::add()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('P10D'));
echo $newDate->format('Y-m-d') . "\n";
?>
```

Exemplo #2 Mais exemplos da função DateTimeImmutable::add()

```php
<?php
$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('PT10H30S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";

$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('P7Y5M4DT4H3M2S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

O exemplo acima produzirá:

```
2000-01-01 10:00:30
2007-06-05 04:03:02
```

Exemplo #3 Cuidado ao adicionar meses

```php
<?php
$date = new DateTimeImmutable('2000-12-31');
$interval = new DateInterval('P1M');

$newDate1 = $date->add($interval);
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->add($interval);
echo $newDate2->format('Y-m-d') . "\n";
?>
```

O exemplo acima produzirá:

```
2001-01-31
2001-03-03
```

Veja também

- DateTimeImmutable::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos
- DateTimeImmutable::diff() - Retorna a diferença entre dois objetos DateTime
- DateTimeImmutable::modify() - Cria um novo objeto com seu timestamp modificado

DateTimeImmutable::__construct

DateTimeImmutable::__construct

date_create_immutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::__construct -- date_create_immutable — Retorna um novo objeto DateTimeImmutable

Descrição

Estilo orientado a objetos

public function **DateTimeImmutable::__construct**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`)

Estilo procedural

function date_create_immutable(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTimeImmutable|false

Retorna um novo objeto DateTimeImmutable.

Parâmetros

`datetime`

Uma string de data/hora. Os formatos válidos são explicados em Formatos de Data e Hora.

Utilize `"now"` para obter o horário atual quando usar o parâmetro`timezone`.

`timezone`

Um objeto DateTimeZone representando o fuso horário do parâmetro `datetime`.

Se `timezone` for `null` ou omitido, o fuso horário atual será utilizado.

> **Nota** :
>
> O parâmetro `timezone` e o fuso horário atual são ignorados quando o parâmetro `datetime` for um timestamp UNIX (ex.: `@946684800`) ou especificar um fuso horário (ex.: `2010-01-28T15:00:00+02:00` ou `2010-07-05T06:00:00Z`).

Valor Retornado

Retorna uma nova instância de DateTimeImmutable.

Erros/Exceções

Se um string inválido de data/horário for passado, uma exceção DateMalformedStringException é disparada. Antes do PHP 8.3, era Exception.

Registro de Alterações

Versão	Descrição
8.3.0	Agora dispara DateMalformedStringException se uma string inválida for passada, em vez de Exception.
7.1.0	De agora em diante os microssegundos são preenchidos com o valor real. Não mais com '00000'.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::__construct()

Estilo orientado a objetos

```php
<?php
try {
    $date = new DateTimeImmutable('2000-01-01');
} catch (Exception $e) {
    echo $e->getMessage();
    exit(1);
}

echo $date->format('Y-m-d');
```

O exemplo acima produzirá:

```
2000-01-01
```

Estilo procedural

```php
<?php
$date = date_create('2000-01-01');
if (!$date) {
    $e = date_get_last_errors();
    foreach ($e['errors'] as $error) {
        echo "$error\n";
    }
    exit(1);
}

echo date_format($date, 'Y-m-d');
```

O exemplo acima produzirá:

```
2000-01-01
```

Exemplo #2 Complexidades de DateTimeImmutable::__construct()

```php
<?php
date_default_timezone_set('America/Jamaica');

// Data/horário especificados no fuso horário padrão do PHP.
$date = new DateTimeImmutable('2000-01-01');
echo $date->format('Y-m-d H:i:sP') . "\n";

// Data/horário especificados no fuso horário informado.
$date = new DateTimeImmutable('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

// Data/horário atual no fuso horário padrão do PHP.
$date = new DateTimeImmutable();
echo $date->format('Y-m-d H:i:sP') . "\n";

// Data/horário atual no fuso horário informado.
$date = new DateTimeImmutable('now', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

// Usando um timestamp UNIX. Note que o resultado é no fuso horário UTC.
$date = new DateTimeImmutable('@946684800');
echo $date->format('Y-m-d H:i:sP') . "\n";

// Valores não existentes são transferidos.
$date = new DateTimeImmutable('2000-02-30');
echo $date->format('Y-m-d H:i:sP') . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
2000-01-01 00:00:00-05:00
2000-01-01 00:00:00+12:00
2010-04-24 10:24:16-04:00
2010-04-25 02:24:16+12:00
2000-01-01 00:00:00+00:00
2000-03-01 00:00:00-05:00
```

> **Nota** :
>
> Datas transferidas podem ser detectadas através da verificação de avisos usando DateTimeImmutable::getLastErrors().

Exemplo #3 Mudando o fuso horário associado

```php
<?php
$timeZone = new \DateTimeZone('Asia/Tokyo');

$time = new \DateTimeImmutable();
$time = $time->setTimezone($timeZone);

echo $time->format('Y/m/d H:i:s e'), "\n";
```

O exemplo acima produzirá algo semelhante a:

```
2022/08/12 23:49:23 Asia/Tokyo
```

Exemplo #4 Usando uma string relativo de data/horário

```php
<?php
$time = new \DateTimeImmutable("-1 year");

echo $time->format('Y/m/d H:i:s'), "\n";
```

O exemplo acima produzirá algo semelhante a:

```
2021/08/12 15:43:51
```

DateTimeImmutable::createFromFormat

DateTimeImmutable::createFromFormat

date_create_immutable_from_format

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::createFromFormat -- date_create_immutable_from_format — Interpreta um string de data/horário de acordo com o formato especificado

Descrição

Estilo orientado a objetos

public static function **DateTimeImmutable::createFromFormat**(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTimeImmutable|false

Estilo procedural

function date_create_immutable_from_format(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTimeImmutable|false

Retorna um novo objeto DateTimeImmutable representando a data e o horário especificados pelo string `datetime`, formatado pelo parâmetro `format`.

Parâmetros

`format`

> O formato no qual a string passada deve estar. Veja as opções de formatação abaixo. Na maioria dos casos, as mesmas letras da função date() podem ser usadas.

Todos os campos são inicializados com a data/horário atual. Na maioria das vezes, a intenção pode ser "zerá-los" (época Unix, `1970-01-01 00:00:00 UTC`). Isto pode ser feito incluindo o caractere `!` como o primeiro no parâmetro `format`, ou `|` como o último. Favor verificar a documentação para cada caractere abaixo para mais informação.

O formato é analisado da esquerda para a direita, o que significa que em algumas situações a ordem na qual os caracteres de formato estão presents afeta o resultado. No caso de `z` (o dia do ano), é necessário que um ano já tenha sido analisado, por exemplo através dos caracteres `Y` ou `y`.

Letras que são usadas para representar números permitem uma ampla gama de valores, fora do que seria a faixa lógica. Por exemplo, a letra `d` (dia do mês) aceita valores na faixa de `00` a `99`. A única restrição é o número de dígitos. O mecanismo de overflow do analisador é usado quando valores fora da faixa são especificados. Os exemplos abaixo mostram alguns destes comportamentos.

Isto também significa que os dados analisados para um caractere de formatação são ambiciosos, e irão ler até a quantidade de dígitos que o formato permite. Isto pode também então significar que não há mais caracteres suficientes na string do parâmetro `datetime` para os caracteres de formato seguintes. Um exemplo nesta página também ilustra este problema.

Os seguintes caracteres são reconhecidos na string do parâmetros `format`

Caractere de `format`	Descrição	Exemplo de valores analisáveis
Dia	---	---
`d` e `j`	Dia do mês, 2 dígitos com ou sem zeros na frente	`01` a `31` ou `1` a `31`. (números de 2 dígitos maiores queo número de dias no mês são aceitos, e neste caso irão causar uma transferência no mês. Por exemplo, usar 33 com janeiro, significa 2 de fevereiro)
`D` e `l`	Uma representação textual de um dia	`Mon` a `Sun` ou `Sunday` a `Saturday`. Se o nome do dia informado for diferente do nome do dia pertencente a uma data analisada (ou data padrão), então uma transferência ocorre para a *próxima* data com o nome do dia informado. Veja os exemplos abaixo para uma explicação.
`S`	Sufixo em inglês para o ordinal do dia do mês, 2 caracteres. É ignorado durante o processamento.	`st`, `nd`, `rd` ou `th`.
`z`	O dia do ano (iniciando em 0); deve ser precedido de `Y` ou `y`.	`0` a `365`. (números de 3 dígitos maiores que o número de dias em um ano são aceitos, e neste caso case irão causar uma transferência de ano. Por exemplo, usar 366 com 2022, significa 2 de janeiro de 2023)
Mês	---	---
`F` e `M`	Uma representação textual do mês, como January ou Sept	`January` a `December` ou `Jan` a `Dec`
`m` e `n`	Representação numérica de um mês, com ou sem zeros na frente	`01` a `12` ou `1` a `12`. (números de 2 dígitos maiores que 12 são aceitos, e neste caso eles irão causar uma transferência de ano. Por exemplo, usar 13 significa janeiro do ano seguinte)
Ano	---	---
`X` e `x`	Uma representação numérica completa de um ano, *com até* 19 dígitos, opcionalmente prefixada por `+` ou `-`	Exemplos: `0055`, `787`, `1999`, `-2003`, `+10191`
`Y`	Uma representação numérica completa de um ano, *com até* 4 dígitos	Exemplos: `25` (o mesmo que `0025`), `787`, `1999`, `2003`
`y`	Uma representação de dois dígitos de um ano (que assume-se estar na faixa 1970-2069, inclusive)	Exemplos: `99` ou `03` (que será interpretado como `1999` e `2003`, respectivamente)
Horário	---	---
`a` e `A`	Ante meridiem e Post meridiem	`am` ou `pm`

Caractere de `format`	Descrição	Exemplo de valores analisáveis
`g` e `h`	Formato de 12 horas para uma hora com ou sem zeros na frente	`1` a `12` ou `01` a `12` (números de 2 dígitos maiores que 12 são aceitos, e neste caso eles irão causar uma transferência de período AM/PM. Por exemplo, usar `14` significa `02` no próximo período AM/PM)
`G` e `H`	Formato de 24 horas para uma hora com ou sem zeros na frente	`0` a `23` ou `00` a `23` (números de 2 dígitos maiores que 24 são aceitos, e neste caso eles irão causar uma transferência de dia. Por exemplo, usar `26` significa `02:00` do dia seguinte)
`i`	Minutos com zero na frente	`00` a `59`. (números de 2 dígitos maiores que 59 são aceitos, e neste caso irão causar uma transferência de hora. Por exemplo, usar `66` significa `:06` da hora seguinte)
`s`	Segundos, com zero na frente	`00` a `59` (números de 2 dígitos maiores que 59 são aceitos, e neste caso irão causar uma transferência de minuto. Por exemplo, usar `90` significa `:30` do próximo minuto)
`v`	Fração em milissegundos (até três dígitos)	Exemplo: `12` (`0.12` segundo), `345` (`0.345` segundo)
`u`	Fração em microssegundos (até seis dígitos)	Exemplo: `45` (`0.45` segundo) `654321` (`0.654321` segundo)
Fuso horário	---	---
`e`, `O`, `p`, `P` e `T`	Identificador do fuso horário, ou a diferença para UTC em horas, ou a diferença para UTC com dois pontos entre horas e minutos, ou abreviação do fuso horário	Exemplos: `UTC`, `GMT`, `Atlantic/Azores` ou `+0200` ou `+02:00` ou `EST`, `MDT`
Data/Horário completos	---	---
`U`	Segundos desde a Época Unix (1 janeiro 1970 00:00:00 GMT)	Exemplo: `1292177455`
Espaços em branco e separadores	---	---
(espaço)	Zero ou mais espaços, tabulações, caracteres NBSP (U+A0), ou NNBSP (U+202F)	Exemplo: `"\t"`, `" "`
`#`	Um dos símbolos de separação: `;`, `:`, `/`, `.`, `,`, `-`, `(` ou `)`	Exemplo: `/`
`;`, `:`, `/`, `.`, `,`, `-`, `(` ou `)`	O caractere especificado.	Exemplo: `-`
`?`	Um byte aleatório	Exemplo: `^` (Esteja ciente que para caracteres UTF-8 pode ser necessário mais de um `?`. Neste caso, usar `*` pode ser o o mais adequado na verdade)
`*`	Bytes aleatórios até o próximo separador ou dígito	Exemplo: `*` em `Y-*-d` com o string `2009-aWord-08` será equivalente a `aWord`
`!`	Reconfigura todos os campos (ano, mês, dia, hora, minuto, segundo, fração e informação de fuso horário) com valores tipo zero (`0` para hora, minuto, segundo e fração, `1` para mês e dia, `1970` para ano e o fuso horário padrão)	Sem `!`, todos os campos serão configurados com a data/horário atual.
`|`	Reconfigura todos os campos (ano, mês, dia, hora, minuto, segundo, fração e informação de fuso horário) com valores tipo zero se eles ainda não foram analisados	`Y-m-d|` irá configurar o ano, mês e dia para as informações encontradas na string a ser analisada, e configura a hora, minuto e segundo para 0.

Caractere de `format`	Descrição	Exemplo de valores analisáveis
`+`	Se esta especificação de formato estiver presente, dados no final da string não irão causar um erro, mas sim um aviso	Use DateTimeImmutable::getLastErrors() para descobrir se dados no final da string estavam presentes.

Caracteres não reconhecidos na string de formatação irão causar falha na análise e uma mensagem de erro será anexada na estrutura retornada. É possível recuperar a mensagem de erro com DateTimeImmutable::getLastErrors().

Para incluir caracteres literais no parâmetro `format`, deve-se usar a barra invertida (\) antes deles.

Se `format` não contiver o caractere `!` então partes da data/horário gerada que não foram especificadas em `format` serão configuradas para a data/horário atual do sistema.

Se `format` contiver o caractere `!`, então partes da data/horário gerada não especificada em `format`, assim como valores à esquerda de `!`, serão configuradas aos valores correspondentes da época Unix.

Se algum caractere de horário for analisado, então todos os outros campos relacionados a horário são configurados para "0", a menos que também tenham sido analisados.

A Época Unix é 1970-01-01 00:00:00 UTC.

`datetime`

String representando a data/horário.

`timezone`

Um objeto DateTimeZone representando o fuso horário desejado.

Se `timezone` for **`null`** ou omitido e `datetime` não contém o fuso horário, o fuso atual será usado.

> **Nota** :
>
> O parâmetro `timezone` e o fuso horário atual são ignorados quando o parâmetro `datetime` contém um timestamp UNIX (ex.: `946684800`) ou especifica um fuso horário (ex.: `2010-01-28T15:00:00+02:00`).

Valor Retornado

Retorna uma nova instância de DateTimeImmutable ou **`false`** em caso de falha.

Erros/Exceções

Este método lança uma exceção ValueError quando o parâmetro `datetime` contiver bytes nulos.

Registro de Alterações

Versão	Descrição
8.2.9	O especificador (space) agora também suporta caracteres NBSP (U+A0) e NNBSP (U+202F).
8.2.0	Os especificadores `X` e `x` de `format` foram adicionados.
8.0.21, 8.1.8, 8.2.0	Agora lança a exceção ValueError quando bytes nulos são passados no parâmetro `datetime`, o que antes era silenciosamente ignorado.
7.3.0	O especificador `v` em `format` foi adicionado.

Examples

Exemplo #1 Exemplo de **DateTimeImmutable::createFromFormat()**

Estilo orientado a objetos

```php
<?php
$date = DateTimeImmutable::createFromFormat('j-M-Y', '15-Feb-2009');
echo $date->format('Y-m-d');
```

Exemplo #2 Usando constantes predefinidas de formato com DateTimeImmutable::createFromFormat()

Estilo orientado a objetos

```php
<?php
$date = DateTimeImmutable::createFromFormat(
    DateTimeInterface::ISO8601,
    '2004-02-12T15:19:21+00:00'
);
echo $date->format('c e') . "\n";

$date = DateTimeImmutable::createFromFormat(
    DateTimeInterface::RFC3339_EXTENDED,
    '2013-10-14T09:00:00.000+02:00'
);
echo $date->format('c e') . "\n";
```

As constantes de formatação como usadas neste exemplo consistem de uma string de caracteres para formatar um objeto DateTimeImmutable. Na maioria dos casos, estas letras são equivalentes aos elementos de data/horário definidos na seção parameters acima, mas elas tendem a ser mais lenientes.

Exemplo #3 Complexidades de DateTimeImmutable::createFromFormat()

```php
<?php
echo 'Horário atual: ' . date('Y-m-d H:i:s') . "\n";

$format = 'Y-m-d';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15');
echo "Formato: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'Y-m-d H:i:s';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15 15:16:17');
echo "Formato: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'Y-m-!d H:i:s';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15 15:16:17');
echo "Formato: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = '!d';
$date = DateTimeImmutable::createFromFormat($format, '15');
echo "Formato: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'i';
$date = DateTimeImmutable::createFromFormat($format, '15');
echo "Formato: $format; " . $date->format('Y-m-d H:i:s') . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
Horário atual: 2022-06-02 15:50:46
Formato: Y-m-d; 2009-02-15 15:50:46
Formato: Y-m-d H:i:s; 2009-02-15 15:16:17
Formato: Y-m-!d H:i:s; 1970-01-15 15:16:17
Formato: !d; 1970-01-15 00:00:00
Formato: i; 2022-06-02 00:15:00
```

Exemplo #4 String de formatação com caracteres literais

```php
<?php
echo DateTimeImmutable::createFromFormat('H\h i\m s\s','23h 15m 03s')->format('H:i:s');
```

O exemplo acima produzirá algo semelhante a:

```
23:15:03
```

Exemplo #5 Comportamento de transferência

```php
<?php
echo DateTimeImmutable::createFromFormat('Y-m-d H:i:s', '2021-17-35 16:60:97')->format(DateTimeImmutable::RFC2822);
```

O exemplo acima produzirá algo semelhante a:

```
Sat, 04 Jun 2022 17:01:37 +0000
```

Embora o resultado pareça estranho, ele está correto, já que as transferências abaixo ocorrem:

1. `97` segundos transferem para `1` minuto, sobrando `37` segundos.
2. `61` minutos transferem para `1` hora, sobrando `1` minuto.
3. `35` dias transferem para `1` mês, sobrando `4` dias. A quantidade de dias que sobram depende do mês, já que nem todo mês tem o mesmo número de dias.
4. `18` meses transferem para `1` ano, sobrando `6` meses.

Exemplo #6 Comportamento de transferência do nome do dia

```php
<?php
$d = DateTime::createFromFormat(DateTimeInterface::RFC1123, 'Mon, 3 Aug 2020 25:00:00 +0000');
echo $d->format(DateTime::RFC1123), "\n";
```

O exemplo acima produzirá algo semelhante a:

```
Mon, 10 Aug 2020 01:00:00 +0000
```

Embora o resultado pareça estranho, ele está correto, já que as transferências abaixo ocorrem:

1. `3 Aug 2020 25:00:00` transfere para `(Tue) 4 Aug 2020 01:00`.
2. `Mon` é aplicado, o que avança a data para `Mon, 10 Aug 2020 01:00:00`. A explicação para as palavras-chaves relativas como `Mon` está disponível na seção sobre formatos relativos.

Para detectar transferências em datas/horas, pode-se usar o método DateTimeImmutable::getLastErrors(), que irá incluir um aviso se uma transferência tiver ocorrido.

Exemplo #7 Detectando transferência de datas/horas

```php
<?php
$d = DateTimeImmutable::createFromFormat('Y-m-d H:i:s', '2021-17-35 16:60:97');
echo $d->format(DateTimeImmutable::RFC2822), "\n\n";

var_dump(DateTimeImmutable::getLastErrors());
```

O exemplo acima produzirá algo semelhante a:

```
Sat, 04 Jun 2022 17:01:37 +0000

array(4) {
  'warning_count' =>
  int(2)
  'warnings' =>
  array(1) {
    [19] =>
    string(27) "The parsed date was invalid"
  }
  'error_count' =>
  int(0)
  'errors' =>
  array(0) {
  }
}
```

Exemplo #8 Comportamento de análise ambicioso

```php
<?php
print_r(date_parse_from_format('Gis', '60101'));
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [year] =>
    [month] =>
    [day] =>
    [hour] => 60
    [minute] => 10
    [second] => 0
    [fraction] => 0
    [warning_count] => 1
    [warnings] => Array
        (
            [5] => The parsed time was invalid
        )

    [error_count] => 1
    [errors] => Array
        (
            [4] => A two digit second could not be found
        )

    [is_localtime] =>
)
```

O formato `G` analisa horas no formato de 24 horas, com ou sem zeros na frente. Isto requer analisar 1 ou 2 dígitos. Como há dois dígitos disponíveis, o analisador ambiciosamente lê o valor como `60`.

Os caracteres seguintes `i` e `s` ambos requerem dois dígitos. Isto significa que `10` é passado como minutos (`i`), e que não há então dígitos suficientes restantes para serem analisados como segundos (`s`).

O array `errors` indica este problema.

Adicionalmente, um valor de hora de `60` está fora da faixa `0-24`, o que faz com que o array de `warnings` inclua um aviso de que o horário é inválido.

Veja também

- DateTimeImmutable::__construct() - Retorna um novo objeto DateTimeImmutable
- DateTimeImmutable::getLastErrors() - Retorna os avisos e erros
- checkdate() - Valida uma data gregoriana
- strptime() - Interpreta uma data/horário gerada pela função strftime

DateTimeImmutable::createFromInterface

DateTimeImmutable::createFromInterface

(PHP 8)

DateTimeImmutable::createFromInterface — Retorna um novo objeto DateTimeImmutable encapsulando o objeto DateTimeInterface informado

Descrição

public static function **DateTimeImmutable::createFromInterface**(DateTimeInterface `$object`): DateTimeImmutable

Parâmetros

`object`

O objeto DateTimeInterface que necessita ser convertido para uma versão imutável. Este objeto não é modificado, em vez disso uma nova instância DateTimeImmutable é criada contendo as mesmas informações de data, horário e fuso.

Valor Retornado

Retorna uma nova instância de DateTimeImmutable.

Examples

Exemplo #1 Criando um objeto de data/horário imutável

```php
<?php
$date = new DateTime("2014-06-20 11:45 Europe/London");
$immutable = DateTimeImmutable::createFromInterface($date);

$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$also_immutable = DateTimeImmutable::createFromInterface($date);
```

DateTimeImmutable::createFromMutable

DateTimeImmutable::createFromMutable

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

DateTimeImmutable::createFromMutable — Retorna uma nova instância DateTimeImmutable que encapsula um objeto DateTime informado

Descrição

public static function **DateTimeImmutable::createFromMutable**(DateTime `$object`): static

Parâmetros

`object`

O objeto mutável DateTime que se deseja converter para uma versão imutável. Este objeto não é modificado, mas em vez disso uma nova instância de DateTimeImmutable é criado contendo a mesma data/horário e informação de fuso horário.

Valor Retornado

Retorna uma nova instância de DateTimeImmutable.

Registro de Alterações

Versão	Descrição
8.0.0	O método agora retorna uma instância da classe atualmente chamada. Antes, ele criava um nova instância de DateTimeImmutable.

Examples

Exemplo #1 Criando um objeto DateTime imutável

```
<?php
$date = new DateTime("2014-06-20 11:45 Europe/London");
$immutable = DateTimeImmutable::createFromMutable( $date );
```

DateTimeImmutable::createFromTimestamp

DateTimeImmutable::createFromTimestamp

(PHP 8 >= 8.4.0)

DateTimeImmutable::createFromTimestamp — Cria uma instância a partir de um timestamp Unix

Descrição

public static function **DateTimeImmutable::createFromTimestamp**(int|float `$timestamp`): static

Cria uma instância a partir de um timestamp Unix.

Parâmetros

`timestamp`

Timestamp Unix representando a data/horário. Um valor float também é aceito, o que permite precisão de microssegundos.

Valor Retornado

Retorna uma nova instância de DateTimeImmutable.

Erros/Exceções

Se `timestamp` estiver fora do intervalo [**PHP_INT_MIN**, **PHP_INT_MAX**], uma DateRangeError é lançada.

Examples

Exemplo #1 Exemplo de DateTimeImmutable::createFromTimestamp()

```
<?php
$date = DateTimeImmutable::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u');
?>
```

O exemplo acima produzirá:

```
1970-01-01 00:02:03.456789
```

DateTimeImmutable::getLastErrors

DateTimeImmutable::getLastErrors

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::getLastErrors — Retorna os avisos e erros

Descrição

public static function **DateTimeImmutable::getLastErrors**(): array|false

Retorna um array de avisos e erros encontrados durante a análise de uma string de data/horário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array contendo informação sobre avisos e erros ou `false` se não houver nem avisos nem erros.

Registro de Alterações

Versão	Descrição
8.2.0	Antes do PHP 8.2.0, esta função não retornava `false` quando não havia warnings ou erros. Em vez disso, ela sempre retornaria a estrutura de array documentada.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::getLastErrors()

```php
<?php
try {
    $date = new DateTimeImmutable('asdfasdf');
} catch (Exception $e) {
    // Apenas para propósito de demonstração...
    print_r(DateTimeImmutable::getLastErrors());

    // A maneira verdadeiramente real orientada a objeto de se fazer isto é
    echo $e->getMessage();
}
?>
```

O exemplo acima produzirá:

```
Array
(
    [warning_count] => 1
    [warnings] => Array
        (
            [6] => Double timezone specification
        )

    [error_count] => 1
    [errors] => Array
        (
            [0] => The timezone could not be found in the database
        )
)
Failed to parse time string (asdfasdf) at position 0 (a): The timezone could not be found in the database
```

Os índices 6 e 0 na saída do exemplo referem-se ao índice do caractere no string onde o erro ocorreu.

Exemplo #2 Detectando datas transferidas

```php
<?php
$date = DateTimeImmutable::createFromFormat('!Y-m-d', '2020-02-30');
print_r(DateTimeImmutable::getLastErrors());
```

O exemplo acima produzirá:

```
Array
(
    [warning_count] => 1
    [warnings] => Array
        (
```

```
            [10] => The parsed date was invalid
        )

    [error_count] => 0
    [errors] => Array
        (
        )
)
```

DateTimeImmutable::modify

DateTimeImmutable::modify

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::modify — Cria um novo objeto com seu timestamp modificado

Descrição

#[\NoDiscard(`message`: "as DateTimeImmutable::modify() does not modify the object itself")]
public function **DateTimeImmutable::modify**(string `$modifier`): DateTimeImmutable

Cria um novo objeto DateTimeImmutable com seu timestamp modificado. O objeto original não é modificado.

Parâmetros

`modifier`

Uma string de data/hora. Os formatos válidos são explicados em Formatos de Data e Hora.

Valor Retornado

Retorna DateTimeImmutable em caso de sucesso. Estilo procedural retorna **`false`** em caso de falha.

Erros/Exceções

Se uma string inválida de data/horário for passada, uma exceção DateMalformedStringException é disparada. Antes do PHP 8.3, era um aviso.

Registro de Alterações

Versão	Descrição
8.4.0	Agora tem um tipo de retorno provisório DateTimeImmutable. Anteriormente era DateTimeImmutable|false.
8.3.0	**DateTimeImmutable::modify()** agora lança uma exceção DateMalformedStringException se uma string inválida for passada. Anteriormente, retornava **`false`** e um alerta era emitido.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::modify()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2006-12-12');
$newDate = $date->modify('+1 day');
echo $newDate->format('Y-m-d');
```

O exemplo acima produzirá:

```
2006-12-13
```

Exemplo #2 Cuidado na adição ou subtração de meses

```php
<?php
$date = new DateTimeImmutable('2000-12-31');

$newDate1 = $date->modify('+1 month');
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->modify('+1 month');
echo $newDate2->format('Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2001-01-31
2001-03-03
```

Veja também

- DateTimeImmutable::add() - Retorna um novo objeto, com adição de dias, meses, anos, horas, minutos e segundos
- DateTimeImmutable::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos
- DateTimeImmutable::setDate() - Define a data
- DateTimeImmutable::setISODate() - Define a data ISO
- DateTimeImmutable::setTime() - Define um horário
- DateTimeImmutable::setTimestamp() - Define a data e o horário com base em um timestamp Unix

DateTimeImmutable::__set_state

DateTimeImmutable::__set_state

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::__set_state — O manipulador __set_state

Descrição

public static function **DateTimeImmutable::__set_state**(array `$array`): DateTimeImmutable

O manipulador __set_state().

Parâmetros

`array`

Array de inicialização.

Valor Retornado

Retorna uma nova instância de um objeto DateTimeImmutable.

DateTimeImmutable::setDate

DateTimeImmutable::setDate

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setDate — Define a data

Descrição

#[\NoDiscard(`message`: "as DateTimeImmutable::setDate() does not modify the object itself")]
public function **DateTimeImmutable::setDate**(int `$year`, int `$month`, int `$day`): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable com a data atual do objeto DateTimeImmutable configurado com a data fornecida.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create(). A função modifica este objeto.

`year`

Ano da data.

`month`

Mês da data.

`day`

Dia da data.

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::setDate()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable();
$newDate = $date->setDate(2001, 2, 3);
echo $newDate->format('Y-m-d');
```

O exemplo acima produzirá:

```
2001-02-03
```

Exemplo #2 Valores que excedem os limites são transferidos para a escala imediatamente superior

```php
<?php
$date = new DateTimeImmutable();

$newDate = $date->setDate(2001, 2, 28);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setDate(2001, 2, 29);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setDate(2001, 14, 3);
echo $newDate->format('Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2001-02-28
2001-03-01
2002-02-03
```

Veja também

- DateTimeImmutable::setISODate() - Define a data ISO
- DateTimeImmutable::setTime() - Define um horário

DateTimeImmutable::setISODate

DateTimeImmutable::setISODate

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setISODate — Define a data ISO

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::setISODate() does not modify the object itself")]
public function **DateTimeImmutable::setISODate**(int $year, int $week, int $dayOfWeek = 1): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable com a data configurada de acordo com o padrão ISO 8601 - usa semanas e deslocamentos de dias no lugar de datas específicas.

Parâmetros

`year`

Ano da data.

`week`

Semana da data.

`dayOfWeek`

Posição em relação ao primeiro dia da semana.

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::setISODate()

Estilo orientado a objetos

```php
<?php

$date = new DateTimeImmutable();

$newDate = $date->setISODate(2008, 2);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 2, 7);
echo $newDate->format('Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2008-01-07
2008-01-13
```

Estilo procedural

```php
<?php

$date = date_create();

date_isodate_set($date, 2008, 2);
```

```php
echo date_format($date, 'Y-m-d') . "\n";

date_isodate_set($date, 2008, 2, 7);
echo date_format($date, 'Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2008-01-07
2008-01-13
```

Exemplo #2 Valores que excedem os limites são transferidos para a escala imediatamente superior

```php
<?php

$date = new DateTimeImmutable();

$newDate = $date->setISODate(2008, 2, 7);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 2, 8);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 53, 7);
echo $newDate->format('Y-m-d') . "\n";
```

O exemplo acima produzirá:

```
2008-01-13
2008-01-14
2009-01-04
```

Exemplo #3 Para encontrar o mês em que uma semana se localiza

```php
<?php

$date = new DateTimeImmutable();
$newDate = $date->setISODate(2008, 14);
echo $newDate->format('n');
```

O exemplo acima produzirá:

```
3
```

Veja também

- DateTimeImmutable::setDate() - Define a data
- DateTimeImmutable::setTime() - Define um horário

DateTimeImmutable::setMicrosecond

DateTimeImmutable::setMicrosecond

(PHP 8 >= 8.4.0)

DateTimeImmutable::setMicrosecond — Define a parte de microssegundos do horário

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::setMicrosecond() does not modify the object itself")]
public function **DateTimeImmutable::setMicrosecond**(int $microsecond): static

Retorna um novo objeto DateTimeImmutable construído a partir do antigo, com a parte de microssegundos modificada.

Parâmetros

`microsecond`

O valor de microssegundos a ser definido (`0` a `999999`).

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Erros/Exceções

Se `microsecond` estiver fora do intervalo [`0`, `999999`], uma DateRangeError é lançada.

Examples

Exemplo #1 Exemplo de DateTimeImmutable::setMicrosecond()

```php
<?php
$date = DateTimeImmutable::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
$date = $date->setMicrosecond(987654);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
?>
```

O exemplo acima produzirá:

```
1970-01-01 00:02:03.456789
1970-01-01 00:02:03.987654
```

Veja também

- DateTimeInterface::getMicrosecond() - Obtém a parte de microssegundos do timestamp Unix

DateTimeImmutable::setTime

DateTimeImmutable::setTime

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTime — Define um horário

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::setTime() does not modify the object itself")]
public function **DateTimeImmutable::setTime**(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable configurado com o horário fornecido.

Parâmetros

`hour`

Hora do horário.

`minute`

Minuto do horário.

`second`

Segundo do horário.

`microsecond`

Microssegundo do horário.

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Registro de Alterações

Versão	Descrição
8.1.0	O comportamento com horários que podem ter duplicidade (durante o retorno do horário de verão) mudou. Anteriormente, o PHP usaria a segunda ocorrência do horário (depois do retorno do horário de verão), em vez da primeira ocorrência (antes do retorno).
7.1.0	O parâmetro `microsecond` foi incluído.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::setTime()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2001-01-01');

$newDate = $date->setTime(14, 55);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
2001-01-01 14:55:00
2001-01-01 14:55:24
```

Exemplo #2 Valores que excedem os limites são transferidos para a escala imediatamente superior

```php
<?php
$date = new DateTimeImmutable('2001-01-01');

$newDate = $date->setTime(14, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 55, 65);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 65, 24);
```

```php
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(25, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

O exemplo acima produzirá:

```
2001-01-01 14:55:24
2001-01-01 14:56:05
2001-01-01 15:05:24
2001-01-02 01:55:24
```

Veja também

- DateTimeImmutable::setDate() - Define a data
- DateTimeImmutable::setISODate() - Define a data ISO

DateTimeImmutable::setTimestamp

DateTimeImmutable::setTimestamp

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTimestamp — Define a data e o horário com base em um timestamp Unix

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::setTimestamp() does not modify the object itself")]
public function **DateTimeImmutable::setTimestamp**(int $timestamp): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable construído a partir do antigo, com a data e o horário configurados com base em um timestamp Unix.

Parâmetros

`timestamp`

> Timestamp Unix representando a data e horário. Configurar timestamps fora dos limites de int é possível usando DateTimeImmutable::modify() com o formato `@`.

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::setTimestamp()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable();
echo $date->format('U = Y-m-d H:i:s') . "\n";

$newDate = $date->setTimestamp(1171502725);
echo $newDate->format('U = Y-m-d H:i:s') . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
1272508903 = 2010-04-28 22:41:43
```

```
1171502725 = 2007-02-14 20:25:25
```

Veja também

- DateTimeImmutable::getTimestamp() - Obtém o timestamp Unix
- DateTimeImmutable::setMicrosecond() - Define a parte de microssegundos do horário

DateTimeImmutable::setTimezone

DateTimeImmutable::setTimezone

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTimezone — Define o fuso horário

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::setTimezone() does not modify the object itself")]
public function **DateTimeImmutable::setTimezone**(DateTimeZone $timezone): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable com um novo fuso horário configurado.

Parâmetros

`timezone`

Um objeto DateTimeZone representando o fuso horário desejado.

Valor Retornado

Retorna um novo objeto DateTimeImmutable modificado para ligação de método. O ponto no tempo subjacente não é modificado quando este médoto é chamado.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::setTimeZone()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

$newDate = $date->setTimezone(new DateTimeZone('Pacific/Chatham'));
echo $newDate->format('Y-m-d H:i:sP') . "\n";
?>
```

O exemplo acima produzirá:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

Veja também

- DateTimeImmutable::getTimezone() - Retorna o fuso horário correspondente a um DateTime informado
- DateTimeZone::__construct() - Cria um novo objeto DateTimeZone

DateTimeImmutable::sub

DateTimeImmutable::sub

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::sub — Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos

Descrição

#[\NoDiscard(message: "as DateTimeImmutable::sub() does not modify the object itself")]
public function **DateTimeImmutable::sub**(DateInterval $interval): DateTimeImmutable

Retorna um novo objeto DateTimeImmutable, com o objeto DateInterval especificado substraído do objeto DateTimeImmutable original.

Parâmetros

`interval`

Um objeto DateInterval

Valor Retornado

Retorna um novo objeto DateTimeImmutable com os dados modificados.

Erros/Exceções

Se uma operação não suportada for solicitada, por exemplo usando um objeto DateInterval que representa especificação relativa de tempo como `next weekday`, uma exceção DateInvalidOperationException é lançada.

Registro de Alterações

Versão	Descrição
8.3.0	Agora lança uma exceção DateInvalidOperationException em vez de um aviso quando uma operação não suportada for solicitada.

Examples

Exemplo #1 Exemplo da função DateTimeImmutable::sub()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('P10D'));
echo $newDate->format('Y-m-d') . "\n";
?>
```

O exemplo acima produzirá:

```
2000-01-10
```

Exemplo #2 Mais exemplos da função DateTimeImmutable::sub()

```php
<?php
$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('PT10H30S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";

$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('P7Y5M4DT4H3M2S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

O exemplo acima produzirá:

```
2000-01-19 13:59:30
1992-08-15 19:56:58
```

Exemplo #3 Cuidado ao subtrair meses

```php
<?php
$date = new DateTimeImmutable('2001-04-30');
$interval = new DateInterval('P1M');

$newDate1 = $date->sub($interval);
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->sub($interval);
echo $newDate2->format('Y-m-d') . "\n";
?>
```

O exemplo acima produzirá:

```
2001-03-30
2001-03-02
```

Veja também

- DateTimeImmutable::add() - Retorna um novo objeto, com adição de dias, meses, anos, horas, minutos e segundos
- DateTimeImmutable::diff() - Retorna a diferença entre dois objetos DateTime
- DateTimeImmutable::modify() - Cria um novo objeto com seu timestamp modificado

DateTimeInterface

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introdução

DateTimeInterface foi criada para que os parâmetros, variáveis de retorno ou declarações de tipo de propriedades aceitem tanto DateTimeImmutable ou DateTime como um valor. Não é possível implementar esta interface em classes no espaço de usuário.

Constantes comuns que permitem formatar objetos DateTimeImmutable ou DateTime através de DateTimeImmutable::format() e DateTime::format() também são definidas nesta interface.

Resumo da Interface

interface **DateTimeInterface** {
/* Constantes */
public const string *ATOM* = "Y-m-d\\TH:i:sP";
public const string *COOKIE* = "l, d-M-Y H:i:s T";
public const string *ISO8601* = "Y-m-d\\TH:i:sO";
public const string *ISO8601_EXPANDED* = "X-m-d\\TH:i:sP";
public const string *RFC822* = "D, d M y H:i:s O";
public const string *RFC850* = "l, d-M-y H:i:s T";
public const string *RFC1036* = "D, d M y H:i:s O";
public const string *RFC1123* = "D, d M Y H:i:s O";
public const string *RFC7231* = "D, d M Y H:i:s \\G\\M\\T";
public const string *RFC2822* = "D, d M Y H:i:s O";
public const string *RFC3339* = "Y-m-d\\TH:i:sP";
public const string *RFC3339_EXTENDED* = "Y-m-d\\TH:i:s.vP";
public const string *RSS* = "D, d M Y H:i:s O";
public const string *W3C* = "Y-m-d\\TH:i:sP";

```
/* Métodos */
public function diff(DateTimeInterface $targetObject, bool $absolute = false): DateInterval
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated]
public function __wakeup(): void
}
```

Constantes predefinidas

DateTimeInterface::ATOM string
DATE_ATOM

Atômico (exemplo: 2005-08-15T15:52:01+00:00); compatível com ISO-8601, RFC 3339 e esquema XML

DateTimeInterface::COOKIE string
DATE_COOKIE

Cookies HTTP (exemplo: Monday, 15-Aug-2005 15:52:01 UTC)

DateTimeInterface::ISO8601 string
DATE_ISO8601

Tipo ISO-8601 (exemplo: 2005-08-15T15:52:01+0000)

> **Nota** :
>
> Este formato não é compatível com a ISO-8601, mas foi deixado assim por motivo de compatibilidade com versões anteriores. Use DateTimeInterface::ISO8601_EXPANDED, DateTimeInterface::ATOM para compatibilidade com a ISO-8601, alternativamente. (ref ISO8601:2004 seção 4.3.3 cláusula d)

DateTimeInterface::ISO8601_EXPANDED string
DATE_ISO8601_EXPANDED

ISO-8601 Expandida (exemplo: +10191-07-26T08:59:52+01:00)

> **Nota** :
>
> Este formato permite valores de ano fora do intervalo normal da ISO-8601 de `0000-9999` porque sempre inclui um caractere de sinal. Ele também garante que a parte do fuso horário (`+01:00`) seja compatível com a ISO-8601.

DateTimeInterface::RFC822 string
DATE_RFC822

RFC 822 (exemplo: Mon, 15 Aug 05 15:52:01 +0000)

DateTimeInterface::RFC850 string
DATE_RFC850

RFC 850 (exemplo: Monday, 15-Aug-05 15:52:01 UTC)

DateTimeInterface::RFC1036 string
DATE_RFC1036

RFC 1036 (exemplo: Mon, 15 Aug 05 15:52:01 +0000)

DateTimeInterface::RFC1123 string
DATE_RFC1123

RFC 1123 (exemplo: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::RFC7231 string
DATE_RFC7231

RFC 7231 (desde o PHP 7.0.19 e 7.1.5) (exemplo: Sat, 30 Apr 2016 17:52:13 GMT)

DateTimeInterface::RFC2822 string
DATE_RFC2822

RFC 2822 (exemplo: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::RFC3339 string
DATE_RFC3339

O mesmo que DATE_ATOM

DateTimeInterface::RFC3339_EXTENDED string
DATE_RFC3339_EXTENDED

RFC 3339 formato ESTENDIDO (exemplo: 2005-08-15T15:52:01.000+00:00)

DateTimeInterface::RSS string
DATE_RSS

RSS (exemplo: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::W3C string
DATE_W3C

World Wide Web Consortium (exemplo: 2005-08-15T15:52:01+00:00)

Registro de Alterações

Versão	Descrição
8.5.0	As constantes `DATE_RFC7231` e `DateTimeInterface::RFC7231` foram descontinuadas.
8.4.0	As constantes de classe agora são tipadas.
8.2.0	A constante `DateTimeInterface::ISO8601_EXPANDED` foi adicionada.
7.2.0	As constantes da classe DateTime agora são definidas na classe **DateTimeInterface**.

Índice

DateTimeInterface::diff

DateTimeInterface::diff

DateTimeImmutable::diff

DateTime::diff

date_diff

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeInterface::diff -- DateTimeImmutable::diff -- DateTime::diff -- date_diff — Retorna a diferença entre dois objetos DateTime

Descrição

Estilo orientado a objetos

public function **DateTimeInterface::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval
public function **DateTimeImmutable::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval
public function **DateTime::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval

Estilo procedural

function date_diff(DateTimeInterface `$baseObject`, DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval

Retorna a diferença entre dois objetos que implementam a classe DateTimeInterface.

Parâmetros

`datetime`

A data a ser comparada.

`absolute`

Informa se o intervalo deve ser forçado como positivo.

Valor Retornado

O objeto DateInterval representa a diferença entre as duas datas.

O parâmetro `absolute` afeta somente a propriedade `invert` de um objeto DateInterval.

O valor de retorno representa mais especificamente o intervalo de tempo a ser aplicado ao objeto original (`$this` ou `$originObject`) para se chegar ao `$targetObject`. Este processo nem sempre é reversível.

O método leva em conta mudanças de horário de verão, e por isso pode retornar um intervalo de `24 horas e 30 minutos`, por exemplo. Se a intenção for fazer cálculos com horários absolutos, será necessário converter tanto `$this`/`$baseObject` quanto `$targetObject` para UTC primeiro.

Examples

Exemplo #1 Exemplo do método DateTimeImmutable::diff()

Estilo orientado a objetos

```php
<?php
$origin = new DateTime('2009-10-11');
$target = new DateTime('2009-10-13');
$interval = $origin->diff($target);
echo $interval->format('%R%a dias');
```

O exemplo acima produzirá:

```
+2 dias
```

Estilo procedural

```php
<?php
$origin = date_create('2009-10-11');
$target = date_create('2009-10-13');
$interval = date_diff($origin, $target);
echo $interval->format('%R%a dias');
```

O exemplo acima produzirá:

```
+2 dias
```

Exemplo #2 DateTimeInterface::diff() durante mudança de horário de verão

```php
<?php
$originalTime = new DateTimeImmutable("2021-10-30 09:00:00 Europe/London");
$targetTime = new DateTimeImmutable("2021-10-31 08:30:00 Europe/London");
```

```php
$interval = $originalTime->diff($targetTime);
echo $interval->format("%H:%I:%S (Dias inteiros: %a)"), "\n";
```

O exemplo acima produzirá:

```
24:30:00 (Dias inteiros: 0)
```

Exemplo #3 Intervalo do método DateTimeInterface::diff()

O valor que o método retorna é a quantidade exata de tempo para sair de `$this` e chegar em `$targetObject`. Comparando 1º de janeiro com 31 de dezembro retorna portanto 364, e não 365 dias (para anos não bissextos).

```php
<?php
$originalTime = new DateTimeImmutable("2023-01-01 UTC");
$targetTime = new DateTimeImmutable("2023-12-31 UTC");
$interval = $originalTime->diff($targetTime);
echo "Dias inteiros: ", $interval->format("%a"), "\n";
```

O exemplo acima produzirá:

```
Dias inteiros: 364
```

Exemplo #4 Comparação entre objetos DateTime

> **Nota** :
>
> Objetos das classes DateTimeImmutable e DateTime podem ser comparados utilizando os operadores de comparação.

```php
<?php
$date1 = new DateTime("now");
$date2 = new DateTime("tomorrow");

var_dump($date1 == $date2);
var_dump($date1 < $date2);
var_dump($date1 > $date2);
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
bool(false)
```

Veja também

- DateInterval::format() - Formata um intervalo
- DateTime::add() - Modifica um objeto DateTime, com adição de dias, meses, anos, horas, minutos e segundos.
- DateTime::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos de um objeto DateTime

DateTimeInterface::format

DateTimeInterface::format

DateTimeImmutable::format

DateTime::format

date_format

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::format -- DateTimeImmutable::format -- DateTime::format -- date_format — Retorna a data formatada de acordo com um formato informado

Descrição

Estilo orientado a objetos

public function **DateTimeInterface::format**(string `$format`): string
public function **DateTimeImmutable::format**(string `$format`): string
public function **DateTime::format**(string `$format`): string

Estilo procedural

function date_format(DateTimeInterface `$object`, string `$format`): string

Retorna a data formatada de acordo com um formato informado.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create()

`format`

O formato da string de data resultante. Veja abaixo as opções de formatação. Existem também várias constantes de data predefinidas que podem ser usadas, por exemplo, **`DATE_RSS`** contém a string de formatação `'D, d M Y H:i:s'`.

Os seguintes caracteres são reconhecidos na string do parâmetro `format`

Caractere em `format`	Descrição	Exemplo de valores retornados
Dia	---	---
`d`	Dia do mês, 2 digits com zeros à esquerda	`01` a `31`
`D`	Uma representação textual de um dia, três letras	`Mon` a `Sun`
`j`	Dia do mês sem zeros à esquerda	`1` a `31`
`l` (lowercase 'L')	Uma representação textual completa do dia da semana	`Sunday` a `Saturday`
`N`	Representação numérica ISO 8601 do dia da semana	`1` (para segunda-feira) a `7` (para domingo)
`S`	Sufixo ordinal inglês para o dia do mês, 2 caracteres	`st`, `nd`, `rd` ou `th`. Funciona bem com `j`
`w`	Representação numérica do dia da semana	`0` (para domingo) a `6` (para sábado)
`z`	O dia do ano (começando em 0)	`0` a `365`
Semana	---	---
`W`	Número da semana do mês ISO 8601, semanas começando na segunda-feira	Exemplo: `42` (a 42ª semana no ano)
Mês	---	---
`F`	Uma representação textual completa do mês, como January ou March	`January` a `December`
`m`	Representação numérica de um mês, com zeros à esquerda	`01` a `12`
`M`	Uma representação textual curta de um mês, três letras	`Jan` a `Dec`
`n`	Representação numérica de um mês, sem zeros à esquerda	`1` a `12`

Caractere em `format`	Descrição	Exemplo de valores retornados
`t`	Número de dias em determinado mês	`28` a `31`
Ano	---	---
`L`	Se é um ano bissexto	`1` se for um ano bissexto, `0` caso contrário.
`o`	Ano de numeração da semana ISO 8601. Tem o mesmo valor que `Y`, exceto que se o número da semana ISO (`W`) pertencer ao ano anterior ou seguinte, esse ano será usado.	Exemplos: `1999` ou `2003`
`X`	Uma representação numérica completa expandida de um ano, pelo menos 4 dígitos, com `-` para anos AEC, e `+` para anos EC.	Exemplos: `-0055`, `+0787`, `+1999`, `+10191`
`x`	Uma representação numérica completa expandida se necessário, ou uma representação numérica completa padrão se possível (como `Y`). Pelo menos quatro dígitos. Anos AEC são prefixados com um `-`. Anos além de (e incluindo) `10000` são prefixados com um `+`.	Exemplos: `-0055`, `0787`, `1999`, `+10191`
`Y`	Uma representação numérica completa de um ano, pelo menos 4 dígitos, com `-` para anos AEC.	Exemplos: `-0055`, `0787`, `1999`, `2003`, `10191`
`y`	Uma representação de dois digitos do ano	Exemplos: `99` ou `03`
Hora	---	---
`a`	Ante meridiem e Post meridiem em letras minúsculas	`am` ou `pm`
`A`	Ante meridiem e Post meridiem em letras maiúsculas	`AM` ou `PM`
`B`	Internet Time da Swatch	`000` a `999`
`g`	Formato de 12 horas de uma hora sem zeros à esquerda	`1` a `12`
`G`	Formato de 24 horas de uma hora sem zeros à esquerda	`0` a `23`
`h`	Formato de 12 horas de uma hora com zeros à esquerda	`01` a `12`
`H`	Formato de 24 horas de uma hora com zeros à esquerda	`00` a `23`
`i`	Minutos com zeros à esquerda	`00` to `59`
`s`	Segundos com zeros à esquerda	`00` a `59`
`u`	Microssegundos. Note que date() sempre irá gerar `000000` uma vez que leva um parâmetro int, enquanto **DateTimeInterface::format()** suporta microssegundos se um objeto do tipo DateTimeInterface foi criado com microssegundos.	Exemplo: `654321`
`v`	Milissegundos. A mesma nota se aplica como no `u`.	Exemplo: `654`
Fuso horário	---	---
`e`	Identificador do fuso horário	Exemplos: `UTC`, `GMT`, `Atlantic/Azores`
`I` (i maiúsculo)	Se a data está ou não no horário de verão	`1` se está no horário de verão, `0` caso contrário.
`O`	Diferença para o horário de Greenwich (GMT) sem dois pontos entre horas e minutos	Example: `+0200`
`P`	Diferença para o horário de Greenwich (GMT) com dois pontos entre horas e minutos	Exemplo: `+02:00`
`p`	O mesmo que `P`, mas retorna `Z` em vez de `+00:00` (disponível a partir do PHP 8.0.0)	Exemplo: `Z` ou `+02:00`
`T`	Abreviação do fuso horário, se conhecido; ou o deslocamento GMT caso contrário.	Exemplos: `EST`, `MDT`, `+05`
`Z`	Deslocamento do fuso horário em segundos. O deslocamento para fusos a oeste de UTC é sempre negativo, e para aqueles a leste de UTC é sempre positivo.	`-43200` a `50400`
Data/Horário completos	---	---
`c`	Data ISO 8601. Compatível somente com o formato não expandido (até o ano 9999). Datas posteriores resultarão em string inválida. Para datas posteriores e formato expandido, consulte `x` e `X`.	2004-02-12T15:19:21+00:00

Caractere em `format`	Descrição	Exemplo de valores retornados
`r`	Data formatada » RFC 2822/» RFC 5322	Exemplo: `Thu, 21 Dec 2000 16:01:07 +0200`
`U`	Segundos desde a Época Unix (1 de janeiro de 1970 00:00:00 GMT)	Veja também time()

Caracteres não reconhecidos na string de formatação serão impressos como estão. O formato `Z` sempre retornará `0` ao usar gmdate().

> **Nota** :
>
> Uma vez que esta função só aceita timestamps int, o caractere de formato `u` só é útil ao usar a função date_format() com timestamps baseados no usuário criados com date_create().

Valor Retornado

Retorna a data formatada em caso de sucesso.

Registro de Alterações

Versão	Descrição
8.2.0	Os caracteres de formatação `X` e `x` foram adicionados.
8.0.0	O caractere de formatação `p` foi adicionado.

Examples

Exemplo #1 Exemplo de DateTimeInterface::format()

Estilo orientado a objetos

```php
<?php
$date = new DateTimeImmutable('2000-01-01');
echo $date->format('Y-m-d H:i:s');
?>
```

O exemplo acima produzirá:

```
2000-01-01 00:00:00
```

Estilo procedural

```php
<?php
$date = date_create('2000-01-01');
echo date_format($date, 'Y-m-d H:i:s');
?>
```

O exemplo acima produzirá:

```
2000-01-01 00:00:00
```

Exemplo #2 Mais exemplos

```php
<?php
// define o fuso horário padrão a ser usado.
date_default_timezone_set('UTC');

// agora
$date = new DateTimeImmutable();

// Imprime algo como: Wednesday
echo $date->format('l'), "\n";

// Imprime algo como: Wednesday 19th of October 2022 08:40:48 AM
echo $date->format('l jS \o\f F Y h:i:s A'), "\n";

/* usa as constantes no parâmetro format */
// Imprime algo como: Wed, 19 Oct 2022 08:40:48 +0000
echo $date->format(DateTimeInterface::RFC2822), "\n";
?>
```

Pode-se evitar que um caractere reconhecido na string de formação seja expandido escapando-o com uma barra invertida antes. Se o caractere com uma barra invertida já for uma sequência especial, pode-se escapar também a barra invertida.

Exemplo #3 Escapando caracteres ao formatar

```php
<?php
$date = new DateTimeImmutable();

// exibe algo como: Wednesday the 19th
echo $date->format('l \t\h\e jS');
?>
```

Para formatar datas em outras línguas, IntlDateFormatter::format() pode ser usado em vez do **DateTimeInterface::format()**.

Notas

Este método não utiliza configurações baseadas no idioma. Toda a saída será em Inglês.

Veja também

- IntlDateFormatter::format() - Formata data/horário como uma string

DateTimeInterface::getMicrosecond

DateTimeInterface::getMicrosecond

DateTimeImmutable::getMicrosecond

DateTime::getMicrosecond

(PHP 8 >= 8.4.0)

DateTimeInterface::getMicrosecond -- DateTimeImmutable::getMicrosecond -- DateTime::getMicrosecond — Obtém a parte de microssegundos do timestamp Unix

Descrição

public function **DateTimeInterface::getMicrosecond**(): int
public function **DateTimeImmutable::getMicrosecond**(): int
public function **DateTime::getMicrosecond**(): int

Obtém a parte de microssegundos do timestamp Unix.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a parte de microssegundos do timestamp Unix que representa a data/horário.

Examples

Exemplo #1 Exemplo de DateTimeInterface::getMicrosecond()

```php
<?php
$date = new DateTimeImmutable('2024-01-01 12:34:56.789123');
var_dump($date->format('u'));
var_dump($date->getMicrosecond());
?>
```

O exemplo acima produzirá:

```
string(6) "789123"
int(789123)
```

Veja também

- DateTimeInterface::getTimestamp() - Obtém o timestamp Unix
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado

DateTimeInterface::getOffset

DateTimeInterface::getOffset

DateTimeImmutable::getOffset

DateTime::getOffset

date_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::getOffset -- DateTimeImmutable::getOffset -- DateTime::getOffset -- date_offset_get — Retorna o deslocamento do fuso horário

Descrição

Estilo orientado a objetos

public function **DateTimeInterface::getOffset**(): int
public function **DateTimeImmutable::getOffset**(): int
public function **DateTime::getOffset**(): int

Estilo procedural

function date_offset_get(DateTimeInterface `$object`): int

Retorna o deslocamento do fuso horário.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create()

Valor Retornado

Retorna o deslocamento do fuso horário, em segundos, para UTC em caso de sucesso.

Examples

Exemplo #1 Exemplo do método DateTime::getOffset()

Estilo orientado a objetos

```php
<?php
$winter = new DateTimeImmutable('2010-12-21', new DateTimeZone('America/New_York'));
$summer = new DateTimeImmutable('2008-06-21', new DateTimeZone('America/New_York'));

echo $winter->getOffset() . "\n";
echo $summer->getOffset() . "\n";
```

O exemplo acima produzirá:

```
-18000
-14400
```

Estilo procedural

```php
<?php
$winter = date_create('2010-12-21', timezone_open('America/New_York'));
$summer = date_create('2008-06-21', timezone_open('America/New_York'));

echo date_offset_get($winter) . "\n";
echo date_offset_get($summer) . "\n";
```

O exemplo acima produzirá:

```
-18000
-14400
```

Nota: -18000 = -5 horas, -14400 = -4 horas.

DateTimeInterface::getTimestamp

DateTimeInterface::getTimestamp

DateTimeImmutable::getTimestamp

DateTime::getTimestamp

date_timestamp_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeInterface::getTimestamp -- DateTimeImmutable::getTimestamp -- DateTime::getTimestamp -- date_timestamp_get — Obtém o timestamp Unix

Descrição

Estilo orientado a objetos

public function **DateTimeInterface::getTimestamp**(): int
public function **DateTimeImmutable::getTimestamp**(): int
public function **DateTime::getTimestamp**(): int

Estilo procedural

function date_timestamp_get(DateTimeInterface `$object`): int

Obtém o timestamp Unix.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o timestamp Unix que representa a data.

Erros/Exceções

Se o timestamp não puder ser representado como um int, uma exceção DateRangeError é disparada. Antes do PHP 8.3.0, uma exceção ValueError era disparada. E, antes do PHP 8.0.0, `false` era retornado neste caso. Ainda, o timestamp pode ser recuperado como string usando o método DateTimeInterface::format() com o formato literal u.

Registro de Alterações

Versão	Descrição
8.3.0	A exceção fora do intervalo é agora DateRangeError.
8.0.0	Esta função não mais retorna `false` em caso de falha.

Examples

Exemplo #1 Exemplo do método DateTime::getTimestamp()

Estilo orientado a objetos

```
<?php
$date = new DateTimeImmutable();
echo $date->getTimestamp();
```

O exemplo acima produzirá algo semelhante a:

```
1272509157
```

Estilo procedural

```
<?php
$date = date_create();
echo date_timestamp_get($date);
```

O exemplo acima produzirá algo semelhante a:

```
1272509157
```

Se for necessário recuperar o timestamp com resolução de milissegundos ou microssegundos, pode ser utilizada a função DateTimeInterface::format().

Exemplo #2 Recuperando o timestamp com resolução de mili e microssegundos

Estilo orientado a objetos

```
<?php
$date = new DateTimeImmutable();
$mili = (int) $date->format('Uv'); // Timestamp em milissegundos
$micro = (int) $date->format('Uu'); // Timestamp em microssegundos
```

```
echo $mili, "\n", $micro, "\n";
```

O exemplo acima produzirá algo semelhante a:

```
1674057635586
1674057635586918
```

Veja também

- DateTime::setTimestamp() - Define a data e hora baseada em um timestamp Unix
- DateTimeImmutable::setTimestamp() - Define a data e o horário com base em um timestamp Unix
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- DateTimeInterface::getMicrosecond() - Obtém a parte de microssegundos do timestamp Unix

DateTimeInterface::getTimezone

DateTimeInterface::getTimezone

DateTimeImmutable::getTimezone

DateTime::getTimezone

date_timezone_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::getTimezone -- DateTimeImmutable::getTimezone -- DateTime::getTimezone -- date_timezone_get — Retorna o fuso horário correspondente a um DateTime informado

Descrição

Estilo orientado a objetos

public function **DateTimeInterface::getTimezone**(): DateTimeZone|false
public function **DateTimeImmutable::getTimezone**(): DateTimeZone|false
public function **DateTime::getTimezone**(): DateTimeZone|false

Estilo procedural

function date_timezone_get(DateTimeInterface `$object`): DateTimeZone|false

Retorna o fuso horário de um DateTime informado

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTime retornado por date_create()

Valor Retornado

Retorna um objeto DateTimeZone em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo do método DateTime::getTimezone()

Estilo orientado a objetos

```
<?php
$date = new DateTimeImmutable("now", new DateTimeZone('Europe/London'));
```

```php
$tz = $date->getTimezone();
echo $tz->getName();
```

O exemplo acima produzirá:

```
Europe/London
```

Estilo procedural

```php
<?php
$date = date_create("now", timezone_open('Europe/London'));
$tz = date_timezone_get($date);
echo timezone_name_get($tz);
```

O exemplo acima produzirá:

```
Europe/London
```

Veja também

- DateTime::setTimezone() - Define o fuso horário de um objeto DateTime

DateTime::__serialize

DateTime::__serialize

DateTimeImmutable::__serialize

DateTimeInterface::__serialize

(PHP 8 >= 8.2.0)

DateTime::__serialize -- DateTimeImmutable::__serialize -- DateTimeInterface::__serialize — Serializa um DateTime

Descrição

public function **DateTime::__serialize**(): array
public function **DateTimeImmutable::__serialize**(): array
public function **DateTimeInterface::__serialize**(): array

O manipulador __serialize().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A representação serializada do objeto DateTime.

Examples

Exemplo #1 Exemplo de DateTime::__serialize()

```php
<?php

class CustomDateTimeImmutable extends DateTimeImmutable
{
```

```php
    #[\Override]
    public function __serialize(): array
    {
        return [
            'date' => $this->format(DateTimeInterface::W3C),
            'timestamp' => $this->getTimestamp(),
            'timezone' => $this->getTimeZone()->getName(),
            'to' => 'Tome uma xícara de café',
        ];
    }
}

$date = new CustomDateTimeImmutable('2025-03-27');
var_dump(serialize($date));
```

O exemplo acima produzirá:

```
string(190) "O:23:"CustomDateTimeImmutable":4:{s:4:"date";s:25:"2025-03-27T00:00:00-
03:00";s:9:"timestamp";i:1743044400;s:8:"timezone";s:17:"America/Sao_Paulo";s:2:"to";s:25:"Tome uma xícara de café";}"
```

Notas

> **Aviso**
>
> Um Error é lançado ao tentar desserializar um objeto DateTime personalizado se __serialize() estiver definido, mas __unserialize() estiver ausente.

Veja também

- DateTime::__unserialize() - Desserializa um Datetime

DateTime::__unserialize

DateTime::__unserialize

DateTimeImmutable::__unserialize

DateTimeInterface::__unserialize

(PHP 8 >= 8.2.0)

DateTime::__unserialize -- DateTimeImmutable::__unserialize -- DateTimeInterface::__unserialize — Desserializa um Datetime

Descrição

public function **DateTime::__unserialize**(array `$data`): void
public function **DateTimeImmutable::__unserialize**(array `$data`): void
public function **DateTimeInterface::__unserialize**(array `$data`): void

O manipulador __unserialize().

Parâmetros

`data`

O DateTime serializado.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de DateTime::__unserialize()

```php
<?php

class CustomDateTimeImmutable extends DateTimeImmutable
{
    #[\Override]
    public function __unserialize(array $data): void
    {
        echo "Hora de `{$data['to']}`:\n\n";

        parent::__construct($data['date'], new DateTimeZone($data['timezone']));
    }
}

$serializedData = 'O:23:"CustomDateTimeImmutable":4:{s:4:"date";s:25:"2025-03-27T00:00:00-
03:00";s:9:"timestamp";i:1743033600;s:8:"timezone";s:17:"America/Sao_Paulo";s:2:"to";s:26:"tomar uma xícara de café";}';

var_dump(unserialize($serializedData));
```

O exemplo acima produzirá:

```
Hora de `tomar uma xícara de café`:

object(CustomDateTimeImmutable)#1 (3) {
  ["date"]=>
  string(26) "2025-03-27 00:00:00.000000"
  ["timezone_type"]=>
  int(1)
  ["timezone"]=>
  string(6) "-03:00"
}
```

Notas

> **Aviso**
>
> Um Error é lançado ao tentar desserializar um objeto DateTime personalizado se __serialize() estiver definido, mas __unserialize() estiver ausente.

Veja também

- DateTime::__serialize() - Serializa um DateTime

DateTime::__wakeup

DateTime::__wakeup

DateTimeImmutable::__wakeup

DateTimeInterface::__wakeup

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::__wakeup -- DateTimeImmutable::__wakeup -- DateTimeInterface::__wakeup — O manipulador __wakeup

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **DateTime::__wakeup**(): void

#[\Deprecated]
public function **DateTimeImmutable::__wakeup**(): void
#[\Deprecated]
public function **DateTimeInterface::__wakeup**(): void

O manipulador __wakeup().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Inicializa o objeto DateTime.

DateTimeZone

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introdução

Representação de um fuso horário.

Resumo da classe

class **DateTimeZone** {
/* Constantes */
public const int *AFRICA*;
public const int *AMERICA*;
public const int *ANTARCTICA*;
public const int *ARCTIC*;
public const int *ASIA*;
public const int *ATLANTIC*;
public const int *AUSTRALIA*;
public const int *EUROPE*;
public const int *INDIAN*;
public const int *PACIFIC*;
public const int *UTC*;
public const int *ALL*;
public const int *ALL_WITH_BC*;
public const int *PER_COUNTRY*;
/* Métodos */
public function __construct(string `$timezone`)
public function getLocation(): array|false
public function getName(): string
public function getOffset(DateTimeInterface `$datetime`): int
public function getTransitions(int `$timestampBegin` = **`PHP_INT_MIN`**, int `$timestampEnd` = 2147483647): array|false
public static function listAbbreviations(): array
public static function listIdentifiers(int `$timezoneGroup` = **`DateTimeZone::ALL`**, ?string `$countryCode` = **`null`**): array
}

Constantes predefinidas

`DateTimeZone::AFRICA` int

Fusos horários da África.

`DateTimeZone::AMERICA` int

Fusos horários da América.

`DateTimeZone::ANTARCTICA` int

Fusos horários da Antártida.

`DateTimeZone::ARCTIC` int

Fusos horários do Ártico.

`DateTimeZone::ASIA` int

Fusos horários da Ásia.

`DateTimeZone::ATLANTIC` int

Fusos horários do Atlântico.

`DateTimeZone::AUSTRALIA` int

Fusos horários da Austrália.

`DateTimeZone::EUROPE` int

Fusos horários da Europa.

`DateTimeZone::INDIAN` int

Fusos horários da Índia.

`DateTimeZone::PACIFIC` int

Fusos horários do Pacífico.

`DateTimeZone::UTC` int

Fusos horários UTC.

`DateTimeZone::ALL` int

Todos os fusos horários.

`DateTimeZone::ALL_WITH_BC` int

Todos os fusos horários, incluindo retrocompatíveis.

`DateTimeZone::PER_COUNTRY` int

Fusos horários por país.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

DateTimeZone::__construct

DateTimeZone::__construct

timezone_open

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::__construct -- timezone_open — Cria um novo objeto DateTimeZone

Descrição

Estilo orientado a objetos

public function **DateTimeZone::__construct**(string `$timezone`)

Estilo procedural

function timezone_open(string `$timezone`): DateTimeZone|false

Cria um novo objeto DateTimeZone.

Um objeto DateTimeZone dá acesso a três diferentes tipos de regras de fuso horário: deslocamento UTC (tipo 1), abreviação de fuso (tipo 2), e identificadores de fuso horário como publicado no banco de dados de fusos horários da IANA (tipo 3).

O objeto DateTimeZone pode ser anexado aos objetos DateTime e DateTimeImmutable para que sejam capazes de mostrar o fuso horário encapsulado por estes objetos em um fuso horário local.

Parâmetros

`timezone`

Um dos nomes de fuso horário suportados, um valor de deslocamento (+0200), ou um identificador de fuso (BST).

Valor Retornado

Retorna um objeto DateTimeZone em caso de sucesso. Estilo procedural retorna `false` em caso de falha.

Erros/Exceções

Este método lança uma exceção DateInvalidTimeZoneException se o fuso horário informado não for reconhecido com um fuso horário válido. Antes do PHP 8.3, era uma exceção Exception.

Registro de Alterações

Versão	Descrição
8.3.0	Valores inválidos agora retornam uma DateInvalidTimeZoneException em vez de uma Exception genérica.

Examples

Exemplo #1 Criando e anexando DateTimeZone a um DateTimeImmutable

```php
<?php
$d = new DateTimeImmutable("2022-06-02 15:44:48 UTC");

$timezones = [ 'Europe/London', 'GMT+04:45', '-06:00', 'CEST' ];

foreach ($timezones as $tz) {
    $tzo = new DateTimeZone($tz);

    $local = $d->setTimezone($tzo);
    echo $local->format(DateTimeInterface::RFC2822 . ' — e') . "\n";
}
```

O exemplo acima produzirá:

```
Thu, 02 Jun 2022 16:44:48 +0100 — Europe/London
Thu, 02 Jun 2022 20:29:48 +0445 — +04:45
Thu, 02 Jun 2022 09:44:48 -0600 — -06:00
Thu, 02 Jun 2022 17:44:48 +0200 — CEST
```

Exemplo #2 Capturando erros na instanciação de DateTimeZone

```php
<?php
// Lidando com erros através de captura de exceções
$timezones = array('Europe/London', 'Mars/Phobos', 'Jupiter/Europa');

foreach ($timezones as $tz) {
    try {
        $mars = new DateTimeZone($tz);
        echo $mars->getName() . "\n";
    } catch(Exception $e) {
        echo $e->getMessage() . "\n";
    }
}
```

O exemplo acima produzirá:

```
Europe/London
DateTimeZone::__construct() [datetimezone.--construct]: Unknown or bad timezone (Mars/Phobos)
DateTimeZone::__construct() [datetimezone.--construct]: Unknown or bad timezone (Jupiter/Europa)
```

DateTimeZone::getLocation

DateTimeZone::getLocation

timezone_location_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeZone::getLocation -- timezone_location_get — Retorna informações da localização de um fuso horário

Descrição

Estilo orientado a objetos

public function **DateTimeZone::getLocation**(): array|false

Estilo procedural

function timezone_location_get(DateTimeZone `$object`): array|false

Retorna informações da localização de um fuso horário, incluindo o código do país, latitude/longitude e comentários.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTimeZone retornado por timezone_open()

Valor Retornado

Array contendo informações da localização de um fuso horário ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo do método DateTimeZone::getLocation()

```php
<?php
$tz = new DateTimeZone("Asia/Jakarta");
print_r($tz->getLocation());
print_r(timezone_location_get($tz));
```

O exemplo acima produzirá:

```
Array
(
    [country_code] => ID
    [latitude] => -6.16667
    [longitude] => 106.8
    [comments] => Java, Sumatra
)
Array
(
    [country_code] => ID
    [latitude] => -6.16667
    [longitude] => 106.8
```

```
    [comments] => Java, Sumatra
)
```

O elemento `country_code` contém o código de país de 2 letras conforme ISO 3166-1 para cada entrada. Os elementos `latitude` e `longitude` contêm as coordenadas da cidade nomeada pelo identificador do fuso horário e o elemento `comments` contém (quando não for **`false`**) uma dica de onde no país informado este fuso horário se aplica. Esta informação é adequada para apresentação ao usuário final.

Veja também

- DateTimeZone::listIdentifiers() - Retorna um array numericamente indexado contendo todos os indenticadores de fuso horário para obter uma lista completa ou parcial de todos os identificadores de fuso horário suportados

DateTimeZone::getName

DateTimeZone::getName

timezone_name_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getName -- timezone_name_get — Retorna o nome do fuso horário

Descrição

Estilo orientado a objetos

public function **DateTimeZone::getName**(): string

Estilo procedural

function timezone_name_get(DateTimeZone `$object`): string

Retorna o nome do fuso horário.

Parâmetros

`object`

O objeto DateTimeZone do qual se deseja obter o nome.

Valor Retornado

Dependendo do tipo do fuso horário, deslocamento UTC (tipo 1), abreviação do fuso (tipo 2), e identificadores de fuso como publicado no banco de dados da IANA (tipo 3), o valor retornado será um descritor string para criar um novo objeto DateTimeZone com o mesmo deslocamento e/ou regras. Por exemplo `02:00`, `CEST`, ou um dos nomes na lista de fusos horários.

DateTimeZone::getOffset

DateTimeZone::getOffset

timezone_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getOffset -- timezone_offset_get — Retorna o deslocamento do fuso horário em relação ao GMT

Descrição

Estilo orientado a objetos

public function **DateTimeZone::getOffset**(DateTimeInterface `$datetime`): int

Estilo procedural

function timezone_offset_get(DateTimeZone `$object`, DateTimeInterface `$datetime`): int

Esta função retorna o deslocamento em relação ao GMT para a data/horário especificada no parâmetro `datetime`. O deslocamento GMT é calculado com informação do fuso horário contido no objeto DateTimeZone utilizado.

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTimeZone retornado por timezone_open()

`datetime`

DateTime que contém a data/horário a partir da qual será computado o deslocamento.

Valor Retornado

Retorna o deslocamento do fuso horário em segundos.

Examples

Exemplo #1 Exemplos do método DateTimeZone::getOffset()

```php
<?php
// Cria dois objetos de fuso horário, um para Taipei (Taiwan) e um para
// Tokyo (Japão)
$dateTimeZoneTaipei = new DateTimeZone("Asia/Taipei");
$dateTimeZoneJapan = new DateTimeZone("Asia/Tokyo");

// Cria dois objetos DateTime que irão conter o mesmo timestamp Unix, mas
// têm fusos horários diferentes anexados a eles.
$dateTimeTaipei = new DateTime("now", $dateTimeZoneTaipei);
$dateTimeJapan = new DateTime("now", $dateTimeZoneJapan);

// Calcula o deslocamento GMT para a data/horário contida no objeto $dateTimeTaipei,
// mas usando as regras de fuso horário como definidas para Tokyo
// ($dateTimeZoneJapan).
$timeOffset = $dateTimeZoneJapan->getOffset($dateTimeTaipei);

// Deve mostrar int(32400) (para datas após Sat Sep 8 01:00:00 1951 JST).
var_dump($timeOffset);
```

O exemplo acima produzirá:

```
int(32400)
```

DateTimeZone::getTransitions

DateTimeZone::getTransitions

timezone_transitions_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getTransitions -- timezone_transitions_get — Retorna todas as transições para o fuso horário

Descrição

Estilo orientado a objetos

public function **DateTimeZone::getTransitions**(int `$timestampBegin` = `PHP_INT_MIN`, int `$timestampEnd` = 2147483647): array|false

Estilo procedural

function timezone_transitions_get(DateTimeZone `$object`, int `$timestampBegin` = `PHP_INT_MIN`, int `$timestampEnd` = 2147483647): array|false

Parâmetros

`object`

Somente no estilo procedural: Um objeto DateTimeZone retornado por timezone_open()

`timestampBegin`

Timestamp inicial.

`timestampEnd`

Timestamp final.

Valor Retornado

Retorna um array numericamente indexado de arrays de transições em caso de sucesso, ou `false` em caso de falha. Objetos DataTimeZone do tipo 1 (deslocamento UTC) e tipo 2 (abreviações) não contêm transições, e se tiverem este método chamado irão retornar `false`.

Se o parâmetro `timestampBegin` for informado, a primeira entrada no array de retorno irá contrer um elemento de transição no horário de `timestampBegin`.

Estrutura do Array de Transição

Key	Type	Description
`ts`	int	Timestamp Unix
`time`	string	string de horário `DateTimeInterface::ISO8601_EXPANDED` (PHP 8.2 e seguintes), ou `DateTimeInterface::ISO8601` (PHP 8.1 e anteriores)
`offset`	int	Deslocamento UTC em segundos
`isdst`	bool	Se o horário de verão está ativo
`abbr`	string	Abreviação do fuso horário

Registro de Alterações

Versão	Descrição
8.1.0	O valor padrão de `timestampEnd` foi alterado para 2147483647. Anteriormente era `PHP_INT_MAX`.

Examples

Exemplo #1 Exemplo do método timezone_transitions_get()

```php
<?php
$timezone = new DateTimeZone("Europe/London");
$transitions = $timezone->getTransitions();
print_r(array_slice($transitions, 0, 3));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [ts] => -2147483648
            [time] => 1901-12-13T20:45:52+00:00
            [offset] => -75
            [isdst] =>
            [abbr] => LMT
        )

    [1] => Array
        (
            [ts] => 442304971
            [time] => 1847-12-01T00:01:15+00:00
            [offset] => 0
```

```
            [isdst] =>
            [abbr] => GMT
        )

    [2] => Array
        (
            [ts] => -1691964000
            [time] => 1916-05-21T02:00:00+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

)
```

Exemplo #2 Um exemplo da função timezone_transitions_get() com o parâmetro `timestampBegin`

```php
<?php
$timezone = new DateTimeZone("Europe/London");
$transitions = $timezone->getTransitions(time());
print_r(array_slice($transitions, 0, 3));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [ts] => 1759058251
            [time] => 2025-09-28T11:17:31+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

    [1] => Array
        (
            [ts] => 1761440400
            [time] => 2025-10-26T01:00:00+00:00
            [offset] => 0
            [isdst] =>
            [abbr] => GMT
        )

    [2] => Array
        (
            [ts] => 1774746000
            [time] => 2026-03-29T01:00:00+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

)
```

DateTimeZone::listAbbreviations

DateTimeZone::listAbbreviations

timezone_abbreviations_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::listAbbreviations -- timezone_abbreviations_list — Retorna um array associativo contendo indicativo de horário de verão, deslocamento e nome do fuso horário

Descrição

Estilo orientado a objetos

public static function **DateTimeZone::listAbbreviations**(): array

Estilo procedural

function timezone_abbreviations_list(): array

A lista de abreviações retornada inclui todo o uso histórico de abreviações, o que pode levar a entradas corretas, porém confusas. Também existem conflitos, já que PST é usado tanto nos EUA como nas Filipinas.

A lista que esta função retora não é portanto adequada para construir um array com opções para apresentar ao usuário para escolha de fusos horários.

> **Nota** :
>
> Os dados desta função são pré compilados por razões de desempenho, e não são atualizadas quado é usado um novo » timezonedb.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o array de abreviações de fuso horário.

Examples

Exemplo #1 Exemplo da função timezone_abbreviations_list()

```php
<?php
$timezone_abbreviations = DateTimeZone::listAbbreviations();
print_r($timezone_abbreviations["acst"]);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Adelaide
        )

    [1] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Broken_Hill
        )

    [2] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Darwin
        )
```

```
    [3] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/North
        )

    [4] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/South
        )

    [5] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Yancowinna
        )

)
```

Veja também

- timezone_identifiers_list() - Sinônimo de DateTimeZone::listIdentifiers

DateTimeZone::listIdentifiers

DateTimeZone::listIdentifiers

timezone_identifiers_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::listIdentifiers -- timezone_identifiers_list — Retorna um array numericamente indexado contendo todos os indenticadores de fuso horário

Descrição

Estilo orientado a objetos

public static function **DateTimeZone::listIdentifiers**(int `$timezoneGroup = DateTimeZone::ALL`, ?string `$countryCode = null`): array

Estilo procedural

function timezone_identifiers_list(int `$timezoneGroup = DateTimeZone::ALL`, ?string `$countryCode = null`): array

Retorna a lista de » identificadores de fuso horário IANA.

> **Nota** :
>
> É possível identificar o fuso horário do cliente (navegador) com JavaScript usando » Intl.DateTimeFormat ou » Temporal.ZonedDateTime.

Parâmetros

`timezoneGroup`

Uma das constantes da classe DateTimeZone (ou uma combinação).

`countryCode`

Um código de país, com duas letras (maiúsculas), compatível com a ISO 3166-1.

> **Nota** :
>
> Esta opção é utilizada somente quando o parâmetro `timezoneGroup` é configurado como `DateTimeZone::PER_COUNTRY`.

Valor Retornado

Retorna o array de identificadores de fuso horário. Apenas itens não desatualizados são retornados. Para obter todos, incluindo identificadores desatualizados, use `DateTimeZone::ALL_WITH_BC` no valor de `timezoneGroup`.

Registro de Alterações

Versão	Descrição
8.0.0	Antes desta versão, `false` era retornado em caso de falha.
7.1.0	O parâmetro `countryCode` agora pode ser nulo.

Examples

Exemplo #1 Listando identificadores com comentários de localização

```php
<?php
$identifiers = DateTimeZone::listIdentifiers(DateTimeZone::ALL);

foreach ($identifiers as $tzid) {
    $tz = new DateTimeZone($tzid);
    $comments = $tz->getLocation()['comments'];
    echo $tzid . " (" . ($comments ?: 'Região inteira') . ")\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
America/Antigua (Região inteira)
America/Araguaina (Tocantins)
America/Argentina/Buenos_Aires (Buenos Aires (BA, CF))
America/Argentina/Catamarca (Catamarca (CT), Chubut (CH))
America/Argentina/Cordoba (Argentina (most areas: CB, CC, CN, ER, FM, MN, SE, SF))
// (Saída truncada devido ao tamanho)
```

Exemplo #2 Listando identificadores para uma região específica

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::ASIA );
for ($i=0; $i < 5; $i++) {
    echo "$timezone_identifiers[$i]\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
Asia/Aden
Asia/Almaty
Asia/Amman
Asia/Anadyr
Asia/Aqtau
```

Exemplo #3 Listando identificadores para múltiplas regiões

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::ASIA | DateTimeZone::PACIFIC );
echo join( ', ', $timezone_identifiers );
```

O exemplo acima produzirá algo semelhante a:

```
Asia/Aden, Asia/Almaty, Asia/Amman, Asia/Anadyr, Asia/Aqtau, Asia/Aqtobe,
Asia/Ashgabat, Asia/Atyrau, Asia/Baghdad, Asia/Bahrain, Asia/Baku,
Asia/Bangkok, Asia/Barnaul, Asia/Beirut, Asia/Bishkek, Asia/Brunei,
Asia/Chita, Asia/Choibalsan, Asia/Colombo, Asia/Damascus, Asia/Dhaka,
Asia/Dili, Asia/Dubai, Asia/Dushanbe, Asia/Famagusta, Asia/Gaza, Asia/Hebron,
Asia/Ho_Chi_Minh, Asia/Hong_Kong, Asia/Hovd, Asia/Irkutsk, Asia/Jakarta,
Asia/Jayapura, Asia/Jerusalem, Asia/Kabul, Asia/Kamchatka, Asia/Karachi,
Asia/Kathmandu, Asia/Khandyga, Asia/Kolkata, Asia/Krasnoyarsk,
Asia/Kuala_Lumpur, Asia/Kuching, Asia/Kuwait, Asia/Macau, Asia/Magadan,
Asia/Makassar, Asia/Manila, Asia/Muscat, Asia/Nicosia, Asia/Novokuznetsk,
Asia/Novosibirsk, Asia/Omsk, Asia/Oral, Asia/Phnom_Penh, Asia/Pontianak,
Asia/Pyongyang, Asia/Qatar, Asia/Qostanay, Asia/Qyzylorda, Asia/Riyadh,
Asia/Sakhalin, Asia/Samarkand, Asia/Seoul, Asia/Shanghai, Asia/Singapore,
Asia/Srednekolymsk, Asia/Taipei, Asia/Tashkent, Asia/Tbilisi, Asia/Tehran,
Asia/Thimphu, Asia/Tokyo, Asia/Tomsk, Asia/Ulaanbaatar, Asia/Urumqi,
Asia/Ust-Nera, Asia/Vientiane, Asia/Vladivostok, Asia/Yakutsk, Asia/Yangon,
Asia/Yekaterinburg, Asia/Yerevan, Pacific/Apia, Pacific/Auckland,
Pacific/Bougainville, Pacific/Chatham, Pacific/Chuuk, Pacific/Easter,
Pacific/Efate, Pacific/Fakaofo, Pacific/Fiji, Pacific/Funafuti,
Pacific/Galapagos, Pacific/Gambier, Pacific/Guadalcanal, Pacific/Guam,
Pacific/Honolulu, Pacific/Kanton, Pacific/Kiritimati, Pacific/Kosrae,
Pacific/Kwajalein, Pacific/Majuro, Pacific/Marquesas, Pacific/Midway,
Pacific/Nauru, Pacific/Niue, Pacific/Norfolk, Pacific/Noumea,
Pacific/Pago_Pago, Pacific/Palau, Pacific/Pitcairn, Pacific/Pohnpei,
Pacific/Port_Moresby, Pacific/Rarotonga, Pacific/Saipan, Pacific/Tahiti,
Pacific/Tarawa, Pacific/Tongatapu, Pacific/Wake, Pacific/Wallis
```

Exemplo #4 Listando identificadores para um único país

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::PER_COUNTRY, "UA" );
foreach( $timezone_identifiers as $identifier ) {
    echo "$identifier\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
Europe/Kyiv
Europe/Simferopol
Europe/Uzhgorod
Europe/Zaporozhye
```

Veja também

- timezone_abbreviations_list() - Sinônimo de DateTimeZone::listAbbreviations

DateInterval

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Representa um intervalo entre datas.

Um intervalo de data armazena, tanto uma quantidade de tempo (em anos, meses, dias, horas, etc), quanto uma string de tempo relativo no formato que os construtores das classes DateTimeImmutable e DateTime suportam.

Mais especificamente, a informação em um objeto da classe **DateInterval** é uma instrução para mover de uma data/horário para outra data/horário. Este processo nem sempre é reversível.

Uma maneira comum de criar um objeto **DateInterval** é calculando a diferença entre dois objetos data/horário através do método DateTimeInterface::diff().

Como não existe uma maneira bem definida de comparar intervalos de datas, as instâncias de **DateInterval** são incomparáveis.

Resumo da classe

class **DateInterval** {
/* Propriedades */
public int *$y*;
public int *$m*;
public int *$d*;
public int *$h*;
public int *$i*;
public int *$s*;
public float *$f*;
public int *$invert*;
public mixed *$days*;
public bool *$from_string*;
public string *$date_string*;
/* Métodos */
public function __construct(string `$duration`)
public static function createFromDateString(string `$datetime`): DateInterval
public function format(string `$format`): string
}

Propriedades

> **Aviso**
>
> As propriedades disponíveis listadas abaixo dependem da versão do PHP, e devem ser consideradas como *somente-leitura*.

y

Número de anos.

m

Número de meses.

d

Número de dias.

h

Número de horas.

i

Número de minutos.

s

Número de segundos.

f

Número de microssegundos, como uma fração de um segundo.

invert

Será `1` se o intervalo representa um período negativo de tempo e `0` caso contrário. Veja DateInterval::format().

days

Se o objeto DateInterval foi criado por DateTimeImmutable::diff() ou DateTime::diff(), então ele representará o número total de dias inteiros entre a data inicial e final. Caso contrário, *days* será `false`.

from_string

Se o objeto DateInterval foi criado por DateInterval::createFromDateString(), então o valor desta propriedade será `true` e a propriedade *date_string* será populada. Caso contrário, o valor será `false` e as propriedades *y* a *f*, *invert* e *days* serão populadas.

date_string

String usado como argumento para DateInterval::createFromDateString().

Registro de Alterações

Versão	Descrição
8.2.0	As propriedades *from_string* e *date_string* foram adicionadas para instâncias **DateInterval** que foram criadas usando o método DateInterval::createFromDateString().
8.2.0	Apenas as propriedades `y` a `f`, `invert`, e `days` serão visíveis.
7.4.0	As instâncias de **DateInterval** agora são incomparáveis; antes, todas as instâncias de **DateInterval** eram consideradas iguais.
7.1.0	A propriedade *f* foi adicionada.

Índice

DateInterval::__construct

DateInterval::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::__construct — Cria um novo objeto DateInterval

Descrição

public function **DateInterval::__construct**(string `$duration`)

Cria um novo objeto DateInterval.

Parâmetros

`duration`

Uma especificação de intervalo.

O formato inicia-se com a letra `P`, de "período". Cada período de duração é representado por um valor inteiro seguido do designador de período. Se a duração contiver elementos de tempo, essa porção da especificação é precedida pela letra `T`.

`duration` Designadores de Período

Designador de Período	Descrição
`Y`	anos
`M`	meses
`D`	dias
`W`	semanas. Convertidas em dias. Antes do PHP 8.0.0, não pode ser combinada com `D`.
`H`	horas
`M`	minutos
`S`	segundos

Alguns exemplos simples. O período de dois dias é representado por `P2D`. O período de dois segundos é representado por `PT2S`. O período de seis anos e cinco minutos é representado por `P6YT5M`.

> **Nota** :
>
> Os tipos de unidade devem ser informados a partir da maior unidade de escala na esquerda até a menor escala na direita. Ou seja, anos antes de meses, meses antes de dias, dias antes de minutos etc. Assim, um ano e quatro dias deve ser representado por `P1Y4D`, e não `P4D1Y`.

A especificação também pode ser representada como data e horário. Um exemplo de um ano e quatro dias seria `P0001-00-04T00:00:00`. Porém, valores neste formato não podem exceder o tamanho máximo do período (por exemplo `25` horas é inválido).

Estes formatos são baseados na » especificação de duração ISO 8601.

Erros/Exceções

Lança uma exceção DateMalformedIntervalStringException quando o parâmetro `duration` não puder ser interpretado como um intervalo. Antes do PHP 8.3, era uma Exception.

Registro de Alterações

Versão	Descrição
8.3.0	Agora lança DateMalformedIntervalStringException em vez de Exception.
8.2.0	Apenas as propriedades `y` a `f`, `invert` e `days` estarão visíveis, incluindo uma nova propriedade booleana `from_string`.
8.0.0	`W` pode ser combinado com `D`.

Examples

Exemplo #1 Construindo e usando objetos da classe DateInterval

```php
<?php
// Cria uma data específica
$someDate = \DateTime::createFromFormat("Y-m-d H:i", "2022-08-25 14:18");

// Cria um intervalo
$interval = new \DateInterval("P7D");

// Adiciona o intervalo
$someDate->add($interval);

// Converte o intervalo para string
echo $interval->format("%d");
```

O exemplo acima produzirá:

```
7
```

Exemplo #2 Exemplo da classe DateInterval

```php
<?php
$interval = new DateInterval('P1W2D');
var_dump($interval);
```

Saída do exemplo acima no PHP 8.2:

```
object(DateInterval)#1 (10) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
```

```
  int(9)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["from_string"]=>
  bool(false)
}
```

Saída do exemplo acima no PHP 8:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
  int(9)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
```

Saída do exemplo acima no PHP 7:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
```

```
  int(0)
  ["d"]=>
  int(2)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
```

Veja também

- DateInterval::format() - Formata um intervalo
- DateTime::add() - Modifica um objeto DateTime, com adição de dias, meses, anos, horas, minutos e segundos.
- DateTime::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos de um objeto DateTime
- DateTime::diff() - Retorna a diferença entre dois objetos DateTime

DateInterval::createFromDateString

DateInterval::createFromDateString

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::createFromDateString — Cria um DateInterval de porções relativas da string

Descrição

Estilo orientado a objetos

public static function **DateInterval::createFromDateString**(string `$datetime`): DateInterval

Estilo procedural

function date_interval_create_from_date_string(string `$datetime`): DateInterval|false

Utiliza o interpretador de data/horário como no construtor da classe DateTimeImmutable para criar um DateInterval a partir das partes relacionadas de uma string analisada.

Parâmetros

`datetime`

> Uma data com porções relativas. Especificamente, os formatos relativos suportados pelo analisador utilizados em DateTimeImmutable, DateTime e strtotime() serão utilizados para construir o DateInterval.
>
> Para usar um string no formato ISO-8601 como `P7D`, deve-se usar o construtor DateInterval::__construct().

Valor Retornado

Retorna DateInterval em caso de sucesso. Estilo procedural retorna `false` em caso de falha.

Erros/Exceções

Apenas para API orientada a objetos: Se uma string de data/horário inválida for passada, uma exceção DateMalformedIntervalStringException é lançada.

Registro de Alterações

Versão	Descrição
8.4.0	Agora possui um tipo de retorno provisório de DateInterval. Anteriormente era DateInterval|false.
8.3.0	**DateInterval::createFromDateString()** agora lança uma exceção DateMalformedIntervalStringException se uma string inválida for passada. Anteriormente, retornava `false` e um alerta era emitido. A função date_interval_create_from_date_string() não foi alterada.
8.2.0	Apenas as propriedades `from_string` e `date_string` estarão visíveis quando um DateInterval for criado com este método.

Examples

Exemplo #1 Analisando intervalos de datas válidos

```php
<?php
// Cada conjunto de intervalos é igual.
$i = new DateInterval('P1D');
$i = DateInterval::createFromDateString('1 day');

$i = new DateInterval('P2W');
$i = DateInterval::createFromDateString('2 weeks');

$i = new DateInterval('P3M');
$i = DateInterval::createFromDateString('3 months');

$i = new DateInterval('P4Y');
$i = DateInterval::createFromDateString('4 years');

$i = new DateInterval('P1Y1D');
$i = DateInterval::createFromDateString('1 year + 1 day');

$i = new DateInterval('P1DT12H');
$i = DateInterval::createFromDateString('1 day + 12 hours');

$i = new DateInterval('PT3600S');
$i = DateInterval::createFromDateString('3600 seconds');
```

Exemplo #2 Analisando combinações e intervalos negativos

```php
<?php
$i = DateInterval::createFromDateString('62 weeks + 1 day + 2 weeks + 2 hours + 70 minutes');
echo $i->format('%d %h %i'), "\n";

$i = DateInterval::createFromDateString('1 year - 10 days');
echo $i->format('%y %d'), "\n";
```

O exemplo acima produzirá:

```
449 2 70
1 -10
```

Exemplo #3 Analisando intervalos especiais de data

```php
<?php
$i = DateInterval::createFromDateString('last day of next month');
var_dump($i);

$i = DateInterval::createFromDateString('last weekday');
var_dump($i);
```

Saída do exemplo acima no PHP 8.2:

```
object(DateInterval)#1 (2) {
  ["from_string"]=>
  bool(true)
  ["date_string"]=>
  string(22) "last day of next month"
}
object(DateInterval)#2 (2) {
  ["from_string"]=>
  bool(true)
  ["date_string"]=>
  string(12) "last weekday"
}
```

A saída do exemplo acima no PHP 8 é semelhante a:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(1)
  ["d"]=>
  int(0)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(2)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
object(DateInterval)#2 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
```

```
  ["d"]=>
  int(0)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(1)
  ["special_amount"]=>
  int(-1)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(1)
}
```

DateInterval::format

DateInterval::format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::format — Formata um intervalo

Descrição

public function **DateInterval::format**(string `$format`): string

Formata um intervalo.

Parâmetros

`format`

Os caracteres a seguir são reconhecidos como string no parâmetro `format`. Cada um dos caracteres de formato deve ser precedidos com o sinal de porcentagem (`%`).

Caractere de `format`	Descrição	Exemplo de valores
`%`	`%` literal	`%`
`Y`	Anos, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`
`y`	Anos, em representação numérica	`1`, `3`
`M`	Meses, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`, `12`
`m`	Meses, em representação numérica	`1`, `3`, `12`
`D`	Dias, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`, `31`
`d`	Dias, em representação numérica	`1`, `3`, `31`
`a`	Número total de dias resultante de DateTime::diff() ou `(unknown)`, caso contrário	`4`, `18`, `8123`
`H`	Horas, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`, `23`
`h`	Horas, em representação numérica	`1`, `3`, `23`

Caractere de `format`	Descrição	Exemplo de valores
`I`	Minutos, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`, `59`
`i`	Minutos, em representação numérica	`1`, `3`, `59`
`S`	Segundos, em representação numérica, com dois dígitos e zero à esquerda	`01`, `03`, `57`
`s`	Segundos, em representação numérica	`1`, `3`, `57`
`F`	Microssegundos, numérico, pelo menos 6 dígitos precedidos com 0	`007701`, `052738`, `428291`
`f`	Microssegundos, numérico	`7701`, `52738`, `428291`
`R`	Sinal "-" quando negativo, "+" quando positivo	`-`, `+`
`r`	Sinal "-" quando negativo, sem sinal quando positivo	`-`,

Valor Retornado

Retorna o intervalo formatado.

Registro de Alterações

Versão	Descrição
7.2.12	Os formatos `F` e `f` agora são sempre positivos.
7.1.0	Os formatos `F` e `f` foram adicionados.

Examples

Exemplo #1 Exemplo da classe DateInterval

```php
<?php
$interval = new DateInterval('P2Y4DT6H8M');
echo $interval->format('%d days');
```

O exemplo acima produzirá:

```
4 days
```

Exemplo #2 A classe DateInterval e os pontos de transferência

```php
<?php
$interval = new DateInterval('P32D');
echo $interval->format('%d days');
```

O exemplo acima produzirá:

```
32 days
```

Exemplo #3 DateInterval e DateTime::diff() com os modificadores %a e %d

```php
<?php
$janeiro = new DateTime('2010-01-01');
$fevereiro = new DateTime('2010-02-01');
$intervalo = $fevereiro->diff($janeiro);

// %a irá resultar no número total de dias.
echo $intervalo->format('%a dias totais')."\n";
```

```php
// Enquanto %d irá apenas resultar no número de dias ainda não cobertos pelo
// mês.
echo $intervalo->format('%m mês, %d dias');
```

O exemplo acima produzirá:

```
31 total days
1 month, 0 days
```

Notas

> **Nota** :
>
> O método **DateInterval::format()** não recalcula pontos de transferência em strings de data/horário. Isto é esperado porque não é possível extrapolar valores como `"32 days"`, o que poderia ser interpretado como qualquer coisa entre `"1 month and 4 days"` e `"1 month and 1 day"`.

Veja também

- DateTime::diff() - Retorna a diferença entre dois objetos DateTime

DatePeriod

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Representa um período entre datas.

O período entre datas permite a interação entre um conjunto de datas e horários recorrentes a intervalos regulares, durante um determinado período.

Resumo da classe

```
class DatePeriod implements IteratorAggregate {
/* Constantes */
public const int EXCLUDE_START_DATE;
public const int INCLUDE_END_DATE;
/* Propriedades */
public readonly ?DateTimeInterface $start;
public readonly ?DateTimeInterface $current;
public readonly ?DateTimeInterface $end;
public readonly ?DateInterval $interval;
public readonly int $recurrences;
public readonly bool $include_start_date;
public readonly bool $include_end_date;
/* Métodos */
public function __construct(
    DateTimeInterface $start,
    DateInterval $interval,
    int $recurrences,
    int $options = 0
)
public function __construct(
    DateTimeInterface $start,
    DateInterval $interval,
    DateTimeInterface $end,
```

```
    int $options = 0
)
public function __construct(string $isostr, int $options = 0)
public static function createFromISO8601String(string $specification, int $options = 0): static
public function getDateInterval(): DateInterval
public function getEndDate(): ?DateTimeInterface
public function getRecurrences(): ?int
public function getStartDate(): DateTimeInterface
}
```

Constantes predefinidas

`DatePeriod::EXCLUDE_START_DATE` int

Exclui a data inicial, utilizada em DatePeriod::__construct().

`DatePeriod::INCLUDE_END_DATE` int

Inclui a data final, utilizado em DatePeriod::__construct().

Propriedades

recurrences

O número mínimo de instâncias retornadas pelo iterador.

O número de recorrências que foi informado explicitamente no parâmetro *recurrences* no construtor da instância de **DatePeriod**, de forma que essa propriedade contém este valor, *mais* um se a data inicial não foi excluída através de **`DatePeriod::EXCLUDE_START_DATE`**, *mais* um se a data final foi incluída através de **`DatePeriod::INCLUDE_END_DATE`**.

Se o número de recorrências não foi informado explicitamente, então esta propriedade conta o número mínimo de recorrências a retornar. Isto pode ser `0`, *mais* um se a data inicial não foi excluída através de **`DatePeriod::EXCLUDE_START_DATE`**, *mais* um se a data final não foi incluída através de **`DatePeriod::INCLUDE_END_DATE`**.

```php
<?php
$start = new DateTime('2018-12-31 00:00:00');
$end   = new DateTime('2021-12-31 00:00:00');
$interval = new DateInterval('P1M');
$recurrences = 5;

// recorrências informadas no construtor
$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::EXCLUDE_START_DATE);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $recurrences);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::INCLUDE_END_DATE);
echo $period->recurrences, "\n";

// recorrências não informadas no construtor
$period = new DatePeriod($start, $interval, $end);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $end, DatePeriod::EXCLUDE_START_DATE);
echo $period->recurrences, "\n";
```

O exemplo acima produzirá:

```
5
6
7
1
0
```

Veja também DatePeriod::getRecurrences().

include_end_date

Informa se deve ser incluída a data final no conjunto de datas a retornar.

include_start_date

Informa se deve ser incluída a data inicial no conjunto de datas a retornar.

start

A data inicial do período.

current

Contém a data atual dentro do período.

end

Data final do período.

interval

Uma especificação ISO 8601 do intervalo repetidor.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.
8.2.0	A constante `DatePeriod::INCLUDE_END_DATE` e a propriedade include_end_date foram adicionados.
8.0.0	**DatePeriod** agora implementa IteratorAggregate. Anteriormente implementava Traversable.

Índice

DatePeriod::__construct

DatePeriod::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DatePeriod::__construct — Cria um novo objeto DatePeriod

Descrição

```
public function DatePeriod::__construct(
    DateTimeInterface $start,
    DateInterval $interval,
    int $recurrences,
    int $options = 0
)
public function DatePeriod::__construct(
    DateTimeInterface $start,
    DateInterval $interval,
    DateTimeInterface $end,
    int $options = 0
)
```

> **Aviso**
>
> A variante de construtor a seguir foi descontinuada:
>
> public function **DatePeriod::__construct**(string `$isostr`, int `$options` = 0)
>
> Esta variante do construtor foi descontinuada a partir do PHP 8.4.0, use o método DatePeriod::createFromISO8601String() em seu lugar.

Cria um objeto DatePeriod.

Objetos DatePeriod podem ser usados como um iterador para gerar um número de objetos DateTimeImmutable ou DateTime a partir de uma data inicial (`start`), um intervalo (`interval`), e uma data final (`end`) ou o número de recorrências (`recurrences`).

A classe do objeto retornado é equivalente à classe ancestral DateTimeImmutable ou DateTime do objeto do parâmetro `start`.

Parâmetros

`start`

A data inicial do período. Incluído por padrão no conjunto de resultados.

`interval`

A recorrência de intervalos entre os períodos.

`recurrences`

O número de recorrências. O número do resultado retornado é uma unidade a mais que isto, já que a data de início é incluída no conjunto de resultados por padrão. Deve ser maior que `0`.

`end`

A data final do período.

`isostr`

Um sobconjunto da » especificação de intervalos recorrented ISO 8601.

Exemplos de algumas características de especificação de intervalos ISO 8601 que o PHP não suporta são:

1. zero ocorrências (`R0/`)
2. fusos horários diferentes de UTC (`Z`), como `+02:00`.

`options`

Um campo de bits que pode ser usado para controlar certos comportamentos com datas de início e fim.

Com **`DatePeriod::EXCLUDE_START_DATE`** exclui-se a data inicial do conjunto de datas recorrentes dentro do período.

Com **`DatePeriod::INCLUDE_END_DATE`** inclui-se a data final do conjunto de datas recorrentes dentro do período.

Erros/Exceções

Lança uma exceção DateMalformedPeriodStringException quando o parâmetro `isostr` não puder ser interpretado como um período ISO 8601 válido. Antes do PHP 8.3, isto era uma Exception.

Registro de Alterações

Versão	Descrição
8.4.0	A assinatura que usa `isostr` foi descontinuada, use DatePeriod::createFromISO8601String() em seu lugar.
8.3.0	Agora dispara DateMalformedPeriodStringException em vez de Exception.
8.2.0	A constante **`DatePeriod::INCLUDE_END_DATE`** foi incluída.
7.2.19, 7.3.6, 7.4.0	O parâmetro `recurrences` agora deve ser maior que `0`.

Examples

Exemplo #1 Exemplos de DatePeriod

```php
<?php
$inicio = new DateTime('2012-07-01');
$intervalo = new DateInterval('P7D');
$fim = new DateTime('2012-07-31');
$recorrencias = 4;
$iso = 'R4/2012-07-01T00:00:00Z/P7D';

// Todos estes períodos são equivalentes.
$periodo = new DatePeriod($inicio, $intervalo, $recorrencias);
$periodo = new DatePeriod($inicio, $intervalo, $fim);
```

```php
$periodo = new DatePeriod($iso);

// Ao iterar sobre o objeto DatePeriod, todas as
// datas recorrentes dentro do período são exibidas.
foreach ($periodo as $data) {
    echo $data->format('Y-m-d')."\n";
}
```

O exemplo acima produzirá:

```
Deprecated: Calling DatePeriod::__construct(string $isostr, int $options = 0) is deprecated, use
DatePeriod::createFromISO8601String() instead in script on line 11
2012-07-01
2012-07-08
2012-07-15
2012-07-22
2012-07-29
```

Exemplo #2 Exemplo de DatePeriod com `DatePeriod::EXCLUDE_START_DATE`

```php
<?php
$inicio = new DateTime('2012-07-01');
$intervalo = new DateInterval('P7D');
$fim = new DateTime('2012-07-31');

$periodo = new DatePeriod($inicio, $intervalo, $fim,
                          DatePeriod::EXCLUDE_START_DATE);

// Ao iterar sobre o objeto DatePeriod, todas as
// datas recorrentes dentro do período são exibidas.
// Note que, neste caso, 2012-07-01 não é mostrada.
foreach ($periodo as $data) {
    echo $data->format('Y-m-d')."\n";
}
```

O exemplo acima produzirá:

```
2012-07-08
2012-07-15
2012-07-22
2012-07-29
```

Exemplo #3 Exemplo de DatePeriod mostrando todas as últimas quintas-feiras do ano

```php
<?php
$inicio = new DateTime('2021-12-31');
$fim = new DateTime('2022-12-31 23:59:59');

$intervalo = DateInterval::createFromDateString('last thursday of next month');
$periodo = new DatePeriod($inicio, $intervalo, $fim, DatePeriod::EXCLUDE_START_DATE);

foreach ($periodo as $data) {
    echo $data->format('l Y-m-d'), "\n";
}
```

O exemplo acima produzirá:

```
Thursday 2022-01-27
Thursday 2022-02-24
Thursday 2022-03-31
Thursday 2022-04-28
Thursday 2022-05-26
Thursday 2022-06-30
Thursday 2022-07-28
Thursday 2022-08-25
Thursday 2022-09-29
Thursday 2022-10-27
Thursday 2022-11-24
Thursday 2022-12-29
```

Notas

Npumeros de repetições não vinculados como especificado pela ISO 8601 seção 4.5 "Recurring time interval" não são suportados, ex.: nem passando passando `"R/..."` como parâmetro `isostr` nem passando `null` como parâmetro `end` funcionaria.

DatePeriod::createFromISO8601String

DatePeriod::createFromISO8601String

(PHP 8 >= 8.3.0)

DatePeriod::createFromISO8601String — Cria um novo objeto DatePeriod a partir de uma string ISO8601

Descrição

public static function **DatePeriod::createFromISO8601String**(string `$specification`, int `$options` = 0): static

Cria um novo objeto DatePeriod a partir de uma string ISO8601, como especificado no parâmetro `specification`.

Parâmetros

`specification`

Um subconjunto da » especificação ISO 8601 de intervalos repetitivos.

Um exemplo de uma especificação de intervalo ISO 8601 aceitável é `R5/2008-03-01T13:00:00Z/P1Y2M10DT2H30M`, que especifica:

- 5 iterações (`R5/`)
- Inicia em `2008-03-01T13:00:00Z`.
- Cada iteração ocorrem em um intervalo de 1 ano, 2 meses, 10 dias, 2 horas e 30 minutos (`/P1Y2M10DT2H30M`).

Exemplo de recursos de especificação de intervalos ISO 8601 que o PHP não suporta são:

1. zero ocorrências (`R0/`)
2. deslocamento de fuso horário que não seja UTC (`Z`), como em `+02:00`.

`options`

Um campo de bits que pode ser utilizado para controlar certos comportamentos com datas iniciais e finais.

Com `DatePeriod::EXCLUDE_START_DATE`, pode-se excluir a data inicial do conjunto de datas recorrentes dentro do período.

Com `DatePeriod::INCLUDE_END_DATE`, pode-se incluir a data final no conjunto de datas recorrentes dentro do período.

Valor Retornado

Retorna um novo objeto DatePeriod.

Objetos DatePeriod criados com este método podem ser usador como um iterador para gerar um número de objetos DateTimeImmutable.

Erros/Exceções

Lanã uma DateMalformedPeriodStringException quando o parâmetro `specification` não puder ser analisado como um período ISO 8601 válido.

Examples

Exemplo #1 Exemplo de DatePeriod::createFromISO8601String

```php
<?php
$iso = 'R4/2023-07-01T00:00:00Z/P7D';
$period = DatePeriod::createFromISO8601String($iso);

// Iterando pelo objeto DatePeriod, todas as
// datas recorrentes dentro do período são exibidas.
foreach ($period as $date) {
    echo $date->format('Y-m-d'), "\n";
}
```

O exemplo acima produzirá:

```
2023-07-01
2023-07-08
2023-07-15
2023-07-22
2023-07-29
```

DatePeriod::getDateInterval

DatePeriod::getDateInterval

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getDateInterval — Obtém o intervalo

Descrição

Estilo orientado a objetos

public function **DatePeriod::getDateInterval**(): DateInterval

Obtém um object da classe DateInterval representando o intervalo usado para o período.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um object DateInterval.

Examples

Exemplo #1 Exemplo de DatePeriod::getDateInterval()

```php
<?php
$period = DatePeriod::createFromIso8601String('R7/2016-05-16T00:00:00Z/P1D');
$interval = $period->getDateInterval();
echo $interval->format('%d day');
```

O exemplo acima produzirá:

```
1 day
```

Veja também

- DatePeriod::getStartDate() - Obtém a data inicial
- DatePeriod::getEndDate() - Obtém a data final

DatePeriod::getEndDate

DatePeriod::getEndDate

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getEndDate — Obtém a data final

Descrição

Estilo orientado a objetos

public function **DatePeriod::getEndDate**(): ?DateTimeInterface

Obtém a data final do período.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `null` se o DatePeriod não tiver uma data final. Por exemplo, quando inicializado com o parâmetro `recurrences` ou `isostr` sem uma data final.

Retorna um object DateTimeImmutable quando o DatePeriod é inicializado com um object DateTimeImmutable no parâmetro `end`.

Caso contrário, retorna um clone do object DateTime representando a data final.

Examples

Exemplo #1 Exemplo de DatePeriod::getEndDate()

```php
<?php
$period = new DatePeriod(
    new DateTime('2016-05-16T00:00:00Z'),
    new DateInterval('P1D'),
    new DateTime('2016-05-20T00:00:00Z')
);
$start = $period->getEndDate();
echo $start->format(DateTime::ISO8601);
```

Os exemplos acima produzirão:

```
2016-05-20T00:00:00+0000
```

Exemplo #2 Exemplo de DatePeriod::getEndDate() sem uma data final

```php
<?php
$period = new DatePeriod(
    new DateTime('2016-05-16T00:00:00Z'),
    new DateInterval('P1D'),
    7
);
var_dump($period->getEndDate());
```

O exemplo acima produzirá:

```
NULL
```

Veja também

- DatePeriod::getStartDate() - Obtém a data inicial
- DatePeriod::getDateInterval() - Obtém o intervalo

DatePeriod::getRecurrences

DatePeriod::getRecurrences

(PHP 7 >= 7.2.17/7.3.4, PHP 8)

DatePeriod::getRecurrences — Obtém o número de recorrências

Descrição

Estilo orientado a objetos

public function **DatePeriod::getRecurrences**(): ?int

Obtém o número de recorrências.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de recorrências como definido explicitamente pelo parâmetro `$recurrences` passado ao construtor da classe DatePeriod, ou `null` caso não tenha sido definido.

Examples

Exemplo #1 Diferentes valores para DatePeriod::getRecurrences()

```php
<?php
$start = new DateTime('2018-12-31 00:00:00');
$end   = new DateTime('2021-12-31 00:00:00');
$interval = new DateInterval('P1M');
$recurrences = 5;

// recorrências definidas explicitamente através do construtor
$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::EXCLUDE_START_DATE);
echo $period->getRecurrences(), "\n";

$period = new DatePeriod($start, $interval, $recurrences);
echo $period->getRecurrences(), "\n";

$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::INCLUDE_END_DATE);
echo $period->getRecurrences(), "\n\n";

// recorrências não definidas no construtor
$period = new DatePeriod($start, $interval, $end);
var_dump($period->getRecurrences());

$period = new DatePeriod($start, $interval, $end, DatePeriod::EXCLUDE_START_DATE);
var_dump($period->getRecurrences());
```

O exemplo acima produzirá:

```
5
5
5

NULL
NULL
```

Veja também

- DatePeriod::$recurrences

DatePeriod::getStartDate

DatePeriod::getStartDate

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getStartDate — Obtém a data inicial

Descrição

Estilo orientado a objetos

public function **DatePeriod::getStartDate**(): DateTimeInterface

Obtém a data inicial do período.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um object DateTimeImmutable quando o DatePeriod é inicializado com um object DateTimeImmutable no parâmetro `start`.

Caso contrário, retorna um object DateTime.

Examples

Exemplo #1 Exemplo de DatePeriod::getStartDate()

```php
<?php
$period = DatePeriod::createFromIso8601String('R7/2016-05-16T00:00:00Z/P1D');
$start = $period->getStartDate();
echo $start->format(DateTime::ISO8601);
```

O exemplo acima produzirá:

```
2016-05-16T00:00:00+0000
```

Veja também

- DatePeriod::getEndDate() - Obtém a data final
- DatePeriod::getDateInterval() - Obtém o intervalo

Funções de Data/Horário

Índice

checkdate

checkdate

(PHP 4, PHP 5, PHP 7, PHP 8)

checkdate — Valida uma data gregoriana

Descrição

function **checkdate**(int `$month`, int `$day`, int `$year`): bool

Verifica a validade da data formada pelos argumentos. Uma data é considerada válida se cada parâmetro é adequadamente definido.

Parâmetros

`month`

O mês entre 1 e 12.

`day`

O dia está dentro do número permitido de dias para o mês (`month`). Anos (`year`) bissextos são levados em consideração.

`year`

O ano está entre 1 e 32767, inclusive.

Valor Retornado

Retorna `true` se a data informada é valida; caso contrário retorna `false`.

Examples

Exemplo #1 Exemplo de checkdate()

```php
<?php
var_dump(checkdate(12, 31, 2000));
var_dump(checkdate(2, 29, 2001));
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
```

Veja também

- mktime() - Obtém um timestamp Unix de uma data
- strtotime() - Interpreta qualquer descrição textual de data/horário em inglês para timestamp Unix

date

date

(PHP 4, PHP 5, PHP 7, PHP 8)

date — Formata um timestamp Unix

Descrição

function **date**(string `$format`, ?int `$timestamp = null`): string

Retorna uma string de acordo com a string de formato informada usando o inteiro `timestamp` (timestamp Unix) informado, ou o horário local atual se nenhum timestamp for informado. Em outras palavras, o parâmetro `timestamp` é opcional e padronizado para o valor de time().

> **Aviso**
>
> Timestamps Unix não lidam com fusos horários. Use a classe DateTimeImmutable e seu método de formatação DateTimeInterface::format() para formatar informações de data/horário com um fuso horário anexado.

Parâmetros

`format`

Formato aceito pelo DateTimeInterface::format().

> **Nota** :
>
> **date()** irá sempre gerar `000000` para os microssegundos já que ele usa um parâmetro int, enquanto que DateTimeInterface::format() suporta microssegundos se um objeto do tipo DateTimeInterface foi criado com microssegundos.

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna uma string formatada de data.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um `E_WARNING` se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplos da função date()

```php
<?php
// Modifica o fuso horário a ser utilizado.
date_default_timezone_set('UTC');


// Exibe algo coisa como: Monday
echo date("l") . "\n";

// Exibe algo como: Monday 8th of August 2005 03:12:46 PM
echo date('l jS \of F Y h:i:s A') . "\n";

// Exibe: July 1, 2000 is on a Saturday
echo "July 1, 2000 is on a " . date("l", mktime(0, 0, 0, 7, 1, 2000)) . "\n";

/* Utiliza as constantes do parâmetro de formato */
// Exibe algo como: Mon, 15 Aug 2005 15:12:46 UTC
echo date(DATE_RFC2822) . "\n";

// Exibe algo como: 2000-07-01T00:00:00+00:00
echo date(DATE_ATOM, mktime(0, 0, 0, 7, 1, 2000));
```

Para evitar que um caractere reconhecido na string de formatação seja expandido, prefixe-o com uma barra invertida. Se o caractere com a barra invertida já for uma sequência especial, pode ser necessário também prefixá-lo com a barra invertida.

Exemplo #2 Caracteres de escape em date()

```php
<?php
// Exibe algo como: Wednesday the 15th
```

```
echo date("l \\t\h\e jS");
```

Alguns exemplos de formatação de **date()**. Observe que qualquer outro caractere deve ser escapado, pois qualquer um que possua um significado especial produzirá resultados indesejáveis, e outros caracteres podem assumir significado em futuras versões do PHP. Quando usar escape, certifique-se de usar aspas simples para evitar que caracteres como \n virem novas linhas.

Exemplo #3 Formatação com a função date()

```
<?php
// Assumindo que hoje é dia 10 de março de 2001, 17:16:18, e que estamos no
// fuso horário Mountain Standard Time (MST)
date_default_timezone_set("America/Phoenix");

$today = date("F j, Y, g:i a") . "\n";                 // March 10, 2001, 5:16 pm
$today = date("m.d.y") . "\n";                         // 03.10.01
$today = date("j, n, Y") . "\n";                       // 10, 3, 2001
$today = date("Ymd") . "\n";                           // 20010310
$today = date('h-i-s, j-m-y, it is w Day') . "\n";     // 05-16-18, 10-03-01, 1631 1618 6 Satpm01
$today = date('\i\t \i\s \t\h\e jS \d\a\y.') . "\n";   // it is the 10th day.
$today = date("D M j G:i:s T Y") . "\n";               // Sat Mar 10 17:16:18 MST 2001
$today = date('H:m:s \m \i\s\ \m\o\n\t\h') . "\n";     // 17:03:18 m is month
$today = date("H:i:s") . "\n";                         // 17:16:18
$today = date("Y-m-d H:i:s") . "\n";                   // 2001-03-10 17:16:18 (o formato DATETIME do MySQL)
```

Para formatar datas em outros idiomas, IntlDateFormatter::format() pode ser utilizada em vez de **date()**.

Notas

> **Nota** :
>
> Para gerar um timestamp de uma string da representação da data, pode-se utilizar as funções strtotime(). Adicionalmente, alguns banco de dados tem funções para converter os formatos de data para timestamps (como a função » UNIX_TIMESTAMP do MySQL).

> **Dica**
>
> O timestamp do início da requisição está disponível na variável *$_SERVER['REQUEST_TIME']*.

Veja também

- DateTimeImmutable::__construct() - Retorna um novo objeto DateTimeImmutable
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- gmdate() - Formata uma data/horário GMT/CUT
- idate() - Formata uma parte de uma data/horário local como um inteiro
- getdate() - Recupera informações de data/horário
- getlastmod() - Obtém o horário da última modificação na página
- mktime() - Obtém um timestamp Unix de uma data
- IntlDateFormatter::format() - Formata data/horário como uma string
- time() - Retorna o timestamp Unix atual
- Constantes Predefinidas da Classe DateTime

date_add

date_add

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_add — Sinônimo de DateTime::add()

Descrição

Esta função é um apelido para: DateTime::add()

date_create

date_create

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_create — Cria um novo objeto DateTime

Descrição

function **date_create**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTime|false

Esta é a versão procedural de DateTime::__construct().

Diferentemente do construtor de DateTime, irá retornar `false` em vez de uma exceção se a string `datetime` for inválida.

Parâmetros

Ver DateTimeImmutable::__construct.

Valor Retornado

Retorna uma nova instância de DateTime ou `false` em caso de falha

Veja também

- DateTimeImmutable::__construct() - Retorna um novo objeto DateTimeImmutable
- DateTimeImmutable::createFromFormat() - Interpreta um string de data/horário de acordo com o formato especificado
- DateTime::__construct() - Retorna um novo objeto DateTime

date_create_from_format

date_create_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_create_from_format — Sinônimo de DateTime::createFromFormat()

Descrição

Esta função é um apelido para: DateTime::createFromFormat()

date_create_immutable

date_create_immutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

date_create_immutable — Cria um novo objeto DateTimeImmutable

Descrição

function **date_create_immutable**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTimeImmutable|false

Esta é a versão procedural de DateTimeImmutable::__construct().

Diferentemente do construtor de DateTimeImmutable, irá retornar `false` em vez de uma exceção se a string `datetime` for inválida.

Parâmetros

Ver DateTimeImmutable::__construct.

Valor Retornado

Retorna uma nova instância de DateTimeImmutable ou `false` em caso de falha

Veja também

- DateTimeImmutable::createFromFormat() - Interpreta um string de data/horário de acordo com o formato especificado

date_create_immutable_from_format

date_create_immutable_from_format

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

date_create_immutable_from_format — Sinônimo de DateTimeImmutable::createFromFormat()

Descrição

Esta função é um apelido para: DateTimeImmutable::createFromFormat()

date_date_set

date_date_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_date_set — Sinônimo de DateTime::setDate()

Descrição

Esta função é um apelido para: DateTime::setDate()

date_default_timezone_get

date_default_timezone_get

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

date_default_timezone_get — Retorna o fuso horário padrão usado por todas as funções de data/horário em um script

Descrição

function **date_default_timezone_get**(): string

Em ordem de preferência, essa função retorna o fuso horário padrão:

- Lendo o fuso horário configurado pela função date_default_timezone_set() (se existir)
- Lendo o valor da opção date.timezone do arquivo INI (se configurada)

Se nenhuma das opções acima tiver êxito, a função **date_default_timezone_get()** retornará o fuso horário padrão UTC.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string.

Examples

Exemplo #1 Lendo o fuso horário padrão

```php
<?php
date_default_timezone_set('Europe/London');

if (date_default_timezone_get()) {
    echo 'date_default_timezone_set: ' . date_default_timezone_get() . "\n";
}

if (ini_get('date.timezone')) {
    echo 'date.timezone: ' . ini_get('date.timezone');
}
```

O exemplo acima produzirá algo semelhante a:

```
date_default_timezone_set: Europe/London
date.timezone: Europe/London
```

Exemplo #2 Lendo a abreviação de um fuso horário

```php
<?php
date_default_timezone_set('America/Los_Angeles');
$ts = mktime(0, 0, 0, 1, 1, 2024);
echo date_default_timezone_get() . ' => ' . date('e') . ' => ' . date('T', $ts);
```

O exemplo acima produzirá:

```
America/Los_Angeles => America/Los_Angeles => PST
```

Veja também

- date_default_timezone_set() - Configura o fuso horário padrão utilizado por todas as funções de data/horário em um script
- Lista de Fusos Horários Suportados

date_default_timezone_set

date_default_timezone_set

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

date_default_timezone_set — Configura o fuso horário padrão utilizado por todas as funções de data/horário em um script

Descrição

function **date_default_timezone_set**(string `$timezoneId`): bool

date_default_timezone_set() configura o fuso horário padrão utilizado por todas as funções de data e hora.

Em vez de utilizar essa função para definir o fuso horário padrão no script, a configuração INI date.timezone também pode ser usada para configurar o fuso horário padrão.

Parâmetros

`timezoneId`

O identificador do fuso horário, como `UTC`, `Africa/Lagos`, `Asia/Hong_Kong`, ou `Europe/Lisbon`. A lista de identificadores válidos está disponível em Lista de Fusos Horários Suportados.

Valor Retornado

A função retorna **`false`** se o `timezoneId` não for válido, ou **`true`** caso contrário.

Examples

Exemplo #1 Lendo o fuso horário padrão

```php
<?php
date_default_timezone_set('America/Los_Angeles');

$script_tz = date_default_timezone_get();
$ini_tz = ini_get('date.timezone');

if (strcmp($script_tz, $ini_tz)){
    echo 'Fuso horário do script é diferente do definido no INI.';
} else {
    echo 'Fuso horário do script corresponde ao definido no INI.';
}
```

Veja também

- date_default_timezone_get() - Retorna o fuso horário padrão usado por todas as funções de data/horário em um script
- Lista de Fusos Horários Suportados

date_diff

date_diff

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_diff — Sinônimo de DateTime::diff()

Descrição

Esta função é um apelido para: DateTime::diff()

date_format

date_format

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_format — Sinônimo de DateTime::format()

Descrição

Esta função é um apelido para: DateTime::format()

date_get_last_errors

date_get_last_errors

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_get_last_errors — Sinônimo de DateTimeImmutable::getLastErrors()

Descrição

Esta função é um apelido para: DateTimeImmutable::getLastErrors()

date_interval_create_from_date_string

date_interval_create_from_date_string

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_interval_create_from_date_string — Sinônimo de DateInterval::createFromDateString()

Descrição

Esta função é um apelido para: DateInterval::createFromDateString()

date_interval_format

date_interval_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_interval_format — Sinônimo de DateInterval::format()

Descrição

Esta função é um apelido para: DateInterval::format()

date_isodate_set

date_isodate_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_isodate_set — Sinônimo de DateTime::setISODate()

Descrição

Esta função é um apelido para: DateTime::setISODate()

date_modify

date_modify

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_modify — Sinônimo de DateTime::modify()

Descrição

Esta função é um apelido para: DateTime::modify()

date_offset_get

date_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_offset_get — Sinônimo de DateTime::getOffset()

Descrição

Esta função é um apelido para: DateTime::getOffset().

date_parse

date_parse

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_parse — Retorna um array associativo com informações detalhadas da data/horário informada

Descrição

function **date_parse**(string `$datetime`): array

A função **date_parse()** analisa a string `datetime` informada conforme as mesmas regras de strtotime() e DateTimeImmutable::__construct(). Ao invés de retornar um timestamp Unix (com strtotime()) ou objeto DateTimeImmutable (com a função DateTimeImmutable::__construct()), ela retorna um array associativo com a informação que ela poderia detectar na string `datetime` fornecida.

Se nenhuma informação sobre um certo grupo de elementos puder ser encontrada, estes elementos do array serão configurados para **`false`** ou omitidos. Se necessário para construção de um timestamp ou objeto DateTimeImmutable a partir da mesma string `datetime`, mais campos podem ser configurados para um valor não-**`false`**. Veja os exemplos para casos onde isso acontece..

Parâmetros

`datetime`

Data/horário no formato aceito pela função DateTimeImmutable::__construct().

Valor Retornado

Retorna um array com informações da data/horário analisada.

O array de retorno tem chaves para `year`, `month`, `day`, `hour`, `minute`, `second`, `fraction`, e `is_localtime`.

Se `is_localtime` estiver presente então `zone_type` indica o tipo do fuso horário. Para o tipo `1` (deslocamento UTC), os campos `zone` e `is_dst` são incluídos; para o tipo `2` (abreviação), os campos `tz_abbr` e `is_dst` são incluídos; e para tipo `3` (identificador de fuso), os campos `tz_abbr` e `tz_id` são incluídos.

Se elementos de tempo relativo estiverem presentes na string `datetime` como `+3 days`, então o array retornado incluirá um array aninhado com a chave `relative`. Este array então conterá as chaves `year`, `month`, `day`, `hour`, `minute`, `second`, e se necessário `weekday` e `weekdays`, dependendo da string que foi passada.

O array inclui os campos `warning_count` e `warnings`. O primeiro indica quantos avisos ocorreram. As chaves dos elementos do array `warnings` indicam as posições na string `datetime` onde os avisos ocorreram, com o valor da string descrevendo o aviso em si.

O array também contém os campos `error_count` e `errors`. O primeiro indica quantos erros foram encontrados. As chaves dos elementos do array `errors` indicam posição na string `datetime` onde os erros ocorreram, com o valor da string descrevendo o erro em si.

> **Aviso**
>
> O número de elementos do arrays `warnings` ou `errors` pode ser menor que `warning_count` ou `error_count` se eles ocorreram na mesma posição.

Erros/Exceções

Se o formato da data/horário contiver erro, o elemento 'errors' conterá as mensagens de erro.

Registro de Alterações

Versão	Descrição
7.2.0	O elemento `zone` do array de retorno agora representa segundos no lugar de minutos, e seu sinal é invertido. Por exemplo, `-120` é agora `7200`.

Examples

Exemplo #1 Um exemplo da função date_parse()com um string `datetime` compreensivo

```php
<?php
var_dump(date_parse("2006-12-12 10:00:00.5"));
```

O exemplo acima produzirá:

```
array(12) {
  ["year"]=>
  int(2006)
  ["month"]=>
  int(12)
  ["day"]=>
  int(12)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
  ["fraction"]=>
  float(0.5)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
}
```

Os elementos de fuso horário só aparecem se eles forem incluídos no parâmetro `datetime`. Neste caso sempre haverá um elemento `zone_type` e alguns mais dependendo do seu valor.

Exemplo #2 Exemplo da função date_parse() com informação de abreviação de fuso horário

```php
<?php
var_dump(date_parse("June 2nd, 2022, 10:28:17 BST"));
```

O exemplo acima produzirá:

```
array(16) {
  ["year"]=>
  int(2022)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(28)
  ["second"]=>
  int(17)
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(true)
  ["zone_type"]=>
  int(2)
  ["zone"]=>
  int(0)
  ["is_dst"]=>
  bool(true)
  ["tz_abbr"]=>
  string(3) "BST"
}
```

Exemplo #3 Exemplo da função date_parse() com informação de identificador de fuso horário

```php
<?php
var_dump(date_parse("June 2nd, 2022, 10:28:17 Europe/London"));
```

O exemplo acima produzirá:

```
array(14) {
  ["year"]=>
  int(2022)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(28)
  ["second"]=>
  int(17)
```

```
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(true)
  ["zone_type"]=>
  int(3)
  ["tz_id"]=>
  string(13) "Europe/London"
}
```

Se uma string mais enxuta de `datetime` for analisada, menos informação estará disponível. Neste exemplo, todas as partes de horário são retornadas como `false`.

Exemplo #4 Função date_parse() com um string minimizado

```php
<?php
var_dump(date_parse("June 2nd, 2022"));
```

O exemplo acima produzirá:

```
array(12) {
  ["year"]=>
  int(2022)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  bool(false)
  ["minute"]=>
  bool(false)
  ["second"]=>
  bool(false)
  ["fraction"]=>
  bool(false)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
}
```

Formatos relativos não influenciam valores interpretados de formatos absolutos, mas são analisados no elemento "relative".

Exemplo #5 date_parse() com formatos relativos

```php
<?php
var_dump(date_parse("2006-12-12 10:00:00.5 +1 week +1 hour"));
```

O exemplo acima produzirá:

```
array(13) {
  ["year"]=>
  int(2006)
  ["month"]=>
  int(12)
  ["day"]=>
  int(12)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
  ["fraction"]=>
  float(0.5)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
  ["relative"]=>
  array(6) {
    ["year"]=>
    int(0)
    ["month"]=>
    int(0)
    ["day"]=>
    int(7)
    ["hour"]=>
    int(1)
    ["minute"]=>
    int(0)
    ["second"]=>
    int(0)
  }
}
```

Algumas strings, como `Thursday` por exemplo, irão configurar a parte de horário da string para `0`. Se `Thursday` for passado para a função DateTimeImmutable::__construct(), ela também teria resultado em hora, minuto, segundo e fração configurados para `0`. No exemplo abaixo, o elemento do ano também é configurado como `false`.

Exemplo #6 Função date_parse() com efeitos colaterais

```php
<?php
var_dump(date_parse("Thursday, June 2nd"));
```

O exemplo acima produzirá:

```
array(13) {
  ["year"]=>
  bool(false)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(0)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
  ["relative"]=>
  array(7) {
    ["year"]=>
    int(0)
    ["month"]=>
    int(0)
    ["day"]=>
    int(0)
    ["hour"]=>
    int(0)
    ["minute"]=>
    int(0)
    ["second"]=>
    int(0)
    ["weekday"]=>
    int(4)
  }
}
```

Veja também

- date_parse_from_format() - Retorna informações sobre uma data informada de acordo com um formato para analisar um `datetime` com um formato específico
- checkdate() - Valida uma data gregoriana para validação da data do calendário gregoriano
- getdate() - Recupera informações de data/horário

date_parse_from_format

date_parse_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_parse_from_format — Retorna informações sobre uma data informada de acordo com um formato

Descrição

function **date_parse_from_format**(string `$format`, string `$datetime`): array

Retorna um array associativo com informação detalhada sobre a data/horário informada.

Parâmetros

`format`

Para documentação sobre como o parâmetro `format` é usado, favor se referir à documentação da função DateTimeImmutable::createFromFormat(). As mesmas regras se aplicam.

`datetime`

String que representa a data/horário.

Valor Retornado

Retorna um array associativo com informação detalhada da data/horário informada.

O array de retorno tem chaves para `year`, `month`, `day`, `hour`, `minute`, `second`, `fraction`, e `is_localtime`.

Se `is_localtime` estiver presente então `zone_type` indica o tipo do fuso horário. Para o tipo `1` (deslocamento UTC), os campos `zone` e `is_dst` são incluídos; para o tipo `2` (abreviação), os campos `tz_abbr` e `is_dst` são incluídos; e para tipo `3` (identificador de fuso), os campos `tz_abbr` e `tz_id` são incluídos.

O array inclui os campos `warning_count` e `warnings`. O primeiro indica quantos avisos ocorreram. As chaves dos elementos do array `warnings` indicam as posições na string `datetime` onde os avisos ocorreram, com o valor da string descrevendo o aviso em si. Um exemplo abaixo mostra tal aviso.

O array também contém os campos `error_count` e `errors`. O primeiro indica quantos erros foram encontrados. As chaves dos elementos do array `errors` indicam posição na string `datetime` onde os erros ocorreram, com o valor da string descrevendo o erro em si. Um exemplo abaixo mostra tal erro.

> **Aviso**
>
> O número de elementos do arrays `warnings` ou `errors` pode ser menor que `warning_count` ou `error_count` se eles ocorreram na mesma posição.

Erros/Exceções

Esta função dispara a exceção ValueError quando o parâmetro `datetime` contém bytes nulos.

Registro de Alterações

Versão	Descrição
8.0.21, 8.1.8, 8.2.0	Agora lança ValueError quando bytes nulos são passados por `datetime`, o que antes era silenciosamente ignorado.
7.2.0	O elemento `zone` do array retornado agora representa segundos em vez de minutos, e seu sinal é invertido. Por exemplo, `-120` agora é `7200`.

Examples

Exemplo #1 Exemplo da função date_parse_from_format()

```php
<?php
$date = "6.1.2009 13:00+01:00";
print_r(date_parse_from_format("j.n.Y H:iP", $date));
```

O exemplo acima produzirá:

```
Array
(
    [year] => 2009
    [month] => 1
    [day] => 6
    [hour] => 13
    [minute] => 0
    [second] => 0
    [fraction] => 0
```

```
    [warning_count] => 0
    [warnings] => Array
        (
        )

    [error_count] => 0
    [errors] => Array
        (
        )

    [is_localtime] => 1
    [zone_type] => 1
    [zone] => 3600
    [is_dst] =>
)
```

Exemplo #2 Exemplo da função date_parse_from_format() com avisos

```php
<?php
$date = "26 August 2022 22:30 pm";
$parsed = date_parse_from_format("j F Y G:i a", $date);

echo "Contagem de avisos: ", $parsed['warning_count'], "\n";
foreach ($parsed['warnings'] as $position => $message) {
    echo "\tNa posição {$position}: {$message}\n";
}
```

O exemplo acima produzirá:

```
Contagem de avisos: 1
Na posição 23: The parsed time was invalid
```

Exemplo #3 Exemplo da função date_parse_from_format() com erros

```php
<?php
$date = "26 August 2022 CEST";
$parsed = date_parse_from_format("j F Y H:i", $date);

echo "Contagem de erros: ", $parsed['error_count'], "\n";
foreach ($parsed['errors'] as $position => $message) {
    echo "\tNa posição {$position}: {$message}\n";
}
```

O exemplo acima produzirá:

```
Contagem de erros: 3
Na posição 15: A two digit hour could not be found
Na posição 19: Not enough data available to satisfy format
```

Veja também

- DateTimeImmutable::createFromFormat() - Interpreta um string de data/horário de acordo com o formato especificado
- checkdate() - Valida uma data gregoriana

date_sub

date_sub

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_sub — Sinônimo de DateTime::sub()

Descrição

Esta função é um apelido para: DateTime::sub()

Veja também

- DateTimeImmutable::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos
- DateTime::sub() - Subtrai uma quantidade de dias, meses, anos, horas, minutos e segundos de um objeto DateTime

date_sun_info

date_sun_info

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

date_sun_info — Retorna um array com informações sobre pôr/nascer do sol e o início/fim do dia

Descrição

function **date_sun_info**(int `$timestamp`, float `$latitude`, float `$longitude`): array

Parâmetros

`timestamp`

Timestamp Unix.

`latitude`

Latitude em graus.

`longitude`

Longitude em graus.

Valor Retornado

Retorna um cuja estrutura é detalhada na lista abaixo:

`sunrise`

O timestamp do nascer do sol (ângulo em relação ao zênite = 90°35').

`sunset`

O timestamp do pôr do sol (ângulo em relação ao zênite = 90°35').

`transit`

O timestamp quando o sol está em seu zênite, ou seja, atingiu seu ponto mais alto.

`civil_twilight_begin`

O início do amanhecer civil (ângulo em relação ao zênite = 96°). Termina no `sunrise`.

`civil_twilight_end`

O final do crepúsculo civil (ângulo em relação ao zênite = 96°). Começa no `sunset`.

`nautical_twilight_begin`

O início do amanhecer náutico (ângulo em relação ao zênite = 102°). Termina no `civil_twilight_begin`.

`nautical_twilight_end`

O final do crepúsculo náutico (ângulo em relação ao zênite = 102°). Começa no `civil_twilight_end`.

`astronomical_twilight_begin`

O início do amanhecer astronômico (ângulo em relação ao zênite = 108°). Termina no `nautical_twilight_begin`.

`astronomical_twilight_end`

O final do crepúsculo astronômico (ângulo em relação ao zênite = 108°). Começa no `nautical_twilight_end`.

Os valores dos elementos do array podem ser timestamps UNIX, `false` se o sol estiver abaixo do respectivo ângulo durante todo o dia, ou `true` se o sol estiver acima do respectivo ângulo durante todo o dia.

Registro de Alterações

Versão	Descrição
7.2.0	Os cálculos foram corrigidos com base na meia-noite local em vez do meio-dia local, o que muda os resultados um pouco.

Examples

Exemplo #1 Um exemplo de date_sun_info()

```php
<?php
date_default_timezone_set('Asia/Jerusalem');
$sun_info = date_sun_info(strtotime("2006-12-12"), 31.7667, 35.2333);
foreach ($sun_info as $key => $val) {
    echo "$key: " . date("H:i:s", $val) . "\n";
}
```

O exemplo acima produzirá:

```
sunrise: 06:29:21
sunset: 16:36:00
transit: 11:32:41
civil_twilight_begin: 06:02:36
civil_twilight_end: 17:02:45
nautical_twilight_begin: 05:32:14
nautical_twilight_end: 17:33:08
astronomical_twilight_begin: 05:02:31
astronomical_twilight_end: 18:02:51
```

Exemplo #2 Noite polar, com algum processamento

```php
<?php
$tz = new \DateTimeZone('America/Anchorage');

$si = date_sun_info(strtotime("2022-12-21"), 70.21, -148.51);
foreach ($si as $key => $value) {
    echo
        match ($value) {
            true => 'always',
            false => 'never',
            default => date_create("@{$value}")->setTimeZone($tz)->format( 'H:i:s T' ),
        },
        ": {$key}",
        "\n";
}
```

O exemplo acima produzirá:

```
never: sunrise
never: sunset
12:52:18 AKST: transit
10:53:19 AKST: civil_twilight_begin
14:51:17 AKST: civil_twilight_end
09:01:47 AKST: nautical_twilight_begin
16:42:48 AKST: nautical_twilight_end
07:40:47 AKST: astronomical_twilight_begin
18:03:49 AKST: astronomical_twilight_end
```

Exemplo #3 Sol da meia-noite (Tromsø, Norway)

```php
<?php
$si = date_sun_info(strtotime("2022-06-26"), 69.68, 18.94);
print_r($si);
```

O exemplo acima produzirá:

```
Array
(
    [sunrise] => 1
    [sunset] => 1
    [transit] => 1656240426
    [civil_twilight_begin] => 1
    [civil_twilight_end] => 1
    [nautical_twilight_begin] => 1
    [nautical_twilight_end] => 1
    [astronomical_twilight_begin] => 1
    [astronomical_twilight_end] => 1
)
```

Exemplo #4 Calculando a duração do dia (Kiev)

```php
<?php
$si = date_sun_info(strtotime('2022-08-26'), 50.45, 30.52);
$diff = $si['sunset'] - $si['sunrise'];
echo "Duração do dia: ",
    floor($diff / 3600), "h ",
    floor(($diff % 3600) / 60), "m\n";
```

O exemplo acima produzirá:

```
Duração do dia: 13h 53m
```

date_sunrise

date_sunrise

(PHP 5, PHP 7, PHP 8)

date_sunrise — Retorna a hora do nascer do sol para um dia e uma localicação informados

> **Aviso**

> Esta função foi *DESCONTINUADA* desde o PHP 8.1.0. Confiar nesta função é altamente desencorajado. Use date_sun_info() no lugar.

Descrição

```
#[\Deprecated]
function date_sunrise(
    int $timestamp,
    int $returnFormat = SUNFUNCS_RET_STRING,
    ?float $latitude = null,
    ?float $longitude = null,
    ?float $zenith = null,
    ?float $utcOffset = null
): string|int|float|false
```

A função **date_sunrise()** retorna a hora do nascer do sol para um dia (especificado como um `timestamp`) e uma localização informados.

Parâmetros

`timestamp`

O `timestamp` do nascer do sol do dia desejado.

`returnFormat`

Constantes de `returnFormat`

Constante	Descrição	Exemplo
SUNFUNCS_RET_STRING	Retorna o resultado como uma string	16:46
SUNFUNCS_RET_DOUBLE	Retorna o resultado como um float	16.78243132
SUNFUNCS_RET_TIMESTAMP	Retorna o resultado como um int (timestamp)	1095034606

`latitude`

Padronizado para o norte, passe um valor negativo para o sul. Veja também: date.default_latitude

`longitude`

Padronizado para o leste, passe um valor negativado para o oeste. Veja também: date.default_longitude

`zenith`

`zenith` é o ângulo entre o centro do sol e uma linha perpendicular à superfície da Terra. É padronizado para date.sunrise_zenith

Ângulos comuns em relação ao `zenith`

Ângulo	Descrição
90°50'	Nascer do sol: o ponto onde o sol se torna visível.
96°	Amanhecer civil: convencionado para significar o início do amanhecer.
102°	Amanhecer náutico: o ponto no qual o horizonte começa a ficar visível no mar.
108°	Amanhecer astronômico: o ponto no qual o sol começa a se tornar fonte de iluminação.

`utcOffset`

Especificado em horas. O parâmetro `utcOffset` é ignorado se `returnFormat` for **`SUNFUNCS_RET_TIMESTAMP`**.

Valor Retornado

Retorna o horário do nascer do sol no `returnFormat` especificado em caso de sucesso ou **`false`** em caso de falha. Um potencial motivo para falha é quando o sol não nasce, o que acontece dentro dos círculos polares durante parte do ano.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um **`E_WARNING`** se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.1.0	Esta função foi descontinuada em favor de date_sun_info().
8.0.0	`latitude`, `longitude`, `zenith` e `utcOffset` agora podem ser nulos.

Examples

Exemplo #1 Exemplo da função date_sunrise()

```php
<?php

/* Calcula o nascer do sol em Lisboa, Portugal
Latitude: 38.4 norte
Longitude: 9 oeste
Zênite ~= 90
deslocamento UTC: +01:00
*/

echo date("D M d Y"). ', horário do nascer do sol : ' .date_sunrise(time(), SUNFUNCS_RET_STRING, 38.4, -9, 90, 1);
```

O exemplo acima produzirá algo semelhante a:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 10
Deprecated: Function date_sunrise() is deprecated since 8.1, use date_sun_info() instead in script on line 10
Mon Dec 20 2004, horário do nascer do sol : 08:54
```

Exemplo #2 Sem nascer do sol

```php
<?php
$solstice = strtotime('2017-12-21');
var_dump(date_sunrise($solstice, SUNFUNCS_RET_STRING, 69.245833, -53.537222));
```

O exemplo acima produzirá:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 3
Deprecated: Function date_sunrise() is deprecated since 8.1, use date_sun_info() instead in script on line 3
bool(false)
```

Veja também

- date_sun_info() - Retorna um array com informações sobre pôr/nascer do sol e o início/fim do dia

date_sunset

date_sunset

(PHP 5, PHP 7, PHP 8)

date_sunset — Retorna a hora do pôr do sol para um dia e uma localização informados.

> **Aviso**
>
> Esta função foi *DESCONTINUADA* desde o PHP 8.1.0. Confiar nesta função é altamente desencorajado. Use date_sun_info() no lugar.

Descrição

```
#[\Deprecated]
function date_sunset(
    int $timestamp,
    int $returnFormat = SUNFUNCS_RET_STRING,
    ?float $latitude = null,
    ?float $longitude = null,
    ?float $zenith = null,
    ?float $utcOffset = null
): string|int|float|false
```

A função **date_sunset()** retorna o horário do pôr do sol para um dia (especificado como um `timestamp`) e uma localização informados.

Parâmetros

`timestamp`

O `timestamp` do dia do pôr do sol desejado.

`returnFormat`

Constantes de `returnFormat`

Constante	Descrição	Exemplo
SUNFUNCS_RET_STRING	Retorna o resultado como uma string	16:46
SUNFUNCS_RET_DOUBLE	Retorna o resultado como um float	16.78243132
SUNFUNCS_RET_TIMESTAMP	Retorna o resultado como um int (timestamp)	1095034606

`latitude`

Padronizado para o norte, passe um valor negativo para o sul. Veja também: date.default_latitude

`longitude`

Padronizado para o leste, passe um valor negativo para o oeste. Veja também: date.default_longitude

`zenith`

`zenith` é o ângulo entre o centro do sol e uma linha perpendicular à superfície da Terra. É padronizado para date.sunset_zenith

Ângulos comuns em relação ao `zenith`

Ângulo	Descrição
90°50'	Pôr do sol: o ponto onde o sol se torna invisível.
96°	Crepúsculo civil: convencionado para significar o final do crepúsculo.
102°	Crepúsculo náutico: o ponto no qual o horizonte termina de ficar visível no mar.
108°	Crepúsculo astronômico: o ponto no qual o sol termina se tornar fonte de iluminação.

`utcOffset`

Especificado em horas. O parâmetro `utcOffset` é ignorado se `returnFormat` for **`SUNFUNCS_RET_TIMESTAMP`**.

Valor Retornado

Retorna o horário do pôr do sol no `returnFormat` especificado em caso de sucesso ou **`false`** em caso de falha. Um potencial motivo para falha é quando o sol não se põe, o que acontece dentro dos círculos polares durante parte do ano.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um **`E_WARNING`** se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.1.0	Esta função foi descontinuada em favor de date_sun_info().
8.0.0	`latitude`, `longitude`, `zenith` e `utcOffset` agora podem ser nulos.

Examples

Exemplo #1 Exemplo da função date_sunset()

```php
<?php

/* Calcula o horário do pôr do sol para Lisboa, Portugal
Latitude: 38.4 norte
Longitude: 9 oeste
Zênite ~= 90
deslocamento UTC: +01:00 GMT
*/

echo date("D M d Y"). ', horário do pôr do sol : ' .date_sunset(time(), SUNFUNCS_RET_STRING, 38.4, -9, 90, 1);
```

O exemplo acima produzirá algo semelhante a:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 10
Deprecated: Function date_sunset() is deprecated since 8.1, use date_sun_info() instead in script on line 10
Mon Dec 20 2004, horário do pôr do sol : 18:13
```

Exemplo #2 Sem pôr do sol

```php
<?php
$solstice = strtotime('2017-12-21');
var_dump(date_sunset($solstice, SUNFUNCS_RET_STRING, 69.245833, -53.537222));
```

O exemplo acima produzirá:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 3
Deprecated: Function date_sunset() is deprecated since 8.1, use date_sun_info() instead in script on line 3
bool(false)
```

Veja também

- date_sun_info() - Retorna um array com informações sobre pôr/nascer do sol e o início/fim do dia

date_time_set

date_time_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_time_set — Sinônimo de DateTime::setTime()

Descrição

Esta função é um apelido para: DateTime::setTime()

date_timestamp_get

date_timestamp_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_timestamp_get — Sinônimo de DateTime::getTimestamp()

Descrição

Esta função é um apelido para: DateTime::getTimestamp()

date_timestamp_set

date_timestamp_set

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_timestamp_set — Sinônimo de DateTime::setTimestamp()

Descrição

Esta função é um apelido para: DateTime::setTimestamp()

date_timezone_get

date_timezone_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_timezone_get — Sinônimo de DateTime::getTimezone()

Descrição

Esta função é um apelido para: DateTime::getTimezone()

date_timezone_set

date_timezone_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_timezone_set — Sinônimo de DateTime::setTimezone()

Descrição

Esta função é um apelido para: DateTime::setTimezone()

getdate

getdate

(PHP 4, PHP 5, PHP 7, PHP 8)

getdate — Recupera informações de data/horário

Descrição

function **getdate**(?int `$timestamp` = `null`): array

Retorna um array associativo contendo a informação da data do `timestamp`, ou o horário atual local se `timestamp` for `null` ou omitido.

Parâmetros

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna um array associativo de informação sobre o `timestamp`. Os elementos do array associativo são os seguintes:

Elementos chave de retorno do array associativo

Chave	Descrição	Exemplo dos valores retornados
`"seconds"`	Representação numérica dos segundos	`0` a `59`
`"minutes"`	Representação numérica dos minutos	`0` a `59`

Chave	Descrição	Exemplo dos valores retornados
`"hours"`	Representação numérica das horas	`0` a `23`
`"mday"`	Representação numérica do dia do mês	`1` a `31`
`"wday"`	Representação numérica do dia da semana	`0` (para Sunday) a `6` (para Saturday)
`"mon"`	Representação numérica de um mês	`1` a `12`
`"year"`	Representação numérica completa do ano, 4 dígitos	Exemplos: `1999` ou `2003`
`"yday"`	Representação numérica do dia do ano	`0` a `365`
`"weekday"`	Representação textual completa do dia da semana	`Sunday` a `Saturday`
`"month"`	Representação textual completa de um mês, tal como January ou March	`January` a `December`
`0`	Segundos desde a época UNIX, similar ao valor retornados pela função time() e utilizado pela função date().	Dependente do sistema, tipicamente `-2147483648` à `2147483647`.

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da função getdate()

```
<?php
$today = getdate();
print_r($today);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [seconds] => 40
    [minutes] => 58
    [hours] => 21
    [mday] => 17
    [wday] => 2
    [mon] => 6
    [year] => 2003
    [yday] => 167
    [weekday] => Tuesday
    [month] => June
    [0] => 1055901520
)
```

Veja também

- date() - Formata um timestamp Unix
- idate() - Formata uma parte de uma data/horário local como um inteiro
- localtime() - Obtém o horário local
- time() - Retorna o timestamp Unix atual
- setlocale() - Define informação de localidade

gettimeofday

gettimeofday

(PHP 4, PHP 5, PHP 7, PHP 8)

gettimeofday — Obtém o horário atual

Descrição

function **gettimeofday**(bool `$as_float` = `false`): array|float

Esta é uma interface para gettimeofday(2). Retorna um array contendo os dados retornando a chamada ao sistema.

Parâmetros

`as_float`

Quando definido como `true`, um float, ao invés de array, é retornado.

Valor Retornado

Por padrão, um array é retornado. Se o parâmetro `as_float` é definido, então um float é retornado.

Chaves do array:

- "sec" - segundos desde a época Unix
- "usec" - microssegundos
- "minuteswest" - minutos a oeste de Greenwich
- "dsttime" - tipo de correção dst

Examples

Exemplo #1 Exemplo da função gettimeofday()

```php
<?php
print_r(gettimeofday());

echo gettimeofday(true);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [sec] => 1073504408
    [usec] => 238215
    [minuteswest] => 0
    [dsttime] => 1
)
1073504408.23910
```

gmdate

gmdate

(PHP 4, PHP 5, PHP 7, PHP 8)

gmdate — Formata uma data/horário GMT/CUT

Descrição

function **gmdate**(string `$format`, ?int `$timestamp` = `null`): string

Idêntica à função date(), exceto que o horário retornado será baseado no Horário de Greenwich (GMT).

Parâmetros

`format`

O formato da saída da string da data. Veja as opções de formatação para a função date().

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna uma string da data formatada.

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora é anulável.

Examples

Exemplo #1 Exemplo da **gmdate()**

```php
<?php
date_default_timezone_set("Europe/Helsinki");

echo date("M d Y H:i:s e", mktime(0, 0, 0, 1, 1, 1998)) . "\n";
echo gmdate("M d Y H:i:s e", mktime(0, 0, 0, 1, 1, 1998));
```

O exemplo acima produzirá:

```
Jan 01 1998 00:00:00 Europe/Helsinki
Dec 31 1997 22:00:00 UTC
```

Veja também

- DateTimeImmutable::__construct() - Retorna um novo objeto DateTimeImmutable
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- date() - Formata um timestamp Unix
- mktime() - Obtém um timestamp Unix de uma data
- gmmktime() - Obtém um timestamp Unix para uma data GMT
- IntlDateFormatter::format() - Formata data/horário como uma string

gmmktime

gmmktime

(PHP 4, PHP 5, PHP 7, PHP 8)

gmmktime — Obtém um timestamp Unix para uma data GMT

Descrição

```
function gmmktime(
    int $hour,
    ?int $minute = null,
    ?int $second = null,
    ?int $month = null,
    ?int $day = null,
    ?int $year = null
): int|false
```

Idêntico ao mktime() exceto que os parâmetros representam uma data GMT. A função **gmmktime()** usa internamente a função mktime(), significando que somente horários válidos derivados de horários locais podem ser utilizados.

Como em mktime(), argumentos opcionais podem ser omitidos da direita para esquerda, com todos os argumentos omitidos sendo definidos para o valor GMT correspondente.

Chamar **gmmktime()** sem nenhum argumento não é suportado, e vai resultar em um ArgumentCountError. time() pode ser usado para obter o horário atual.

Parâmetros

`hour`

O número de horas relativas ao início do dia determinadas pelos parâmetros `month`, `day` e `year`. Valores negativos são referenciados à horas antes da meia noite do dia em questão. Valores maiores que 23 referenciam horas correspondentes ao(s) próximo(s) dia(s).

`minute`

O número de minutos relativos ao início da `hour`. Valores negativos referenciam minutos da hora anterior. Valores maiores que 23 referenciam os minutos correspondentes a(s) próxima(s) horas(s).

`second`

O número de segundos relativos ao início do `minute`. Valores negativos referenciam segundos do minuto anterior. Valores maiores que 23 referenciam os segundos correspondentes ao(s) próximo(s) minuto(s).

`month`

O número do mês relativo ao fim do anterior. Valores de 1 a 12 referenciam o calendário normal de meses do ano em questão. Valores menores que 1 (valores negativos, incluso) referenciam meses do ano anterior em ordem reversa, então 0 é Dezembro, -1 Novembro, e etc. Valores maiores que 12 referenciam meses correspondentes ao(s) próximo(s) ano(s).

`day`

O número do dia relativo ao final do mês anterior. Valores de 1 a 28, 29, 30 e 31 (dependendo do mês) corresponde a dias normais no mês. Valores menores que 1 (valores negativos, incluso) corresponde a dias no mês anterior, então 0 é o último dia do mês anterior, -1 é o dia anterior a este, e etc. Valores maiores que o número de dias do mês, referenciam dias correspondentes ao(s) próximo(s) mês(es).

`year`

O ano

Valor Retornado

Retorna um int timestamp Unix em caso de sucesso, ou **`false`** se o timestamp não couber em um número inteiro do PHP.

Registro de Alterações

Versão	Descrição
8.0.0	`hour` não é mais opcional. Se for necessário um timestamp Unix, use time().
8.0.0	`minute`, `second`, `month`, `day` e `year` agora podem ser nulos.

Examples

Exemplo #1 A função **gmmktime()** no Windows

```php
<?php
date_default_timezone_set("Indian/Maldives");

echo "July 1, 2000 is on a " . date("l", gmmktime(0, 0, 0, 7, 1, 2000));
```

O exemplo acima produzirá:

```
July 1, 2000 is on a Saturday
```

Veja também

- A classe DateTimeImmutable
- mktime() - Obtém um timestamp Unix de uma data
- date() - Formata um timestamp Unix
- time() - Retorna o timestamp Unix atual

gmstrftime

gmstrftime

(PHP 4, PHP 5, PHP 7, PHP 8)

gmstrftime — Formata uma data/horário GMT/UTC de acordo com as configurações de localidade

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.1.0. O uso desta função é fortemente desencorajado.

Alternativas a esta função incluem:

- gmdate()
- IntlDateFormatter::format()

Descrição

#[\Deprecated]
function **gmstrftime**(string `$format`, ?int `$timestamp` = null): string|false

Possui o mesmo comportamento que a função strftime() exceto que o tempo retornado será no Horário de Greenwich (GMT). Por exemplo, quando executada no fuso horário EST (GMT -0500), a primeira linha abaixo imprime "Dec 31 1998 20:00:00", enquanto a segunda linha imprime "Jan 01 1999 01:00:00".

> **Aviso**
>
> Esta função depende das informações de localidade do sistema operacional, que podem ser inconsistentes ou não disponíveis. Ao invés disso, use o método IntlDateFormatter::format().

Parâmetros

`format`

Veja a descrição em strftime().

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna uma string formatada de acordo com a string de formatação, usando o `timestamp` informado, ou o horário local se nenhum timestamp é informado. Nomes de meses, dias da semana e outras strings que dependem do idioma respeitam a configuração de localidade atual definido com a função setlocale(). Em caso de falha, é retornado `false`.

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da função gmstrftime()

```
<?php
```

```php
setlocale(LC_TIME, 'es_ES.UTF-8');
date_default_timezone_set('EST');

echo strftime("%B %d %Y %H:%M:%S",   mktime(20, 0, 0, 12, 31, 98)) . "\n";
echo gmstrftime("%B %d %Y %H:%M:%S", mktime(20, 0, 0, 12, 31, 98));
```

O exemplo acima produzirá:

```
Deprecated: Function strftime() is deprecated since 8.1, use IntlDateFormatter::format() instead in script on line 6
diciembre 31 1998 20:00:00

Deprecated: Function gmstrftime() is deprecated since 8.1, use IntlDateFormatter::format() instead in script on line 7
enero 01 1999 01:00:00
```

Veja também

- IntlDateFormatter::format() - Formata data/horário como uma string
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- strftime() - Formata uma data/horário de acordo com as configurações locais

idate

idate

(PHP 5, PHP 7, PHP 8)

idate — Formata uma parte de uma data/horário local como um inteiro

Descrição

function **idate**(string `$format`, ?int `$timestamp` = `null`): int|false

Retorna um número formatado de acordo com um formato informado utilizando o inteiro `timestamp`, ou o horário local atual se nenhum timestamp foi informado. Em outras palavras, `timestamp` é opcional e tem como padrão o valor da função time().

Ao contrário da função date(), a **idate()**, aceita apenas um caractere no parâmetro `format`.

Parâmetros

`format`

Os caracteres a seguir são reconhecidos como válidos no parâmetro `format`

Caractere `format`	Descrição
`B`	Swatch Beat/Horário da Internet
`d`	Dia do mês
`h`	Hora (no formato de 12 horas)
`H`	Hora (no formato de 24 horas)
`i`	Minutos
`I` (i maiúsculo)	retorna `1` se o horário de verão está ativo, `0`, caso contrário
`L` (l maiúsculo)	retorna `1` para anos bissextos, `0`, caso contrário
`m`	Número do mês
`N`	Dia da semana, baseado na ISO-8601 (`1` para segunda-feira até`7` para domingo)
`o`	ISO-8601 year (4 digits)
`s`	Segundos
`t`	Dias do mês atual
`U`	Segundos desde a época Unix - January 1 1970 00:00:00 UTC - isso é o mesmo que a função time()

Caractere `format`	Descrição
`w`	Dia da semana (`0` para domingo)
`W`	Semana do ano, baseado na ISO-8601, com a semana começando na segunda-feira
`y`	Ano (1 ou 2 dígitos - veja a nota abaixo)
`Y`	Ano (4 dígitos)
`z`	Dia do ano
`Z`	Deslocamento do fuso horário em segundos

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna um int em caso de sucesso, ou `false` em caso de falha.

Como a função **idate()** sempre retorna um int e como ela não começa com um "0", **idate()** pode retornar menos dígitos do que se espera. Veja o exemplo abaixo.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um `E_WARNING` se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.2.0	Inclui os caracteres de formato `N` (dia da semana ISO-8601) e `o` (ano ISO-8601).
8.0.0	`timestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da função idate()

```php
<?php
$timestamp = strtotime('1st January 2004'); // 1072915200

// este código imprime o ano em um formato de dois dígitos
// entretanto, como começaria com um "0", ele
// só imprime "4"
echo idate('y', $timestamp) . "\n";

$timestamp = strtotime('1st January 2024'); // 1704067200
echo idate('y', $timestamp);
```

O exemplo acima produzirá:

```
4
24
```

Veja também

- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- date() - Formata um timestamp Unix
- getdate() - Recupera informações de data/horário
- time() - Retorna o timestamp Unix atual

localtime

localtime

(PHP 4, PHP 5, PHP 7, PHP 8)

localtime — Obtém o horário local

Descrição

function **localtime**(?int `$timestamp` = `null`, bool `$associative` = `false`): array

A função **localtime()** retorna um array idêntico à estrutura retornada pela chamada da função em C.

Parâmetros

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

`associative`

Valor Retornado

Se `associative` for definido para `false` ou não fornecido, o retorno é um array normal, indexado numericamente. Se `associative` for definido para `true`, então **localtime()** retorna um array associativo contendo os elementos da estrutura retornados pela chamada da função C para o horário local. Os nomes das chaves do array associativo são as seguintes:

- "tm_sec" - segundos, `0` a `59`
- "tm_min" - minutos, `0` a `59`
- "tm_hour" - horas, `0` a `23`
- "tm_mday" - dia do mês, `1` a `31`
- "tm_mon" - mês do ano, `0` (Jan) a `11` (Dez)
- "tm_year" - anos desde 1900
- "tm_wday" - dia da semana, `0` (Dom) a `6` (Sáb)
- "tm_yday" - dia do ano, `0` a `365`
- "tm_isdst" - Se está em horário de verão

  Positivo se sim, `0` se não, negativo se é desconhecido.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um `E_WARNING` se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da função localtime()

```php
<?php
$localtime = localtime();
$localtime_assoc = localtime(time(), true);
print_r($localtime);
print_r($localtime_assoc);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => 24
    [1] => 3
    [2] => 19
    [3] => 3
    [4] => 3
    [5] => 105
    [6] => 0
    [7] => 92
    [8] => 1
)

Array
(
    [tm_sec] => 24
    [tm_min] => 3
    [tm_hour] => 19
    [tm_mday] => 3
    [tm_mon] => 3
    [tm_year] => 105
    [tm_wday] => 0
    [tm_yday] => 92
    [tm_isdst] => 1
)
```

Veja também

- getdate() - Recupera informações de data/horário

microtime

microtime

(PHP 4, PHP 5, PHP 7, PHP 8)

microtime — Retorna o timestamp Unix atual com microssegundos

Descrição

function **microtime**(bool `$as_float = false`): string|float

A função **microtime()** retorna o timestamp atual com microssegundos. Esta função está disponível apenas em sistemas operacionais que suportam a chamada de sistema gettimeofday().

Para medições de desempenho, é recomendado utilizar hrtime().

Parâmetros

`as_float`

> Se utilizada e definida como `true`, a função **microtime()** retornará um float em vez de uma string, como descrito na seção de valores de retorno a seguir.

Valor Retornado

Por padrão, a função **microtime()** retorna uma string no formato "msec sec", onde `sec` é o número de segundos desde a época Unix (1 de Janeiro de 1970, 00:00:00 GMT), e `msec` mensura os microssegundos que se passaram desde `sec` e também é expressa em segundos, na forma de fração decimal.

Se o parâmetro opcional `as_float` for definido para `true`, a função **microtime()** retorna um float que representa a hora atual em segundos desde a época Unix com precisão de microssegundos.

Examples

Exemplo #1 Cronometrando a execução do script

```php
<?php
$time_start = microtime(true);

// Espera um momento
usleep(10_000);

$time_end = microtime(true);
$time = $time_end - $time_start;

print "Nada foi feito em $time segundo(s).\n";
```

Exemplo #2 microtime() e `REQUEST_TIME_FLOAT`

```php
<?php
// Tempo de espera aleatório
usleep(random_int(10_000, 1_000_000));

// REQUEST_TIME_FLOAT está disponível no array superglobal $_SERVER.
// Ele contém o timestamp do início da requisição com precisão de microssegundos.
$time = microtime(true) - $_SERVER["REQUEST_TIME_FLOAT"];

echo "Nada foi feito em $time segundo(s).\n";
```

Veja também

- time() - Retorna o timestamp Unix atual
- hrtime() - Obtém o tempo de alta resolução do sistema

mktime

mktime

(PHP 4, PHP 5, PHP 7, PHP 8)

mktime — Obtém um timestamp Unix de uma data

Descrição

```
function mktime(
    int $hour,
    ?int $minute = null,
    ?int $second = null,
    ?int $month = null,
    ?int $day = null,
    ?int $year = null
): int|false
```

Retorna o timestamp Unix correspondente aos argumentos informados. Este timestamp é um inteiro longo contendo o número de segundos entre a Época Unix (January 1 1970 00:00:00 GMT), e o horário especificado.

Quaisquer argumentos opcionais omitidos ou `null` serão definidos para os valores correntes de acordo com a data e horário locais.

> **Aviso**
>
> Favor notar que os argumentos estão em uma ordem não usual: `month`, `day`, `year`, e não na ordem mais razoável `year`, `month`, `day`.

Chamar **mktime()** sem argumentos não é suportado, e irá resultar em um ArgumentCountError. time() pode ser usada para obter o timestamp atual.

Parâmetros

`hour`

O número da hora relativa ao início de determinado dia do `month`, `day` e `year`. Valores negativos referenciam a hora anterior a meia noite do dia em questão. Valores maiores que 23 referenciam a hora correspondente no(s) próximo(s) dia(s).

`minute`

O número de minutos relativos ao início da `hour`. Valores negativos referenciam minutos da hora anterior. Valores maiores que 23 referenciam os minutos correspondentes a(s) próxima(s) horas(s).

`second`

O número de segundos relativos ao início do `minute`. Valores negativos referenciam segundos do minuto anterior. Valores maiores que 23 referenciam os segundos correspondentes ao(s) próximo(s) minuto(s).

`month`

O número do mês relativo ao fim do anterior. Valores de 1 a 12 referenciam o calendário normal de meses do ano em questão. Valores menores que 1 (valores negativos, inclusos) referenciam meses do ano anterior em ordem reversa, então 0 é Dezembro, -1 Novembro, e etc. Valores maiores que 12 referenciam meses correspondentes ao(s) próximo(s) ano(s).

`day`

O número do dia relativo ao final do mês anterior. Valores de 1 a 28, 29, 30 e 31 (dependendo do mês) corresponde a dias normais no mês. Valores menores que 1 (valores negativos, inclusos) corresponde a dias no mês anterior, então 0 é o último dia do mês anterior, -1 é o dia anterior a este, e etc. Valores maiores que o número de dias do mês, referenciam dias correspondentes ao(s) próximo(s) mês(es).

`year`

O número do ano. Pode conter dois ou quatro dígitos, com os valores entre 0-69 significando 2000-2069 e 70-100 para 1970-2000. Em sistemas aonde o time_t é um inteiro assinado de 32 bit, como é mais comum, o faixa de valores para `year` é algo entre 1901 e 2038.

Valor Retornado

A função **mktime()** retorna o timestamp Unix dos argumentos informados ou `false` se o timestamp não couber em um inteiro do PHP.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `hour` não é mais opcional. Se for necessário um timestamp Unix, use a função time().
8.0.0	`minute`, `second`, `month`, `day` e `year` agora podem ser nulos.

Examples

Exemplo #1 Exemplo básico da função mktime()

```php
<?php
// Configura o fuso horário padrão a ser utilizado.
date_default_timezone_set('UTC');

// Imprime: July 1, 2000 is on a Saturday
echo "July 1, 2000 is on a " . date("l", mktime(0, 0, 0, 7, 1, 2000)) . "\n";

// Imprime algo como: 2006-04-05T01:02:03+00:00
echo date('c', mktime(1, 2, 3, 4, 5, 2006)) . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
July 1, 2000 is on a Saturday
2006-04-05T01:02:03+00:00
```

Exemplo #2 Exemplo da mktime()

mktime() é útil para a aritmética e validação de data, já que calculará automaticamente o valor correto para a entrada fora do intervalo. Por exemplo, cada uma das seguintes linhas produzirá a string "Jan-01-1998".

```php
<?php
date_default_timezone_set('America/New_York');

echo date("c", mktime(0, 0, 0, 12, 32, 1997)) . "\n";
echo date("c", mktime(0, 0, 0, 13, 1, 1997)) . "\n";
echo date("c", mktime(0, 0, 0, 1, 1, 1998)) . "\n";
echo date("c", mktime(0, 0, 0, 1, 1, 98)) . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
```

Exemplo #3 Usando mktime para encontrar datas relativas

```php
<?php
date_default_timezone_set('Asia/Tokyo');

$tomorrow  = mktime(0, 0, 0, date("m")  , date("d")+1, date("Y"));
print date('c', $tomorrow) . "\n";

$lastmonth = mktime(0, 0, 0, date("m")-1, date("d"),   date("Y"));
print date('c', $lastmonth) . "\n";

$nextyear  = mktime(0, 0, 0, date("m"),   date("d"),   date("Y")+1) . "\n";
print date('c', $nextyear) . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
2025-09-30T00:00:00+09:00
2025-08-29T00:00:00+09:00
2026-09-29T00:00:00+09:00
```

> **Nota** :
>
> Isto pode ser mais confiável que simplesmente adicionar ou substrair o número de segundos em um dia ou mês de um timestamp por causa do fuso horário.

Exemplo #4 Último dia do próximo mês

O último dia de um mês informado pode ser expressado como o dia "0" do mês seguinte, não o dia -1. Os dois exemplos a seguir produzirão a string "Last day in Feb 2000 is: 29".

```php
<?php

$lastday = mktime(0, 0, 0, 3, 0, 2000);
echo 'Last day in Feb 2000 is: ', date('d', $lastday) . "\n";

$lastday = mktime(0, 0, 0, 4, -31, 2000);
echo 'Last day in Feb 2000 is: ', date('d', $lastday) . "\n";
```

O exemplo acima produzirá:

```
Last day in Feb 2000 is: 29
Last day in Feb 2000 is: 29
```

Veja também

- A classe DateTimeImmutable
- checkdate() - Valida uma data gregoriana
- gmmktime() - Obtém um timestamp Unix para uma data GMT
- date() - Formata um timestamp Unix
- time() - Retorna o timestamp Unix atual

strftime

strftime

(PHP 4, PHP 5, PHP 7, PHP 8)

strftime — Formata uma data/horário de acordo com as configurações locais

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.1.0. O uso desta função é fortemente desencorajado.

Alternativas a esta função incluem:

- date()
- IntlDateFormatter::format()

Descrição

#[\Deprecated]
function **strftime**(string `$format`, ?int `$timestamp` = null): string|false

Formata uma data/horário local de acordo com a configuração do idioma. Nome do mês e dia da semana e outras strings respeitam o idioma corrente definido com a função setlocale().

> **Aviso**
>
> Nem todos os especificadores de conversão podem ser suportados por suas bibliotecas C, e nestes casos eles não serão suportados pela função do PHP **strftime()**. Além disso, nem todas as plataformas suportam timestamps negativos, então sua faixa de datas pode ficar limitada a um valor não anterior à época Unix. Isto significa que %e, %T, %R e %D (e possivelmente outras) - assim como datas anteriores a `1 de janeiro de 1970` - não funcionarão no Windows, algumas distribuições Linux, e alguns outros sistemas operacionais. Para sistemas Windows, uma visão completa dos especificadores de conversão suportados podem ser encontrados em » MSDN. No lugar desta função, use o método IntlDateFormatter::format().

Parâmetros

`format`

Os seguintes caracteres são reconhecidos na string do parâmetro `format`

`format`	Descrição	Exemplo de valores retornados
Dia	---	---
`%a`	Representação textual abreviada do dia	`Sun` até `Sat`
`%A`	Representação textual completa do dia	`Sunday` até `Saturday`
`%d`	Dia do mês, com dois dígitos (com zeros à esquerda)	`01` a `31`
`%e`	Dia do mês com um dígito, precedido com um espaço. Não implementado como descrito no Windows. Veja abaixo para mais informações.	`1` a `31`

`format`	Descrição	Exemplo de valores retornados
`%j`	Dia do ano, com 3 dígitos e zeros à esquerda	`001` a `366`
`%u`	Representação numérica, do dia da semana, compatível com a ISO-8601	`1` (para segunda-feira) até `7` (para domingo)
`%w`	Representação numérica do dia da semana	`0` (para domingo) até `6` (para sábado)
Semana	---	---
`%U`	Número da semana de um dado ano, iniciado com o primeiro domingo sendo a primeira semana	`13` (como a 13ª semana completa do ano)
`%V`	Número da semana, compatível com a ISO-8601:1988 de um dado ano, iniciada com a primeira semana do ano com pelo menos 4 finais de semana, sendo a segunda-feira como o início da semana.	`01` até `53` (onde 53 é responsável por uma sobreposição)
`%W`	Representação numérica da semana do ano, começando pela primeira segunda-feira como primeira semana	`46` (como a 46ª semana do do ano iniciando na segunda-feira)
Mês	---	---
`%b`	Nome do mês abreviado, baseado no idioma	`jan` até `dez`
`%B`	Nome completo do mês, baseado no idioma	`janeiro` até `dezembro`
`%h`	Nome do mês abreviado, baseado no idioma (sinônimo de %b)	`jan` até `dez`
`%m`	Representação com dois dígitos do mês	`01` (para janeiro) até `12` (para dezembro)
Ano	---	---
`%C`	Representação, com dois dígitos, do século (ano dividido por 100, truncado como inteiro)	`19` para o século 20
`%g`	Representação do ano, com dois dígitos, seguindo o padrão ISO-8601:1988 (veja %V)	Exemplo: `09` de 6 de janeiro de 2009
`%G`	Versão de 4 dígitos de %g	Exemplo: `2008` de 3 de janeiro de 2009
`%y`	Representação, com dois dígitos, do ano	Exemplo: `09` de 2009, `79` de 1979
`%Y`	Representação, com quatro dígitos, do ano	Exemplo: `2038`
Hora	---	---
`%H`	Representação, com dois dígitos, da hora no formato 24 horas	`00` até `23`
`%k`	Representação da hora no formato 24 horas, com um espaço precedendo dígito único	`0` até `23`
`%I`	Representação, com dois dígitos, da hora no formato 12 horas	`01` até `12`
`%l ('L' minúsculo)`	Representação da hora no formato 12 horas, com um espaço precedendo dígito único	`1` até `12`
`%M`	Representação, com dois dígitos, do minuto	`00` até `59`
`%p`	'AM' ou 'PM' maiúsculo baseado na hora informada	Exemplo: `AM` para 00:31, `PM` para 22:23. O resultado exato depende do sistema operacional, e pode retornar variantes em minúsculoas, ou variantes com pontos (como `a.m.`).
`%P`	'am' ou 'pm' maiúsculo baseado na hora informada	Exemplo: `am` para 00:31, `pm` para 22:23. Não suportado por todos os sistemas operacionais.
`%r`	O mesmo que "%I:%M:%S %p"	Exemplo: `09:34:17 PM` de 21:34:17
`%R`	O mesmo que "%H:%M"	Exemplo: `00:35` de 12:35 AM, `16:44` de 4:44 PM
`%S`	Representação, com dois dígitos, do segundo	`00` até `59`
`%T`	O mesmo que "%H:%M:%S"	Exemplo: `21:34:17` para 09:34:17 PM
`%X`	Representação escolhida baseada no idioma, sem a data	Exemplo: `03:59:16` ou `15:59:16`

`format`	Descrição	Exemplo de valores retornados
`%z`	O deslocamento do fuso horário. Não implementado como descrito no Windows. Veja mais informações a seguir.	Exemplo: `-0500` para hora à leste dos EUA
`%Z`	A abreviação do fuso horário. Não implementado como descrito no Windows. Veja mais informações a seguir.	Exemplo: `EST` para Eastern Time
Carimbos de Data e Hora	---	---
`%c`	Carimbo escolhido de data e hora baseado no idioma	Exemplo: `Ter Fev 5 00:45:10 2009` de February 5, 2009 at 12:45:10 AM
`%D`	O mesmo que "%m/%d/%y"	Exemplo: `02/05/09` de February 5, 2009
`%F`	O mesmo que "%Y-%m-%d" (geralmente utilizado em carimbos de data em bancos de dados)	Exemplo: `2009-02-05` de February 5, 2009
`%s`	Timestamp Unix(o mesmo que a função time() function)	Exemplo: `305815200` de September 10, 1979 08:40:00 AM
`%x`	Representação da data baseada na preferência da localidade, sem a hora	Exemplo: `02/05/09` de February 5, 2009
Miscelânea	---	---
`%n`	Uma nova linha ("\n")	---
`%t`	Um caractere TAB ("\t")	---
`%%`	Um caractere literal de porcento ("%")	---

> **Aviso**
>
> Contrário a ISO-9899:1999, o sistema Sun Solaris começa a semana no Domingo, como 1. Resultando em `%u` não funcionando como descrito neste manual.

> **Aviso**
>
> *Somente no Windows:*
>
> O modificador `%e` não possui suporte a implementação desta função no Windows. Para conseguir este valor, o modificador `%#d` pode ser utilizado em substituição. O exemplo a seguir ilustra como escrever uma função com compatibilidade multiplataforma.
>
> Os modificadores `%z` e `%Z` retornam o nome do fuso horário em vez do deslocamento ou abreviação do mesmo.

> **Aviso**
>
> *macOS e musl somente:* O modificador `%P` não é suportado na implementação macOS desta função.

`timestamp`

O parâmetro opcional `timestamp` é um int de timestamp Unix cujo padrão é a hora local atual se `timestamp` não for informado ou for `null`. Em outras palavras, o padrão é o valor de time().

Valor Retornado

Retorna uma string formatada de acordo com o `format` informado em `timestamp` ou o horário local atual se nenhum timestamp foi informado. Nomes de meses e dia da semana, e outras strings dependentes de linguagens respeitam o idioma definido com a função setlocale(). A função retorna `false` se `format` for vazio, contém especificadores de conversão não suportados, ou se o comprimento da string retornada fosse maior que `4095`.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um `E_WARNING` se o fuso horário não for válido. Veja também date_default_timezone_set()

Como a saída depende da biblioteca C disponível, alguns especificadores de conversão não possuem suporte. No Windows, informar especificadores de conversão desconhecidos resultará em mensagens `E_WARNING` e retornará `false`. Em outros sistemas operacionais, pode não haver nenhuma mensagem `E_WARNING` e a saída poderá conter os especificadores de conversão não convertidos.

Registro de Alterações

Versão	Descrição
8.0.0	`timestamp` agora pode ser nulo.

Examples

Este exemplo funciona se os respectivos idiomas estiverem instalados no sistema.

Exemplo #1 Exemplo da função strftime() com idiomas

```php
<?php
setlocale(LC_TIME, "C");
echo strftime("%A");
setlocale(LC_TIME, "fi_FI");
echo strftime(" em finlandês é %A,");
setlocale(LC_TIME, "fr_FR");
echo strftime(" em francês %A e");
setlocale(LC_TIME, "de_DE");
echo strftime(" em alemão %A.\n");
```

Exemplo #2 Exemplo do número de semana ISO 8601:1988

```php
<?php
/*     Dezembro 2002 / Janeiro 2003
ISOWk  M   Tu  W   Thu F   Sa  Su
-----  --------------------------
51     16  17  18  19  20  21  22
52     23  24  25  26  27  28  29
1      30  31   1   2   3   4   5
2       6   7   8   9  10  11  12
3      13  14  15  16  17  18  19   */

// Mostra: 12/28/2002 - %V,%G,%Y = 52,2002,2002
echo "12/28/2002 - %V,%G,%Y = " . strftime("%V,%G,%Y", strtotime("12/28/2002")) . "\n";

// Mostra: 12/30/2002 - %V,%G,%Y = 1,2003,2002
echo "12/30/2002 - %V,%G,%Y = " . strftime("%V,%G,%Y", strtotime("12/30/2002")) . "\n";

// Mostra: 1/3/2003 - %V,%G,%Y = 1,2003,2003
echo "1/3/2003 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/3/2003")) . "\n";

// Mostra: 1/10/2003 - %V,%G,%Y = 2,2003,2003
echo "1/10/2003 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/10/2003")) . "\n";



/*     Dezembro 2004 / Janeiro 2005
ISOWk  M   Tu  W   Thu F   Sa  Su
-----  --------------------------
51     13  14  15  16  17  18  19
52     20  21  22  23  24  25  26
53     27  28  29  30  31   1   2
1       3   4   5   6   7   8   9
2      10  11  12  13  14  15  16   */

// Mostra: 12/23/2004 - %V,%G,%Y = 52,2004,2004
echo "12/23/2004 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("12/23/2004")) . "\n";

// Mostra: 12/31/2004 - %V,%G,%Y = 53,2004,2004
echo "12/31/2004 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("12/31/2004")) . "\n";

// Mostra: 1/2/2005 - %V,%G,%Y = 53,2004,2005
echo "1/2/2005 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/2/2005")) . "\n";

// Mostra: 1/3/2005 - %V,%G,%Y = 1,2005,2005
echo "1/3/2005 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/3/2005")) . "\n";
```

Exemplo #3 Exemplo do uso do modificador `%e` compatível com múltiplas plataformas

```php
<?php
```

```php
// Jan 1: resulta em: '%e%1%' (%%, e, %%, %e, %%)
$format = '%%e%%%e%%';

// Verifica se está no Windows e substitui o modificador %e
// adequadamente
if (strtoupper(substr(PHP_OS, 0, 3)) == 'WIN') {
    $format = preg_replace('#(?<!%)((?:%%)*)%e#', '\1%#d', $format);
}

echo strftime($format);
```

Exemplo #4 Mostra os formatos conhecidos e desconhecidos

```php
<?php

// Descreve os formatos
$strftimeFormats = array(
    'A' => 'A full textual representation of the day',
    'B' => 'Full month name, based on the locale',
    'C' => 'Two digit representation of the century (year divided by 100, truncated to an integer)',
    'D' => 'Same as "%m/%d/%y"',
    'E' => '',
    'F' => 'Same as "%Y-%m-%d"',
    'G' => 'The full four-digit version of %g',
    'H' => 'Two digit representation of the hour in 24-hour format',
    'I' => 'Two digit representation of the hour in 12-hour format',
    'J' => '',
    'K' => '',
    'L' => '',
    'M' => 'Two digit representation of the minute',
    'N' => '',
    'O' => '',
    'P' => 'lower-case "am" or "pm" based on the given time',
    'Q' => '',
    'R' => 'Same as "%H:%M"',
    'S' => 'Two digit representation of the second',
    'T' => 'Same as "%H:%M:%S"',
    'U' => 'Week number of the given year, starting with the first Sunday as the first week',
    'V' => 'ISO-8601:1988 week number of the given year, starting with the first week of the year with at least 4 weekdays,
with Monday being the start of the week',
    'W' => 'A numeric representation of the week of the year, starting with the first Monday as the first week',
    'X' => 'Preferred time representation based on locale, without the date',
    'Y' => 'Four digit representation for the year',
    'Z' => 'The time zone offset/abbreviation option NOT given by %z (depends on operating system)',
    'a' => 'An abbreviated textual representation of the day',
    'b' => 'Abbreviated month name, based on the locale',
    'c' => 'Preferred date and time stamp based on local',
    'd' => 'Two-digit day of the month (with leading zeros)',
    'e' => 'Day of the month, with a space preceding single digits',
    'f' => '',
    'g' => 'Two digit representation of the year going by ISO-8601:1988 standards (see %V)',
    'h' => 'Abbreviated month name, based on the locale (an alias of %b)',
    'i' => '',
    'j' => 'Day of the year, 3 digits with leading zeros',
    'k' => 'Hour in 24-hour format, with a space preceding single digits',
    'l' => 'Hour in 12-hour format, with a space preceding single digits',
    'm' => 'Two digit representation of the month',
    'n' => 'A newline character ("\n")',
    'o' => '',
    'p' => 'UPPER-CASE "AM" or "PM" based on the given time',
    'q' => '',
    'r' => 'Same as "%I:%M:%S %p"',
    's' => 'Unix Epoch Time timestamp',
    't' => 'A Tab character ("\t")',
    'u' => 'ISO-8601 numeric representation of the day of the week',
    'v' => '',
    'w' => 'Numeric representation of the day of the week',
    'x' => 'Preferred date representation based on locale, without the time',
    'y' => 'Two digit representation of the year',
    'z' => 'Either the time zone offset from UTC or the abbreviation (depends on operating system)',
    '%' => 'A literal percentage character ("%")',
);
```

```php
// Resultados
$strftimeValues = array();

// Avalia os formatos enqunto suprime os erros.
foreach ($strftimeFormats as $format => $description) {
    if (false !== ($value = @strftime("%{$format}"))) {
        $strftimeValues[$format] = $value;
    }
}

// Encontra o valor mais longo
$maxValueLength = 2 + max(array_map('strlen', $strftimeValues));

// Reporta formatos conhecidos
foreach ($strftimeValues as $format => $value) {
    echo "Formato conhecido    : '{$format}' = ", str_pad("'{$value}'", $maxValueLength), " ( {$strftimeFormats[$format]}
)\n";
}

// Reporta formatos desconhecidos
foreach (array_diff_key($strftimeFormats, $strftimeValues) as $format => $description) {
    echo "Formato desconhecido : '{$format}'   ", str_pad(' ', $maxValueLength), ($description ? " ( {$description} )" : ''),
"\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
Formato conhecido    : 'A' = 'Friday'            ( A full textual representation of the day )
Formato conhecido    : 'B' = 'December'          ( Full month name, based on the locale )
Formato conhecido    : 'H' = '11'                ( Two digit representation of the hour in 24-hour format )
Formato conhecido    : 'I' = '11'                ( Two digit representation of the hour in 12-hour format )
Formato conhecido    : 'M' = '24'                ( Two digit representation of the minute )
Formato conhecido    : 'S' = '44'                ( Two digit representation of the second )
Formato conhecido    : 'U' = '48'                ( Week number of the given year, starting with the first Sunday as the
first week )
Formato conhecido    : 'W' = '48'                ( A numeric representation of the week of the year, starting with the first
Monday as the first week )
Formato conhecido    : 'X' = '11:24:44'          ( Preferred time representation based on locale, without the date )
Formato conhecido    : 'Y' = '2010'              ( Four digit representation for the year )
Formato conhecido    : 'Z' = 'GMT Standard Time' ( The time zone offset/abbreviation option NOT given by %z (depends on
operating system) )
Formato conhecido    : 'a' = 'Fri'               ( An abbreviated textual representation of the day )
Formato conhecido    : 'b' = 'Dec'               ( Abbreviated month name, based on the locale )
Formato conhecido    : 'c' = '12/03/10 11:24:44' ( Preferred date and time stamp based on local )
Formato conhecido    : 'd' = '03'                ( Two-digit day of the month (with leading zeros) )
Formato conhecido    : 'j' = '337'               ( Day of the year, 3 digits with leading zeros )
Formato conhecido    : 'm' = '12'                ( Two digit representation of the month )
Formato conhecido    : 'p' = 'AM'                ( UPPER-CASE "AM" or "PM" based on the given time )
Formato conhecido    : 'w' = '5'                 ( Numeric representation of the day of the week )
Formato conhecido    : 'x' = '12/03/10'          ( Preferred date representation based on locale, without the time )
Formato conhecido    : 'y' = '10'                ( Two digit representation of the year )
Formato conhecido    : 'z' = 'GMT Standard Time' ( Either the time zone offset from UTC or the abbreviation (depends on
operating system) )
Formato conhecido    : '%' = '%'                 ( A literal percentage character ("%") )
Formato desconhecido : 'C'                       ( Two digit representation of the century (year divided by 100, truncated
to an integer) )
Formato desconhecido : 'D'                       ( Same as "%m/%d/%y" )
Formato desconhecido : 'E'
Formato desconhecido : 'F'                       ( Same as "%Y-%m-%d" )
Formato desconhecido : 'G'                       ( The full four-digit version of %g )
Formato desconhecido : 'J'
Formato desconhecido : 'K'
Formato desconhecido : 'L'
Formato desconhecido : 'N'
Formato desconhecido : 'O'
Formato desconhecido : 'P'                       ( lower-case "am" or "pm" based on the given time )
Formato desconhecido : 'Q'
Formato desconhecido : 'R'                       ( Same as "%H:%M" )
```

```
Formato desconhecido : 'T'                      ( Same as "%H:%M:%S" )
Formato desconhecido : 'V'                      ( ISO-8601:1988 week number of the given year, starting with the first week
of the year with at least 4 weekdays, with Monday being the start of the week )
Formato desconhecido : 'e'                      ( Day of the month, with a space preceding single digits )
Formato desconhecido : 'f'
Formato desconhecido : 'g'                      ( Two digit representation of the year going by ISO-8601:1988 standards
(see %V) )
Formato desconhecido : 'h'                      ( Abbreviated month name, based on the locale (an alias of %b) )
Formato desconhecido : 'i'
Formato desconhecido : 'k'                      ( Hour in 24-hour format, with a space preceding single digits )
Formato desconhecido : 'l'                      ( Hour in 12-hour format, with a space preceding single digits )
Formato desconhecido : 'n'                      ( A newline character ("\n") )
Formato desconhecido : 'o'
Formato desconhecido : 'q'
Formato desconhecido : 'r'                      ( Same as "%I:%M:%S %p" )
Formato desconhecido : 's'                      ( Unix Epoch Time timestamp )
Formato desconhecido : 't'                      ( A Tab character ("\t") )
Formato desconhecido : 'u'                      ( ISO-8601 numeric representation of the day of the week )
Formato desconhecido : 'v'
```

Notas

> **Nota** :
>
> %G e %V, que são baseadas nos números de semana da ISO 8601:1988, podem dar resultados inesperados (embora corretos), se o sistema numérico não for reconhecido perfeitamente. Veja exemplos da %V nesta página do manual.

Veja também

- IntlDateFormatter::format() - Formata data/horário como uma string
- DateTimeInterface::format() - Retorna a data formatada de acordo com um formato informado
- » Ferramenta online para criação de formatos compatíveis com o strftime()
- setlocale() - Define informação de localidade
- mktime() - Obtém um timestamp Unix de uma data
- strptime() - Interpreta uma data/horário gerada pela função strftime
- gmstrftime() - Formata uma data/horário GMT/UTC de acordo com as configurações de localidade
- » Especificação Open Group da **strftime()**

strptime

strptime

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

strptime — Interpreta uma data/horário gerada pela função strftime()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.1.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **strptime**(string `$timestamp`, string `$format`): array|false

strptime() retorna um array com a `timestamp` analisada, ou false em caso de erro.

Nomes de meses e semanas, e outras strings que dependem do idioma, respeitam o idioma corrente definido com a função setlocale() (LC_TIME).

Parâmetros

`timestamp` (string)

A string a ser interpretada (retornada da função strftime()).

`format` (string)

O formato utilizado em `timestamp` (o mesmo utilizado na função strftime()). Note que algumas opções de formato disponíveis na função strftime() podem não ter nenhum efeito com **strptime()**; o subconjunto exato que é suportado vai variar por ser baseado no sistema operacional e biblioteca C em uso.

Para mais informações sobre as opções de formato, veja a página da função strftime().

Valor Retornado

Retorna um array ou `false` em caso de falha.

Os seguintes parâmetros são retornados no array

Parâmetro	Descrição
`"tm_sec"`	Segundos após o minuto (0-61)
`"tm_min"`	Minutos após a hora (0-59)
`"tm_hour"`	Horas passadas desde a meia-noite (0-23)
`"tm_mday"`	Dia do mês (1-31)
`"tm_mon"`	Meses passados desde janeiro (0-11)
`"tm_year"`	Anos passados desde 1900
`"tm_wday"`	Dias passados desde domingo (0-6)
`"tm_yday"`	Dias passados desde 1º de janeiro 1 (0-365)
`"unparsed"`	A parte `timestamp` que não foi reconhecida usando o `format` especificado

Registro de Alterações

Versão	Descrição
8.1.0	Esta função foi descontinuada. Use date_parse_from_format() no lugar (para análise independente de localidade), ou IntlDateFormatter::parse() (para análise dependente de localidade).

Examples

Exemplo #1 Exemplo da função **strptime()**

```php
<?php
$format = '%d/%m/%Y %H:%M:%S';
$strf = strftime($format);

echo "$strf\n";

print_r(strptime($strf, $format));
```

O exemplo acima produzirá algo semelhante a:

```
03/10/2004 15:54:19

Array
(
    [tm_sec] => 19
    [tm_min] => 54
    [tm_hour] => 15
    [tm_mday] => 3
    [tm_mon] => 9
    [tm_year] => 104
    [tm_wday] => 0
    [tm_yday] => 276
    [unparsed] =>
)
```

Notas

> **Nota** :
>
> Esta função não foi implementada em plataformas Windows.

> **Nota** :
>
> Internamente, esta função chama a função `strptime()` presente na biblioteca C do sistema. Esta função pode apresentar um comportamento diferente entre diferentes sistemas operacionais. A utilização da função date_parse_from_format(), que não é afetada por esta questão, é recomendada.

> **Nota** :
>
> `"tm_sec"` inclui todos os segundos intercalares (atualmente 2 por ano). Para mais informações sobre segundos intercalares, veja o » o artigo da Wikipedia sobre segundos intercalares.

Veja também

- IntlDateFormatter::parse() - Interpreta uma string para uma valor de timestamp
- DateTime::createFromFormat() - Interpreta um string de data/horário de acordo com um formato especificado.
- checkdate() - Valida uma data gregoriana
- strftime() - Formata uma data/horário de acordo com as configurações locais
- date_parse_from_format() - Retorna informações sobre uma data informada de acordo com um formato

strtotime

strtotime

(PHP 4, PHP 5, PHP 7, PHP 8)

strtotime — Interpreta qualquer descrição textual de data/horário em inglês para timestamp Unix

Descrição

function **strtotime**(string `$datetime`, ?int `$baseTimestamp` = `null`): int|false

A função espera que seja informada uma string contendo um formato de data em inglês US, e tentará interpretá-lo para um timestamp Unix (o número de segundos desde 1º de janeiro de 1970 00:00:00 GMT), relativo ao timestamp dado em `baseTimestamp`, ou o horário atual se `baseTimestamp` não for fornecido. A interpretação da string de data está definida em Formatos de Data e Horário, e tem várias considerações sutis. Rever os detalhes completos dessas definições é fortemente recomendado.

> **Aviso**
>
> O timestamp Unix que esta função retorna não contém informação sobre fuso horário. Para realizar cálculos com informação de data/horário, deve-se usar a classe com mais capacidade DateTimeImmutable.

Cada parâmetro desta função utiliza o fuso horário padrão, a não ser que um fuso horário seja especificado neste parâmetro. Tome cuidado para não utilizar fusos horários diferentes em cada um dos parâmetros a não ser que seja o pretendido. Veja a função date_default_timezone_get() para verificar as diversas formas de definir um fuso horário padrão.

Parâmetros

`datetime`

Uma string de data/hora. Os formatos válidos são explicados em Formatos de Data e Hora.

`baseTimestamp`

O timestamp que será utilizado como base no cálculo das datas relativas.

Valor Retornado

Retorna um timestamp em sucesso, `false` caso contrário.

Erros/Exceções

Toda chamada a uma função de data/hora gerará um `E_WARNING` se o fuso horário não for válido. Veja também date_default_timezone_set()

Registro de Alterações

Versão	Descrição
8.0.0	`baseTimestamp` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da função strtotime()

```php
<?php
echo strtotime("now"), "\n";
echo strtotime("10 September 2000"), "\n";
echo strtotime("+1 day"), "\n";
echo strtotime("+1 week"), "\n";
echo strtotime("+1 week 2 days 4 hours 2 seconds"), "\n";
echo strtotime("next Thursday"), "\n";
echo strtotime("last Monday"), "\n";
```

Exemplo #2 Verificando erros

```php
<?php
$str = 'Not Good';

if (($timestamp = strtotime($str)) === false) {
    echo "A string ($str) tem problemas";
} else {
    echo "$str == " . date('l dS \o\f F Y h:i:s A', $timestamp);
}
```

Notas

> **Nota** :
>
> Data "relativa" neste caso também significa que se um componente em particular do timestamp de data/horário não for fornecido, ele será removido literalmente do parâmetro `baseTimestamp`. Isto é, `strtotime('February')`, se executado em 31 de maio de 2022, será interpretado como `31 de fevereiro de 2022`, o que vai transferir para um timestamp de `3 de março` (em um ano bissexto, seria `2 de março`). Usar `strtotime('1 February')` ou `strtotime('first day of February')` evitaria este problema.

> **Nota** :
>
> Se o número do ano for especificado no formato com dois dígitos, o valor entre 00-60 será mapeado para 2000-2069 e 70-99 para 1970-1999. Veja as notas a seguir para possíveis diferenças em sistemas de 32 bits (as datas possíveis acabam em 2038-01-19 03:14:07),

> **Nota** :
>
> O intervalo válido de um timestamp é tipicamente de Sex, 13 Dez 1901 20:45:54 GMT até Ter, 19 Jan 2038 03:14:07 GMT. Estas são datas que correspondem aos valores máximos e mínimos para um inteiro sem sinal em sistemas de 32 bits.
>
> Em versões do PHP em 64 bits, a faixa válida de um timestamp é praticamente infinita, já que 64 bits podem representar aproximadamente 293 bilhões de anos em ambas direções.

> **Nota** :

> O uso desta função para operações matemáticas não é recomendado. É melhor utilizar os métodos DateTime::add() e DateTime::sub().

Veja também

- DateTimeImmutable
- DateTimeImmutable::createFromFormat() - Interpreta um string de data/horário de acordo com o formato especificado
- Formatos de Data e Horário
- checkdate() - Valida uma data gregoriana
- strptime() - Interpreta uma data/horário gerada pela função strftime

time

time

(PHP 4, PHP 5, PHP 7, PHP 8)

time — Retorna o timestamp Unix atual

Descrição

function **time**(): int

Retorna o horário atual medido como o número de segundos desde a Época Unix (1 de janeiro de 1970 00:00:00 GMT).

> **Nota** :
>
> Timestamps Unix não contêm nenhuma informação sobre fuso horário local. Recomenda-se usar a classe DateTimeImmutable para lidar com informações de data e horário para evitar as armadilhas que vêm com o uso exclusivo de timestamps Unix.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o timestamp atual.

Examples

Exemplo #1 Exemplo de **time()**

```php
<?php
echo 'Agora: '. time();
```

O exemplo acima produzirá algo semelhante a:

```
Agora: 1660338149
```

Notas

> **Dica**
>
> O timestamp do início da requisição está disponível na variável *$_SERVER['REQUEST_TIME']*.

Veja também

- DateTimeImmutable
- date() - Formata um timestamp Unix
- microtime() - Retorna o timestamp Unix atual com microssegundos

timezone_abbreviations_list

timezone_abbreviations_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_abbreviations_list — Sinônimo de DateTimeZone::listAbbreviations()

Descrição

Esta função é um apelido para: DateTimeZone::listAbbreviations()

timezone_identifiers_list

timezone_identifiers_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_identifiers_list — Sinônimo de DateTimeZone::listIdentifiers()

Descrição

Esta função é um apelido para: DateTimeZone::listIdentifiers()

timezone_location_get

timezone_location_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

timezone_location_get — Sinônimo de DateTimeZone::getLocation()

Descrição

Esta função é um apelido para: DateTimeZone::getLocation()

timezone_name_from_abbr

timezone_name_from_abbr

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

timezone_name_from_abbr — Retorna um nome de fuso horário através da abreviação e do deslocamento UTC

Descrição

function **timezone_name_from_abbr**(string `$abbr`, int `$utcOffset = -1`, int `$isDST = -1`): string|false

Parâmetros

`abbr`

A abreviação do fuso horário.

`utcOffset`

Deslocamento em relação ao GMT em segundos. Sendo -1 o padrão, que significa que o primeiro fuso horário correspondente encontrado com `abbr` é retornado. Caso contrário o deslocamento é buscao, e caso não encontrado, o o primeiro fuso horário, com qualquer deslocamento, é retornado.

`isDST`

Indicador do horário de verão. O padrão é -1, e informa que tendo o fuso horário horário de verão, ou não, este não será levado em consideração na busca. Se configurado para 1, então `utcOffset` é utilizado como diferença no horário de verão; se 0, então `utcOffset` é utilizado sem a diferença do horário de verão. Se `abbr` não existir, o fuso horário será buscado exclusivamente pelo `utcOffset` e `isDST`.

Valor Retornado

Retorna o fuso horário em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Um exemplo da função timezone_name_from_abbr()

```php
<?php
echo timezone_name_from_abbr("CET") . "\n";
echo timezone_name_from_abbr("", 3600, 0) . "\n";
```

O exemplo acima produzirá algo semelhante a:

```
Europe/Berlin
Europe/Paris
```

Veja também

- timezone_abbreviations_list() - Sinônimo de DateTimeZone::listAbbreviations

timezone_name_get

timezone_name_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_name_get — Sinônimo de DateTimeZone::getName()

Descrição

Esta função é um apelido para: DateTimeZone::getName()

timezone_offset_get

timezone_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_offset_get — Sinônimo de DateTimeZone::getOffset()

Descrição

Esta função é um apelido para: DateTimeZone::getOffset()

timezone_open

timezone_open

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_open — Sinônimo de DateTimeZone::__construct()

Descrição

Esta função é um apelido para: DateTimeZone::__construct()

timezone_transitions_get

timezone_transitions_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_transitions_get — Sinônimo de DateTimeZone::getTransitions()

Descrição

Esta função é um apelido para: DateTimeZone::getTransitions()

timezone_version_get

timezone_version_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

timezone_version_get — Obtém a versão do banco de dados de fuso horário.

Descrição

function **timezone_version_get**(): string

Obtém a versão atual do banco de dados de fuso horário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string no formato `YYYY.incremento`, como em `2022.2`.

Se a versão instalada do banco de dados de fuso horário for antiga (por exemplo, ela não mostra o ano atual), pode-se atualizar a informação do fuso horário tanto atualizando a versão do PHP, ou instalando o pacote PECL » timezonedb.

Algumas distribuições Linux alteram o suporte do PHP para data/horário para usar uma fonte alternativa de informação de fuso horário. Neste caso, esta função irá retornar `0.system`. Recomenda-se instalar o pacote PECL » timezonedb neste caso também.

Examples

Exemplo #1 Obtendo a versão do banco de dados de fuso horário

```php
<?php
echo timezone_version_get();
```

O exemplo acima produzirá algo semelhante a:

```
2022.2
```

Veja também

- Lista de fusos horários suportados

Erros e Exceções de Data/Horário

- DateError (herda de Error)
    - DateObjectError
    - DateRangeError
- DateException (herda de Exception)
    - DateInvalidOperationException
    - DateInvalidTimezoneException
    - DateMalformedIntervalStringException
    - DateMalformedPeriodStringException
    - DateMalformedStringException

Formatos de Data e Horário Suportados

Esta seção descreve todos os diferentes formatos em uma sintaxe BNF, que os analisadores de DateTimeImmutable, DateTime, date_create_immutable(), date_create(), date_parse() e strtotime() entendem. Os formatos estão agrupados por seção. Na maioria dos casos, formatos de diferentes seções, separados por espaços, vírgula ou ponto, podem ser utilizados com a mesma string data/horário. Para cada um dos formatos suportados, um ou mais exemplos estão disponíveis, assim como a descrição do formato. Nos formatos, caracteres entre aspas simples são insensíveis a maiúsculas/minúsculas (`'t'` pode ser `t` ou `T`), caracteres entre aspas duplas são sensíveis a maiúsculas/minúsculas (`"T"` só pode ser `T`).

Para formatar objetos DateTimeImmutable e DateTime, favor se referir à documentação do método DateTimeInterface::format().

Um conjunto geral de regras deve ser considerado.

1. O analisador permite para cada unidade (ano, mês, dia, hora, minuto, segundo) o intervalo completo de valores. Para um ano são apenas 4 dígitos, para um mês o intervalo é 0-12, dia é 0-31, hora é 0-24, e minuto é 0-59.
2. 60 é permitido para segundos, e algumas vezes strings com este segundo adicional aparecem. Porém, o PHP implementa o horário Unix onde "60" não é um segundo válido e por isso ocorre sobrecarga.
3. strtotime() retorna **`false`** se quaisquer números estiverem fora dos intervalos, e DateTimeImmutable::__construct() dispara uma exceção.
4. Se um string contém somente uma data, todos os elementos de horário são inicializados em 0.
5. Todos os elementos menos significativos são inicializados em 0 se alguma parte do horário estiver presente no string.
6. O analisador é muito simples, e não faz verificações adicionais, o que o torna mais rápido (e mais genérico).
7. Além das regras para os elementos de horário individuais, o analisador também entende formatos compostos mais específicos, como análise de timestamps Unix (`@1690388256`) e números de semana ISO (`2008-W28-3`).
8. Há uma verificação adicional se uma data inválida é fornecida:

   ```php
   <?php
   $res = date_parse("2015-09-31");
   var_dump($res["warnings"]);
   ```

   O exemplo acima produzirá:

   ```
   array(1) {
     [11]=>
     string(27) "The parsed date was invalid"
   }
   ```

9. Já é possível lidar com casos extremos, mas neste caso a função DateTimeImmutable::createFromFormat() deve ser usada e o formato correto fornecido.

   ```php
   <?php
   $res = DateTimeImmutable::createFromFormat("Y-m-d", "2015-09-34");
   var_dump($res);
   ```

   O exemplo acima produzirá:

   ```
   object(DateTimeImmutable)#1 (3) {
     ["date"]=>
     string(26) "2015-10-04 17:24:43.000000"
     ["timezone_type"]=>
     int(3)
     ["timezone"]=>
     string(13) "Europe/London"
   }
   ```

Formatos de Horário

Esta página descreve os diferentes formatos de data/horário em uma sintaxe BNF, que os analisadores de DateTimeImmutable, DateTime, date_create(), date_create_immutable() e strtotime() entendem.

Para formatar objetos DateTimeImmutable e DateTime, favor se referir à documentação do método DateTimeInterface::format().

Símbolos utilizados

Descrição	Formatos	Exemplos
`frac`	. [0-9]+	".21342", ".85"
`hh`	"0"?[1-9] | "1"[0-2]	"04", "7", "12"
`HH`	[01][0-9] | "2"[0-4]	"04", "07", "19"
`meridian`	[AaPp] .? [Mm] .? [\0\t]	"A.m.", "pM", "am."
`MM`	[0-5][0-9]	"00", "12", "59"
`II`	[0-5][0-9]	"00", "12", "59"
`space`	[\t]	
`tz`	"("? [A-Za-z]{1,6} ")"? | [A-Z][a-z]+([_/][A-Z][a-z]+)+	"CEST", "Europe/Amsterdam", "America/Indiana/Knox"
`tzcorrection`	"GMT"? [+-] `hh` ":"? `MM`?	"+0400", "GMT-07:00", "-07:00"

Notação de 12 Horas

Descrição	Formatos	Exemplos
Somente as horas, com o meridiano	`hh space? meridian`	"4 am", "5PM"
Horas e minutos, com o meridiano	`hh` [.:] `MM space? meridian`	"4:08 am", "7:19P.M."
Horas, minutos e segundos, com o meridiano	`hh` [.:] `MM` [.:] `II space? meridian`	"4:08:37 am", "7:19:19P.M."
MS SQL (Horas, minutos, segundos e a fração com o meridiano)	`hh` ":" `MM` ":" `II` [.:] [0-9]+ `meridian`	"4:08:39:12313am"

Notação de 24 Horas

Descrição	Formato	Exemplos
Horas e minutos	't'? `HH` [.:] `MM`	"04:08", "19.19", "T23:43"
Horas e minutos, sem dois-pontos	't'? `HH MM`	"0408", "t1919", "T2343"
Horas, minutos e segundos	't'? `HH` [.:] `MM` [.:] `II`	"04.08.37", "t19:19:19"
Horas, minutos e segundos, sem dois-pontos	't'? `HH MM II`	"040837", "T191919"
Horas, minutos, segundos e fuso horário	't'? `HH` [.:] `MM` [.:] `II space?` (`tzcorrection` | `tz`)	"040837CEST", "T191919-0700"
Horas, minutos, segundos e fração	't'? `HH` [.:] `MM` [.:] `II frac`	"04.08.37.81412", "19:19:19.532453"
Informações do fuso horário	`tz` | `tzcorrection`	"CEST", "Europe/Amsterdam", "+0430", "GMT-06:00"

Formatos de Data

Esta página descreve os diferentes formatos de data em uma sintaxe BNF, que os analisadores de DateTimeImmutable, DateTime, date_create(), date_create_immutable() e strtotime() entendem.

Para formatar objetos DateTimeImmutable e DateTime, favor se referir à documentação do método DateTimeInterface::format().

Simbolos Utilizados

Descrição	Formato	Exemplos
`daysuf`	"st" | "nd" | "rd" | "th"	
`dd`	([0-2]?[0-9] | "3"[01]) `daysuf`?	"7th", "22nd", "31"
`DD`	"0" [0-9] | [1-2][0-9] | "3" [01]	"07", "31"

Descrição	Formato	Exemplos
`m`	'january' | 'february' | 'march' | 'april' | 'may' | 'june' | 'july' | 'august' | 'september' | 'october' | 'november' | 'december' | 'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec' | "I" | "II" | "III" | "IV" | "V" | "VI" | "VII" | "VIII" | "IX" | "X" | "XI" | "XII"	
`M`	'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec'	
`mm`	"0"? [0-9] | "1"[0-2]	"0", "04", "7", "12"
`MM`	"0" [0-9] | "1"[0-2]	"00", "04", "07", "12"
`y`	[0-9]{1,4}	"00", "78", "08", "8", "2008"
`yy`	[0-9]{2}	"00", "08", "78"
`YY`	[0-9]{4}	"2000", "2008", "1978"
`YYY`	[0-9]{5,19}	"81412", "20192"

Notações Localizadas

Descrição	Formato	Exemplos
Dia e mês americano	`mm "/" dd`	"5/12", "10/27"
Dia, mês e ano americano	`mm "/" dd "/" y`	"12/22/78", "1/17/2006", "1/17/6"
Ano com quatro dígitos, mês e dia com barras	`YY "/" mm "/" dd`	"2008/6/30", "1978/12/22"
Ano com quatro dígitos e mês (GNU)	`YY "-" mm`	"2008-6", "2008-06", "1978-12"
Ano, mês e dia com traços	`y "-" mm "-" dd`	"2008-6-30", "78-12-22", "8-6-21"
Dia, mês e ano, com quatro dígitos, com pontos, tabulações ou traços	`dd [.\t-] mm [.-] YY`	"30-6-2008", "22.12.1978"
Dia, mês e ano, com dois dígitos, com pontos ou tabulações	`dd [.\t] mm "." yy`	"30.6.08", "22\t12.78"
Dia, mês textual e ano	`dd ([ \t.-])* m ([ \t.-])* y`	"30-June 2008", "22DEC78", "14 III 1879"
Mês textual e ano com quatro dígitos (O dia é redefinido para 1)	`m ([ \t.-])* YY`	"June 2008", "DEC1978", "March 1879"
Ano com quatro dígitos e mês textual (O dia é redefinido para 1)	`YY ([ \t.-])* m`	"2008 June", "1978-XII", "1879.MArCH"
Mês textual, dia e ano	`m ([ .\t-])* dd [,.stndrh\t ]+ y`	"July 1st, 2008", "April 17, 1790", "May.9,78"
Mês textual e dia	`m ([ .\t-])* dd [,.stndrh\t ]*`	"July 1st,", "Apr 17", "May.9"
Dia e mês textual	`dd ([ .\t-])* m`	"1 July", "17 Apr", "9.May"
Mês abreviado, dia e ano	`M "-" DD "-" y`	"May-09-78", "Apr-17-1790"
Ano, mês abreviado e dia	`y "-" M "-" DD`	"78-Dec-22", "1814-MAY-17"
Ano (somente o ano)	`YY`	"1978", "2008"
Year (expandido, 5-19 dígitos com sinal)	`[+-] YYY`	"-81120", "+20192"
Mês textual (somente o mês)	`m`	"March", "jun", "DEC"

Notação ISO8601

Descrição	Formato	Exemplos
Ano com oito dígitos, mês e dia	`YY MM DD`	"15810726", "19780417", "18140517"
Ano com quatro dígitos, mês e dia com barras	`YY "/" MM "/" DD`	"2008/06/30", "1978/12/22"
Ano com dois dígitos, mês e dia com traços	`yy "-" MM "-" DD`	"08-06-30", "78-12-22"
Ano com quatro dígitos, com sinal opcional, mês e dia	`[+-]? YY "-" MM "-" DD`	"-0002-07-26", "+1978-04-17", "1814-05-17"
Ano com 5+ dígitos com sinal, mês e dia	`[+-] YYY "-" MM "-" DD`	"-81120-02-26", "+20192-04-17"

> **Nota** :

Para os formatos `y` e `yy`, anos abaixo de 100 são manipulados de maneira especial quando os símbolos `y` ou `yy` são utilizados. Se o ano cair no período de 0 (incluso) a 69 (incluso), 2000 anos são adicionados. Se o ano cair no período de 70 (incluso) a 99 (incluso), então 1900 são adicionados. Isso significa que "00-01-01" será interpretado como "2000-01-01".

> **Nota** :
>
> O formato "Dia, mês e ano com dois dígitos, com pontos ou tabulações" (`dd` [.\t] `mm` "." `yy`) somente funciona para valores de ano que variam 61 (incluso) até 99 (incluso) - fora destes anos, o *formato de tempo* "`HH` [.:] `MM` [.:] `SS`" possui precedência.

> **Nota** :
>
> O formato "Ano (e somente o ano)", só funcionará de maneira confiável se uma string de horário já tiver sido encontrada. Caso contrário, se o ano de quatro dígitos corresponder a `HH MM`, estes dois elementos serão definidos no lugar.
>
> Para interpretear consistentemente só um valor de ano, use DateTimeImmutable::createFromFormat() com o especificador `Y`.

> **Cuidado**
>
> É possível exceder positivamente e negativamente os formatos `dd` e `DD`. Dia 0 corresponde a último dia do mês anterior, assim como excedências positivas contam no próximo mês. Isso faz com que "2008-08-00" seja equivalente a "2008-07-31" e "2008-07-31" seja equivalente a "2008-07-01" (junho possui somente 30 dias).
>
> Note que o intervalo de dia é restrito a 0-31 como indicado pela expressão regular acima. Por isso, "2008-06-32" não é um string de data válida, por exemplo.
>
> É possível exceder negativamente os formatos `mm` e `MM` com o valor 0. O valor de mês 0 corresponde a dezembro do mês anterior. Como exemplo, "2008-00-22" é equivalente a "2007-12-22".
>
> Se os dois fatos forem combinados e o dia e o mês forem excedidos negativamente, o que segue acontece: "2008-00-00" primeiro será convertido para "2007-12-00" que então será convertido para "2007-11-30". Isso também acontece com a string "0000-00-00", que é transformada em "-0001-11-30" (o ano -1 ano calendário ISO 8601, que é 2 anos DC no calendário que antecede o calendário gregoriano).

Formatos Compostos

Esta página descreve os diferentes formatos compostos de data/horário em uma sintaxe BNF, que os analisadores de DateTimeImmutable, DateTime, date_create(), date_create_immutable() e strtotime() entendem.

Para formatar objetos DateTimeImmutable e DateTime, favor se referir à documentação do método DateTimeInterface::format().

Símbolos Utilizados

Descrição	Formato	Exemplos
`DD`	"0" [0-9] | [1-2][0-9] | "3" [01]	"02", "12", "31"
`doy`	"00"[1-9] | "0"[1-9][0-9] | [1-2][0-9][0-9] | "3"[0-5][0-9] | "36"[0-6]	"001", "012", "180", "350", "366"
`frac`	. [0-9]+	".21342", ".85"
`hh`	"0"?[1-9] | "1"[0-2]	"04", "7", "12"
`HH`	[01][0-9] | "2"[0-4]	"04", "07", "19"
`meridian`	[AaPp] .? [Mm] .? [\0\t]	"A.m.", "pM", "am."
`ii`	[0-5]?[0-9]	"04", "8", "59"
`II`	[0-5][0-9]	"04", "08", "59"
`M`	'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec'	
`MM`	[0-1][0-9]	"00", "12"
`space`	[\t]	
`ss`	[0-5]?[0-9]|60	"04", "8", "59"

Descrição	Formato	Exemplos
`SS`	[0-5][0-9]	"04", "08", "59", "60" (segundo intercalar)
`W`	"0"[1-9] | [1-4][0-9] | "5"[0-3]	"05", "17", "53"
`tzcorrection`	"GMT"? [+-] `hh` ":"? `II`?	"+0400", "GMT-07:00", "-07:00"
`YY`	[0-9]{4}	"2000", "2008", "1978"

Formatos padrões

Descrição	Exemplos
ATOM	"2022-06-02T16:58:35+00:00"
COOKIE	"Thursday, 02-Jun-2022 16:58:35 UTC"
ISO8601	"2022-06-02T16:58:35+0000"
» RFC 822	"Thu, 02 Jun 22 16:58:35 +0000"
» RFC 850	"Thursday, 02-Jun-22 16:58:35 UTC"
» RFC 1036	"Thu, 02 Jun 22 16:58:35 +0000"
» RFC 1123	"Thu, 02 Jun 2022 16:58:35 +0000"
» RFC 2822	"Thu, 02 Jun 2022 16:58:35 +0000"
» RFC 3339	"2022-06-02T16:58:35+00:00"
» RFC 3339 estendida	"2022-06-02T16:58:35.698+00:00"
» RFC 7231	"Thu, 02 Jun 2022 16:58:35 GMT"
RSS	"Thu, 02 Jun 2022 16:58:35 +0000"
W3C	"2022-06-02T16:58:35+00:00"

Notações Localizadas

Descrição	Formato	Exemplos
Formato de Log Popular	`dd` "/" `M` "/" `YY` : `HH` ":" `II` ":" `SS` space `tzcorrection`	"10/Oct/2000:13:55:36 -0700"
EXIF	`YY` ":" `MM` ":" `DD` " " `HH` ":" `II` ":" `SS`	"2008:08:07 18:11:31"
Ano ISO com Semana ISO	`YY` "-"? "W" `W`	"2008W27", "2008-W28"
Ano ISO com Semana ISO e dia	`YY` "-"? "W" `W` "-"? [0-7]	"2008W273", "2008-W28-3"
MySQL	`YY` "-" `MM` "-" `DD` " " `HH` ":" `II` ":" `SS`	"2008-08-07 18:11:31"
PostgreSQL: Ano com dia do ano	`YY` "."? `doy`	"2008.197", "2008197"
SOAP	`YY` "-" `MM` "-" `DD` "T" `HH` ":" `II` ":" `SS` `frac` `tzcorrection`?	"2008-07-01T22:35:17.02", "2008-07-01T22:35:17.03+08:00"
Timestamp Unix	"@" "-"? [0-9]+	"@1215282385"
Timestamp Unix com microssegundos	"@" "-"? [0-9]+ "." [0-9]{0,6}	"@1607974647.503686"
XMLRPC	`YY` `MM` `DD` "T" `hh` ":" `II` ":" `SS`	"20080701T22:38:07", "20080701T9:38:07"
XMLRPC (Compacto)	`YY` `MM` `DD` 't' `hh` `II` `SS`	"20080701t223807", "20080701T093807"
WDDX	`YY` "-" `mm` "-" `dd` "T" `hh` ":" `ii` ":" `ss`	"2008-7-1T9:3:37"

> **Nota** :
>
> O "W" nos formatos "Ano ISO com semana ISO" e "Ano ISO com semana ISO e dia" são sensíveis a maiúsculas/minúsculas, só pode ser usado o "W" maiúsculo.
>
> O "T", nos formatos SOAP, XMLRPC e WDDX é sensível a maiúsculas/minúsculas, só pode ser usado o "T" maiúsculo.
>
> O formato "Timestamp Unix" define o fuso horário para UTC.

Formatos relativos

Esta página descreve os diferentes formatos de data/horário relativos em uma sintaxe BNF, que os analisadores de DateTimeImmutable, DateTime, date_create(), date_create_immutable() e strtotime() entendem.

Para formatar objetos DateTimeImmutable e DateTime, favor se referir à documentação do método DateTimeInterface::format().

Símbolos Utilizados

Descrição	Formato
`dayname`	'sunday' | 'monday' | 'tuesday' | 'wednesday' | 'thursday' | 'friday' | 'saturday' | 'sun' | 'mon' | 'tue' | 'wed' | 'thu' | 'fri' | 'sat'
`daytext`	'weekday' | 'weekdays'
`number`	[+-]?[0-9]+
`ordinal`	'first' | 'second' | 'third' | 'fourth' | 'fifth' | 'sixth' | 'seventh' | 'eighth' | 'ninth' | 'tenth' | 'eleventh' | 'twelfth' | 'next' | 'last' | 'previous' | 'this'
`reltext`	'next' | 'last' | 'previous' | 'this'
`space`	[\t]+
`unit`	'ms' | 'µs' | (('msec' | 'millisecond' | 'µsec' | 'microsecond' | 'usec' | 'sec' | 'second' | 'min' | 'minute' | 'hour' | 'day' | 'fortnight' | 'forthnight' | 'month' | 'year') 's'?) | 'weeks' | `daytext`

Notação baseada no dia

Formato	Descrição	Exemplos
'yesterday'	Meia-noite de ontem	"yesterday 14:00"
'midnight'	O horário é definido para 00:00:00	
'today'	O horário é definido para 00:00:00	
'now'	Agora - isto é simplesmente ignorado	
'noon'	O horário é definido para 12:00:00	"yesterday noon"
'tomorrow'	Meia-noite de amanhã	
'back of' `hour`	15 minutos após o horário informado	"back of 7pm", "back of 15"
'front of' `hour`	15 minutos antes do horário informado	"front of 5am", "front of 23"
'first day of'	Define o dia para o primeiro dia do mês atual. Essa frase é normalmente usada da melhor forma em conjunto com um nome de mês, já que ela somente afeta o mês atual.	"first day of January 2008"
'last day of'	Define o dia para o último do mês corrente. Essa frase é normalmente usada da melhor forma em conjunto com um nome de mês, já que ela somente afeta o mês atual.	"last day of next month"
`ordinal space dayname space` 'of'	Calcula o número ordinal `x`-th da semana do mês corrente.	"first sat of July 2008"
'last' `space dayname space` 'of'	Calcular o *último* dia da semana do mês corrente.	"last sat of July 2008"
`number space?` (`unit` | 'week')	Manipula porções de tempo relativas onde o valor seja um número.	"+5 weeks", "12 day", "-7 weekdays"
(`ordinal` | `reltext`) `space unit`	Manipula porções de tempo relativas onde o valor seja um texto. `last` e `previous` são equivalentes a `-1`, `this` equivalente a nada, e `next` equivamente a `+1`.	"fifth day", "second month", "last day", "previous year"
'ago'	Nega todos os valores das porções de tempo relativas anteriores.	"2 days ago", "8 days ago 14:00", "2 months 5 days ago", "2 months ago 5 days", "2 days ago"
`dayname`	Move para o próximo dia deste nome.(See note)	"Monday"
`reltext space` 'week'	Manipula o formato especial "weekday + last/this/next week".	"Monday next week"

> **Nota** :
>
> Declarações relativas são sempre processadas *depois* de declarações não relativas. Isso faz com que "+1 week july 2008" e "july 2008 +1 week" sejam equivalentes.

As exceções desta regra são "yesterday", "midnight", "today", "noon" e "tomorrow". Observe que "tomorrow 11:00" e "11:00 tomorrow" são diferentes. Considerando que a data hoje como "July 23rd, 2008", o primeiro produzirá "2008-07-24 11:00" assim como o segundo produzirá "2008-07-24 00:00". Isso acontece por que as cinco declarações influenciam diretamente o tempo corrente.

Palavras-chave como "first day of" dependem do contexto no qual o string de formato relativo é usado. Se usado com um método ou funcão estáticos, a referência é o timestamp atual do sistema. Entretanto, se usado em DateTime::modify() ou DateTimeImmutable::modify(), a referência é o objeto no qual o método `modify()` é chamado.

Nota :

Observe as seguintes considerações quando o dia da semana corrente é o mesmo que o dia da semana utilizada na string date/horário. O dia da semana corrente pode ser (re-)calculado para porções não relativas de strings de data/horário.

1. "`dayname`" *não* avança para outro dia. (Exemplo: "Wed July 23rd, 2008" referencia "2008-07-23").
2. "`number dayname`" *não* avança para outro dia. (Exemplo: "1 wednesday july 23rd, 2008" referencia "2008-07-23").
3. "`number` week `dayname`" adicionará primeiro o dia da semana, mas *not* avançará para outro dia. Neste caso, "`number` week" e "`dayname`" são dois blocos distintos. (Exemplo: "+1 week wednesday july 23rd, 2008" referencia "2008-07-30").
4. "`ordinal dayname`" *avançará*para outro dia. (Exemplo "first wednesday july 23rd, 2008" referencia "2008-07-30").
5. "`number` week `ordinal dayname`" adicionará primeiro o número de semanas, e então *avançará* para outro dia. Neste caso "`number` week" e "`ordinal dayname` são dois blocos distintos. (Exemplo: "+1 week first wednesday july 23rd, 2008" referencia "2008-08-06").
6. "`ordinal dayname` 'of' " *não* avança para outro dia. (Exemplo: "first wednesday of july 23rd, 2008" referencia "2008-07-02" porque a frase específica com 'of' redefine o dia do mês para '1' e o '23rd' é ignorado).

Também observe que o "of" em "`ordinal space dayname space` 'of' " e "'last' `space dayname space` 'of' " fazem coisas especiais.

1. Ele definirá o dia do mês para 1.
2. "`ordinal dayname` 'of' " *não* avançará para outro dia. (Exemplo: "first tuesday of july 2008" referencia "2008-07-01").
3. "`ordinal dayname` " *avançará* para outro dia. (Exemplo: "first tuesday july 2008" referencia "2008-07-08", veja também o item 4 na lista acima).
4. "'last' `dayname` 'of' " captura o último `dayname` do mês corrente. (Exemplo: "last wed of july 2008" referencia "2008-07-30")
5. "'last' `dayname`" captura o último `dayname` do mês corrente. (Exemplo: "last wed july 2008" referencia "2008-06-25"; "july 2008" primeiro definirá a data corrente para "2008-07-01" e então "last wed" moverá para a última Quarta-feira que será "2008-06-25").

Nota :

Meses com valores relativos são calculados baseados no tamanho do mês passado. Um exemplo seria "+2 month 2011-11-30",que produziria "2012-01-30". Isso é devido a Novembro tendo 30 dias como tamanho, e Dezembro tendo 31, produzindo um total de 61 dias.

Nota :

`number` é um número *inteiro*; se um número decimal é informado, o ponto (ou vírgula) provavelmente será interpretado como delimitador. Por exemplo, `'+1.5 hours'` é analisado como `'+1 5 hours'`, não como `'+1 hour +30 minutes'`.

Registro de Alterações

Versão	Descrição
8.4.0	`number` agora aceita novamente um sinal positivo seguido por um sinal negativo, por exemplo, `+-2` e outras combinações de múltiplos sinais.
8.2.0	`number` não aceita mais um sinal positivo seguido por um sinal negativo, por exemplo, `+-2`.

Versão	Descrição
7.0.8	Semanas sempre começam na segunda-feira. Antes, considerava-se que o domingo também poderia começar uma semana.

Lista de Fusos Horários Suportados

Índice

Aqui pode ser encontrada a lista completa de fusos horários suportados pelo PHP, os quais podem ser usados, por exemplo, com date_default_timezone_set().

> **Cuidado**
>
> O comportamento dos fusos horários não listados aqui é indefinido.

> **Nota** :
>
> A versão mais recente do banco de dados de fusos horários pode ser instalada através do pacote PECL » timezonedb.

> **Nota** :
>
> Esta lista é baseada na versão do banco de dados de fusos horários 2026.5.

África

África			
Africa/Abidjan	Africa/Accra	Africa/Addis_Ababa	Africa/Algiers
Africa/Asmara	Africa/Bamako	Africa/Bangui	Africa/Banjul
Africa/Bissau	Africa/Blantyre	Africa/Brazzaville	Africa/Bujumbura
Africa/Cairo	Africa/Casablanca	Africa/Ceuta	Africa/Conakry
Africa/Dakar	Africa/Dar_es_Salaam	Africa/Djibouti	Africa/Douala
Africa/El_Aaiun	Africa/Freetown	Africa/Gaborone	Africa/Harare
Africa/Johannesburg	Africa/Juba	Africa/Kampala	Africa/Khartoum
Africa/Kigali	Africa/Kinshasa	Africa/Lagos	Africa/Libreville
Africa/Lome	Africa/Luanda	Africa/Lubumbashi	Africa/Lusaka
Africa/Malabo	Africa/Maputo	Africa/Maseru	Africa/Mbabane
Africa/Mogadishu	Africa/Monrovia	Africa/Nairobi	Africa/Ndjamena
Africa/Niamey	Africa/Nouakchott	Africa/Ouagadougou	Africa/Porto-Novo
Africa/Sao_Tome	Africa/Tripoli	Africa/Tunis	Africa/Windhoek

América

América			
America/Adak	America/Anchorage	America/Anguilla	America/Antigua
America/Araguaina	America/Argentina/Buenos_Aires	America/Argentina/Catamarca	America/Argentina/Cordoba
America/Argentina/Jujuy	America/Argentina/La_Rioja	America/Argentina/Mendoza	America/Argentina/Rio_Gallegos
America/Argentina/Salta	America/Argentina/San_Juan	America/Argentina/San_Luis	America/Argentina/Tucuman
America/Argentina/Ushuaia	America/Aruba	America/Asuncion	America/Atikokan
America/Bahia	America/Bahia_Banderas	America/Barbados	America/Belem
America/Belize	America/Blanc-Sablon	America/Boa_Vista	America/Bogota
America/Boise	America/Cambridge_Bay	America/Campo_Grande	America/Cancun
America/Caracas	America/Cayenne	America/Cayman	America/Chicago
America/Chihuahua	America/Ciudad_Juarez	America/Costa_Rica	America/Coyhaique

America/Creston	America/Cuiaba	America/Curacao	America/Danmarkshavn
America/Dawson	America/Dawson_Creek	America/Denver	America/Detroit
America/Dominica	America/Edmonton	America/Eirunepe	America/El_Salvador
America/Fort_Nelson	America/Fortaleza	America/Glace_Bay	America/Goose_Bay
America/Grand_Turk	America/Grenada	America/Guadeloupe	America/Guatemala
America/Guayaquil	America/Guyana	America/Halifax	America/Havana
America/Hermosillo	America/Indiana/Indianapolis	America/Indiana/Knox	America/Indiana/Marengo
America/Indiana/Petersburg	America/Indiana/Tell_City	America/Indiana/Vevay	America/Indiana/Vincennes
America/Indiana/Winamac	America/Inuvik	America/Iqaluit	America/Jamaica
America/Juneau	America/Kentucky/Louisville	America/Kentucky/Monticello	America/Kralendijk
America/La_Paz	America/Lima	America/Los_Angeles	America/Lower_Princes
America/Maceio	America/Managua	America/Manaus	America/Marigot
America/Martinique	America/Matamoros	America/Mazatlan	America/Menominee
America/Merida	America/Metlakatla	America/Mexico_City	America/Miquelon
America/Moncton	America/Monterrey	America/Montevideo	America/Montserrat
America/Nassau	America/New_York	America/Nome	America/Noronha
America/North_Dakota/Beulah	America/North_Dakota/Center	America/North_Dakota/New_Salem	America/Nuuk
America/Ojinaga	America/Panama	America/Paramaribo	America/Phoenix
America/Port-au-Prince	America/Port_of_Spain	America/Porto_Velho	America/Puerto_Rico
America/Punta_Arenas	America/Rankin_Inlet	America/Recife	America/Regina
America/Resolute	America/Rio_Branco	America/Santarem	America/Santiago
America/Santo_Domingo	America/Sao_Paulo	America/Scoresbysund	America/Sitka
America/St_Barthelemy	America/St_Johns	America/St_Kitts	America/St_Lucia
America/St_Thomas	America/St_Vincent	America/Swift_Current	America/Tegucigalpa
America/Thule	America/Tijuana	America/Toronto	America/Tortola
America/Vancouver	America/Whitehorse	America/Winnipeg	America/Yakutat

Antártica

Antártica			
Antarctica/Casey	Antarctica/Davis	Antarctica/DumontDUrville	Antarctica/Macquarie
Antarctica/Mawson	Antarctica/McMurdo	Antarctica/Palmer	Antarctica/Rothera
Antarctica/Syowa	Antarctica/Troll	Antarctica/Vostok	

Ártico

Ártico
Arctic/Longyearbyen

Ásia

Ásia			
Asia/Aden	Asia/Almaty	Asia/Amman	Asia/Anadyr
Asia/Aqtau	Asia/Aqtobe	Asia/Ashgabat	Asia/Atyrau
Asia/Baghdad	Asia/Bahrain	Asia/Baku	Asia/Bangkok
Asia/Barnaul	Asia/Beirut	Asia/Bishkek	Asia/Brunei
Asia/Chita	Asia/Colombo	Asia/Damascus	Asia/Dhaka
Asia/Dili	Asia/Dubai	Asia/Dushanbe	Asia/Famagusta
Asia/Gaza	Asia/Hebron	Asia/Ho_Chi_Minh	Asia/Hong_Kong
Asia/Hovd	Asia/Irkutsk	Asia/Jakarta	Asia/Jayapura

Asia/Jerusalem	Asia/Kabul	Asia/Kamchatka	Asia/Karachi
Asia/Kathmandu	Asia/Khandyga	Asia/Kolkata	Asia/Krasnoyarsk
Asia/Kuala_Lumpur	Asia/Kuching	Asia/Kuwait	Asia/Macau
Asia/Magadan	Asia/Makassar	Asia/Manila	Asia/Muscat
Asia/Nicosia	Asia/Novokuznetsk	Asia/Novosibirsk	Asia/Omsk
Asia/Oral	Asia/Phnom_Penh	Asia/Pontianak	Asia/Pyongyang
Asia/Qatar	Asia/Qostanay	Asia/Qyzylorda	Asia/Riyadh
Asia/Sakhalin	Asia/Samarkand	Asia/Seoul	Asia/Shanghai
Asia/Singapore	Asia/Srednekolymsk	Asia/Taipei	Asia/Tashkent
Asia/Tbilisi	Asia/Tehran	Asia/Thimphu	Asia/Tokyo
Asia/Tomsk	Asia/Ulaanbaatar	Asia/Urumqi	Asia/Ust-Nera
Asia/Vientiane	Asia/Vladivostok	Asia/Yakutsk	Asia/Yangon
Asia/Yekaterinburg	Asia/Yerevan		

Atlântico

Atlântico			
Atlantic/Azores	Atlantic/Bermuda	Atlantic/Canary	Atlantic/Cape_Verde
Atlantic/Faroe	Atlantic/Madeira	Atlantic/Reykjavik	Atlantic/South_Georgia
Atlantic/St_Helena	Atlantic/Stanley		

Austrália

Austrália			
Australia/Adelaide	Australia/Brisbane	Australia/Broken_Hill	Australia/Darwin
Australia/Eucla	Australia/Hobart	Australia/Lindeman	Australia/Lord_Howe
Australia/Melbourne	Australia/Perth	Australia/Sydney	

Europa

Europa			
Europe/Amsterdam	Europe/Andorra	Europe/Astrakhan	Europe/Athens
Europe/Belgrade	Europe/Berlin	Europe/Bratislava	Europe/Brussels
Europe/Bucharest	Europe/Budapest	Europe/Busingen	Europe/Chisinau
Europe/Copenhagen	Europe/Dublin	Europe/Gibraltar	Europe/Guernsey
Europe/Helsinki	Europe/Isle_of_Man	Europe/Istanbul	Europe/Jersey
Europe/Kaliningrad	Europe/Kirov	Europe/Kyiv	Europe/Lisbon
Europe/Ljubljana	Europe/London	Europe/Luxembourg	Europe/Madrid
Europe/Malta	Europe/Mariehamn	Europe/Minsk	Europe/Monaco
Europe/Moscow	Europe/Oslo	Europe/Paris	Europe/Podgorica
Europe/Prague	Europe/Riga	Europe/Rome	Europe/Samara
Europe/San_Marino	Europe/Sarajevo	Europe/Saratov	Europe/Simferopol
Europe/Skopje	Europe/Sofia	Europe/Stockholm	Europe/Tallinn
Europe/Tirane	Europe/Ulyanovsk	Europe/Vaduz	Europe/Vatican
Europe/Vienna	Europe/Vilnius	Europe/Volgograd	Europe/Warsaw
Europe/Zagreb	Europe/Zurich		

Índia

Índia			
Indian/Antananarivo	Indian/Chagos	Indian/Christmas	Indian/Cocos

Indian/Comoro	Indian/Kerguelen	Indian/Mahe	Indian/Maldives
Indian/Mauritius	Indian/Mayotte	Indian/Reunion	

Pacífico

Pacífico

Pacific/Apia	Pacific/Auckland	Pacific/Bougainville	Pacific/Chatham
Pacific/Chuuk	Pacific/Easter	Pacific/Efate	Pacific/Fakaofo
Pacific/Fiji	Pacific/Funafuti	Pacific/Galapagos	Pacific/Gambier
Pacific/Guadalcanal	Pacific/Guam	Pacific/Honolulu	Pacific/Kanton
Pacific/Kiritimati	Pacific/Kosrae	Pacific/Kwajalein	Pacific/Majuro
Pacific/Marquesas	Pacific/Midway	Pacific/Nauru	Pacific/Niue
Pacific/Norfolk	Pacific/Noumea	Pacific/Pago_Pago	Pacific/Palau
Pacific/Pitcairn	Pacific/Pohnpei	Pacific/Port_Moresby	Pacific/Rarotonga
Pacific/Saipan	Pacific/Tahiti	Pacific/Tarawa	Pacific/Tongatapu
Pacific/Wake	Pacific/Wallis		

Outros

> **Aviso**
>
> Não use nenhum dos fusos horários listados aqui (exceto o UTC), pois eles existem apenas por motivos de compatibilidade com versões anteriores e podem ter comportamento errôneo. Além disso, esses fusos horários podem ser removidos do banco de dados de fusos horários da IANA a qualquer momento.

> **Aviso**
>
> Se o aviso acima for desconsiderado, observe também que o banco de dados de fusos horários da IANA que fornece suporte aos fusos horários do PHP usa sinais no estilo POSIX, o que resulta nos fusos horários `Etc/GMT+n` e `Etc/GMT-n` sendo invertidos do uso comum.
>
> Por exemplo, o fuso horário 8 horas à frente do GMT usado na China e na Austrália Ocidental (entre outros lugares) é, na verdade, `Etc/GMT-8` neste banco de dados, e não `Etc/GMT+8`, como normalmente esperado.
>
> Novamente, é altamente recomendável que seja usado o fuso horário correto para sua localização, como `Asia/Shanghai` ou `Australia/Perth` nos exemplos acima.

Outros

Africa/Asmera	Africa/Timbuktu	America/Argentina/ComodRivadavia	America/Atka
America/Buenos_Aires	America/Catamarca	America/Coral_Harbour	America/Cordoba
America/Ensenada	America/Fort_Wayne	America/Godthab	America/Indianapolis
America/Jujuy	America/Knox_IN	America/Louisville	America/Mendoza
America/Montreal	America/Nipigon	America/Pangnirtung	America/Porto_Acre
America/Rainy_River	America/Rosario	America/Santa_Isabel	America/Shiprock
America/Thunder_Bay	America/Virgin	America/Yellowknife	Antarctica/South_Pole
Asia/Ashkhabad	Asia/Calcutta	Asia/Choibalsan	Asia/Chongqing
Asia/Chungking	Asia/Dacca	Asia/Harbin	Asia/Istanbul
Asia/Kashgar	Asia/Katmandu	Asia/Macao	Asia/Rangoon
Asia/Saigon	Asia/Tel_Aviv	Asia/Thimbu	Asia/Ujung_Pandang
Asia/Ulan_Bator	Atlantic/Faeroe	Atlantic/Jan_Mayen	Australia/ACT
Australia/Canberra	Australia/Currie	Australia/LHI	Australia/North
Australia/NSW	Australia/Queensland	Australia/South	Australia/Tasmania
Australia/Victoria	Australia/West	Australia/Yancowinna	Brazil/Acre

Brazil/DeNoronha	Brazil/East	Brazil/West	Canada/Atlantic
Canada/Central	Canada/Eastern	Canada/Mountain	Canada/Newfoundland
Canada/Pacific	Canada/Saskatchewan	Canada/Yukon	CET
Chile/Continental	Chile/EasterIsland	CST6CDT	Cuba
EET	Egypt	Eire	EST
EST5EDT	Etc/GMT	Etc/GMT+0	Etc/GMT+1
Etc/GMT+10	Etc/GMT+11	Etc/GMT+12	Etc/GMT+2
Etc/GMT+3	Etc/GMT+4	Etc/GMT+5	Etc/GMT+6
Etc/GMT+7	Etc/GMT+8	Etc/GMT+9	Etc/GMT-0
Etc/GMT-1	Etc/GMT-10	Etc/GMT-11	Etc/GMT-12
Etc/GMT-13	Etc/GMT-14	Etc/GMT-2	Etc/GMT-3
Etc/GMT-4	Etc/GMT-5	Etc/GMT-6	Etc/GMT-7
Etc/GMT-8	Etc/GMT-9	Etc/GMT0	Etc/Greenwich
Etc/UCT	Etc/Universal	Etc/UTC	Etc/Zulu
Europe/Belfast	Europe/Kiev	Europe/Nicosia	Europe/Tiraspol
Europe/Uzhgorod	Europe/Zaporozhye	Factory	GB
GB-Eire	GMT	GMT+0	GMT-0
GMT0	Greenwich	Hongkong	HST
Iceland	Iran	Israel	Jamaica
Japan	Kwajalein	Libya	MET
Mexico/BajaNorte	Mexico/BajaSur	Mexico/General	MST
MST7MDT	Navajo	NZ	NZ-CHAT
Pacific/Enderbury	Pacific/Johnston	Pacific/Ponape	Pacific/Samoa
Pacific/Truk	Pacific/Yap	Poland	Portugal
PRC	PST8PDT	ROC	ROK
Singapore	Turkey	UCT	Universal
US/Alaska	US/Aleutian	US/Arizona	US/Central
US/East-Indiana	US/Eastern	US/Hawaii	US/Indiana-Starke
US/Michigan	US/Mountain	US/Pacific	US/Samoa
UTC	W-SU	WET	Zulu

DateError

(PHP 8 >= 8.3.0)

Introdução

Disparada quando o banco de dados de fuso horário não for encontrado ou contiver dados inválidos.

Este erro nunca deveria ocorrer, e não é dependente do código. Existem duas exceções-filhas (DateObjectError e DateRangeError) que são disparadas dependendo do erro do programados ou de problemas relacionados a faixas de datas.

Resumo da classe

```
class DateError extends Error {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
```

/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Veja também

- DateObjectError
- DateRangeError

DateObjectError

(PHP 8 >= 8.3.0)

Introdução

Disparada quando uma das classe de Data/Horário não foi corretamente inicializada.

Como as classes de Data/Horário não são finais, eles podem ser herdadas. Quando um construtor da classe superior não for chamado, este erro é lançado. Este é sempre um erro de programação.

Resumo da classe

class **DateObjectError** extends DateError {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Veja também

- DateError
- DateRangeError

DateRangeError

(PHP 8 >= 8.3.0)

Introdução

Disparada por DateTime::getTimestamp(), DateTimeImmutable::getTimestamp() e date_timestamp_get()em plataformas de 32 bits se o objeto representa uma data fora da faixa de inteiros de 32 bits com sinal.

Resumo da classe

```
class DateRangeError extends DateError {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

Veja também

- DateError
- DateObjectError

DateException

(PHP 8 >= 8.3.0)

Introdução

Classe mãe das exceções de Data/Horário, para problemas que surgem devido a entradas do usuário ou argumentos texto em formato livre que precisam ser analisados.

As exceções-filhas a seguir são disparadas pela extensão:

- DateInvalidOperationException
- DateInvalidTimezoneException
- DateMalformedIntervalStringException
- DateMalformedPeriodStringException
- DateMalformedStringException

Resumo da classe

```
class DateException extends Exception {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
```

final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateInvalidOperationException

(PHP 8 >= 8.3.0)

Introdução

Disparada por DateTimeImmutable::sub() e DateTime::sub() quando uma operação não suportada é solicitada.

Um exemplo de operação não suportada é o uso de um objeto DateInterval representando especificação de tempo relativo como `next weekday`, que não permite construir um objeto por ser inválido.

Resumo da classe

class **DateInvalidOperationException** extends DateException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateInvalidTimeZoneException

(PHP 8 >= 8.3.0)

Introdução

Disparada quando um valor incorreto é passado para DateTimeZone::__construct().

Resumo da classe

class **DateInvalidTimeZoneException** extends DateException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];

private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedIntervalStringException

(PHP 8 >= 8.3.0)

Introdução

Disparada quando um argumento `duration` inválido é passado para DateInterval::__construct().

Resumo da classe

class **DateMalformedIntervalStringException** extends DateException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedPeriodStringException

(PHP 8 >= 8.3.0)

Introdução

Disparaqa quando um argumento `isostr` inválido é passado para DatePeriod::__construct().

Resumo da classe

class **DateMalformedPeriodStringException** extends DateException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];

private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedStringException

(PHP 8 >= 8.3.0)

Introdução

Disparada quando uma string inválida de Data/Horário é detectada.

Pode ser um valor para DateTimeImmutable::__construct(), DateTimeImmutable::modify(), DateTime::__construct() ou DateTime::modify().

Resumo da classe

class **DateMalformedStringException** extends DateException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Notação de Objetos do JavaScript

Introdução

Esta extensão implementa o formato de intercâmbio de dados » Notação de Objetos do JavaScript (JSON). O analisador foi escrito especificamente para o PHP e licenciado sob a licença do PHP.

> **Nota** :
>
> O PHP implementa um superconjunto de JSON conforme especificado na » RFC 7159 original.

Instalação/Configuração

Índice

Instalação

A extensão JSON é uma extensão do núcleo do PHP, por isso está sempre ativada.

Anteriormente ao PHP 8.0.0, a extensão JSON era compilada e incluída no PHP por padrão mas podia ser desativada explicitamente usando **--disable-json**.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

As seguintes constantes indicam o tipo de erro retornado pela função json_last_error() ou armazenadas no *code* de uma JsonException

`JSON_ERROR_NONE` (int)

Sem erros.

`JSON_ERROR_DEPTH` (int)

O limite de pilha de chamadas foi ultrapassado.

`JSON_ERROR_STATE_MISMATCH` (int)

Ocorre em underflows ou com incongruência de modos.

`JSON_ERROR_CTRL_CHAR` (int)

Erro em caracter de controle, possivelmente erro de codificação.

`JSON_ERROR_SYNTAX` (int)

Erro de sintaxe.

`JSON_ERROR_UTF8` (int)

Caracteres UTF-8 mal formados, possivelmente erro de codificação.

`JSON_ERROR_RECURSION` (int)

O objeto ou array passado para json_encode() inclui referências recursivas, e não pode ser formatada. Se a opção `JSON_PARTIAL_OUTPUT_ON_ERROR` foi informada, `null` será substituído no lugar da referência recursiva.

`JSON_ERROR_INF_OR_NAN` (int)

Um valor passado para json_encode() inclui `NAN` ou `INF`. Se a opção `JSON_PARTIAL_OUTPUT_ON_ERROR` foi informada, `0` será substituído no lugar do número especial.

`JSON_ERROR_UNSUPPORTED_TYPE` (int)

Um valor de um tipo não suportado foi informado para json_encode(), por exemplo um resource. Se a opção `JSON_PARTIAL_OUTPUT_ON_ERROR` foi informada, `null` será substituída ao invés do valor não suportado.

`JSON_ERROR_INVALID_PROPERTY_NAME` (int)

Uma chave começando com caracter \u0000 estava na chave string foi passada para json_decode() quando decodificando um objeto JSON para um objeto PHP.

`JSON_ERROR_UTF16` (int)

Substituto UTF-16 único não pareado em escape unicode contido na string JSON passada para json_decode().

`JSON_ERROR_NON_BACKED_ENUM` (int)

O valor passado para json_encode() inclui uma enumeração não apoiada que não pode ser serializada. Disponível a partir do PHP 8.1.0.

As constantes a seguir podem ser combinadas para formar opções na função json_decode().

`JSON_BIGINT_AS_STRING` (int)

Decodifica inteiros grandes como suas strings originais.

`JSON_OBJECT_AS_ARRAY` (int)

Decodifica objetos JSON como arrays PHP. Esta opção pode ser adicionada automaticamente quando chamando json_decode() com o segundo parâmetro igual a `true`.

As seguintes constantes podem ser combinadas para formar opções para a função json_encode().

`JSON_HEX_TAG` (int)

Todos os caracteres < e > serão convertidos para \u003C e \u003E.

`JSON_HEX_AMP` (int)

Todos os caracteres & serão convertidos para \u0026.

`JSON_HEX_APOS` (int)

Todos os caracteres ' serão convertidos para \u0027.

`JSON_HEX_QUOT` (int)

Todos os caracteres " serão convertidos para \u0022.

`JSON_FORCE_OBJECT` (int)

Imprime um objeto em vez de um array quando um array não recursivo é usado. Muito útil quando o chamador espera um objeto mas o array está vazio.

`JSON_NUMERIC_CHECK` (int)

Codifica strings numéricas como números.

`JSON_PRETTY_PRINT` (int)

Formata os dados retornados com espaços em branco.

`JSON_UNESCAPED_SLASHES` (int)

Não escapa o caracter `/`.

`JSON_UNESCAPED_UNICODE` (int)

Codifica caracteres Unicode multibyte literalmente (default é formatar como \uXXXX).

`JSON_PARTIAL_OUTPUT_ON_ERROR` (int)

Substitui valores não identificados ao invés de falhar.

`JSON_PRESERVE_ZERO_FRACTION` (int)

Garante que float sempre são codificados como um número float.

`JSON_UNESCAPED_LINE_TERMINATORS` (int)

Os fins de linha são preservados não escapados quando `JSON_UNESCAPED_UNICODE` é informado. Este é o mesmo comportamento existente anteriormente ao PHP 7.1 sem essa constante. Disponível desde o PHP 7.1.0.

As constantes a seguir podem ser combinadas para formar opções para a função json_decode() e json_encode().

`JSON_INVALID_UTF8_IGNORE` (int)

Ignora caracteres inválidos UTF-8. Disponível desde o 7.2.0.

`JSON_INVALID_UTF8_SUBSTITUTE` (int)

Converte caracteres inválidos UTF-8 para \0xfffd (Caracter Unicode 'REPLACEMENT CHARACTER') Disponível desde o 7.2.0.

`JSON_THROW_ON_ERROR` (int)

Lança um JsonException se ocorrer um erro ao invés de carregar o erro global que pode ser obtido com json_last_error() e json_last_error_msg(). `JSON_PARTIAL_OUTPUT_ON_ERROR` tem precedência sobre `JSON_THROW_ON_ERROR`. Disponível desde o PHP 7.3.0.

JsonException

(PHP 7 >= 7.3.0, PHP 8)

Introdução

Uma exceção será lançada se a opção `JSON_THROW_ON_ERROR` for definida para json_encode() ou json_decode(). *code* contém o tipo de erro, para valores possíveis veja json_last_error().

Resumo da classe

class **JsonException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

JsonSerializable

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introdução

Os objetos que implementam **JsonSerializable** podem personalizar sua representação JSON quando codificados com json_encode().

Resumo da Interface

interface **JsonSerializable** {
/* Métodos */

public function jsonSerialize(): mixed
}

Índice

JsonSerializable::jsonSerialize

JsonSerializable::jsonSerialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

JsonSerializable::jsonSerialize — Especifica os dados que devem ser serializados para JSON

Descrição

public function **JsonSerializable::jsonSerialize**(): mixed

Serializa o objeto para um valor que pode ser serializado nativamente por json_encode().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna dados que podem ser serializados por json_encode(), que é um valor de qualquer tipo que não seja um resource.

Examples

Exemplo #1 Exemplo de JsonSerializable::jsonSerialize() retornando um array

```php
<?php
class ArrayValue implements JsonSerializable {
    private $array;
    public function __construct(array $array) {
        $this->array = $array;
    }

    public function jsonSerialize(): mixed {
        return $this->array;
    }
}

$array = [1, 2, 3];
echo json_encode(new ArrayValue($array), JSON_PRETTY_PRINT);
?>
```

O exemplo acima produzirá:

```
[
    1,
    2,
    3
]
```

Exemplo #2 Exemplo de JsonSerializable::jsonSerialize() retornando um array associativo

```php
<?php
class ArrayValue implements JsonSerializable {
    private $array;
    public function __construct(array $array) {
        $this->array = $array;
    }

    public function jsonSerialize(): mixed {
```

```
        return $this->array;
    }
}

$array = ['foo' => 'bar', 'quux' => 'baz'];
echo json_encode(new ArrayValue($array), JSON_PRETTY_PRINT);
?>
```

O exemplo acima produzirá:

```
{
    "foo": "bar",
    "quux": "baz"
}
```

Exemplo #3 Exemplo de JsonSerializable::jsonSerialize() retornando um int

```
<?php
class IntegerValue implements JsonSerializable {
    private $number;
    public function __construct($number) {
        $this->number = (int) $number;
    }

    public function jsonSerialize(): mixed {
        return $this->number;
    }
}

echo json_encode(new IntegerValue(1), JSON_PRETTY_PRINT);
?>
```

O exemplo acima produzirá:

```
1
```

Exemplo #4 Exemplo de JsonSerializable::jsonSerialize() retornando uma string

```
<?php
class StringValue implements JsonSerializable {
    private $string;
    public function __construct($string) {
        $this->string = (string) $string;
    }

    public function jsonSerialize(): mixed {
        return $this->string;
    }
}

echo json_encode(new StringValue('Hello!'), JSON_PRETTY_PRINT);
?>
```

O exemplo acima produzirá:

```
"Hello!"
```

Funções da JSON

Índice

json_decode

json_decode

(PHP 5 >= 5.2.0, PHP 7, PHP 8, PECL json >= 1.2.0)

json_decode — Decodifica uma string JSON

Descrição

```
function json_decode(
    string $json,
    ?bool $associative = null,
    int $depth = 512,
    int $flags = 0
): mixed
```

Recebe uma string codificada em JSON e converte em um valor do PHP.

Parâmetros

`json`

A string `json` a ser decodificada.

Esta função funciona apenas com strings codificadas em UTF-8.

> **Nota** :
>
> O PHP implementa um superconjunto de JSON conforme especificado na » RFC 7159 original.

`associative`

Quando **`true`**, objetos JSON serão retornados como arrays associativo; quando **`false`**, objetos JSON serão retornados como objects. Quando **`null`**, objetos JSON serão retornados como arrays ou objects dependendo se `flags` **`JSON_OBJECT_AS_ARRAY`** estiver definida.

`depth`

Profundidade máxima da estrutura que será decodificada. O valor deve ser maior que `0`, e menor ou igual a `2147483647`.

`flags`

Máscara de bits de **`JSON_BIGINT_AS_STRING`**, **`JSON_INVALID_UTF8_IGNORE`**, **`JSON_INVALID_UTF8_SUBSTITUTE`**, **`JSON_OBJECT_AS_ARRAY`**, **`JSON_THROW_ON_ERROR`**. O comportamento dessas constantes é descrito na página de constantes JSON.

Valor Retornado

Retorna o valor codificado em `json` como um tipo apropriado do PHP. Valores sem aspas `true`, `false` e `null` são retornados como **`true`**, **`false`** e **`null`** respectivamente. **`null`** é retornado se o `json` não puder ser decodificado ou se os dados codificados tiverem profundidade maior que o limite de aninhamento.

Erros/Exceções

Se a `depth` estiver fora do intervalo permitido, um ValueError é lançado a partir do PHP 8.0.0, enquanto anteriormente, era gerado um erro do nível **`E_WARNING`**.

Registro de Alterações

Versão	Descrição
7.3.0	Foi adicionado `JSON_THROW_ON_ERROR` em `flags`
7.2.0	`associative` agora é nullable.
7.2.0	Foi adicionado `JSON_INVALID_UTF8_IGNORE`, e `JSON_INVALID_UTF8_SUBSTITUTE` em `flags`.
7.1.0	Uma chave JSON vazia ("") pode ser codificada para a propriedade vazia de objeto, em vez de usar uma chave com o valor `_empty_`.

Examples

Exemplo #1 **json_decode()** exemplos

```php
<?php
$json = '{"a":1,"b":2,"c":3,"d":4,"e":5}';

var_dump(json_decode($json));
var_dump(json_decode($json, true));

?>
```

O exemplo acima produzirá:

```
object(stdClass)#1 (5) {
    ["a"] => int(1)
    ["b"] => int(2)
    ["c"] => int(3)
    ["d"] => int(4)
    ["e"] => int(5)
}

array(5) {
    ["a"] => int(1)
    ["b"] => int(2)
    ["c"] => int(3)
    ["d"] => int(4)
    ["e"] => int(5)
}
```

Exemplo #2 Acessando propriedades inválidas do objeto

Acessar elementos dentro de um objeto que contém caracteres não permitidos pela convenção de nomenclatura do PHP (por exemplo, o hífen) pode ser realizado encapsulando o nome do elemento entre chaves e apóstrofo.

```php
<?php

$json = '{"foo-bar": 12345}';

$obj = json_decode($json);
print $obj->{'foo-bar'}; // 12345

?>
```

Exemplo #3 erros comuns usando **json_decode()**

```php
<?php

// as seguintes strings são JavaScript válidas, mas não JSON válidos

// o nome e o valor devem ser colocados entre aspas duplas
// aspas simples não são válidas
$bad_json = "{ 'bar': 'baz' }";
json_decode($bad_json); // null
```

```php
// o nome deve ser colocado entre aspas duplas
$bad_json = '{ bar: "baz" }';
json_decode($bad_json); // null

// vírgulas à direita não são permitidas
$bad_json = '{ bar: "baz", }';
json_decode($bad_json); // null

?>
```

Exemplo #4 erros de `depth`

```php
<?php
// Codifica alguns dados com uma profundidade máxima de 4 (array -> array -> array -> string)
$json = json_encode(
    array(
        1 => array(
            'English' => array(
                'One',
                'January'
            ),
            'French' => array(
                'Une',
                'Janvier'
            )
        )
    )
);

// Mostrar os erros para diferentes profundidades.
var_dump(json_decode($json, true, 4));
echo 'Last error: ', json_last_error_msg(), PHP_EOL, PHP_EOL;

var_dump(json_decode($json, true, 3));
echo 'Last error: ', json_last_error_msg(), PHP_EOL, PHP_EOL;
?>
```

O exemplo acima produzirá:

```
array(1) {
  [1]=>
  array(2) {
    ["English"]=>
    array(2) {
      [0]=>
      string(3) "One"
      [1]=>
      string(7) "January"
    }
    ["French"]=>
    array(2) {
      [0]=>
      string(3) "Une"
      [1]=>
      string(7) "Janvier"
    }
  }
}
Last error: No error

NULL
Last error: Maximum stack depth exceeded
```

Exemplo #5 json_decode() de inteiros grandes

```php
<?php
$json = '{"number": 12345678901234567890}';

var_dump(json_decode($json));
var_dump(json_decode($json, false, 512, JSON_BIGINT_AS_STRING));

?>
```

O exemplo acima produzirá:

```
object(stdClass)#1 (1) {
  ["number"]=>
  float(1.2345678901235E+19)
}
object(stdClass)#1 (1) {
  ["number"]=>
  string(20) "12345678901234567890"
}
```

Notas

> **Nota** :
>
> A especificação JSON não é JavaScript, mas um subconjunto de JavaScript.

> **Nota** :
>
> No caso de falha na decodificação, json_last_error() pode ser usado para determinar a exata natureza do erro.

Veja também

- json_encode() - Retorna a representação JSON de um valor
- json_last_error() - Retorna o último erro ocorrido
- json_last_error_msg() - Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode()

json_encode

json_encode

(PHP 5 >= 5.2.0, PHP 7, PHP 8, PECL json >= 1.2.0)

json_encode — Retorna a representação JSON de um valor

Descrição

function **json_encode**(mixed `$value`, int `$flags` = 0, int `$depth` = 512): string|false

Retorna uma string contendo a representação JSON de `value` fornecido. Se o parâmetro for um array ou object, ele será serializado recursivamente.

Se um valor a ser serializado for um objeto, então, por padrão, apenas propriedades visíveis publicamente serão incluídas. Alternativamente, uma classe pode implementar JsonSerializable para controlar como seus valores são serializados para JSON.

A codificação é afetada pelas `flags` fornecidas e além disso a codificação de valores com ponto flutuante depende do valor de serialize_precision.

Parâmetros

`value`

O `value` a ser codificado. Pode ser qualquer tipo, exceto um resource.

Toda a string deve ser codificada como UTF-8.

> **Nota** :
>
> O PHP implementa um superconjunto de JSON conforme especificado na » RFC 7159 original.

`flags`

Máscara de bits consistindo de `JSON_FORCE_OBJECT`, `JSON_HEX_QUOT`, `JSON_HEX_TAG`, `JSON_HEX_AMP`, `JSON_HEX_APOS`, `JSON_INVALID_UTF8_IGNORE`, `JSON_INVALID_UTF8_SUBSTITUTE`, `JSON_NUMERIC_CHECK`, `JSON_PARTIAL_OUTPUT_ON_ERROR`, `JSON_PRESERVE_ZERO_FRACTION`, `JSON_PRETTY_PRINT`, `JSON_UNESCAPED_LINE_TERMINATORS`, `JSON_UNESCAPED_SLASHES`, `JSON_UNESCAPED_UNICODE`, `JSON_THROW_ON_ERROR`. O comportamento destas constantes é descrito na página de constantes JSON.

`depth`

Define a profundidade máxima. Deve ser maior do que zero.

Valor Retornado

Retorna um JSON codificado como string em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.3.0	Adicionado `JSON_THROW_ON_ERROR` em `flags`.
7.2.0	Adicionado `JSON_INVALID_UTF8_IGNORE` e `JSON_INVALID_UTF8_SUBSTITUTE` em `flags`.
7.1.0	Adicionado `JSON_UNESCAPED_LINE_TERMINATORS` em `flags`.
7.1.0	É usado serialize_precision em vez de precision quando codificado valores float.

Examples

Exemplo #1 Um exemplo da json_encode()

```php
<?php
$arr = array('a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5);

echo json_encode($arr);
?>
```

O exemplo acima produzirá:

```
{"a":1,"b":2,"c":3,"d":4,"e":5}
```

Exemplo #2 Um exemplo de json_encode() mostrando algumas opções em uso

```php
<?php
$a = array('<foo>',"'bar'",'"baz"','&blong&', "\xc3\xa9");

echo "Normal: ",  json_encode($a), "\n";
echo "Tags: ",    json_encode($a, JSON_HEX_TAG), "\n";
echo "Apos: ",    json_encode($a, JSON_HEX_APOS), "\n";
echo "Quot: ",    json_encode($a, JSON_HEX_QUOT), "\n";
echo "Amp: ",     json_encode($a, JSON_HEX_AMP), "\n";
echo "Unicode: ", json_encode($a, JSON_UNESCAPED_UNICODE), "\n";
echo "All: ",     json_encode($a, JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_QUOT | JSON_HEX_AMP | JSON_UNESCAPED_UNICODE),
"\n\n";

$b = array();

echo "Saída de um array vazio como array: ", json_encode($b), "\n";
```

```php
echo "Saída de um array vazio como objeto: ", json_encode($b, JSON_FORCE_OBJECT), "\n\n";

$c = array(array(1,2,3));

echo "Saída de um array não-associativo como array: ", json_encode($c), "\n";
echo "Saída de um array não-associativo como objeto: ", json_encode($c, JSON_FORCE_OBJECT), "\n\n";

$d = array('foo' => 'bar', 'baz' => 'long');

echo "Array associativo sempre tem saída como objeto: ", json_encode($d), "\n";
echo "Array associativo sempre tem saída como objeto: ", json_encode($d, JSON_FORCE_OBJECT), "\n\n";
?>
```

O exemplo acima produzirá:

```
Normal: ["<foo>","'bar'","\"baz\"","&blong&","\u00e9"]
Tags: ["\u003Cfoo\u003E","'bar'","\"baz\"","&blong&","\u00e9"]
Apos: ["<foo>","\u0027bar\u0027","\"baz\"","&blong&","\u00e9"]
Quot: ["<foo>","'bar'","\u0022baz\u0022","&blong&","\u00e9"]
Amp: ["<foo>","'bar'","\"baz\"","\u0026blong\u0026","\u00e9"]
Unicode: ["<foo>","'bar'","\"baz\"","&blong&","é"]
All: ["\u003Cfoo\u003E","\u0027bar\u0027","\u0022baz\u0022","\u0026blong\u0026","é"]

Saída de um array vazio como array: []
Saída de um array vazio como objeto: {}

Saída de um array não-associativo como array: [[1,2,3]]
Saída de um array não-associativo como objeto: {"0":{"0":1,"1":2,"2":3}}

Array associativo sempre tem saída como objeto: {"foo":"bar","baz":"long"}
Array associativo sempre tem saída como objeto: {"foo":"bar","baz":"long"}
```

Exemplo #3 Exemplo com a opção JSON_NUMERIC_CHECK

```php
<?php
echo "Strings representando números automaticamente transformados em números".PHP_EOL;
$numbers = array('+123123', '-123123', '1.2e3', '0.00001');
var_dump(
 $numbers,
 json_encode($numbers, JSON_NUMERIC_CHECK)
);
echo "Strings contendo números formatados incorretamente".PHP_EOL;
$strings = array('+a33123456789', 'a123');
var_dump(
 $strings,
 json_encode($strings, JSON_NUMERIC_CHECK)
);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Strings representando números automaticamente transformados em números
array(4) {
  [0]=>
  string(7) "+123123"
  [1]=>
  string(7) "-123123"
  [2]=>
  string(5) "1.2e3"
  [3]=>
  string(7) "0.00001"
}
string(28) "[123123,-123123,1200,1.0e-5]"
```

```
Strings contendo números formatados incorretamente
array(2) {
  [0]=>
  string(13) "+a33123456789"
  [1]=>
  string(4) "a123"
}
string(24) "["+a33123456789","a123"]"
```

Exemplo #4 Exemplo de array sequencial versus não-sequencial

```php
<?php
echo "Array sequencial".PHP_EOL;
$sequential = array("foo", "bar", "baz", "blong");
var_dump(
 $sequential,
 json_encode($sequential)
);

echo PHP_EOL."Array não-sequencial".PHP_EOL;
$nonsequential = array(1=>"foo", 2=>"bar", 3=>"baz", 4=>"blong");
var_dump(
 $nonsequential,
 json_encode($nonsequential)
);

echo PHP_EOL."Array sequencial com uma chave não definida".PHP_EOL;
unset($sequential[1]);
var_dump(
 $sequential,
 json_encode($sequential)
);
?>
```

O exemplo acima produzirá:

```
Array sequencial
array(4) {
  [0]=>
  string(3) "foo"
  [1]=>
  string(3) "bar"
  [2]=>
  string(3) "baz"
  [3]=>
  string(5) "blong"
}
string(27) "["foo","bar","baz","blong"]"

Array não-sequencial
array(4) {
  [1]=>
  string(3) "foo"
  [2]=>
  string(3) "bar"
  [3]=>
  string(3) "baz"
  [4]=>
  string(5) "blong"
}
string(43) "{"1":"foo","2":"bar","3":"baz","4":"blong"}"

Array sequencial com uma chave não definida
array(3) {
  [0]=>
```

```
  string(3) "foo"
  [2]=>
  string(3) "baz"
  [3]=>
  string(5) "blong"
}
string(33) "{"0":"foo","2":"baz","3":"blong"}"
```

Exemplo #5 Exemplo com a opção `JSON_PRESERVE_ZERO_FRACTION`

```php
<?php
var_dump(json_encode(12.0, JSON_PRESERVE_ZERO_FRACTION));
var_dump(json_encode(12.0));
?>
```

O exemplo acima produzirá:

```
string(4) "12.0"
string(2) "12"
```

Notas

> **Nota** :
>
> Em uma eventual falha ao codificar, json_last_error() pode ser usado para determinar a natureza exata do erro.

> **Nota** :
>
> Quando codificando um array, se as chaves não são uma sequência numérica contínua começando por 0, todas as chaves são codificadas como strings, e especificadas explicitamente para cada par chave-valor.

> **Nota** :
>
> Assim como o codificador JSON referenciado, **json_encode()** irá gerar JSON que é um valor simples (isto é, nem um objeto e nem um array) se informado um string, int, float ou bool como uma `value`. Enquanto a maioria dos decodificadores aceitará esses valores como JSON válido, alguns podem não aceitar, já que a especificação é ambígua neste ponto.
>
> Para resumir, sempre teste se o seu decodificador JSON pode dar conta da saída que você gerar a partir de **json_encode()**.

Veja também

- JsonSerializable
- json_decode() - Decodifica uma string JSON
- json_last_error() - Retorna o último erro ocorrido
- json_last_error_msg() - Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode()
- serialize() - Gera uma representação armazenável de um valor

json_last_error

json_last_error

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

json_last_error — Retorna o último erro ocorrido

Descrição

function **json_last_error**(): int

Retorna o erro ocorrido (se houver) durante a última validação/codificação/decodificação JSON que não tenha especificado `JSON_THROW_ON_ERROR`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um inteiro, o valor pode ser uma das seguintes constantes:

Códigos de erro JSON

Constante	Significado	Disponibilidade
`JSON_ERROR_NONE`	Não ocorreu nenhum erro	
`JSON_ERROR_DEPTH`	A profundidade máxima da pilha foi excedida	
`JSON_ERROR_STATE_MISMATCH`	JSON inválido ou mal formado	
`JSON_ERROR_CTRL_CHAR`	Erro de caractere de controle, possivelmente codificado incorretamente	
`JSON_ERROR_SYNTAX`	Erro de sintaxe	
`JSON_ERROR_UTF8`	Caracteres UTF-8 malformados, possivelmente codificados incorretamente	
`JSON_ERROR_RECURSION`	Uma ou mais referências recursivas no valor a ser codificado	
`JSON_ERROR_INF_OR_NAN`	Um ou mais valores `NAN` ou `INF` no valor a ser codificado	
`JSON_ERROR_UNSUPPORTED_TYPE`	Fornecido um valor de um tipo que não pode ser codificado	
`JSON_ERROR_INVALID_PROPERTY_NAME`	Fornecido um nome de propriedade que não pode ser codificado	
`JSON_ERROR_UTF16`	Caracteres UTF-16 malformados, possivelmente codificados incorretamente	
`JSON_ERROR_NON_BACKED_ENUM`	Valor contém uma enumeração não apoiada que não pode ser serializada. Disponível a partir do PHP 8.1.0.	

Examples

Exemplo #1 Exemplo de **json_last_error()**

```php
<?php
// Uma string json válida
$json[] = '{"Organização": "Time de Documentação PHP"}';

// Uma string json inválida que causará uma sintaxe
// erro, neste caso usamos ' em vez de " para cotação
$json[] = "{'Organização': 'Time de Documentação PHP'}";


foreach ($json as $string) {
    echo 'Decodificando: ' . $string;
    json_decode($string);

    switch (json_last_error()) {
        case JSON_ERROR_NONE:
            echo ' - Sem Erros';
        break;
        case JSON_ERROR_DEPTH:
            echo ' - Profundidade máxima da pilha foi excedida';
        break;
        case JSON_ERROR_STATE_MISMATCH:
            echo ' - subfluxo ou os modos não combinam';
        break;
        case JSON_ERROR_CTRL_CHAR:
            echo ' - Caractere de controle inesperado encontrado';
        break;
        case JSON_ERROR_SYNTAX:
            echo ' - Erro de sintaxe, JSON malformado';
        break;
```

```
        case JSON_ERROR_UTF8:
            echo ' - Caracteres UTF-8 malformados, possivelmente codificados incorretamente';
        break;
        default:
            echo ' - Erro desconhecido';
        break;
    }

    echo PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
Decodificando: {"Organização": "Time de Documentação PHP"} - Sem erros
Decodificando: {'Organização': 'Time de Documentação PHP'} - Erro de sintaxe, JSON mal formatado
```

Exemplo #2 json_last_error() com json_encode()

```
<?php
// Uma sequência UTF8 inválida
$text = "\xB1\x31";

$json  = json_encode($text);
$error = json_last_error();

var_dump($json, $error === JSON_ERROR_UTF8);
?>
```

O exemplo acima produzirá:

```
string(4) "null"
bool(true)
```

Exemplo #3 json_last_error() e JSON_THROW_ON_ERROR

```
<?php
// Uma sequência UTF8 inválida que causa JSON_ERROR_UTF8
json_encode("\xB1\x31");

// O seguinte não causa um erro JSON
json_encode('okay', JSON_THROW_ON_ERROR);

// O estado de erro global não foi alterado pelo antigo json_encode()
var_dump(json_last_error() === JSON_ERROR_UTF8);
?>
```

O exemplo acima produzirá:

```
bool(true)
```

Veja também

- json_last_error_msg() - Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode()
- json_decode() - Decodifica uma string JSON
- json_encode() - Retorna a representação JSON de um valor

json_last_error_msg

json_last_error_msg

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

json_last_error_msg — Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode()

Descrição

function **json_last_error_msg**(): string

Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode() que não tenha especificado `JSON_THROW_ON_ERROR`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a mensagem de erro em caso de sucesso, ou `"No error"` se não ocorreu erro.

Veja também

- json_last_error() - Retorna o último erro ocorrido

json_validate

json_validate

(PHP 8 >= 8.3.0)

json_validate — Verifica se uma string contém JSON válido

Descrição

function **json_validate**(string `$json`, int `$depth` = 512, int `$flags` = 0): bool

Retorna se a string informada é um JSON válido sintaticamente. Se **json_validate()** retornar `true`, json_decode() irá decodificar com sucesso a string informada ao usar a mesma profundidade de `depth` e as opções em `flags`.

Se **json_validate()** retornar `false`, a causa pode ser obtida usando json_last_error() e json_last_error_msg().

json_validate() usa menos memória que json_decode() se o JSON decodificado não for usado, porque ela não precisa construir o array ou a estrutura de objeto que contém a decodificação.

> **Cuidado**
>
> Chamar **json_validate()** imediatamente antes de json_decode() irá analisar a string duas vezes desnecessariamente, já que json_decode() implicitamente realiza a validação durante a decodificação.
>
> Portanto, **json_validate()** deve somente ser utilizada se o JSON decodificado não for usado imediatamente e se for necessário saber se a string contém JSON válido.

Parâmetros

`json`

A string a ser validada.

Esta função funciona somente com strings codificadas com UTF-8.

> **Nota** :

> O PHP implementa um superconjunto de JSON conforme especificado na » RFC 7159 original.

`depth`

A máxima profundidade de aninhamento da estrutura sendo decodificada. O valor precisa ser maior que `0`, e menor ou igual a `2147483647`.

`flags`

Atualmente apenas a constante **`JSON_INVALID_UTF8_IGNORE`** é aceita.

Valor Retornado

Retorna **`true`** se a string fornecida for um JSON válido sintaticamente, caso contrário retorna **`false`**.

Erros/Exceções

Se `depth` estiver fora do intervalo permitido, uma exceção ValueError é lançada.

Se `flags` não for uma opção válida, uma exceção ValueError é lançada.

Examples

Exemplo #1 Exemplos de json_validate()

```php
<?php
var_dump(json_validate('{ "test": { "foo": "bar" } }'));
var_dump(json_validate('{ "": "": "" } }'));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
```

Veja também

- json_decode() - Decodifica uma string JSON
- json_last_error() - Retorna o último erro ocorrido
- json_last_error_msg() - Retorna a string de erro da última chamada a json_validate(), json_encode() ou json_decode()

Manipulação de Sessão

Introdução

O suporte a sessões no PHP consiste em uma maneira de preservar certos dados atráves dos acessos subsequentes.

Um visitante acessando o seu web site ganha um identificador único, o assim chamado id de sessão. Ele é salvo em um cookie do lado do usuário ou propagado via URL.

O suporte à sessão permite armazenar dados entre as requisições no array super global *$_SESSION*. Quando um visitante acessar seu site, o PHP vai conferir automaticamente (se session.auto_start estiver definido como 1) ou quando você pedir (explicitamente atráves de session_start()) se um id de sessão específico foi enviado com a requisição. Se este for o caso, o ambiente anteriormente salvo é recriado.

> **Cuidado**
>
> Se você ativar session.auto_start, então a única maneira de colocar objetos em suas sessões é carregando a definição da classe usando auto_prepend_file, onde você pode carregar a definição da classe, caso contrário você terá que usar serialize() no objeto, e unserialize() nele depois.

$_SESSION (e todas as variáveis registradas) são serializadas internamente pelo PHP usando o manipulador de serialização especificado pela configuração INI session.serialize_handler depois que a requisição terminar. Variáveis registradas que estejam indefinidas são marcadas como não definidas. Nos acessos subsequentes, elas não são definidas pelo módulo da sessão, a menos que o usuário as definam posteriormente.

> **Aviso**
>
> Porque as variáveis de sessão são serializadas, variáveis resource não podem ser armazenadas em sessão.
>
> Manipuladores de serialização (`php` e `php_binary`) herdam as limitações de register_globals. Portanto, índices numéricos ou strings contendo caracteres especiais (`|` e `!`) não podem ser usados. Se usados, resultará em erros na finalização do script. `php_serialize` não possui tais limitações.

> **Nota** :
>
> Por favor, note que ao trabalhar com sessões, um registro de uma sessão não é criado até que uma variável tenha sindo registrada pela adição de uma nova chave ao array super global *$_SESSION*. Isto continua valendo mesmo se uma sessão tenha sido iniciada usando a função session_start().

Instalação/Configuração

Índice

Dependências

Nenhuma biblioteca externa é necessária para compilar esta extensão.

> **Nota** :
>
> Opcionalmente você pode usar alocação de memória compartilhada (mm), desenvolvida por Ralf S. Engelschall, para armazenar a sessão. Você terá que baixar » mm e instalá-lo. Esta opção não está disponível para plataformas Windows. Note que o módulo de armazenamento de sessões para o mm não garante que acessos concorrentes à mesma sessão sejam travados apropriadamente. Pode ser mais apropriado usar um sistema de arquivos baseado em memória compartilhada (como tmpfs em Solaris/Linux, ou */dev/md* em BSD) para armazenar sessões em arquivos, por que eles são travados apropriadamente. Os dados das sessões são salvos na memória então se o servidor for reiniciado os dados são excluídos.

Instalação

Esta extensão está habilitada por padrão. Ela pode ser desabilitada usando a seguinte opção em tempo de compilação: **--disable-session**

Para usar alocação de memória compartilhada (mm) para armazenamento de sessão, configure o PHP com **--with-mm[=DIR]**.

A versão do PHP para Windows tem suporte nativo para esta extensão. Nenhuma extensão adicional precisa ser carregada para usar essas funções.

> **Nota** :
>
> Por padrão, todos os dados relacionados com uma sessão em particular serão armazenados em um arquivo no diretório especificado pela configuração INI session.save_path. Um arquivo para cada sessão (independente se alguma informação está associada com esta sessão) será criado. Isto é devido ao fato de que uma sessão é aberta (um arquivo é criado) mas nenhum dado é escrito neste arquivo. Note que este comportamento é um efeito colateral das limitações de se trabalhar com o sistema de arquivos e é possível que um manipulador de sessão personalizado (como um que utilize banco de dados) não mantenha registros de sessões que não armazenem dados.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração de sessão

Nome	Padrão	Modificável	Registro de Alterações
session.save_path	""	INI_ALL	
session.name	"PHPSESSID"	INI_ALL	
session.save_handler	"files"	INI_ALL	
session.auto_start	"0"	INI_PERDIR	
session.gc_probability	"1"	INI_ALL	
session.gc_divisor	"100"	INI_ALL	
session.gc_maxlifetime	"1440"	INI_ALL	
session.serialize_handler	"php"	INI_ALL	
session.cookie_lifetime	"0"	INI_ALL	
session.cookie_path	"/"	INI_ALL	
session.cookie_domain	""	INI_ALL	
session.cookie_secure	"0"	INI_ALL	Antes do PHP 7.2.0, o padrão era "".
session.cookie_httponly	"0"	INI_ALL	Antes do PHP 7.2.0, o padrão era "".
session.cookie_samesite	""	INI_ALL	Disponível a partir do PHP 7.3.0.
session.use_strict_mode	"0"	INI_ALL	
session.use_cookies	"1"	INI_ALL	
session.use_only_cookies	"1"	INI_ALL	Desabilitar esta configuração foi descontinuado a partir do PHP 8.4.0.
session.referer_check	""	INI_ALL	Definir um valor não vazio foi descontinuado a partir do PHP 8.4.0.
session.cache_limiter	"nocache"	INI_ALL	
session.cache_expire	"180"	INI_ALL	
session.use_trans_sid	"0"	INI_ALL	Habilitar esta configuração foi descontinuado a partir do PHP 8.4.0.
session.trans_sid_tags	"a=href,area=href,frame=src,form="	INI_ALL	Disponível a partir do PHP 7.1.0. Alterar esta configuração foi descontinuado a partir do PHP 8.4.0.
session.trans_sid_hosts	$_SERVER['HTTP_HOST']	INI_ALL	Disponível a partir do PHP 7.1.0. Definir um valor não vazio foi descontinuado a partir do PHP 8.4.0.
session.sid_length	"32"	INI_ALL	Disponível desde o PHP 7.1.0. Descontinuada a partir do PHP 8.4.0.

Nome	Padrão	Modificável	Registro de Alterações
session.sid_bits_per_character	"5"	INI_ALL	Disponível desde o PHP 7.1.0. Descontinuada a partir do PHP 8.4.0.
session.upload_progress.enabled	"1"	INI_PERDIR	
session.upload_progress.cleanup	"1"	INI_PERDIR	
session.upload_progress.prefix	"upload_progress_"	INI_PERDIR	
session.upload_progress.name	"PHP_SESSION_UPLOAD_PROGRESS"	INI_PERDIR	
session.upload_progress.freq	"1%"	INI_PERDIR	
session.upload_progress.min_freq	"1"	INI_PERDIR	
session.lazy_write	"1"	INI_ALL	
session.hash_function	"0"	INI_ALL	Removido no PHP 7.1.0.
session.hash_bits_per_character	"4"	INI_ALL	Removido no PHP 7.1.0.
session.entropy_file	""	INI_ALL	Removido no PHP 7.1.0.
session.entropy_length	"0"	INI_ALL	Removido no PHP 7.1.0.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

O sistema de gerenciamento de sessões suporta várias opções de configurações que podem ser colocados no arquivo *php.ini*. A seguir um breve resumo.

`session.save_handler` string

> `session.save_handler` define o nome do manipulador que será utilizado para armazenar e recuperar dados associados à sessão. Padrão para `files`. Note que extensões podem registrar seus próprios `save_handlers`; manipuladores registrados podem ser obtidos ao se vefificar o phpinfo(). Veja também session_set_save_handler().

`session.save_path` string

> `session.save_path` define o argumento que é passado para o manipulador de gravação. Se você escolher o manipulador padrão (de arquivos), este é o caminho onde os arquivos serão criados. Veja também session_save_path().
>
> Há um argumento opcional `N` para esta diretiva que determina o número de níveis de diretório que seus arquivos de sessão serão espalhados. Por exemplo, definindo para `'5;/tmp'` pode levar a criação de um arquivo de sessão e diretórios como `/tmp/4/b/1/e/3/sess_4b1e384ad74619bd212e236e52a5a174If` . Para fazer uso do `N` você deve criar todos estes diretórios antes do uso. Um pequeno script shell existe em *ext/session* para fazer isto, chamado *mod_files.sh*, com uma versão para o Windows chamada *mod_files.bat*. Também note que se `N` é usado e maior do que 0 então a limpeza de sessões não será executada, veja uma cópia do *php.ini* para mais informações. Também, se você usar `N`, certifique-se de cercar `session.save_path` com "aspas" porque o separador (`;`) é usado também para comentátios no *php.ini*.
>
> O módulo de armazenamento em arquivo cria arquivos com permissões 600 por padrão. Esse padrão pode ser alterado com o argumento opcional `MODE`: `N;MODE;/path` onde `MODE` é o valor octal representando as permissões. Configurar `MODE` não afeta o umask do processo.
>
> > **Aviso**
> >
> > Se você deixar isto definido num diretório de leitura público, como */tmp* (o padrão), outros usuários no servidor poderão raptar sessões pegando a lista de arquivos nesse diretório.
>
> > **Cuidado**
> >
> > Ao utilizar o argumento opcional de nível de diretório `N`, como descrito acima, note que utilizar um valor maior que 1 ou 2 é inapropriado na maioria dos sites por conta da geração de um grande número de diretórios exigidos: por exemplo, um valor de 3 implica que existirão `(2 ** session.sid_bits_per_character) ** 3` diretórios em disco, o que pode resultar numa quantidade enorme de espaço e inodes desperdiçados.
> >
> > Somente use `N` maiores que 2 se você estiver absolutamente certo que o seu site é grande o suficiente para exigí-lo.

`session.name` string

> `session.name` especifica o nome da sessão que é usada como o nome do cookie. Deve conter apenas caracteres alfanuméricos. O padrão é `PHPSESSID`. Veja também session_name().

`session.auto_start` bool

`session.auto_start` especifica se o módulo da sessão deve iniciar uma sessão automaticamente no início da requisição. Padrão `0` (desabilitado).

`session.serialize_handler` string

`session.serialize_handler` define o nome do manipulador utilizado para serializar/desserializar dados. São suportados o formato serial do PHP (`php_serialize`), o formato interno do PHP (`php` e `php_binary`) e WDDX (`wddx`). O WDDX somente está disponível se o PHP foi compilado com suporte WDDX. `php_serialize` utiliza as funções serialize/unserialize internamente e não tem as limitações que `php` e `php_binary` tem. Serializadores antigos não conseguem gravar índices numéricos ou índices string que contenham caracteres especiais (`|` e `!`) no array $_SESSION. Utilize `php_serialize` para evitar erros de índices numéricos ou índices com caracteres especiais no final do script. O padrão é `php`.

`session.gc_probability` int

`session.gc_probability` em conjunto com `session.gc_divisor` é usado para gerenciar a probabilidade que o gc (coletor de lixo) seja iniciado. O padrão é `1`. Precisa ser maior ou igual a `0`. Veja session.gc_divisor para detalhes.

`session.gc_divisor` int

`session.gc_divisor` em conjunto com `session.gc_probability` define a probabilidade que o processo do gc (coletor de lixo) seja iniciado na inicialização de cada sessão. A probabilidade é calculada usando gc_probability/gc_divisor, ex. 1/100 indica que existe 1% de chance que o processo GC inicie em cada requisição. O padrão para `session.gc_divisor` é `100`. Precisa ser maior que `0`.

`session.gc_maxlifetime` int

`session.gc_maxlifetime` especifica o número de segundos, que, depois de decorridos, os dados serão considerados como lixo ('garbage') e eventualmente apagados. Isso pode ocorrer no início da sessão (dependendo de session.gc_probability e session.gc_divisor). O padrão é `1440` (24 minutos).

> **Nota** :
>
> Se scripts diferentes tem valores diferentes para `session.gc_maxlifetime` mas compartilham o mesmo lugar para guardar os dados da sessão então o script com o menor valor estará limpando os dados. Neste caso, use esta diretiva em conjunto com session.save_path.

`session.referer_check` string

`session.referer_check` contém a substring que você quer checar contra cada HTTP Referer. Se o Referer for enviado pelo cliente e a substring não foi encontrada, a id de sessão embutida será marcada como inválida. O padrão é uma string vazia.

> **Aviso**
>
> Definir `session.referer_check` para um valor não vazio foi descontinuado a partir do PHP 8.4.0.

`session.entropy_file` string

`session.entropy_file` informa o caminho para um recurso externo (arquivo) que será usado como uma fonte de entropia no processo de criação da id de sessão. Exemplos são `/dev/random` ou `/dev/urandom` que estão disponíveis em muitos sistemas UNIX.

Este recurso é suportado no Windows. Configurando `session.entropy_length` para um valor diferente de zero fará o PHP utilizar o Windows Random API como fonte de entropia.

> **Nota** :
>
> Removido no PHP 7.1.0.
>
> `session.entropy_file` tem como padrão `/dev/urandom` ou `/dev/arandom`, se disponível.

`session.entropy_length` int

`session.entropy_length` especifica o número de bytes que serão lidos do arquivo especificado acima. Padrão em `32`.

Removido no PHP 7.1.0.

`session.use_strict_mode` bool

`session.use_strict_mode` especifica se o módulo utilizará o modo id de sessão rigoroso (strict). Se esse modo estiver ativo, o módulo não aceitará um id de sessão não inicializado. Se um id de sessão não inicializado for enviado pelo browser, um novo id de sessão é devolvido para o browser. Aplicações são protegidas da fixação de sessão adotando o modo de sessão rigoroso (strict). Padrão é `0` (desativado).

> **Nota** :
>
> Ativar `session.use_strict_mode` é mandatório para segurança geral das sessões. A todos os sites é recomendado mantê-lo ativado. Veja o código em session_create_id() para mais detalhes.

> **Aviso**
>
> Se um manipulador de sessão personalizado registrado via session_set_save_handler() não implementa SessionUpdateTimestampHandlerInterface::validateId(), nem fornece o callback `validate_sid`, respectivamente, o modo rigoroso(strict) do ID da sessão é efetivamente desativado, independentemente do valor desta diretiva. É importante observar que SessionHandler *não* implementa **SessionHandler::validateId()**.

`session.use_cookies` bool

`session.use_cookies` especifica se o módulo utilizará cookies para guardar a id da sessão no lado do cliente. O padrão é `1` (habilitado).

`session.use_only_cookies` bool

`session.use_only_cookies` especifica se o módulo usará **somente** cookies para guardar a id no lado do cliente. Habilitar esta configuração previne ataques envolvendo passagem de IDs de sessão nas URLs. Padrão `1` (ativado).

> **Aviso**
>
> Desabilitar `session.use_only_cookies` foi descontinuado a partir do PHP 8.4.0.

`session.cookie_lifetime` int

`session.cookie_lifetime` especifica o tempo de vida do cookie em segundos que é enviado para o browser. O valor 0 significa "até o browser ser fechado". O padrão é `0`. Veja também session_get_cookie_params() e session_set_cookie_params().

> **Nota** :
>
> O timestamp de expiração é informado em relação à hora do servidor, que não necessariamente coincide com o horário do navegador do cliente.

`session.cookie_path` string

`session.cookie_path` especifica o caminho para definir em session_cookie. O padrão é `/`. Veja também session_get_cookie_params() e session_set_cookie_params().

`session.cookie_domain` string

`session.cookie_domain` especifica o domínio para definir no cookie de sessão. O padrão é nenhum significando o nome do servidor que gerou o cookie de acordo com a especificação dos cookies. Veja também session_get_cookie_params() e session_set_cookie_params().

`session.cookie_secure` bool

`session.cookie_secure` especifica se os cookies devem ser enviados apenas em conexões seguras. Com esta opção definida para `on`, as sessões só funcionam com conexões HTTPS. Se estiver em `off`, as sessões funcionam tanto com HTTP quanto com HTTPS. O padrão é `off`. Veja também session_get_cookie_params() e session_set_cookie_params().

`session.cookie_httponly` bool

Marca o cookie para ser acessível apenas atráves do protocolo HTTP. Isto significa que o cookie não será acessível por linguagens de script, como o JavaScript. Esta configuração pode efetivamente reduzir o roubo de identidade atráves de ataques XSS (apesar de

não ser suportado por todos os browsers).

`session.cookie_samesite` string

Permite que os servidores declarem que um cookie não deve ser enviado junto com solicitações entre sites. Essa afirmação permite que os agentes do usuário reduzam o risco de vazamento de informações entre origens e fornece alguma proteção contra ataques de falsificação de solicitações entre sites (CSRF). Observe que isso não é suportado por todos os navegadores. Um valor vazio significa que nenhum atributo de cookie SameSite será definido. `Lax` e `Strict` significam que o cookie não será enviado entre domínios para solicitações POST; `Lax` enviará o cookie para solicitações GET entre domínios, enquanto `Strict` não enviará.

`session.cache_limiter` string

`session.cache_limiter` especifica o método de controle de cache para usar em páginas de sessão. Pode ser um dos seguintes valores: `nocache`, `private`, `private_no_expire` ou `public`. Padrão `nocache`. Veja também a documentação de session_cache_limiter() para ver o que os valores significam.

`session.cache_expire` int

`session.cache_expire` especifica o time-to-live (tempo de vida) para páginas de sessão em cache, em minutos, não tendo efeito na limitação de nocache. O padrão é `180`. Veja também session_cache_expire().

`session.use_trans_sid` bool

`session.use_trans_sid` se o suporte a sid transparente está habilitado ou não. O padrão é `0` (desabilitado).

> **Aviso**
>
> Habilitar `session.use_trans_sid` foi descontinuado a partir do PHP 8.4.0.

> **Nota** :
>
> Gerenciamento de sessões baseadas em URLs tem riscos de segurança adicionais comparados ao gerenciamento baseado em cookies. Usuários podem enviar uma URL que contenha uma ID de sessão ativa para seus amigos por e-mail ou usuários podem salvar uma URL que contenha uma ID de sessão em seus bookmarks e acessar seu site sempre com a mesma ID de sessão, por exemplo.
>
> Desde o PHP 7.1.0 uma URL completa, por exemplo https://php.net/, é manipulada pelo recurso trans sid. Versões anteriores do PHP manipulavam apenas URLs relativas. Alvos de rewrite estão definidos em session.trans_sid_hosts.

`session.trans_sid_tags` string

`session.trans_sid_tags` especifica quais tags HTML serão reescritas para incluir IDs de sessão quando o suporte ao SID transparente estiver ativo. Por padrão: `a=href,area=href,frame=src,form=`

`form` é uma tag especial. `<input hidden="session_id" name="session_name">` é adicionado a variável de formulário.

> **Aviso**
>
> Alterar `session.trans_sid_tags` de seu valor padrão foi descontinuado a partir do PHP 8.4.0.

> **Nota** :
>
> Antes do PHP 7.1.0 url_rewriter.tags era utilizada para essa funcionalidade. Desde o PHP 7.1.0, `fieldset` não é mais considerada uma tag especial.

`session.trans_sid_hosts` string

`session.trans_sid_hosts` especifica quais hosts serão reescritos para incluir IDs de sessão quando o suporte ao SID transparente estiver ativo. Padrão para `$_SERVER['HTTP_HOST']` Vários hosts podem ser especificados, separados por ",", mas sem espaços entre os hosts. Por exemplo: `php.net,wiki.php.net,bugs.php.net`

> **Aviso**

> Definir `session.trans_sid_hosts` para um valor não vazio foi descontinuado a partir do PHP 8.4.0.

`session.sid_length` int

`session.sid_length` permite especificar o tamanho do texto do ID de sessão. IDs de sessão podem ter tamanhos entre 22 a 256 letras.

O padrão é 32. Para compatibilidade, você pode especificar 32, 40, etc. IDs de sessão maiores são mais difíceis de adivinhar. Ao menos 32 letras é o recomendados.

> **Dica**
>
> Nota de compatibilidade: Utilize 32 para que `session.hash_function=0` (MD5) e `session.hash_bits_per_character=4`, `session.hash_function=1` (SHA1) e `session.hash_bits_per_character=6`. Utilize 26 para que `session.hash_function=0` (MD5) e `session.hash_bits_per_character=5`. Utilize 22 para que `session.hash_function=0` (MD5) e `session.hash_bits_per_character=6`. Você precisa configurar os valores INI para ao menos 128 bits no ID de sessão. Não se esqueça de configurar o valor apropriado em `session.sid_bits_per_character`, de outra forma você terá um ID de sessão fraco.

> **Nota** :
>
> Esta configuração foi introduzida no PHP 7.1.0.

`session.sid_bits_per_character` int

`session.sid_bits_per_character` permite especificar o número de bits codificados no ID de sessão. Os valores possíveis são '4' (0-9, a-f), '5' (0-9, a-v), e '6' (0-9, a-z, A-Z, "-", ",").

O padrão é 4. Mais bits resultam em IDs de sessão mais fortes. 5 é recomendado para a maioria dos elementos.

> **Nota** :
>
> Esta configuração foi introduzida no PHP 7.1.0.

`session.hash_function` mixed

`session.hash_function` permite especificar o algoritimo de hash usado para gerar os IDs de sessão. '0' indica MD5 (128 bits) e '1' indica SHA-1 (160 bits).

Também é possível especificar qualquer um dos algoritmos disponibilizados pela extensão hash (se habilitada), como `sha512` ou `whirlpool`. A lista completa de algoritmos suportados pode ser obtida com a função hash_algos().

> **Nota** :
>
> Removido no PHP 7.1.0.

`session.hash_bits_per_character` int

`session.hash_bits_per_character` permite a você definir quantos bits são guardados em cada caractere ao converter os dados binários de hash para algo legível. Os valores possíveis são '4' (0-9, a-f), '5' (0-9, a-v) e '6' (0-9, a-z, A-Z, "-", ",").

> **Nota** :
>
> Removido no PHP 7.1.0.

`session.upload_progress.enabled` bool

Permite o rastreamento do progresso de upload, populando a variável *$_SESSION*. O padrão é 1, habilitado.

`session.upload_progress.cleanup` bool

Limpa a informação de progresso assim que todos os dados POST forem lidos (upload completado). Padrão 1, habilitado.

> **Nota** :
>
> É altamente recomendável manter esse recurso ligado.

`session.upload_progress.prefix` string

Um prefixo utilizado na chave da variável *$_SESSION*. Essa chave será concatenada com o valor de `$_POST[ini_get("session.upload_progress.name")]` para prover um índice único.

Padrão "upload_progress_".

`session.upload_progress.name` string

O nome da chave utilizada em *$_SESSION* para armazenar a informação de progresso. Veja também session.upload_progress.prefix.

Se `$_POST[ini_get("session.upload_progress.name")]` não for passado ou estiver indisponível, o progresso de upload não será registrado.

Padrão "PHP_SESSION_UPLOAD_PROGRESS".

`session.upload_progress.freq` mixed

Define a frequência com que a informação de progresso de upload será atualizada. Isso pode ser definido em bytes ("atualize a informação a cada 100 bytes") ou em percentagens ("atualize a informação de progresso a cada 1% do total do arquivo").

Padrão "1%".

`session.upload_progress.min_freq` int

O intervalo mínimo entre as atualizações, em segundos. Padrão "1" (um segundo).

`session.lazy_write` bool

`session.lazy_write`, quando configurado para 1, significa que os dados de sessão somente serão reescrito caso eles mudem. O padrão é 1, habilitado.

O progresso de upload não será registrado, a não ser que session.upload_progress.enabled esteja habilitado e a variável $_POST[ini_get("session.upload_progress.name")] esteja definida. Veja Progresso de Upload em sessões para mais detalhes dessa funcionalidade.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

`SID` (string)

Constante contendo nome da sessão e id da sessão na forma de `"name=ID"` ou string vazia se o ID da sessão foi definido em um apropriado cookie de sessão. Este é o mesmo id que é retornado por session_id().

> **Aviso**
>
> Esta constante foi descontinuada a partir do PHP 8.4.0.

`PHP_SESSION_DISABLED` (int)

Retorna o valor de session_status() se as sessões estiverem desabilitadas.

`PHP_SESSION_NONE` (int)

Retorna o valor de session_status() se as sessões estiverem habilitadas, mas nenhuma sessão existir.

`PHP_SESSION_ACTIVE` (int)

Retorna o valor de session_status() se as sessões estiverem habilitadas, e uma sessão existir.

Examples

Índice

Uso básico

Sessões são uma forma simples de armazenar dados para usuários individuais usando um ID de sessão único. Sessões podem ser usadas para persistir informações entre requisições de páginas. IDs de sessão normalmente são enviados ao navegador através de cookies de sessão e o ID é usado para recuperar dados da sessão existente. A ausência de um ID ou cookie de sessão permite que o PHP saiba que deve criar uma nova sessão e gerar um novo ID de sessão.

Sessões seguem um fluxo simples. Quando uma sessão é iniciada, o PHP recupera uma sessão existente usando o ID informado (normalmente de um cookie de sessão) ou se nenhum é informado então será criada uma nova sessão. O PHP preencherá a super global *$_SESSION* com todos os dados de sessão depois que a sessão iniciar. Quando o PHP finalizar, automaticamente ele pegará o conteúdo da super global *$_SESSION*, então vai serializá-lo e enviá-lo para armazenamento usando o manipulador de gravação da sessão.

Por padrão, o PHP usa o manipulador de gravação interno `files` que é configurado por session.save_handler. Isto salva os dados da sessão no servidor no local configurado pela diretiva de configuração session.save_path.

Sessões podem ser iniciadas manualmente usando a função session_start(). Se a diretiva session.auto_start estiver configurada como `1`, a sessão será iniciada automaticamente no início da requisição.

Sessões normalmente se encerram automaticamente quando o PHP termina de executar um script, mas podem ser encerradas manualmente usando a função session_write_close().

Exemplo #1 Registrando uma variável com *$_SESSION*.

```php
<?php
session_start();
if (!isset($_SESSION['count'])) {
  $_SESSION['count'] = 0;
} else {
  $_SESSION['count']++;
}
?>
```

Exemplo #2 Desregistrando uma variável com *$_SESSION*.

```php
<?php
session_start();
unset($_SESSION['count']);
?>
```

> **Cuidado**
>
> NÃO desregistre toda a *$_SESSION* com `unset($_SESSION)` já que isso desativará o registro de variáveis de sessões da super global *$_SESSION*.

> **Aviso**
>
> Referências não podem ser usadas nas variáveis de sessão já que não existe uma maneira de restaurar uma referência para outra variável.

> **Nota** :
>
> Sessões baseadas em arquivos (padrão no PHP) adicionam travas no arquivo de sessão assim que a sessão é iniciada via session_start() ou simplesmente via session.auto_start. Assim que adicionada a trava, nenhum outro script pode acessar o mesmo arquivo de sessão até que ela seja finalizada pelo encerramento do primeiro script ou pela chamada de session_write_close().
>
> Isto é possivelmente um problema para web sites que utilizam fortemente o AJAX e que tem múltiplas solicitações concorrentes. A maneira mais fácil de lidar com isto é chamar session_write_close() assim que as alterações necessárias na sessão forem feitas, de preferência no início do script. Como alternativa, um manipulador de sessão diferente que suporte concorrência poderia ser usado.

Passando o ID de sessão

Existem dois métodos para a propagação do id de sessão:

- Cookies
- URL parameter

O módulo de sessão suporta ambos os métodos. Cookies são melhores, mas como nem sempre eles estão disponíveis, também é oferecido um método alternativo. O segundo método embute o id de sessão diretamente nas URLs.

O PHP é capaz de transformar links de forma transparente. Se a opção de tempo de execução `session.use_trans_sid` estiver habilitada, URIs relativas serão alteradas automaticamente para conter o ID da sessão.

> **Nota** :
>
> O arg_separator.output, diretiva *php.ini*, permite configurar o separador de argumentos. Para completa conformidade com XHTML, especifique & aqui.

Alternativamente, pode ser usada a constante **`SID`**, a qual é definida se uma sessão é iniciada. Se o cliente não enviou um cookie de sessão apropriado, a constante terá o formato `session_name=session_id`. Se não, ela será uma string vazia. Dessa forma, ela pode ser colocada incondicionalmente em URLs.

O exemplo a seguir demonstra como registrar uma variável e como criar um link corretamente para outra página usando o **`SID`**.

Exemplo #1 Contando o número de acessos de um único usuário

```
<?php

session_start();

if (empty($_SESSION['count'])) {
   $_SESSION['count'] = 1;
} else {
   $_SESSION['count']++;
}
?>

<p>
Olá visitante, você acessou esta página <?php echo $_SESSION['count']; ?> vezes.
</p>

<p>
Para continuar, <a href="nextpage.php?<?php echo htmlspecialchars(SID); ?>">clique
aqui</a>.
</p>
```

A função htmlspecialchars() pode ser usada ao mostrar o **`SID`** com o intuito de prevenir ataques relacionados a XSS.

Mostrar o **`SID`**, como mostrado acima, não é necessário se --enable-trans-sid foi usado para compilar o PHP.

> **Nota** :
>
> Para URLs não-relativas, assume-se que elas apontam para sites externos e por isso não é adicionado o **`SID`**, já que seria um risco de segurança vazar o **`SID`** para um servidor diferente.

Manipuladores de Sessão Personalizados

Para implementar as sessões em banco de dados, ou qualquer outro método de armazenamento, é preciso usar session_set_save_handler() para criar um conjunto de funções de armazenamento a nível de usuário. Um manipulador de sessão pode ser criado usando SessionHandlerInterface ou estender os manipuladores internos do PHP herdando de SessionHandler.

Os callbacks especificados em session_set_save_handler() são métodos executados pelo PHP durante o ciclo de vida de uma sessão: `open`, `read`, `write` e `close` e para as tarefas de manutenção: `destroy` para deletar uma sessão e `gc` para recolha periódica de lixo.

Portanto, o PHP sempre requer manipuladores de gravação de sessão. O padrão normalmente é o manipulador de gravação interno de 'arquivos'. Um manipulador personalizado pode ser configurado usando session_set_save_handler(). Manipuladores internos alternativos também são disponibilizados por extensões do PHP, como `sqlite`, `memcache` e `memcached` e podem ser definidos com session.save_handler.

Quando a sessão inicia, o PHP internamente executa o manipulador `open` seguido pelo callback `read` que deve retornar uma string codificada exatamente como foi originalmente passado para armazenamento. Após o `read` retornar a string codificada, o PHP vai decodificá-la e colocar o array resultante na super global *$_SESSION*.

Quando o PHP é encerrado (ou quando session_write_close() for chamada), o PHP internamente vai codificar a super global *$_SESSION* e passá-la junto com o ID de sessão para o callback `write`. Depois que o callback `write` finalizar, o PHP internamente invocará o callback `close`.

Quando uma sessão é destruída, o PHP chamará o manipulador `destroy` com o ID de sessão.

O PHP executará a chamada de retorno `gc` de tempo em tempo para apagar quaisquer informações da sessão de acordo com o tempo de vida máximo definido para ela. Esta rotina deve apagar todas as informações do armazenamento persistente onde a diferença de tempo do último acesso até o momento atual seja maior que `lifetime`.

Progresso de Upload em Sessão

Quando a configuração INI session.upload_progress.enabled estiver habilitada, o PHP será capaz de rastrear o progresso do upload de arquivos individuais que estiverem sendo feito upload. Esta informação não é muito útil para a requisição atual, mas durante o upload do arquivo uma aplicação pode enviar uma requisição POST para um endpoint separado (via XHR por exemplo) para checar o status.

O progresso do upload estará disponível na variável super global *$_SESSION* quando um upload estiver em progresso e quando a requisição POST tiver uma variável com o mesmo nome que a configuração INI session.upload_progress.name estiver configurada. Quando o PHP detectar requisições como essa, ele preencherá um array em *$_SESSION*, onde o índice é um valor resultante da concatenação de session.upload_progress.prefix e session.upload_progress.name (configurações INI). O índice normalmente é obtido lendo essas configurações INI, ou seja

```php
<?php
$key = ini_get("session.upload_progress.prefix") . $_POST[ini_get("session.upload_progress.name")];
var_dump($_SESSION[$key]);
?>
```

Também é possível *cancelar* o upload do arquivo em andamento ao definir o índice `$_SESSION[$key]["cancel_upload"]` para `true`. Quando houver upload de múltiplos arquivos em uma mesma requisição, isso vai cancelar apenas o upload dos arquivos que ainda estiverem em andamento e pendentes, mas não removerá uploads já concluídos com sucesso. Quando um upload é cancelado dessa forma, o índice `error` no array *$_FILES* será alterado para `UPLOAD_ERR_EXTENSION`.

As configurações INI session.upload_progress.freq e session.upload_progress.min_freq controlam a frequência com que a informação do progresso do upload deve ser recalculado. Com um valor razoável para estas duas configurações, a sobrecarga desse recurso é quase inexistente.

Exemplo #1 Exemplo de informação

Exemplo de estrutura do array de progresso de upload.

```php
<form action="upload.php" method="POST" enctype="multipart/form-data">
 <input type="hidden" name="<?php echo ini_get("session.upload_progress.name"); ?>" value="123" />
 <input type="file" name="file1" />
 <input type="file" name="file2" />
 <input type="submit" />
</form>
```

Os dados armazenados na sessão serão parecidos com isto:

```php
<?php
$_SESSION["upload_progress_123"] = array(
 "start_time" => 1234567890,   // A hora da requisição
 "content_length" => 57343257, // tamanho do conteúdo de POST
 "bytes_processed" => 453489,  // Quantidade de bytes recebidos e processados
 "done" => false,              // true quando o manipulador do POST finalizar, com sucesso ou não
 "files" => array(
  0 => array(
   "field_name" => "file1",       // Nome do campo <input/>
   // Os próximos 3 elementos se assemelham aos de $_FILES
   "name" => "foo.avi",
   "tmp_name" => "/tmp/phpxxxxxx",
   "error" => 0,
   "done" => true,                // True quando o manipulador do POST terminar de manipular este arquivo
```

```
    "start_time" => 1234567890,    // Quando este arquivo começou a ser processado
    "bytes_processed" => 57343250, // Quantidade de bytes recebidos e processados para este arquivo
   ),
   // Um outro arquivo, em que o upload ainda não acabou, na mesma requisição
   1 => array(
    "field_name" => "file2",
    "name" => "bar.avi",
    "tmp_name" => NULL,
    "error" => 0,
    "done" => false,
    "start_time" => 1234567899,
    "bytes_processed" => 54554,
   ),
  )
);
```

> **Aviso**
>
> O buffer de requisições do servidor web deve estar desabilitado para que isso funcione corretamente ou o PHP pode enxergar o upload apenas quando ele já estiver completado. É de conhecimento que servidores como Nginx criam buffer para requisições maiores.

> **Cuidado**
>
> A informação do progresso do upload é escrita na sessão antes da execução de quaisquer scripts. Portanto, alterar o nome da sessão através da função ini_set() ou session_name() resultará em uma sessão sem as informações de progresso do upload.

Sessões e Segurança

Índice

Links externos: » Fixação de Sessão

O gerenciamento de sessões HTTP representa o centro da segurança na web. Todas as medídas possíveis de mitigação *devem* ser adotadas para garantir que as sessões estão seguras. Os desenvolvedores devem também habilitar/utilizar as medidas de segurança aplicáveis.

Gerencimento básico de sessão

Segurança da sessão

O módulo de sessão não garante que a informação armazenada em uma sessão seja visualizada apenas pelo usuário que criou a sessão. Medidas adicionais devem ser tomadas para proteger a confidencialidade da sessão, dependendo do valor associado à ela.

A importância dos dados carregados na sessão precisa ser avaliada e proteções adicionais podem ser disparadas; isso normalmente tem um preço, como alguns inconveninentes para o usuário. Por exemplo, para proteger os usuários de uma tática simples de engenharia social, a opção session.use_only_cookies precisa ser habilitada. Neste caso, os cookies devem obrigatoriamente estar habilitados no lado do cliente, ou as sessões não irão funcionar.

Existem várias maneiras de expor um ID de sessão existente para terceiros. Exemplos: injeção de JavaScript, ID de sessão nas URLs, interceptação de pacotes, acesso físico ao dispositivo, etc. Um ID de sessão exposto possibilita que terceiros tenham acesso a todos os recursos que estão associados a um ID específico. Primeiro, URLs que contenham IDs de sessão. Se houver ligações para um site ou recurso externo, a URL que contém o ID de sessão pode ser armazenada nos registros do site externo. Segundo, um atacante mais ativo pode interceptar o tráfego de rede. Caso não estejam criptografados, os IDs de sessão serão transmitidos em texto puro pela rede. A solução é implementar SSL/TLS no servidor e torná-lo obrigatório para os usuários. HSTS deve ser usado para uma melhor segurança.

> **Nota** :
>
> Até mesmo o HTTPS não consegue proteger dados confidenciais em todos os momentos. Por exemplo, as vulnerabilidades "CRIME" e "Beast" podem permitir que um atacante leia os dados. Além disso, existem muitas redes que utilizam proxy HTTPS MITM com o propósito de auditoria. Atacantes também podem configurar um proxy semelhante.

Gerenciamento de sessão não adaptativo

O gerenciador de sessão do PHP atualmente é adaptativo por padrão atualmente. Os gerenciadores de sessão adaptativos possuem riscos adicionais.

Quando session.use_strict_mode está habilitada e o manipulador de armazenamento de sessão suportá-la, um ID de sessão que não inicializado é rejeitado, e um novo é criado. Isso protege contra ataques que forçam o usuário a usar um ID de sessão conhecido. Atacantes podem colar links ou enviar e-mail que contém ID de sessão. Ex.: `http://example.com/page.php?PHPSESSID=123456789`, se session.use_trans_sid estiver habilitado, a vítima iniciará uma sessão usando o ID de sessão fornecido pelo atacante. A opção session.use_strict_mode mitiga este risco.

> **Aviso**
>
> Um manipulador de armazenamento definido pelo usuário também pode suportar o modo de sessão estrita implementando validação do ID de sessão. Todos os manipuladores de armazenamento definidos pelo usuário devem implementar uma validação do ID de sessão.

O cookie de ID de sessão pode ser definido usando os atributos domain, path, httponly e secure; e a partir do PHP 7.3, SameSite. Existe uma certa precedência que é definida pelos navegadores. Ao usar a precedência, o atacante pode definir um ID de sessão que pode ser usado permanentemente. O uso de session.use_only_cookies não resolverá essa questão. session.use_strict_mode mitiga este risco. Com session.use_strict_mode=On, um ID de sessão não inicializado será recusado.

> **Nota** :
>
> Apesar da opção session.use_strict_mode diminuir o risco do gerenciamento de sessão adaptativo, um atacante pode forçar os usuários a usarem um ID de sessão inicializado e que foi criado pelo atacante. Por exemplo, com injeção de JavaScript. Esse tipo de ataque pode ser evitado seguindo as recomendações deste manual.
>
> Ao seguir este manual, desenvolvedores devem habilitar a opção session.use_strict_mode, e também usa um gerenciamento de sessão baseado em timestamp e gerar novamente um ID de sessão usando session_regenerate_id() com os procedimentos recomendados. Se tudo isso for seguido, o ID de sessão gerado por um atacante eventualmente será deletado.
>
> Quando ocorre um acesso à uma sessão obsoleta, todos os dados da sessão ativa devem ser salvos. Isso será útil para uma investigação mais tarde. Será feito logout forçado do usuário de todas as sessões, ou seja, o usuário será obrigado a se autenticar novamente. Desta forma se evita que atacantes abusem de sessões roubadas.

> **Aviso**
>
> O acesso aos dados de sessões obsoletas nem sempre é por causa de ataques. Redes instáveis e/ou remoção imediata de sessão ativa farão com que usuários legítimos usem sessões obsoletas.

A partir do PHP 7.1.0, session_create_id() foi adicionado. Essa função poderia ser usada para adicionar o ID do usuário como prefixo no ID de sessão para acessar a sessão ativa do usuário de forma eficiente. Habilitar session.use_strict_mode é muito importante nesta configuração. Caso contrário, usuários mal intencionados podem definir IDs de sessão maliciosos para outros usuários.

> **Nota** :
>
> Usuários de versões anteriores ao PHP 7.1.0 *devem* usar CSPRNG, como por exemplo */dev/urandom* ou random_bytes(), e funções de hash para gerar um novo ID de sessão. session_create_id() possui detecção de colisão e gera o ID de sessão de acordo com as configurações INI de sessão. O uso de session_create_id() é preferível.

Renovação do ID de sessão

session.use_strict_mode é uma boa prevenção, mas não é o suficiente. Desenvolvedores devem usar session_regenerate_id() para a segurança da sessão.

A renovação do ID de sessão reduz o risco de roubo do ID de sessão, sendo assim, session_regenerate_id() deve ser chamada periodicamente. Por exemplo, a renovação do ID de sessão a cada 15 minutos para a segurança de conteúdos sensíveis. Mesmo se o ID de sessão for roubado, tanto a sessão do usuário legítimo quanto a do atacante terão expirados. Ou seja, o acesso do usuário ou do atacante irá gerar erro de acesso à sessão obsoleta.

O ID de sessão *deve* ser renovado quando o privilégios do usuário são elevados, por exemplo após o usuário se autenticar. session_regenerate_id() deve ser chamado antes de salvar as informações de autenticação em *$_SESSION*. (session_regenerate_id() salva os

dados da sessão atual automaticamente com o intuito de salvar o timestamp/etc na sessão atual.) Assegure-se de que apenas a nova sessão contenha flag autenticada.

Desenvolvedores *não devem* depender da expiração do ID de sessão proveniente de session.gc_maxlifetime. Atacantes podem acessar o ID de sessão da vítima periodicamente para impedir que ele expire e para poder continuar explorando o ID, inclusive as sessões autenticadas.

Ao invés disso, desenvolvedores devem implementar um gerenciamento de sessão baseada em timestamp.

> **Aviso**
>
> Embora o gerenciador de sessão possa gerenciar o timestamp de forma transparente, essa funcionalidade não está implementada. Dados de sessões antigas devem ser mantidos até a execução do GC (coletor de lixo). Ao mesmo tempo, desenvolvedores devem assegurar-se de remover dados de sessões obsoletas. Porém, os desenvolvedores não devem remover dados de sessões ativas imediatamente. Isto é, `session_regenerate_id(true);` e session_destroy() nunca devem ser chamados juntos para uma sessão ativa. Isso pode soar contraditório, mas é um requisito obrigatório.

session_regenerate_id() *não* apaga sessão antiga por padrão. Uma sessão obsoleta e autenticada pode estar disponível para uso. Os desenvolvedores devem impedir que sessões desatualizadas sejam utilizadas por qualquer pessoa. Eles devem impedir acesso acesso aos dados de sessões obsoletas por conta própria utilizando timestamps.

> **Aviso**
>
> A remoção repentina de sessão ativa produz efeitos colaterais indesejados. A sessão pode desaparecer quando houver conexões concorrentes em uma aplicação web e/ou a rede estiver instável.
>
> Possíveis acessos maliciosos também não podem ser detectados com a remoção imediata de sessões ativas.
>
> Ao invés de remover a sessão antiga imediatamente, deve ser configurado, na *$_SESSION*, um tempo curto (timestamp) para a expiração e proibir acesso aos dados da sessão (essa implementação é por contra própria).
>
> O acesso aos dados de sessões antigas não deve ser bloqueado imediatamente depois de executar session_regenerate_id(). O acesso deve ser bloqueado um pouco depois, como por exemplo, alguns segundos depois para redes estáveis, como redes cabeadas, e alguns minutos depois para redes instáveis como celulares ou sem Wi-Fi.
>
> Se um usuário tentar acessar uma sessão obsoleta (que já tenha expirado), o acesso deve ser proibido. É recomendável que seja removido o status de autenticação de todas as sessões do usuário, porque é provável que seja um ataque.

O uso adequado de session.use_only_cookies e session_regenerate_id() poderia causar um ataque DoS pessoal por causa de cookies impossíveis de serem removidos e que foram configurados por atacantes. Quando isso acontecer, o desenvolvedor pode solicitar aos usuários que removam os cookies e avisá-los que podem existir problemas de segurança. Atacantes podem configurar cookies maliciosos via aplicações web vulneráveis, extensões maliciosas para navegadores, dispositivos comprometidos fisicamente, etc.

> **Aviso**
>
> Não interprete de forma equivocada o risco de DoS. session.use_strict_mode=On é obrigatório para a segurança geral do ID de sessão! É recomendável que todos os sites habilitem session.use_strict_mode.
>
> DoS somente pode acontecer quando a conta está sob ataque. Uma vulnerabilidade de injeção de JavaScript na aplicação representa a causa mais comum.

Remoção dos dados de sessão

Os dados de sessões obsoletas devem ser inacessíveis e removidos. O módulo de sessão atual não manipula isso muito bem.

É melhor remover os dados de sessões obsoletas o mais cedo possível. Ainda assim, sessões ativas não devem ser removidas imediatamente. Para preencher esses requisitos, o gerenciamento dos dados de sessões baseadas em timestamp deve ser implementado pelo próprio desenvolvedor.

Configure e gerencie um timestamp de expiração na $_SESSION. Bloqueie os acessos aos dados de sessões desatualizadas. Quando um acesso aos dados de uma sessão obsoleta for detectado, é aconselhável remover todos os status de autenticação das sessões dos usuários e forçá-los a refazer a autenticação. O acesso aos dados de uma sessão obsoleta pode ser um ataque. Para fazer isso, deve ser mantido um registro das sessões ativas por usuário.

> **Nota** :
>
> O acesso à uma sessão obsoleta pode ocorrer por causa de redes instáveis e/ou acessos concorrentes ao website. O servidor tenta definir um novo ID de sessão via cookie, mas o pacote que define o cookie (Set-Cookie) pode não ter chegado ao cliente por perda de conexão. Uma conexão pode gerar um novo ID de sessão executando session_regenerate_id(), mas uma outra conexão concorrente pode não ter pego ainda o novo ID de sessão. Além disso, o acesso aos dados da sessão obsoleta deve ser bloqueado algum tempo depois. Ou seja, o gerenciamento de sessão baseada em timestamp é necessário.

Em poucas palavras, dados de sessão não devem ser destruídos com session_regenerate_id() ou session_destroy(), mas timestamps dem ser utilizados para controlar o acesso aos dados da sessão. Deixe que session_gc() remova os dados obsoletos do armazenamento de dados da sessão.

Sessão e Travamento dos dados

Os dados de sessão são travados para evitar condição de corrida (data race) por padrão. A trava é necessária para manter os dados consistentes entre as requisições.

Contudo, o travamento pode ser abusado por um atacante para realizar um ataque DoS. Para diminuir os riscos de DoS por travamento de sessão, reduza o travamento. Use sessões somente leitura quando a alteração dos dados não for necessária. Use a opção 'read_and_close' com session_start(). `session_start(['read_and_close'=>1]);` Feche a sessão assim que você terminar de alterar $_SESSION, usando session_commit().

O módulo de sessão atual *não* detecta modificações em $_SESSION enquanto a sessão está inativa. É responsabilidade do desenvolvedor não modificar a variável $_SESSION quando a sessão está inativa.

Sessões ativas

Os desenvolvedores devem manter um registro de sessões ativas por usuário. E notificar o usuário sobre quantas sessões ativas, de qual IP (e área), há quanto tempo a sessão está ativa, etc. O PHP não mantém um registro dessas informações. É o desenvolvedor quem deve manter.

Existem inúmeras formas de implementação. Pode ser configurado um banco de dados que mantém um registro dos dados necessários e armazena as informações nele. Como os dados de sessão são removidos pelo coletor de lixo, o desenvolvedor deve cuidar dos dados removidos para manter a consistência do banco de dados das sessões ativas.

Uma das implementações mais simples é prefixar o ID de sessão com o ID do usuário e armazenar as informações necessárias na $_SESSION. Muitos bancos de dados têm bom desempenho para selecionar o prefixo de uma string. Desemvolvedores PODEM usar session_regenerate_id() e session_create_id() para isso.

> **Aviso**
>
> Nunca utilize informações confidenciais como prefixo. Se o ID do usuário for confidencial, considere a utilização de hash_hmac().

> **Aviso**
>
> Habilitar session.use_strict_mode é obrigatório para esse setup. Certifique-se de que essa opção esteja habilitada, caso contrário o banco de dados de sessões ativas pode ser comprometido.

O gerenciamento de sessão baseada em timestamp é necessário para detectar acesso em sessões obsoletas. Quando um acesso à uma sessão obsoleta é detectado, as flags de autenticação devem ser removidas de todas as sessões ativas para esse usuário. Isso evita que atacantes explorem sessões roubadas.

Sessão e login automático

Desenvolvedores não devem utilizar ID de sessão de longa vida para o login automático, pois isso aumenta o risco de roubo da sessão. O login automático deve ser implementado pelo desenvolvedor.

Use um hash seguro de uso único como chave para o login automático usando setcookie(). Use um hash seguro e mais forte que SHA-2, como por exemplo, SHA-256 ou maior com dados randômicos provenientes de random_bytes() ou */dev/urandom*.

Se o usuário não estiver autenticado, verifique se a chave de login automático é válida ou não. Se a chave é válida, autentique o usuário e configure uma nova chave "one-time hash" segura. Deve ser possível usar a chave de login automático apenas uma vez, ou seja, nunca reutilize uma chave de login automático; ao invés disso, sempre gere uma nova chave.

A chave de login automático é uma chave de autenticação de longa vida, portanto essa chave deve estar o mais protegida possível. Use os atributos de cookie path/httponly/secure para protegê-la. Ou seja, nunca transmita a chave de login automático a não ser que seja realmente necessário.

Desenvolvedores devem implementar funcionalidades que desabilitam login automático e removem cookies de login automático desnecessários.

Ataques CSRF (Cross-Site Request Forgeries)

Sessão e autenticação não protegem contra ataques CSRF. Os desenvolvedores devem implementar proteções contra CSRF por conta própria.

output_add_rewrite_var() pode ser usada para proteção contra CSRF. Visite o manual para mais detalhes.

> **Nota** :
>
> Versões do PHP anteriores à 7.2.0 utilizam o mesmo buffer de saída e as mesmas configurações INI que o recurso trans sid. Portanto, o uso de output_add_rewrite_var() em versões do PHP anteriores à 7.2.0 não é recomendado.

A maioria dos frameworks para aplicações web tem suporte à proteção CSRF. Visite o manual do framework de sua aplicação para mais detalhes.

A partir do PHP 7.3, o atributo SameSite para cookies de sessão pode ser utilizado. É uma medida adicional que pode mitigar vulnerabilidades CSRF.

Protegendo as configurações INI relacionadas à sessão

Ao proteger as configurações INI relacionadas à sessão, a segurança das sessões também aumenta. Algumas configurações INI importantes não possuem recomendações. O desenvolvedor é o responsável em garantir a segurança das configurações de sessão.

- session.cookie_lifetime=0

  `0` tem um sentido especial. Ele diz para o navegador não armazenar cookies no armazenamento permanente. Sendo assim, quando o navegador é encerrado, o cookie com o ID de sessão é deletado imediatamente. Se o desenvolvedor configurar um valor diferente de "0", pode permitir que outros usuários utilizem o ID de sessão. A maioria das aplicações devem utilizar `0` para isto.

  Se a funcionalidade de login automático é necessária, o desenvolvedor deve implementá-la por conta própria e de forma segura. Não utilize um ID de sessão de longa vida para isso. Mais informações podem ser encontradas acima na seção sobre este assunto.

- session.use_cookies=On

  session.use_only_cookies=On

  Embora o cookie HTTP tenha alguns problemas, ele é o modo preferido para gerenciar o ID de sessão. Utilize apenas cookies para o gerenciamento do ID de sessão quando possível. A maioria das aplicações devem utilizar cookie para o ID de sessão.

  Se session.use_only_cookies=Off, o módulo de sessão usará valores para o ID de sessão definidos por GET ou POST desde que o cookie de ID de sessão não tenha sido inicializado.

- session.use_strict_mode=On

  Contudo, habilitar session.use_strict_mode é obrigatório para sessões seguras. Essa opção está desabilitada por padrão.

  Essa opção evita que o módulo de sessão utilize um ID de sessão que não tenha sido inicializado. Em outras palavras, o módulo de sessão aceita apenas ID de sessão válido e que tenha sido gerado pelo módulo de sessão. O módulo de sessão rejeita o ID caso ele tenha sido fornecido pelo usuário.

  Por causa das especificações dos cookies, atacantes podem configurar cookies de ID de sessão indeletáveis por causa de configurações locais ou por injeção de JavaScript. session.use_strict_mode pode evitar que um ID de sessão inicializado por um atacante seja utilizado.

  > **Nota** :

> Atacantes podem inicializar um ID de sessão utilizando os próprios computadores e depois configurá-lo como o da vítima. No entanto, eles precisariam manter o ID de sessão ativo. Também seria necessário alguns passos adicionais para executar um ataque nesse cenário. Portanto, a opção session.use_strict_mode funciona como prevenção.

- session.cookie_httponly=On

  Proíbe o acesso ao cookie de sessão através do JavaScript. Essa configuração evita o roubo de cookies por injeção de JavaScript.

  É possível usar um ID de sessão como chave de proteção contra CSRF, mas isso não é recomendado. Por exemplo, o código HTML pode ser salvo e enviado para outros usuários. Os desenvolvedores não devem escrever o ID de sessão nas páginas, para evitar problemas de segurança. Quase todas as aplicações devem utilizar o atributo httponly para o cookie do ID de sessão.

  > **Nota** :
  >
  > O token de proteção contra CSRF deve ser renovado periodicamente, exatamente como o ID de sessão.

- session.cookie_secure=On

  Permite o acesso ao cookie de ID de sessão apenas quando o protocolo é HTTPS. Se seu website utiliza apenas HTTPS, então essa opção deve ser habilitada.

  O uso de HSTS deve ser considerado em websites que utilizem apenas HTTPS.

- session.cookie_samesite="Lax" ou session.cookie_samesite="Strict"

  A partir do PHP 7.3 o atributo `"SameSite"` pode ser definido para o cookie de ID de sessão. Este atributo é uma forma de se mitigar ataques CSRF (Cross Site Request Forgery).

  A diferença entre Lax e Strict é a acessibilidade do cookie em requisições que se original de outro domínio registrável empregando o método HTTP GET. Cookies usando Lax serão acessíveis em uma requisição GET originada de outro domínio registrável, enquanto que cookies que usam Strict não serão.

- session.gc_maxlifetime=[choose smallest possible]

  session.gc_maxlifetime é a configuração para remoção de ID de sessão obsoleta. Depender dessa configuração *não* é recomendado. O gerenciamento do tempo de vida da sessão deve ser feito utilizando timestamp e por contra própria.

  É melhor que a coleta de lixo (garbage collection) da sessão seja feita pela função session_gc(). session_gc() é recomendada ser executada pelos gerenciadores de tarefas, como, por exemplo, cron em sistemas tipo Unix.

  Por padrão, a coleta de lixo (GC) é executada por probabilidade. Essa configuração *não* garante a remoção de sessões antigas. Embora o desenvolvedor não possa depender dessa configuração, defini-la com o menor valor possível é recomendado. Ajuste session.gc_probability e session.gc_divisor para que as sessões obsoletas sejam deletadas na frequência apropriada. Se a funcionalidade de login automático é necessária, o desenvolvedor deve implementá-la por contra própria e de forma segura, veja acima para mais informação. Nunca utilize ID de sessão de longa vida para isso.

  > **Nota** :
  >
  > Alguns módulos de manipuladores de armazenamento de sessão não utilizam essa configuração para a expiração baseada em probabilidade, como, por exemplo, memcached e memcache. Visite a documentação do manipulador de armazenamento de sessão para mais detalhes.

- session.use_trans_sid=Off

  O uso de gerenciamento transparente do ID de sessão não é proibido. Ele pode ser utilizado quando necessário. Contudo, desabilitar o gerenciamento transparente do ID de sessão melhora a segurança geral do ID de sessão, pois remove a possibilidade de injeção e vazamento do ID de sessão.

  > **Nota** :
  >
  > O ID de sessão pode ser exposto através de uma URL salva nos Favoritos do navegador, uma URL enviada por e-mail ou mesmo caso o código-fonte HTML seja salvo.

- session.trans_sid_tags=[tags limitadas]

  (PHP 7.1.0 >=) Desenvolvedores não devem reescrever tags HTML sem necessidade. A configuração padrão é suficiente para a maioria dos casos. Versões anteriores do PHP utilizam a configuração url_rewriter.tags para tal.

- session.trans_sid_hosts=[hosts limitados]

  (PHP 7.1.0 >=) Essa configuração INI define uma lista de hosts que permitem a reescrita do recurso trans sid. Nunca adicione hosts que não sejam confiáveis. O módulo de sessão permite apenas o valor de `$_SERVER['HTTP_HOST']` quando essa configuração está vazia.

- session.referer_check=[sua URL de origem]

  Quando session.use_trans_sid estiver habilitada, o uso dessa opção é recomendado. Ela reduz o risco de injeção do ID de sessão. Se seu site é `http://example.com/`, defina essa opção como `http://example.com/`. Note que quando HTTPS é utilizado, o navegador não enviará o cabeçalho relacionado ao "referer". O navegador também pode não enviar o "referer" dependendo da configuração. Portanto, essa configuração não é uma medida de segurança confiável. O uso desta configuração é recomendado.

- session.cache_limiter=nocache

  Certifique-se de que o conteúdo HTTP não é armazenado em cache para sessões autenticadas. Permita o armazenamento em cache apenas quando o conteúdo não é privado. Caso contrário, o conteúdo pode ser exposto. `"private"` pode ser usado se o conteúdo HTTP não incluir dados sensíveis/confidenciais. Note que `"private"` pode fazer com que dados privados sejam armazenados em cache em clientes compartilhados. `"public"` pode ser usado somente quando o conteúdo HTTP não contém dados privados e ou confidenciais.

- session.hash_function="sha256"

  (PHP 7.1.0 <) Funções de hash mais fortes também geram um ID de sessão mais forte. Embora a colisão de hash é pouco provável até mesmo com MD5, desenvolvedores devem usar SHA-2 ou funções de hash ainda mais fortes para essa tarefa. Os desenvolvedores podem usar um hash mais forte como sha384 ou sha512. Certifique-se de informar um valor suficientemente longo em entropy para a função de hash usada.

- session.save_path=[diretório não público]

  Se for definido para um diretório público, como */tmp* (o padrão), outros usuários no servidor podem ser capazes de sequestrar seções obtendo a lista de arquivos nesse diretório.

Funções para Sessão

Índice

session_abort

session_abort

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

session_abort — Descarta as alterações no array da sessão e encerra a sessão

Descrição

function **session_abort**(): bool

session_abort() encerra a sessão sem salvar dados. Consequentemente, os valores originais da sessão são mantidos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.2.0	O tipo de retorno dessa função agora é bool. Antigamente, era void.

Veja também

- *$_SESSION*
- Diretiva de configuração session.auto_start
- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session_reset() - Reinicializa um array de sessão com os valores originais
- session_commit() - Sinônimo de session_write_close

session_cache_expire

session_cache_expire

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

session_cache_expire — Obter e/ou definir a expiração do cache atual

Descrição

function **session_cache_expire**(?int `$value` = `null`): int|false

session_cache_expire() retorna a configuração atual de `session.cache_expire`.

O prazo para o cache expirar é redefinido para o padrão de 180 guardado em session.cache_expire na inicialização da requisição. Então você precisa chamar **session_cache_expire()** para cada requesição (e antes que session_start() seja utilizada).

Parâmetros

`value`

Se `value` é informado e não `null`, a expiração do cache atual é modificado para `value`.

> **Nota** :
>
> Definir o valor de `value` tem valor apenas se `session.cache_limiter` é definido como um valor *diferente* de `nocache`.

Valor Retornado

Retorna a configuração atual de `session.cache_expire`. O valor retornado deve ser lido em minutos, padrão de 180. Caso não seja possível alterar o valor, `false` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`value` é anulável agora.

Examples

Exemplo #1 Exemplo de session_cache_expire()

```php
<?php

/* define o limitador de cache para 'private' */

session_cache_limiter('private');
$cache_limiter = session_cache_limiter();

/* define o prazo do cache em 30 minutos */
session_cache_expire(30);
$cache_expire = session_cache_expire();

/* inicia a sessão */

session_start();

echo "O limitador de cache está definido agora como $cache_limiter<br />";
echo "As sessões em cache irão expirar depois de $cache_expire minutos";
?>
```

Veja também

- session.cache_expire
- session.cache_limiter
- session_cache_limiter() - Obtém e/ou define o limitador do cache atual

session_cache_limiter

session_cache_limiter

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

session_cache_limiter — Obtém e/ou define o limitador do cache atual

Descrição

function **session_cache_limiter**(?string `$value` = `null`): string|false

session_cache_limiter() retorna o nome do atual limitador do cache.

O limitador do cache controla quais HTTP headers de controle de cache são enviados para o cliente. Estes headers determinam as regras pelas quais o conteúdo da página pode ser guardado em cache pelo cliente e proxy intermediário. Definindo o limitador do cache para `nocache` rejeitaria qualquer cache do cliente/proxy. Um valor como `public` permitiria o cache do proxy e do cliente, e `private` rejeitaria o cache do proxy mas permite que o cliente armazene o conteúdo em cache.

No modo `private`, o cabeçalho Expire enviado para o cliente pode provocar confusão em alguns browsers, incluindo o Mozilla. Você pode evitar este problema usando o modo `private_no_expire`. O cabeçalho `Expire` nunca é enviado para o cliente nesse modo.

Definindo o limitador do cache como `''` desativará completamente o envio automático de cabeçalhos de cache.

O limitador do cache é redefinido para o valor padrão guardado em session.cache_limiter na inicialização da requisição. Então você precisa chamar **session_cache_limiter()** para cada requisição (e antes que session_start() seja chamada).

Parâmetros

`value`

Se `value` é especificado e não `null`, o nome do limitador de cache atual é modificado para o novo valor.

Valores possíveis

Valor	Cabeçalhos enviados

`public`	`Expires: (em algum momento do futuro, de acordo com session.cache_expire)`
	`Cache-Control: public, max-age=(em algum momento do futuro, de acordo com session.cache_expire)`
	`Last-Modified: (o timestamp do script atual)`

`private_no_expire`	`Cache-Control: private, max-age=(session.cache_expire no futuro)` `Last-Modified: (o timestamp do script atual)`

`private`	`Expires: Thu, 19 Nov 1981 08:52:00 GMT`
	`Cache-Control: private, max-age=(session.cache_expire no futuro)`
	`Last-Modified: (o timestamp do script atual)`

`nocache`	`Expires: Thu, 19 Nov 1981 08:52:00 GMT`
	`Cache-Control: no-store, no-cache, must-revalidate`
	`Pragma: no-cache`

Valor Retornado

Retorna o nome do limitador de cache atual. Caso não seja possível alterar o valor, `false` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`value` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de session_cache_limiter()

```php
<?php

/* define o limitador do cache para 'private' */

session_cache_limiter('private');
$cache_limiter = session_cache_limiter();

echo "O limitador do cache está definido agora para $cache_limiter<br />";
?>
```

Veja também

- session.cache_limiter

session_commit

session_commit

(PHP 4 >= 4.4.0, PHP 5, PHP 7, PHP 8)

session_commit — Sinônimo de session_write_close()

Descrição

Esta função é um apelido para: session_write_close().

session_create_id

session_create_id

(PHP 7 >= 7.1.0, PHP 8)

session_create_id — Cria novo ID de sessão

Descrição

function **session_create_id**(string `$prefix = ""`): string|false

session_create_id() é usado para criar um novo id de sessão para a sessão atual. Ele retorna um ID de sessão livre de colisão.

Se a sessão não estiver ativa, a verificação de colisão será omitida.

O id de sessão é criado de acordo com a configuração no php.ini.

É importante usar o mesmo ID de usuário do seu servidor web para o script de tarefa do GC. Caso contrário, você poderá ter problemas de permissão, especialmente com o gerenciador de salvamento de arquivos.

Parâmetros

`prefix`

> Se `prefix` for especificado, o novo id de sessão é prefixado por `prefix`. Nem todos os caracteres são permitidos no ID da sessão. Caracteres na faixa `[a-zA-Z0-9,-]` são permitidos. O comprimento máximo é de 256 caracteres.

Valor Retornado

session_create_id() retorna um novo id de sessão livre de colisão para a sessão atual. Se for usado sem uma sessão ativa, a verificação de colisão é omitida. Em caso de falha, `false` é retornado.

Examples

Exemplo #1 session_create_id() exemplo com session_regenerate_id()

```php
<?php
// Minha função de início de sessão suporta gerenciamento de timestamp
function my_session_start() {
    session_start();
    // Não permitir o uso de ID de sessão muito antigo
    if (!empty($_SESSION['deleted_time']) && $_SESSION['deleted_time'] < time() - 180) {
        session_destroy();
        session_start();
    }
}

// Minha função de regeneração de id de sessão
function my_session_regenerate_id() {
    // Chame session_create_id() enquanto a sessão estiver ativa
    // para garantir que não haja colisões.
    if (session_status() != PHP_SESSION_ACTIVE) {
        session_start();
    }
    // AVISO: Nunca use strings confidenciais para prefixo!
    $newid = session_create_id('myprefix-');
    // Definir timestamp de exclusão. Os dados da sessão não devem ser excluídos imediatamente por motivos.
    $_SESSION['deleted_time'] = time();
    // Terminar sessão
    session_commit();
    // Certifique-se de aceitar o ID de sessão definido pelo usuário
    // NOTA: Você deve ativar use_strict_mode para operações normais.
    ini_set('session.use_strict_mode', 0);
    // Define novo ID de sessão personalizado
    session_id($newid);
    // Comece com um ID de sessão personalizado
    session_start();
}

// Certifique-se de que use_strict_mode está habilitado.
// use_strict_mode é obrigatório por motivos de segurança.
ini_set('session.use_strict_mode', 1);
my_session_start();

// O ID da sessão deve ser regenerado quando
// - Usuário se loga
// - Usuário se desloga
// - Certo período já passou
my_session_regenerate_id();

// Escreva códigos úteis
?>
```

Veja também

- session_regenerate_id() - Atualiza o id da sessão atual com um novo id gerado
- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session.use_strict_mode
- SessionHandler::create_sid() - Retorna um novo ID de sessão

session_decode

session_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

session_decode — Decodifica dados de sessão de uma sessão codificada em formato string

Descrição

function **session_decode**(string `$data`): bool

session_decode() decodifica os dado de sessão serializados fornecidos em `data` e preenche a super global $_SESSION com o resultado.

Por padrão, o método de deserialização usado é interno do PHP e não é o mesmo que unserialize(). O método de deserialização pode ser configurado usando session.serialize_handler.

Parâmetros

`data`

Os dados codificados para serem armazenados.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- session_encode() - Codifica os dados atuais da sessão como uma sessão codificada em formato string
- session.serialize_handler

session_destroy

session_destroy

(PHP 4, PHP 5, PHP 7, PHP 8)

session_destroy — Destrói todos os dados registrados em uma sessão

Descrição

function **session_destroy**(): bool

session_destroy() destrói todos os dados associados com a sessão atual. Ela não apaga nenhuma das variáveis globais associadas à sessão atual, nem apaga o cookie de sessão. Para usar as variáveis de sessão novamente, session_start() deve ser chamada.

> **Nota** :
>
> Não é necessário chamar **session_destroy()** em um código habitual. Ao invés de destruir os dados da sessão, limpe o array $_SESSION.

Para matar a sessão completamente, o ID da sessão também deve ser apagado. Se um cookie for usado para propagar o ID da sessão (funcionamento padrão), então o cookie de sessão deve ser excluído. setcookie() pode ser usado para isso.

Quando session.use_strict_mode está habilitado, não é necessário remover o cookie de ID das sessões obsoletas porque o módulo de sessão não aceitará um cookie de ID de sessão quando não houver dados associados ao ID de sessão, e irá criar um novo cookie de ID de sessão. Habilitar session.use_strict_mode é recomendado para todos os sites.

> **Aviso**
>
> A remoção imediata da sessão pode causar resultados indesejados. Quando há requisições concorrentes, outras conexões podem perder de forma repentina os dados da sessão, como por exemplo, requisições originadas no JavaScript e/ou requisições de links.
>
> Embora o módulo de sessão atual não aceita um cookie de ID de sessão vazio, a remoção imediata da sessão pode resultar em um cookie de ID de sessão vazio por causa da condição de corrida no navegador do usuário. Isso fará com que o usuário crie vários IDs de sessão desnecessariamente.
>
> Para evitar isso, deve ser definido um timestamp de removação em $_SESSION e o acesso à sessão deve ser rejeitado algum tempo depois. Ou então a aplicação não pode ter requisições concorrentes. Isso se aplica à session_regenerate_id() também.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Destruindo uma sessão com *$_SESSION*

```php
<?php
// Inicializa a sessão.
// Se estiver sendo usado session_name("something"), não esqueça de usá-lo agora!
session_start();

// Apaga todas as variáveis da sessão
$_SESSION = array();

// Se é preciso matar a sessão, então os cookies de sessão também devem ser apagados.
// Nota: Isto destruirá a sessão, e não apenas os dados!
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// Por último, destrói a sessão
session_destroy();
?>
```

Veja também

- session.use_strict_mode
- session_reset() - Reinicializa um array de sessão com os valores originais
- session_regenerate_id() - Atualiza o id da sessão atual com um novo id gerado
- unset() - unset remove a definição de uma variável informada
- setcookie() - Envia um cookie

session_encode

session_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

session_encode — Codifica os dados atuais da sessão como uma sessão codificada em formato string

Descrição

function **session_encode**(): string|false

session_encode() retorna uma string serializada do conteúdo atual da sessão armazenados na super global $_SESSION.

Por padrão, o método de serialização usado é interno do PHP e não é o mesmo que serialize(). O método de serialização pode ser configurado usando session.serialize_handler.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o conteúdo da sessão atual codificado, ou `false` em caso de falha.

Notas

> **Aviso**
>
> É necessário chamar session_start() antes de usar **session_encode()**.

Veja também

- session_decode() - Decodifica dados de sessão de uma sessão codificada em formato string
- session.serialize_handler

session_gc

session_gc

(PHP 7 >= 7.1.0, PHP 8)

session_gc — Executa a coleta de lixo de dados da sessão

Descrição

function **session_gc**(): int|false

Por padrão, o PHP usa session.gc_probability para executar o coletor de lixo da sessão probabilisticamente em cada requisição. Existem algumas limitações com essa abordagem:

- Sites com baixo tráfego podem não ter seus dados de sessão excluídos dentro do período recomendado.
- Em sites com alto tráfego, o coletor de lixo pode ser acionado com muita frequência, realizando trabalho extra desnecessário.
- A coleta de lixo é realizada mediante solicitação do usuário, e o usuário pode experimentar um atraso.

Para sistemas de produção, recomenda-se desativar a coleta de lixo baseada em probabilidade, definindo session.gc_probability para `0` e acionar explicitamente o coletor de lixo periodicamente, por exemplo, usando "cron" em sistemas do tipo UNIX para executar um script que chama **session_gc()**.

> **Nota** :
>
> Ao chamar a função **session_gc()** a partir de um script PHP de linha de comando, a diretiva session.save_path deve ser definida para o mesmo valor das requisições web, e o script deve ter permissões de acesso e exclusão para os arquivos de sessão. Isso pode ser afetado pelo usuário que executa o script, e por recursos de contêiner ou "caixa de areia", como a opção `PrivateTmp=` do systemd.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

session_gc() retorna o número de dados de sessão excluídos em caso de sucesso, ou `false` em caso de falha.

> **Nota** :
>
> Os manipuladores de salvamento de sessão antigos não retornam o número de dados de sessão excluídos, apenas um indicativo de sucesso ou falha. Se este for o caso, o número de dados de sessão excluídos se torna `1`, independentemente dos dados realmente excluídos.

Examples

Exemplo #1 Exemplo de session_gc() para gerenciadores de tarefas como o cron

```php
<?php
// Observação: Este script deve ser executado pelo mesmo usuário do processo do servidor web.

// É necessário ter uma sessão ativa para inicializar o acesso ao armazenamento de dados da sessão.
session_start();

// Executa a coleta de lixo (GC) imediatamente.
session_gc();

// Limpa o ID da sessão criado pelo session_start()
session_destroy();
?>
```

Exemplo #2 Exemplo de session_gc() para script acessível pelo usuário

```php
<?php
// Observação: session_gc() é recomendado para ser usado por um script de gerenciador de tarefas, mas
// pode ser utilizado da forma abaixo.

// Usado para verificar o último horário de coleta de lixo (GC).
$gc_time = '/tmp/php_session_last_gc';
$gc_period = 1800;
```

```php
session_start();
// Execute a coleta de lixo (GC) apenas quando o período de GC tiver decorrido.
// Ou seja, chamar session_gc() a cada solicitação é desperdício de recursos.
if (file_exists($gc_time)) {
    if (filemtime($gc_time) < time() - $gc_period) {
        session_gc();
        touch($gc_time);
    }
} else {
    touch($gc_time);
}
?>
```

Veja também

- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session_destroy() - Destrói todos os dados registrados em uma sessão
- session.gc_probability

session_get_cookie_params

session_get_cookie_params

(PHP 4, PHP 5, PHP 7, PHP 8)

session_get_cookie_params — Obtém os parâmetros do cookie da sessão

Descrição

function **session_get_cookie_params**(): array

Obtém os parâmetros do cookie da sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com informação do cookie da sessão atual. O array contém os seguintes items:

- "lifetime" - A duração do cookie em segundos.
- "path" - O caminho onde a informação está armazenada.
- "domain" - O domínio do cookie.
- "secure" - O cookie deve ser enviado apenas em conexões seguras.
- "httponly" - O cookie pode ser acessado somente atráves do protocolo HTTP.
- "samesite" - Controla o envio entre domínios do cookie.

Registro de Alterações

Versão	Descrição
7.3.0	A entrada "samesite" foi adicionada ao array retornado.

Veja também

- session.cookie_lifetime
- session.cookie_path
- session.cookie_domain
- session.cookie_secure
- session.cookie_httponly
- session.cookie_samesite
- session_set_cookie_params() - Define os parâmetros do cookie de sessão

session_id

session_id

(PHP 4, PHP 5, PHP 7, PHP 8)

session_id — Obtém e/ou define o id de sessão atual

Descrição

function **session_id**(?string `$id` = `null`): string|false

session_id() é usado para obter ou definir o id de sessão para a sessão atual.

A constante `SID` também pode ser usada para obter o nome e id da sessão atual como uma string adequado para adicionar em URLs. Veja também Manipulação de Sessão.

Parâmetros

`id`

Se `id` for especificado e não `null`, ele substituirá o id de sessão atual. **session_id()** precisa ser chamado antes de session_start() para este fim. Dependendo do manipulador de sessão, nem todos os caracteres são permitidos em um id de sessão. Por exemplo, o manipulador de sessão em arquivo permite apenas caracteres no intervalo `[a-zA-Z0-9,-]`!

> **Nota** :
>
> Quando estiver usando cookies de sessão, especificar um `id` para **session_id()** sempre enviará um novo cookie quando session_start() for chamada, sem importar se o id da sessão atual for idêntico ao que está sendo definido.

Valor Retornado

session_id() retorna o id de sessão para a sessão atual ou uma string vazia (`""`) se não houver sessão atual (não existe um id de sessão atual). Em caso de falha, `false` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`id` é anulável agora.

Veja também

- session_regenerate_id() - Atualiza o id da sessão atual com um novo id gerado
- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session_set_save_handler() - Define funções de armazenamento de sessão à nível de usuário
- session.save_handler

session_module_name

session_module_name

(PHP 4, PHP 5, PHP 7, PHP 8)

session_module_name — Obtém e/ou define o módulo da sessão atual

Descrição

function **session_module_name**(?string `$module` = `null`): string|false

session_module_name() obtém o nome do módulo da sessão atual, que também é conhecido como session.save_handler.

Parâmetros

`module`

Se `module` é especificado e não `null`, então é este módulo que será usado. É proibido passar `"user"` para este parâmetro. Em vez de session_set_save_handler() deve ser chamado para definir um usuário manipulador de sessão definido.

Valor Retornado

Retorna o nome do módulo da sessão atual, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`module` é anulável agora.
7.2.0	Agora é explicitamente proibido definir o nome do módulo como `"user"`. Anteriormente, isso era silenciosamente ignorado.

session_name

session_name

(PHP 4, PHP 5, PHP 7, PHP 8)

session_name — Obtém e/ou define o nome da sessão atual

Descrição

function **session_name**(?string `$name` = `null`): string|false

session_name() retorna o nome da sessão atual. Se `name` for informado, **session_name()** irá atualizar o nome da sessão e retornar o nome da sessão *antiga*.

Se um novo nome de sessão for informado no parâmetro `name`, **session_name()** modificará o cookie HTTP (e exibir o conteúdo quando session.use_trans_sid estiver habilitado). Assim que o cookie HTTP tiver sido enviado, chamar **session_name()** gerará um `E_WARNING`. **session_name()** deve ser chamada antes de session_start() para a sessão funcionar apropriadamente.

O nome da sessão é redefinido para o padrão guardado em `session.name` na inicialização da requisição. Por isso, deve-se chamar **session_name()** para cada requisição (e antes que session_start() seja chamada).

Parâmetros

`name`

O nome da sessão, que é usado em cookies e URLs (por exemplo, `PHPSESSID`). Ele deve conter apenas caracteres alfanuméricos; deve ser curto e descritivo (para usuários com avisos de cookie habilitados). Se `name` for informado e diferente de `null`, o nome da sessão atual será modificado para o novo valor.

> **Aviso**
>
> O nome da sessão não pode consistir apenas de dígitos, pelo menos uma letra deve estar presente. Caso contrário um novo ID de sessão é gerado toda vez.

Valor Retornado

Retorna o nome da sessão atual. Se `name` for informado e a função atualizar o nome da sessão, o nome da sessão *antiga* será retornado, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`name` é anulável agora.
7.2.0	**session_name()** verifica status de sessão; anteriormente, apenas verificava status de cookie. Portanto, a versão antiga de **session_name()** permitia chamar **session_name()** após session_start(), que poderia travar o PHP e resultar em comportamento inesperado.

Examples

Exemplo #1 Exemplo de session_name()

```php
<?php

/* define o nome da sessão para WebsiteID */

$previous_name = session_name("WebsiteID");
```

```
echo "O nome da sessão anterior era $previous_name<br />";
?>
```

Veja também

- session.name (diretiva de configuração)

session_regenerate_id

session_regenerate_id

(PHP 4 >= 4.3.2, PHP 5, PHP 7, PHP 8)

session_regenerate_id — Atualiza o id da sessão atual com um novo id gerado

Descrição

function **session_regenerate_id**(bool `$delete_old_session = false`): bool

session_regenerate_id() substituirá o id da sessão atual com um novo id e manterá a informação da sessão atual.

Quando session.use_trans_sid estiver habilitada, saídas (output) devem ser iniciadas depois de **session_regenerate_id()** ser chamada. Caso contrário, o ID da sessão antiga é utilizado.

> **Aviso**
>
> Atualmente, session_regenerate_id não lida muito bem em redes instáveis como, por exemplo, móvel e WiFi. Pode ocorrer perda da sessão ao chamar session_regenerate_id.
>
> Os dados de sessões antigas não devem ser destruídos imediatamente; ao invés disso, deve ser usado timestamp para destruir a sessão e controlar o acesso aos dados de sessões antigas. Caso contrário, acessos concorrentes à página podem resultar em um estado inconsistente, em perda da sessão, ou pode causar uma condição de corrida no lado do cliente (navegador) e pode criar vários IDs de sessão desnecessariamente. A remoção imediata dos dados de sessão impossibilita a detecção de ataques de roubo de sessão, e também a prevenção.

Parâmetros

`delete_old_session`

Se o arquivo associado à sessão anterior deve ser excluído ou não. A sessão antiga não deve ser removida se for necessário evitar condições de corrida causadas pela remoção, ou se for necessário detectar/evitar ataques de roubo de sessão.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de session_regenerate_id()

```
<?php
// NOTA: Este código não é totalmente funcional, mas apenas um exemplo!

session_start();

// Verifica o timestamp para remoção
if (isset($_SESSION['destroyed'])
    && $_SESSION['destroyed'] < time() - 300) {
    // Normalmente não deve acontecer. Isso pode ser um ataque ou então causado por uma rede instável.
    // Remove todos os status de autenticação da sessão deste usuário.
    remove_all_authentication_flag_from_active_sessions($_SESSION['userid']);
    throw(new DestroyedSessionAccessException);
}

$old_sessionid = session_id();

// Define o timestamp para remoção
$_SESSION['destroyed'] = time(); // session_regenerate_id() salva os dados da sessão anterior
```

```php
// Apenas chamar session_regenerate_id() pode causar a perda da sessão, etc.
// Veja o próximo exemplo.
session_regenerate_id();

// A nova sessão não precisa da flag destroyed
unset($_SESSION['destroyed']);

$new_sessionid = session_id();

echo "Old Session: $old_sessionid<br />";
echo "New Session: $new_sessionid<br />";

print_r($_SESSION);
?>
```

O módulo de sessão atual não trabalha bem com redes instáveis. O programador deve gerenciar o ID de sessão para evitar a perda da sessão causada por session_regenerate_id.

Exemplo #2 Evitando a perda de sessão causada por session_regenerate_id()

```php
<?php
// NOTA: Este código não é totalmente funcional, mas apenas um exemplo!
// my_session_start() e my_session_regenerate_id() evitam a perda da sessão causada por
// redes instáveis. Além disso, este código pode evitar que a sessão seja
// roubada por atacantes.

function my_session_start() {
    session_start();
    if (isset($_SESSION['destroyed'])) {
       if ($_SESSION['destroyed'] < time()-300) {
           // Normalmente não deve acontecer. Isso pode ser um ataque ou então causado por uma rede instável.
           // Remove todos os status de autenticação da sessão deste usuário.
           remove_all_authentication_flag_from_active_sessions($_SESSION['userid']);
           throw(new DestroyedSessionAccessException);
       }
       if (isset($_SESSION['new_session_id'])) {
           // Não está completamente expirado ainda. O cookie pode ter sido perdido por causa de uma rede instável.
           // Tenta novamente definir o cookie de ID de sessão.
           // NOTA: Não tente definir o ID de sessão novamente se você quiser remover
           // as flags de autenticação
           session_commit();
           session_id($_SESSION['new_session_id']);
           // O novo ID de sessão deve existir
           session_start();
           return;
       }
   }
}

function my_session_regenerate_id() {
    // Um novo ID de sessão é necessário para definir o ID de sessão corretamente
    // quando o ID de sessão não é configurado por causa de instabilidade na rede.
    $new_session_id = session_create_id();
    $_SESSION['new_session_id'] = $new_session_id;

    // Definie o timestamp para remoção
    $_SESSION['destroyed'] = time();

    // Escreve e fecha a sessão
    session_commit();

    // Inicia a sessão com um novo ID de sessão
    session_id($new_session_id);
    ini_set('session.use_strict_mode', 0);
    session_start();
    ini_set('session.use_strict_mode', 1);

    // A nova sessão não precisa destas informações
    unset($_SESSION['destroyed']);
    unset($_SESSION['new_session_id']);
```

```
}
?>
```

Veja também

- session_id() - Obtém e/ou define o id de sessão atual
- session_create_id() - Cria novo ID de sessão
- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session_destroy() - Destrói todos os dados registrados em uma sessão
- session_reset() - Reinicializa um array de sessão com os valores originais
- session_name() - Obtém e/ou define o nome da sessão atual

session_register_shutdown

session_register_shutdown

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

session_register_shutdown — Função de finalização da sessão

Descrição

function **session_register_shutdown**(): void

Registra session_write_close() como uma função de finalização.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Dispara um `E_WARNING` se o processo de registrar a função falhar.

session_reset

session_reset

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

session_reset — Reinicializa um array de sessão com os valores originais

Descrição

function **session_reset**(): bool

session_reset() reinicializa uma sessão com os valores originais salvos no armazenamento de sessão. Esta função requer uma sessão ativa e descarta as alterações em $_SESSION.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.2.0	O tipo de retorno desta função é bool agora. Anteriormente, era void.

Veja também

- *$_SESSION*
- A diretiva de configuração session.auto_start

- session_start() - Inicia uma nova sessão ou continua uma sessão existente
- session_abort() - Descarta as alterações no array da sessão e encerra a sessão
- session_commit() - Sinônimo de session_write_close

session_save_path

session_save_path

(PHP 4, PHP 5, PHP 7, PHP 8)

session_save_path — Obtém e/ou define o caminho para armazenamento da sessão atual

Descrição

function **session_save_path**(?string `$path = null`): string|false

session_save_path() retorna o caminho do diretório atual utilizado para salvar os dados de sessão.

Parâmetros

`path`

Caminho dos dados de sessão. Se informado e não `null`, o caminho onde os dados são salvos será alterado. **session_save_path()** precisa ser chamada antes de session_start() para isto.

> **Nota** :
>
> Em alguns sistemas operacionais, talvez seja melhor especificar um caminho para um sistema de arquivos que possa lidar eficientemente com uma grande quantidade de arquivos pequenos.

Valor Retornado

Retorna o caminho atual do diretório usado para o armazenamento de dados, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`path` agora pode ser nulo.

Veja também

- session.save_path (diretiva de configuração)

session_set_cookie_params

session_set_cookie_params

(PHP 4, PHP 5, PHP 7, PHP 8)

session_set_cookie_params — Define os parâmetros do cookie de sessão

Descrição

```
function session_set_cookie_params(
    int $lifetime_or_options,
    ?string $path = null,
    ?string $domain = null,
    ?bool $secure = null,
    ?bool $httponly = null
): bool
```

Assinatura alternativa disponível a partir do PHP 7.3.0:

function **session_set_cookie_params**(array `$lifetime_or_options`): bool

Define parâmetros dos cookies configurados no arquivo *php.ini*. O efeito desta função é apenas pela duração do script. Então, você precisa chamar **session_set_cookie_params()** para cada requisição e antes que session_start() seja chamada.

Esta função atualiza os valores em tempo de execução correspondentes às configurações INI que podem ser obtidos com ini_get().

Parâmetros

`lifetime_or_options`

Ao usar a primeira assinatura, este parâmetro indica o tempo de vida do cookie de sessão, definido em segundos.

Ao usar a segunda assinatura, este parâmetro deve conter um array associativo que pode ter qualquer uma das chaves `lifetime` (tempo de vida), `path` (caminho), `domain` (domínio), `secure` (seguro), `httponly` (somente http) e `samesite` (mesmo site). Os valores têm o mesmo significado descrito para os parâmetros com o mesmo nome. O valor do elemento `samesite` deve ser `Lax` ou `Strict`. Se alguma das opções permitidas não for fornecida, seus valores padrão serão os iguais aos valores padrão dos parâmetros explícitos. Se o O elemento `samesite` for omitido, nenhum atributo SameSite do cookie é definido.

`path`

Caminho no domínio onde o cookie irá funcionar. Use uma única barra ('/') para que funcione em todos os caminhos do domínio.

`domain`

Domínio do cookie, por exemplo 'www.php.net'. Para tornar os cookies visíveis em todos os subdomínios, o domínio deve ser prefixado com um ponto, como '.php.net'.

`secure`

Se **`true`**, o cookie só será enviado em conexões seguras.

`httponly`

Se **`true`**, então o PHP tentará enviar a opção httponly ao definir o cookie de sessão.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`path`, `domain`, `secure` e `httponly` são anuláveis agora.
7.3.0	Uma assinatura alternativa que suporta `lifetime_or_options` array foi adicionada. Esta assinatura suporta também a configuração do atributo de cookie SameSite.
7.2.0	Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha. Anteriormente a função retornava void.

Veja também

- session.cookie_lifetime
- session.cookie_path
- session.cookie_domain
- session.cookie_secure
- session.cookie_httponly
- session.cookie_samesite
- session_get_cookie_params() - Obtém os parâmetros do cookie da sessão

session_set_save_handler

session_set_save_handler

(PHP 4, PHP 5, PHP 7, PHP 8)

session_set_save_handler — Define funções de armazenamento de sessão à nível de usuário

Descrição

function **session_set_save_handler**(
  callable `$open`,
  callable `$close`,
  callable `$read`,

    callable $write,
    callable $destroy,
    callable $gc,
    callable $create_sid = ?,
    callable $validate_sid = ?,
    callable $update_timestamp = ?
): bool

É possível registrar o seguinte protótipo:

function **session_set_save_handler**(object $sessionhandler, bool $register_shutdown = **true**): bool

session_set_save_handler() define, a nível de usuário, as funções de armazenamento de sessão que serão usadas para guardar e buscar dados associados à uma sessão. Ela é mais útil quando um método de armazenamento diferente do método disponibilizado pelas sessões do PHP é preferido, por exemplo armazenar os dados de sessão em um banco de dados.

Parâmetros

Essa função tem dois protótipos.

`sessionhandler`

Uma instância de uma classe implementando SessionHandlerInterface, e opcionalmente SessionIdInterface e/ou SessionUpdateTimestampHandlerInterface, como SessionHandler, para registrá-la como manipulador de sessão.

`register_shutdown`

Registra session_write_close() como uma função register_shutdown_function().

ou

`open`

Um callable com a seguinte assinatura:

function open(string $savePath, string $sessionName): bool

O callback open funciona igual um construtor em classes e é executado quando a sessão está sendo aberta. É a primeira função de callback executada quando a sessão é iniciada automaticamente ou manualmente com session_start(). O valor de retorno é **true** para sucesso, **false** para falha.

`close`

Um callable com a seguinte assinatura:

function close(): bool

O callback close funciona igual um destrutor em classes e é executado depois que o callback write da sessão for chamado. Ele também é invocado quando a função session_write_close() é chamada. O valor de retorno é **true** para sucesso, **false** para falha.

`read`

Um callable com a seguinte assinatura:

function read(string $sessionId): string

O callback `read` sempre deve retornar uma sessão codificada (serializada) no formato string ou uma string vazia se não há dados para leitura.

Esse callback é chamado internamente pelo PHP quando a sessão inicia ou quando a função session_start() é chamada. Antes que esse callback seja invocado, o PHP invocará o callback `open`.

O valor que esse callback retorna deve estar exatamente no mesmo formato de serialização que originalmente foi passado para armazenamento ao callback `write`. O valor retornado será desserializado automaticamente pelo PHP e usado para preencher a super global *$_SESSION*. Mesmo que os dados pareçam similares ao de serialize(), note que é um formato diferente, especificado na configuração INI session.serialize_handler.

`write`

Um callable com a seguinte assinatura:

function write(string `$sessionId`, string `$data`): bool

O callback `write` é chamado quando a sessão precisa ser salva e fechada. Esse callback recebe o ID da sessão atual e uma versão serializada da super global *$_SESSION*. O método de serialização usado internamente pelo PHP é definido pela configuração INI session.serialize_handler.

Os dados de sessão serializados passado para esse callback devem ser armazenados junto com o ID de sessão passado. Ao buscar esses dados, o callback `read` deve retornar exatamente o mesmo valor que foi passado originalmente para o callback `write`.

Esse callback é invocado quando o PHP é encerrado ou explicitamente quando a função session_write_close() é chamada. Note que depois de executar esta função, o PHP executará internamente o callback `close`.

> **Nota** :
>
> O manipulador "write" não é executado enquanto o fluxo de saída não for encerrado. Portanto, saídas de comandos de debug no manipulador "write" nunca serão vistas pelo browser. Se a saída de debug é necessária, é recomendado que ela seja escrita em arquivo.

`destroy`

Um callable com a seguinte assinatura:

function destroy(string `$sessionId`): bool

Esse callback é executado quando uma sessão é destruída com session_destroy() ou com session_regenerate_id() com o parâmetro de destruição definido como **`true`**. O valor de retorno deve ser **`true`** para sucesso, **`false`** para falha.

`gc`

Um callable com a seguinte assinatura:

function gc(int `$lifetime`): bool

O callback de limpeza (gc) é invocado pelo PHP, internamente e periodicamente, para apagar dados de sessão antiga. A frequência é controlada por session.gc_probability e session.gc_divisor. O valor do parâmetro que é passado para esse callback pode ser definido em session.gc_maxlifetime. O valor de retorno deve ser **`true`** para sucesso, **`false`** para falha.

`create_sid`

Um callable com a seguinte assinatura:

function create_sid(): string

Esse callback é executado quando um novo ID de sessão é necessário. Nenhum parâmetro é necessário e o valor de retorno deve ser uma string que seja um ID de sessão válido para seu manipulador.

`validate_sid`

Um callable com a seguinte assinatura:

function validate_sid(string `$key`): bool

Este callback é executado quando uma sessão é iniciada, um ID de sessão é fornecido e session.use_strict_mode é habilitado. A `key` é o ID da sessão para validar. Um ID de sessão é válido se já existir uma sessão com esse ID. O valor retornado deve ser **`true`** em caso de sucesso, **`false`** em caso de falha.

`update_timestamp`

Um callable com a seguinte assinatura:

function update_timestamp(string `$key`, string `$val`): bool

Este callback é executado quando uma sessão é atualizada. `key` é o ID da sessão, `val` são os dados da sessão. O valor retornado deve ser **`true`** em caso de sucesso, **`false`** em caso de falha.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.4.0	Chamar **session_set_save_handler()** com mais de dois argumentos agora foi descontinuado. Use a assinatura com dois argumentos.

Examples

Exemplo #1 Manipulador de sessão personalizado: veja o código completo na sinópse de SessionHandlerInterface.

Aqui é demonstrado apenas a execução, o código completo pode ser visto na sinópse de SessionHandlerInterface no link acima.

Note que é usado orientação à objetos com **session_set_save_handler()** e a função de encerramento (register_shutdown) é registrada usando sua respectiva flag. Isto geralmente é aconselhável ao registrar objetos como manipuladores de gravação de sessão.

```php
<?php
class MySessionHandler implements SessionHandlerInterface
{
    // implementa a interface aqui
}

$handler = new MySessionHandler();
session_set_save_handler($handler, true);
session_start();

// proceder para definir e recuperar os valores pela chave de $_SESSION
```

Notas

> **Aviso**
>
> The `write` e `close` são executados depois da destruição de objetos e portanto não podem usar objetos ou lançar exceções. Não é possível capturar nem exibir o rastro (trace) de exceções e a execução simplesmente será interrompida de forma inesperada. Contudo, os destrutores do objeto podem usar sessões.
>
> É possível chamar session_write_close() do destrutor para resolver esse problema de 'o ovo e a galinha' mas a forma mais confiável é registrar a função de finalização como descrito acima.

> **Aviso**
>
> O diretório de trabalho atual é alterado com algumas SAPIs (Server API) se a sessão for fechada na finalização do script. É possível fechar a sessão antecipadamente com session_write_close().

Veja também

- session.save_handler (diretiva de configuração)
- session.serialize_handler (diretiva de configuração)
- A register_shutdown_function() - Registra uma função para ser executada no final da execução do script
- The session_register_shutdown() - Função de finalização da sessão
- Visite » save_handler.inc para ver uma implementação completamente procedural

session_start

session_start

(PHP 4, PHP 5, PHP 7, PHP 8)

session_start — Inicia uma nova sessão ou continua uma sessão existente

Descrição

function **session_start**(array `$options = []`): bool

session_start() cria uma sessão ou continua a sessão atual baseado em um ID de sessão passado via GET ou POST, ou passado via cookie.

Quando **session_start()** é chamada ou quando uma sessão inicia automaticamente, o PHP chamará os manipuladores de gravação de sessão de abertura e de leitura. Eles serão manipuladores de gravação internos fornecidos por padrão ou por extensões do PHP (como

SQLite ou Memcached); ou podem ser manipuladores personalizados definidos por session_set_save_handler(). A função de leitura recuperará qualquer informação de sessão existente (armazenada em um formato serializado especial) e será desserializado e usado para preencher automaticamente a variável super global $_SESSION quando a função de leitura retornar os dados de sessão gravados para o tratamento de sessão do PHP.

Para utilizar uma sessão com nome, session_name() deve ser chamada antes de **session_start()**.

Quando session.use_trans_sid está habilitada, a função **session_start()** registrará um manipulador de saída interno para a reescrita da URL.

Se um usuário usar `ob_gzhandler` ou similar com ob_start(), a ordem da função é importante para saída/output adequado. Por exemplo, `ob_gzhandler` deve ser registrado antes de iniciar a sessão.

Parâmetros

`options`

> Se informado, este é um array associativo de opções que irão substituir os que estiverem configurados nas diretivas de configuração de sessão. As chaves não devem incluir o prefixo `session.`.
>
> Além do conjunto de diretivas de configuração comum, a opção `read_and_close` pode ser informada. Se definida como **`true`**, fará com que a sessão seja fechada imediatamente depois de ser lida, evitando o bloqueio desnecessário caso os dados da sessão não sejam alterados.

Valor Retornado

Esta função retorna **`true`** se uma sessão foi iniciada com sucesso, caso contrário **`false`**.

Registro de Alterações

Versão	Descrição
7.1.0	**session_start()** agora retorna **`false`** e não mais inicializa *$_SESSION* quando este falha em iniciar a sessão.

Examples

Um exemplo básico de sessão

Exemplo #1 *page1.php*

```php
<?php
// page1.php

session_start();

echo 'Bem vindo à página #1';

$_SESSION['favcolor'] = 'green';
$_SESSION['animal']   = 'cat';
$_SESSION['time']     = time();

// Funciona se o cookie de sessão foi aceito
echo '<br /><a href="page2.php">page 2</a>';

// Ou talvez passando o ID da sessão se necessário
echo '<br /><a href="page2.php?' . SID . '">page 2</a>';
?>
```

Após acessar *page1.php*, a segunda página *page2.php* magicamente terá os dados da sessão. Leia funções para sessão para informações sobre propagação de IDs de sessão já que, por exemplo, explica o que é a constante **`SID`**.

Exemplo #2 *page2.php*

```php
<?php
// page2.php

session_start();

echo 'Bem vindo à página #2<br />';

echo $_SESSION['favcolor']; // green
```

```php
echo $_SESSION['animal'];   // cat
echo date('Y m d H:i:s', $_SESSION['time']);

// Pode ser desejado usar o SID aqui, como foi feito em page1.php
echo '<br /><a href="page1.php">page 1</a>';
?>
```

Fornecendo configurações para **session_start()**

Exemplo #3 Sobrepondo o tempo de duração de cookie

```php
<?php
// Isto envia um cookie persistente que dura um dia
session_start([
    'cookie_lifetime' => 86400,
]);
?>
```

Exemplo #4 Lendo e fechando a sessão

```php
<?php
// Se não houver necessidade de alterar nada na
// sessão, pode-se apenas lê-la e já fechá-la para evitar
// travar o arquivo de sessão e bloquear outras páginas
session_start([
    'cookie_lifetime' => 86400,
    'read_and_close'  => true,
]);
```

Notas

> **Nota** :
>
> Para usar sessões baseadas em cookies, **session_start()** deve ser chamada antes de enviar qualquer saída para o browser.

> **Nota** :
>
> O uso de zlib.output_compression é recomendado ao invés de ob_gzhandler()

> **Nota** :
>
> Esta função envia vários cabeçalhos HTTP dependendo da configuração. Veja session_cache_limiter() para personalizar estes cabeçalhos.

Veja também

- *$_SESSION*
- session.auto_start (diretiva de configuração)
- session_id() - Obtém e/ou define o id de sessão atual

session_status

session_status

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

session_status — Retorna o status atual da sessão

Descrição

function **session_status**(): int

session_status() é usada para retornar o status atual da sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

- `PHP_SESSION_DISABLED` se as sessões estiverem desabilitadas.
- `PHP_SESSION_NONE` se as sessões estiverem habilitadas, mas nenhuma existir.
- `PHP_SESSION_ACTIVE` se as sessões estiverem habilitadas, e uma existir.

Veja também

- session_start() - Inicia uma nova sessão ou continua uma sessão existente

session_unset

session_unset

(PHP 4, PHP 5, PHP 7, PHP 8)

session_unset — Libera todas as variáveis de sessão

Descrição

function **session_unset**(): bool

A função **session_unset()** libera todas as variáveis atualmente registradas na sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.2.0	O tipo de retorno desta função é bool agora. Anteriormente, era void.

Notas

> **Nota** :
>
> Se *$_SESSION* é usado, use unset() para desregistrar a variável da sessão, isto é, `unset($_SESSION['varname']);`.

> **Cuidado**
>
> NÃO elimine completamente *$_SESSION* com `unset($_SESSION)` já que isto desabilitará o registro de variáveis de sessão através da super global *$_SESSION*.

> **Nota** :
>
> O uso de **session_unset()** é idêntico a `$_SESSION = []`.

> **Cuidado**
>
> Esta função funciona apenas se uma sessão estiver ativa. Não limpará o array *$_SESSION* se a sessão ainda não foi iniciada ou já foi destruída. Use `$_SESSION = []` para desmontar todas as variáveis de sessão mesmo que a sessão não esteja ativa.

session_write_close

session_write_close

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

session_write_close — Guarda os dados de sessão e fecha a sessão

Descrição

function **session_write_close**(): bool

Fecha a sessão atual e guarda os dados.

Os dados de sessão geralmente são guardados depois que o script termina, sem a necessidade de chamar **session_write_close()**; mas como dados de sessão são travados para evitar escritas concorrentes, apenas um script pode operar em uma sessão por vez. Ao usar iframe/framesets junto com sessões, os frames serão lidos um a um devido à esta trava. Pode-se reduzir o tempo necessário para carregar todos os frames fechando a sessão o mais cedo possível, assim que todas as alterações na variável de sessão tiverem sido feitas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.2.0	O tipo de retorno desta função é bool agora. Anteriormente, era void.

Veja também

- A session_register_shutdown() - Função de finalização da sessão

SessionHandler

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introdução

SessionHandler é uma classe especial que pode ser usada para expor o manipulador interno atual do PHP de gravação de sessão por herança. Existem sete métodos que envolvem (wrap) as sete funções internas de callbacks do manipulador de gravação de sessão (`open`, `close`, `read`, `write`, `destroy`, `gc` e `create_sid`). Por padrão, esta classe vai envolver qualquer manipulador de gravação interno definido pela diretiva de configuração session.save_handler, que normalmente é `files` por padrão. Outros manipuladores internos de gravação de sessão podem ser fornecidos por extensões do PHP, como por exemplo SQLite (como `sqlite`), Memcache (como `memcache`), e Memcached (como `memcached`).

Quando uma instância de **SessionHandler** é definida como manipulador de gravação usando session_set_save_handler(), ela envolverá o manipulador de gravação atual. Uma classe que estende **SessionHandler** permite sobrescrever os métodos, interceptá-los ou filtrá-los chamando os métodos da classe pai que envolvem os manipuladores de sessão internos do PHP.

Isto permite, por exemplo, interceptar os métodos `read` e `write` para criptografar/descriptografar os dados de sessão e então passar o resultado de e para a classe pai. Alternativamente, pode-se sobrescrever completamente um método como o callback de limpeza `gc`.

Como o **SessionHandler** envolve os manipuladores de gravação atuais, o exemplo acima de criptografia pode ser aplicado em qualquer manipulador de gravação interno sem precisar saber o funcionamento interno dos manipuladores.

Para usar esta classe, primeiro configure o manipulador de gravação que você quer expor usando session.save_handler e então passe uma instância de **SessionHandler** ou uma classe que a estenda para session_set_save_handler().

Note que os métodos de callbacks desta classe são projetados para serem chamados internamente pelo PHP e não para serem chamados pelo código do usuário. Os valores de retorno são igualmente processados internamente pelo PHP. Para mais informações do fluxo de trabalho da sessão, consulte session_set_save_handler().

Resumo da classe

class **SessionHandler** implements SessionHandlerInterface, SessionIdInterface {
/* Métodos */
public function close(): bool
public function create_sid(): string

public function destroy(string $id): bool
public function gc(int $max_lifetime): int|false
public function open(string $path, string $name): bool
public function read(string $id): string|false
public function write(string $id, string $data): bool
}

Notas

> **Aviso**
>
> Esta classe é projetada para expor o manipulador interno do PHP de gravação de sessão; se você quiser escrever manipuladores de gravação personalizados, implemente a interface SessionHandlerInterface ao invés de estender a classe **SessionHandler**.

Examples

Exemplo #1 Usando SessionHandler para adicionar criptografia aos manipuladores internos do PHP de gravação.

```php
<?php

 /**
  * decrypt AES 256
  *
  * @param data $edata
  * @param string $password
  * @return decrypted data
  */
function decrypt($edata, $password) {
    $data = base64_decode($edata);
    $salt = substr($data, 0, 16);
    $ct = substr($data, 16);

    $rounds = 3; // depende do comprimento da chave
    $data00 = $password.$salt;
    $hash = array();
    $hash[0] = hash('sha256', $data00, true);
    $result = $hash[0];
    for ($i = 1; $i < $rounds; $i++) {
        $hash[$i] = hash('sha256', $hash[$i - 1].$data00, true);
        $result .= $hash[$i];
    }
    $key = substr($result, 0, 32);
    $iv  = substr($result, 32,16);

    return openssl_decrypt($ct, 'AES-256-CBC', $key, true, $iv);
  }

/**
 * crypt AES 256
 *
 * @param data $data
 * @param string $password
 * @return base64 encrypted data
 */
function encrypt($data, $password) {
    // Gera um salt aleatório criptograficamente seguro usando random_bytes()
    $salt = random_bytes(16);

    $salted = '';
    $dx = '';
    // Salt em key(32) e iv(16) = 48
    while (strlen($salted) < 48) {
        $dx = hash('sha256', $dx.$password.$salt, true);
        $salted .= $dx;
    }

    $key = substr($salted, 0, 32);
    $iv  = substr($salted, 32,16);

    $encrypted_data = openssl_encrypt($data, 'AES-256-CBC', $key, true, $iv);
```

```
    return base64_encode($salt . $encrypted_data);
}

class EncryptedSessionHandler extends SessionHandler
{
    private $key;

    public function __construct($key)
    {
        $this->key = $key;
    }

    public function read($id)
    {
        $data = parent::read($id);

        if (!$data) {
            return "";
        } else {
            return decrypt($data, $this->key);
        }
    }

    public function write($id, $data)
    {
        $data = encrypt($data, $this->key);

        return parent::write($id, $data);
    }
}

// iremos interceptar o manipulador nativo de arquivos, mas trabalharemos igualmente
// com outros manipuladores internos como 'sqlite', 'memcache' ou 'memcached'
// que são disponibilizados por extensões do PHP
ini_set('session.save_handler', 'files');

$key = 'secret_string';
$handler = new EncryptedSessionHandler($key);
session_set_save_handler($handler, true);
session_start();

// proceder para definir e recuperar os valores pela chave de $_SESSION
```

> **Nota** :
>
> Como os métodos dessa classe são projetados para serem chamados internamente pelo PHP como parte do fluxo normal da sessão, chamadas à esses métodos (ou seja, os manipuladores internos nativos reais) por classes filhas irão retornar `false`, a não ser que a sessão tenha sido iniciada (independente se foi de forma automática ou pela chamada de session_start()). É importante levar isto em consideração quando escrever testes unitários onde os métodos da classe podem ser invocados manualmente.

Índice

SessionHandler::close

SessionHandler::close

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::close — Fecha a sessão

Descrição

public function **SessionHandler::close**(): bool

Fecha a sessão atual. Este método é executado automaticamente e internamente pelo PHP ao fechar a sessão, ou explicitamente com session_write_close() (que primeiro chama SessionHandler::write()).

Este método envolve o manipulador interno de gravação do PHP definido na configuração ini session.save_handler que foi definido antes deste manipulador ser ativado por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `close` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído ou interceptado.

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::close().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor retornado (geralmente **`true`** em caso de sucesso, **`false`** em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

SessionHandler::create_sid

SessionHandler::create_sid

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

SessionHandler::create_sid — Retorna um novo ID de sessão

Descrição

public function **SessionHandler::create_sid**(): string

Gera e retorna um novo ID de sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um ID de sessão válido para o manipulador de sessão padrão.

Veja também

- session_id() - Obtém e/ou define o id de sessão atual
- session_create_id() - Cria novo ID de sessão

SessionHandler::destroy

SessionHandler::destroy

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::destroy — Destrói uma sessão

Descrição

public function **SessionHandler::destroy**(string `$id`): bool

Destrói uma sessão. Chamada internamente pelo PHP com session_regenerate_id() (desde que `destroy` seja **`true`**), por session_destroy() ou quando session_decode() falhar.

Este método envolve o manipulador de gravação interno do PHP, definido na configuração ini session.save_handler, configurado antes da definição deste manipulador por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `destroy` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído e ou interceptado e filtrado.

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::destroy().

Parâmetros

`id`

O ID de sessão que estiver sendo destruído.

Valor Retornado

O valor retornado (geralmente `true` em caso de sucesso, `false` em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

SessionHandler::gc

SessionHandler::gc

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::gc — Apaga sessões antigas

Descrição

public function **SessionHandler::gc**(int `$max_lifetime`): int|false

Apaga sessões expiradas. Chamada internamente e aleatoriamente pelo PHP quando uma sessão inicia ou quando session_start() é invocada. A frequência que ela é chamada é baseada em session.gc_divisor e session.gc_probability (diretivas de configuração).

Este método envolve o manipulador de gravação interno do PHP definido na configuração ini session.save_handler e que foi configurado antes desse manipulador ser definido por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `gc` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído e ou interceptado e filtrado.

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::gc().

Parâmetros

`max_lifetime`

Sessões que não foram atualizados nos últimos `max_lifetime` segundos serão removidas.

Valor Retornado

Retornar o número de sessões eliminadas no caso de sucesso, ou `false` em caso de falha. Observe que esse valor é retornado internamente ao PHP para processamento.

Registro de Alterações

Versão	Descrição
7.1.0	Antes dessa versão, a função retornava `true` no caso de sucesso.

SessionHandler::open

SessionHandler::open

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::open — Inicia uma sessão

Descrição

public function **SessionHandler::open**(string `$path`, string `$name`): bool

Cria uma nova sessão, ou reinicializa uma sessão existente. Chamada internamente pelo PHP quando uma sessão inicia, quer seja de forma automática ou quando session_start() é invocada.

Este método envolve o manipulador de gravação interno do PHP definido na configuração ini session.save_handler e que foi configurado antes desse manipulador ser definido por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `open` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído e ou interceptado e filtrado.

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::open().

Parâmetros

`path`

O caminho para armazenar/recuperar a sessão.

`name`

O nome da sessão.

Valor Retornado

O valor retornado (geralmente `true` em caso de sucesso, `false` em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

Veja também

- session.auto-start (diretiva de configuração).

SessionHandler::read

SessionHandler::read

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::read — Lê os dados de sessão

Descrição

public function **SessionHandler::read**(string `$id`): string|false

Lê os dados de sessão do armazenamento de sessão e retorna o resultado de volta para o PHP para processamento interno. Este método é chamado automaticamente pelo PHP quando uma sessão é iniciada (quer seja de forma automática ou explicitamente com session_start()) e é precedida por uma chamada interna de SessionHandler::open().

Este método envolve o manipulador interno de gravação do PHP definido na configuração ini session.save_handler que foi definido antes deste manipulador ser ativado por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `read` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído ou interceptado e filtrado (por exemplo, descriptografar o valor de `data` retornado pelo método `read` da classe pai).

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::read().

Parâmetros

`id`

O id da sessão onde deve ser feita a leitura dos dados.

Valor Retornado

Retorna uma string, codificada, dos dados lidos. Se nada foi lido, deve retornar `false`. Note que este valor é retornado internamente ao PHP para processamento.

Veja também

- session.serialize_handler (diretiva de configuração).

SessionHandler::write

SessionHandler::write

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::write — Escreve os dados de sessão

Descrição

public function **SessionHandler::write**(string `$id`, string `$data`): bool

Escreve os dados de sessão no armazenamento de sessão. Chamada na finalização normal do PHP; por session_write_close(); ou quando session_register_shutdown() falha. O PHP chamará SessionHandler::close() imediatamente depois que este método retornar.

Este método envolve o manipulador interno de gravação do PHP definido na configuração ini session.save_handler que foi definido antes deste manipulador ser ativado por session_set_save_handler().

Se esta classe for estendida por herança, chamar o método `write` da classe pai invocará a função que envolve este método, portanto invocará o callback interno. Isto permite que esse método seja substituído ou interceptado e filtrado (por exemplo, descriptografar o valor de `data` antes de enviar para o método `write` da classe pai).

Para mais informações relacionadas ao que é esperado que este método faça, por favor leia a documentação de SessionHandlerInterface::write().

Parâmetros

`id`

O id da sessão.

`data`

Os dados de sessão codificados. Esta informação é o resultado codificado internamente pelo PHP da super global *$_SESSION* em uma string serializada e passada como este parâmetro. Note que sessões usam um método alternativo de serialização.

Valor Retornado

O valor retornado (geralmente **`true`** em caso de sucesso, **`false`** em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

Veja também

- session.serialize_handler (diretiva de configuração).

SessionHandlerInterface

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introdução

SessionHandlerInterface é uma interface que define o protótipo mínimo para criar um manipulador de sessão personalizado. Para passar um manipulador de sessão personalizado para session_set_save_handler() usando invocação OOP (Orientada à Objetos), a classe deve implementar esta interface.

Note que os métodos de callbacks desta classe são projetos para serem chamados internamente pelo PHP e não para serem chamados pelo código do usuário.

Resumo da Interface

interface **SessionHandlerInterface** {
/* Métodos */
public function close(): bool
public function destroy(string $id): bool
public function gc(int $max_lifetime): int|false
public function open(string $path, string $name): bool
public function read(string $id): string|false
public function write(string $id, string $data): bool
}

Examples

Exemplo #1 Exemplo usando **SessionHandlerInterface**

O exemplo a seguir fornece um armazenamento de sessão baseado em arquivos similar ao manipulador de gravação padrão de sessões do PHP, `files`. Este exemplo poderia facilmente ser estendido para cobrir armazenamento em banco de dados usando o motor de banco de dados de sua preferência suportado pelo PHP.

Observe que é usado o protótipo orientado a objetos com session_set_save_handler() e a função de encerramento (register_shutdown) é registrada usando a opção de parâmetro da função. Isto é geralmente aconselhável ao registrar objetos como manipuladores de gravação

de sessão.

> **Cuidado**
>
> Por brevidade, este exemplo omite a validação de entrada. No entanto, o parâmetros `$id` são, na verdade, valores fornecidos pelo usuário que exigem validação e sanitização adequadas para evitar vulnerabilidades, como problemas de passagem de caminho. *Portanto, não use este exemplo não modificado em ambientes de produção.*

```php
<?php
class MySessionHandler implements SessionHandlerInterface
{
    private $savePath;

    public function open($savePath, $sessionName): bool
    {
        $this->savePath = $savePath;
        if (!is_dir($this->savePath)) {
            mkdir($this->savePath, 0777);
        }

        return true;
    }

    public function close(): bool
    {
        return true;
    }

    #[\ReturnTypeWillChange]
    public function read($id)
    {
        return (string) @file_get_contents("$this->savePath/sess_$id");
    }

    public function write($id, $data): bool
    {
        return file_put_contents("$this->savePath/sess_$id", $data) === false ? false : true;
    }

    public function destroy($id): bool
    {
        $file = "$this->savePath/sess_$id";
        if (file_exists($file)) {
            unlink($file);
        }

        return true;
    }

    #[\ReturnTypeWillChange]
    public function gc($maxlifetime)
    {
        foreach (glob("$this->savePath/sess_*") as $file) {
            if (filemtime($file) + $maxlifetime < time() && file_exists($file)) {
                unlink($file);
            }
        }

        return true;
    }
}

$handler = new MySessionHandler();
session_set_save_handler($handler, true);
session_start();

// proceder para definir e recuperar valores pela chave de $_SESSION
```

Índice

SessionHandlerInterface::close

SessionHandlerInterface::close

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::close — Fecha a sessão

Descrição

public function **SessionHandlerInterface::close**(): bool

Fecha a sessão atual. Esta função é executada automaticamente quando a sessão é fechada ou explicitamente com session_write_close().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor retornado (geralmente **`true`** em caso de sucesso, **`false`** em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

SessionHandlerInterface::destroy

SessionHandlerInterface::destroy

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::destroy — Destrói uma sessão

Descrição

public function **SessionHandlerInterface::destroy**(string `$id`): bool

Destrói uma sessão. Chamada por session_regenerate_id() (se $destroy = **`true`**), session_destroy() e quando session_decode() falhar.

Parâmetros

`id`

O ID da sessão a ser destruída.

Valor Retornado

O valor retornado (geralmente **`true`** em caso de sucesso, **`false`** em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

SessionHandlerInterface::gc

SessionHandlerInterface::gc

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::gc — Remove sessões antigas

Descrição

public function **SessionHandlerInterface::gc**(int `$max_lifetime`): int|false

Remove sessões expiradas. Chamada por session_start(), baseado nas configurações session.gc_divisor, session.gc_probability e session.gc_maxlifetime.

Parâmetros

`max_lifetime`

Sessões que não foram atualizados nos últimos `max_lifetime` segundos serão removidas.

Valor Retornado

Retorna o número de sessões excluídas com sucesso, ou `false` em caso de falha. Observe que este valor é retornado internamente ao PHP para processamento.

Registro de Alterações

Versão	Descrição
7.1.0	Antes desta versão, a função retornava `true` em caso de sucesso.

SessionHandlerInterface::open

SessionHandlerInterface::open

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::open — Inicializa uma sessão

Descrição

public function **SessionHandlerInterface::open**(string `$path`, string `$name`): bool

Reinicializa uma sessão existente ou cria uma nova. Chamada quando uma sessão inicia ou quando session_start() é invocada.

Parâmetros

`path`

O caminho para armazenar/recuperar a sessão.

`name`

O nome da sessão.

Valor Retornado

O valor retornado (geralmente `true` em caso de sucesso, `false` em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

Veja também

- session_name() - Obtém e/ou define o nome da sessão atual
- session.auto-start (diretiva de configuração).

SessionHandlerInterface::read

SessionHandlerInterface::read

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::read — Lê os dados de sessão

Descrição

public function **SessionHandlerInterface::read**(string `$id`): string|false

Lê os dados de sessão do armazenamento de sessão e retorna os resultados. Chamada logo depois que a sessão inicia ou quando session_start() é chamada. Note que antes desse método ser chamado, o método SessionHandlerInterface::open() é invocado.

Este método é chamado pelo próprio PHP quando uma sessão é iniciada. Este método deve recuperar os dados de sessão do armazenamento de sessão relacionados ao ID de sessão fornecido. A string retornada por este método deve estar no mesmo formato de serialização de quando ela foi originalmente passada para SessionHandlerInterface::write(). Se a informação não for encontrada, retorna `false`

Os dados retornados por este método serão decodificados internamente pelo PHP usando o método de deserialização especificado em session.serialize_handler. Os dados resultantes serão usados para preencher a super global *$_SESSION*.

Note que o esquema de serialização não é o mesmo que unserialize() e pode ser acessado por session_decode().

Parâmetros

`id`

O id da sessão.

Valor Retornado

Retorna uma string codificada dos dados lidos. Se nada foi lido, então deve retornar `false`. Note que este valor é retornado internamente para o PHP para processamento.

Veja também

- session.serialize_handler (diretiva de configuração).

SessionHandlerInterface::write

SessionHandlerInterface::write

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::write — Escreve os dados de sessão

Descrição

public function **SessionHandlerInterface::write**(string `$id`, string `$data`): bool

Escreve os dados de sessão no armazenamento de sessão. Chamada por session_write_close(), quando session_register_shutdown() falhar ou na finalização normal. Note que o método SessionHandlerInterface::close() é chamado imediatamente depois desta função.

O PHP chamará este método quando a sessão estiver pronta para salvar e fechar. O PHP codifica os dados da super global *$_SESSION* em uma string serializada e passa o resultado junto com o ID de sessão à este método para armazenamento. O método de serialização usado é especificado na configuração session.serialize_handler.

Note que este método normalmente é chamado pelo PHP depois que o buffer de saída é fechado, a não ser que ele seja chamado explicitamente por session_write_close()

Parâmetros

`id`

O id da sessão.

`data`

Os dados de sessão codificados. Esta informação é o resultado codificado internamente pelo PHP da super global *$_SESSION* em uma string serializada e passada como este parâmetro. Note que sessões usam um método alternativo de serialização.

Valor Retornado

O valor retornado (geralmente `true` em caso de sucesso, `false` em caso de falha). Note que este valor é retornado internamente ao PHP para processamento.

Veja também

- session.serialize_handler (diretiva de configuração).

SessionIdInterface

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

Introdução

SessionIdInterface é uma interface que define métodos opcionais para criar um manipulador de sessão personalizado. Para passar um manipulador de sessão personalizado para session_set_save_handler() usando sua invocação OOP, a classe pode implementar esta interface.

Observe que os métodos de retorno de chamada das classes que implementam esta interface são projetados para serem chamados internamente pelo PHP e não devem ser chamados a partir do código do espaço do usuário.

Resumo da Interface

interface **SessionIdInterface** {

/* Métodos */
public function create_sid(): string
}

Índice

SessionIdInterface::create_sid

SessionIdInterface::create_sid

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

SessionIdInterface::create_sid — Cria um ID de sessão.

Descrição

public function **SessionIdInterface::create_sid**(): string

Cria um novo ID de sessão. Esta função é executada automaticamente quando é necessário criar um novo ID de sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O novo ID de sessão. Observe que este valor é retornado internamente para o PHP para processamento.

Veja também

- SessionHandler::create_sid() - Retorna um novo ID de sessão

SessionUpdateTimestampHandlerInterface

(PHP 7, PHP 8)

Introdução

SessionUpdateTimestampHandlerInterface é uma interface que define métodos opcionais para criar um manipulador de sessão personalizado. Para passar um manipulador de sessão personalizado para session_set_save_handler() usando sua invocação OOP, a classe pode implementar esta interface.

Observe que os métodos de retorno de chamada das classes que implementam esta interface são projetados para serem chamados internamente pelo PHP e não devem ser chamados a partir do código do espaço do usuário.

Resumo da classe

interface **SessionUpdateTimestampHandlerInterface** {
/* Métodos */
public function updateTimestamp(string `$id`, string `$data`): bool
public function validateId(string `$id`): bool
}

Índice

SessionUpdateTimestampHandlerInterface::updateTimestamp

SessionUpdateTimestampHandlerInterface::updateTimestamp

(PHP 7, PHP 8)

SessionUpdateTimestampHandlerInterface::updateTimestamp — Atualiza o timestamp

Descrição

public function **SessionUpdateTimestampHandlerInterface::updateTimestamp**(string `$id`, string `$data`): bool

Atualiza o último timestamp de modificação da sessão. Esta função é executada automaticamente quando uma sessão é atualizada.

Parâmetros

`id`

O id da sessão

`data`

Os dados da sessão. Os dados da sessão serializados podem ser usados para determinar se os dados da sessão foram alterados e, portanto, devem ser atualizados.

Valor Retornado

Retorna `true` se o timestamp foi atualizado, `false` caso contrário. Observe que esse valor é retornado internamente para o processamento pelo PHP.

SessionUpdateTimestampHandlerInterface::validateId

SessionUpdateTimestampHandlerInterface::validateId

(PHP 7, PHP 8)

SessionUpdateTimestampHandlerInterface::validateId — Valida o ID

Descrição

public function **SessionUpdateTimestampHandlerInterface::validateId**(string `$id`): bool

Valida um ID de sessão fornecido. Um ID de sessão é válido se já existir uma sessão com esse ID. Essa função é executada automaticamente quando uma sessão está prestes a ser iniciada, um ID de sessão é fornecido e session.use_strict_mode é habilitado.

Parâmetros

`id`

O ID da sessão.

Valor Retornado

Retorna `true` para ID válido, `false` caso contrário. Observe que este valor é retornado internamente ao PHP para processamento.

Extensão Filesystem (Sistema de Arquivos)

Introdução

Nenhuma bilbioteca externa é necessária para compilar esta extensão, porém se for desejado que o PHP suporte LFS (grandes arquivos) no Linux, será necessária uma biblioteca glibc recente e o PHP precisa ser compilado com as seguintes opções: `-D_LARGEFILE_SOURCE -D_FILE_OFFSET_BITS=64`.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do Sistema de Arquivos e de Fluxos

Nome	Padrão	Modificável	Registro de Alterações
allow_url_fopen	"1"	INI_SYSTEM	
allow_url_include	"0"	INI_SYSTEM	Descontinuada a partir do PHP 7.4.0.
user_agent	NULL	INI_ALL	
default_socket_timeout	"60"	INI_ALL	
from	""	INI_ALL	
auto_detect_line_endings	"0"	INI_ALL	Descontinuada a partir do PHP 8.1.0.
sys_temp_dir	""	INI_SYSTEM	

Aqui está uma breve explicação das diretivas de configuração.

`allow_url_fopen` bool

Esta opção habilita os empacotadores fopen sensíveis a URL que permitem acessar objetos URL como se fossem arquivos. Empacotadores padrão são fornecidos para o acesso a arquivos remotos usando o protocolo ftp ou http, e algumas extensões como zlib podem registrar empacotadores adicionais.

`allow_url_include` bool

Esta opção permite o uso de empacotadores fopen sensíveis a URL com as seguintes funções: include, include_once, require e require_once.

> **Nota** :
>
> Esta configuração requer que a opção allow_url_fopen esteja habilitada.

`user_agent` string

Define o agente do usuário que o PHP irá enviar.

`default_socket_timeout` int

Limite padrão de tempo (em segundos) para fluxos baseados em soquete. Especificar um valor negativo significa um limite de tempo infinito.

`from` string

O endereço de e-mail que será usado em conexões FTP não autenticadas e como o valor do cabeçalho From em conexões HTTP, ao usar os empacotadores ftp e http, respectivamente.

`auto_detect_line_endings` bool

Quando habilitada, o PHP examinará os dados lidos por fgets() e file() para verificar se está usando a convenção de fim de linha do Unix, MS-DOS ou Macintosh.

Isso permite que o PHP interopere com sistemas Macintosh, mas o padrão é Off (desligado), pois há uma penalidade de desempenho muito pequena ao detectar as convenções de fim de linha para a primeira linha, e também porque as pessoas que usam retornos de carro como separadores de itens nos sistemas Unix podem experimentar problemas de incompatibilidade com versões anteriores.

`sys_temp_dir` string

Tipos de Recurso

O sistema de arquivos usa fluxos (`streams`) como seu tipo de recurso. Fluxos estão documentados em seu próprio capítulo de referência, Fluxos.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

SEEK_SET (int)

SEEK_CUR (int)

SEEK_END (int)

LOCK_SH (int)

LOCK_EX (int)

LOCK_UN (int)

LOCK_NB (int)

Valores para o parâmetro `flags` da função file()

FILE_USE_INCLUDE_PATH (int)

Procura por `filename` no include_path.

FILE_NO_DEFAULT_CONTEXT (int)

FILE_APPEND (int)

Adiciona conteúdo ao arquivo existente.

FILE_IGNORE_NEW_LINES (int)

Remove caracteres de fim de linha.

FILE_SKIP_EMPTY_LINES (int)

Pula linhas em branco.

FILE_BINARY (int)

Modo binário.

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.1.0.

FILE_TEXT (int)

Modo texto.

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.1.0.

Valores para o parâmetro `flags` da função glob()

`GLOB_AVAILABLE_FLAGS` (int)

Todas as constantes `GLOB_*` combinadas. Equivalente a `0` | `GLOB_BRACE` | `GLOB_MARK` | `GLOB_NOSORT` | `GLOB_NOCHECK` | `GLOB_NOESCAPE` | `GLOB_ERR` | `GLOB_ONLYDIR`

`GLOB_BRACE` (int)

Expande {a,b,c} para corresponder a 'a', 'b', ou 'c'

> **Nota** :
>
> `GLOB_BRACE` não está disponível em alguns sistemas não-GNU, como Solaris ou Alpine Linux.

`GLOB_ERR` (int)

Para em erros de leitura (como diretórios não legíveis), por padrão erros são ignorados.

`GLOB_MARK` (int)

Adiciona uma barra (ou barra invertida no Windows) para cada diretório retornado

`GLOB_NOCHECK` (int)

Retorna a expressão de pesquisa se nenhum arquivo correspondente a ela for encontrado.

`GLOB_NOESCAPE` (int)

Barras invertidas não escapam meta-caracteres.

`GLOB_NOSORT` (int)

Retorna os arquivos conforme aparecem no diretório (sem ordenação). Quando esta opção não é usada, os nomes são ordenados alfabeticamente.

`GLOB_ONLYDIR` (int)

Retorna somente as entradas de diretório que corresponderem à expressão.

Valores para o parâmetro `flags` da função pathinfo()

`PATHINFO_ALL` (int)

Todas as partes da informação do caminho retornadas como um array associativo.

`PATHINFO_DIRNAME` (int)

O caminho do diretório ou arquivo.

`PATHINFO_BASENAME` (int)

O nome do diretório ou o nome e extensão do arquivo.

`PATHINFO_EXTENSION` (int)

A extensão do arquivo.

`PATHINFO_FILENAME` (int)

O nome do arquivo (sem a extensão) ou diretório.

Valores para o parâmetro `scanner_mode` das funções parse_ini_file() e parse_ini_string()

`INI_SCANNER_NORMAL` (int)

Modo de scanner INI normal.

`INI_SCANNER_RAW` (int)

Modo de scanner INI original.

`INI_SCANNER_TYPED` (int)

Modo de scanner INI tipado.

Valores para o parâmetro `flags` da função fnmatch()

`FNM_NOESCAPE` (int)

Desativa o escape de barra invertida.

`FNM_PATHNAME` (int)

A barra na string corresponde apenas à barra no padrão fornecido.

`FNM_PERIOD` (int)

O ponto inicial na string deve corresponder exatamente ao ponto no padrão fornecido.

`FNM_CASEFOLD` (int)

Corresponde sem diferenciar maiúsculas e minúsculas. Parte da extensão GNU.

Constantes de Envio de Arquivo do PHP

`UPLOAD_ERR_CANT_WRITE` (int)

Falha ao gravar o arquivo no disco. O valor da constante é `7`.

`UPLOAD_ERR_EXTENSION` (int)

Uma extesão PHP parou o envio do arquivo. O PHP não fornece um meio de certificar que extensão causou a parada do envio; examinar a lista de extensões carregadas com phpinfo() pode ajudar. O valor da constante é `8`.

`UPLOAD_ERR_FORM_SIZE` (int)

O arquivo enviado excede o tamanho da diretiva *MAX_FILE_SIZE* que foi especificada no formulário HTML. O valor da constante é `2`.

`UPLOAD_ERR_INI_SIZE` (int)

O arquivo enviado excede o tamanho especificado na diretiva upload_max_filesize do *php.ini*. O valor da constante é `1`.

`UPLOAD_ERR_NO_FILE` (int)

Nenhum arquivo foi enviado. O valor da constante é `4`.

`UPLOAD_ERR_NO_TMP_DIR` (int)

Falta um diretório temporário. O valor da constante é `6`.

`UPLOAD_ERR_OK` (int)

Não houve erro, o arquivo foi enviado com sucesso. O valor da constante é `0`.

`UPLOAD_ERR_PARTIAL` (int)

O arquivo foi enviado apenas parcialmente. O valor da constante é `3`.

Funções de Filesystem

Veja também

Para funções relacionadas, consulte também as seções Diretório e Execução de Programas.

Para obter uma lista e a descrição dos vários empacotadors de URL que podem ser usados como arquivos remotos, veja também Protocolos e empacotadores suportados.

Índice

basename

basename

(PHP 4, PHP 5, PHP 7, PHP 8)

basename — Retorna a última parte do caminho

Descrição

function **basename**(string `$path`, string `$suffix = ""`): string

Dada uma string contendo o caminho para um arquivo ou diretório, esta função retornará a última parte do caminho.

> **Nota** :
>
> A função **basename()** opera de forma ingênua na string fornecida e não tem conhecimento do sistema de arquivos real ou dos componentes de caminho como "`..`".

> **Cuidado**
>
> A função **basename()** reconhece a localidade, portanto, para obter o nome correto ao usar caminhos com caracteres multibyte, a localidade correspondente deve ser definida com a função setlocale(). Se o parâmetro `path` contiver caracteres inválidos para a localidade atual, o comportamento de **basename()** será indefinido.

Parâmetros

`path`

Um caminho.

No Windows, tanto a barra (`/`) quanto a barra invertida (`\`) são usadas como caractere separador de diretório. Em outros ambientes, apenas a barra (`/`) é usada.

`suffix`

Se a última parte do caminho terminar em `suffix`, ele será removido.

Valor Retornado

Retorna a última parte de um dado `path`.

Examples

Exemplo #1 Exemplo de basename()

```php
<?php
echo "1) ".basename("/etc/sudoers.d", ".d").PHP_EOL;
echo "2) ".basename("/etc/sudoers.d").PHP_EOL;
echo "3) ".basename("/etc/passwd").PHP_EOL;
echo "4) ".basename("/etc/").PHP_EOL;
echo "5) ".basename(".").PHP_EOL;
echo "6) ".basename("/");
?>
```

O exemplo acima produzirá:

```
1) sudoers
2) sudoers.d
3) passwd
4) etc
5) .
6)
```

Veja também

- dirname() - Retorna o caminho para o diretório pai
- pathinfo() - Retorna informações sobre um caminho de arquivo

chgrp

chgrp

(PHP 4, PHP 5, PHP 7, PHP 8)

chgrp — Altera o grupo do arquivo

Descrição

function **chgrp**(string `$filename`, string|int `$group`): bool

Tenta alterar o grupo do arquivo `filename` para `group`.

Somente o superusuário pode alterar arbitrariamente o grupo de um arquivo; outros usuários podem alterar o grupo de um arquivo para qualquer grupo do qual sejam membros.

Parâmetros

`filename`

Caminho para o arquivo.

`group`

O nome ou número do grupo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Alterando o grupo de um arquivo

```php
<?php
$arquivo = 'arquivo_compartilhado.txt';
$formato = "ID do grupo de %s em %s: %d\n";
printf($formato, $arquivo, date('r'), filegroup($arquivo));
chgrp($arquivo, 8);
clearstatcache(); // não armazena em cache os resultados de filegroup()
printf($formato, $arquivo, date('r'), filegroup($arquivo));
?>
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :
>
> No Windows, esta função falha silenciosamente quando aplicada em um arquivo normal.

Veja também

- chown() - Modifica o proprietário do arquivo
- chmod() - Modifica as permissões do arquivo

chmod

chmod

(PHP 4, PHP 5, PHP 7, PHP 8)

chmod — Modifica as permissões do arquivo

Descrição

function **chmod**(string `$filename`, int `$permissions`): bool

Tenta mudar as permissões do arquivo especificado para o especificado em `permissions`.

Parâmetros

`filename`

O caminho para o arquivo.

`permissions`

Note que `permissions` não é automaticamente assumido como valor em octal, então para assegurar a operação esperada, é necessário prefixar `permissions` com um zero (0). Strings como "g+w" não irão funcionar adequadamente.

```php
<?php
chmod ("/arquivo/diretorio", 755);   // decimal; provavelmente incorreto
chmod ("/arquivo/diretorio", "u+rwx,go+rx"); // string; incorreto
chmod ("/arquivo/diretorio", 0755);  // octal; representação correta do modo
?>
```

O parâmetro `permissions` consiste de três componentes numéricos em octal especificando restrições de acesso para o proprietário, para o grupo de usuário onde está o proprietário e para todos os outros usuários, nesta ordem. Um componente pode ser calculado adicionando-se as permissões necessárias para o usuário alvo. O número 1 significa direito de execução, 2 significa direito de escrita, 4 significa direito de leitura. Some esses números para especificar os direitos desejados. Pode-se consultar mais sobre modos em sistemas Unix com os comandos '**man 1 chmod**' e '**man 2 chmod**'.

```php
<?php
// Escrita e leitura para o proprietário, nada mais para ninguém
chmod ("/somedir/somefile", 0600);

// Escrita e leitura para o proprietário, leitura para todos os outros
chmod ("/somedir/somefile", 0644);

// Tudo para o proprietário, leitura e execução para os outros
chmod ("/somedir/somefile", 0755);

// Tudo para o proprietário, leitura e execução para o grupo do prop
chmod ("/somedir/somefile", 0750);
?>
```

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Em caso de falha, um aviso **`E_WARNING`** é emitido.

Notas

> **Nota** :
>
> O usuário atual é aquele sob o qual o PHP é executado. Provavelmente não é o mesmo usuário que acessa linha de comando ou FTP. O modo pode ser modificado apenas pelo usuário proprietário do arquivo na maioria dos sistemas.

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

Veja também

- chown() - Modifica o proprietário do arquivo

- chgrp() - Altera o grupo do arquivo
- fileperms() - Obtém permissões de arquivo
- stat() - Obtém informações sobre um arquivo

chown

chown

(PHP 4, PHP 5, PHP 7, PHP 8)

chown — Modifica o proprietário do arquivo

Descrição

function **chown**(string `$filename`, string|int `$user`): bool

Tenta modificar o proprietário do `filename` para `user`. Somente o superusuário pode modificar o proprietário de um arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

`user`

O nome do usuário ou número.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Uso simples de chown()

```php
<?php

// Nome de usuário e do arquivo que serão usados
$file_name= "foo.php";
$path = "/home/sites/php.net/public_html/sandbox/" . $file_name ;
$user_name = "root";

// Define o usuário proprietário
chown($path, $user_name);

// Verifica o resultado
$stat = stat($path);
print_r(posix_getpwuid($stat['uid']));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [name] => root
    [passwd] => x
    [uid] => 0
    [gid] => 0
    [gecos] => root
    [dir] => /root
    [shell] => /bin/bash
)
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :
>
> No Windows, esta função falha silenciosamente quando aplicada a um arquivo normal.

Veja também

- chmod() - Modifica as permissões do arquivo
- chgrp() - Altera o grupo do arquivo

clearstatcache

clearstatcache

(PHP 4, PHP 5, PHP 7, PHP 8)

clearstatcache — Limpa o cache de estado de arquivos

Descrição

function **clearstatcache**(bool `$clear_realpath_cache = false`, string `$filename = ""`): void

Quando são chamadas as funções stat(), lstat() ou qualquer uma das funções afetadas (listadas abaixo), o PHP mantém em cache as informações que essas funções retornam para fornecer desempenho mais rápido. Entretanto, em certos casos pode ser necessário limpar as informações em cache. Por exemplo, se um mesmo arquivo é verificado várias vezes em um único script, e esse arquivo corre o risco de ser apagado ou modificado durante a operação do script, pode ser necessário limpar os dados do cache. Nesses casos, pode-se utilizar a função **clearstatcache()** para limpar as informações que o PHP mantém em cache sobre um arquivo.

Deve-se notar também que o PHP não guarda informação em cache sobre arquivos não existente. Assim, a função file_exists() for chamada em um arquivo que não existe, ela retornará `false` até que o arquivo seja criado. Se o arquivo for criado, ela retornará `true` mesmo que o arquivo seja removido. Entretanto, a função unlink() limpa o cache automaticamente.

> **Nota** :
>
> Esta função guarda informações em cache sobre nomes de arquivos específicos, de forma que somente será necessário chamar **clearstatcache()** se estão sendo realizadas várias operações sobre o mesmo nome de arquivo e seja necessário que a informação sobre esse arquivo em particular não seja armazenada em cache.

As funções afetadas incluem stat(), lstat(), file_exists(), is_writable(), is_readable(), is_executable(), is_file(), is_dir(), is_link(), filectime(), fileatime(), filemtime(), fileinode(), filegroup(), fileowner(), filesize(), filetype(), e fileperms().

Parâmetros

`clear_realpath_cache`

Se deve-se *também* limpar o cache de caminho real.

`filename`

Limpa o cache de caminho real para um arquivo específico somente; apenas usado se `clear_realpath_cache` for `true`.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de clearstatcache()

```php
<?php
$file = 'output_log.txt';

function get_owner($file)
{
    $stat = stat($file);
    $user = posix_getpwuid($stat['uid']);
    return $user['name'];
}

$format = "UID @ %s: %s\n";

printf($format, date('r'), get_owner($file));

chown($file, 'ross');
printf($format, date('r'), get_owner($file));

clearstatcache();
printf($format, date('r'), get_owner($file));
?>
```

O exemplo acima produzirá algo semelhante a:

```
UID @ Sun, 12 Oct 2008 20:48:28 +0100: root
UID @ Sun, 12 Oct 2008 20:48:28 +0100: root
UID @ Sun, 12 Oct 2008 20:48:28 +0100: ross
```

copy

copy

(PHP 4, PHP 5, PHP 7, PHP 8)

copy — Copia arquivo

Descrição

function **copy**(string `$from`, string `$to`, ?resource `$context = null`): bool

Faz uma cópia do arquivo informado em `from` para o arquivo informado em `to`.

Se a intenção for mover um arquivo, utilize a função rename().

Parâmetros

`from`

Caminho para o arquivo de origem.

`to`

O caminho de destino. Se o parâmetro `to` for uma URL, a operação de cópia pode falhar se o empacotador não suportar a sobrescrita de arquivos existentes.

> **Aviso**
>
> Se o arquivo de destino já existir, ele será sobrescrito.

`context`

Um recurso de contexto válido criado com stream_context_create().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de copy()

```php
<?php
$file = 'example.txt';
$newfile = 'example.txt.bak';

if (!copy($file, $newfile)) {
    echo "falha ao copiar $file...\n";
}
?>
```

Veja também

- move_uploaded_file() - Move um arquivo enviado para uma nova localização
- rename() - Renomeia um arquivo ou diretório
- A seção do manual sobre Gerenciamento de envio de arquivos

delete

delete

delete — Consulte unlink() ou unset()

Descrição

Não existe função ou instrução "delete" no PHP. Se você chegou nesta página procurando por deletar arquivos, consulte unlink(). Para excluir uma variável de um escopo local, consulte unset().

Veja também

- unlink() - Apaga um arquivo
- unset() - unset remove a definição de uma variável informada

dirname

dirname

(PHP 4, PHP 5, PHP 7, PHP 8)

dirname — Retorna o caminho para o diretório pai

Descrição

function **dirname**(string `$path`, int `$levels` = 1): string

Dada uma string contendo um caminho para um arquivo ou diretório, esta função irá retornar o caminho do diretório pai que é `levels` acima do diretório atual.

> **Nota** :
>
> **dirname()** opera ingenuamente sobre a string de entrada, e não tem conhecimento do sistema de arquivos real, ou componentes de caminho de diretório como "`..`".

> **Cuidado**
>
> No Windows, **dirname()** assume o código de página atual, portanto para que possa ver os nomes corretos de diretório com caracteres multi-byte, o código de página correspondente deve ser configurado. Se `path` contiver caracteres que são inválidos para o código de página ativo, o comportamento de **dirname()** será indefinido.
>
> Em outros sistemas, **dirname()** assume que `path` estará codificado em formato compatível com ASCII. Caso contrário o comportamento da função será indefinido.

Parâmetros

`path`

Um caminho.

No Windows, tanto a barra (`/`) quanto a barra invertida (`\`) são usadas como caractere separador de diretório. Em outros ambientes, é utilizada a barra (`/`).

`levels`

O número de diretórios pai para subir.

Deve ser um inteiro maior que 0.

Valor Retornado

Retorna o caminho do diretório pai. Se não houver nenhuma barra no `path`, um ponto ('`.`') é retornado, indicando o diretório atual. Do contrário, a string retornada é o `path` com qualquer `/componente` ao final da string removido.

> **Cuidado**
>
> Deve-se tomar cuidado ao usar esta função em um loop que alcança o diretório raiz pois isto pode resultar em um loop infinito.
>
> ```
> <?php
> dirname('.');    // Retornará '.'.
> dirname('/');    // Retornará `\` no Windows e '/' em sistemas *nix.
> dirname('\\');   // Retornará `\` no Windows e '.' em sistemas *nix.
> dirname('C:\\'); // Retornará 'C:\' no Windows e '.' em sistemas *nix.
> ?>
> ```

Examples

Exemplo #1 Exemplo de dirname()

```
<?php
 echo dirname("/etc/passwd") . PHP_EOL;
 echo dirname("/etc/") . PHP_EOL;
 echo dirname(".") . PHP_EOL;
 echo dirname("C:\\") . PHP_EOL;
 echo dirname("/usr/local/lib", 2);
```

O exemplo acima produzirá algo semelhante a:

```
/etc
 / (ou \ no Windows)
 .
 C:\
 /usr
```

Veja também

- basename() - Retorna a última parte do caminho
- pathinfo() - Retorna informações sobre um caminho de arquivo
- realpath() - Retorna o path absoluto canonizado

disk_free_space

disk_free_space

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

disk_free_space — Retorna o espaço disponível no sistema de arquivos ou partição de disco

Descrição

function **disk_free_space**(string `$directory`): float|false

Dado uma string contendo um diretório, esta função retornará o número de bytes disponíveis no sistema de arquivos ou partição de disco.

Parâmetros

`directory`

Um diretório do sistema de arquivos ou partição de disco.

> **Nota** :
>
> Dado um nome de arquivo ao invés de um diretório, o comportamento da função não é especificado e pode ser diferente entre sistemas operacionais e versões do PHP.

Valor Retornado

Retorna o número de bytes disponíveis como um float ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de disk_free_space()

```php
<?php
// $df contém o número de bytes disponíveis em "/"
$df = disk_free_space("/");

// No Windows:
$df_c = disk_free_space("C:");
$df_d = disk_free_space("D:");
?>
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

Veja também

- disk_total_space() - Retorna o tamanho total de um sistema de arquivos ou partição de disco

disk_total_space

disk_total_space

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

disk_total_space — Retorna o tamanho total de um sistema de arquivos ou partição de disco

Descrição

function **disk_total_space**(string `$directory`): float|false

Dado uma string contendo um diretório, esta função retornará o número total de bytes do sistema de arquivos ou da partição de disco.

Parâmetros

`directory`

Um diretório do sistema de arquivos ou uma partição de disco.

Valor Retornado

Retorna o número total de bytes como um float ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de disk_total_space()

```php
<?php
// $ds contém o numero total de bytes disponíveis em "/"
$ds = disk_total_space("/");

// No Windows:
disk_total_space("C:");
disk_total_space("D:");
?>
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

Veja também

- disk_free_space() - Retorna o espaço disponível no sistema de arquivos ou partição de disco

diskfreespace

diskfreespace

(PHP 4, PHP 5, PHP 7, PHP 8)

diskfreespace — Sinônimo de disk_free_space()

Descrição

Esta função é um apelido para: disk_free_space().

fclose

fclose

(PHP 4, PHP 5, PHP 7, PHP 8)

fclose — Fecha um ponteiro de arquivo aberto

Descrição

function **fclose**(resource `$stream`): bool

O arquivo apontado por `stream` é fechado.

Parâmetros

`stream`

O ponteiro para o arquivo tem que ser válido e tem que apontar para um arquivo aberto por fopen() ou fsockopen().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de fclose()

```php
<?php

  $handle = fopen('qualquerarquivo.txt', 'r');
```

```
  fclose($handle);

?>
```

Veja também

- fopen() - Abre um arquivo ou URL
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet

fdatasync

fdatasync

(PHP 8 >= 8.1.0)

fdatasync — Sincroniza dados (mas não metadados) ao arquivo

Descrição

function **fdatasync**(resource `$stream`): bool

Esta função sincroniza o conteúdo do fluxo definido em `stream` para a mídia de armazenamento, da mesma forma que fsync(), mas não sincroniza os metadados de arquivo. Note que esta função só é efetivamente diferente em sistemas POSIX. No Windows, esta função é apelidada de fsync().

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de fdatasync()

```
<?php

$file = 'test.txt';

$stream = fopen($file, 'w');
fwrite($stream, 'test data');
fwrite($stream, "\r\n");
fwrite($stream, 'dados adicionais');

fdatasync($stream);
fclose($stream);
?>
```

Veja também

- fflush() - Força a liberação do buffer para um arquivo
- fsync() - Sincroniza mudanças ao arquivo (incluindo metadados)

feof

feof

(PHP 4, PHP 5, PHP 7, PHP 8)

feof — Testa pelo fim-de-arquivo em um ponteiro de arquivo

Descrição

function **feof**(resource `$stream`): bool

Testa pelo fim-de-arquivo em um ponteiro de arquivo.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna `true` se o ponteiro estiver no fim do arquivo ou um erro ocorrer (incluindo um limite de tempo de socket); caso contrário retorna `false`.

Notas

> **Aviso**
>
> Se uma conexão aberta por fsockopen() não foi fechada pelo servidor, **feof()** irá travar. Para contornar isto, veja o exemplo abaixo:
>
> **Exemplo #1 Manipulando limites de tempo com feof()**

```php
<?php
function safe_feof($fp, &$start = NULL) {
 $start = microtime(true);

 return feof($fp);
}

/* Assumindo que $fp foi previamente aberto por fsockopen() */

$start = NULL;
$timeout = ini_get('default_socket_timeout');

while(!safe_feof($fp, $start) && (microtime(true) - $start) < $timeout)
{
 /* Manipulação */
}
?>
```

> **Aviso**
>
> Se o ponteiro de arquivo passado não for válido pode-se chegar a um loop infinito, porque **feof()** falha em retornar `true`.
>
> **Exemplo #2 Exemplo de feof() com um ponteiro de arquivo inválido**

```php
<?php
// se o arquivo não puder ser lido ou não existir a função fopen retorna FALSE
$file = @fopen("no_such_file", "r");

// FALSE a partir do fopen irá emitir um aviso (warning) e resultar em um loop infinito aqui
while (!feof($file)) {
}

fclose($file);
?>
```

fflush

fflush

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

fflush — Força a liberação do buffer para um arquivo

Descrição

function **fflush**(resource `$stream`): bool

Esta função força a escrita de toda saída cacheada para o recurso apontado fluxo de arquivo `stream`.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de escrita em arquivo usando fflush()

```php
<?php
$filename = 'bar.txt';

$file = fopen($filename, 'r+');
rewind($file);
fwrite($file, 'Foo');
fflush($file);
ftruncate($file, ftell($file));
fclose($file);
?>
```

Veja também

- clearstatcache() - Limpa o cache de estado de arquivos
- fwrite() - Escrita binary-safe em arquivos

fgetc

fgetc

(PHP 4, PHP 5, PHP 7, PHP 8)

fgetc — Lê um caractere do ponteiro de arquivo

Descrição

function **fgetc**(resource `$stream`): string|false

Obtém um caractere de um dado ponteiro de arquivo.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna uma string contendo um único caractere lido do ponteiro do arquivo passado por `stream`. Retorna `false` em EOF.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Examples

Exemplo #1 Um exemplo da fgetc()

```php
<?php
$fp = fopen('algumarquivo.txt', 'r');
if (!$fp) {
    echo 'Não é possivel abrir algumarquivo.txt';
```

```
}
while (false !== ($char = fgetc($fp))) {
    echo "$char\n";
}
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

Veja também

- fread() - Leitura de arquivo segura para binário
- fopen() - Abre um arquivo ou URL
- popen() - Abre um processo como ponteiro de arquivo
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- fgets() - Lê uma linha de um ponteiro de arquivo

fgetcsv

fgetcsv

(PHP 4, PHP 5, PHP 7, PHP 8)

fgetcsv — Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV

Descrição

```
function fgetcsv(
    resource $stream,
    ?int $length = null,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\"
): array|false
```

Similar à função fgets(), exceto que a **fgetcsv()** interpreta a linha em busca de campos no formato CSV e retorna um array contendo os campos lidos.

> **Nota** :
>
> As configurações de localidade são consideradas por esta função. Por exemplo, dados em certas codificações de um byte podem ser interpretados incorretamente se `LC_CTYPE` for `en_US.UTF-8`.

Parâmetros

`stream`

Um ponteiro de arquivo válido para um arquivo aberto com sucesso por fopen(), popen() ou fsockopen().

`length`

Deve ser maior do que a maior linha (em caracteres) a ser encontrada no arquivo CSV (considerando também os caracteres de fim-de-linha). Caso contrário a linha é dividida em partes de `length` caracteres, a não ser que a divisão ocorra dentro de um caractere que cerca os campos (enclosure).

Se este parâmetro for omitido (ou definido em 0 ou `null` a partir do PHP 8.0.0), o tamanho máximo das linhas não é limitado, o que é um pouco mais lento.

`separator`

O parâmetro `separator` define o delimitador de campo. Precisa ser um caractere de um byte.

`enclosure`

O parâmetro `enclosure` define o caractere que cerca um campo. Precisa ser um caractere de um byte.

`escape`

O parâmetro `escape` define o caractere de escape. Precisa ser um caractere de um byte ou uma string vazia. Uma string vazia (`""`) desabilita o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado. Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados.

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

Valor Retornado

Retorna um array indexado contendo os campos lidos em caso de sucesso, ou **`false`** em caso de falha.

> **Nota** :
>
> Uma linha em branco em um arquivo CSV será retornada como um array contendo um único campo nulo (null) e não será tratada como um erro.

> **Nota** :
>
> Se o PHP não estiver reconhecendo corretamente os finais de linha ao ler arquivos em um computador Macintosh ou que foram criados em um, habilitar a opção de configuração em tempo de execução auto_detect_line_endings pode ajudar a resolver o problema.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
8.3.0	Uma string vazia é retornada ao invés de uma string com um único byte nulo para o último campo, se ele contiver apenas um demarcador não terminado.
8.0.0	O parâmetro `length` agora pode ser nulo.
7.4.0	O parâmetro `escape` agora aceita uma string vazia para desabilitar o mecanismo de escape proprietário.

Examples

Exemplo #1 Lê e imprime todo o conteúdo de um arquivo CSV

```php
<?php

if (($handle = fopen("test.csv", "r")) !== false) {
    $row = 1;

    while (($fields = fgetcsv($handle, 1000, ",", "\"", "\\")) !== false) {
        $num = count($fields);

        echo "\n<p>$num campos na linha $row:</p>\n<ol>";

        for ($c = 0; $c < $num; $c++) {
            echo "\n\t<li>", $fields[$c], "</li>";
        }

        echo "\n</ol>";

        $row++;
    }

    fclose($handle);
}
```

Veja também

- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array
- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV
- explode() - Divide uma string com base em outra string
- file() - Lê todo o arquivo para um array
- pack() - Empacota dados em uma string binária

fgets

fgets

(PHP 4, PHP 5, PHP 7, PHP 8)

fgets — Lê uma linha de um ponteiro de arquivo

Descrição

function **fgets**(resource `$stream`, ?int `$length = null`): string|false

Retorna uma linha do ponteiro do arquivo.

Parâmetros

`stream`

> O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

`length`

> A leitura termina quando `length` - 1 bytes tenham sido lidos, ou uma quebra de linha (que é incluída no retorno), ou no final do arquivo (EOF), o que acontecer primeiro. Se nenhum comprimento for especificado, a leitura do fluxo continuará até chegar ao final da linha.

Valor Retornado

Retorna uma string de até `length` - 1 bytes lida do arquivo apontado por `stream`. Se não tiver mais dados para ler no ponteiro do arquivo, então retorna `false`.

Se um erro ocorrer, é retornado `false`.

Examples

Exemplo #1 Lendo um arquivo linha por linha

```php
<?php

$fp = @fopen("/tmp/arquivodeentrada.txt", "r");

if ($fp) {
    while (($buffer = fgets($fp, 4096)) !== false) {
        echo $buffer, PHP_EOL;
    }

    if (!feof($fp)) {
        echo "Erro: falha inexperada de fgets()\n";
    }

    fclose($fp);
}

?>
```

Notas

> **Nota** :
>
> Se o PHP não estiver reconhecendo corretamente os finais de linha ao ler arquivos em um computador Macintosh ou que foram criados em um, habilitar a opção de configuração em tempo de execução auto_detect_line_endings pode ajudar a resolver o problema.

> **Nota** :
>
> Pessoas acostumadas à semântica do **fgets()** em 'C' devem notar a diferença em como o fim do arquivo (EOF) é retornado.

Veja também

- fgetss() - Ler uma linha de um ponteiro de arquivo e retira as tags HTML
- fread() - Leitura de arquivo segura para binário
- fgetc() - Lê um caractere do ponteiro de arquivo
- stream_get_line() - Obtém uma linha do recurso do fluxo até um delimitador informado
- fopen() - Abre um arquivo ou URL
- popen() - Abre um processo como ponteiro de arquivo
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- stream_set_timeout() - Define o limite de tempo em um fluxo

fgetss

fgetss

(PHP 4, PHP 5, PHP 7)

fgetss — Ler uma linha de um ponteiro de arquivo e retira as tags HTML

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.3.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **fgetss**(resource `$handle`, int `$length = ?`, string `$allowable_tags = ?`): string

Idêntico a fgets(), exceto que **fgetss()** tenta retirar qualquer byte NUL, tag HTML ou PHP do texto que ela lê. A função retém o estado de interpretação de chamada para chamada, e desta forma não É equivalente a chamar strip_tags() no valor de retorno de fgets().

Parâmetros

`handle`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

`length`

Tamanho da informação a ser recebida.

`allowable_tags`

Pode-se usar o terceiro parâmetro opcional para especificar as tags que não devem ser removidas. Consulte strip_tags() para detalhes sobre `allowable_tags`.

Valor Retornado

Retorna uma string de até `length` - 1 bytes lidos do arquivo apontado por `handle`, com todo o código HTML e PHP removidos.

Se um erro ocorrer, retorna `false`.

Examples

Exemplo #1 Lendo um arquivo PHP linha por linha

```
<?php
$str = <<<EOD
<html><body>
 <p>Bem-vindo! Hoje é <?php echo(date('jS')); ?> de <?= date('F'); ?>.</p>
</body></html>
Texto fora do bloco HTML.
EOD;
file_put_contents('sample.php', $str);

$handle = @fopen("sample.php", "r");
if ($handle) {
    while (!feof($handle)) {
        $buffer = fgetss($handle, 4096);
        echo $buffer;
    }
    fclose($handle);
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Bem-vindo! Hoje é  de .

Texto fora do bloco HTML.
```

Notas

> **Nota** :
>
> Se o PHP não estiver reconhecendo corretamente os finais de linha ao ler arquivos em um computador Macintosh ou que foram criados em um, habilitar a opção de configuração em tempo de execução auto_detect_line_endings pode ajudar a resolver o problema.

Veja também

- fgets() - Lê uma linha de um ponteiro de arquivo
- fopen() - Abre um arquivo ou URL
- popen() - Abre um processo como ponteiro de arquivo

- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- strip_tags() - Retira as tags HTML e PHP de uma string
- SplFileObject::fgetss() - Obtém linha do arquivo e remove tags HTML
- O filtro string.strip_tags

file

file

(PHP 4, PHP 5, PHP 7, PHP 8)

file — Lê todo o arquivo para um array

Descrição

function **file**(string `$filename`, int `$flags` = 0, ?resource `$context` = `null`): array|false

Carrega todo um arquivo em um array.

> **Nota** :
>
> Pode-se utilizar file_get_contents() para retornar o conteúdo de um arquivo como uma string.

Parâmetros

`filename`

Caminho para o arquivo.

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

`flags`

O parâmetro opcional `flags` pode ser uma ou mais das seguintes constantes:

`FILE_USE_INCLUDE_PATH`

Procurar pelo arquivo no include_path.

`FILE_IGNORE_NEW_LINES`

Omite a quebra de linha no final de cada elemento do array.

`FILE_SKIP_EMPTY_LINES`

Pula linhas vazias.

`FILE_NO_DEFAULT_CONTEXT`

Não usa o contexto padrão.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna o arquivo em um array. Cada elemento do array corresponde a uma linha no arquivo, ainda com a quebra de linha. Em caso de falha, **file()** retorna `false`

> **Nota** :

> Cada linha no array incluirá a quebra de linha, a não ser que `FILE_IGNORE_NEW_LINES` seja usado.

> **Nota** :
>
> Se o PHP não estiver reconhecendo corretamente os finais de linha ao ler arquivos em um computador Macintosh ou que foram criados em um, habilitar a opção de configuração em tempo de execução auto_detect_line_endings pode ajudar a resolver o problema.

Erros/Exceções

A partir do PHP 8.3.0, lança uma exceção ValueError se `flags` incluir qualquer valor inválido, tal como `FILE_APPEND`.

Emite um aviso `E_WARNING` se o arquivo não existe.

Registro de Alterações

Versão	Descrição
8.3.0	Uma exceção ValueError é lançada para qualquer valor inválido no parâmetro `flags`.

Examples

Exemplo #1 Exemplo de file()

```php
<?php
// Lê um arquivo em um array.  Nesse exemplo será obtido o código fonte HTML de
// uma URL via HTTP
$lines = file ('http://www.example.com/');

// Percorre o array, mostrando o fonte HTML com numeração de linhas.
foreach ($lines as $line_num => $line) {
    echo "Linha #<b>{$line_num}</b> : " . htmlspecialchars($line) . "<br />\n";
}

// Usando o parâmetro de opções opcionais
$trimmed = file('somefile.txt', FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
?>
```

Notas

> **Aviso**
>
> Ao usar SSL, o Microsoft IIS violará o protocolo fechando a conexão sem enviar uma notificação `close_notify`. O PHP reportará isso como "SSL: Fatal Protocol Error" quando chegar ao final dos dados. Para contornar isso, o valor de error_reporting deve ser reduzido a um nível que não inclua avisos. O PHP pode detectar servidores IIS defeituosos quando o fluxo é aberto usando o empacotador `https://` e suprimirá o aviso. Ao usar fsockopen() para criar um soquete `ssl://`, o desenvolvedor é responsável por detectar e suprimir este aviso.

Veja também

- file_get_contents() - Lê todo o conteúdo de um arquivo para uma string
- readfile() - Exibe o conteúdo de um arquivo
- fopen() - Abre um arquivo ou URL
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- popen() - Abre um processo como ponteiro de arquivo
- include - include
- stream_context_create() - Cria um contexto de fluxo

file_exists

file_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

file_exists — Verifica se um arquivo ou diretório existe

Descrição

function **file_exists**(string `$filename`): bool

Verifica se um arquivo ou diretório existe.

Parâmetros

`filename`

Caminho para o arquivo ou diretório.

No Windows, use *//computername/share/filename* ou *\\computername\share\filename* para checar arquivos em compartilhamentos de rede.

Valor Retornado

Retorna **`true`** se o arquivo ou diretório especificado por `filename` existe; **`false`** caso contrário.

> **Nota** :
>
> Esta função retorna **`false`** para ligações simbólicas apontando para arquivos inexistentes.

> **Nota** :
>
> O teste é realizado utilizando o UID/GID real, em vez de outro valor efetivo.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Erros/Exceções

Em caso de falha, um **`E_WARNING`** será emitido.

Examples

Exemplo #1 Testando se um arquivo existe

```php
<?php
$filename = '/caminho/para/arquivo.txt';

if (file_exists($filename)) {
    echo "O arquivo $filename existe";
} else {
    echo "O arquivo $filename não existe";
}
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- is_readable() - Informa se um arquivo existe e se pode ser lido
- is_writable() - Informa se o arquivo pode ser escrito
- is_file() - Informa se o arquivo é um arquivo comum
- file() - Lê todo o arquivo para um array
- SplFileInfo

file_get_contents

file_get_contents

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

file_get_contents — Lê todo o conteúdo de um arquivo para uma string

Descrição

```
function file_get_contents(
    string $filename,
    bool $use_include_path = false,
    ?resource $context = null,
    int $offset = 0,
    ?int $length = null
): string|false
```

Esta função é semelhante à file(), exceto que **file_get_contents()** retorna o arquivo em uma string, começando a partir de `offset` até `length` bytes. Em caso de falha, **file_get_contents()** retornará `false`.

file_get_contents() é o método preferível para ler o conteúdo de um arquivo em uma string. Ela usa técnicas de mapeamento de memória suportadas pelo sistema operacional para melhorar o desempenho.

> **Nota** :
>
> Se for aberta uma URI com caracteres especiais, como espaços, é necessário codificar a URI com urlencode().

Parâmetros

`filename`

Nome do arquivo para ler.

`use_include_path`

> **Nota** :
>
> A constante `FILE_USE_INCLUDE_PATH` pode ser usada para acionar a busca no include path. Isto não é possível se a tipagem estrita estiver habilitada, já que `FILE_USE_INCLUDE_PATH` é um int. Em vez disso, use `true`.

`context`

Um recurso de contexto válido, criado com stream_context_create(). Se você não precisa usar um contexto personalizado, você pode ignorar este parâmetro passando `null`.

`offset`

O ponto onde a leitura deve começar no fluxo original. Offsets negativos contam a partir do final do fluxo.

`offset` não é suportado em arquivos remotos. Tentar fazer busca em arquivos não locais pode funcionar com offsets pequenos, mas isso é imprevisível pois só funciona no fluxo com buffer.

`length`

Comprimento máximo dos dados lidos. O padrão é ler até que o fim do arquivo seja alcançado. Note que este parâmetro é aplicado para o fluxo processado pelos filtros.

Valor Retornado

A função retorna os dados lidos ou `false` em caso de falha.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

Um erro de nível `E_WARNING` é gerado se `filename` não puder ser encontrado, `length` é menor que zero, ou se a busca no fluxo pelo `offset` especificado falhar.

Quando **file_get_contents()** é chamada em um diretório, um erro nível `E_WARNING` é gerado no Windows, e a partir do PHP 7.4 em outros sistemas operacionais também.

Registro de Alterações

Versão	Descrição
8.0.0	`length` agora é anulável.
7.1.0	Suporte para `offsets` negativos foi adicionado.

Examples

Exemplo #1 Obtém e exibe o código-fonte da página inicial de um website

```php
<?php
$homepage = file_get_contents('http://www.example.com/');
echo $homepage;
?>
```

Exemplo #2 Buscando no include_path

```php
<?php
// Se strict types estiver ativado com declare(strict_types=1);
$file = file_get_contents('./people.txt', true);
// Se não
$file = file_get_contents('./people.txt', FILE_USE_INCLUDE_PATH);
?>
```

Exemplo #3 Lendo uma sessão de um arquivo

```php
<?php
// Lê 14 letras começando da letra 21
$section = file_get_contents('./people.txt', FALSE, NULL, 20, 14);
var_dump($section);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(14) "lle Bjori Ro"
```

Exemplo #4 Usando contextos de fluxo

```php
<?php
// Criam um fluxo
$opts = [
    'http' => [
        'method' => "GET",
        'header' => "Accept-language: en\r\n" .
                "Cookie: foo=bar\r\n"
```

```
    ]
];

$context = stream_context_create($opts);

// Abre o arquivo utilizando os headers HTTP craidos acima
$file = file_get_contents('http://www.example.com/', false, $context);
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

> **Aviso**
>
> Ao usar SSL, o Microsoft IIS violará o protocolo fechando a conexão sem enviar uma notificação `close_notify`. O PHP reportará isso como "SSL: Fatal Protocol Error" quando chegar ao final dos dados. Para contornar isso, o valor de error_reporting deve ser reduzido a um nível que não inclua avisos. O PHP pode detectar servidores IIS defeituosos quando o fluxo é aberto usando o empacotador `https://` e suprimirá o aviso. Ao usar fsockopen() para criar um soquete `ssl://`, o desenvolvedor é responsável por detectar e suprimir este aviso.

Veja também

- file() - Lê todo o arquivo para um array
- fgets() - Lê uma linha de um ponteiro de arquivo
- fread() - Leitura de arquivo segura para binário
- readfile() - Exibe o conteúdo de um arquivo
- file_put_contents() - Escreve dados em um arquivo
- stream_get_contents() - Lê o restante de um fluxo em uma string
- stream_context_create() - Cria um contexto de fluxo
- $http_response_header

file_put_contents

file_put_contents

(PHP 5, PHP 7, PHP 8)

file_put_contents — Escreve dados em um arquivo

Descrição

```
function file_put_contents(
    string $filename,
    mixed $data,
    int $flags = 0,
    ?resource $context = null
): int|false
```

Esta função é idêntica a chamar fopen(), fwrite() e fclose() sucessivamente para escrever dados em um arquivo.

Se o arquivo informado em `filename` não existir, o arquivo será criado. Do contrário, o arquivo existente será sobrescrito, a não ser que a opção `FILE_APPEND` esteja definida.

Parâmetros

`filename`

O caminho para o arquivo onde os dados serão escritos.

`data`

Os dados a serem escritos. Pode ser uma string, um array ou um recurso stream.

Se `data` for um recurso stream, o buffer restante desse fluxo será copiado para o arquivo especificado, semelhante à utilização de stream_copy_to_stream().

Também pode ser especificado o parâmetro `data` como um array unidimensional. Isso é equivalente a `file_put_contents($filename, implode('', $array))`.

`flags`

O valor de `flags` pode ser qualquer combinação das seguintes opções, adicionadas com o operador binário OR (`|`).

Opções disponíveis

Opção	Descrição
`FILE_USE_INCLUDE_PATH`	Procura o arquivo `filename` no diretório de 'include'. Consulte include_path para mais informações.
`FILE_APPEND`	Se o arquivo `filename` já existir, acrescenta os dados ao final do arquivo em vez de sobrescrevê-lo.
`LOCK_EX`	Adquire uma trava exclusiva no arquivo ao prosseguir para a escrita. Em outras palavras, uma chamada a flock() acontece entre a chamada fopen() e a chamada fwrite(). Isso não é idêntico à uma chamada fopen() com modo "x".

`context`

Um recurso de contexto válido criado com stream_context_create().

Valor Retornado

Esta função retorna o número de bytes que foram escritos no arquivo ou **`false`** em caso de falha.

> **Aviso**
>
> Esta função pode retornar o valor booleano **`false`**, mas também pode retornar um valor não booleano que pode ser avaliado como **`false`**. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Examples

Exemplo #1 Exemplo de uso simples

```php
<?php
$file = 'pessoas.txt';
// Abre o arquivo para obter o conteúdo existente
$current = file_get_contents($file);
// Acrescenta a nova pessoa no arquivo
$current .= "Fulano de Tal\n";
// Escreve o conteúdo de volta no arquivo
file_put_contents($file, $current);
?>
```

Exemplo #2 Usando opções

```php
<?php
$file = 'pessoas.txt';
// A nova pessoa para adicionar no arquivo
$person = "Cicrano de Tal\n";
// Escreve o conteúdo no arquivo
// usando a opção FILE_APPEND para acrescentar o conteúdo ao final do arquivo
// e a opção LOCK_EX para evitar que outra pessoa escreva no arquivo ao mesmo tempo.
file_put_contents($file, $person, FILE_APPEND | LOCK_EX);
?>
```

Notas

> **Nota** :
>
> Esta função é compatível com dados binários.

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

Veja também

- fopen() - Abre um arquivo ou URL
- fwrite() - Escrita binary-safe em arquivos
- file_get_contents() - Lê todo o conteúdo de um arquivo para uma string
- stream_context_create() - Cria um contexto de fluxo

fileatime

fileatime

(PHP 4, PHP 5, PHP 7, PHP 8)

fileatime — Obtém o último horário de acesso do arquivo

Descrição

function **fileatime**(string `$filename`): int|false

Obtém o último horário de acesso do arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o horário em que um arquivo foi acessado pela última vez, ou `false` em caso de falha. O horário é retornado como um timestamp Unix.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de fileatime()

```php
<?php

// Exibe algo como: arquivo.txt teve o último acesso em: December 29 2002 22:16:23.

$filename = 'arquivo.txt';
if (file_exists($filename)) {
    echo "$filename teve o último acesso em: " . date ("F d Y H:i:s.", fileatime($filename));
}

?>
```

Notas

> **Nota** :

> O horário de acesso de um arquivo deve mudar sempre os blocos de dados de um arquivo estão sendo lidos. Isto pode ser custoso do ponto de vista de desempenho quando uma aplicação acessa regularmente um grande número de arquivos ou diretórios.
>
> Alguns sistemas de arquivos Unix podem ser montados com atualizações de horário de acesso desabilitadas para aumentar o desempenho de tais aplicações; os spools de notícias da USENET são um exemplo comum. Nestes sistemas de arquivos esta função não terá utilidade.

> **Nota** :
>
> Note que a resolução de tempo pode variar de um sistema de arquivos para outro.

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- filemtime() - Obtém o horário de modificação do arquivo
- fileinode() - Lê o inode do arquivo
- date() - Formata um timestamp Unix

filectime

filectime

(PHP 4, PHP 5, PHP 7, PHP 8)

filectime — Obtém o horário de modificação de inode do arquivo

Descrição

function **filectime**(string `$filename`): int|false

Obtém o horário de modificação do inode de um arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o horário em que o arquivo foi modificado pela última vez, ou `false` em caso de falha. O horário é retornado como um timestamp Unix.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de filectime()

```php
<?php

// Exibe algo como: arquivo.txt foi modificado em: December 29 2002 22:16:23.

$filename = 'arquivo.txt';
if (file_exists($filename)) {
```

```
    echo "$filename foi modificado em: " . date ("F d Y H:i:s.", filectime($filename));
}

?>
```

Notas

> **Nota** :
>
> Nota: Na maioria dos sistemas de arquivos Unix, um arquivo é considerado modificado quando seus dados de inode são modificados; isto é, quando as permissões, proprietário, grupo ou outros metadados do inode são atualizados. Consulte também filemtime() (que é usado quando se deseja criar rodapés de páginas web do tipo "Última Modificação") e fileatime().

> **Nota** :
>
> Note também que em alguns textos sobre Unix o horário de modificação de um arquivo é definido como sendo o horário de criação do arquivo. Isto está errado. Não existe horário de criação para arquivos na maioria dos sistemas de arquivos Unix.

> **Nota** :
>
> Note que a resolução de tempo pode variar de um sistema de arquivos para outro.

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- filemtime() - Obtém o horário de modificação do arquivo

filegroup

filegroup

(PHP 4, PHP 5, PHP 7, PHP 8)

filegroup — Lê o grupo do arquivo

Descrição

function **filegroup**(string `$filename`): int|false

Obtém o grupo do arquivo. O ID do grupo é retornado no formato numérico, use posix_getgrgid() para resolvê-lo para um nome do grupo.

Parâmetros

`filename`

Caminho do arquivo.

Valor Retornado

Retorna o group ID do arquivo, ou **`false`** no caso de um erro. O group ID é retornado em um formato numérico. Use posix_getgrgid() para resolver para o nome do grupo. Em caso de falha, **`false`** é retornado.

Erros/Exceções

Em caso de falha. um `E_WARNING` é emitido.

Examples

Exemplo #1 Descobrindo o grupo de um arquivo

```php
<?php
$filename = 'index.php';
print_r(posix_getgrgid(filegroup($filename)));
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- fileowner() - Obtem o proprietário do arquivo
- posix_getgrgid() - Retorna informação sobre um grupo a partir de um ID de grupo

fileinode

fileinode

(PHP 4, PHP 5, PHP 7, PHP 8)

fileinode — Lê o inode do arquivo

Descrição

function **fileinode**(string `$filename`): int|false

Obtém o inode do arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o número do inode do arquivo, ou `false` em caso de falha.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Comparando o inode de um arquivo com o arquivo atual

```php
<?php
$filename = 'index.php';
if (getmyinode() == fileinode($filename)) {
    echo 'Você está verificando o arquivo atual.';
}
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- getmyinode() - Obtém o inode do script atual
- stat() - Obtém informações sobre um arquivo

filemtime

filemtime

(PHP 4, PHP 5, PHP 7, PHP 8)

filemtime — Obtém o horário de modificação do arquivo

Descrição

function **filemtime**(string `$filename`): int|false

Esta função retorna o horário em que os blocos de dados de um arquivo estavam sendo escritos, isto é, o horário em que o conteúdo do arquivo foi modificado.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o horário em que o arquivo foi modificado pela última vez, ou `false` em caso de falha. O horário é retornado como um timestamp Unix, que é adequado para a função date().

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de filemtime()

```php
<?php
// exibe algo como: arquivo.txt foi modificado em December 29 2002 22:16:23.

$filename = 'arquivo.txt';
if (file_exists($filename)) {
    echo "$filename foi modificado em " . date ("F d Y H:i:s.", filemtime($filename));
}
?>
```

Notas

> **Nota** :
>
> Note que a resolução de tempo pode variar de um sistema de arquivos para outro.

> **Nota** :

> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- filectime() - Obtém o horário de modificação de inode do arquivo
- stat() - Obtém informações sobre um arquivo
- touch() - Define horário de acesso e de modificação de arquivo
- getlastmod() - Obtém o horário da última modificação na página

fileowner

fileowner

(PHP 4, PHP 5, PHP 7, PHP 8)

fileowner — Obtem o proprietário do arquivo

Descrição

function **fileowner**(string `$filename`): int|false

Obtém o proprietário do arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna a ID do usuário do proprietário do arquivo, ou `false` em caso de falha. A ID do usuário é retornada em formato numérico, use a função posix_getpwuid() para obter o nome do usuário.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Encontrando o proprietário de um arquivo

```php
<?php
$filename = 'index.php';
print_r(posix_getpwuid(fileowner($filename)));
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- filegroup() - Lê o grupo do arquivo

- stat() - Obtém informações sobre um arquivo
- posix_getpwuid() - Retorna informações sobre um usuário por ID de usuário

fileperms

fileperms

(PHP 4, PHP 5, PHP 7, PHP 8)

fileperms — Obtém permissões de arquivo

Descrição

function **fileperms**(string `$filename`): int|false

Obtém as permissões para o arquivo informado.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna as permissões do arquivo em modo numérico. Os bits menos significativos deste modo são os mesmos das permissões esperadas por chmod(), entretanto na maioria das plataformas o valor de retorno também inclui informação sobre o tipo do arquivo informado em `filename`. Os exemplos abaixo demonstram como testar o valor de retorno para permissões e tipos de arquivos específicos em sistemas POSIX, incluindo Linux e macOS.

Para aquivos locais, o valor de retorno específico é o do membro `st_mode` da estrutura retornada pela função stat() da biblioteca C. Exatamente quais bits são definidos pode variar de plataforma para plataforma, e é recomendado consultar a documentação específica da plataforma se for necessária uma interpretação dos bits de não-permissão do valor de retorno.

Retorna `false` em caso de falha.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Mostra as permissões como um valor em octal

```php
<?php
echo substr(sprintf('%o', fileperms('/tmp')), -4);
echo substr(sprintf('%o', fileperms('/etc/passwd')), -4);
?>
```

O exemplo acima produzirá:

```
1777
0644
```

Exemplo #2 Mostra as permissões completas

```php
<?php
$perms = fileperms('/etc/passwd');

switch ($perms & 0xF000) {
    case 0x1000: $info = 'p'; break; // pipe FIFO
    case 0x2000: $info = 'c'; break; // caractere especial
    case 0x4000: $info = 'd'; break; // diretório
    case 0x6000: $info = 'b'; break; // bloco especial
```

```
    case 0x8000: $info = '-'; break; // comum
    case 0xA000: $info = 'l'; break; // ligação simbólica (disponível ao utilizar a função lstat)
    case 0xC000: $info = 's'; break; // socket

    // desconhecido
    default: $info = 'u';
}

// Proprietário
$setuid = $perms & 0x0800;
$info .= (($perms & 0x0100) ? 'r' : '-');
$info .= (($perms & 0x0080) ? 'w' : '-');
$info .= (($perms & 0x0040) ? ($setuid ? 's' : 'x') : ($setuid ? 'S' : '-'));

// Grupo
$setgid = $perms & 0x0400;
$info .= (($perms & 0x0020) ? 'r' : '-');
$info .= (($perms & 0x0010) ? 'w' : '-');
$info .= (($perms & 0x0008) ? ($setgid ? 's' : 'x') : ($setgid ? 'S' : '-'));

// Outros
$sticky = $perms & 0x0200;
$info .= (($perms & 0x0004) ? 'r' : '-');
$info .= (($perms & 0x0002) ? 'w' : '-');
$info .= (($perms & 0x0001) ? ($sticky ? 't' : 'x') : ($sticky ? 'T' : '-'));

echo $info;
?>
```

O exemplo acima produzirá:

```
-rw-r--r--
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- chmod() - Modifica as permissões do arquivo
- is_readable() - Informa se um arquivo existe e se pode ser lido
- stat() - Obtém informações sobre um arquivo

filesize

filesize

(PHP 4, PHP 5, PHP 7, PHP 8)

filesize — Obtém o tamanho do arquivo

Descrição

function **filesize**(string `$filename`): int|false

Obtém o tamanho do arquivo.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o tamanho do arquivo em bytes, ou `false` (e gera um aviso `E_WARNING`) em caso de erro.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de filesize()

```php
<?php

// Exibe algo como: arquivo.txt: 1024 bytes

$filename = 'arquivo.txt';
echo $filename . ': ' . filesize($filename) . ' bytes';

?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- file_exists() - Verifica se um arquivo ou diretório existe

filetype

filetype

(PHP 4, PHP 5, PHP 7, PHP 8)

filetype — Obtém o tipo do arquivo

Descrição

function **filetype**(string `$filename`): string|false

Retorna o tipo do arquivo informado.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna o tipo do arquivo. Os valores possíveis são 'fifo' (fila), 'char' (caractere), 'dir' (diretório), 'block' (bloco), 'link' (ligação), 'file' (arquivo), 'socket' (soquete) e 'unknown' (desconhecido).

Retorna `false` se ocorrer um erro. **filetype()** também produzirá uma mensagem `E_NOTICE` se a chamada 'stat' falhar ou se o tipo do arquivo for desconhecido.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de filetype()

```php
<?php

echo filetype('/etc/passwd');
echo "\n";
echo filetype('/etc/');

?>
```

O exemplo acima produzirá:

```
file
dir
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- is_dir() - Diz se o caminho é um diretório
- is_file() - Informa se o arquivo é um arquivo comum
- is_link() - Informa se o arquivo é uma ligação simbólica
- file_exists() - Verifica se um arquivo ou diretório existe
- mime_content_type() - Detecta o tipo de conteúdo MIME de um arquivo
- pathinfo() - Retorna informações sobre um caminho de arquivo
- stat() - Obtém informações sobre um arquivo

flock

flock

(PHP 4, PHP 5, PHP 7, PHP 8)

flock — Travamento consultivo portável de arquivo

Descrição

function **flock**(resource `$stream`, int `$operation`, int `&$would_block` = `null`): bool

flock() permite realizar um modelo simples de leitura/escrita que pode ser usado virtualmente em qualquer plataforma (incluindo na maioria dos derivados Unix e até mesmo no Windows).

A trava é liberada também por fclose(), ou quando o fluxo `stream` sofrer coleta de lixo.

O PHP suporta uma maneira portável de travamento de arquivos completos de forma consultiva (que significa que todos programas de acesso devem usar a mesma forma de travamento ou ela não irá funcionar). Por padrão, esta função irá bloquear até que a trava solicitada seja adquirida; isto pode ser controlado com a opção `LOCK_NB` documentada abaixo.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`operation`

`operation` deve ser uma das opções a seguir:

- `LOCK_SH` para obter uma trava compartilhada (leitura).
- `LOCK_EX` para obter uma trava exclusiva (escrita).
- `LOCK_UN` para liberar uma trava (compartilhada ou exclusiva).

É possível também adicionar `LOCK_NB` como uma máscara de bits a uma das operações acima, caso seja necessário que **flock()** não bloqueie durante a tentativa de travamento.

`would_block`

O terceiro argumento opcional é definido como 1 se a trava for bloquear (condição de erro número EWOULDBLOCK).

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de flock()

```php
<?php

$fp = fopen("/tmp/lock.txt", "r+");

if (flock($fp, LOCK_EX)) {  // adquire uma trava exclusiva
    ftruncate($fp, 0);      // trunca o arquivo
    rewind($fp);
    fwrite($fp, "Escreve alguma coisa aqui\n");
    fflush($fp);            // descarrega a saída antes de liberar a trava
    flock($fp, LOCK_UN);    // libera a trava
} else {
    echo "Não foi possível obter a trava!";
}

fclose($fp);

?>
```

Exemplo #2 flock() usando a opção `LOCK_NB`

```php
<?php
$fp = fopen('/tmp/lock.txt', 'r+');

/* Ativa a opção LOCK_NB em uma operação LOCK_EX */
if(!flock($fp, LOCK_EX | LOCK_NB)) {
    echo 'Não foi possível obter a trava';
    exit(-1);
```

```
}

/* ... */

fclose($fp);
?>
```

Notas

> **Nota** :
>
> **flock()** usa travamento mandatório no lugar de travamento consultivo no Windows. Travamento mandatório também é suportado no Linux e em sistemas operacionais baseados em System V através do mecanismo usual suportado pela chamada de sistema fcntl(): isto é, se o arquivo em questão tem a o bit de permissão setgid ligado e o bit de execução do grupo desligado. No Linux, o sistema de arquivos também precisará ser montado com a opção mand para que isto funcione.

> **Nota** :
>
> Pelo motivo de **flock()** requerer um ponteiro de arquivo, pode ser necessário usar um arquivo de trava especial para proteger o acesso a um aqruivo que deseja-se truncar abrindo-o em modo de escrita (com um argumento "w" ou "w+" para fopen()).

> **Nota** :
>
> Pode somente ser usado em ponteiros de arquivos retornados por fopen() para arquivos locais, ou ponteiros de arquivos que apontam para fluxo no espaço do usuário que implementam o método streamWrapper::stream_lock().

> **Aviso**
>
> Atribuir outro valor ao argumento `stream` em código subsequente irá liberar a trava.

> **Aviso**
>
> Em alguns sistemas operacionais, **flock()** é implementado no nível de processos. Ao usar uma API de servidor multi-tarefa pode ser que se consiga confiar em **flock()** para proteger arquivos contra outros scripts PHP sendo executados em tarefas parelelas da mesma instância do servidor!
>
> **flock()** não é suportado em sistemas de arquivos antiquados como `FAT` e seus derivativos e portanto sempre retornará `false` nestes ambientes.

> **Nota** :
>
> No Windows, se o processo de travamento abrir o arquivo uma segunda vez, ele não poderá acessar o arquivo através deste segundo manipulador até que destrave o arquivo.

fnmatch

fnmatch

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

fnmatch — Compara nome de arquivo com um padrão

Descrição

function **fnmatch**(string `$pattern`, string `$filename`, int `$flags` = 0): bool

fnmatch() verifica se o parâmetro `filename` informado corresponde com o padrão de curingas shell `pattern`.

Parâmetros

`pattern`

O parâmetro `pattern` deve conter o padrão para correspondência. Normalmente, `pattern` conterá curingas como `'?'` e `'*'`.

Curingas a serem usados no parâmetro `pattern`

Curinga	Descrição
`?`	Ponto de interrogação irá corresponder a um caractere único. Por exemplo, o padrão `"file?.txt"` corresponderá a `"file1.txt"` e `"fileA.txt"`, mas não a `"file10.txt"`.
`*`	Asteriscos corresponderá zero ou mais caracteres. Por exemplo, o padrão `"foo*.xml"` corresponderá a `"foo.xml"` e `"foobar.xml"`.
`[ ]`	Colchetes são usados para criar intervalos de caracteres ASCII ou conjuntos de caracteres. Por exemplo, o padrão `"index.php[45]"` corresponderá a `"index.php4"` e `"index.php5"`, mas não a `"index.phpt"`. Intervalos bem conhecidos são `[0-9]`, `[a-z]` e `[A-Z]`. Conjuntos e intervalos múltiplos podem ser usados ao mesmo tempo, por exemplo `[0-9a-zABC]`.
`!`	Ponto de exclamação é usado para negar caracteres dentro de colchetes. Por exemplo, `"[!A-Z]*.html"` corresponderá a `"demo.html"`, mas não a `"Demo.html"`.
`\`	Barra invertida é usada para escapar caracteres especiais. Por exemplo, `"Name\?"` corresponderá a `"Name?"`, mas não a `"Names"`.

`filename`

A string testada. Esta função é especialmente útil para nomes de arquivo, mas também pode ser usada em strings normais.

O usuário comum pode estar acostumado com padrões shell ou pelo menos, na sua forma mais simples, aos curingas `'?'` e `'*'`. Então usar **fnmatch()** ao invés de preg_match() para entrada de expressões de busca na interface com o usuário pode ser muito mais conveniente para não programadores.

`flags`

O valor do parâmetro `flags` pode ser qualquer combinação das opções a seguir, unidos com o operador binário OR (|).

Uma lista de opções possíveis para fnmatch()

Flag	Descrição
`FNM_NOESCAPE`	Desabilita escape por barra invertida.
`FNM_PATHNAME`	Barra na string somente corresponde a barra no padrão fornecido.
`FNM_PERIOD`	Um ponto no início da string deve corresponder a exatamente um ponto no padrão fornecido.
`FNM_CASEFOLD`	Não diferencia maiúsculas de minúsculas. Parte da extensão GNU.

Valor Retornado

Retorna `true` se houver correspondência, `false` caso contrário.

Examples

Exemplo #1 Comparando uma cor com um padrão de curingas shell

```
<?php
if (fnmatch("*gr[ae]y", $color)) {
  echo "alguma forma de cinza (grey ou gray) ...";
}
?>
```

Notas

> **Aviso**
>
> Por enquanto esta função não está disponível em sistemas não-POSIX exceto pelo Windows.

Veja também

- glob() - Encontra caminhos que combinam com uma expressão
- preg_match() - Realiza uma correspondência com expressão regular
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada

fopen

fopen

(PHP 4, PHP 5, PHP 7, PHP 8)

fopen — Abre um arquivo ou URL

Descrição

```
function fopen(
    string $filename,
    string $mode,
    bool $use_include_path = false,
    ?resource $context = null
): resource|false
```

fopen() conecta um recurso nomeado, especificado por `filename`, a um fluxo.

Parâmetros

`filename`

Se `filename` estiver na forma de "esquema://...", assume-se que seja uma URL, e o PHP buscará por um manipulador de protocolo (também conhecido como empacotador) para aquele esquema. Se nenhum empacotador para aquele protocolo estiver registrado, o PHP emitirá uma aviso para auxiliar no rastreio de potenciais problemas no script, e então continuará presumindo que `filename` especifica um arquivo normal.

Se o PHP decidiu que `filename` se refere a um arquivo local, então ele tentará abrir um fluxo para aquele arquivo. O arquivo precisa ser acessível pelo PHP, portanto é necessário garantir que as permissões de acesso do arquivo possibilitem este acesso. Se estiver habilitado open_basedir, outras restrições podem se aplicar.

Se o PHP decidiu que `filename` se refere a um protocolo registrado e esse protocolo estiver registrado como uma URL de rede, o PHP irá verificar para ter certeza que allow_url_fopen está habilitado. Se não estiver, o PHP emitirá um alerta e a chamada a **fopen()** irá falhar.

> **Nota** :
>
> A lista de protocolos suportados pode ser encontrada em Protocolos e empacotadores suportados. Alguns protocolos (também mencionados como empacotadores - `wrappers`) suportam `context` e/ou opções do *php.ini*. Recorra à página específica do protocolo em uso para uma lista das opções que podem ser definidas. (ex.: o valor *php.ini* `user_agent` utilizado pelo protocolo `http`).

Na plataforma Windows, tenha cuidado de escapar qualquer barra invertida usada no caminho do arquivo, ou use barras normais.

```php
<?php
$handle = fopen("c:\\data\\info.txt", "r");
?>
```

`mode`

O parâmetro `mode` especifica o tipo de acesso que você precisa ao fluxo. Pode ser um dos seguintes:

Lista dos possíveis modos de fopen() utilizando `mode`

`mode`	Descrição
`'r'`	Abre somente para leitura; coloca o ponteiro do arquivo no começo do arquivo.
`'r+'`	Abre para leitura e escrita; coloca o ponteiro do arquivo no começo do arquivo.
`'w'`	Abre somente para escrita; coloca o ponteiro do arquivo no começo do arquivo e reduz o comprimento do arquivo para zero. Se o arquivo não existir, tenta criá-lo.
`'w+'`	Abre para leitura e escrita; fora isso, tem o mesmo comportamento de `'w'`.
`'a'`	Abre somente para escrita; coloca o ponteiro do arquivo no final do arquivo. Se o arquivo não existir, tenta criá-lo. Neste modo, fseek() não tem efeito, a escrita é sempre adicionada.
`'a+'`	Abre para leitura e escrita; coloca o ponteiro do arquivo no final do arquivo. Se o arquivo não existir, tenta criá-lo. Neste modo, fseek() somente afeta a posição de leitura, escrita é sempre adicionada.

mode	Descrição
'x'	Cria e abre o arquivo somente para escrita; coloca o ponteiro no começo do arquivo. Se o arquivo já existir, a chamada a **fopen()** falhará, retornando `false` e gerando um erro de nível `E_WARNING`. Se o arquivo não existir, tenta criá-lo. Isto é equivalente a especificar as opções `O_EXCL|O_CREAT` para a chamada de sistema `open(2)` subjacente.
'x+'	Cria e abre o arquivo para leitura e escrita; fora isso, tem o mesmo comportamento de 'x'.
'c'	Abre o arquivo somente para escrita. Se o arquivo não existir, ele é criado. Se existir, ele não é truncado (ao contrário de 'w') e nem a chamada a esta função falha (como no caso de 'x'). O ponteiro é posicionado no início do arquivo. Isto pode ser útil se for desejado obter uma trava consultiva (consulte flock()) antes de tentar modificar o arquivo, já que usar 'w' poderia trincar o arquivo antes que a trava fosse obtida (se o truncamento for desejado, ftruncate() pode ser usando depois que a trava for requisitada).
'c+'	Abre o arquivo para leitura e escrita; fora isso, tem o mesmo comportamento de 'c'.
'e'	Define a opção close-on-exec no descritor do arquivo aberto. Somente disponível no PHP compilado em sistemas compatíveis com POSIX.1-2008.
'n'	Define o sinalizador não bloqueante no descritor de arquivo aberto. Disponível apenas no PHP compilado em sistemas compatíveis com POSIX.1-2008.

> **Nota** :
>
> Diferentes famílias de sistemas operacionais têm diferentes convenções de final de linha. Quando um arquivo texto é escrito e deseja-se inserir uma quebra de linha, é necessário usar o caractere correto de fim de linha para o sistema operacional. Sistemas baseados no Unix utilizam `\n` como o caractere de fim de linha, sistemas baseados no Windows utilizam `\r\n` e sistemas baseados no Macintosh (Mac OS Classic) utilizavam `\r`.
>
> Se forem usados os caracteres de fim de linha inadequados em seus arquivos, pode ser que esses arquivos "parecer estranhos" quando forem abertos em outras aplicações.
>
> O Windows oferece uma opção de tradução do modo texto (`'t'`) que traduz, transparentemente, `\n` para `\r\n` quando estiver trabalhando com o arquivo. Em contraste, pode-se também utilizar `'b'` para forçar o modo binário, que não irá traduzir os dados. Para usar essas opções, especifique `'b'` ou `'t'` como o último caractere do parâmetro `mode`.
>
> O modo de tradução padrão é `'b'`. Pode-se usar o modo `'t'` ao trabalhar com arquivos texto e usar `\n` para delimitar as linhas no script, de forma que pode-se esperar que eles sejam lidos em aplicações como versões antigas do bloco de notas. Deve-se usar `'b'` em todos os outros casos.
>
> Se for especificada a opção 't' ao trabalhar com arquivos binários, pode-se experimentar problemas estranhos com os dados, incluindo arquivos de imagens corrompidos e problemas estranhos com os caracteres `\r\n`.

> **Nota** :
>
> Para portabilidade, é fortemente recomendado que se reescreva códigos que usem ou dependam do modo `'t'` de forma que passem a usar o fim de linha correto e o modo `'b'` em seu lugar.

> **Nota** :
>
> O parâmetro `mode` é ignorado para os empacotadores de fluxo *php://output*, *php://input*, *php://stdin*, *php://stdout*, *php://stderr* e *php://fd*.

`use_include_path`

O terceiro parâmetro opcional `use_include_path` pode ser definido `true` se for desejado buscar o arquivo também no include_path.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna um recurso de ponteiro de arquivo em caso de sucesso, ou `false` em caso de falha

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Registro de Alterações

Versão	Descrição
7.0.16, 7.1.2	A opção `'e'` foi adicionada.

Examples

Exemplo #1 Exemplos de fopen()

```
<?php
$handle = fopen("/home/rasmus/file.txt", "r");
$handle = fopen("/home/rasmus/file.gif", "wb");
$handle = fopen("http://www.example.com/", "r");
$handle = fopen("ftp://user:password@example.com/somefile.txt", "w");
?>
```

Notas

> **Aviso**
>
> Ao usar SSL, o Microsoft IIS violará o protocolo fechando a conexão sem enviar uma notificação `close_notify`. O PHP reportará isso como "SSL: Fatal Protocol Error" quando chegar ao final dos dados. Para contornar isso, o valor de error_reporting deve ser reduzido a um nível que não inclua avisos. O PHP pode detectar servidores IIS defeituosos quando o fluxo é aberto usando o empacotador `https://` e suprimirá o aviso. Ao usar fsockopen() para criar um soquete `ssl://`, o desenvolvedor é responsável por detectar e suprimir este aviso.

> **Nota** :
>
> Se houver problemas com leitura e gravação de arquivos e a versão de módulo de servidor do PHP estiver sendo usada, deve-se garantir que os arquivos e diretórios sendo usados são acessíveis ao processo do servidor.

> **Nota** :
>
> Esta função também pode ter sucesso quando `filename` for um diretório. Se não houver certeza se `filename` é um arquivo ou um diretório, pode ser necessário usar a função is_dir() antes de chamar **fopen()**.

Veja também

- Protocolos e empacotadores suportados
- fclose() - Fecha um ponteiro de arquivo aberto
- fgets() - Lê uma linha de um ponteiro de arquivo
- fread() - Leitura de arquivo segura para binário
- fwrite() - Escrita binary-safe em arquivos
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- file() - Lê todo o arquivo para um array
- file_exists() - Verifica se um arquivo ou diretório existe
- is_readable() - Informa se um arquivo existe e se pode ser lido
- stream_set_timeout() - Define o limite de tempo em um fluxo
- popen() - Abre um processo como ponteiro de arquivo
- stream_context_create() - Cria um contexto de fluxo
- umask() - Modificar a umask atual
- SplFileObject

fpassthru

fpassthru

(PHP 4, PHP 5, PHP 7, PHP 8)

fpassthru — Imprime todos os dados restantes de um ponteiro de arquivo

Descrição

function **fpassthru**(resource `$stream`): int

Lê até o fim do arquivo (EOF) do ponteiro de arquivo dado e imprime os resultados para a buffer de saída.

Pode ser necessário chamar rewind() para redefinir o ponteiro do arquivo para início do arquivo se dados já tiverem sido escritos no arquivo.

Se for desejado apenas descarregar o conteúdo de um arquivo para o buffer de saída, sem primeiro modificá-lo ou deslocar o ponteiro até uma posição, pode-se usar a função readfile(), que vai economizar uma chamada à função fopen().

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna o número de caracteres lidos do fluxo `stream` e passados para a saída.

Examples

Exemplo #1 Usando fpassthru() com arquivo binário

```php
<?php

// Abre o arquivo em modo binário
$name = './img/ok.png';
$fp = fopen($name, 'rb');

// Envia os cabeçalhos
header("Content-Type: image/png");
header("Content-Length: ".filesize($name));

// Manda a imagem e para o script
fpassthru($fp);
exit;

?>
```

Notas

> **Nota** :
>
> Ao utilizar **fpassthru()** em um arquivo binário em sistemas Windows, assegure-se de abrir o arquivo em modo binário, acrescentando um `b` ao modo informado na chamada a fopen().
>
> Recomenda-se utilizar o modo `b` ao lidar com arquivos binários, mesmo que seu sistema não exiga isso, de forma que seus scripts se tornem mais portáveis.

Veja também

- readfile() - Exibe o conteúdo de um arquivo
- fopen() - Abre um arquivo ou URL
- popen() - Abre um processo como ponteiro de arquivo
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet

fputcsv

fputcsv

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

fputcsv — Formata a linha como CSV e a escreve em um ponteiro de arquivo

Descrição

```
function fputcsv(
    resource $stream,
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
```

fputcsv() formata uma linha (passada como um array de campos `fields`) como CSV e a escreve (terminando com uma nova linha `eol`) no fluxo de arquivo especificado por `stream`.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

`fields`

Um array de strings.

`separator`

O parâmetro `separator` define o delimitador de campo. Precisa ser um caractere de um byte.

`enclosure`

O parâmetro `enclosure` define o caractere que cerca um campo. Precisa ser um caractere de um byte.

`escape`

O parâmetro `escape` define o caractere de escape. Precisa ser um caractere de um byte ou uma string vazia. Uma string vazia (`""`) desabilita o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado. Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados.

`eol`

O parâmetro opcional `eol` define uma sequência customizada de Fim de Linha.

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

> **Nota** :

> Se um caractere `enclosure` estiver contido em um campo, ele sofrerá escape por duplicação, a menos que seja imediatamente precedido por um `escape`.

Valor Retornado

Retorna o comprimento da string escrita ou `false` em caso de falha.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
8.1.0	O parâmetro opcional `eol` foi adicionado.
7.4.0	O parâmetro `escape` agora aceita uma string vazia para desabilitar o mecanismo de escape proprietário.

Examples

Exemplo #1 Exemplo da fputcsv()

```php
<?php

$lista = [
    ['aaa', 'bbb', 'ccc', 'dddd'],
    ['123', '456', '789'],
    ['"aaa"', '"bbb"']
];

$fp = fopen('file.csv', 'w');

foreach ($lista as $linha) {
    fputcsv($fp, $linha, ',', '"', '');
}

fclose($fp);
?>
```

O exemplo acima irá escrever o seguinte no arquivo `file.csv`:

```
aaa,bbb,ccc,dddd
123,456,789
"""aaa""","""bbb"""
```

Veja também

- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array
- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV

fputs

fputs

(PHP 4, PHP 5, PHP 7, PHP 8)

fputs — Sinônimo de fwrite()

Descrição

Esta função é um apelido para: fwrite().

fread

fread

(PHP 4, PHP 5, PHP 7, PHP 8)

fread — Leitura de arquivo segura para binário

Descrição

function **fread**(resource `$stream`, int `$length`): string|false

fread() lê até `length` bytes do ponteiro de arquivo informado em `stream`. A leitura é interrompida quando uma das seguintes condições são satisfeitas:

- `length` bytes foram lidos
- EOF (final do arquivo) é alcançado
- um pacote tornou-se disponível ou o limite de tempo de socket foi atingido (para fluxos de rede)
- se o fluxo é lido com buffer e não representa um arquivo, é realizada no máximo uma leitura de até o número de bytes igual ao tamanho do bloco (normalmente 8192); dependendo dos dados previamente armazenados em buffer, o tamanho dos dados retornados pode ser maior que o tamanho do bloco.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`length`

Limite de até `length` bytes lidos.

Valor Retornado

Retorna a string lida ou `false` em caso de falha.

Examples

Exemplo #1 Um simples exemplo de fread()

```php
<?php
// lê o conteúdo do arquivo para uma string
$filename = "/usr/local/qualquer.txt";
$handle = fopen ($filename, "r");
$conteudo = fread ($handle, filesize ($filename));
fclose ($handle);
?>
```

Exemplo #2 Exemplo de fread() binário

> **Aviso**
>
> Em sistemas que diferenciam entre arquivos binários e texto (por exemplo Windows) o arquivo tem que ser aberto com 'b' incluído no parâmetro 'mode' da função fopen().

```php
<?php
$filename = "c:\\files\\figura.gif";
$handle = fopen ($filename, "rb");
$conteudo = fread ($handle, filesize ($filename));
```

```
fclose ($handle);
?>
```

Exemplo #3 Exemplos de fread() remoto

> **Aviso**
>
> Ao ler de qualquer coisa que não seja um arquivo local comum, tal como de fluxos retornados ao ler arquivos remotos, ou de popen() e fsockopen(), a leitura irá parar depois que um pacote estiver disponível. Isto significa que deve-se juntar os dados em blocos como demonstrado nos exemplos abaixo.

```
<?php
$handle = fopen("http://www.example.com/", "rb");
$contents = stream_get_contents($handle);
fclose($handle);
?>
```

```
<?php
$handle = fopen("http://www.example.com/", "rb");
if (FALSE === $handle) {
    exit("Falha ao abrir fluxo para a URL");
}

$contents = '';

while (!feof($handle)) {
    $contents .= fread($handle, 8192);
}
fclose($handle);
?>
```

Notas

> **Nota** :
>
> Se for desejado apenas obter o conteúdo de um arquivo para uma string, utilize file_get_contents() que tem um desempenho muito melhor que o código acima.

> **Nota** :
>
> Note que **fread()** lê da posição atual do ponteiro de arquivo. Use ftell() para encontrar a posição atual do ponteiro e rewind() para retroceder a posição do ponteiro.

Veja também

- fwrite() - Escrita binary-safe em arquivos
- fopen() - Abre um arquivo ou URL
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- popen() - Abre um processo como ponteiro de arquivo
- fgets() - Lê uma linha de um ponteiro de arquivo
- fgetss() - Ler uma linha de um ponteiro de arquivo e retira as tags HTML
- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- file() - Lê todo o arquivo para um array
- fpassthru() - Imprime todos os dados restantes de um ponteiro de arquivo
- fseek() - Aponta para uma posição no arquivo
- ftell() - Retorna a posição atual do ponteiro de leitura/gravação do arquivo
- rewind() - Retrocede a posição de um ponteiro de arquivos
- unpack() - Desempacota dados de uma string binária

fscanf

fscanf

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

fscanf — Interpreta a leitura de um arquivo de acordo com um formato

Descrição

function **fscanf**(resource `$stream`, string `$format`, mixed `&...$vars`): array|int|false|null

A função **fscanf()** é semelhante à sscanf(), mas usa como entrada um arquivo associado com o fluxo `stream` e interpreta a entrada de acordo com o especificado em `format`.

Qualquer espaço em branco na string do formato corresponde a quaisquer espaços em branco no fluxo de entrada. Isto significa que até mesmo uma tabulação (`\t`) na string do formato pode corresponder a um único caractere de espaço no fluxo de entrada.

Cada chamada a **fscanf()** lê uma linha do arquivo.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`format`

O formato interpretado para `string`, que é descrito na documentação para sprintf() com as seguintes diferenças:

- A função não reconhece a localidade.
- `F`, `g`, `G` e `b` não são suportados.
- `D` significa número decimal.
- `i` significa inteiro com detecção de base.
- `n` representa o número de caracteres processados até o momento.
- `s` pára de ler em qualquer caractere de espaço em branco.
- `*` em vez de `argnum$` suprime a atribuição desta especificação de conversão.

`vars`

Os valores opcionais atribuídos.

Valor Retornado

Se somente dois parâmetros forem passados a esta função, os valores interpretados serão retornados como um array. Do contrário, se parâmetros opcionais forem passados, a função retornará o número de valores atribuídos. Os parâmetros opcionais devem ser passados por referência.

Se houver mais substrings esperadas no parâmetro `format` do que estiverem disponíveis na `string`, **`null`** será retornado. Para outros erros, **`false`** será retornado.

Quando parâmetros opcionais são usados e o fim da entrada lida de `stream` é alcançado antes que qualquer valor tenha sido interpretado, `-1` é retornado.

Examples

Exemplo #1 Exemplo de **fscanf()**

```php
<?php
$handle = fopen("users.txt", "r");
while ($userinfo = fscanf($handle, "%s\t%s\t%s\n")) {
    list ($name, $profession, $countrycode) = $userinfo;
    //... faz algo com os valores
}
fclose($handle);
?>
```

Exemplo #2 Conteúdo de users.txt

```
javier  argonaut        pe
hiroshi sculptor        jp
robert  slacker us
luigi   florist it
```

Veja também

- fread() - Leitura de arquivo segura para binário
- fgets() - Lê uma linha de um ponteiro de arquivo
- fgetss() - Ler uma linha de um ponteiro de arquivo e retira as tags HTML
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada

fseek

fseek

(PHP 4, PHP 5, PHP 7, PHP 8)

fseek — Aponta para uma posição no arquivo

Descrição

function **fseek**(resource `$stream`, int `$offset`, int `$whence` = **`SEEK_SET`**): int

Modifica o indicador de posição do arquivo referenciado por `stream`. A nova posição, medida em bytes a partir do início do arquivo, é obtida ao adicionar `offset` à posição especificada por `whence`.

Geralmente é permitido apontar para além do fim-do-arquivo; se algum dado for então escrito, leituras além de qualquer região escrita entre o fim-do-arquivo e a região desejada irão trazer bytes com valor 0. Entretanto, alguns fluxos podem não suportar esse comportamento, especialmente quando têm definido um tamanho de armazenamento fixo.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`offset`

A posição.

Para apontar para uma posição anterior ao fim-do-arquivo, um valor negativo em `offset` deve ser passado e o parâmetro `whence` deve ser definido para **`SEEK_END`**.

`whence`

Valores de `whence` são:

- **`SEEK_SET`** - Define a posição igual ao número de bytes informado em `offset` desde o início do arquivo.
- **`SEEK_CUR`** - Define a posição para a localização atual mais o número de bytes informado em `offset`.
- **`SEEK_END`** - Define a posição para o final do arquivo mais o número de bytes informado em `offset`.

Valor Retornado

Em caso de sucesso, retorna `0`; caso contrário, retorna `-1`.

> **Aviso**
>
> Esta função foi criada para imitar a função de mesmo nome da linguagem C. Atenção aos valores de retorno, pois eles diferem do esperado no PHP.

Examples

Exemplo #1 Exemplo de fseek()

```php
<?php

$fp = fopen('arquivo.txt');

// lê alguns dados
$data = fgets($fp, 4096);

// move de volta para o inicio do arquivo
// o mesmo que rewind($fp);
```

```php
fseek($fp, 0);

?>
```

Notas

> **Nota** :
>
> Se o arquivo foi aberto em modo de adição (`a` ou `a+`), quaisquer dados que sejam escritos no arquivo serão sempre acrescentados ao final, independente da posição do arquivo, e o resultado da chamada a **fseek()** será indefinido.

> **Nota** :
>
> Nem todos os fluxos suportam pesquisa. Para aqueles que não suportam, pesquisas além da posição atual podem ser feitas através da leitura e descarte de dados; outras formas de pesquisa irão falhar.

Veja também

- ftell() - Retorna a posição atual do ponteiro de leitura/gravação do arquivo
- rewind() - Retrocede a posição de um ponteiro de arquivos

fstat

fstat

(PHP 4, PHP 5, PHP 7, PHP 8)

fstat — Lê informações sobre um arquivo usando um ponteiro de arquivo aberto

Descrição

function **fstat**(resource `$stream`): array|false

Obtém estatísticas do arquivo aberto pelo ponteiro `stream`. Esta função é similar à função stat() exceto que ela opera em um ponteiro de arquivo aberto em vez de um nome de arquivo.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

Valor Retornado

Retorna um array com as estatísticas de um arquivo. O formato do array é descrito em detalhes na página do manual sobre stat(). Retorna `false` em caso de falha.

Examples

Exemplo #1 Exemplo de fstat()

```php
<?php

// abre um arquivo
$fp = fopen("/etc/passwd", "r");

// colhe as estatísticas
$fstat = fstat($fp);

// fecha o arquivo
fclose($fp);

// imprime somente a parte de índices associativos
print_r(array_slice($fstat, 13));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [dev] => 771
    [ino] => 488704
    [mode] => 33188
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 1114
    [atime] => 1061067181
    [mtime] => 1056136526
    [ctime] => 1056136526
    [blksize] => 4096
    [blocks] => 8
)
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

fsync

fsync

(PHP 8 >= 8.1.0)

fsync — Sincroniza mudanças ao arquivo (incluindo metadados)

Descrição

function **fsync**(resource `$stream`): bool

Este função sincroniza mudanças ao arquivo, incluindo seus metadados. Isto é semelhante a fflush(), mas também instrui o sistema operacional a escrever na mídia de armazenamento.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen() ou fsockopen() (e ainda não fechado por fclose()).

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de fsync()

```php
<?php

$file = 'test.txt';

$stream = fopen($file, 'w');
fwrite($stream, 'dados de teste');
fwrite($stream, "\r\n");
```

```php
fwrite($stream, 'dados adicionais');

fsync($stream);
fclose($stream);
?>
```

Veja também

- fdatasync() - Sincroniza dados (mas não metadados) ao arquivo
- fflush() - Força a liberação do buffer para um arquivo

ftell

ftell

(PHP 4, PHP 5, PHP 7, PHP 8)

ftell — Retorna a posição atual do ponteiro de leitura/gravação do arquivo

Descrição

function **ftell**(resource `$stream`): int|false

Retorna a posição do ponteiro do arquivo referenciado por `stream`.

Parâmetros

`stream`

O ponteiro de arquivo precisa ser válido, e deve apontar para um arquivo aberto com sucesso por fopen() ou popen(). **ftell()** dá resultados indefinidos para fluxos no modo de acréscimo (abertos com a opção "a").

Valor Retornado

Retorna a posição do ponteiro do arquivo referenciado por `stream` como um inteiro; isto é, seu deslocamento no fluxo do arquivo.

Se um erro ocorrer, retorna `false`.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Examples

Exemplo #1 Exemplo de ftell()

```php
<?php

// abre o arquivo e lê alguns dados
$fp = fopen("/etc/passwd", "r");
$data = fgets($fp, 12);

// onde estamos?
echo ftell($fp); // 11

fclose($fp);

?>
```

Veja também

- fopen() - Abre um arquivo ou URL
- popen() - Abre um processo como ponteiro de arquivo
- fseek() - Aponta para uma posição no arquivo
- rewind() - Retrocede a posição de um ponteiro de arquivos

ftruncate

ftruncate

(PHP 4, PHP 5, PHP 7, PHP 8)

ftruncate — Trunca um arquivo a um tamanho especificado

Descrição

function **ftruncate**(resource `$stream`, int `$size`): bool

Toma o ponteiro de arquivo `stream` e trunca o arquivo para o comprimento definido em `size`.

Parâmetros

`stream`

O ponteiro do arquivo.

> **Nota** :
>
> O fluxo `stream` deve estar aberto para escrita.

`size`

O tamanho para o qual será truncado.

> **Nota** :
>
> Se o parâmetro `size` for maior que o arquivo, o arquivo será estendido com bytes nulos.
>
> Se o parâmetro `size` for menor que o arquivo, o arquivo será truncado para este tamanho.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de truncamento de arquivo

```php
<?php
$filename = 'lorem_ipsum.txt';

$handle = fopen($filename, 'r+');
ftruncate($handle, rand(1, filesize($filename)));
rewind($handle);
echo fread($handle, filesize($filename));
fclose($handle);
?>
```

Notas

> **Nota** :
>
> O ponteiro do arquivo *não* é alterado.

Veja também

- fopen() - Abre um arquivo ou URL
- fseek() - Aponta para uma posição no arquivo

fwrite

fwrite

(PHP 4, PHP 5, PHP 7, PHP 8)

fwrite — Escrita binary-safe em arquivos

Descrição

function **fwrite**(resource `$stream`, string `$data`, ?int `$length = null`): int|false

fwrite() grava o conteúdo de `data` no ponteiro de arquivo informado em `stream`.

Parâmetros

`stream`

Um resource de ponteiro do sistema de arquivos que normalmente é criado usando fopen().

`data`

A string a ser escrita.

`length`

Se `length` for um int, a escrita irá parar depois que`length` forem escritos, ou o o fim de `data` for alcançado, o que ocorrer primeiro.

Valor Retornado

fwrite() retorna o número de bytes escritos, ou `false` em caso de falha.

Erros/Exceções

fwrite() emite um `E_WARNING` em caso falhas.

Registro de Alterações

Versão	Descrição
8.0.0	`length` pode ser null.

Examples

Exemplo #1 Exemplo de **fwrite()**

```php
<?php
$filename = 'test.txt';
$somecontent = "Acrescente isso no arquivo\n";

// Vamos garantir que o arquivo existe e pode ser escrito
if (is_writable($filename)) {

    // Nesse exemplo estamos abrindo o $filename em modo append.
    // O ponteiro do arquivo estará no final do arquivo
    // e portanto é aqui que $somecontent será posicionado pelo fwrite().
    if (!$fp = fopen($filename, 'a')) {
         echo "Erro ao abrir o ($filename)";
         exit;
    }

    // Escrever alguma coisa no arquivo.
    if (fwrite($fp, $somecontent) === FALSE) {
        echo "Erro ao escrever no arquivo ($filename)";
        exit;
    }

    echo "Sucesso, escrito ($somecontent) no arquivo ($filename)";

    fclose($fp);

} else {
    echo "O arquivo não permite escrita";
}
?>
```

Notas

> **Nota** :
>
> Escrever em um fluxo de rede pode acabar antes de toda a string ser transmitida. O retorno de **fwrite()** precisa ser inspecionado:

```php
<?php
function fwrite_stream($fp, $string) {
    for ($written = 0; $written < strlen($string); $written += $fwrite) {
        $fwrite = fwrite($fp, substr($string, $written));
        if ($fwrite === false) {
            return $written;
        }
    }
    return $written;
}
?>
```

> **Nota** :
>
> Em sistemas que diferenciam entre arquivos binários e texto (por exemplo Windows) o arquivo tem que ser aberto com 'b' incluído no parâmetro 'mode' na fopen().

> **Nota** :
>
> Se o `stream` foi aberto com fopen() em modo de adição, escritas com **fwrite()** são atômicas (a não ser que o tamanho da `data` exceda o tamanho de bloco do sistema de arquivos, em algumas plataformas, e contanto que o arquivo esteja em um sistema de arquivos local). Sendo assim, não há necessidade de bloquear um recurso com flock() antes de chamar **fwrite()**; todos os dados serão escritos sem interrupção.

> **Nota** :
>
> Se escrever duas vezes no ponteiro do arquivo, então a informação será adicionado ao final do contéudo do arquivo:

```php
<?php
$fp = fopen('data.txt', 'w');
fwrite($fp, '1');
fwrite($fp, '23');
fclose($fp);

// O conteúdo de 'data.txt' é 123, e não 23
?>
```

Veja também

- fread() - Leitura de arquivo segura para binário
- fopen() - Abre um arquivo ou URL
- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet
- popen() - Abre um processo como ponteiro de arquivo
- file_get_contents() - Lê todo o conteúdo de um arquivo para uma string
- pack() - Empacota dados em uma string binária

glob

glob

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

glob — Encontra caminhos que combinam com uma expressão

Descrição

function **glob**(string `$pattern`, int `$flags` = 0): array|false

A função **glob()** procura por todos os caminhos que correspondem à expressão em `pattern` de acordo com as regras usadas pela função glob() da libc, que é semelhante às regras usadas por shells comuns.

O comportamento em sistemas Unix e macOS é determinado pela implementação de glob() no sistema. No Windows, é usada uma implementação que está em conformidade com a definição POSIX 1003.2 para glob() e inclui uma extensão para lidar com a convenção `[!...]` para negar um intervalo.

Parâmetros

`pattern`

O padrão. Não é feita nenhuma expansão de til ou substituição de parâmetros.

Caracteres especiais:

- `*` - Corresponde a zero ou mais caracteres.
- `?` - Corresponde a exatamente um caractere (qualquer caractere).
- `[...]` - Corresponde a um caractere de um grupo de caracteres. Se o primeiro caractere for `!`, corresponde a qualquer caractere que não estiver no grupo.
- `{a,b,c}` - Corresponde a uma string de um grupo de strings delimitadas por vírgula quando a opção `GLOB_BRACE` for usada.
- `\` - Faz escape no caractere seguinte, exceto quando a opção `GLOB_NOESCAPE` for usada.

`flags`

Umas das constantes `GLOB_*`.

Valor Retornado

Retorna um array contendo os arquivos/diretórios que corresponderam, um array vazio se nenhum arquivo correspondeu ou `false` em caso de erro. A menos que `GLOB_NOSORT` tenha sido usada, os nomes serão ordenados alfabeticamente e numericamente.

Examples

Exemplo #1 Modo conveniente de como glob() pode susbstituir opendir() e companhia.

```php
<?php
foreach (glob("*.txt") as $arquivo) {
    echo "tamanho de $arquivo " . filesize($arquivo) . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
tamanho de funclist.txt 44686
tamanho de funcsummary.txt 267625
tamanho de quickref.txt 137820
```

Exemplo #2 Exemplo com uma expressão mais complexa

```php
<?php
foreach (glob("caminho/*/*.{txt,md}", \GLOB_BRACE) as $filename) {
    echo "$filename\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
caminho/docs/mailinglist-rules.md
caminho/docs/README.md
caminho/docs/release-process.md
```

```
caminho/pear/install-pear.txt
caminho/Zend/README.md
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :
>
> Esta função não está disponível em alguns sistemas (ex.: antigo Sun OS).

Veja também

- opendir() - Abre um manipulador de diretório
- readdir() - Lê entrada do manipulador do diretório
- closedir() - Fecha o manipulador do diretório
- fnmatch() - Compara nome de arquivo com um padrão

is_dir

is_dir

(PHP 4, PHP 5, PHP 7, PHP 8)

is_dir — Diz se o caminho é um diretório

Descrição

function **is_dir**(string `$filename`): bool

Informa se o dado nome de arquivo é um diretório.

Parâmetros

`filename`

> Caminho para o arquivo. Se `filename` for um nome de arquivo relativo, será verificado relativamente ao diretório atual de trabalho. Se `filename` for uma ligação simbólica ou ligação rígida então a ligação será resolvida e validado. Se open_basedir estiver habilitado restrições adicionais podem ser aplicáveis.

Valor Retornado

Retorna **`true`** se o nome do arquivo existe e é um diretório, **`false`** caso contrário.

Erros/Exceções

Em caso de falha, um **`E_WARNING`** será emitido.

Examples

Exemplo #1 Exemplo da is_dir()

```php
<?php
var_dump(is_dir('a_file.txt'));
var_dump(is_dir('bogus_dir/abc'));

var_dump(is_dir('..')); // um diretório acima
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(false)
bool(true)
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- chdir() - Muda o diretório
- dir() - Retorna uma instância da classe Directory
- opendir() - Abre um manipulador de diretório
- is_file() - Informa se o arquivo é um arquivo comum
- is_link() - Informa se o arquivo é uma ligação simbólica

is_executable

is_executable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_executable — Diz se um arquivo é executável

Descrição

function **is_executable**(string `$filename`): bool

Diz se um arquivo é executável.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna `true` se o arquivo existe e é executável, ou `false` em caso de erro. Em sistemas POSIX, um arquivo é executável se o bit de executável das permissões de arquivo estiver ligado. Para Windows, veja a nota abaixo.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo da is_executable()

```php
<?php

$file = '/home/vincent/arquivo.sh';

if (is_executable($file)) {
    echo $file.' é executável';
```

```
} else {
    echo $file.' não é executável';
}

?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

> **Nota** :
>
> No Windows, um arquivo é considerado executável se for assim reportado pela função `GetBinaryType()` da API do Windows; por razões de compatibilidade, arquivos com extensões *.bat* ou *.cmd* também são considerados executáveis. Antes do PHP 7.4.0, qualquer arquivo não vazio com extensão *.exe* ou *.com* era considerado executável. Note que *PATHEXT* é irrelevante para a função **is_executable()**.

Veja também

- is_file() - Informa se o arquivo é um arquivo comum
- is_link() - Informa se o arquivo é uma ligação simbólica

is_file

is_file

(PHP 4, PHP 5, PHP 7, PHP 8)

is_file — Informa se o arquivo é um arquivo comum

Descrição

function **is_file**(string `$filename`): bool

Informa se o arquivo é um arquivo comum. Se o parâmetro `filename` apontar para uma ligação simbólica, a função resolverá a ligação e dará informação sobre o arquivo referenciado.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna `true` se o arquivo existe e é um arquivo comum, `false` caso contrário.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo da is_file()

```php
<?php
var_dump(is_file('a_file.txt')) . "\n";
var_dump(is_file('/usr/bin/')) . "\n";
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- is_dir() - Diz se o caminho é um diretório
- is_link() - Informa se o arquivo é uma ligação simbólica
- SplFileInfo

is_link

is_link

(PHP 4, PHP 5, PHP 7, PHP 8)

is_link — Informa se o arquivo é uma ligação simbólica

Descrição

function **is_link**(string `$filename`): bool

Informa se o arquivo informado é uma ligação simbólica (symbolic link).

Parâmetros

`filename`

O caminho para o arquivo.

Valor Retornado

Retorna `true` se o nome do arquivo existe e se é uma ligação simbólica, `false` caso contrário.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Cria e confirma se um arquivo é uma ligação simbólica

```php
<?php
$link = 'uploads';

if (is_link($link)) {
    echo readlink($link);
} else {
    symlink('uploads.php', $link);
}
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- is_dir() - Diz se o caminho é um diretório
- is_file() - Informa se o arquivo é um arquivo comum
- readlink() - Retornar o alvo de uma ligação simbólica
- symlink() - Cria uma ligação simbólica

is_readable

is_readable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_readable — Informa se um arquivo existe e se pode ser lido

Descrição

function **is_readable**(string `$filename`): bool

Informa se um arquivo existe e se pode ser lido.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Retorna `true` se o arquivo ou diretório especificado por `filename` existe e pode ser lido, `false` caso contrário.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de is_readable()

```php
<?php
$filename = 'teste.txt';
if (is_readable($filename)) {
    echo 'O arquivo pode ser lido.';
} else {
    echo 'O arquivo não pode ser lido.';
```

```
}
?>
```

Notas

Lembre-se que o PHP pode acessar o arquivo como o ID de usuário com o qual o servidor web é executado (geralmente 'nobody').

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

> **Nota** :
>
> A verificação é feita usando os UID/GID reais em vez dos efetivos.

Essa função pode retornar `true` para diretórios. Use a função is_dir() para distinguir arquivo de diretório.

Veja também

- is_writable() - Informa se o arquivo pode ser escrito
- file_exists() - Verifica se um arquivo ou diretório existe
- fgets() - Lê uma linha de um ponteiro de arquivo

is_uploaded_file

is_uploaded_file

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

is_uploaded_file — Informa se o arquivo foi enviado por POST HTTP

Descrição

function **is_uploaded_file**(string `$filename`): bool

Retorna `true` se o arquivo com o nome `filename` foi enviado por POST HTTP. Isto é útil para garantir que um usuário malicioso não está tentando levar o script a trabalhar em arquivos que não deveria estar trabalhando -- por exemplo, */etc/passwd*.

Este tipo de confirmação é importante principalmente se existe alguma chance que qualquer coisa feita com os arquivos enviados possa revelar seu conteúdo para o usuário, ou mesmo para outros usuários no mesmo sistema.

Para o funcionamento apropriado, a função **is_uploaded_file()** precisa de um argumento como *$_FILES['userfile']['tmp_name']*, - o nome do arquivo enviado na máquina do cliente *$_FILES['userfile']['name']* não funciona.

Parâmetros

`filename`

O nome do arquivo sendo verificado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de is_uploaded_file()

```
<?php

if (is_uploaded_file($_FILES['userfile']['tmp_name'])) {
```

```
    echo "O arquivo ". $_FILES['userfile']['name'] ." foi enviado com sucesso.\n";
    echo "Mostrando o conteúdo\n";
    readfile($_FILES['userfile']['tmp_name']);
} else {
    echo "Possível ataque de envio de arquivo: ";
    echo "nome do arquivo '". $_FILES['userfile']['tmp_name'] . "'.";
}

?>
```

Veja também

- move_uploaded_file() - Move um arquivo enviado para uma nova localização
- *$_FILES*
- Consulte Gerenciando Envio de Arquivos para um exemplo de uso básico.

is_writable

is_writable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_writable — Informa se o arquivo pode ser escrito

Descrição

function **is_writable**(string `$filename`): bool

Retorna `true` se o arquivo informado por `filename` existe e pode ser escrito. O argumento pode ser um nome de diretório, permitindo a verificação de permissão de escrita em um diretório.

Lembre-se que o acesso ao arquivo pelo PHP pode ser feito com o ID de usuário com o qual o servidor web é executado (geralmente 'nobody').

Parâmetros

`filename`

O arquivo a ser verificado.

Valor Retornado

Retorna `true` se o arquivo em `filename` existe e pode ser escrito.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de is_writable()

```
<?php
$filename = 'teste.txt';
if (is_writable($filename)) {
    echo 'O arquivo possui permissão de escrita';
} else {
    echo 'O arquivo não possui permissão de escrita';
}
?>
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**

> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- is_readable() - Informa se um arquivo existe e se pode ser lido
- file_exists() - Verifica se um arquivo ou diretório existe
- fwrite() - Escrita binary-safe em arquivos

is_writeable

is_writeable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_writeable — Sinônimo de is_writable()

Descrição

Esta função é um apelido para: is_writable().

lchgrp

lchgrp

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

lchgrp — Modifica o grupo de proprietário da ligação simbólica

Descrição

function **lchgrp**(string `$filename`, string|int `$group`): bool

Tenta modificar o grupo da ligação simbólica `filename` para o valor do parâmetro `group`.

Somente o superusuário pode alterar arbitrariamente o grupo de uma ligação simbólica; outros usuários podem alterar o grupo de uma ligação simbólica para qualquer grupo do qual esse usuário seja um membro.

Parâmetros

`filename`

Caminho para a ligação simbólica.

`group`

O grupo especificado por nome ou número.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Alterando um grupo de um ligação simbólica

```php
<?php
$target = 'output.php';
$link = 'output.html';
symlink($target, $link);

lchgrp($link, 8);
?>
```

Notas

> **Nota** :

> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :
>
> Esta função não foi implementada em plataformas Windows.

Veja também

- chgrp() - Altera o grupo do arquivo
- lchown() - Modifica o proprietário da ligação simbólica
- chown() - Modifica o proprietário do arquivo
- chmod() - Modifica as permissões do arquivo

lchown

lchown

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

lchown — Modifica o proprietário da ligação simbólica

Descrição

function **lchown**(string `$filename`, string|int `$user`): bool

Tenta modificar o proprietário da ligação simbólica `filename` para o usuário `user`.

Somente o superusuário pode mudar o proprietário de uma ligação simbólica.

Parâmetros

`filename`

Caminho para o arquivo.

`user`

O nome do usuário ou número.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Alterando o proprietário de uma ligação simbólica

```php
<?php
$target = 'output.php';
$link = 'output.html';
symlink($target, $link);

lchown($link, 8);
?>
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :

> Esta função não foi implementada em plataformas Windows.

Veja também

- chown() - Modifica o proprietário do arquivo
- lchgrp() - Modifica o grupo de proprietário da ligação simbólica
- chgrp() - Altera o grupo do arquivo
- chmod() - Modifica as permissões do arquivo

link

link

(PHP 4, PHP 5, PHP 7, PHP 8)

link — Criando uma ligação rígida

Descrição

function **link**(string `$target`, string `$link`): bool

link() cria uma ligação rígida.

Parâmetros

`target`

Alvo da ligação.

`link`

O nome da ligação.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

A função falha e emite um `E_WARNING` se `link` já existir, ou se `target` não existir.

Examples

Exemplo #1 Criando uma ligação rígida

```php
<?php
$target = 'source.ext'; // Esse é o arquivo existente
$link = 'newfile.ext';  // Esse é o nome da ligação

link($target, $link);
?>
```

Notas

> **Nota** :
>
> Esta função não funcionará em arquivos remotos, pois o arquivo a ser examinado deve ser acessível através do sistema de arquivos do servidor.

> **Nota** :
>
> No Windows, essa função requer o PHP rodando em nível elevado de privilégios ou com o UAC desabilitado.

Veja também

- symlink() - Cria uma ligação simbólica

- readlink() - Retornar o alvo de uma ligação simbólica
- linkinfo() - Obtém informação sobre uma ligação
- unlink() - Apaga um arquivo

linkinfo

linkinfo

(PHP 4, PHP 5, PHP 7, PHP 8)

linkinfo — Obtém informação sobre uma ligação

Descrição

function **linkinfo**(string `$path`): int|false

Obtém informação sobre uma ligação.

Esta função é usada para verificar se uma ligação (apontada por `path`) realmente existe (usando o mesmo método da macro S_ISLNK definida em *stat.h*).

Parâmetros

`path`

Caminho para a ligação.

Valor Retornado

linkinfo() retorna o campo `st_dev` da estrutura stat do Unix C retornado pela chamada de sistema `lstat`. Retorna um inteiro não negativo em caso de sucesso, -1 caso a ligação não seja encontrada, ou **`false`** se uma violação de open.base_dir ocorrer.

Examples

Exemplo #1 Exemplo de linkinfo()

```php
<?php

echo linkinfo('/vmlinuz'); // 835

?>
```

Veja também

- symlink() - Cria uma ligação simbólica
- link() - Criando uma ligação rígida
- readlink() - Retornar o alvo de uma ligação simbólica

lstat

lstat

(PHP 4, PHP 5, PHP 7, PHP 8)

lstat — Obtém informações sobre um arquivo ou ligação simbólica

Descrição

function **lstat**(string `$filename`): array|false

Obtém estatísticas do arquivo ou ligação simbólica com o nome de `filename`.

Parâmetros

`filename`

Caminho para um arquivo ou ligação simbólica.

Valor Retornado

Veja a página do manual para stat() para informações sobre a estrutura do array retornado que **lstat()** retorna. Esta função é idêntica à stat() exceto que se o parâmetro `filename` é uma ligação simbólica, e o status da ligação simbólica é retornado e não o status do arquivo apontado pela ligação simbólica.

Em caso de falhas retorna `false`.

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Examples

Exemplo #1 Comparação de stat() e lstat()

```
<?php
symlink('uploads.php', 'uploads');

// Compara a informação de uploads.php e uploads
array_diff(stat('uploads'), lstat('uploads'));
?>
```

O exemplo acima produzirá algo semelhante a:

Informação que difere entre os dois arquivos:

```
Array
(
    [ino] => 97236376
    [mode] => 33188
    [size] => 34
    [atime] => 1223580003
    [mtime] => 1223581848
    [ctime] => 1223581848
    [blocks] => 8
)
```

Notas

> **Nota** :
>
> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções stat().

Veja também

- stat() - Obtém informações sobre um arquivo

mkdir

mkdir

(PHP 4, PHP 5, PHP 7, PHP 8)

mkdir — Cria um diretório

Descrição

```
function mkdir(
    string $directory,
    int $permissions = 0777,
    bool $recursive = false,
    ?resource $context = null
): bool
```

Tenta criar o diretório especificado pelo parâmetro `directory`.

Parâmetros

`directory`

O caminho do diretório.

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

`permissions`

A permissão padrão é o octal 0777, que significa o acesso mais abrangente possível. Para mais informações sobre permissões, leia os detalhes na página da função chmod().

> **Nota** :
>
> O parâmetro `permissions` é ignorado no Windows.

Note que provavelmente será desejado especificar `permissions` como um número octal, o que significa que ele deve ser precedido por zero. `permissions` também é modificado inicial. O mode é também modificado pela umaks atual, pela umask atual, que pode ser modificada usando umask().

`recursive`

Se for `true`, quaisquer diretórios acima do especificado em `directory` serão criados, com as mesmas permissões.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

> **Nota** :
>
> Se o diretório a ser criado já existir, será considerado um erro e `false` será retornado. Use is_dir() ou file_exists() para verificar se o diretório já existe antes de tentar criá-lo.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o diretório já existir.

Emite um erro de nível `E_WARNING` se as permissões relevantes impedem a criação do diretório.

Examples

Exemplo #1 Exemplo de mkdir()

```php
<?php
mkdir("/caminho/para/meu/diretorio", 0700);
?>
```

Exemplo #2 mkdir() usando o parâmetro `recursive`

```php
<?php
// Estrutura de diretório desejada
$structure = './profundidade1/profundidade2/profundidade3/';

// Para criar a estrutura aninhada, o parâmetro $recursive
// para mkdir() deve ser especificado.

if (!mkdir($structure, 0777, true)) {
    die('Falha ao criar os diretórios...');
}

// ...
?>
```

Veja também

- is_dir() - Diz se o caminho é um diretório
- rmdir() - Remove um diretório
- umask() - Modificar a umask atual

move_uploaded_file

move_uploaded_file

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

move_uploaded_file — Move um arquivo enviado para uma nova localização

Descrição

function **move_uploaded_file**(string `$from`, string `$to`): bool

Esta função faz uma verificação para garantir que o arquivo designado por `from` é um arquivo de envio válido (que tenha sido enviado pelo mecanismo de envio POST HTTP do PHP). Se o arquivo for válido, ele será movido para o nome de arquivo dado por `to`.

Este tipo de verificação é especialmente imporante se existir alguma chance de que qualquer coisa feita com os arquivos enviados possa revelar seus conteúdos ao usuário, ou mesmo para outros usuários no mesmo sistema.

Parâmetros

`from`

O nome do arquivo enviado.

`to`

O destino do arquivo movido.

Valor Retornado

Retorna `true` em caso de sucesso.

Se `from` não for um arquivo enviado válido, nenhuma ação ocorrerá e **move_uploaded_file()** retornará `false`.

Se `from` for um arquivo enviado válido, mas não puder ser movido por alguma razão, nenhuma ação ocorrerá e **move_uploaded_file()** retornará `false`. Adicionalmente, um alerta será emitido.

Examples

Exemplo #1 Enviando múltiplos arquivos

```php
<?php
$uploads_dir = '/uploads';
foreach ($_FILES["pictures"]["error"] as $key => $error) {
    if ($error == UPLOAD_ERR_OK) {
        $tmp_name = $_FILES["pictures"]["tmp_name"][$key];
        // basename() pode evitar ataques no sistema de arquivos;
        // valização e sanitização adicionais no nome do arquivo podem ser necessárias
```

```
        $name = basename($_FILES["pictures"]["name"][$key]);
        move_uploaded_file($tmp_name, "$uploads_dir/$name");
    }
}
?>
```

Notas

> **Nota** :
>
> **move_uploaded_file()** é afetada por open_basedir. Entretanto, restrições são aplicadas apenas no caminho de `to` para permitir a movimentação de arquivos enviados nos quais `from` possa conflitar com tais restrições. **move_uploaded_file()** assegura a segurança desta operação permitindo que apenas arquivos enviados através do PHP possam ser movidos.

> **Aviso**
>
> Se o arquivo de destino já existir, ele será sobrescrito.

Veja também

- is_uploaded_file() - Informa se o arquivo foi enviado por POST HTTP
- rename() - Renomeia um arquivo ou diretório
- Consulteja Gerenciando envio de arquivos para um exemplo de uso básico

parse_ini_file

parse_ini_file

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_ini_file — Interpreta um arquivo de configuração

Descrição

function **parse_ini_file**(string `$filename`, bool `$process_sections` = **`false`**, int `$scanner_mode` = **`INI_SCANNER_NORMAL`**): array|false

parse_ini_file() carrega o arquivo INI especificado em `filename` e retorna as configurações contidas nele em um array associativo.

A estrutura do arquivo INI é a mesma do *php.ini*.

> **Aviso**
>
> Esta função não deve ser usada com entradas não confiáveis a menos que `scanner_mode` seja igual a **`INI_SCANNER_RAW`**, já que a saída analisada pode conter valores de constantes sensíveis como as que contêm senhas de banco de dados.

Parâmetros

`filename`

O nome do arquivo INI sendo interpretado. Se um caminho relativo for usado, será avaliado relativamente ao diretório atual de trabalho, e depois no caminho include_path.

`process_sections`

Definindo o parâmetro `process_sections` para **`true`**, obtém-se um array multidimensional com os nomes das seções e configurações incluídos. O padrão para `process_sections` é **`false`**

`scanner_mode`

Pode ser **`INI_SCANNER_NORMAL`** (padrão) ou **`INI_SCANNER_RAW`**. Se **`INI_SCANNER_RAW`** for fornecido, os valores das opções não serão interpretados.

A partir do PHP 5.6.1, também pode ser especificado como **`INI_SCANNER_TYPED`**. Nesse modo, os tipos boolean, null e integer são preservados quando possível. Os valores string `"true"`, `"on"` e `"yes"` são convertidos em **`true`**. `"false"`, `"off"`, `"no"` e `"none"` são considerados **`false`**. `"null"` é convertido para **`null`** no modo tipado. Além disso, todas as strings numéricas são convertidas para o tipo inteiro, se possível.

Valor Retornado

As configurações são retornadas como um array associativo em caso de sucesso, e `false` em caso de falha.

Examples

Exemplo #1 Conteúdo de *sample.ini*

```
; Este é um arquivo de configuração de exemplo
; Comentários iniciam com ';', como no php.ini

[primeira_secao]
um = 1
cinco = 5
animal = PASSARO

[segunda_secao]
path = "/usr/local/bin"
URL = "http://www.example.com/~username"

[terceira_secao]
versaophp[] = "5.0"
versaophp[] = "5.1"
versaophp[] = "5.2"
versaophp[] = "5.3"

urls[svn] = "http://svn.php.net"
urls[git] = "http://git.php.net"
```

Exemplo #2 Exemplo de parse_ini_file()

Constantes (mas não "constantes mágicas" como `__FILE__`) também podem ser interpretadas no arquivo INI. Se for definida uma constante como um valor INI antes de executar **parse_ini_file()**, ela será intergrada aos resultados. Somente valores INI são avaliados e o valor deve ser apenas a constante. Por exemplo:

```
<?php

define('PASSARO', 'Pássaro Dodo');

// Interpreta sem as seções
$ini_array = parse_ini_file("sample.ini");
print_r($ini_array);

// Interpreta com as seções
$ini_array = parse_ini_file("sample.ini", true);
print_r($ini_array);

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [um] => 1
    [cinco] => 5
    [animal] => Pássaro Dodo
    [path] => /usr/local/bin
    [URL] => http://www.example.com/~username
    [versaophp] => Array
        (
            [0] => 5.0
            [1] => 5.1
            [2] => 5.2
            [3] => 5.3
        )

    [urls] => Array
        (
```

```
            [svn] => http://svn.php.net
            [git] => http://git.php.net
        )

)
Array
(
    [primeira_secao] => Array
        (
            [um] => 1
            [cinco] => 5
            [animal] = Pássaro Dodo
        )

    [segunda_secao] => Array
        (
            [path] => /usr/local/bin
            [URL] => http://www.example.com/~username
        )

    [terceira_secao] => Array
        (
            [versaophp] => Array
                (
                    [0] => 5.0
                    [1] => 5.1
                    [2] => 5.2
                    [3] => 5.3
                )

            [urls] => Array
                (
                    [svn] => http://svn.php.net
                    [git] => http://git.php.net
                )

        )

)
```

Exemplo #3 parse_ini_file() interpretando um arquivo php.ini

```php
<?php
// Uma função simples usada para comparar os resultados abaixo
function sim_nao($expression)
{
    return($expression ? 'Sim' : 'Não');
}

// Obtém o caminho para o php.ini usando a função php_ini_loaded_file()
$ini_path = php_ini_loaded_file();

// Interpreta o php.ini
$ini = parse_ini_file($ini_path);

// Mostra e compara os valores, note que usar get_cfg_var()
// levará aos mesmos resultados para interpretado e carregado aqui
echo '(interpretado) magic_quotes_gpc = ' . sim_nao($ini['magic_quotes_gpc']) . PHP_EOL;
echo '(carregado) magic_quotes_gpc = ' . sim_nao(get_cfg_var('magic_quotes_gpc')) . PHP_EOL;
?>
```

O exemplo acima produzirá algo semelhante a:

```
(parsed) magic_quotes_gpc = Sim
(loaded) magic_quotes_gpc = Sim
```

Exemplo #4 Interpolação de Valor

Além de avaliar constantes, alguns caracteres possuem significado especial em um valor INI. Adicionamente, variáveis de ambiente e opções de configuração previamente definidas (consulte get_cfg_var()) podem ser lidas usando a sintaxe `${}`.

```
; | é usado par OR bit a bit
tres = 2|3

; & é usado para AND bit a bit
quatro = 6&5

; ^ é usado para XOR bit a bit
cinco = 3^6

; ~ é usado para inversão bit a bit
dois_negativo = ~1

; () é usado para agrupamento
sete = (8|7)&(6|5)

; Interpola a variável de ambiente PATH
path = ${PATH}

; Interpola a opção de configuração 'memory_limit'
limite_de_memoria_configurado = ${memory_limit}
```

Exemplo #5 Fazendo Escape de Caracteres

Alguns caracteres têm significado especial em strings com aspas duplas e devem ser escapados pela barra invertida prefixada. Primeiramente, eles são as aspas duplas `"` como marcação dos limites, e a barra invertida `\` em si (se seguida por um dos caracteres especiais):

```
fala = "Ela disse \"Exatamente meu ponto\"." ; Resulta em uma string contendo aspas duplas.
dica = "Use \\\" para escapar aspas duplas" ; Resulta em: Use \" para escapar aspas duplas
```

Há uma excecão feita para caminhos do tipo Windows: é possível não escapar barra invertida no final se a string entre aspas for diretamente seguida por uma quebra de linha:

```
save_path = "C:\Temp\"
```

Se for necessário escapar aspas duplas seguidas pode quebra de linha em um valor com várias linhas, é possível usar concatenação de valores da seguinte forma (há uma string com aspas duplas diretamente seguida por outra):

```
long_text = "Lorem \"ipsum\""""
 dolor" ; Resulta em: Lorem "ipsum"\n dolor
```

Outro caractere com significado especial é `$` (sinal de cifrão). Deve ser escapado se for seguido de chave:

```
code = "\${test}"
```

Escapar caracteres não é suportado no modo **`INI_SCANNER_RAW`** (neste modo todos os caracateres são processados como eles são).

Note que o interpretador INI não suporta sequências escape padrões (`\n`, `\t` etc.). Se necessário, faça pós processamento do resultado de **parse_ini_file()** com a função stripcslashes().

Notas

> **Nota** :
>
> Essa função não tem relação nenhuma com o arquivo *php.ini*. Ele já está processado no momento em o script é executado. Esta função pode ser usada para ler os arquivos de configuração da própria aplicação do usuário.

> **Nota** :
>
> Se um valor no arquivo INI tiver algum caractere não alfanumérico, ele precisará ser envolvido em aspas duplas (").

> **Nota** :
>
> Existem algumas palavras reservadas que não podem ser usadas como chaves em arquivos INI. Elas incluem: `null`, `yes`, `no`, `true`, `false`, `on`, `off` e `none`. Valores `null`, `off`, `no` e `false` resultam em `""`, e os valores `on`, `yes` e `true` resultam em `"1"`, a menos que o modo `INI_SCANNER_TYPED` seja usado. Os caracteres `?{}|&~!()^"` não devem ser usados em nenhum lugar da chave e têm um significado especial no valor.

> **Nota** :
>
> Entradas sem um sinal de igualdade são ignoradas. Por exemplo, "foo" é ignorado enquanto que "bar =" é interpretado e adicionado com um valor vazio. Por exemplo, o MySQL tem uma configuração "no-auto-rehash" no arquivo *my.cnf* que não leva um valor, portanto ela é ignorada.

> **Nota** :
>
> Arquivos INI são feralmente tratados como texto puro por servidores web e depois enviados aos navegadores se requisitados. Isto quer dizer que para segurança deve-se manter os arquivos INI fora da raiz de documentos da web ou reconfigurar seu servidor web para não disponibilizá-los. Falha em fazer isso pode introduzir um risco de segurança.

Veja também

- parse_ini_string() - Interpreta uma string de configuração

parse_ini_string

parse_ini_string

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

parse_ini_string — Interpreta uma string de configuração

Descrição

function **parse_ini_string**(string `$ini_string`, bool `$process_sections = false`, int `$scanner_mode = INI_SCANNER_NORMAL`): array|false

parse_ini_string() retorna as configurações da string `ini_string` em um array associativo.

A estrutura da string INI é a mesma dos *php.ini*'s.

> **Aviso**
>
> Esta função não deve ser usada com entradas não confiáveis a menos que `scanner_mode` seja igual a `INI_SCANNER_RAW`, já que a saída analisada pode conter valores de constantes sensíveis como as que contêm senhas de banco de dados.

Parâmetros

`ini_string`

O conteúdo do arquivo ini sendo interpretado.

`process_sections`

Definindo o último parâmetro `process_sections` como `true`, você recupera um array multidimensional com o nome das seções e configurações inclusas. O padrão para `process_sections` é `false`

`scanner_mode`

O parâmetro pode ser **INI_SCANNER_NORMAL** (default) ou **INI_SCANNER_RAW**. Se **INI_SCANNER_RAW** for fornecido, então os valores das opçoes não serão analisados.

A partir do PHP 5.6.1, também pode ser especificado como **INI_SCANNER_TYPED**. Nesse modo, os tipos boolean, null e integer são preservados quando possível. Os valores string "true", "on" e "yes" são convertidos em **true**. "false", "off", "no" e "none" são considerados **false**. "null" é convertido para **null** no modo tipado. Além disso, todas as strings numéricas são convertidas para o tipo inteiro, se possível.

Valor Retornado

As configurações são retornadas como um array associativo, se sucesso e **false** caso falhe.

Notas

> **Nota** :
>
> Existem algumas palavras reservadas que não podem ser usadas como chaves em arquivos INI. Elas incluem: null, yes, no, true e false, on, off, none. Valores null, off, no e false retornam como "", e valores on, yes e true resultam em "1", a menos que o modo **INI_SCANNER_TYPED** seja usado. Os caracteres ?{}|&~![()^" não podem ser usados em lugar algum na chave e têm significado especial no valor.

Veja também

- parse_ini_file() - Interpreta um arquivo de configuração

pathinfo

pathinfo

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

pathinfo — Retorna informações sobre um caminho de arquivo

Descrição

function **pathinfo**(string $path, int $flags = **PATHINFO_ALL**): array|string

pathinfo() retorna informações sobre o caminho em path: ou um array associativo ou uma string, dependendo das opções em flags.

> **Nota** :
>
> Para informação sobre obtenção de informação sobre o caminho atual, leia a seção sobre variáveis reservadas predefinidas.

> **Nota** :
>
> **pathinfo()** opera ingenuamente na string de entrada, e não tem ciência sobre o sistema de arquivos real, ou componentes de caminho como "..".

> **Nota** :
>
> Apenas nos sistemas Windows, o caractere \ será interpretado como um separador de diretório. Em outros sistemas será tratado como qualquer outro caractere.

> **Cuidado**
>
> **pathinfo()** é afetado pela localidade, então para que seja interpretado corretamente um caminho contendo caracteres multibyte, a localização correspondente deve ser definida pela função setlocale().

Parâmetros

path

O caminho a ser analisado.

`flags`

Se presente, define o elemento expecífico a ser retornado; pode ser **PATHINFO_DIRNAME**, **PATHINFO_BASENAME**, **PATHINFO_EXTENSION** ou **PATHINFO_FILENAME**.

Se `flags` não for especificado, retorna todos os elementos disponíveis.

Valor Retornado

Se o parâmetro `flags` não for passado, um array associativo contendo os seguintes elementos é retornado: `dirname`, `basename`, `extension` (se houver), e `filename`.

> **Nota** :
>
> Se `path` tiver mais de uma extensão, **PATHINFO_EXTENSION** retorna apenas a última e **PATHINFO_FILENAME** apenas remove a última. (veja o primeiro exemplo abaixo).

> **Nota** :
>
> Se `path` não tiver uma extensão, nenhum elemento `extension` será retornado (veja o segundo exemplo abaixo).

> **Nota** :
>
> Se o `basename` do parâmetro `path` começa com um ponto, os seguintes caracteres são interpretados como `extension`, e o `filename` estará vazio (veja o terceiro exemplo abaixo).

Se `flags` estiver presente, retorna uma string contendo o elemento solicitado.

Examples

Exemplo #1 Exemplo de pathinfo()

```php
<?php
$path_parts = pathinfo('/www/htdocs/inc/lib.inc.php');

echo $path_parts['dirname'], "\n";
echo $path_parts['basename'], "\n";
echo $path_parts['extension'], "\n";
echo $path_parts['filename'], "\n";
?>
```

O exemplo acima produzirá:

```
/www/htdocs/inc
lib.inc.php
php
lib.inc
```

Exemplo #2 Exemplo de pathinfo() mostrando a diferença entre null e nenhuma extensão

```php
<?php
$path_parts = pathinfo('/path/emptyextension.');
var_dump($path_parts['extension']);

$path_parts = pathinfo('/path/noextension');
```

```php
var_dump($path_parts['extension']);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(0) ""

Notice: Undefined index: extension in test.php on line 6
NULL
```

Exemplo #3 Exemplo de pathinfo() para um arquivo iniciado por ponto

```php
<?php
print_r(pathinfo('/some/path/.test'));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [dirname] => /some/path
    [basename] => .test
    [extension] => test
    [filename] =>
)
```

Exemplo #4 Exemplo de pathinfo() com desconstrução de array

O parâmetro `flags` não é uma máscara de bits. Apenas um único valor pode ser fornecido. Para selecionar apenas um conjunto limitado de valores interpretados, use desconstrução de array desta forma:

```php
<?php
['basename' => $basename, 'dirname' => $dirname] = pathinfo('/www/htdocs/inc/lib.inc.php');

var_dump($basename, $dirname);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(11) "lib.inc.php"
string(15) "/www/htdocs/inc"
```

Veja também

- dirname() - Retorna o caminho para o diretório pai
- basename() - Retorna a última parte do caminho
- parse_url() - Analisa uma URL e retorna seus componentes
- realpath() - Retorna o path absoluto canonizado

pclose

pclose

(PHP 4, PHP 5, PHP 7, PHP 8)

pclose — Fecha o ponteiro de arquivo de processo

Descrição

function **pclose**(resource `$handle`): int

Fecha um ponteiro de arquivo a um tubo aberto por popen().

Parâmetros

`handle`

O ponteiro de arquivo deve ser válido e deve ter sido retornado por uma chamada bem sucedida à função popen().

Valor Retornado

Retorna o estado da finalização do processo que estava em execução. Em caso de erro, `-1` é retornado.

> **Nota** :
>
> Se o PHP foi compilado com --enable-sigchild, o valor retornado por esta função é indefinido.

Examples

Exemplo #1 Exemplo de pclose()

```php
<?php
$handle = popen('/bin/ls', 'r');
pclose($handle);
?>
```

Notas

> **Nota** : **Somente Unix:**
>
> **pclose()** é implementada internamente usando a chamada de sistema `waitpid(3)`. Para obter o código real do estado de saída, a função pcntl_wexitstatus() deve ser usada.

Veja também

- popen() - Abre um processo como ponteiro de arquivo

popen

popen

(PHP 4, PHP 5, PHP 7, PHP 8)

popen — Abre um processo como ponteiro de arquivo

Descrição

function **popen**(string `$command`, string `$mode`): resource|false

Abre uma pipe para um processo executado ao se dar um fork para o comando dado por `command`.

Parâmetros

`command`

O comando

`mode`

O modo. Pode ser `'r'` para leitura ou `'w'` para escrita.

No Windows, o padrão de **popen()** é modo texto, isto é, qualquer caractere `\n` escrito ou lido do tubo será traduzido para `\r\n`. Se isto não for desejado, o modo binário pode ser forçado definindo-se o `mode` para `'rb'` e `'wb'`, respectivamente.

Valor Retornado

Retorna um ponteiro de arquivo idêntico ao retornado por fopen(), exceto que ele é unidirecional (pode ser usado somente para leitura ou gravação) e tem que ser fechado com pclose(). Este ponteiro pode ser usado com fgets(), fgetss() e fwrite(). Quando o modo é 'r', o ponteiro de arquivo retornado é igual ao STDOUT do comando; quando o modo é 'w', o ponteiro de arquivo é igual ao STDIN do comando.

Se um erro ocorrer, retorna `false`.

Examples

Exemplo #1 Exemplo de popen()

```php
<?php
$handle = popen("/bin/ls", "r");
?>
```

Se o comando a ser executado não for encontrado, um resource válido é retornado. Isso pode parecer estranho, mas tem sentido: permite que você acesse qualquer mensagem de erro retornado pelo sistema operacional.

Exemplo #2 Exemplo da popen()

```php
<?php
error_reporting(E_ALL);

/* Adiciona redirecionamento para que seja obtido stderr. */
$handle = popen('/caminho/para/executavel 2>&1', 'r');
echo "'$handle'; " . gettype($handle) . "\n";
$read = fread($handle, 2096);
echo $read;
pclose($handle);
?>
```

Notas

> **Nota** :
>
> Se for necessário suporte bidirecional (duas mãos), use proc_open().

Veja também

- pclose() - Fecha o ponteiro de arquivo de processo
- fopen() - Abre um arquivo ou URL
- proc_open() - Executa um comando e abre ponteiros de arquivo para entrada/saída

readfile

readfile

(PHP 4, PHP 5, PHP 7, PHP 8)

readfile — Exibe o conteúdo de um arquivo

Descrição

function **readfile**(string `$filename`, bool `$use_include_path = false`, ?resource `$context = null`): int|false

Lê um arquivo e escreve o seu conteúdo para o buffer de saída.

Parâmetros

`filename`

O nome do arquivo a ser lido.

`use_include_path`

Pode-se usar o segundo parâmetro opcional e configurá-lo para **`true`**, se for desejado procurar o arquivo no include_path também.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna o número de bytes lidos do arquivo em caso de sucesso, ou **`false`** em caso de falha

Erros/Exceções

Em caso de falha, um **`E_WARNING`** será emitido.

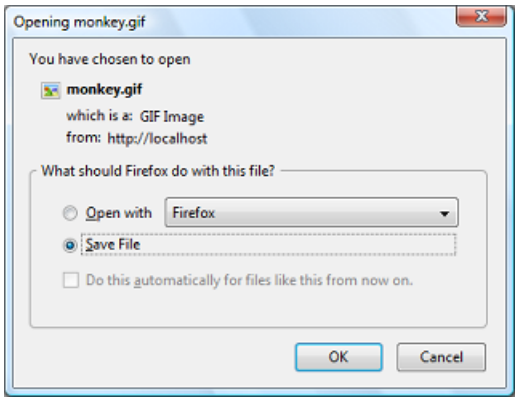
Examples

Exemplo #1 Forçando um recebimento usando readfile()

```php
<?php
$file = 'monkey.gif';

if (file_exists($file)) {
    header('Content-Description: File Transfer');
    header('Content-Type: application/octet-stream');
    header('Content-Disposition: attachment; filename="'.basename($file).'"');
    header('Expires: 0');
    header('Cache-Control: must-revalidate');
    header('Pragma: public');
    header('Content-Length: ' . filesize($file));
    readfile($file);
    exit;
}
?>
```

O exemplo acima produzirá algo semelhante a:



Notas

> **Nota** :
>
> **readfile()** não apresentará por si só nenhum problema de memória, mesmo no envio de grandes arquivos. Se for encontrado um erro de falta de memória, certifique-se que o buffer de saída está desligado com ob_get_level().

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

Veja também

- fpassthru() - Imprime todos os dados restantes de um ponteiro de arquivo
- file() - Lê todo o arquivo para um array
- fopen() - Abre um arquivo ou URL
- include - include
- require - require
- virtual() - Realiza uma sub-requisição Apache
- file_get_contents() - Lê todo o conteúdo de um arquivo para uma string
- Protocolos e empacotadores suportados

readlink

readlink

(PHP 4, PHP 5, PHP 7, PHP 8)

readlink — Retornar o alvo de uma ligação simbólica

Descrição

function **readlink**(string `$path`): string|false

readlink() faz o mesmo que a função do C.

Parâmetros

`path`

O caminho para a ligação simbólica.

Valor Retornado

Retorna o conteúdo do caminho da ligação simbólica ou `false` em erro.

> **Nota** :
>
> Esta função falha se `path` não é um symlink, exceto no Windows, onde o caminho normalizado é retornado.

Examples

Exemplo #1 Exemplo de readlink()

```php
<?php

// Exibe algo como /boot/vmlinux-2.4.20-xfs
echo readlink('/vmlinuz');

?>
```

Veja também

- is_link() - Informa se o arquivo é uma ligação simbólica
- symlink() - Cria uma ligação simbólica
- linkinfo() - Obtém informação sobre uma ligação

realpath

realpath

(PHP 4, PHP 5, PHP 7, PHP 8)

realpath — Retorna o path absoluto canonizado

Descrição

function **realpath**(string `$path`): string|false

realpath() expande todas as ligações simbólicas e resolve referências para `/./`, `/../` e caracteres `/` extras no caminho `path`, e retorna o caminho absoluto canonizado.

Parâmetros

`path`

O caminho a ser verificado.

> **Nota** :
>
> Ainda que caminho precise ser fornecido, o valor pode ser uma string vazia. Neste caso, o valor é interpretado como o diretório atual.

Valor Retornado

Retorna o caminho absouto canonizado em caso de sucesso. O caminho resultante não conterá nenhuma ligação simbólica ou componentes `/./` e `/../`. Delimitadores no final da string, como `\` e `/`, também são removidos.

realpath() retorna `false` em caso de falha, por exemplo, se o caminho não existir.

> **Nota** :
>
> O script em execução deve ter permissões de executável em todos os diretórios na hierarquia, caso contrário **realpath()** retornará `false`.

> **Nota** :
>
> Para sistemas de arquivos insensíveis a maiúsculas/minúsculas, **realpath()** pode ou não normalizar a forma maiúscula/minúscula do caractere.

> **Nota** :
>
> A função **realpath()** não funcionará para um arquivo que estiver dentro de um Phar pois este caminho será virtual, e não real.

> **Nota** :
>
> No Windows, junções e ligações simbólicas a diretórios somente são expandidas em um nível.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Examples

Exemplo #1 Exemplo de realpath()

```php
<?php
chdir('/var/www/');
echo realpath('./../../etc/passwd') . PHP_EOL;

echo realpath('/tmp/') . PHP_EOL;
?>
```

O exemplo acima produzirá:

```
```

```
/etc/passwd
```

Exemplo #2 realpath() no Windows

No Windows, **realpath()** modificará o estilo Unix de diretórios para o estilo Windows.

```php
<?php
echo realpath('/windows/system32'), PHP_EOL;

echo realpath('C:\Program Files\\'), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
C:\WINDOWS\System32
C:\Program Files
```

Veja também

- basename() - Retorna a última parte do caminho
- dirname() - Retorna o caminho para o diretório pai
- pathinfo() - Retorna informações sobre um caminho de arquivo

realpath_cache_get

realpath_cache_get

(PHP 5 >= 5.3.2, PHP 7, PHP 8)

realpath_cache_get — Obter entradas de cache do caminho absoluto

Descrição

function **realpath_cache_get**(): array

Obtém o conteúdo do cache do caminho absoluto.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com as entradas do caminho absoluto. As chaves são o caminho absoluto, e os valores são arrays de informações dos dados, contendo o caminho absoluto, data de expiração, e outras informações mantidas no cache.

Examples

Exemplo #1 Exemplo de realpath_cache_get()

```php
<?php
var_dump(realpath_cache_get());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  ["/test"]=>
  array(4) {
    ["key"]=>
    int(123456789)
    ["is_dir"]=>
    bool(true)
    ["realpath"]=>
    string(5) "/test"
    ["expires"]=>
    int(1260318939)
  }
  ["/test/test.php"]=>
  array(4) {
    ["key"]=>
    int(987654321)
    ["is_dir"]=>
    bool(false)
    ["realpath"]=>
    string(12) "/root/test.php"
    ["expires"]=>
    int(1260318939)
  }
}
```

Veja também

- realpath_cache_size() - Obter o tamanho do cache do caminho absoluto

realpath_cache_size

realpath_cache_size

(PHP 5 >= 5.3.2, PHP 7, PHP 8)

realpath_cache_size — Obter o tamanho do cache do caminho absoluto

Descrição

function **realpath_cache_size**(): int

Obtém a quantidade de memória utilizada pelo cache do caminho absoluto.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna quanta memória o cache do caminho absoluto está utilizando.

Examples

Exemplo #1 Exemplo de realpath_cache_size()

```php
<?php
var_dump(realpath_cache_size());
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(412)
```

Veja também

- realpath_cache_get() - Obter entradas de cache do caminho absoluto
- As opções de configurações de realpath_cache_size

rename

rename

(PHP 4, PHP 5, PHP 7, PHP 8)

rename — Renomeia um arquivo ou diretório

Descrição

function **rename**(string `$from`, string `$to`, ?resource `$context` = `null`): bool

Tenta renomear `from` para `to`, movendo entre diretórios se necessário. Se o arquivo `to` já existir, ele será sobrescrito. Se renomeando um diretório e `to` já existir, esta função emite um warning.

Parâmetros

`from`

O nome anterior.

> **Nota** :
>
> O empacotador utilizado em `from` *precisa* ser o mesmo empacotador utilizado em `to`.

`to`

O novo nome.

> **Nota** :
>
> No Windows, se `to` já existir, ele precisa ter permissão de escrita. Se não a função **rename()** falhará e emitirá um `E_WARNING`.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo da rename()

```
<?php
rename("/tmp/tmp_file.txt", "/home/user/login/docs/my_file.txt");
?>
```

Veja também

- copy() - Copia arquivo
- unlink() - Apaga um arquivo
- move_uploaded_file() - Move um arquivo enviado para uma nova localização

rewind

rewind

(PHP 4, PHP 5, PHP 7, PHP 8)

rewind — Retrocede a posição de um ponteiro de arquivos

Descrição

function **rewind**(resource `$stream`): bool

Define o indicador de posição de arquivo de `stream` para o início do fluxo do arquivo.

> **Nota** :
>
> Se o arquivo foi aberto no modo de acréscimo ("a" ou "a+"), qualquer dado que for escrito no arquivo será sempre acrescentado, independente da posição do ponteiro.

Parâmetros

`stream`

O ponteiro de arquivo deve ser válido e deve apontar para um arquivo aberto com sucesso por fopen().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de rewind() sobrescrevendo

```php
<?php
$handle = fopen('output.txt', 'r+');

fwrite($handle, 'Frase realmente longa.');
rewind($handle);
fwrite($handle, 'Foo');
rewind($handle);

echo fread($handle, filesize('output.txt'));

fclose($handle);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Foose realmente longa.
```

Veja também

- fread() - Leitura de arquivo segura para binário
- fseek() - Aponta para uma posição no arquivo
- ftell() - Retorna a posição atual do ponteiro de leitura/gravação do arquivo
- fwrite() - Escrita binary-safe em arquivos

rmdir

rmdir

(PHP 4, PHP 5, PHP 7, PHP 8)

rmdir — Remove um diretório

Descrição

function **rmdir**(string `$directory`, ?resource `$context = null`): bool

Tenta remover o diretório com o nome de `directory`. O diretório deve estar vazio e as permissões relevantes devem autorizar a operação. Um erro de nível `E_WARNING` será gerado em caso de falha.

Parâmetros

`directory`

Caminho para o diretório.

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de rmdir()

```
<?php
if (!is_dir('examples')) {
    mkdir('examples');
}

rmdir('examples');
?>
```

Veja também

- is_dir() - Diz se o caminho é um diretório
- mkdir() - Cria um diretório
- unlink() - Apaga um arquivo

set_file_buffer

set_file_buffer

(PHP 4, PHP 5, PHP 7, PHP 8)

set_file_buffer — Sinônimo de stream_set_write_buffer()

Descrição

Esta função é um apelido para: stream_set_write_buffer().

stat

stat

(PHP 4, PHP 5, PHP 7, PHP 8)

stat — Obtém informações sobre um arquivo

Descrição

function **stat**(string `$filename`): array|false

Coleta estatísticas do arquivo informado em `filename`. Se `filename` for uma ligação simbólica, estatísticas serão do próprio arquivo, não da ligação. Antes do PHP 7.4.0, nas compilações NTS do Windows, as estatísticas `size`, `atime`, `mtime` e `ctime` são da ligação simbólica, neste caso.

lstat() é idêntica à **stat()** exceto que ela foi baseada no estado de ligações simbólicas.

Parâmetros

`filename`

Caminho para o arquivo.

Valor Retornado

Formato de retorno de stat() e fstat()

Numérico	Associativo	Descrição
0	dev	número do dispositivo ***
1	ino	número do inode ***
2	mode	modo de proteção do inode *****
3	nlink	número de ligações
4	uid	id de usuário do proprietário *
5	gid	id de grupo do proprietário *
6	rdev	tipo de dispositivo, se for inode
7	size	tamanho em bytes
8	atime	horário do último acesso (timestamp Unix)
9	mtime	horário da última modificação (timestamp Unix)
10	ctime	hora da última modificação do inode (timestamp Unix)
11	blksize	tamanho do bloco de E/S no sistema de arquivos **
12	blocks	número de blocos de 512 bytes alocados **

* No Windows será sempre `0`.

** Válido apenas em sistemas que suportam o tipo st_blksize type - outros sistemas (ex.: Windows) retornam `-1`.

*** No Windows, a partir do PHP 7.4.0, este é o número de série do volume que contém o arquivo, que é um inteiro de 64-bits *sem sinal*, ou seja, pode sobrecarregar. Anteriormente, era uma representação numérica da letra do dispositivo (ex.: `2` para `C:`) para **stat()**, e `0` para lstat().

**** No Windows, a partir do PHP 7.4.0, este é o identificador associado com o arquivo, que é um inteiro de 64-bits *sem sinal*, ou seja, pode sobrecarregar. Anteriormente, era sempre `0`.

***** No Windows, o bit de permissão de escrita é definido de acordo com o atributo de somente-leitura do arquivo, e o mesmo valor é reportado para todos os usuários, grupos e proprietários. A ACL não é levada em consideração, contrário a is_writable().

O valor de `mode` contém informação lida por várias funções. Quando escrita em octal, começando da direita, os primeiros três dígitos são retornados por chmod(). O dígito seguinte é ignorado pelo PHP. Os dois dígitos seguintes indicam o tipo de arquivo:

Tipos de arquivo para `mode`

`mode` em octal	Significado
`0140000`	socket
`0120000`	ligação
`0100000`	arquivo normal
`0060000`	dispositivo de bloco
`0040000`	diretório
`0020000`	dispositivo de caracteres
`0010000`	fifo

Por exemplo, um arquivo normal poderia ser `0100644` e um diretório poderia ser `0040755`.

Em caso de erro, **stat()** retorna `false`.

> **Nota** :
>
> Como o tipo inteiro do PHP possui sinal e muitas plataformas usam números inteiros de 32 bits, algumas funções do sistema de arquivos podem retornar resultados inesperados para arquivos maiores que 2GB.

Erros/Exceções

Em caso de falha, um `E_WARNING` é emitido.

Registro de Alterações

Versão	Descrição
7.4.0	No Windows, o número do dispositivo agora é o número de série do volume que contém o arquivo, e o número do inode é o identificador associado com o arquivo.
7.4.0	As estatísticas `size`, `atime`, `mtime` e `ctime` de ligações simbólicas agora são sempre aquelas do alvo. Anteriormente não era o caso para compilações NTS do Windows.

Examples

Exemplo #1 Exemplo de stat()

```php
<?php
/* Obtém estatística sobre o arquivo */
$stat = stat('C:\php\php.exe');

/*
 * Mostra o horário de acesso ao arquivo, é o mesmo que
 * chamar fileatime()
 */
echo 'Horário de acesso: ' . $stat['atime'];

/*
 * Mostra o horário de modificação do arquivo, é o mesmo que
 * chamar filemtime()
 */
echo 'Horário de modificação: ' . $stat['mtime'];

/* Mostra o número de dispositivo */
echo 'Número de dispositivo: ' . $stat['dev'];
?>
```

Exemplo #2 Usando informação de stat() junto com a função touch()

```php
<?php
/* Obtém estatística sobre o arquivo */
$stat = stat('C:\php\php.exe');

/* Falhou a obtenção de estatística? */
if (!$stat) {
    echo 'stat() falhou...';
} else {
    /*
     * Queremos o horário de acesso 1 semana
     * depois do atual.
     */
    $atime = $stat['atime'] + 604800;

    /* Toca o arquivo */
    if (!touch('some_file.txt', time(), $atime)) {
        echo 'Falha ao tocar o arquivo...';
    } else {
        echo 'touch() retorno com sucesso...';
    }
}
?>
```

Notas

> **Nota** :
>
> Note que a resolução de tempo pode variar de um sistema de arquivos para outro.

> **Nota** :

> Os resultados desta função são armazenados em cache. Consulte a função clearstatcache() para mais detalhes.

> **Dica**
>
> A partir do PHP 5.0.0, esta função também pode ser usada com *alguns* empacotadores de URL. Consulte os Protocolos e empacotadores suportados para determinar quais empacotadores suportam a família de funções **stat()**.

Veja também

- lstat() - Obtém informações sobre um arquivo ou ligação simbólica
- fstat() - Lê informações sobre um arquivo usando um ponteiro de arquivo aberto
- filemtime() - Obtém o horário de modificação do arquivo
- filegroup() - Lê o grupo do arquivo
- SplFileInfo

symlink

symlink

(PHP 4, PHP 5, PHP 7, PHP 8)

symlink — Cria uma ligação simbólica

Descrição

function **symlink**(string `$target`, string `$link`): bool

symlink() cria uma ligação simbólica ao alvo existente em `target` com o nome especificado em `link`.

Parâmetros

`target`

Alvo da ligação.

`link`

O nome da ligação.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

A função falha e emite um `E_WARNING`, se `link` já existir. No Windows, a função também falha e emite um `E_WARNING` se `target` não existir.

Examples

Exemplo #1 Cria uma ligação simbólica

```php
<?php
$target = 'uploads.php';
$link = 'uploads';
symlink($target, $link);

echo readlink($link);
?>
```

Veja também

- is_link() - Informa se o arquivo é uma ligação simbólica
- link() - Criando uma ligação rígida
- readlink() - Retornar o alvo de uma ligação simbólica
- linkinfo() - Obtém informação sobre uma ligação
- unlink() - Apaga um arquivo

tempnam

tempnam

(PHP 4, PHP 5, PHP 7, PHP 8)

tempnam — Cria arquivo com nome único

Descrição

function **tempnam**(string `$directory`, string `$prefix`): string|false

Cria um arquivo com nome único, com permissões de acesso definidas para 0600, no diretório especificado. Se o diretório não existir ou não puder ser escrito, **tempnam()** pode gerar um arquivo no diretório temporário do sistema e retornar o caminho completo ao arquivo, incluindo seu nome.

Parâmetros

`directory`

O diretório onde o arquivo temporário será criado.

`prefix`

O prefixo do nome do arquivo temporário gerado.

> **Nota** :
>
> Apenas os primeiros 63 caracteres do prefixo são usados, o restante é ignorado. Windows usa apenas os primeiros três caracteres do prefixo.

Valor Retornado

Retorna o novo nome do arquivo temporário (com caminho), ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.4.0	O nome dos arquivos criados por **tempnam()** agora é 13 bytes mais longo. O comprimento total ainda depende da plataforma.
7.1.0	**tempnam()** agora emite um aviso ao cair de volta no diretório temporário do sistema.

Examples

Exemplo #1 Exemplo de tempnam()

```php
<?php
$tmpfname = tempnam ("/tmp", "FOO");

$handle = fopen($tmpfname, "w");
fwrite($handle, "writing to tempfile");
fclose($handle);

// faz alguma coisa

unlink($tmpfname);
?>
```

Notas

> **Nota** :
>
> Se o PHP não conseguir criar um arquivo no diretório informado `directory`, ele cria no diretório padrão do sistema. No sistema de arquivos NTFS isto também acontece se o diretório especificado em `directory` contiver mais que 65534 arquivos.

Veja também

- tmpfile() - Cria um arquivo temporário
- sys_get_temp_dir() - Retorna o caminho do diretório usado para arquivos temporários
- unlink() - Apaga um arquivo

tmpfile

tmpfile

(PHP 4, PHP 5, PHP 7, PHP 8)

tmpfile — Cria um arquivo temporário

Descrição

function **tmpfile**(): resource|false

Cria um arquivo temporário com um nome único em modo de leitura-gravação-binário (w+b) e retorna um manipulador do arquivo.

O arquivo é automaticamente removido quando ele é fechado (por exemplo, ao chamar fclose(), ou quando não há mais referências para o manipulador do arquivo retornado por **tmpfile()**), ou quando o script finalizar.

> **Cuidado**
>
> Se o script terminar inesperadamente, o arquivo temporário pode não ser removido.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o manipulador de arquivo, similar ao que é retornado por fopen(), para o novo arquivo ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo da **tmpfile()**

```php
<?php
$temp = tmpfile();
fwrite($temp, "escrevendo no arquivo temporario");
fseek($temp, 0);
echo fread($temp, 1024);
fclose($temp); // isto remove o arquivo
?>
```

O exemplo acima produzirá:

```
writing to tempfile
```

Veja também

- tempnam() - Cria arquivo com nome único
- sys_get_temp_dir() - Retorna o caminho do diretório usado para arquivos temporários

touch

touch

(PHP 4, PHP 5, PHP 7, PHP 8)

touch — Define horário de acesso e de modificação de arquivo

Descrição

function **touch**(string `$filename`, ?int `$mtime` = `null`, ?int `$atime` = `null`): bool

Tenta mudar os horários de acesso e modificação do arquivo no parâmetro `filename` para o valor informado em `mtime`. Note que o horário de acesso é sempre modificado, independente do número de parâmetros.

Se o arquivo não existe, ele será criado.

Parâmetros

`filename`

O nome do arquivo a ser atualizado.

`mtime`

Horário da atualização. Se `mtime` for `null`, o horário atual do sistema (time()) é usado.

`atime`

Se não `null`, o horário de acesso do arquivo é definido para o valor de `atime`. Caso contrário, é definido para o valor passado ao parâmetro `mtime`. Se ambos forem `null`, o horário atual do sistema é usado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`mtime` e `atime` agora podem ser nulos.

Examples

Exemplo #1 Exemplo da função touch()

```php
<?php
if (touch ($nomedoarquivo)) {
    echo 'O horário de ' . $nomedoarquivo . ' foi modificado para o horário atual';
} else {
    echo 'Desculpe, não foi possivel modificar o horário de ' . $nomedoarquivo;
}
?>
```

Exemplo #2 touch() usando o parâmetro `mtime`

```php
<?php
// Este é o horário de modificação, vamos defini-lo para uma hora no passado.
$hora = time() - 3600;

// Toca o arquivo
if (!touch('algum_arquivo.txt', $hora)) {
    echo 'Opa, algo deu errado...';
} else {
    echo 'Horário modificado com sucesso';
}
?>
```

Notas

> **Nota** :
>
> Note que a resolução de tempo pode variar de um sistema de arquivos para outro.

umask

umask

(PHP 4, PHP 5, PHP 7, PHP 8)

umask — Modificar a umask atual

Descrição

function **umask**(?int `$mask` = `null`): int

umask() modifica o umask do PHP para a `mask` & 0777 e retorna o umask antigo. Quando o PHP está sendo usado como um módulo de servidor, o umask é restaurado ao final de cada requisição.

Parâmetros

`mask`

A nova máscara.

Valor Retornado

Se `mask` for `null`, **umask()** simplesmente retorna a umask atual senão a umask antiga é retornada.

Registro de Alterações

Versão	Descrição
8.0.0	`mask` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da umask()

```php
<?php
$umaskAntigo = umask(0);
chmod("/caminho/diretorio/arquivo.txt", 0755);
umask($umaskAntigo);

// Checking
if ($umaskAntigo != umask()) {
    die('Ocorreu um erro ao voltar para o umask antigo');
}
?>
```

Notas

> **Nota** :
>
> Evite usar esta função em servidores web multithreaded. É melhor modificar as permissões do arquivo com chmod() depois de criar o arquivo. Usando **umask()** pode levar a comportamento inesperado de scripts executados concorrentemente e do próprio webserver porque todos eles usam a mesma umask.

unlink

unlink

(PHP 4, PHP 5, PHP 7, PHP 8)

unlink — Apaga um arquivo

Descrição

function **unlink**(string `$filename`, ?resource `$context` = `null`): bool

Apaga o arquivo `filename`. Similar a função do C no Unix unlink(). Emite um erro `E_WARNING` level no caso de falha.

Parâmetros

`filename`

Caminho para o arquivo.

Se o arquivo for uma ligação simbólica, a ligação será apagada. No Windows, para apagar uma ligação para um diretório, deve ser utilizada a função rmdir().

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.3.0	No Windows, agora é possível realizar o **unlink()** em arquivos abertos, quando antes isso falhava. Entretanto, ainda não é possível de recriar o arquivo apagado, até que o acessos ao arquivo anterior tenham sido fechados.

Examples

Exemplo #1 Exemplo de unlink()

```php
<?php
$fh = fopen('test.html', 'a');
fwrite($fh, '<h1>Hello world!</h1>');
fclose($fh);

unlink('test.html');
?>
```

Veja também

- rmdir() - Remove um diretório

Diretórios

Constantes predefinidas

DIRECTORY_SEPARATOR (string)

PATH_SEPARATOR (string)

Ponto-e-vírgula no Windows, dois-pontos nos demais.

SCANDIR_SORT_ASCENDING (int)

SCANDIR_SORT_DESCENDING (int)

SCANDIR_SORT_NONE (int)

Directory

(PHP 4, PHP 5, PHP 7, PHP 8)

Introdução

Instâncias de **Directory** são criadas chamando-se a função dir(), e não através do operador new.

Resumo da classe

final class **Directory** {
/* Propriedades */
public readonly string *$path*;
public readonly resource *$handle*;
/* Métodos */
public function close(): void
public function read(): string|false
public function rewind(): void
}

Propriedades

path

O diretórioque foi aberto.

handle

Pode ser usada com outras funções de diretório como readdir(), rewinddir() e closedir().

Registro de Alterações

Versão	Descrição
8.5.0	A class agora é final.
8.1.0	As propriedades *path* e *handle* agora são somente-leitura.

Índice

Directory::close

Directory::close

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::close — Fecha o manipulador de diretório

Descrição

public function **Directory::close**(): void

Registro de Alterações

Versão	Descrição
8.0.0	Nenhum parâmetro é aceito. Anteriormente, um manipulador de diretório poderia ser passado como argumento.

Directory::read

Directory::read

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::read — Lê entrada de um manipulador de diretório

Descrição

public function **Directory::read**(): string|false

Registro de Alterações

Versão	Descrição
8.0.0	Nenhum parâmetro é aceito. Anteriormente, um manipulador de diretório poderia ser passado como argumento.

Directory::rewind

Directory::rewind

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::rewind — Retrocede um manipulador de diretório

Descrição

public function **Directory::rewind**(): void

Registro de Alterações

Versão	Descrição
8.0.0	Nenhum parâmetro é aceito. Anteriormente, um manipulador de diretório poderia ser passado como argumento.

Funções de Diretórios

Veja também

Para funções relacionadas como dirname(), is_dir(), mkdir() e rmdir(), consulte a seção Filesystem.

Índice

chdir

chdir

(PHP 4, PHP 5, PHP 7, PHP 8)

chdir — Muda o diretório

Descrição

function **chdir**(string `$directory`): bool

Muda o diretório atual do PHP para `directory`.

Parâmetros

`directory`

O novo diretório atual

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Dispara um aviso `E_WARNING` em caso de falha.

Examples

Exemplo #1 Exemplo de chdir()

```php
<?php

// diretório atual
echo getcwd() . "\n";

chdir('public_html');

// diretório atual
echo getcwd() . "\n";

?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/vincent
/home/vincent/public_html
```

Notas

> **Cuidado**
>
> Se o interpretador PHP foi compilado com ZTS (Zend Thread Safety) habilitado, quaisquer mudanças ao diretório atual através de **chdir()** serão invisíveis ao sistema operacional. Todas as funções internas do PHP ainda irão respeitar a mudança do diretório atual; mas funções de bibliotecas externas, chamadas usando FFI não irão. Pode-se verificar se a cópia do PHP foi compilada com ZTS habilitado usando o comando **php -i** ou a constante interna `PHP_ZTS`.

Veja também

- getcwd() - Obtém o diretório atual

chroot

chroot

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

chroot — Muda o diretório raiz

Descrição

function **chroot**(string `$directory`): bool

Muda o diretório raiz do processo atual para `directory`, e muda o diretório de trabalho atual para "/".

Esta função está disponível somente para sistemas GNU e BSD, e apenas quando CLI, CGI ou Embed SAPI estiverem sendo usados. Além disso, esta função requer privilégio de administrador.

Chamar esta função não muda os valores das constantes mágicas `__DIR__` e `__FILE__`.

Parâmetros

`directory`

O caminho para o qual apontar o novo diretório raiz.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de chroot()

```php
<?php
chroot("/caminho/para/seu/chroot/");
echo getcwd();
?>
```

O exemplo acima produzirá:

```
/
```

Notas

> **Nota** :
>
> Esta função não foi implementada em plataformas Windows.

> **Nota** :
>
> Esta função não está disponível em interpretadores PHP compilados com o recurso ZTS (Zend Thread Safety) habilitado. Para verificar se sua cópia do PHP foi compilada com o ZTS habilitado, use o comando **php -i** ou teste a constante `PHP_ZTS`.

closedir

closedir

(PHP 4, PHP 5, PHP 7, PHP 8)

closedir — Fecha o manipulador do diretório

Descrição

function **closedir**(?resource `$dir_handle = null`): void

Fecha o fluxo de diretório indicado por `dir_handle`.

Parâmetros

`dir_handle`

O manipulador de diretório do tipo resource aberto previamente com opendir(). Se `dir_handle` for `null`, o último manipulador aberto por opendir() será usado.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Usar `null` para `dir_handle` foi descontinuado. Em vez disso, o manipulador do último diretório aberto deve ser fornecido explicitamente.

Versão	Descrição
8.0.0	`dir_handle` agora pode ser nulo.

Examples

Para um exemplo completo, consulte a documentação de opendir().

Veja também

- opendir() - Abre um manipulador de diretório
- readdir() - Lê entrada do manipulador do diretório
- rewinddir() - Retroceder o manipulador de diretório
- dir() - Retorna uma instância da classe Directory
- is_dir() - Diz se o caminho é um diretório
- glob() - Encontra caminhos que combinam com uma expressão
- scandir() - Lista arquivos e diretórios dentro do caminho especificado

dir

dir

(PHP 4, PHP 5, PHP 7, PHP 8)

dir — Retorna uma instância da classe Directory

Descrição

function **dir**(string `$directory`, ?resource `$context = null`): Directory|false

Um mecanismo pseudo-orientado a objeto para ler um diretório. O diretório fornecido pelo parâmetro `directory` é aberto.

Parâmetros

`directory`

Diretório a ser aberto

`context`

Um resource de contexto de fluxo.

Valor Retornado

Retorna uma instância de Directory, ou `false` em caso de erro.

Registro de Alterações

Versão	Descrição
8.0.0	`context` agora pode ser nulo.

Examples

Exemplo #1 Exemplo da dir()

Repare nos detalhes de como o valor retornado de Directory::read() é verificado no exemplo abaixo. É testado explicitamente se o valor de retorno é idêntico a (igual e do mesmo tipo que - consulte operadores de comparação para mais informação) `false`, caso contrário, qualquer entrada de diretório cujo nome seja interpretado como `false` fará parar o loop.

```
<?php
$d = dir("/etc/php5");
echo "Handle: " . $d->handle . "\n";
echo "Caminho: " . $d->path . "\n";
while (false !== ($entry = $d->read())) {
   echo $entry."\n";
}
$d->close();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Handle: Resource id #2
Caminho: /etc/php5
.
..
apache
cgi
cli
```

Notas

> **Nota** :
>
> A ordem em que as entradas do diretório são retornadas pelo método de leitura é dependente do sistema operacional.

getcwd

getcwd

(PHP 4, PHP 5, PHP 7, PHP 8)

getcwd — Obtém o diretório atual

Descrição

function **getcwd**(): string|false

Obtém o diretório atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o diretório atual em caso de sucesso ou `false` em caso de falha.

Em algunmas variantes Unix, **getcwd()** irá retornar `false` se qualquer um dos diretórios-pai não tiver o modo leitura ou pesquisa definidos, mesmo se diretório atual tiver. Consulte chmod() para mais informação sobre modos e permissões.

Examples

Exemplo #1 Exemplo de getcwd()

```php
<?php

// diretório atual
echo getcwd() . "\n";

chdir('cvs');

// diretório atual
echo getcwd() . "\n";

?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/didou
```

```
/home/didou/cvs
```

Notas

> **Cuidado**
>
> Se o interpretador PHP foi compilado com ZTS (Zend Thread Safety) habilitado, o diretório de trabalho atual retornado por **getcwd()** pode ser diferente do retornado pelas interfaces do sistema operacional. Bibliotecas externas (chamadas usando FFI) que dependem do diretório de trabalho atual serão afetadas.

Veja também

- chdir() - Muda o diretório
- chmod() - Modifica as permissões do arquivo

opendir

opendir

(PHP 4, PHP 5, PHP 7, PHP 8)

opendir — Abre um manipulador de diretório

Descrição

function **opendir**(string `$directory`, ?resource `$context = null`): resource|false

Abre um manipulador de diretório para ser usado em chamadas subsequentes de closedir(), readdir() e rewinddir().

Parâmetros

`directory`

O caminho do diretório a ser aberto.

`context`

Para descrição do parâmetro `context`, consulte a seção sobre fluxos do manual.

Valor Retornado

Retorna um manipulador de diretório em caso de sucesso, ou `false` em caso de falha

Erros/Exceções

Em caso de falha, um `E_WARNING` será emitido.

Isto pode ocorrer se `directory` não for um nome de diretório válido, o diretório não puder ser aberto por restrições de permissão, ou por erros no sistema de arquivos.

Registro de Alterações

Versão	Descrição
8.0.0	`context` agora pode ser nulo.

Examples

Exemplo #1 Lista todas as entradas de um diretório, pulando os especiais `.` e `..`

Como os nomes de diretório e arquivo podem ser strings interpretadas como "falso" pelo PHP (por exemplo um diretório com nome `"0"`) e readdir() retorna `false` quando tiver lido todas as entradas em um diretório, deve-se usar o operador de comparação `===` para distinguir adequadamente entre uma entrada de diretório cujo nome seja interpretado como "falso" e a função ter terminado de ler todas as entradas.

```php
<?php

if ($handle = opendir('/path/to/files')) {
    echo "Entradas:\n";

    /* Lidando corretamente com entradas de diretório que podem ser interpretadas como "falso" */
    while (false !== ($entry = readdir($handle))) {
        if ($entry === '.' || $entry === '..') {
            continue;
        }
        echo "$entry\n";
    }

    closedir($handle);
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Entradas:
base
en
fr
output.md
test.php
```

Veja também

- readdir() - Lê entrada do manipulador do diretório
- rewinddir() - Retroceder o manipulador de diretório
- closedir() - Fecha o manipulador do diretório
- dir() - Retorna uma instância da classe Directory
- is_dir() - Diz se o caminho é um diretório
- glob() - Encontra caminhos que combinam com uma expressão
- scandir() - Lista arquivos e diretórios dentro do caminho especificado

readdir

readdir

(PHP 4, PHP 5, PHP 7, PHP 8)

readdir — Lê entrada do manipulador do diretório

Descrição

function **readdir**(?resource `$dir_handle = null`): string|false

Retorna o nome da próxima entrada do diretório. As entradas são retornadas na ordem em que estão armazenadas no sistema de arquivos.

Parâmetros

`dir_handle`

O manipulador de diretório do tipo resource previamente aberto por opendir(). Se `dir_handle` for `null`, o último manipulador aberto por opendir() será usado.

Valor Retornado

Retorna o nome do arquivo em caso de sucesso, ou `false` em caso de falha.

> **Aviso**

> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Registro de Alterações

Versão	Descrição
8.5.0	Usar `null` para `dir_handle` foi descontinuado. Em vez disso, o manipulador do último diretório aberto deve ser fornecido explicitamente.
8.0.0	`dir_handle` agora pode ser nulo.

Examples

Para um exemplo completo, consulte a documentação de opendir().

Veja também

- opendir() - Abre um manipulador de diretório
- rewinddir() - Retroceder o manipulador de diretório
- closedir() - Fecha o manipulador do diretório
- dir() - Retorna uma instância da classe Directory
- is_dir() - Diz se o caminho é um diretório
- glob() - Encontra caminhos que combinam com uma expressão
- scandir() - Lista arquivos e diretórios dentro do caminho especificado

rewinddir

rewinddir

(PHP 4, PHP 5, PHP 7, PHP 8)

rewinddir — Retroceder o manipulador de diretório

Descrição

function **rewinddir**(?resource `$dir_handle = null`): void

Reinicia o fluxo do diretório indicado por `dir_handle` para o início do diretório.

Parâmetros

`dir_handle`

O manipulador de diretório do tipo resource previamente aberto por opendir(). Se `dir_handle` for `null`, o último manipulador aberto por opendir() será usado.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Usar `null` para `dir_handle` foi descontinuado. Em vez disso, o manipulador do último diretório aberto deve ser fornecido explicitamente.
8.0.0	`dir_handle` agora pode ser nulo.

Examples

Para um exemplo completo, consulte a documentação de opendir().

Veja também

- opendir() - Abre um manipulador de diretório
- readdir() - Lê entrada do manipulador do diretório
- closedir() - Fecha o manipulador do diretório
- dir() - Retorna uma instância da classe Directory

- is_dir() - Diz se o caminho é um diretório
- glob() - Encontra caminhos que combinam com uma expressão
- scandir() - Lista arquivos e diretórios dentro do caminho especificado

scandir

scandir

(PHP 5, PHP 7, PHP 8)

scandir — Lista arquivos e diretórios dentro do caminho especificado

Descrição

function **scandir**(string `$directory`, int `$sorting_order` = **`SCANDIR_SORT_ASCENDING`**, ?resource `$context` = **`null`**): array|false

Retorna um array de arquivos e diretórios dentro de `directory`.

Parâmetros

`directory`

O diretório que será pesquisado.

`sorting_order`

Por padrão, a lista está em ordem alfabética crescente. Se o parâmetro opcional `sorting_order` for definido para **`SCANDIR_SORT_DESCENDING`**, a ordenação será alfabética decrescente. Se definida para **`SCANDIR_SORT_NONE`**, o resultado não será ordenado.

`context`

Para uma descrição do parâmetro `context` refira-se à seção de fluxos do manual.

Valor Retornado

Retorna um array com nomes de arquivos em caso de sucesso, ou **`false`** em caso de falha. Se `directory` não for um diretório, **`false`** é retornado e um aviso **`E_WARNING`** é gerado.

Registro de Alterações

Versão	Descrição
8.0.0	`context` agora pode ser nulo.

Examples

Exemplo #1 Um exemplo simples de scandir()

```php
<?php
$dir    = '/tmp';
$files1 = scandir($dir);
$files2 = scandir($dir, 1);

print_r($files1);
print_r($files2);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => .
    [1] => ..
    [2] => bar.php
    [3] => foo.txt
    [4] => algumdiretorio
)
```

```
Array
(
    [0] => algumdiretorio
    [1] => foo.txt
    [2] => bar.php
    [3] => ..
    [4] => .
)
```

Notas

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

Veja também

- opendir() - Abre um manipulador de diretório
- readdir() - Lê entrada do manipulador do diretório
- glob() - Encontra caminhos que combinam com uma expressão
- is_dir() - Diz se o caminho é um diretório
- sort() - Ordena um array em ordem ascendente

Manipulação de variável

Introdução

Para maiores informações do comportamento de variáveis, consulte Variavéis localizada na seção Referências da Linguagem do manual.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração de Variáveis

Nome	Padrão	Modificável	Registro de Alterações
unserialize_callback_func	`null`	`INI_ALL`	
unserialize_max_depth	"4096"	`INI_ALL`	Disponível a partir do PHP 7.4.0.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`unserialize_callback_func` string

A função de retorno é chamada quando a função unserialize() tentar utilizar uma classe indefinida. Um aviso aparecerá se a função de retorno especificada não estiver definida, ou se a função de retorno falhar ao definir a classe ausente.

Veja também unserialize() e Autoloading de Classes.

`unserialize_max_depth` int

A profundidade máxima de estruturas permitida durante a desserialização ao usar unserialize(), com a intenção de evitar sobrecargas de pilha. Este limite pode ser desativado definindo `unserialize_max_depth=0`.

Veja também unserialize() e Autoloading de Classes.

Funções para Manipulação de variáveis

Índice

boolval

boolval

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

boolval — Obtém o valor booleano de uma variável

Descrição

function **boolval**(mixed `$value`): bool

Retorna o valor bool do parâmetro `value`.

Parâmetros

`value`

O valor escalar a ser convertido para um bool.

Valor Retornado

O valor bool do parâmetro`value`.

Examples

Exemplo #1 Exemplos da função boolval()

```php
<?php
echo '0:        '.(boolval(0) ? 'true' : 'false')."\n";
echo '42:       '.(boolval(42) ? 'true' : 'false')."\n";
echo '0.0:      '.(boolval(0.0) ? 'true' : 'false')."\n";
echo '4.2:      '.(boolval(4.2) ? 'true' : 'false')."\n";
echo '"":       '.(boolval("") ? 'true' : 'false')."\n";
echo '"string": '.(boolval("string") ? 'true' : 'false')."\n";
echo '"0":      '.(boolval("0") ? 'true' : 'false')."\n";
echo '"1":      '.(boolval("1") ? 'true' : 'false')."\n";
echo '[1, 2]:   '.(boolval([1, 2]) ? 'true' : 'false')."\n";
echo '[]:       '.(boolval([]) ? 'true' : 'false')."\n";
echo 'stdClass: '.(boolval(new stdClass) ? 'true' : 'false')."\n";
?>
```

O exemplo acima produzirá:

```
0:        false
42:       true
0.0:      false
4.2:      true
"":       false
"string": true
"0":      false
"1":      true
[1, 2]:   true
[]:       false
stdClass: true
```

Veja também

- floatval() - Obtém o valor em ponto flutuante da variável
- intval() - Obtém o valor em número inteiro de uma variável
- strval() - Retorna o valor string de uma variável
- settype() - Define o tipo de uma variável
- is_bool() - Verifica se a variável é um booleano
- Manipulação de tipos

debug_zval_dump

debug_zval_dump

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

debug_zval_dump — Exibe na saída uma representação em string de uma estrutura zval interna

Descrição

function **debug_zval_dump**(mixed `$value`, mixed `...$values`): void

Descarrega para a saída uma representação em string de uma estrutura zval interna (valor Zend). Muito útil para entender ou depurar detalhes de implementação do motor Zend ou de extensões PHP.

Parâmetros

`value`

A variável ou valor a exibir.

`values`

Mais variáveis ou valores a exibir.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.4.0	**debug_zval_dump()** agora indica se um array é compactado (packed).

Examples

Exemplo #1 Exemplo de **debug_zval_dump()**

```php
<?php
$var1 = 'Olá,';
$var1 .= ' Mundo';
$var2 = $var1;

debug_zval_dump($var1);
?>
```

O exemplo acima produzirá:

```
string(10) "Olá, Mundo" refcount(3)
```

> **Nota** : **Entendendo o `refcount` (contagem de referências)**
>
> O valor de `refcount` mostrado por este função pode ser surpreendente sem um entendimento detalhado da implementação interna do motor.
>
> O Motor Zend usa contagem de referência para dois propósitos diferentes:
>
> - Otimização de uso de memória usando a técnica chamada "cópia na escrita", onde múltiplas variáveis com o mesmo valor apontam para o mesma posição na memória. Quando alguma dessas variáveis é modificada, ela é apontada para uma nova cópia na memória, e a contagem de referências na original é diminuída em 1.
> - Rastrear variáveis que foram atribuídas ou passadas por referência (consulte Referências). Esta contagem de referências é armazenada em um zval de referência separado, apontando para o zval correspondente ao valor atual. Este zval adicional atualmente não é mostrado pela função **debug_zval_dump()**.
>
> Como a função **debug_zval_dump()** recebe suas entradas como parâmetros normais, passados por valor, a técnica de cópia na escrita será usada para passá-los: em vez de copiar os dados, a contagem de referências será adicionada em uma unidade durante toda a duração da chamada à função. Se a função modificasse o parâmetro depois de recebê-lo, então seria feita uma cópia; como ela não modifica, ela irá mostrar uma contagem uma unidade acima em relação ao escopo que a chamou.
>
> A passagem de parâmetros também evita que **debug_zval_dump()** mostre variáveis que foram atribuídas por referência. Para ilustrar, considere uma versão levemente modificada do exemplo acima:
>
> ```php
> <?php
> $var1 = 'Olá,';
> $var1 .= ' Mundo';
> // Aponta três variáveis como referências ao mesmo valor
> $var2 =& $var1;
> $var3 =& $var1;
>
> debug_zval_dump($var1);
> ?>
> ```
>
> O exemplo acima produzirá:

```
string(10) "Olá, Mundo" refcount(2)
```

Embora *$var1*, *$var2* e *$var3* estejam ligadas como referências, apenas o *valor* é passado a **debug_zval_dump()**. Este valor é usado uma vez pelo conjunto de referências, e uma vez dentro de **debug_zval_dump()**, por isso mostra uma contagem de 2.

Complicações adicionais aparecem por conta de otimizações implementadas no motor para diferentes tipos de dados. Alguns tipos como inteiros não usam "cópia na escrita", por isso não exibem nenhuma contagem de referência. Em outros casos, a contagem mostra cópias extras usadas internamente, como quando uma string ou array literais são armazenados como parte de uma instrução de código.

Veja também

- var_dump() - Despeja na saída informações sobre uma variável
- debug_backtrace() - Gera uma rastreamento retroativo
- Referências
- » Referências Explicadas (por Derick Rethans)

doubleval

doubleval

(PHP 4, PHP 5, PHP 7, PHP 8)

doubleval — Sinônimo de floatval()

Descrição

Esta função é um apelido para: floatval().

empty

empty

(PHP 4, PHP 5, PHP 7, PHP 8)

empty — Determina se uma variável está vazia

Descrição

function **empty**(mixed `$var`): bool

Determina se uma variável é considerada vazia. Uma variável é considerada vazia se não existir ou seu valor for igual a `false`. A função **empty()** não gera um aviso se a variável não existir.

Parâmetros

`var`

Variável a ser verificada

Nenhum aviso é gerado se a variável não existir. Isso significa que a função **empty()** é essencialmente o equivalente sucinto de **!isset($var) || $var == false**. Isto também se aplica a estruturas aninhadas, tais como um array multidimensional ou propriedades em encadeadas.

Valor Retornado

Retorna `true` se `var` não existir ou se tiver um valor que é vazio ou igual a zero. Consulte conversão para booleano. Caso contrário, retorna `false`.

Examples

Exemplo #1 Uma simples comparação entre as funções empty() / isset().

```php
<?php
$var = 0;

// Avaliado como verdadeiro porque $var está vazia
if (empty($var)) {
    echo '$var é 0, vazia ou não definida';
}

// Avaliado como verdadeiro porque $var está definida
if (isset($var)) {
    echo '$var está definida embora esteja vazia';
}
?>
```

Exemplo #2 Função empty() em posições de string

```php
<?php
$expected_array_got_string = 'somestring';
var_dump(empty($expected_array_got_string['some_key']));
var_dump(empty($expected_array_got_string[0]));
var_dump(empty($expected_array_got_string['0']));
var_dump(empty($expected_array_got_string['0.5']));
var_dump(empty($expected_array_got_string['0 Mostel']));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
bool(false)
bool(true)
bool(true)
```

Exemplo #3 empty() on array multidimensional

```php
<?php
$multidimensional = [
    'some' => [
        'deep' => [
            'nested' => 'value'
        ]
    ]
];

if (!empty($multidimensional['some']['some']['nested'])) {
    $someVariable = $multidimensional['some']['deep']['nested'];
}

var_dump(empty($multidimensional['some-undefined-key']));
var_dump(empty($multidimensional['some']['deep']['unknown']));
var_dump(empty($multidimensional['some']['deep']['nested']));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
bool(false)
```

Notas

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

> **Nota** :
>
> Ao utilizar a função **empty()** em propriedades de objetos inacessíveis, o método de sobrecarga __isset() será chamado, se declarado.

Veja também

- isset() - Determina se uma variável está declarada e é diferente de null
- __isset()
- unset() - unset remove a definição de uma variável informada
- array_key_exists() - Verifica se uma chave ou índice existe em um array
- count() - Conta todos os elementos de um array ou de um objeto Countable
- strlen() - Retorna o tamanho de uma string
- A tabela de comparação de tipos

floatval

floatval

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

floatval — Obtém o valor em ponto flutuante da variável

Descrição

function **floatval**(mixed `$value`): float

Obtém o valor float de `value`.

Parâmetros

`value`

Pode ser qualquer tipo escalar. **floatval()** não deve ser utilizada em objetos, pois isso ocasionará emissão de um erro de nível `E_WARNING` e retornará 1.

Valor Retornado

O valor em ponto flutuante da variável informada. Arrays vazios retornam 0, arrays com valores retoram 1.

Strings provavelmente retornarão 0, embora esse comportamento dependa dos caracteres à esquerda da string. A regra geral de conversão para ponto flutuante é aplicada.

Registro de Alterações

Versão	Descrição
8.0.0	O nível de erro ao converter um objeto mudou de `E_NOTICE` para `E_WARNING`.

Examples

Exemplo #1 Exemplo da função floatval()

```php
<?php
$var = '122.34343The';
$float_value_of_var = floatval($var);
echo $float_value_of_var; // 122.34343
?>
```

Exemplo #2 Exemplo da função floatval() em string com caracteres não numéricos à esquerda

```php
<?php
$var = 'The122.34343';
$float_value_of_var = floatval($var);
echo $float_value_of_var; // 0
?>
```

Veja também

- boolval() - Obtém o valor booleano de uma variável
- intval() - Obtém o valor em número inteiro de uma variável
- strval() - Retorna o valor string de uma variável
- settype() - Define o tipo de uma variável
- Manipulação de Tipos

get_debug_type

get_debug_type

(PHP 8)

get_debug_type — Obtém o nome do tipo de uma variável de forma adequada a depuração

Descrição

function **get_debug_type**(mixed `$value`): string

Retorna o nome resolvido da variável PHP `value`. Esta função resolverá objetos ao nome da classe deles, recursos ao nome do tipo de recurso deles, e valores escalares ao nome comum deles como seria usado em declarações de tipo.

Este função difere de gettype() pelo fato de retornar nomes de tipo que são mais consistentes com o uso real, ao invés daqueles existentes por razões históricas.

Parâmetros

`value`

A variável a ter o tipo verificado.

Valor Retornado

Valores possíveis para a string de retorno são:

Tipo + Estado	Valor de Retorno	Notas
nulo	`"null"`	-
Booleanos (`true` ou `false`)	`"bool"`	-
Inteiros	`"int"`	-
Números de ponto flutuante	`"float"`	-
Strings	`"string"`	-
Arrays	`"array"`	-
Recursos	`"resource (resourcename)"`	-
Recursos (Fechados)	`"resource (closed)"`	Exemplo: um fluxo de arquivo depois de ter sido fechado com fclose().
Objetos de Classes com Nome	O nome completo da classe incluindo seu espaço de nomes, ex.: `Foo\Bar`	-
Objetos de Classes Anônimas	`"class@anonymous"` ou nome da classe pai/interface se a classe estende outra classe ou implementa uma interface, p.ex.: `"Foo\Bar@anonymous"`	Classes anônimas são aquelas criadas através da sintaxe `$x = new class { ... }`

Examples

Exemplo #1 Exemplo de get_debug_type()

```php
<?php

namespace Foo;

echo get_debug_type(null), PHP_EOL;
echo get_debug_type(true), PHP_EOL;
echo get_debug_type(1), PHP_EOL;
echo get_debug_type(0.1), PHP_EOL;
echo get_debug_type("foo"), PHP_EOL;
echo get_debug_type([]), PHP_EOL;

$fp = fopen('/examples/book.xml', 'rb');
echo get_debug_type($fp), PHP_EOL;

fclose($fp);
echo get_debug_type($fp), PHP_EOL;

echo get_debug_type(new \stdClass), PHP_EOL;
echo get_debug_type(new class {}), PHP_EOL;

interface A {}
interface B {}
class C {}

echo get_debug_type(new class implements A {}), PHP_EOL;
echo get_debug_type(new class implements A,B {}), PHP_EOL;
echo get_debug_type(new class extends C {}), PHP_EOL;
echo get_debug_type(new class extends C implements A {}), PHP_EOL;

?>
```

O exemplo acima produzirá algo semelhante a:

```
null
bool
int
float
string
array
resource (stream)
resource (closed)
stdClass
class@anonymous
Foo\A@anonymous
Foo\A@anonymous
Foo\C@anonymous
Foo\C@anonymous
```

Veja também

- gettype() - Obtém o tipo de uma variável
- get_class() - Retorna o nome da classe de um objeto

get_defined_vars

get_defined_vars

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

get_defined_vars — Retorna um array com todas variáveis definidas

Descrição

function **get_defined_vars**(): array

Essa função retorna um array multimensional contendo uma lista com todas as variáveis definidas, incluindo variáveis de ambiente, servidor ou variáveis definidas pelo usuário, dentro do escopo que **get_defined_vars()** foi chamada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array multidimensional com todas as variáveis.

Examples

Exemplo #1 Exemplo da get_defined_vars()

```php
<?php
$b = array(1,1,2,3,5,8);

$arr = get_defined_vars();

// mostra $b
print_r($arr["b"]);

/* mostra o caminho do interpretador do PHP (se for usado como CGI)
 * por exemplo /usr/local/bin/php */
echo $arr["_"];

// mostra os parâmetros da linha de comando se for usado
print_r($arr["argv"]);

// mostra todas variáveis de servidor
print_r($arr["_SERVER"]);

// mostra todos os indíces possíveis para o array de variáveis
print_r(array_keys(get_defined_vars()));
?>
```

Veja também

- isset() - Determina se uma variável está declarada e é diferente de null
- get_defined_functions() - Retorna um array com todas as funções definidas
- get_defined_constants() - Retorna um array associativo com os nomes de todas as constantes e seus valores

get_resource_id

get_resource_id

(PHP 8)

get_resource_id — Retorna um identificador inteiro para o recurso informado

Descrição

function **get_resource_id**(resource `$resource`): int

Esta função fornece uma maneira segura (do ponto de vista de tipo) de gerar o identificador inteiro para um recurso.

Parâmetros

`resource`

O manipulador de recurso a ser avaliado.

Valor Retornado

O identificador int para o recurso informado em `resource`.

Esta função é essencialmente uma conversão para int do parâmetro `resource` para tornar mais fácil a obtenção do ID do recurso.

Examples

Exemplo #1 get_resource_id() produz o mesmo resultado de uma conversão int

```php
<?php
$handle = fopen("php://stdout", "w");

echo (int) $handle . "\n";

echo get_resource_id($handle);

?>
```

O exemplo acima produzirá algo semelhante a:

```
698
698
```

Veja também

- get_resource_type() - Retorna o tipo de recurso

get_resource_type

get_resource_type

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

get_resource_type — Retorna o tipo de recurso

Descrição

function **get_resource_type**(resource `$resource`): string

Essa função obtém o tipo do recurso informado.

Parâmetros

`resource`

O manipulador de recurso a ser avaliado.

Valor Retornado

Se o parâmetro `resource` for um recurso, esta função retornará uma string representando seu tipo. Se o tipo não for identificado por esta função, o valor de retorno será a string `Unknown`.

Esta função retornará `null` e gerará um erro se `resource` não for um recurso do tipo resource.

Examples

Exemplo #1 Exemplo de get_resource_type()

```php
<?php
$fp = fopen("foo", "w");
echo get_resource_type($fp) . "\n";
?>
```

Saída do exemplo acima no PHP 7:

```
stream
```

Veja também

- get_resource_id() - Retorna um identificador inteiro para o recurso informado

gettype

gettype

(PHP 4, PHP 5, PHP 7, PHP 8)

gettype — Obtém o tipo de uma variável

Descrição

function **gettype**(mixed `$value`): string

Retorna o tipo da variável PHP `value`. Para checagem de tipo, utilize as funções `is_*`.

Parâmetros

`value`

A variável a ter o tipo verificado.

Valor Retornado

Os valores possíveis para a string retornada são:

- `"boolean"` - booleano
- `"integer"` - inteiro
- `"double"` - ponto flutuante (por razões históricas, `"double"` é retornado no caso de um float, e não simplesmente `"float"`)
- `"string"` - cadeia de caracteres
- `"array"`
- `"object"` - objeto
- `"resource"` - recurso
- `"resource (closed)"` a partir do PHP 7.2.0 - recurso (fechado)
- `"NULL"` - nulo
- `"unknown type"` - tipo desconhecido

Registro de Alterações

Versão	Descrição
7.2.0	Recursos já fechados agora são reportados como `'resource (closed)'`. Anteriormente os valores retornados para recursos fechados eram `'unknown type'`.

Examples

Exemplo #1 Exemplo da função gettype()

```php
<?php

$data = array(1, 1., NULL, new stdClass, 'foo');

foreach ($data as $value) {
    echo gettype($value), "\n";
}

?>
```

O exemplo acima produzirá algo semelhante a:

```
integer
double
NULL
```

```
object
string
```

Veja também

- get_debug_type() - Obtém o nome do tipo de uma variável de forma adequada a depuração
- settype() - Define o tipo de uma variável
- get_class() - Retorna o nome da classe de um objeto
- is_array() - Verifica se a variável é um array
- is_bool() - Verifica se a variável é um booleano
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual
- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_null() - Verifica se uma variável é null
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_object() - Verifica se uma variável é um objeto
- is_resource() - Verifica se uma variável é um recurso
- is_scalar() - Verifica se é uma variável é escalar
- is_string() - Verifica se o tipo de uma variável é string
- function_exists() - Retorna true se a função informada estiver definida
- method_exists() - Verifica se o método da classe existe

intval

intval

(PHP 4, PHP 5, PHP 7, PHP 8)

intval — Obtém o valor em número inteiro de uma variável

Descrição

function **intval**(mixed `$value`, int `$base` = 10): int

Retorna o valor do tipo int do parâmetro `value`, usando a `base` especificada para a conversão (o padrão é base 10). **intval()** não deve ser usada em objetos, se isto for feito emitirá um erro de nível `E_WARNING` e retornará 1.

Parâmetros

`value`

O valor escalar a ser convertido para um número inteiro

`base`

A base para conversão

> **Nota** :
>
> Se `base` for 0, a base usada será determinada pelo formato de `value`:
>
> - se a string incluir um prefixo "0x" (ou "0X"), a base será 16 (hexadecimal); senão,
> - se a string começar com um "0b" (ou "0B"), a base será 2 (binária); senão,
> - se a string começar com "0", a base será 8 (octal); senão,
> - a base será 10 (decimal).

Valor Retornado

O valor inteiro de `value` em caso de sucesso ou 0 em caso de falhar. Arrays vazios retornam 0, arrays não vazios retornam 1.

O valor máximo depende do sistema. Sistemas de 32 bits têm uma faixa inteiros com sinal de -2147483648 to 2147483647. Então, por exemplo, em um sistema desses, `intval('1000000000000')` retornará 2147483647. O máximo inteiro com sinal para sistemas de 64 bits é 9223372036854775807.

Strings na maior parte das vezes retornarão 0 embora isso dependa do caractere mais à esquerda dele da string. As regras comuns à conversão para inteiros se aplicam.

> **Nota** :
>
> Strings numéricas que usam notação científica (contendo a letra `e` ou `E`) são primeiro interpretadas como números antes de ser convertidas para um inteiro.
>
> Como a parte numérica da string é interpretada como um todo, o resultado não é simplesmente a parte inteira na frente. Além disso, expoentes grandes podem ser convertidos para `PHP_INT_MAX` em caso de sobrecarga:
>
> ```php
> <?php
> echo intval('42.42e42'); // 9223372036854775807 on 64-bit systems
> ?>
> ```
>
> Consulte strings numéricas para detalhes sobre como tais strings são interpretadas.

Registro de Alterações

Versão	Descrição
8.0.0	O nível de erro na conversão de objeto mudou de `E_NOTICE` para `E_WARNING`.

Examples

Exemplo #1 Exemplos de intval()

Os seguintes exemplos são baseados em um sistema de 64 bits.

```php
<?php
echo intval(42), PHP_EOL;                      // 42
echo intval(4.7), PHP_EOL;                     // 4
echo intval('42'), PHP_EOL;                    // 42
echo intval('+42'), PHP_EOL;                   // 42
echo intval('-42'), PHP_EOL;                   // -42
echo intval(042), PHP_EOL;                     // 34
echo intval('042'), PHP_EOL;                   // 42
echo intval(1e10), PHP_EOL;                    // 10000000000
echo intval('1e10'), PHP_EOL;                  // 10000000000
echo intval(0x1A), PHP_EOL;                    // 26
echo intval('0x1A'), PHP_EOL;                  // 0
echo intval('0x1A', 0), PHP_EOL;               // 26
echo intval(42000000), PHP_EOL;                // 42000000
echo intval(420000000000000000), PHP_EOL;      // -4275113695319687168
echo intval('420000000000000000'), PHP_EOL;    // 9223372036854775807
echo intval(42, 8), PHP_EOL;                   // 42
echo intval('42', 8), PHP_EOL;                 // 34
echo intval(array()), PHP_EOL;                 // 0
echo intval(array('foo', 'bar')), PHP_EOL;     // 1
echo intval(false), PHP_EOL;                   // 0
echo intval(true), PHP_EOL;                    // 1
?>
```

Notas

> **Nota** :
>
> O parâmetro `base` não terá efeito a menos que o parâmetro `value` seja uma string.

Veja também

- boolval() - Obtém o valor booleano de uma variável

- floatval() - Obtém o valor em ponto flutuante da variável
- strval() - Retorna o valor string de uma variável
- settype() - Define o tipo de uma variável
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- Manipulação de Tipos
- BCMath Funções Matemáticas de Precisão Arbitrária

is_array

is_array

(PHP 4, PHP 5, PHP 7, PHP 8)

is_array — Verifica se a variável é um array

Descrição

function **is_array**(mixed `$value`): bool

Verifica se uma dada variável é um array.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna **`true`** se `value` é um array, **`false`** caso contrário.

Examples

Exemplo #1 Verificando se a variável é um array

```php
<?php
$sim = array('este', 'é', 'um array');

echo is_array($sim) ? 'Array' : 'não Array';
echo "\n";

$nao = 'esta é uma string';

echo is_array($nao) ? 'Array' : 'não Array';
?>
```

O exemplo acima produzirá:

```
Array
não Array
```

Veja também

- array_is_list() - Verifica se um array é uma lista
- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_string() - Verifica se o tipo de uma variável é string
- is_object() - Verifica se uma variável é um objeto

is_bool

is_bool

(PHP 4, PHP 5, PHP 7, PHP 8)

is_bool — Verifica se a variável é um booleano

Descrição

function **is_bool**(mixed `$value`): bool

Verifica se uma dada variável é um booleano.

Parâmetros

`value`

A variável a ser verificada.

Valor Retornado

Retorna **`true`** se `value` for um bool, **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de is_bool()

```php
<?php
 $a = false;
 $b = 0;

 // Como $a é boolean, retornará true
 if (is_bool($a) === true) {
    echo "Sim, isso é booleano\n";
 }

 // Como $b não é boolean, retornará false
 if (is_bool($b) === false) {
     echo "Não, isso não é booleano\n";
 }
 ?>
```

Veja também

- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_string() - Verifica se o tipo de uma variável é string
- is_object() - Verifica se uma variável é um objeto
- is_array() - Verifica se a variável é um array

is_callable

is_callable

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

is_callable — Verifica se um valor pode ser chamado como uma função a partir do escopo atual

Descrição

function **is_callable**(mixed `$value`, bool `$syntax_only` = **`false`**, string `&$callable_name` = **`null`**): bool

Verifica se o valor informado em `value` é do tipo callable, ou se pode ser chamado usando a função call_user_func().

Parâmetros

`value`

O valor a ser verificado.

`syntax_only`

Se definido para **`true`** a função apenas verifica se `value` pode ser uma função ou método. Ela irá rejeitar quaisquer valores que não sejam objetos chamáveis, Closure, strings ou arrays que não tenham uma estrutura válida para ser usado como uma função de

retorno. Um array chamável válido tem 2 entradas, a primeira delas é um objeto ou uma string, e a segunda uma string.

`callable_name`

Recebe o "nome da função chamável", por exemplo, `"AlgumaClasse::algumMétodo"`. Observe, entretanto, que embora a implicação de que `AlgumaClasse::algumMétodo()` seja um método estático chamável, este não é o caso.

Valor Retornado

Retorna **`true`** se `value` puder ser chamado como função, **`false`** caso contrário.

Examples

Exemplo #1 Verificando se uma string pode ser chamada como uma função

```php
<?php

function someFunction() {}

$functionVariable = 'someFunction';

var_dump(is_callable($functionVariable, false, $callable_name));

var_dump($callable_name);

?>
```

O exemplo acima produzirá:

```
bool(true)
string(12) "someFunction"
```

Exemplo #2 Verificando se um array pode ser chamado como uma função

```php
<?php

class SomeClass
{
    function someMethod() {}
}

$anObject = new SomeClass();

$methodVariable = [$anObject, 'someMethod'];

var_dump(is_callable($methodVariable, true, $callable_name));

var_dump($callable_name);

?>
```

O exemplo acima produzirá:

```
bool(true)
string(21) "SomeClass::someMethod"
```

Exemplo #3 Exemplo de is_callable() e construtores

Apesar do fato que construtores são métodos que são chamados quando um objeto é criado, eles não são métodos estáticos e **is_callable()** retornará `false` para eles. Não é possível usar **is_callable()** para verificar se uma classe pode ser instanciada a partir do escopo atual.

```php
<?php

class Foo
{
    public function __construct() {}

    public function foo() {}
}

var_dump(
    is_callable(['Foo', '__construct']),
    is_callable(['Foo', 'foo'])
);

$foo = new Foo();
var_dump(is_callable([$foo, '__construct']));

?>
```

O exemplo acima produzirá:

```
bool(false)
bool(false)
bool(true)
```

Notas

- Um objeto é sempre chamável se ele implementar o método __invoke(), e este método estiver visível no escopo atual.
- Um nome de classe é chamável se ela implementa o método __callStatic().
- Se um objeto implementa o método __call(), então esta função retornará `true` para qualquer método neste objeto, mesmo se o método não estiver definido.
- Esta função pode disparar o auto-carregamento se chamada com o nome de uma classe.

Veja também

- call_user_func() - Chama a função de retorno informada no primeiro parâmetro
- function_exists() - Retorna true se a função informada estiver definida
- method_exists() - Verifica se o método da classe existe

is_countable

is_countable

(PHP 7 >= 7.3.0, PHP 8)

is_countable — Verifica se o conteúdo de uma variável é um valor contável

Descrição

function **is_countable**(mixed `$value`): bool

Verifica se o conteúdo de uma variável é um array ou um objeto que implementa a classe Countable.

Parâmetros

`value`

O valor a ser verificado

Valor Retornado

Retorna `true` se `value` for contável, `false` caso contrário.

Registro de Alterações

Versão	Descrição
7.3.0	A função **is_countable()** foi adicionada.

Examples

Exemplo #1 Exemplos de is_countable()

```
<?php
var_dump(is_countable([1, 2, 3])); // bool(true)
var_dump(is_countable(new ArrayIterator(['foo', 'bar', 'baz']))); // bool(true)
var_dump(is_countable(new ArrayIterator())); // bool(true)
var_dump(is_countable(new stdClass())); // bool(false)
?>
```

Veja também

- is_array() - Verifica se a variável é um array
- is_object() - Verifica se uma variável é um objeto
- is_iterable() - Verifica se o conteúdo de uma variável é um valor iterável
- is_bool() - Verifica se a variável é um booleano

is_double

is_double

(PHP 4, PHP 5, PHP 7, PHP 8)

is_double — Sinônimo de is_float()

Descrição

Esta função é um apelido para: is_float().

is_float

is_float

(PHP 4, PHP 5, PHP 7, PHP 8)

is_float — Verifica se a variável é do tipo float

Descrição

function **is_float**(mixed `$value`): bool

Verifica se a variável informada é do tipo ponto flutuante.

> **Nota** :
>
> Para verificar se a variável é um número ou uma string numérica (como as entradas de formulário, que são sempre strings), deve ser usada a função is_numeric().

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for um float, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_float()

```php
<?php

var_dump(is_float(27.25));
var_dump(is_float('abc'));
var_dump(is_float(23));
var_dump(is_float(23.5));
var_dump(is_float(1e7));  // Notação científica
var_dump(is_float(true));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
bool(false)
bool(true)
bool(true)
bool(false)
```

Veja também

- is_bool() - Verifica se a variável é um booleano
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_string() - Verifica se o tipo de uma variável é string
- is_array() - Verifica se a variável é um array
- is_object() - Verifica se uma variável é um objeto

is_int

is_int

(PHP 4, PHP 5, PHP 7, PHP 8)

is_int — Informa se o tipo de uma variável é um inteiro

Descrição

function **is_int**(mixed `$value`): bool

Verifica se o tipo da variável informada é um inteiro.

> **Nota** :
>
> Para verificar se a variável é um número ou uma string numérica (como em entrada de formulário, que é sempre uma string), deve-se usar is_numeric().

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for um int, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_int()

```php
<?php
$values = array(23, "23", 23.5, "23.5", null, true, false);
foreach ($values as $value) {
    echo "is_int(";
    var_export($value);
    echo ") = ";
    var_dump(is_int($value));
}
?>
```

O exemplo acima produzirá:

```
is_int(23) = bool(true)
is_int('23') = bool(false)
is_int(23.5) = bool(false)
is_int('23.5') = bool(false)
is_int(NULL) = bool(false)
is_int(true) = bool(false)
is_int(false) = bool(false)
```

Veja também

- is_bool() - Verifica se a variável é um booleano
- is_float() - Verifica se a variável é do tipo float
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_string() - Verifica se o tipo de uma variável é string
- is_array() - Verifica se a variável é um array
- is_object() - Verifica se uma variável é um objeto

is_integer

is_integer

(PHP 4, PHP 5, PHP 7, PHP 8)

is_integer — Sinônimo de is_int()

Descrição

Esta função é um apelido para: is_int().

is_iterable

is_iterable

(PHP 7 >= 7.1.0, PHP 8)

is_iterable — Verifica se o conteúdo de uma variável é um valor iterável

Descrição

function **is_iterable**(mixed `$value`): bool

Verifica se o conteúdo de uma variável é aceita pelo pseudo-tipo iterable, ou seja, ela é um array ou um objeto que implementa a classe Traversable

Parâmetros

`value`

O valor a ser verificado

Valor Retornado

Retorna `true` se `value` for iterável, `false` caso contrário.

Examples

Exemplo #1 Exemplos de is_iterable()

```php
<?php

var_dump(is_iterable([1, 2, 3]));  // bool(true)
var_dump(is_iterable(new ArrayIterator([1, 2, 3])));  // bool(true)
var_dump(is_iterable((function () { yield 1; })()));  // bool(true)
var_dump(is_iterable(1));  // bool(false)
var_dump(is_iterable(new stdClass()));  // bool(false)

?>
```

Veja também

- is_array() - Verifica se a variável é um array

is_long

is_long

(PHP 4, PHP 5, PHP 7, PHP 8)

is_long — Sinônimo de is_int()

Descrição

Esta função é um apelido para: is_int().

is_null

is_null

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

is_null — Verifica se uma variável é `null`

Descrição

function **is_null**(mixed `$value`): bool

Verifica se a variável fornecida é `null`.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for null, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_null()

```php
<?php

error_reporting(E_ALL);

$foo = NULL;
var_dump(is_null($inexistent), is_null($foo));

?>
```

Veja também

- O tipo `null`
- isset() - Determina se uma variável está declarada e é diferente de null
- is_bool() - Verifica se a variável é um booleano
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_string() - Verifica se o tipo de uma variável é string
- is_object() - Verifica se uma variável é um objeto
- is_array() - Verifica se a variável é um array

is_numeric

is_numeric

(PHP 4, PHP 5, PHP 7, PHP 8)

is_numeric — Verifica se uma variável é um número ou uma string numérica

Descrição

function **is_numeric**(mixed `$value`): bool

Determina se a variável fornecida é um número ou uma string numérica.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for um número ou uma string numérica, `false` caso contrário.

Registro de Alterações

Versão	Descrição
8.0.0	String numéricas terminando com espaços (`"42 "`) agora retornam `true`. Anteriormente, `false` era retornado.

Examples

Exemplo #1 Exemplos de is_numeric()

```php
<?php
$tests = array(
    "42",
    1337,
    0x539,
    02471,
    0b10100111001,
    1337e0,
    "0x539",
    "02471",
    "0b10100111001",
    "1337e0",
    "not numeric",
    array(),
    9.1,
    null,
    '',
);

foreach ($tests as $element) {
    if (is_numeric($element)) {
        echo var_export($element, true) . " é numérico", PHP_EOL;
    } else {
        echo var_export($element, true) . " NÃO é numérico", PHP_EOL;
    }
```

```
}
?>
```

O exemplo acima produzirá:

```
'42' é numérico
1337 é numérico
1337 é numérico
1337 é numérico
1337 é numérico
1337.0 é numérico
'0x539' NÃO é numérico
'02471' é numérico
'0b10100111001' NÃO é numérico
'1337e0' é numérico
'not numeric' NÃO é numérico
array (
) NÃO é numérico
9.1 é numérico
NULL NÃO é numérico
'' NÃO é numérico
```

Exemplo #2 is_numeric() com espaço em branco

```php
<?php
$tests = [
    " 42",
    "42 ",
    "\u{A0}9001", // caractere de espaço sem quebra
    "9001\u{A0}", // caractere de espaço sem quebra
];

foreach ($tests as $element) {
    if (is_numeric($element)) {
        echo var_export($element, true) . " é numérico", PHP_EOL;
    } else {
        echo var_export($element, true) . " NÃO é numérico", PHP_EOL;
    }
}
?>
```

Saída do exemplo acima no PHP 8:

```
' 42' é numérico
'42 ' é numérico
' 9001' NÃO é numérico
'9001 ' NÃO é numérico
```

Saída do exemplo acima no PHP 7:

```
' 42' é numérico
'42 ' NÃO é numérico
' 9001' NÃO é numérico
'9001 ' NÃO é numérico
```

Veja também

- Strings numéricas
- ctype_digit() - Verifica se os caracteres são numéricos
- is_bool() - Verifica se a variável é um booleano
- is_null() - Verifica se uma variável é null
- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_string() - Verifica se o tipo de uma variável é string
- is_object() - Verifica se uma variável é um objeto
- is_array() - Verifica se a variável é um array
- filter_var() - Filtra uma variável com um filtro especificado

is_object

is_object

(PHP 4, PHP 5, PHP 7, PHP 8)

is_object — Verifica se uma variável é um objeto

Descrição

function **is_object**(mixed `$value`): bool

Verifica se a variável fornecida é um objeto.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for um object, `false` caso contrário.

Registro de Alterações

Versão	Descrição
7.2.0	**is_object()** agora retorna `true` para objetos desserializados sem definição de classe (classe de __PHP_Incomplete_Class). Anteriormente `false` era retornado.

Examples

Exemplo #1 Exemplo de is_object()

```php
<?php
// Declara uma função simples para retornar um
// array do nosso objeto
function get_students($obj)
{
    if (!is_object($obj)) {
        return false;
    }

    return $obj->students;
}

// Declara uma nova instância de classe e preenche
// alguns valores
$obj = new stdClass();
$obj->students = array('Kalle', 'Ross', 'Felipe');

var_dump(get_students(null));
```

```
var_dump(get_students($obj));
?>
```

Veja também

- is_bool() - Verifica se a variável é um booleano
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_float() - Verifica se a variável é do tipo float
- is_string() - Verifica se o tipo de uma variável é string
- is_array() - Verifica se a variável é um array

is_real

is_real

(PHP 4, PHP 5, PHP 7, PHP 8)

is_real — Sinônimo de is_float()

Descrição

Esta função é um apelido para: is_float().

> **Aviso**
>
> Este apelido foi *DESCONTINUADO* a partir do PHP 7.4.0 e foi *REMOVIDO* a partir do PHP 8.0.0.

is_resource

is_resource

(PHP 4, PHP 5, PHP 7, PHP 8)

is_resource — Verifica se uma variável é um recurso

Descrição

function **is_resource**(mixed `$value`): bool

Verifica se a variável informada é do tipo resource.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna **`true`** se `value` for do tipo resource, **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de is_resource()

```
<?php

$handle = fopen("php://stdout", "w");
if (is_resource($handle)) {
    echo '$handle é do tipo resource';
}

?>
```

O exemplo acima produzirá:

```

```

```
$handle é do tipo resource
```

Notas

> **Nota** :
>
> **is_resource()** não é um método estrito de validação de tipo: retornará `false` se `value` for uma variável do tipo resource que foi fechada.

Veja também

- Documentação sobre o tipo Resource
- get_resource_type() - Retorna o tipo de recurso

is_scalar

is_scalar

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

is_scalar — Verifica se é uma variável é escalar

Descrição

function **is_scalar**(mixed `$value`): bool

Verifica se uma expressão é avaliada como um valor escalar.

Consulte Tipos Escalares para mais informação.

> **Nota** :
>
> **is_scalar()** não considera valores de recurso do tipo resource como escalares, porque recursos são tipos de dados abstratos que atualmente são baseados em inteiros. Esse detalhe de implementação não deve ser levado em consideração pois pode ser modificado futuramente.

> **Nota** :
>
> **is_scalar()** não considera valores nulos como sendo do tipo escalar.

Parâmetros

`value`

A variável a ser avaliada.

Valor Retornado

Retorna `true` se `value` for uma variável escalar, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_scalar()

```php
<?php
function show_var($var)
{
    if (is_scalar($var)) {
        echo $var, PHP_EOL;
    } else {
        var_dump($var);
    }
```

```
}

$pi = 3.1416;
$proteins = array("hemoglobin", "cytochrome c oxidase", "ferredoxin");

show_var($pi);
show_var($proteins)

?>
```

O exemplo acima produzirá:

```
3.1416
array(3) {
  [0]=>
  string(10) "hemoglobin"
  [1]=>
  string(20) "cytochrome c oxidase"
  [2]=>
  string(10) "ferredoxin"
}
```

Veja também

- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_real() - Sinônimo de is_float
- is_string() - Verifica se o tipo de uma variável é string
- is_bool() - Verifica se a variável é um booleano
- is_object() - Verifica se uma variável é um objeto
- is_array() - Verifica se a variável é um array

is_string

is_string

(PHP 4, PHP 5, PHP 7, PHP 8)

is_string — Verifica se o tipo de uma variável é string

Descrição

function **is_string**(mixed `$value`): bool

Verifica se o tipo da variável informada é string.

Parâmetros

`value`

A variável a ser analisada.

Valor Retornado

Retorna `true` se `value` for do tipo string, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_string()

```
<?php
$values = array(false, true, null, 'abc', '23', 23, '23.5', 23.5, '', ' ', '0', 0);
foreach ($values as $value) {
```

```
    echo "is_string(";
    var_export($value);
    echo ") = ";
    echo var_dump(is_string($value));
}
?>
```

O exemplo acima produzirá:

```
is_string(false) = bool(false)
is_string(true) = bool(false)
is_string(NULL) = bool(false)
is_string('abc') = bool(true)
is_string('23') = bool(true)
is_string(23) = bool(false)
is_string('23.5') = bool(true)
is_string(23.5) = bool(false)
is_string('') = bool(true)
is_string(' ') = bool(true)
is_string('0') = bool(true)
is_string(0) = bool(false)
```

Veja também

- is_float() - Verifica se a variável é do tipo float
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_bool() - Verifica se a variável é um booleano
- is_object() - Verifica se uma variável é um objeto
- is_array() - Verifica se a variável é um array

isset

isset

(PHP 4, PHP 5, PHP 7, PHP 8)

isset — Determina se uma variável está declarada e é diferente de `null`

Descrição

function **isset**(mixed `$var`, mixed `...$vars`): bool

Determina se uma variável é considerada definida, isto é, está declarada e é diferente de `null`.

Se a variável tornou-se indefinida com a função unset(), ela não é mais considerada definida.

isset() retornará `false` ao verificar uma variável que foi atribuída com o valor `null`. Note também que um caractere nulo (`"\0"`) não é equivalente à constante `null` do PHP.

Se múltiplos parâmetros forem fornecidos então **isset()** retornará `true` apenas se todos os parâmetros forem considerados definidos. A avaliação ocorre da esquerda para a direira e para assim que uma variável indefinida for encontrada.

Parâmetros

`var`

A variável a ser verificada.

`vars`

Mais variáveis a verificar.

Valor Retornado

Retorna `true` se `var` existe e tem qualquer valor diferente de `null`. `false` caso contrário.

Examples

Exemplo #1 Exemplos de isset()

```php
<?php

$var = '';

// Isto será avaliado como VERDADEIRO e por isso o texto será mostrado.
if (isset($var)) {
    echo "Esta variável está definida então vou imprimir.", PHP_EOL;
}

// Nos próximos exemplos, será utilizado var_dump() para mostrar
// o valor de retorno de isset().

$a = "teste";
$b = "outroteste";

var_dump(isset($a));     // VERDADEIRO
var_dump(isset($a, $b)); // VERDADEIRO

unset ($a);

var_dump(isset($a));     // FALSO
var_dump(isset($a, $b)); // FALSO

$foo = NULL;
var_dump(isset($foo));   // FALSO

?>
```

Esta função também funciona para elementos em arrays:

Exemplo #2 Exemplo de isset() com elementos de arrays

```php
<?php

$a = array ('test' => 1, 'hello' => NULL, 'pie' => array('a' => 'apple'));

var_dump(isset($a['test']));            // VERDADEIRO
var_dump(isset($a['foo']));             // FALSO
var_dump(isset($a['hello']));           // FALSO

// A chave 'hello' é igual a NULO então é considerada indefinida
// Se deseja-se verificar por chaves de valores NULOS, deve-se tentar:
var_dump(array_key_exists('hello', $a)); // VERDADEIRO

// Verificando valores de array em maior profundidade
var_dump(isset($a['pie']['a']));        // VERDADEIRO
var_dump(isset($a['pie']['b']));        // FALSO
var_dump(isset($a['cake']['a']['b']));  // FALSO

?>
```

Exemplo #3 isset() em Deslocamento de Strings

```php
<?php
$expected_array_got_string = 'somestring';
var_dump(isset($expected_array_got_string['some_key']));
var_dump(isset($expected_array_got_string[0]));
var_dump(isset($expected_array_got_string['0']));
var_dump(isset($expected_array_got_string[0.5]));
var_dump(isset($expected_array_got_string['0.5']));
var_dump(isset($expected_array_got_string['0 Mostel']));
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
bool(true)
bool(true)
bool(false)
bool(false)
```

Notas

> **Aviso**
>
> **isset()** somente funciona com variáveis, pois passar qualquer outra coisa resultará em erro de interpretação. Para verificar se constantes estão definidas, use a função defined().

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

> **Nota** :
>
> Ao usar **isset()** em propriedades inacessíveis de objetos, o método de sobrecarga __isset() será chamado, se declarado.

Veja também

- empty() - Determina se uma variável está vazia
- __isset()
- unset() - unset remove a definição de uma variável informada
- defined() - Verifica se existe uma constante com o nome informado
- Tabelas de comparação de tipos
- array_key_exists() - Verifica se uma chave ou índice existe em um array
- is_null() - Verifica se uma variável é null
- Operador @ de controle de erro

print_r

print_r

(PHP 4, PHP 5, PHP 7, PHP 8)

print_r — Exibe informação legível sobre uma variável

Descrição

function **print_r**(mixed `$value`, bool `$return` = `false`): string|true

print_r() exibe informação sobre uma variável de uma forma legível para humanos.

print_r(), var_dump() e var_export() mostrarão também propriedades protegidas e privadas de objetos. Membros de classes estáticas não serão mostrados.

Parâmetros

`value`

A expressão a ser exibida.

`return`

Se for desejado capturar a saída de **print_r()**, utilize o parâmetro `return`. Quando esse parâmetro for definido para `true`, **print_r()** retornará a informação, ao invés de exibi-la.

Valor Retornado

Se for fornecido uma string, um int ou um float, o próprio valor será impresso. Se for fornecido um array, valores serão apresentados em um formato que mostra chaves e elementos. Notação similar é utilizada para objetos (objects).

Quando o parâmetro `return` for **`true`**, esta função retornará uma string. Caso contrário, o valor retornado será **`true`**.

Registro de Alterações

Versão	Descrição
8.4.0	O tipo de retorno foi alterado de string|bool para string|true.

Examples

Exemplo #1 Exemplo de print_r()

```
<pre>
<?php
$a = array ('a' => 'apple', 'b' => 'banana', 'c' => array ('x', 'y', 'z'));
print_r($a);
?>
</pre>
```

O exemplo acima produzirá:

```
<pre>
Array
(
    [a] => apple
    [b] => banana
    [c] => Array
        (
            [0] => x
            [1] => y
            [2] => z
        )
)
</pre>
```

Exemplo #2 Exemplo do parâmetro `return`

```
<?php
$b = array ('m' => 'monkey', 'foo' => 'bar', 'x' => array ('x', 'y', 'z'));
$results = print_r($b, true); // $results agora contém a saída de print_r

print_r($results);
?>
```

Notas

> **Nota** :
>
> Quando o parâmetro `return` é usado, esta função usa um buffer de saída interno antes do PHP 7.1.0, portanto não pode ser usada em uma função callback da função ob_start().

Veja também

- ob_start() - Ativa o buffer de saída
- var_dump() - Despeja na saída informações sobre uma variável
- var_export() - Mostra ou retorna uma representação string analisável de uma variável

serialize

serialize

(PHP 4, PHP 5, PHP 7, PHP 8)

serialize — Gera uma representação armazenável de um valor

Descrição

function **serialize**(mixed `$value`): string

Gera uma representação armazenável de um valor.

É útil para armazenar ou passar um valor do PHP sem perder seu tipo ou estrutura.

Para restaurar uma string serializada de volta a um valor PHP, use unserialize().

Parâmetros

`value`

O valor a ser serializado. **serialize()** lida com todos os tipos, exceto recursos (resource) e alguns objetos (object)s (veja nota abaixo). Pode-se até usar **serialize()** para array que contenha referência a si mesmo. Referências circulaes dentro do array/objeto a ser serializado também serão armazenadas. Qualquer outra referência será perdida.

Ao serializar objetos, o PHP tentará chamar as funções membro __serialize() ou __sleep() antes da serialização. Isto serve para permitir ao objeto alguma limpeza de última hora, etc. antes de ser realizado. Da mesma forma, quando o objeto é restaurado usando unserialize(), a função membro __unserialize() ou __wakeup() será chamada.

> **Nota** :
>
> Os membros privados terão o nome da classe prefixando o nome do membro; membros protegidos terão um '*' prefixando o nome. Estes prefixos serão envolvidos por bytes nulos, um de cada lado.

Valor Retornado

Retorna uma string contendo uma representação em fluxo de bytes de `value` que pode ser armazenada em qualquer lugar.

Note que esta string é binária e pode incluir bytes nulos, e precisa ser armazenada e gerenciada como tal. Por exemplo, a saída de **serialize()** deveria ser armazenada geralmente em um campo BLOB de um banco de dados, e não em um campo tipo CHAR ou TEXT.

Examples

Exemplo #1 Exemplo de serialize()

```php
<?php
// $session_data contém um array multi-dimensional com informação
// de sessão do usuário atual. serialize() será utilizada para armazenar
// a variável em um banco de dados ao final da requisição.

$conn = odbc_connect("webdb", "php", "chicken");
$stmt = odbc_prepare($conn,
      "UPDATE sessions SET data = ? WHERE id = ?");
$sqldata = array (serialize($session_data), $_SERVER['PHP_AUTH_USER']);
if (!odbc_execute($stmt, $sqldata)) {
    $stmt = odbc_prepare($conn,
     "INSERT INTO sessions (id, data) VALUES(?, ?)");
    if (!odbc_execute($stmt, array_reverse($sqldata))) {
        /* Algo deu errado.. */
    }
}
?>
```

Notas

> **Nota** :

> Note que muitos objetos internos do PHP não podem ser serializados. Entretanto, os que tiverem esta habilidade ou implementam a interface Serializable ou os métodos mágicos __serialize()/__unserialize() ou __sleep()/__wakeup(). Se uma classe interna não preencher algum desses requisitos, ela não poderá ser serializada com confiança.
>
> Existem algumas exceções histórias à regra acima, onde alguns objetos internos poderiam ser serializados sem implementar a interface ou expor os métodos.

> **Aviso**
>
> Quando **serialize()** serializa objetos, a barra invertida no início não é incluída nas classes com espaço de nomes para máxima compatibilidade.

Veja também

- unserialize() - Cria um valor PHP a partir de uma representação armazenada
- var_export() - Mostra ou retorna uma representação string analisável de uma variável
- json_encode() - Retorna a representação JSON de um valor
- Serializando Objetos
- __sleep()
- __wakeup()
- __serialize()
- __unserialize()

settype

settype

(PHP 4, PHP 5, PHP 7, PHP 8)

settype — Define o tipo de uma variável

Descrição

function **settype**(mixed `&$var`, string `$type`): bool

Define o tipo da variável `var` para `type`.

Parâmetros

`var`

A variável a ser convertida.

`type`

Os valores possíveis para `type` são:

- "boolean" ou "bool"
- "integer" ou "int"
- "float" ou "double"
- "string"
- "array"
- "object"
- "null"

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Lança um ValueError se o valor de `type` não for um tipo válido, a partir do PHP 8.0.0. Antes do PHP 8.0.0, um `E_WARNING` era emitido e a função retornava `false`.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança um ValueError quando um tipo inválido é passado ao parâmetro `type`. Anteriormente, um `E_WARNING` era emitido e a função retornava `false`.

Examples

Exemplo #1 Exemplo da função settype()

```php
<?php
$foo = "5bar"; // string
$bar = true;   // boolean

settype($foo, "integer"); // $foo é agora 5   (integer)
settype($bar, "string");  // $bar é agora "1" (string)

var_dump($foo, $bar);
?>
```

Notas

> **Nota** :
>
> O valor máximo para "int" é `PHP_INT_MAX`.

Veja também

- gettype() - Obtém o tipo de uma variável
- Conversão de tipos
- Manipulação de tipos

strval

strval

(PHP 4, PHP 5, PHP 7, PHP 8)

strval — Retorna o valor string de uma variável

Descrição

function **strval**(mixed `$value`): string

Obtém o valor string de uma variável. Consulte a documentação sobre string para maiores informações sobre conversão para o tipo string.

Esta função não faz nenhuma formatação no valor retornado. Se o objetivo for formatar um valor numérico como uma string, favor consultar sprintf() ou number_format().

Parâmetros

`value`

A variável que será convertida para o tipo string.

`value` pode ser qualquer tipo escalar, null, ou um objeto (object) que implemente o método __toString(). **strval()** não pode ser usado em arrays ou em objetos que não implementem o método __toString().

Valor Retornado

O valor em string do parâmetro `value`.

Examples

Exemplo #1 Exemplo de strval() usando o método mágico __toString().

```php
<?php
class StrValTest
{
    public function __toString()
    {
        return __CLASS__;
    }
}

// Prints 'StrValTest'
echo strval(new StrValTest);
?>
```

Veja também

- boolval() - Obtém o valor booleano de uma variável
- floatval() - Obtém o valor em ponto flutuante da variável
- intval() - Obtém o valor em número inteiro de uma variável
- settype() - Define o tipo de uma variável
- sprintf() - Retona uma string formatada
- number_format() - Formata um número com milhares agrupados
- Manipulação de tipos
- __toString()

unserialize

unserialize

(PHP 4, PHP 5, PHP 7, PHP 8)

unserialize — Cria um valor PHP a partir de uma representação armazenada

Descrição

function **unserialize**(string `$data`, array `$options` = []): mixed

unserialize() toma uma única variável serializada e converte-a de volta em um valor PHP.

> **Aviso**
>
> Entradas de usuário não confiáveis não devem ser passadas a **unserialize()** independente dos valores do elemento `allowed_classes` do parâmetro `options`. Desserialização pode resultar em um código carregado e executado devido a uma instanciação de objeto e auto-carregamento, e um usuário malicioso pode ser capaz de explorar isto. Use um formato padrão e seguro de troca de dados como JSON (via json_decode() e json_encode()) se for necessário passar dados serializados ao usuário.
>
> Se for necessário desserializar dados armazenados externamente, considere o uso de hash_hmac() para validação de dados. Certifique-se que os dados não foram modificados por outras pessoas.

Parâmetros

`data`

A string serializada.

Se a variável a ser desserializada for um objeto, após a reconstrução bem sucedida do objeto o PHP tentará automaticamente chamar os métodos __unserialize() ou __wakeup() (se existirem).

> **Nota** : **Diretiva unserialize_callback_func**
>
> A função de retorno especificada na diretiva unserialize_callback_func é chamada quando uma classe indefinida é desserializada. Se nenhuma função for especificada, o objeto será instanciado como __PHP_Incomplete_Class.

`options`

Quaisquer opções podem ser fornecidas a **unserialize()** como um array associativo.

Opções válidas

Nome	Tipo	Descrição
`allowed_classes`	array|bool	Pode ser um array de nomes de classe que devem ser aceitas, `false` para não aceitar nenhuma classe, ou `true` para aceitar todas as classes. Se esta opção for definida e **unserialize()** encontrar um objeto de uma classe que não deve ser aceita, o objeto será instanciado como uma __PHP_Incomplete_Class. Omitir esta opção é o mesmo que defini-la como `true`: o PHP tentará instanciar objetos de qualquer classe. Esta opção não afeta Enumerações.
`max_depth`	int	A profundidade máxima de estruturas permitidas durante a desserialização, com a intenção de prevenir sobrecarga de pilha. O limite padrão de profundidade é `4096` e pode ser desabilitado definindo-se `max_depth` para `0`.

Valor Retornado

O valor convertido é retornado, e pode ser bool, int, float, string, array ou object.

Caso a string informada não for desserializável, `false` é retornado e um erro de nível `E_WARNING` é emitido.

Erros/Exceções

Objetos podem lançar exceções da classe Throwable em seus manipuladores de desserialização.

A partir do PHP 8.4.0, se o elemento `allowed_classes` do parâmetro `options` não for um array de nomes de classes nem um bool, **unserialize()** lança exceções TypeError e ValueError.

Registro de Alterações

Versão	Descrição
8.4.0	Agora lança exceções TypeError e ValueError se o elemento `allowed_classes` de `options` não for um array de nomes de classes nem um bool.
8.4.0	Desserializar strings usando a tag maiúscula `"S"` foi descontinuado; use a tag minúscula `"s"` em vez disso.
8.3.0	Agora emite um `E_WARNING` quando a string de entrada tem dados não consumidos.
8.3.0	Agora emite um `E_WARNING` quando a string informada não for desserializável; anteriormente um `E_NOTICE` era emitido.
7.4.0	Adicionado elemento `max_depth` do array `options` para definir a profundidade máxima de estruturas permitidas durante desserialização.
7.1.0	O elemento `allowed_classes` de `options` agora é estritamente tipado, isto é, se algo que não seja um array ou um bool for fornecido, **unserialize()** retorna `false` e emite um erro de nível `E_WARNING`.

Examples

Exemplo #1 Exemplo de unserialize()

```php
<?php
// Aqui, será usado unserialize() para carregar dados de sessão para o
// array $session_data a partir de uma string selecionada de um banco de dados.
// Este exemplo complementa o descrito em serialize().

$conn = odbc_connect("webdb", "php", "chicken");
$stmt = odbc_prepare($conn, "SELECT data FROM sessions WHERE id = ?");
$sqldata = array($_SERVER['PHP_AUTH_USER']);
if (!odbc_execute($stmt, $sqldata) || !odbc_fetch_into($stmt, $tmp)) {
    // se a execute ou fetch falharem, inicializa como um array vazio
    $session_data = array();
} else {
    // agora os dados serializados devem estar em $tmp[0].
    $session_data = unserialize($tmp[0]);
    if (!is_array($session_data)) {
        // algo deu errado, inicializa como um array vazio
        $session_data = array();
    }
}
?>
```

Exemplo #2 Exemplo de unserialize_callback_func

```php
<?php
$serialized_object='O:1:"a":1:{s:5:"value";s:3:"100";}';

ini_set('unserialize_callback_func', 'mycallback'); // define a função de retorno de chamada

function mycallback($classname)
{
    // apenas inclua um arquivo contendo sua definição de classe
    // $classname serve para determinar que definição de classe é requerida
    var_dump($classname);
}

unserialize($serialized_object);
?>
```

Notas

> **Aviso**
>
> `false` é retornado tanto em caso de um erro quanto no caso de desserializar o valor `false` serializado. É possível capturar este caso especial comparando-se `data` com `serialize(false)` ou capturando-se o erro de nível `E_WARNING` emitido.

Veja também

- json_encode() - Retorna a representação JSON de um valor
- json_decode() - Decodifica uma string JSON
- hash_hmac() - Gera um valor de hash com chave usando o método HMAC
- serialize() - Gera uma representação armazenável de um valor
- Auto-carregamento de Classes
- unserialize_callback_func
- unserialize_max_depth
- __wakeup()
- __serialize()
- __unserialize()

unset

unset

(PHP 4, PHP 5, PHP 7, PHP 8)

unset — **unset()** remove a definição de uma variável informada

Descrição

function **unset**(mixed `$var`, mixed `...$vars`): void

unset() destrói as variáveis especificadas.

O comportamento de **unset()** dentro de uma função pode variar dependendo de qual tipo de variável está tentando-se destruir.

Se uma variável global tornar-se indefinida por **unset()** dentro de uma função, apenas a variável local é destruída. A variável no ambiente que chamou a função manterá o mesmo valor anterior à chamada a **unset()**.

Exemplo #1 Usando unset()

```php
<?php
function destroy_foo()
{
    global $foo;
    unset($foo);
}

$foo = 'bar';
destroy_foo();
echo $foo;
?>
```

Para remover a definição de uma variável global usando **unset()** dentro de uma função, use o array *$GLOBALS*:

Exemplo #2 unset() com variável global

```php
<?php
function foo()
{
    unset($GLOBALS['bar']);
}

$bar = "alguma coisa";
foo();
?>
```

Se uma variável que é PASSADA POR REFERÊNCIA tiver sua definição removida por **unset()** dentro de uma função, apenas a variável local é destruída. A variável no ambiente que chamou a função manterá o mesmo valor anterior à chamada a **unset()**.

Exemplo #3 unset() com referência

```php
<?php
function foo(&$bar)
{
    unset($bar);
    $bar = "blah";
}

$bar = 'alguma coisa';
echo "$bar\n";

foo($bar);
echo "$bar\n";
?>
```

Se uma variável estática tiver a definição removida por **unset()** dentro de uma função, **unset()** destrói a variável apenas no contexto do resto da função. Chamadas subsequentes à função irão restaurar o valor anterior da variável.

Exemplo #4 unset() com variável estática

```php
<?php
function foo()
{
    static $bar;
    $bar++;
    echo "Antes de unset: $bar, ";
    unset($bar);
    $bar = 23;
    echo "depois de unset: $bar\n";
}

foo();
foo();
foo();
?>
```

Parâmetros

`var`

A variável que terá sua definição removida.

`vars`

Mais variáveis que terão definições removidas.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #5 Exemplo de unset()

```php
<?php
// destrói uma única variável
unset($foo);

// destrói um único elemento de um array
unset($bar['quux']);

// destrói mais de uma variável
unset($foo1, $foo2, $foo3);
?>
```

Notas

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

> **Nota** :
>
> É possível remover definições de propriedades de objeto visíveis no contexto atual.
>
> Se declarados, o método __get() é chamado ao acessar uma propriedade indefinida, e o método __set() é chamado ao definir uma propriedade indefinida.

> **Nota** :
>
> Não é possível remover a definição de `$this` dentro de um método do objeto.

> **Nota** :
>
> Ao usar **unset()** em propriedades de objeto inacessíveis, o método de sobrecarga __unset() será chamado, caso esteja declarado.

Veja também

- isset() - Determina se uma variável está declarada e é diferente de null
- empty() - Determina se uma variável está vazia
- __unset()
- array_splice() - Remove uma parte array e a substitui por outra coisa
- Conversão (unset)

var_dump

var_dump

(PHP 4, PHP 5, PHP 7, PHP 8)

var_dump — Despeja na saída informações sobre uma variável

Descrição

function **var_dump**(mixed `$value`, mixed `...$values`): void

Esta função mostra na saída uma informação estruturada sobre uma ou mais expressões, incluindo o tipo e o valor. Arrays e objetos são explorados recursivamente com valores identados para mostrar a estrutura.

Todas as propriedades públicas, privadas e protegidas de objetos serão retornadas na saída a menos que o objeto implemente um método __debugInfo().

> **Dica**

> Assim como qualquer coisa que envia seu resultado diretamente para o navegador, as funções de controle de saída podem ser usadas para capturar a saída desta função e salvá-la em uma string, por exemplo.

Parâmetros

`value`

A expressão a ser despejada.

`values`

Mais expressões a despejar.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de var_dump()

```php
<?php
$a = array(1, 2, array("a", "b", "c"));
var_dump($a);
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  array(3) {
    [0]=>
    string(1) "a"
    [1]=>
    string(1) "b"
    [2]=>
    string(1) "c"
  }
}
```

```php
<?php

$b = 3.1;
$c = true;
var_dump($b,$c);

?>
```

O exemplo acima produzirá:

```
float(3.1)
bool(true)
```

Veja também

- print_r() - Exibe informação legível sobre uma variável
- debug_zval_dump() - Exibe na saída uma representação em string de uma estrutura zval interna
- var_export() - Mostra ou retorna uma representação string analisável de uma variável
- __debugInfo()

var_export

var_export

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

var_export — Mostra ou retorna uma representação string analisável de uma variável

Descrição

function **var_export**(mixed `$value`, bool `$return = false`): ?string

var_export() obtém informação estruturada sobre a variável informada. Ela é similar a var_dump() com uma exceção: a representação retornada é um código PHP válido.

Parâmetros

`value`

A variável a ser exportada.

`return`

Se usado e definido para `true`, **var_export()** irá retornar a representação da variável ao invés de exibí-la.

Valor Retornado

Retorna a representação da variável quando o parâmetro `return` é usado e avaliado para `true`. Caso contrário, esta função retornará `null`.

Registro de Alterações

Versão	Descrição
8.2.0	Nomes de classe exportados agora são completamente qualificadas; anteriormente, a barra invertida no início era omitida.
7.3.0	Agora exporta objetos stdClass como um array convertido para um objeto (`(object) array( ... )`), ao invés de usar o método inexistente **stdClass::__setState()**. O efeito prático é que agora stdClass é exportável e o código resultante funcionará até mesmo em versões anteriores do PHP.

Examples

Exemplo #1 Exemplos de var_export()

```php
<?php
$a = array (1, 2, array ("a", "b", "c"));
var_export($a);
?>
```

O exemplo acima produzirá:

```
array (
  0 => 1,
  1 => 2,
  2 =>
  array (
    0 => 'a',
    1 => 'b',
    2 => 'c',
  ),
)
```

```php
<?php

$b = 3.1;
$v = var_export($b, true);
echo $v;

?>
```

O exemplo acima produzirá:

```
3.1
```

Exemplo #2 Exportando stdClass (a partir do PHP 7.3.0)

```php
<?php
$person = new stdClass;
$person->name = 'ElePHPant ElePHPantsdotter';
$person->website = 'https://php.net/elephpant.php';

var_export($person);
```

O exemplo acima produzirá:

```
(object) array(
   'name' => 'ElePHPant ElePHPantsdotter',
   'website' => 'https://php.net/elephpant.php',
)
```

Exemplo #3 Exportando classes

```php
<?php
class A { public $var; }
$a = new A;
$a->var = 5;
var_export($a);
?>
```

O exemplo acima produzirá:

```
\A::__set_state(array(
   'var' => 5,
))
```

Exemplo #4 Usando __set_state()

```php
<?php
class A
{
    public $var1;
    public $var2;
```

```php
    public static function __set_state($an_array)
    {
        $obj = new A;
        $obj->var1 = $an_array['var1'];
        $obj->var2 = $an_array['var2'];
        return $obj;
    }
}

$a = new A;
$a->var1 = 5;
$a->var2 = 'foo';

eval('$b = ' . var_export($a, true) . ';'); // $b = A::__set_state(array(
                                            //    'var1' => 5,
                                            //    'var2' => 'foo',
                                            // ));

var_dump($b);
?>
```

O exemplo acima produzirá:

```
object(A)#2 (2) {
  ["var1"]=>
  int(5)
  ["var2"]=>
  string(3) "foo"
}
```

Notas

> **Nota** :
>
> Variáveis do tipo resource não podem ser exportadas por esta função.

> **Nota** :
>
> **var_export()** não lida com referência circular já que seria praticamente impossível gerar código PHP analisável para isto. Se for desejado fazer algo com a representação completa de um array ou objeto, use serialize().

> **Aviso**
>
> Antes do PHP 8.2.0, quando **var_export()** exporta objetos, a barra inicial no início não é incluída no nome de classes com namespace para máxima compatibilidade.

> **Nota** :
>
> Para ser possível avaliar o código PHP gerado por **var_export()**, todos os objetos processados devem implementar o método mágico __set_state. A única exceção é stdClass, que é exportada usando um array convertido para um objeto.

Veja também

- print_r() - Exibe informação legível sobre uma variável
- serialize() - Gera uma representação armazenável de um valor
- var_dump() - Despeja na saída informações sobre uma variável

Expressões Regulares (Compatíveis com Perl)

Introdução

A sintaxe para padrões usados nestas funções assemelham-se com Perl. A expressão precisa estar incluída entre delimitadores, uma barra (/), por exemplo. Qualquer caractere pode ser usado para delimitar desde que não seja alfanumérico ou barra invertida (\). Se o caractere delimitador tiver de ser usado na própria expressão, ele necessitará de escape. Desde o PHP 4.0.4, pode-se também usar delimitadores no estilo Perl (), {}, [], e <>. Veja Sintaxe de Expressões para mais detalhes.

No final do delimitador pode ser seguido por vários modificadores que afetam a combinação. Veja Modificadores de Expressão.

> **Nota** :
>
> Esta extensão mantém um cache global por thread de expressões regulares compiladas (até 4096).

> **Aviso**
>
> É necessário estar ciente de algumas limitações da PCRE. Leia » http://www.pcre.org/pcre.txt para mais informação.

A biblioteca PCRE fornece um conjunto de funções que implementam correspondência de expressões regulares usando a mesma sintaxe e semântica do Perl 5, com apenas algumas diferenças (veja abaixo). A implementação atual corresponde à do Perl 5.005.

Instalação/Configuração

Índice

Instalação

A extensão PCRE é uma extensão do núcleo do PHP, portanto está sempre habilitada. Por padrão, esta extensão é compilada usando a biblioteca PCRE incluída. Alternativamente, uma biblioteca PCRE externa pode ser usada passando a opção de configuração **--with-pcre-regex=DIR** onde `DIR` é a localização do PCRE incluir e arquivos de biblioteca. Recomenda-se utilizar PCRE 8.10 ou mais recente; a partir do PHP 7.3.0, o PCRE2 é necessário.

A compilação just-in-time do PCRE é suportada por padrão, e pode ser desabilitada com a opção de configuração **--without-pcre-jit** a partir do PHP 7.0.12.

A versão do PHP para Windows tem suporte nativo para esta extensão. Nenhuma extensão adicional precisa ser carregada para usar essas funções.

PCRE é um projeto ativo e, à medida que muda, também muda a funcionalidade do PHP que depende dele. É possível que certas partes da documentação do PHP estejam desatualizadas, pois podem não cobrir os recursos mais recentes que o PCRE oferece. Para obter uma lista de alterações, consulte o » registro de alterações da biblioteca PCRE e também o seguinte histórico do PCRE incluído:

Histórico de atualização da biblioteca PCRE incluída

Versão do PHP	Versão da PCRE atualizada	Notas
8.2.0	10.40	
8.1.0	10.39	
7.4.12, 8.0.0	10.35	
7.4.6	10.34	
7.4.0	10.33	
7.3.0	10.32	
7.2.0	8.41	
7.0.3	8.38	Consulte: CVE-2015-8383, CVE-2015-8386, CVE-2015-8387, CVE-2015-8389, CVE-2015-8390, CVE-2015-8391, CVE-2015-8393, CVE-2015-8394
7.0.0	8.37	Consulte: CVE-2015-2325, CVE-2015-2326

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração da PCRE

Nome	Padrão	Modificável	Registro de Alterações
pcre.backtrack_limit	"1000000"	INI_ALL	
pcre.recursion_limit	"100000"	INI_ALL	
pcre.jit	"1"	INI_ALL	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`pcre.backtrack_limit` int

Limite de rastreamento retroativo da PCRE. Padrão é de 100000 para PHP < 5.3.7.

`pcre.recursion_limit` int

Limite de recursões da PCRE. Nota que se você definir este valor para um número bem alto poderá consumir toda a pilha de processo disponível e eventualmente travar o PHP (devido a alcançar o limite do tamanho da pilha imposto pelo sistema operacional).

`pcre.jit` bool

Configura se a compilação just-in-time do PCRE será utilizada.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

Constantes PREG

Constantes	Descrição	Desde
PREG_PATTERN_ORDER (int)	Ordena os resultados de modo que *$matches[0]* seja um array de todas as correspondências da expressão, *$matches[1]* seja um array de strings combinadas pela primeira sub-expressão, e assim por diante. Esta opção é somente usada com preg_match_all().	
PREG_SET_ORDER (int)	Ordena os resultados de modo que *$matches[0]* seja um array do primeiro conjunto de correspondências, *$matches[1]* seja um array do segundo conjunto de correspondências, e assim por diante. Esta opção é somente usada com preg_match_all().	
PREG_OFFSET_CAPTURE (int)	Se esta opção for definida, para cada correspondência a posição do byte também será retornada. Observe que isto muda os valores de retorno para um array onde cada elemento é um array consistindo da string correspondida no elemento 0 e sua posição na string de entrada no elemento 1.	
PREG_SPLIT_NO_EMPTY (int)	Esta opção configura preg_split() para retornar apenas pedaços não vazios.	
PREG_SPLIT_DELIM_CAPTURE (int)	Esta opção configura preg_split() para capturar expressões entre parênteses em expressões delimitadas.	
PREG_SPLIT_OFFSET_CAPTURE (int)	Veja a descrição de PREG_OFFSET_CAPTURE. Esta opção é usada somente para preg_split().	
PREG_UNMATCHED_AS_NULL (int)	Esta opção informa a preg_match() e preg_match_all() para incluir sub-expressões não correspondidas em *$matches* como null. Sem esta opção, sub-expressões sem correspondência são reportadas como strings vazias, como se fossem correspondências vazias. Informar esta opção permite diferenciar entre esses dois casos. Com preg_match(), subpadrões não correspondidos no final são omitidos de *$matches* completamente, a menos que esta opção seja usada; a partir do PHP 7.4.0, a opção os reporta como null, de modo que *$matches* sempre tenha o mesmo tamanho.	7.2.0
PREG_NO_ERROR (int)	Retornada pela preg_last_error() se não houver erros.	5.2.0
PREG_INTERNAL_ERROR (int)	Retornada pela preg_last_error() se houver um erro interno na PCRE.	5.2.0

Constantes	Descrição	Desde
PREG_BACKTRACK_LIMIT_ERROR (int)	Retornada pela preg_last_error() se o limite de rastreamento retroativo tiver sido ultrapassado.	5.2.0
PREG_RECURSION_LIMIT_ERROR (int)	Retornada pela preg_last_error() se o limite de recursividade tiver sido ultrapassado.	5.2.0
PREG_BAD_UTF8_ERROR (int)	Retornada pela preg_last_error() se o último erro foi causado por dados UTF-8 mal formados (somente ao usar uma expressão regular em modo UTF-8).	5.2.0
PREG_BAD_UTF8_OFFSET_ERROR (int)	Retornada pela preg_last_error() se o deslocamento não tiver correspondido ao início de um ponto de código UTF-8 válido (somente ao usar uma expressão regular em modo UTF-8).	5.3.0
PREG_JIT_STACKLIMIT_ERROR (int)	Retornada por preg_last_error() se a última função PCRE falhou por falta de espaço em pilha do JIT.	7.0.0
PCRE_VERSION (string)	Versão da PCRE e data de lançamento (e.g. "`7.0 18-Dec-2006`").	5.2.4
PCRE_VERSION_MAJOR (int)	Número de versão principal do PCRE.	
PCRE_VERSION_MINOR (int)	Número de versão secundária do PCRE.	
PCRE_JIT_SUPPORT (bool)	Indica se o suporte PCRE JIT está disponível.	
PREG_GREP_INVERT (int)	Retorna os elementos que não correspondem à expressão informada.	

Examples

Exemplo #1 Exemplos de expressões válidas

- `/<\/\w+>/`
- `|(\d{3})-\d+|Sm`
- `/^(?i)php[34]/`
- `{^\s+(\s+)?$}`

Exemplo #2 Exemplos de expressões inválidas

- `/href='(.*)'` - faltando o delimitador final
- `/\w+\s*\w+/J` - modificador 'J' desconhecido
- `1-\d3-\d3-\d4|` - faltando delimitador inicial

Expressões PCRE

Índice

Sintaxe das expressões regulares PCRE

Índice

Introdução

A sintaxe e a semântica das expressões regulares suportadas pelo PCRE estão descritas nesta seção. Expressões regulares também estão descritas na documentação Perl e em vários outros livros, alguns dos quais possuem exemplos abundantes. "Dominando Expressões Regulares" de Jeffrey Friedl, publicado pela O'Reilly e Editora Alta Books (ISBN 978-85-7608-394-8), cobre-as com riqueza de detalhe. A descrição aqui serve como documentação de referência.

Uma expressão regular é um padrão que corresponde a uma string de entrada da esquerda para a direita. Na maior parte das vezes, um caractere representa a si mesmo em uma expressão e casa com o caractere correspondente na string de entrada. Como um exemplo trivial, a expressão `The quick brown fox` casa com uma parte de uma string de entrada que é idêntica a si mesma.

Delimitadores

Ao usar as funções PCRE, é necessário que a expressão esteja entre *delimitadores*. Um delimitador pode ser qualquer caractere não alfanumérico que não seja uma barra invertida e que não corresponda a um espaço em branco. Espaços em branco antes de um delimitador válido são silenciosamente ignorados.

Delimitadores comumente usados são barras (`/`), cerquilhas (`#`) e til (`~`). Os exemplos a seguir são padrões com delimitadores válidos.

```
/foo bar/
#^[^0-9]$#
+php+
%[a-zA-Z0-9_-]%
```

Também é possível usar delimitadores no estilo parênteses, onde os delimitadores inicial e final são os caracteres de abertura e fechamento, respectivamente. `()`, `{}`, `[]` e `<>` são todos pares válidos de delimitadores nesse estilo.

```
(esta [é] uma (expressão))
{esta [é] uma (expressão)}
[esta [é] uma (expressão)]
<esta [é] uma (expressão)>
```

Delimitadores no estilo parênteses não precisam ser escapados quando são usados como metacaracteres dentro da expressão, mas como acontece com outros delimitadores, eles precisam ser escapados quando são usados como caracteres literais.

Se o delimitador precisa ser correspondido dentro da expressão, ele precisa ser escapado usando uma barra invertida. Se o delimitador aparece com muita frequência dentro da expressão, é uma boa ideia escolher um delimitador diferente para melhorar a legibilidade.

```
/http:\/\//
#http://#
```

A função preg_quote() pode ser usada para escapar uma string a ser injetada em uma expressão e seu segundo parâmetro opcional pode ser usado para especificar o delimitador a ser escapado.

Pode-se adicionar modificadores de expressão após o delimitador final. Este é um exemplo de correspondência insensível a maiúsculas/minúsculas usando o modificador "i":

```
#[a-z]#i
```

Metacaracteres

O poder das expressões regulares vem da habilidade de incluir alternativas e repetições na expressão. Elas são codificadas na expressão pelo uso de *metacaracteres*, que não representam a si mesmos mas ao invés disso são interpretados de um modo especial.

Existem dois conjuntos diferentes de metacaracteres: aqueles que são reconhecidos em qualquer lugar na expressão exceto quando estão dentro de colchetes, e aqueles que são reconhecidos dentro de colchetes. Fora de colchetes, os metacaracteres são:

Metacaracteres fora de colchetes

Metacaractere	Descrição
\	caractere de escape geral com vários usos
^	declara início da string (ou da linha, em modo multi-linhas)
$	declara o final da string ou posição antes de um caractere de nova linha (ou final da linha, em modo multi-linhas)
.	corresponde a qualquer caractere exceto nova linha (por padrão)
[	inicia definição de classe de caractere
]	termina definição de classe de caractere
|	início de ramo alternativo
(	início de sub-expressão
)	fim de sub-expressão
?	estende o significado de (, também significa quantificador 0 ou 1, e também torna preguiçosos os quantificadores gananciosos (consulte repetição)
*	quantificador de 0 ou mais
+	quantificador de 1 ou mais
{	inicia quantificador de mínimo/máximo
}	termina quantificador de mínimo/máximo

A parte de uma expressão dentro de colchetes é chamada de classe de caractere. Em uma classe de caractere os únicos metacaracteres são:

Metacaracteres dentro de colchetes (*classes de caracteres*)

Metacaractere	Descrição
\	caractere de escape geral
^	nega a classe, mas apenas se for o primeiro caractere
-	indica a faixa de caracteres

As seções a seguir descrevem o uso de cada um dos metacaracteres.

Sequências de escape

A barra invertida tem vários usos. Primeiramente, se ela for seguida por um caractere não alfanumérico, ela remove qualquer significado especial que este caractere possa ter. Este uso da barra invertida como um caractere de escape se aplica tanto dentro quanto fora de classes de caracteres.

Por exemplo, se a intenção é casar com um caractere "*", deve-se escrever "*" na expressão. Isto é aplicável mesmo que o próximo caractere pudesse ser interpretado como um metacaractere, por isso é sempre seguro preceder um caractere não alfanumérico com "\" para especificar que ele irá representar a si mesmo. Em particular, se a intenção for casar com uma barra invertida, deve-se escrever "\\".

> **Nota** :
>
> strings do PHP com aspas simples e aspas duplas têm significado especial para a barra invertida. Assim, se \ precisa corresponder a uma expressão regular \\, então "\\\\" ou '\\\\' precisam ser usadas no código PHP.

Se uma expressão for compilada com a opção PCRE_EXTENDED, espaços em branco na expressão (que não estejam em uma classe de caracteres) e caracteres entre um "#" fora de uma classe de caracteres e o próximo caractere de nova linha são ignorados. Uma barra invertida de escape pode ser usada para incluir um espaço ou um caractere "#" como parte da expressão.

Um segundo uso da barra invertida fornece um meio de codificar caracteres não imprimíveis de uma maneira visível. Não há restrição para presença de caracteres não imprimíveis, exceto pelo zero binário que termina uma expressão, mas quando uma expressão está sendo preparada durante a edição de texto, normalmente é mais fácil usar uma das sequências de escape a seguir do que usar o caractere binário que ela representa:

\a

sinal sonoro, isto é, o caracater BEL (hexa 07)

\cx

"Crtl-x", onde x é qualquer caractere

\e

escape (hexa 1B)

\f

alimentação de formulário (hexa 0C)

\n

nova linha (hexa 0A)

\p{xx}

um caractere com a propriedade xx, veja propriedades Unicode para mais informação

\P{xx}

um caractere sem a propriedade xx, veja propriedades Unicode para mais informação

\r

retorno de carro (hexa 0D)

\R

quebra de linha: corresponde a \n, \r e \r\n

t

tabulação (hexa 09)

xhh

caractere com o código hexadecimal hh

ddd

caractere com o código octal ddd, ou referência retroativa

O efeito preciso de "`\cx`" é o seguinte: se "`x`" for uma letra minúscula, ela é convertida para maiúscula. Depois, o bit 6 do caractere (hexa 40) é invertido. Portanto, "`\cz`" se torna hexa 1A, mas "`\c{`" se torna hexa 3B, enquanto que "`\c;`" se torna hexa 7B.

Depois de "`\x`", até dois dígitos hexadecimais são lidos (letras podem ser minúsculas ou maiúsculas). No *modo UTF-8*, "`\x{...}`" é permitido, onde o conteúdo entre chaves é uma string de dígitos hexadecimais. É interpretado como um caractere UTF-8 cujo número de código é o número hexadecimal informado. A sequência de escape hexadecimal original, `\xhh`, corresponde a um caractere UTF-8 de dois bytes se o valor for maior que 127.

Depois de "`\0`" até dois dígitos octais adicionais são lidos. Em ambos os casos, se eles forem menos que dois, apenas os presentes são usados. Assim, a sequência "`\0\x\07`" especifica dois zeros binários seguidos por um caractere BEL. Certifique-se de fornecer dois dígitos depois do zero inicial se o caractere seguinte for, por acaso, um dígito octal.

A interpretação de uma barra invertida seguida por um dígito que não seja 0 é complicada. Fora de uma classe de caracteres, o PCRE lê este dígito e os seguintes como um número decimal. Se o número for menor que 10, ou se houve pelo menos esta quantidade de grupos em parênteses na expressão, a sequência inteira é interpretada como uma *referência retroativa*. Uma descrição de como isto funciona é fornecida mais abaixo, logo após a discussão sobre sub-expressões entre parênteses.

Dentro de uma classe de caracteres, ou se o número decimal for maior que 9 e não houve essa quantidade de sub-expressões de captura, o PCRE lê novamente até três dígitos octais depois da barra invertida, e gera um único byte a partir dos 8 bits menos significativos do valor. Quaisquer dígitos subsequentes representam a si mesmos. Por exemplo:

040

é uma outra forma de escrever um espaço em branco (decimal 32)

40

é a mesma coisa, desde que haja menos que 40 sub-expressões de captura

7

é sempre uma referência retroativa

11

pode ser uma referência retroativa, ou outra forma de escrever uma tabulação (decimal 9)

011

é sempre uma tabulação (decimal 9)

0113

é uma tabulação seguida pelo caractere "3"

113

é o caractere com o código octal 113 (decimal 75), já que não pode haver mais que 99 referências retroativas

377

é um byte que consiste inteiramente de bits com o valor 1 (decimal 255)

81

pode ser uma referência retroativa, ou um zero binário seguido pelos caracteres "8" e "1"

Observe que valores octais de 100 ou maiores não podem ser introduzidos com um zero na frente, porque não mais que três dígitos octais são lidos no total.

Todas as sequências que definem um valor de byte único podem ser usadas tanto dentro quanto fora de classes de caracteres. Além disso, dentro de uma classe de caracteres, a sequência "`\b`" é interpretada como o caractere "backspace" (hexa 08). Fora de uma classe de caracteres ela tem um significado diferente (veja abaixo).

O terceiro uso da barra invertida é para especificar tipos genéricos de caracteres:

\d

qualquer dígito decimal

\D

qualquer caractere que não seja um dígito decimal

\h

qualquer caractere de espaço em branco horizontal

\H

qualquer caractere que não seja um espaço em branco horizontal

\s

qualquer caractere que represente um espaço em branco

\S

qualquer caractere que não represente um espaço em branco

\v

qualquer caractere de espaço em branco vertical

\V

qualquer caractere que não seja um espaço em branco vertical

\w

qualquer caractere de uma "palavra"

\W

qualquer caractere que não seja de uma "palavra"

Cada par de sequência de escape particiona o conjunto completo de caracteres em dois conjuntos desconexos. Cada caractere informado corresponde a um, e apenas um, de cada par.

Os caracteres que representam um espaço em branco horizontal são tabulação horizontal (9), nova linha (10), alimentação de formulário (12), retorno de carro (13), e espaço (32). No entanto, em casos de correspondências específicas de localidade, caracteres com pontos de código na faixa de 128 a 255 podem também ser considerados como caracteres de espaço em branco, por exemplo, NBSP (A0).

Uma caractere de uma "palavra" é qualquer letra ou dígito ou o caractere de sublinhado "_", isto é, qualquer caractere que possa fazer parte de uma "*palavra*" em Perl. A definição de letras e dígitos é controlada pelas tabelas de caracteres do PCRE, e pode varia se correspondências específicas de localidade estiverem ocorrendo. Por exemplo, na localidade "pt-BR" (português do Brasil), alguns códigos de caracteres maiores que 127 são usados para letras acentuadas, e estes são correspondidos por `\w`.

Estas sequências de tipo de caracteres podem aparecer tanto dentro como fora de classes de caracteres. Cada uma corresponde a um caracteres do tipo apropriado. Se o ponto de correspondência atual estiver no final da string de entrada, todos eles irão falhar, já que não haverá caractere para correspondência.

O quarto uso da barra invertida é para certas afirmações simples. Uma afirmação especifica uma condição que deve ser encontrada em um ponto particular de uma correspondência, sem consumir nenhum caractere da string de entrada. O uso de sub-expressões para afirmações mais complicadas é descrito abaixo. As afirmaçõs que usam barra invertida são:

\b

borda de palavra

\B

não é borda de palavra

\A

início da string de entrada (independente do modo multi-linhas)

\Z

final da string de entrada ou caractere de nova linha no final (independente do modo multi-linhas)

\z

final da string de entrada (independente do modo multi-linhas)

\G

primeira posição de correspondência na string de entrada

Estas afirmações podem não aparecer em classes de caracteres (mas observe que "`\b`" tem um significado diferente, que é o caractere de "backspace" dentro de uma classe de caracteres).

Uma borda de palavra é uma posição na string de entrada onde nem o caractere atual nem o caractere anterior correspondem a `\w` ou `\W` ao mesmo tempo (isto é, um deles corresponde a `\w` e o outro corresponde a `\W`), ou o início ou final da string se o primeiro ou último caractere corresponder a `\w`, respectivamente.

As afirmações `\A`, `\Z` e `\z` diferem dos tradicionais circunflexo "^" e cifrão "$" (descritos em âncoras) porque elas somente são correspondidas exatamente no início e no final da string de entrada, não importa que opções estão definidas. Elas não são afetada pelas opções PCRE_MULTILINE ou PCRE_DOLLAR_ENDONLY. A diferença entre `\Z` e `\z` é que `\Z` corresponde antes de um caractere de nova linha que é o último caractere da string e também no final da string, enquanto que `\z` corresponde apenas no final.

A afirmação `\G` é verdadeira somente quando a posição de correspondência atual é no ponto inicial da corresponência, como especificado pelo argumento `offset` da função preg_match(). Ela difere de `\A` quando o valor de `offset` é não-zero.

`\Q` e `\E` podem ser usadas para ignorar metacaracteres na expressão. Por exemplo: `\w+\Q.$.\E$` irão corresponder a um ou mais caracteres de palavras, seguidos pelos literais `.$.` e ancorados no final da string. Observe que isto não muda o comportamento dos delimitadores; por exemplo, a expressão `#\Q#\E#$` não é válida, porque o segundo `#` marca o final da expressão, e o `\E#` é interpretado como modificador inválido.

`\K` pode ser usado para redefinir o início da correspondência. Por exemplo, a expressão `foo\Kbar` corresponde a "foobar", mas reporta que correspondeu a "bar". O uso de `\K` não interfere com a configuração de substrings capturadas. Por exemplo, quando a expressão `(foo)\Kbar` corresponde a "foobar", a primeira substring ainda será definida como "foo".

Propriedades de caracteres Unicode

Desde a versão 5.1.0, três sequências de escape adicionais para corresponder a tipo de caracteres genéricos estão disponíveis quando o *modo UTF-8* é selecionado. Elas são:

\p{xx}

um caractere com a propriedade xx

\P{xx}

um caractere sem a propriedade xx

\X

uma sequência Unicode estendida

Os nomes de propriedade representados por `xx` acima são limitados às propriedades de categoria geral do Unicode. Cada caractere tem exatamente uma propriedade deste tipo, especificada por uma abreviação de duas letras. Para compatibilidade com o Perl, uma negação pode ser especificada incluindo um circunflexo "^" entre a chave de abertura e o nome da propriedade. Por exemplo, `\p{^Lu}` é o mesmo que `\P{Lu}`.

Se somente uma letra for especificada com `\p` ou `\P`, serão incluídas todas as propriedades que iniciam com essa letra. Neste caso, na ausência de negação, as chaves na sequência de escape são opcionais; estes dois exemplos têm o mesmo efeito:

```
\p{L}
\pL
```

Códigos de propriedade suportados

Propriedade	Correspondências	Observações
`C`	Outro	
`Cc`	Controle	
`Cf`	Formato	
`Cn`	Não atribuído	
`Co`	Uso privado	
`Cs`	Substituto	
`L`	Letra	Inclui as seguintes propriedades: `Ll`, `Lm`, `Lo`, `Lt` e `Lu`.
`Ll`	Letra minúscula	
`Lm`	Letra modificadora	
`Lo`	Outra letra	
`Lt`	Letra de título	
`Lu`	Letra maiúscula	
`M`	Marca	
`Mc`	Marca especial	
`Me`	Marca de envolvimento	
`Mn`	Marca de largura zero	
`N`	Número	
`Nd`	Número decimal	
`Nl`	Letra numérica	
`No`	Outro número	
`P`	Pontuação	
`Pc`	Pontuação de conexão	
`Pd`	Pontuação de traço	
`Pe`	Pontuação de fechamento	
`Pf`	Pontuação final	
`Pi`	Pontuação inicial	
`Po`	Outra pontuação	
`Ps`	Pontuação aberta	
`S`	Símbolo	
`Sc`	Símbolo de moeda	
`Sk`	Símbolo modificador	
`Sm`	Símbolo matemático	
`So`	Outro símbolo	Inclui emojis
`Z`	Separador	
`Zl`	Separador de linha	
`Zp`	Separador de parágrafo	
`Zs`	Separador de espaço	

Propriedades estendidas tais como `InMusicalSymbols` (símbolos musicais) não são suportadas pelo PCRE.

Especificar correspondência insensível a maiúsculas/minúsculas não afeta estas sequências de escape. Por exemplo, `\p{Lu}` sempre irá corresponder somente a letras maiúsculas.

Conjuntos de caracteres Unicode são definidos como pertencentes a certos tipos de escrita (ou tipo de alfabeto). Um caractere de um destes conjuntos pode ser correspondido usando um nome de tipo de escrita. Por exemplo:

- `\p{Greek}`

- `\P{Han}`

Aqueles que não são parte de um tipo de escrita identificado são agrupados como `Common` (comuns). A lista atual de tipos de escrita é:

Tipos de escrita suportados (nomes em inglês)

Arabic	Armenian	Avestan	Balinese	Bamum
Batak	Bengali	Bopomofo	Brahmi	Braille
Buginese	Buhid	Canadian_Aboriginal	Carian	Chakma
Cham	Cherokee	Common	Coptic	Cuneiform
Cypriot	Cyrillic	Deseret	Devanagari	Egyptian_Hieroglyphs
Ethiopic	Georgian	Glagolitic	Gothic	Greek
Gujarati	Gurmukhi	Han	Hangul	Hanunoo
Hebrew	Hiragana	Imperial_Aramaic	Inherited	Inscriptional_Pahlavi
Inscriptional_Parthian	Javanese	Kaithi	Kannada	Katakana
Kayah_Li	Kharoshthi	Khmer	Lao	Latin
Lepcha	Limbu	Linear_B	Lisu	Lycian
Lydian	Malayalam	Mandaic	Meetei_Mayek	Meroitic_Cursive
Meroitic_Hieroglyphs	Miao	Mongolian	Myanmar	New_Tai_Lue
Nko	Ogham	Old_Italic	Old_Persian	Old_South_Arabian
Old_Turkic	Ol_Chiki	Oriya	Osmanya	Phags_Pa
Phoenician	Rejang	Runic	Samaritan	Saurashtra
Sharada	Shavian	Sinhala	Sora_Sompeng	Sundanese
Syloti_Nagri	Syriac	Tagalog	Tagbanwa	Tai_Le
Tai_Tham	Tai_Viet	Takri	Tamil	Telugu
Thaana	Thai	Tibetan	Tifinagh	Ugaritic
Vai	Yi			

O escape `\X` corresponde a um agrupamento de grafema estendido Unicode. Um agrupamento de grafema estendido consiste em um ou mais caracteres Unicode que se combinam para formar um único glifo. Na verade, isso pode ser considerado como o equivalente Unicode de `.`, já que irá corresponder a um caractere composto, independente de quantos caracteres individuais são realmente usados para gerá-lo.

Em versões do PCRE mais antigas que 8.32 (que corresponde às versões do PHP antes da 5.4.14 que usavam a biblioteca PCRE interna), `\X` é equivalente a `(?>\PM\pM*)`. Isto é, corresponde a um caractere sem a propriedade de "marca", seguida por zero ou mais caracteres com a propriedadede "marca", e trata a sequência como um grupo atômico (veja abaixo). Caracteres com a propriedade de "marca" são tipicamente acentuações que afetam o caractere precedente.

A correspondência de caracteres por propriedade Unicode não é rápida porque o PCRE precisa pesquisar uma estrutura que contenha dados para mais que quinze mil caracteres. É por isso que as sequências de escape tradicionais como `\d` e `\w` não usam as propriedades Unicode no PCRE.

Âncoras

Fora de uma classe de caracteres, no modo padrão de correspondência, o caractere de circunflexo (`^`) é uma afirmação que é verdadeira somente se o ponto de correspondência atual estiver no início da string de entrada. Dentro de uma classe de caracteres, o circunflexo (`^`) tem um significado completamente diferente (veja abaixo).

O circunflexo (`^`) não precisa ser o primeiro caractere da expressão se um número de alternativas estiver envolvido, mas deve ser a primeira coisa em cada alternativa em que aparece se a expressão acabar correspondendo a esse ramo. Se todas as alternativas possíveis iniciarem com um circunflexo (`^`), isto é, se a expressão for restringida a corresponder apenas ao início da string, é dito que essa é uma expressão "ancorada". Também há outras construções que podem fazer com que uma expressão seja ancorada.

Um caractere de cifrão (`$`) é uma afirmação que é `true` somente se o ponto atual de correspondência estiver no final da string de entrada, ou imediatamente antes de um caractere de nova linha que é o último caractere na string (por padrão). Cifrão (`$`) não precisa ser o último caractere da expressão se um número de alternativas estiver envolvido, mas deve ser o último item em qualquer ramo em que ele aparecer. O cifrão não tem nenhum significado especial em uma classe de caracteres.

O significado do cifrão pode ser alterado de forma que ele corresponda apenas ao final da string, configurando-se a opção PCRE_DOLLAR_ENDONLY no momento da compilação ou da correspondência. Isto não afeta a afirmação \Z.

Os significados dos caracteres circunflexo e cifrão são alterados se a opção PCRE_MULTILINE estiver definida. Quando esse é o caso, eles irão corresponder imediatamente após e imediatamente antes de um caractere "\n" interno, respectivamente, além de corresponder ao início e ao final da string de entrada. Por exemplo, a expressão /^abc$/ corresponde à string de entrada "def\nabc" em modo multi-linhas, mas não em modo de linha única. Consequentemente, expressões que são ancoradas em modo de linha única porque todos os ramos iniciam com "^" não são ancoradas em modo multi-linhas. A opção PCRE_DOLLAR_ENDONLY é ignorada se PCRE_MULTILINE estiver definida.

Observe que as sequências \A, \Z, e \z podem ser usadas para corresponder ao início e ao final da string em ambos os modos e, se todos os ramos de uma expressão iniciarem com \A, ela estará sempre ancorada, independente se PCRE_MULTILINE estiver definida ou não.

Ponto

Fora de uma classe de caracteres, um ponto na expressão corresponde a qualquer caractere na string, incluindo os não imprimíveis, mas não o caractere de nova linha (por padrão). Se a opção PCRE_DOTALL estiver definida, os pontos corresponderão aos caracteres de nova linha. A manipulação do ponto é completamente independente da manipulação do circunflexo e do cifrão, sendo que a única relação entre eles é que ambos envolvem caracteres de nova linha. O ponto não tem nenhum significado especial em uma classe de caracteres.

C pode ser usada para corresponder a um único byte. Faz sentido no *modo UTF-8* onde o ponto final corresponde ao caractere inteiro que pode consistir de múltiplos bytes.

Classes de caracteres

Uma abertura de colchete introduz uma classe de caracteres, terminada por um fechamento de colchete. Um colchete de fechamento em si não é especial. Se um colchete de fechamento for requerido como membro da classe, ele deve ser o primeiro caractere na classe (após um circunflexo inicial, se presente) ou escapado com uma barra invertida.

Uma classe de caracteres corresponde a um único caractere na string; o caractere precisa estar no conjunto de caracteres definido pela classe, a menos que o primeiro caractere na classe seja um circunflexo, que nesse caso o caractere na string não pode estar no conjunto definido pela classe. Se um circunflexo for requerido como membro da classe, é preciso garantir que ele não seja o primeiro caractere, ou que seja escapado com uma barra invertida.

Por exemplo, a classe de caracteres [aeiou] corresponde a qualquer vogal minúscula sem acento, enquanto que [^aeiou] corresponde a qualquer caractere que não seja uma vogal minúscula sem acento. Observe que um circunflexo é apenas uma notação conveniente para especificar os caracteres que estão na classe enumerando aqueles que não estão. Não é uma afirmação: ela ainda consome um caractere da string de entrada, e falha se o ponteiro atual estiver no final da string.

Quando uma correspodência insensível a maiúsculas/minúsculas é definida, qualquer letra em uma classe representa tanto a forma maiúscula quanto a minúscula, então, por exemplo, uma classe [aeiou] insensível à forma corresponde tanto a "A" quanto a "a", e uma classe [^aeiou] insensível à forma não corresponde a "A", enquanto que um versão sensível à forma corresponderia.

Um caractere de nova linha nunca é tratado de modo especial em classes de caractere, independente da configuração da opção PCRE_DOTALL ou da opção PCRE_MULTILINE. Uma classe como [^a] sempre corresponderá a uma nova linha.

O caractere hífen (sinal de subtração) pode ser usado para especificar uma faixa de caracteres em uma classe. Por exemplo, [d-m] corresponde a qualquer letra entre d e m, inclusive. Se um hífen literal for requerido na classe, ele precisa ser escapado com uma barra invertida ou aparecer em uma posição onde não possa ser confundido com um indicador de faixa, tipicamente como o primeiro ou último caractere na classe.

Não é possível ter o caractere literal "]" como o caractere final de uma faixa. Uma expressão como [W-]46] é interpretada como uma classe de dois caracteres ("W" e "-") seguida pela string literal "46]", portanto corresponderia a "W46]" ou "-46]". Entretanto, se o "]" for escapado com uma barra invertida ele será interpretado como o final da faixa. Assim, [W-\]46] é interpretado como uma classe única contendo uma faixa única seguida por dois caracteres separados. A representação octal ou hexadecimal de "]" também pode ser usada para finalizar uma faixa.

As faixas operam na sequência da tabela ASCII. Elas também podem ser usadas para caracteres especificados numericamente, por exemplo [\000-\037]. Se uma faixa que inclui letras for usada quando uma correspondência insensível a maiúsculas/minúsculas estiver definida, ela corresponderá às letras em ambas as formas. Por exemplo, [W-c] é equivalente a [][\^_`wxyzabc], correspondida de maneira insensível à forma, e se tabelas de caracteres para a localidade "pt-BR" estiverem em uso, [\xc0-\xc3] correspondem à letra A com acentos tanto em maiúsuculas quanto em minúsculas.

Os tipos de caractere \d, \D, \s, \S, \w e \W também podem aparecer em uma classe de caracteres e adicionar à classe os caractares aos quais eles correspondem. Por exemplo, [\dABCDEF] corresponde a qualquer dígito hexadecimal. Um circunflexo pode convenientemente

ser usado com os tipos de caracteres maiúsculos para especificar um conjunto mais restrito de caracteres que a correspondência ao tipo de minúsculas. Por exemplo, a classe [^\W_] corresponde a qualquer letra ou dígito, mas não ao sublinhado "_".

Todos os caracteres não alfanuméricos que não sejam \, -, ^ (no início) e o terminador] não são especiais em classes de caracteres, mas não faz mal agum se forem escapados. O terminador de expressão é sempre especial e precisa ser escapado quando usado dentro de uma expressão.

O Perl suporta a notação POSIX para classes de caracteres. Ela usa nomes envolvidos por `[:` e `:]` dentro de colchetes. O PCRE também suporta esta notação. Por exemplo, `[01[:alpha:]%]` corresponde a "0", "1", qualquer caractere alfabético ou "%". Os nomes de classe suportados são:

Classes de caracteres

`alnum`	letras e dígitos
`alpha`	letras
`ascii`	caracteres com códigos 0 - 127
`blank`	espaço ou tabulação somente
`cntrl`	caracteres de controle
`digit`	dígitos decimais (o mesmo que \d)
`graph`	caracteres imprimíveis, excluindo o espaço
`lower`	letras minúsculas
`print`	caracteres imprimíveis, incluindo o espaço
`punct`	caracteres imprimíveis, excluindo letras e dígitos
`space`	espaço (não é idêntico a \s)
`upper`	letras maiúsculas
`word`	caracteres de "palavra" (o mesmo que \w)
`xdigit`	dígitos hexadecimais

Os caracteres da classe `space` são: tabulação horizontal (9), nova linha (10), tabulação vertical (11), alimentação de formulário (12), retorno de carro (13), e espaço (32). Observe que esta lista inclui o caractere de tabulação vertical (código 11). Isso faz com que "space" seja diferente de `\s`, que não inclui a tabulação vertical (para compatibilidade com o Perl).

O nome `word` é uma extensão do Perl, e `blank` é uma extensão GNU do Perl 5.8. Uma outra extensão do Perl é a negação, que é indicada por um caractere `^` após o caractere de dois-pontos. Por exemplo, `[12[:^digit:]]` corresponde a "1", "2" ou qualquer não-dígito.

No modo UTF-8, caracteres com valores maiores que 127 não correspondem a nenhuma das classes de caracteres POSIX. A partir da libpcre 8.10 algumas classes de caracteres são modificadas para usar proprieadades de caracteres Unicode, e nesse caso a restrição mencionada não se aplica. Refira-se ao » manual PCRE(3) para detalhes.

Propriedades de caracteres Unicode podem aparecer dentro de uma classe de caracteres. Elas não podem ser parte de uma faixa. O caractere hífen após uma classe de caractere Unicode irá corresponder literalmente. Tentar finalizar uma faixa com uma propriedade de caractere Unicode resultará em um alerta.

Alternância

Caracteres de barra vertical "|" são usados para separar expressões alternativas. Por exemplo, a expressão `gilbert|sullivan` correspondem tanto a "gilbert" quanto a "sullivan". Qualquer número de alternativas pode ser usado, e uma alternativa vazia é permitida (correspondendo a uma string vazia). O processo de correspondência testa uma alternativa de cada vez, da esquerda para a direita, e a primeira bem-sucedida é usada. Se as alternativas estiverem dentro de uma sub-expressão (definida abaixo), "bem-sucedida" significa correspondência ao restante da expressão principal assim como à alternativa dentro da sub-expressão.

É possível registrar qual alternativa foi correspondida usando `(*MARK:NAME)` ou `(*:NAME)`. Qualquer número desses verbos `(*MARK)` pode aparecer e seus nomes não precisam ser únicos. Quando uma correspondência for bem-sucedida, o nome do último `(*MARK:NAME)` encontrado será colocado entre as correspondências como se fosse um grupo de captura chamado `MARK` para que possa ser lido a partir dos `matches` de preg_match() e será passado para o `callback` de preg_replace_callback() etc.

Configuração de opções dentro da expressão

As configurações de PCRE_CASELESS, PCRE_MULTILINE, PCRE_DOTALL, PCRE_UNGREEDY, PCRE_EXTRA, PCRE_EXTENDED e PCRE_DUPNAMES podem ser alteradas de dentro da expressão através de uma sequência de letras de opção do Perl envolvidas entre "(?" e ")". As letras de opções são:

Letras de opções internas	
`i`	para PCRE_CASELESS
`m`	para PCRE_MULTILINE
`s`	para PCRE_DOTALL
`x`	para PCRE_EXTENDED
`U`	para PCRE_UNGREEDY
`X`	para PCRE_EXTRA (não mais suportada a partir do PHP 7.3.0)
`J`	para PCRE_INFO_JCHANGED

Por exemplo, (?im) define uma correspondência insensível a maiúsculas/minúsculas, com multi-linhas. Também é possível remover estas opções precendendo a letra com um hífen, e combinar definição com remoção como em (?im-sx), que define PCRE_CASELESS e PCRE_MULTILINE enquanto remove PCRE_DOTALL e PCRE_EXTENDED. Se uma letra aparecer tanto antes quanto depois do hífen, a opção é removida.

Quando uma mudança de opção ocorre no nível mais alto (isto é, não ocorre dentro de parênteses de sub-expressão), a mudança se aplica ao restante da expressão que se segue. Assim, `/ab(?i)c/` corresponde apenas a "abc" e "abC".

Se uma mudança de opção ocorre dentro de uma sub-expressão, o efeito é diferente. Isto é uma mudança de comportamento no Perl 5.005. Uma mudança de opção dentro de uma sub-expressão afeta somente aquela parte da sub-expressão que a segue, assim `(a(?i)b)c` corresponde a "abc" e "aBc" e nenhuma outra string (assumindo que PCRE_CASELESS não seja usada). Por este meio, opções podem ser usadas para ter configurações diferentes em partes diferentes da sub-expressão. Qualquer mudança feita em uma alternativa é carregada nos ramos subsequentes dentro da mesma sub-expressão. Por exemplo, `(a(?i)b|c)` corresponde a "ab", "aB", "c" e "C", embora quando corresponde a "C", o primeiro ramo é abandonado antes da definição da opção. Isto porque os efeitos de configurações de opções acontecem em tempo de compilação. Haveria comportamento muito estranho caso não fosse desta maneira.

As opções específicas do PCRE PCRE_UNGREEDY e PCRE_EXTRA podem ser alteradas da mesma forma que as compatíveis com o Perl, usando os caracteres U e X respectivamente. A opção (?X) é especial pelo fato de que deve sempre ocorrer na expressão antes de qualquer recurso adicional que ela ativa, mesmo quando está no nível mais alto. É melhor quando colocada no início.

Sub-expressões

Sub-expressões são delimitadas por parênteses e podem ser aninhadas. Marcar parte de uma expressão como uma sub-expressão tem dois objetivos:

1. Localizar um conjunto de alternativas. Por exemplo, a expressão `cat(aract|erpillar|)` corresponde a uma das três palavras: "cat", "cataract" ou "caterpillar". Sem os parênteses, corresponderia a "cataract", "erpillar" ou a uma string vazia.

2. Definir a sub-expressão como um grupo de captura (como definido acima). Quando toda a expressão tem correspondência, a parte da string de entrada que correspondeu à sub-expressão é passada de volta à função chamadora através do argumento *ovector* da função **pcre_exec()**. Parênteses de abertura são contados da esquerda para a direita (iniciando de 1) para obter os números dos grupos de captura.

Por exemplo, se a string "the red king" é correspondida pela expressão `the ((red|white) (king|queen))` os grupos de captura são "red king", "red" e "king", e são numerados com 1, 2 e 3.

O fato dos parênteses terem duas funções nem sempre ajuda. Frequentemente é necessária uma sub-expressão de agrupamento sem a necessidade de captura. Se um parêntese de abertura for seguido por "?:", a sub-expressão não faz nenhuma captura, e não é contado ao computar o número de sub-expressões de captura subsequentes. Por exemplo, se a string "the white queen" é correspondida pela expressão `the ((?:red|white) (king|queen))` as substrings capturadas são "white queen" e "queen", e são numeradas com 1 e 2. O número máximo de subtrings capturadas é 65535. Porém, pode não ser possível compilar expressões deste tamanho, dependendo das opções de configuração da biblioteca libpcre.

Como um atalho conveniente, se qualquer configuração de opção for necessária no início de uma sub-expressão sem captura, as letras de opções podem aparecer entre o "?" e o ":". Assim, as duas expressões

```
(?i:saturday|sunday)
(?:(?i)saturday|sunday)
```

correspondem exatamente ao mesmo conjunto de strings. Como os ramos alternativos são testados da esquerda para a direita, e as opções não são redefinidas até que se alcance o final da sub-expressão, uma configuração de opção em um ramo não afeta ramos subsequentes, portanto as expressões acima correspondem a "SUNDAY" e também a "Saturday".

É possível nomear uma sub-expressão usando a sintaxe `(?P<nome>expressão)`. Esta sub-expressão será então indexada no array de correspondências pela sua posição numérica normal e também pelo nome. Há duas sintaxes alternativas: `(?<nome>expressão)` e `(?'nome'expressão)`.

A partir do PHP 8.4.0, que inclui o PCRE2 10.44, o comprimento máximo de um nome de sub-expressão é 128 caracteres; anteriormente era 32 caracteres.

Algumas vezes é necessário ter correspondências múltiplas, mas não ter sub-grupos alternados em uma expressão regular. Normalmente, cada um destes receberia seu próprio número de referência embora apenas um seria possivelmente correspondido. Para contornar isto, a sintaxe `(?|` permite ter números duplicados. Considere o caso abaixo, onde a expressão regular corresponde à string `Sunday`:

```
(?:(Sat)ur|(Sun))day
```

Aqui, `Sun` é armazenado na referência 2, enquanto que a referência 1 está vazia. Corresponder `Saturday` gera a referência 1 para `Sat` enquanto que a referência 2 não existirá. Modificar a expressão usando `(?|` corrige este problema:

```
(?|(Sat)ur|(Sun))day
```

Usando esta expressão, tanto `Sun` quanto `Sat` seriam armazenadas na referência 1.

Repetição

Repetição é especificada por quantificadores, que podem acompanhar qualquer dos itens abaixo:

- um único caractere, possivelmente escapado
- o metacaractere "."
- uma classe de caracteres
- uma referência retroativa (veja a próxima seção)
- uma sub-expressão entre parênteses (a menos que seja uma afirmação - veja abaixo)

O quantificador de repetição geral especifica um número mínimo e um máximo de correspondências permitidas, informando os dois números dentro de chaves e separados por uma vírgula. Os números precisam ser menores que 65536, e o primeiro precisa ser menor ou igual ao segundo. Por exemplo: `z{2,4}` corresponde a "zz", "zzz" ou "zzzz". Um fechamento de chave em si não é um caractere especial, Se o segundo número for omitido, mas a vírgula estiver presente, não há limite superior; se o segundo número e a vírgula são ambos omitidos, o quantificador especifica um número exato de correspondências requeridas. Assim, `[aeiou]{3,}` corresponde a pelo menos 3 vogais sem acento sucessivas, mas também podem corresponder a muito mais, enquanto que `\d{8}` corresponde a exatamente 8 dígitos.

Antes do PHP 8.4.0, uma chave de abertura que aparece em uma posição onde um quantificador não é permitido, ou que não corresponda à sintaxe de um quantificador, é considerada como um caractere literal, Por exemplo, `{,6}` não é um quantificador, mas uma string literal de quatro caracteres. A partir do 8.4.0, a extensão PCRE é incluída com o PCRE2 versão 10.44, que permite expressões como `\d{,8}` e elas são interpretadas como `\d{0,8}`. Além disso, a partir do PHP 8.4.0, caracteres de espaço em torno de quantificadores como em `\d{0 , 8}` e `\d{ 0 , 8 }` são permitidos.

O quantificador {0} é permitido, fazendo com que a expressão se comporte como se o item anterior e o quantificador não estivessem presentes.

Para conveniência (e compatibilidade histórica), os três quantificadores mais comuns têm abreviaturas de apenas um caractere:

Quantificadores de um caractere	
`*`	equivalente a `{0,}`
`+`	equivalente a `{1,}`
`?`	equivalente a `{0,1}`

É possível construir laços de repetição infinitos acompanhando uma sub-expressão que pode corresponder a nenhum caractere com um quantificador que não tem limite superior, por exemplo: `(a?)*`

Versões mais antigas do Perl e do PCRE costumavam emitir um erro em tempo de compilação para tais expressões. No entanto, por haver casos onde isto pode ser útil, essas expressões agora são aceitas, mas se alguma repetição na sub-expressão de fato corresponder a nenhum caractere, o laço é quebrado à força.

Por padrão, os quantificadores são "gananciosos", isto é, eles irão corresponder ao máximo de caracteres possível (até o limite do número máximo permitido), sem fazer com que o resto da expressão falhe. O exemplo clássico de onde isso dá problema é a tentativa de correspondência de comentários nos programas em linguagem C. Eles aparecem entre as sequências /* e */ e dentro da sequência, caracteres individuais * e / podem aparecer. Uma tentativa de corresponder a comentários C aplicando a expressão `/\*.*\*/` à string `/* primeiro comentário */ não comentário /* segundo comentário */` falha, porque ela corresponde à string completa devido à ganância do item ".*".

Porém, se um quantificador for seguido por um ponto de interrogação, ele se torna preguiçoso, e agora irá corresponder ao mínimo número de vezes possível, e portanto a expressão `/\*.*?\*/` faz o certo com os comentários C. O significado dos vários quantificadores não é alterado, apenas o número preferido de correspondências. Não confunda este uso do ponto de interrogação com o seu uso como um quantificador em si. Como ele tem dois usos, pode algumas vezes aparecer duplicado, como em `\d??\d` que corresponde a um dígito por preferência, mas pode corresponder a dois se esta for a única maneira que o resto da expressão terá correspondência.

Se a opção PCRE_UNGREEDY estiver definida (uma opção que não está disponível no Perl), os quantificadores não serão gananciosos por padrão, mas os individuais podem se tornar gananciosos inserindo-se um ponto de interrogação na sequência deles. Em outras palavras, o ponto de interrogação inverte o comportamento padrão.

Quantificadores seguidos por + são "possessivos". Eles consomem o máximo de caracteres possível e não retornam para corresponder ao restante da expressão. Assim, `.*abc` corresponde a "aabc" mas `.*+abc` não, porque `.*+` consome a string inteira. Quantificadores possessivos podem ser usados para acelerar o processamento.

Quando uma sub-expressão entre parênteses é quantificada com uma contagem mínima de repetições maior que um ou com um máximo limitado, mais espaço de armazenamento é necessário para a expressão compilada, em proporção ao tamanho do mínimo ou do máximo.

Se uma expressão começa com .* ou .{0,} e a opção PCRE_DOTALL (equivalente ao /s do Perl) está definida, assim permitindo que o "." corresponda a caracteres de nova linha, a expressão estará implicitamente ancorada, porque qualquer coisa que estiver na sequência será testada em cada posição de caractere na string de entrada. Portanto, não faz sentido tentar novamente a correspondência geral em qualquer posição após a primeira. O PCRE trata tal expressão como se ela estivesse precedida por \A. Em casos onde é sabido que a string de entrada não contém caracteres de nova linha, vale a pena configurar PCRE_DOTALL quando a expressão inicia com .* para obter esta otimização, ou alternativamente usar ^ para indicar a ancoragem explicitamente.

Quando uma sub-expressão de captura é repetida, o valor capturado é a substring que correspondeu à iteração final. Por exemplo, depois que `(tweedle[dume]{3}\s*)+` tenha correspondido a "tweedledum tweedledee" o valor da substring capturada é "tweedledee". Porém se houver sub-expressões de captura aninhadas, os valores de captura correspondentes podem ter sido definidos em iterações prévias. Por exemplo, depois que `/(a|(b))+/` corresponder a "aba" o valor da segunda substring capturada é "b".

Referências retroativas

Fora de uma classe de caracteres, uma barra invertida seguida por um dígito maior que 0 (e possivelmente dígitos adicionais) é uma referência retroativa a um grupo de captura realizada anteriormente (isto é, à sua esquerda) na expressão, desde que tenha havido previamente essa quantidade de captura entre parênteses.

Porém, se um número decimal seguindo a barra invertida for menor que 10, ele sempre será interpretado como uma referência retroativa e causará um erro somente se não tiver havido essa quantidade de grupos de captura em toda a expressão. Em outras palavras, os parênteses que são referenciados não precisam estar à esquerda da referência para números menores que 10. Uma "referência retroativa à frente" pode fazer sentido quando uma repetição está envolvida e a sub-expressão à direita tenha participado de uma iteração anterior. Consulte a seção sobre sequências de escape para detalhes adicionais da manipulação de dígitos que seguem uma barra invertida.

Uma referência retroativa corresponde a qualquer coisa que verdadeiramente correspondeu à sub-expressão de captura na string de entrada atual, e não a qualquer coisa que corresponda à sub-expressão em si. Portanto, a expressão `(sens|respons)e and \1ibility` corresponde a "sense and sensibility" e "response and responsibility", mas não a "sense and responsibility". Se a correspondência for sensível a maiúsculas/minúsculas no momento da referência retroativa, a forma das letras será relevante. Por exemplo, `((?i)rah)\s+\1` corresponde a "rah rah" e "RAH RAH", mas não a "RAH rah", embora a sub-expressão de captura original seja correspondida de forma insensível a maiúsculas/minúsculas.

Pode haver mais que uma referência retroativa à mesma sub-expressão. Se uma sub-expressão não tiver sido usada em uma correspondência em particular, qualquer referência retroativa a ela sempre falhará. Por exemplo, a expressão `(a|(bc))\2` sempre falhará se ele iniciar correspondendo a "a" ao invés de "bc". Como podem haver até 99 referências retroativas, todos os dígitos seguindo a barra invertida são consideradas como parte de um potencial número de identificador de referência retroativa. Se a expressão continua com um caractere de dígito, algum delimitador precisa ser usado para terminar a referência retroativa. Se a opção PCRE_EXTENDED estiver definida, o delimitador pode ser um espaço em branco. Caso contrário um comentário vazio pode ser usado.

Uma referência retroativa que ocorre dentro dos parênteses aos quais ela se refere falha quando a sub-expressão é usada pela primeira vez, portanto, por exemplo, (a\1) nunca será correspondida. No entanto, tais referências podem ser úteis dentro de sub-expressões repetidas. Por exemplo, a expressão `(a|b\1)+` corresponde a qualquer número de "a"s e também a "aba", "ababba" etc. A cada iteração da

sub-expressão, a referência retroativa corresponde à string de caracteres relativa à iteração anterior. Para que isso funcione, a expressão precisa ser tal que a primeira iteração não precise corresponder à referência retroativa. Isto pode ser feito usando alternância, como no exemplo acima, ou por um quantificador com um mínimo de zero.

A sequência de escape `\g` pode ser usada para referências absolutas e relativas de sub-expressões. Esta sequência de escape precisa ser seguida por um número sem sinal ou negativo, opcionalmente envolvido por chaves. As sequências `\1`, `\g1` e `\g{1}` são sinônimos entre si. O uso desta expressão com um número sem sinal pode ajudar a remover a ambiguidade inerente ao usar dígitos após uma barra invertida. A sequência ajuda a distinguir referências de caracteres octais e também torna mais fácil ter uma referência seguida por um número literal, ex.: `\g{2}1`.

O uso da sequência `\g` com um número negativo significa uma referência relativa. Por exemplo, `(foo)(bar)\g{-1}` corresponderia à sequência "foobarbar" e `(foo)(bar)\g{-2}` corresponderia a "foobarfoo". Isto pode ser útil em expressões longas como uma alternativa para manter rastreamento do número de sub-expressões para referenciar uma sub-expressão anterior específica.

Referências às sub-expressões nomeadas podem ser obtidas com `(?P=nome)`, `\k<nome>`, `\k'nome'`, `\k{nome}`, `\g{nome}`, `\g<nome>` ou `\g'nome'`.

Afirmações

Uma afirmação é um teste nos caracteres posteriores ou anteriores ao ponto atual de correspondência que efetivamente não consome nenhum caractere. As afirmações simples codificadas como \b, \B, \A, \Z, \z, ^ e $ estão descritas na seção sobre sequências de escape. Afirmações mais complexas são codificadas como sub-expressões. Existem dois tipos: aquelas que *olham para frente* da posição atual na string de entrada e aquelas que *olham para trás* dela.

Uma sub-expressão de afirmação é correspondida da maneira usual, exceto que ela não faz com que a posição atual de correspondência seja alterada. Afirmações que olham *para frente* iniciam com (?= para afirmações positivas e com (?! para afirmações negativas. Por exemplo, `\w+(?=;)` corresponde a uma palavra seguida por ponto-e-vírgula, mas não inclui o ponto-e-vírgula na correspondência, e `foo(?!bar)` corresponde a qualquer ocorrência de "foo" que não seja seguida por "bar". Observe que a expressão aparentemente similar `(?!foo)bar` não encontra uma ocorrência de "bar" que seja precedida por algo além de "foo"; ela encontra qualquer ocorrência de "bar" de todo modo, porque a afirmação (?!foo) é sempre `true` quando os três próximos caracteres são "bar". Uma afirmação que olha para trás é necessária para conseguir este efeito.

Afirmações que olham *para trás* iniciam com (?<= para afirmações positivas e (?<! para afirmações negativas. Por exemplo, `(?<!foo)bar` encontra uma ocorrência de "bar" que não seja precedida por "foo". Uma afirmação que olha para trás pode corresponder a strings de tamanhos variáveis, até um limite definido pela implementação. Alternativas de tamanhos diferentes como `(?<=dogs?|cats?)` e ramos de nível superior que podem corresponder a mais que um comprimento como `(?<=ab(c|de))` são permitidos. A implementação de afirmações que olham para trás é, para cada alternativa, mover temporariamente a posição atual para trás pela largura fixa e então tentar a correspodência. Se houver caracteres insuficientes antes da posição atual, a correspondência falhará. Afirmações que olham para trás em conjunção com sub-expressões de ocorrência única podem ser particularmente úteis para ter correspondência nos finais de strings; um exemplo é mostrado no final da seção sobre sub-expressões de ocorrência única.

Antes do PHP 8.4.0, que inclui o PCRE2 10.44, o conteúdo de uma afirmação que olha para trás era restrito de forma que todas as strings que ela correspondia tinham que ter um comprimento fixo. Se houvessem várias alternativas, nem todas elas precisavam ter o mesmo comprimento fixo, então `(?<=bullock|donkey)` era permitido, mas `(?<!dogs?|cats?)` causava um erro no momento da compilação. Ramos que correspondiam a strings de diferentes comprimentos eram permitidas somente no nível superior de uma afirmação que olha para trás. Isto era uma extensão comparado com o Perl 5.005, que requeria que todos os ramos correspondessem o mesmo comprimento de string. Um afirmação como `(?<=ab(c|de))` não era permitida porque seu ramo superior único poderia corresponder a dois comprimentos diferentes mas era aceita se fosse reescrita para usar dois ramos de nível superior: `(?<=abc|abde)`

Várias afirmações (de qualquer tipo) podem ocorrer sucessivamente. Por exemplo, `(?<=\d{3})(?<!999)foo` corresponde a "foo" precedida por três dígitos que não são "999". Note que cada afirmação é aplicada independentemente no mesmo ponto da string de entrada. Primeiro é verificado se os três caracteres anteriores são todos dígitos e depois é verificado se os mesmos três caracteres não são "999". Esta expressão não corresponde a "foo" precedida por seis caracteres, os primeiros três dos quais são dígitos e os últimos três deles não são "999". Por exemplo, ela não corresponde a "123abcfoo". Uma expressão para fazer isso é `(?<=\d{3}...)(?<!999)foo`

Agora a primeira afirmação olha para os seis caracteres precedentes, verificando se os primeiros três são dígitos e depois a segunda afirmação verifica se os três caracteres precedentes não são "999".

Afirmações podem ser aninhadas em qualquer combinação. Por exemplo, `(?<=(?<!foo)bar)baz` corresponde a uma ocorrência de "baz" que é precedida por "bar" que por sua vez não é precedida por "foo", enquanto que `(?<=\d{3}...(?<!999))foo` é outra expressão que corresponde a "foo" precedida por três dígitos e quaisquer três caracteres que não sejam "999".

Sub-expressões de afirmação não são sub-expressões de captura, e podem não ser repetidas, porque não faz sentido afirmar a mesmo coisa várias vezes. Se qualquer tipo de afirmação coniver sub-expressões de captura dentro dela, estas são contadas para os propósitos de numeração dos grupos de captura em toda a expressão. Porém, a captura de substring acontece somente para afirmações positivas, porque não faz sentido para afirmações negativas.

Afirmações contam até o máximo de 200 sub-expressões entre parênteses.

Sub-expressões de ocorrência única

Tanto com a maximização quanto com a minimização da repetição, a falha no que se segue normalmente faz com que o item repetido seja reavaliado para ver se um número diferente de repetições permite que o resto da expressão corresponda. Às vezes é útil evitar isso, seja para mudar a natureza da correspondência ou para fazer com que ela falhe antes do que aconteceria de outra forma, quando o autor da expressão sabe que não há sentido em continuar.

Considere, por exemplo, a expressão \d+foo quando aplicada à linha `123456bar`

Depois de corresponder a todos os 6 dígitos e depois falhar em corresponder a "foo", a ação normal do algoritmo de correspondência é tentar novamente com somente 5 dígitos correspondendo ao item \d+, e depois com 4, e assim por diante, até falhar na última vez. Sub-expressões de ocorrência única fornecem o meio de especificar que uma vez que a parte da expressão tenha sido correspondida, ela não deve ser reavaliada desta forma, portanto o algoritmo desistiria imediatamente ao falhar na correspondência a "foo" na primeira vez. A notação é um outro tipo de parêntese especial, iniciando com (?> como neste exemplo: `(?>\d+)bar`

Este tipo de parênteses "trava" a parte da expressão que eles contém uma vez que tenha sido correspondida, e uma falha mais para a frente na expressão é impedida de rastreá-la retroativamente. O restreamento retroativo depois deles para itens anteriores, entretanto, funciona normalmente.

Uma descrição alternativa é que uma sub-expressão deste tipo corresponde à string de caracteres que uma expressão idêntica avulsa corresponderia, se ancorada no ponto atual da string de entrada.

Sub-expressões de ocorrência única não são sub-expressões de captura. Casos simples como o exemplo acima podem sem pensados como uma repetição de maximização que precisa consumir tudo o que ela puder. Portanto, enquanto ambos \d+ e \d+? são preparados para ajustar o número de dígitos que eles correspondem para fazer com o que o resto da expressão tenha correspondência, (?>\d+) pode somente corresponder a uma sequência inteira de dígitos.

Obviamente, esta contrução pode conter sub-expressões arbitrariamente complexas, e ela pode ser aninhada.

Sub-expressões de ocorrência única podem ser usadas em conjunção com afirmações que olham para trás para especificar correspondência eficiente no final da string. Considere a expressão simples `abcd$` quando aplicada a uma string longa que não corresponde. Como a correspondência procede da esquerda para a direita, o PCRE irá procurar cada "a" na string e depois ver se o que vem a seguir corresponde ao resto da expressão. Se a expressão for especificada como `^.*abcd$`, o .* inicial corresponderá à string inteira primeito, mas quando isso falhar (porque não existe um "a" a seguir), ela faz um rastreamento retroativo para corresponder a todos os caracteres exceto ao último, e depois todos exceto os últimos dois, e assim por diante. Uma vez mais a pesuisa por "a" cobre a string inteira, da direita para a esquerda, portanto não melhorou em nada. Porém, se a expressão for escrita como `^(?>.*)(?<=abcd)`, não poderá haver rastreamento retroativo para o item ".*". ele pode corresponder somente à string inteira. A afirmação subsequente faz um teste simples nos últimos quatro caracteres. Se ela falhar, a correspondência falha imediatamente. Para strings longas, este métodos faz uma diferença significativa no tempo de processamento.

Quando uma expressão contém uma repetição ilimitada dentro de uma sub-expressão que pode ser repetida um número ilimitado de vezes, o uso de uma sub-expressão de ocorrência única é a única maneira de evitar que algumas correspondências com falha demorem muito tempo. A expressão `(\D+|<\d+>)*[!?]` corresponde a um número ilimitado de sub-expressões que consistem de não-dígitos, ou de dígitos envolvidos por <>, seguidos por ! ou ?. Quando ela é correspondida, é executada rapidamente. Porém, se for aplicada a `aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa`, ela leva um longo tempo antes de reportar falha. Isso ocorre porque a string pode ser dividida entre as duas repetições de várias maneiras e todas precisam ser tentadas. (O exemplo usou [!?] em vez de um único caractere no final, porque tanto o PCRE quanto o Perl têm uma otimização que permite falha rápida quando um único caractere é usado. Eles lembram o último caractere único necessário para uma correspondência, e falhar antecipadamente se não estiver presente na string.) Se a expressão for alterada para `((?>\D+)|<\d+>)*[!?]`, sequências de não-dígitos não podem ser quebradas, e a falha acontece rapidamente.

Sub-expressões condicionais

É possível fazer com que o processo de correspondência obedeça condicionalmente a uma sub-expressão ou escolha entre duas sub-expressões alternativas, dependendo do resultado de uma afirmação, ou se uma sub-expressão de captura anterior correspondeu ou não. As duas formas possíveis de sub-expressão condicional são

```
(?(condição)expressão-sim)
(?(condição)expressão-sim|expressão-não)
```

Se a condição for satisfeita, a expressão-sim é usada; caso contrário a expressão-não (se presente) é usada. Se houver mais que duas alternativas na sub-expressão, ocorrerá um erro no momento da compilação.

Existem dois tipos de condição. Se o texto entre parênteses consistir em uma sequência de dígitos, então a condição será satisfeita se a sub-expressão de captura desse número tiver correspondido anteriormente. Considere a seguinte expressão, que contém espaços em branco não significativos para torná-la mais legível (assumindo que a opção PCRE_EXTENDED está definida) e para dividi-la em três partes para facilidade de discussão:

```
( \( )? [^()]+ (?(1) \) )
```

A primeira parte corresponde a um parêntese de abertura opcional e, se esse caractere estiver presente, define-o como a primeira substring capturada. A segunda parte corresponde a um ou mais caracteres que não são parênteses. A terceira parte é uma sub-expressão condicional que testa se o primeiro conjunto de parênteses corresponde ou não. Se sim, ou seja, se a string começou com um parêntese de abertura, a condição é `true`, e então a expressão-sim é executada e um parêntese de fechamento é necessário. Caso contrário, como a expressão-não não está presente, A sub-expressão não corresponde a nada. Em outras palavras, essa expressão corresponde a uma sequência de não-parênteses, opcionalmente entre parênteses.

Se a condição for a string `(R)`, ela será satisfeita se uma chamada recursiva à expressão ou sub-expressão tiver sido feita. No "nível superior", a condição é falsa.

Se a condição não for uma sequência de dígitos ou (R), deverá ser uma afirmação. Esta pode ser uma afirmação positiva ou negativa que olha para frente ou que olha para trás. Considere esta expressão, novamente contendo espaços em branco não significativos e com as duas alternativas na segunda linha:

```
(?(?=[^a-z]*[a-z])
\d{2}-[a-z]{3}-\d{2} | \d{2}-\d{2}-\d{2} )
```

A condição é uma afirmação que olha para frente positiva que corresponde a uma sequência opcional de não-letras seguidas por uma letra. Em outras palavras, testa a presença de pelo menos uma letra na string. Se for encontrada uma letra, a string é comparada com a primeira alternativa; caso contrário, será comparada com o segunda. Esta expressão corresponde a strings em uma das duas formas dd-aaa-dd ou dd-dd-dd, onde aaa são letras e dd são dígitos.

Comentários

A sequência (?# marca o início de um comentário que continua até o próximo parêntese de fechamento. Parênteses aninhados não são permitidos. Os caracteres que compõem um comentário não desempenham nenhum papel na correspondência de expressões.

Se a opção PCRE_EXTENDED estiver definida, um caractere # sem escape fora de uma classe de caracteres introduz um comentário que continua até o próximo caractere de nova linha na expressão.

Exemplo #1 Uso de comentários em expressões PCRE

```php
<?php

$subject = 'test';

/* (?# pode ser usado para adicionar comentários sem habilitar PCRE_EXTENDED */
$match = preg_match('/te(?# isto é um comentário)st/', $subject);
var_dump($match);

/* Espaço em branco e # são tratados como parte da expressão a menos que PCRE_EXTENDED esteja habilitada */
$match = preg_match('/te   #~~~~
st/', $subject);
var_dump($match);

/* Quando PCRE_EXTENDED está habilitada, todos os espaços em branco e qualquer coisa que
   esteja na sequência de um # sem escape na mesma linha são ignorados */
$match = preg_match('/te    #~~~~
st/x', $subject);
var_dump($match);
```

O exemplo acima produzirá:

```
int(1)
int(0)
int(1)
```

Expressões recursivas

Considere o problema de corresponder a uma string entre parênteses, permitindo parênteses aninhados ilimitados. Sem o uso de recursão, o melhor que pode ser feito é usar uma expressão que corresponda a uma profundidade fixa de aninhamento. Não é possível lidar com uma profundidade de aninhamento arbitrária. O Perl 5.6 forneceu um recurso experimental que permite a recursividade de expressões regulares (entre outras coisas). O item especial (?R) é fornecido para o caso específico de recursão. Esta expressão PCRE resolve o problema dos parênteses (suponha que a opção PCRE_EXTENDED esteja definida para que o espaço em branco seja ignorado): `\( ( (?> [^()]+) | (?R) )* \)`

Primeiro, ela corresponde a um parêntese de abertura. Em seguida, ela corresponde a qualquer número de substrings que podem ser uma sequência de não-parênteses ou uma correspondência recursiva da própria expressão (ou seja, uma substring colocada de forma correta entre parênteses). Finalmente há um parêntese de fechamento.

Esta expressão de exemplo em particular contém repetições ilimitadas aninhadas e, portanto, o uso de uma sub-expressão de ocorrência única para correspondência a strings de não-parênteses é importante ao aplicar a expressão em strings que não correspondem. Por exemplo, quando é aplicada a `(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa()` ela gera uma "não correspondência" rapidamente. Entretanto, se uma expressão de ocorrência única nao for usada, a correspondência é executada por um longo tempo porque existem muitas formas diferentes das repetições + e * dividirem a string, e todas têm que ser testadas antes que a falha possa ser reportada.

Os valores definidos para quaisquer sub-expressões de captura são aqueles do nível mais externo da recursão no qual o valor da sub-expressão é definido. Se a expressão acima for comparada com `(ab(cd)ef)`, o valor para os parênteses de captura será "ef", que é o último valor considerado no nível mais alto. Se parênteses extras forem acrescentados, resultando em `\( ( ( (?>[^()]+) | (?R) )* ) \)`, a string que eles irão capturar será "ab(cd)ef", o conteúdo dos parênteses do nível mais alto. Se houver mais de 15 parênteses de captura em uma expressão, o PCRE precisa obter memória extra para armazenar dados durante uma recursão, o que ele faz usando pcre_malloc, liberando-a posteriormente via pcre_free. Se nenhuma memória puder ser obtida, ele salva os dados apenas dos primeiros 15 parênteses de captura, pois não há como gerar um erro de falta de memória dentro de uma recursão.

`(?1)`, `(?2)`, e assim por diante, também podem ser usados para sub-expressões recursivas. também é possível usar sub-expressões nomeadas: `(?P>nome)` ou `(?&nome)`.

Se a sintaxe para uma referência de sub-expressão recursiva (seja por número ou por nome) for usada fora dos parênteses aos quais se refere, ela funcionará como uma sub-rotina em uma linguagem de programação. Um exemplo anterior apontou que a expressão `(sens|respons)e and \1ibility` corresponde a "sense and sensibility" e "response and responsibility", mas não a "sense and responsibility". Se, em vez disso, a expressão `(sens|respons)e and (?1)ibility` for usada, ela corresponderá a "sense and responsibility", bem como às outras duas strings. Tais referências devem, contudo, seguir a sub-expressão a que se referem.

O comprimento máximo de uma string de entrada é o maior número positivo que uma variável inteira pode conter. No entanto, o PCRE usa recursão para lidar com sub-expressões e repetições indefinidas. Isso significa que o espaço de pilha disponível pode limitar o tamanho de uma string de entrada que pode ser processada por determinadas expressões.

Desempenho

Certos itens que podem aparecer nas expressões são mais eficientes que outros. É mais eficiente usar uma classe de caracteres como [aeiou] do que um conjunto de alternativas como (a|e|i|o|u). Em geral, a construção mais simples que proporciona o comportamento requerido é normalmente a mais eficiente. O livro de Jeffrey Friedl contém muita discussão sobre como otimizar expressões regulares para um desempenho eficiente.

Quando uma expressão começa com .* e a opção PCRE_DOTALL está definida, a expressão é implicitamente ancorada pelo PCRE, pois ela pode corresponder apenas ao início de um string de entrada. Porém, se PCRE_DOTALL não estiver definida, o PCRE não poderá fazer esta otimização, pois o metacaractere . não corresponderá a uma nova linha e, se a string de entrada contiver novas linhas, a expressão poderá corresponder a partir do caractere imediatamente após um deles, em vez de desde o início. Por exemplo, a expressão `(.*) second` corresponde à string "first\nand second" (onde \n representa um caractere de nova linha) com a primeira substring capturada igual a "and". Para fazer isso, o PCRE precisa tentar a correspondência novamente iniciando após cada caractere de nova linha na string.

Se uma expressão como essa estiver sendo usada com uma string de entrada que não contêm novas linhas, o melhor desempenho será obtido definindo PCRE_DOTALL ou iniciando a expressão com ^.* para indicar ancoragem explícita. Isso evita que o PCRE tenha que varrer a string em busca de uma nova linha para reiniciar.

Cuidado com expressões que contêm repetições indefinidas aninhadas. Elas podem levar muito tempo para serem executadas quando aplicados a uma string que não corresponde. Considere o fragmento de expressão `(a+)*`

Isso pode corresponder a "aaaa" de 33 maneiras diferentes, e esse número aumenta muito rapidamente à medida que a string fica mais longa. (A repetição * pode corresponder a 0, 1, 2, 3 ou 4 vezes e, para cada um desses casos diferentes de 0, as repetições + podem corresponder a diferentes números de vezes.) Quando o restante da expressão é tal que toda a correspondência irá falhar, o PCRE tem que, em princípio, tentar todas as variações possíveis, e isso pode levar muito tempo.

Uma otimização captura alguns dos casos mais simples como `(a+)*b`, onde um carectere literal aparece na sequência. Antes de embarcar no procedimento padrão de correspondência, o PCRE verifica se existe um "b" mais à frente na string de entrada, e se não houver, ele falha a correspondência imediatamente. Porém, quando não há um caractere literal na sequência, esta otimização não pode ser usada. Pode-se ver a deferença comparando o comportamento de `(a+)*\d` com a expressão mais acima. A anterior falha quase instantaneamente quando aplicada a uma linha inteira de caracteres "a", enquanto que a última leva um tempo apreciável com strings mais longas que aproximadamente 20 caracteres.

Possíveis modificadores em expressões regulares

Os possíveis modificadores PCRE atuais estão listados abaixo. Os nomes entre parênteses referem-se a nomes PCRE internos para esses modificadores. Espaços e novas linhas são ignorados nos modificadores, outros caracteres causam erro.

> *i* (PCRE_CASELESS)
>
> Se este modificador estiver definido, as letras na expressão corresponderão às letras maiúsculas e minúsculas.
>
> *m* (PCRE_MULTILINE)
>
> Por padrão, o PCRE trata a string de entrada como uma única "linha" de caracteres (mesmo que na verdade contenha vários caracteres de nova linha). O metacaractere "início de linha" (^) corresponde apenas ao início da string, enquanto que o metacaractere "fim de linha" ($) corresponde apenas ao final da string ou antes de um caractere final de nova linha (a menos que o modificador *D* esteja definido). O comportamento é o mesmo no Perl.
>
> Quando este modificador é definido, as construções "início de linha" e "fim de linha" correspondem imediatamente após ou imediatamente antes de qualquer caractere de nova linha na string de entrada, respectivamente, bem como no início e no final. Isso é equivalente ao modificador /m do Perl. Se não houver caracteres "\n" em uma string de entrada ou nenhuma ocorrência de ^ ou $ em uma expressão, a configuração desse modificador não terá efeito.
>
> *s* (PCRE_DOTALL)
>
> Se este modificador estiver definido, um metacaractere de ponto na expressão corresponderá a todos os caracteres, incluindo os de nova linha. Sem ele, os de nova linha são excluídas. Este modificador é equivalente ao modificador /s do Perl. Uma classe negativa como [^a] sempre corresponde a um caractere de nova linha, independente da configuração deste modificador.
>
> *x* (PCRE_EXTENDED)
>
> Se este modificador estiver definido, os caracteres de espaço em branco de dados na expressão serão totalmente ignorados, exceto quando escapados ou dentro de uma classe de caracteres, e os caracteres entre um # sem escape fora de uma classe de caracteres e o próximo caractere de nova linha, inclusive, também serão ignorados. Isto é equivalente ao modificador /x do Perl e torna possível incluir comentários dentro de padrões complexos. Observe, entretanto, que isso se aplica apenas a caracteres de dados. Os caracteres de espaço em branco nunca podem aparecer em sequências de caracteres especiais em uma expressão, por exemplo, dentro da sequência (?(que introduz uma sub-expressão condicional.
>
> *A* (PCRE_ANCHORED)
>
> Se este modificador estiver definido, a expressão é forçada a ser "ancorada", ou seja, é restrita a corresponder apenas ao início da string que está sendo pesquisada (a "string de entrada"). Este efeito também pode ser alcançado por construções apropriadas na própria expressão, que é a única maneira de fazer isso em Perl.
>
> *D* (PCRE_DOLLAR_ENDONLY)
>
> Se esse modificador estiver definido, um metacaractere de cifrão na expressão corresponderá apenas ao final da string de entrada. Sem esse modificador, um cifrão também corresponde imediatamente antes do caractere final se for um de nova linha (mas não antes de qualquer outro de nova linha). Este modificador é ignorado se o modificador *m* estiver definido. Não há equivalente a este modificador no Perl.
>
> *S*
>
> Quando uma expressão vai ser usada várias vezes, vale a pena gastar mais tempo analisando-a para agilizar o tempo de correspondência. Se este modificador estiver definido, esta análise extra será executada. Atualmente, estudar uma expressão é

útil apenas para expressões não ancoradas que não possuem um caractere inicial único fixo. A partir do PHP 7.3.0, este modificador não tem efeito.

U (PCRE_UNGREEDY)

Este modificador inverte a "ganância" dos quantificadores para que eles não sejam gananciosos por padrão, mas se tornem gananciosos se seguidos por `?`. Não é compatível com o Perl. Também pode ser definido por uma configuração de modificador dentro da expressão (`?U`) ou por um ponto de interrogação atrás de um quantificador (por exemplo, `.*?`).

> **Nota** :
>
> Geralmente não é possível corresponder mais do que pcre.backtrack_limit caracteres no modo não ganancioso.

X (PCRE_EXTRA)

Este modificador ativa funcionalidades adicionais do PCRE que são incompatíveis com o Perl. Qualquer barra invertida em uma expressão seguida por uma letra sem significado especial causa um erro, reservando assim essas combinações para expansão futura. Por padrão, como no Perl, uma barra invertida seguida por uma letra sem significado especial é tratada como um literal. Atualmente não há outras características controladas por este modificador.

J (PCRE_INFO_JCHANGED)

A configuração de opção interna (?J) altera a opção local PCRE_DUPNAMES. Permite nomes duplicados para sub-expressões. A partir do PHP 7.2.0, `J` também é suportado como modificador.

u (PCRE_UTF8)

Este modificador ativa funcionalidades adicionais do PCRE que são incompatíveis com o Perl. As strings de expressão e de entrada são tratadas como UTF-8. Uma string de entrada inválida fará com que a função preg_* não corresponda a nada; uma expressão inválida irá causar um erro de nível E_WARNING. Sequências UTF-8 de cinco e seis octetos são consideradas inválidas.

n (PCRE_NO_AUTO_CAPTURE)

Este modificador torna grupos (`xyz`) simples não capturáveis. Somente grupos nomeados como (`?<nome>xyz`) serão capturáveis. Isso afeta apenas quais grupos estão sendo capturados, ainda é possível usar referências numeradas de sub-expressões e o array de correspondências ainda conterá resultados numerados. Disponível a partir do PHP 8.2.0.

r (PCRE2_EXTRA_CASELESS_RESTRICT)

Quando *u* (PCRE_UTF8) e *i* (PCRE_CASELESS) estão ativos, este modificador evita correspondência entre caracteres ASCII e não-ASCII.

Por exemplo, `preg_match('/\x{212A}/iu', "K")` corresponde ao símbolo de Kelvin K (U+212A). Quando *r* é usado (`preg_match('/\x{212A}/iur', "K")`), não há correspondência.

Disponível a partir do PHP 8.4.0.

Diferenças para o Perl

As diferenças descritas aqui referem-se ao Perl 5.005.

1. Por padrão, um caractere de espaço em branco é qualquer caractere que a função isspace() da biblioteca C reconhece, embora seja possível compilar PCRE com tabelas de tipos de caracteres alternativas. Normalmente isspace() corresponde a espaço, alimentação de formulário, nova linha, retorno de carro, tabulação horizontal e tabulação vertical. Perl 5 não inclui mais a tabulação vertical em seu conjunto de caracteres de espaço em branco. O escape \v que esteve na documentação do Perl por muito tempo nunca foi de fato reconhecido. No entanto, o caractere em si foi tratado como espaço em branco pelo menos até a versão 5.002. Na 5.004 e na 5.005, ele não corresponde a \s.

2. O PCRE não permite quantificadores repetidos em afirmações antecipadas. O Perl permite isso, mas eles não significam o que se imagina. Por exemplo, (?!a){3} não afirma que os próximos três caracteres não são "a". Apenas afirma que o próximo caractere não é “a” três vezes.

3. A captura de sub-expressões que ocorrem dentro de afirmações antecipadas negativas são contadas, mas suas entradas no vetor de deslocamentos nunca são definidas. O Perl define suas variáveis numéricas a partir de quaisquer expressões que sejam

correspondidas antes que a afirmação falhe em corresponder a algo (deste modo tendo sucesso), mas apenas se a afirmação antecipada negativa contiver apenas uma ramificação.

4. Embora caracteres zero binários sejam suportados na string de entrada, eles não são permitidos na string da expressão (antes do PHP 8.2) porque ela é passada como uma string C normal, terminada por zero. A sequência de escape "\x00" pode ser usada na expressão para representar um zero binário. A partir do PHP 8.2, caracteres NUL \0 são suportados na string da expressão regular.
5. As seguintes sequências de escape do Perl não são suportadas: \l, \u, \L, \U. Na verdade, eles são implementados pelo tratamento geral de strings do Perl e não fazem parte de seu mecanismo de correspondência de expressões.
6. A asserção Perl \G não é suportada porque não é relevante para correspondências de expressões únicas.
7. Obviamente, a PCRE não suporta a construção (?{code}) e (??{code}). No entanto, há suporte para expressões recursivas.
8. No momento em que este artigo foi escrito, havia algumas curiosidades no Perl 5.005_02 relacionadas às configurações de strings capturadas quando parte de uma expressão é repetida. Por exemplo, corresponder "aba" com a expressão /^(a(b)?)+$/ define $2 como o valor "b", mas corresponder "aabbaa" com /^(aa(bb)?)+$/ deixa $2 indefinido. No entanto, se a expressão for alterada para /^(aa(b(b))?)+$/ então $2 (e $3) serão definidos. No Perl 5.004, $2 é definido em ambos os casos, e isso também é `true` na PCRE. Se no futuro o Perl mudar para um estado consistente diferente, a PCRE poderá mudar para seguir o mesmo comportamento.
9. Outra discrepância ainda não resolvida é que no Perl 5.005_02 o padrão /^(a)?(?(1)a|b)+$/ corresponde à string "a", enquanto que na PCRE isso não acontece. No entanto, tanto no Perl quanto na PCRE, /^(a)?a/ correspondido com "a" deixa $1 indefinido.
10. PCRE fornece algumas extensões para os recursos de expressão regular Perl:
    1. Embora as afirmações retroativas devam corresponder a strings de comprimento fixo, cada ramificação alternativa de uma afirmação retroativa pode corresponder a um comprimento diferente de string. O Perl 5.005 exige que todas tenham o mesmo comprimento.
    2. Se PCRE_DOLLAR_ENDONLY estiver definida e PCRE_MULTILINE não estiver definida, o metacaractere $ corresponderá apenas ao final da string.
    3. Se PCRE_EXTRA estiver definida, uma barra invertida seguida por uma letra sem significado especial será uma falha.
    4. Se PCRE_UNGREEDY estiver definida, a ganância dos quantificadores de repetição é invertida, ou seja, por padrão eles não serão gananciosos, mas se seguidos de um ponto de interrogação eles serão.

Funções da PCRE

Índice

preg_filter

preg_filter

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

preg_filter — Executa pesquisa e substituição usando expressões regulares

Descrição

```
function preg_filter(
    string|array $pattern,
    string|array $replacement,
    string|array $subject,
    int $limit = -1,
    int &$count = null
): string|array|null
```

preg_filter() é idêntica à preg_replace() exceto que ela somente retorna strings de entrada (possivelmente transformadas) onde ocorreu uma correspondência. Para detalhes sobre como esta função trabalha, leia a documentação sobre preg_replace().

Parâmetros

Parâmetros estão descritos na documentação sobre preg_replace().

Valor Retornado

Retorna um array se o parâmetro `subject` for um array, ou uma string caso contrário.

Se nenhuma correspondência for encontrada ou se ocorrer um erro, um array vazio é retornado quando `subject` for um array ou `null` caso contrário.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo comparando preg_filter() com preg_replace()

```php
<?php
$subject = array('1', 'a', '2', 'b', '3', 'A', 'B', '4');
$pattern = array('/\d/', '/[a-z]/', '/[1a]/');
$replace = array('A:$0', 'B:$0', 'C:$0');

echo "preg_filter retorna\n";
print_r(preg_filter($pattern, $replace, $subject));

echo "preg_replace retorna\n";
print_r(preg_replace($pattern, $replace, $subject));
?>
```

O exemplo acima produzirá:

```
preg_filter retorna
Array
(
    [0] => A:C:1
    [1] => B:C:a
    [2] => A:2
    [3] => B:b
    [4] => A:3
    [7] => A:4
)
preg_replace retorna
Array
(
    [0] => A:C:1
    [1] => B:C:a
    [2] => A:2
    [3] => B:b
    [4] => A:3
    [5] => A
    [6] => B
    [7] => A:4
)
```

Veja também

- Expressões regulares PCRE
- preg_quote() - Escapa caracteres de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_replace_callback() - Executa uma busca por expressão regular e substitui usando uma chamada de retorno
- preg_grep() - Retorna entradas de array que correspondem à expressão regular
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada

preg_grep

preg_grep

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_grep — Retorna entradas de array que correspondem à expressão regular

Descrição

function **preg_grep**(string `$pattern`, array `$array`, int `$flags` = 0): array|false

Retorna o array consistindo dos elementos do parâmetro `array` que correspondem à expressão regular informada em `pattern`.

Parâmetros

`pattern`

A expressão a ser pesquisada, como uma string.

`array`

O array de entrada.

`flags`

Se definido para **`PREG_GREP_INVERT`**, esta função retornará os elementos do array de entrada que *não* correspondem ao `pattern` informado.

Valor Retornado

Retorna um array indexado usando as chaves do parâmetro `array`, ou **`false`** em caso de falha.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um **`E_WARNING`** será emitido.

Examples

Exemplo #1 Exemplo de preg_grep()

```php
<?php
$array = [ "4", M_PI, "2.74", 42 ];

// retorna todos os elementos do array contendo número em ponto flutuante
$fl_array = preg_grep("/^(\d+)?\.\d+$/", $array);

var_dump($fl_array);
?>
```

Veja também

- Expressões regulares PCRE
- preg_quote() - Escapa caracteres de expressão regular
- preg_match_all() - Realiza uma correspondência global de expressão regular
- preg_filter() - Executa pesquisa e substituição usando expressões regulares
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada

preg_last_error

preg_last_error

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

preg_last_error — Retorna o código de erro da última expressão regular PCRE executada

Descrição

function **preg_last_error**(): int

Retorna o código de erro da última expressão regular PCRE executada.

Exemplo #1 Exemplo de preg_last_error()

```php
<?php

preg_match('/(?:\D+|<\d+>)*[!?]/', 'foobar foobar foobar');
```

```php
if (preg_last_error() == PREG_BACKTRACK_LIMIT_ERROR) {
    echo 'Limite de rastreamento retroativo ultrapassado!';
}

?>
```

O exemplo acima produzirá:

```
Limite de rastreamento retroativo ultrapassado!
```

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma das seguintes constantes (explicadas em sua própria página):

- **PREG_NO_ERROR**
- **PREG_INTERNAL_ERROR**
- **PREG_BACKTRACK_LIMIT_ERROR** (veja também pcre.backtrack_limit)
- **PREG_RECURSION_LIMIT_ERROR** (veja também pcre.recursion_limit)
- **PREG_BAD_UTF8_ERROR**
- **PREG_BAD_UTF8_OFFSET_ERROR**
- **PREG_JIT_STACKLIMIT_ERROR**

Veja também

- preg_last_error_msg() - Retorna a mensagem de erro da última execução de expressão regular PCRE

preg_last_error_msg

preg_last_error_msg

(PHP 8)

preg_last_error_msg — Retorna a mensagem de erro da última execução de expressão regular PCRE

Descrição

function **preg_last_error_msg**(): string

Retorna a mensagem de erro da última execução de expressão regular PCRE.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a mensagem de erro em caso de sucesso, ou `"No error"` se nenhum erro tiver ocorrido.

Examples

Exemplo #1 Exemplo de preg_last_error_msg()

```php
<?php

preg_match('/(?:\D+|<\d+>)*[!?]/', 'foobar foobar foobar');

if (preg_last_error() !== PREG_NO_ERROR) {
    echo preg_last_error_msg();
}
```

```
?>
```

O exemplo acima produzirá:

```
Backtrack limit exhausted
```

Veja também

- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada

preg_match

preg_match

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_match — Realiza uma correspondência com expressão regular

Descrição

```
function preg_match(
    string $pattern,
    string $subject,
    array &$matches = null,
    int $flags = 0,
    int $offset = 0
): int|false
```

Pesquisa `subject` por uma correspondência à expressão regular informada em `pattern`.

Parâmetros

`pattern`

A expressão sendo procurada, como uma string.

`subject`

A string de entrada.

`matches`

Se `matches` for informada, será preenchida com os resultados da pesquisa. *$matches[0]* contéra o texto que correspondeu à expressão inteira, *$matches[1]* terá o texto que correspondeu à primeira sub-expressão entre parênteses capturada, e assim por diante.

`flags`

`flags` pode ser uma combinação das seguintes constantes:

`PREG_OFFSET_CAPTURE`

Se esta constante for passada, para cada correspondência o deslocamento (em bytes) da string também será retornado. Observe que isto modifica o valor de `matches` para um array de arrays onde cada elemento é um array que consiste na string correspondida no elemento `0` e seu deslocamento na string `subject` no elemento `1`.

```
<?php
preg_match('/(foo)(bar)(baz)/', 'foobarbaz', $matches, PREG_OFFSET_CAPTURE);
print_r($matches);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => foobarbaz
            [1] => 0
        )

    [1] => Array
        (
            [0] => foo
            [1] => 0
        )

    [2] => Array
        (
            [0] => bar
            [1] => 3
        )

    [3] => Array
        (
            [0] => baz
            [1] => 6
        )

)
```

PREG_UNMATCHED_AS_NULL

Se esta constante for passada, sub-expressões não correspondidas são reportadas como `null` e sempre são incluídas nos resultados (incluindo as finais). Sem esta constante, subpadrões não correspondidos que são seguidos por um subpadrão correspondente são reportados como uma string vazia, enquanto subpadrões não correspondidos finais são omitidos dos resultados completamente.

```php
<?php
preg_match('/(a)(b)*(c)/', 'ac', $matches);
var_dump($matches);
preg_match('/(a)(b)*(c)/', 'ac', $matches, PREG_UNMATCHED_AS_NULL);
var_dump($matches);

// Trailing unmatched subpatterns:
preg_match('/(a)(b)?(c)?/', 'a', $matches);
var_dump($matches);
preg_match('/(a)(b)?(c)?/', 'a', $matches, PREG_UNMATCHED_AS_NULL);
var_dump($matches);
?>
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  string(2) "ac"
  [1]=>
  string(1) "a"
  [2]=>
  string(0) ""
  [3]=>
  string(1) "c"
}
array(4) {
  [0]=>
  string(2) "ac"
```

```
  [1]=>
  string(1) "a"
  [2]=>
  NULL
  [3]=>
  string(1) "c"
}
array(2) {
  [0]=>
  string(1) "a"
  [1]=>
  string(1) "a"
}
array(4) {
  [0]=>
  string(1) "a"
  [1]=>
  string(1) "a"
  [2]=>
  NULL
  [3]=>
  NULL
}
```

`offset`

Normalmente, a pesquisa começa no início da string de entrada. O parâmetro opcional `offset` pode ser usado para especificar o deslocamento alternativo (em bytes) a partir do qual a pesquisa será iniciada.

> **Nota** :
>
> Usar `offset` não é equivalente a passar `substr($subject, $offset)` para **preg_match()** no lugar da string de entrada, porque `pattern` pode conter afirmações como ^, *$* ou *(?<=x)* que depende da string inteira. Compare:
>
> ```php
> <?php
> $subject = "abcdef";
> $pattern = '/^def/';
> preg_match($pattern, $subject, $matches, PREG_OFFSET_CAPTURE, 3);
> print_r($matches);
> ?>
> ```
>
> O exemplo acima produzirá:
>
> ```
> Array
> (
> )
> ```
>
> enquanto este exemplo
>
> ```php
> <?php
> $subject = "abcdef";
> $pattern = '/^def/';
> preg_match($pattern, substr($subject,3), $matches, PREG_OFFSET_CAPTURE);
> print_r($matches);
> ?>
> ```
>
> produz

```
Array
(
    [0] => Array
        (
            [0] => def
            [1] => 0
        )

)
```

Alternativamente, para evitar o uso de substr(), use a afirmação `\G` no lugar da âncora `^`, ou o modificador `A`; ambos não funcionam com o parâmetro `offset`.

Valor Retornado

preg_match() retorna 1 se `pattern` corresponder a `subject`, 0 se não corresponder, ou **`false`** em caso de falha.

> **Aviso**
>
> Esta função pode retornar o valor booleano **`false`**, mas também pode retornar um valor não booleano que pode ser avaliado como **`false`**. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um **`E_WARNING`** será emitido.

Registro de Alterações

Versão	Descrição
7.4.0	Quando a constante **`PREG_UNMATCHED_AS_NULL`** é usada, os grupos de captura finais não correspondidos agora também são incluídos no resultado com o valor **`null`** (ou `[null, -1]` quando **`PREG_OFFSET_CAPTURE`** também é usado), de forma que *$matches* sempre tenha o mesmo tamanho. Anteriormente, eles eram omitidos.
7.2.0	A constante **`PREG_UNMATCHED_AS_NULL`** agora é suportada para o parâmetro `flags`.

Examples

Exemplo #1 Encontrando a string de texto "php"

```php
<?php
// A letra "i" após o delimitador da expressão indica uma pesquisa insensível a maiúsculas/minúsculas
if (preg_match("/php/i", "PHP is the web scripting language of choice.")) {
    echo "Uma correspondência foi encontrada.";
} else {
    echo "Uma correspondência foi encontrada.";
}
?>
```

Exemplo #2 Encontrando a palavra "web"

```php
<?php
/* \b na expressão indica uma borda de palavra, portanto somente a palavra
 * separada "web" é correspondida, e não uma palavra que contém "web" como "webbing" ou "cobweb" */
if (preg_match("/\bweb\b/i", "PHP is the web scripting language of choice.")) {
    echo "Uma correspondência foi encontrada.";
} else {
    echo "Uma correspondência foi encontrada.";
}

echo "\n";

if (preg_match("/\bweb\b/i", "PHP is the website scripting language of choice.")) {
    echo "Uma correspondência foi encontrada.";
} else {
    echo "Uma correspondência foi encontrada.";
```

```
}
?>
```

Exemplo #3 Obtendo o nome de domínio de uma URL

```php
<?php
// obtem o nome do servidor da URL
preg_match('@^(?:http://)?([^/]+)@i',
    "http://www.php.net/index.html", $matches);
$host = $matches[1];

// obtém os últimos dois segmentos do nome do servidor
preg_match('/[^.]+\.[^.]+$/', $host, $matches);
echo "nome de domínio é: {$matches[0]}\n";
?>
```

O exemplo acima produzirá:

```
nome de domínio é: php.net
```

Exemplo #4 Usando sub-expressão nomeada

```php
<?php

$str = 'foobar: 2008';

preg_match('/(?P<name>\w+): (?P<digit>\d+)/', $str, $matches);

/* Alternativa */
// preg_match('/(?<name>\w+): (?<digit>\d+)/', $str, $matches);

print_r($matches);

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => foobar: 2008
    [name] => foobar
    [1] => foobar
    [digit] => 2008
    [2] => 2008
)
```

Notas

> **Dica**
>
> Não use **preg_match()** se for requerido apenas verificar se uma string está contida em outra. Use strpos() em seu lugar, pois será mais rápida.

Veja também

- Expressões PCRE
- preg_quote() - Escapa caracteres de expressão regular

- preg_match_all() - Realiza uma correspondência global de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_split() - Divide uma string com base em expressão regular
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada
- preg_last_error_msg() - Retorna a mensagem de erro da última execução de expressão regular PCRE

preg_match_all

preg_match_all

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_match_all — Realiza uma correspondência global de expressão regular

Descrição

```
function preg_match_all(
    string $pattern,
    string $subject,
    array &$matches = null,
    int $flags = 0,
    int $offset = 0
): int|false
```

Pesquisa `subject` por todas as correspondências à expressão regular informada em `pattern` e insere-as em `matches` na ordem especificada por `flags`.

Depois que a primeira correspondência é encontrada, as pesquisas subsequentes continuam a partir do final da última correspondência.

Parâmetros

`pattern`

A expressão regular a ser procurada, como uma string.

`subject`

A string de entrada.

`matches`

Array de todas as correspondências em um array multi-dimensional ordenado de acordo com `flags`.

`flags`

Se nenhuma opção de ordenamento for informada, **`PREG_PATTERN_ORDER`** será usada.

Pode ser uma combinação das seguintes constantes (observe que não faz sentido usar **`PREG_PATTERN_ORDER`** junto com **`PREG_SET_ORDER`**):

`PREG_PATTERN_ORDER`

Ordena os resultados de forma que *$matches[0]* será um array das correspondências completas à expressão, *$matches[1]* será um array de strings correspondidas pela primeira sub-expressão entre parênteses, e assim por diante.

```php
<?php
preg_match_all("|<[^>]+>(.*)</[^>]+>|U",
    "<b>exemplo: </b><div align=left>isto é um teste</div>",
    $out, PREG_PATTERN_ORDER);
echo $out[0][0] . ", " . $out[0][1] . "\n";
echo $out[1][0] . ", " . $out[1][1] . "\n";
?>
```

O exemplo acima produzirá:

```
<b>exemplo: </b>, <div align=left>isto é um teste</div>
exemplo: , isto é um teste
```

Portanto, *$out[0]* contém um array de strings que corresponderam a toda a expressão, e *$out[1]* contém um array de strings envolvidas por etiquetas HTML.

Se a expressão contiver sub-expressões nomeadas, *$matches* conterá, adicionalmente, entradas para chaves com o nome da sub-expressão.

Se a expressão contiver sub-expressões nomeadas duplicadas, somente a mais à direita será armazenada em *$matches[NOME]*.

```php
<?php
preg_match_all(
    '/(?J)(?<match>foo)|(?<match>bar)/',
    'foo bar',
    $matches,
    PREG_PATTERN_ORDER
);
print_r($matches['match']);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] =>
    [1] => bar
)
```

PREG_SET_ORDER

Ordena os resultados de forma que *$matches[0]* será um array do primeiro conjunto de correspondências, *$matches[1]* será um array do segundo conjunto, e assim por diante.

```php
<?php
preg_match_all("|<[^>]+>(.*)</[^>]+>|U",
    "<b>exemplo: </b><div align=\"left\">isto é um teste</div>",
    $out, PREG_SET_ORDER);
echo $out[0][0] . ", " . $out[0][1] . "\n";
echo $out[1][0] . ", " . $out[1][1] . "\n";
?>
```

O exemplo acima produzirá:

```
<b>exemplo:: </b>, exemplo::
<div align="left">isto é um teste</div>, isto é um teste
```

PREG_OFFSET_CAPTURE

Se esta constante for passada, para cada correspondência o deslocamento (em bytes) da string também será retornado. Observe que isto modifica o valor de `matches` para um array de arrays onde cada elemento é um array que consiste na string correspondida no elemento `0` e seu deslocamento na string `subject` no elemento `1`.

```php
<?php
preg_match_all('/(foo)(bar)(baz)/', 'foobarbaz', $matches, PREG_OFFSET_CAPTURE);
print_r($matches);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => Array
                (
                    [0] => foobarbaz
                    [1] => 0
                )

        )

    [1] => Array
        (
            [0] => Array
                (
                    [0] => foo
                    [1] => 0
                )

        )

    [2] => Array
        (
            [0] => Array
                (
                    [0] => bar
                    [1] => 3
                )

        )

    [3] => Array
        (
            [0] => Array
                (
                    [0] => baz
                    [1] => 6
                )

        )

)
```

`PREG_UNMATCHED_AS_NULL`

Se esta constante for passada, sub-expressões não correspondidas são reportadas como `null`; caso contrário elas são reportadas como strings vazias.

`offset`

Normalmente, a pesquisa começa no início da string de entrada. O parâmetro opcional `offset` pode ser usado para especificar o deslocamento alternativo (em bytes) a partir do qual a pesquisa será iniciada.

> **Nota** :
>
> Usar `offset` não é equivalente a passar `substr($subject, $offset)` para **preg_match_all()** no lugar da string de entrada, porque `pattern` pode conter afirmações como `^`, *$* ou *(?<=x)* que depende da string inteira. Consulte preg_match() para exemplos.

Valor Retornado

Retorna o número de correspondências completas à expressão (que pode ser zero), ou `false` em caso de falha.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um `E_WARNING` será emitido.

Registro de Alterações

Versão	Descrição
7.2.0	A constante `PREG_UNMATCHED_AS_NULL` agora é suportada para o parâmetro `flags`.

Examples

Exemplo #1 Obtendo todos os números de telefine de algum texto.

```
<?php
preg_match_all("/\(?  (\d{3})?  \)?  (?(1)  [\-\s] ) \d{3}-\d{4}/x",
                "Ligue para 555-1212 ou 1-800-555-1212", $phones);
?>
```

Exemplo #2 Encontrando etiquetas HTML (ganancioso)

```
<?php
// A expressão \\2 é um exemplo de referência retroativa. Isto informa ao PCRE que
// ele deve corresponder ao segundo conjunto de parênteses na expressão regular
// em si, que seria ([\w]+) neste caso. A barra invertida extra em \\2 é
// necessária porque a string está em aspas duplas.
$html = "<b>texto negrito</b><a href=howdy.html>clique aqui</a>";

preg_match_all("/(<([\w]+)[^>]*>)(.*?)(<\/\\2>)/", $html, $matches, PREG_SET_ORDER);

foreach ($matches as $val) {
    echo "correspondeu: " . $val[0] . "\n";
    echo "parte 1: " . $val[1] . "\n";
    echo "parte 2: " . $val[2] . "\n";
    echo "parte 3: " . $val[3] . "\n";
    echo "parte 4: " . $val[4] . "\n\n";
}
?>
```

O exemplo acima produzirá:

```
correspondeu: <b>texto negrito</b>
parte 1: <b>
parte 2: b
parte 3: texto negrito
parte 4: </b>

correspondeu: <a href=howdy.html>clique aqui</a>
parte 1: <a href=howdy.html>
parte 2: a
parte 3: clique aqui
parte 4: </a>
```

Exemplo #3 Usando sub-expressões nomeadas

```
<?php

$str = <<<FOO
a: 1
b: 2
c: 3
FOO;

preg_match_all('/(?P<name>\w+): (?P<digit>\d+)/', $str, $matches);
```

```php
/* Alternativa */
// preg_match_all('/(?<name>\w+): (?<digit>\d+)/', $str, $matches);

print_r($matches);

?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => a: 1
            [1] => b: 2
            [2] => c: 3
        )

    [name] => Array
        (
            [0] => a
            [1] => b
            [2] => c
        )

    [1] => Array
        (
            [0] => a
            [1] => b
            [2] => c
        )

    [digit] => Array
        (
            [0] => 1
            [1] => 2
            [2] => 3
        )

    [2] => Array
        (
            [0] => 1
            [1] => 2
            [2] => 3
        )

)
```

Veja também

- Expressões regulares PCRE
- preg_quote() - Escapa caracteres de expressão regular
- preg_match() - Realiza uma correspondência com expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_split() - Divide uma string com base em expressão regular
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada

preg_quote

preg_quote

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_quote — Escapa caracteres de expressão regular

Descrição

function **preg_quote**(string `$str`, ?string `$delimiter` = `null`): string

preg_quote() insere uma barra invertida na frente de cada caractere de `str` que seja parte da sintaxe de expressões regulares. Isto é útil caso exista uma string no momento da execução que precisa ser correspondida a algum texto e que possa conter caracteres especiais de expressões regulares.

Os caracteres especiais de expressões regulares são: `. \ + * ? [ ^ ] $ ( ) { } = ! < > | : - #`

Observe que `/` não é um caractere especial de expressões regulares.

> **Nota** :
>
> Veja que **preg_quote()** não foi projetada para ser aplicada às strings de substituição da função preg_replace() e outras.

Parâmetros

`str`

A string de entrada.

`delimiter`

Se o parâmetro opcional `delimiter` for especificado, o ele também será escapado. Isto é útil para escapar o delimitador requerido pelas funções PCRE. A barra `/` é o delimitador mais comum.

Valor Retornado

Retorna a string escapada.

Registro de Alterações

Versão	Descrição
7.3.0	O caractere # agora é escapado.
7.2.0	`delimiter` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de **preg_quote()**

```php
<?php
$keywords = '$40 por um g3/400';
$keywords = preg_quote($keywords, '/');
echo $keywords; // retorna \$40 por um g3\/400
?>
```

Exemplo #2 Transformando uma palavra em itálico dentro de um texto

```php
<?php
// Neste exemplo, preg_quote($word) é usada para impedir que
// os asteriscos tenham um significado especial na expressão
// regular.

$textbody = "Este livro é *muito* difícil de encontrar.";
$word = "*muito*";
$textbody = preg_replace ("/" . preg_quote($word, '/') . "/",
                          "<i>" . $word . "</i>",
                          $textbody);
?>
```

Notas

> **Nota** :

> Esta função é compatível com dados binários.

Veja também

- Expressões regulares PCRE
- escapeshellcmd() - Escapa meta-caracteres shell

preg_replace

preg_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_replace — Realiza uma pesquisa por uma expressão regular e substitui

Descrição

```
function preg_replace(
    string|array $pattern,
    string|array $replacement,
    string|array $subject,
    int $limit = -1,
    int &$count = null
): string|array|null
```

Pesquisa `subject` por correspondências a `pattern` e substitui-as por `replacement`.

Para procurar correspondência a uma string exata, e não a uma expressão, considere usar str_replace() ou str_ireplace() no lugar desta função.

Parâmetros

`pattern`

A expressão a ser pesquisada. Pode ser uma string ou um array com strings.

Diversos modificadores PCRE também estão disponíveis.

`replacement`

A string ou array com strings para substituir. Se este parâmetro for uma string e o parâmetro `pattern` for um array, todas as expressões do array serão substituídos por esta string. Se ambos os parâmetros `pattern` e `replacement` forem arrays, cada expressão correspondida de `pattern` será substituída pelo seu `replacement` correspondente. Se houver menos elementos no array `replacement` que no array `pattern` array, qualquer `pattern` extra será substituído por uma string vazia.

`replacement` pode conter referências na forma `\n` ou `$n`, com a última forma sendo a preferível. Cada referência desse tipo será substituída pelo texto capturado pelo n-ésimo padrão entre parentêses. n pode ser de 0 to 99, e `\0` ou `$0` referem-se ao texto correspondente à expressão inteira. Parênteses de abertura são contados da esquerda para direita (iniciando em 1) para obter o número da sub-expressão de captura. Note que a barra invertida em strings literais pode precisar ser escapada.

Quando se trabalha com uma expressão de substituição, onde a referência retroativa é imediatamente seguida por outro número (isto é, inserido um número literal imediatamente após uma expressão correspondente), não se pode usar a habitual notação `\1` para a referência retroativa. `\11`, por exemplo, iria confundir a função **preg_replace()** pois ela não saberá se o desejado é a referência `\1` seguida por um literal `1`, ou a referência `\11` seguida por nada. Neste caso, a solução é usar `${1}1`. Isto cria uma referência `$1` isolada, deixando o `1` como um literal.

`subject`

String ou um array com strings para pesquisar e substituir.

Se `subject` for um array, então a pesquisa e a substituição serão executadas em cada entrada de `subject`, e o valor de retorno será um array também.

Se o array `subject` for associativo, as chaves são preservadas no valor de retorno.

`limit`

O máximo possível de substituições para cada expressão em cada string de `subject`. O padrão é `-1` (sem limite).

`count`

Se especificada, essa variável será preenchida com o número de substiuições realizadas.

Valor Retornado

preg_replace() retorna um array se o parâmetro `subject` for um array, caso contrário retorna uma string.

Se correspondências forem encontradas, o novo `subject` será devolvido, caso contrário `subject` será devolvido inalterado ou `null` se um erro tiver ocorrido.

Erros/Exceções

Usar o modificador "\e" é um erro; um erro de nível `E_WARNING` é emitido neste caso.

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um `E_WARNING` será emitido.

Examples

Exemplo #1 Usando referências retroativas seguidas por literais numéricos

```php
<?php
$string = 'April 15, 2003';
$pattern = '/(\w+) (\d+), (\d+)/i';
$replacement = '${1}1,$3';
echo preg_replace($pattern, $replacement, $string);
?>
```

O exemplo acima produzirá:

```
April1,2003
```

Exemplo #2 Usando arrays indexados com preg_replace()

```php
<?php
$string = 'The quick brown fox jumped over the lazy dog.';
$patterns = array();
$patterns[0] = '/quick/';
$patterns[1] = '/brown/';
$patterns[2] = '/fox/';
$replacements = array();
$replacements[2] = 'bear';
$replacements[1] = 'black';
$replacements[0] = 'slow';
echo preg_replace($patterns, $replacements, $string);
?>
```

O exemplo acima produzirá:

```
The bear black slow jumped over the lazy dog.
```

Ao ordenar expressões e substituições com ksort, obtém-se o desejado.

```php
<?php
$string = 'The quick brown fox jumps over the lazy dog.';
$patterns = array();
$patterns[0] = '/quick/';
$patterns[1] = '/brown/';
```

```php
$patterns[2] = '/fox/';
$replacements = array();
$replacements[2] = 'bear';
$replacements[1] = 'black';
$replacements[0] = 'slow';
ksort($patterns);
ksort($replacements);
echo preg_replace($patterns, $replacements, $string);
?>
```

O exemplo acima produzirá:

```
The slow black bear jumped over the lazy dog.
```

Exemplo #3 Substituindo vários valores

```php
<?php
$patterns = array ('/(19|20)(\d{2})-(\d{1,2})-(\d{1,2})/',
                   '/^\s*{(\w+)}\s*=/');
$replace = array ('\3/\4/\1\2', '$\1 =');
echo preg_replace($patterns, $replace, '{startDate} = 1999-5-27');
?>
```

O exemplo acima produzirá:

```
$startDate = 5/27/1999
```

Exemplo #4 Removendo espaços

Este exemplo retira o excesso de espaçoes em branco de uma string.

```php
<?php
$str = 'foo   o';
$str = preg_replace('/\s\s+/', ' ', $str);
// O resultado será 'foo o'
echo $str;
?>
```

Exemplo #5 Usando o parâmetro `count`

```php
<?php
$count = 0;

echo preg_replace(array('/\d/', '/\s/'), '*', 'xp 4 to', -1 , $count);
echo "\n";
echo $count; //3
?>
```

O exemplo acima produzirá:

```
xp***to
3
```

Notas

> **Nota** :
>
> Ao usar arrays com `pattern` e `replacement`, as chaves são processadas na ordem em que aparecem no array. Isto *não é necessariamente* o mesmo que a ordem dos índices numéricos. Se os índices forem utilizados para indentificar qual `pattern` deve ser substituído por qual `replacement`, é necessário executar a função ksort() em cada array antes de chamar a função **preg_replace()**.

> **Nota** :
>
> Quando ambos `pattern` e `replacement` forem arrays, as regras de correspondência irão operar sequencialmente. Isto é, o segundo par `pattern`/`replacement` irá operar na string que resulta do primeiro par `pattern`/`replacement`, e não na string original. Se for desejado simular substituições operando em paralelo, como inversão de dois valores, substitua uma expressão por uma referência intermediária, e então em um par posterior substitua a referência intermediária pelo substituto desejado.
>
> ```php
> <?php
> $p = array('/a/', '/b/', '/c/');
> $r = array('b', 'c', 'd');
> print_r(preg_replace($p, $r, 'a'));
> // prints d
> ?>
> ```

Veja também

- Expressões PCRE
- preg_quote() - Escapa caracteres de expressão regular
- preg_filter() - Executa pesquisa e substituição usando expressões regulares
- preg_match() - Realiza uma correspondência com expressão regular
- preg_replace_callback() - Executa uma busca por expressão regular e substitui usando uma chamada de retorno
- preg_split() - Divide uma string com base em expressão regular
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada
- str_replace() - Substitui todas as ocorrências da string de pesquisa com a string de substituição

preg_replace_callback

preg_replace_callback

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

preg_replace_callback — Executa uma busca por expressão regular e substitui usando uma chamada de retorno

Descrição

```
function preg_replace_callback(
    string|array $pattern,
    callable $callback,
    string|array $subject,
    int $limit = -1,
    int &$count = null,
    int $flags = 0
): string|array|null
```

O comportamento desta função é quase idêntico ao da preg_replace(), exceto pelo fato de que ao invés do parâmetro `replacement`, deve-se especificar o parâmetro `callback`.

Parâmetros

`pattern`

A padrão usado para a busca. Pode ser tanto uma string como um array de strings.

`callback`

Uma função de retorno que será chamada e passado um array dos elementos correspondentes na string `subject`. A função deve retornar a string substituta. Esta é a assinatura da função de retorno:

function handler(array `$matches`): string

Normalmente será necessária uma função `callback` para **preg_replace_callback()** em apenas um lugar. Neste caso pode-se usar uma função anônima para declarar a função de retorno dentro da chamada a **preg_replace_callback()**. Desta forma, todas as informações para a chamada estarão no mesmo lugar e não poluem o espaço de nomes com um nome de função que não é usado em nenhum outro lugar.

Exemplo #1 preg_replace_callback() e função anônima

```php
<?php
/* um filtro de linha de comando em estilo Unix para converter maiúsculas
 * no início de parágrafos para letras minúsculas */
$fp = fopen("php://stdin", "r") or die("não foi possível ler a entrada padrão");
while (!feof($fp)) {
    $line = fgets($fp);
    $line = preg_replace_callback(
        '|<p>\s*\w|',
        function ($matches) {
            return strtolower($matches[0]);
        },
        $line
    );
    echo $line;
}
fclose($fp);
?>
```

`subject`

A string ou array com strings para procurar e modificar.

`limit`

O máximo de possíveis substituições para cada expressão em cada string de `subject`. O padrão é `-1` (sem limite).

`count`

Se especificado, esta variável será preenchida com o número de substituições feitas.

`flags`

`flags` pode ser uma combinação das opções `PREG_OFFSET_CAPTURE` e `PREG_UNMATCHED_AS_NULL`, que influenciam o formato do array de correspondências. Veja a descrição em preg_match() para mais detalhes.

Valor Retornado

preg_replace_callback() retorna um array se o parâmetro `subject` for um array, ou uma string caso contrário. Em caso de erro o valor de retorno é `null`.

Se ocorrer correspondência, a nova string será retornada, caso contrário `subject` será retornada inalterada.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um `E_WARNING` será emitido.

Registro de Alterações

Versão	Descrição
7.4.0	O parâmetro `flags` foi adicionado.

Examples

Exemplo #2 Exemplo de preg_replace_callback()

```php
<?php
// este texto foi usado em 2002
// se quisermos atualizá-lo para 2003
```

```php
$text = "Dia da mentira é em 01/04/2002\n";
$text.= "Último Natal foi em 25/12/2001\n";
// a chamada de retorno
function next_year($matches)
{
    // como sempre: $matches[0] é a correspondência completa
    // $matches[1] é a correspondência da primeira sub-expressão
    // envolvida por '(...)' e assim por diante
    return $matches[1].($matches[2]+1);
}
echo preg_replace_callback(
            "|(\d{2}/\d{2}/)(\d{4})|",
            "next_year",
            $text);

?>
```

O exemplo acima produzirá:

```
Dia da mentira é em 01/04/2003
Último Natal foi em 25/12/2002
```

Exemplo #3 preg_replace_callback() usando estrutura recursiva para manipular código BB encapsulado

```php
<?php
$input = "plain [indent] deep [indent] deeper [/indent] deep [/indent] plain";

function parseTagsRecursive($input)
{

    $regex = '#\[indent]((?:[^[]|\[(?!/?indent])|(?R))+)\[/indent]#';

    if (is_array($input)) {
        $input = '<div style="margin-left: 10px">'.$input[1].'</div>';
    }

    return preg_replace_callback($regex, 'parseTagsRecursive', $input);
}

$output = parseTagsRecursive($input);

echo $output;
?>
```

Veja também

- Expressões regulares PCRE
- preg_replace_callback_array() - Realiza pesquisa e substituição em expressão regular usando chamadas de retorno
- preg_quote() - Escapa caracteres de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada
- Funções Anônimas

preg_replace_callback_array

preg_replace_callback_array

(PHP 7, PHP 8)

preg_replace_callback_array — Realiza pesquisa e substituição em expressão regular usando chamadas de retorno

Descrição

```
function preg_replace_callback_array(
    array $pattern,
    string|array $subject,
    int $limit = -1,
    int &$count = null,
    int $flags = 0
): string|array|null
```

O comportamento desta função é similar a preg_replace_callback(), exceto que chamadas de retorno são executada para cada expressão.

Parâmetros

`pattern`

Um array associativo mapeando expressões (chaves) a funções de retorno do tipo callable (valores).

`subject`

A string ou um array com strings para pesquisa e substituição.

`limit`

O número máximo de substituições para cada expressão em cada string de entrada do parâmetro `subject`. O padrão é `-1` (sem limite).

`count`

Se especificada, esta variável será preenchida com o número de substituições realizadas.

`flags`

`flags` pode ser uma combinação das opções **`PREG_OFFSET_CAPTURE`** e **`PREG_UNMATCHED_AS_NULL`**, que influenciam o formato do array de correspondências. Consulte a descrição em preg_match() para mais detalhes.

Valor Retornado

preg_replace_callback_array() retorna um array se o parâmetro `subject` for um array, ou uma string caso contrário. Em caso de erros, o valor de retorno é **`null`**

Se corresponências forem encontradas, a nova string será retornada, caso contrário `subject` será retornado sem alteração.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um **`E_WARNING`** será emitido.

Registro de Alterações

Versão	Descrição
7.4.0	O parâmetro `flags` foi adicionado.

Examples

Exemplo #1 Exemplo de preg_replace_callback_array()

```php
<?php
$subject = 'Aaaaaa Bbb';

preg_replace_callback_array(
    [
        '~[a]+~i' => function ($match) {
            echo strlen($match[0]), ' correspondências a "a" encontradas', PHP_EOL;
        },
        '~[b]+~i' => function ($match) {
            echo strlen($match[0]), ' correspondências a "b" encontradas', PHP_EOL;
        }
    ],
    $subject
);
?>
```

O exemplo acima produzirá:

```
6 correspondências a "a" encontradas
3 correspondências a "b" encontradas
```

Veja também

- Expressões Regulares PCRE
- preg_replace_callback() - Executa uma busca por expressão regular e substitui usando uma chamada de retorno
- preg_quote() - Escapa caracteres de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada
- Funções Anônimas

preg_split

preg_split

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_split — Divide uma string com base em expressão regular

Descrição

```
function preg_split(
    string $pattern,
    string $subject,
    int $limit = -1,
    int $flags = 0
): array|false
```

Divide a string informada com base em uma expressão regular.

Parâmetros

`pattern`

A expressão a ser pesquisada, como uma string.

`subject`

A string de entrada.

`limit`

Se especificado, somente substrings de até `limit` bytes são retornadas com o resto da string sendo colocado na última substring. Um valor de -1 ou 0 em `limit` significa "sem limite".

`flags`

`flags` pode ser qualquer combinação das seguintes opções (combinadas com o operador binário |):

`PREG_SPLIT_NO_EMPTY`

Se esta opção for definida, somente partes não vazias serão retornadas por **preg_split()**.

`PREG_SPLIT_DELIM_CAPTURE`

Se esta opção for definida, expressões entre parênteses na expressão serão capturadas e também retornadas.

`PREG_SPLIT_OFFSET_CAPTURE`

Se esta opção for definida, para cada correspondência que ocorrer, a sua posição na string de entrada também será retornada. Observe que isto altera o valor de retorno para um array onde cada elemento é um array consistindo da string

correspondida no índice `0` e seu deslocamento na string de entrada (`subject`) no índice `1`.

Valor Retornado

Retorna um array contendo substrings de `subject` divididas ao longo de limites correspondidos por `pattern`, ou `false` em caso de falha.

Erros/Exceções

Se o padrão de expressão regular passado não for compilado para uma expressão regular válida, um `E_WARNING` será emitido.

Examples

Exemplo #1 Exemplo de preg_split(): Obter partes de uma string de pesquisa

```php
<?php
// divide a frase por qualquer número de vírgulas ou caracteres representativos de espaço,
// que incluem " ", \r, \t, \n e \f
$keywords = preg_split("/[\s,]+/", "hypertext language, programming");
print_r($keywords);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => hypertext
    [1] => language
    [2] => programming
)
```

Exemplo #2 Dividindo uma string em seus caracteres componentes

```php
<?php
$str = 'string';
$chars = preg_split('//', $str, -1, PREG_SPLIT_NO_EMPTY);
print_r($chars);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => s
    [1] => t
    [2] => r
    [3] => i
    [4] => n
    [5] => g
)
```

Exemplo #3 Dividindo uma string em correspondências e suas posições

```php
<?php
$str = 'hypertext language programming';
$chars = preg_split('/ /', $str, -1, PREG_SPLIT_OFFSET_CAPTURE);
print_r($chars);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [0] => hypertext
            [1] => 0
        )

    [1] => Array
        (
            [0] => language
            [1] => 10
        )

    [2] => Array
        (
            [0] => programming
            [1] => 19
        )

)
```

Notas

> **Dica**
>
> Se o poder das expressões regulares não for necessário, pode-se optar por alternativas mais rápidas (e simples) como explode() ou str_split().

> **Dica**
>
> Se a correspondência falhar, um array com um único elemento contendo a string de entrada será retornado.

Veja também

- Expressões Regulares PCRE
- preg_quote() - Escapa caracteres de expressão regular
- explode() - Divide uma string com base em outra string
- preg_match() - Realiza uma correspondência com expressão regular
- preg_match_all() - Realiza uma correspondência global de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_last_error() - Retorna o código de erro da última expressão regular PCRE executada

String Multibyte

Introdução

Embora existam muitas linguagens nas quais cada caractere necessário pode ser representado por um mapeamento de um para um para um valor de 8 bits, há também várias linguagens que exigem tantos caracteres para a comunicação escrita que não podem ser contidos dentro da faixa que um simples byte pode codificar (um byte é composto por oito bits. Cada bit pode conter apenas dois valores distintos, um ou zero. Devido a isso, um byte só pode representar 256 valores únicos (dois elevado à potência de oito)). Esquemas de codificação de caracteres multibyte foram desenvolvidos para expressar mais de 256 caracteres no sistema de codificação de bytes regular.

Ao manipular (cortar, dividir, emendar, etc.) strings codificadas em uma codificação multibyte, você precisa usar funções especiais, pois dois ou mais bytes consecutivos podem representar um único caractere em tais esquemas de codificação. Caso contrário, se você aplicar uma função de string que não é sensível a multibyte à string, ela provavelmente falhará em detectar o início ou o final do caractere multibyte e acabará com uma string corrompida e inútil que provavelmente perde seu significado original.

`mbstring` fornece funções de string específicas para multibyte que ajudam a lidar com codificações multibyte em PHP. Além disso, `mbstring` lida com a conversão de codificação de caracteres entre os pares de codificação possíveis. `mbstring` é projetada para lidar com codificações baseadas em Unicode, como UTF-8 e UCS-2, e muitas codificações de byte único por conveniência (listadas em Codificações de Caracteres Suportadas).

Instalação/Configuração

Índice

Instalação

`mbstring` é uma extensão não padrão. Isso significa que ela não está habilitada por padrão. Você deve habilitar explicitamente o módulo com a opção `configure`. Veja a seção Instalação para detalhes.

As seguintes opções de configuração estão relacionadas ao módulo `mbstring`.

- **--enable-mbstring**: Habilita as funções do `mbstring`. Esta opção é necessária para usar as funções do `mbstring`.

  libmbfl é necessário para o `mbstring`. libmbfl é incluído com o `mbstring`. Antes do PHP 7.3.0, se libmbfl já está instalado no sistema, **--with-libmbfl[=DIR]** pode ser especificado para usar a biblioteca instalada.

- **--disable-mbregex**: Desabilita funções de expressão regular com suporte a caracteres multibyte.

  Oniguruma é necessário para as funções de expressão regular com suporte a caracteres multibyte. A partir do PHP 7.4.0, pkg-config é usado para detectar a biblioteca libonig. Antes do PHP 7.4.0, Oniguruma estava incluído com o `mbstring`, mas era possível compilá-lo com uma libonig já instalada passando **--with-onig[=DIR]**.

  É possível desabilitar a verificação de retrocesso multibyte na expressão regular especificando **--disable-mbregex-backtrack**.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do mbstring

Nome	Padrão	Modificável	Registro de Alterações
mbstring.language	"neutral"	`INI_ALL`	
mbstring.detect_order	NULL	`INI_ALL`	
mbstring.http_input	"pass"	`INI_ALL`	Descontinuado
mbstring.http_output	"pass"	`INI_ALL`	Descontinuado
mbstring.internal_encoding	NULL	`INI_ALL`	Descontinuado
mbstring.substitute_character	NULL	`INI_ALL`	
mbstring.func_overload	"0"	`INI_SYSTEM`	Descontinuado a partir do PHP 7.2.0; removido a partir do PHP 8.0.0.
mbstring.encoding_translation	"0"	`INI_PERDIR`	

Nome	Padrão	Modificável	Registro de Alterações
mbstring.http_output_conv_mimetypes	"^(text/|application/xhtml\+xml)"	INI_ALL	
mbstring.strict_detection	"0"	INI_ALL	
mbstring.regex_retry_limit	"1000000"	INI_ALL	Disponível a partir do PHP 7.4.0.
mbstring.regex_stack_limit	"100000"	INI_ALL	Disponível a partir do PHP 7.3.5.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`mbstring.language` string

A configuração padrão do idioma nacional (NLS) usada no mbstring. Note que esta opção define automaticamente `mbstring.internal_encoding` e `mbstring.internal_encoding` deve ser colocada após `mbstring.language` no *php.ini*

`mbstring.encoding_translation` bool

Ativa o filtro de codificação de caracteres transparente para as consultas HTTP de entrada, que realiza a detecção e conversão da codificação de entrada para a codificação de caracteres interna.

`mbstring.internal_encoding` string

> **Aviso**
>
> Este recurso descontinuado *será* certamente *removido* no futuro.

Define a codificação de caracteres interna padrão.

Os usuários devem deixar isso vazio e definir `default_charset` em vez disso.

`mbstring.http_input` string

> **Aviso**
>
> Este recurso descontinuado *será* certamente *removido* no futuro.

Define a codificação de caracteres de entrada HTTP padrão.

Os usuários devem deixar isso vazio e definir `default_charset` em vez disso.

`mbstring.http_output` string

> **Aviso**
>
> Este recurso descontinuado *será* certamente *removido* no futuro.

Define a codificação de caracteres de saída HTTP padrão (a saída será convertida da codificação interna para a codificação de saída HTTP na saída).

Os usuários devem deixar isso vazio e definir `default_charset` em vez disso.

`mbstring.detect_order` string

Define a ordem padrão de detecção de código de caracteres. Veja também mb_detect_order().

`mbstring.substitute_character` string

Define o caractere a ser substituído por codificação de caracteres inválida. Veja mb_substitute_character() para os valores suportados.

`mbstring.func_overload` string

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 7.2.0 e foi *REMOVIDO* a partir do PHP 8.0.0. O uso deste recurso é fortemente desencorajado.

Sobrecarrega um conjunto de funções de byte único pelas contrapartes do mbstring. Veja Sobrecarga de Funções para mais informações.

Esta configuração só pode ser alterada no arquivo *php.ini*.

`mbstring.http_output_conv_mimetypes` string

`mbstring.strict_detection` bool

Ativa a detecção estrita de codificação. Veja mb_detect_encoding() para uma descrição e exemplos.

`mbstring.regex_retry_limit` int

Limita a quantidade de retrocesso que pode ser executada durante uma correspondência de mbregex.

Esta configuração só tem efeito ao vincular contra oniguruma >= 6.8.0.

`mbstring.regex_stack_limit` int

Limita a profundidade da pilha de expressões regulares mbstring.

De acordo com a » especificação HTML 4.01, os navegadores podem codificar um formulário sendo enviado com uma codificação de caracteres diferente da usada para a página. Veja mb_http_input() para detectar a codificação de caracteres usada pelos navegadores.

Embora os navegadores populares sejam capazes de fazer uma suposição razoavelmente precisa sobre a codificação de caracteres de um documento HTML específico, seria melhor definir o parâmetro `charset` no cabeçalho HTTP `Content-Type` para o valor apropriado usando a função header() ou a configuração ini default_charset.

Exemplo #1 Exemplos de configuração do *php.ini*

```
; Define idioma padrão
mbstring.language        = Neutral; Define o idioma padrão como neutro (UTF-8) (padrão)
mbstring.language        = English; Define o idioma padrão para inglês
mbstring.language        = Japanese; Define o idioma padrão para japonês

;; Define a codificação interna padrão
;; Nota: Certifique-se de usar a codificação de caracteres que funciona com PHP
mbstring.internal_encoding    = UTF-8  ; Define a codificação interna para UTF-8

;; A tradução da codificação de entrada HTTP está habilitada.
mbstring.encoding_translation = On

;; Define a codificação de caracteres de entrada HTTP padrão
;; Nota: O script não pode alterar a configuração http_input.
mbstring.http_input           = pass    ; Sem conversão.
mbstring.http_input           = auto    ; Define a entrada HTTP como automática
                                ; "auto" é expandido de acordo com mbstring.language
mbstring.http_input           = SJIS    ; Define a entrada HTTP para SJIS
mbstring.http_input           = UTF-8,SJIS,EUC-JP ; Especifica a ordenação

;; Define a codificação de caracteres de saída HTTP padrão
mbstring.http_output          = pass    ; Sem conversão
mbstring.http_output          = UTF-8   ; Define a codificação de saída HTTP para UTF-8

;; Define a ordem de detecção de codificação de caracteres padrão
mbstring.detect_order         = auto    ; Define a ordem de detecção como automática
mbstring.detect_order         = ASCII,JIS,UTF-8,SJIS,EUC-JP ; Especifica o pedido

;; Define o caractere substituto padrão
mbstring.substitute_character = 12307   ; Especifica o valor Unicode
mbstring.substitute_character = none    ; Não imprime caractere
mbstring.substitute_character = long    ; Exemplo longo: U+3000,JIS+7E7E
```

Exemplo #2 *php.ini* configuração para usuários `EUC-JP`

```
;; Desativa buffer de saída
output_buffering      = Off

;; Define conjunto de caracteres do cabeçalho HTTP
default_charset       = EUC-JP
```

```
;; Define o idioma padrão para japonês
mbstring.language = Japanese

;; A tradução da codificação de entrada HTTP está habilitada
mbstring.encoding_translation = On

;; Define a conversão de codificação de entrada HTTP como automática
mbstring.http_input   = auto

;; Converte saída HTTP para EUC-JP
mbstring.http_output  = EUC-JP

;; Define a codificação interna para EUC-JP
mbstring.internal_encoding = EUC-JP

;; Não imprime caracteres inválidos
mbstring.substitute_character = none
```

Exemplo #3 *php.ini* configuração para usuários `SJIS`

```
;; Habilita buffer de saída
output_buffering     = On

;; Define mb_output_handler para ativar a conversão de saída
output_handler       = mb_output_handler

;; Define o conjunto de caracteres do cabeçalho HTTP
default_charset      = Shift_JIS

;; Define o idioma padrão para japonês
mbstring.language = Japanese

;; Define a conversão de codificação de entrada http como automática
mbstring.http_input  = auto

;; Converte para SJIS
mbstring.http_output = SJIS

;; Define a codificação interna para EUC-JP
mbstring.internal_encoding = EUC-JP

;; Não imprime caracteres inválidos
mbstring.substitute_character = none
```

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

`MB_OVERLOAD_MAIL` (int)

Removido a partir do PHP 8.0.0

`MB_OVERLOAD_STRING` (int)

Removido a partir do PHP 8.0.0

`MB_OVERLOAD_REGEX` (int)

Removido a partir do PHP 8.0.0

`MB_CASE_UPPER` (int)

Realiza uma dobragem completa em maiúsculas. Isso pode alterar o comprimento da string. Este é o modo usado por mb_strtoupper().

`MB_CASE_LOWER` (int)

Realiza uma dobragem completa em minúsculas. Isso pode alterar o comprimento da string. Este é o modo usado por mb_strtolower().

`MB_CASE_TITLE` (int)

Realiza uma conversão completa de título com base nas propriedades Unicode derivadas de Cased e CaseIgnorable. Em particular, isso melhora o manuseio de aspas e apóstrofos. Isso pode alterar o comprimento da string.

`MB_CASE_FOLD` (int)

Realiza uma conversão completa de dobragem de caso que remove distinções de caso presentes na string. Isso é usado para correspondência sem distinção de maiúsculas e minúsculas. Isso pode alterar o comprimento da string. Disponível desde o PHP 7.3.

`MB_CASE_LOWER_SIMPLE` (int)

Realiza uma conversão simples de dobragem em minúsculas. Isso não altera o comprimento da string. Disponível desde o PHP 7.3.

`MB_CASE_UPPER_SIMPLE` (int)

Realiza uma conversão simples de dobragem em maiúsculas. Isso não altera o comprimento da string. Disponível desde o PHP 7.3.

`MB_CASE_TITLE_SIMPLE` (int)

Realiza uma conversão simples de dobragem de título. Isso não altera o comprimento da string. Disponível desde o PHP 7.3.

`MB_CASE_FOLD_SIMPLE` (int)

Realiza uma conversão simples de dobragem de caso que remove distinções de caso presentes na string. Isso é usado para correspondência sem distinção de maiúsculas e minúsculas. Isso não altera o comprimento da string. Usado internamente pela extensão MBString em operações insensíveis a maiúsculas e minúsculas. Disponível desde o PHP 7.3.

`MB_ONIGURUMA_VERSION` (string)

A versão do Oniguruma, por exemplo, `6.9.4`. Disponível desde o PHP 7.4.

Sumários de Codificações Suportadas

Sumários de Codificações Suportadas

Nome no registro de conjunto de caracteres da IANA	Conjunto de caracteres subjacente	Descrição	Nota Adicional
ISO-10646-UCS-4	ISO 10646	O Conjunto de Caracteres Universal com espaço de código de 31 bits, padronizado como UCS-4 pela ISO/IEC 10646. Mantido sincronizado com a versão mais recente do mapa de códigos Unicode.	Se este nome for usado na facilidade de conversão de codificação, o conversor tenta identificar pelo BOM precedente (marcador de ordem de byte) em qual endian os bytes subsequentes estão representados.
ISO-10646-UCS-4	UCS-4	Ver acima.	Ao contrário do `UCS-4`, as strings são sempre assumidas estar na forma big endian.
ISO-10646-UCS-4	UCS-4	Ver acima.	Ao contrário do `UCS-4`, as strings são sempre assumidas estar na forma little endian.
ISO-10646-UCS-2	UCS-2	O Conjunto de Caracteres Universal com espaço de código de 16 bits, padronizado como UCS-2 pela ISO/IEC 10646. Mantido sincronizado com a versão mais recente do mapa de códigos Unicode.	Se este nome for usado na facilidade de conversão de codificação, o conversor tenta identificar pelo BOM precedente (marcador

Nome no registro de conjunto de caracteres da IANA	Conjunto de caracteres subjacente	Descrição	Nota Adicional
			de ordem de byte) em qual endian os bytes subsequentes estão representados.
ISO-10646-UCS-2	UCS-2	Ver acima.	Ao contrário do UCS-2, as strings são sempre assumidas estar na forma big endian.
ISO-10646-UCS-2	UCS-2	Ver acima.	Ao contrário do UCS-2, as strings são sempre assumidas estar na forma big endian.
UTF-32	Unicode	Formato de Transformação Unicode de largura de unidade de 32 bits, cujo espaço de codificação refere-se ao padrão de conjunto de códigos Unicode. Este esquema de codificação não era idêntico ao UCS-4 porque o espaço de código do Unicode era limitado a um valor de 21 bits.	Se este nome for usado na facilidade de conversão de codificação, o conversor tenta identificar pelo BOM precedente (marcador de ordem de byte) em qual endian os bytes subsequentes estão representados.
UTF-32BE	Unicode	Ver acima	Ao contrário do UTF-32, as strings são sempre assumidas estar na forma big endian.
UTF-32LE	Unicode	Ver acima	Ao contrário do UTF-32, as strings são sempre assumidas estar na forma little endian.
UTF-16	Unicode	Formato de Transformação Unicode de largura de unidade de 16 bits. Vale a pena observar que o UTF-16 não é mais a mesma especificação que UCS-2 porque o mecanismo de substituição foi introduzido desde o Unicode 2.0 e UTF-16 agora se refere a um espaço de código de 21 bits.	Se este nome for usado na facilidade de conversão de codificação, o conversor tenta identificar pelo BOM precedente (marcador de ordem de byte) em qual endian os bytes subsequentes estão representados.
UTF-16BE	Unicode	Ver acima.	Ao contrário do UTF-16, as strings são sempre assumidas estar na forma big endian.
UTF-16LE	Unicode	Ver acima.	Ao contrário do UTF-16, as strings são sempre assumidas estar na forma little endian.
UTF-8	Unicode / UCS	Formato de Transformação Unicode de largura de unidade de 8 bits.	none
UTF-7	Unicode	Um formato de transformação seguro para e-mails do Unicode, especificado em » RFC2152.	none

Nome no registro de conjunto de caracteres da IANA	Conjunto de caracteres subjacente	Descrição	Nota Adicional
(none)	Unicode	Uma variante de UTF-7 especializada para uso no » protocolo IMAP.	none
US-ASCII (preferred MIME name) / iso-ir-6 / ANSI_X3.4-1986 / ISO_646.irv:1991 / ASCII / ISO646-US / us / IBM367 / CP367 / csASCII	ASCII / ISO 646	Código Padrão Americano para Intercâmbio de Informações é uma codificação comumente usada de 7 bits. Também padronizado como um padrão internacional, ISO 646.	(none)
EUC-JP (preferred MIME name) / Extended_UNIX_Code_Packed_Format_for_Japanese / csEUCPkdFmtJapanese	Compound of US-ASCII / JIS X0201:1997 (hankaku kana part) / JIS X0208:1990 / JIS X0212:1990	Como pode ser visto, o nome é derivado de uma abreviação de Formato Padrão de Código UNIX Packed para Japonês, esta codificação é principalmente usada em UNIX ou plataformas semelhantes. O esquema de codificação original, Extended UNIX Code, é projetado com base no ISO 2022.	O conjunto de caracteres referido pelo EUC-JP é diferente do IBM932 / CP932, que são usados pelo OS/2® e Microsoft® Windows®. Para a troca de informações com essas plataformas, use EUCJP-WIN em vez disso.
Shift_JIS (preferred MIME name) / MS_Kanji / csShift_JIS	Compound of JIS X0201:1997 / JIS X0208:1997	Shift_JIS foi desenvolvido no início dos anos 80, na época dos processadores de palavras japoneses pessoais foram lançados no mercado, a fim de manter compatibilidades com o esquema de codificação legado JIS X 0201:1976. De acordo com a definição da IANA, o conjunto de códigos do Shift_JIS é ligeiramente diferente de IBM932 / CP932. No entanto, os nomes "SJIS" / "Shift_JIS" são frequentemente usados erroneamente para se referir a esses conjuntos de códigos.	Para o mapa de código CP932, use SJIS-WIN em vez disso.
(nenhum)	Composto de JIS X0201:1997 / JIS X0208:1997 / extensões IBM / extensões NEC	Embora esta "codificação" use o mesmo esquema de codificação que EUC-JP, o conjunto de caracteres subjacente é diferente. Ou seja, alguns pontos de código mapeiam para caracteres diferentes do EUC-JP.	nenhum
Windows-31J / csWindows31J	Composto de JIS X0201:1997 / JIS X0208:1997 / extensões IBM / extensões NEC	Embora esta "codificação" use o mesmo esquema de codificação que Shift_JIS, o conjunto de caracteres subjacente é diferente. Isso significa que alguns códigos pontos mapeiam para caracteres diferentes do Shift_JIS.	(none)
ISO-2022-JP (preferred MIME name) / csISO2022JP	US-ASCII / JIS X0201:1976 / JIS X0208:1978 / JIS X0208:1983	» RFC1468	(nenhum)
JIS			

Nome no registro de conjunto de caracteres da IANA	Conjunto de caracteres subjacente	Descrição	Nota Adicional
ISO-8859-1			
ISO-8859-2			
ISO-8859-3			
ISO-8859-4			
ISO-8859-5			
ISO-8859-6			
ISO-8859-7			
ISO-8859-8			
ISO-8859-9			
ISO-8859-10			
ISO-8859-13			
ISO-8859-14			
ISO-8859-15			
ISO-8859-16			
byte2be			
byte2le			
byte4be			
byte4le			
BASE64			
HTML-ENTITIES			
7bit			
8bit			
EUC-CN			
CP936			
HZ			
EUC-TW			
CP950			
BIG-5			
EUC-KR			
UHC (CP949)			
ISO-2022-KR			
Windows-1251 (CP1251)			
Windows-1252 (CP1252)			
CP866 (IBM866)			
KOI8-R			
KOI8-U			

Noções Básicas de Codificações Multibyte Japonesas

Os caracteres japoneses só podem ser representados por codificações multibyte, e vários padrões de codificação são usados dependendo da plataforma e da finalidade do texto. Para piorar a situação, esses padrões de codificação diferem um pouco um do outro. Para criar uma aplicação web que possa ser usada em um ambiente japonês, o desenvolvedor precisa ter em mente essas complexidades para garantir que as codificações de caracteres adequadas sejam usadas.

- O armazenamento de um caractere pode ter até seis bytes.

- A maioria dos caracteres multibyte japoneses aparece com o dobro da largura dos caracteres de byte único. Esses caracteres são chamados "zen-kaku" em japonês, que significa "largura total". Outros caracteres mais estreitos, são chamados "han-kaku", que significa "meia largura". As propriedades gráficas dos caracteres, no entanto, dependem das fontes usadas para exibi-las.
- Algumas codificações de caracteres usam sequências de deslocamento (escape) definidas na ISO-2022 para alternar o mapa de código da área de código específica (`00h` a `7fh`).
- A ISO-2022-JP deve ser usada no SMTP/NNTP, e cabeçalhos e entidades devem ser recodificados de acordo com os requisitos da RFC. Embora esses não sejam requisitos, ainda é uma boa ideia porque vários agentes de usuário populares não podem reconhecer nenhum outro método de codificação.
- As páginas web criadas para serviços de telefonia móvel, como » i-mode ou » EZweb devem usar Shift_JIS.
- Emoji usados em serviços de telefonia móvel, como » i-mode ou » EZweb são suportados.

Entrada e Saída HTTP

A conversão de codificação de caracteres de entrada/saída HTTP pode converter dados binários também. Os usuários devem controlar a conversão de codificação de caracteres se dados binários forem usados para entrada/saída HTTP.

> **Nota** :
>
> Se `enctype` para o formulário HTML estiver configurado como `multipart/form-data` e `mbstring.encoding_translation` estiver ativado no *php.ini*, as variáveis POST'ed e os nomes dos arquivos enviados serão convertidos para a codificação de caracteres interna também. No entanto, a conversão não é aplicada às chaves de consulta.

- Entrada HTTP

  Não há maneira de controlar a conversão de caracteres de entrada HTTP a partir de um script PHP. Para desativar a conversão de caracteres de entrada HTTP, isso deve ser feito no *php.ini*.

  Exemplo #1 Desativando a conversão de entrada HTTP no *php.ini*

  ```
  ;; Disable HTTP Input conversion
  mbstring.http_input = pass
  ;; Disable HTTP Input conversion
  mbstring.encoding_translation = Off
  ```

  Ao usar o PHP como um módulo do Apache, é possível substituir essas configurações em cada diretiva Virtual Host no *httpd.conf* ou por diretório com *.htaccess*. Consulte a seção Configuração e o Manual do Apache para obter detalhes.

- Saída HTTP

  Existem várias maneiras de habilitar a conversão de codificação de caracteres de saída. Uma é usando o *php.ini*, outra é usando ob_start() com mb_output_handler() como a `ob_start` função de retorno de chamada.

Exemplo #2 Exemplo de configuração no *php.ini*

```
;; Enable output character encoding conversion for all PHP pages

;; Enable Output Buffering
output_buffering    = On

;; Set mb_output_handler to enable output conversion
output_handler      = mb_output_handler
```

Exemplo #3 Exemplo de script

```
<?php

// Habilitar a conversão de codificação de caracteres de saída apenas para esta página

// Definir a codificação de caracteres de saída HTTP para SJIS
mb_http_output('SJIS');

// Iniciar o buffer e especificar "mb_output_handler" como
```

```
// função de callback
ob_start('mb_output_handler');

?>
```

Codificações de Caracteres Suportadas

Atualmente, as seguintes codificações de caracteres são suportadas pelo módulo `mbstring`. Qualquer uma dessas codificações pode ser especificada no parâmetro `encoding` das funções `mbstring`.

As seguintes codificações de caracteres são suportadas nesta extensão PHP:

- UCS-4*
- UCS-4BE
- UCS-4LE*
- UCS-2
- UCS-2BE
- UCS-2LE
- UTF-32*
- UTF-32BE*
- UTF-32LE*
- UTF-16*
- UTF-16BE*
- UTF-16LE*
- UTF-7
- UTF7-IMAP
- UTF-8*
- ASCII*
- EUC-JP*
- SJIS*
- eucJP-win*
- SJIS-win*
- ISO-2022-JP
- ISO-2022-JP-MS
- CP932
- CP51932
- SJIS-mac (alias: MacJapanese)
- SJIS-Mobile#DOCOMO (alias: SJIS-DOCOMO)
- SJIS-Mobile#KDDI (alias: SJIS-KDDI)
- SJIS-Mobile#SOFTBANK (alias: SJIS-SOFTBANK)
- UTF-8-Mobile#DOCOMO (alias: UTF-8-DOCOMO)
- UTF-8-Mobile#KDDI-A

- UTF-8-Mobile#KDDI-B (alias: UTF-8-KDDI)
- UTF-8-Mobile#SOFTBANK (alias: UTF-8-SOFTBANK)
- ISO-2022-JP-MOBILE#KDDI (alias: ISO-2022-JP-KDDI)
- JIS
- JIS-ms
- CP50220
- CP50220raw
- CP50221
- CP50222
- ISO-8859-1*
- ISO-8859-2*
- ISO-8859-3*
- ISO-8859-4*
- ISO-8859-5*
- ISO-8859-6*
- ISO-8859-7*
- ISO-8859-8*
- ISO-8859-9*
- ISO-8859-10*
- ISO-8859-13*
- ISO-8859-14*
- ISO-8859-15*
- ISO-8859-16*
- byte2be
- byte2le
- byte4be
- byte4le
- BASE64
- HTML-ENTITIES (alias: HTML)
- 7bit
- 8bit
- EUC-CN*
- CP936
- GB18030
- HZ
- EUC-TW*
- CP950

- BIG-5*
- EUC-KR*
- UHC (alias: CP949)
- ISO-2022-KR
- Windows-1251 (alias: CP1251)
- Windows-1252 (alias: CP1252)
- CP866 (alias: IBM866)
- KOI8-R*
- KOI8-U*
- ArmSCII-8 (alias: ArmSCII8)

* denota codificações também utilizáveis em expressões regulares.

Qualquer entrada *php.ini* que aceite um nome de codificação também pode usar os valores "`auto`" e "`pass`". Funções `mbstring` que aceitam um nome de codificação também podem usar o valor "`auto`".

Se "`pass`" estiver definido, nenhuma conversão de codificação de caracteres será realizada.

Se "`auto`" estiver definido, ele é expandido para a lista de codificações definidas de acordo com o NLS. Por exemplo, se o NLS estiver definido como `Japanese`, o valor é assumido como "`ASCII,JIS,UTF-8,EUC-JP,SJIS`".

Veja também mb_detect_order()

Recurso de Sobrecarga de Funções

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 7.2.0 e foi *REMOVIDO* a partir do PHP 8.0.0. O uso deste recurso é fortemente desencorajado.

É possível que frequentemente sejam encontradas dificuldades ao fazer com que uma aplicação PHP existente funcione em um ambiente multibyte específico. Isso acontece porque a maioria das aplicações PHP por aí são escritas com funções de string padrão como substr(), que são conhecidas por não lidar corretamente com strings codificadas em multibyte.

mbstring suporta um recurso de 'sobrecarga de funções' que permite adicionar suporte a multibyte a uma aplicação dessas sem modificar o código, sobrecarregando as contrapartes multibyte das funções de string padrão. Por exemplo, mb_substr() é chamada em vez de substr() se a sobrecarga de funções estiver ativada. Esse recurso facilita a migração de aplicações que suportam apenas codificações de byte único para um ambiente multibyte em muitos casos.

Para usar a sobrecarga de funções, defina `mbstring.func_overload` no *php.ini* para um valor positivo que represente uma combinação de máscaras de bits especificando as categorias de funções a serem sobrecarregadas. Deve ser definido como 1 para sobrecarregar a função mail(), 2 para funções de string, 4 para funções de expressão regular. Por exemplo, se for definido como 7, as funções de correio, strings e expressões regulares serão sobrecarregadas. A lista de funções sobrecarregadas é mostrada abaixo.

Funções a serem sobrecarregadas

valor de mbstring.func_overload	função original	função sobrecarregada
1	mail()	mb_send_mail()
2	strlen()	mb_strlen()
2	strpos()	mb_strpos()
2	strrpos()	mb_strrpos()
2	substr()	mb_substr()
2	strtolower()	mb_strtolower()
2	strtoupper()	mb_strtoupper()
2	stripos()	mb_stripos()

valor de mbstring.func_overload	função original	função sobrecarregada
2	strripos()	mb_strripos()
2	strstr()	mb_strstr()
2	stristr()	mb_stristr()
2	strrchr()	mb_strrchr()
2	substr_count()	mb_substr_count()

> **Nota** :
>
> Não é recomendado usar a opção de sobrecarga de funções no contexto por diretório, porque ainda não foi confirmado como sendo estável o suficiente em um ambiente de produção e pode levar a um comportamento indefinido.

Requisitos de Codificação de Caracteres em PHP

As codificações dos seguintes tipos são seguras para uso com o PHP.

- Uma codificação de byte único,
  - que possui mapeamentos compatíveis com ASCII (compatíveis com ISO646) para os caracteres no intervalo de `00h` a `7fh`.
- Uma codificação multibyte,
  - que possui mapeamentos compatíveis com ASCII para os caracteres no intervalo de `00h` a `7fh`.
  - que não usa sequências de escape ISO2022.
  - que não usa um valor de `00h` a `7fh` em nenhum dos bytes compostos que representam um único caractere.

Esses são exemplos de codificações de caracteres que provavelmente não funcionarão corretamente com o PHP.

```
JIS, SJIS, ISO-2022-JP, BIG-5
```

Embora scripts PHP escritos em qualquer uma dessas codificações possam não funcionar, especialmente no caso em que strings codificadas aparecem como identificadores ou literais no script, você pode evitar o uso dessas codificações configurando a função de filtro de codificação transparente do `mbstring` para consultas HTTP de entrada.

> **Nota** :
>
> É altamente desencorajado usar SJIS, BIG5, CP936, CP949 e GB18030 para a codificação interna, a menos que você esteja familiarizado com o analisador, o scanner e a codificação de caracteres.

> **Nota** :
>
> Se você estiver se conectando a um banco de dados com PHP, é recomendável que você use a mesma codificação de caracteres para tanto o banco de dados quanto o `codificação interna` para facilitar o uso e obter melhor desempenho.
>
> Se estiver usando o PostgreSQL, a codificação de caracteres usada no banco de dados e aquela usada no PHP podem diferir, pois ele oferece suporte à conversão automática de conjunto de caracteres entre o backend e o frontend.

Strings Multibyte e Funções

Referências

Esquemas de codificação de caracteres multibyte e seus problemas relacionados são bastante complicados e estão além do escopo desta documentação. Consulte as seguintes URLs e outros recursos para obter mais informações sobre esses tópicos.

- Materiais Unicode

  » http://www.unicode.org/

- Informações sobre caracteres Japoneses/Coreanos/Chineses

  » https://resources.oreilly.com/examples/9781565922242/blob/master/doc/cjk.inf

Índice

mb_check_encoding

mb_check_encoding

(PHP 4 >= 4.4.3, PHP 5 >= 5.1.3, PHP 7, PHP 8)

mb_check_encoding — Verifica se as strings são válidas para a codificação especificada

Descrição

function **mb_check_encoding**(array|string|null `$value = null`, ?string `$encoding = null`): bool

Verifica se o fluxo de bytes especificado é válido para a codificação especificada. Se `value` for do tipo array, todas as chaves e valores são validados recursivamente. É útil para prevenir ataques chamados "Invalid Encoding Attack".

Parâmetros

`value`

O fluxo de bytes ou array a ser verificado. Se omitido, esta função verifica toda a entrada desde o início da requisição.

> **Aviso**
>
> A partir do PHP 8.1.0, omitir este parâmetro ou passar `null` está descontinuado.

`encoding`

A codificação esperada.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar esta função com `null` como `value` ou sem argumento está descontinuado.
8.0.0	`value` e `encoding` são anuláveis agora.
7.2.0	Esta função agora aceita também um array como `value`. Anteriormente, apenas strings eram suportadas.

mb_chr

mb_chr

(PHP 7 >= 7.2.0, PHP 8)

mb_chr — Retorna um caractere pelo valor do ponto de código Unicode

Descrição

function **mb_chr**(int `$codepoint`, ?string `$encoding = null`): string|false

Retorna uma string contendo o caractere especificado pelo valor do ponto de código Unicode, codificado na codificação especificada.

Esta função complementa mb_ord().

Parâmetros

`codepoint`

Um valor de ponto de código Unicode, por exemplo, `128024` para *U+1F418 ELEPHANT*

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Uma string contendo o caractere solicitado, se puder ser representado na codificação especificada ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Testando diferentes pontos de código

```
<?php
$values = [65, 63, 0x20AC, 128024];
foreach ($values as $value) {
    var_dump(mb_chr($value, 'UTF-8'));
    var_dump(mb_chr($value, 'ISO-8859-1'));
}
?>
```

O exemplo acima produzirá:

```
string(1) "A"
string(1) "A"
string(1) "?"
string(1) "?"
string(3) "€"
bool(false)
string(4) "🐘"
bool(false)
```

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_ord() - Obtém o ponto de código Unicode do caractere
- IntlChar::ord() - Retorna o valor do ponto de código Unicode do caractere
- chr() - Gera uma string de um byte a partir de um número

mb_convert_case

mb_convert_case

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_convert_case — Realiza a conversão de caixa em uma string

Descrição

function **mb_convert_case**(string `$string`, int `$mode`, ?string `$encoding` = `null`): string

Realiza a conversão de caixa em uma string, convertida da maneira especificada por `mode`.

Parâmetros

`string`

A string sendo convertida.

`mode`

O modo de conversão. Pode ser um dos `MB_CASE_UPPER`, `MB_CASE_LOWER`, `MB_CASE_TITLE`, `MB_CASE_FOLD`, `MB_CASE_UPPER_SIMPLE`, `MB_CASE_LOWER_SIMPLE`, `MB_CASE_TITLE_SIMPLE`, `MB_CASE_FOLD_SIMPLE`.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Uma versão da `string` com conversão de caixa conforme especificado por `mode`.

Registro de Alterações

Versão	Descrição
8.3.0	Implementadas regras condicionais de maiúsculas/minúsculas para a letra grega sigma que só se aplicam aos modos `MB_CASE_LOWER` e `MB_CASE_TITLE`, e não a `MB_CASE_LOWER_SIMPLE` e `MB_CASE_TITLE_SIMPLE`.
7.3.0	Adicionado suporte para `MB_CASE_FOLD`, `MB_CASE_UPPER_SIMPLE`, `MB_CASE_LOWER_SIMPLE`, `MB_CASE_TITLE_SIMPLE`, e `MB_CASE_FOLD_SIMPLE` como `mode`.

Examples

Exemplo #1 Exemplo de mb_convert_case()

```php
<?php
$str = "mary had a Little lamb and she loved it so";
$str = mb_convert_case($str, MB_CASE_UPPER, "UTF-8");
echo $str, PHP_EOL;
$str = mb_convert_case($str, MB_CASE_TITLE, "UTF-8");
echo $str, PHP_EOL;
?>
```

Exemplo #2 Exemplo de mb_convert_case() com texto UTF-8 não latino

```php
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_convert_case($str, MB_CASE_UPPER, "UTF-8");
echo $str, PHP_EOL;
$str = mb_convert_case($str, MB_CASE_TITLE, "UTF-8");
echo $str, PHP_EOL;
?>
```

Notas

Ao contrário das funções padrão de conversão de caixa, como strtolower() e strtoupper(), a conversão de caixa é realizada com base nas propriedades dos caracteres Unicode. Assim, o comportamento desta função não é afetado pelas configurações de localidade e ela pode converter qualquer caractere que tenha a propriedade 'alfabética', como o a-umlaut (ä).

Para obter mais informações sobre as propriedades Unicode, consulte » http://www.unicode.org/reports/tr21/.

Veja também

- mb_strtolower() - Torna uma string minúscula
- mb_strtoupper() - Faz uma string ficar em maiúsculas
- strtolower() - Converte uma string para minúsculas
- strtoupper() - Converte uma string em maiúsculas
- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo
- ucwords() - Converte para maiúsculas o primeiro caractere de cada palavra

mb_convert_encoding

mb_convert_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_encoding — Converte uma string de uma codificação de caracteres para outra

Descrição

function **mb_convert_encoding**(array|string `$string`, string `$to_encoding`, array|string|null `$from_encoding` = **`null`**): array|string|false

Converte `string` de `from_encoding`, ou a codificação interna atual, para `to_encoding`. Se `string` for um array, todos os seus valores string serão convertidos recursivamente.

Parâmetros

`string`

A string ou array a ser convertida.

`to_encoding`

A codificação desejada para o resultado.

`from_encoding`

A codificação atual usada para interpretar `string`. Múltiplas codificações podem ser especificadas como um array ou uma lista separada por vírgulas, nesse caso, a codificação correta será detectada usando o mesmo algoritmo que mb_detect_encoding().

Se `from_encoding` for omitida ou **`null`**, a configuração mbstring.internal_encoding será usada se definida, caso contrário, a configuração default_charset.

Consulte codificações suportadas para os valores válidos de `to_encoding` e `from_encoding`.

Valor Retornado

A string ou array codificada em caso de sucesso, ou **`false`** em caso de falha.

Erros/Exceções

A partir do PHP 8.0.0, um ValueError é lançado se o valor de `to_encoding` ou `from_encoding` for uma codificação inválida. Antes do PHP 8.0.0, um **`E_WARNING`** era emitido.

Registro de Alterações

Versão	Descrição
8.2.0	**mb_convert_encoding()** não retornará mais as seguintes codificações não textuais: `"Base64"`, `"QPrint"`, `"UUencode"`, `"HTML entities"`, `"7 bit"` e `"8 bit"`.
8.0.0	**mb_convert_encoding()** agora lançará um ValueError quando `to_encoding` receber uma codificação inválida.
8.0.0	**mb_convert_encoding()** agora lançará um ValueError quando `from_encoding` receber uma codificação inválida.
8.0.0	`from_encoding` agora é anulável.
7.2.0	Esta função agora aceita um array como `string`. Anteriormente, apenas strings eram suportadas.

Examples

Exemplo #1 Exemplo de **mb_convert_encoding()**

```php
<?php
/* Converte a codificação interna para SJIS */
$str = mb_convert_encoding($str, "SJIS");

/* Converte EUC-JP para UTF-7 */
$str = mb_convert_encoding($str, "UTF-7", "EUC-JP");

/* Detecta automaticamente a codificação a partir de JIS, eucjp-win, sjis-win, e converte str para UCS-2LE */
$str = mb_convert_encoding($str, "UCS-2LE", "JIS, eucjp-win, sjis-win");

/* Se mbstring.language for "Japanese", "auto" é expandido para "ASCII,JIS,UTF-8,EUC-JP,SJIS" */
$str = mb_convert_encoding($str, "EUC-JP", "auto");
?>
```

Veja também

- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres
- UConverter::transcode() - Converte uma string de uma codificação de caracteres para outra
- iconv() - Converte uma string de uma codificação de caracteres para outra

mb_convert_kana

mb_convert_kana

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_kana — Converte "kana" de um tipo para outro ("zen-kaku", "han-kaku" e mais)

Descrição

function **mb_convert_kana**(string `$string`, string `$mode` = "KV", ?string `$encoding` = **`null`**): string

Realiza uma conversão "han-kaku" - "zen-kaku" para a string `string`. Esta função é útil apenas para o japonês.

Parâmetros

`string`

A string a ser convertida.

`mode`

A opção de conversão.

Especifique com uma combinação das seguintes opções.

Opções de Conversão Aplicáveis

Opção	Significado
`r`	Converte alfabetos "zen-kaku" para "han-kaku"
`R`	Converte alfabetos "han-kaku" para "zen-kaku"
`n`	Converte números "zen-kaku" para "han-kaku"
`N`	Converte números "han-kaku" para "zen-kaku"
`a`	Converte alfabetos e números "zen-kaku" para "han-kaku"
`A`	Converte alfabetos e números "han-kaku" para "zen-kaku" (Caracteres incluídos nas opções "a", "A" são U+0021 - U+007E, excluindo U+0022, U+0027, U+005C, U+007E)
`s`	Converte espaço "zen-kaku" para "han-kaku" (U+3000 -> U+0020)
`S`	Converte espaço "han-kaku" para "zen-kaku" (U+0020 -> U+3000)
`k`	Converte "zen-kaku kata-kana" para "han-kaku kata-kana"
`K`	Converte "han-kaku kata-kana" para "zen-kaku kata-kana"
`h`	Converte "zen-kaku hira-gana" para "han-kaku kata-kana"
`H`	Converte "han-kaku kata-kana" para "zen-kaku hira-gana"
`c`	Converte "zen-kaku kata-kana" para "zen-kaku hira-gana"
`C`	Converte "zen-kaku hira-gana" para "zen-kaku kata-kana"
`V`	Colapsa a notação de som sonoro e converte-os em um caractere. Use com "K","H"

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou **`null`**, o valor da codificação de caracteres interna será usado.

Valor Retornado

A string convertida.

Erros/Exceções

Gera um ValueError se a combinação de diferentes `modes` for inválida. Por exemplo, `"sS"`.

Registro de Alterações

Versão	Descrição
8.2.0	Um ValueError agora é gerado se a combinação de diferentes `modes` for inválida.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de mb_convert_kana()

```php
<?php
/* Converte todos "han-kaku" "kata-kana" para "zen-kaku" "hira-gana" */
echo mb_convert_kana('ﾔﾏﾀﾞ ﾊﾅｺ', "HV") . "\n";

/* Converte "han-kaku" "kata-kana" para "zen-kaku" "kata-kana"
   e alfanuméricos "zen-kaku" para "han-kaku" */
echo mb_convert_kana('ｺｳｻﾞﾊﾞﾝｺﾞｳ ０１２３４５６', "KVa") . "\n";
?>
```

O exemplo acima produzirá:

```
やまだ はなこ
コウザバンゴウ 0123456
```

mb_convert_variables

mb_convert_variables

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_variables — Converte o código de caracteres em variável(is)

Descrição

function **mb_convert_variables**(
    string `$to_encoding`,
    array|string `$from_encoding`,
    mixed `&$var`,
    mixed `&...$vars`
): string|false

Converte a codificação de caracteres das variáveis `var` e `vars` de codificação `from_encoding` para a codificação `to_encoding`.

mb_convert_variables() une strings em Arrays ou Objetos para detectar a codificação, já que a detecção de codificação tende a falhar para strings curtas. Portanto, é impossível misturar codificações em um único array ou objeto.

Parâmetros

`to_encoding`

A codificação para a qual a string está sendo convertida.

`from_encoding`

Se `from_encoding` for especificado como um array ou uma string separada por vírgula, ele tenta detectar a codificação de `from-coding`. Quando `from_encoding` é omitido, é usado `detect_order`.

`var`

`var` é a referência para a variável sendo convertida. String, Array e Object são aceitos. **mb_convert_variables()** assume que todos os parâmetros têm a mesma codificação.

`vars`

Variáveis adicionais `vars`.

Valor Retornado

A codificação de caracteres antes da conversão para sucesso, ou `false` para falha.

Examples

Exemplo #1 Exemplo de mb_convert_variables()

```php
<?php
/* Converte as variáveis $post1, $post2 para a codificação interna */
$interenc = mb_internal_encoding();
$inputenc = mb_convert_variables($interenc, "ASCII,UTF-8,SJIS-win", $post1, $post2);
?>
```

mb_decode_mimeheader

mb_decode_mimeheader

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_decode_mimeheader — Decodifica string no campo de cabeçalho MIME

Descrição

function **mb_decode_mimeheader**(string `$string`): string

Decodifica a string codificada `string` no cabeçalho MIME.

Parâmetros

`string`

A string sendo decodificada.

Valor Retornado

A string decodificada na codificação interna de caracteres.

Registro de Alterações

Versão	Descrição
8.3.0	Caracteres de sublinhado (_) são convertidos para espaços conforme especificado na » RFC 2047.

Veja também

- mb_encode_mimeheader() - Codificar string para cabeçalho MIME

mb_decode_numericentity

mb_decode_numericentity

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_decode_numericentity — Decodifica referência de string numérica HTML para caractere

Descrição

function **mb_decode_numericentity**(string `$string`, array `$map`, ?string `$encoding` = **`null`**): string

Converte a referência de string numérica de string `string`, em um bloco especificado, para caractere.

Parâmetros

`string`

A string sendo decodificada.

`map`

`map` é um array que especifica a área de código a ser convertida.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou **`null`**, o valor da codificação de caracteres interna será usado.

Valor Retornado

A string convertida.

Erros/Exceções

Lança uma exceção ValueError se `map` não for uma lista de ints.

Registro de Alterações

Versão	Descrição
8.4.0	**mb_decode_numericentity()** agora lança uma exceção ValueError se `map` não for uma lista de ints.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de `map`

```
<?php
$convmap = array (
   int start_code1, int end_code1, int offset1, int mask1,
   int start_code2, int end_code2, int offset2, int mask2,
   //  ........
   int start_codeN, int end_codeN, int offsetN, int maskN );
// Especifique o valor Unicode para start_codeN e end_codeN
// Adicione offsetN ao valor e faça 'AND' bit a bit com maskN,
// em seguida, converta o valor para referência de string numérica.
?>
```

Exemplo #2 Exemplo de `map` escapando string JavaScript

```
<?php
function escape_javascript_string($str) {
  $map = [
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,0,0, // 49
          0,0,0,0,0,0,0,0,1,1,
          1,1,1,1,1,0,0,0,0,0,
          0,0,0,0,0,0,0,0,0,0,
          0,0,0,0,0,0,0,0,0,0,
          0,1,1,1,1,1,1,0,0,0, // 99
          0,0,0,0,0,0,0,0,0,0,
          0,0,0,0,0,0,0,0,0,0,
          0,0,0,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1, // 149
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1, // 199
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1, // 249
          1,1,1,1,1,1,1, // 255
          ];
  // Codificação de caractere é UTF-8
  $mblen = mb_strlen($str, 'UTF-8');
  $utf32 = bin2hex(mb_convert_encoding($str, 'UTF-32', 'UTF-8'));
  for ($i=0, $encoded=''; $i < $mblen; $i++) {
      $u = substr($utf32, $i*8, 8);
      $v = base_convert($u, 16, 10);
      if ($v < 256 && $map[$v]) {
        $encoded .= '\\x'.substr($u, 6,2);
      } else if ($v == 2028) {
```

```
        $encoded .= '\\u2028';
      } else if ($v == 2029) {
        $encoded .= '\\u2029';
      } else {
        $encoded .= mb_convert_encoding(hex2bin($u), 'UTF-8', 'UTF-32');
      }
    }
    return $encoded;
}

// Dados de teste
$convmap = [ 0x0, 0xffff, 0, 0xffff ];
$msg = '';
for ($i=0; $i < 1000; $i++) {
  // chr() não pode gerar dados UTF-8 corretos com valor maior que 128, use mb_decode_numericentity().
  $msg .= mb_decode_numericentity('&#'.$i.';', $convmap, 'UTF-8');
}

// var_dump($msg);
var_dump(escape_javascript_string($msg));
```

Veja também

- mb_encode_numericentity() - Codificar caractere para referência numérica HTML

mb_detect_encoding

mb_detect_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_detect_encoding — Detectar a codificação de caracteres

Descrição

function **mb_detect_encoding**(string `$string`, array|string|null `$encodings` = `null`, bool `$strict` = `false`): string|false

Detecta a codificação de caracteres mais provável para a string `string` a partir de uma lista de candidatos.

A partir do PHP 8.1, esta função usa heurística para detectar qual das codificações de texto válidas na lista especificada tem maior probabilidade de estar correta e pode não estar na ordem das codificações fornecidas no parâmetro `encodings`.

A detecção automática da codificação de caracteres pretendida nunca pode ser totalmente confiável; sem informações adicionais, é semelhante a decodificar uma string criptografada sem a chave. É sempre preferível usar uma indicação de codificação de caracteres armazenada ou transmitida com os dados, como um cabeçalho HTTP "Content-Type".

Esta função é mais útil com codificações multibyte, onde nem todas as sequências de bytes formam uma string válida. Se a string de entrada contiver tal sequência, essa codificação será rejeitada.

> **Aviso**
>
> #### O resultado não é preciso
>
> O nome dessa função é enganoso: ela realiza "adivinhação" em vez de "detecção".
>
> Os palpites estão longe de ser precisos e, portanto, esta função não pode ser usada para detectar com precisão a codificação correta de caracteres.

Parâmetros

`string`

A string sendo inspecionada.

`encodings`

Uma lista de codificações de caracteres para tentar. A lista pode ser especificada como um array de strings ou uma única string separada por vírgulas.

Se `encodings` for omitido ou `null`, a detect_order atual (definida com a opção de configuração mbstring.detect_order, ou função mb_detect_order()) será usada.

`strict`

Controla o comportamento quando `string` não é válida em nenhuma das `encodings` listadas. Se `strict` for definido como `false`, a codificação mais próxima correspondente será retornada; se `strict` for definido como `true`, `false` será retornado.

O valor padrão para `strict` pode ser definido com a opção de configuração mbstring.strict_detection.

Valor Retornado

A codificação de caracteres detectada ou `false` se a string não for válida em nenhuma das codificações listadas.

Registro de Alterações

Versão	Descrição
8.2.0	**mb_detect_encoding()** não retornará mais as seguintes codificações não textuais: `"Base64"`, `"QPrint"`, `"UUencode"`, `"HTML entities"`, `"7 bit"` e `"8 bit"`.

Examples

Exemplo #1 Exemplo de mb_detect_encoding()

```php
<?php

$str = "\x95\xB6\x8E\x9A\x83\x52\x81\x5B\x83\x68";

// Detecta codificação de caracteres com a ordem de detecção atual
var_dump(mb_detect_encoding($str));

// "auto" é expandido de acordo com mbstring.language
var_dump(mb_detect_encoding($str, "auto"));

// Especifica parâmetro "encodings" com lista separada por vírgula
var_dump(mb_detect_encoding($str, "JIS, eucjp-win, sjis-win"));

// Usa array para especificar parâmetro "encodings"
$encodings = [
  "ASCII",
  "JIS",
  "EUC-JP"
];
var_dump(mb_detect_encoding($str, $encodings));
?>
```

O exemplo acima produzirá:

```
string(5) "ASCII"
string(5) "ASCII"
string(8) "SJIS-win"
string(5) "ASCII"
```

Exemplo #2 Efeito do parâmetro `strict`

```php
<?php
// 'áéóú' codificado em ISO-8859-1
$str = "\xE1\xE9\xF3\xFA";

// A string não é válida em ASCII ou UTF-8, mas UTF-8 é considerado uma correspondência mais próxima
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8'], false));
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8'], true));

// Se uma codificação válida for encontrada, o parâmetro strict não muda o resultado
```

```php
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8', 'ISO-8859-1'], false));
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8', 'ISO-8859-1'], true));
?>
```

O exemplo acima produzirá:

```
string(5) "UTF-8"
bool(false)
string(10) "ISO-8859-1"
string(10) "ISO-8859-1"
```

Em alguns casos, a mesma sequência de bytes pode formar uma string válida em várias codificações de caracteres, e é impossível saber qual interpretação foi pretendida. Por exemplo, entre muitos outros, a sequência de bytes "\xC4\xA2" poderia ser:

- "Ä¢" (U+00C4 LATIN CAPITAL LETTER A WITH DIAERESIS seguido por U+00A2 CENT SIGN) codificado em qualquer um de of ISO-8859-1, ISO-8859-15, ou Windows-1252
- "ФЂ" (U+0424 CYRILLIC CAPITAL LETTER EF seguido por U+0402 CYRILLIC CAPITAL LETTER DJE) codificado em ISO-8859-5
- "Ģ" (U+0122 LATIN CAPITAL LETTER G WITH CEDILLA) codificado em UTF-8

Exemplo #3 Efeito da ordem quando várias codificações correspondem

```php
<?php
$str = "\xC4\xA2";

// A string é válida em todas as três codificações, mas a primeira listada pode nem sempre ser a retornada
var_dump(mb_detect_encoding($str, ['UTF-8']));
var_dump(mb_detect_encoding($str, ['UTF-8', 'ISO-8859-1', 'ISO-8859-5'])); // a partir do PHP 8.1, retorna ISO-8859-1 em vez de UTF-8
var_dump(mb_detect_encoding($str, ['ISO-8859-1', 'ISO-8859-5', 'UTF-8']));
var_dump(mb_detect_encoding($str, ['ISO-8859-5', 'UTF-8', 'ISO-8859-1']));
?>
```

O exemplo acima produzirá:

```
string(5) "UTF-8"
string(10) "ISO-8859-1"
string(10) "ISO-8859-1"
string(10) "ISO-8859-5"
```

Veja também

- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres

mb_detect_order

mb_detect_order

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_detect_order — Define ou obtém a ordem de detecção de codificação de caracteres

Descrição

function **mb_detect_order**(array|string|null `$encoding = null`): array|bool

Define a ordem automática de detecção de codificação de caracteres para `encoding`.

Parâmetros

`encoding`

`encoding` é um array ou lista de codificações de caracteres separadas por vírgula. Veja codificações suportadas.

Se `encoding` for omitido ou `null`, ele retorna a ordem atual de detecção de codificação de caracteres como um array.

Essa configuração afeta mb_detect_encoding() e mb_send_mail().

`mbstring` implementa atualmente os seguintes filtros de detecção de codificação. Se houver uma sequência de bytes inválida para as seguintes codificações, a detecção de codificação falhará.

`UTF-8`, `UTF-7`, `ASCII`, `EUC-JP`,`SJIS`, `eucJP-win`, `SJIS-win`, `JIS`, `ISO-2022-JP`

Para `ISO-8859-*`, `mbstring` sempre detecta como `ISO-8859-*`.

Para `UTF-16`, `UTF-32`, `UCS2` e `UCS4`, a detecção de codificação sempre falhará.

Valor Retornado

Ao definir a ordem de detecção de codificação, `true` é retornado em caso de sucesso ou `false` em caso de falha.

Ao obter a ordem de detecção de codificação, um array ordenado das codificações é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplos de mb_detect_order()

```php
<?php
/* Define ordem de detecção por lista enumerada */
mb_detect_order("eucjp-win,sjis-win,UTF-8");

/* Define ordem de detecção por array */
$ary[] = "ASCII";
$ary[] = "JIS";
$ary[] = "EUC-JP";
mb_detect_order($ary);

/* Exibe ordem de detecção atual */
echo implode(", ", mb_detect_order());
?>
```

Exemplo #2 Exemplo mostrando ordens de detecção inúteis

```
; Sempre detecta como ISO-8859-1
detect_order = ISO-8859-1, UTF-8

; Sempre detecta como UTF-8, porque os valores ASCII/UTF-7 são
; válidos para UTF-8
detect_order = UTF-8, ASCII, UTF-7
```

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_http_input() - Detecta a codificação de caracteres de entrada HTTP
- mb_http_output() - Define/Obtém a codificação de caracteres de saída HTTP
- mb_send_mail() - Envia email codificado

mb_encode_mimeheader

mb_encode_mimeheader

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_encode_mimeheader — Codificar string para cabeçalho MIME

Descrição

function **mb_encode_mimeheader**(
  string $string,
  ?string $charset = null,
  ?string $transfer_encoding = null,
  string $newline = "\r\n",
  int $indent = 0
): string

Codifica uma string `string` pelo esquema de codificação de cabeçalho MIME.

Parâmetros

`string`

A string a ser codificada. Sua codificação deve ser a mesma que mb_internal_encoding().

`charset`

`charset` especifica o nome do conjunto de caracteres no qual `string` está representada. O valor padrão é determinado pela configuração NLS atual (`mbstring.language`).

`transfer_encoding`

`transfer_encoding` especifica o esquema de codificação MIME. Deve ser ou `"B"` (Base64) ou `"Q"` (Quoted-Printable). Retrocede para `"B"` se não fornecido.

`newline`

`newline` especifica o marcador de EOL (end-of-line) com o qual **mb_encode_mimeheader()** realiza a quebra de linha (um termo » RFC, o ato de quebrar uma linha mais longa que um certo comprimento em várias linhas. O comprimento é atualmente codificado como 74 caracteres). Retrocede para `"\r\n"` (CRLF) se não fornecido.

`indent`

Recuo da primeira linha (número de caracteres no cabeçalho antes de `string`).

Valor Retornado

Uma versão convertida da string representada em ASCII.

Registro de Alterações

Versão	Descrição
8.3.0	Bytes `NUL` (0) não são mais excluídos quando codificados através da codificação Quoted-Printable, mas são codificados como `=00`.
8.0.0	`charset` e `transfer_encoding` agora são anuláveis.

Examples

Exemplo #1 Exemplo de **mb_encode_mimeheader()**

```
<?php
$name = "太郎"; // kanji
$mbox = "kru";
$doma = "gtinn.mon";
$addr = '"' . addcslashes(mb_encode_mimeheader($name, "UTF-7", "Q"), '"') . '" <' . $mbox . "@" . $doma . ">";
echo $addr;
?>
```

O exemplo acima produzirá:

```
"=?UTF-7?Q?+WSqQzg-?=" <kru@gtinn.mon>
```

Notas

> **Nota** :
>
> Esta função não é projetada para quebrar linhas em pontos de quebra de contexto de nível superior (limites de palavra, etc.). Esse comportamento pode poluir a string original com espaços inesperados.

Veja também

- mb_decode_mimeheader() - Decodifica string no campo de cabeçalho MIME

mb_encode_numericentity

mb_encode_numericentity

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_encode_numericentity — Codificar caractere para referência numérica HTML

Descrição

```
function mb_encode_numericentity(
    string $string,
    array $map,
    ?string $encoding = null,
    bool $hex = false
): string
```

Converte códigos de caractere especificados em string `string` de códigos de caractere para referência numérica de caractere HTML.

Parâmetros

`string`

A string a ser codificada.

`map`

`map` é um array que especifica a área de código para converter.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

`hex`

Se a referência de entidade retornada deve estar em notação hexadecimal (caso contrário, estará em notação decimal).

Valor Retornado

A string convertida.

Erros/Exceções

Lança uma exceção ValueError se `map` não for uma lista de ints.

Registro de Alterações

Versão	Descrição
8.4.0	**mb_encode_numericentity()** agora lança uma exceção ValueError se `map` não for uma lista de ints.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de `map`

```php
<?php

$convmap = array (
 int start_code1, int end_code1, int offset1, int mask1,
 int start_code2, int end_code2, int offset2, int mask2,
 ........
 int start_codeN, int end_codeN, int offsetN, int maskN );
// Especifique o valor Unicode para start_codeN e end_codeN
// Adicione offsetN ao valor e faça um 'AND' bit a bit com maskN, então
// converta o valor para referência numérica de string.
?>
```

Exemplo #2 Exemplo de mb_encode_numericentity()

```php
<?php
$str = "aAæÆあア鮎";

/* Converte todos os caracteres UTF8 de até 4 bytes para referência numérica de caracteres HTML */
$convmap = [0, 0x1FFFFF, 0, 0x10FFFF];
var_dump(mb_encode_numericentity($str, $convmap, "utf8"));

/* Converte somente caracteres UTF8 de 2 bytes e de 4 bytes para referência numérica de caracteres HTML */
$convmap = [
    0x80, 0x7FF, 0, 0x10FFFF,
    0x10000, 0x1FFFFF, 0, 0x10FFFF,
];
var_dump(mb_encode_numericentity($str, $convmap, "utf8"));
?>
```

O exemplo acima produzirá:

```
string(46) "&#97;&#65;&#230;&#198;&#12354;&#12450;&#40509;"
string(28) "aA&#230;&#198;あア&#40509;"
```

Veja também

- mb_decode_numericentity() - Decodifica referência de string numérica HTML para caractere

mb_encoding_aliases

mb_encoding_aliases

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mb_encoding_aliases — Obtém aliases de um tipo de codificação conhecido

Descrição

function **mb_encoding_aliases**(string `$encoding`): array

Retorna um array de aliases para um tipo de `encoding` conhecido.

Parâmetros

`encoding`

O tipo de codificação a ser verificado, para aliases.

Valor Retornado

Retorna um array indexada numericamente de aliases de codificação.

Erros/Exceções

Lança um ValueError se `encoding` for desconhecido.

Registro de Alterações

Versão	Descrição
8.0.0	Se o `encoding` for desconhecido, agora é lançado um ValueError; anteriormente, um `E_WARNING` era emitido, e a função retornava `false`.

Examples

Exemplo #1 Exemplo de mb_encoding_aliases()

```
<?php
$encoding        = 'ASCII';
$known_encodings = mb_list_encodings();

if (in_array($encoding, $known_encodings)) {

    $aliases = mb_encoding_aliases($encoding);
    print_r($aliases);

} else {

    echo "Unknown ($encoding) encoding.\n";

}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => ANSI_X3.4-1968
    [1] => iso-ir-6
    [2] => ANSI_X3.4-1986
    [3] => ISO_646.irv:1991
    [4] => US-ASCII
    [5] => ISO646-US
    [6] => us
    [7] => IBM367
    [8] => cp367
    [9] => csASCII
)
```

Veja também

- mb_list_encodings() - Retorna um array de todas as codificações suportadas

mb_ereg

mb_ereg

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg — Correspondência de expressão regular com suporte multibyte

Descrição

function **mb_ereg**(string `$pattern`, string `$string`, array `&$matches = null`): bool

Executa a correspondência de expressão regular com suporte multibyte..

Parâmetros

`pattern`

O padrão de busca.

`string`

A string de busca.

`matches`

Se forem encontradas correspondências para substrings entre parênteses de `pattern` e a função for chamada com o terceiro argumento `matches`, as correspondências serão armazenadas nos elementos do array `matches`. Se não forem encontradas correspondências, `matches` é definido como um array vazio.

$matches[1] conterá a substring que começa no primeiro parêntese à esquerda; *$matches[2]* conterá a substring que começa no segundo parêntese, e assim por diante. *$matches[0]* conterá uma cópia da string completa correspondida.

Valor Retornado

Retorna se `pattern` corresponde a `string`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora retorna `true` em caso de sucesso. Anteriormente, ela retornava o comprimento em bytes da string correspondida se uma correspondência para `pattern` fosse encontrada em `string` e `matches` fosse passado. Se o parâmetro opcional `matches` não fosse passado ou o comprimento da string correspondida fosse `0`, esta função retornava `1`.
7.1.0	**mb_ereg()** agora define `matches` como um array vazio se nada for correspondido. Anteriormente, `matches` não era modificado nesse caso.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_eregi() - Correspondência de expressão regular sem diferenciar maiúsculas e minúsculas com suporte multibyte

mb_ereg_match

mb_ereg_match

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_match — Coincidência de expressão regular para string multibyte

Descrição

function **mb_ereg_match**(string `$pattern`, string `$string`, ?string `$options = null`): bool

Uma correspondência de expressão regular para uma string multibyte

> **Nota** :
>
> `pattern` só é correspondido no início de `string`.

Parâmetros

`pattern`

O padrão de expressão regular.

`string`

A string sendo avaliada.

`options`

A opção de pesquisa. Consulte mb_regex_set_options() para explicação.

Valor Retornado

Retorna **`true`** se `string` coincide com a expressão regular `pattern`, **`false`** se não.

Registro de Alterações

Versão	Descrição
8.0.0	`options` é anulável agora.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg() - Correspondência de expressão regular com suporte multibyte

mb_ereg_replace

mb_ereg_replace

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_replace — Substituir expressão regular com suporte a multibyte

Descrição

```
function mb_ereg_replace(
    string $pattern,
    string $replacement,
    string $string,
    ?string $options = null
): string|false|null
```

Examina `string` em busca de correspondências com `pattern`, em seguida, substitui o texto correspondido por `replacement`

Parâmetros

`pattern`

O padrão de expressão regular.

Caracteres multibyte podem ser usados em `pattern`.

`replacement`

O texto de substituição.

`string`

A string sendo verificada.

`options`

A opção de pesquisa. Consulte mb_regex_set_options() para explicação.

Valor Retornado

A string resultante em caso de sucesso, ou `false` em caso de erro. Se `string` não for válida para a codificação atual, `null` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`options` é anulável agora.
7.1.0	A função verifica se `string` é válida para a codificação atual.
7.1.0	O modificador `e` foi descontinuado.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

> **Aviso**
>
> Nunca use o modificador `e` ao trabalhar com entradas não confiáveis. Nenhum escape automático acontecerá (como é conhecido em preg_replace()). Não cuidar disso provavelmente criará vulnerabilidades de execução remota de código na sua aplicação.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_eregi_replace() - Substituir expressão regular com suporte multibyte, ignorando maiúsculas e minúsculas

mb_ereg_replace_callback

mb_ereg_replace_callback

(PHP 5 >= 5.4.1, PHP 7, PHP 8)

mb_ereg_replace_callback — Realiza uma busca e substituição de expressão regular com suporte a string multibyte usando um callback

Descrição

```
function mb_ereg_replace_callback(
    string $pattern,
    callable $callback,
    string $string,
    ?string $options = null
): string|false|null
```

Examina `string` em busca de correspondências com `pattern`, em seguida, substitui o texto correspondido pelo resultado da função `callback`.

O comportamento desta função é quase idêntico ao de mb_ereg_replace(), exceto pelo fato de que, em vez de `replacement`, deve-se especificar um `callback`.

Parâmetros

`pattern`

O padrão de expressão regular.

Caracteres multibyte podem ser usados em `pattern`.

`callback`

Um callback que será chamado e receberá um array de elementos correspondidos na `string`. O callback deve retornar a string de substituição.

Você frequentemente precisará da função `callback` para um **mb_ereg_replace_callback()** em apenas um lugar. Nesse caso, você pode usar uma função anônima para declarar o retorno de chamada dentro da chamada para **mb_ereg_replace_callback()**. Fazendo isso, você tem todas as informações para a chamada em um só lugar e não polui o namespace da função com o nome de uma função de callback não utilizada em nenhum outro lugar.

`string`

A string sendo verificada.

`options`

A opção de pesquisa. Consulte mb_regex_set_options() para explicação.

Valor Retornado

A string resultante em caso de sucesso ou `false` em caso de erro. Se `string` não for válida para a codificação atual, `null` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`options` é anulável agora.
7.1.0	A função verifica se `string` é válida para a codificação atual.

Examples

Exemplo #1 Exemplo de mb_ereg_replace_callback()

```php
<?php
// este texto foi usado em 2002
// queremos atualizá-lo para 2003
$text = "April fools day is 04/01/2002\n";
$text.= "Last christmas was 12/24/2001\n";
// a função callback
function next_year($matches)
{
  // como de costume: $matches[0] é a correspondência completa
  // $matches[1] é a correspondência para a primeira sub-expressão
  // envolvida por '(...)' e assim por diante
  return $matches[1].($matches[2]+1);
}
echo mb_ereg_replace_callback(
            "(\d{2}/\d{2}/)(\d{4})",
            "next_year",
            $text);

?>
```

O exemplo acima produzirá:

```
April fools day is 04/01/2003
Last christmas was 12/24/2002
```

Exemplo #2 mb_ereg_replace_callback() usando função anônima

```php
<?php
// este texto foi usado em 2002
// queremos atualizá-lo para 2003
$text = "April fools day is 04/01/2002\n";
$text.= "Last christmas was 12/24/2001\n";

echo mb_ereg_replace_callback(
            "(\d{2}/\d{2}/)(\d{4})",
            function ($matches) {
```

```
            return $matches[1].($matches[2]+1);
        },
        $text);
?>
```

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_replace() - Substituir expressão regular com suporte a multibyte
- Funções Anônimas

mb_ereg_search

mb_ereg_search

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search — Correspondência de expressão regular multibyte para uma string multibyte predefinida

Descrição

function **mb_ereg_search**(?string `$pattern = null`, ?string `$options = null`): bool

Realiza uma correspondência de expressão regular multibyte para uma string multibyte predefinida.

Parâmetros

`pattern`

O padrão de busca.

`options`

A opção de busca. Veja mb_regex_set_options() para explicação.

Valor Retornado

mb_ereg_search() retorna `true` se a string multibyte corresponder à expressão regular, ou `false` caso contrário. A string para correspondência é definida por mb_ereg_search_init(). Se `pattern` não for especificado, o anterior é utilizado.

Registro de Alterações

Versão	Descrição
8.0.0	`pattern` e `options` agora são anuláveis.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_init() - Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

mb_ereg_search_getpos

mb_ereg_search_getpos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_getpos — Retorna o ponto de início para a próxima correspondência de expressão regular

Descrição

function **mb_ereg_search_getpos**(): int

Retorna o ponto de início para a próxima correspondência de expressão regular.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

mb_ereg_search_getpos() retorna o ponto de início da correspondência de expressão regular para mb_ereg_search(), mb_ereg_search_pos(), mb_ereg_search_regs(). A posição é representada em bytes a partir do início da string.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_setpos() - Define o ponto de início para a próxima correspondência de expressão regular

mb_ereg_search_getregs

mb_ereg_search_getregs

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_getregs — Recupera o resultado da última correspondência de expressão regular multibyte

Descrição

function **mb_ereg_search_getregs**(): array|false

Recupera o resultado da última correspondência de expressão regular multibyte

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array incluindo a sub-string da parte correspondente pela última mb_ereg_search(), mb_ereg_search_pos(), mb_ereg_search_regs(). Se houver algumas correspondências, o primeiro elemento terá a sub-string correspondente, o segundo elemento terá a primeira parte agrupada com colchetes, o terceiro elemento terá a segunda parte agrupada com colchetes, e assim por diante. Retorna `false` em caso de erro.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_init() - Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

mb_ereg_search_init

mb_ereg_search_init

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_init — Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

Descrição

function **mb_ereg_search_init**(string `$string`, ?string `$pattern` = `null`, ?string `$options` = `null`): bool

mb_ereg_search_init() configura `string` e `pattern` para uma expressão regular multibyte. Esses valores são usados para mb_ereg_search(), mb_ereg_search_pos() e mb_ereg_search_regs().

Parâmetros

`string`

A string de busca.

`pattern`

O padrão de busca.

`options`

A opção de busca. Consulte mb_regex_set_options() para obter uma explicação.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`pattern` e `options` são anuláveis agora.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_regs() - Retorna a parte correspondida de uma expressão regular multibyte

mb_ereg_search_pos

mb_ereg_search_pos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_pos — Retorna a posição e o comprimento de uma parte correspondida da expressão regular multibyte para uma string multibyte predefinida

Descrição

function **mb_ereg_search_pos**(?string `$pattern` = `null`, ?string `$options` = `null`): array|false

Retorna a posição e o comprimento de uma parte correspondida da expressão regular multibyte para uma string multibyte predefinida.

A string para a correspondência é especificada por mb_ereg_search_init(). Se não for especificada, a anterior será usada.

Parâmetros

`pattern`

O padrão de busca.

`options`

A opção de busca. Consulte mb_regex_set_options() para obter uma explicação.

Valor Retornado

Um array contendo dois elementos. O primeiro elemento é o deslocamento, em bytes, onde a correspondência começa em relação ao início da string de busca, e o segundo elemento é o comprimento, em bytes, da correspondência.

Se ocorrer um erro, é retornado `false`.

Registro de Alterações

Versão	Descrição
8.0.0	`pattern` e `options` são anuláveis agora.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_init() - Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

mb_ereg_search_regs

mb_ereg_search_regs

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_regs — Retorna a parte correspondida de uma expressão regular multibyte

Descrição

function **mb_ereg_search_regs**(?string `$pattern = null`, ?string `$options = null`): array|false

Retorna a parte correspondida de uma expressão regular multibyte.

Parâmetros

`pattern`

O padrão de busca.

`options`

A opção de busca. Consulte mb_regex_set_options() para obter uma explicação.

Valor Retornado

mb_ereg_search_regs() executa a correspondência da expressão regular multibyte e, se houver alguma parte correspondida, ela retorna um array contendo a substring da parte correspondida como primeiro elemento, a primeira parte agrupada com colchetes como segundo elemento, a segunda parte agrupada como terceiro elemento, e assim por diante. Retorna `false` em caso de erro.

Registro de Alterações

Versão	Descrição
8.0.0	`pattern` e `options` agora são anuláveis.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_init() - Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

mb_ereg_search_setpos

mb_ereg_search_setpos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_setpos — Define o ponto de início para a próxima correspondência de expressão regular

Descrição

function **mb_ereg_search_setpos**(int `$offset`): bool

mb_ereg_search_setpos() define o ponto de início de uma correspondência para mb_ereg_search().

Parâmetros

`offset`

A posição a definir. Se for negativo, conta a partir do final da string.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
7.1.0	Suporte para `offsets` negativo foi adicionado.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_search_init() - Configura a string e a expressão regular para uma correspondência de expressão regular multibyte

mb_eregi

mb_eregi

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_eregi — Correspondência de expressão regular sem diferenciar maiúsculas e minúsculas com suporte multibyte

Descrição

function **mb_eregi**(string `$pattern`, string `$string`, array `&$matches = null`): bool

Executa a correspondência de expressão regular sem diferenciar maiúsculas e minúsculas com suporte multibyte.

Parâmetros

`pattern`

O padrão de expressão regular.

`string`

A string sendo pesquisada.

`matches`

Se correspondências forem encontradas para substrings entre parênteses de `pattern` e a função for chamada com o terceiro argumento `matches`, as correspondências serão armazenadas nos elementos do array `matches`. Se nenhuma correspondência for encontrada, `matches` será configurado como um array vazio.

$matches[1] conterá a substring que começa em o primeiro parêntese à esquerda; *$matches[2]* conterá a substring que começa no segundo, e assim por diante. *$matches[0]* conterá uma cópia da string completa correspondida.

Valor Retornado

Retorna se `pattern` corresponde a `string`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora retorna `true` em caso de sucesso. Anteriormente, ela retornava o comprimento em bytes da string correspondida se uma correspondência para `pattern` fosse encontrada em `string` e `matches` fosse passado. Se o parâmetro opcional `matches` não fosse passado ou o comprimento da string correspondida fosse `0`, esta função retornava `1`.
7.1.0	**mb_eregi()** agora define `matches` como um array vazio, se nada coincidir. Anteriormente, `matches` não era modificado nesse caso.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg() - Correspondência de expressão regular com suporte multibyte

mb_eregi_replace

mb_eregi_replace

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_eregi_replace — Substituir expressão regular com suporte multibyte, ignorando maiúsculas e minúsculas

Descrição

```
function mb_eregi_replace(
    string $pattern,
    string $replacement,
    string $string,
    ?string $options = null
): string|false|null
```

Examina `string` em busca de correspondências com `pattern`, e substitui o texto correspondido por `replacement`.

Parâmetros

`pattern`

O padrão de expressão regular. Caracteres multibyte podem ser usados. A diferenciação entre maiúsculas e minúsculas será ignorada.

`replacement`

O texto de substituição.

`string`

A string pesquisada.

`options`

A opção de pesquisa. Veja mb_regex_set_options() para explicação.

Valor Retornado

A string resultante ou **`false`** em caso de erro. Se `string` não for válida para a codificação atual, **`null`** é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`options` é anulável agora.
7.1.0	A função verifica se `string` é válida para a codificação atual.
7.1.0	O modificador `e` foi descontinuado.

Notas

> **Nota** :
>
> A codificação interna ou a codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função.

> **Aviso**
>
> Nunca use o modificador `e` ao trabalhar com entradas não confiáveis. Nenhum escape automático acontecerá (como é conhecido em preg_replace()). Não cuidar disso provavelmente criará vulnerabilidades de execução remota de código na sua aplicação.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg_replace() - Substituir expressão regular com suporte a multibyte

mb_get_info

mb_get_info

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_get_info — Obtém configurações internas do mbstring

Descrição

function **mb_get_info**(string `$type` = "all"): array|string|int|false|null

mb_get_info() retorna os parâmetros de configuração internos do mbstring.

Parâmetros

`type`

Se `type` não for especificado ou for especificado como `"all"`, `"internal_encoding"`, `"http_input"`, `"http_output"`, `"http_output_conv_mimetypes"`, `"mail_charset"`, `"mail_header_encoding"`, `"mail_body_encoding"`, `"illegal_chars"`, `"encoding_translation"`, `"language"`, `"detect_order"`, `"substitute_character"` e `"strict_detection"` serão retornados.

Se `type` for especificado como `"internal_encoding"`, `"http_input"`, `"http_output"`, `"http_output_conv_mimetypes"`, `"mail_charset"`, `"mail_header_encoding"`, `"mail_body_encoding"`, `"illegal_chars"`, `"encoding_translation"`, `"language"`, `"detect_order"`, `"substitute_character"` ou `"strict_detection"`, o parâmetro de configuração especificado será retornado.

Valor Retornado

Um array de informações do tipo se `type` não for especificado, caso contrário, um `type` específico, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	Os `types` `"func_overload"` e `"func_overload_list"` não são mais suportados.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_http_output() - Define/Obtém a codificação de caracteres de saída HTTP

mb_http_input

mb_http_input

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_http_input — Detecta a codificação de caracteres de entrada HTTP

Descrição

function **mb_http_input**(?string `$type` = `null`): array|string|false

Detecta a codificação de caracteres de entrada HTTP.

Parâmetros

`type`

A string de entrada especifica o tipo de entrada. `"G"` para GET, `"P"` para POST, `"C"` para COOKIE, `"S"` para string, `"L"` para lista, e `"I"` para a lista inteira (retornará um array). Se o tipo for omitido, ele retorna o último tipo de entrada processado.

Valor Retornado

O nome da codificação de caracteres, conforme o `type`, ou um array de nomes de codificação de caracteres, se `type` for `"I"`. Se **mb_http_input()** não processar a entrada HTTP especificada, ele retorna `false`.

Erros/Exceções

Lança uma exceção ValueError se `type` for inválido.

Registro de Alterações

Versão	Descrição
8.0.0	**mb_http_input()** agora lança uma exceção ValueError se `type` for inválido. `type` agora é anulável.

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_http_output() - Define/Obtém a codificação de caracteres de saída HTTP
- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres

mb_http_output

mb_http_output

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_http_output — Define/Obtém a codificação de caracteres de saída HTTP

Descrição

function **mb_http_output**(?string `$encoding` = `null`): string|bool

Define/obtém a codificação de caracteres de saída HTTP. A saída após a chamada desta função será convertida da codificação interna definida para `encoding`.

Parâmetros

`encoding`

> Se `encoding` estiver definido, **mb_http_output()** define a codificação de caracteres de saída HTTP para `encoding`.
>
> Se `encoding` for omitido, **mb_http_output()** retorna a codificação de caracteres de saída HTTP atual.

Valor Retornado

Se `encoding` for omitido, **mb_http_output()** retorna a codificação de caracteres de saída HTTP atual. Caso contrário, Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Lança uma exceção ValueError se `encoding` contiver bytes nulos.

Registro de Alterações

Versão	Descrição
8.4.0	**mb_http_output()** agora lança uma exceção ValueError se `encoding` contiver bytes nulos.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_http_input() - Detecta a codificação de caracteres de entrada HTTP
- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres

mb_internal_encoding

mb_internal_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_internal_encoding — Define/Obtém a codificação de caracteres interna

Descrição

function **mb_internal_encoding**(?string `$encoding` = `null`): string|bool

Define/Obtém a codificação de caracteres interna

Parâmetros

`encoding`

> `encoding` é o nome da codificação de caracteres usado para a conversão de codificação de caracteres de entrada HTTP, conversão de codificação de caracteres de saída HTTP, e a codificação de caracteres padrão para funções de string definidas pelo módulo mbstring. Deve-se observar que a codificação interna é totalmente diferente da utilizada em regex multibyte.

Valor Retornado

Se `encoding` for definido, então Retorna `true` em caso de sucesso ou `false` em caso de falha. Nesse caso, a codificação de caracteres para regex multibyte NÃO é alterada. Se `encoding` for omitido, então o nome atual da codificação de caracteres é retornado.

Erros/Exceções

A partir do PHP 8.0.0, um ValueError é lançado se o valor de `encoding` for uma codificação inválida. Antes do PHP 8.0.0, um `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.
8.0.0	Agora lança um ValueError se `encoding` for uma codificação inválida. Anteriormente, um `E_WARNING` era emitido.

Examples

Exemplo #1 Exemplo de mb_internal_encoding()

```php
<?php
/* Define a codificação de caracteres interna como UTF-8 */
mb_internal_encoding("UTF-8");

/* Exibe a codificação de caracteres interna atual */
echo mb_internal_encoding();
?>
```

Veja também

- mb_http_input() - Detecta a codificação de caracteres de entrada HTTP
- mb_http_output() - Define/Obtém a codificação de caracteres de saída HTTP
- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres
- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte

mb_language

mb_language

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_language — Define/Obtém o idioma atual

Descrição

function **mb_language**(?string `$language = null`): string|bool

Define/Obtém o idioma atual.

Parâmetros

`language`

Usado para codificar mensagens de e-mail. Os idiomas válidos estão listados na tabela a seguir. mb_send_mail() usa essa configuração para codificar e-mails.

Idioma	Charset	Codificação	Alias
German/de	ISO-8859-15	Quoted-Printable	Deutsch
English/en	ISO-8859-1	Quoted-Printable	
Armenian/hy	ArmSCII-8	Quoted-Printable	
Japanese/ja	ISO-2022-JP	BASE64	
Korean/ko	ISO-2022-KR	BASE64	
neutral	UTF-8	BASE64	
Russian/ru	KOI8-R	Quoted-Printable	
Turkish/tr	ISO-8859-9	Quoted-Printable	
Ukrainian/ua	KOI8-U	Quoted-Printable	
uni	UTF-8	BASE64	universal
Simplified Chinese/zh-cn	HZ	BASE64	
Traditional Chinese/zh-tw	BIG-5	BASE64	

Valor Retornado

Se `language` for definido e `language` for válido, retorna **`true`**. Caso contrário, retorna **`false`**. Quando `language` é omitido ou **`null`**, retorna o nome do idioma como uma string.

Registro de Alterações

Versão	Descrição
8.0.0	`language` é anulável agora.

Veja também

- mb_send_mail() - Envia email codificado

mb_lcfirst

mb_lcfirst

(PHP 8 >= 8.4.0)

mb_lcfirst — Converte parap minúscula o primeiro caractere de uma string

Descrição

function **mb_lcfirst**(string `$string`, ?string `$encoding` = **`null`**): string

Realiza uma operação lcfirst() segura para caracteres multi-byte, e retorna uma string com o primeiro caractere de `string` convertido para minúscula.

Parâmetros

`string`

A string de entrada.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou **`null`**, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a string resultante.

Veja também

- mb_ucfirst() - Converte para maiúscula o primeiro caractere de uma string
- lcfirst() - Torna minúsculo o primeiro caractere de uma string

mb_list_encodings

mb_list_encodings

(PHP 5, PHP 7, PHP 8)

mb_list_encodings — Retorna um array de todas as codificações suportadas

Descrição

function **mb_list_encodings**(): array

Retorna um array contendo todas as codificações suportadas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array indexado numericamente.

Erros/Exceções

Esta função não emite erros.

Examples

Exemplo #1 Exemplo de **mb_list_encodings()**

```php
<?php

print_r(mb_list_encodings());

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => pass
    [1] => auto
    [2] => wchar
    [3] => byte2be
    [4] => byte2le
    [5] => byte4be
    [6] => byte4le
    [7] => BASE64
    [8] => UUENCODE
    [9] => HTML-ENTITIES
    [10] => Quoted-Printable
    [11] => 7bit
    [12] => 8bit
    [13] => UCS-4
    [14] => UCS-4BE
    [15] => UCS-4LE
    [16] => UCS-2
    [17] => UCS-2BE
    [18] => UCS-2LE
    [19] => UTF-32
    [20] => UTF-32BE
    [21] => UTF-32LE
    [22] => UTF-16
    [23] => UTF-16BE
    [24] => UTF-16LE
    [25] => UTF-8
    [26] => UTF-7
    [27] => UTF7-IMAP
    [28] => ASCII
    [29] => EUC-JP
    [30] => SJIS
    [31] => eucJP-win
    [32] => SJIS-win
    [33] => JIS
    [34] => ISO-2022-JP
    [35] => Windows-1252
    [36] => ISO-8859-1
    [37] => ISO-8859-2
    [38] => ISO-8859-3
    [39] => ISO-8859-4
    [40] => ISO-8859-5
    [41] => ISO-8859-6
    [42] => ISO-8859-7
    [43] => ISO-8859-8
    [44] => ISO-8859-9
    [45] => ISO-8859-10
    [46] => ISO-8859-13
    [47] => ISO-8859-14
    [48] => ISO-8859-15
    [49] => EUC-CN
    [50] => CP936
    [51] => HZ
    [52] => EUC-TW
    [53] => BIG-5
    [54] => EUC-KR
```

```
    [55] => UHC
    [56] => ISO-2022-KR
    [57] => Windows-1251
    [58] => CP866
    [59] => KOI8-R
)
```

Veja também

- mb_encoding_aliases() - Obtém aliases de um tipo de codificação conhecido

mb_ltrim

mb_ltrim

(PHP 8 >= 8.4.0)

mb_ltrim — Remove espaços em branco (ou outros caracteres) do início de uma string

Descrição

function **mb_ltrim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Realiza uma operação ltrim() segura para strings multi-byte. Remove espaços em branco (ou outros caracteres) do início de uma string.

Sem o segundo parâmetro, **mb_ltrim()** removerá estes caracteres:

- `" "` (Unicode U+0020), um espaço em branco normal.
- `"\t"` (Unicode U+0009), uma tabulação horizontal.
- `"\n"` (Unicode U+000A), uma nova linha (alimentação de linha).
- `"\r"` (Unicode U+000D), um retorno de carro.
- `"\0"` (Unicode U+0000), o byte NUL.
- `"\v"` (Unicode U+000B), uma tabulação vertical.
- `"\f"` (Unicode U+000C), uma alimentação de formulário.
- `"\u00A0"` (Unicode U+00A0), um espaço sem quebra de palavra (NO-BREAK SPACE).
- `"\u1680"` (Unicode U+1680), uma marca de espaço Ogham.
- `"\u2000"` (Unicode U+2000), um EN QUAD.
- `"\u2001"` (Unicode U+2001), um EM QUAD.
- `"\u2002"` (Unicode U+2002), um EN SPACE.
- `"\u2003"` (Unicode U+2003), um EM SPACE.
- `"\u2004"` (Unicode U+2004), um THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), um FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), um SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), um FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), um PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), um THIN SPACE.
- `"\u200A"` (Unicode U+200A), um HAIR SPACE.
- `"\u2028"` (Unicode U+2028), um LINE SEPARATOR.

- `"\u2029"` (Unicode U+2029), um PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), um NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), um MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), um IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), um NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), um MONGOLIAN VOWEL SEPARATOR.

Parâmetros

`string`

A string de entrada.

`characters`

Opcionalmente, os caracteres a serem removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Esta função retorna uma string com espaços em branco removidos do início da `string`.

Veja também

- mb_trim() - Remove espaços em branco (ou outros caracteres) do início e do final de uma string
- mb_rtrim() - Remove espaços em branco (ou outros caracteres) do final de uma string
- ltrim() - Retira espaços em branco (ou outros caracteres) do início de uma string

mb_ord

mb_ord

(PHP 7 >= 7.2.0, PHP 8)

mb_ord — Obtém o ponto de código Unicode do caractere

Descrição

function **mb_ord**(string `$string`, ?string `$encoding` = `null`): int|false

Retorna o valor do ponto de código Unicode do caractere fornecido.

Esta função complementa mb_chr().

Parâmetros

`string`

Uma string

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

O ponto de código Unicode para o primeiro caractere de `string` ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Um exemplo básico de mb_ord()

```
<?php
var_dump(mb_ord("A", "UTF-8"));
var_dump(mb_ord("🐘", "UTF-8"));
var_dump(mb_ord("\x80", "ISO-8859-1"));
var_dump(mb_ord("\x80", "Windows-1252"));
?>
```

O exemplo acima produzirá:

```
int(65)

int(128024)

int(128)

int(8364)
```

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_chr() - Retorna um caractere pelo valor do ponto de código Unicode
- IntlChar::ord() - Retorna o valor do ponto de código Unicode do caractere
- ord() - Converte o primeiro byte de uma string para um valor entre 0 e 255

mb_output_handler

mb_output_handler

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_output_handler — Callback converte a codificação de caracteres no buffer de saída

Descrição

function **mb_output_handler**(string `$string`, int `$status`): string

mb_output_handler() é um callback ob_start(). **mb_output_handler()** converte caracteres no buffer de saída da codificação de caracteres interna para a codificação de caracteres de saída HTTP.

Parâmetros

`string`

O conteúdo do buffer de saída.

`status`

O status do buffer de saída.

Valor Retornado

A string convertida.

Examples

Exemplo #1 Exemplo de mb_output_handler()

```
<?php
mb_http_output("UTF-8");
ob_start("mb_output_handler");
?>
```

Notas

> **Nota** :
>
> Se você deseja enviar dados binários, como uma imagem, um cabeçalho Content-Type: deve ser definido usando header() antes que quaisquer dados binários sejam enviados ao cliente (por exemplo, header("Content-Type: image/png")). Se o cabeçalho Content-Type: for enviado, a conversão de codificação de caracteres de saída não será realizada.
>
> Observe que se 'Content-Type: text/*' for enviado, o corpo do conteúdo é considerado como texto; a conversão será realizada.

Veja também

- ob_start() - Ativa o buffer de saída

mb_parse_str

mb_parse_str

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_parse_str — Analisa dados GET/POST/COOKIE e define variável global

Descrição

function **mb_parse_str**(string `$string`, array `&$result`): bool

Analisa dados GET/POST/COOKIE e define variáveis globais. Como o PHP não fornece dados brutos POST/COOKIE, ele só pode ser usado para dados GET por enquanto. Analisa dados codificados em URL, detecta a codificação, converte a codificação para a codificação interna e define valores para o `result` array ou variáveis globais.

Parâmetros

`string`

Os dados codificados em URL.

`result`

Um array contendo valores decodificados e convertidos de caracteres.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O segundo parâmetro não é mais opcional.
7.2.0	Chamar **mb_parse_str()** sem o segundo parâmetro foi descontinuado.

Veja também

- mb_detect_order() - Define ou obtém a ordem de detecção de codificação de caracteres
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna

mb_preferred_mime_name

mb_preferred_mime_name

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_preferred_mime_name — Obter uma string MIME charset

Descrição

function **mb_preferred_mime_name**(string `$encoding`): string|false

Obtém MIME charset string para uma codificação específica.

Parâmetros

`encoding`

A codificação que está sendo verificada.

Valor Retornado

A string contendo o conjunto de caracteres MIME (`charset`) para a codificação de caracteres especificada em `encoding`, ou `false` se nenhum conjunto de caracteres for preferido para a codificação informada em `encoding`.

Examples

Exemplo #1 Exemplo de mb_preferred_mime_name()

```
<?php
$outputenc = "sjis-win";
mb_http_output($outputenc);
ob_start("mb_output_handler");
header("Content-Type: text/html; charset=" . mb_preferred_mime_name($outputenc));
?>
```

mb_regex_encoding

mb_regex_encoding

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_regex_encoding — Define/Obtém codificação de caracteres para regex multibyte

Descrição

function **mb_regex_encoding**(?string `$encoding` = `null`): string|bool

Define/obtém a codificação de caracteres para uma regex multibyte.

Parâmetros

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Se `encoding` for definido, então Retorna `true` em caso de sucesso ou `false` em caso de falha. Neste caso, a codificação de caracteres interna NÃO é alterada. Se `encoding` for omitido, então o nome da codificação de caracteres atual para uma regex multibyte é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- mb_ereg() - Correspondência de expressão regular com suporte multibyte

mb_regex_set_options

mb_regex_set_options

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_regex_set_options — Define/Obtém as opções padrão para funções mbregex

Descrição

function **mb_regex_set_options**(?string `$options` = `null`): string

Define/obtém as opções padrão descritas por `options` para funções de regex multibyte.

Parâmetros

`options`

As opções a serem definidas. Esta é uma string onde cada caractere é uma opção. Para definir um modo, o caractere de modo deve ser o último a ser definido, no entanto, pode haver apenas um modo, mas várias opções podem ser definidas.

Opções de Regex

Opção	Significado	Registro de Alterações
i	correspondência ambígua ativada	
x	Ativa a forma de padrão estendida	
m	`'.'` corresponde a quebras de linha	
s	`'^' -> '\A', '$' -> '\Z'`	
p	Mesmo que as opções `m` e `s`	
l	Encontrar as correspondências mais longas	
n	Ignorar correspondências vazias	
e	eval() código resultante	Descontinuado desde o PHP 7.1.0 e removido a partir do PHP 8.0.0

> **Nota** :
>
> A opção `"e"` não tem efeito quando definida através de **mb_regex_set_options()**. Use-a com mb_ereg_replace() ou mb_eregi_replace().

Modos de sintaxe Regex (apenas um pode ser definido)

Modo	Significado
j	Java (Sun java.util.regex)
u	GNU regex
g	grep
c	Emacs
r	Ruby
z	Perl
b	Regex Básico POSIX
d	Regex Estendido POSIX

Valor Retornado

As opções anteriores. Se `options` for omitido ou `null`, retorna a string que descreve as opções atuais.

Registro de Alterações

Versão	Descrição
8.0.0	Se o parâmetro `options` for fornecido e não for `null`, as opções *anteriores* são retornadas. Anteriormente, as opções *atuais* eram retornadas.
8.0.0	`options` é anulável agora.
8.0.0	A opção `"e"` agora lança um ValueError.
7.1.0	A opção `"e"` agora emite um `E_DEPRECATED`.

Veja também

- mb_split() - Divide uma string multibyte usando expressão regular
- mb_ereg() - Correspondência de expressão regular com suporte multibyte
- mb_eregi() - Correspondência de expressão regular sem diferenciar maiúsculas e minúsculas com suporte multibyte

mb_rtrim

mb_rtrim

(PHP 8 >= 8.4.0)

mb_rtrim — Remove espaços em branco (ou outros caracteres) do final de uma string

Descrição

function **mb_rtrim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Realiza uma operação rtrim() regura para strings multi-byte, e retorna uma string com espaços em branco (ou outros caracteres) removidos do final da `string`.

Sem o segundo parâmetro, **mb_rtrim()** removerá estes caracteres:

- `" "` (Unicode U+0020), um espaço em branco normal.
- `"\t"` (Unicode U+0009), uma tabulação horizontal.
- `"\n"` (Unicode U+000A), uma nova linha (alimentação de linha).
- `"\r"` (Unicode U+000D), um retorno de carro.
- `"\0"` (Unicode U+0000), o byte NUL.
- `"\v"` (Unicode U+000B), uma tabulação vertical.
- `"\f"` (Unicode U+000C), uma alimentação de formulário.
- `"\u00A0"` (Unicode U+00A0), um espaço sem quebra de palavra (NO-BREAK SPACE).
- `"\u1680"` (Unicode U+1680), uma marca de espaço Ogham.
- `"\u2000"` (Unicode U+2000), um EN QUAD.
- `"\u2001"` (Unicode U+2001), um EM QUAD.
- `"\u2002"` (Unicode U+2002), um EN SPACE.
- `"\u2003"` (Unicode U+2003), um EM SPACE.
- `"\u2004"` (Unicode U+2004), um THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), um FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), um SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), um FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), um PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), um THIN SPACE.
- `"\u200A"` (Unicode U+200A), um HAIR SPACE.
- `"\u2028"` (Unicode U+2028), um LINE SEPARATOR.
- `"\u2029"` (Unicode U+2029), um PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), um NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), um MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), um IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), um NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), um MONGOLIAN VOWEL SEPARATOR.

Parâmetros

`string`

A string de entrada.

`characters`

Opcionalmente, os caracteres a serem removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a string modificada.

Veja também

- mb_trim() - Remove espaços em branco (ou outros caracteres) do início e do final de uma string
- mb_ltrim() - Remove espaços em branco (ou outros caracteres) do início de uma string
- rtrim() - Retira espaços em branco (ou outros caracteres) do final de uma string

mb_scrub

mb_scrub

(PHP 7 >= 7.2.0, PHP 8)

mb_scrub — Substitui sequências de bytes malformadas pelo caractere substituto

Descrição

function **mb_scrub**(string `$string`, ?string `$encoding` = `null`): string

Realiza uma conversão de conjunto de caracteres a partir da codificação especificada, ou a codificação padrão se nenhuma codificação for especificada, para a mesma codificação. Isso tem o efeito de substituir qualquer sequência de bytes inválida pelo caractere substituto.

Parâmetros

`string`

A string de entrada.

`encoding`

A codificação usada para interpretar `string`. Se for omitida ou `null`, o mbstring.internal_encoding setting será usado se configurado, caso contrário, o default_charset setting será usado.

Valor Retornado

O resultado string com sequências de bytes inválidas substituídas.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

mb_send_mail

mb_send_mail

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_send_mail — Envia email codificado

Descrição

function **mb_send_mail**(
  string `$to`,
  string `$subject`,
  string `$message`,

```
    array|string $additional_headers = [],
    ?string $additional_params = null
): bool
```

Envia e-mails. Cabeçalhos e mensagens são convertidos e codificados de acordo com a configuração de mb_language(). É uma função encapsuladora para mail(), então veja também mail() para detalhes.

Parâmetros

`to`

Os endereços de e-mail para os quais o e-mail será enviado. Múltiplos destinatários podem ser especificados separando cada endereço em `to` por vírgula. Este parâmetro não é automaticamente codificado.

`subject`

O assunto do e-mail.

`message`

A mensagem do e-mail.

`additional_headers` (opcional)

String ou array a ser inserido no final do cabeçalho do e-mail.

Isso é tipicamente usado para adicionar cabeçalhos extras (De, Cc e Bcc). Múltiplos cabeçalhos extras devem ser separados por um CRLF (\r\n). Valide o parâmetro para não injetar cabeçalhos indesejados por atacantes.

Se um array for passado, suas chaves são os nomes dos cabeçalhos e seus valores são os respectivos valores dos cabeçalhos.

> **Nota** :
>
> Ao enviar e-mails, o e-mail *deve* conter um cabeçalho `From`. Isso pode ser configurado com o parâmetro `additional_headers`, ou um padrão pode ser configurado no *php.ini*.
>
> Falhar em fazer isso resultará em uma mensagem de erro semelhante a `Warning: mail(): "sendmail_from" not set in php.ini or custom "From:" header missing`. O cabeçalho `From` também define `Return-Path` no Windows.

> **Nota** :
>
> Se as mensagens não forem recebidas, tente usar apenas LF (\n). Alguns agentes de transferência de email Unix (notavelmente » qmail) substituem LF por CRLF automaticamente (o que leva a duplicação de CR se CRLF for usado). Isso deve ser usado apenas como último recurso, pois não está em conformidade com » RFC 2822.

`additional_params`

`additional_params` é um parâmetro de linha de comando MTA útil ao definir o cabeçalho Return-Path correto ao usar sendmail.

Este parâmetro é escapado internamente por escapeshellcmd() para evitar a execução de comandos. escapeshellcmd() evita a execução de comandos, mas permite adicionar parâmetros adicionais. Por razões de segurança, este parâmetro deve ser validado.

Como escapeshellcmd() é aplicado automaticamente, alguns caracteres que são permitidos como endereços de e-mail por RFCs da Internet não podem ser usados. Programas que são obrigados a usar esses caracteres mail() não podem ser usados.

O usuário que o servidor da web executa deve ser adicionado como um usuário confiável para o arquivo de configuração do sendmail para evitar a adição de um cabeçalho 'X-Warning' à mensagem quando o remetente do envelope (-f) é definido usando este método. Para usuários do sendmail, este arquivo é */etc/mail/trusted-users*.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`additional_params` é anulável agora.
7.2.0	O parâmetro `additional_headers` agora também aceita um array.

Veja também

- mail() - Envia e-mail
- mb_encode_mimeheader() - Codificar string para cabeçalho MIME
- mb_language() - Define/Obtém o idioma atual

mb_split

mb_split

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_split — Divide uma string multibyte usando expressão regular

Descrição

function **mb_split**(string `$pattern`, string `$string`, int `$limit` = -1): array|false

Divide uma `string` multibyte usando expressão regular `pattern` e retorna o resultado como um array.

Parâmetros

`pattern`

O padrão da expressão regular.

`string`

A string sendo dividida.

`limit`

Se o parâmetro opcional `limit` for especificado, ele será dividido em `limit` elementos no máximo.

Valor Retornado

O resultado como um array, ou **`false`** em caso de falha.

Notas

> **Nota** :
>
> A codificação de caracteres especificada por mb_regex_encoding() será usada como a codificação de caracteres para esta função por padrão.

Veja também

- mb_regex_encoding() - Define/Obtém codificação de caracteres para regex multibyte
- mb_ereg() - Correspondência de expressão regular com suporte multibyte
- explode() - Divide uma string com base em outra string

mb_str_pad

mb_str_pad

(PHP 8 >= PHP 8.3.0)

mb_str_pad — Preenche uma string multibyte para um comprimento específico com outra string multibyte

Descrição

function **mb_str_pad**(
  string `$string`,

```
    int $length,
    string $pad_string = " ",
    int $pad_type = STR_PAD_RIGHT,
    ?string $encoding = null
): string
```

Esta função retorna a `string` preenchida à esquerda, à direita ou em ambos os lados até o comprimento de preenchimento especificado, onde o comprimento é medido em pontos de código Unicode. Se o argumento opcional `pad_string` não for fornecido, a `string` será preenchida com espaços, caso contrário, ela será preenchida com caracteres de `pad_string` até o limite.

Parâmetros

`string`

A string de entrada.

`length`

Se o valor de `length` for negativo, menor ou igual ao comprimento da string de entrada, nenhum preenchimento ocorre, e a `string` será retornada.

`pad_string`

> **Nota** :
>
> O `pad_string` pode ser truncado se o número necessário de caracteres de preenchimento não puder ser dividido uniformemente pelo comprimento de `pad_string`.

`pad_type`

O argumento opcional `pad_type` pode ser `STR_PAD_RIGHT`, `STR_PAD_LEFT`, ou `STR_PAD_BOTH`. Por padrão, `STR_PAD_RIGHT`.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a string preenchida.

Examples

Exemplo #1 Exemplo de mb_str_pad()

```
<?php
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_RIGHT)); // string(18) "▶▶❤❓✳❤"
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_LEFT));  // string(18) "❤❓✳❤▶▶"
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_BOTH));  // string(18) "❤❓▶▶❤❓"

var_dump(mb_str_pad("🎉", 3, "祝", STR_PAD_LEFT));   // string(10) "祝祝🎉"
?>
```

mb_str_split

mb_str_split

(PHP 7 >= 7.4.0, PHP 8)

mb_str_split — Dada uma string multibyte, retorna um array de seus caracteres

Descrição

function **mb_str_split**(string `$string`, int `$length` = 1, ?string `$encoding` = `null`): array

Esta função retornará um array de strings, sendo uma versão de str_split() com suporte para codificações de tamanho de caractere variável, bem como codificações de tamanho fixo de caracteres de 1, 2 ou 4 bytes. Se o parâmetro `length` for especificado, a string será dividida em partes do comprimento especificado em caracteres (não bytes). O parâmetro `encoding` pode ser especificado opcionalmente e é uma boa prática fazê-lo.

Parâmetros

`string`

A string para dividir em caracteres ou partes.

`length`

Se especificado, cada elemento do array retornado será composto por vários caracteres em vez de um único caractere.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Uma string especificando uma das codificações suportadas.

Valor Retornado

mb_str_split() retorna um array de strings.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.
8.0.0	Esta função não retorna mais `false` em caso de falha.

Veja também

- str_split() - Converte uma string em um array
- grapheme_str_split() - Separa uma string em um array
- explode() - Divide uma string com base em outra string

mb_strcut

mb_strcut

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strcut — Obtém parte da string

Descrição

```
function mb_strcut(
    string $string,
    int $start,
    ?int $length = null,
    ?string $encoding = null
): string
```

mb_strcut() extrai uma substring de uma string de forma semelhante a mb_substr(), mas opera em bytes em vez de caracteres. Se a posição de corte estiver entre dois bytes de um caractere multibyte, o corte é realizado a partir do primeiro byte desse caractere. Essa também é a diferença para a função substr(), que simplesmente cortaria a string entre os bytes e resultaria em uma sequência de bytes malformada.

Parâmetros

`string`

A string a ser cortada.

`start`

Se `start` for não negativo, a string retornada começará na posição de *byte* `start` em `string`, contando a partir do zero. Por exemplo, na string `'abcdef'`, o byte na posição `0` é `'a'`, o byte na posição `2` é `'c'`, e assim por diante.

Se `start` for negativo, a string retornada começará no byte `start` contando a partir do final de `string`. No entanto, se o valor absoluto de um `start` negativo for maior que o comprimento da string, a parte retornada começará do início de `string`.

`length`

Comprimento em *bytes*. Se omitido ou `null` for passado, extrai todos os bytes até o final da string.

Se `length` for negativo, a string retornada terminará no byte de número `length` contando a partir do final de `string`. No entanto, se o valor absoluto de um `length` negativo for maior que o número de caracteres após a posição de `start`, uma string vazia será retornada.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

mb_strcut() retorna a parte de `string` especificada pelos parâmetros `start` e `length`.

Registro de Alterações

Versão	Descrição
8.4.0	O comportamento agora é mais consistente em strings `UTF-8` e `UTF-16` inválidas.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_substr() - Obtém parte da string
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna

mb_strimwidth

mb_strimwidth

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strimwidth — Obtém string truncada com largura especificada

Descrição

```
function mb_strimwidth(
    string $string,
    int $start,
    int $width,
    string $trim_marker = "",
    ?string $encoding = null
): string
```

Trunca a string `string` para a largura especificada `width`, onde caracteres de meia largura contam como `1`, e caracteres de largura total contam como `2`. Consulte » http://www.unicode.org/reports/tr11/ para detalhes sobre larguras de caracteres do leste asiático.

Parâmetros

`string`

A string a ser decodificada.

`start`

O deslocamento da posição inicial. Número de caracteres a partir do início da string (o primeiro caractere é 0), ou se start for negativo, o número de caracteres a partir do final da string.

`width`

A largura do corte desejado. Se uma largura negativa for especificada, conta a partir do final da string.

> **Nota** :
>
> Passar uma largura negativa foi descontinuado a partir do PHP 8.3.0.

`trim_marker`

Uma string que é adicionada ao final de string quando a string é truncada.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

A string truncada. Se `trim_marker` estiver definido, `trim_marker` substitui os últimos caracteres para corresponder à `width`.

Registro de Alterações

Versão	Descrição
8.3.0	Passar uma largura negativa `width` para **mb_strimwidth()** agora foi descontinuado.
8.0.0	O parâmetro `encoding` agora pode ser nulo.
7.1.0	Adicionado suporte para `starts` e `widths` negativos.

Examples

Exemplo #1 Exemplo de **mb_strimwidth()**

```php
<?php
echo mb_strimwidth("Hello World", 0, 10, "...");
// saída: "Hello W..."
?>
```

Veja também

- mb_strwidth() - Retorna a largura da string
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna

mb_stripos

mb_stripos

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_stripos — Encontra a posição da primeira ocorrência de uma string dentro de outra, sem diferenciar maiúsculas de minúsculas

Descrição

```
function mb_stripos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

mb_stripos() retorna a posição numérica da primeira ocorrência de `needle` na string `haystack`. Ao contrário de mb_strpos(), **mb_stripos()** não faz distinção entre maiúsculas e minúsculas. Se `needle` não for encontrado, retorna `false`.

Parâmetros

`haystack`

A string da qual deve ser obtida a posição da primeira ocorrência de `needle`

`needle`

A string a ser encontrada em `haystack`

`offset`

A posição em `haystack` a partir da qual a pesquisa deve ser iniciada. Um valor negativo é contado no sentido contrário a partir do final da string.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a posição numérica da primeira ocorrência de `needle` na string `haystack`, ou `false` se `needle` não for encontrado.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.
7.1.0	Adicionado suporte para `offset` negativo.

Veja também

- stripos() - Encontra a posição da primeira ocorrência de uma substring em uma string, de forma insensível a maiúsculas/minúsculas
- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string
- mb_strpos() - Encontra a posição da primeira ocorrência de uma string em outra

mb_stristr

mb_stristr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_stristr — Encontra a primeira ocorrência de uma string dentro de outra, sem diferenciar maiúsculas de minúsculas

Descrição

```
function mb_stristr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_stristr() encontra a primeira ocorrência de `needle` em `haystack` e retorna a parte de `haystack`. Ao contrário de mb_strstr(), **mb_stristr()** não faz distinção entre maiúsculas e minúsculas. Se `needle` não for encontrado, retorna `false`.

Parâmetros

`haystack`

A string da qual obter a primeira ocorrência de `needle`

`needle`

A string a ser encontrada em `haystack`

`before_needle`

Determina qual parte de `haystack` esta função retorna. Se definido como `true`, retorna toda `haystack` desde o início até a primeira ocorrência de `needle` (excluindo needle). Se definido como `false`, retorna toda `haystack` desde a primeira ocorrência de `needle` até o final (incluindo needle).

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a parte de `haystack`, ou `false` se `needle` não for encontrado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- stristr() - strstr insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string
- mb_strstr() - Encontra a primeira ocorrência de uma string em outra

mb_strlen

mb_strlen

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strlen — Obtém o comprimento da string

Descrição

function **mb_strlen**(string `$string`, ?string `$encoding` = **`null`**): int

Obtém o comprimento de uma string.

Parâmetros

`string`

A string sendo verificada quanto ao comprimento.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou **`null`**, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna o número de caracteres em string `string` tendo codificação de caracteres `encoding`. Um caractere multibyte é contado como 1.

Erros/Exceções

Se a codificação for desconhecida, um erro do nível **`E_WARNING`** é gerado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- grapheme_strlen() - Obtém o comprimento de uma string em unidade de grafema
- iconv_strlen() - Retorna a contagem de caracteres da string
- strlen() - Retorna o tamanho de uma string

mb_strpos

mb_strpos

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strpos — Encontra a posição da primeira ocorrência de uma string em outra

Descrição

function **mb_strpos**(
  string `$haystack`,
  string `$needle`,
  int `$offset` = 0,

```
    ?string $encoding = null
): int|false
```

Encontra a posição da primeira ocorrência de `needle` na string de `haystack`.

Realiza uma operação strpos() segura para multibyte com base no número de caracteres. A posição do primeiro caractere é 0, a posição do segundo caractere é 1, e assim por diante.

Parâmetros

`haystack`

A string da qual será obtida a posição da primeira ocorrência de `needle`.

`needle`

A string a ser encontrada em `haystack`. Em contraste com strpos(), valores numéricos não são aplicados como o valor ordinal de um caractere.

`offset`

A posição de início da pesquisa. Se não for especificada, 0 é usado. Uma posição negativa conta a partir do final da string no sentido contrário.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a posição numérica da primeira ocorrência de `needle` na string `haystack`. Se `needle` não for encontrada, retorna `false`.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.
7.1.0	Adicionado suporte para posições negativas no parâmetro `offset`.

Veja também

- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- strpos() - Encontra a posição da primeira ocorrência de uma substring em uma string

mb_strrchr

mb_strrchr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strrchr — Encontra a última ocorrência de um caractere em uma string dentro de outra

Descrição

```
function mb_strrchr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strrchr() encontra a última ocorrência de `needle` em `haystack` e retorna a porção de `haystack`. Se `needle` não for encontrada, retorna `false`.

Parâmetros

`haystack`

A string da qual será obtida a última ocorrência de `needle`.

`needle`

A string a ser encontrada em `haystack`.

`before_needle`

Determina qual porção de `haystack` esta função retorna. Se definido como `true`, retorna toda `haystack` desde o início até a última ocorrência de `needle`. Se definido como `false`, retorna toda `haystack` desde a última ocorrência de `needle` até o final.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a porção de `haystack`. ou `false` se `needle` não for encontrada.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- strrchr() - Encontra a última ocorrência de um caractere em uma string
- mb_strstr() - Encontra a primeira ocorrência de uma string em outra
- mb_strrichr() - Encontra a última ocorrência de um caractere em uma string dentro de outra, sem diferenciar maiúsculas de minúsculas

mb_strrichr

mb_strrichr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strrichr — Encontra a última ocorrência de um caractere em uma string dentro de outra, sem diferenciar maiúsculas de minúsculas

Descrição

```
function mb_strrichr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strrichr() encontra a última ocorrência de `needle` em `haystack` e retorna a porção de `haystack`. Ao contrário de mb_strrchr(), **mb_strrichr()** é insensível a maiúsculas e minúsculas. Se `needle` não for encontrada, retorna `false`.

Parâmetros

`haystack`

A string da será obtida a última ocorrência de `needle`.

`needle`

A string a ser encontrada em `haystack`.

`before_needle`

Determina qual porção de `haystack` esta função retorna. Se definido como `true`, retorna toda `haystack` desde o início até a última ocorrência de `needle`. Se definido como `false`, retorna toda `haystack` desde a última ocorrência de `needle` até o final.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a porção de `haystack`. ou `false` se `needle` não for encontrada.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_stristr() - Encontra a primeira ocorrência de uma string dentro de outra, sem diferenciar maiúsculas de minúsculas
- mb_strrchr() - Encontra a última ocorrência de um caractere em uma string dentro de outra

mb_strripos

mb_strripos

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strripos — Encontra a posição da última ocorrência de uma string em outra, sem diferenciar maiúsculas de minúsculas

Descrição

```
function mb_strripos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

mb_strripos() realiza uma operação segura para multibyte de strripos() com base no número de caracteres. A posição de `needle` é contada a partir do início de `haystack`. A posição do primeiro caractere é 0. A posição do segundo caractere é 1. Ao contrário de mb_strrpos(), **mb_strripos()** é insensível a maiúsculas e minúsculas.

Parâmetros

`haystack`

A string da qual deve ser obtida a posição da última ocorrência de `needle`.

`needle`

A string a ser encontrada em `haystack`.

`offset`

Pode ser especificado para iniciar a busca a partir de um número arbitrário de caracteres de `haystack`. Valores negativos encerram a busca em um ponto arbitrário antes do final de `haystack`.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a posição numérica da última ocorrência de `needle` na string `haystack` ou `false` se `needle` não for encontrada.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- strripos() - Encontra a posição da última ocorrência de uma substring em uma string, insensível a maiúsculas/minúsculas
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string
- mb_strrpos() - Encontra a posição da última ocorrência de uma string em outra string

mb_strrpos

mb_strrpos

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strrpos — Encontra a posição da última ocorrência de uma string em outra string

Descrição

```
function mb_strrpos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

Realiza uma operação segura para multibyte strrpos() com base no número de caracteres. A posição de `needle` é contada a partir do início de `haystack`. A posição do primeiro caractere é 0. A posição do segundo caractere é 1.

Parâmetros

`haystack`

A string que está sendo verificada, para a última ocorrência de `needle`

`needle`

A string a ser encontrada em `haystack`.

`offset`

Pode ser especificado para começar a busca a partir de um número arbitrário de caracteres em `haystack`. Valores negativos interromperão a busca em um ponto arbitrário antes do final de `haystack`.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a posição numérica da última ocorrência de `needle` na string `haystack`. Se `needle` não for encontrada, retorna `false`.

Erros/Exceções

- Se `offset` for maior que o comprimento de `haystack`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O envio de `encoding` como terceiro argumento em vez de `offset` foi removido.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_strpos() - Encontra a posição da primeira ocorrência de uma string em outra
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna
- strrpos() - Encontra a posição da última ocorrência de uma substring em uma string

mb_strstr

mb_strstr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strstr — Encontra a primeira ocorrência de uma string em outra

Descrição

```
function mb_strstr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strstr() encontra a primeira ocorrência de `needle` em `haystack` e retorna a porção de `haystack`. Se `needle` não for encontrado, retorna `false`.

Parâmetros

`haystack`

A string da qual obter a primeira ocorrência de `needle`

`needle`

A string a ser encontrada em `haystack`

`before_needle`

Determina qual parte de `haystack` essa função retorna. Se definido como `true`, retorna toda a `haystack` desde o início até a primeira ocorrência de `needle` (excluindo needle). Se definido como `false`, retorna toda a `haystack` da primeira ocorrência de `needle` até o final (incluindo needle).

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a porção de `haystack`, ou `false` se `needle` não for encontrado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `needle` agora aceita uma string vazia.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- stristr() - strstr insensível a maiúsculas/minúsculas
- strstr() - Encontra a primeira ocorrência de uma string
- mb_stristr() - Encontra a primeira ocorrência de uma string dentro de outra, sem diferenciar maiúsculas de minúsculas

mb_strtolower

mb_strtolower

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_strtolower — Torna uma string minúscula

Descrição

function **mb_strtolower**(string `$string`, ?string `$encoding` = `null`): string

Retorna a `string` com todos os caracteres alfabéticos convertidos em minúsculas.

Parâmetros

`string`

A string a ser convertida em minúsculas.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

`string` com todos os caracteres alfabéticos convertidos em minúsculas.

Registro de Alterações

Versão	Descrição
8.3.0	Implementadas regras condicionais de maiúscula/minúscula para a letra grega sigma.

Examples

Exemplo #1 Exemplo de mb_strtolower()

```php
<?php
$str = "Maria Tinha Um Cordeirinho e Ela O AMAVA";
$str = mb_strtolower($str);
echo $str; // Imprime maria tinha um cordeirinho e ela o amava
?>
```

Exemplo #2 Exemplo de mb_strtolower() com texto UTF-8 não-Latin

```php
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_strtolower($str, 'UTF-8');
echo $str; // Imprime τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός
?>
```

Notas

Ao contrário de strtolower(), 'alphabetic' é determinado pelas propriedades de caractere Unicode. Portanto, o comportamento desta função não é afetado pelas configurações de localidade e pode converter qualquer caractere com uma propriedade 'alphabetic', como um trema (ä).

Para obter mais informações sobre as propriedades Unicode, consulte » http://www.unicode.org/reports/tr21/.

Veja também

- mb_strtoupper() - Faz uma string ficar em maiúsculas
- mb_convert_case() - Realiza a conversão de caixa em uma string
- strtolower() - Converte uma string para minúsculas

mb_strtoupper

mb_strtoupper

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_strtoupper — Faz uma string ficar em maiúsculas

Descrição

function **mb_strtoupper**(string `$string`, ?string `$encoding` = `null`): string

Retorna `string` com todos os caracteres alfabéticos convertidos para maiúsculas.

Parâmetros

`string`

A string que será convertida para maiúsculas.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

`string` com todos os caracteres alfabéticos convertidos para maiúsculas.

Examples

Exemplo #1 Exemplo de mb_strtoupper()

```php
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = mb_strtoupper($str);
echo $str; // Imprime MARY HAD A LITTLE LAMB AND SHE LOVED IT SO
?>
```

Exemplo #2 Exemplo de mb_strtoupper() com texto UTF-8 não latino

```php
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_strtoupper($str, 'UTF-8');
echo $str; // Imprime ΤΆΧΙΣΤΗ ΑΛΏΠΗΞ ΒΑΦΗΣ ΨΗΜΕΝΗ ΓΗ, ΔΡΑΣΚΕΛΊΖΕΙ ΥΠΈΡ ΝΩΘΡΟΥ ΚΥΝΟΣ
?>
```

Notas

Ao contrário de strtoupper(), 'alfabético' é determinado pelas propriedades de caracteres Unicode. Assim, o comportamento desta função não é afetado pelas configurações de localidade e pode converter qualquer caractere que tenha a propriedade 'alfabético', como a-umlaut (ä).

Para obter mais informações sobre as propriedades Unicode, consulte » http://www.unicode.org/reports/tr21/.

Veja também

- mb_strtolower() - Torna uma string minúscula
- mb_convert_case() - Realiza a conversão de caixa em uma string
- strtoupper() - Converte uma string em maiúsculas

mb_strwidth

mb_strwidth

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strwidth — Retorna a largura da string

Descrição

function **mb_strwidth**(string `$string`, ?string `$encoding` = `null`): int

Retorna a largura da string `string`, onde os caracteres de meia largura contam como `1`, e os caracteres de largura total contam como `2`. Consulte » http://www.unicode.org/reports/tr11/ para detalhes sobre larguras de caracteres no leste asiático.

Os caracteres de largura total são: `U+1100-U+115F`, `U+11A3-U+11A7`, `U+11FA-U+11FF`, `U+2329-U+232A`, `U+2E80-U+2E99`, `U+2E9B-U+2EF3`, `U+2F00-U+2FD5`, `U+2FF0-U+2FFB`, `U+3000-U+303E`, `U+3041-U+3096`, `U+3099-U+30FF`, `U+3105-U+312D`, `U+3131-U+318E`, `U+3190-U+31BA`, `U+31C0-U+31E3`, `U+31F0-U+321E`, `U+3220-U+3247`, `U+3250-U+32FE`, `U+3300-U+4DBF`, `U+4E00-U+A48C`, `U+A490-U+A4C6`, `U+A960-U+A97C`, `U+AC00-U+D7A3`, `U+D7B0-U+D7C6`, `U+D7CB-U+D7FB`, `U+F900-U+FAFF`, `U+FE10-U+FE19`, `U+FE30-U+FE52`, `U+FE54-U+FE66`, `U+FE68-U+FE6B`, `U+FF01-U+FF60`, `U+FFE0-U+FFE6`, `U+1B000-U+1B001`, `U+1F200-U+1F202`, `U+1F210-U+1F23A`, `U+1F240-U+1F248`, `U+1F250-U+1F251`, `U+20000-U+2FFFD`, `U+30000-U+3FFFD`. Todos os outros caracteres são de meia largura.

Parâmetros

`string`

A string a ser decodificada.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

A largura da string `string`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de mb_strwidth()

```
<?php
var_dump(
    mb_strwidth('a'),       // LATIN SMALL LETTER A
    mb_strwidth("\u{ff41}") // FULLWIDTH LATIN SMALL LETTER A
);
?>
```

O exemplo acima produzirá:

```
int(1)
int(2)
```

Veja também

- mb_strimwidth() - Obtém string truncada com largura especificada
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna

mb_substitute_character

mb_substitute_character

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_substitute_character — Define/Obtém caractere de substituição

Descrição

function **mb_substitute_character**(string|int|null `$substitute_character = null`): string|int|bool

Especifica um caractere de substituição quando a codificação de caracteres de entrada é inválida ou o código de caractere não existe na codificação de caracteres de saída. Caracteres inválidos podem ser substituídos por `"none"` (sem saída), string ou valor int (valor do código de caractere Unicode).

Essa configuração afeta as funções mb_convert_encoding(), mb_convert_variables(), mb_output_handler(), mb_scrub() e mb_send_mail().

Parâmetros

`substitute_character`

Especifique o valor Unicode como um int, ou como uma das seguintes strings:

- `"none"`: sem saída
- `"long"`: Valor do código de caractere de saída (Exemplo: `U+3000`, `JIS+7E7E`)
- `"entity"`: Entidade de caractere de saída (Exemplo: `Ȁ`)

Valor Retornado

Se `substitute_character` for configurado, retorna `true` para sucesso, caso contrário, retorna `false`. Se `substitute_character` não estiver configurado, retorna a configuração atual.

Registro de Alterações

Versão	Descrição
8.0.0	Passar uma string vazia para `substitute_character` não é mais suportado; deve-se passar `"none"` em vez disso.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de mb_substitute_character()

```php
<?php
/* Definir com Unicode U+3013 (GETA MARK) */
mb_substitute_character(0x3013);

/* Definir no formato hexadecimal */
mb_substitute_character("long");

/* Exibir configuração atual */
echo mb_substitute_character();
?>
```

mb_substr

mb_substr

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_substr — Obtém parte da string

Descrição

```
function mb_substr(
    string $string,
    int $start,
    ?int $length = null,
    ?string $encoding = null
): string
```

Realiza uma operação de substr() segura para múltiplos bytes baseada no número de caracteres. A posição é contada a partir do início de `string`. A posição do primeiro caractere é 0. A posição do segundo caractere é 1, e assim por diante.

Parâmetros

`string`

A string da qual extrair a substring.

`start`

Se `start` for não negativo, a string retornada começará na posição `start`-ésima em `string`, contando a partir de zero. Por exemplo, na string `'abcdef'`, o caractere na posição `0` é `'a'`, o caractere na posição `2` é `'c'`, e assim por diante.

Se `start` for negativo, a string retornada começará no caractere `start`-ésimo a partir do final de `string`.

`length`

Número máximo de caracteres a serem usados de `string`. Se omitido ou `null` for passado, extrai todos os caracteres até o final da string.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

mb_substr() retorna a porção de `string` especificada pelos parâmetros `start` e `length`.

Registro de Alterações

Versão	Descrição
8.4.0	Em strings inválidas (aquelas com erros de codificação), os índices de caractere agora são interpretados da mesma maneira que a maioria das outras funções mbstring. Isso significa que os índices de caractere retornados por mb_strpos() podem ser passados diretamente.
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Veja também

- mb_strcut() - Obtém parte da string
- mb_internal_encoding() - Define/Obtém a codificação de caracteres interna

mb_substr_count

mb_substr_count

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_substr_count — Conta o número de ocorrências de substring

Descrição

function **mb_substr_count**(string `$haystack`, string `$needle`, ?string `$encoding` = `null`): int

Conta o número de vezes que a substring `needle` ocorre na `haystack` string.

Parâmetros

`haystack`

A string sendo verificada.

`needle`

A string sendo procurada.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

O número de vezes que a substring `needle` ocorre na `haystack` string.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `encoding` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de **mb_substr_count()**

```
<?php
echo mb_substr_count("This is a test", "is"); // Imprime 2
?>
```

Veja também

- mb_strpos() - Encontra a posição da primeira ocorrência de uma string em outra

- mb_substr() - Obtém parte da string
- substr_count() - Conta o número de ocorrências de uma substring

mb_trim

mb_trim

(PHP 8 >= 8.4.0)

mb_trim — Remove espaços em branco (ou outros caracteres) do início e do final de uma string

Descrição

function **mb_trim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Realiza uma operação trim() segura para strings multi-byte, e retorna uma string com os espaços em branco (ou outros caracteres) removidos do início e do final da `string`. Sem o segundo parâmetro, **mb_trim()** removerá estes caracteres:

- `" "` (Unicode U+0020), um espaço em branco normal.
- `"\t"` (Unicode U+0009), uma tabulação horizontal.
- `"\n"` (Unicode U+000A), uma nova linha (alimentação de linha).
- `"\r"` (Unicode U+000D), um retorno de carro.
- `"\0"` (Unicode U+0000), o byte NUL.
- `"\v"` (Unicode U+000B), uma tabulação vertical.
- `"\f"` (Unicode U+000C), uma alimentação de formulário.
- `"\u00A0"` (Unicode U+00A0), um espaço sem quebra de palavra (NO-BREAK SPACE).
- `"\u1680"` (Unicode U+1680), uma marca de espaço Ogham.
- `"\u2000"` (Unicode U+2000), um EN QUAD.
- `"\u2001"` (Unicode U+2001), um EM QUAD.
- `"\u2002"` (Unicode U+2002), um EN SPACE.
- `"\u2003"` (Unicode U+2003), um EM SPACE.
- `"\u2004"` (Unicode U+2004), um THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), um FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), um SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), um FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), um PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), um THIN SPACE.
- `"\u200A"` (Unicode U+200A), um HAIR SPACE.
- `"\u2028"` (Unicode U+2028), um LINE SEPARATOR.
- `"\u2029"` (Unicode U+2029), um PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), um NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), um MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), um IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), um NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), um MONGOLIAN VOWEL SEPARATOR.

Parâmetros

`string`

A string de entrada.

`characters`

Opcionalmente, os caracteres a serem removidos também podem ser especificados usando o parâmetro `characters`. Simplesmente liste todos os caracteres que precisam ser removidos.

`encoding`

O parâmetro `encoding` é a codificação de caracteres. Se for omitido ou `null`, o valor da codificação de caracteres interna será usado.

Valor Retornado

Retorna a string resultante.

Veja também

- mb_ltrim() - Remove espaços em branco (ou outros caracteres) do início de uma string
- mb_rtrim() - Remove espaços em branco (ou outros caracteres) do final de uma string
- trim() - Retira espaços (ou outros caracteres) do início e do final de uma string

mb_ucfirst

mb_ucfirst

(PHP 8 >= 8.4.0)

mb_ucfirst — Converte para maiúscula o primeiro caractere de uma string

Descrição

function **mb_ucfirst**(string `$string`, ?string `$encoding = null`): string

Realizad uma operação ucfirst() segura para strings multi-byte, e retorna uma string com o primeiro caractere do parâmetro `string` convertido para letra de título.

Parâmetros

`string`

A string de entrada.

`encoding`

A codificação da string.

Valor Retornado

Retorna a string resultante.

Notas

> **Nota** :
>
> Em contraste com as funções padrão de conversão de maiúsculas e minúsculas, como strtolower() e strtoupper(), a conversão é executada com base nas propriedades dos caracteres Unicode. Assim, o comportamento desta função não é afetado pelas configurações de localidade e pode converter quaisquer caracteres que tenham propriedade 'alphabetic' (alfabética), como trema (ä) por exemplo.

Para obter mais informações sobre as propriedades Unicode, consulte » http://www.unicode.org/reports/tr21/.

Veja também

- mb_lcfirst() - Converte parap minúscula o primeiro caractere de uma string
- mb_convert_case() - Realiza a conversão de caixa em uma string

- ucfirst() - Transforma o primeiro caractere de uma string em maiúsculo

Verificação de tipo de caractere

Introdução

As funções providas por esta extensão verificam se um caractere ou uma string está dentro de uma certa classe de caracteres de acordo com a localidade atual (veja também setlocale()).

Quando chamada com um argumento inteiro, estas funções comportam-se exatamente como ocorre na linguagem C usando *ctype.h*. Significa que, se for passado um inteiro menor que 256, será usado o valor ASCII dele para conferir se está dentro do intervalo especificado (dígitos estão entre 0x30-0x39). Se o número estiver entre -128 e -1 (inclusive), será adicionado 256 e a verificação estará concluída.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Quando chamada com uma string como argumento ela irá verificar todos os caracteres na string e somente retornará `true` se todos os caracteres da string corresponderem ao o critério solicitado. Quando chamada com uma string vazia, o resultado sempre será `false`.

Se for passado qualquer tipo diferente de string ou inteiro a função retornará `false`.

Note que funções "ctype" são sempre preferenciais em relação às expressões regulares, e em relação às funções equivalentes `"str_*"` e `"is_*"`. O motivo é que "ctype" usa a biblioteca nativa C e assim tem um processamento significativamente mais rápido.

> **Nota** :
>
> Estas funções não estão relacionadas às da biblioteca "ctypes" do Python. O nome da extensão vem do arquivo de cabeçalho *ctype.h* onde suas funções equivalentes em C são definidas.
>
> Esta extensão também é anterior à biblioteca "ctypes" do Python, então qualquer confusão causada por conta da nomenclatura dificilmente é por conta do PHP.

Instalação/Configuração

Índice

Dependências

Nenhum adicional para estas funções da biblioteca padrão C que estão sempre disponíveis.

Instalação

Esta extensão está habilitada por padrão. Ela pode ser desabilitada usando a seguinte opção em tempo de compilação: **--disable-ctype**

A versão do PHP para Windows tem suporte nativo para esta extensão. Nenhuma extensão adicional precisa ser carregada para usar essas funções.

Funções da Ctype

Índice

ctype_alnum

ctype_alnum

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_alnum — Verifica se os caracteres são alfanuméricos

Descrição

function **ctype_alnum**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são alfanuméricos.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna **`true`** se todos os caracteres em `text` são letras ou dígitos, **`false`** caso contrário. Quando chamada com uma string vazia, o resultado sempre será **`false`**.

Examples

Exemplo #1 Um exemplo de ctype_alnum() (usando a localidade padrão)

```php
<?php
$strings = array('AbCd1zyZ9', 'foo!#$bar');
foreach ($strings as $testcase) {
    if (ctype_alnum($testcase)) {
        echo "A string $testcase consiste inteiramente de letras ou dígitos.\n";
    } else {
        echo "A string $testcase não consiste inteiramente de letras ou dígitos.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string AbCd1zyZ9 consiste inteiramente de letras ou dígitos.
A string foo!#$bar não consiste inteiramente de letras ou dígitos.
```

Veja também

- ctype_alpha() - Verifica se os caracteres são alfabéticos
- ctype_digit() - Verifica se os caracteres são numéricos
- setlocale() - Define informação de localidade
- IntlChar::isalnum() - Verifica se um ponto de código é um caractere alfanumérico

ctype_alpha

ctype_alpha

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_alpha — Verifica se os caracteres são alfabéticos

Descrição

function **ctype_alpha**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são alfabéticos. No padrão C locale letras são `[A-Za-z]` e **ctype_alpha()** é equivalente a `(ctype_upper($text) || ctype_lower($text))` se $text é um único caractere, mas outras linguagens possuem letras que não são considerada nem maiúscula nem minúscula.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são letra da localidade atual, `false` caso contrário. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_alpha() (usando a localidade padrão)

```php
<?php
$strings = array('KjgWZC', 'arf12');
foreach ($strings as $testcase) {
    if (ctype_alpha($testcase)) {
        echo "A string $testcase consiste inteiramente de letras.\n";
    } else {
        echo "A string $testcase não consiste inteiramente de letras.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string KjgWZC consiste inteiramente de letras.
A string arf12 não consiste inteiramente de letras.
```

Veja também

- ctype_upper() - Verifica se os caracteres são maiúsculos
- ctype_lower() - Verifica se os caracteres são minúsculos
- setlocale() - Define informação de localidade
- IntlChar::isalpha() - Verifica se um ponto de código é uma letra

ctype_cntrl

ctype_cntrl

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_cntrl — Verifica se os caracteres são de controle

Descrição

function **ctype_cntrl**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são caracteres de controle. Caracteres de controle são e.g. line feed, tab, escape.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna **`true`** se todos caracteres em `text` são caracteres de controle da localidade atual, **`false`** caso contrário. Quando chamada com uma string vazia, o resultado sempre será **`false`**.

Examples

Exemplo #1 Um exemplo de ctype_cntrl()

```php
<?php
$strings = array('string1' => "\n\r\t", 'string2' => 'arf12');
foreach ($strings as $name => $testcase) {
    if (ctype_cntrl($testcase)) {
        echo "A string '$name' consiste inteiramente de caracteres de controle.\n";
    } else {
        echo "A string '$name' não consiste inteiramente de caracteres de controle.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string 'string1' consiste inteiramente de caracteres de controle.
A string 'string2' não consiste inteiramente de caracteres de controle.
```

Veja também

- ctype_print() - Verifica se os caracteres são imprimíveis
- IntlChar::iscntrl() - Verifica se um ponto de código é um caractere de controle

ctype_digit

ctype_digit

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_digit — Verifica se os caracteres são numéricos

Descrição

function **ctype_digit**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são numéricos.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna **`true`** se todos caracteres na string `text` forem dígitos decimais, **`false`** caso contrário. Quando chamada com uma string vazia, o resultado sempre será **`false`**.

Examples

Exemplo #1 Um exemplo de ctype_digit()

```php
<?php
$strings = array('1820.20', '10002', 'wsl!12');
foreach ($strings as $caso_de_teste) {
    if (ctype_digit($caso_de_teste)) {
        echo "A string $caso_de_teste consiste inteiramente de dígitos.\n";
    } else {
        echo "A string $caso_de_teste não consiste inteiramente de dígitos.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string 1820.20 não consiste inteiramente de dígitos.
A string 10002 consiste inteiramente de dígitos.
A string wsl!12 não consiste inteiramente de dígitos.
```

Exemplo #2 Um exemplo de ctype_digit() comparando strings com inteiros

```php
<?php

$string_numerica = '42';
$inteiro         = 42;

ctype_digit($string_numerica);  // verdadeiro
ctype_digit($inteiro);          // false (código ASCII 42 é o caractere *)

is_numeric($string_numerica);   // verdadeiro
is_numeric($inteiro);           // verdadeiro
?>
```

Veja também

- ctype_alnum() - Verifica se os caracteres são alfanuméricos
- ctype_xdigit() - Verifica se os caracteres representam um dígito hexadecimal
- is_numeric() - Verifica se uma variável é um número ou uma string numérica
- is_int() - Informa se o tipo de uma variável é um inteiro
- is_string() - Verifica se o tipo de uma variável é string
- IntlChar::isdigit() - Verifica se um ponto de código é um caractere de dígito

ctype_graph

ctype_graph

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_graph — Verifica se os caracteres são imprimíveis exceto espaço

Descrição

function **ctype_graph**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são visíveis.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são imprimíveis e criam uma saída visível (sem espaço em branco), `false` caso contrário. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Exemplo de ctype_graph()

```php
<?php
$strings = array('string1' => "asdf\n\r\t", 'string2' => 'arf12', 'string3' => 'LKA#@%.54');
foreach ($strings as $name => $testcase) {
    if (ctype_graph($testcase)) {
```

```
        echo "A string '$name' consiste inteiramente de caracteres (visivelmente) imprimíveis.\n";
    } else {
        echo "A string '$name' não consiste inteiramente de caracteres (visivelmente) imprimíveis.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string 'string1' não consiste inteiramente de caracteres (visivelmente) imprimíveis.
A string 'string2' consiste inteiramente de caracteres (visivelmente) imprimíveis.
A string 'string3' consiste inteiramente de caracteres (visivelmente) imprimíveis.
```

Veja também

- ctype_alnum() - Verifica se os caracteres são alfanuméricos
- ctype_print() - Verifica se os caracteres são imprimíveis
- ctype_punct() - Verifica se é um caractere imprimível que não é um caractere de espaço ou alfanumério
- IntlChar::isgraph() - Verifica se um ponto de código é um caractere gráfico

ctype_lower

ctype_lower

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_lower — Verifica se os caracteres são minúsculos

Descrição

function **ctype_lower**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são letras minúsculas.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são letras minúsculas na localidade atual. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_lower() (usando a localidade padrão)

```php
<?php
$strings = array('aac123', 'qiutoas', 'QASsdks');
foreach ($strings as $testcase) {
    if (ctype_lower($testcase)) {
        echo "A string $testcase consiste inteiramente de letras minúsculas.\n";
    } else {
        echo "A string $testcase não consiste inteiramente de letras minúsculas.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string aac123 não consiste inteiramente de letras minúsculas.
A string qiutoas consiste inteiramente de letras minúsculas.
A string QASsdks não consiste inteiramente de letras minúsculas.
```

Veja também

- ctype_alpha() - Verifica se os caracteres são alfabéticos
- ctype_upper() - Verifica se os caracteres são maiúsculos
- setlocale() - Define informação de localidade
- IntlChar::islower() - Verifica se um ponto de código é uma letra minúscula

ctype_print

ctype_print

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_print — Verifica se os caracteres são imprimíveis

Descrição

function **ctype_print**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são imprimíveis.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna **`true`** se todos caracteres em `text` produzem saída (incluindo em branco). Retorna **`false`** se `text` contiver caracteres de controle ou caracteres que não têm nenhuma saída ou função de controle. Quando chamada com uma string vazia, o resultado sempre será **`false`**.

Examples

Exemplo #1 Um exemplo da ctype_print()

```php
<?php
$strings = array('string1' => "asdf\n\r\t", 'string2' => 'arf12', 'string3' => 'LKA#@%.54');
foreach ($strings as $name => $testcase) {
    if (ctype_print($testcase)) {
        echo "A string '$name' consiste inteiramente de caracteres imprimíveis.\n";
    } else {
        echo "A string '$name' não consiste inteiramente de caracteres imprimíveis.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string 'string1' não consiste inteiramente de caracteres imprimíveis.
A string 'string2' consiste inteiramente de caracteres imprimíveis.
A string 'string3' consiste inteiramente de caracteres imprimíveis.
```

Veja também

- ctype_cntrl() - Verifica se os caracteres são de controle
- ctype_graph() - Verifica se os caracteres são imprimíveis exceto espaço
- ctype_punct() - Verifica se é um caractere imprimível que não é um caractere de espaço ou alfanumério
- IntlChar::isprint() - Verifica se um ponto de código é um caractere imprimível

ctype_punct

ctype_punct

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_punct — Verifica se é um caractere imprimível que não é um caractere de espaço ou alfanumério

Descrição

function **ctype_punct**(mixed `$text`): bool

Verifica se todos os caracteres da string fornecida, `text`, são caracteres de pontuação.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se cada caractere em `text` é imprimível, não sendo letra, dígito ou espaço em branco, `false` caso contrário. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_punct()

```
<?php
$strings = array('ABasdk!@!$#', '!@ # $', '*&$()');
foreach ($strings as $testcase) {
    if (ctype_punct($testcase)) {
        echo "A string $testcase consiste inteiramente de caracteres de pontuação.\n";
    } else {
        echo "A string $testcase não consiste inteiramente de caracteres de pontuação.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string ABasdk!@!$# não consiste inteiramente de caracteres de pontuação.
A string !@ # $ não consiste inteiramente de caracteres de pontuação.
A string *&$() consiste inteiramente de caracteres de pontuação.
```

Veja também

- ctype_cntrl() - Verifica se os caracteres são de controle
- ctype_graph() - Verifica se os caracteres são imprimíveis exceto espaço
- IntlChar::ispunct() - Verifica se um ponto de código é um caractere de pontuação

ctype_space

ctype_space

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_space — Verifica se os caracteres são espaço em branco

Descrição

function **ctype_space**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são espaços em branco.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são espaços em branco, `false` caso contrário. Além de caracteres de espaço, também são considerados em branco os caracteres de tabulação horizontal, tabulação vertical, alimentação de linha, retorno de carro e alimentação de formulário. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_space()

```php
<?php
$strings = array(
    'string1' => "\n\r\t",
    'string2' => "\narf12",
    'string3' => '\n\r\t' // observe as aspas simples
);
foreach ($strings as $name => $testcase) {
    if (ctype_space($testcase)) {
        echo "A string '$name' consiste apenas em caracteres de espaço em branco.\n";
    } else {
        echo "A string '$name' contém caracteres sem espaço em branco.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string 'string1' consiste apenas em caracteres de espaço em branco.
A string 'string2' contém caracteres sem espaço em branco.
A string 'string3' contém caracteres sem espaço em branco.
```

Veja também

- ctype_cntrl() - Verifica se os caracteres são de controle
- ctype_graph() - Verifica se os caracteres são imprimíveis exceto espaço
- ctype_punct() - Verifica se é um caractere imprimível que não é um caractere de espaço ou alfanumério
- IntlChar::isspace() - Verifica se um ponto de código é um caractere de espaçamento

ctype_upper

ctype_upper

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_upper — Verifica se os caracteres são maiúsculos

Descrição

function **ctype_upper**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são maiúsculos.

Parâmetros

`text`

A string a ser testada.

> **Nota** :

> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são letras maiúsculas na localidade atual. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_upper() (usando a localidade padrão)

```
<?php
$strings = array('AKLWC139', 'LMNSDO', 'akwSKWsm');
foreach ($strings as $testcase) {
    if (ctype_upper($testcase)) {
        echo "A string $testcase consiste inteiramente de letras maiúsculas.\n";
    } else {
        echo "A string $testcase não consiste inteiramente de letras maiúsculas.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string AKLWC139 não consiste inteiramente de letras maiúsculas.
A string LMNSDO consiste inteiramente de letras maiúsculas.
A string akwSKWsm não consiste inteiramente de letras maiúsculas.
```

Veja também

- ctype_alpha() - Verifica se os caracteres são alfabéticos
- ctype_lower() - Verifica se os caracteres são minúsculos
- setlocale() - Define informação de localidade
- IntlChar::isupper() - Verifica se um ponto de código tem a categoria geral "Lu" (letra maiúscula)

ctype_xdigit

ctype_xdigit

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_xdigit — Verifica se os caracteres representam um dígito hexadecimal

Descrição

function **ctype_xdigit**(mixed `$text`): bool

Verifica se todos os caracteres na string fornecida, `text`, são 'dígitos' hexadecimais.

Parâmetros

`text`

A string a ser testada.

> **Nota** :
>
> Se um int entre -128 e 255 inclusive for fornecido, ele será interpretado como o valor ASCII de um único caractere (valores negativos são somados a 256 para permitir caracteres no intervalo ASCII estendido). Qualquer outro inteiro será interpretado como uma string contendo os dígitos decimais do inteiro.

> **Aviso**
>
> A partir do PHP 8.1.0, passar um argumento que não seja string foi descontinuado. No futuro, o argumento será interpretado como uma string em vez de um ponto de código ASCII. Dependendo do comportamento esperado, o argumento deve ser convertido para string ou uma chamada explícita para chr() deve ser feita.

Valor Retornado

Retorna `true` se todos caracteres em `text` são 'dígitos' hexadecimais, que são os dígitos e caracteres no intervalo `[A-Fa-f]`, `false` caso contrário. Quando chamada com uma string vazia, o resultado sempre será `false`.

Examples

Exemplo #1 Um exemplo de ctype_xdigit()

```php
<?php
$strings = array('AB10BC99', 'AR1012', 'ab12bc99');
foreach ($strings as $testcase) {
    if (ctype_xdigit($testcase)) {
        echo "A string $testcase consiste inteiramente de dígitos hexadecimais.\n";
    } else {
        echo "A string $testcase consiste inteiramente de dígitos hexadecimais.\n";
    }
}
?>
```

O exemplo acima produzirá:

```
A string AB10BC99 consiste inteiramente de dígitos hexadecimais.
A string AR1012 não consiste inteiramente de dígitos hexadecimais.
A string ab12bc99 consiste inteiramente de dígitos hexadecimais.
```

Veja também

- ctype_digit() - Verifica se os caracteres são numéricos
- IntlChar::isxdigit() - Verifica se um ponto de código é um dígito hexadecimal

URLs

Introdução

Lidando com URLs: codificando, decodificando e interpretando.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

As seguintes constantes são para uso com parse_url().

PHP_URL_SCHEME (int)

PHP_URL_HOST (int)

Produz o nome do host da URL analisada.

PHP_URL_PORT (int)

Produz a porta da URL analisada.

PHP_URL_USER (int)

Produz o usuário do URL analisado.

PHP_URL_PASS (int)

Produz a senha do URL analisado.

PHP_URL_PATH (int)

Produz o caminho do URL analisado.

PHP_URL_QUERY (int)

Produz a query string do URL analisado.

PHP_URL_FRAGMENT (int)

Produz o fragmento (string após o hashmark #) do URL analisado.

As seguintes constantes devem ser usadas com http_build_query().

PHP_QUERY_RFC1738 (int)

A codificação é realizada por » RFC 1738 e o tipo de mídia `application/x-www-form-urlencoded`, que implica que os espaços são codificados como sinais de mais (+).

PHP_QUERY_RFC3986 (int)

A codificação é realizada de acordo com » RFC 3986, e os espaços serão codificados em porcentagem (`%20`).

Funções de URL

Índice

base64_decode

base64_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

base64_decode — Decodifica dados no formato MIME base64

Descrição

function **base64_decode**(string `$string`, bool `$strict = false`): string|false

Decodifica uma `string` codificada em base64.

Parâmetros

`string`

Os dados codificados.

`strict`

Se o parâmetro `strict` for definido como `true`, então a função **base64_decode()** retornará `false` se a entrada contiver caracteres fora do alfabeto base64. Caso contrário, os caracteres inválidos serão descartados silenciosamente.

Valor Retornado

Retorna os dados decodificados ou `false` em caso de falha. Os dados retornados podem ser binários.

Examples

Exemplo #1 Exemplo base64_decode()

```php
<?php
$str = 'RXN0YSDDqSB1bWEgc3RyaW5nIGNvZGlmaWNhZGEu';
echo base64_decode($str);
?>
```

O exemplo acima produzirá:

```
Esta é uma string codificada.
```

Veja também

- base64_encode() - Codifica dados com MIME base64
- Seção 6.8 do » RFC 2045

base64_encode

base64_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

base64_encode — Codifica dados com MIME base64

Descrição

function **base64_encode**(string `$string`): string

Codifica a dada `string` com base64.

Esta codificação é designada para que dados binários durem no transporte sobre camadas de transorte que não são 8-bit clean, como mensagens de e-mail.

Os dados codificados em base64 ocupam cerca de 33% a mais de espaço do que os dados originais.

Parâmetros

`string`

Os dados a serem codificados.

Valor Retornado

Os dados codificados, como uma string.

Examples

Exemplo #1 Exemplo base64_encode()

```php
<?php
$str = 'Esta é uma string codificada';
echo base64_encode($str);
?>
```

O exemplo acima produzirá:

```
RXN0YSDDqSB1bWEgc3RyaW5nIGNvZGlmaWNhZGEu
```

Veja também

- base64_decode() - Decodifica dados no formato MIME base64
- chunk_split() - Divide uma string em pedaços menores
- convert_uuencode() - Codifica uma string com uuencode
- Seção 6.8 do » RFC 2045

get_headers

get_headers

(PHP 5, PHP 7, PHP 8)

get_headers — Retorna todos os cabeçalhos enviados pelo servidor em resposta a uma requisição HTTP

Descrição

function **get_headers**(string `$url`, bool `$associative = false`, ?resource `$context = null`): array|false

get_headers() retorna um array com os cabeçalhos enviados pelo servidor em resposta a uma requisição HTTP.

Parâmetros

`url`

A URL alvo.

`associative`

Se o parâmetro opcional `associative` estiver definido como true, **get_headers()** avalia a resposta e define as chaves do array.

`context`

Um recurso de contexto válido criado com stream_context_create(), ou `null` para usar o contexto padrão

Valor Retornado

Retorna um array indexado ou associativo com os cabeçalhos, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `associative` foi modificado de int para bool.
7.1.0	O parâmetro `context` foi adicionado.

Examples

Exemplo #1 Exemplo de get_headers()

```php
<?php
$url = 'http://www.example.com';

print_r(get_headers($url));

print_r(get_headers($url, true));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => HTTP/1.1 200 OK
    [1] => Date: Sat, 29 May 2004 12:28:13 GMT
    [2] => Server: Apache/1.3.27 (Unix)  (Red-Hat/Linux)
    [3] => Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
    [4] => ETag: "3f80f-1b6-3e1cb03b"
    [5] => Accept-Ranges: bytes
    [6] => Content-Length: 438
    [7] => Connection: close
    [8] => Content-Type: text/html
)

Array
(
    [0] => HTTP/1.1 200 OK
    [Date] => Sat, 29 May 2004 12:28:14 GMT
    [Server] => Apache/1.3.27 (Unix)  (Red-Hat/Linux)
    [Last-Modified] => Wed, 08 Jan 2003 23:11:55 GMT
    [ETag] => "3f80f-1b6-3e1cb03b"
    [Accept-Ranges] => bytes
    [Content-Length] => 438
    [Connection] => close
    [Content-Type] => text/html
)
```

Exemplo #2 Exemplo de get_headers() usando HEAD

```php
<?php
// Por padrão get_headers usa uma requisição GET para obter os cabeçalhos. Se
// a intenção for enviar uma requisição HEAD, isso pode ser feito utilizando um contexo de fluxo:
$context = stream_context_create(
    [
        'http' => array(
            'method' => 'HEAD'
        )
    ]
);
$headers = get_headers('http://example.com', false, $context);
?>
```

Veja também

- apache_request_headers() - Obtém todos os cabeçalhos da requisição HTTP

get_meta_tags

get_meta_tags

(PHP 4, PHP 5, PHP 7, PHP 8)

get_meta_tags — Extrai todos os atributos de conteúdo de metatags de um arquivo e retorna um array

Descrição

function **get_meta_tags**(string `$filename`, bool `$use_include_path = false`): array|false

Abre o arquivo informado em `filename` e analisa-o linha a linha procurando por <meta>-tags. A análise termina em `</head>`.

Parâmetros

`filename`

O caminho para o arquivo HTML, como uma string. Pode ser um arquivo local ou uma URL.

Exemplo #1 O que get_meta_tags() analisa

```
<meta name="author" content="name">
<meta name="keywords" content="php documentation">
<meta name="DESCRIPTION" content="a php manual">
<meta name="geo.position" content="49.33;-86.59">
</head> <!-- análise termina aqui -->
```

`use_include_path`

Definir `use_include_path` para `true` fará com que o PHP tente abrir o arquivo ao longo do caminho de inclusão padrão de acordo com a diretiva include_path. Isto é útil para arquivos locais, mas não para URLs.

Valor Retornado

Retorna um array com todas as metatags analisadas.

O valor da propriedade "name" torna-se a chave, o valor da propriedade "content" torna-se o valor do array retornada, para que se possa usar facilmente funções padrão de arrays para percorrê-lo ou acessar valores únicos. Caracteres especiais no valor da propriedade "name" são substituídos por '_', o restante é convertido para minúsculas. Se duas metatags tiverem o mesmo nome, apenas a última será retornada.

Retorna `false` em caso de falha.

Examples

Exemplo #2 O que get_meta_tags() retorna

```
<?php
// Assumindo que as tags acima estão em www.example.com
$tags = get_meta_tags('http://www.example.com/');

// Note como as chaves estão todas em minúsculas agora, e
// como o ponto (.) foi substituído pelo sublinhado (_) na chave.
echo $tags['author'];       // name
echo $tags['keywords'];     // php documentation
echo $tags['description'];  // a php manual
echo $tags['geo_position']; // 49.33;-86.59
?>
```

Notas

> **Nota** :
>
> Somente metatags com atributos "name" serão analisadas. As aspas não são obrigatórias.

Veja também

- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- urlencode() - Codifica uma string no formato URL

http_build_query

http_build_query

(PHP 5, PHP 7, PHP 8)

http_build_query — Gera a string de consulta em formato URL

Descrição

```
function http_build_query(
    array|object $data,
    string $numeric_prefix = "",
    ?string $arg_separator = null,
    int $encoding_type = PHP_QUERY_RFC1738
): string
```

Gera a string de consulta em formato URL a partir de um array associativo (ou indexado).

Parâmetros

`data`

Pode ser um array ou objeto contendo propriedades.

Se `data` é um array, este pode ser de estrutura simples de uma dimensão, ou um array multidimensional (que por sua vez pode conter outros arrays).

Se `data` é um objeto, então somente propriedades públicas serão incorporadas ao resultado.

> **Nota** :
>
> O método mágico __toString() não é chamado quando um objeto é avaliado. Para usar a representação em string de um objeto na query string, o objeto precisa ser explicitamente convertido para uma string.

`numeric_prefix`

Se índices numéricos são usados no array base e esse parâmetro for fornecido, o número índice dos elementos será prefixado com seu valor, apenas no array base.

Isso é feito para permitir nomes de variáveis aceitáveis quando os dados forem decodificados pelo PHP ou por outra aplicação CGI posteriormente.

`arg_separator`

O argumento separador. Se não estiver definido ou `null`, arg_separator.output é usado para separar argumentos.

`encoding_type`

Por padrão, `PHP_QUERY_RFC1738`.

Se `encoding_type` é `PHP_QUERY_RFC1738`, então a codificação é executada por » RFC 1738 e o media type `application/x-www-form-urlencoded`, o que implica que os espaços são codificados com o sinal de mais(`+`).

Se `encoding_type` é `PHP_QUERY_RFC3986`, então a codificação é executada de acordo com » RFC 3986, e os espaços serão codificados em porcentagem (`%20`).

Valor Retornado

Retorna um string no formato de uma URL.

Registro de Alterações

Versão	Descrição
8.4.0	Anteriormente ao PHP 8.4.0, a propriedades BackedEnum de `data` eram convertidas para objetos, em vez de seus equivalentes escalares.
8.0.0	`arg_separator` agora é anulável.

Examples

Exemplo #1 Exemplo de uso de **http_build_query()**

```
<?php
$data = array(
    'foo' => 'bar',
```

```
    'baz' => 'boom',
    'cow' => 'milk',
    'null' => null,
    'php' => 'hypertext processor'
);

echo http_build_query($data) . "\n";
echo http_build_query($data, '', '&amp;');

?>
```

O exemplo acima produzirá:

```
foo=bar&baz=boom&cow=milk&php=hypertext+processor
foo=bar&amp;baz=boom&amp;cow=milk&amp;php=hypertext+processor
```

Exemplo #2 http_build_query() com elementos indexados numericamente.

```
<?php
$data = array('foo', 'bar', 'baz', null, 'boom', 'cow' => 'milk', 'php' => 'hypertext processor');

echo http_build_query($data) . "\n";
echo http_build_query($data, 'myvar_');
?>
```

O exemplo acima produzirá:

```
0=foo&1=bar&2=baz&4=boom&cow=milk&php=hypertext+processor
myvar_0=foo&myvar_1=bar&myvar_2=baz&myvar_4=boom&cow=milk&php=hypertext+processor
```

Exemplo #3 http_build_query() com arrays complexos

```
<?php
$data = array(
    'user' => array(
        'name' => 'Bob Smith',
        'age'  => 47,
        'sex'  => 'M',
        'dob'  => '5/12/1956'
    ),
    'pastimes' => array('golf', 'opera', 'poker', 'rap'),
    'children' => array(
        'bobby' => array('age'=>12, 'sex'=>'M'),
        'sally' => array('age'=>8, 'sex'=>'F')
    ),
    'CEO'
);

echo http_build_query($data, 'flags_');
?>
```

O exemplo acima produzirá: (quebra de linhas para legibilidade)

```
user%5Bname%5D=Bob+Smith&user%5Bage%5D=47&user%5Bsex%5D=M&
user%5Bdob%5D=5%2F12%2F1956&pastimes%5B0%5D=golf&pastimes%5B1%5D=opera&
pastimes%5B2%5D=poker&pastimes%5B3%5D=rap&children%5Bbobby%5D%5Bage%5D=12&
```

```
children%5Bbobby%5D%5Bsex%5D=M&children%5Bsally%5D%5Bage%5D=8&
children%5Bsally%5D%5Bsex%5D=F&flags_0=CEO
```

> **Nota** :
>
> Apenas o elemento indexado numericamente no array base "CEO" recebeu um prefixo. Os outros índices numéricos, encontrados abaixo de "pastimes", não requerem uma string de prefixo para serem nomes de variáveis aceitáveis.

Exemplo #4 Usando http_build_query() com um objeto

```php
<?php
class parentClass {
    public    $pub      = 'publicParent';
    protected $prot     = 'protectedParent';
    private   $priv     = 'privateParent';
    public    $pub_bar  = null;
    protected $prot_bar = null;
    private   $priv_bar = null;

    public function __construct(){
        $this->pub_bar  = new childClass();
        $this->prot_bar = new childClass();
        $this->priv_bar = new childClass();
    }
}

class childClass {
    public    $pub  = 'publicChild';
    protected $prot = 'protectedChild';
    private   $priv = 'privateChild';
}

$parent = new parentClass();

echo http_build_query($parent);
?>
```

O exemplo acima produzirá:

```
pub=publicParent&pub_bar%5Bpub%5D=publicChild
```

Exemplo #5 Usando http_build_query() com objetos contendo __toString()

```php
<?php
class Foo {
    public $publicProperty = 'visible';

    public function __toString() {
        return "bar";
    }
}

$params = array(
    'a' => 'b',
    'foo' => new Foo()
);

// Sem conversão, http_build_query lê as propriedades públicas
echo http_build_query($params) . "\n";
```

```php
// Com conversão explícita, http_build_query usa o output de __toString()
$params['foo'] = (string) new Foo();
echo http_build_query($params) . "\n";
?>
```

O exemplo acima produzirá:

```
a=b&foo%5BpublicProperty%5D=visible
a=b&foo=bar
```

Veja também

- parse_str() - Interpreta uma string como uma requisição URL
- parse_url() - Analisa uma URL e retorna seus componentes
- urlencode() - Codifica uma string no formato URL
- array_walk() - Aplica uma determinada função em cada elemento de um array

parse_url

parse_url

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_url — Analisa uma URL e retorna seus componentes

Descrição

function **parse_url**(string `$url`, int `$component = -1`): int|string|array|null|false

Esta função analisa uma URL e retorna um array associativo contendo cada um dos vários componentes da URL presentes. Os valores dos elementos do array *não* são decodificados como URL.

Esta função **não** se destina a validar o URL fornecido, apenas o divide nas partes listadas abaixo. URLs parciais e inválidos também são aceitos. **parse_url()** faz o possível para analisá-los corretamente.

> **Cuidado**
>
> Esta função não segue nenhum padrão estabelecido de URI ou URL. Ela retornará resultados incorretos ou sem sentido para URLs relativas ou malformadas. Mesmo para URLs válidas, o resultado pode diferir do de um analisador de URL diferente, já que existem vários padrões diferentes relacionados a URLs que visam casos de uso diferentes e que diferem em seus requisitos.
>
> O processamento de uma URL com analisadores que seguem diferentes padrões de URL é uma fonte comum de vulnerabilidades de segurança. Por exemplo, validar uma URL em relação a uma lista de permissões de nomes de host aceitáveis com o analisador A pode ser ineficaz quando a recuperação real do recurso usa o analisador B que extrai os nomes de host de forma diferente.
>
> As classes Uri\Rfc3986\Uri e Uri\WhatWg\Url seguem rigorosamente os padrões de URL RFC 3986 e WHATWG, respectivamente. É altamente recomendável usar essas classes para todo o código recém-escrito e migrar os usos existentes da função **parse_url()** para essas classes, a menos que o comportamento de **parse_url()** precise ser preservado por motivos de compatibilidade.

Parâmetros

`url`

A URL a ser analisada.

`component`

Deve ser especificada uma das constantes **PHP_URL_SCHEME**, **PHP_URL_HOST**, **PHP_URL_PORT**, **PHP_URL_USER**, **PHP_URL_PASS**, **PHP_URL_PATH**, **PHP_URL_QUERY** ou **PHP_URL_FRAGMENT** para recuperar apenas um componente específico da URL como uma string (exceto quando **PHP_URL_PORT** for informada, que no caso o valor de retorno será um int).

Valor Retornado

Em URLs gravemente malformados, **parse_url()** pode retornar `false`.

Se o parâmetro `component` for omitido, um array associativo será retornado. Pelo menos um elemento estará presente no array. As potenciais chaves dentro deste array são:

- *scheme* - ex.: `http`
- *host*
- *port*
- *user*
- *pass*
- *path*
- *query* - após o ponto de interrogação `?`
- *fragment* - após a cerquilha `#`

Se o parâmetro `component` for especificado, **parse_url()** retornará uma string (ou um int, no caso de `PHP_URL_PORT`) em vez de um array. Se o componente solicitado não existir no URL fornecido, `null` será retornado. A partir do PHP 8.0.0, **parse_url()** diferencia consultas e fragmentos ausentes de vazios:

```
http://example.com/foo → consulta = null, fragmento = null
http://example.com/foo? → consulta = "",   fragmento = null
http://example.com/foo# → consulta = null, fragmento = ""
http://example.com/foo?# → consulta = "",   fragmento = ""
```

Anteriormente, todos os casos resultavam em consulta e fragmento `null`.

Observe que os caracteres de controle (compare com ctype_cntrl()) nos componentes são substituídos por sublinhados (_).

Registro de Alterações

Versão	Descrição
8.0.0	**parse_url()** agora diferencia consultas e fragmentos vazios de ausentes.

Examples

Exemplo #1 Um exemplo de parse_url()

```php
<?php
$url = 'http://username:password@hostname:9090/path?arg=value#anchor';

var_dump(parse_url($url));
var_dump(parse_url($url, PHP_URL_SCHEME));
var_dump(parse_url($url, PHP_URL_USER));
var_dump(parse_url($url, PHP_URL_PASS));
var_dump(parse_url($url, PHP_URL_HOST));
var_dump(parse_url($url, PHP_URL_PORT));
var_dump(parse_url($url, PHP_URL_PATH));
var_dump(parse_url($url, PHP_URL_QUERY));
var_dump(parse_url($url, PHP_URL_FRAGMENT));
?>
```

O exemplo acima produzirá:

```
array(8) {
  ["scheme"]=>
  string(4) "http"
  ["host"]=>
  string(8) "hostname"
  ["port"]=>
  int(9090)
  ["user"]=>
  string(8) "username"
  ["pass"]=>
  string(8) "password"
  ["path"]=>
  string(5) "/path"
  ["query"]=>
  string(9) "arg=value"
  ["fragment"]=>
  string(6) "anchor"
}
string(4) "http"
string(8) "username"
string(8) "password"
string(8) "hostname"
int(9090)
string(5) "/path"
string(9) "arg=value"
string(6) "anchor"
```

Exemplo #2 Um exemplo de parse_url() com esquema ausente

```php
<?php
$url = '//www.example.com/path?googleguy=googley';

// Antes do PHP 5.4.7, o "path" seria "//www.example.com/path"
var_dump(parse_url($url));
?>
```

O exemplo acima produzirá:

```
array(3) {
  ["host"]=>
  string(15) "www.example.com"
  ["path"]=>
  string(5) "/path"
  ["query"]=>
  string(17) "googleguy=googley"
}
```

Notas

> **Nota** :
>
> Esta função destina-se especificamente à análise de URLs e não de URIs. No entanto, para cumprir os requisitos de compatibilidade com versões anteriores do PHP, ele abre uma exceção para o esquema `file://` onde barras triplas (`file:///...`) são permitidas. Para qualquer outro esquema isto é inválido.

Veja também

- pathinfo() - Retorna informações sobre um caminho de arquivo
- parse_str() - Interpreta uma string como uma requisição URL

- http_build_query() - Gera a string de consulta em formato URL
- dirname() - Retorna o caminho para o diretório pai
- basename() - Retorna a última parte do caminho
- » RFC 3986

rawurldecode

rawurldecode

(PHP 4, PHP 5, PHP 7, PHP 8)

rawurldecode — Decodifica uma URL

Descrição

function **rawurldecode**(string `$string`): string

Retorna uma string em que as sequências com o sinal porcento (%) seguida por dois dígitos hexadecimais são substituídas pelos caracteres correspondentes.

Parâmetros

`string`

A URL a ser decodificada.

Valor Retornado

Retorna a URL decodificada, como uma string.

Examples

Exemplo #1 Exemplo da rawurldecode()

```php
<?php

echo rawurldecode('foo%20bar%40baz'); // foo bar@baz

?>
```

Notas

> **Nota** :
>
> **rawurldecode()** não decodifica os sinais de adição ('+') em espaços. A função urldecode() faz isto.

Veja também

- rawurlencode() - Codifica uma URL de acordo com RFC 3986
- urldecode() - Decodifica uma URL codificada
- urlencode() - Codifica uma string no formato URL
- » RFC 3986

rawurlencode

rawurlencode

(PHP 4, PHP 5, PHP 7, PHP 8)

rawurlencode — Codifica uma URL de acordo com RFC 3986

Descrição

function **rawurlencode**(string `$string`): string

Codifica uma dada string de acordo com » RFC 3986.

Parâmetros

`string`

A URL a ser codificada.

Valor Retornado

Retorna uma string na qual todos os caracteres alfa-numericos com exceção de `-_.~` são substituídos com um sinal de porcento (`%`) seguido de dois digitos hexadecimais. Esta codificação esta descrita na » RFC 3986 para proteger caracteres de serem interpretados como delimitadores de URL, e para proteger URLs de serem embaralhadas por transmissão em meios com conversões de caracteres (como alguns sistemas de email).

Examples

Exemplo #1 Incluindo uma senha em uma URL de FTP

```php
<?php
echo '<a href="ftp://user:', rawurlencode('foo @+%/'),
     '@ftp.example.com/x.txt">';
?>
```

O exemplo acima produzirá:

```
<a href="ftp://user:foo%20%40%2B%25%2F@ftp.example.com/x.txt">
```

Ou, se você quiser passar informação no componente PATH_INFO se uma URL:

Exemplo #2 Exemplo 2 de rawurlencode()

```php
<?php
echo '<a href="http://example.com/department_list_script/',
    rawurlencode('sales and marketing/Miami'), '">';
?>
```

O exemplo acima produzirá:

```
<a href="http://example.com/department_list_script/sales%20and%20marketing%2FMiami">
```

Veja também

- rawurldecode() - Decodifica uma URL
- urldecode() - Decodifica uma URL codificada
- urlencode() - Codifica uma string no formato URL
- » RFC 3986

urldecode

urldecode

(PHP 4, PHP 5, PHP 7, PHP 8)

urldecode — Decodifica uma URL codificada

Descrição

function **urldecode**(string `$string`): string

Decodifica qualquer `%##` codificado na string. Os símbolos 'mais' ('+') são decodificados para um caractere de espaço.

Parâmetros

`string`

A string a ser decodificada.

Valor Retornado

Retorna a string decodificada.

Examples

Exemplo #1 Exemplo de urldecode()

```php
<?php
$query = "my=apples&are=green+and+red";

foreach (explode('&', $query) as $chunk) {
    $param = explode("=", $chunk);

    if ($param) {
        printf("O valor para o parâmetro \"%s\" é \"%s\"<br/>\n", urldecode($param[0]), urldecode($param[1]));
    }
}
?>
```

Notas

> **Aviso**
>
> As superglobais *$_GET* e *$_REQUEST* já são decodificadas. Usar **urldecode()** em um elemento *$_GET* ou *$_REQUEST* podem gerar resultados inesperados e perigosos.

Veja também

- urlencode() - Codifica uma string no formato URL
- rawurlencode() - Codifica uma URL de acordo com RFC 3986
- rawurldecode() - Decodifica uma URL
- » RFC 3986

urlencode

urlencode

(PHP 4, PHP 5, PHP 7, PHP 8)

urlencode — Codifica uma string no formato URL

Descrição

function **urlencode**(string `$string`): string

Esta função é conveniente ao codificar uma string a ser usada em uma parte de consulta de uma URL, como uma maneira conveniente de passar variáveis para a próxima página.

Parâmetros

`string`

A string a ser codificada.

Valor Retornado

Retorna uma string na qual todos os caracteres não alfanuméricos, exceto `-_.`, foram substituídos por um sinal de porcentagem (`%`) seguido por dois dígitos hexadecimais e espaços codificados como sinais de adição (`+`). Ela é codificada da mesma forma que os dados enviados por um formulário WWW são codificados, ou seja, da mesma forma que no tipo de mídia `application/x-www-form-urlencoded`. Isso difere da codificação » RFC 3986 (consulte rawurlencode()) porque, por razões históricas, os espaços são codificados como sinais de adição (+).

Examples

Exemplo #1 Exemplo de urlencode()

```php
<?php
$userinput = 'Dados123!@-_ +';
echo "Entrada do usuário: $userinput\n";
echo '<a href="mycgi?foo=', urlencode($userinput), '">';
?>
```

O exemplo acima produzirá:

```
Entrada do usuário: Dados123!@-_ +
<a href="mycgi?foo=Dados123%21%40-_+%2B">
```

Exemplo #2 Exemplo de urlencode() e htmlentities()

```php
<?php
$foo = 'Dados123!@-_ +';
$bar = "Diferente de $foo";
echo "foo: $foo\n";
echo "bar: $bar\n";
$query_string = 'foo=' . urlencode($foo) . '&bar=' . urlencode($bar);
echo '<a href="mycgi?' . htmlentities($query_string) . '">';
?>
```

O exemplo acima produzirá:

```
foo: Dados123!@-_ +
bar: Diferente de Dados123!@-_ +
<a href="mycgi?foo=Dados123%21%40-_+%2B&amp;bar=Diferente+de+Dados123%21%40-_+%2B">
```

Notas

> **Nota** :
>
> Deve-se ter cuidado com variáveis que possam corresponder a entidades HTML. Coisas como &, &copy e &pound são analisadas pelo navegador e a entidade real é usada no lugar do nome da variável desejada. Este é um incômodo óbvio sobre o qual o W3C vem falando às pessoas há anos. A referência está aqui: » http://www.w3.org/TR/html4/appendix/notes.html#h-B.2.2.
>
> O PHP suporta a alteração do separador de argumentos para o ponto e vírgula sugerido pelo W3C por meio da diretiva INI arg_separator. Infelizmente, a maioria dos agentes de usuário não envia dados de formulário neste formato separado por ponto e vírgula. Uma maneira mais portável de contornar isso é usar & em vez de & como separador. Não é necessário alterar o arg_separator do PHP para isso. Deve ser mantido como &, mas as URLs devem simplesmente ser codificadas usando htmlentities() ou htmlspecialchars().

Veja também

- urldecode() - Decodifica uma URL codificada
- htmlentities() - Converte todos os caracteres aplicáveis em entidades HTML
- rawurlencode() - Codifica uma URL de acordo com RFC 3986
- rawurldecode() - Decodifica uma URL
- » RFC 3986

Network

Introdução

Fornece várias funções para rede.

Instalação/Configuração

Índice

Dependências

As funções checkdnsrr(), getmxrr() e dns_get_record() requerem o Bind no Linux.

Tipos de Recurso

Esta extensão define um recurso de ponteiro de arquivo retornado pelas funções fsockopen() e pfsockopen().

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

Opções em `flags` disponíveis para openlog()

`LOG_CONS` (int)

Se houver um erro ao enviar dados para o registrador do sistema, grava diretamente no console do sistema.

`LOG_NDELAY` (int)

Abre a conexão com o registrador imediatamente.

`LOG_ODELAY` (int)

Atrasa a abertura da conexão até que a primeira mensagem seja registrada. Este é o comportamento padrão.

`LOG_NOWAIT` (int)

`LOG_PERROR` (int)

Além disso, registra mensagens em `STDERR`.

`LOG_PID` (int)

Inclui o PID com cada mensagem de log.

Opções de `facility` para openlog()

`LOG_AUTH` (int)

Para mensagens de segurança/autorização.

> **Nota** :
>
> Use `LOG_AUTHPRIV` quando disponível.

`LOG_AUTHPRIV` (int)

Para mensagens privadas de segurança/autorização.

`LOG_CRON` (int)

Para mensagens do daemon de relógio. Por exemplo **cron** ou **at**.

`LOG_DAEMON` (int)

Para mensagens do daemon do sistema.

`LOG_KERN` (int)

Para mensagens do kernel.

`LOG_LOCAL0` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL1` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL2` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL3` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL4` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL5` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL6` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LOCAL7` (int)

Reservado para uso local.

> **Aviso**
>
> Não disponível no Windows.

`LOG_LPR` (int)

Para mensagens do subsistema de impressora de linha.

`LOG_MAIL` (int)

Para mensagens do subsistema de correio.

`LOG_NEWS` (int)

Para mensagens do subsistema de notícias da USENET.

`LOG_SYSLOG` (int)

Para mensagens geradas internamente por **syslogd**.

`LOG_USER` (int)

Para mensagens genéricas no nível do usuário.

`LOG_UUCP` (int)

Para mensagens do subsistema UUCP.

Opções de `priority` para syslog()

As constantes de prioridade são listadas desde urgentes até mensagens de depuração.

`LOG_EMERG` (int)

Emergência, o sistema está inutilizável. Isso corresponde a uma condição de pânico. Geralmente transmitida para todos os processos.

`LOG_ALERT` (int)

Alerta, é necessária ação imediata. Por exemplo, um banco de dados do sistema corrompido.

`LOG_CRIT` (int)

Crítico, é necessária ação. Por exemplo, erros de dispositivo rígido.

`LOG_ERR` (int)

Mensagens de erros.

`LOG_WARNING` (int)

Mensagens de alerta.

`LOG_NOTICE` (int)

Mensagens de aviso correspondem a condições que não são condições de erro, mas que podem exigir tratamento especial.

`LOG_INFO` (int)

Mensagens informativas.

`LOG_DEBUG` (int)

Mensagens de depuração que contêm informações normalmente úteis apenas ao depurar um programa.

Opções de `type` para dns_get_record()

`DNS_ANY` (int)

Qualquer registro de recurso. Na maioria dos sistemas isso retorna todos os registros de recursos, porém devido a excentricidades no desempenho do `libresolv` entre plataformas isso não é garantido.

O `DNS_ALL` mais lento coletará todos os registros de forma mais confiável.

`DNS_ALL` (int)

Consulta iterativamente o servidor de nomes para cada tipo de registro disponível.

`DNS_A` (int)

Recurso de endereço IPv4.

`DNS_AAAA` (int)

Recurso de endereço IPv6.

`DNS_A6` (int)

Definido como parte do IPv6 inicial, mas rebaixado para histórico pela » RFC 6563.

`DNS_CAA` (int)

Recurso de autorização da autoridade de certificação. Disponível a partir do PHP 7.0.16 e 7.1.2.

> **Aviso**
>
> Não disponível no Windows.

`DNS_CNAME` (int)

Recurso de Alias (Canonical Name).

`DNS_HINFO` (int)

Recurso de informações do host. Para obter mais explicações e significados desses valores, visite a página da IANA em » Nomes de sistemas operacionais.

`DNS_MX` (int)

Recurso de troca de correio.

`DNS_NAPTR` (int)

Ponteiro de autoridade de nomes.

`DNS_NS` (int)

Recurso de servidor de nomes autoritativo.

`DNS_PTR` (int)

Recurso de ponteiro.

`DNS_SOA` (int)

Início do recurso de autoridade.

`DNS_SRV` (int)

Localizador de serviço.

`DNS_TXT` (int)

Recurso de texto.

Funções de Network

Índice

checkdnsrr

checkdnsrr

(PHP 4, PHP 5, PHP 7, PHP 8)

checkdnsrr — Verifica os registros DNS correspondentes para o nome do host ou endereço IP informado

Descrição

function **checkdnsrr**(string `$hostname`, string `$type` = "MX"): bool

Pesquisa o DNS para os registros do tipo informado em `type` correspondentes ao nome do host definido em `hostname`.

Parâmetros

`hostname`

`hostname` pode ser o endereço IP em notação quádrupla separada por pontos ou o nome do host.

`type`

`type` pode ser: A, MX, NS, SOA, PTR, CNAME, AAAA, A6, SRV, NAPTR, TXT ou ANY.

Valor Retornado

Retorna **`true`** se o registro for encontrado; retorna **`false`** se não forem encontrados registros ou se um erro ocorrer.

Notas

> **Nota** :
>
> Para compatibilidade com Windows antes desta implementação, deve ser usada a classe » Net_DNS do » PEAR.

Veja também

- dns_get_record() - Busca registros de recursos DNS associados a um nome de host
- getmxrr() - Obtém registros MX correspondentes a um determinado nome de host da Internet
- gethostbyaddr() - Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido
- gethostbyname() - Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet
- gethostbynamel() - Obtém uma lista de endereços IPv4 correspondentes a um determinado nome de host da Internet
- a página named(8) do manual

closelog

closelog

(PHP 4, PHP 5, PHP 7, PHP 8)

closelog — Fecha conexão com o registrador do sistema

Descrição

function **closelog**(): true

closelog() fecha o descritor que está sendo usado para gravar no registrador do sistema. O uso de **closelog()** é opcional.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna **`true`**.

Veja também

- syslog() - Gera uma mensagem de registro do sistema
- openlog() - Abre conexão com o registrador do sistema

dns_check_record

dns_check_record

(PHP 5, PHP 7, PHP 8)

dns_check_record — Sinônimo de checkdnsrr()

Descrição

Esta função é um apelido para: checkdnsrr().

dns_get_mx

dns_get_mx

(PHP 5, PHP 7, PHP 8)

dns_get_mx — Sinônimo de getmxrr()

Descrição

Esta função é um apelido para: getmxrr().

dns_get_record

dns_get_record

(PHP 5, PHP 7, PHP 8)

dns_get_record — Busca registros de recursos DNS associados a um nome de host

Descrição

```
function dns_get_record(
    string $hostname,
    int $type = DNS_ANY,
    array &$authoritative_name_servers = null,
    array &$additional_records = null,
    bool $raw = false
): array|false
```

Busca registros de recursos DNS associados ao nome do host informado no parâmetro `hostname`.

Parâmetros

`hostname`

`hostname` deve ser um nome de host DNS válido, como "`www.example.com`". Pesquisas reversas podem ser geradas usando a notação `in-addr.arpa`, mas gethostbyaddr() é mais adequada para a maioria das pesquisas reversas.

> **Nota** :
>
> De acordo com os padrões DNS, os endereços de e-mail são fornecidos no formato `user.host` (por exemplo: `hostmaster.example.com` em oposição a `hostmaster@example.com`). Certifique-se de verificar este valor e modificá-lo se necessário antes de usá-lo com funções como mail().

`type`

Por padrão, **dns_get_record()** procurará quaisquer registros de recursos associados ao `hostname`. Para limitar a consulta, use uma das constantes `DNS_*`

`authoritative_name_servers`

Passado por referência e, se fornecido, será preenchido com Registros de Recursos para os *Servidores de Nomes Autoritativos*.

`additional_records`

Passado por referência e, se fornecido, será preenchido com quaisquer *Registros Adicionais*.

`raw`

O parâmetro `type` será interpretado como um ID de tipo DNS bruto (as constantes `DNS_*` não podem ser usadas). O valor de retorno conterá uma chave `data`, que precisa ser analisada manualmente.

Valor Retornado

Esta função retorna um array de arrays associativos, ou `false` em caso de falha. Cada array associativo contém, *no mínimo*, as seguintes chaves:

Atributos DNS Básicos

Atributo	Significado
host	O registro no namespace DNS ao qual o restante dos dados associados se refere.
class	**dns_get_record()** retorna apenas registros de classes da Internet e como tal este parâmetro sempre retornará `IN`.
type	String contendo o tipo de registro. Atributos adicionais também estarão contidos no array resultante, dependendo do valor do tipo. Veja a tabela abaixo.
ttl	`"Time To Live"` (tempo de vida) restante para este registro. Isto *não* será igual ao TTL original do registro, mas será igual ao TTL original menos qualquer período de tempo decorrido desde que o servidor de nomes autoritativo foi consultado.

Outras chaves nos arrays associativos dependentes de `type`

Tipo	Colunas Extras
`A`	`ip`: Endereços IPv4 em notação decimal com pontos.
`MX`	`pri`: Prioridade do trocador de mensagens. Números mais baixos indicam maior prioridade. `target`: FQDN do trocador de correio. Consulte também dns_get_mx().
`CNAME`	`target`: FQDN do local no namespace DNS ao qual o registro tem alias.
`NS`	`target`: FQDN do servidor de nomes que tem autoridade para este nome de host.
`PTR`	`target`: Local no namespace DNS para o qual esse registro aponta.
`TXT`	`txt`:Dados de string arbitrários associados a este registro.
`HINFO`	`cpu`: Número IANA que designa a CPU da máquina referenciada por este registro. `os`: Número IANA que designa o sistema operacional na máquina referenciada por este registro. Consulte os » Nomes de Sistemas Operacionais (`Operating System Names`) da IANA para o significado desses valores.
`CAA`	`flags`: Um campo de bits de um byte; atualmente apenas o bit 0 está definido, significando 'crítico'; outros bits são reservados e devem ser ignorados. `tag`: O nome da tag CAA (sequência alfanumérica ASCII). `value`: O valor da tag CAA (string binária, pode usar subformatos). Para obter informações adicionais, consulte: » RFC 6844
`SOA`	`mname`: FQDN da máquina da qual os registros de recursos foram originados. `rname`: Endereço de e-mail do contato administrativo deste domínio. `serial`: Número de série desta revisão do domínio solicitado. `refresh`: Intervalo de atualização (segundos) que os servidores de nomes secundários devem usar ao atualizar cópias remotas deste domínio. `retry`: Tempo (segundos) de espera após uma atualização com falha antes de fazer uma segunda tentativa. `expire`: Período máximo de tempo (segundos) que um servidor DNS secundário deve reter cópias remotas dos dados da zona sem uma atualização bem-sucedida antes de descartar. `minimum-ttl`: Período mínimo de tempo (segundos) que um cliente pode continuar a usar uma resolução DNS antes de solicitar uma nova resolução do servidor. Pode ser substituído por registros de recursos individuais.
`AAAA`	`ipv6`: Endereço IPv6
`A6`	`masklen`: Comprimento (em bits) a ser herdado do destino especificado por `chain`. `ipv6`: Endereço para este registro específico ser mesclado com `chain`. `chain`: Registro pai para mesclar com dados `ipv6`.
`SRV`	`pri`: (Prioridade) as prioridades mais baixas devem ser usadas primeiro. `weight`: Classificação para ponderar quais dos `targets` comumente priorizados devem ser escolhidos aleatoriamente. `target` e `port`: nome do host e porta onde o serviço solicitado pode ser encontrado. Para obter informações adicionais, consulte: » RFC 2782.
`NAPTR`	`order` e `pref`: Equivalente a `pri` e `weight` acima. `flags`, `services`, `regex` e `replacement`: Parâmetros como definidos pela » RFC 2915.

Registro de Alterações

Versão	Descrição
7.0.16, 7.1.2	Adicionado suporte para tipo de registro CAA.

Examples

Exemplo #1 Usando dns_get_record()

```php
<?php
$result = dns_get_record("php.net");
print_r($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => MX
            [pri] => 5
            [target] => pair2.php.net
            [class] => IN
            [ttl] => 6765
        )

    [1] => Array
        (
            [host] => php.net
            [type] => A
            [ip] => 64.246.30.37
            [class] => IN
            [ttl] => 8125
        )

)
```

Exemplo #2 Usando dns_get_record() e DNS_ANY

Como é muito comum pesquisar o endereço IP de um servidor de e-mail depois que o registro MX tenha sido resolvido, **dns_get_record()** também retorna um array em `additional_records` que contém registros associados. `authoritative_name_servers` também é retornado contendo uma lista de servidores de nomes autoritativos.

```php
<?php
/* Solicita o registro "ANY" para php.net
   e cria arrays $authns e $addtl
   contendo uma lista de servidores de nomes e
   quaisquer registros adicionais que os
   acompanham */
$result = dns_get_record("php.net", DNS_ANY, $authns, $addtl);
echo "Resultado = ";
print_r($result);
echo "Servidor de Nomes Autoritativo = ";
print_r($authns);
echo "Adicional = ";
print_r($addtl);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Resultado = Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => MX
            [pri] => 5
            [target] => pair2.php.net
            [class] => IN
```

```
            [ttl] => 6765
        )

    [1] => Array
        (
            [host] => php.net
            [type] => A
            [ip] => 64.246.30.37
            [class] => IN
            [ttl] => 8125
        )

)
Servidor de Nomes Autoritativo = Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => remote1.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [1] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => remote2.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [2] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => ns1.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [3] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => ns2.easydns.com
            [class] => IN
            [ttl] => 10722
        )

)
Adicional = Array
(
    [0] => Array
        (
            [host] => pair2.php.net
            [type] => A
            [ip] => 216.92.131.5
            [class] => IN
            [ttl] => 6766
        )

    [1] => Array
        (
            [host] => remote1.easydns.com
            [type] => A
            [ip] => 64.39.29.212
            [class] => IN
            [ttl] => 100384
        )
```

```
    [2] => Array
        (
            [host] => remote2.easydns.com
            [type] => A
            [ip] => 212.100.224.80
            [class] => IN
            [ttl] => 81241
        )

    [3] => Array
        (
            [host] => ns1.easydns.com
            [type] => A
            [ip] => 216.220.40.243
            [class] => IN
            [ttl] => 81241
        )

    [4] => Array
        (
            [host] => ns2.easydns.com
            [type] => A
            [ip] => 216.220.40.244
            [class] => IN
            [ttl] => 81241
        )

)
```

Veja também

- dns_get_mx() - Sinônimo de getmxrr
- dns_check_record() - Sinônimo de checkdnsrr

fsockopen

fsockopen

(PHP 4, PHP 5, PHP 7, PHP 8)

fsockopen — Abre uma conexão socket de domínio Unix ou de Internet

Descrição

```
function fsockopen(
    string $hostname,
    int $port = -1,
    int &$error_code = null,
    string &$error_message = null,
    ?float $timeout = null
): resource|false
```

Inicia uma conexão socket para o recurso especificado em `hostname`.

O PHP suporta destinos na Internet ou domínios Unix, conforme descrito em Lista de Transportes via Soquete Suportados. Uma lista dos transportes suportados pode ser encontrada utilizando-se a função stream_get_transports().

Por padrão, o socket será aberto em modo de bloqueio. Pode-se mudar para o modo de não-bloqueio utilizando-se a função stream_set_blocking().

A função stream_socket_client() é similar mas fornece um número maior de opções, incluindo conexão não bloqueada e a habilidade de fornecer um contexto de fluxo.

Parâmetros

`hostname`

Se o suporte Open SSL estiver instalado, pode-se prefixar o `hostname` com `ssl://` ou `tls://` para utilizar uma conexão SSL ou TLS sobre TCP/IP para conectar-se ao servidor remoto.

`port`

O número da porta. Pode ser omitido e ignorado com `-1` para transportes que não usam portas, como `unix://`.

`error_code`

Se informado, guarda o número do erro do sistema que aconteceu na chamada de `connect()` no nível do sistema.

Se o valor retornado em `error_code` for `0` e a função retornou **`false`**, é uma indicação que o erro aconteceu antes da chamada a `connect()`. A causa mais provável é um problema na inicialização do socket.

`error_message`

A mensagem de erro como uma string.

`timeout`

Limite de tempo de conexão, em segundos. Quando igual a **`null`**, a configuração default_socket_timeout *php.ini* é usada.

> **Nota** :
>
> Se for necessário um limite de tempo para ler/escrever dados através do socket, use stream_set_timeout(), já que o parâmetro `timeout` de **fsockopen()** somente é aplicável enquanto o socket estiver conectando.

Valor Retornado

fsockopen() retorna um ponteiro de arquivo que pode ser usado em em conjunto com outras funções de arquivo (como fgets(), fgetss(), fwrite(), fclose() e feof()). Se a chamada falhar, ela retornará **`false`**.

Erros/Exceções

Dispara um **`E_WARNING`** se `hostname` não for um domínio válido.

Registro de Alterações

Versão	Descrição
8.0.0	`timeout` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de **fsockopen()**

```php
<?php
$fp = fsockopen("www.example.com", 80, $errno, $errstr, 30);
if (!$fp) {
    echo "$errstr ($errno)<br />\n";
} else {
    $out = "GET / HTTP/1.1\r\n";
    $out .= "Host: www.example.com\r\n";
    $out .= "Connection: Close\r\n\r\n";
    fwrite($fp, $out);
    while (!feof($fp)) {
        echo fgets($fp, 128);
    }
    fclose($fp);
}
?>
```

Exemplo #2 Utilizando conexão UDP

O exemplo abaixo mostra como obter a data e o horário do serviço UDP (porta 13) da própria máquina.

```php
<?php
$fp = fsockopen("udp://127.0.0.1", 13, $errno, $errstr);
if (!$fp) {
```

```
    echo "ERRO: $errno - $errstr<br />\n";
} else {
    fwrite($fp, "\n");
    echo fread($fp, 26);
    fclose($fp);
}
?>
```

Notas

> **Nota** :
>
> Dependedendo do ambiente, o domínio Unix ou o limite opcional de tempo de conexão podem não estar disponíveis.

> **Aviso**
>
> Sockets UDP algumas vezes irão parecer que foram abertos sem erro, mesmo que o servidor remoto seja inalcançável. O erro só se tornará aparente quando dados são lidos ou escritos no socket. A razão disso é porque o UDP é um protocolo "sem conexão", o que significa que o sistema operacional não tenta estabelecer uma ligação para o socket até que ele realmente precise enviar ou receber dados.

> **Nota** :
>
> Ao especificar um endereço IPv6 numérico (por exemplo, `fe80::1`), o IP deve ser colocado entre colchetes — por exemplo, `tcp://[fe80::1]:80`.

Veja também

- pfsockopen() - Abra uma conexão persistente de soquete de domínio Unix ou de Internet
- stream_socket_client() - Abre conexão de socket de domínio Internet ou Unix
- stream_set_blocking() - Definir modo de bloqueio/não-bloqueio em um fluxo
- stream_set_timeout() - Define o limite de tempo em um fluxo
- fgets() - Lê uma linha de um ponteiro de arquivo
- fgetss() - Ler uma linha de um ponteiro de arquivo e retira as tags HTML
- fwrite() - Escrita binary-safe em arquivos
- fclose() - Fecha um ponteiro de arquivo aberto
- feof() - Testa pelo fim-de-arquivo em um ponteiro de arquivo
- socket_connect() - Inicia uma conexão em um soquete
- A extensão Curl

gethostbyaddr

gethostbyaddr

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbyaddr — Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido

Descrição

function **gethostbyaddr**(string `$ip`): string|false

Retorna o nome do servidor de Internet especificado por `ip`.

Parâmetros

`ip`

O endereço IP do host.

Valor Retornado

Retorna o nome do servidor em caso de sucesso, o `ip` não modificado em caso de falha, ou **`false`** em caso de entrada mal formatada.

Examples

Exemplo #1 Um exemplo simples de gethostbyaddr()

```php
<?php
$hostname = gethostbyaddr($_SERVER['REMOTE_ADDR']);

echo $hostname;
?>
```

Veja também

- gethostbyname() - Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet
- gethostbynamel() - Obtém uma lista de endereços IPv4 correspondentes a um determinado nome de host da Internet

gethostbyname

gethostbyname

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbyname — Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet

Descrição

function **gethostbyname**(string `$hostname`): string

Retorna o endereço IPv4 do host da Internet especificado por `hostname`.

Parâmetros

`hostname`

O nome do host.

Valor Retornado

Retorna o endereço IPv4 ou uma string contendo o endereço não modificado `hostname` em caso de falha.

Examples

Exemplo #1 Um exemplo simples de gethostbyname()

```php
<?php
$ip = gethostbyname('www.example.com');

echo $ip;
?>
```

Veja também

- gethostbyaddr() - Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido
- gethostbynamel() - Obtém uma lista de endereços IPv4 correspondentes a um determinado nome de host da Internet
- inet_pton() - Converte um endereço IP legível em sua representação de endereço de Internet compactado
- inet_ntop() - Converte um endereço de Internet compactado em uma representação legível

gethostbynamel

gethostbynamel

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbynamel — Obtém uma lista de endereços IPv4 correspondentes a um determinado nome de host da Internet

Descrição

function **gethostbynamel**(string `$hostname`): array|false

Retorna uma lista de endereços IPv4 aos quais o host da Internet especificado por `hostname` é resolvido.

Parâmetros

`hostname`

O nome do host.

Valor Retornado

Retorna um array de endereços IPv4 ou `false` se o `hostname` não puder ser resolvido.

Examples

Exemplo #1 Exemplo de **gethostbynamel()**

```php
<?php
$hosts = gethostbynamel('www.example.com');
print_r($hosts);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 192.0.34.166
)
```

Veja também

- gethostbyname() - Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet
- gethostbyaddr() - Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido
- checkdnsrr() - Verifica os registros DNS correspondentes para o nome do host ou endereço IP informado
- getmxrr() - Obtém registros MX correspondentes a um determinado nome de host da Internet
- a página `named(8)` do manual

gethostname

gethostname

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gethostname — Obtém o nome do host

Descrição

function **gethostname**(): string|false

gethostname() obtém o nome de host padrão da máquina local.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string com o nome do host em caso de sucesso, caso contrário, `false` é retornado.

Examples

Exemplo #1 Um exemplo simples de **gethostname()**

```php
<?php
echo gethostname(); // pode exibir, por exemplo, "sandie"
?>
```

Veja também

- gethostbyname() - Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet
- gethostbyaddr() - Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido
- php_uname() - Retorna informações sobre o sistema operacional em que o PHP está sendo executado

getmxrr

getmxrr

(PHP 4, PHP 5, PHP 7, PHP 8)

getmxrr — Obtém registros MX correspondentes a um determinado nome de host da Internet

Descrição

function **getmxrr**(string `$hostname`, array `&$hosts`, array `&$weights = null`): bool

Pesquisa DNS por registros MX correspondentes a `hostname`.

Parâmetros

`hostname`

O nome do host da Internet.

`hosts`

Uma lista dos registros MX encontrados é colocada no array `hosts`.

`weights`

Se o array `weights` for fornecido, ele será preenchido com as informações de peso coletadas.

Valor Retornado

Retorna `true` se algum registro for encontrado; retorna `false` se nenhum registro for encontrado ou se ocorrer um erro.

Notas

> **Nota** :
>
> Esta função não deve ser usada para fins de verificação de endereço. Apenas os trocadores de e-mail encontrados no DNS são retornados, no entanto, de acordo com a » RFC 2821, quando nenhum trocador de e-mail estiver listado, o próprio `hostname` deverá ser usado como o único trocador de e-mail com prioridade `0`.

> **Nota** :
>
> Para compatibilidade com o Windows, antes dessa implementação, pode ser usada a classe » Net_DNS do » PEAR.

Veja também

- checkdnsrr() - Verifica os registros DNS correspondentes para o nome do host ou endereço IP informado
- dns_get_record() - Busca registros de recursos DNS associados a um nome de host
- gethostbyname() - Obtém o endereço IPv4 correspondente a um determinado nome de host da Internet
- gethostbynamel() - Obtém uma lista de endereços IPv4 correspondentes a um determinado nome de host da Internet
- gethostbyaddr() - Obtém nome do servidor de Internet correspondente ao endereço de IP fornecido
- A página `named(8)` do manual

getprotobyname

getprotobyname

(PHP 4, PHP 5, PHP 7, PHP 8)

getprotobyname — Obtém o número do protocolo associado ao nome do protocolo

Descrição

function **getprotobyname**(string `$protocol`): int|false

getprotobyname() retorna o número do protocolo associado ao protocolo `protocol` conforme */etc/protocols*.

Parâmetros

`protocol`

O nome do protocolo.

Valor Retornado

Retorna o número do protocolo, ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **getprotobyname()**

```php
<?php
$protocol = 'tcp';
$get_prot = getprotobyname($protocol);
if ($get_prot === FALSE) {
    echo 'Protocolo inválido';
} else {
    echo 'Protocolo número ' . $get_prot;
}
?>
```

Veja também

- getprotobynumber() - Obtém o nome do protocolo associado ao número do protocolo

getprotobynumber

getprotobynumber

(PHP 4, PHP 5, PHP 7, PHP 8)

getprotobynumber — Obtém o nome do protocolo associado ao número do protocolo

Descrição

function **getprotobynumber**(int `$protocol`): string|false

getprotobynumber() retorna o nome do protocolo associado ao protocolo `protocol` conforme */etc/protocols*.

Parâmetros

`protocol`

O número do protocolo.

Valor Retornado

Retorna o nome do protocolo como uma string, ou **`false`** em caso de falha.

Veja também

- getprotobyname() - Obtém o número do protocolo associado ao nome do protocolo

getservbyname

getservbyname

(PHP 4, PHP 5, PHP 7, PHP 8)

getservbyname — Obtém o número da porta associado a um serviço e protocolo de Internet

Descrição

function **getservbyname**(string `$service`, string `$protocol`): int|false

getservbyname() retorna a porta da Internet que corresponde ao serviço informado no parâmetro `service`, para o protocolo informado em `protocol`, especificado conforme */etc/services*.

Parâmetros

`service`

O nome do serviço de Internet, como uma string.

`protocol`

`protocol` é o protocolo: `"tcp"` ou `"udp"` (em letras minúsculas).

Valor Retornado

Retorna o número da porta ou `false` se o `service` ou o `protocol` não for encontrado.

Examples

Exemplo #1 Exemplo de getservbyname()

```php
<?php
$services = array('http', 'ftp', 'ssh', 'telnet', 'imap',
'smtp', 'nicname', 'gopher', 'finger', 'pop3', 'www');

foreach ($services as $service) {
    $port = getservbyname($service, 'tcp');
    echo $service . ": " . $port . "<br />\n";
}
?>
```

Veja também

- getservbyport() - Obtém o serviço de Internet que corresponde à porta e ao protocolo
- » http://www.iana.org/assignments/port-numbers para uma lista completa de números de portas.

getservbyport

getservbyport

(PHP 4, PHP 5, PHP 7, PHP 8)

getservbyport — Obtém o serviço de Internet que corresponde à porta e ao protocolo

Descrição

function **getservbyport**(int `$port`, string `$protocol`): string|false

getservbyport() retorna o serviço de Internet associado à porta informada no parâmetro `port`, para o protocolo informado em `protocol`, especificado conforme */etc/services*.

Parâmetros

`port`

O número da porta.

`protocol`

`protocol` pode ser: `"tcp"` ou `"udp"` (em minúsculas).

Valor Retornado

Retorna o nome do serviço de Internet como uma string, ou `false` em caso de falha.

Veja também

- getservbyname() - Obtém o número da porta associado a um serviço e protocolo de Internet

header

header

(PHP 4, PHP 5, PHP 7, PHP 8)

header — Envia um cabeçalho HTTP bruto

Descrição

function **header**(string `$header`, bool `$replace = true`, int `$response_code = 0`): void

header() é usada para enviar um cabeçalho HTTP bruto. Consulte a » especificação HTTP/1.1 para obter mais informações sobre cabeçalhos HTTP.

Lembre-se de que **header()** deve ser chamada antes de qualquer saída real ser enviada, seja por tags HTML normais, linhas em branco em um arquivo ou pelo PHP. É um erro muito comum ler código com include, require, funções ou outra função de acesso a arquivo, e ter espaços ou linhas vazias que são exibidas antes de **header()** ser chamada. O mesmo problema existe ao usar um único arquivo PHP/HTML.

```
<html>
<?php
/* Isso causará um erro. Observe a saída
 * acima, que é antes da chamada header() */
header('Location: http://www.example.com/');
exit;
?>
```

Parâmetros

`header`

A string do cabeçalho.

Existem dois casos especiais de chamadas de cabeçalho. O primeiro é um cabeçalho que começa com a string "`HTTP/`" (maiúsculas ou minúsculas, não importa), que será usado para descobrir o código de status HTTP a ser enviado. Por exemplo, se o Apache foi configurado para usar um script PHP para lidar com solicitações de arquivos ausentes (usando a diretiva `ErrorDocument`), é importante ter certeza de que o script gera o código de status adequado.

```
<?php
// Este exemplo ilustra o caso especial "HTTP/"
// Alternativas melhores em casos de uso típico incluem:
// 1. header($_SERVER["SERVER_PROTOCOL"] . " 404 Not Found");
//    (para substituir as mensagens de status HTTP para clientes que ainda estão usando HTTP/1.0)
// 2. http_response_code(404); (para usar a mensagem padrão)
header("HTTP/1.1 404 Not Found");
?>
```

O segunda caso especial é o cabeçalho "Location:". Ele não só envia o cabeçalho de volta para o navegador, mas também retorna um código de status `REDIRECT` (302) para o navegador a menos que o código de status `201` ou um `3xx` já tenha sido enviado.

```
<?php
header("Location: http://www.example.com/"); /* Redireciona o navegador */

/* Certificando que o restante do código não seja executado quando o redirecionamento é feito. */
exit;
?>
```

`replace`

O parâmetro opcional `replace` indica se o cabeçalho deve substituir um cabeçalho semelhante anterior ou adicionar um segundo cabeçalho do mesmo tipo. Por padrão ele irá substituir, mas se for passado `false` como segundo argumento podem ser forçados vários cabeçalhos do mesmo tipo. Por exemplo:

```
<?php
header('WWW-Authenticate: Negotiate');
header('WWW-Authenticate: NTLM', false);
?>
```

`response_code`

Força o código de resposta HTTP para o valor especificado. Observe que este parâmetro só tem efeito se o parâmetro `header` não estiver vazio.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Em caso de falha ao programar o envio do cabeçalho, **header()** emitirá um erro de nível `E_WARNING`.

Examples

Exemplo #1 Caixa de diálogo para salvar arquivo

Se quiser que o usuário seja solicitado a salvar os dados que estão sendo enviados, como um arquivo PDF gerado por exemplo, pode ser usado o cabeçalho » Content-Disposition para fornecer um nome de arquivo recomendado e forçar o navegador a exibir a caixa de diálogo para salvá-lo.

```
<?php
// A saída será um PDF
header('Content-Type: application/pdf');

// Será chamado de downloaded.pdf
header('Content-Disposition: attachment; filename="downloaded.pdf"');

// A fonte do PDF está em original.pdf
readfile('original.pdf');
?>
```

Exemplo #2 Diretivas de cache

Os scripts PHP geralmente geram conteúdo dinâmico que não deve ser armazenado em cache pelo navegador do cliente ou por qualquer cache proxy entre o servidor e o navegador do cliente. Muitos proxies e clientes podem ser forçados a desabilitar o cache com:

```
<?php
header("Cache-Control: no-cache, must-revalidate"); // HTTP/1.1
header("Expires: Sat, 03 Dec 2016 17:15:00 GMT"); // Data no passado
?>
```

> **Nota** :
>
> Pode acontecer das páginas não serem armazenadas em cache mesmo que todos os cabeçalhos acima não sejam gerados. Há uma série de opções que os usuários podem definir para seus navegadores que alteram seu comportamento padrão de cache. Ao enviar os cabeçalhos acima, devem ser substituídas quaisquer configurações que possam fazer com que a saída do script seja armazenada em cache.
>
> Além disso, session_cache_limiter() e a configuração `session.cache_limiter` podem ser usadas para gerar automaticamente os cabeçalhos corretos relacionados ao cache quando sessões estão sendo usadas.

Exemplo #3 Definindo um cookie

setcookie() fornece uma maneira conveniente de definir cookies. Para definir um cookie que inclui atributos que setcookie() não suporta, **header()** pode ser usada.

Por exemplo, o código a seguir define um cookie que inclui um atributo `Partitioned`.

```
<?php
header('Set-Cookie: name=value; Secure; Path=/; SameSite=None; Partitioned;');
?>
```

Notas

> **Nota** :

> Os cabeçalhos só serão acessíveis e enviados quando uma SAPI que os suporta estiver em uso.

> **Nota** :
>
> Para contornar esse problema, o buffer de saída pode ser usado, com a sobrecarga de toda a saída para o navegador sendo armazenada em buffer no servidor até seja enviada. Isso pode ser feito chamando ob_start() e ob_end_flush() no script, ou definindo a diretiva de configuração `output_buffering` no arquivo *php.ini* ou nos arquivos de configuração do servidor.

> **Nota** :
>
> A linha do cabeçalho de status HTTP sempre será a primeira enviada ao cliente, independentemente de a chamada a **header()** ser a primeira ou não. O status pode ser substituído chamando **header()** com uma nova linha de status a qualquer momento, a menos que os cabeçalhos HTTP já tenham sido enviados.

> **Nota** :
>
> A maioria dos clientes atuais aceita URI relativo como argumento para » Local:, mas alguns clientes mais antigos exigem um URI absoluto incluindo o esquema, nome do host e caminho absoluto. Geralmente, podem ser usados *$_SERVER['HTTP_HOST']*, *$_SERVER['PHP_SELF']* e dirname() para criar um URI absoluto a partir de um relativo:
>
> ```php
> <?php
> /* Redireciona para uma página diferente no diretório atual que foi solicitado */
> $host  = $_SERVER['HTTP_HOST'];
> $uri   = rtrim(dirname($_SERVER['PHP_SELF']), '/\\');
> $extra = 'minha_pagina.php';
> header("Location: http://$host$uri/$extra");
> exit;
> ?>
> ```

> **Nota** :
>
> O ID da sessão não é passado com o cabeçalho "Location:" mesmo se session.use_trans_sid estiver ativada. Deve ser passado manualmente usando a constante **SID**.

Veja também

- headers_sent() - Verifica se ou onde os cabeçalhos foram enviados
- setcookie() - Envia um cookie
- http_response_code() - Obtém ou define o código de resposta HTTP
- header_remove() - Remove cabeçalhos definidos anteriormente
- headers_list() - Retorna uma lista de cabeçalhos de resposta enviados (ou prontos para enviar)
- A seção sobre autenticação HTTP

header_register_callback

header_register_callback

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

header_register_callback — Chama uma função de cabeçalho

Descrição

function **header_register_callback**(callable `$callback`): bool

Registra uma função que será chamada quando o PHP começar a enviar a saída.

A função `callback` é executada logo após o PHP preparar todos os cabeçalhos a serem enviados, e antes de qualquer outra saída ser enviada, criando uma janela para manipular os cabeçalhos de saída antes de serem enviados.

> **Aviso**

> As funções de retorno não são empilhadas. Chamadas subsequentes a **header_register_callback()** vão desregistrar silenciosamente qualquer `callback` existente, independente de onde ela foi definida. Esteja ciente que dependências podem sobrescrever a função de retorno. A depuração de erros neste cenário pode ser difícil.

Parâmetros

`callback`

Função de retorno chamada logo antes dos cabeçalhos serem enviados. Não recebe parâmetros e o valor de retorno é ignorado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de header_register_callback()

```php
<?php

header('Content-Type: text/plain');
header('X-Test: foo');

function foo() {
 foreach (headers_list() as $header) {
   if (strpos($header, 'X-Powered-By:') !== false) {
     header_remove('X-Powered-By');
   }
   header_remove('X-Test');
 }
}

$result = header_register_callback('foo');
echo "a";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Content-Type: text/plain

a
```

Notas

header_register_callback() é executado quando os cabeçalhos estão prestes a ser enviados, portanto, qualquer saída desta função pode quebrar a saída.

> **Nota** :
>
> Os cabeçalhos só serão acessíveis e enviados quando uma SAPI que os suporta estiver em uso.

Veja também

- headers_list() - Retorna uma lista de cabeçalhos de resposta enviados (ou prontos para enviar)
- header_remove() - Remove cabeçalhos definidos anteriormente
- header() - Envia um cabeçalho HTTP bruto

header_remove

header_remove

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

header_remove — Remove cabeçalhos definidos anteriormente

Descrição

function **header_remove**(?string `$name` = `null`): void

Remove um cabeçalho HTTP previamente definido usando header().

Parâmetros

`name`

O nome do cabeçalho a ser removido. Quando igual a `null`, todos os cabeçalhos anteriormente definidos são removidos.

> **Nota** :
>
> Este parâmetro é insensível a maiúsculas/minúsculas.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`name` agora pode ser nulo.

Examples

Exemplo #1 Removendo um cabeçalho específico

```
<?php
header("X-Foo: Bar");
header("X-Bar: Baz");
header_remove("X-Foo");
?>
```

O exemplo acima produzirá algo semelhante a:

```
X-Bar: Baz
```

Exemplo #2 Removendo todos os cabeçalhos definidos anteriormente

```
<?php
header("X-Foo: Bar");
header("X-Bar: Baz");
header_remove();
?>
```

O exemplo acima produzirá algo semelhante a:

```
```

Notas

> **Cuidado**
>
> Esta função irá remover *todos* os cabeçalhos definidos pelo PHP, incluindo cabeçalhos de cookies, de sessão e `X-Powered-By`.

> **Nota** :
>
> Os cabeçalhos só serão acessíveis e enviados quando uma SAPI que os suporta estiver em uso.

Veja também

- header() - Envia um cabeçalho HTTP bruto
- headers_sent() - Verifica se ou onde os cabeçalhos foram enviados

headers_list

headers_list

(PHP 5, PHP 7, PHP 8)

headers_list — Retorna uma lista de cabeçalhos de resposta enviados (ou prontos para enviar)

Descrição

function **headers_list**(): array

headers_list() retornará uma lista de cabeçalhos a serem enviados ao navegador/cliente. Para determinar se esses cabeçalhos já foram ou não enviados, a função headers_sent() deve ser usada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array de cabeçalhos indexado numericamente.

Examples

Exemplo #1 Exemplo de uso de headers_list()

```php
<?php

/* setcookie() irá adicionar um cabeçalho de resposta por conta própria */
setcookie('foo', 'bar');

/* Define um cabeçalho de resposta personalizado
   Isso será ignorado pela maioria dos clientes */
header("Example-Test: foo");

/* Especifica conteúdo em texto simples na resposta */
header('Content-Type: text/plain; charset=UTF-8');

/* Quais cabeçalhos serão enviados? */
var_dump(headers_list());

?>
```

O exemplo acima produzirá algo semelhante a:

```
array(3) {
  [0]=>
  string(19) "Set-Cookie: foo=bar"
  [1]=>
  string(17) "Example-Test: foo"
  [2]=>
```

```
  string(39) "Content-Type: text/plain; charset=UTF-8"
}
```

Notas

> **Nota** :
>
> Os cabeçalhos só serão acessíveis e enviados quando uma SAPI que os suporta estiver em uso.

Veja também

- headers_sent() - Verifica se ou onde os cabeçalhos foram enviados
- header() - Envia um cabeçalho HTTP bruto
- setcookie() - Envia um cookie
- apache_response_headers() - Obtém todos os cabeçalhos da resposta HTTP
- http_response_code() - Obtém ou define o código de resposta HTTP

headers_sent

headers_sent

(PHP 4, PHP 5, PHP 7, PHP 8)

headers_sent — Verifica se ou onde os cabeçalhos foram enviados

Descrição

function **headers_sent**(string `&$filename = null`, int `&$line = null`): bool

Verifica se ou onde os cabeçalhos foram enviados.

Não é possível adicionar mais linhas de cabeçalho usando a função header() uma vez que o bloco de cabeçalho já tiver sido enviado. Usando esta função, é possível pelo menos evitar mensagens de erro relacionadas ao cabeçalho HTTP. Outra opção é usar Buffer de Saída.

Parâmetros

`filename`

Se os parâmetros opcionais `filename` e `line` forem informados, **headers_sent()** colocará o nome do arquivo fonte PHP e o número da linha onde a saída iniciou nas variáveis `filename` e `line`, respectivamente.

> **Nota** :
>
> Se a saída foi iniciada antes da execução do arquivo fonte PHP (por exemplo, devido a um erro de inicialização), o parâmetro `filename` será definido como uma string vazia.

`line`

O número da linha onde a saída iniciou.

Valor Retornado

headers_sent() retornará `false` se nenhum cabeçalho HTTP ainda tiver sido enviado, ou `true` caso contrário.

Examples

Exemplo #1 Exemplos de uso de headers_sent()

```php
<?php

// Se nenhum cabeçalho tiver sido enviado, envia um
if (!headers_sent()) {
    header('Location: http://www.example.com/');
```

```
    exit;
}

// Um exemplo usando os parâmetros opcionais de arquivo e linha
// Observe que $filename e $linenum são passados para uso posterior.
// Não devem ser atribuídos valores a eles antecipadamente.
if (!headers_sent($filename, $linenum)) {
    header('Location: http://www.example.com/');
    exit;

// Aqui é exibida uma mensagem de erro.
} else {

    echo "Cabeçalhos já enviados no arquivo $filename na linha $linenum\n" .
         "Não é possível redirecionar, favor clicar neste atalho: <a " .
         "href=\"http://www.example.com\">link</a>\n";
    exit;
}

?>
```

Notas

> **Nota** :
>
> Os cabeçalhos só serão acessíveis e enviados quando uma SAPI que os suporta estiver em uso.

Veja também

- ob_start() - Ativa o buffer de saída
- trigger_error() - Gera uma mensagem a nível de usuário de erro/alerta/nota
- headers_list() - Retorna uma lista de cabeçalhos de resposta enviados (ou prontos para enviar)
- Função header() - Envia um cabeçalho HTTP bruto para uma descrição mais detalhada sobre os assuntos envolvidos.

http_clear_last_response_headers

http_clear_last_response_headers

(PHP 8 >= 8.4.0)

http_clear_last_response_headers — Limpa os cabeçalhos de resposta HTTP armazenados

Descrição

function **http_clear_last_response_headers**(): void

Limpa os cabeçalhos de resposta HTTP que foram recebidos usando o empacotador HTTP.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- http_get_last_response_headers() - Recupera os últimos cabeçalhos de reposta HTTP

http_get_last_response_headers

http_get_last_response_headers

(PHP 8 >= 8.4.0)

http_get_last_response_headers — Recupera os últimos cabeçalhos de reposta HTTP

Descrição

function **http_get_last_response_headers**(): ?array

Retorna um array contendo os últimos cabeçalhos de resposta HTTP recebidos através do empacotador HTTP. Se não houver nenhum, retorna `null`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array indexado de cabeçalhos HTTP que foram recebidos ao usar o empacotador HTTP. Se não houver nenhum, retorna `null`.

Examples

Exemplo #1 Exemplo de http_get_last_response_headers()

Obtendo os cabeçalhos de exemplo.com.br

```php
<?php
file_get_contents("http://examplo.com.br");
var_dump(http_get_last_response_headers());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(14) {
  [0]=>
  string(15) "HTTP/1.1 200 OK"
  [1]=>
  string(20) "Accept-Ranges: bytes"
  [2]=>
  string(11) "Age: 326940"
  [3]=>
  string(29) "Cache-Control: max-age=604800"
  [4]=>
  string(38) "Content-Type: text/html; charset=UTF-8"
  [5]=>
  string(35) "Date: Mon, 11 Nov 2024 13:34:09 GMT"
  [6]=>
  string(23) "Etag: "3147526947+gzip""
  [7]=>
  string(38) "Expires: Mon, 18 Nov 2024 13:34:09 GMT"
  [8]=>
  string(44) "Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT"
  [9]=>
  string(24) "Server: ECAcc (nyd/D16C)"
  [10]=>
  string(21) "Vary: Accept-Encoding"
  [11]=>
  string(12) "X-Cache: HIT"
  [12]=>
  string(20) "Content-Length: 1256"
  [13]=>
  string(17) "Connection: close"
}
```

Veja também

- http_clear_last_response_headers() - Limpa os cabeçalhos de resposta HTTP armazenados

http_response_code

http_response_code

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

http_response_code — Obtém ou define o código de resposta HTTP

Descrição

function **http_response_code**(int `$response_code` = 0): int|bool

Obtém ou define o código de resposta HTTP.

Parâmetros

`response_code`

O opcional `response_code` definirá o código de resposta.

Valor Retornado

Se `response_code` for fornecido, o código de status anterior será retornado. Se `response_code` não for fornecido, o código de status atual será retornado. Ambos os valores terão como padrão um código de status `200` se usados em um ambiente de servidor web.

`false` será retornado se `response_code` não for fornecido e a função não for chamada em um ambiente de servidor web (como em uma aplicação CLI). `true` será retornado se `response_code` for fornecido e a função não for chamada em um ambiente de servidor web (mas somente quando nenhum status de resposta anterior tiver sido definido).

Examples

Exemplo #1 Usando http_response_code() em um ambiente de servidor web

```php
<?php

// Obtém o código de resposta atual e define um novo
var_dump(http_response_code(404));

// Obtém o novo código de resposta
var_dump(http_response_code());
?>
```

O exemplo acima produzirá:

```
int(200)
int(404)
```

Exemplo #2 Usando http_response_code() em um ambiente CLI

```php
<?php

// Obtém o código de resposta padrão atual
var_dump(http_response_code());

// Define um código de resposta
var_dump(http_response_code(201));

// Obtém o novo código de resposta
var_dump(http_response_code());
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
int(201)
```

Veja também

- header() - Envia um cabeçalho HTTP bruto
- headers_list() - Retorna uma lista de cabeçalhos de resposta enviados (ou prontos para enviar)

inet_ntop

inet_ntop

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

inet_ntop — Converte um endereço de Internet compactado em uma representação legível

Descrição

function **inet_ntop**(string `$ip`): string|false

Esta função converte um endereço IPv4 de 32 bits ou IPv6 de 128 bits (se o PHP foi construído com suporte IPv6 habilitado) em uma representação de string apropriada para a família de endereços.

Parâmetros

`ip`

Um endereço IPv4 de 32 bits ou IPv6 de 128 bits.

Valor Retornado

Retorna uma representação em string do endereço ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de inet_ntop()

```php
<?php
$packed = chr(127) . chr(0) . chr(0) . chr(1);
$expanded = inet_ntop($packed);

/* Exibe: 127.0.0.1 */
echo $expanded;

$packed = str_repeat(chr(0), 15) . chr(1);
$expanded = inet_ntop($packed);

/* Exibe: ::1 */
echo $expanded;
?>
```

Veja também

- long2ip() - Converte um endereço inteiro longo em uma string no formato padrão com pontos da Internet (IPv4)
- ip2long() - Converte uma string contendo um endereço com pontos do protocolo da Internet (IPv4) em um número inteiro longo
- inet_pton() - Converte um endereço IP legível em sua representação de endereço de Internet compactado

inet_pton

inet_pton

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

inet_pton — Converte um endereço IP legível em sua representação de endereço de Internet compactado

Descrição

function **inet_pton**(string `$ip`): string|false

Esta função converte um endereço IPv4 ou IPv6 legível (se o PHP foi construído com suporte IPv6 habilitado) em uma estrutura binária de 32 ou 128 bits apropriada para a família de endereços.

Parâmetros

`ip`

Um endereço IPv4 ou IPv6 legível.

Valor Retornado

Retorna a representação `in_addr` (endereço de Internet compactado) do `ip` fornecido, ou **`false`** se um `ip` sintaticamente inválido for fornecido (por exemplo, um endereço IPv4 sem pontos ou um endereço IPv6 sem dois pontos).

Examples

Exemplo #1 Exemplo de inet_pton()

```php
<?php
$in_addr = inet_pton('127.0.0.1');

$in6_addr = inet_pton('::1');
?>
```

Veja também

- ip2long() - Converte uma string contendo um endereço com pontos do protocolo da Internet (IPv4) em um número inteiro longo
- long2ip() - Converte um endereço inteiro longo em uma string no formato padrão com pontos da Internet (IPv4)
- inet_ntop() - Converte um endereço de Internet compactado em uma representação legível

ip2long

ip2long

(PHP 4, PHP 5, PHP 7, PHP 8)

ip2long — Converte uma string contendo um endereço com pontos do protocolo da Internet (IPv4) em um número inteiro longo

Descrição

function **ip2long**(string `$ip`): int|false

A função **ip2long()** gera uma representação inteira longa do endereço de rede da Internet IPv4 a partir de sua representação no formato padrão da Internet (string com pontos).

ip2long() também funcionará com endereços IP incompletos. Leia » http://ps-2.kev009.com/wisclibrary/aix52/usr/share/man/info/en_US/a_doc_lib/libs/commtrf2/inet_addr.htm para mais informações.

Parâmetros

`ip`

Um endereço no formato padrão.

Valor Retornado

Retorna o número inteiro longo ou **`false`** se `ip` for inválido.

Examples

Exemplo #1 Exemplo de ip2long()

```php
<?php
$ip = gethostbyname('www.example.com');
$out = "Os seguintes URLs são equivalentes:<br />\n";
$out .= 'http://www.example.com/, http://' . $ip . '/, e http://' . sprintf("%u", ip2long($ip)) . "/<br />\n";
echo $out;
?>
```

Exemplo #2 Exibindo um endereço IP

Este segundo exemplo mostra como exibir um endereço convertido com a função printf():

```php
<?php
$ip   = gethostbyname('www.example.com');
$long = ip2long($ip);

if ($long == -1 || $long === FALSE) {
    echo 'IP inválido, tente novamente';
} else {
    echo $ip   . "\n";            // 192.0.34.166
    echo $long . "\n";            // 3221234342 (-1073732954 em sistemas de 32 bits, devido ao estouro de número inteiro)
    printf("%u\n", ip2long($ip)); // 3221234342
}
?>
```

Notas

> **Nota** :
>
> Como o tipo int do PHP é com sinal e muitos endereços IP resultarão em números inteiros negativos em arquiteturas de 32 bits, é necessário usar o formatador "%u" de sprintf() ou printf() para obter a representação em string do endereço IP sem sinal.

> **Nota** :
>
> **ip2long()** retornará `-1` para o IP `255.255.255.255` em sistemas de 32 bits devido ao estouro do valor inteiro.

Veja também

- long2ip() - Converte um endereço inteiro longo em uma string no formato padrão com pontos da Internet (IPv4)
- sprintf() - Retona uma string formatada

long2ip

long2ip

(PHP 4, PHP 5, PHP 7, PHP 8)

long2ip — Converte um endereço inteiro longo em uma string no formato padrão com pontos da Internet (IPv4)

Descrição

function **long2ip**(int `$ip`): string

A função **long2ip()** gera um endereço de Internet em formato com pontos (ou seja: aaa.bbb.ccc.ddd) a partir da representação de inteiro longo.

Parâmetros

`ip`

Uma representação de endereço adequada em número inteiro longo.

Valor Retornado

Retorna o endereço IP da Internet como uma string.

Registro de Alterações

Versão	Descrição
8.4.0	O tipo do retorno foi alterado de string|false para string.
7.1.0	O tipo do parâmetro `ip` foi alterado de string para int.

Notas

> **Nota** :

> Em arquiteturas de 32 bits, converter representações inteiras de endereços IP de string para int não fornecerá resultados corretos para números que excedam `PHP_INT_MAX`.

Veja também

- ip2long() - Converte uma string contendo um endereço com pontos do protocolo da Internet (IPv4) em um número inteiro longo

net_get_interfaces

net_get_interfaces

(PHP 7 >= 7.3, PHP 8)

net_get_interfaces — Obtém interfaces de rede

Descrição

function **net_get_interfaces**(): array|false

Retorna uma enumeração de interfaces de rede (adaptadores) na máquina local.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array associativo onde a chave é o nome da interface e o valor é um array associativo de atributos de interface, ou `false` em caso de falha.

Cada array associativo de interface contém

Atributos de interface

Nome	Descrição
description	Valor de string opcional para descrição da interface. Somente Windows.
mac	Valor de string opcional para endereço MAC da interface. Somente Windows.
mtu	Valor inteiro para a unidade máxima de transmissão (MTU) da interface. Somente Windows.
unicast	Array de arrays associativos, consulte atributos Unicast abaixo.
up	Status booleano (ligado/desligado) para a interface.

Atributos Unicast

Nome	Descrição
flags	Valor inteiro.
family	Valor inteiro.
address	Valor string para o endereço em IPv4 ou IPv6.
netmask	Valor string para a máscara de rede em either IPv4 ou IPv6.

Erros/Exceções

Emite um `E_WARNING` em caso de falha ao obter informação de interface.

openlog

openlog

(PHP 4, PHP 5, PHP 7, PHP 8)

openlog — Abre conexão com o registrador do sistema

Descrição

function **openlog**(string `$prefix`, int `$flags`, int `$facility`): true

openlog() abre uma conexão com o registrador do sistema para um programa.

O uso de **openlog()** é opcional. Ela será automaticamente chamada por syslog() se necessário, e nesse caso o padrão de `prefix` será uma string vazia.

Parâmetros

`prefix`

O parâmetro `prefix` define uma string de prefixo que é adicionado a cada mensagem.

`flags`

Máscara de bits das seguinte constantes:

- `LOG_CONS`
- `LOG_NDELAY`
- `LOG_ODELAY`
- `LOG_NOWAIT`
- `LOG_PERROR`
- `LOG_PID`

`facility`

O argumento `facility` é usado para especificar que tipo de módulo está registrando a mensagem. Isto permite que o arquivo de configuração que mensagens provenientes de diferentes módulos serão tratadas diferentemente. Deve ser uma das constantes a seguir:

- `LOG_AUTH`
- `LOG_AUTHPRIV`
- `LOG_CRON`
- `LOG_DAEMON`
- `LOG_KERN`
- `LOG_LOCAL[0-7]`
- `LOG_LPR`
- `LOG_MAIL`
- `LOG_NEWS`
- `LOG_SYSLOG`
- `LOG_USER`
- `LOG_UUCP`

> **Nota** :
>
> Este parâmetro é ignorado no Windows.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	A função agora sempre retorna `true`. Anteriormente, retornava `false` em caso de falha.

Veja também

- syslog() - Gera uma mensagem de registro do sistema
- closelog() - Fecha conexão com o registrador do sistema

pfsockopen

pfsockopen

(PHP 4, PHP 5, PHP 7, PHP 8)

pfsockopen — Abra uma conexão persistente de soquete de domínio Unix ou de Internet

Descrição

```
function pfsockopen(
    string $hostname,
    int $port = -1,
    int &$error_code = null,
    string &$error_message = null,
    ?float $timeout = null
): resource|false
```

Esta função se comporta exatamente como fsockopen() com a diferença de que a conexão não é fechada após o término do script. É a versão persistente de fsockopen().

Parâmetros

Para obter informações sobre parâmetros, consulte a documentação da função fsockopen().

Valor Retornado

pfsockopen() retorna um ponteiro de arquivo que pode ser usado junto com outras funções de arquivo (como fgets(), fgetss(), fwrite(), fclose() e feof()), ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`timeout` agora pode ser nulo.

Veja também

- fsockopen() - Abre uma conexão socket de domínio Unix ou de Internet

request_parse_body

request_parse_body

(PHP 8 >= 8.4.0)

request_parse_body — Lê e analisa o corpor da requisição e retorna o resultado

Descrição

```
function request_parse_body(?array $options = null): array
```

Esta função lê o corpo da requisição e analisa-o de acordo com o cabeçalho `Content-Type`. Atualmente, dois tipos de conteúdo são suportados:

- `application/x-www-form-urlencoded`
- `multipart/form-data`

Esta função é usada primariamente para analisar requisições `multipart/form-data` com verbos HTTP diferentes de `POST` que não populam automaticamente as superglobais *$_POST* e *$_FILES*.

> **Cuidado**
>
> O corpo da requisição pode ser consumido apenas uma vez. **request_parse_body()** consome o corpo da requisição sem fazer buffer para o fluxo `php://input`. Por outro lado, se o corpo já tiver sido lido (ex.: por `php://input`), **request_parse_body()** retornará dados vazios.

Parâmetros

`options`

O parâmetro `options` aceita um array associativo para substituir as seguintes configurações globais do *php.ini* para analisar o corpo da requisição.

- `max_file_uploads`
- `max_input_vars`

- `max_multipart_body_parts`
- `post_max_size`
- `upload_max_filesize`

Valor Retornado

request_parse_body() retorna um array com o equivalente de *$_POST* no índice `0` e o de *$_FILES* no índice `1`.

Erros/Exceções

Quando o corpo da requisição for inválido, de acordo com o cabeçalho `Content-Type`, uma exceção RequestParseBodyException é lançada.

Uma exceção ValueError é lançada quando `options` contiver chaves inválidas ou valores inválidos para a chave correspondente.

Examples

Exemplo #1 Exemplo de request_parse_body()

```php
<?php
// Analisa o resultado e armazena-o nas superglobais $_POST e $_FILES.
[$_POST, $_FILES] = request_parse_body();
// Exibe o conteúdo de um arquivo transferido
echo file_get_contents($_FILES['file_name']['tmp_name']);
?>
```

Exemplo #2 Exemplo de request_parse_body() com opções personalizadas

```php
<?php
// form.php

assert_logged_in();

// Apenas para este formulário, um tamanho maior de envio é permitido.
[$_POST, $_FILES] = request_parse_body([
    'post_max_size' => '10M',
    'upload_max_filesize' => '10M',
]);

// Faz algo com os arquivos enviados.
?>
```

setcookie

setcookie

(PHP 4, PHP 5, PHP 7, PHP 8)

setcookie — Envia um cookie

Descrição

function **setcookie**(
  string `$name`,
  string `$value` = "",
  int `$expires_or_options` = 0,
  string `$path` = "",
  string `$domain` = "",
  bool `$secure` = **false**,
  bool `$httponly` = **false**
): bool

Assinatura alternativa disponível a partir do PHP 7.3.0 (sem suporte a parâmetros nomeados):

function **setcookie**(string `$name`, string `$value` = "", array `$options` = []): bool

A função **setcookie()** define um cookie para ser enviado juntamente com os cabeçalhos HTTP. Como outros cabeçalhos, os cookies devem ser enviados *antes* de qualquer saída do script (isso é uma restrição do protocolo). Isso requer que esta função seja chamada antes de

qualquer saída, incluindo as tags `<html>` e `<head>`, assim como espaços em branco.

Uma vez que os cookies foram definidos, eles podem ser acessados no próximo carregamento da página através do array *$_COOKIE*. Os valores dos cookies também podem existir no array *$_REQUEST*.

Parâmetros

A » RFC 6265 fornece a referência normativa de como cada parâmetro de **setcookie()** é interpretado.

`name`

O nome do cookie.

`value`

O valor do cookie. Esse valor é armazenado no computador do cliente; não deve ser armazenada informação sensível. Supondo que o `name` seja `'cookiename'`, o valor pode ser lido através de *$_COOKIE['cookiename']*

`expires_or_options`

O tempo para o cookie expirar. Esse valor é um timestamp Unix, portanto é o número de segundos desde a época Unix. Uma maneira de configurar esse valor é adicionando o número de segundos em que o cookie deve durar antes de expirar ao resultado da função time(). Por exemplo, `time()+60*60*24*30` irá configurar o cookie para expirar em 30 dias. Outra opção é usar a função mktime(). Se configurado para `0` ou omitido, o cookie vai expirar no final da sessão (quando o navegador fechar).

> **Nota** :
>
> O parâmetro `expires_or_options` recebe um timestamp Unix, ao contrário do formato de data `Wdy, DD-Mon-YYYY HH:MM:SS GMT`, isso acontece porque o PHP faz essa conversão internamente.

`path`

O caminho no servidor onde o cookie estará disponível. Se configurado para `'/'`, o cookie estará disponível para todo o `domain`. Se configurado para o diretório `'/foo/'`, o cookie estará disponível apenas dentro do diretório `/foo/` e todos os subdiretórios como `/foo/bar/` do `domain`. O valor padrão é o diretório atual onde o cookie está sendo configurado.

`domain`

O (sub)domínio para qual o cookie estará disponível. Definindo para um subdomínio (como `'www.example.com'`) deixará o cookie disponível para aquele subdomínio e todos os outros sub-domínios abaixo dele (exemplo w2.www.example.com). Para deixar o cookie disponível para todo o domínio (incluindo todos os subdomínios dele), basta definir o valor para o nome do domínio (`'example.com'`, nesse caso).

Navegadores antigos ainda implementam a » RFC 2109 e podem requerer um `.` no início para funcionar com todos os subdomínios.

`secure`

Indica que o cookie só poderá ser transmitido sob uma conexão segura HTTPS do cliente. Quando configurado para **`true`**, o cookie será enviado somente se uma conexão segura existir. No lado do servidor, fica por conta do programador enviar esse tipo de cookie somente sob uma conexão segura (ex respeitando *$_SERVER["HTTPS"]*).

`httponly`

Quando for **`true`**, o cookie será acessível somente sob o protocolo HTTP. Isso significa que o cookie não será acessível por linguagens de script, como JavaScript. É dito que essa configuração pode ajudar a reduzir ou identificar roubos de identidade através de ataques do tipo XSS (entretanto ela não é suportada por todos os navegadores), mas essa informação é constantemente discutida. Foi adicionada no PHP 5.2.0. **`true`** ou **`false`**

`options`

Um array associativo que pode ter qualquer uma das chaves `expires`, `path`, `domain`, `secure`, `httponly` e `samesite`.

Os valores têm o mesmo efeito como descrito para os parâmetros de mesmo nome. O valor do elemento `samesite` deve ser `None`, `Lax` ou `Strict`. Se uma das opções não for informada, os valores padrão serão os mesmos valores padrão dos parâmetros explícitos. Se `samesite` for omitido, o atributo SameSite do cookie não será atribuído.

> **Nota** :

> Para definir um cookie que inclui atributos que não estão entre as chaves listadas, use a função header().

> **Nota** :
>
> Se `samesite` for igual a `"None"` então `secure` também precisa ser habilitado senão o cookie será bloqueado pelo cliente.

Valor Retornado

Se houver saída antes da chamada dessa função, **setcookie()** irá falhar e retornará `false`. Se a função **setcookie()** for executada com sucesso, ela retornará `true`. Isso não indica que o usuário aceitou o cookie.

Erros/Exceções

Se o array `options` contiver chaves não suportadas:

- Antes do PHP 8.0.0, era gerado um `E_WARNING`.
- A partir do PHP 8.0.0, é lançado um ValueError.

Registro de Alterações

Versão	Descrição
8.2.0	O formato de data do cookie agora é `'D, d M Y H:i:s \G\M\T'`; anteriormente era `'D, d-M-Y H:i:s T'`.
8.0.0	Passar chaves não suportadas agora lança um ValueError em vez de emitir um `E_WARNING`.
7.3.0	Uma assinatura alternativa para suportar o array `options` foi adicionado. Essa assinatura também permite configurar o atributo SameSite do cookie.

Examples

Os efeitos dos exemplos a seguir podem ser observados usando a lista de cookies da ferramenta de desenvolvedor do navegador (normalmente na aba de Armazenamento ou Aplicação).

Exemplo #1 Exemplo de setcookie() para enviar cookies

```php
<?php

$value = 'alguma coisa de algum lugar';

// Define um "cookie de sessão" que expira quando o navegador é fechado
setcookie("CookieTeste", $value);
// Define um cookie que expira em 1 hora
setcookie("CookieTeste", $value, time()+3600);
// Define um cookie que se aplica somente a um caminho específico de um domnínio específico
// Observe que o domínio usado deve corresponder ao domínio do site
setcookie("CookieTeste", $value, time()+3600, "/~rasmus/", "example.com", true);

?>
```

Observe que a porção do valor do cookie será automaticamente codificada em URL e decodificada pelo PHP. Isto pode ser evitado usando setrawcookie() em seu lugar.

Para ver o conteúdo dos cookies definidos no exemplo acima em uma requisição posterior:

```php
<?php
// Mostra um cookie individual
echo $_COOKIE["CookieTeste"];

// Outra maneira de depurar/testar é vendo todos os cookies
print_r($_COOKIE);
?>
```

Exemplo #2 Exemplo de setcookie() para apagar cookies

Para excluir um cookie, defina a data de expiração para uma data no passado (porém diferente de zero, que é reservado para cookies de sessão).

Para excluir os cookies definidos no exemplo anterior:

```php
<?php
// Configura a data de expiração para uma hora atrás
setcookie("CookieTeste", "", time() - 3600);
setcookie("CookieTeste", "", time() - 3600, "/~rasmus/", ".example.com", 1);
?>
```

Exemplo #3 setcookie() e arrays

"Arrays de cookies" podem ser definidos usando a notação de array no nome do cookie. Isso tem o efeito de definir tantos cookies quantos elementos houver no array, mas quando o cookie for recebido pelo script, os valores serão todos colocados em um array com o nome do cookie:

```php
<?php
// define os cookies
setcookie("cookie[tres]", "cookietres");
setcookie("cookie[dois]", "cookiedois");
setcookie("cookie[um]", "cookieum");

// Exibe os cookies depois que a página recarregar
if (isset($_COOKIE['cookie'])) {
    foreach ($_COOKIE['cookie'] as $nome => $valor) {
        $nome = htmlspecialchars($nome);
        $valor = htmlspecialchars($valor);
        echo "$nome : $valor <br />\n";
    }
}
?>
```

O exemplo acima produzirá:

```
tres : cookietres
dois : cookiedois
um : cookieum
```

> **Nota** :
>
> Utilizar caracteres como `[` e `]` no nome do cookie não é válido conforme a RFC 6265, seção 4, mas deve ser suportado por navegadores conforme a RFC 6265, seção 5.

Notas

> **Nota** :
>
> Buffer de saída pode ser usado para permitir saída do script antes de chamar essa função. Toda a saída será armazenada em buffer até que seja enviada (explicitamente ou ao final da execução do script). Isso é feito chamando ob_start() e ob_end_flush() no script, ou configurando a diretiva `output_buffering` no *php.ini* ou nos arquivos de configuração do servidor.

Problemas comuns:

- Os cookies não estarão disponíveis até o próximo carregamento da página para a qual o cookie deverá estar visível. Para testar se um cookie foi enviado com sucesso, deve ser verificado o cookie no próximo carregamento da página antes que ele expire. O tempo para expirar é configurado pelo parâmetro `expires_or_options`. Uma boa maneira de depurar a existência dos cookies é chamando a função `print_r($_COOKIE);`.
- Os cookies devem ser excluídos com os mesmos parâmetros com os quais foram configurados. Se o argumento `value` for uma string vazia, e todos os outros argumentos forem iguais à chamada anterior de **setcookie()**, o cookie com o nome especificado será

excluído do cliente remoto. Internamente, isso é feito colocando o valor do cookie para `'deleted'` e a data de expiração para um ano no passado.

- Quando um cookie for configurado com o valor **`false`**, será realizada uma tentativa de excluí-lo. Por isso, valores booleanos não devem ser usados. No lugar deles, utilize *0* para **`false`** e *1* para **`true`**.
- Nomes de cookies podem ser definidos como nomes de arrays e estarão disponíves para os scripts PHP como arrays, mas cookies separados serão armazenados pelo navegador. Considere utilizar json_encode() para definir um cookie com nomes e valores múltiplos. Não é recomendado usar serialize() para esse propósito pois pode resultar em furos de segurança.

Várias chamadas para a função **setcookie()** são feitas na ordem em que são chamadas.

Veja também

- header() - Envia um cabeçalho HTTP bruto
- setrawcookie() - Envie um cookie sem codificar em URL o valor do cookie
- Seção de cookies
- » RFC 6265
- » RFC 2109

setrawcookie

setrawcookie

(PHP 5, PHP 7, PHP 8)

setrawcookie — Envie um cookie sem codificar em URL o valor do cookie

Descrição

```
function setrawcookie(
    string $name,
    string $value = ?,
    int $expires_or_options = 0,
    string $path = ?,
    string $domain = ?,
    bool $secure = false,
    bool $httponly = false
): bool
```

Assinatura alternativa disponível a partir do PHP 7.3.0 (não suportada com parâmetros nomeados):

function **setrawcookie**(string `$name`, string `$value` = ?, array `$options` = []): bool

setrawcookie() é exatamente o mesmo que setcookie() exceto que o valor do cookie não será automaticamente codificado em URL quando enviado ao navegador.

Parâmetros

Para informações sobre parâmetros, consulte a documentação da função setcookie().

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
7.3.0	Uma assinatura alternativa que suporta um array `options` foi adicionada. Esta assinatura também suporta a configuração do atributo de cookie "SameSite".

Veja também

- setcookie() - Envia um cookie

socket_get_status

socket_get_status

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_get_status — Sinônimo de stream_get_meta_data()

Descrição

Esta função é um apelido para: stream_get_meta_data().

socket_set_blocking

socket_set_blocking

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_set_blocking — Sinônimo de stream_set_blocking()

Descrição

Esta função é um apelido para: stream_set_blocking().

socket_set_timeout

socket_set_timeout

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_set_timeout — Sinônimo de stream_set_timeout()

Descrição

Esta função é um apelido para: stream_set_timeout().

syslog

syslog

(PHP 4, PHP 5, PHP 7, PHP 8)

syslog — Gera uma mensagem de registro do sistema

Descrição

function **syslog**(int `$priority`, string `$message`): true

syslog() gera uma mensagem de registro que será distribuída pelo registrador do sistema.

Para obter informações sobre como configurar um manipulador de registros definido pelo usuário, consulte a página de manual do Unix syslog.conf (5). Mais informações sobre os módulos e opções do syslog podem ser encontradas nas páginas man do syslog (3) em máquinas Unix.

Parâmetros

`priority`

Uma das constantes: `LOG_EMERG`, `LOG_ALERT`, `LOG_CRIT`, `LOG_ERR`, `LOG_WARNING`, `LOG_NOTICE`, `LOG_INFO`, `LOG_DEBUG` .

`message`

A mensagem a ser enviada.

Valor Retornado

Sempre retorna `true`.

Examples

Exemplo #1 Usando syslog()

```php
<?php
// abre o registrador do sistema, inclui o ID do processo e também envia
// o registro para a saída de erro padrão; e usa um mecanismo de registro
// definido pelo usuário
openlog("meuScriptDeRegistro", LOG_PID | LOG_PERROR, LOG_LOCAL0);

// algum código

if (authorized_client()) {
    // faz alguma coisa
} else {
    // cliente não autorizado!
    // registra a tentativa
    $access = date("d/m/Y H:i:s");
    syslog(LOG_WARNING, "Cliente não autorizado: $access {$_SERVER['REMOTE_ADDR']} ({$_SERVER['HTTP_USER_AGENT']})");
}

closelog();
?>
```

Notas

No Windows, o serviço de registrador do sistema é emulado pelo Log de Eventos.

> **Nota** :
>
> O uso das constantes **`LOG_LOCAL0`** até `LOG_LOCAL7` para o parâmetro `facility` da função openlog() não está disponível no Windows.

Veja também

- openlog() - Abre conexão com o registrador do sistema
- closelog() - Fecha conexão com o registrador do sistema
- Configuração INI syslog.filter (a partir do PHP 7.3)

Hash de Senhas

Introdução

A API de hash de senhas fornece uma interface em torno de crypt() de fácil utilização e alguns outros algoritmos de hash de senha, para facilitar a criação e o gerenciamento de senhas de forma segura.

Instalação/Configuração

Índice

Dependências

Nenhuma biblioteca externa é necessária para compilar esta extensão.

Para o hash de senha Argon2, é necessário o uso do » libargon2 ou, a partir do PHP 8.4.0, OpenSSL versão 3.2 ou superior. A partir do PHP 7.3.0, é necessário ter a versão 20161029 ou posterior do libargon2.

Instalação

Nenhuma instalação é necessária para usar estas funções; elas fazem parte do núcleo do PHP.

No entanto, para habilitar o hash de senha Argon2, o PHP deve ser compilado com suporte ao libargon2 utilizando a opção de configuração **--with-password-argon2** ou a partir do PHP 8.4.0, com o OpenSSL usando **--with-openssl** e **--with-openssl-argon2**.

Antes do PHP 8.1.0, era possível especificar o diretório argon2 com **--with-password-argon2[=DIR]**.

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

`PASSWORD_BCRYPT` (string)

> `PASSWORD_BCRYPT` é usada para criar novas hashes de senha usando o algoritmo `CRYPT_BLOWFISH`.
>
> Sempre terminará com um hash usando o formato de criptografia "$2y$", com 60 caracteres.
>
> Opções suportadas:
>
> - `salt` (string) - para fornecer manualmente um salt que será usado ao fazer o hash da senha. Observe que isso irá substituir e evitar que um salt seja gerado automaticamente.
>
> Se omitido, um salt aleatório será gerado pela função password_hash() para cada senha sofrendo hash. Esse é o modo de operação desejado e a partir do PHP 7.0.0 a opção salt foi descontinuada.
>
> - `cost` (int) - indica o custo de algoritmo que deve ser usado. Exemplos desses valores podem ser encontrados na página da função crypt().
>
> Se omitido, um valor padrão de `12` será usado. Esta é uma boa base de custo, mas pode-se considerar aumentar esse valor dependendo do equipamento.

`PASSWORD_BCRYPT_DEFAULT_COST` (int)

> Custo algorítmico padrão usado por password_hash() ao fazer hash de senhas com `PASSWORD_BCRYPT`.
>
> Disponível a partir do PHP 7.0.0.

`PASSWORD_ARGON2I` (string)

> `PASSWORD_ARGON2I` é utilizado para criar novos hashes utilizando o algoritmo Argon2i.
>
> Opções:
>
> - `memory_cost` (int) - Memória máxima (em bytes) que poderá ser utilizada para calcular o hash Argon2. Padrão: `PASSWORD_ARGON2_DEFAULT_MEMORY_COST`.

- `time_cost` (int) - Número de iterações (passagens) usadas para calcular o hash Argon2. Padrão: `PASSWORD_ARGON2_DEFAULT_TIME_COST`.
- `threads` (int) - Número de threads utilizada para calcular o hash Argon2. Padrão: `PASSWORD_ARGON2_DEFAULT_THREADS`. Somente disponível com libargon2, mas não na implementação libsodium.

Disponível a partir do PHP 7.2.0.

`PASSWORD_ARGON2ID` (string)

`PASSWORD_ARGON2ID` é utilizado para criar novos hashes de senha com o algoritmo Argon2id. Ele suporta as mesmas opções de `PASSWORD_ARGON2I`.

Disponível a partir do PHP 7.3.0.

`PASSWORD_ARGON2_DEFAULT_MEMORY_COST` (int)

Quantidade de memória padrão, em quilobytes (KiB) que será utilizada para calcular um hash.

Disponível a partir do PHP 7.2.0.

`PASSWORD_ARGON2_DEFAULT_TIME_COST` (int)

Número padrão de iterações (passagens) usadas para calcular um hash.

Disponível a partir do PHP 7.2.0.

`PASSWORD_ARGON2_DEFAULT_THREADS` (int)

Número de threads padrão qual a Argon2lib irá utilizar. Não disponível na implementação libsodium.

Disponível a partir do PHP 7.2.0.

`PASSWORD_ARGON2_PROVIDER` (string)

Disponível a partir do PHP 7.4.0.

`PASSWORD_DEFAULT` (string)

O algoritmo padrão a ser usado para fazer o hash se nenhum algoritmo for fornecido. Ele pode mudar com as novas versões do PHP quando algoritmos de hash novos e mais fortes forem suportados.

É importante notar que ao longo do tempo essa constante pode mudar. Assim sendo, deve-se estar ciente que o comprimento do hash resultante pode mudar. Portanto, ao usar `PASSWORD_DEFAULT`, o hash resultante deve ser armazanado de uma forma que possa armazenar hashes arbitrários, o comprimento recomendado é de 255 bytes.

Atualmente é um sinônimo de `PASSWORD_BCRYPT`.

Registro de Alterações

Versão	Descrição
7.4.0	Os valores informados como IDs de algoritimo (`PASSWORD_BCRYPT`, `PASSWORD_ARGON2I`, `PASSWORD_ARGON2ID` e `PASSWORD_DEFAULT`) agora são do tipo string. Anteriormente eles eram do tipo int.

Funções da extensão Password Hashing

Índice

password_algos

password_algos

(PHP 7 >= 7.4.0, PHP 8)

password_algos — Obtém IDs de algoritmos disponíveis de hash de senhas

Descrição

function **password_algos**(): array

Retorna uma lista completa de todos os IDs de algoritmos registrados de hash de senhas como um array de strings.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna os IDs de algoritmos disponíveis de hash de senhas.

Examples

Exemplo #1 Uso básico de **password_algos()**

```
<?php
print_r(password_algos());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => 2y
    [1] => argon2i
    [2] => argon2id
)
```

password_get_info

password_get_info

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_get_info — Retorna a informações sobre o hash fornecido

Descrição

function **password_get_info**(string `$hash`): array

Quando for fornecido um hash válido criado por um algoritmo suportado pela função password_hash(), esta função retornará um array com as informações sobre esse hash.

Parâmetros

`hash`

Um hash criado por password_hash().

Valor Retornado

Retorna um array associativo com três elementos:

- `algo`, que corresponderá a uma constante de algoritmo de senha
- `algoName`, que terá o nome legível do algoritmo
- `options`, que inclui as opções fornecidas ao se chamar a função password_hash()

password_hash

password_hash

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_hash — Cria um hash de senha

Descrição

function **password_hash**(#[\SensitiveParameter] string `$password`, string|int|null `$algo`, array `$options = []`): string

password_hash() cria um novo hash de senha usando um algoritmo forte de hash de mão única.

Os seguintes algoritmos são suportados atualmente:

- `PASSWORD_DEFAULT` - Usa o algoritmo bcrypt (padrão desde o PHP 5.5.0). Perceba que essa constante foi desenhada para mudar ao longo do tempo a medida que novos algoritmos mais fortes forem adicionados ao PHP. Por essa razão, o comprimento do resultado da utilização desse identificador pode mudar ao longo do tempo. Por isso, é recomendado que armazene o resultado em uma coluna do banco de dados que possa ser expandida além dos 60 bytes (255 bytes seria uma boa escolha).
- `PASSWORD_BCRYPT` - Usa o algoritmo bcrypt para criar o hash. Produzirá um hash compatível com o padrão crypt() usando o identificador `$2y$`.
- `PASSWORD_ARGON2I` - Usa o algoritmo Argon2i para criar o hash. Este algoritmo somente estará disponível se o PHP tiver sido compilado com suporte a Argon2.
- `PASSWORD_ARGON2ID` - Usa o algoritmo Argon2id para criar o hash. Este algoritmo somente estará disponível se o PHP tiver sido compilado com suporte a Argon2.

Opções suportadas por `PASSWORD_BCRYPT`:

- `salt` (string) - para fornecer manualmente um salt a ser usado quando estiver sendo feito o hash da senha. Perceba que isso irá sobrepor e evitar que um salt seja gerado automaticamente.

  Se omitido, um salt aleatório será gerado pela função **password_hash()** para cada senha sofrendo hash. Esse é o modo de operação desejado.

  > **Aviso**
  >
  > A opção salt foi descontinuada. Agora é preferível que simplesmente se utilize o salt que é gerado por padrão. A partir do PHP 8.0.0, um salt fornecido explicitamente será ignorado.

- `cost` (int) - indica o custo de algoritmo que deve ser usado. Exemplos desses valores podem ser encontrados na página da função crypt().

  Se omitido, um valor padrão `12` será usado. Este é um bom patamar de custo, mas deve ser ajustado dependendo do equipamento em uso.

Opções suportadas por `PASSWORD_ARGON2I` e `PASSWORD_ARGON2ID`:

- `memory_cost` (int) - Máximo de mémoria (em kibibytes) que pode ser utilizado para computar o hash Argon2. O valor padrão é `PASSWORD_ARGON2_DEFAULT_MEMORY_COST`.
- `time_cost` (int) - Número de iterações (passagens) usadas para computar o hash Argon2. O valor padrão é `PASSWORD_ARGON2_DEFAULT_TIME_COST`.
- `threads` (int) - Número de threads para computar o hash Argon2. O valor padrão é `PASSWORD_ARGON2_DEFAULT_THREADS`.

  > **Aviso**
  >
  > Disponível somente quando o PHP usa a libargon2, e não com a implementação libsodium.

Parâmetros

`password`

A senha do usuário.

> **Cuidado**
>
> Usando `PASSWORD_BCRYPT` como algoritmo, resultará no parâmetro `password` sendo truncado em um comprimento máximo de 72 bytes.

`algo`

Uma constante de algoritmo de senha denotando o algoritmo a ser usado ao fazer o hash da senha.

`options`

Um array associativo contendo opções. Consulte as constantes de algoritmo de senha para obter a documentação sobre as opções suportadas por cada algoritmo.

Valor Retornado

Retorna o hash da senha.

O algoritmo, o custo e o salt utilizados são retornados como parte do hash. Dessa forma, toda informação necessária para verificar o hash é incluída nele. Isso permite que a função password_verify() verifique o hash sem precisar de um armazenamento separado para a informação do salt ou do algoritmo.

Registro de Alterações

Versão	Descrição
8.4.0	O valor padrão da opção `cost` do algoritmo **`PASSWORD_BCRYPT`** foi aumentado de `10` para `12`.
8.3.0	**password_hash()** agora define a exceção Random\RandomException subjacente como a Exception::$previous quando uma exceção ValueError é lançada devido a uma falha na geração do salt.
8.0.0	**password_hash()** não mais retorna **`false`** em caso de falha, em vez disso uma exceção ValueError será lançada se o algoritmo de hash da senha não for válido, ou uma exceção Error se o cálculo do hash falhou por motivo desconhecido.
8.0.0	O parâmetros `algo` agora pode ser nulo.
7.4.0	O parâmetro `algo` agora espera uma string, mas ainda aceita ints para compatibilidade com versões anteriores.
7.4.0	A extensão sodium fornece uma implementação alternativa para senhas Argon2.
7.3.0	Suporte para senhas Argon2id usando **`PASSWORD_ARGON2ID`** foi adicionado.
7.2.0	Suporte para senhas Argon2i usando **`PASSWORD_ARGON2I`** foi adicionado.

Examples

Exemplo #1 Exemplo de password_hash()

```php
<?php
echo password_hash("rasmuslerdorf", PASSWORD_DEFAULT);
?>
```

O exemplo acima produzirá algo semelhante a:

```
$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.
```

Exemplo #2 Exemplo de configuração do custo de password_hash() manualmente

```php
<?php
$options = [
    // Aumenta o custo bcrypt de 12 para 13.
    'cost' => 13,
];
echo password_hash("rasmuslerdorf", PASSWORD_BCRYPT, $options);
?>
```

O exemplo acima produzirá algo semelhante a:

```
$2y$13$xeDfQumlmdm0Sco.4qmH1OGfUUmOcuRmfae0dPJhjX1Bq0yYhqbNi
```

Exemplo #3 Exemplo de busca de um bom custo para password_hash()

Este código fará um benchmark da máquina para determinar o quão alto um custo pode ser usado sem deteriorar a experiência do usuário. É recomendado definir o custo mais alto que não desacelere outras operações que a máquina precisa executar. 11 é uma boa linha de base, e mais é melhor se a máquina for rápida o suficiente. O código abaixo visa um tempo de alongamento ≤ 350 milissegundos, que é um atraso apropriado para sistemas que lidam com logins interativos.

```php
<?php
$timeTarget = 0.350; // 350 milissegundos

$cost = 11;
do {
    $cost++;
    $start = microtime(true);
    password_hash("test", PASSWORD_BCRYPT, ["cost" => $cost]);
    $end = microtime(true);
} while (($end - $start) < $timeTarget);

echo "Custo Apropriado Encontrado: " . $cost - 1;
?>
```

O exemplo acima produzirá algo semelhante a:

```
Custo Apropriado Encontrado: 13
```

Exemplo #4 Exemplo de password_hash() usando Argon2i

```php
<?php
echo 'Argon2i hash: ' . password_hash('rasmuslerdorf', PASSWORD_ARGON2I);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Argon2i hash: $argon2i$v=19$m=1024,t=2,p=2$YzJBSzV4TUhkMzc3d3laeg$zqU/1IN0/AogfP4cmSJI1vc8lpXRW9/S0sYY2i2jHT0
```

Notas

> **Cuidado**
>
> É fortemente recomendado não fornecer um salt explícito para esta função. Ela criará um salt seguro automaticamente se um não for especificado.
>
> Como apontado acima, fornecer a opção `salt` no PHP 7.0.0 gerará um alerta de descontinuação. O suporte para fornecer um salt explicitamente foi removido no PHP 8.0.

> **Nota** :
>
> É recomendado testar esta função na máquina usada, ajustando o parâmetro custo para que a execução da função leve menos do que 350 milissegundos em sistemas interativos. O script do exemplo acima ajudará a escolher um custo bcrypt apropriado para uma determinada máquina.

> **Nota** :
>
> Atualizações dos algoritmos suportados por esta função (ou alterações no algoritmo padrão) precisam seguir as seguintes regras:
>
> - Qualquer novo algoritmo precisa estar no core por pelo menos 1 versão completa do PHP antes de se tornar padrão. Assim se, por exemplo, um novo algoritmo for adicionado na versão 7.5.5, ela não seria elegível para padrão até a versão 7.7 (uma vez que a 7.6 seria a primeira versão completa). Mas se um algoritmo diferente for adicionado na versão 7.6.0, ela seria elegível para padrão na 7.7.0.
> - O padrão deve mudar apenas em uma versão completa (7.3.0, 8.0.8 etc.) e não em uma versão de revisão. A única exceção seria uma emergênca caso uma falha de segurança crítica fosse encontrada no padrão atual.

Veja também

- password_verify() - Verifica se uma senha corresponde a um hash
- password_needs_rehash() - Verifica se o hash informado corresponde às opções especificadas
- crypt() - Hash unidirecional de string
- sodium_crypto_pwhash_str() - Obtém um hash codificado em ASCII

password_needs_rehash

password_needs_rehash

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_needs_rehash — Verifica se o hash informado corresponde às opções especificadas

Descrição

function **password_needs_rehash**(string `$hash`, string|int|null `$algo`, array `$options = []`): bool

Esta função verifica se o hash fornecido implementa o algoritmo e as opções indicadas. Se não, ela assume que o hash precisa ser recalculado.

Parâmetros

`hash`

Um hash criado por password_hash().

`algo`

Uma constante de algoritmo de senha denotando o algoritmo a ser usado ao fazer o hash da senha.

`options`

Um array associativo contendo opções. Consulte as constantes de algoritmo de senha para obter a documentação sobre as opções suportadas por cada algoritmo.

Valor Retornado

Retorna **`true`** se o hash deve ser recalculado para corresponder ao algortimo informado em `algo` e às opções em `options`, ou **`false`** caso contrário.

Registro de Alterações

Versão	Descrição
7.4.0	O parâmetro `algo` agora espera uma string, mas ainda aceita ints para compatibilidade com versões anteriores.

Examples

Exemplo #1 Utilização de **password_needs_rehash()**

```
<?php

$password = 'rasmuslerdorf';
$hash = '$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.';
```

```php
$algorithm = PASSWORD_BCRYPT;
// o parâmetro de custo de bcrypt pode mudar com o tempo com a evloução dos equipamentos
$options = ['cost' => 13];

// Verifica o hash armazenado comparando com uma senha em texto puro
if (password_verify($password, $hash)) {
    // Verifica se o algoritmo ou o custo mudaram
    if (password_needs_rehash($hash, $algorithm, $options)) {
        // Caso positivo, cria um hash novo, e substitui o antigo
        $newHash = password_hash($password, $algorithm, $options);

        // Atualiza o registro do usuário com $newHash
    }

    // Faz login
}
?>
```

password_verify

password_verify

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_verify — Verifica se uma senha corresponde a um hash

Descrição

function **password_verify**(#[\SensitiveParameter] string `$password`, string `$hash`): bool

Verifica se o hash fornecido corresponde à senha informada **password_verify()** é compatível com crypt(). Portanto, hashes de senha criadas por crypt() podem ser usados com **password_verify()**.

Perceba que password_hash() retorna o algoritmo, custo e o salt como parte do hash retornado. Por isso, toda a informação necessária para verificar o hash está incluída nele. Isso permite que a função de verificação faça isso sem precisar de um armazenamento separado para a informação do salt ou do algoritmo.

Esta função é segura contra ataques de sincronismo.

Parâmetros

`password`

A senha do usuário.

`hash`

Um hash criado por password_hash().

Valor Retornado

Retorna **`true`** se o password e o hash corresponderem, ou **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de password_verify()

Este é um exemplo simplificado; é recomendável recalcular o hash de uma senha correta se necessário; consulte a página da função password_needs_rehash() para um exemplo.

```php
<?php
// Veja o exemplo de password_hash() para ver de onde veio esta string.
$hash = '$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.';

if (password_verify('rasmuslerdorf', $hash)) {
    echo 'Senha válida!';
} else {
    echo 'Senha inválida';
}
?>
```

O exemplo acima produzirá:

```
Senha válida!
```

Veja também

- password_needs_rehash() - Verifica se o hash informado corresponde às opções especificadas
- password_hash() - Cria um hash de senha
- sodium_crypto_pwhash_str_verify() - Verifica se uma senha corresponde a um hash

Filtragem de dados

Introdução

Esta extensão fornece filtros que podem ser usados para validar ou sanitizar dados. Isto é especialmente útil quando a fonte contém dados desconhecidos (ou externos), como dados enviados pelo usuário. Por exemplo, esses dados podem vir de um formulário HTML.

Há dois tipos principais de filtragem: *validação* e *higienização*.

Um filtro de validação é utilizado para verificar se os dados cumprem certos critérios. Estes filtros são identificados pelas constantes `FILTER_VALIDATE_*`. Por exemplo, a constante `FILTER_VALIDATE_EMAIL` pode ser usada para determinar se o dado é um endereço de e-mail válido. Entretanto, ele nunca alterará os dados de entrada.

A sanitização, por outro lado, "limpará" os dados, portanto pode alterar os dados de entrada adicionando ou removendo caracteres. Estes filtros são identificados pelas constantes `FILTER_SANITIZE_*`. Por exemplo, o filtro `FILTER_SANITIZE_EMAIL` removerá caracteres que são inadequados para um endereço de e-mail. Entretanto, os dados sanitizados não são validados para verificar se o endereço é válido.

A maioria dos filtros suporta *sinalizadores* opcionais que podem ajustar o comportamento do filtro. Estes sinalizadores são identificados pelas constantes `FILTER_FLAG_*`. Por exemplo, usar `FILTER_FLAG_PATH_REQUIRED` com um filtro de validação `FILTER_VALIDATE_URL` irá requerer que um URL tenha um caminho (por exemplo, `/foo` em `https://example.org/foo`).

Instalação/Configuração

Índice

Instalação

Esta extensão está habilitada por padrão. Ela pode ser desabilitada usando a seguinte opção em tempo de compilação: **--disable-filter**

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração da Filter

Nome	Padrão	Modificável	Registro de Alterações
filter.default	"unsafe_raw"	`INI_PERDIR`	Descontinuado desde o PHP 8.1.0.
filter.default_flags	NULL	`INI_PERDIR`	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`filter.default` string

Filtra todas informações de *$_GET*, *$_POST*, *$_COOKIE*, *$_REQUEST* e *$_SERVER* com este filtro. A informação original pode ser acessada através da filter_input().

Deve ser o nome de um filtro que pode ser determinado através de filter_list() e filter_id().

> **Nota** :
>
> Cuidado com as opções padrão para os filtros padrão. Eles devem ser definidos explicitamente. Por exemplo, para configurar o filtro padrão para se comportar exatamente como htmlspecialchars(), as opções padrão devem ser definidas para `0` como mostrado no exemplo abaixo.
>
> **Exemplo #1 Configurando o filtro padrão para agir como htmlspecialchars**
>
> ```
> filter.default = full_special_chars
> filter.default_flags = 0
> ```

> **Aviso**

> Esta configuração INI está descontinuada a partir do PHP 8.1.0.

`filter.default_flags` int

Opções padrão a serem aplicados quando o filtro padrão está definido. É definido como `FILTER_FLAG_NO_ENCODE_QUOTES` por padrão para compatibilidade com versões anteriores. Consulte as constantes `FILTER_FLAG_*` para conhecer as opções disponíveis.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

Constantes de Entrada

Estas constantes são usadas por filter_input() e filter_input_array().

`INPUT_POST` (int)

Variáveis POST.

`INPUT_GET` (int)

Variáveis GET.

`INPUT_COOKIE` (int)

Variáveis COOKIE.

`INPUT_ENV` (int)

Variáveis ENV.

`INPUT_SERVER` (int)

Variáveis SERVER.

`INPUT_SESSION` (int)

Variáveis SESSION. (Removida a partir do PHP 8.0.0; não estava implementada anteriormente)

`INPUT_REQUEST` (int)

Variáveis REQUEST. (Removida a partir do PHP 8.0.0; não estava implementada anteriormente)

Opções Genéricas de Filtros

`FILTER_FLAG_NONE` (int)

Sem flags.

`FILTER_REQUIRE_SCALAR` (int)

Opção usada para requerer que a entrada do filtro seja um escalar.

`FILTER_REQUIRE_ARRAY` (int)

Opção usada para requerer que a entrada do filtro seja um array.

`FILTER_FORCE_ARRAY` (int)

Esta opção encapsula entradas encalares em um array de um elemento para filtros que operam em arrays.

`FILTER_NULL_ON_FAILURE` (int)

Usa `null` ao invés de `false` em caso de falha.

Utilizável com qualquer filtro `FILTER_VALIDATE_*` de validação.

`FILTER_THROW_ON_FAILURE` (int)

Lança uma Filter\FilterFailedException quando um filtro de validação falhar, em vez de retornar `false`.

Pode ser usado com qualquer filtro de validação `FILTER_VALIDATE_*`.

Disponível a partir do PHP 8.5.0.

Opções de Filtro de Sanitização

`FILTER_FLAG_STRIP_LOW` (int)

Remove caracteres com valor ASCII menor que 32.

`FILTER_FLAG_STRIP_HIGH` (int)

Remove caracteres com valor ASCII maior que 127.

`FILTER_FLAG_STRIP_BACKTICK` (int)

Remove caracteres de acento grave (`).

`FILTER_FLAG_ENCODE_LOW` (int)

Codifica caracteres com valor ASCII menor que 32.

`FILTER_FLAG_ENCODE_HIGH` (int)

Codifica caracteres com valor ASCII maior que 127.

`FILTER_FLAG_ENCODE_AMP` (int)

Codifica `&`.

`FILTER_FLAG_NO_ENCODE_QUOTES` (int)

Não codifica aspas simples e duplas (`'` e `"`).

`FILTER_FLAG_EMPTY_STRING_NULL` (int)

Se a sanitização de uma string resultar em uma string vazia, converte o valor para `null`.

Filtros de Validação

`FILTER_VALIDATE_BOOL` (int)

Retorna `true` para `"1"`, `1` incluindo as notações binária, octal e hexadecimal, `1.0` incluindo notação científica, `"true"`, `true`, `"on"` e `"yes"`.

Retorna `false` para `"0"`, `0` incluindo as notações binária, octal e hexadecimal, `0.0` incluindo notação científica, `"false"`, `false`, `"off"`, `"no"` e `""`.

Valores string são comparados de forma insensível a maiúsculas/minúsculas. O valor de retorno para valores não booleanos depende da opção `FILTER_NULL_ON_FAILURE`. Se estiver definida, retorna `null`, caso contrário, `false`.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Disponível a partir do PHP 8.0.0.

`FILTER_VALIDATE_BOOLEAN` (int)

Sinônimo de `FILTER_VALIDATE_BOOL`. O sinônimo estava disponível antes da introdução de seu nome canônico no PHP 8.0.0.

`FILTER_VALIDATE_INT` (int)

Valida se o valor é um inteiro, em caso de sucesso é convertido para o tipo int.

> **Nota** :
>
> Valores string são aparados usando trim() antes da validação.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

`min_range`

Valor só é válido se for maior ou igual que o valor fornecido.

`max_range`

Valor só é válido se for menor ou igual que o valor fornecido.

Constantes Opcionais

`FILTER_FLAG_ALLOW_OCTAL` (int)

Permite inteiros em notação octal (`0[0-7]+`).

`FILTER_FLAG_ALLOW_HEX` (int)

Permite inteiros em notação hexadecimal (`0x[0-9a-fA-F]+`).

`FILTER_VALIDATE_FLOAT` (int)

Valida se o valor é um float, em caso de sucesso é convertido para o tipo float.

> **Nota** :
>
> Valores string são aparados usando trim() antes da validação.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

`decimal`

`min_range`

Valor só é válido se for maior ou igual que o valor fornecido. Disponível a partir do PHP 7.4.0.

`max_range`

Valor só é válido se for menor ou igual que o valor fornecido. Disponível a partir do PHP 7.4.0.

Constantes opcionais

`FILTER_FLAG_ALLOW_THOUSAND` (int)

Aceita vírgulas (`,`), que normalmente representam separador de milhares em inglês.

`FILTER_VALIDATE_REGEXP` (int)

Valida o valor comparando com a expressão regular informada pela opção `regexp`.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

`regexp`

Expressão regular compatível com Perl.

`FILTER_VALIDATE_URL` (int)

Verifica se o URL é válido conforme a » RFC 2396.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Constantes Opcionais

`FILTER_FLAG_SCHEME_REQUIRED` (int)

Requer que o URL contenha uma parte de esquema.

> **Aviso**
>
> *DESCONTINUADA* a partir do PHP 7.3.0 e *REMOVIDA* a partir do PHP 8.0.0. A razão é que fica sempre implícita pelo filtro `FILTER_VALIDATE_URL`.

`FILTER_FLAG_HOST_REQUIRED` (int)

Requer que o URL contenha uma parte de host.

> **Aviso**
>
> *DESCONTINUADA* a partir do PHP 7.3.0 e *REMOVIDA* a partir do PHP 8.0.0. A razão é que fica sempre implícita pelo filtro `FILTER_VALIDATE_URL`.

`FILTER_FLAG_PATH_REQUIRED` (int)

Requer que o URL contenha uma parte de caminho.

`FILTER_FLAG_QUERY_REQUIRED` (int)

Requer que o URL contenha uma parte de consulta.

> **Aviso**
>
> Um URL válido pode não especificar o protocolo HTTP (`http://`). Portanto, uma validação adicional pode ser necessária para determinar se o URL usa um protocolo esperado, por exemplo, `ssh://` ou `mailto:`.

> **Aviso**
>
> Este filtro só funciona em URLs ASCII. Isto significa que os Nomes de Domínio Internacionalizados (IDN) serão sempre rejeitados.

`FILTER_VALIDATE_DOMAIN` (int)

Valida se o nome de domínio é válido de acordo com » RFC 952, » RFC 1034, » RFC 1035, » RFC 1123, » RFC 2732, e » RFC 2181.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Constantes Opcionais

`FILTER_FLAG_HOSTNAME` (int)

Requer que hostnames iniciem com caracteres alfanuméricos e contenham apenas caracteres alfanuméricos ou hífen.

`FILTER_VALIDATE_EMAIL` (int)

Verifica se o valor é um endereço válido de e-mail.

A validação é realizada conforme a sintaxe `addr-spec` na » RFC 822. Contudo, comentários, espaços em branco e domínios sem ponto não são suportados, e por isso serão rejeitados.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Constantes Opcionais

`FILTER_FLAG_EMAIL_UNICODE` (int)

Aceita caracteres Unicode na parte local. Disponível a partir do PHP 7.1.0.

> **Aviso**
>
> A validação de e-mail é complexa e a única maneira verdadeira de confirmar se um e-mail é válido e existe é enviando um e-mail para o endereço.

`FILTER_VALIDATE_IP` (int)

Valida o valor como um endereço IP.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Constantes Opcionais

`FILTER_FLAG_IPV4` (int)

Permite endereço IPv4.

`FILTER_FLAG_IPV6` (int)

Permite endereço IPv6.

`FILTER_FLAG_NO_RES_RANGE` (int)

Nega endereços reservados.

Estes são os intervalos marcados como `Reserved-By-Protocol` (reservados por protocolo) na » RFC 6890.

Para IPv4, corresponden aos seguintes intervalos: `0.0.0.0/8`, `169.254.0.0/16`, `127.0.0.0/8`, `240.0.0.0/4`.

E para IPv6, corresponden aos seguintes intervalos: `::1/128`, `::/128`, `::FFFF:0:0/96`, `FE80::/10`.

`FILTER_FLAG_NO_PRIV_RANGE` (int)

Nega endereços privados.

São os endereços IPv4 que estão nestas faixas: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.

E são os endereços IPv6 que inicial com `FD` ou `FC`.

`FILTER_FLAG_GLOBAL_RANGE` (int)

Permite apenas endereços globais. Podem ser encontrados na » RFC 6890 onde o atributo `Global` é `True`. Disponível a partir do PHP 8.2.0.

`FILTER_VALIDATE_MAC` (int)

Valida se o valor é um endereço MAC.

Opções disponíveis

`default`

Valor a retornar caso o filtro falhe.

Filtros de Sanitização

`FILTER_UNSAFE_RAW` (int)

Este filtro não faz nada.

Entretanto, ele pode remover ou codificar caracteres especiais se usado em conjunto com as opções `FILTER_FLAG_STRIP_*` e `FILTER_FLAG_ENCODE_*` de filtro de sanitização.

`FILTER_DEFAULT` (int)

Sinônimo de `FILTER_UNSAFE_RAW`.

`FILTER_SANITIZE_STRING` (int)

Este filtro remove etiquetas e codifica HTML com aspas duplas e simples.

Opcionalmente, ele pode remover ou codificar caracteres especificados se usado junto com as opções de filtro de sanitização `FILTER_FLAG_STRIP_*` e `FILTER_FLAG_ENCODE_*`.

O comportamento da codificação de aspas pode ser desativado usando a opção de filtro `FILTER_FLAG_NO_ENCODE_QUOTES`

> **Aviso**
>
> *Descontinuado* a partir do PHP 8.1.0, use htmlspecialchars() em seu lugar.

> **Aviso**
>
> A maneira como esse filtro remove as etiquetas não é equivalente a strip_tags().

`FILTER_SANITIZE_STRIPPED` (int)

Sinônimo de `FILTER_SANITIZE_STRING`.

> **Aviso**
>
> *Descontinuado* a partir do PHP 8.1.0, use htmlspecialchars() em seu lugar.

`FILTER_SANITIZE_ENCODED` (int)

Este filtra codifica uma string em URL.

Opcionalmente pode codificar caracteres especificados se usado junto com as opções de filtro de sanitização `FILTER_FLAG_STRIP_*` e `FILTER_FLAG_ENCODE_*`.

`FILTER_SANITIZE_SPECIAL_CHARS` (int)

Este filtra codifica, em HTML, `'`, `"`, `<`, `>`, `&` e caracteres com um valor ASCII menor que 32. Diferentemente do filtro `FILTER_SANITIZE_FULL_SPECIAL_CHARS`, o filtro `FILTER_SANITIZE_SPECIAL_CHARS` ignora a opção `FILTER_FLAG_NO_ENCODE_QUOTES`.

Opcionalmente pode codificar caracteres especificados se usado junto com as opções de filtro de sanitização `FILTER_FLAG_STRIP_*`, e pode codificar caracteres com valores ASCII maiores que 127 usando `FILTER_FLAG_ENCODE_HIGH`.

`FILTER_SANITIZE_FULL_SPECIAL_CHARS` (int)

Este filtro é equivalente a chamar htmlspecialchars() com `ENT_QUOTES` definida.

O comportamento da codificação de aspas pode ser desabilitado usando a opção de filtro `FILTER_FLAG_NO_ENCODE_QUOTES`.

> **Aviso**
>
> Assim como htmlspecialchars(), este filtro reconhece a configuração INI default_charset. Se for detectada uma sequência de bytes que constitui um caractere inválido no conjunto de caracteres atual, a string inteira será rejeitada, resultando no retorno de uma string vazia.

`FILTER_SANITIZE_EMAIL` (int)

Sanitiza a string removendo todos os caracteres exceto as letras latinas (`[a-zA-Z]`), dígitos (`[0-9]`) e os caracteres especiais `!#$%&'*+-=?^_`{|}~@.[]`.

`FILTER_SANITIZE_URL` (int)

Sanitiza a string removendo todos os caracteres exceto as letras latinas (`[a-zA-Z]`), dígitos (`[0-9]`) e os caracteres especiais `$-_.+!*'(),{}|\\^~[]`<>#%";/?:@&=`.

`FILTER_SANITIZE_NUMBER_INT` (int)

Sanitiza a string removendo todos os caracteres exceto (`[0-9]`), sinal de adição (`+`), e sinal de subtração (`-`).

`FILTER_SANITIZE_NUMBER_FLOAT` (int)

Sanitiza a string removendo todos os caracteres exceto (`[0-9]`), sinal de adição (`+`), e sinal de subtração (`-`).

Constantes Opcionais

`FILTER_FLAG_ALLOW_FRACTION` (int)

Aceita o caractere ponto (`.`), que normalmente representa o separador entre as partes inteira e fracionária em inglês.

`FILTER_FLAG_ALLOW_THOUSAND` (int)

Aceita vírgulas (`,`), que normalmente representam separador de milhares em inglês.

`FILTER_FLAG_ALLOW_SCIENTIFIC` (int)

Aceita números em notação científica permitindo os caracteres `e` e `E`.

> **Aviso**
>
> Se a opção `FILTER_FLAG_ALLOW_FRACTION` não for usada, o separador de decimais será removido, alterando o valor recebido.
>
> ```php
> <?php
> $number = '12.34';
>
> var_dump(filter_var($number, FILTER_SANITIZE_NUMBER_FLOAT));
> var_dump(filter_var($number, FILTER_SANITIZE_NUMBER_FLOAT, FILTER_FLAG_ALLOW_FRACTION));
> ?>
> ```
>
> O exemplo acima produzirá:
>
> ```
> string(4) "1234"
> string(5) "12.34"
> ```

`FILTER_SANITIZE_ADD_SLASHES` (int)

Aplica addslashes() à entrada. Disponível a partir do PHP 7.3.0.

`FILTER_SANITIZE_MAGIC_QUOTES` (int)

Sinônimo de `FILTER_SANITIZE_ADD_SLASHES`.

> **Aviso**
>
> *DESCONTINUADA* a partir do PHP 7.3.0 e *REMOVIDA* a partir do PHP 8.0.0.

Filtro Definido pelo Usuário

`FILTER_CALLBACK` (int)

Este filtro delega a filtragem a uma função definida pelo usuário. O callable é passado através do parâmetro `options` comom o valor associado à chave `'options'`.

A função deve ter a seguinte assinatura:

function callback(string `$value`): mixed

`value`

O valor a ser filtrado.

> **Nota** :
>
> O valor retornado pela função será o valor retornado pela função de filtro chamada.

Exemplo #1 Exemplo de uso de `FILTER_CALLBACK` para validar um login

```php
<?php
function validate_login(string $value): ?string
{
    if (strlen($value) >= 5 && ctype_alnum($value)) {
```

```
        return $value;
    }
    return null;
}

$login = "L0ginValido";
$filtered_login = filter_var($login, FILTER_CALLBACK, ['options' => 'validate_login']);
var_dump($filtered_login);

$login = "login f&lso";
$filtered_login = filter_var($login, FILTER_CALLBACK, ['options' => 'validate_login']);
var_dump($filtered_login);
?>
```

O exemplo acima produzirá:

```
string(11) "L0ginValido"
NULL
```

> **Nota** :
>
> A partir do PHP 8.1.25 e 8.2.12, as opções estruturais `FILTER_REQUIRE_ARRAY`, `FILTER_FORCE_ARRAY` e `FILTER_REQUIRE_SCALAR` se aplicam a este filtro e `FILTER_NULL_ON_FAILURE` altera como tais falhas estruturais são reportadas. Em versões anteriores, todas as opções eram ignoradas. Opções específicas de filtro são sempre ignoradas, e como o valor de retorno da função de retorno é o resultado, a função de retorno em si não pode falhar: nenhuma opção transforma um `false` retornado pela função de retorno em um `null` ou em uma exceção.

Examples

Índice

Validação

Exemplo #1 Validando um endereço de e-mail com filter_var()

```
<?php
$email_a = 'joe@example.com';
$email_b = 'bogus';

if (filter_var($email_a, FILTER_VALIDATE_EMAIL)) {
    echo "O endereço de e-mail '$email_a' é considerado válido.\n";
}
if (filter_var($email_b, FILTER_VALIDATE_EMAIL)) {
    echo "O endereço de e-mail '$email_b' é considerado válido.\n";
} else {
    echo "O endereço de e-mail '$email_b' é considerado inválido.\n";
}
?>
```

O exemplo acima produzirá:

```
O endereço de e-mail 'joe@example.com' é considerado válido.
O endereço de e-mail 'bogus' é considerado inválido.
```

Exemplo #2 Validando endereço IP com filter_var()

```php
<?php
$ip_a = '127.0.0.1';
$ip_b = '42.42';

if (filter_var($ip_a, FILTER_VALIDATE_IP)) {
    echo "O endereço IP '$ip_a' é considerado válido.";
}
if (filter_var($ip_b, FILTER_VALIDATE_IP)) {
    echo "O endereço IP '$ip_b' é considerado inválido.";
}
?>
```

O exemplo acima produzirá:

```
O endereço IP '127.0.0.1' é considerado válido.
```

Exemplo #3 Passando opções para filter_var()

```php
<?php
$int_a = '1';
$int_b = '-1';
$int_c = '4';
$options = array(
    'options' => array(
        'min_range' => 0,
        'max_range' => 3,
    )
);
if (filter_var($int_a, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "O inteiro A '$int_a' é considerado válido (entre 0 e 3).\n";
}
if (filter_var($int_b, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "O inteiro B '$int_b' é considerado válido (entre 0 e 3).\n";
}
if (filter_var($int_c, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "O inteiro C '$int_c' é considerado válido (entre 0 e 3).\n";
}

$options['options']['default'] = 1;
if (($int_c = filter_var($int_c, FILTER_VALIDATE_INT, $options)) !== FALSE) {
    echo "O inteiro C '$int_c' é considerado válido (entre 0 e 3).";
}
?>
```

O exemplo acima produzirá:

```
O inteiro A '1' é considerado válido (entre 0 e 3).
O inteiro C '1' é considerado válido (entre 0 e 3).
```

Sanitização

Exemplo #1 Sanitização e validação de endereços de e-mail

```php
<?php
$a = 'joe@example.org';
$b = 'bogus - at - example dot org';
$c = '(bogus@example.org)';

$sanitized_a = filter_var($a, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_a, FILTER_VALIDATE_EMAIL)) {
```

```
    echo "Este endereço de e-mail (a) sanitizado é considerado válido.\n";
}

$sanitized_b = filter_var($b, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_b, FILTER_VALIDATE_EMAIL)) {
    echo "Este endereço de e-mail sanitizado é considerado válido.";
} else {
    echo "Este endereço de e-mail (b) sanitizado é considerado inválido.\n";
}

$sanitized_c = filter_var($c, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_c, FILTER_VALIDATE_EMAIL)) {
    echo "Este endereço de e-mail (c) sanitizado é considerado válido.\n";
    echo "Antes: $c\n";
    echo "Depois:  $sanitized_c\n";
}
?>
```

O exemplo acima produzirá:

```
Este endereço de e-mail (a) sanitizado é considerado válido.
Este endereço de e-mail (b) sanitizado é considerado inválido.
Este endereço de e-mail (c) sanitizado é considerado válido.
Antes: (bogus@example.org)
Depois: bogus@example.org
```

Funções da Filter

Índice

filter_has_var

filter_has_var

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_has_var — Verifica se uma variável de um tipo especificado existe

Descrição

function **filter_has_var**(int `$input_type`, string `$var_name`): bool

Parâmetros

`input_type`

Uma das constantes **`INPUT_GET`**, **`INPUT_POST`**, **`INPUT_COOKIE`**, **`INPUT_SERVER`** ou **`INPUT_ENV`**.

`var_name`

Nome da variável a ser verificada.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

filter_id

filter_id

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_id — Retorna o ID de filtro pertencente a um filtro nomeado

Descrição

function **filter_id**(string `$name`): int|false

Parâmetros

`name`

Nome do filtro a ser obtido.

Valor Retornado

ID do filtro em caso de sucesso ou `false` se o filtro não existir.

Veja também

- filter_list() - Retorna a lista de todos os filtros suportados

filter_input

filter_input

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_input — Obtém uma variável externa específica por nome e filtra-a opcionalmente

Descrição

```
function filter_input(
    int $type,
    string $var_name,
    int $filter = FILTER_DEFAULT,
    array|int $options = 0
): mixed
```

Parâmetros

`type`

Uma das constantes `INPUT_*`.

> **Aviso**
>
> O conteúdo da superglobal que está sendo filtrada é o conteúdo "bruto" original fornecido pela SAPI, antes de qualquer modificação de usuário na superglobal. Para filtrar uma superglobal modificada, use a função filter_var().

`var_name`

Nome de uma variável a filtrar dentro da superglobal `type` correspondente.

`filter`

O filtro a ser aplicado. Pode ser um filtro de validação pelo uso de uma das constantes `FILTER_VALIDATE_*`, um filtro de sanitização pelo uso de uma das constantes `FILTER_SANITIZE_*` ou de `FILTER_UNSAFE_RAW`, ou um filtro cutomizado com o uso de `FILTER_CALLBACK`.

> **Nota** :
>
> O padrão é `FILTER_DEFAULT`, que é um sinônimo de `FILTER_UNSAFE_RAW`. Isto resultará em nenhuma filtragem por padrão.

`options`

Um array associativo de opções, ou uma máscara de bits de constantes de opções de filtro `FILTER_FLAG_*`.

Se `filter` aceitar opções, elas podem ser fornecidas pelo uso do campo `"flags"` do array.

Valor Retornado

Valor da variável solicitada em caso de sucesso, `false` se o filtro falhar ou `null` se a varíavel `var_name` não estiver definida. Se o sinalizador `FILTER_NULL_ON_FAILURE` for usado, retorna `false` se a variável não estiver definida e `null` e o filtro falhar.

Examples

Exemplo #1 Um exemplo de filter_input()

```php
<?php
$search_html = filter_input(INPUT_GET, 'search', FILTER_SANITIZE_SPECIAL_CHARS);
$search_url = filter_input(INPUT_GET, 'search', FILTER_SANITIZE_ENCODED);
echo "Você pesquisou por $search_html.\n";
echo "<a href='?search=$search_url'>Pesquise novamente.</a>";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Você pesquisou por Me &#38; son.
<a href='?search=Me%20%26%20son'>Pesquise novamente.</a>
```

Veja também

- filter_input_array() - Obtem variáveis externas e opcionalmente as filtra
- filter_var() - Filtra uma variável com um filtro especificado
- filter_var_array() - Obtêm e opcionalmente filtra múltiplas variáveis
- Filtros de validação `FILTER_VALIDATE_*`
- Filtros de sanitização `FILTER_SANITIZE_*`

filter_input_array

filter_input_array

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_input_array — Obtem variáveis externas e opcionalmente as filtra

Descrição

function **filter_input_array**(int `$type`, array|int `$options` = `FILTER_DEFAULT`, bool `$add_empty` = `true`): array|false|null

Esta função é útil para receber muitos valores sem repetidamente chamar a função filter_input().

Parâmetros

`type`

Uma das constantes `INPUT_*`.

> **Aviso**
>
> O conteúdo da superglobal que está sendo filtrada é o conteúdo "bruto" original fornecido pela SAPI, antes de qualquer modificação de usuário na superglobal. Para filtrar uma superglobal modificada, use a função filter_var_array().

`options`

Pode ser um array associativo de opções, ou o filtro a aplicar a cada entrada, que pode ser um filtro de validação usando uma das constantes `FILTER_VALIDATE_*`, ou um filtro de sanitização, usando uma das constantes `FILTER_SANITIZE_*`.

O array de opções é um array associativo onde a chave corresponde a uma chave no `array` de dados e o valor associado é o filtro a ser aplicado a esta entrada ou um array associativo descrevendo como e qual filtro deve ser aplicado a esta entrada.

O array associativo que descreve como um filtro deve ser aplicado deve conter a chave `'filter'` cujo valor associado é o filtro a ser aplicado, que pode ser uma das constantes `FILTER_VALIDATE_*`, `FILTER_SANITIZE_*`, `FILTER_UNSAFE_RAW` ou `FILTER_CALLBACK`. Opcionalmente, ele pode conter a chave `'flags'` que especifica os sinalizadores que se aplicam ao filtro, e a chave `'options'` que especifica quaisquer opções que se aplicam ao filtro.

`add_empty`

Adiciona chaves ausentes como `null` ao valor de retorno.

Valor Retornado

Em caso de sucesso, um array contendo os valores das variáveis solicitadas.

Em caso de falha, retorna `false`. Exceto se a falha é aquela na qual o array de entrada designado por `type` não é populado onde `null` é retornado se a opção `FILTER_NULL_ON_FAILURE` for utilizada.

Entradas faltantes do array de entrada serão populadas no array retornado se `add_empty` for `true`. Neste caso, entradas faltantes serão definidas para `null`, a menos que a opção `FILTER_NULL_ON_FAILURE` seja usada, que neste caso serão `false`.

Uma entrada do array retornado será `false` se o filtro falhar, a menos que a opção `FILTER_NULL_ON_FAILURE` seja usada, que neste caso será `null`.

Notas

> **Nota** :
>
> Não existe a chave `REQUEST_TIME` no array `INPUT_SERVER` porque ela será inserida na variável *$_SERVER* mais tarde.

Veja também

- filter_input() - Obtém uma variável externa específica por nome e filtra-a opcionalmente
- filter_var() - Filtra uma variável com um filtro especificado
- filter_var_array() - Obtêm e opcionalmente filtra múltiplas variáveis
- Filtros de validação `FILTER_VALIDATE_*`
- Filtros de sanitização `FILTER_SANITIZE_*`

filter_list

filter_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_list — Retorna a lista de todos os filtros suportados

Descrição

function **filter_list**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array de nomes de todos os filtros suportados, um array vazio se não houver filtros. Índices deste array não são IDs de filtros, estes podem ser obtidos com filter_id() a partir do nome.

Examples

Exemplo #1 Um exemplo de filter_list()

```php
<?php
print_r(filter_list());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => int
    [1] => boolean
    [2] => float
    [3] => validate_regexp
```

```
    [4] => validate_url
    [5] => validate_email
    [6] => validate_ip
    [7] => string
    [8] => stripped
    [9] => encoded
    [10] => special_chars
    [11] => unsafe_raw
    [12] => email
    [13] => url
    [14] => number_int
    [15] => number_float
    [16] => magic_quotes
    [17] => callback
)
```

filter_var

filter_var

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_var — Filtra uma variável com um filtro especificado

Descrição

function **filter_var**(mixed `$value`, int `$filter` = `FILTER_DEFAULT`, array|int `$options` = 0): mixed

Filtra uma variável usando filtros de validação `FILTER_VALIDATE_*`, filtros de sanitização `FILTER_SANITIZE_*`, ou um filtro customizado.

Parâmetros

`value`

Valor a filtrar.

> **Aviso**
>
> Valores escalares são convertidos para strings internamente antes de serem filtrados.

`filter`

O filtro a ser aplicado. Pode ser um filtro de validação pelo uso de uma das constantes `FILTER_VALIDATE_*`, um filtro de sanitização pelo uso de uma das constantes `FILTER_SANITIZE_*` ou de `FILTER_UNSAFE_RAW`, ou um filtro cutomizado com o uso de `FILTER_CALLBACK`.

> **Nota** :
>
> O padrão é `FILTER_DEFAULT`, que é um sinônimo de `FILTER_UNSAFE_RAW`. Isto resultará em nenhuma filtragem por padrão.

`options`

Um array associativo de opções, ou uma máscara de bits de constantes de opções de filtro `FILTER_FLAG_*`.

Se `filter` aceitar opções, elas podem ser fornecidas pelo uso do campo `"flags"` do array.

Valor Retornado

Em caso de sucesso, retorna os dados filtrados. Em caso de falha, retorna `false`, a não ser que a opção `FILTER_NULL_ON_FAILURE` seja usada, neste caso retorna `null`.

Examples

Exemplo #1 Um exemplo de filter_var()

```php
<?php
var_dump(filter_var('bob@example.com', FILTER_VALIDATE_EMAIL));
var_dump(filter_var('https://example.com', FILTER_VALIDATE_URL, FILTER_FLAG_PATH_REQUIRED));
?>
```

O exemplo acima produzirá:

```
string(15) "bob@example.com"
bool(false)
```

Exemplo #2 Exemplo de validação de entradas de um array

```php
<?php
$emails = [
    "bob@example.com",
    "test@example.local",
    "invalidemail"
];

var_dump(filter_var($emails, FILTER_VALIDATE_EMAIL, FILTER_REQUIRE_ARRAY));
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  string(15) "bob@example.com"
  [1]=>
  string(18) "test@example.local"
  [2]=>
  bool(false)
}
```

Exemplo #3 Exemplo de passagem de um array em `options`

```php
<?php

$options = [
    'options' => [
        'min_range' => 10,
    ],
    'flags' => FILTER_FLAG_ALLOW_OCTAL,
];

var_dump(filter_var('0755', FILTER_VALIDATE_INT, $options));
var_dump(filter_var('011', FILTER_VALIDATE_INT, $options));

?>
```

O exemplo acima produzirá:

```
int(493)
bool(false)
```

Exemplo #4 Fornecendo opções diretamente ou através de um array

```php
<?php

$str = 'string';

var_dump(filter_var($str, FILTER_VALIDATE_BOOLEAN, FILTER_NULL_ON_FAILURE));
var_dump(filter_var($str, FILTER_VALIDATE_BOOLEAN, ['flags' => FILTER_NULL_ON_FAILURE]));

?>
```

O exemplo acima produzirá:

```
NULL
NULL
```

Veja também

- filter_var_array() - Obtêm e opcionalmente filtra múltiplas variáveis
- filter_input() - Obtém uma variável externa específica por nome e filtra-a opcionalmente
- filter_input_array() - Obtem variáveis externas e opcionalmente as filtra
- Filtros de validação `FILTER_VALIDATE_*`
- Filtros de sanitização `FILTER_SANITIZE_*`

filter_var_array

filter_var_array

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_var_array — Obtêm e opcionalmente filtra múltiplas variáveis

Descrição

function **filter_var_array**(array `$array`, array|int `$options` = `FILTER_DEFAULT`, bool `$add_empty` = `true`): array|false|null

Filtra um array associativo de valores usando filtros de validação `FILTER_VALIDATE_*`, filtros de sanitização `FILTER_SANITIZE_*`, ou filtros customizados.

Parâmetros

`array`

Um array associativo contendo os dados a filtrar.

`options`

Pode ser um array associativo de opções, ou o filtro a aplicar a cada entrada, que pode ser um filtro de validação usando uma das constantes `FILTER_VALIDATE_*`, ou um filtro de sanitização, usando uma das constantes `FILTER_SANITIZE_*`.

O array de opções é um array associativo onde a chave corresponde a uma chave no `array` de dados e o valor associado é o filtro a ser aplicado a esta entrada ou um array associativo descrevendo como e qual filtro deve ser aplicado a esta entrada.

O array associativo que descreve como um filtro deve ser aplicado deve conter a chave `'filter'` cujo valor associado é o filtro a ser aplicado, que pode ser uma das constantes `FILTER_VALIDATE_*`, `FILTER_SANITIZE_*`, `FILTER_UNSAFE_RAW` ou `FILTER_CALLBACK`. Opcionalmente, ele pode conter a chave `'flags'` que especifica os sinalizadores que se aplicam ao filtro, e a chave `'options'` que especifica quaisquer opções que se aplicam ao filtro.

`add_empty`

Adiciona chaves ausentes como `null` ao valor de retorno.

Valor Retornado

Um array contendo valores das variáveis requisitadas em caso de sucesso, ou `false` em caso de falha. Um valor de array será `false` se o filtro falhar, ou `null` se a variável não estiver definida.

Examples

Exemplo #1 Um exemplo de filter_var_array()

```php
<?php

$data = [
    'product_id' => 'libgd<script>',
    'component'  => '10',
    'versions'   => '2.0.33',
    'testscalar' => ['2', '23', '10', '12'],
    'testarray'  => '2',
];

$filters = [
    'product_id'   => FILTER_SANITIZE_ENCODED,
    'component'    => [
        'filter'   => FILTER_VALIDATE_INT,
        'flags'    => FILTER_FORCE_ARRAY,
        'options'  => [
            'min_range' => 1,
            'max_range' => 10,
        ],
    ],
    'versions'     => [
        'filter' => FILTER_SANITIZE_ENCODED
    ],
    'testscalar'   => [
        'filter' => FILTER_VALIDATE_INT,
        'flags'  => FILTER_REQUIRE_SCALAR,
    ],
    'testarray'    => [
        'filter' => FILTER_VALIDATE_INT,
        'flags'  => FILTER_FORCE_ARRAY,
    ],
    'doesnotexist' => FILTER_VALIDATE_INT,
];

var_dump(filter_var_array($data, $filters));

?>
```

O exemplo acima produzirá:

```
array(6) {
  ["product_id"]=>
  string(17) "libgd%3Cscript%3E"
  ["component"]=>
  array(1) {
    [0]=>
    int(10)
  }
  ["versions"]=>
  string(6) "2.0.33"
  ["testscalar"]=>
  bool(false)
  ["testarray"]=>
  array(1) {
    [0]=>
    int(2)
  }
  ["doesnotexist"]=>
```

```
  NULL
}
```

Veja também

- filter_input_array() - Obtem variáveis externas e opcionalmente as filtra
- filter_var() - Filtra uma variável com um filtro especificado
- filter_input() - Obtém uma variável externa específica por nome e filtra-a opcionalmente
- Filtros de validação `FILTER_VALIDATE_*`
- Filtros de sanitização `FILTER_SANITIZE_*`

Filter\FilterException

(No version information available, might only be in Git)

Introdução

A classe base para Exceptions lançadas pela extensão Filter.

Resumo da classe

namespace **Filter**;
class **FilterException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Filter\FilterFailedException

(No version information available, might only be in Git)

Introdução

Lançada quando um filtro de validação falha e a opção `FILTER_THROW_ON_FAILURE` está definida.

Resumo da classe

namespace **Filter**;
class **FilterFailedException** extends Filter\FilterException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;

```
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Gerenciamento de funções

Introdução

Todas estas funções lidam com várias operações envolvidas no trabalho com funções.

Funções para Manuseamento de Funções

Índice

call_user_func

call_user_func

(PHP 4, PHP 5, PHP 7, PHP 8)

call_user_func — Chama a função de retorno informada no primeiro parâmetro

Descrição

function **call_user_func**(callable `$callback`, mixed `...$args`): mixed

Chama a função informada no parâmetro `callback` e passa os argumentos restantes como argumentos para essa função.

Parâmetros

`callback`

A função do tipo callable a ser chamada.

`args`

Zero ou mais parâmetros a serem passados à função.

> **Nota** :
>
> Observe que os parâmetros para **call_user_func()** não são passados por referência.
>
> **Exemplo #1 Exemplo de call_user_func() e referências**

```php
<?php
error_reporting(E_ALL);
function increment(&$var)
{
    $var++;
}

$a = 0;
call_user_func('increment', $a);
echo $a."\n";

// é possível usar esta outra função alternativamente
call_user_func_array('increment', array(&$a));
echo $a."\n";

// é também possível usar uma função-variável
$increment = 'increment';
$increment($a);
echo $a."\n";
?>
```

O exemplo acima produzirá:

```
Warning: Parameter 1 to increment() expected to be a reference, value given in …
0
1
2
```

Valor Retornado

Retorna o mesmo valor retornado pela função de retorno.

Examples

Exemplo #2 Exemplo de call_user_func()

```php
<?php
function barbeiro($tipo)
{
    echo "Você quer um corte do tipo $tipo, sem problemas\n";
}
call_user_func('barbeiro', "cogumelo");
call_user_func('barbeiro', "moicano");
?>
```

O exemplo acima produzirá:

```
Você quer um corte do tipo cogumelo, sem problemas
Você quer um corte do tipo moicano, sem problemas
```

Exemplo #3 call_user_func() usando espaço de nomes

```php
<?php

namespace Foobar;

class Foo {
    static public function test() {
        print "Olá, mundo!\n";
    }
}

call_user_func(__NAMESPACE__ .'\Foo::test');
call_user_func(array(__NAMESPACE__ .'\Foo', 'test'));

?>
```

O exemplo acima produzirá:

```
Olá, mundo!
Olá, mundo!
```

Exemplo #4 Usando um método de classe com call_user_func()

```php
<?php

class myclass {
```

```php
    static function say_hello()
    {
        echo "Hello!\n";
    }
}

$classname = "myclass";

call_user_func(array($classname, 'say_hello'));
call_user_func($classname .'::say_hello');

$myobject = new myclass();

call_user_func(array($myobject, 'say_hello'));

?>
```

O exemplo acima produzirá:

```
Hello!
Hello!
Hello!
```

Exemplo #5 Usando função lambda com call_user_func()

```php
<?php
call_user_func(function($arg) { print "[$arg]\n"; }, 'test');
?>
```

O exemplo acima produzirá:

```
[test]
```

Notas

> **Nota** :
>
> Callbacks registrados com funções como **call_user_func()** e call_user_func_array() não serão chamados se houver uma exceção não capturada que foi lançada em um callback anterior.

Veja também

- call_user_func_array() - Chama uma função de retorno com um array de parâmetros
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual
- Funções-Variáveis
- ReflectionFunction::invoke() - Invoca função
- ReflectionMethod::invoke() - Invoca

call_user_func_array

call_user_func_array

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

call_user_func_array — Chama uma função de retorno com um array de parâmetros

Descrição

function **call_user_func_array**(callable `$callback`, array `$args`): mixed

Chama a função de retorno do definida no primeiro parâmetro `callback` com os parâmetros informados no parâmetro `args`.

Parâmetros

`callback`

A função do tipo callable a ser chamada.

`args`

Os parâmetros a serem passados à função de retorno, como um array.

Se as chaves do array de `args` forem todas numéricas, elas serão ignoradas e cada elemento será passado à função informada no parâmetro `callback` como um argumento potencial, em ordem.

Se qualquer chave do array `args` for uma string, esses elementos serão passados à função informada no parâmetro `callback` como argumentos nomeados, com o mesmo nome dado pela chave.

É um erro fatal quando uma chave numérica no array do parâmetro `args` aparece depois de uma chave string, ou se uma chave string não corresponder a nenhum parâmetro da função informada em `callback`.

Valor Retornado

Retorna o mesmo valor retornado pela função de retorno, ou `false` em caso de erro.

Registro de Alterações

Versão	Descrição
8.0.0	As chaves de `args` agora serão interpretadas como nomes de parâmetros, ao invés de serem silenciosamente ignorados.

Examples

Exemplo #1 Exemplo de call_user_func_array()

```php
<?php
function foobar($arg, $arg2) {
    echo __FUNCTION__, " recebeu $arg e $arg2\n";
}
class foo {
    function bar($arg, $arg2) {
        echo __METHOD__, " recebeu $arg e $arg2\n";
    }
}


// Chama a função foobar() com 2 argumentos
call_user_func_array("foobar", array("um", "dois"));

// Chama o método $foo->bar() com 2 argumentos
$foo = new foo;
call_user_func_array(array($foo, "bar"), array("três", "quatro"));
?>
```

O exemplo acima produzirá algo semelhante a:

```
foobar recebeu um e dois
foo::bar recebeu três e quatro
```

Exemplo #2 call_user_func_array() usando espaço de nomes

```php
<?php

namespace Foobar;

class Foo {
    static public function test($name) {
        print "Olá, {$name}!\n";
    }
}

call_user_func_array(__NAMESPACE__ .'\Foo::test', array('João'));

call_user_func_array(array(__NAMESPACE__ .'\Foo', 'test'), array('Felipe'));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Olá, João!
Olá, Felipe!
```

Exemplo #3 Usando função lambda

```php
<?php

$func = function($arg1, $arg2) {
    return $arg1 * $arg2;
};

var_dump(call_user_func_array($func, array(2, 4)));

?>
```

O exemplo acima produzirá:

```
int(8)
```

Exemplo #4 Passando valores por referência

```php
<?php

function mega(&$a){
    $a = 55;
    echo "função mega \$a=$a\n";
}
$bar = 77;
call_user_func_array('mega',array(&$bar));
echo "global \$bar=$bar\n";

?>
```

O exemplo acima produzirá:

```
função mega $a=55
global $bar=55
```

Exemplo #5 call_user_func_array() usando parâmetros nomeados

```php
<?php
function foobar($primeiro, $segundo) {
    echo __FUNCTION__, " recebeu $primeiro e $segundo\n";
}

// Chama a função foobar() com argumentos nomeados em ordem não posicional
call_user_func_array("foobar", array("segundo" => "dois", "primeiro" => "um"));

// Chama a função foobar() com um argumento nomeado
call_user_func_array("foobar", array("foo", "segundo" => "bar"));

// Erro fatal: Não é possível usar argumento posicional após um argumento nomeado
call_user_func_array("foobar", array("primeiro" => "um", "bar"));

?>
```

O exemplo acima produzirá algo semelhante a:

```
foobar recebeu um e dois
foobar recebeu foo e bar

Fatal error: Uncaught Error: Cannot use positional argument after named argument
```

Notas

> **Nota** :
>
> Callbacks registrados com funções como call_user_func() e **call_user_func_array()** não serão chamados se houver uma exceção não capturada que foi lançada em um callback anterior.

Veja também

- call_user_func() - Chama a função de retorno informada no primeiro parâmetro
- ReflectionFunction::invokeArgs() - Invoca argumentos de função
- ReflectionMethod::invokeArgs() - Invoca argumentos

create_function

create_function

(PHP 4 >= 4.0.1, PHP 5, PHP 7)

create_function — Cria uma função dinamicamente avaliando uma string de código

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.2.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **create_function**(string `$args`, string `$code`): string

Cria uma função dinamicamente a partir dos parâmetros passados e retorna um nome único para ela.

> **Cuidado**

> Essa função executa um eval() internamente e, por isso, tem os mesmos problemas de segurança de eval(). Além disso, ela tem características ruins de desempenho e uso de memória, porque as funções criadas são globais e não podem ser liberadas da memória.
>
> Uma função anônima nativa deve ser utilizada alternativamente.

Parâmetros

É recomendável passar estes parâmetros como strings em aspas simples. Se forem usadas strings em aspas duplas, os nomes de variáveis no código devem ser escapados com cuidado, ex.: `\$somevar`.

`args`

Os argumentos da função, em uma string única, separados por vírgulas.

`code`

O código da função.

Valor Retornado

Retorna um nome de função único como uma string, ou `false` em caso de falha. Note que o nome contém um caractere não imprimível (`"\0"`), portanto deve-se tomar cuidado ao imprimir o nome ou incorporá-lo em qualquer outra string.

Examples

Exemplo #1 Criando uma função dinamicamente, com create_function() ou funções anônimas

Pode-se usar uma função criada dinamicamente para (por exemplo) criar uma função a partir de informação obtida em tempo de execução. Primeiro, usando **create_function()**:

```php
<?php
$newfunc = create_function('$a,$b', 'return "ln($a) + ln($b) = " . log($a * $b);');
echo $newfunc(2, M_E) . "\n";
?>
```

Agora o mesmo código, usando uma função anônima; note que o código e os argumentos não estão mais contidos em strings:

```php
<?php
$newfunc = function($a,$b) { return "ln($a) + ln($b) = " . log($a * $b); };
echo $newfunc(2, M_E) . "\n";
?>
```

O exemplo acima produzirá:

```
ln(2) + ln(2.718281828459) = 1.6931471805599
```

Exemplo #2 Criação de uma função de processamento geral, com create_function() ou funções anônimas

Outro uso seria ter uma função manipuladora geral que possa aplicar um conjunto de operações a uma lista de parâmetros:

```php
<?php
function process($var1, $var2, $farr)
{
    foreach ($farr as $f) {
        echo $f($var1, $var2) . "\n";
    }
}

// cria uma porção de funções matemáticas
$farr = array(
    create_function('$x,$y', 'return "um pouco de trigonometria: ".(sin($x) + $x*cos($y));'),
    create_function('$x,$y', 'return "uma hipotenusa: ".sqrt($x*$x + $y*$y);'),
    create_function('$a,$b', 'if ($a >=0) {return "b*a^2 = ".$b*sqrt($a);} else {return false;}'),
```

```
    create_function('$a,$b', "return \"min(b^2+a, a^2,b) = \".min(\$a*\$a+\$b,\$b*\$b+\$a);"),
    create_function('$a,$b', 'if ($a > 0 && $b != 0) {return "ln(a)/b = ".log($a)/$b; } else { return false; }')
);

echo "\nUsando o primeiro array de funções dinâmicas\n";
echo "parâmetros: 2.3445, M_PI\n";
process(2.3445, M_PI, $farr);

// agora cria uma porção de funções de processamento de strings
$garr = array(
    create_function('$b,$a', 'if (strncmp($a, $b, 3) == 0) return "** \"$a\" '.
        'e \"$b\"\n** Parecem iguais para mim! (olhando para os 3 primeiros caracteres)";'),
    create_function('$a,$b', 'return "CRCs: " . crc32($a) . ", ".crc32($b);'),
    create_function('$a,$b', 'return "similar(a,b) = " . similar_text($a, $b, $p) . "($p%)";')
);
echo "\nUsando o segundo array de funções dinâmicas\n";
process("Twas brilling and the slithy toves", "Twas the night", $garr);
?>
```

Novamente, aqui está o mesmo código usando funções anônimas. Note que os nomes das variáveis no código não precisam mais ser escapados, porque não estão em uma string.

```
<?php
function process($var1, $var2, $farr)
{
    foreach ($farr as $f) {
        echo $f($var1, $var2) . "\n";
    }
}

// cria uma porção de funções matemáticas
$farr = array(
    function($x,$y) { return "um pouco de trigonometria: ".(sin($x) + $x*cos($y)); },
    function($x,$y) { return "uma hipotenusa: ".sqrt($x*$x + $y*$y); },
    function($a,$b) { if ($a >=0) {return "b*a^2 = ".$b*sqrt($a);} else {return false;} },
    function($a,$b) { return "min(b^2+a, a^2,b) = " . min($a*$a+$b, $b*$b+$a); },
    function($a,$b) { if ($a > 0 && $b != 0) {return "ln(a)/b = ".log($a)/$b; } else { return false; } }
);

echo "\nUsando o primeiro array de funções dinâmicas\n";
echo "parâmetros: 2.3445, M_PI\n";
process(2.3445, M_PI, $farr);

// agora cria uma porção de funções de processamento de strings
$garr = array(
    function($b,$a) { if (strncmp($a, $b, 3) == 0) return "** \"$a\" " .
        "e \"$b\"\n** Parecem iguais para mim! (olhando para os 3 primeiros caracteres)"; },
    function($a,$b) { return "CRCs: " . crc32($a) . ", ".crc32($b); },
    function($a,$b) { return "similar(a,b) = " . similar_text($a, $b, $p) . "($p%)"; }
);
echo "\nUsando o segundo array de funções dinâmicas\n";
process("Twas brilling and the slithy toves", "Twas the night", $garr);
?>
```

O exemplo acima produzirá:

```
Usando o primeiro array de funções dinâmicas
parâmetros: 2.3445, M_PI
um pouco de trigonometria: -1.6291725057799
uma hipotenusa: 3.9199852871011
b*a^2 = 4.8103313314525
min(b^2+a, a^2,b) = 8.6382729035898
ln(a)/b = 0.27122299212594

Usando o segundo array de funções dinâmicas
** "Twas the night" e "Twas brilling and the slithy toves"
** Parecem iguais para mim! (olhando para os 3 primeiros caracteres)
```

```
CRCs: -725381282, 342550513
similar(a,b) = 11(45.833333333333%)
```

Exemplo #3 Uso de funções anônimas como funções callback

Talvez o uso mais comum para funções dinâmicas seja passá-las como chamadas de retorno, por exemplo ao usar array_walk() ou usort().

```php
<?php
$av = array("a ", "uma ", "aquela ", "esta ");
array_walk($av, create_function('&$v,$k', '$v = $v . "manga";'));
print_r($av);
?>
```

Converted to an anonymous function:

```php
<?php
$av = array("a ", "uma ", "aquela ", "esta ");
array_walk($av, function(&$v,$k) { $v = $v . "manga"; });
print_r($av);
?>
```

O exemplo acima produzirá:

```
Array
(
  [0] => a manga
  [1] => uma manga
  [2] => aquela manga
  [3] => esta manga
)
```

Ordenando strings do mais longo ao mais curto com **create_function()**:

```php
<?php
$sv = array("pequena", "uma string grande", "pouco maior", "é uma coisa essa string");
echo "Original:\n";
print_r($sv);
echo "Ordenado:\n";
usort($sv, create_function('$a,$b','return strlen($b) - strlen($a);'));
print_r($sv);
?>
```

Convertida em função anônima:

```php
<?php
$sv = array("pequena", "uma string grande", "pouco maior", "é uma coisa essa string");
echo "Original:\n";
print_r($sv);
echo "Ordenado:\n";
usort($sv, function($a,$b) { return strlen($b) - strlen($a); });
print_r($sv);
?>
```

O exemplo acima produzirá:

```
Original:
Array
```

```
(
  [0] => pequena
  [1] => uma string grande
  [2] => pouco maior
  [3] => é uma coisa essa string
)
Sorted:
Array
(
  [0] => é uma coisa essa string
  [1] => uma string grande
  [2] => pouco maior
  [3] => pequena
)
```

Veja também

- Funções anônimas

forward_static_call

forward_static_call

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

forward_static_call — Chama um método estático

Descrição

function **forward_static_call**(callable `$callback`, mixed `...$args`): mixed

Chama uma função ou método definido pelo usuário fornecido pelo parâmetro `callback`, com os argumentos na sequência. Esta função deve ser chamada dentro de um contexto de método, não pode ser usada fora de uma classe. Ele usa a ligação estática tardia.

Parâmetros

`callback`

A função ou método a ser chamado. Este parâmetro pode ser um array, com o nome da classe e do método, ou uma string, com um nome de função.

`args`

Zero ou mais parâmetros a serem passados para a função.

Valor Retornado

Retorna o resultado da função ou `false` em caso de erro.

Examples

Exemplo #1 Exemplo de forward_static_call()

```php
<?php

class A
{
    const NAME = 'A';
    public static function test() {
        $args = func_get_args();
        echo static::NAME, " ".join(',', $args)." \n";
    }
}

class B extends A
{
    const NAME = 'B';
```

```
    public static function test() {
        echo self::NAME, "\n";
        forward_static_call(array('A', 'test'), 'mais', 'argumentos');
        forward_static_call( 'test', 'outros', 'argumentos');
    }
}

B::test('foo');

function test() {
        $args = func_get_args();
        echo "C ".join(',', $args)." \n";
    }

?>
```

O exemplo acima produzirá:

```
B
B mais,argumentos
C outros,argumentos
```

Veja também

- forward_static_call_array() - Chama um método estático e passa os argumentos como array
- call_user_func_array() - Chama uma função de retorno com um array de parâmetros
- call_user_func() - Chama a função de retorno informada no primeiro parâmetro
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual

forward_static_call_array

forward_static_call_array

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

forward_static_call_array — Chama um método estático e passa os argumentos como array

Descrição

function **forward_static_call_array**(callable `$callback`, array `$args`): mixed

Chama uma função ou método definido pelo usuário fornecido pelo parâmetro `callback`, com os argumentos passados como um array, de forma semelhante a call_user_func_array(). Quando chamada de dentro de um contexto de método, ela usa a ligação estática tardia. Quando chamada fora de um contexto de classe, ela se comporta como call_user_func_array(), sem ligação estática tardia.

> **Nota** :
>
> Diferentemente de forward_static_call(), esta função não requer um escopo de classe ativo: chamar forward_static_call() fora de uma classe lança um Error, enquanto esta função não.

Parâmetros

`callback`

A função ou método a ser chamado. Este parâmetro pode ser um array, com o nome da classe e do método, ou uma string, com um nome de função.

`args`

Um parâmetro, reunindo todos os parâmetros do método em um array.

> **Nota** :
>
> Observe que os parâmetros para **forward_static_call_array()** não são passados por referência.

Valor Retornado

Retorna o resultado da função ou `false` em caso de erro.

Examples

Exemplo #1 Exemplo de forward_static_call_array()

```
<?php

class A
{
    const NAME = 'A';
    public static function test() {
        $args = func_get_args();
        echo static::NAME, " ".join(',', $args)." \n";
    }
}

class B extends A
{
    const NAME = 'B';

    public static function test() {
        echo self::NAME, "\n";
        forward_static_call_array(array('A', 'test'), array('mais', 'argumentos'));
        forward_static_call_array( 'test', array('outros', 'argumentos'));
    }
}

B::test('foo');

function test() {
        $args = func_get_args();
        echo "C ".join(',', $args)." \n";
    }

?>
```

O exemplo acima produzirá:

```
B
B mais,argumentos
C outros,argumentos
```

Veja também

- forward_static_call() - Chama um método estático
- call_user_func() - Chama a função de retorno informada no primeiro parâmetro
- call_user_func_array() - Chama uma função de retorno com um array de parâmetros
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual

func_get_arg

func_get_arg

(PHP 4, PHP 5, PHP 7, PHP 8)

func_get_arg — Retorna um item de uma lista de argumentos

Descrição

function **func_get_arg**(int `$position`): mixed

Obtém o argumento especificado de uma lista de argumentos de uma função definida pelo usuário.

Esta função pode ser usada juntamente com func_get_args() e func_num_args() para permitir que funções definidas pelo usuário aceitem listas de argumentos com comprimento variável.

Parâmetros

`position`

O índice do argumento. Argumentos de função são contados iniciando de zero.

Valor Retornado

Retorna o argumento especificado, ou `false` em caso de erro.

Erros/Exceções

Gera um alerta se chamada fora de uma função definida pelo usuário, ou se `position` for maior que o número de argumentos realmente passados.

Examples

Exemplo #1 Exemplo de func_get_arg()

```php
<?php
function foo()
{
     $numargs = func_num_args();
     echo "Número de argumentos: $numargs<br />\n";
     if ($numargs >= 2) {
     echo "Segundo argumento é: " . func_get_arg(1) . "<br />\n";
     }
}

foo (1, 2, 3);
?>
```

O exemplo acima produzirá:

```
Número de argumentos: 3
Segundo argumento é: 2
```

Exemplo #2 Exemplo de func_get_arg() com argumentos por valor e por referência

```php
<?php
function byVal($arg) {
    echo 'Como passado     : ', var_export(func_get_arg(0)), PHP_EOL;
    $arg = 'baz';
    echo 'Depois da mudança: ', var_export(func_get_arg(0)), PHP_EOL;
}

function byRef(&$arg) {
    echo 'Como passado     : ', var_export(func_get_arg(0)), PHP_EOL;
    $arg = 'baz';
    echo 'Depois da mudança: ', var_export(func_get_arg(0)), PHP_EOL;
}

$arg = 'bar';
byVal($arg);
```

```
byRef($arg);
?>
```

O exemplo acima produzirá:

```
Como passado     : 'bar'
Depois da mudança: 'baz'
Como passado     : 'bar'
Depois da mudança: 'baz'
```

Notas

> **Nota** :
>
> A partir do PHP 8.0.0, a família de funções func_*() deve ser mais transparente em relação aos argumentos nomeados, tratando os argumentos como se fossem todos passados posicionalmente, e os argumentos ausentes são substituídos por seus valores padrão. Esta função ignora a coleção de argumentos variádicos nomeados desconhecidos. Argumentos nomeados desconhecidos que são coletados só podem ser acessados por meio do parâmetro variádico.

> **Nota** :
>
> Se os argumentos forem passados por referência, qualquer alteração nos argumentos será refletida nos valores retornados por esta função. A partir do PHP 7, os valores atuais também serão retornados se os argumentos forem passados por valor.

> **Nota** :
>
> Esta função retorna uma cópia apenas dos argumentos informados, e não leva em consideração argumentos padrão (não informados).

Veja também

- Sintaxe ...
- func_get_args() - Retorna um array contendo uma lista de argumentos da função
- func_num_args() - Retorna o número de argumentos passados para a função

func_get_args

func_get_args

(PHP 4, PHP 5, PHP 7, PHP 8)

func_get_args — Retorna um array contendo uma lista de argumentos da função

Descrição

function **func_get_args**(): array

Obtém um array de argumentos da função.

Esta função pode ser usada em conjunto com func_get_arg() e func_num_args() para permitir que funções definidas pelo usuário aceitem listas de argumentos de tamanhos variáveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array no qual cada elemento é uma cópia do membro correspondente da lista de argumentos da função atual definida pelo usuário.

Erros/Exceções

Gera um alerta se for chamado de fora de uma função definida pelo usuário.

Examples

Exemplo #1 Exemplo de func_get_args()

```php
<?php
function foo()
{
    $numargs = func_num_args();
    echo "Número de argumentos: $numargs \n";
    if ($numargs >= 2) {
        echo "O segundo argumento é: " . func_get_arg(1) . "\n";
    }
    $arg_list = func_get_args();
    for ($i = 0; $i < $numargs; $i++) {
        echo "Argumento $i é: " . $arg_list[$i] . "\n";
    }
}

foo(1, 2, 3);
?>
```

O exemplo acima produzirá:

```
Número de argumentos: 3
O segundo argumento é: 2
Argumento 0 é: 1
Argumento 1 é: 2
Argumento 2 é: 3
```

Exemplo #2 Exemplo de func_get_args() com argumentos por valor e por referência

```php
<?php
function byVal($arg) {
    echo 'Como informado : ', var_export(func_get_args()), PHP_EOL;
    $arg = 'baz';
    echo 'Após alterado  : ', var_export(func_get_args()), PHP_EOL;
}

function byRef(&$arg) {
    echo 'Como informado : ', var_export(func_get_args()), PHP_EOL;
    $arg = 'baz';
    echo 'Após alterado  : ', var_export(func_get_args()), PHP_EOL;
}

$arg = 'bar';
byVal($arg);
byRef($arg);
?>
```

O exemplo acima produzirá:

```
Como informado : array (
  0 => 'bar',
```

```
)
Após alterado  : array (
  0 => 'baz',
)
Como informado : array (
  0 => 'bar',
)
Após alterado  : array (
  0 => 'baz',
)
```

Notas

> **Nota** :
>
> A partir do PHP 8.0.0, a família de funções func_*() deve ser mais transparente em relação aos argumentos nomeados, tratando os argumentos como se fossem todos passados posicionalmente, e os argumentos ausentes são substituídos por seus valores padrão. Esta função ignora a coleção de argumentos variádicos nomeados desconhecidos. Argumentos nomeados desconhecidos que são coletados só podem ser acessados por meio do parâmetro variádico.

> **Nota** :
>
> Se os argumentos forem passados por referência, qualquer alteração nos argumentos será refletida nos valores retornados por esta função. A partir do PHP 7, os valores atuais também serão retornados se os argumentos forem passados por valor.

> **Nota** :
>
> Esta função retorna apenas uma cópia dos argumentos passados e não leva em conta os argumentos padrão (não passados).

Veja também

- Sintaxe `...`
- func_get_arg() - Retorna um item de uma lista de argumentos
- func_num_args() - Retorna o número de argumentos passados para a função
- ReflectionFunctionAbstract::getParameters() - Obtém parâmetros

func_num_args

func_num_args

(PHP 4, PHP 5, PHP 7, PHP 8)

func_num_args — Retorna o número de argumentos passados para a função

Descrição

function **func_num_args**(): int

Obtém o número de argumentos passados para a função.

Esta função pode ser usada em conjunto com func_get_arg() e func_get_args() para permitir que funções definidas pelo usuário aceitem listas de argumentos de comprimento variável.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de argumentos passados para a função atual definida pelo usuário.

Erros/Exceções

Gera um aviso se for chamado de fora de uma função definida pelo usuário.

Examples

Exemplo #1 Exemplo de func_num_args()

```php
<?php
function foo()
{
    echo "Número de argumentos: ", func_num_args(), PHP_EOL;
}

foo(1, 2, 3);
?>
```

O exemplo acima produzirá:

```
Número de argumentos: 3
```

Notas

> **Nota** :
>
> A partir do PHP 8.0.0, a família de funções func_*() deve ser mais transparente em relação aos argumentos nomeados, tratando os argumentos como se fossem todos passados posicionalmente, e os argumentos ausentes são substituídos por seus valores padrão. Esta função ignora a coleção de argumentos variádicos nomeados desconhecidos. Argumentos nomeados desconhecidos que são coletados só podem ser acessados por meio do parâmetro variádico.

Veja também

- Sintaxe ...
- func_get_arg() - Retorna um item de uma lista de argumentos
- func_get_args() - Retorna um array contendo uma lista de argumentos da função
- ReflectionFunctionAbstract::getNumberOfParameters() - Obtém o número de parâmetros

function_exists

function_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

function_exists — Retorna `true` se a função informada estiver definida

Descrição

function **function_exists**(string `$function`): bool

Verifica a lista de funções definidas, tanto as internas quanto as definidas pelo usuário, procurando por `function`.

Parâmetros

`function`

O nome da função, como uma string.

Valor Retornado

Retorna `true` se `function` existir e for uma função, `false` caso contrário.

> **Nota** :
>
> Esta função retornará `false` para instruções de linguagem, como include_once e echo.

Examples

Exemplo #1 Exemplo de function_exists()

```php
<?php
if (function_exists('imap_open')) {
    echo "Funções IMAP estão disponíveis.<br />\n";
} else {
    echo "Funções IMAP não estão disponíveis.<br />\n";
}
?>
```

Notas

> **Nota** :
>
> Um nome de função pode existir mesmo se a própria função não puder ser usada devido a opções de configuração ou compilação (por exemplo, as funções image).

Veja também

- method_exists() - Verifica se o método da classe existe
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual
- get_defined_functions() - Retorna um array com todas as funções definidas
- class_exists() - Verifica se uma classe foi definida
- extension_loaded() - Descobre se uma extensão está carregada

get_defined_functions

get_defined_functions

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

get_defined_functions — Retorna um array com todas as funções definidas

Descrição

function **get_defined_functions**(bool `$exclude_disabled = true`): array

Obtém um array com todas as funções definidas.

Parâmetros

`exclude_disabled`

Opção para definir se funções desabilitadas devem ser excluídas do valor de retorno. Este parâmetro não tem efeito desde o PHP 8.0.0.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.5.0. O uso deste recurso é fortemente desencorajado.

Valor Retornado

Retorna um array multidimensional contendo uma lista de todas as funções definidas, tanto as internas quanto as definidas pelo usuário. As funções internas podem ser acessadas em *$arr["internal"]*, e as definidas por usuário em *$arr["user"]* (veja exemplo abaixo).

Registro de Alterações

Versão	Descrição
8.5.0	O parâmetro `exclude_disabled` foi descontinuado, pois não tem mais efeito.
8.0.0	O valor padrão do parâmetro `exclude_disabled` mudou de `false` para `true`. Entretanto, ele não terá qualquer efeito já que as funções desabilitadas são removidas da tabela de funções no momento da compilação.
7.0.15, 7.1.1	O parâmetro `exclude_disabled` foi adicionado.

Examples

Exemplo #1 Exemplo de get_defined_functions()

```php
<?php
function myrow($id, $data)
{
    return "<tr><th>$id</th><td>$data</td></tr>\n";
}

$arr = get_defined_functions();

print_r($arr);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [internal] => Array
        (
            [0] => zend_version
            [1] => func_num_args
            [2] => func_get_arg
            [3] => func_get_args
            [4] => strlen
            [5] => strcmp
            [6] => strncmp
            ...
            [750] => bcscale
            [751] => bccomp
        )

    [user] => Array
        (
            [0] => myrow
        )

)
```

Veja também

- function_exists() - Retorna true se a função informada estiver definida
- get_defined_vars() - Retorna um array com todas variáveis definidas
- get_defined_constants() - Retorna um array associativo com os nomes de todas as constantes e seus valores
- get_declared_classes() - Retorna um array com os nomes das classes definidas

register_shutdown_function

register_shutdown_function

(PHP 4, PHP 5, PHP 7, PHP 8)

register_shutdown_function — Registra uma função para ser executada no final da execução do script

Descrição

function **register_shutdown_function**(callable `$callback`, mixed `...$args`): void

Registra a função nomeada por `callback` para ser executada após o término da execução do script ou após a chamada a exit().

Múltiplas chamadas a **register_shutdown_function()** podem ser feitas, e cada uma irá ser chamada na mesma ordem em que foram registradas. Se exit() for chamada dentro de uma função de desligamento, o processamento irá parar completamente e nenhuma outra função de desligamento será chamada.

> **Cuidado**
>
> Desde o PHP 8.4.0, uma chamada a exit() sem parâmetros dentro de uma função de desligamento registrada redefine o código de saída para `0`. Chamar exit() com um status específico substitui o código de saída anterior em todas as versões.

As próprias funções de desligamento também podem chamar **register_shutdown_function()** para adicionar uma função de desligamento ao final da fila.

Parâmetros

`callback`

A função de desligamento a ser registrada.

As chamadas às funções de desligamento são executadas como parte da requisição, portanto é possível enviar resultados a partir delas e acessar buffers de saída.

`args`

É possível passar parâmetros para a função de desligamento passando parâmetros adicionais.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de register_shutdown_function()

```php
<?php
function desligamento()
{
    // Esta é uma função de desligamento, aqui
    // podem ser feitas as últimas operações
    // antes que o script seja concluído.

    echo 'Script executado com sucesso', PHP_EOL;
}

register_shutdown_function('desligamento');
?>
```

Notas

> **Nota** :
>
> O diretório de trabalho do script pode mudar dentro da função de desligamento em alguns servidores web, por exemplo, no Apache.

> **Nota** :

> As funções de desligamento não serão executadas se o processo for encerrado com um sinal SIGTERM ou SIGKILL. Embora um SIGKILL não possa ser interceptado, pode ser usada a função pcntl_signal() para instalar um manipulador para um SIGTERM que usa exit() para terminar de forma limpa.

> **Nota** :
>
> As funções de desligamento são executadas separadamente do tempo monitorado por max_execution_time. Isso significa que mesmo que um processo seja encerrado por ter sido executado por muito tempo, as funções de desligamento ainda serão chamadas. Além disso, se o `max_execution_time` acabar enquanto uma função de desligamento estiver em execução, ela não será encerrada.

Veja também

- auto_append_file
- exit() - Termina o script atual com um código ou uma mensagem de status
- fastcgi_finish_request() - Envia todos os dados de resposta ao cliente
- A seção sobre manipulação de conexões

register_tick_function

register_tick_function

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

register_tick_function — Registra uma função para execução em cada tique

Descrição

function **register_tick_function**(callable `$callback`, mixed `...$args`): bool

Registra a função informada no parâmetro `callback` para ser executada quando um tique for chamado.

Parâmetros

`callback`

A função a ser registrada.

`args`

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de register_tick_function()

```php
<?php
declare(ticks=1);

function my_tick_function($param) {
    echo "Função de retorno de tique chamada com o parâmetro: $param\n";
}

register_tick_function('my_tick_function', true);
?>
```

Veja também

- declare
- unregister_tick_function() - Desregistra uma função executada a cada tique

unregister_tick_function

unregister_tick_function

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

unregister_tick_function — Desregistra uma função executada a cada tique

Descrição

function **unregister_tick_function**(callable `$callback`): void

Desregistra a função nomeada por `callback` de forma que não seja mais executada quando um tique for chamado.

Parâmetros

`callback`

O nome da função a ser desregistrada.

Valor Retornado

Nenhum valor é retornado.

Veja também

- register_tick_function() - Registra uma função para execução em cada tique

Informação de Classe/Objeto

Introdução

Estas funções permitem obter informações sobre classes e instâncias de objetos. Pode-se obter o nome da classe ao qual um objeto pertence, bem como suas propriedades e métodos. Usando estas funções, pode-se descobrir não apenas a classe de um objeto, mas também seu parentesco (no caso, que classe este objeto de classe estende).

Por favor, veja a seção Objetos do manual para uma explicação detalhada de como classes e objetos são implementados e utilizados no PHP.

Examples

Neste exemplo, nós primeiramente definiremos uma classe base e uma extensão da classe. A classe base descreve um vegetal comum, que é comestível ou não e qual é sua cor. A sub-classe *Espinafre* adiciona um método para cozinhá-lo e outro para descobrir se ele está cozido.

Exemplo #1 Definições da Classe

Vegetal

```php
<?php

class Vegetal {
    public $comestivel;

    public $cor;

    public function __construct($comestivel, $cor = "verde")
    {
        $this->comestivel = $comestivel;
        $this->cor = $cor;
    }

    public function comestivel()
    {
        return $this->comestivel;
    }

    public function qual_cor()
    {
        return $this->cor;
    }
}

?>
```

Espinafre

```php
<?php

class Espinafre extends Vegetal {
    public $cozido = false;

    public function __construct()
    {
        parent::__construct(true, "verde");
    }

    public function cozinhe()
    {
        $this->cozido = true;
    }

    public function esta_cozido()
    {
```

```
        return $this->cozido;
    }
}

?>
```

Agora serão instanciados objetos destas classes e exibidas informações sobre elas, incluindo o parentesco entre elas. Também serão definidas algumas funções úteis, principalmente para ter uma saída bem formatada de variáveis.

Exemplo #2 test_script.php

```
<?php

// register autoloader to load classes
spl_autoload_register();

function printProperties($obj)
{
    foreach (get_object_vars($obj) as $prop => $val) {
        echo "\t$prop = $val\n";
    }
}

function printMethods($obj)
{
$arr = get_class_methods(get_class($obj));
    foreach ($arr as $metodo) {
        echo "\tfunção $metodo()\n";
    }
}

function objetoPertenceA($obj, $classe)
{
    if (is_subclass_of($obj, $classe)) {
        echo "Objeto pertence à classe " . get_class($obj);
        echo ", uma sub-classe de $classe\n";
    } else {
        echo "Objeto não pertence a uma sub-classe de $classe\n";
    }
}

// instancia 2 objetos

$vegetal = new Vegetal(true, "azul");
$folha = new Espinafre();

// mostra informação sobre os objetos
echo "vegetal: CLASSE " . get_class($vegetal) . "\n";
echo "folha: CLASSE " . get_class($folha);
echo ", CLASSE PAI " . get_parent_class($folha) . "\n";

// mostra propriedades do vegetal
echo "\nvegetal: Propriedades\n";
printProperties($vegetal);

// e métodos de folha
echo "\nfolha: Métodos\n";
printMethods($folha);

echo "\nParentesco:\n";
objetoPertenceA($folha, Espinafre::class);
objetoPertenceA($folha, Vegetal::class);
?>
```

Os exemplos acima produzirão:

```
vegetal: CLASSE Vegetal
folha: CLASSE Espinafre, CLASSE PAI Vegetal
```

```
vegetal: Propriedades
        comestivel = 1
        cor = azul

folha: Métodos
        function __construct()
        function cozinhe()
        function esta_cozido()
        function comestivel()
        function qual_cor()

Parentesco:
Objeto não pertence a uma sub-classe de Espinafre
Objeto pertence à classe Espinafre, uma sub-classe de Vegetal
```

Uma nota importante sobre o exemplo acima é que o objeto *$folha* é uma instância da classe **Espinafre** que é uma sub-classe de **Vegetal**.

Funções para Classes/Objetos

Índice

__autoload

__autoload

(PHP 5, PHP 7)

__autoload — Tenta carregar uma classe indefinida

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.2.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **__autoload**(string `$class`): void

Pode-se definir esta função para habilitar carregamento automático de classes.

Parâmetros

`class`

Nome da classe a ser carregada

Valor Retornado

Nenhum valor é retornado.

Veja também

- spl_autoload_register() - Registra a função dada como implementação __autoload()

class_alias

class_alias

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

class_alias — Cria um apelido para uma classe

Descrição

function **class_alias**(string `$class`, string `$alias`, bool `$autoload = true`): bool

Cria um apelido com o nome informado em `alias` com base na classe definida pelo usuário `class`. A classe com apelido é exatamente igual à classe original.

O apelido de classe não pode ser nenhuma das palavras reservadas do PHP.

> **Nota** :
>
> A partir do PHP 8.3.0, **class_alias()** também suporta a criação de um apelido de uma classe interna do PHP.

Parâmetros

`class`

A classe original.

`alias`

O nome do alias da classe.

`autoload`

Se deve-se carregar automaticamente caso a classe original não seja encontrada.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.3.0	**class_alias()** agora suporta criação de apelido de uma classe interna.

Examples

Exemplo #1 Exemplo de class_alias()

```php
<?php

class Foo { }

class_alias('Foo', 'Bar');

$a = new Foo;
$b = new Bar;

// os objetos são os mesmos
var_dump($a == $b, $a === $b);
var_dump($a instanceof $b);

// as classes são as mesmas
var_dump($a instanceof Foo);
var_dump($a instanceof Bar);

var_dump($b instanceof Foo);
var_dump($b instanceof Bar);

?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
bool(true)
bool(true)
bool(true)
```

```
bool(true)
bool(true)
```

Notas

> **Nota** :
>
> Nomes de classe não são sensíveis a maiúsculas/minúsculas no PHP, e isto é refletido nesta função. Apelidos criados por **class_alias()** são declarados em minúsculas. Isto significa que para uma classe `MinhaClasse`, a chamada a `class_alias('MinhaClasse', 'ApelidoDaMinhaClasse')` irá declarar um novo apelido de classe chamado `apelidodaminhaclasse`.

Veja também

- get_parent_class() - Obtém o nome da classe pai para o objeto ou classe
- is_subclass_of() - Verifica se o objeto tem esta classe como uma de suas classes pai

class_exists

class_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

class_exists — Verifica se uma classe foi definida

Descrição

function **class_exists**(string `$class`, bool `$autoload` = `true`): bool

Esta função verifica se uma classe foi definida.

Parâmetros

`class`

O nome da classe. O nome da classe é verificada sem fazer distinção de maiúscula e minúscula.

`autoload`

Se deve-se carregar automaticamente caso não tenha sido carregada.

Valor Retornado

Retorna `true` se `class` é uma classe definida, caso contrário retorna `false`.

Examples

Exemplo #1 Exemplo de class_exists()

```php
<?php
// Verifica que a classe existe antes de se tentar usá-la
if (class_exists('MinhaClasse')) {
    $minhaclasse = new MinhaClasse();
}

?>
```

Exemplo #2 Exemplo do parâmetro `autoload`

```php
<?php
spl_autoload_register(function ($nome_classe) {
    include $nome_classe . '.php';

    // Verifica se o arquivo incluído declarou a classe
    if (!class_exists($nome_classe, false)) {
        throw new LogicException("Não foi possível carregar a classe: $nome_classe");
```

```
    }
});

if (class_exists(MinhaClasse::class)) {
    $minhaclasse = new MinhaClasse();
}

?>
```

Veja também

- function_exists() - Retorna true se a função informada estiver definida
- enum_exists() - Verifica se uma enumeração foi definida
- interface_exists() - Verifica se a interface foi definida
- get_declared_classes() - Retorna um array com os nomes das classes definidas

enum_exists

enum_exists

(PHP 8 >= 8.1.0)

enum_exists — Verifica se uma enumeração foi definida

Descrição

function **enum_exists**(string `$enum`, bool `$autoload` = **`true`**): bool

Esta função verifica se o enum fornecido foi definido.

Parâmetros

`enum`

O nome da enumetação. O nome é verificado sem sensibilidade a maiúsculas/minúsculas.

`autoload`

Se deve-se carregar automaticamente a enumeração caso não tenha sido carregada.

Valor Retornado

Retorna **`true`** se `enum` é uma enumeração definida, **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de enum_exists()

```
<?php
// Verifica se a enumeração existe antes de tentar usá-la
if (enum_exists(Suit::class)) {
    $myclass = Suit::Hearts;
}
?>
```

Veja também

- function_exists() - Retorna true se a função informada estiver definida
- class_exists() - Verifica se uma classe foi definida
- interface_exists() - Verifica se a interface foi definida
- get_declared_classes() - Retorna um array com os nomes das classes definidas

get_called_class

get_called_class

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

get_called_class — O nome da classe da qual o método estático foi chamado

Descrição

function **get_called_class**(): string

Obtém o nome da classe da qual o método estático foi chamado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome da classe.

Erros/Exceções

Se **get_called_class()** for chamada de fora de uma classe, um Error é disparado. Antes do PHP 8.0.0, um aviso `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.0.0	Chamar esta função de fora de uma classe agora irá disparar um Error. Anteriormente, um aviso `E_WARNING` era emitido e a função retornava `false`.

Examples

Exemplo #1 Usando get_called_class()

```php
<?php

class foo {
    static public function test() {
        var_dump(get_called_class());
    }
}

class bar extends foo {
}

foo::test();
bar::test();

?>
```

O exemplo acima produzirá:

```
string(3) "foo"
string(3) "bar"
```

Veja também

- get_parent_class() - Obtém o nome da classe pai para o objeto ou classe
- get_class() - Retorna o nome da classe de um objeto
- is_subclass_of() - Verifica se o objeto tem esta classe como uma de suas classes pai

get_class

get_class

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class — Retorna o nome da classe de um objeto

Descrição

function **get_class**(object `$object` = ?): string

Obtém o nome da classe do `object` informado.

Parâmetros

`object`

O objeto testado.

> **Nota** :
>
> Passar explicitamente `null` no parâmetro `object` não é mais permitido a partir do PHP 7.2.0 e emite um `E_WARNING`. A partir do 8.0.0, um TypeError é emitido quando `null` é usado.

Valor Retornado

Retorna o nome da classe da qual `object` é uma instância.

Se `object` for uma instância de uma classe que existe em um espaço de nomes, o nome da classe qualificado com espaço de nomes é retornado.

Erros/Exceções

Se **get_class()** for chamada com qualquer outro parâmetro que não seja um objeto, TypeError é disparado. Antes do PHP 8.0.0, um alerta `E_WARNING` era emitido.

Se **get_class()** for chamada sem parâmetros de fora de uma classe, um Error é disparado. Antes do PHP 8.0.0, um alerta `E_WARNING` era emitido.

Registro de Alterações

Versão	Descrição
8.3.0	Chamar **get_class()** sem um argumento agora emite um alerta `E_DEPRECATED`; anteriormente, chamar esta função de dentro de uma classe retornava o nome dessa classe.
8.0.0	Chamar esta função de fora de uma classe, sem argumentos, agora dispara um Error. Anteriormente, um alerta `E_WARNING` era emitido e a função retornava `false`.
7.2.0	Antes desta versão o valor padrão para `object` era `null` e tinha o mesmo efeito de não se passar nenhum valor. Agora, `null` foi removido do valor padrão para `object`, e não é mais uma entrada válida.

Examples

Exemplo #1 Usando **get_class()**

```php
<?php

class foo {
    function name()
    {
        echo "Meu nome é " , get_class($this) , "\n";
    }
}

// cria um objeto
$bar = new foo();

// chamada externa
echo "Seu nome é " , get_class($bar) , "\n";

// chamada interna
$bar->name();

?>
```

O exemplo acima produzirá:

```
Meu nome é foo
Seu nome é foo
```

Exemplo #2 Usando get_class() em superclasse

```php
<?php

abstract class bar {
    public function __construct()
    {
        var_dump(get_class($this));
        var_dump(get_class());
    }
}

class foo extends bar {
}

new foo;

?>
```

O exemplo acima produzirá:

```
string(3) "foo"
string(3) "bar"
```

Exemplo #3 Usando get_class() em classe com espaço de nomes

```php
<?php

namespace Foo\Bar;

class Baz {
    public function __construct()
    {

    }
}

$baz = new \Foo\Bar\Baz;

var_dump(get_class($baz));
?>
```

O exemplo acima produzirá:

```
string(11) "Foo\Bar\Baz"
```

Veja também

- get_called_class() - O nome da classe da qual o método estático foi chamado
- get_parent_class() - Obtém o nome da classe pai para o objeto ou classe
- gettype() - Obtém o tipo de uma variável
- get_debug_type() - Obtém o nome do tipo de uma variável de forma adequada a depuração
- is_subclass_of() - Verifica se o objeto tem esta classe como uma de suas classes pai

get_class_methods

get_class_methods

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class_methods — Obtém o nome dos métodos da classe

Descrição

function **get_class_methods**(object|string `$object_or_class`): array

Obtém o nome dos métodos da classe.

Parâmetros

`object_or_class`

O nome da classe ou a instância do objeto

Valor Retornado

Retorna um array de nome de métodos definidos para a classe especificada por `object_or_class`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `object_or_class` agora aceita apenas objetos ou nomes de classe válidos.

Examples

Exemplo #1 Exemplo de get_class_methods()

```php
<?php

class minhaclasse {
    // construtor
    function __construct()
    {
        return(true);
    }

    // método 1
    function minhafuncao1()
    {
        return(true);
    }

    // método 2
    function minhafuncao2()
    {
        return(true);
    }
}

$metodos_da_classe = get_class_methods('minhaclasse');
// ou
$metodos_da_classe = get_class_methods(new minhaclasse());

foreach ($metodos_da_classe as $nome_do_metodo) {
    echo "$nome_do_metodo\n";
}

?>
```

O exemplo acima produzirá:

```
__construct
minhafuncao1
minhafuncao2
```

Veja também

- get_class() - Retorna o nome da classe de um objeto
- get_class_vars() - Obtém as propriedades padrão da classe
- get_object_vars() - Obtém as propriedades de um objeto

get_class_vars

get_class_vars

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class_vars — Obtém as propriedades padrão da classe

Descrição

function **get_class_vars**(string `$class`): array

Obtém as propriedades padrão de uma dada classe.

Parâmetros

`class`

O nome da classe

Valor Retornado

Retorna um array associativo das propriedades declaradas visíveis no escopo atual, com seu valor padrão. Os elementos do array resultante estão na forma de `varname => value`. Em caso de um erro, retornará **`false`**.

Examples

Exemplo #1 Exemplo da get_class_vars()

```php
<?php

class MyClass
{
    public $var1; // Esta não tem valor padrão explícito (tecnicamente tem NULL como padrão)...
    public $var2 = "xyz";
    public $var3 = 100;
    private $var4;

    public function __construct()
    {
        // altera algumas propriedades
        $this->var1 = "foo";
        $this->var2 = "bar";
        return true;
    }
}

$my_class = new MyClass();

$class_vars = get_class_vars(get_class($my_class));

foreach ($class_vars as $name => $value) {
    echo "{$name}: ", var_export($value, true), "\n";
```

```
}

?>
```

O exemplo acima produzirá:

```
var1: NULL
var2: 'xyz'
var3: 100
```

Exemplo #2 get_class_vars() e comportamento de escopo

```
<?php

function format($array)
{
    return implode('|', array_keys($array)) . "\r\n";
}

class TestCase
{
    public $a    = 1;
    protected $b = 2;
    private $c   = 3;

    public static function expose()
    {
        echo format(get_class_vars(__CLASS__));
    }
}

TestCase::expose();
echo format(get_class_vars('TestCase'));

?>
```

O exemplo acima produzirá:

```
// 5.0.0
a| * b| TestCase c
a| * b| TestCase c

// 5.0.1 - 5.0.2
a|b|c
a|b|c

// 5.0.3 +
a|b|c
a
```

Veja também

- get_class_methods() - Obtém o nome dos métodos da classe
- get_object_vars() - Obtém as propriedades de um objeto

get_declared_classes

get_declared_classes

(PHP 4, PHP 5, PHP 7, PHP 8)

get_declared_classes — Retorna um array com os nomes das classes definidas

Descrição

function **get_declared_classes**(): array

Obtém as classes declaradas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array de nomes das classes declaradas no script atual.

> **Nota** :
>
> Observe que, dependendo de quais bibliotecas tenham sido compiladas ou carregadas no PHP, classes adicionais podem estar presentes. Isto significa que não será possível definir classes de usuário usando estes nomes. Há uma lista de classes predefinidas na seção Classes Predefinidas dos apêndices.

Registro de Alterações

Versão	Descrição
7.4.0	Anteriormente a função **get_declared_classes()** sempre retornada classes pais antes das classes filhas. Isto não é mais o que acontece. Nenhuma ordem em particular é garantida para o valor de retorno de **get_declared_classes()**.

Examples

Exemplo #1 Exemplo de get_declared_classes()

```php
<?php
print_r(get_declared_classes());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => stdClass
    [1] => __PHP_Incomplete_Class
    [2] => Directory
)
```

Veja também

- class_exists() - Verifica se uma classe foi definida
- get_declared_interfaces() - Retorna um array com todas as interfaces declaradas
- get_defined_functions() - Retorna um array com todas as funções definidas

get_declared_interfaces

get_declared_interfaces

(PHP 5, PHP 7, PHP 8)

get_declared_interfaces — Retorna um array com todas as interfaces declaradas

Descrição

function **get_declared_interfaces**(): array

Retorna as interfaces declaradas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Essa função retorna um array com os nomes de interfaces declaradas no script atual.

Examples

Exemplo #1 Exemplo de uso de get_declared_interfaces()

```php
<?php
print_r(get_declared_interfaces());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Traversable
    [1] => IteratorAggregate
    [2] => Iterator
    [3] => ArrayAccess
    [4] => reflector
    [5] => RecursiveIterator
    [6] => SeekableIterator
)
```

Veja também

- interface_exists() - Verifica se a interface foi definida
- get_declared_classes() - Retorna um array com os nomes das classes definidas
- class_implements() - Retorna as interfaces que são implementadas pela classe ou interface fornecida

get_declared_traits

get_declared_traits

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

get_declared_traits — Retorna um array de todas as Traits declaradas

Descrição

function **get_declared_traits**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com todos os nomes das Traits declaradas em seus valores.

Veja também

- class_uses() - Retorna os traits usados pela classe fornecida
- trait_exists() - Verifica se a trait existe

get_mangled_object_vars

get_mangled_object_vars

(PHP 7 >= 7.4.0, PHP 8)

get_mangled_object_vars — Retorna um array de nomes únicos e valores das propriedades do objeto

Descrição

function **get_mangled_object_vars**(object `$object`): array

Retorna um array cujos elementos são as propriedades de `object`. As chaves são os nomes das variáveis membras, com algumas exceções notáveis: variáveis privadas têm o nome da classe prefixada ao nome da variável, e variáveis protegidas têm um `*` prefixado ao nome da variável. Estes valores prefixados têm bytes `NUL` de cada um dos lados. Propriedades tipadas não inicializadas são silenciosamente descartadas.

Parâmetros

`object`

Uma instância de objeto.

Valor Retornado

Retorna um array contendo todas as propriedades de `object`, independente da visibilidade.

Examples

Exemplo #1 Exemplo de get_mangled_object_vars()

```php
<?php

class A
{
    public $public = 1;

    protected $protected = 2;

    private $private = 3;
}

class B extends A
{
    private $private = 4;
}

$object = new B;
$object->dynamic = 5;
$object->{'6'} = 6;

var_dump(get_mangled_object_vars($object));

class AO extends ArrayObject
{
    private $private = 1;
}

$arrayObject = new AO(['x' => 'y']);
$arrayObject->dynamic = 2;

var_dump(get_mangled_object_vars($arrayObject));
```

O exemplo acima produzirá:

```
array(6) {
  ["Bprivate"]=>
  int(4)
  ["public"]=>
```

```
  int(1)
  ["*protected"]=>
  int(2)
  ["Aprivate"]=>
  int(3)
  ["dynamic"]=>
  int(5)
  [6]=>
  int(6)
}
array(2) {
  ["AOprivate"]=>
  int(1)
  ["dynamic"]=>
  int(2)
}
```

Veja também

- get_class_vars() - Obtém as propriedades padrão da classe
- get_object_vars() - Obtém as propriedades de um objeto

get_object_vars

get_object_vars

(PHP 4, PHP 5, PHP 7, PHP 8)

get_object_vars — Obtém as propriedades de um objeto

Descrição

function **get_object_vars**(object `$object`): array

Obtém as propriedades não estáticas acessíveis do `object` informado de acordo com o escopo.

Parâmetros

`object`

Uma instância de objeto.

Valor Retornado

Retorna um array associativo das propriedades não estáticas acessíveis definidas para o `object` especificado dentro do escopo.

Examples

Exemplo #1 Uso da get_object_vars()

```php
<?php

class foo {
    private $a;
    public $b = 1;
    public $c;
    private $d;
    static $e;

    public function test() {
        var_dump(get_object_vars($this));
    }
}

$test = new foo;
var_dump(get_object_vars($test));

$test->test();
```

```
?>
```

O exemplo acima produzirá:

```
array(2) {
  ["b"]=>
  int(1)
  ["c"]=>
  NULL
}
array(4) {
  ["a"]=>
  NULL
  ["b"]=>
  int(1)
  ["c"]=>
  NULL
  ["d"]=>
  NULL
}
```

> **Nota** :
>
> Propriedades não inicializadas são consideradas inacessíveis, e portanto não serão incluídas no array.

Veja também

- get_class_methods() - Obtém o nome dos métodos da classe
- get_class_vars() - Obtém as propriedades padrão da classe

get_parent_class

get_parent_class

(PHP 4, PHP 5, PHP 7, PHP 8)

get_parent_class — Obtém o nome da classe pai para o objeto ou classe

Descrição

function **get_parent_class**(object|string `$object_or_class` = ?): string|false

Obtém o nome da classe pai para o objeto ou classe.

Parâmetros

`object_or_class`

O objeto testado ou o nome da classe.

Valor Retornado

Retorna o nome da classe pai da classe da qual `object_or_class` é uma instância ou o nome.

Se o objeto não tiver um pai ou a classe informada não existir, será retornado `false`.

Registro de Alterações

Versão	Descrição
8.3.0	Chamar **get_parent_class()** sem um argumento agora emite um alerta `E_DEPRECATED`; anteriormente, chamar esta função de dentro de uma classe retornava o nome dessa classe.

Versão	Descrição
8.0.0	O parâmetro `object_or_class` agora aceita apenas objetos ou nomes de classe válidos.

Examples

Exemplo #1 Usando get_parent_class()

```php
<?php

class Pai {
    function __construct()
    {
    // implemente alguma lógica
    }
}

class Filha extends Pai {
    function __construct()
    {
        echo "Eu sou filha de " , get_parent_class($this) , "\n";
    }
}

class Filha2 extends Pai {
    function __construct()
    {
        echo "Eu também sou filha de " , get_parent_class('filha2') , "\n";
    }
}

$foo = new Filha();
$bar = new Filha2();

?>
```

O exemplo acima produzirá:

```
Eu sou filha de Pai
Eu também sou filha de Pai
```

Veja também

- get_class() - Retorna o nome da classe de um objeto
- is_subclass_of() - Verifica se o objeto tem esta classe como uma de suas classes pai
- class_parents() - Retorna as classes pai da classe fornecida

interface_exists

interface_exists

(PHP 5 >= 5.0.2, PHP 7, PHP 8)

interface_exists — Verifica se a interface foi definida

Descrição

function **interface_exists**(string `$interface`, bool `$autoload = true`): bool

Verifica se a interface foi definida.

Parâmetros

`interface`

O nome da interface.

`autoload`

Se deve-se carregar automaticamente a interface se já não tiver sido carregada.

Valor Retornado

Retorna `true` se a interface fornecida pelo parâmetro `interface` já foi definida, `false` caso contrário.

Examples

Exemplo #1 Exemplo de interface_exists()

```php
<?php
// Verifica se a interface existe antes de tentar usá-la
if (interface_exists('MyInterface')) {
    class MyClass implements MyInterface
    {
        // Métodos
    }
}

?>
```

Veja também

- get_declared_interfaces() - Retorna um array com todas as interfaces declaradas
- class_implements() - Retorna as interfaces que são implementadas pela classe ou interface fornecida
- class_exists() - Verifica se uma classe foi definida
- enum_exists() - Verifica se uma enumeração foi definida

is_a

is_a

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_a — Verifica se um objeto é ou herda de uma classe

Descrição

function **is_a**(mixed `$object_or_class`, string `$class`, bool `$allow_string = false`): bool

Determina se o `object_or_class` é do tipo `class`, ou se tem `class` como um dos tipos herdados.

Parâmetros

`object_or_class`

O nome da classe ou o objeto.

`class`

O nome da classe.

`allow_string`

Se esse parâmetro for configurado para `false`, um nome de classe em string em `object_or_class` não será permitido. Isso evita chamar o autoloader de uma classe inexistente.

Valor Retornado

Retorna `true` se o `object_or_class` for do tipo `class` ou se tiver `class` como um dos tipos herdados, `false` caso contrário.

Examples

Exemplo #1 Exemplo de is_a()

```php
<?php
// Define uma classe
class WidgetFactory
{
  var $oink = 'moo';
}

// Cria um objeto
$WF = new WidgetFactory();

if (is_a($WF, 'WidgetFactory')) {
  echo "\$WF implementa WidgetFactory\n";
}
?>
```

Exemplo #2 Utilizando o operador *instanceof*

```php
<?php
// define uma classe
class WidgetFactory
{
  var $oink = 'moo';
}

// cria um novo objeto
$WF = new WidgetFactory();

if ($WF instanceof WidgetFactory) {
    echo "\$WF implementa WidgetFactory\n";
}
?>
```

Veja também

- get_class() - Retorna o nome da classe de um objeto
- get_parent_class() - Obtém o nome da classe pai para o objeto ou classe
- is_subclass_of() - Verifica se o objeto tem esta classe como uma de suas classes pai

is_subclass_of

is_subclass_of

(PHP 4, PHP 5, PHP 7, PHP 8)

is_subclass_of — Verifica se o objeto tem esta classe como uma de suas classes pai

Descrição

function **is_subclass_of**(mixed `$object_or_class`, string `$class`, bool `$allow_string = true`): bool

Verifica se `object_or_class` informado tem a classe `class` como uma de suas classes pai ou o/a implementa.

Parâmetros

`object_or_class`

O nome da classe ou um instância do objeto. Nenhum erro é gerado se a classe não existir.

`class`

O nome da classe

`allow_string`

Se este parâmetro for definido para falso, nome de classe em string no parâmetro `object_or_class` não será permitido. Isto também evita chamada ao carregamento automático se a classe não existir.

Valor Retornado

Esta função retorna `true` se o objeto `object_or_class`, pertencer a uma classe que é uma subclasse de `class`, `false` caso contrário.

Examples

Exemplo #1 Exemplo de **is_subclass_of()**

```php
<?php
// define uma classe
class WidgetFactory
{
  var $oink = 'moo';
}

// define uma classe filha
class WidgetFactory_Child extends WidgetFactory
{
  var $oink = 'oink';
}

// cria um novo objeto
$WF = new WidgetFactory();
$WFC = new WidgetFactory_Child();

if (is_subclass_of($WFC, 'WidgetFactory')) {
  echo "sim, \$WFC é uma subclasse de WidgetFactory\n";
} else {
  echo "não, \$WFC não é uma subclasse de WidgetFactory\n";
}


if (is_subclass_of($WF, 'WidgetFactory')) {
  echo "sim, \$WF é uma subclasse de WidgetFactory\n";
} else {
  echo "não, \$WF não é uma subclasse de WidgetFactory\n";
}


if (is_subclass_of('WidgetFactory_Child', 'WidgetFactory')) {
  echo "sim, WidgetFactory_Child é uma subclasse de WidgetFactory\n";
} else {
  echo "não, WidgetFactory_Child não é uma subclasse de WidgetFactory\n";
}
?>
```

O exemplo acima produzirá:

```
sim, $WFC é uma subclasse de WidgetFactory
não, $WF não é uma subclasse de WidgetFactory
sim, WidgetFactory_Child é uma subclasse de WidgetFactory
```

Exemplo #2 Exemplo de **is_subclass_of()** usando interface

```php
<?php
// Define a Interface
interface MyInterface
{
  public function MyFunction();
}

// Define a implementação de classe da interface
class MyClass implements MyInterface
{
  public function MyFunction()
  {
    return "MyClass Implementa MyInterface!";
  }
}
```

```php
// Instancia o objeto object
$my_object = new MyClass;

// Funciona desde 5.3.7

// Teste usando a instância do objeto da classe
if (is_subclass_of($my_object, 'MyInterface')) {
  echo "Sim, \$my_object é uma subclasse de MyInterface\n";
} else {
  echo "No, \$my_object não é uma subclasse de MyInterface\n";
}

// Teste usando a string do nome da classe
if (is_subclass_of('MyClass', 'MyInterface')) {
  echo "Sim, MyClass é uma subclasse de MyInterface\n";
} else {
  echo "No, MyClass não é uma subclasse de MyInterface\n";
}
?>
```

O exemplo acima produzirá:

```
Sim, $my_object é uma subclasse de MyInterface
Sim, MyClass é uma subclasse de MyInterface
```

Notas

> **Nota** :
>
> Usar esta função fará com que qualquer autoloader registrado seja usado se a classe ainda não for conhecida.

Veja também

- get_class() - Retorna o nome da classe de um objeto
- get_parent_class() - Obtém o nome da classe pai para o objeto ou classe
- is_a() - Verifica se um objeto é ou herda de uma classe
- class_parents() - Retorna as classes pai da classe fornecida

method_exists

method_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

method_exists — Verifica se o método da classe existe

Descrição

function **method_exists**(object|string `$object_or_class`, string `$method`): bool

Verifica se o método da classe existe no `object_or_class` fornecido.

Parâmetros

`object_or_class`

Uma instância de objeto ou um nome de classe

`method`

O nome do método

Valor Retornado

Retorna `true` se método informado no parâmetro `method` foi definido para o `object_or_class` informado, `false` caso contrário.

Registro de Alterações

Versão	Descrição
7.4.0	As verificações de classe em relação aos métodos privados herdados agora retornam `false`.

Examples

Exemplo #1 Exemplo de method_exists()

```
<?php
$directory = new Directory('.');
var_dump(method_exists($directory,'read'));
?>
```

O exemplo acima produzirá:

```
bool(true)
```

Exemplo #2 Exemplo de method_exists() com método estático

```
<?php
var_dump(method_exists('Directory','read'));
?>
```

O exemplo acima produzirá:

```
bool(true)
```

Notas

> **Nota** :
>
> Usar esta função fará com que qualquer autoloader registrado seja usado se a classe ainda não for conhecida.

> **Nota** :
>
> A função **method_exists()** não pode detectar métodos que são magicamente acessíveis usando o método mágico `__call`.

Veja também

- function_exists() - Retorna true se a função informada estiver definida
- is_callable() - Verifica se um valor pode ser chamado como uma função a partir do escopo atual
- class_exists() - Verifica se uma classe foi definida

property_exists

property_exists

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

property_exists — Verifica se o objeto ou a classe tem uma propriedade

Descrição

function **property_exists**(object|string `$object_or_class`, string `$property`): bool

Essa função verifica se a propriedade informada em `property` existe na classe especificada.

> **Nota** :
>
> Ao contrário de isset(), **property_exists()** retorna `true` mesmo se a propriedade tiver o valor `null`.

Parâmetros

`object_or_class`

O nome da classe ou um objeto da classe a ser testada

`property`

O nome da propriedade

Valor Retornado

Retorna `true` se a propriedade existe, `false` se ela não existir.

Examples

Exemplo #1 Exemplo de property_exists()

```php
<?php

class myClass {
    public $mine;
    private $xpto;
    static protected $test;

    static function test() {
        var_dump(property_exists('myClass', 'xpto')); //true
    }
}

var_dump(property_exists('myClass', 'mine'));   //true
var_dump(property_exists(new myClass, 'mine')); //true
var_dump(property_exists('myClass', 'xpto'));   //true
var_dump(property_exists('myClass', 'bar'));    //false
var_dump(property_exists('myClass', 'test'));   //true
myClass::test();

?>
```

Notas

> **Nota** :
>
> Usar esta função fará com que qualquer autoloader registrado seja usado se a classe ainda não for conhecida.

> **Nota** :
>
> A função **property_exists()** não consegue detectar propriedades que são magicamente acessíveis usando o método mágico `__get`.

Veja também

- method_exists() - Verifica se o método da classe existe

trait_exists

trait_exists

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

trait_exists — Verifica se a trait existe

Descrição

function **trait_exists**(string `$trait`, bool `$autoload = true`): bool

Parâmetros

`trait`

Nome da trait para verificação

`autoload`

Se deve-se carregar automaticamente a trait caso não tenha sido caregada.

Valor Retornado

Retorna `true` se a Trait existir, `false` caso não exista.

Gerenciamento e Registro de Erros

Introdução

Estas são funções para lidar com gerenciamento e registro de erros. Elas permitem a definição de regras personalizadas de gerenciamento de erros e e de maneiras de se registrá-los. Isto possibilita que os relators de erros sejam modificados e melhorados para se adequar às necessidades do programador.

Com as funções de registro, pode-se enviar mensagens diretamente para outras máquinas, para um email (ou email para um portal de pagers), para os registros do sistema etc., de forma que seja possível registrar e monitorar as partes mais importantes das aplicações e sites da internet.

As funções relatórios de erro permitem customizar quais níveis e tipos de retorno de erro são fornecidos, desde notas simples até funções personalizadas retornadas durante os erros.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração de Erros e de Registros

Nome	Padrão	Modificável	Registro de Alterações
error_reporting	NULL	INI_ALL	
fatal_error_backtraces	"1"	INI_ALL	Disponível a partir do PHP 8.5.0.
display_errors	"1"	INI_ALL	
display_startup_errors	"1"	INI_ALL	Antes do PHP 8.0.0, o valor padrão era "0".
log_errors	"0"	INI_ALL	
log_errors_max_len	"1024"	INI_ALL	Sem efeito a partir do PHP 8.0.0, removido a partir do PHP 8.1.0.
ignore_repeated_errors	"0"	INI_ALL	
ignore_repeated_source	"0"	INI_ALL	
report_memleaks	"1"	INI_ALL	Descontinuada a partir do PHP 8.5.0
track_errors	"0"	INI_ALL	Descontinuado a partir do PHP 7.2.0, removido a partir do PHP 8.0.0.
html_errors	"1"	INI_ALL	
xmlrpc_errors	"0"	INI_SYSTEM	
xmlrpc_error_number	"0"	INI_ALL	
docref_root	""	INI_ALL	
docref_ext	""	INI_ALL	
error_prepend_string	NULL	INI_ALL	
error_append_string	NULL	INI_ALL	
error_log	NULL	INI_ALL	
error_log_mode	0o644	INI_ALL	Disponível a partir do PHP 8.2.0
syslog.facility	"LOG_USER"	INI_SYSTEM	Disponível a partir do PHP 7.3.0.
syslog.filter	"no-ctrl"	INI_ALL	Disponível a partir do PHP 7.3.0.
syslog.ident	"php"	INI_SYSTEM	Disponível a partir do PHP 7.3.0.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`error_reporting` int

Define o nível das reportagens de erros. O parâmetro é ou um inteiro representando um campo de bits ou constantes nomeadas. Os níveis e constantes de reportagens de erros estão descritos em Constantes Predefinidas e no arquivo *php.ini*. Para definir em tempo de execução, use a função error_reporting(). Consulte também a diretiva display_errors.

O padrão é `E_ALL`.

Antes do PHP 8.0.0, o valor padrão era: `E_ALL & ~E_NOTICE & ~E_STRICT & ~E_DEPRECATED`. Isto significa que os diagnósticos de nível `E_NOTICE`, `E_STRICT` e `E_DEPRECATED` não eram mostrados.

> **Nota** : **Constantes do PHP fora do PHP**
>
> Usar constantes do PHP fora do PHP, como no *httpd.conf*, não terão significado algum, portanto, nesses casos, os valores int são necessários. E, como níveis de erro serão adicionados ao longo do tempo, o valor máximo (para `E_ALL`) provavelmente será alterado. Portanto, no lugar de `E_ALL`, considere usar um valor maior para cobrir todos os campos de bits, tanto os de hoje como os do futuro, um valor numérico como `2147483647` (inclui todos os erros, não apenas `E_ALL`).

`fatal_error_backtraces` bool

Constrola se erros fatais devem incluir um rastreamento de pilha. Erros fatais são `E_ERROR`, `E_CORE_ERROR`, `E_COMPILE_ERROR`, `E_USER_ERROR`, `E_RECOVERABLE_ERROR` e `E_PARSE`.

`display_errors` string

Este parâmetro determina se os erros devem ser mostrados na tela como parte da saída ou se devem ser ocultados do usuário.

- `Off` - não mostra nenhum erro.
- `On` ou `stdout` - mostra erros em `stdout`. É o padrão.
- `stderr` - mostra erros em `stderr`. Efetivo somento nos SAPIs `CLI`, `phpdbg` e `CGI`.

O arquivo `php.ini-development` define para `On`; já o arquivo `php.ini-production` define para `Off`.

> **Nota** :
>
> Este é um recurso para dar suporte ao desenvolvimento e nunca deve ser usado em sistemas de produção (por exemplo, sistemas conectados à internet).

> **Nota** :
>
> Embora display_errors possa ser configurado em tempo de execução (com ini_set()), ele não terá nenhum efeito se o script tiver erros fatais. Isso ocorre porque a ação desejada em tempo de execução não é executada.

`display_startup_errors` bool

Mesmo quando display_errors está ativado, erros que ocorrem durante a sequência de inicialização do PHP não são exibidos. É altamente recomendável manter display_startup_errors desativado, exceto para depuração.

`log_errors` bool

Informa se as mensagens de erro de script devem ser gravadas no registro de erros do servidor ou no local indicado por error_log. Essa opção é, portanto, específica do servidor.

> **Nota** :
>
> É altamente aconselhável usar o registro de erros em vez de exibir erros em sites de produção.

`log_errors_max_len` int

Define o comprimento máximo de log_errors em bytes. Em error_log, informações sobre a fonte são adicionadas. O padrão é 1024, e 0 permite não aplicar limite de comprimento. Esse tamanho é aplicado a erros registrados, erros exibidos e também a *$php_errormsg*, mas não a funções chamadas explicitamente como error_log().

Quando um int é usado, o valor é medido em bytes. A notação abreviada, como descrita nesta FAQ, também pode ser usada.

`ignore_repeated_errors` bool

Não registra mensagens repetidas. Erros repetidos devem ocorrer no mesmo arquivo e na mesma linha, a menos que ignore_repeated_source seja definido como verdadeiro.

`ignore_repeated_source` bool

Ignora a fonte da mensagem quando estiver ignorando mensagens repetidas. Quando esta configuração estiver On, não serão registrados erros com mensagens repetidas de arquivos ou linhas diferentes.

`report_memleaks` bool

Se este parâmetro estiver definido como On (o padrão), este parâmetro mostrará um relatório de vazamentos de memória detectados pelo gerenciador de memória do Zend. Este relatório será enviado para stderr em plataformas Posix. No Windows, ele será enviado ao depurador usando o OutputDebugString() e poderá ser visualizado com ferramentas como » DbgView. Este parâmetro só tem efeito em uma compilação de depuração e se error_reporting incluir **`E_WARNING`** na lista de erros permitidos.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.5.0. O uso deste recurso é fortemente desencorajado.

`track_errors` bool

Se habilitado, a última mensagem de erro estará sempre presente na variável *$php_errormsg*.

`html_errors` bool

Se ativado, as mensagens de erro incluirão tags HTML. O formato para erros de HTML produz mensagens clicáveis que direcionam o usuário para uma página descrevendo o erro ou a função que está causando o erro. Essas referências são afetadas por docref_root e docref_ext.

Se desativado, a mensagem de erro será apenas texto simples.

`xmlrpc_errors` bool

Se ativado, desativa o relatório de erros normal e formata os erros como mensagem de erro XML-RPC.

`xmlrpc_error_number` int

Usado como o valor do elemento faultCode do XML-RPC.

`docref_root` string

O novo formato de erro contém uma referência a uma página que descreve o erro ou a função que está causando o erro. No caso de páginas de manual, pode-se baixar o manual no idioma próprio e definir esta diretiva ini para a URL da cópia local. Se a cópia local do manual puder ser alcançada por `"/manual/"` pode-se simplesmente usar **`docref_root=/manual/`**. Além disso, é necessário definir docref_ext para corresponder às extensões de arquivo da cópia local **`docref_ext=.html`**. É possível usar referências externas. Por exemplo, pode-se usar **`docref_root=http://manual/en/`** ou **`docref_root="http://landonize.it/?how=url&theme=classic&filter=Landon &url=http%3A%2F%2Fwww.php.net%2F"`**

Na maioria das vezes, é desejado que o valor docref_root termine com uma barra `"/"`. Mas veja o segundo exemplo acima, que não tem nem precisa dela.

> **Nota** :
>
> Este é um recurso para dar suporte ao desenvolvimento, pois facilita a pesquisa da descrição de uma função. No entanto, nunca deve ser usado em sistemas de produção (por exemplo, sistemas conectados à internet).

`docref_ext` string

Consulte docref_root.

> **Nota** :
>
> O valor de docref_ext deve começar com um ponto `"."`.

`error_prepend_string` string

String a ser mostrado antes de uma mensagem de erro. Usado apenas quando a mensagem de erro é mostrada na tela. O propósito principal é prover a capacidade de prefixar uma marcação HTML adicional à mensagem de erro.

`error_append_string` string

String para mostrar após uma mensagem de erro. Usado apenas quando a mensagem de erro é mostrada na tela. O propósito principal é prover a capacidade de posfixar uma marcação HTML adicional à mensagem de erro.

`error_log` string

Nome do arquivo onde os erros de script devem ser registrados. O arquivo deve ter permissão de escrita pelo usuário do servidor da web. Se o valor especial `syslog` for usado, os erros serão enviados ao registrador de erros do sistema. No Unix, isso significa syslog(3) e no Windows significa o Log de Eventos. Veja também: syslog(). Se esta diretiva não estiver definida, os erros serão enviados para o registrador de erros SAPI. Por exemplo, é um log de erros no Apache ou `stderr` no CLI. Consulte também error_log().

`error_log_mode` int

Modo de arquivo para o arquivo definido em error_log.

`syslog.facility` string

Especifica qual tipo de programa está registrando a mensagem. Apenas efetivo se error_log estiver definido como "syslog".

`syslog.filter` string

Especifica o tipo de filtro para filtrar as mensagens registradas. Caracteres permitidos são passados sem modificações; todos os outros são escritos em sua representação hexadecimal prefixados com `\x`.

- `all` - a string registrada será partida nos caracteres de novas linhas, e todos os caracteres serão passados inalterados
- `ascii` - a string registrada será partida nos caracteres de novas linhas, e qualquer caractere ASCII de 7 bits não imprimível será escapado
- `no-ctrl` - a string registrada será partida nos caracteres de novas linhas, e qualquer caractere não imprimível será escapado
- `raw` - todos os caracteres são passados ao registrador do sistema logger inalterados, sem separação novas linhas (idêntico ao PHP antes do 7.3)

Esta configuração irá afetar o registro através de error_log definido para "syslog" e chamadas a syslog().

> **Nota** :
>
> O tipo de filtro `raw` está disponível a partir do PHP 7.3.8 e do PHP 7.4.0.

Esta diretiva não é suportada no Windows.

`syslog.ident` string

Especifica a string de identificação que é anexada a todas as mensagens. Somente é efetivo se error_log estiver definido como "syslog".

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

As constantes a seguir (seus valores numéricos ou seus nomes simbólicos) são usadas como máscaras de bits para especificar que erros devem ser reportados. É possível usar operadores binários para combinar estes valores ou mascarar certos tipos de erro.

> **Dica**
>
> Os nomes destas constantes podem ser usados no *php.ini*, no lugar de valores numéricos puros aos quais elas correspondem. Entretanto, apenas os operadores `|`, `~`, `^`, `!` e `&` são compreendidos dentro do *php.ini*.

> **Aviso**

> Não é possível usar os nomes simbólicos fora do PHP. Por exemplo, no *httpd.conf*, deve ser usado o valor da máscara de bits computada.

E_ERROR (int)

Erros fatais em tempo de execução. Indicam erros a partir dos quais não é possível retomar a execução, como problema de alocação de memória por exemplo. A execução do script é terminada.

Valor da constante: 1

E_WARNING (int)

Alertas em tempo de executação (erros não fatais). A execução do script não é terminada.

Valor da constante: 2

E_PARSE (int)

Erro de análise em tempo de compilação. Erros de análise somente devem ser gerados pelo analisador.

Valor da constante: 4

E_NOTICE (int)

Avisos em tempo de execução. Indicam que o script encontrou algo que pode indicar um erro, mas também pode acontecer no curso normal de execução de um script.

Valor da constante: 8

E_CORE_ERROR (int)

Erros fatais que ocorrem durante a inicialização do PHP. É como um **E_ERROR**, exceto por ser gerado pelo núcleo do PHP.

Valor da constante: 16

E_CORE_WARNING (int)

Alertas (erros não fatais) que ocorrem na inicialização do PHP. É como um **E_WARNING**, exceto por ser gerado pelo núcleo do PHP.

Valor da constante: 32

E_COMPILE_ERROR (int)

Erros fatais em tempo de compilação. É como um **E_ERROR**, exceto por ser gerado pelo mecanismo de scripts Zend.

Valor da constante: 64

E_COMPILE_WARNING (int)

Alertas em tempo de compilação (erros não fatais). É como um **E_WARNING**, exceto por ser gerado pelo mecanismo de scripts Zend.

Valor da constante: 128

E_DEPRECATED (int)

Avisos de descontinuação em tempo de execução. Habilite para receber alertas sobre código que não funcionará em versões futuras.

Valor da constante: 8192

E_USER_ERROR (int)

Mensagem de erro gerada pelo usuário. É como um **E_ERROR**, exceto por ser gerada no código PHP através do uso da função trigger_error().

Valor da constante: 256

> **Aviso**

> O uso desta constante com trigger_error() foi descontinuado a partir do PHP 8.4.0. É recomendado, alternativamente, fazer uso de `throw` com uma Exception ou chamar exit().

E_USER_WARNING (int)

Mensagem de alerta gerada pelo usuário. É como um **E_WARNING**, exceto por ser gerada no código PHP através do uso da função trigger_error().

Valor da constante: `512`

E_USER_NOTICE (int)

Mensagem de aviso gerada pelo usuário. É como um **E_NOTICE**, exceto por ser gerada no código PHP através do uso da função trigger_error().

Valor da constante: `1024`

E_USER_DEPRECATED (int)

Mensagem de descontinuação gerada pelo usuário. É como **E_DEPRECATED**, exceto por ser gerada nop código PHP através do uso da função trigger_error().

Valor da constante: `16384`

E_STRICT (int)

Sugestões em tempo de execução emitidas pelo PHP sobre o código executado para assegurar compatibilidade com versões futuras.

Valor da constante: `2048`

> **Aviso**
>
> Este nível de erro não é utilizado e foi descontinuado a partir do PHP 8.4.0.

E_RECOVERABLE_ERROR (int)

"Exceções" legadas do mecanismo, que correspondem a um erro fatal capturável. Parecida com Error mas precisam ser capturadas através de um manipulador de erro definido pelo usuário (consulte set_error_handler()). Se não tratado, se comporta como **E_ERROR**.

Valor da constante: `4096`

> **Nota** :
>
> Este nível de erro está efetivamente sem uso, o único caso onde pode acontecer é na falha ao interpretar um object como um bool. Isto só pode acontecer para objetos internos.
>
> O exemplo mais comum, antes do PHP 8.4.0, é usar uma instância de GMP em uma condicional.

E_ALL (int)

Máscara de bits que contém todos os erros, alertas e avisos.

Valor da constante: `30719`

> **Aviso**
>
> Antes do PHP 8.4, o valor da constante era `32767`.

Examples

Pode-se ver abaixo um exemplo de uso das capacidades de gerenciamento de erros no PHP. Define-se uma função de gerenciamento de erros que registra as informações dentro de um arquivo (usando um formato XML), e envia um e-mail ao desenvolvedor se um erro crítico na lógica acontecer.

Exemplo #1 Usando gerenciamento de erro em um script

```php
<?php
// fazendo o próprio gerenciamento de erro
error_reporting(0);

// função de gerenciamento de erro definida pelo usuário
function userErrorHandler($errno, $errmsg, $filename, $linenum, $vars)
{
    // horário do erro
    $dt = date("Y-m-d H:i:s (T)");

    // Define um array associativo de strings de erro.
    // Na realidade, as únicas entradas que devem ser
    // consideradas são E_WARNING, E_NOTICE, E_USER_ERROR,
    // E_USER_WARNING e E_USER_NOTICE
    $errortype = array (
                E_ERROR              => 'Erro',
                E_WARNING            => 'Alerta',
                E_PARSE              => 'Erro de Interpretação',
                E_NOTICE             => 'Nota',
                E_CORE_ERROR         => 'Erro de Núcleo',
                E_CORE_WARNING       => 'Alerta de Núcleo',
                E_COMPILE_ERROR      => 'Erro de Compilação',
                E_COMPILE_WARNING    => 'Alerta de Compilação',
                E_USER_ERROR         => 'Erro de Usuário',
                E_USER_WARNING       => 'Alerta de Usuário',
                E_USER_NOTICE        => 'Nota de Usuário',
                E_STRICT             => 'Nota em Tempo de Execução',
                E_RECOVERABLE_ERROR  => 'Erro Fatal Recuperável'
                );
    // conjunto de erros para os quais um rastreamento de variáveis será gravado
    $user_errors = array(E_USER_ERROR, E_USER_WARNING, E_USER_NOTICE);

    $err = "<errorentry>\n";
    $err .= "\t<datetime>" . $dt . "</datetime>\n";
    $err .= "\t<errornum>" . $errno . "</errornum>\n";
    $err .= "\t<errortype>" . $errortype[$errno] . "</errortype>\n";
    $err .= "\t<errormsg>" . $errmsg . "</errormsg>\n";
    $err .= "\t<scriptname>" . $filename . "</scriptname>\n";
    $err .= "\t<scriptlinenum>" . $linenum . "</scriptlinenum>\n";

    if (in_array($errno, $user_errors)) {
        $err .= "\t<vartrace>" . wddx_serialize_value($vars, "Variáveis") . "</vartrace>\n";
    }
    $err .= "</errorentry>\n\n";

    // para testes:
    // echo $err;

    // grava no registro de erros, e envia e-mail se houver um erro crítico
    error_log($err, 3, "/usr/local/php/error.log");
    if ($errno == E_USER_ERROR) {
        mail("phpdev@example.com", "Critical User Error", $err);
    }
}


function distance($vect1, $vect2)
{
    if (!is_array($vect1) || !is_array($vect2)) {
        trigger_error("Parâmetros incorretos, arrays esperados", E_USER_ERROR);
        return NULL;
    }

    if (count($vect1) != count($vect2)) {
        trigger_error("Vetores devem ser do mesmo tamanho", E_USER_ERROR);
        return NULL;
    }

    for ($i=0; $i<count($vect1); $i++) {
        $c1 = $vect1[$i]; $c2 = $vect2[$i];
        $d = 0.0;
        if (!is_numeric($c1)) {
```

```
            trigger_error("Coordenada $i no vetor 1 não é um número, usando zero",
                          E_USER_WARNING);
            $c1 = 0.0;
        }
        if (!is_numeric($c2)) {
            trigger_error("Coordenada $i no vetor 2 não é um número, usando zero",
                          E_USER_WARNING);
            $c2 = 0.0;
        }
        $d += $c2*$c2 - $c1*$c1;
    }
    return sqrt($d);
}

$old_error_handler = set_error_handler("userErrorHandler");

// constante não definida, gera um alerta
$t = I_AM_NOT_DEFINED;

// define alguns "vetores"
$a = array(2, 3, "foo");
$b = array(5.5, 4.3, -1.6);
$c = array(1, -3);

// gera um erro de usuário
$t1 = distance($c, $b) . "\n";

// gera um outro erro de usuário
$t2 = distance($b, "eu não sou um array") . "\n";

// gera um alerta
$t3 = distance($a, $b) . "\n";

?>
```

Funções para Manuseamento de Erros

Veja também

Veja também syslog().

Índice

debug_backtrace

debug_backtrace

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

debug_backtrace — Gera uma rastreamento retroativo

Descrição

function **debug_backtrace**(int `$options` = `DEBUG_BACKTRACE_PROVIDE_OBJECT`, int `$limit` = 0): array

debug_backtrace() gera um rastreamento retroativo do PHP.

Parâmetros

`options`

Este parâmetro é uma máscara de bits para as seguintes opções:

Opções de debug_backtrace()

DEBUG_BACKTRACE_PROVIDE_OBJECT	Define se o índice "object" deve ou não ser preenchido.
DEBUG_BACKTRACE_IGNORE_ARGS	Define se o índice "args" deve ou não ser omitido, e consequentemente todos os argumentos de métodos e funções, para economizar memória.

> **Nota** :

Existem quatro combinações possíveis:

Opções de debug_backtrace()

`debug_backtrace()`	Preenche ambos os índices
`debug_backtrace(DEBUG_BACKTRACE_PROVIDE_OBJECT)`	
`debug_backtrace(~DEBUG_BACKTRACE_PROVIDE_OBJECT)`	Omite o índice `"object"` e preenche o índice `"args"`.
`debug_backtrace(0)`	
`debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)`	Omite *ambos* os índices `"object"` *e* `"args"`.
`debug_backtrace(~DEBUG_BACKTRACE_PROVIDE_OBJECT | DEBUG_BACKTRACE_IGNORE_ARGS)`	
`debug_backtrace(DEBUG_BACKTRACE_PROVIDE_OBJECT|DEBUG_BACKTRACE_IGNORE_ARGS)`	Preenche o índice `"object"` *e* omite o índice `"args"`.

`limit`

Este parâmetro pode ser usado para limitar o número de estruturas de pilha retornados. Por padrão (`limit=0`) todas as estruturas são retornadas.

Valor Retornado

Retorna um array de arrays associativos. Os elementos possíveis retornados são os seguintes:

Elementos possíveis retornados de debug_backtrace()

Nome	Tipo	Descrição
function	string	O nome da função atual. Consulte também __FUNCTION__.
line	int	O número da linha atual. Consulte também __LINE__.
file	string	O nome do arquivo atual. Consulte também __FILE__.
class	string	O nome da classe atual. Consulte também __CLASS__
object	object	O objeto atual de **DEBUG_BACKTRACE_PROVIDE_OBJECT** for informada.
type	string	O tipo de chamada atual. Se for chamada de método, "->" é retornado. Se for chamada de método estático, "::" é retornado. Se for chamada de função, nada é retornado.
args	array	Se for dentro de uma função, isto lista os arumentos da função. Se for dentro de um arquivo incluído, isto lista o nome do arquivo incluído. A menos que **DEBUG_BACKTRACE_IGNORE_ARGS** seja informada.

Examples

Exemplo #1 Exemplo de debug_backtrace()

```php
<?php
// filename: /tmp/a.php

function a_test($str)
{
    echo "\nHi: $str";
    var_dump(debug_backtrace());
}

a_test('friend');
?>

<?php
// filename: /tmp/b.php
include_once '/tmp/a.php';
?>
```

Resultados similares ao seguinte ao executar */tmp/b.php*:

```
Hi: friend
array(2) {
```

```
[0]=>
array(4) {
    ["file"] => string(10) "/tmp/a.php"
    ["line"] => int(10)
    ["function"] => string(6) "a_test"
    ["args"]=>
    array(1) {
      [0] => &string(6) "friend"
    }
}
[1]=>
array(4) {
    ["file"] => string(10) "/tmp/b.php"
    ["line"] => int(2)
    ["args"] =>
    array(1) {
      [0] => string(10) "/tmp/a.php"
    }
    ["function"] => string(12) "include_once"
  }
}
```

Veja também

- trigger_error() - Gera uma mensagem a nível de usuário de erro/alerta/nota
- debug_print_backtrace() - Mostra um rastreamento retroativo

debug_print_backtrace

debug_print_backtrace

(PHP 5, PHP 7, PHP 8)

debug_print_backtrace — Mostra um rastreamento retroativo

Descrição

function **debug_print_backtrace**(int `$options` = 0, int `$limit` = 0): void

A função **debug_print_backtrace()** mostra um rastreamento retroativo do PHP. Ela mostra as chamadas de função, arquivos incluídos por include ou requeridos por require, e resultados de avaliações de eval().

Parâmetros

`options`

Este parâmetro é uma máscara de bits para as opções a seguir:

Opções de debug_print_backtrace()

DEBUG_BACKTRACE_IGNORE_ARGS	Omitir os índices de "args", e portanto todos os argumentos de funções/métodos, para economizar memória.
DEBUG_BACKTRACE_PROVIDE_OBJECT	Aceito para compatibilidade com debug_backtrace(), mas não tem efeito: o rastreamento retroativo gerado é renderizado como uma string e a entrada "object" nunca é impressa.

`limit`

Este parâmetro pode ser usado para limitar o número de seções de pilha mostradas. Por padrão (`limit=0`) ela mostra todas as seções da pilha.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de debug_print_backtrace()

```php
<?php
// arquivo include.php

function a() {
    b();
}

function b() {
    c();
}

function c(){
    debug_print_backtrace();
}

a();

?>
```

```php
<?php
// arquivo test.php
// este é o arquivo que deve ser executado

include 'include.php';
?>
```

O exemplo acima produzirá algo semelhante a:

```
#0  c() called at [/tmp/include.php:10]
#1  b() called at [/tmp/include.php:6]
#2  a() called at [/tmp/include.php:17]
#3  include(/tmp/include.php) called at [/tmp/test.php:3]
```

Veja também

- debug_backtrace() - Gera uma rastreamento retroativo

error_clear_last

error_clear_last

(PHP 7, PHP 8)

error_clear_last — Limpa o erro mais recente

Descrição

function **error_clear_last**(): void

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Limpa o erro mais recente, fazendo com que não seja possível recuperá-los com error_get_last().

Examples

Exemplo #1 Um exemplo de error_clear_last()

```php
<?php
var_dump(error_get_last());
error_clear_last();
var_dump(error_get_last());
```

```php
@$a = $b;

var_dump(error_get_last());
error_clear_last();
var_dump(error_get_last());
?>
```

O exemplo acima produzirá algo semelhante a:

```
NULL
NULL
array(4) {
  ["type"]=>
  int(8)
  ["message"]=>
  string(21) "Undefined variable: b"
  ["file"]=>
  string(9) "%s"
  ["line"]=>
  int(6)
}
NULL
```

Veja também

- Constantes de erro

error_get_last

error_get_last

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

error_get_last — Obtém o último erro ocorrido

Descrição

function **error_get_last**(): ?array

Obtém informações sobre o último erro que ocorreu.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array associativo que descreve o último erro com as chaves "type", "message", "file" e "line". Se o erro foi causado por uma função interna do PHP, então a "message" começa com o nome da função. Retorna `null` se ainda não houve erro.

Examples

Exemplo #1 Um exemplo de error_get_last()

```php
<?php
echo $a;
print_r(error_get_last());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [type] => 8
    [message] => Undefined variable: a
    [file] => C:\WWW\index.php
    [line] => 2
)
```

Veja também

- Constantes de erro
- Variável *$php_errormsg*
- error_clear_last() - Limpa o erro mais recente
- Diretiva `display_errors`
- Diretiva `html_errors`
- Diretiva `xmlrpc_errors`

error_log

error_log

(PHP 4, PHP 5, PHP 7, PHP 8)

error_log — Envia uma mensagem de erro para as rotinas de tratamento de erros definidas

Descrição

```
function error_log(
    string $message,
    int $message_type = 0,
    ?string $destination = null,
    ?string $additional_headers = null
): bool
```

Envia uma mensagem de erro para o registro de erros do servidor web ou para um arquivo.

Parâmetros

`message`

A mensagem de erro que deve ser registrada.

`message_type`

Indica para onde o erro deve ser direcionado. Os possíveis tipos de mensagem são os seguintes:

Tipos de logs de error_log()

0	`message` é enviado para sistema de logs do PHP, usando o mecanismo de log do sistema operacional ou um arquivo, dependendo do que a diretiva de configuração error_log estiver definida. Esta é a opção padrão.
1	`message` é enviado por e-mail para o endereço no parâmetro `destination`. Este é o único tipo de mensagem em que o quarto parâmetro, `additional_headers`, é utilizado.
2	Não é mais uma opção.
3	`message` é adicionada ao arquivo `destination`. Uma nova linha não é automaticamente adicionada ao final da string `message`.
4	`message` é enviado diretamente para o manipulador de log do SAPI.

`destination`

O destino. Seu significado depende do parâmetro `message_type`, conforme descrito acima.

`additional_headers`

Os cabeçalhos adicionais. É usado quando o parâmetro `message_type` está definido como `1`. Este tipo de mensagem usa a mesma função interna que mail() utiliza.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha. Se `message_type` for zero, esta função sempre retorna `true`, independentemente de o erro poder ser registrado ou não.

Registro de Alterações

Versão	Descrição
8.0.0	`destination` e `additional_headers` agora permitem null.

Examples

Exemplo #1 Exemplos error_log()

```php
<?php
// Envia uma notificação através do registro do servidor se não conseguirmos
// conectar ao banco de dados.
if (!Ora_Logon($username, $password)) {
    error_log("Banco de dados Oracle não disponível!", 0);
}

// Notificar o administrador por e-mail se ficarmos sem FOO
if (!($foo = allocate_new_foo())) {
    error_log("Grande problema, estamos sem FOOs!", 1,
              "operator@example.com");
}

// Outra forma de chamar error_log():
error_log("Você cometeu um erro!", 3, "/var/tmp/my-errors.log");
?>
```

Notas

> **Aviso**
>
> **error_log()** não é seguro para binários. `message` será truncado pelo caractere nulo.

> **Dica**
>
> `message` não deve conter o caractere nulo. Observe que `message` pode ser enviado para arquivo, e-mail, syslog, etc. Use a função apropriada de conversão/escape, como base64_encode(), rawurlencode() ou addslashes() antes de chamar **error_log()**.

error_reporting

error_reporting

(PHP 4, PHP 5, PHP 7, PHP 8)

error_reporting — Define quais erros do PHP são relatados

Descrição

function **error_reporting**(?int `$error_level = null`): int

A função **error_reporting()** define a diretiva error_reporting em tempo de execução. O PHP possui muitos níveis de erros, e o uso desta função define esse nível para a duração (tempo de execução) do seu script. Se o parâmetro opcional `error_level` não for definido, a função **error_reporting()** retornará apenas o nível de relatório de erros atual.

Parâmetros

`error_level`

O novo nível de error_reporting. Pode ser uma máscara de bits ou constantes nomeadas. É altamente recomendado o uso de constantes nomeadas para garantir compatibilidade com futuras versões. Conforme são adicionados níveis de erro, o intervalo de inteiros aumenta, então os níveis de erro baseados em inteiros mais antigos nem sempre se comportarão como esperado.

As constantes de nível de erro disponíveis e os significados reais desses níveis de erro são descritos nas constantes predefinidas.

Valor Retornado

Retorna o nível de error_reporting *antes* que ele seja alterado para `error_level`.

> **Nota** :
>
> O operador de controle de erro `@` altera o `error_level` durante o tratamento de erros.

Registro de Alterações

Versão	Descrição
8.0.0	`error_level` agora permite null.

Examples

Exemplo #1 Exemplo error_reporting()

```php
<?php

// Desativar todos os relatórios de erros
error_reporting(0);

// Relatar erros simples durante a execução
error_reporting(E_ERROR | E_WARNING | E_PARSE);

// Relatar E_NOTICE também pode ser útil (para relatar variáveis não inicializadas
// ou capturar erros de digitação nos nomes das variáveis...)
error_reporting(E_ERROR | E_WARNING | E_PARSE | E_NOTICE);

// Relatar todos os erros, exceto E_NOTICE
error_reporting(E_ALL & ~E_NOTICE);

// Relatar todos os erros do PHP
error_reporting(E_ALL);

// Relatar todos os erros do PHP
error_reporting(-1);

// O mesmo que error_reporting(E_ALL);
ini_set('error_reporting', E_ALL);

?>
```

Notas

> **Dica**
>
> Passar o valor `-1` mostrará todos os erros possíveis, mesmo quando novos níveis e constantes forem adicionados em futuras versões do PHP. O comportamento é equivalente a passar a constante `E_ALL`.

Veja também

- A diretiva display_errors
- A diretiva html_errors
- A diretiva xmlrpc_errors
- O operador error control
- ini_set() - Define o valor de uma opção de configuração

get_error_handler

get_error_handler

(PHP 8 >= 8.5.0)

get_error_handler — Obtém a função de manipulação de erro definida pelo usuário

Descrição

function **get_error_handler**(): ?callable

Retorna a função de manipulação de erro atual, se houver.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a função de manipulação de erro atualmente definida (se houver). Se o manipulador interno estiver sendo usado, retorna `null`.

O manipulador retornado é o valor chamável exato que foi passado a set_error_handler() para defini-lo.

Examples

Exemplo #1 Exemplo de get_error_handler()

```php
<?php

$handler = function (int $errno, string $errstr, ?string $errfile, ?int $errline) {
     echo "Erro: " . $errstr . "\n";
};

var_dump(get_error_handler()); // NULL

set_error_handler($handler);

var_dump(get_error_handler() === $handler); // bool(true)

?>
```

Notas

> **Dica**
>
> Antes do PHP 8.5.0, esta funcionalidade poderia ser fornecida pelo código a seguir:
>
> ```php
> <?php
> if (!function_exists('get_error_handler')) {
>     function noop_error_handler() {
>     }
>     function get_error_handler(): ?callable {
>         $handler = set_error_handler('noop_error_handler');
>         restore_error_handler();
>         return $handler;
>     }
> }
> ?>
> ```

Veja também

- error_reporting() - Define quais erros do PHP são relatados
- set_error_handler() - Define uma função de usuário para manipulação de erro
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- trigger_error() - Gera uma mensagem a nível de usuário de erro/alerta/nota
- Constantes de nível de erro

get_exception_handler

get_exception_handler

(PHP 8 >= 8.5.0)

get_exception_handler — Obtém a função de manipulação de exceção definida pelo usuário

Descrição

function **get_exception_handler**(): ?callable

Retorna a função de manipulação de exceção atual, se houver.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a função de manipulação de exceção atualmente definida (se houver). Se nenhum manipulador estiver definido, retorna `null`.

O manipulador retornado é o valor chamável exato que foi passado a set_exception_handler() para defini-lo.

Examples

Exemplo #1 Exemplo de get_exception_handler()

```php
<?php

$handler = function (Throwable $ex) {
     echo "Exceção: " . $ex::class . ": " . $ex->getMessage() . "\n";
};

var_dump(get_exception_handler()); // NULL

set_exception_handler($handler);

var_dump(get_exception_handler() === $handler); // bool(true)

?>
```

Notas

> **Dica**
>
> Antes do PHP 8.5.0, esta funcionalidade poderia ser fornecida pelo código a seguir:
>
> ```php
> <?php
> if (!function_exists('get_exception_handler')) {
>     function noop_exception_handler() {
>     }
>     function get_exception_handler(): ?callable {
>         $handler = set_exception_handler('noop_exception_handler');
>         restore_exception_handler();
>         return $handler;
>     }
> }
> ?>
> ```

Veja também

- set_exception_handler() - Define uma função de manipulação de exceção definida pelo usuário
- restore_exception_handler() - Restaura a função de manipulação de exceção anteriormente definida
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- error_reporting() - Define quais erros do PHP são relatados
- Exceções

restore_error_handler

restore_error_handler

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

restore_error_handler — Restaura a função anterior de gerenciamento de erro

Descrição

function **restore_error_handler**(): true

Usada após mudar a função que gerencia os erros usando set_error_handler(), para reverter para a função anterior (que pode ser a função interna ou outra função definida pelo usuário).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Examples

Exemplo #1 Exemplo de **restore_error_handler()**

Decide se unserialize() causou um erro, então restaura o gerenciador de erro original.

```php
<?php
function unserialize_handler($errno, $errstr)
{
    echo "Valor serializado inválido.\n";
}

$serialized = 'foo';
set_error_handler('unserialize_handler');
$original = unserialize($serialized);
restore_error_handler();
?>
```

O exemplo acima produzirá:

```
Valor serializado inválido.
```

Veja também

- error_reporting() - Define quais erros do PHP são relatados
- set_error_handler() - Define uma função de usuário para manipulação de erro
- get_error_handler() - Obtém a função de manipulação de erro definida pelo usuário
- restore_exception_handler() - Restaura a função de manipulação de exceção anteriormente definida
- trigger_error() - Gera uma mensagem a nível de usuário de erro/alerta/nota

restore_exception_handler

restore_exception_handler

(PHP 5, PHP 7, PHP 8)

restore_exception_handler — Restaura a função de manipulação de exceção anteriormente definida

Descrição

function **restore_exception_handler**(): true

Usada após alteração da função de manipulação de exceção ao usar set_exception_handler(), para reverter ao manipulador de exceção anterior (que pode ser o integrado ou uma função definida pelo usuário).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Examples

Exemplo #1 Exemplo restore_exception_handler()

```php
<?php
    function manipulador_excecao_1(Exception $e)
    {
        echo '[' . __FUNCTION__ . '] ' . $e->getMessage();
    }

    function manipulador_excecao_2(Exception $e)
    {
        echo '[' . __FUNCTION__ . '] ' . $e->getMessage();
    }

    set_exception_handler('manipulador_excecao_1');
    set_exception_handler('manipulador_excecao_2');

    restore_exception_handler();

    throw new Exception('Isto dispara o primeiro manipulador de exceção...');
?>
```

O exemplo acima produzirá:

```
[manipulador_excecao_1] Isto dispara o primeiro manipulador de exceção...
```

Veja também

- set_exception_handler() - Define uma função de manipulação de exceção definida pelo usuário
- get_exception_handler() - Obtém a função de manipulação de exceção definida pelo usuário
- set_error_handler() - Define uma função de usuário para manipulação de erro
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- error_reporting() - Define quais erros do PHP são relatados

set_error_handler

set_error_handler

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

set_error_handler — Define uma função de usuário para manipulação de erro

Descrição

function **set_error_handler**(?callable `$callback`, int `$error_levels` = **`E_ALL`**): ?callable

Define uma função de usuário (`callback`) para lidar com erros em um script.

Esta função pode ser usada para definir manipuladores de erros customizados em tempo de execução, por exemplo em aplicações que precisem limpar arquivos/dados quando um erro crítico ocorrer, ou no disparo de um erro em resposta a certas condições (usando trigger_error()).

É importante ressaltar que o manipulador padrão de erros do PHP é completamente ignorado para os tipos de erro especificados por `error_levels` a menos que a função de retorno devolva **`false`**. As configurações de error_reporting() não terão efeito e o manipulador de erro será chamado de qualquer maneira - entretanto, ainda será possível ler o valor atual de error_reporting e atuar apropriadamente.

Note também que é responsabilidade do manipulador para a execução do script se for necessário, chamando-se exit(). Se a função de manipulação de erro simplesmente retornar, a execução do script continuará com a declaração seguinte à que causou o erro.

Os seguintes tipos de erro não podem ser manipulados com uma função definida pelo usuário: **`E_ERROR`**, **`E_PARSE`**, **`E_CORE_ERROR`**, **`E_CORE_WARNING`**, **`E_COMPILE_ERROR`**, **`E_COMPILE_WARNING`** independentemente de onde eles surgirem, e a maior parte dos erros **`E_STRICT`** originados no arquivo onde a função **set_error_handler()** for chamada.

Se erros ocorrerem antes que o script seja executado (ex.: em envios de arquivos), o manipulador customizado de erros não poderá ser chamado já que ainda não estará registrado nesse momento.

Parâmetros

`callback`

Se `null` for passado, o manipulador será redefinido a seu estado padrão. Caso contrário, o manipulador é uma função de retorno com a seguinte assinatura:

```
function handler(
    int $errno,
    string $errstr,
    string $errfile = ?,
    int $errline = ?,
    array $errcontext = ?
): bool
```

`errno`

O primeiro parâmetro, `errno`, que conterá o nível do erro, como um inteiro.

`errstr`

O segundo parâmetro, `errstr`, que conterá a mensagem de erro, como uma string.

`errfile`

Se a função de retorno aceitar um terceiro parâmetro, `errfile`, será passado o nome do arquivo no qual o erro se originou, como uma string.

`errline`

Se a função de retorno aceitar um quarto parâmetro, `errline`, será passado o número da linha onde o erro ocorreu, como um inteiro.

`errcontext`

Se a função de retorno aceitar um quinto parâmetro, `errcontext`, será passado um array que aponta para a tabela de símbolos ativa no ponto onde o erro ocorreu. Em outras palavras, `errcontext` conterá um array com cada variável existente no escopo em que o erro surgiu. Manipuladores de erro de usuário não devem modificar o contexto de erro.

> **Aviso**
>
> Este parâmetro tornou-se *DESCONTINUADO* a partir do PHP 7.2.0, e foi *REMOVIDO* a partir do PHP 8.0.0. Se a função definir este parâmetro sem um valor padrão, um erro do tipo "too few arguments" será emitido quando ela for chamada.

Se a função retornar `false`, o manipulador de erros normal continua.

`error_levels`

Pode ser usado para mascarar a chamada da função `callback` da mesma forma que a configuração ini error_reporting controla quais erros são mostrados. Sem a definição desta máscara, a função `callback` será chamada para todo erro, indenpendentemente da configuração de error_reporting.

Valor Retornado

Retorna o manipulador de erro anterior (se existente) como um callable. Se o manipulador interno for usado, `null` é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	`errcontext` foi removido e não mais será passado para chamadas de retorno de usuário.
7.2.0	`errcontext` foi descontinuado. O uso deste parâmetro agora emite uma nota `E_DEPRECATED`.

Examples

Exemplo #1 Manipulação de erro com set_error_handler() e trigger_error()

O exemplo abaixo mostra a manipulação de exceções internas através de disparo de erros e uso de função definida pelo usuário:

```php
<?php
// função de manipulação de erro
function myErrorHandler($errno, $errstr, $errfile, $errline)
{
    if (!(error_reporting() & $errno)) {
        // Este código de erro não está incluído em error_reporting, então será
        // passado ao manipulador padrão de erros do PHP
        return false;
    }

    // $errstr pode precisar de escape:
    $errstr = htmlspecialchars($errstr);

    switch ($errno) {
    case E_USER_ERROR:
        echo "<b>Meu ERRO</b> [$errno] $errstr<br />\n";
        echo "  Erro fatal na linha $errline no arquivo $errfile";
        echo ", PHP " . PHP_VERSION . " (" . PHP_OS . ")<br />\n";
        echo "Abortando...<br />\n";
        exit(1);

    case E_USER_WARNING:
        echo "<b>Meu ALERTA</b> [$errno] $errstr<br />\n";
        break;

    case E_USER_NOTICE:
        echo "<b>Minha NOTA</b> [$errno] $errstr<br />\n";
        break;

    default:
        echo "Tipo de erro desconhecido: [$errno] $errstr<br />\n";
        break;
    }

    /* Não executa o manipulador interno do PHP */
    return true;
}

// função para testar a manipulação de erro
function scale_by_log($vect, $scale)
{
    if (!is_numeric($scale) || $scale <= 0) {
        trigger_error("log(x) para x <= 0 é indefinido, foi usado: escala = $scale", E_USER_ERROR);
    }

    if (!is_array($vect)) {
        trigger_error("Vetor de entrada incorreto, esperado array de valores", E_USER_WARNING);
        return null;
    }

    $temp = array();
    foreach($vect as $pos => $value) {
        if (!is_numeric($value)) {
            trigger_error("Valor na posição $pos não é um número, usando 0 (zero)", E_USER_NOTICE);
            $value = 0;
        }
        $temp[$pos] = log($scale) * $value;
    }

    return $temp;
}

// define para o manipulador de erro de usuário
$old_error_handler = set_error_handler("myErrorHandler");

// dispara alguns erros, primeiro define um array misto com um item não numérico
echo "vetor a\n";
$a = array(2, 3, "foo", 5.5, 43.3, 21.11);
print_r($a);
```

```php
// agora gera um segundo array
echo "----\nvetor b - uma nota (b = log(PI) * a)\n";
/* Valor na posição $pos não é um número, usando 0 (zero) */
$b = scale_by_log($a, M_PI);
print_r($b);

// isto é um problema, passando uma string ao invés de um array
echo "----\nvetor c - um alerta\n";
/* Vetor de entrada incorreto, esperado array de valores */
$c = scale_by_log("not array", 2.3);
var_dump($c); // NULL

// aqui um erro crítico, log de zero ou número negativo não é definido
echo "----\nvetor d - erro fatal\n";
/* log(x) para x <= 0 é indefinido, foi usado: escala = $scale" */
$d = scale_by_log($a, -2.5);
var_dump($d); // Nunca alcançado
?>
```

O exemplo acima produzirá algo semelhante a:

```
vetor a
Array
(
    [0] => 2
    [1] => 3
    [2] => foo
    [3] => 5.5
    [4] => 43.3
    [5] => 21.11
)
----
vetor b - uma nota (b = log(PI) * a)
<b>Minha NOTA</b> [1024] Valor na posição 2 não é um número, usando 0 (zero)<br />
Array
(
    [0] => 2.2894597716988
    [1] => 3.4341896575482
    [2] => 0
    [3] => 6.2960143721717
    [4] => 49.566804057279
    [5] => 24.165247890281
)
----
vetor c - um alerta
<b>Meu ALERTA</b> [512] Vetor de entrada incorreto, esperado array de valores<br />
NULL
----
vetor d - erro fatal
<b>Meu ERRO</b> [256] log(x) para x <= 0 é indefinido, foi usado: escala = -2.5<br />
  Erro fatal na linha 35 no arquivo trigger_error.php, PHP 5.2.1 (FreeBSD)<br />
Abortando...<br />
```

Veja também

- ErrorException
- error_reporting() - Define quais erros do PHP são relatados
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- get_error_handler() - Obtém a função de manipulação de erro definida pelo usuário
- trigger_error() - Gera uma mensagem a nível de usuário de erro/alerta/nota
- Constantes de nível de erro

set_exception_handler

set_exception_handler

(PHP 5, PHP 7, PHP 8)

set_exception_handler — Define uma função de manipulação de exceção definida pelo usuário

Descrição

function **set_exception_handler**(?callable `$callback`): ?callable

Define o manipulador de exceção padrão se uma exceção não for capturada dentro de um bloco try/catch. A execução irá parar após a função definida no parâmetro `callback` for chamada.

Parâmetros

`callback`

> A função a ser chamada quando uma exceção não capturada ocorrer. Esta função de manipulação precisa aceitar um parâmetro, que será o objeto Throwable que foi lançado. Tanto Error quanto Exception implementam a interface Throwable. Esta é a assinatura do manipulador:
>
> function handler(Throwable `$ex`): void
>
> **`null`** pode ser passado no lugar, para redefinir este manipulador ao seu estado inicial.

Valor Retornado

Retorna o manipulador de exceção anteriormente definido, ou **`null`** em caso de erro. Se nenhum manipulador anterior estava definido, **`null`** também é retornado.

Examples

Exemplo #1 Exemplo de set_exception_handler()

```php
<?php
function exception_handler(Throwable $exception) {
  echo "Exceção não capturada: " , $exception->getMessage(), "\n";
}

set_exception_handler('exception_handler');

throw new Exception('Exceção Não Capturada');
echo "Não Executado\n";
?>
```

Veja também

- get_exception_handler() - Obtém a função de manipulação de exceção definida pelo usuário
- restore_exception_handler() - Restaura a função de manipulação de exceção anteriormente definida
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- error_reporting() - Define quais erros do PHP são relatados
- Exceções

trigger_error

trigger_error

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

trigger_error — Gera uma mensagem a nível de usuário de erro/alerta/nota

Descrição

function **trigger_error**(string `$message`, int `$error_level` = **`E_USER_NOTICE`**): true

Usado para disparar uma condição de erro, pode ser usado em conjunto com o gerenciador de erro interno, ou com uma função definida pelo usuário que tenha sido definida como novo gerenciador de erro (set_error_handler()).

Esta função é útil quando for necessário gerar uma resposta em particular a uma exceção em tempo de execução.

Parâmetros

`message`

A mensagem de erro designada para este erro. É limitada a 1024 bytes em tamanho. Quaisquer caracteres adicionais além de 1024 bytes serão truncados.

`error_level`

O tipo de erro designado para este erro. Funciona apenas com a família de constantes **`E_USER_*`**, e o padrão é **`E_USER_NOTICE`**.

> **Aviso**
>
> Passar **`E_USER_ERROR`** como o `error_level` agora foi descontinuado. Alternativamente, uma Exception pode ser lançada ou exit() pode ser chamada.

Valor Retornado

Sempre retorna **`true`**.

Erros/Exceções

Esta função lança uma exceção ValueError se `error_level` não for uma das constantes **`E_USER_ERROR`**, **`E_USER_WARNING`**, **`E_USER_NOTICE`** ou **`E_USER_DEPRECATED`**.

Registro de Alterações

Versão	Descrição
8.4.0	Passar **`E_USER_ERROR`** como `error_level` agora foi descontinuado. Alternativamente, uma Exception pode ser lançada ou exit() pode ser chamada.
8.4.0	A função agora tem um tipo de retorno true ao invés de bool.
8.0.0	A função agora lança uma exceção ValueError se um `error_level` inválido for especificado. Anteriormente, retornava **`false`**.

Examples

Exemplo #1 Exemplo de trigger_error()

Consulte set_error_handler() para um exemplo mais extensivo.

```php
<?php
$password = $_POST['password'] ?? '';
if ($password === '') {
    trigger_error("Usar uma senha vazia não é seguro", E_USER_WARNING);
}
$hash = password_hash($password, PASSWORD_DEFAULT);
?>
```

Notas

> **Aviso**
>
> As entidades HTML contidas no parâmetro `message` não são escapadas. Utilize htmlentities() na mensagem se for necessário exibir o erro em um navegador.

Veja também

- error_reporting() - Define quais erros do PHP são relatados
- set_error_handler() - Define uma função de usuário para manipulação de erro
- restore_error_handler() - Restaura a função anterior de gerenciamento de erro
- As constantes de nível de erro
- O atributo Deprecated

user_error

user_error

(PHP 4, PHP 5, PHP 7, PHP 8)

user_error — Sinônimo de trigger_error()

Descrição

Esta função é um apelido para: trigger_error().

Controle de Buffer de Saída

Introdução

As funções de Controle de Saída permitem controlar quando a saída é enviada do script. Isto pode ser útil em situações diversas, especialmente se for necessário enviar cabeçalhos para o navegador depois que o script começou a enviar dados. As funções de Controle de Saída não afetam os cabeçalhos enviados por funções como header() ou setcookie(), apenas funções como echo e dados entre blocos de código PHP.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração de Controle de Saída

Nome	Padrão	Modificável	Registro de Alterações
output_buffering	`"0"`	`INI_PERDIR`	
output_handler	`null`	`INI_PERDIR`	
implicit_flush	`"0"`	`INI_ALL`	
url_rewriter.tags	`"form="`	`INI_ALL`	A partir do PHP 7.1.0, esta configuração INI afeta somente output_add_rewrite_var(). Antes do PHP 7.1.0, esta configuração INI habilitava duporte a ID de sessão transparente (veja session.trans_sid_tags).
url_rewriter.hosts	`$_SERVER['HTTP_HOST']` é usado como padrão.	`INI_ALL`	Disponível a partir do PHP 7.1.0

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`output_buffering` bool/int

Buffer de saída para todos os arquivos pode ser habilitado configurando-se esta diretiva para `"On"`. Para limitar o tamanho do buffer, um número/quantidade correspondendo ao número máximo de bytes permitidos pode ser usado no lugar de `"On"` para o valor desta diretiva. Por exemplo, `output_buffering=4096`. Esta diretiva é sempre "Off" no PHP-CLI.

`output_handler` string

A saída de scripts pode ser redirecionada para uma função. Por exemplo, ao configurar `output_handler` para mb_output_handler(), a codificação de caracteres será convertida de forma trasnparente para a codificação especificada. Configurar qualquer manipulador de saída automaticamente habilita o buffer de saída.

> **Nota** :
>
> mb_output_handler() e ob_iconv_handler() não podem ser usadas juntas; e ob_gzhandler() e zlib.output_compression não podem ser usadas com nenhuma das funções a seguir: mb_output_handler(), ob_gzhandler(), zlib.output_compression, o manipulador 'URL-Rewriter' (veja session.use_trans_sid e output_add_rewrite_var()).

> **Nota** :
>
> Apenas funções internas do PHP podem ser usadas com esta diretiva. Para funções definidas pelo usuário, use ob_start().

`implicit_flush` bool

`false` por padrão. Alterar para `true` indica ao PHP que a camada de saída deve se descarregar automaticamente após cada bloco de saída. Isto é equivalente a chamar a função do PHP flush() após cada chamada a qualquer função que produza saída (tal como print

ou echo) e cada bloco HTML.

Ao usar o PHP em um ambiente web, habilitar esta opção tem implicações sérias em desempenho e é geralmente recomendada apenas para depuração. O valor deste parâmetro é **`true`** ao operar sob o `CLI SAPI`.

Consulte também ob_implicit_flush().

`url_rewriter.tags` string

`url_rewriter.tags` especifica etiquetas e atributos HTML cujas URLs são re-escritas pelos valores de output_add_rewrite_var(). O padrão é `"form="`.

Adicionar `"form="` ou qualquer atributo `form` irá adicionar um elemento `input` oculto ao `form` com um atributo de nome e valor para cada par de nome-valor passado para output_add_rewrite_var().

> **Cuidado**
>
> Caso seja adicionada a mesma etiqueta mais de uma vez a `url_rewriter.tags`, apenas a primeira será usada durante o processo de re-escrita da URL.

> **Nota** :
>
> Antes do PHP 7.1.0, url_rewriter.tags era usado para especificar session.trans_sid_tags.

`url_rewriter.hosts` string

`url_rewriter.hosts` especifica quais hosts são re-escritos para incluir os valores output_add_rewrite_var(). O padrão é `$_SERVER['HTTP_HOST']`. Múltiplos hosts podem ser especificados por uma lista separada por vírgulas que não contenha espaços. Por exemplo, `"php.net,wiki.php.net,bugs.php.net"`.

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

Opções de status passadas ao manipulador de saída

As opções a seguir são passadas no segundo parâmetro (`phase`) do manipulador de saída definido por ob_start() como parte de uma máscara de bits:

`PHP_OUTPUT_HANDLER_START` (int)

Indica que o buffer de saída iniciou.

`PHP_OUTPUT_HANDLER_WRITE` (int)

Indica que o buffer de saída está sendo despejado, e tinha dados para enviar.

`PHP_OUTPUT_HANDLER_FLUSH` (int)

Indica que o buffer de saída foi despejado.

`PHP_OUTPUT_HANDLER_CLEAN` (int)

Indica que o buffer de saída foi limpo.

`PHP_OUTPUT_HANDLER_FINAL` (int)

Indica que esta é a operação de buffer de saída final.

`PHP_OUTPUT_HANDLER_CONT` (int)

Indica que o buffer de saída foi despejado, mas que operação de buffer de saída irá continuar.

Este é um apelido para **`PHP_OUTPUT_HANDLER_WRITE`**.

`PHP_OUTPUT_HANDLER_END` (int)

Indica que o buffer de saída terminou.

Este é um apelido para `PHP_OUTPUT_HANDLER_FINAL`.

Opções de controle de buffer de saída

As opções a seguir podem ser passadas no terceiro parâmetro (`flags`) do manipulador de saída definido por ob_start() como uma máscara de bits.:

`PHP_OUTPUT_HANDLER_CLEANABLE` (int)

Controla se um buffer de saída criado por ob_start() pode ser limpo por ob_clean(). Esta opção não controla o comportamento de ob_end_clean() ou ob_get_clean().

`PHP_OUTPUT_HANDLER_FLUSHABLE` (int)

Controla se um buffer de saída criado por ob_start() pode ser despejado por ob_flush(). Esta opção não controla o comportamento de ob_end_flush() ou ob_get_flush().

`PHP_OUTPUT_HANDLER_REMOVABLE` (int)

Controla se um buffer de saída criado por ob_start() pode ser removido antes do final do script ou ao chamar ob_end_clean(), ob_end_flush(), ob_get_clean() ou ob_get_flush().

`PHP_OUTPUT_HANDLER_STDFLAGS` (int)

O conjunto padrão de opções de buffer de saída; atualmente equivalente a `PHP_OUTPUT_HANDLER_CLEANABLE` | `PHP_OUTPUT_HANDLER_FLUSHABLE` | `PHP_OUTPUT_HANDLER_REMOVABLE`.

Opções de status do manipulador de saída

As opções a seguir são parte da máscara de bits `flags` retornada por ob_get_status():

`PHP_OUTPUT_HANDLER_STARTED` (int)

Indica que um manipulador de saída foi chamado.

`PHP_OUTPUT_HANDLER_DISABLED` (int)

Indica que o manipulador de saída está desabilitado. Esta opção é definida quando o manipulador de saída retorna `false` ou falha durante o processamento do buffer. Antes do PHP 8.4.0, esta opção poderia ser definida ao iniciar um buffer de saída.

`PHP_OUTPUT_HANDLER_PROCESSED` (int)

Indica que o manipulador de saída processou o buffer com sucesso. Disponível a partir do PHP 8.4.0.

Buffer de Saída

O buffer de saída é o buffer (armazenamento temporário) da saída antes de ela ser descarregada (enviada e descartada) para o navegador (em um contexto da web) ou para o shell (na linha de comando). Enquanto o buffer de saída estiver ativo, nenhuma saída será enviada do script; em vez disso, a saída será armazenada em um buffer interno.

Buffer Afetando o PHP

O PHP depende da infraestrutura de software/hardware subjacente ao descarregar a saída. O buffer implementado por consoles na linha de comando (por exemplo, buffer de linha) ou servidores web e navegador em um contexto web (por exemplo, totalmente bufferizado) afeta quando a saída é exibida para o usuário final. Alguns desses efeitos podem ser eliminados ajustando as configurações do servidor e/ou alinhando os tamanhos dos buffers das diversas camadas.

Controle de Buffer de Saída no PHP

O PHP fornece um buffer de saída em nível de usuário totalmente armazenado em buffer com funções para iniciar, manipular e desligar o buffer (a maioria das funções ob_*) e duas funções para descarregar os buffers subjacentes do sistema (flush() e ob_implicit_flush()). Algumas dessas funcionalidades podem ser definidas e/ou configuradas usando também as configurações apropriadas do *php.ini*.

Casos de Uso

O buffer de saída geralmente é útil em situações em que a saída armazenada em buffer é modificada ou inspecionada ou é usada mais de uma vez em uma solicitação; ou quando a descarga controlada da saída é desejada. Os casos de uso específicos incluem:

- Armazenar em cache o resultado de scripts com uso intensivo de tempo/computação, por exemplo, gerando páginas HTML estáticas.

- Reutilizar a saída gerada exibindo-a, salvando-a em um arquivo e/ou enviando-a por e-mail.
- Enviar a tag `head` de uma página `HTML`, antes da tag `body` permite que os navegadores carreguem recursos externos enquanto o script executa processos potencialmente mais demorados (por exemplo, acesso a banco de dados e arquivos, conexão de rede externa). Isto só é útil se o código de status `HTTP` não puder ser alterado após o envio dos cabeçalhos.
- Extrair informações de funções que de outra forma produziriam saída (por exemplo, phpinfo()).
- Controlar a saída de código de terceiros modificando/usando partes (por exemplo, extraindo dados, substituindo palavras/frases, adicionando tags `HTML` ausentes) ou descartando-o inteiramente sob certas condições (por exemplo, erros).
- Preenchendo certas funcionalidades indisponíveis do servidor web (por exemplo, compactação ou codificação de saída)

Descarregando Buffers de Sistema

O PHP fornece duas maneiras relacionadas para descarregar os buffers de sistema (enviar e descartar o conteúdo): chamando-se a função flush() e habilitando o descarregamento implicitamente com a função ob_implicit_flush() ou com a configuração implicit_flush do *php.ini*.

Comportamento do Descarregameto da Saída

Com o descarregamento implícito desabilitado, o PHP irá descarregar a saída somente quendo flush() for chamada ou quando o script terminar.

Com o descarregamento implícito habilitado, o PHP tentará descarregar após cada bloco de código que resulte em uma saída. A saída neste contexto é um dado de comprimento maior que zero que:

- esteja fora das tags `<?php ?>`
- seja exibido por construções de linguagem e funções cujo propósito explícito é mostrar variáveis fornecidas pelo usuário ou strings tais como echo, print, printf(), var_dump(), var_export() e vprintf()
- seja exibido por funções cujo propósito é coletar e mostrar dados/informações no script em execução ou no PHP tais como debug_print_backtrace(), phpcredits() e phpinfo(), ReflectionExtension::info()
- seja mostrado pelo PHP em uma exceção não capturada ou erro não gerenciado (sujeitos às configurações de display_errors e error_reporting)
- seja qualquer coisa escrita em `php://output`

> **Nota** :
>
> Exibir strings vazias ou enviar cabeçalhos não é considerado saída e não resultará em operação de descarregamento.

> **Aviso**
>
> Se o descarregamento implícito estiver habilitado, caracteres de controle (ex.: `"\n"`, `"\r"`, `"\0"`) também irão ativar o descarregamento.

Limitações

Esta funcionalidade não pode descarregar buffers de saída no nível do usuário. Para usá-los juntos, buffers de saída no nível do usuário devem ser descarregados antes do descarregamento ds buffers de sistema para que o PHP produza qualquer saída.

> **Aviso**
>
> Chamar flush() ou habilitar o descarregamento implícito pode interferir com os manipuladores de saída de buffers no nível do usuário que definem e enviam cabeçalhos em um contexto web (ex.: ob_gzhandler()) enviando cabeçalhos antes que estes manipuladores o façam.

Buffer implementado pelo software/hardware subjacente não podem ser substituídos pelo PHP e devem ser levados em consideração ao se trabalhar com as funções de controle de buffer do PHP. Verificar as configurações de buffer dos servidores web, navegadores e consoles e trabalhar com elas pode aliviar possíveis problemas. Trabalhar em um contexto web, tanto as configurações de buffer do servidor quanto de buffer do script podem ser ajustadas para operar sequencialmente enquanto que contornar as estratégias de buffer de vários navegadores pode ser conseguido ajustando o buffer no script PHP. Em consoles que implementam buffer de linha, caracteres de nova linha podem ser inseridos nos locais apropriados antes de liberar a saída.

Diferenças SAPI No Descarregamento

Embora o descarregamento seja implementado por cada SAPI de uma maneira ligeiramente diferente, essas implementações se enquadram em uma de duas categorias:

- SAPIs usadas em um contexto web irão descarregar cabeçalhos primeiro, e depois a saída. `Apache2Handler`, `CGI`, `FastCGI` e `FPM` são tais SAPIs.
- Outras SAPIs tais como `CLI` e `embed` irão descarregar somente a saída.

Buffers de Saída em Nível de Usuário

Índice

Os buffers de saída no nível do usuário podem ser iniciados, manipulados e finalizados a partir do código PHP. Cada um desses buffers inclui um buffer de saída e uma função de manipulador de saída associada.

Ativando o buffer de saída

O buffer de saída pode ser ativado usando a função ob_start() ou definindo as configurações output_buffering e output_handler do *php.ini*. Embora ambas possam criar buffers de saída, ob_start() é mais flexível, pois aceita funções definidas pelo usuário como manipuladores de saída e as operações permitidas no buffer (liberar, limpar, remover) também podem ser definidas. Buffers iniciados com ob_start() estarão ativos a partir da linha em que a função foi chamada, enquanto aqueles iniciados com output_buffering irão armazenar a saída em buffer a partir da primeira linha do script.

O PHP também é fornecido com um manipulador de saída `"URL-Rewriter"` integrado que inicia seu próprio buffer de saída e permite até no máximo duas instâncias dele em execução a qualquer momento (uma para reescrita de URL no nível de usuário e uma para suporte transparente ao ID de sessão). Esses buffers podem ser iniciados chamando a função output_add_rewrite_var() e/ou ativando a configuração session.use_trans_sid do *php.ini*.

A extensão `zlib` incluída tem seu próprio buffer de saída que pode ser ativado usando a configuração zlib.output_compression do *php.ini*.

> **Nota** :
>
> Embora `"URL-Rewriter"` seja especial porque permite apenas até duas instâncias dele em execução ao mesmo tempo, todos os buffers de saída em nível de usuário usam os mesmos buffers subjacentes usados por ob_start() com sua funcionalidade implementada por uma função de manipulador de saída personalizada. Como tal, todas as suas funcionalidades podem ser emuladas pelo código do usuário.

Descarregando, Acessando e Limpando o Conteúdo do Buffer

A descarga envia e descarta o conteúdo do buffer ativo. Os buffers de saída são descarregados quando o tamanho da saída excede o tamanho do buffer; quando o script termina; ou quando ob_flush(), ob_end_flush() ou ob_get_flush() são chamados.

> **Cuidado**
>
> Chamar ob_end_flush() ou ob_get_flush() irá desligar o buffer ativo.

> **Cuidado**
>
> A descarga de buffers liberará o valor de retorno do manipulador de saída, que pode ser diferente do conteúdo do buffer. Por exemplo, usar ob_gzhandler() compactará a saída e liberará a saída compactada.

O conteúdo do buffer ativo pode ser recuperado chamando ob_get_contents(), ob_get_clean() ou ob_get_flush().

Se apenas o comprimento do conteúdo do buffer for necessário, ob_get_length() ou ob_get_status() retornação o comprimento do conteúdo em bytes.

> **Cuidado**
>
> Chamar ob_get_clean() ou ob_get_flush() irá desligar o buffer ativo depois de retornar seu conteúdo.

O conteúdo do buffer ativo pode ser limpo chamando ob_clean(), ob_end_clean() ou ob_get_clean().

> **Cuidado**
>
> Chamar ob_end_clean() ou ob_get_clean() desligará o buffer ativo.

Desligando Buffers

Buffers de saída podem ser desligados chamando ob_end_clean(), ob_end_flush(), ob_get_flush() ou ob_get_clean().

> **Aviso**
>
> Buffers de saída iniciados sem a opção `PHP_OUTPUT_HANDLER_REMOVABLE` não podem ser desligados e podem gerar um `E_NOTICE`.

Todo buffer de saída que não tiver sido fechado até o final do script, ou quando a função exit() for chamada, será descarregado e desligado pelo processo de desligamento do PHP. Os buffers serão descarregados e desligados na ordem inversa de sua inicialização. O último buffer iniciado será o primeiro, o primeiro buffer iniciado será o último a ser descarregado e desligado.

> **Cuidado**
>
> Se a descarga do conteúdo do buffer não for desejada, um manipulador de saída personalizado deverá ser usado para evitar a descarga durante o desligamento.

Exceções Lançadas em Manipuladores de Saída

Se uma exceção não capturada é lançada em um manipulador de saída, o programa termina e o manipulador é invocado pelo processo de desligamento depois do qual a mensagem de erro `"Uncaught Exception"` (exceção não capturada) é descarregada.

Se a exceção não capturada é lançada em um manipulador invocado por ob_flush(), ob_end_flush() ou ob_get_flush(), o conteúdo do buffer é descarregado antes da mensagem de erro.

Se uma exceção não capturada é lançada em um manipulador de saída durante o desligamento, o manipulador é terminado e nem o conteúdo do buffer nem a mensagem de erro são descarregados.

> **Nota** :
>
> Se um manipulador lança uma exceção, sua constante de estado `PHP_OUTPUT_HANDLER_DISABLED` é definida.

Erros Emitidos em Manipuladores de Saída

Se um erro não fatal for gerado em um manipulador de saída, o programa continuará a execução.

Se o erro não fatal for gerado em um manipulador invocado por ob_flush(), ob_end_flush() ou ob_get_flush(), o buffer descarrega determinados dados dependendo do valor de retorno do manipulador. Se o manipulador retornar `false`, o buffer e a mensagem de erro são descarregados. Se retornar qualquer outra coisa, o valor de retorno do manipulador será descarregado, mas a mensagem de erro não.

> **Nota** :
>
> Se um manipulador retornar `false`, sua constante de estado `PHP_OUTPUT_HANDLER_DISABLED` será definida.

Se um erro fatal for gerado em um manipulador de saída, o programa será encerrado e o manipulador será invocado pelo processo de desligamento, após o qual a mensagem de erro será descarregada.

Se o erro fatal for gerado em um manipulador invocado por ob_flush(), ob_end_flush() ou ob_get_flush(), o conteúdo dos buffers será descarregado antes da mensagem de erro.

Se um erro fatal for gerado em um manipulador de saída durante o desligamento, o programa será encerrado sem descarregar o buffer ou a mensagem de erro.

Saída nos Manipuladores de Saída

Em circunstâncias específicas, a saída produzida no manipulador é descarregada junto com o conteúdo do buffer. Esta saída não é anexada ao buffer e não faz parte da string retornada por ob_get_flush().

Durante operações de descarga (ao chamar ob_flush(), ob_end_flush(), ob_get_flush() e durante o desligamento), se o valor de retorno de um manipulador for `false`, o conteúdo do buffer é descarregado seguido pela saída. Se o manipulador não for invocado durante o desligamento, o fato do manipulador lançar uma exceção ou chamar a função exit() resultará no mesmo comportamento.

> **Nota** :
>
> Se um manipulador retornar `false`, sua constante de estado `PHP_OUTPUT_HANDLER_DISABLED` é definida.

Constantes de Estado do Manipulador de Saída

As constantes de estado de manipulador da máscara de bits `flags` do buffer são definidas toda vez que o manipulador de saída é invocado e são parte das `flags` retornadas por ob_get_status(). Se o manipulador executa com sucesso e não retorna `false`, `PHP_OUTPUT_HANDLER_STARTED` e `PHP_OUTPUT_HANDLER_PROCESSED` são definidas. Se o manipulador retonar `false` ou lança uma exceção durante a execução, `PHP_OUTPUT_HANDLER_STARTED` e `PHP_OUTPUT_HANDLER_DISABLED` são definidas.

> **Nota** :
>
> Se a constante `PHP_OUTPUT_HANDLER_DISABLED` de um manipulador for definida, o manipulador não será invocado chamando-se ob_end_clean(), ob_end_flush(), ob_get_clean(), ob_get_flush(), ob_clean(), ob_flush() ou durante o processo de desligamento do PHP. Antes do PHP 8.4.0, esta constante não tinha efeito ao chamar ob_clean() ou ob_flush().

Que Saída é Armazenada em Buffer?

Os buffers de saída em nível de usuário do PHP armazenam toda a saída após serem iniciados, até que sejam desligados ou o script termine. A saída no contexto do buffer de saída no nível do usuário do PHP é tudo o que o PHP exibiria ou enviaria de volta ao navegador. Em termos práticos, a saída são dados de comprimento diferente de zero que são:

- esteja fora das tags `<?php ?>`
- seja exibido por construções de linguagem e funções cujo propósito explícito é mostrar variáveis fornecidas pelo usuário ou strings tais como echo, print, printf(), var_dump(), var_export() e vprintf()
- seja exibido por funções cujo propósito é coletar e mostrar dados/informações no script em execução ou no PHP tais como debug_print_backtrace(), phpcredits() e phpinfo(), ReflectionExtension::info()
- seja mostrado pelo PHP em uma exceção não capturada ou erro não gerenciado (sujeitos às configurações de display_errors e error_reporting)
- seja qualquer coisa escrita em `php://output`

> **Nota** :
>
> Os dados gravados diretamente em `stdout` ou passados para uma função SAPI com funcionalidade semelhante não serão capturados pelos buffers de saída no nível do usuário. Isso inclui escrever dados em `stdout` com fwrite() ou enviar cabeçalhos usando header() ou setcookie().

Aninhando Buffers de Saída

Se houver um buffer de saída ativo quando um novo buffer for iniciado, o novo buffer será aninhado dentro do buffer ativo anteriormente. O buffer interno se comportará da mesma maneira, independentemente de estar aninhado, mas a saída armazenada em buffer por ele não será armazenada em buffer pelo buffer externo. Somente a saída liberada pelo buffer interno será armazenada no buffer externo.

A maioria das funções `ob_*` só funcionam com o buffer de saída ativo (o último iniciado), portanto, apenas o buffer ativo pode ser descarregado, limpo e desligado. As funções que funcionam com outros buffers são ob_list_handlers() que retorna a lista de todos os manipuladores de saída em uso e ob_get_status() que pode retornar informações apenas sobre o buffer ativo ou sobre todos os buffers em uso .

Chamar ob_get_level() ou ob_get_status() retornará o nível de aninhamento do buffer de saída ativo.

> **Cuidado**

> O valor para níveis idênticos entre ob_get_level() e ob_get_status() está desfasado em um. Para ob_get_level() o primeiro nível é `1`, enquanto para ob_get_status() o primeiro nível é `0`.

Tamanho do Buffer

Os tamanhos dos buffers são expressos por números inteiros e representam o número de bytes que o buffer pode armazenar sem descarregar. Quando o tamanho da saída no buffer excede o tamanho do buffer, o conteúdo do buffer é enviado para o manipulador de saída, seu valor de retorno é descarregado e o buffer é limpo.

Com exceção de `"URL-Rewriter"`, o tamanho dos buffers de saída pode ser definido quando o buffer é iniciado. Se definido como `0`, o buffer de saída será limitado apenas pela memória disponível para o PHP. Se definido como `1`, o buffer será descarregado após cada bloco de código produzir qualquer saída de comprimento diferente de zero.

O tamanho dos buffers de saída pode ser recuperado chamando ob_get_status().

Buffers de saída iniciados com ob_start() terão seus tamanhos de buffer definidos para o valor inteiro passado para o segundo parâmetro `chunk_size` da função. Se omitido, será definido como `0`.

O buffer de saída iniciado com output_buffering definido como `"On"` terá seu tamanho de buffer definido como 0. Se definido como um número inteiro, o tamanho do buffer corresponderá a esse número.

O tamanho do buffer do `"URL-Rewriter"`está definido como `0`, portanto ele é limitado apenas pela memória disponível para o PHP.

O tamanho do buffer de saída do `zlib` é controlado pela configuração zlib.output_compression do *php.ini*. Se definido como `"On"` o tamanho do buffer será `"16K"`(`16384`). Se definido como um número inteiro, o tamanho do buffer corresponderá a esse número em bytes.

Operações permitidas em buffers

As operações permitidas em buffers podem ser controladas passando uma das opções de controle de buffer para o terceiro parâmetro `flags` de ob_start(). Se omitido, todas as operações serão permitidas por padrão. Se `0` for usado, o buffer não poderá ser descarregado, limpo ou removido, mas seu conteúdo ainda poderá ser recuperado.

`PHP_OUTPUT_HANDLER_CLEANABLE` permite que ob_clean() limpe o conteúdo do buffer.

> **Aviso**
>
> A ausência da opção **`PHP_OUTPUT_HANDLER_CLEANABLE`** não impedirá que ob_end_clean() ou ob_get_clean() limpem o conteúdo do buffer.

`PHP_OUTPUT_HANDLER_FLUSHABLE` permite que ob_flush() descarregue o conteúdo do buffer.

> **Aviso**
>
> A ausência da opção **`PHP_OUTPUT_HANDLER_FLUSHABLE`** não impedriá que ob_end_flush() ou ob_get_flush() descarreguem o conteúdo do buffer.

`PHP_OUTPUT_HANDLER_REMOVABLE` permite que ob_end_clean(), ob_end_flush(), ob_get_clean() ou ob_get_flush() desliguem o buffer.

`PHP_OUTPUT_HANDLER_STDFLAGS`, a combinação das três opções irá permitir que cada uma das três operações seja realizada no buffer.

Manipuladores de Saída

Manipuladores de saída são funções do tipo callable associadas com buffers de saída que são invocados ao chamar ob_clean(), ob_flush(), ob_end_flush(), ob_get_flush(), ob_end_clean(), ob_get_clean() ou durante o processo de desligamento do PHP.

> **Nota** :
>
> O processo de desligamento irá descarregar o valor de retorno do manipulador.

Se omitido ou **`null`** ao iniciar o buffer de saída, o `"default output handler"` (manipulador de saída padão) interno será usado, retornando o conteúdo não modificado do buffer quando invocado. Manipuladores de saída podem ser usados para retornar uma versão modificada do conteúdo do buffer e/ou ter efeitos colaterais (por exemplo, enviar cabeçalhos).

O PHP vem com dois manipuladores de saída internos: `"default output handler"` e `"URL-Rewriter"` (que é integrado em seu próprio buffer de saída e no máximo duas instâncias dele podem ser iniciadas).

As extensões incluídas no PHP são equipadas com quatro manipuladores de saída adicionais: mb_output_handler(), ob_gzhandler(), ob_iconv_handler() e ob_tidyhandler().

Trabalhando com Manipuladores de Saída

Quando invocados, os manipuladores de saída recebem o conteúdo do buffer e uma máscara de bits indicando o status do buffer de saída.

function handler (string `$buffer` , int `$phase` = ?): string

`buffer`

Conteúdo do buffer de saída.

`phase`

Máscara de bits das constantes **`PHP_OUTPUT_HANDLER_*`** .

> **Aviso**
>
> Chamar qualquer das funções a seguir de dentro de um manipulador de saída resultará em um erro fatal: ob_clean(), ob_end_clean(), ob_end_flush(), ob_flush(), ob_get_clean(), ob_get_flush() ou ob_start().

> **Nota** :
>
> Se a constante **`PHP_OUTPUT_HANDLER_DISABLED`** de um manipulador estiver definida, ele não será invocado chamando ob_end_clean(), ob_end_flush(), ob_get_clean(), ob_get_flush(), ob_clean(), ob_flush() ou durante o processo de desligamento do PHP. Antes do PHP 8.4.0 esta opção não tinha efeito ao chamar ob_clean() ou ob_flush().

> **Nota** :
>
> O diretório de trabalho do script pode mudar dentro da função de desligamento em alguns servidores web, por ex. Apache ou o servidor web integrado.

Opções Passadas aos Manipuladores de Saída

A máscara de bits passada para o segundo parâmetro `phase` do manipulador de saída fornece informações sobre a invocação do manipulador.

> **Nota** :
>
> A máscara de bits pode incluir mais de uma opção e o operador binário `&` deve ser usado para verificar se uma opção está definida.

> **Aviso**
>
> O valor de **`PHP_OUTPUT_HANDLER_WRITE`** e seu apelido **`PHP_OUTPUT_HANDLER_CONT`** é `0` portanto, se ele está definido ou não só pode ser determinado usando-se um operador de igualdade (`==` ou `===`).

As seguintes opções são definidas em uma fase específica do ciclo de vida do manipulador: **`PHP_OUTPUT_HANDLER_START`** é definida quando um manipulador é invocado pela primeira vez. **`PHP_OUTPUT_HANDLER_FINAL`** ou seu apelido **`PHP_OUTPUT_HANDLER_END`** é definida quando um manipulador é invocado pela última vez, ou seja, ele está sendo desligado. Esta opção também é definida quando os buffers estão sendo desligados pelo processo de desligamento do PHP.

As seguintes opções são definidas por uma invocação específica do manipulador: **`PHP_OUTPUT_HANDLER_FLUSH`** é definida quando o manipulador é invocado chamando-se ob_flush(). **`PHP_OUTPUT_HANDLER_WRITE`** ou seu apelido **`PHP_OUTPUT_HANDLER_CONT`** é definida quando o tamanho de seu conteúdo é igual ou excede o tamanho do buffer e o manipulador é invocado enquanto o buffer está sendo descarregado automaticamente. **`PHP_OUTPUT_HANDLER_FLUSH`** é definida quando o manipulador é invocado chamando-se ob_clean(), ob_end_clean() ou ob_get_clean(). Quando ob_end_clean() ou ob_get_clean() são chamados, **`PHP_OUTPUT_HANDLER_FINAL`** também é definida.

> **Nota** :
>
> Quando ob_end_flush() ou ob_get_flush() são chamadas, `PHP_OUTPUT_HANDLER_FINAL` é definida mas `PHP_OUTPUT_HANDLER_FLUSH` não é.

Valores de Retorno do Menipulador de Saída

O valor de retorno do manipulador de saída é internamente forçado para uma string seguindo a semântica padrão de tipos do PHP, com duas exceções: arrays e booleanos.

Arrays são convertidos na string `"Array"` mas o alerta `Array to string conversion` (conversão de array para string) não é emitido.

Se o manipulador retorna `false`, o conteúdo do buffer é retornado. Se o manipulador retorna `true`, uma string vazia é retornada.

> **Nota** :
>
> Se o manipulador retorna `false` ou lança uma exceção, sua constante de estado `PHP_OUTPUT_HANDLER_DISABLED` é definida.

Examples

Índice

Uso básico

Exemplo #1 Exemplo de Controle de Saída

```php
<?php

ob_start();
echo "Hello\n";

setcookie("cookiename", "cookiedata");

ob_end_flush();

?>
```

No exemplo acima, a saída de echo seria armazenada no buffer de saída até que ob_end_flush() fosse chamada. Neste meio tempo, a chamada a setcookie() armazenou com sucesso um cookie sem causar um erro. (Normalmente não se pode enviar cabeçalhos ao navegador depois que dados já foram enviados.)

Uso de re-escrita na saída

Desde o PHP 7.1.0, output_add_rewrite_var() e output_reset_rewrite_vars() usam buffer dedicado de saída, isto é, não usam o buffer de saída trans sid.

Exemplo #1 Exemplo de re-escrita na saída

```php
<?php
// Este código funciona com PHP 7.1.0, 7.0.10, 5.6.25 e posteriores.

// HTTP_HOST é o servidor alvo padrão. Defina manualmente para fazer com o que o código de exemplo funcione.
$_SERVER['HTTP_HOST'] = 'php.net';

// Re-escrita na saída re-escreve apenas formulários. Adicione a=href.
// Etiquetas podem ser especificadas nome_da_etiqueta=atribulo_url, ex.: img=src,iframe=src
// Não é permitido espaço em branco entre as definições.
// Etiqueta 'form' é uma etiqueta especial que adiciona entrada oculta.
ini_set('url_rewriter.tags','a=href,form=');
var_dump(ini_get('url_rewriter.tags'));

// Isto é adicionado à URL e ao formulário
output_add_rewrite_var('test', 'value');
?>
<a href="//php.net/index.php?bug=1234">bug1234</a>
```

```
<form action="https://php.net/?bug=1234&edit=1" method="post">
 <input type="text" name="title" />
</form>
```

O exemplo acima produzirá:

```
<a href="//php.net/?bug=1234&test=value">bug1234</a>
<form action="https://php.net/?bug=1234&edit=1" method="post"><input type="hidden" name="test" value="value" />
 <input type="text" name="title" />
</form>
```

Desde o PHP 7.1.0, funções de re-escrita na saída têm suas próprias configurações INI, url_rewriter.tags e url_rewriter.hosts.

Funções para Control de Saída

Veja também

Veja também header() e setcookie().

Índice

flush

flush

(PHP 4, PHP 5, PHP 7, PHP 8)

flush — Descarrega o buffer na saída do sistema

Descrição

function **flush**(): void

Descarrega os buffers de escrita do PHP e do backend usado pelo pelo PHP (ex.: CGI, um servidor web). Em um ambiente de linha de comando, **flush()** tentará descarregar somente o conteúdo do buffer enquanto que em um contexto web, os cabeçalhos e o conteúdo do buffer são descarregados.

> **Nota** :
>
> **flush()** pode não ser capaz de passar por cima do esquema de buffer do servidor web e não tem efeito sobre nenhum sistema de buffer cliente-servidor no navegador.

> **Nota** :
>
> Esta função não tem nenhum efeito sobre manipuladores de saída de usuário como os inicializados por ob_start() ou output_add_rewrite_var().

> **Aviso**
>
> **flush()** pode interferir com manipuladores de saída que definem e enviam cabeçalhos em um contexto web (ex.: ob_gzhandler()), enviando cabeçalhos antes que esses manipuladores possam fazê-lo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.4.0	Descarregar cabeçalhos sem um corpo agora terá sucesso no FastCGI.

Veja também

- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo
- ob_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo
- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o

ob_clean

ob_clean

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_clean — Limpa (apaga) o conteúdo do buffer de saída ativo

Descrição

function **ob_clean**(): bool

Esta função chama o manipulador de saída (com a opção `PHP_OUTPUT_HANDLER_CLEAN`), descarta seu valor de retorno e limpa (apaga) o conteúdo do buffer de saída ativo.

Esta função não desliga o buffer de saída ativo como a função ob_end_clean() ou ob_get_clean() fazem.

ob_clean() falhará sem um buffer de saída ativo iniciado com a opção `PHP_OUTPUT_HANDLER_CLEANABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se a função falhar ela gera um `E_NOTICE`.

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o
- ob_get_clean() - Obtém o conteúdo do buffer de saída ativo e desliga-o
- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo

ob_end_clean

ob_end_clean

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_end_clean — Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o

Descrição

function **ob_end_clean**(): bool

Esta função chama o manipulador de saída (com as opções `PHP_OUTPUT_HANDLER_CLEAN` e `PHP_OUTPUT_HANDLER_FINAL`), descarta seu valor de retorno, descarta o conteúdo do buffer de saída ativo e desliga o mesmo.

ob_end_clean() falhará sem um buffer de saída ativo com a opção `PHP_OUTPUT_HANDLER_REMOVABLE`.

ob_end_clean() irá descartar o conteúdo do buffer de saída ativo mesmo se tiver sido iniciado sem a opção `PHP_OUTPUT_HANDLER_CLEANABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se a função falhar, ela gera um `E_NOTICE`.

Examples

O exemplo a seguir mostra uma maneira fácil de se livrar do conteúdo do buffer de saída ativo:

Exemplo #1 Exemplo de ob_end_clean()

```php
<?php
ob_start();
echo 'Este texto não será mostrado.';
ob_end_clean();
?>
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo
- ob_get_clean() - Obtém o conteúdo do buffer de saída ativo e desliga-o
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo

ob_end_flush

ob_end_flush

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_end_flush — Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo

Descrição

function **ob_end_flush**(): bool

Esta função chama o manipulador de saída (com a opção `PHP_OUTPUT_HANDLER_FINAL`), descarrega (envia) seu valor de retorno, descarta o conteúdo do buffer de saída ativo e o desliga.

ob_end_flush() irá falhar sem um buffer de saída ativo iniciado com a opção `PHP_OUTPUT_HANDLER_REMOVABLE`.

ob_end_flush() irá descarregar (enviar) e retornar o valor do manipulador de saída mesmo se o buffer de saída ativo tiver iniciado sem a opção `PHP_OUTPUT_HANDLER_FLUSHABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se a função falhar, ela gera um `E_NOTICE`.

Examples

Exemplo #1 Exemplo de ob_end_flush()

O exemplo a seguir mostra uma maneira fácil de descarregar e terminar todos os buffers de saída:

```php
<?php
  while (@ob_end_flush());
?>
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo
- ob_get_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo, retorna o conteúdo do buffer de saída ativo e desliga-o
- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o

ob_flush

ob_flush

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_flush — Descarrega (envia) o valor de retorno do manipulador de saída ativo

Descrição

function **ob_flush**(): bool

Esta função chama o manipulador de saída (com a opção `PHP_OUTPUT_HANDLER_FLUSH`), descarrega (envia) seu valor de retorno e descarta o conteúdo do buffer de saída ativo.

Esta função não desliga o buffer de saída ativo como as funções ob_end_flush() ou ob_get_flush() fazem.

ob_flush() falhará sem um buffer de saída ativo iniciado com a opção `PHP_OUTPUT_HANDLER_FLUSHABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se a função falhar ela gera um `E_NOTICE`.

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo
- ob_get_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo, retorna o conteúdo do buffer de saída ativo e desliga-o
- ob_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo

ob_get_clean

ob_get_clean

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_get_clean — Obtém o conteúdo do buffer de saída ativo e desliga-o

Descrição

function **ob_get_clean**(): string|false

Esta função chama o manipulador de saída (com as opções `PHP_OUTPUT_HANDLER_CLEAN` e `PHP_OUTPUT_HANDLER_FINAL`), descarta seu valor de retorno, retorna o conteúdo do buffer de saída ativo e desliga o mesmo.

ob_get_clean() irá falhar sem um buffer de saída ativo iniciado com a opção `PHP_OUTPUT_HANDLER_REMOVABLE`.

ob_get_clean() irá descartar o conteúdo do buffer de saída ativo mesmo se tiver sido iniciado sem a opção `PHP_OUTPUT_HANDLER_CLEANABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o conteúdo do buffer de saída ativo em caso de sucesso ou `false` em caso de falha.

> **Cuidado**
>
> **ob_get_clean()** retornará false mas não irá gerar um `E_NOTICE` se não houver buffer de saída ativo.

Erros/Exceções

Se a função falhar, ela gerará um `E_NOTICE`.

Examples

Exemplo #1 Um exemplo simples de ob_get_clean()

```php
<?php

ob_start();

echo "Olá, Mundo!";

$out = ob_get_clean();
$out = strtolower($out);

var_dump($out);
?>
```

O exemplo acima produzirá:

```
string(11) "olá, mundo!"
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo
- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o
- ob_get_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo, retorna o conteúdo do buffer de saída ativo e desliga-o

ob_get_contents

ob_get_contents

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_get_contents — Retorna o conteúdo do buffer de saída

Descrição

function **ob_get_contents**(): string|false

Obtém o conteúdo do buffer de saída sem limpá-lo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o conteúdo do buffer de saída ou `false`, se o buffer de saída não estiver ativo.

Examples

Exemplo #1 Um simples exemplo de ob_get_contents()

```php
<?php

ob_start();

echo "Olá, ";

$out1 = ob_get_contents();

echo "Mundo";

$out2 = ob_get_contents();

ob_end_clean();

var_dump($out1, $out2);
?>
```

O exemplo acima produzirá:

```
string(5) "Olá, "
string(10) "Olá, Mundo"
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_length() - Retorna o comprimento do buffer de saída

ob_get_flush

ob_get_flush

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_get_flush — Descarrega (envia) o valor de retorno do manipulador de saída ativo, retorna o conteúdo do buffer de saída ativo e desliga-o

Descrição

function **ob_get_flush**(): string|false

Esta função chama o manipulador de saída (com a opção `PHP_OUTPUT_HANDLER_FINAL`), descarrega (envia) seu valor de retorno, retorna o conteúdo do buffer de saída ativo e desliga o mesmo.

ob_get_flush() irá falhar sem um buffer de saída ativo iniciado com a opção `PHP_OUTPUT_HANDLER_REMOVABLE`.

ob_get_flush() irá descarregar (enviar) o valor de retorno do manipulador de saída mesmo se o buffer de saída ativo tiver sido iniciado sem a opção `PHP_OUTPUT_HANDLER_FLUSHABLE`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o conteúdo do buffer de saída ativo em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se a função falhar, ela gera um `E_NOTICE`.

Examples

Exemplo #1 Exemplo de ob_get_flush()

```php
<?php
// Usando output_buffering=On
print_r(ob_list_handlers());

// Grava o buffer em um arquivo
$buffer = ob_get_flush();
file_put_contents('buffer.txt', $buffer);

print_r(ob_list_handlers());
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => default output handler
)
Array
(
)
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo
- ob_get_clean() - Obtém o conteúdo do buffer de saída ativo e desliga-o

ob_get_length

ob_get_length

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

ob_get_length — Retorna o comprimento do buffer de saída

Descrição

function **ob_get_length**(): int|false

Esta função retornará o comprimento do conteúdo do buffer de saída, em bytes.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o comprimento do conteúdo do buffer de saída ou `false` se o buffer não estiver ativo.

Examples

Exemplo #1 Um simples exemplo de ob_get_length()

```php
<?php

ob_start();

echo "Olá, ";

$len1 = ob_get_length();

echo "Mundo";

$len2 = ob_get_length();

ob_end_clean();

echo $len1 . ", ." . $len2;
?>
```

O exemplo acima produzirá:

```
5, 10
```

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída

ob_get_level

ob_get_level

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_get_level — Retorna o nível de aninhamento do mecanismo de buffer de saída

Descrição

function **ob_get_level**(): int

Retorna o nível de aninhamento do mecanismo de buffer de saída.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nível de manipuladores de buffer de saída aninhados ou zero se o buffer de saída não estiver ativo.

> **Cuidado**
>
> Os valores para níveis idênticos entre **ob_get_level()** e ob_get_status() são deslocados em uma unidade. Para **ob_get_level()** o primeiro nível é `1`, enquanto que para ob_get_status() o primeiro nível é `0`.

Veja também

- ob_start() - Ativa o buffer de saída
- ob_get_status() - Obtém status dos buffers de saída
- ob_get_contents() - Retorna o conteúdo do buffer de saída

ob_get_status

ob_get_status

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_get_status — Obtém status dos buffers de saída

Descrição

function **ob_get_status**(bool `$full_status = false`): array

ob_get_status() retorna informação de status do buffer de saída do nível mais alto ou de todos os níveis de buffers de saída ativos se `full_status` estiver definido para `true`.

Parâmetros

`full_status`

> `true` para retornar todos os níveis de buffers de saída ativos. Se for `false` ou não definido, somente o buffer de saída de nível mais alto é retornado.

Valor Retornado

Se `full_status` for omitido ou `false` um array simples contendo informações de status no nível de saída ativo é retornado.

Se `full_status` for `true` um array com um elemento para cada nível de buffer de saída ativo é retornado. O nível de saída é usado como chave do array de nível superior e cada elemento do array em si é outro array que contém informações de status sobre um nível de saída ativo.

Um array vazio será retornada se o buffer de saída não estiver ativado.

Elementos do array de retorno de ob_get_status()

Chave	Valor
name	Nome do manipulador de saída ativo (veja os valores de retorno de ob_list_handlers() para detalhes)
type	`0` (manipulador interno) ou `1` (manipulador fornecido pelo usuário)
flags	Máscara de bits das opções definidas por ob_start(), o tipo de manipulador de saída (veja acima) e o status do processo de buffer (constantes `PHP_OUTPUT_HANDLER_*`). Se o manipulador processou o buffer com sucesso e não retornou `false`, `PHP_OUTPUT_HANDLER_STARTED` e `PHP_OUTPUT_HANDLER_PROCESSED` serão definidas. Se o manipulador falhou ao processar o buffer ou retornou `false`, `PHP_OUTPUT_HANDLER_STARTED` e `PHP_OUTPUT_HANDLER_DISABLED` serão definidas.
level	Nível de aninhamento de saída (com base em zero). Observe que o valor retornado para o mesmo nível por ob_get_level() está deslocado em uma unidade. O primeiro nível é `0` para **ob_get_status()** e `1` para ob_get_level().
chunk_size	Tamanho do bloco em bytes. Definido por ob_start() ou se output_buffering estiver habilitado e seu valor estiver definido como um número inteiro positivo.
buffer_size	Tamanho do buffer de saída em bytes
buffer_used	Tamanho dos dados no buffer de saída em bytes (o mesmo que o valor de retorno do tipo inteiro de ob_get_length())

Examples

Exemplo #1 Array retornado quando `full_status` é `false`

```
Array
(
    [name] => URL-Rewriter
    [type] => 0
    [flags] => 112
    [level] => 2
    [chunk_size] => 0
    [buffer_size] => 16384
    [buffer_used] => 1024
)
```

Exemplo #2 Array retornado quando `full_status` é `true`

```
Array
(
    [0] => Array
        (
            [name] => default output handler
            [type] => 0
            [flags] => 112
            [level] => 1
            [chunk_size] => 0
            [buffer_size] => 16384
            [buffer_used] => 2048
        )

    [1] => Array
        (
            [name] => URL-Rewriter
            [type] => 0
            [flags] => 112
            [level] => 2
            [chunk_size] => 0
            [buffer_size] => 16384
            [buffer_used] => 1024
        )

)
```

Veja também

- ob_get_level() - Retorna o nível de aninhamento do mecanismo de buffer de saída
- ob_list_handlers() - Lista todos os manipuladores de saída em uso
- ob_get_length() - Retorna o comprimento do buffer de saída
- ob_start() - Ativa o buffer de saída

ob_implicit_flush

ob_implicit_flush

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_implicit_flush — Liga ou desliga o descarregamento implícito

Descrição

function **ob_implicit_flush**(bool `$enable = true`): void

ob_implicit_flush() ativará ou desativará o descarregamento implícito. O descarregamento implícito resultará em uma operação de envio após cada bloco de código que resulte em saída, de modo que chamadas explícitas a flush() não serão mais necessárias.

> **Nota** :
>
> Imprimir strings vazias ou enviar cabeçalhos não é considerado saída e não resultará em uma operação de descarregamento.

> **Nota** :
>
> Esta função não tem nenhum efeito nos manipuladores de saída no nível do usuário, como aqueles iniciados por ob_start() ou output_add_rewrite_var().

Parâmetros

`enable`

`true` para ligar o descarregamento implícito, `false` para desligar.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `enable` agora espera um valor do tipo bool; anteriormente, um int era esperado.

Veja também

- flush() - Descarrega o buffer na saída do sistema
- ob_start() - Ativa o buffer de saída
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo

ob_list_handlers

ob_list_handlers

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_list_handlers — Lista todos os manipuladores de saída em uso

Descrição

function **ob_list_handlers**(): array

Lista todos os manipuladores de saída em uso.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Isso retornará um array com os manipuladores de saída em uso (se houver).

Se output_buffering estiver ativado e nenhum output_handler estiver definido, ou nenhuma função de retorno ou `null` foi passada para ob_start(), `"default output handler"` é retornado. Ativar output_buffering e definir um output_handler é equivalente a passar uma função interna (integrada) para ob_start().

Se um callable foi passado para ob_start(), o nome totalmente qualificado do callable é retornado. Se o callable for um objeto que implementa __invoke(), o nome totalmente qualificado do método __invoke() do objeto é retornado. Se o callable for um Closure, `"Closure::__invoke"` será retornado.

Examples

Exemplo #1 Exemplo de ob_list_handlers()

```php
<?php
// usando output_buffering=On, nenhum manipulador de saída definido
var_dump(ob_list_handlers());
ob_end_flush();

// nenhuma chamada de retorno ou null
ob_start();
var_dump(ob_list_handlers());
ob_end_flush();

// função anônima
ob_start(function($string) { return $string; });
var_dump(ob_list_handlers());
ob_end_flush();

// função seta
ob_start(fn($string) => $string);
var_dump(ob_list_handlers());
ob_end_flush();
```

```php
// chamável de primeira classe
$firstClassCallable = userDefinedFunction(...);

ob_start([$firstClassCallable, '__invoke']);
var_dump(ob_list_handlers());
ob_end_flush();

// função interna (integrada)
ob_start('print_r');
var_dump(ob_list_handlers());
ob_end_flush();

// função definida pelo usuário
function userDefinedFunction($string, $flags) { return $string; };

ob_start('userDefinedFunction');
var_dump(ob_list_handlers());
ob_end_flush();

class MyClass {
    public static function staticHandle($string) {
        return $string;
    }

    public static function handle($string) {
        return $string;
    }

    public function __invoke($string) {
        return $string;
    }
}

// classe e método estático
ob_start(['MyClass','staticHandle']);
var_dump(ob_list_handlers());
ob_end_flush();

// objeto e método não estático
ob_start([new MyClass,'handle']);
var_dump(ob_list_handlers());
ob_end_flush();

// objeto invocável
ob_start(new MyClass);
var_dump(ob_list_handlers());
ob_end_flush();
?>
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  string(22) "default output handler"
}
array(1) {
  [0]=>
  string(22) "default output handler"
}
array(1) {
  [0]=>
  string(7) "print_r"
}
array(1) {
  [0]=>
  string(19) "userDefinedFunction"
}
array(1) {
```

```
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(21) "MyClass::staticHandle"
}
array(1) {
  [0]=>
  string(15) "MyClass::handle"
}
array(1) {
  [0]=>
  string(17) "MyClass::__invoke"
}
```

Veja também

- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo
- ob_get_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo, retorna o conteúdo do buffer de saída ativo e desliga-o
- ob_start() - Ativa o buffer de saída

ob_start

ob_start

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_start — Ativa o buffer de saída

Descrição

function **ob_start**(?callable `$callback` = **`null`**, int `$chunk_size` = 0, int `$flags` = **`PHP_OUTPUT_HANDLER_STDFLAGS`**): bool

Esta função irá ativar o buffer de saída. Enquanto o buffer de saída estiver ativo, não é enviada a saída do script, (outros que não sejam cabeçalhos), ao invés disto a saída é guardada em um buffer interno. Consulte Que Saída é Armazenada em Buffer? sobre exatamente que saída é afetada.

Buffers de saída são empilháveis, ou seja, **ob_start()** pode ser chamada enquanto outro buffer estiver ativo. Se múltiplos buffers de saída estiverem ativos, a saída será filtrada sequencialmente atráves de cada uma delas na ordem de aninhamento. Consulte Aninhando Buffers de Saída para mais detalhes.

Consulte Buffers de Saída em Nível de Usuário para uma descrição detalhada de buffers de saída.

Parâmetros

`callback`

> Uma função `callback` do tipo callable pode ser especificada. Ela também pode ser ignorada passando-se **`null`**.
>
> `callback` é chamada quando o buffer de saída é descarregado (enviado), limpo, ou quando o buffer de saída é descarregado no final da execução do script.
>
> A assinatura da função `callback` deve ser da seguinte forma:
>
> function handler(string `$buffer`, int `$phase` = ?): string
> `buffer`

O conteúdo original do buffer.

`phase`

Máscara de bits das constantes **`PHP_OUTPUT_HANDLER_*`** . Consulte Opções Passadas aos Manipuladores de Saída para mais detalhes.

Se `callback` retornar **`false`**, o conteúdo do buffer é retornado. Consulte Valores de Retorno do Menipulador de Saída para mais detalhes.

> **Aviso**
>
> Chamar qualquer das funções a seguir de dentro de um manipulador de saída resultará em erro fatal: ob_clean(), ob_end_clean(), ob_end_flush(), ob_flush(), ob_get_clean(), ob_get_flush() e **ob_start()**.

Consulte Manipuladores de Saída e Trabalhando com Manipuladores de Saída para mais detalhes sobre as funções `callback` (manipuladores de saída).

`chunk_size`

Se o parâmetro opcional `chunk_size` for informado, o buffer será descarregado depois que algum bloco de código que produza saída faça com que o tamanho do buffer fique igual ou maior que `chunk_size`. O valor padrão é `0`, que significa que toda a saída fica em buffer até que ele seja desligado. Consulte Tamanho do Buffer para mais detalhes.

`flags`

O parâmetro `flags` é uma máscara de bits que controla as operações que podem ser realizadas no buffer de saída. O padrão é permitir que o buffer de saída possa ser limpo, descarregado ou removido, o que pode ser configurado explicitamente através das opções de controle de buffer . Consulte Operações permitidas em buffers para mais detalhes.

Cada opção controla o acesso das funções, como descrito a seguir:

Constante	Funções
`PHP_OUTPUT_HANDLER_CLEANABLE`	ob_clean()
`PHP_OUTPUT_HANDLER_FLUSHABLE`	ob_flush()
`PHP_OUTPUT_HANDLER_REMOVABLE`	ob_end_clean(), ob_end_flush(), ob_get_clean() e ob_get_flush()

> **Nota** :
>
> Antes do PHP 8.4.0, o parâmetro flags poderia definir também as opções de status do manipulador de saída.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de uma função de callback

```
<?php

function callback($buffer)
{
  // substitui apples por oranges
  return (str_replace("apples", "oranges", $buffer));
}

ob_start("callback");

?>
<html>
<body>
<p>It's like comparing apples to oranges.</p>
</body>
</html>
<?php
```

```php
ob_end_flush();

?>
```

O exemplo acima produzirá:

```
<html>
<body>
<p>It's like comparing oranges to oranges.</p>
</body>
</html>
```

Exemplo #2 Criando um buffer de saída não apagável

```php
<?php

ob_start(null, 0, PHP_OUTPUT_HANDLER_STDFLAGS ^ PHP_OUTPUT_HANDLER_REMOVABLE);

?>
```

Veja também

- ob_get_contents() - Retorna o conteúdo do buffer de saída
- ob_end_clean() - Limpa (apaga) o conteúdo do buffer de saída ativo e desliga-o
- ob_end_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo e desliga o buffer de saída ativo
- ob_implicit_flush() - Liga ou desliga o descarregamento implícito
- ob_gzhandler() - Função de chamada de retorno de ob_start para comprimir o buffer de saída com gzip
- ob_iconv_handler() - Converte codificação de caracteres como manipulador de buffer de saída
- mb_output_handler() - Callback converte a codificação de caracteres no buffer de saída
- ob_tidyhandler() - Função de retorno ob_start para reparar o buffer

output_add_rewrite_var

output_add_rewrite_var

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

output_add_rewrite_var — Adiciona valores de reescritor de URL

Descrição

function **output_add_rewrite_var**(string `$name`, string `$value`): bool

Esta função inicia o manipulador de buffer de saída `'URL-Rewriter'` se ele não estiver ativo, armazena os parâmetros `name` e `value`, e quando o buffer é descarregado reescreve os URLs e formulários com base nas configurações ini aplicáveis. As chamadas subsequentes a esta função armazenarão todos os pares nome/valor adicionais até que o manipulador seja desligado.

Quando o buffer de saída é descarregado (chamando ob_flush(), ob_end_flush(), ob_get_flush() ou no final do script), o manipulador `'URL-Rewriter'` adiciona os pares nome/valor como parâmetros de consulta para URLs em atributos de etiquetas HTML e adiciona campos ocultos aos formulários com base nos valores das diretivas de configuração url_rewriter.tags e url_rewriter.hosts.

Cada par nome/valor adicionado ao manipulador `'URL-Rewriter'` é adicionado aos URLs e/ou formulários, mesmo que isso resulte em parâmetros de consulta de URL duplicados ou elementos com atributos de mesmo nome.

> **Nota** :
>
> Depois que o manipulador `'URL-Rewriter'` for desligado, ele não poderá ser iniciado novamente.

> **Aviso**

> Antes do PHP 8.4.0, os hosts a serem sobrescritos eram definidos em session.trans_sid_hosts ao invés de url_rewriter.hosts.

Parâmetros

`name`

O nome da variável.

`value`

O valor da variável.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.1.0	A partir do PHP 7.1.0, um buffer de saída dedicado é usado, url_rewriter.tags é usado somente para funções de saída e url_rewriter.hosts está disponível. Antes do PHP 7.1.0, variáveis de reescrita definidas por **output_add_rewrite_var()** compartilhavam um buffer de saída com suporte transparente para ID de sessão (veja session.trans_sid_tags).

Examples

Exemplo #1 Exemplo de **output_add_rewrite_var()**

```php
<?php
ini_set('url_rewriter.tags', 'a=href,form=');

output_add_rewrite_var('var', 'value');

// alguns "links"
echo '<a href="file.php">link</a>
<a href="http://example.com">link2</a>';

// um formulário
echo '<form action="script.php" method="post">
<input type="text" name="var2" />
</form>';

print_r(ob_list_handlers());
?>
```

O exemplo acima produzirá:

```
<a href="file.php?var=value">link</a>
<a href="http://example.com">link2</a>

<form action="script.php" method="post">
<input type="hidden" name="var" value="value" />
<input type="text" name="var2" />
</form>

Array
(
    [0] => URL-Rewriter
)
```

Veja também

- output_reset_rewrite_vars() - Redefine valores de reescritor de URL

- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo
- ob_list_handlers() - Lista todos os manipuladores de saída em uso
- url_rewriter.tags
- url_rewriter.hosts

output_reset_rewrite_vars

output_reset_rewrite_vars

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

output_reset_rewrite_vars — Redefine valores de reescritor de URL

Descrição

function **output_reset_rewrite_vars**(): bool

Esta função remove todas as variáveis de reescrita previamente definidas pela função output_add_rewrite_var().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
7.1.0	Antes do PHP 7.1.0, as variáveis de reescrita definidas por output_add_rewrite_var() usavam o mesmo buffer de saída "trans sid" do módulo de sessão. Desde o PHP 7.1.0, um buffer de saída dedicado é usado e **output_reset_rewrite_vars()** apenas remove variáveis de reescrita definidas por output_add_rewrite_var().

Examples

Exemplo #1 Exemplo de output_reset_rewrite_vars()

```php
<?php
ini_set('url_rewriter.tags', 'a=href');

output_add_rewrite_var('var', 'value');

echo '<a href="file.php">link</a>';
ob_flush();

output_reset_rewrite_vars();
echo '<a href="file.php">link</a>';
?>
```

O exemplo acima produzirá:

```
<a href="file.php?var=value">link</a>
<a href="file.php">link</a>
```

Veja também

- output_add_rewrite_var() - Adiciona valores de reescritor de URL
- ob_flush() - Descarrega (envia) o valor de retorno do manipulador de saída ativo
- ob_list_handlers() - Lista todos os manipuladores de saída em uso
- url_rewriter.tags
- url_rewriter.hosts

- session.trans_sid_tags
- session.trans_sid_hosts

Informações e Opções do PHP

Introdução

Estas funções permitem obter muitas informações sobre o PHP em si, por exemplo, configurações em tempo de execução, extensões carregadas, versão e muito mais. Também podem ser encontradas funções para definir opções para o PHP em execução. A função que é provavelmente a mais conhecida do PHP - phpinfo() - pode ser encontrada aqui.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do PHP

Nome	Padrão	Modificável	Registro de Alterações
assert.active	"1"	INI_ALL	Descontinuado desde o PHP 8.3.0
assert.bail	"0"	INI_ALL	Descontinuado desde o PHP 8.3.0
assert.warning	"1"	INI_ALL	Descontinuado desde o PHP 8.3.0
assert.callback	NULL	INI_ALL	Descontinuado desde o PHP 8.3.0
assert.quiet_eval	"0"	INI_ALL	Removido desde o PHP 8.0.0
assert.exception	"1"	INI_ALL	Anteriormente ao PHP 8.0.0 o padrão era "0". Desencorajado desde o PHP 8.3.0
enable_dl	"1"	INI_SYSTEM	Este recurso descontinuado *será* certamente *removido* no futuro.
max_execution_time	"30"	INI_ALL	
max_input_time	"-1"	INI_PERDIR	
max_input_nesting_level	"64"	INI_PERDIR	
max_input_vars	1000	INI_PERDIR	
zend.enable_gc	"1"	INI_ALL	
zend.max_allowed_stack_size	"0"	INI_SYSTEM	Disponível a partir do PHP 8.3.0.
zend.reserved_stack_size	"0"	INI_SYSTEM	Disponível a partir do PHP 8.3.0.
fiber.stack_size		INI_ALL	Disponível a partir do PHP 8.1.0.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`assert.active` bool

Ativa a avaliação de assert(). zend.assertions deve ser utilizado para controlar o comportamento de assert().

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

`assert.bail` bool

Termina a execução de scripts se uma asserção falharem.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

`assert.warning` bool

Emite um warning para cada falha de assert.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

`assert.callback` string

Função a chamar em caso de asserts falharem.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

`assert.quiet_eval` bool

> **Aviso**
>
> Este recurso foi *REMOVIDO* a partir do PHP 8.0.0.

Utilize a configuração atual de error_reporting() durante a avaliação de expressões assert. Se ativo então nenhum erro é mostrado (padrão é error_reporting(0)) nas avaliações. Se desativa erros são mostrados de acordo com as configurações de error_reporting()

`assert.exception` bool

Lança uma exceção AssertionError no caso de um assert falhar.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

`enable_dl` bool

Esta diretiva permite ativar ou desativar o carregamento dinâmico de extensões do PHP com dl() .

A razão principal para desligar o carregamento dinâmico é segurança. Com carregamento dinâmico é possível ignorar todas as restrições de open_basedir. O padrão é permitir carregamento dinâmico.

`max_execution_time` int

Isso configura o tempo máximo, em segundos, que um script é permitido executar antes de ser terminado. Isso ajuda a prevenir que scripts mal escritos de lotar os servidores. O padrão é de `30` segundos. Quando rodando o PHP a partir da linha de comando o padrão é zero `0`.

O tempo máximo de execução não é afetado por chamadas de sistema, operações em fluxos, etc. Veja a documentação da função set_time_limit() para mais detalhes.

Quando o PHP é compilado com --enable-zend-max-execution-timers (o padrão para builds ZTS desde o PHP 8.3.0), o limite é rastreado com temporizadores POSIX por thread que medem o tempo decorrido, então o tempo gasto dormindo ou esperando por E/S conta para ele. O rastreamento padrão baseado em `setitimer(ITIMER_PROF)` mede o tempo de CPU em vez disso.

Seu servidor web pode ter outras configurações de timeout que também interrompam a execução do PHP. Apache possui uma diretiva `Timeout` e o IIS tem uma função de timeout CGI. Ambos com padrão de 300 segundos. Veja a documentação do servidor web para mais detalhes.

`max_input_time` int

Configura o tempo máximo, em segundos, que um script é permitido dispensar interpretando dados de entrada, como GET e POST. O cronômetro começa no momento que o PHP é chamado pelo servidor e termina quando a execução começa. A configuração padrão é `-1`, o que significa que o valor de max_execution_time será usado. Defina como `0` para permitir tempo ilimitado.

`max_input_nesting_level` int

Configura a profundidade máxima de níveis das variáveis de entrada (*$_GET*, *$_POST*).

`max_input_vars` int

Configura quantas variáveis de entrada serão aceitas, com o limte aplicado a cada super global $_GET, $_POST e $_COOKIE separadamente). O uso dessa diretiva mitiga a possibilidade de ataques de negação de serviço que utilizam colisões de hash. Se

houver mais variáveis do que o especificado nessa diretiva um `E_WARNING` é lançado, e variáveis de entrada adicionais são ignorados.

`zend.enable_gc` bool

Ativa ou desativa o coletor de refefências circulares.

`zend.max_allowed_stack_size` int

O espaço máximo de pilha nativa que o sistema operacional permite que o programa consuma. Tentar consumir mais do que o sistema operacional permite normalmente resulta em uma falha grave, sem informações de depuração facilmente disponíveis. Para facilitar a depuração, o mecanismo lança uma exceção Error antes que isso aconteça (quando o programa usa mais que zend.max_allowed_stack_size-zend.reserved_stack_size bytes de pilha).

A recursão no código definido pelo usuário não consome espaço de pilha nativo. No entanto, funções internas e métodos mágicos sim. A recursão muito profunda envolvendo essas funções pode fazer com que o programa esgote todo o espaço de pilha nativa disponível.

Os valores possíveis para este parâmetro são:

- `0`: Detecta automaticamente o espaço máximo de pilha nativa que o sistema operacional permite que o programa consuma. Este é o padrão. Quando a detecção não é possível, um padrão conhecido do sistema é usado.
- `-1`: Desativa a verificação do tamanho da pilha no mecanismo.
- Inteiro positivo: um tamanho fixo, em bytes. Definir esse valor muito alto tem o mesmo efeito que desabilitar a verificação do tamanho da pilha.

Como o tamanho da pilha dos fibers é determinado por fiber.stack_size, o valor deste parâmetro é usado em vez do zend.max_allowed_stack_size ao verificar o uso da pilha durante a execução de um Fiber.

> **Nota** :
>
> Isso não está relacionado a estouros de *buffer* de pilha e não é um recurso de segurança.

`zend.reserved_stack_size` int

O tamanho da pilha reservada, em bytes. Isso é subtraído do tamanho máximo permitido da pilha, como um buffer, ao verificar o tamanho da pilha.

Os valores possíveis para este parâmetro são:

- `0`: Detecta automaticamente um tamanho razoável.
- Inteiro positivo: um tamanho fixo, em bytes.

`fiber.stack_size` int

O tamanho da pilha nativa, em bytes, alocado para cada Fiber.

O valor padrão é 1MiB em sistemas com tamanho de ponteiro inferior a 8 bytes ou 2MiB caso contrário.

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

Constantes predefinidas da função phpcredits()

Constantes	Descrição
`CREDITS_GROUP` (int)	Uma lista do núcleo de desenvolvedores
`CREDITS_GENERAL` (int)	Créditos em geral: design e conceito da linguagem, autores do PHP e módulo SAPI.
`CREDITS_SAPI` (int)	Uma lista dos módulos API dos servidores para o PHP, e seus autores.
`CREDITS_MODULES` (int)	Uma lista dos módulos de extensão para o PHP, e seus autores.

CREDITS_DOCS (int)	Os créditos para a equipe de documentação.
CREDITS_FULLPAGE (int)	Normalmente usada em combinação com as outras opções. Indica que uma página HTML completa precisa ser mostrada incluindo a informação indicada pelas outras opções.
CREDITS_QA (int)	Os créditos para a equipe de controle de qualidade.
CREDITS_ALL (int)	Todos os créditos, equivalente a usar: `CREDITS_DOCS | CREDITS_GENERAL | CREDITS_GROUP | CREDITS_MODULES | CREDITS_QA | CREDITS_FULLPAGE`. Gera uma página HTML completa com as etiquetas apropriadas. Este é o valor padrão.

Constantes da função phpinfo()

Constantes	Descrição
INFO_GENERAL (int)	A linha da configuração, localização do *php.ini*, data de construção, servidor web, sistema e mais.
INFO_CREDITS (int)	Créditos do PHP. Veja também phpcredits().
INFO_CONFIGURATION (int)	Valores atuais e principais para as diretivas de configuração do PHP. Veja também ini_get().
INFO_MODULES (int)	Módulos carregados e suas respectivas configurações.
INFO_ENVIRONMENT (int)	Informações das variáveis de ambiente que também estão disponíveis em *$_ENV*.
INFO_VARIABLES (int)	Mostra todas as variáveis predefinidas do EGPCS (ENV, GET, POST, COOKIE, SERVER).
INFO_LICENSE (int)	Informação sobre a licença do PHP. Veja também » a seção sobre licença.
INFO_ALL (int)	Mostra todas as opções acima. Este é o valor padrão.

Constantes de modo INI

Constantes	Descrição
INI_USER (int)	A entrada pode der definida em scripts do usuário (como com ini_set()) ou no registro do Windows. A entrada pode ser definida no *.user.ini*.
INI_PERDIR (int)	A entrada pode der definida no *php.ini*, *.htaccess*, *httpd.conf* ou *.user.ini*.
INI_SYSTEM (int)	A entrada pode der definida no *php.ini* ou *httpd.conf*.
INI_ALL (int)	A entrada pode der definida em qualquer lugar.

Constantes de assetividade. Estes valores são usados para definir as opções de assertividade em assert_options().

Constantes da função assert()

Constantes	Descrição
ASSERT_ACTIVE (int)	Habilita avaliação da função assert().

	Aviso Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.
ASSERT_CALLBACK (int)	Função de retorno a ser chamada em afirmações falhadas. **Aviso** Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.
ASSERT_BAIL (int)	Termina execução em afirmações falhadas. **Aviso** Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.
ASSERT_EXCEPTION (int)	Emite uma AssertionError para cada afirmação falhada. **Aviso** Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.
ASSERT_WARNING (int)	Emite um alerta do PHP para cada afirmação falhada. **Aviso** Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.
ASSERT_QUIET_EVAL (int)	Desabilita `error_reporting` durante avaliação de expressão de afirmação. **Aviso** Este recurso foi *REMOVIDO* a partir do PHP 8.0.0.

As constantes a seguir somente estão disponíveis se o sistema operacional do servidor for Windows, e detalham diferentes informações de versão de forma a possibilitar detecção de vários recursos e fazer uso deles.

Constantes específicas do Windows

Constantes	Descrição
PHP_WINDOWS_VERSION_MAJOR (int)	A versão principal do Windows, que pode ser `4` (NT4/Me/98/95), `5` (XP/2003 R2/2003/2000) ou `6` (Vista/2008/7/8/8.1).
PHP_WINDOWS_VERSION_MINOR (int)	A versão secundária do Windows, que pode ser `0` (Vista/2008/2000/NT4/95), `1` (XP), `2` (2003 R2/2003/XP x64), `10` (98) ou `90` (ME).
PHP_WINDOWS_VERSION_BUILD (int)	O número de construção do Windows (por exemplo, Windows Vista com SP1 aplicado tem o número 6001)
PHP_WINDOWS_VERSION_PLATFORM (int)	A plataforma em que o PHP está atualmente em execução, este valor é `2` no Windows Vista/XP/2000/NT4, Server 2008/2003 e `1` no Windows ME/98/95.

PHP_WINDOWS_VERSION_SP_MAJOR (int)	A versão principal do pacote de serviço instalado, este valor é 0 se não houver nenhum instalado. Por exemplo, Windows XP com pacote de serviço 3 instalado terá valor de 3.
PHP_WINDOWS_VERSION_SP_MINOR (int)	A versão secundária do pacote de serviço instalado, este valor é 0 se não houver nenhum.
PHP_WINDOWS_VERSION_SUITEMASK (int)	Uma máscara de bits que informa se vários recursos do Windows estão instalados, consulte a tabela abaixo para possível valores.
PHP_WINDOWS_VERSION_PRODUCTTYPE (int)	Contém o valor usado para determinar as constantes PHP_WINDOWS_NT_*. Este valor pode ser uma das constantes PHP_WINDOWS_NT_* que indicam o tipo de plataforma.
PHP_WINDOWS_NT_DOMAIN_CONTROLLER (int)	Este é um controlador de domínio.
PHP_WINDOWS_NT_SERVER (int)	Este é um sistema de servidor (ex.: Server 2008/2003/2000), observe que se este for um controlador de domínio, será reportado como PHP_WINDOWS_NT_DOMAIN_CONTROLLER.
PHP_WINDOWS_NT_WORKSTATION (int)	Este é um sistema de estação de trabalho (ex.: Vista/XP/2000/NT4).

Esta tabela mostra uma lista de recursos que podem ser verificados para uso da máscara de bits PHP_WINDOWS_VERSION_SUITEMASK.

Campos de bits para suítes Windows

Bits	Descrição
0x00000004	Componentes do Microsoft BackOffice estão instalados.
0x00000400	Windows Server 2003, Web Edition está instalado.
0x00004000	Windows Server 2003, Compute Cluster Edition está instalado.
0x00000080	Windows Server 2008 Datacenter, Windows Server 2003, Datacenter Edition ou Windows 2000 Datacenter Server estão instalados.
0x00000002	Windows Server 2008 Enterprise, Windows Server 2003, Enterprise Edition, Windows 2000 Advanced Server ou Windows NT Server 4.0 Enterprise Edition estão instalados.
0x00000040	Windows XP Embedded está instalado.
0x00000200	Windows Vista Home Premium, Windows Vista Home Basic ou Windows XP Home Edition estão instalados.
0x00000100	Área de Trabalho Remota é suportada, mas somente uma sessão interativa é suportada. Este valor estará definido a menos que o sistema esteja sendo executado em modo de servidor de aplicação.
0x00000001	Microsoft Small Business Server foi instalado uma vez no sistema, mas pode ter sido atualizado para outra versão do Windows.
0x00000020	Microsoft Small Business Server está instalado com a licença de cliente restritiva.
0x00002000	Windows Storage Server 2003 R2 ou Windows Storage Server 2003 está instalado.
0x00000010	Terminal Services está instalado. Este valor sempre estará definido. Se este valor estiver definido mas 0x00000100 não estiver, então o sistema está sendo executado em modo de servidor de aplicação.
0x00008000	Windows Home Server está instalado.

Funções para Opções/Info do PHP

Índice

assert

assert

(PHP 4, PHP 5, PHP 7, PHP 8)

assert — Executa uma asserção

Descrição

function **assert**(mixed `$assertion`, Throwable|string|null `$description = null`): bool

assert() permite a definição de expectativas: asserções são verificadas em ambientes de desenvolvimento ou testes, mas que são removidas por otimização a custo zero em produção.

Asserções podem ser utilizadas para ajudar em depuração. Um dos usos é para funcionar como verificações de sanidade para precondições que precisam ser sempre `true` e caso elas não sejam confirmadas, isto indica algum erro de programação. Outro uso é garantir a presença de certos recursos, como por exemplo funções de extensões ou certos recursos ou limites de sistema.

As asserções podem ser configurados para serem eliminados, e elas *não* não devem ser utilizados em operações comuns como por exemplo a validação de parâmetros. Como regra geral, o código deve se comportar igual mesmo no caso das asserções estarem desativadas.

assert() verificará se a expectativa informada no parâmetro `assertion` é válida. Se não, o resultado avaliado é `false`, ocorrerá a ação apropriada, dependendo em como **assert()** está configurado.

O comportamento de **assert()** é ditado pelas seguintes configurações INI:

Opções de Configuração de Assert

Nome	Padrão	Descrição	Registro de Alterações
zend.assertions	`1`	• `1`: gera e executa o código (modo de desenvolvimento) • `0`: gera o código mas ignora durante a execução • `-1`: não gera o código (modo de produção)	
assert.active	`true`	Se `false`, **assert()** não verifica a expectativa e retorna `true` imediatamente.	Descontinuado desde o PHP 8.3.0.
assert.callback	`null`	Uma função definida pelo usuário a ser chamada quando a asserção falha. Sua assinatura deve ser: function **assert_callback**(string `$file`, int `$line`, null `$assertion`, string `$description = ?`): void	Anteriormente ao PHP 8.0.0, a assinatura deveria ser: function **assert_callback**(string `$file`, int `$line`, string `$assertion`, string `$description = ?`): void Descontinuado a partir do PHP 8.3.0.
assert.exception	`true`	Se `true` irá lançar um AssertionError no caso da expectativa não ser válida.	Descontinuado desde o PHP 8.3.0.
assert.bail	`false`	Se `true` então a execução do script PHP será abortada caso a expectativa não seja válida.	Descontinuado desde o PHP 8.3.0.
assert.warning	`true`	Se `true`, será emitido um `E_WARNING` no caso da expectativa não seja válida. Essa configuração INI é inefetiva caso assert.exception esteja ativo.	Descontinuado desde o PHP 8.3.0.

Parâmetros

`assertion`

Pode ser qualquer expressão que retorna um valor, qual será executado e o resultado é utilizado para indicar se a asserção deve passar ou falhar.

> **Aviso**
>
> Anteriormente ao PHP 8.0.0, se `assertion` era uma string ela era interpretada como código PHP e executada via eval(). A string era passada para o argumento do callback no terceiro argumento. Este comportamento está *DESCONTINUADO* desde o PHP 7.2.0, e foi *REMOVIDO* no PHP 8.0.0.

`description`

Se `description` é uma instância de Throwable, ela será lançada somente se `assertion` for executada e falhar.

> **Nota** :
>
> A partir do 8.0.0, isto ocorria *antes* de chamar a função de callback potencialmente definida.

> **Nota** :
>
> A partir do PHP 8.0.0, o object será lançado independente da configuração de assert.exception.

> **Nota** :
>
> A partird do PHP 8.0.0, a configuração assert.bail não terá efeito nesse caso.

Se `description` é uma string esta mensagem será utilizada em uma exceção ou em um aviso. Uma descrição opcional que será incluída na mensagem de falha no caso de `assertion` falhar.

Se `description` é omitido. Uma descrição padrão, igual ao código fonte na invocação de **assert()** é criada em tempo de compilação.

Valor Retornado

assert() sempre retorna **`true`** se ao menos uma das seguintes condições forem verdadeiras:

- `zend.assertions=0`
- `zend.assertions=-1`
- `assert.active=0`
- `assert.exception=1`
- `assert.bail=1`
- Um objeto exception foi passado em `description`.

Se nenhuma das condições forem verdadeiras **assert()** ainda pode retornar **`true`** se `assertion` possa ser convertido em um valor verdadeiro, ou **`false`** nos demais casos.

Registro de Alterações

Versão	Descrição
8.3.0	Todas as configurações INI `assert.` estão desencorajadas.
8.0.0	**assert()** não mais avaliará argumentos strings, e por outro lado elas serão tratadas como um argumento comum. `assert($a == $b)` deve ser utilizado ao invés de `assert('$a == $b')`. A diretiva `assert.quiet_eval` *php.ini* e a constante **`ASSERT_QUIET_EVAL`** também foram removidos, dado que elas não tem mais nenhum efeito.
8.0.0	Se `description` é uma instância de Throwable, a exceção é lançada no caso da asserção falhar, independentemente do valor de assert.exception.
8.0.0	Se `description` é uma instância de Throwable, o callback não é chamado, mesmo que ele seja informado.
8.0.0	Declarar uma função chamada `assert()` dentro de um namespace não é mais permitido, e emite um **`E_COMPILE_ERROR`**.
7.3.0	Declarar uma função chamada `assert()` dentro de um namespace foi descontinuado. Uma declaração assim emite um **`E_DEPRECATED`**.
7.2.0	Uso de strings no argumento `assertion` foi descontinuado. Isto agora emite um aviso **`E_DEPRECATED`** quando ambos assert.active e zend.assertions estão configurados para `1`.

Examples

Exemplo #1 Exemplo de **assert()**

```php
<?php
assert(1 > 2);
echo 'Hi!';
```

Se as asserções estiverem ativadas (`zend.assertions=1`), o exemplo acima emitirá:

```
```

```
Fatal error: Uncaught AssertionError: assert(1 > 2) in example.php:2
Stack trace:
#0 example.php(2): assert(false, 'assert(1 > 2)')
#1 {main}
  thrown in example.php on line 2
```

Se as asserções estiverem desativadas (`zend.assertions=0` ou `zend.assertions=-1`), o exemplo acima emitirá:

```
Hi!
```

Exemplo #2 Utilizando uma mensagem customizada

```php
<?php
assert(1 > 2, "Esperado que um seja maior que dois");
echo 'Hi!';
```

Se as asserções estiverem ativadas, o código acima emitirá:

```
Fatal error: Uncaught AssertionError: Esperado que um seja maior que dois in example.php:2
Stack trace:
#0 example.php(2): assert(false, 'Esperado que um...')
#1 {main}
  thrown in example.php on line 2
```

Se as asserções estiverem desativadas, o código acima emitirá:

```
Hi!
```

Exemplo #3 Utilizando uma classe de exceção

```php
<?php
class ArithmeticAssertionError extends AssertionError {}

assert(1 > 2, new ArithmeticAssertionError("Esperado que um seja maior que dois"));
echo 'Hi!';
```

Se as asserções estiverem ativas, o exemplo acima emitirá:

```
Fatal error: Uncaught ArithmeticAssertionError: Esperado que um seja maior que dois in example.php:4
Stack trace:
#0 {main}
  thrown in example.php on line 4
```

Se as asserções estiverem desativadas o exemplo acima emitirá:

```
Hi!
```

Veja também

- assert_options() - Define/obtém as várias opções de asserção

assert_options

assert_options

(PHP 4, PHP 5, PHP 7, PHP 8)

assert_options — Define/obtém as várias opções de asserção

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.3.0. O uso desta função é fortemente desencorajado.

Descrição

function **assert_options**(int `$option`, mixed `$value` = ?): mixed

Define as várias opções de controle de assert() ou apenas consulta suas várias configurações.

> **Nota** :
>
> O uso de **assert_options()** é desencorajado em favor de definir e obter as diretivas zend.assertions e assert.exception do *php.ini* com ini_set() e ini_get(), respectivamente.

Parâmetros

`option`

Opções de Asserção

Opção	Configuração INI	Valor padrão	Descrição
ASSERT_ACTIVE	assert.active	1	habilita avaliação de assert()
ASSERT_EXCEPTION	assert.exception	1	lança uma exceção AssertionError para cada asserção falhada
ASSERT_WARNING	assert.warning	1	emite um alerta PHP para cada asserção falhada
ASSERT_BAIL	assert.bail	0	termina a execução em asserções falhadas
ASSERT_QUIET_EVAL	assert.quiet_eval	0	desabilita registro de erros durante avaliação de expressões de asserção. Removido a partir do PHP 8.0.0.
ASSERT_CALLBACK	assert.callback	(`null`)	Função de retorno a ser chamada em asserções falhadas

`value`

Um novo valor opcional para a opção.

A função de retorno definida via `ASSERT_CALLBACK` ou assert.callback deve ter a seguinte assinatura:

```
function assert_callback(
    string $file,
    int $line,
    ?string $assertion,
    string $description = ?
): void
```

`file`

O arquivo onde assert() foi chamada.

`line`

A linha onde assert() foi chamada.

`assertion`

Antes do PHP 8.0.0, a asserção que foi passada a assert(), mas somente quando a asserção for informada como uma string. (Se a asserção for uma condição booleana, este parâmetro será uma string vazia.) A partir do PHP 8.0.0, este parâmetro é sempre `null`.

`description`

A descrição que foi passada a assert().

Passar uma string vazia no parâmetro `value` redefine a função de retorno da asserção.

Valor Retornado

Retorna a configuração original de qualquer opção.

Erros/Exceções

Se `option` não for uma opção válida, uma exceção ValueError é lançada.

Registro de Alterações

Versão	Descrição
8.3.0	**assert_options()** agora foi descontinuada.
8.0.0	Se `option` não for uma opção válida, uma exceção ValueError é lançada. Anteriormente, `false` era retornado.

Examples

Exemplo #1 Exemplo de assert_options()

```php
<?php
// Esta é nossa função para manipular
// falhas de asserção
function assert_failure($file, $line, $assertion, $message)
{
    echo "A asserção $assertion em $file na linha $line falhou: $message";
}

// Esta é nossa função de teste
function test_assert($parameter)
{
    assert(is_bool($parameter));
}

// Define as opções de asserção
assert_options(ASSERT_ACTIVE,   true);
assert_options(ASSERT_BAIL,     true);
assert_options(ASSERT_WARNING,  false);
assert_options(ASSERT_CALLBACK, 'assert_failure');

// Cria uma asserção que iria falhar
test_assert(1);

// Isto nunca é alcançado devido a ASSERT_BAIL
// definido como verdadeiro
echo 'Nunca alcançado';
?>
```

Veja também

- assert() - Executa uma asserção

cli_get_process_title

cli_get_process_title

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

cli_get_process_title — Retorna o título do processo atual

Descrição

function **cli_get_process_title**(): ?string

Retorna o título do processo atual, conforme definido por cli_set_process_title(). Observe que isso pode não corresponder exatamente ao que é mostrado em **ps** ou **top**, dependendo do seu sistema operacional.

Esta função está disponível somente no modo CLI.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string com o título do processo atual ou `null` em caso de erro.

Erros/Exceções

Um `E_WARNING` será gerado se o sistema operacional não for compatível.

Examples

Exemplo #1 Exemplo de cli_get_process_title()

```php
<?php
echo "Título do processo: " . cli_get_process_title() . "\n";
?>
```

Veja também

- cli_set_process_title() - Define o título do processo

cli_set_process_title

cli_set_process_title

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

cli_set_process_title — Define o título do processo

Descrição

function **cli_set_process_title**(string `$title`): bool

Define o título do processo, visível em ferramentas como **top** e **ps**. Esta função está disponível apenas no modo CLI.

Parâmetros

`title`

O novo título.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Um `E_WARNING` será gerado se o sistema operacional não for compatível.

Registro de Alterações

Versão	Descrição
8.5.0	**cli_set_process_title()** agora emite um `E_WARNING` ao definir um título muito longo para um processo; anteriormente o título seria truncado.

Examples

```php
<?php
$title = "Meu Super Script PHP";
$pid = getmypid(); // pode-se usar isto para ver o título do processo no comando "ps"

if (!cli_set_process_title($title)) {
    echo "Não foi possível definir o título do processo para o PID $pid...\n";
    exit(1);
} else {
    echo "O título '$title' para o PID $pid foi definido para o seu processo!\n";
    sleep(5);
}
?>
```

Veja também

- cli_get_process_title() - Retorna o título do processo atual

dl

dl

(PHP 4, PHP 5, PHP 7, PHP 8)

dl — Carrega uma extensão do PHP durante a execução

Descrição

function **dl**(string `$extension_filename`): bool

Carrega a extensão do PHP indicada pelo parâmetro `extension_filename`.

Use extension_loaded() para testar se uma extensão está disponível ou não. Isto funciona para extensões internas e para aquelas carregadas dinamicamente (através do *php.ini* ou da função **dl()**).

> **Aviso**
>
> Esta função somente está disponível no modos CLI e SAPI internalizado, e na SAPI CGI quando rodado em linha de comando.

Parâmetros

`extension_filename`

Este parâmetro é *somente* o nome do arquivo da extensão a carregar o qual também depende da sua plataforma. Por exemplo, a extensão sockets (se compilada como módulo compartilhado, não o padrão!) seria chamada *sockets.so* em plataformas Unix enquanto é chamada *php_sockets.dll* na plataforma Windows.

O diretório a partir do qual a extensão é carregada depende da sua plataforma:

Windows - Se não for explicitamente definido no *php.ini*, a extensão é carregada a partir de *C:\php5* por padrão.

Unix - Se não for explicitamente definido no *php.ini*, o diretório padrão das extensões depende de

- se o PHP foi compilado com `--enable-debug` ou não
- se o PHP foi compilado com suporte ZTS (Zend Thread Safety) ou não
- O `ZEND_MODULE_API_NO` (o número Zend internal module API, o que é basicamente a data onde houve uma mudança maior na API ex. `20010901`)

Levando em conta o acima, o diretório padrão poderá ser `<install-dir>/lib/php/extensions/ <debug-or-not>-<zts-or-not>-ZEND_MODULE_API_NO`, e.g. */usr/local/php/lib/php/extensions/debug-non-zts-20010901* ou */usr/local/php/lib/php/extensions/no-debug-zts-20010901*.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha. Se a funcionalidade de carregar módulos não estiver disponível ou foi desabilitada (por configurar enable_dl como off no *php.ini*) então um erro `E_ERROR` é disparado e a execução é interrompida. Se **dl()** falhar porque a biblioteca especificada não puder ser carregada, além de retornar `false` uma mensagem `E_WARNING` é emitida.

Examples

Exemplo #1 Exemplos de dl()

```php
<?php
// Exemplo de carregamento de uma extensão com base no sistema operacional
if (!extension_loaded('sqlite')) {
    if (strtoupper(substr(PHP_OS, 0, 3)) === 'WIN') {
        dl('php_sqlite.dll');
    } else {
        dl('sqlite.so');
    }
}

// Ou, a constante PHP_SHLIB_SUFFIX está disponível desde o PHP 4.3.0
if (!extension_loaded('sqlite')) {
    $prefix = (PHP_SHLIB_SUFFIX === 'dll') ? 'php_' : '';
    dl($prefix . 'sqlite.' . PHP_SHLIB_SUFFIX);
}
?>
```

Notas

> **Nota** :
>
> **dl()** diferencia maiúsculas e minúsculas em sistemas Unix.

Veja também

- Diretivas de carregamento de extensões
- extension_loaded() - Descobre se uma extensão está carregada

extension_loaded

extension_loaded

(PHP 4, PHP 5, PHP 7, PHP 8)

extension_loaded — Descobre se uma extensão está carregada

Descrição

function **extension_loaded**(string `$extension`): bool

Descobre se uma extensão está carregada.

Parâmetros

`extension`

O nome da extensão. O parâmetro não faz distinção de maiúsculas/minúsculas.

Pode-se ver os nomes de várias extensões usando phpinfo(); ou se estiver sendo usada a versão `CGI` ou `CLI` do PHP, pode-se usar a opção **-m** para listar todas as extensões disponíveis:

```
$ php -m
[PHP Modules]
xml
tokenizer
standard
```

```
sockets
session
posix
pcre
overload
mysql
mbstring
ctype

[Zend Modules]
```

Valor Retornado

Retorna `true` se a extensão identificada por `extension` estiver carregada, `false` caso contrário.

Examples

Exemplo #1 Exemplo de extension_loaded()

```php
<?php
if (!extension_loaded('gd')) {
    if (!dl('gd.so')) {
        exit;
    }
}
?>
```

Veja também

- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- get_extension_funcs() - Retorna um array com os nomes das funções de um módulo
- phpinfo() - Mostra informações sobre a configuração do PHP
- dl() - Carrega uma extensão do PHP durante a execução
- function_exists() - Retorna true se a função informada estiver definida

gc_collect_cycles

gc_collect_cycles

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_collect_cycles — Força a coleta de quaisquer ciclos de lixo existentes.

Descrição

function **gc_collect_cycles**(): int

Força a coleta de quaisquer ciclos de lixo existentes.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de ciclos coletados.

Registro de Alterações

Versão	Descrição
8.5.0	O valor de retorno não mais inclui strings e recursos que foram indiretamente coletados através dos ciclos.

Veja também

- Coleta de Lixo

gc_disable

gc_disable

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_disable — Desativa o coletor de referência circular

Descrição

function **gc_disable**(): void

Desativa o coletor de referência circular, definindo zend.enable_gc para `0`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- Coleta de Lixo

gc_enable

gc_enable

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_enable — Ativa o coletor de referência circular

Descrição

function **gc_enable**(): void

Ativa o coletor de referência circular, definindo zend.enable_gc para `1`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- Coleta de Lixo

gc_enabled

gc_enabled

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_enabled — Retorna o status do coletor de referência circular

Descrição

function **gc_enabled**(): bool

Retorna o status do coletor de referência circular.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o coletor de lixo estiver habilitado, `false` caso contrário.

Examples

Exemplo #1 Um exemplo de gc_enabled()

```
<?php
if(gc_enabled()) gc_collect_cycles();
?>
```

Veja também

- Coleta de Lixo

gc_mem_caches

gc_mem_caches

(PHP 7, PHP 8)

gc_mem_caches — Recupera memória usada pelo gerenciador de memória Zend Engine

Descrição

function **gc_mem_caches**(): int

Recupera memória usada pelo gerenciador de memória Zend Engine.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de bytes liberados.

Veja também

- Coleta de Lixo

gc_status

gc_status

(PHP 7 >= 7.3.0, PHP 8)

gc_status — Obtém informações sobre o coletor de lixo

Descrição

function **gc_status**(): array

Obtém informações sobre o status atual do coletor de lixo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array associativo com os seguintes elementos:

- `"runs"`
- `"collected"`
- `"threshold"`
- `"roots"`
- `"running"`

- `"protected"`
- `"full"`
- `"buffer_size"`
- `"application_time"`
- `"collector_time"`
- `"destructor_time"`
- `"free_time"`

Registro de Alterações

Versão	Descrição
8.3.0	**gc_status()** agora retorna os seguintes campos adicionais: `"running"`, `"protected"`, `"full"`, `"buffer_size"`, `"application_time"`, `"collector_time"`, `"destructor_time"` e `"free_time"`.

Examples

Exemplo #1 Uso de gc_status()

```php
<?php

// cria árvore de objetos que precisam de coleta de lixo
$a = new stdClass();
$a->b = [];
for ($i = 0; $i < 100000; $i++) {
    $b = new stdClass();
    $b->a = $a;
    $a->b[] = $b;
}
unset($a);
unset($b);
gc_collect_cycles();

var_dump(gc_status());
```

O exemplo acima produzirá algo semelhante a:

```
array(4) {
  ["runs"]=>
  int(5)
  ["collected"]=>
  int(100002)
  ["threshold"]=>
  int(50001)
  ["roots"]=>
  int(0)
}
```

A saída do exemplo acima no PHP 8.3 é semelhante a:

```
array(12) {
  ["running"]=>
  bool(false)
  ["protected"]=>
  bool(false)
  ["full"]=>
  bool(false)
```

```
  ["runs"]=>
  int(5)
  ["collected"]=>
  int(100002)
  ["threshold"]=>
  int(50001)
  ["buffer_size"]=>
  int(131072)
  ["roots"]=>
  int(0)
  ["application_time"]=>
  float(0.031182458)
  ["collector_time"]=>
  float(0.020106291)
  ["destructor_time"]=>
  float(0)
  ["free_time"]=>
  float(0.003707167)
}
```

Veja também

- Coleta de Lixo

get_cfg_var

get_cfg_var

(PHP 4, PHP 5, PHP 7, PHP 8)

get_cfg_var — Obtém o valor de uma opção de configuração do PHP

Descrição

function **get_cfg_var**(string `$option`): string|array|false

Obtém o valor de uma opção de configuração do PHP definida em `option`.

Esta função não retornará informações de configuração definidas quando o PHP foi compilado ou lidas de um arquivo de configuração do Apache.

Para verificar se o sistema está usando um arquivo de configuração, tente recuperar o valor da configuração cfg_file_path. Se estiver disponível, um arquivo de configuração está sendo usado.

Parâmetros

`option`

O nome da opção de configuração.

Valor Retornado

Retorna o valor atual da variável de configuração do PHP especificada por `option` ou **`false`** se ocorrer um erro.

Veja também

- ini_get() - Obtém o valor de uma opção de configuração
- ini_get_all() - Obtém todas as opções de configuração

get_current_user

get_current_user

(PHP 4, PHP 5, PHP 7, PHP 8)

get_current_user — Obtém o nome do proprietário do script PHP atual

Descrição

function **get_current_user**(): string

Obtém o nome do proprietário do script PHP atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome de usuário como uma string.

Examples

Exemplo #1 Exemplo de get_current_user()

```
<?php
echo 'Proprietário do script atual: ' . get_current_user();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Proprietário do script atual: SYSTEM
```

Veja também

- getmyuid() - Obtém o UID do proprietário do script PHP
- getmygid() - Obtém o GID do proprietário do script PHP
- getmypid() - Obtém o ID do processo PHP
- getmyinode() - Obtém o inode do script atual
- getlastmod() - Obtém o horário da última modificação na página

get_defined_constants

get_defined_constants

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

get_defined_constants — Retorna um array associativo com os nomes de todas as constantes e seus valores

Descrição

function **get_defined_constants**(bool `$categorize = false`): array

Retorna os nomes e valores de todas as constantes definidas atualmente. Isto inclui aquelas criadas por extensões, bem como aquelas criados com a função define().

Parâmetros

`categorize`

Faz com que esta função retorne um array multidimensional com categorias nas chaves da primeira dimensão e constantes e seus valores na segunda dimensão.

```
<?php
define("MY_CONSTANT", 1);
print_r(get_defined_constants(true));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [Core] => Array
        (
            [E_ERROR] => 1
            [E_WARNING] => 2
            [E_PARSE] => 4
            [E_NOTICE] => 8
            [E_CORE_ERROR] => 16
            [E_CORE_WARNING] => 32
            [E_COMPILE_ERROR] => 64
            [E_COMPILE_WARNING] => 128
            [E_USER_ERROR] => 256
            [E_USER_WARNING] => 512
            [E_USER_NOTICE] => 1024
            [E_ALL] => 2047
            [TRUE] => 1
        )

    [pcre] => Array
        (
            [PREG_PATTERN_ORDER] => 1
            [PREG_SET_ORDER] => 2
            [PREG_OFFSET_CAPTURE] => 256
            [PREG_SPLIT_NO_EMPTY] => 1
            [PREG_SPLIT_DELIM_CAPTURE] => 2
            [PREG_SPLIT_OFFSET_CAPTURE] => 4
            [PREG_GREP_INVERT] => 1
        )

    [user] => Array
        (
            [MY_CONSTANT] => 1
        )

)
```

Valor Retornado

Retorna um array de nome de constante => valor da constante, opcionalmente agrupado por nome de extensão que registrou a constante.

Examples

Exemplo #1 Exemplo de **get_defined_constants()**

```php
<?php
print_r(get_defined_constants());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [E_ERROR] => 1
    [E_WARNING] => 2
    [E_PARSE] => 4
    [E_NOTICE] => 8
    [E_CORE_ERROR] => 16
    [E_CORE_WARNING] => 32
    [E_COMPILE_ERROR] => 64
    [E_COMPILE_WARNING] => 128
```

```
    [E_USER_ERROR] => 256
    [E_USER_WARNING] => 512
    [E_USER_NOTICE] => 1024
    [E_ALL] => 2047
    [TRUE] => 1
)
```

Veja também

- defined() - Verifica se existe uma constante com o nome informado
- constant() - Retorna o valor de uma constante
- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- get_defined_functions() - Retorna um array com todas as funções definidas
- get_defined_vars() - Retorna um array com todas variáveis definidas

get_extension_funcs

get_extension_funcs

(PHP 4, PHP 5, PHP 7, PHP 8)

get_extension_funcs — Retorna um array com os nomes das funções de um módulo

Descrição

function **get_extension_funcs**(string `$extension`): array|false

Esta função retorna os nomes de todas as funções definidas no módulo indicado por `extension`.

Parâmetros

`extension`

O nome do módulo.

> **Nota** :
>
> Este parâmetro precisa estar em *minúsculas*.

Valor Retornado

Retorna um array com todas as funções, ou **`false`** se `extension` não for uma extensão válida.

Examples

Exemplo #1 Imprime as funções de XML

```php
<?php
print_r(get_extension_funcs("xml"));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => xml_parser_create
    [1] => xml_parser_create_ns
    [2] => xml_set_object
    [3] => xml_set_element_handler
    [4] => xml_set_character_data_handler
    [5] => xml_set_processing_instruction_handler
    [6] => xml_set_default_handler
    [7] => xml_set_unparsed_entity_decl_handler
```

```
    [8] => xml_set_notation_decl_handler
    [9] => xml_set_external_entity_ref_handler
    [10] => xml_set_start_namespace_decl_handler
    [11] => xml_set_end_namespace_decl_handler
    [12] => xml_parse
    [13] => xml_parse_into_struct
    [14] => xml_get_error_code
    [15] => xml_error_string
    [16] => xml_get_current_line_number
    [17] => xml_get_current_column_number
    [18] => xml_get_current_byte_index
    [19] => xml_parser_free
    [20] => xml_parser_set_option
    [21] => xml_parser_get_option
)
```

Veja também

- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- ReflectionExtension::getFunctions() - Obtém funções de extensão

get_include_path

get_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

get_include_path — Obtém a opção de configuração include_path atual

Descrição

function **get_include_path**(): string|false

Obtém a opção de configuração include_path atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho, como uma string, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de get_include_path()

```php
<?php
echo get_include_path();

// Ou usando ini_get()
echo ini_get('include_path');
?>
```

Veja também

- ini_get() - Obtém o valor de uma opção de configuração
- restore_include_path() - Restaura o valor da opção de configuração include_path
- set_include_path() - Define a opção de configuração include_path
- include - include

get_included_files

get_included_files

(PHP 4, PHP 5, PHP 7, PHP 8)

get_included_files — Retorna um array com os nomes dos arquivos incluídos ou requeridos

Descrição

function **get_included_files**(): array

Obtém os nomes de todos os arquivos que foram incluídos usando include, include_once, require ou require_once.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com os nomes de todos os arquivos.

O script chamado originalmente é considerado um "arquivo incluído", portanto será listado junto com os arquivos referenciados por include e cia.

Os arquivos incluídos ou requeridos várias vezes aparecem apenas uma vez no array retornado.

Examples

Exemplo #1 Exemplo de get_included_files()

```php
<?php
// Este arquivo é o abc.php

include 'test1.php';
include_once 'test2.php';
require 'test3.php';
require_once 'test4.php';

$included_files = get_included_files();

foreach ($included_files as $filename) {
    echo "$filename\n";
}

?>
```

O exemplo acima produzirá:

```
/caminho/para/abc.php
/caminho/para/test1.php
/caminho/para/test2.php
/caminho/para/test3.php
/caminho/para/test4.php
```

Veja também

- include - include
- include_once - include_once
- require - require
- require_once - require_once
- get_required_files() - Sinônimo de get_included_files

get_loaded_extensions

get_loaded_extensions

(PHP 4, PHP 5, PHP 7, PHP 8)

get_loaded_extensions — Retorna um array com os nomes de todos os módulos compilados e carregados

Descrição

function **get_loaded_extensions**(bool `$zend_extensions = false`): array

Esta função retorna os nomes de todos os módulos compilados e carregados no interpretador PHP.

Parâmetros

`zend_extensions`

Retorna apenas extensões Zend, caso contrário, extensões comuns, como mysqli, serão listadas. O padrão é `false` (retorna extensões comuns).

Valor Retornado

Retorna um array indexado com todos os nomes dos módulos.

Examples

Exemplo #1 Exemplo de **get_loaded_extensions()**

```php
<?php
print_r(get_loaded_extensions());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Core
    [1] => date
    [2] => libxml
    [3] => pcre
    [4] => sqlite3
    [5] => zlib
    [6] => ctype
    [7] => dom
    [8] => fileinfo
    [9] => filter
    [10] => hash
    [11] => json
    [12] => mbstring
    [13] => SPL
    [14] => PDO
    [15] => session
    [16] => posix
    [17] => Reflection
    [18] => standard
    [19] => SimpleXML
    [20] => pdo_sqlite
    [21] => Phar
    [22] => tokenizer
    [23] => xml
    [24] => xmlreader
    [25] => xmlwriter
    [26] => gmp
    [27] => iconv
    [28] => intl
    [29] => bcmath
    [30] => sodium
    [31] => Zend OPcache
)
```

Veja também

- get_extension_funcs() - Retorna um array com os nomes das funções de um módulo
- extension_loaded() - Descobre se uma extensão está carregada
- dl() - Carrega uma extensão do PHP durante a execução
- phpinfo() - Mostra informações sobre a configuração do PHP

get_magic_quotes_gpc

get_magic_quotes_gpc

(PHP 4, PHP 5, PHP 7)

get_magic_quotes_gpc — Obtém a configuração atual de magic_quotes_gpc

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **get_magic_quotes_gpc**(): false

Sempre retorna `false`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `false`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Veja também

- addslashes() - Adiciona barras invertidas a uma string
- stripslashes() - Desfaz os escapes de uma string escapada
- get_magic_quotes_runtime() - Obtém a configuração ativa para magic_quotes_runtime
- ini_get() - Obtém o valor de uma opção de configuração

get_magic_quotes_runtime

get_magic_quotes_runtime

(PHP 4, PHP 5, PHP 7)

get_magic_quotes_runtime — Obtém a configuração ativa para magic_quotes_runtime

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **get_magic_quotes_runtime**(): false

Sempre retorna `false`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `false`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Veja também

- get_magic_quotes_gpc() - Obtém a configuração atual de magic_quotes_gpc

get_required_files

get_required_files

(PHP 4, PHP 5, PHP 7, PHP 8)

get_required_files — Sinônimo de get_included_files()

Descrição

Esta função é um apelido para: get_included_files().

get_resources

get_resources

(PHP 7, PHP 8)

get_resources — Retorna recursos ativos

Descrição

function **get_resources**(?string `$type = null`): array

Retorna um array de todos os recursos do tipo resource atualmente ativos, opcionalmente filtrados por tipo de recurso.

> **Nota** :
>
> Esta função destina-se a fins de depuração e teste. Não deve ser usada em ambientes de produção, especialmente para não acessar ou mesmo manipular recursos que normalmente não são acessíveis (por exemplo, o recurso de fluxo subjacente de instâncias de SplFileObject).

Parâmetros

`type`

Se definido, isso fará com que **get_resources()** retorne apenas recursos do tipo especificado. Uma lista de tipos de recursos está disponível.

Se a string `Unknown` for fornecida como o tipo, somente os recursos de tipo desconhecido serão retornados.

Se omitido, todos os recursos serão devolvidos.

Valor Retornado

Retorna um array de recursos atualmente ativos, indexados pelo número do recurso.

Registro de Alterações

Versão	Descrição
8.0.0	`type` agora pode ser nulo.

Examples

Exemplo #1 get_resources() sem filtro

```php
<?php
$fp = tmpfile();
var_dump(get_resources());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [1]=>
  resource(1) of type (stream)
}
```

Exemplo #2 get_resources() com filtro

```php
<?php
$fp = tmpfile();
var_dump(get_resources('stream'));
var_dump(get_resources('curl'));
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [1]=>
  resource(1) of type (stream)
}
array(0) {
}
```

Veja também

- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- get_defined_constants() - Retorna um array associativo com os nomes de todas as constantes e seus valores
- get_defined_functions() - Retorna um array com todas as funções definidas
- get_defined_vars() - Retorna um array com todas variáveis definidas

getenv

getenv

(PHP 4, PHP 5, PHP 7, PHP 8)

getenv — Obtém o valor de uma única ou de todas as variáveis de ambiente

Descrição

function **getenv**(?string `$name` = `null`, bool `$local_only` = `false`): string|array|false

Obtém o valor de uma única ou de todas as variáveis de ambiente.

Uma lista de todas as variáveis dem ambiente pode ser vista usando phpinfo(). Muitas dessas variáveis estão listadas na » RFC 3875, especificamente na seção 4.1, "Request Meta-Variables".

Parâmetros

`name`

O nome da variável como uma string ou `null`.

`local_only`

Quando definido como `true`, apenas variáveis de ambiente locais são retornadas, definidas pelo sistema operacional ou por putenv. Só tem efeito quando `name` é uma string.

Valor Retornado

Retorna o valor da variável de ambiente `name` ou `false` se a variável de ambiente `name` não existir. Se `name` for `null`, todas as variáveis de ambiente serão retornadas como um array associativo.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `name` agora pode ser nulo.
7.1.0	O `name` agora pode ser omitido para recuperar um array associativo de todas as variáveis de ambiente.
7.0.9	O parâmetro `local_only` foi adicionado.

Examples

Exemplo #1 Exemplo de getenv()

```php
<?php
// Exemplo de uso de getenv()
$ip = getenv('REMOTE_ADDR');

// Ou simplesmente usa uma variável super global ($_SERVER ou $_ENV)
$ip = $_SERVER['REMOTE_ADDR'];

// Obtenha com segurança o valor de uma variável de ambiente, ignorando se
// ela foi definida ou não por uma SAPI ou se foi alterada com putenv
$ip = getenv('REMOTE_ADDR', true) ?: getenv('REMOTE_ADDR')
?>
```

Notas

> **Aviso**
>
> Se o PHP estiver sendo executado em uma SAPI como o Fast CGI, esta função sempre retornará o valor de uma variável de ambiente definida pela SAPI, mesmo que putenv() tenha sido usada para definir uma variável de ambiente local de mesmo nome. Use o parâmetro `local_only` para retornar o valor das variáveis de ambiente definidas localmente.

Veja também

- putenv() - Define o valor de uma variável de ambiente
- apache_getenv() - Retorna uma variável subprocess_env do Apache
- Super Globais

getlastmod

getlastmod

(PHP 4, PHP 5, PHP 7, PHP 8)

getlastmod — Obtém o horário da última modificação na página

Descrição

function **getlastmod**(): int|false

Obtém o horário da última modificação do script principal de execução.

Se o objetivo for obter o horário da última modificação de um arquivo diferente, considere usar filemtime().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tempo da última modificação na página atual. O valor é retornado como um timestamp Unix, útil para usar com a função date(). Retorna `false` em caso de erro.

Examples

Exemplo #1 Exemplo de **getlastmod()**

```php
<?php
// mostra por exemplo 'Última Modificação: March 04 1998 20:43:59.'
echo "Última Modificação: " . date ("F d Y H:i:s.", getlastmod());
?>
```

Veja também

- date() - Formata um timestamp Unix
- getmyuid() - Obtém o UID do proprietário do script PHP
- getmygid() - Obtém o GID do proprietário do script PHP
- get_current_user() - Obtém o nome do proprietário do script PHP atual
- getmyinode() - Obtém o inode do script atual
- getmypid() - Obtém o ID do processo PHP
- filemtime() - Obtém o horário de modificação do arquivo

getmygid

getmygid

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

getmygid — Obtém o GID do proprietário do script PHP

Descrição

function **getmygid**(): int|false

Obtém o ID do grupo do script atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o ID do grupo do script atual, ou `false` em caso de erro.

Veja também

- getmyuid() - Obtém o UID do proprietário do script PHP
- getmypid() - Obtém o ID do processo PHP
- get_current_user() - Obtém o nome do proprietário do script PHP atual
- getmyinode() - Obtém o inode do script atual
- getlastmod() - Obtém o horário da última modificação na página

getmyinode

getmyinode

(PHP 4, PHP 5, PHP 7, PHP 8)

getmyinode — Obtém o inode do script atual

Descrição

function **getmyinode**(): int|false

Obtém o inode do script atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o inode do script atual como um número inteiro ou `false` em caso de erro.

Veja também

- getmygid() - Obtém o GID do proprietário do script PHP
- getmyuid() - Obtém o UID do proprietário do script PHP
- getmypid() - Obtém o ID do processo PHP
- get_current_user() - Obtém o nome do proprietário do script PHP atual
- getlastmod() - Obtém o horário da última modificação na página

getmypid

getmypid

(PHP 4, PHP 5, PHP 7, PHP 8)

getmypid — Obtém o ID do processo PHP

Descrição

function **getmypid**(): int|false

Obtém o ID do processo atual do PHP.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o ID do processo atual do PHP, ou `false` em caso de erro.

Notas

> **Aviso**
>
> IDs de processos não são únicos, portanto são uma fonte de entropia fraca. Recomenda-se não confiar em PIDs em contextos que dependam de segurança.

Veja também

- getmygid() - Obtém o GID do proprietário do script PHP
- getmyuid() - Obtém o UID do proprietário do script PHP
- get_current_user() - Obtém o nome do proprietário do script PHP atual
- getmyinode() - Obtém o inode do script atual
- getlastmod() - Obtém o horário da última modificação na página

getmyuid

getmyuid

(PHP 4, PHP 5, PHP 7, PHP 8)

getmyuid — Obtém o UID do proprietário do script PHP

Descrição

function **getmyuid**(): int|false

Obtém o ID do usuário do script atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o ID do usuário do script atual, ou `false` em caso de erro.

Veja também

- getmygid() - Obtém o GID do proprietário do script PHP
- getmypid() - Obtém o ID do processo PHP
- get_current_user() - Obtém o nome do proprietário do script PHP atual
- getmyinode() - Obtém o inode do script atual
- getlastmod() - Obtém o horário da última modificação na página

getopt

getopt

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

getopt — Obtêm opções da linha de comando

Descrição

function **getopt**(string `$short_options`, array `$long_options = []`, int `&$rest_index = null`): array|false

Extrai as opções passadas para o script.

Parâmetros

`short_options`

> Cada caracter nessa string será usada como uma opção e comparada contra as opções passadas para o script prefixadas com um único hífen (-).
>
> Por exemplo, a string `"x"` reconhece a opção `-x`.
>
> Somente a-z, A-Z e 0-9 são permitidos.

`long_options`

> Um array de opções. Cada elemento desse array será usado como uma string de opção e comparada contra as opções passadas para o script com dois hífens (--).
>
> Por exemplo, um elemento longopts `"opt"` reconhece a opção `--opt`.

`rest_index`

> Se o parâmetro `rest_index` estiver presente, então o índice onde a interpretação de argumentos parou será escrita nessa variável.

O parâmetro `short_options` pode conter os elementos a seguir:

- Letras individuais (não aceita valores)
- Letras seguidas de um dois pontos (parâmetro requer valores)
- Letras seguidas de dois dois pontos(valor opcional)

Valores de opções são o primeiro argumento depois da string. Se o valor é exigido então não importa se o valor tem um espaço na frente ou não. Veja nota.

> **Nota** :
>
> Valores opcionais não aceitam `" "` (espaço) como separador.

O array `long_options` pode conter:

- String (parâmetro que não aceita valores)
- String seguida por único dois pontos parâmetro requer valor)
- String seguida por dois dois pontos (valor opcional)

> **Nota** :

> O formado para `short_options` e `long_options` é quase o mesmo, com a única diferença sendo que `long_options` é informado como um array de opções (onde cada elemento é a opção), enquanto `short_options` é informado como uma string (onde cada caracter é um opção).

Valor Retornado

Esta função retorna um array de pares de opções e argumentos, ou **`false`** em caso de falha.

> **Nota** :
>
> A interpretação de opções irá parar na primeira opção desconhecida, e tudo o mais será descartado.

Registro de Alterações

Versão	Descrição
7.1.0	Acrescentando o parâmetro `rest_index`.

Examples

Exemplo #1 Exemplo básico de getopt()

```php
<?php
// Script example.php
$options = getopt("f:hp:");
var_dump($options);
?>
```

```
shell> php example.php -fvalue -h
```

O exemplo acima produzirá:

```
array(2) {
  ["f"]=>
  string(5) "value"
  ["h"]=>
  bool(false)
}
```

Exemplo #2 Exemplo de getopt() com opções longas

```php
<?php
// Script example.php
$shortopts  = "";
$shortopts .= "f:";  // Valor exigido
$shortopts .= "v::"; // Valor opcional
$shortopts .= "abc"; // Essas opções não aceitam valores

$longopts  = array(
    "required:",     // Valor exigido
    "optional::",    // Valor opcional
    "option",        // Sem valor
    "opt",           // Sem valor
);
$options = getopt($shortopts, $longopts);
var_dump($options);
?>
```

```
shell> php example.php -f "value for f" -v -a --required value --optional="optional value" --option
```

O exemplo acima produzirá:

```
array(6) {
  ["f"]=>
  string(11) "value for f"
  ["v"]=>
  bool(false)
  ["a"]=>
  bool(false)
  ["required"]=>
  string(5) "value"
  ["optional"]=>
  string(14) "optional value"
  ["option"]=>
  bool(false)
}
```

Exemplo #3 Exemplo de getopt(): passando múltiplas opções como uma só

```php
<?php
// Script example.php
$options = getopt("abc");
var_dump($options);
?>
```

```
shell> php example.php -aaac
```

O exemplo acima produzirá:

```
array(2) {
  ["a"]=>
  array(3) {
    [0]=>
    bool(false)
    [1]=>
    bool(false)
    [2]=>
    bool(false)
  }
  ["c"]=>
  bool(false)
}
```

Exemplo #4 Exemplo de getopt() com `rest_index`

```php
<?php
// Script example.php
$rest_index = null;
$opts = getopt('a:b:', [], $rest_index);
$pos_args = array_slice($argv, $rest_index);
var_dump($pos_args);
```

```
shell> php example.php -a 1 -b 2 -- test
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  string(4) "test"
}
```

Veja também

- $argv

getrusage

getrusage

(PHP 4, PHP 5, PHP 7, PHP 8)

getrusage — Obtém os usos atuais de recursos

Descrição

function **getrusage**(int `$mode` = 0): array|false

Esta é uma interface para **getrusage(2)**. Ele obtém dados retornados a partir da chamada de sistema.

Parâmetros

`mode`

Se `mode` for 1, getrusage será chamado com **`RUSAGE_CHILDREN`**.

Valor Retornado

Retorna um array associativo contendo os dados retornados da chamada de sistema. Todas as entradas são acessíveis usando seus nomes de campo documentados. Retorna **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de getrusage()

```php
<?php
$dat = getrusage();
echo $dat["ru_oublock"];       // número de operações de saída em bloco
echo $dat["ru_inblock"];       // número de operações de entrada em bloco
echo $dat["ru_msgsnd"];        // número de mensagens IPC enviadas
echo $dat["ru_msgrcv"];        // número de mensagens IPC recebidas
echo $dat["ru_maxrss"];        // tamanho máximo de conjunto residente
echo $dat["ru_ixrss"];         // tamanho integral da memória compartilhada
echo $dat["ru_idrss"];         // tamanho integral de dados não compartilhados
echo $dat["ru_minflt"];        // número de recuperações de página (falhas de página soft)
echo $dat["ru_majflt"];        // número de falhas de página (falhas de página graves)
echo $dat["ru_nsignals"];      // número de sinais recebidos
echo $dat["ru_nvcsw"];         // número de mudanças voluntárias de contexto
echo $dat["ru_nivcsw"];        // número de mudanças de contexto involuntárias
echo $dat["ru_nswap"];         // número de trocas de páginas
echo $dat["ru_utime.tv_usec"]; // tempo usado pelo usuário (microssegundos)
echo $dat["ru_utime.tv_sec"];  // tempo usado pelo usuário (segundos)
echo $dat["ru_stime.tv_usec"]; // tempo do sistema usado (microssegundos)
echo $dat["ru_stime.tv_sec"];  // tempo do sistema usado (segundos)
?>
```

Notas

> **Nota** :
>
> No Windows, **getrusage()** irá retornar somente os seguintes membros:
>
> - `"ru_stime.tv_sec"`

- "ru_stime.tv_usec"
- "ru_utime.tv_sec"
- "ru_utime.tv_usec"
- "ru_majflt" (somente se mode for **RUSAGE_SELF**)
- "ru_maxrss" (somente se mode for **RUSAGE_SELF**)

Se **getrusage()** for chamada com mode definido como 1 (**RUSAGE_CHILDREN**), então o uso de recursos para threads será coletado (ou seja, que internamente a função é chamada com **RUSAGE_THREAD**).

> **Nota** :
>
> No BeOS 2000, apenas os seguintes membros são retornados:
>
> - "ru_stime.tv_sec"
> - "ru_stime.tv_usec"
> - "ru_utime.tv_sec"
> - "ru_utime.tv_usec"

Veja também

- Página do manual do sistema operacional sobre o comando **getrusage(2)**

ini_alter

ini_alter

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_alter — Sinônimo de ini_set()

Descrição

Esta função é um apelido para: ini_set().

ini_get

ini_get

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_get — Obtém o valor de uma opção de configuração

Descrição

function **ini_get**(string $option): string|false

Retorna o valor da opção de configuração em caso de sucesso.

Parâmetros

option

O nome da opção de configuração.

Valor Retornado

Retorna o valor da opção de configuração como uma string em caso de sucesso ou uma string vazia para valores null. Retorna false se a opção de configuração não existir.

Examples

Exemplo #1 Alguns exemplos de ini_get()

```
<?php

/*
Considere o php.ini com as seguintes configurações:
```

```
display_errors = On
opcache.enable_cli = Off
post_max_size = 8M
*/

echo 'display_errors = ' . ini_get('display_errors') . "\n";
echo 'opcache.enable_cli = ' . (int) ini_get('opcache.enable_cli') . "\n";
echo 'post_max_size = ' . ini_get('post_max_size') . "\n";
echo 'post_max_size + 1 = ' . (rtrim(ini_get('post_max_size'), 'KMG') + 1) . "\n";
echo 'post_max_size in bytes = ' . ini_parse_quantity(ini_get('post_max_size'));

?>
```

O exemplo acima produzirá algo semelhante a:

```
display_errors = 1
opcache.enable_cli = 0
post_max_size = 8M
post_max_size+1 = 9
post_max_size em bytes = 8388608
```

Notas

> **Nota** : **Ao consultar valores booleanos**
>
> Um valor booleano ini `off` será retornado como uma string vazia ou "0", enquanto um valor booleano ini `on` será retornado como "1". A função também pode retornar a string literal do valor INI.

> **Nota** : **Ao consultar valores de tamanho de memória**
>
> Muitos valores de tamanho de memória ini, como upload_max_filesize, são armazenados no arquivo *php.ini* em notação abreviada. **ini_get()** retornará a string exata armazenada no arquivo *php.ini* e *NÃO* o seu equivalente int. Tentar funções aritméticas normais com esses valores não trará os resultados esperados. A função ini_parse_quantity() pode ser usada para converter a notação abreviada em bytes.

> **Nota** :
>
> **ini_get()** não consegue ler opções ini de "array" como, por exemplo, `pdo.dsn.*`, e retorna `false` neste caso.

Veja também

- get_cfg_var() - Obtém o valor de uma opção de configuração do PHP
- ini_get_all() - Obtém todas as opções de configuração
- ini_parse_quantity() - Obtém o tamanho interpretado da sintaxe abreviada do ini
- ini_restore() - Restaura o valor de uma opção de configuração
- ini_set() - Define o valor de uma opção de configuração

ini_get_all

ini_get_all

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ini_get_all — Obtém todas as opções de configuração

Descrição

function **ini_get_all**(?string `$extension` = `null`, bool `$details` = `true`): array|false

Retorna todas as opções de configuração cadastradas.

Parâmetros

`extension`

Um nome de extensão opcional. Se diferente de **`null`** ou igual à string `core`, a função retorna apenas opções específicas para aquela extensão.

`details`

Recupera configurações detalhadas ou apenas o valor atual de cada configuração. O padrão é **`true`** (recuperar detalhes).

Valor Retornado

Retorna um array associativo com o nome da diretiva como chave do array. Retorna **`false`** e gera um erro de nível **`E_WARNING`** se a extensão informada em `extension` não existir.

Quando `details` for **`true`** (padrão) o array conterá `global_value` (definido no *php.ini*), `local_value` (talvez definido com ini_set() ou *.htaccess*) e `access` (o nível de acesso).

Quando `details` for **`false`** o valor será o atual da opção.

Consulte a seção do manual para informação sobre o que significam os níveis de acesso.

> **Nota** :
>
> É possível que uma diretiva tenha múltiplos níveis de acesso, e é por isso que `access` mostra os valores apropriados da máscara de bits.

Examples

Exemplo #1 Exemplos de ini_get_all()

```php
<?php
print_r(ini_get_all("pcre"));
print_r(ini_get_all());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [pcre.backtrack_limit] => Array
        (
            [global_value] => 100000
            [local_value] => 100000
            [access] => 7
        )

    [pcre.recursion_limit] => Array
        (
            [global_value] => 100000
            [local_value] => 100000
            [access] => 7
        )

)
Array
(
    [allow_call_time_pass_reference] => Array
        (
            [global_value] => 0
            [local_value] => 0
            [access] => 6
        )
```

```
    [allow_url_fopen] => Array
        (
            [global_value] => 1
            [local_value] => 1
            [access] => 4
        )

    ...

)
```

Exemplo #2 Desabilitando `details`

```php
<?php
print_r(ini_get_all("pcre", false)); // Adicionado no PHP 5.3.0
print_r(ini_get_all(null, false)); // Adicionado no PHP 5.3.0
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [pcre.backtrack_limit] => 100000
    [pcre.recursion_limit] => 100000
)
Array
(
    [allow_call_time_pass_reference] => 0
    [allow_url_fopen] => 1
    ...
)
```

Notas

> **Nota** :
>
> **ini_get_all()** ignora opções ini de "array" como, por exemplo, `pdo.dsn.*`.

Veja também

- Como mudar as configurações
- ini_get() - Obtém o valor de uma opção de configuração
- ini_restore() - Restaura o valor de uma opção de configuração
- ini_set() - Define o valor de uma opção de configuração
- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- phpinfo() - Mostra informações sobre a configuração do PHP
- ReflectionExtension::getINIEntries() - Obtém entradas de extensão ini

ini_parse_quantity

ini_parse_quantity

(PHP 8 >= 8.2.0)

ini_parse_quantity — Obtém o tamanho interpretado da sintaxe abreviada do ini

Descrição

function **ini_parse_quantity**(string `$shorthand`): int

Retorna o tamanho interpretado em bytes em caso de sucesso a partir de uma notação abreviada ini.

Parâmetros

`shorthand`

> Notação abreviada ini a ser interpretada, precisa ser um número seguido por um multiplicador opcional. Os seguintes multiplicadores são suportados: `k`/`K` (`1024`), `m`/`M` (`1048576`), `g`/`G` (`1073741824`). O número pode ser um decimal, hexadecimal (prefixado com `0x` ou `0X`), octal (prefixado com `0o`, `0O` ou `0`) ou binário (prefixado com `0b` ou `0B`)

Valor Retornado

Retorna o tamanho interpretado em bytes como um número int.

Erros/Exceções

Se o valor não puder ser interpretado, ou se um multiplicador inválido for utilizado, um `E_WARNING` é emitido.

Examples

Exemplo #1 Alguns exemplos de ini_parse_quantity()

```php
<?php

var_dump(ini_parse_quantity('1024'));
var_dump(ini_parse_quantity('1024M'));
var_dump(ini_parse_quantity('512K'));
var_dump(ini_parse_quantity('0xFFk'));
var_dump(ini_parse_quantity('0b1010k'));
var_dump(ini_parse_quantity('0o1024'));
var_dump(ini_parse_quantity('01024'));
var_dump(ini_parse_quantity('Foobar'));
var_dump(ini_parse_quantity('10F'));

?>
```

O exemplo acima produzirá:

```
int(1024)
int(1073741824)
int(524288)
int(261120)
int(10240)
int(532)
int(532)

Warning: Invalid quantity "Foobar": no valid leading digits, interpreting as "0" for backwards compatibility
int(0)

Warning: Invalid quantity "10F": unknown multiplier "F", interpreting as "10" for backwards compatibility
int(10)
```

Veja também

- ini_get() - Obtém o valor de uma opção de configuração

ini_restore

ini_restore

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_restore — Restaura o valor de uma opção de configuração

Descrição

function **ini_restore**(string `$option`): void

Restaura uma determinada opção de configuração ao seu valor original.

Parâmetros

`option`

O nome da opção de configuração.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ini_restore()

```php
<?php
$setting = 'html_errors';

echo 'Valor atual para \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;

ini_set($setting, ini_get($setting) ? 0 : 1);
echo 'Novo valor para \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;

ini_restore($setting);
echo 'Valor original para \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;
?>
```

O exemplo acima produzirá:

```
Valor atual para 'html_errors': 1
Novo valor para 'html_errors': 0
Valor original para 'html_errors': 1
```

Veja também

- ini_get() - Obtém o valor de uma opção de configuração
- ini_get_all() - Obtém todas as opções de configuração
- ini_set() - Define o valor de uma opção de configuração

ini_set

ini_set

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_set — Define o valor de uma opção de configuração

Descrição

function **ini_set**(string `$option`, string|int|float|bool|null `$value`): string|false

Define o valor da opção de configuração fornecida. A opção de configuração manterá este novo valor durante a execução do script e será restaurada ao final do script.

Parâmetros

`option`

Nem todas as opções disponíveis podem ser alteradas usando **ini_set()**. Há uma lista de todas as opções disponíveis no apêndice.

`value`

O novo valor da opção.

Valor Retornado

Retorna o valor antigo em caso de sucesso, `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.1.0	`value` agora aceita qualquer tipo escalar (incluindo `null`). Anteriormente, apenas valores string eram aceitos.

Examples

Exemplo #1 Configurando uma opção ini

```php
<?php
echo ini_get('display_errors');

if (!ini_get('display_errors')) {
    ini_set('display_errors', '1');
}

echo ini_get('display_errors');
?>
```

Veja também

- get_cfg_var() - Obtém o valor de uma opção de configuração do PHP
- ini_get() - Obtém o valor de uma opção de configuração
- ini_get_all() - Obtém todas as opções de configuração
- ini_restore() - Restaura o valor de uma opção de configuração
- Como alterar as definições de configuração

memory_get_peak_usage

memory_get_peak_usage

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

memory_get_peak_usage — Retorna o pico de memória alocada pelo PHP

Descrição

function **memory_get_peak_usage**(bool `$real_usage = false`): int

Retorna o pico de memória, em bytes, que foi alocado para o script PHP.

Parâmetros

`real_usage`

Defina este parâmetro como `true` para obter o tamanho real da memória alocada do sistema. Se não estiver definido ou se for informado como `false`, apenas a memória usada por `emalloc()` é relatada.

Valor Retornado

Retorna o pico de memória em bytes.

Veja também

- memory_get_usage() - Retorna a quantidade de memória alocada para o PHP
- memory_reset_peak_usage() - Redefine o pico de uso de memória
- memory_limit

memory_get_usage

memory_get_usage

(PHP 4 >= 4.3.2, PHP 5, PHP 7, PHP 8)

memory_get_usage — Retorna a quantidade de memória alocada para o PHP

Descrição

function **memory_get_usage**(bool `$real_usage = false`): int

Retorna a quantidade de memória, em bytes, que está sendo alocada atualmente para o script PHP.

Parâmetros

`real_usage`

Defina isso como `true` para obter a memória total alocada do sistema, incluindo páginas não utilizadas. Se não estiver definido ou informado como `false`, apenas a memória usada é relatada.

> **Nota** :
>
> O PHP não rastreia memória que não é alocada por `emalloc()`

Valor Retornado

Retorna a quantidade de memória em bytes.

Examples

Exemplo #1 Um exemplo de memory_get_usage()

```php
<?php
// Este é apenas um exemplo, os número abaixo serão
// diferentes dependendo do sistema

echo memory_get_usage() . "\n"; // 36640

$a = str_repeat("Hello", 4242);

echo memory_get_usage() . "\n"; // 57960

unset($a);

echo memory_get_usage() . "\n"; // 36744

?>
```

Veja também

- memory_get_peak_usage() - Retorna o pico de memória alocada pelo PHP
- memory_limit

memory_reset_peak_usage

memory_reset_peak_usage

(PHP 8 >= 8.2.0)

memory_reset_peak_usage — Redefine o pico de uso de memória

Descrição

function **memory_reset_peak_usage**(): void

Redefine o pico de uso de memória retornado pela função memory_get_peak_usage().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de memory_reset_peak_usage()

```php
<?php

var_dump(memory_get_peak_usage());

$a = str_repeat("Hello", 424242);
var_dump(memory_get_peak_usage());

unset($a);
memory_reset_peak_usage();

$a = str_repeat("Hello", 2424);
var_dump(memory_get_peak_usage());

?>
```

O exemplo acima produzirá algo semelhante a:

```
int(422440)
int(2508672)
int(399208)
```

Veja também

- memory_get_peak_usage() - Retorna o pico de memória alocada pelo PHP

php_ini_loaded_file

php_ini_loaded_file

(PHP 5 >= 5.2.4, PHP 7, PHP 8)

php_ini_loaded_file — Obtém o caminho para o arquivo php.ini carregado

Descrição

function **php_ini_loaded_file**(): string|false

Verifica se um *php.ini* arquivo está carregado e retorna seu caminho.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O caminho do arquivo *php.ini*, ou `false` se nenhum tiver sido carregado.

Examples

Exemplo #1 Exemplo de php_ini_loaded_file()

```php
<?php
$inipath = php_ini_loaded_file();

if ($inipath) {
    echo 'Arquivo php.ini carregado: ' . $inipath;
} else {
    echo 'Nenhum arquivo php.ini carregado';
```

```
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Arquivo php.ini carregado: /usr/local/php/php.ini
```

Veja também

- php_ini_scanned_files() - Retorna uma lista de arquivos .ini analisados do diretório ini adicional
- phpinfo() - Mostra informações sobre a configuração do PHP
- O arquivo de configuração

php_ini_scanned_files

php_ini_scanned_files

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

php_ini_scanned_files — Retorna uma lista de arquivos .ini analisados do diretório ini adicional

Descrição

function **php_ini_scanned_files**(): string|false

php_ini_scanned_files() retorna uma lista separada por vírgulas de arquivos de configuração analisados após o *php.ini*. Os diretórios pesquisados são definidos por uma opção em tempo de compilação e, opcionalmente, por uma variável de ambiente em tempo de execução: mais informações podem ser encontradas no guia de instalação.

Os arquivos de configuração retornados incluem o caminho completo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma sequência separada por vírgulas de arquivos .ini em caso de sucesso. Cada vírgula é seguida por uma nova linha. Se a diretiva **--with-config-file-scan-dir** do comando configure não tiver sido definida e a variável de ambiente *PHP_INI_SCAN_DIR* também não estiver definida, `false` é retornado. Se foi definida e o diretório estiver vazio, uma string vazia será retornada. Se um arquivo estiver irreconhecível, o arquivo ainda aparecerá na string retornada, mas também ocorrerá um erro de PHP. Este erro do PHP será visto tanto em tempo de compilação quanto durante o uso de **php_ini_scanned_files()**.

Examples

Exemplo #1 Um exemplo simples para listar os arquivos ini retornados

```
<?php
if ($filelist = php_ini_scanned_files()) {
    if (strlen($filelist) > 0) {
        $files = explode(',', $filelist);

        foreach ($files as $file) {
            echo "<li>" . trim($file) . "</li>\n";
        }
    }
}
?>
```

Veja também

- ini_set() - Define o valor de uma opção de configuração
- phpinfo() - Mostra informações sobre a configuração do PHP

- php_ini_loaded_file() - Obtém o caminho para o arquivo php.ini carregado

php_sapi_name

php_sapi_name

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

php_sapi_name — Retorna o tipo de interface entre o servidor web e o PHP

Descrição

function **php_sapi_name**(): string|false

Retorna uma string em minúsculas que descreve o tipo de interface (API do Servidor, SAPI) que o PHP está usando. Por exemplo, no PHP CLI esta string será "cli" e no Apache poderá ter vários valores diferentes dependendo da SAPI exata sendo usada. Os valores possíveis estão listados abaixo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tipo de interface como uma string em minúsculas ou `false` em caso de falha.

Embora não seja uma lista exaustiva, os valores possíveis incluem `apache`, `apache2handler`, `cgi` (até o PHP 5.3), `cgi-fcgi`, `cli`, `cli-server`, `embed`, `fpm-fcgi`, `litespeed`, `phpdbg`.

Examples

Exemplo #1 Exemplo de php_sapi_name()

Este exemplo procura pela sub-string `cgi` porque a interface também pode ser `cgi-fcgi`.

```
<?php
$sapi_type = php_sapi_name();
if (substr($sapi_type, 0, 3) == 'cgi') {
    echo "Você está usando o PHP CGI.\n";
} else {
    echo "Você não está usando o PHP CGI.\n";
}
?>
```

Notas

> **Nota** : **Uma abordagem alternativa**
>
> A constante `PHP_SAPI` do PHP tem o mesmo valor retornado pela função **php_sapi_name()**.

> **Dica**
>
> #### Uma potencial armadilha
>
> A SAPI definida pode não ser óbvia porque, por exemplo, ao invés de `apache` ela pode estar definida como `apache2handler`.

Veja também

- PHP_SAPI

php_uname

php_uname

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

php_uname — Retorna informações sobre o sistema operacional em que o PHP está sendo executado

Descrição

function **php_uname**(string `$mode` = "a"): string

php_uname() retorna uma descrição do sistema operacional em que o PHP está sendo executado. Esta é a mesma string que você vê no topo da saída de phpinfo(). Para o nome apenas do sistema operacional, considere usar a constante **PHP_OS**, mas tenha em mente que esta constante conterá o sistema operacional no qual o PHP foi *construído*.

Em algumas plataformas UNIX mais antigas, pode não ser possível determinar as informações atuais do sistema operacional e, nesse caso, a função reverterá para a exibição do sistema operacional no qual o PHP foi construído. Isso só acontecerá se a chamada da biblioteca uname() não existir ou não funcionar.

Parâmetros

`mode`

`mode` é um único caractere que define quais informações são retornadas:

- `'a'`: É o padrão. Retorna a mesma informação que os modos individuais `'s'`, `'n'`, `'r'`, `'v'`, `'m'` separados por espaços.
- `'s'`: Nome do sistema operacional. Ex: `FreeBSD`.
- `'n'`: Nome do servidor. Ex.: `localhost.example.com`.
- `'r'`: Nome da distribuição. Ex.: `5.1.2-RELEASE`.
- `'v'`: Informação de versão. Varia muito entre sistemas operacionais.
- `'m'`: Tipo de máquina. Ex.: `i386`.

Valor Retornado

Retorna a descrição, como uma string.

Registro de Alterações

Versão	Descrição
8.4.0	Lança um ValueError quando um `mode` inválido for especificado.

Examples

Exemplo #1 Alguns exemplos de php_uname()

```php
<?php
echo php_uname();
echo PHP_OS;

/* Alguns resultados possíveis:
Linux localhost 2.4.21-0.13mdk #1 Fri Mar 14 15:08:06 EST 2003 i686
Linux

FreeBSD localhost 3.2-RELEASE #15: Mon Dec 17 08:46:02 GMT 2001
FreeBSD

Windows NT XN1 5.1 build 2600
WINNT
*/

if (strtoupper(substr(PHP_OS, 0, 3)) === 'WIN') {
    echo 'Este é um servidor que usa Windows!';
} else {
    echo 'Este é um servidor que não usa Windows!';
}

?>
```

Também existem algumas Constantes Predefinidas do PHP relacionadas que podem ser úteis, por exemplo:

Exemplo #2 Exemplos de algumas constantes relacionadas a sistema operacional

```php
<?php
// *nix
echo DIRECTORY_SEPARATOR; // /
echo PHP_SHLIB_SUFFIX;    // so
echo PATH_SEPARATOR;      // :

// Win*
echo DIRECTORY_SEPARATOR; // \
echo PHP_SHLIB_SUFFIX;    // dll
echo PATH_SEPARATOR;      // ;
?>
```

Veja também

- phpversion() - Obtém a versão atual do PHP
- php_sapi_name() - Retorna o tipo de interface entre o servidor web e o PHP
- phpinfo() - Mostra informações sobre a configuração do PHP

phpcredits

phpcredits

(PHP 4, PHP 5, PHP 7, PHP 8)

phpcredits — Mostra os créditos para o PHP

Descrição

function **phpcredits**(int `$flags` = `CREDITS_ALL`): true

Esta função imprime os créditos listando os desenvolvedores do PHP, dos módulos, etc. Ela gera os códigos HTML apropriados para inserir as informações em uma página.

Parâmetros

`flags`

Para gerar uma página de créditos personalizada, pode-se usar o parâmetro `flags`.

Opções predefinidas de phpcredits()

nome	descrição
CREDITS_ALL	Todos os créditos, equivalente a usar: `CREDITS_DOCS` + `CREDITS_GENERAL` + `CREDITS_GROUP` + `CREDITS_MODULES` + `CREDITS_FULLPAGE`. Gera uma página HTML completa e independente com as etiquetas HTML apropriadas.
CREDITS_DOCS	Os créditos para a equipe de documentação
CREDITS_FULLPAGE	Geralmente usada em combinação com outras opções. Indica que uma página HTML independente completa precisa ser mostrada incluindo as informações indicadas pelas outras opções.
CREDITS_GENERAL	Créditos gerais: Projeto e conceito de linguagem, autores do PHP e módulo SAPI.
CREDITS_GROUP	Uma lista dos desenvolvedores do núcleo
CREDITS_MODULES	Uma lista dos módulos de extensão para PHP e seus autores.
CREDITS_SAPI	Uma lista dos módulos de API de servidor para PHP e seus autores.

Valor Retornado

Sempre retorna `true`.

Examples

Exemplo #1 Mostra os créditos gerais

```php
<?php
phpcredits(CREDITS_GENERAL);
?>
```

Exemplo #2 Mostra os desenvolvedores do núcleo e o grupo de documentação

```php
<?php
phpcredits(CREDITS_GROUP | CREDITS_DOCS | CREDITS_FULLPAGE);
?>
```

Exemplo #3 Mostra todos os créditos

```php
<html>
 <head>
  <title>Minha página de créditos</title>
 </head>
 <body>
<?php
// algum código
phpcredits(CREDITS_ALL - CREDITS_FULLPAGE);
// mais código
?>
 </body>
</html>
```

Notas

> **Nota** :
>
> **phpcredits()** mostra texto puro ao invés de HTML quando o modo CLI está sendo usado.

Veja também

- phpversion() - Obtém a versão atual do PHP
- phpinfo() - Mostra informações sobre a configuração do PHP

phpinfo

phpinfo

(PHP 4, PHP 5, PHP 7, PHP 8)

phpinfo — Mostra informações sobre a configuração do PHP

Descrição

function **phpinfo**(int `$flags = INFO_ALL`): true

Mostra uma grande quantidade de informações sobre o estado atual do PHP. Isso inclui informações sobre opções e extensões de compilação do PHP, a versão do PHP, informações do servidor e ambiente (se compilado como um módulo), o ambiente do PHP, informações da versão do sistema operacional, caminhos, valores principais e locais das opções de configuração, cabeçalhos HTTP e a Licença do PHP.

Como cada sistema é configurado de maneira diferente, **phpinfo()** é comumente usado para verificar definições de configuração e disponibilidade de variáveis predefinidas em um determinado sistema.

phpinfo() também é uma ferramenta de depuração valiosa, pois contém todos os dados EGPCS (Environment - ambiente, GET, POST, Cookie, Servidor).

Parâmetros

`flags`

A saída pode ser personalizada passando um ou mais dos seguintes valores de *constantes* bit a bit somados no parâmetro opcional `flags`. Também é possível combinar as respectivas constantes ou valores bit a bit com o operador binário OR.

Opções de phpinfo()

Nome (constante)	Valor	Descrição
INFO_GENERAL	1	A linha de configuração, localização do *php.ini*, data de construção, servidor Web, sistema e muito mais.
INFO_CREDITS	2	Créditos do PHP. Veja também phpcredits().

Nome (constante)	Valor	Descrição
INFO_CONFIGURATION	4	Valores atuais das diretivas do PHP locais e principais. Veja também ini_get().
INFO_MODULES	8	Módulos carregados e suas respectivas configurações. Veja também get_loaded_extensions().
INFO_ENVIRONMENT	16	Informações sobre variáveis de ambiente que também estão disponíveis em *$_ENV*.
INFO_VARIABLES	32	Mostra todas as variáveis predefinidas de EGPCS (Environment - ambiente, GET, POST, Cookie, Servidor).
INFO_LICENSE	64	Informações de Licença do PHP. Veja também a seção de » Perguntas e Respostas sobre Licença.
INFO_ALL	-1	Mostra todas as informações acima.

Valor Retornado

Sempre retorna `true`.

Examples

Exemplo #1 Exemplo de phpinfo()

```php
<?php

// Mostra todas as informações, equivalente ao padrão INFO_ALL
phpinfo();

// Mostra apenas informações sobre módulos.
// phpinfo(8) gera resultados idênticos.
phpinfo(INFO_MODULES);

?>
```

Notas

> **Nota** :
>
> Nas versões do PHP anteriores à 5.5, partes das informações exibidas eram desativadas quando a configuração expose_php estava definida como `off`. Isso incluía os logotipos PHP e Zend e os créditos.

> **Nota** :
>
> **phpinfo()** gera texto puro ao invés de HTML ao usar o modo CLI.

Veja também

- phpversion() - Obtém a versão atual do PHP
- phpcredits() - Mostra os créditos para o PHP
- ini_get() - Obtém o valor de uma opção de configuração
- ini_set() - Define o valor de uma opção de configuração
- get_loaded_extensions() - Retorna um array com os nomes de todos os módulos compilados e carregados
- Variáveis Predefinidas

phpversion

phpversion

(PHP 4, PHP 5, PHP 7, PHP 8)

phpversion — Obtém a versão atual do PHP

Descrição

function **phpversion**(?string `$extension = null`): string|false

Retorna uma string contendo a versão do interpretador ou extensão do PHP atualmente em execução.

Parâmetros

`extension`

Um nome de extensão opcional.

Valor Retornado

Retorna a versão atual do PHP como uma string. Se um argumento string for fornecido para o parâmetro `extension`, **phpversion()** retornará a versão dessa extensão, ou `false` se não houver informações de versão associadas ou se a extensão não estiver habilitada.

Registro de Alterações

Versão	Descrição
8.0.0	`extension` agora pode ser nulo.

Examples

Exemplo #1 Exemplo de phpversion()

```php
<?php

// mostra por exemplo 'Versão atual do PHP: 8.3.12'
echo 'Versão atual do PHP: ' . phpversion();

// mostra por exemplo '1.22.3' ou nada se a extensão não estiver habilitada
echo phpversion('zip');

?>
```

Exemplo #2 Exemplo e uso de `PHP_VERSION_ID`

```php
<?php

/**
 * PHP_VERSION_ID é definida como um número, onde quanto mais alto ele
 * for, mais recente é a versão do PHP usada. É definida de acordo com a expressão
 * abaixo:
 *
 * $id_versao = $versao_principal * 10000 + $versao_secundaria * 100 + $versao_distribuicao;
 *
 * Com PHP_VERSION_ID, pode-se procurar por recursos que esta versão do PHP
 * possa ter, e não é necessário usar version_compare() toda vez
 * para verificar se a versão atual do PHP pode ou não suportar um recurso.
 *
 * Por exemplo, pode-se definir aqui as constantes PHP_*_VERSION que
 * não estão disponíveis em versões anteriores à 5.2.7
 */

if (PHP_VERSION_ID < 50207) {
    define('PHP_MAJOR_VERSION',   $version[0]);
    define('PHP_MINOR_VERSION',   $version[1]);
    define('PHP_RELEASE_VERSION', $version[2]);

    // e assim por diante, ...
}

?>
```

Notas

> **Nota** :
>
> Esta informação também está disponível na constante predefinida `PHP_VERSION`. Mais informações sobre versionamento estão disponíveis usando as constantes `PHP_*_VERSION`.

> **Nota** :

> Algumas extensões podem definir seu próprio número de versão. No entanto, a maioria das extensões incluídas usará a versão do PHP como número de versão.

Veja também

- Constantes PHP_VERSION
- version_compare() - Compara duas strings de número de versão no "padrão PHP"
- phpinfo() - Mostra informações sobre a configuração do PHP
- phpcredits() - Mostra os créditos para o PHP
- zend_version() - Obtém a versão do motor Zend atual

putenv

putenv

(PHP 4, PHP 5, PHP 7, PHP 8)

putenv — Define o valor de uma variável de ambiente

Descrição

function **putenv**(string `$assignment`): bool

Adiciona `assignment` no ambiente do servidor. A variável de ambiente irá existir somente durante a requisição atual. Ao final da requisição o ambiente é restaurado ao seu estado original.

Parâmetros

`assignment`

A definição, como `"FOO=BAR"`

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Definindo uma variável de ambiente

```
<?php
putenv("UNIQID=$uniqid");
?>
```

Veja também

- getenv() - Obtém o valor de uma única ou de todas as variáveis de ambiente
- apache_setenv() - Define uma variável subprocess_env do Apache

restore_include_path

restore_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7)

restore_include_path — Restaura o valor da opção de configuração include_path

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **restore_include_path**(): void

Restaura a opção de configuração include_path de volta ao seu valor original master como definido no *php.ini*

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Examples

Exemplo #1 Exemplo de restore_include_path()

```php
<?php

echo get_include_path();  // .:/usr/local/lib/php

set_include_path('/inc');

echo get_include_path();  // /inc

restore_include_path();

// Ou usando ini_restore()
ini_restore('include_path');

echo get_include_path();  // .:/usr/local/lib/php

?>
```

Veja também

- ini_restore() - Restaura o valor de uma opção de configuração
- get_include_path() - Obtém a opção de configuração include_path atual
- set_include_path() - Define a opção de configuração include_path
- include - include

set_include_path

set_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

set_include_path — Define a opção de configuração include_path

Descrição

function **set_include_path**(string `$include_path`): string|false

Define a opção de configuração include_path para toda a duração do script.

Parâmetros

`include_path`

O novo valor para o include_path

Valor Retornado

Retorna o include_path antigo em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de set_include_path()

```php
<?php
set_include_path('/usr/lib/pear');

// Ou usando ini_set()
ini_set('include_path', '/usr/lib/pear');
?>
```

Exemplo #2 Adicionando ao caminho de inclusão

Fazendo uso da constante `PATH_SEPARATOR`, é possível estender o caminho de inclusão independentemente do sistema operacional.

Neste exemplo foi adicionado o caminho */usr/lib/pear* no final do `include_path` existente.

```php
<?php
$path = '/usr/lib/pear';
set_include_path(get_include_path() . PATH_SEPARATOR . $path);
?>
```

Veja também

- ini_set() - Define o valor de uma opção de configuração
- get_include_path() - Obtém a opção de configuração include_path atual
- restore_include_path() - Restaura o valor da opção de configuração include_path
- include - include

set_time_limit

set_time_limit

(PHP 4, PHP 5, PHP 7, PHP 8)

set_time_limit — Limita o máximo tempo de execução

Descrição

function **set_time_limit**(int `$seconds`): bool

Define um tempo em segundos em que um script poderá ser executado. Se este tempo for alcançado, o script retorna um erro fatal. O limite padrão é de 30 segundos ou, se exitente, o valor de `max_execution_time` definido no *php.ini*.

Quando chamada, **set_time_limit()** reinicia o contador do limite a partir do zero. Em outras palavras, se o limite for o padrão de 30 segundos, e já houver 25 segundos decorridos desde o início do script, quando uma chamada chamada tipo `set_time_limit(20)` é feita, o script será executado por um total de 45 segundos antes de expirar.

Parâmetros

`seconds`

O tempo máximo de execução, em segundos. Se definido para zero, nenhum limite será imposto.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Notas

> **Nota** :
>
> A função **set_time_limit()** e a diretiva de configuração max_execution_time afetam apenas o tempo de execução do próprio script. Qualquer tempo gasto em atividades que acontecem fora da execução do script, como chamadas de sistema usando system(), operações de fluxo, consultas de banco de dados, etc., não é incluído na determinação do tempo máximo de execução do script. Isso não acontece no Windows, onde o tempo medido é real.

Veja também

- max_execution_time
- max_input_time

sys_get_temp_dir

sys_get_temp_dir

(PHP 5 >= 5.2.1, PHP 7, PHP 8)

sys_get_temp_dir — Retorna o caminho do diretório usado para arquivos temporários

Descrição

function **sys_get_temp_dir**(): string

Retorna o caminho do diretório onde o PHP armazena arquivos temporários por padrão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho do diretório de arquivos temporários.

Examples

Exemplo #1 Exemplo de sys_get_temp_dir()

```php
<?php
// Cria um arquivo no diretório de
// arquivos temporários usando sys_get_temp_dir()
$temp_file = tempnam(sys_get_temp_dir(), 'Tux');

echo $temp_file;
?>
```

O exemplo acima produzirá algo semelhante a:

```
C:\Windows\Temp\TuxA318.tmp
```

Veja também

- tmpfile() - Cria um arquivo temporário
- tempnam() - Cria arquivo com nome único

version_compare

version_compare

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

version_compare — Compara duas strings de número de versão no "padrão PHP"

Descrição

function **version_compare**(string `$version1`, string `$version2`, ?string `$operator` = `null`): int|bool

version_compare() compara duas strings de número de versão no "padrão PHP".

A função primeiro substitui _, - e + por um ponto . nas strings de versão e também insere pontos . antes e depois de qualquer não-número para que, por exemplo, '4.3.2RC1' se torne '4.3.2.RC.1'. Em seguida, compara as partes começando da esquerda para a direita. Se uma parte contiver strings especiais de versão, elas serão tratadas na seguinte ordem: `qualquer string não encontrada nesta lista` < `dev` < `alpha` = `a` < `beta` = `b` < `RC` = `rc` < `#` < `pl` = `p`. Desta forma, não apenas versões com níveis diferentes como '4.1' e '4.1.2' podem ser comparadas, mas também qualquer versão específica do PHP contendo estado de desenvolvimento.

Parâmetros

`version1`

Primeiro número de versão.

`version2`

Segundo número de versão.

`operator`

Um operador opcional. Os operadores possíveis são: `<`, `lt`, `<=`, `le`, `>`, `gt`, `>=`, `ge`, `==`, `=`, `eq`, `!=`, `<>`, `ne` respectivamente.

Este parâmetro diferencia maiúsculas de minúsculas, os valores devem estar em minúsculas.

Valor Retornado

Por padrão, **version_compare()** retorna `-1` se a primeira versão for inferior à segunda, `0` se forem iguais e `1` se a segunda for menor.

Ao usar o argumento opcional `operator`, a função retornará `true` se o relacionamento for aquele especificado pelo operador, `false` caso contrário.

Examples

Os exemplos abaixo utilizam a constante `PHP_VERSION`, pois ela contém o valor da versão do PHP que está executando o código.

Exemplo #1 Exemplos de version_compare()

```php
<?php
if (version_compare(PHP_VERSION, '7.0.0') >= 0) {
    echo 'Estou pelo menos na versão 7.0.0 do PHP, minha versão: ' . PHP_VERSION . "\n";
}

if (version_compare(PHP_VERSION, '5.3.0') >= 0) {
    echo 'Estou pelo menos na versão 5.3.0 do PHP, minha versão: ' . PHP_VERSION . "\n";
}

if (version_compare(PHP_VERSION, '5.0.0', '>=')) {
    echo 'Estou pelo menos na versão 5.0.0 do PHP, minha versão: ' . PHP_VERSION . "\n";
}

if (version_compare(PHP_VERSION, '5.0.0', '<')) {
    echo 'Ainda estou na versão 4 do PHP, minha versão: ' . PHP_VERSION . "\n";
}
?>
```

Notas

> **Nota** :
>
> A constante `PHP_VERSION` contém a versão atual do PHP.

> **Nota** :
>
> Observe que as versões de pré-lançamento, como 5.3.0-dev, são consideradas inferiores às suas versões finais correspondentes (como 5.3.0).

> **Nota** :
>
> Strings especiais de versões como `alpha` e `beta` diferenciam maiúsculas de minúsculas. Strings de versão de fontes arbitrárias que não aderem ao padrão do PHP podem precisar ser convertidas para minúsculas com strtolower() antes de chamar **version_compare()**.

Veja também

- phpversion() - Obtém a versão atual do PHP
- php_uname() - Retorna informações sobre o sistema operacional em que o PHP está sendo executado
- function_exists() - Retorna true se a função informada estiver definida

zend_thread_id

zend_thread_id

(PHP 5, PHP 7, PHP 8)

zend_thread_id — Retorna um identificador exclusivo para o thread atual

Descrição

function **zend_thread_id**(): int

Retorna um identificador exclusivo para o thread atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o ID do thread como um número inteiro.

Examples

Exemplo #1 Exemplo de zend_thread_id()

```php
<?php
$thread_id = zend_thread_id();

echo 'ID do thread atual é: ' . $thread_id;
?>
```

O exemplo acima produzirá algo semelhante a:

```
ID do thread atual é: 7864
```

Notas

> **Nota** :
>
> Esta função só está disponível se o PHP tiver sido construído com suporte ao ZTS (Zend Thread Safety) e com modo de depuração (`--enable-debug`).

zend_version

zend_version

(PHP 4, PHP 5, PHP 7, PHP 8)

zend_version — Obtém a versão do motor Zend atual

Descrição

function **zend_version**(): string

Retorna uma string contendo a versão atual do motor Zend Engine atualmente em execução.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número da versão do motor Zend, como uma string.

Examples

Exemplo #1 Exemplo de **zend_version()**

```php
<?php
echo "Versão do motor Zend: " . zend_version();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Versão do motor Zend: 2.2.0
```

Veja também

- phpinfo() - Mostra informações sobre a configuração do PHP
- phpcredits() - Mostra os créditos para o PHP
- phpversion() - Obtém a versão atual do PHP

Funções Miscelâneas

Introdução

Estas funções foram colocadas aqui porque pareciam não caber em nenhuma outra categoria.

Instalação/Configuração

Índice

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração Miscelâneas

Nome	Padrão	Modificável	Registro de Alterações
ignore_user_abort	"0"	INI_ALL	
highlight.string	"#DD0000"	INI_ALL	
highlight.comment	"#FF8000"	INI_ALL	
highlight.keyword	"#007700"	INI_ALL	
highlight.default	"#0000BB"	INI_ALL	
highlight.html	"#000000"	INI_ALL	
browscap	NULL	INI_SYSTEM	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`ignore_user_abort` bool

> `false` por padrão. Se modificado para `true`, os scripts não serão terminados depois que um cliente tiver abortado sua conexão.
>
> Consulte também ignore_user_abort().

`highlight.bg` string
`highlight.comment` string
`highlight.default` string
`highlight.html` string
`highlight.keyword` string
`highlight.string` string

> Cores para o modo de Destaque de Sintaxe. Qualquer argumento que seja aceito em <font color="??????"> irá funcionar.

`browscap` string

> Nome (ex.: *browscap.ini*) e localização do arquivo de capacidades do navegador. Consulte também get_browser().

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

`CONNECTION_ABORTED` (int)

`CONNECTION_NORMAL` (int)

`CONNECTION_TIMEOUT` (int)

`__COMPILER_HALT_OFFSET__` (int)

Funções Miscelâneas

Índice

connection_aborted

connection_aborted

(PHP 4, PHP 5, PHP 7, PHP 8)

connection_aborted — Verifica se o cliente está desconectado

Descrição

function **connection_aborted**(): int

Verifica se o cliente está desconectado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna 1 se o cliente está desconectado, 0 caso contrário.

Veja também

- connection_status() - Retorna o campo de bits do estado da conexão
- ignore_user_abort() - Define se uma desconexão de cliente deve abortar execução de script
- Tratamento de conexões para um descrição completa sobre tratamento de conexões no PHP.

connection_status

connection_status

(PHP 4, PHP 5, PHP 7, PHP 8)

connection_status — Retorna o campo de bits do estado da conexão

Descrição

function **connection_status**(): int

Obtém o campo de bits do estado da conexão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o campo de bits do estado da conexão, que pode ser confrontado com as constantes `CONNECTION_*` para determinar o estado da conexão.

Veja também

- connection_aborted() - Verifica se o cliente está desconectado
- ignore_user_abort() - Define se uma desconexão de cliente deve abortar execução de script
- Manipulação de Conexões para uma descrição completa de manipulação de conexões no PHP.

constant

constant

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

constant — Retorna o valor de uma constante

Descrição

function **constant**(string `$name`): mixed

Retorna o valor de uma constante indicada pelo parâmetro `name`.

A função **constant()** é útil se você precisa pegar o valor de uma constante, mas não sabe o seu nome. I.e. esta guardada em uma variável ou é retornada por uma função.

Esta função funciona também com constantes de classes e casos de enumeração.

Parâmetros

`name`

O nome da constante.

Valor Retornado

Retorna o valor da constante.

Erros/Exceções

Se a constante não estiver definida, uma exceção Error é lançada. Antes do PHP 8.0.0, um erro de nível `E_WARNING` era gerado nesse caso.

Registro de Alterações

Versão	Descrição
8.0.0	Se a constante não estiver definida, **constant()** agora lança uma exceção Error; anteriormente um `E_WARNING` era gerado e `null` era retornado.

Examples

Exemplo #1 Usando constant() com Constantes

```php
<?php

define("MAXSIZE", 100);

echo MAXSIZE;
echo constant("MAXSIZE"); // mesma coisa que a linha anterior


interface bar {
    const test = 'foobar!';
}

class foo {
    const test = 'foobar!';
}

$const = 'test';

var_dump(constant('bar::'. $const)); // string(7) "foobar!"
var_dump(constant('foo::'. $const)); // string(7) "foobar!"

?>
```

Exemplo #2 Usando constant()com Casos de Enumeração (a partir do PHP 8.1.0)

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

$case = 'Hearts';
```

```php
var_dump(constant('Suit::'. $case)); // enum(Suit::Hearts)

?>
```

Veja também

- define() - Define uma constante com nome
- defined() - Verifica se existe uma constante com o nome informado
- get_defined_constants() - Retorna um array associativo com os nomes de todas as constantes e seus valores
- A seção sobre Constantes

define

define

(PHP 4, PHP 5, PHP 7, PHP 8)

define — Define uma constante com nome

Descrição

function **define**(string `$constant_name`, mixed `$value`, bool `$case_insensitive = false`): bool

Define uma constante com nome em tempo de execução.

Parâmetros

`constant_name`

O nome da constante.

> **Nota** :
>
> É possível definir constante pela função **define()** com nome reservado ou até mesmo com nome inválido, cujo valor pode (somente) ser recuperado com a função constant(). Entretanto, isto não é recomendado.

`value`

O valor da constante.

> **Aviso**
>
> Apesar de ser possível definir constantes do tipo resource, não é recomendado e pode causar comportamento imprevisível.

`case_insensitive`

Se definido para `true`, a constante será definida sem diferenciação de maiúsculas/minúsculas. O comportamento padrão é diferenciar, isto é, `CONSTANT` e `Constant` representam valores diferentes.

> **Aviso**
>
> Definir constantes sem diferenciar maiúsculas/minúsculas foi descontinuado a partir do PHP 7.3.0. A partir do PHP 8.0.0, somente `false` é um valor aceitável para este parâmetro, passar `true` irá gerar um alerta.

> **Nota** :
>
> Constantes insensíveis a maiúsculas/minúsculas são armazenadas em minúsculas.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.1.0	`value` agora pode ser um objeto.
8.0.0	Passar `true` para `case_insensitive` agora emite um `E_WARNING`. Passar `false` ainda é permitido.
7.3.0	`case_insensitive` foi descontinuado e será removido na versão 8.0.0.

Examples

Exemplo #1 Definindo Constantes

```
<?php
define("CONSTANTE", "Olá, mundo.");
echo CONSTANTE; // exibe "Olá, mundo."
echo Constante; // exibe "Constante" e emite um aviso.

define("CUMPRIMENTO", "Tudo bem?", true);
echo CUMPRIMENTO; // exibe "Tudo bem?"
echo Cumprimento; // exibe "Tudo bem?"

// Funciona a partir do PHP 7
define('ANIMAIS', array(
    'cachorro',
    'gato',
    'pássaro'
));
echo ANIMAIS[1]; // exibe "gato"

?>
```

Exemplo #2 Constantes com Nomes Reservados

Este exemplo ilustra a *possibilidade* de definir uma constante com o mesmo nome de uma constante mágica. Como o comportamento resultante obviamente é confuso, não é portanto recomendado fazê-lo na prática.

```
<?php
var_dump(defined('__LINE__'));
var_dump(define('__LINE__', 'test'));
var_dump(constant('__LINE__'));
var_dump(__LINE__);
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
string(4) "test"
int(5)
```

Veja também

- defined() - Verifica se existe uma constante com o nome informado
- constant() - Retorna o valor de uma constante
- A seção sobre Constantes

defined

defined

(PHP 4, PHP 5, PHP 7, PHP 8)

defined — Verifica se existe uma constante com o nome informado

Descrição

function **defined**(string `$constant_name`): bool

Verifica se uma constante com o nome informado em `constant_name` está definida.

A função também funciona com constantes de classe e casos de enumeração.

> **Nota** :
>
> Se o objetivo for verificar se uma variável existe, use a função isset() já que a função **defined()** somente se aplica a constantes. Se o objetivo for verificar se uma função existe, use a função function_exists().

Parâmetros

`constant_name`

O nome da constante.

Valor Retornado

Retorna `true` se a constante nomeada informada por `constant_name` já está definida, `false` caso contrário.

Examples

Exemplo #1 Verificando constantes

```php
<?php

/* Observe o uso de aspas, isto é importante. Este exemplo está verificando
 * se a string 'TEST' é o nome de uma constante chamada TEST */
if (defined('TEST')) {
    echo TEST;
}


interface bar {
    const test = 'foobar!';
}

class foo {
    const test = 'foobar!';
}

var_dump(defined('bar::test')); // bool(true)
var_dump(defined('foo::test')); // bool(true)

?>
```

Exemplo #2 Verificando Casos de Enumeração (a partir do PHP 8.1.0)

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

var_dump(defined('Suit::Hearts')); // bool(true)

?>
```

Veja também

- define() - Define uma constante com nome
- constant() - Retorna o valor de uma constante

- get_defined_constants() - Retorna um array associativo com os nomes de todas as constantes e seus valores
- function_exists() - Retorna true se a função informada estiver definida
- A seção sobre Constantes

die

die

(PHP 4, PHP 5, PHP 7, PHP 8)

die — Sinônimo de exit()

Descrição

Esta função é um apelido para: exit().

eval

eval

(PHP 4, PHP 5, PHP 7, PHP 8)

eval — Avalia uma string como código PHP

Descrição

function **eval**(string `$code`): mixed

Avalia a string fornecida no parâmetro `code` como um código PHP.

O código a ser avaliado herda o escopo de variáveis da linha na qual a chamada a **eval()** ocorrer. Todas as variáveis disponíveis naquela linha estarão disponíveis para leitura e modificação no código avaliado. Entretanto, todas as funções e classes definidas serão definidas no espaço de nomes global. Em outras palavras, o compilador considera o código avaliado como se ele fosse um arquivo incluído separadamente.

> **Cuidado**
>
> A construção de linguagem **eval()** é *bastante perigosa* porque permite execução de código PHP arbitrário. *Seu uso é contudo desencorajado.* Se foi cuidadosamente verificado que não há outra opção a não ser usar esta construção, deve-se prestar atenção especial para *não passar nenhum dado fornecido pelo usuário* no parâmetro sem antes validá-lo.

Parâmetros

`code`

Código PHP válido a ser avaliado.

O código não pode ser envolvido em etiquetas PHP de abertura e de fechamento, isto é, `'echo "Hi!";'` deve ser passado ao invés de `'<?php echo "Hi!"; ?>'`. Ainda é possível sair e re-entrar no modo PGP usando-se os marcadores apropriados, ex.: `'echo "In PHP mode!"; ?>In HTML mode!<?php echo "Back in PHP mode!";'`.

Além disso, o código informado deve ser código PHP válido. Isto inclui o fato de que todas as instruções devem ser adequadamente terminadas com ponto-e-vírgula. `'echo "Hi!"'` por exemplo causará um erro de avaliação, enquanto que `'echo "Hi!";'` funcionará.

Uma instrução `return` irá terminar imediatamente a avaliação do código.

o código será executado no escopo do código que está chamando **eval()**. Todas as variáveis definidas ou alteradas na chamada a **eval()** permanecerão visíveis depois que ela terminar.

Valor Retornado

eval() retorna `null` a menos que `return` seja chamado no código avaliado, neste caso o valor passado a `return` será o retornado. A partir do PHP 7, se houver um erro de avaliação no código, **eval()** lança uma exceção ParseError. Antes do PHP 7, neste caso **eval()** retornava `false` e a execução do código seguinte continuava normalmente. Não é possível capturar um erro de avaliação em **eval()** usando set_error_handler().

Examples

Exemplo #1 Exemplo de eval() - fusão simples de texto

```php
<?php
$string = 'xícara';
$nome = 'café';
$str = 'Isto é uma $string com meu $nome.';
echo $str. "\n";
eval("\$str = \"$str\";");
echo $str. "\n";
?>
```

O exemplo acima produzirá:

```
Isto é uma $string com meu $nome.
Isto é uma xícara com meu café.
```

Notas

> **Nota** :
>
> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

> **Dica**
>
> Assim como qualquer coisa que envia seu resultado diretamente para o navegador, as funções de controle de saída podem ser usadas para capturar a saída desta função e salvá-la em uma string, por exemplo.

> **Nota** :
>
> Em caso de erro fatal no código avaliado, todo o script é terminado.

Veja também

- call_user_func() - Chama a função de retorno informada no primeiro parâmetro

exit

exit

(PHP 4, PHP 5, PHP 7, PHP 8)

exit — Termina o script atual com um código ou uma mensagem de status

Descrição

function **exit**(string|int `$status` = 0): never

Termina a execução do script. Funções de desligamento e destrutores de objeto sempre serão executados mesmo que **exit()** seja chamada. Entretanto, blocos `finally` nunca são executados.

Um código de saída igual a `0` é usado para indicar que o programa foi bem sucedido em suas tarefas. Qualquer outro valor indica que algum tipo de erro ocorreu durante a execução.

exit() é uma função especial porque tem um token dedicado no analisador, e como tal pode ser usada como uma instrução (ou seja, sem parênteses) para terminar o script com o código de status padrão.

> **Cuidado**

> Não é possível desabilitar ou criar uma função com namespace ocultando a função global **exit()**.

Parâmetros

`status`

Se `status` for uma string, esta função exibe o `status` logo antes de sair. O código de saída retornado pelo PHP será `0`.

Se `status` for um int, o código de saída retornado pelo PHP será `status`.

> **Nota** :
>
> Códigos de saída devem estar no intervalo `0` a `254`, o código de saída `255` é reservado pelo PHP e não deve ser usado.

> **Aviso**
>
> Antes do PHP 8.4.0, **exit()** não seguia o padrão do PHP de conversão automática de tipos e nem respeitava a declaração `strict_types`.
>
> Qualquer valor que não fosse do tipo int era convertido para string, incluindo valores dos tipos resource e array. A partir do PHP 8.4.0, ela segue a conversão automática de tipos e lança uma exceção do tipo TypeError para valores inválidos.

Valor Retornado

Como a função termina o script PHP, nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.4.0	**exit()** agora é uma função verdadeira, portanto segue a conversão automática de tipos usual, é afetada pela declaração `strict_types`, pode ser chamada com argumentos nomeados, e pode ser uma função variável.
8.4.0	Uma chamada a **exit()** sem parâmetros de dentro de funções de desligamento ou de destrutores de objetos agora redefine o código de saída para `0`; anteriormente, o código de saída definido por uma chamada anterior de **exit()** era mantido.

Examples

Exemplo #1 Exemplo básico de exit()

```php
<?php

// sai do programa normalmente
exit();
exit(0);

// sai com um código de erro
exit(1);

?>
```

Exemplo #2 Exemplo de exit() com uma string

```php
<?php

$filename = '/caminho/para/arquivo-de-dados';
$file = fopen($filename, 'r')
    or exit("não foi possível abrir o arquivo ($filename)");

?>
```

Exemplo #3 Funções de desligamento e destrutores de objeto são executados independentemente

```php
<?php
class Foo
{
    public function __destruct()
```

```
    {
        echo 'Destrutor: ' . __METHOD__ . '()' . PHP_EOL;
    }
}

function shutdown()
{
    echo 'Desligamento: ' . __FUNCTION__ . '()' . PHP_EOL;
}

$foo = new Foo();
register_shutdown_function('shutdown');

exit();
echo 'Este texto não será mostrado.';
?>
```

O exemplo acima produzirá:

```
Desligamento: shutdown()
Destrutor: Foo::__destruct()
```

Exemplo #4 exit() como uma instrução

```
<?php

// sai do programa normalmente com um código de saída igual a 0
exit;

?>
```

Notas

> **Aviso**
>
> Antes do PHP 8.4.0, **exit()** era uma construção de linguagem e não uma função, portanto não era possível chamá-la usando funções variáveis, ou argumentos nomeados.

Veja também

- register_shutdown_function() - Registra uma função para ser executada no final da execução do script
- Funções de desligamento
- Destrutores de objeto

get_browser

get_browser

(PHP 4, PHP 5, PHP 7, PHP 8)

get_browser — Diz o que o navegador do usuário é capaz fazer

Descrição

function **get_browser**(?string `$user_agent = null`, bool `$return_array = false`): object|array|false

Tenta determinar as capacidades do navegador do usuário, consultando as informações do navegador no arquivo *browscap.ini*.

Parâmetros

`user_agent`

O agente do usuário (navegador) a ser analisado. Por padrão, o valor do cabeçalho HTTP User-Agent é usado; no entanto, pode-se alterar isso (ou seja, procurar informações sobre outro navegador) passando este parâmetro.

Pode-se ignorar esse parâmetro com um valor **`null`**.

`return_array`

Se definido como **`true`**, esta função retornará um array, em vez de um object.

Valor Retornado

As informações são retornadas num objeto ou array que conterá vários elementos de dados representando, por exemplo, os números de versão principal e secundário e a string de ID; valores **`true`**/**`false`** para recursos como frames, JavaScript e cookies; e assim por diante.

O valor `cookies` significa simplesmente que o navegador é capaz de aceitar cookies e não significa que o usuário habilitou ou não o navegador para aceitar cookies. A única maneira de testar se os cookies são aceitos é definir um com setcookie(), recarregar, e verificar o valor.

Retorna **`false`** quando nenhuma informação puder ser obtida, como quando de configuração browscap no *php.ini* não tiver sido configurada.

Examples

Exemplo #1 Listando todas as informações sobre o navegador do usuário

```php
<?php
echo $_SERVER['HTTP_USER_AGENT'] . "\n\n";

$browser = get_browser(null, true);
print_r($browser);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040803 Firefox/0.9.3

Array
(
    [browser_name_regex] => ^mozilla/5\.0 (windows; .; windows nt 5\.1; .*rv:.*) gecko/.* firefox/0\.9.*$
    [browser_name_pattern] => Mozilla/5.0 (Windows; ?; Windows NT 5.1; *rv:*) Gecko/* Firefox/0.9*
    [parent] => Firefox 0.9
    [platform] => WinXP
    [browser] => Firefox
    [version] => 0.9
    [majorver] => 0
    [minorver] => 9
    [cssversion] => 2
    [frames] => 1
    [iframes] => 1
    [tables] => 1
    [cookies] => 1
    [backgroundsounds] =>
    [vbscript] =>
    [javascript] => 1
    [javaapplets] => 1
    [activexcontrols] =>
    [cdf] =>
    [aol] =>
    [beta] => 1
    [win16] =>
    [crawler] =>
    [stripper] =>
    [wap] =>
    [netclr] =>
)
```

Notas

> **Nota** :
>
> Para que isso funcione, sua diretiva de configuração browscap no *php.ini* deve apontar para a localização correta do arquivo *browscap.ini* no seu sistema.
>
> O arquivo *browscap.ini* não vem com o PHP, mas pode-se encontrar um arquivo » php_browscap.ini atualizado aqui.
>
> Embora o arquivo *browscap.ini* contenha informações sobre muitos navegadores, ele depende de atualizações de usuários para manter o banco de dados atualizado. O formato do arquivo é bastante autoexplicativo.

__halt_compiler

__halt_compiler

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

__halt_compiler — Para a execução do compilador

Descrição

function **__halt_compiler**(): void

Para a execução do compilador. Pode ser útil para embutir dados em scripts PHP, como os arquivos de instalação.

A posição do byte do início dos dados pode ser determinada pela constante `__COMPILER_HALT_OFFSET__` que é definida somente se houver uma função **__halt_compiler()** presente no arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Um exemplo de __halt_compiler()

```php
<?php

// abre este arquivo
$fp = fopen(__FILE__, 'r');

// procura por dados usando o ponteiro do arquivo
fseek($fp, __COMPILER_HALT_OFFSET__);

// e mostra os dados
var_dump(stream_get_contents($fp));

// final da execução do script
__halt_compiler(); os dados de instalação (ex.: tar, gz, PHP, etc.)
```

Notas

> **Nota** :
>
> **__halt_compiler()** somente pode ser usada a partir do escopo mais externo.

highlight_file

highlight_file

(PHP 4, PHP 5, PHP 7, PHP 8)

highlight_file — Destaca a sintaxe de um arquivo

Descrição

function **highlight_file**(string `$filename`, bool `$return` = **`false`**): string|bool

Imprime ou retorna a versão com destaque de sintaxe do código contigo no arquivo `filename` utilizando as cores definidas no destacador de sintaxe nativo do PHP.

Muitos servidores são configurados para destacar automaticamente os arquivos com extensão *phps*. Por exemplo, o arquivo *example.phps* quando acessado irá mostrar o código fonte com destaque de cores. Para ativar isso, acrescente a seguinte linha no *httpd.conf*:

```
AddType application/x-httpd-php-source .phps
```

Parâmetros

`filename`

Caminho do arquivo PHP a ser destacado.

`return`

Informe esse parâmetro como **`true`** para fazer essa função retornar o código destacado (ao invés de imprimir).

Valor Retornado

Se `return` for **`true`**, retorna o código destacado como uma string ao invés de imprimir. Se não, irá retornar **`true`** no sucesso e **`false`** no caso de falhas.

Registro de Alterações

Versão	Descrição
8.3.0	O HTML resultante foi alterado.

Notas

> **Cuidado**
>
> Cuidados devem ser tomados quando utilizando a função **highlight_file()** para prevenir que você não revele informações sensíveis inadvertidamente, como senhas e outros tipos de informações que possam criar risco de segurança.

> **Nota** :
>
> Quando o parâmetro `return` é usado, esta função usa um buffer de saída interno, portanto não pode ser usada em uma função callback da função ob_start().

Veja também

- highlight_string() - Destaque de sintaxe de uma string
- Diretivas INI de highlight

highlight_string

highlight_string

(PHP 4, PHP 5, PHP 7, PHP 8)

highlight_string — Destaque de sintaxe de uma string

Descrição

function **highlight_string**(string `$string`, bool `$return = false`): string|true

Mostra ou retorna uma marcação html para uma versão destacada do código PHP fornecido usando as cores definidas no marcador de sintaxe integrado do PHP.

Parâmetros

`string`

O código PHP a ser destacado. Isso deve incluir a etiqueta de abertura.

`return`

Defina este parâmetro como `true` para fazer esta função retornar o código destacado.

Valor Retornado

Se `return` estiver definido para `true`, retorna o código destacado como uma string ao invés de exibi-lo. Caso contrário, retorna `true`.

Registro de Alterações

Versão	Descrição
8.4.0	O tipo de retorno foi alterado de string|bool para string|true.
8.3.0	O HTML resultante foi alterado.

Examples

Exemplo #1 Exemplo de highlight_string()

```
<?php
highlight_string('<?php phpinfo(); ?>');
?>
```

O exemplo acima produzirá:

```
<code><span style="color: #000000">
<span style="color: #0000BB">&lt;?php phpinfo</span><span style="color: #007700">(); </span><span style="color: #0000BB">?
&gt;</span>
</span>
</code>
```

Saída do exemplo acima no PHP 8.3:

```
<pre><code style="color: #000000"><span style="color: #0000BB">&lt;?php phpinfo</span><span style="color: #007700">();
</span><span style="color: #0000BB">?&gt;</span></code></pre>
```

Notas

> **Nota** :
>
> Quando o parâmetro `return` é usado, esta função usa um buffer de saída interno, portanto não pode ser usada em uma função callback da função ob_start().

A marcação HTML gerada está sujeita a alterações.

Veja também

- highlight_file() - Destaca a sintaxe de um arquivo
- Diretivas INI para destaque

hrtime

hrtime

(PHP 7 >= 7.3.0, PHP 8)

hrtime — Obtém o tempo de alta resolução do sistema

Descrição

function **hrtime**(bool `$as_number = false`): array|int|float|false

Retorna o tempo em alta resolução do sistema, contado a partir de um ponto arbitrário no tempo. O timestamp fornecido é monotônico e não pode ser ajustado.

Parâmetros

`as_number`

Define se o tempo em alta resolução deve ser retornado como array ou como número.

Valor Retornado

Retorna um array de inteiros na forma [segundos, nanossegundos], se o parâmetro `as_number` for falso. Caso contrário, os nanossegundos são retornados como int (plataformas 64 bits) ou float (plataformas 32 bits). Retorna `false` em caso de falha.

Examples

Exemplo #1 Uso de hrtime()

```php
<?php
echo hrtime(true), PHP_EOL;
print_r(hrtime());
?>
```

O exemplo acima produzirá algo semelhante a:

```
10444739687370679
Array
(
    [0] => 10444739
    [1] => 687464812
)
```

Veja também

- A extensão de Tempo de alta resolução
- microtime() - Retorna o timestamp Unix atual com microssegundos

ignore_user_abort

ignore_user_abort

(PHP 4, PHP 5, PHP 7, PHP 8)

ignore_user_abort — Define se uma desconexão de cliente deve abortar execução de script

Descrição

function **ignore_user_abort**(?bool `$enable = null`): int

Define se a desconexão do cliente deve abortar a execução do script.

Ao executar o PHP como um script de linha de comando, e o terminal do script for fechado antes do script ser terminado, o script irá morrer na próxima vez que tentar escreve algo, a menos que o parâmetro `enable` seja definido para `true`

Parâmetros

`enable`

> Se definido e não `null`, esta função definirá a configuração ini ignore_user_abort para o valor definido em `enable`. Caso contrário, esta função irá somente retornar a configuração anterior sem alterá-la.

Valor Retornado

Retorna a configuração anterior, como um inteiro.

Registro de Alterações

Versão	Descrição
8.0.0	`enable` agora pode ser nulo.

Examples

Exemplo #1 Um exemplo de ignore_user_abort()

```php
<?php
// Ignora o cancelamento pelo usuário e permite que o script
// seja executado para sempre
ignore_user_abort(true);
set_time_limit(0);

echo 'Testando gerenciamento de conexões no PHP';

// Executa uma repetição sem sentido que em algum momento
// talvez nos faça clicar fora da página ou
// clicar no botão de "Stop" do navegador.
while(1)
{
    // A conexão falhou?
    if(connection_status() != CONNECTION_NORMAL)
    {
        break;
    }

    // Espera por 10 segundos
    sleep(10);
}

// Se esta linha for alcançada, então o 'break'
// foi ativado de dentro da repetição 'while'

// Aqui pode-se fazer algum registro, ou executar outras tarefas
// conforme necessário sem realmente ficar dependente do
// navegador.
?>
```

Notas

O PHP não irá detectar que o usuário cancelou a conexão até que uma tentativa seja feita de enviar informação ao cliente. Simplesmente usar uma isntrução 'echo' não garante que a informação é enviada, consulte a função flush().

Veja também

- connection_aborted() - Verifica se o cliente está desconectado
- connection_status() - Retorna o campo de bits do estado da conexão
- Manipulação de Conexões para uma descrição completa de manipulação de conexões no PHP.

pack

pack

(PHP 4, PHP 5, PHP 7, PHP 8)

pack — Empacota dados em uma string binária

Descrição

function **pack**(string `$format`, mixed `...$values`): string

Empacota os argumentos fornecidos em uma string binária de acordo com o formato especificado em `format`.

A ideia para esta função foi tirada do Perl e todos os códigos de formatação funcionam da mesma forma que no Perl. Entretanto, há alguns códigos de formatação que não existem, como código "u" do Perl.

Observe que a distinção entre valores com e sem sinal somente afeta a função unpack(), enquanto que a função **pack()** fornece o mesmo resultado para códigos de formato com e sem sinal.

Parâmetros

`format`

A string `format` consistem em códigos de formato seguidos de um argumento repetidor opcional. O argumento repetidor pode ser um valor inteiro ou `*` para repetir até o final dos dados de entrada. Para a, A, h, H a contagem de repetição especifica quantos caracteres de um argumento de dados são tomados, para @ é a posição absoluta onde será colocado o próximo dado, para todo o restante a contagem de repetição especifica quantos argumentos de dados são consumidos e empacotados na string binária resultante.

Os formatos atualmente implementados são:

Caracteres de formato de pack()

Código	Descrição
a	String com bytes NUL à esquerda
A	String com espaços à esquerda
h	String hexadecimal, nibble menos significativo primeiro
H	String hexadecimal, nibble mais significativo primeiro
c	signed char
C	unsigned char
s	signed short (sempre 16 bits, ordem de bytes da máquina)
S	unsigned short (sempre 16 bits, ordem de bytes da máquina)
n	unsigned short (sempre 16 bits, ordem de bytes Big Endian)
v	unsigned short (sempre 16 bits, ordem de bytes Little Endian)
i	signed integer (tamanho e ordem de bytes dependentes da máquina)
I	unsigned integer (tamanho e ordem de bytes dependentes da máquina)
l	signed long (sempre 32 bits, ordem de bytes da máquina)
L	unsigned long (sempre 32 bits, ordem de bytes da máquina)
N	unsigned long (sempre 32 bits, ordem de bytes Big Endian)
V	unsigned long (sempre 32 bits, ordem de bytes Little Endian)
q	signed long long (sempre 64 bits, ordem de bytes da máquina)
Q	unsigned long long (sempre 64 bits, ordem de bytes da máquina)
J	unsigned long long (sempre 64 bits, ordem de bytes Big Endian)
P	unsigned long long (sempre 64 bits, ordem de bytes Little Endian)
f	float (representação e tamanho dependentes da máquina)
g	float (tamanho dependente da máquina, ordem de bytes Little Endian)
G	float (tamanho dependente da máquina, ordem de bytes Big Endian)

Código	Descrição
d	double (representação e tamanho dependentes da máquina)
e	double (tamanho dependente da máquina, ordem de bytes Little Endian)
E	double (tamanho dependente da máquina, ordem de bytes Big Endian)
x	byte NUL
X	Volta um byte
Z	String terminada em NUL (ASCIIZ), preenchida com bytes NUL
@	Preenche com bytes NUL até a posição absoluta

`values`

Valor Retornado

Retorna uma string binária contendo dados.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função não mais retorna `false` em caso de falha.
7.2.0	Tipos float e double suportam tanto Big Endian quanto Little Endian.
7.0.15, 7.1.1	Os códigos "e", "E", "g" e "G" foram adicionados para habilitar suporte a ordem de byte para float e double.

Examples

Exemplo #1 Exemplo de pack()

```php
<?php
$binarydata = pack("nvc*", 0x1234, 0x5678, 65, 66);
?>
```

A string binária resultante terá 6 bytes de comprimento e conterá a sequência de bytes 0x12, 0x34, 0x78, 0x56, 0x41, 0x42.

Notas

> **Cuidado**
>
> Os códigos de formato `q`, `Q`, `J` e `P` não estão disponíveis em versões do PHP de 32 bits.

> **Cuidado**
>
> Observe que, internamente, o PHP armazena valores int como valores com sinal de tamanho dependente da máquina. Literais inteiros e operações que resultam em números fora da faixa do tipo int serão armazenados como float. Ao empacotar estes floats como inteiros, eles primeiro são convertidos para o tipo inteiro. Isto pode ou não resultar no padrão de bytes desejado.
>
> O caso mais relevante ocorre ao empacotar números sem sinal que poderiam ser representáveis pelo tipo int se eles fossem sem sinal. Em sistemas onde o tipo int tem 32 bits, a conversão normalmente resulta no mesmo padrão de byte como se o int fosse sem sinal (embora isso dependa das conversões de sem sinal para com sinal definidas pela implementação, conforme o padrão C). Em sistemas onte o tipo int tem 64 bits, o tipo float provavelmente não terá a mantissa grande o suficiente para representar o valor sem perda de precisão. Se estes sistemas também tiverem o tipo `int` nativo de 64 bits (a maioria dos sistemas tipo UNIX não têm), a única maneira de usar o formato `I` de empacotamento na faixa superior é criar valores negativos int com a mesma representação de bytes do valor sem sinal desejado.

Veja também

- unpack() - Desempacota dados de uma string binária

php_strip_whitespace

php_strip_whitespace

(PHP 5, PHP 7, PHP 8)

php_strip_whitespace — Retorna o código com comentários e espaços em branco removidos

Descrição

function **php_strip_whitespace**(string `$filename`): string

Retorna o código-fonte do PHP do arquivo `filename` com comentários PHP e espaços em branco removidos. Isto pode ser útil para determina a quantidade atual de códigos em seu script comparado com a quantidade de comentários. Similar ao uso de **php -w** pela linha de comando.

Parâmetros

`filename`

Caminho para o arquivo PHP.

Valor Retornado

O código-fonte filtrado será retornado em caso de sucesso, ou uma string vazia em falha.

> **Nota** :
>
> Esta função respeita o valor da diretiva ini short_open_tag.

Examples

Exemplo #1 Exemplo de php_strip_whitespace()

```
<?php
// Comentário do PHP aqui

/*
 * Outro comentário do PHP
 */

echo        php_strip_whitespace(__FILE__);
// Novas linhas são consideradas espaço em branco, e também são removidas:
do_nothing();
?>
```

O exemplo acima produzirá:

```
<?php
 echo php_strip_whitespace(__FILE__); do_nothing(); ?>
```

Note que os comentários do PHP foram removidos, assim como os espaços em branco e caracteres de novas linhas depois da primeira declaração echo.

sapi_windows_cp_conv

sapi_windows_cp_conv

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_conv — Converte string de um código de página para outro

Descrição

function **sapi_windows_cp_conv**(int|string `$in_codepage`, int|string `$out_codepage`, string `$subject`): ?string

Converte string de um código de página para outro.

Parâmetros

`in_codepage`

O código de página da string definida pelo parâmetro `subject`. Pode ser um nome ou um identificador de código de página.

`out_codepage`

O código de página para o qual a string definida por `subject` será convertida. Pode ser um nome ou um identificador de código de página.

`subject`

A string a ser convertida.

Valor Retornado

A string em `subject` convertida para `out_codepage`, ou **`null`** em caso de falha.

Erros/Exceções

Esta função emite erros de nível E_WARNING, se códigos de página inválidos forem informados, ou se a string não for válida para o código de página informado em `in_codepage`.

Veja também

- sapi_windows_cp_get() - Obtém o código de página atual
- iconv() - Converte uma string de uma codificação de caracteres para outra

sapi_windows_cp_get

sapi_windows_cp_get

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_get — Obtém o código de página atual

Descrição

function **sapi_windows_cp_get**(string `$kind = ""`): int

Obtém o código de página atual.

Parâmetros

`kind`

O tipo de código de página do sistema operacional a se obter, pode ser `'ansi'` ou `'oem'`. Qualquer outro valor refere-se ao código de páginal atual do processo.

Valor Retornado

Se `kind` for `'ansi'`, o código de página ANSI atual do sistema operacional é retornado. Se `kind` for `'oem'`, o código de página OEM do sistema operacional é retornado. Caso contrário, o código de página atual do processo é retornado.

Veja também

- sapi_windows_cp_set() - Define o código de página do processo

sapi_windows_cp_is_utf8

sapi_windows_cp_is_utf8

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_is_utf8 — Indica se o código de página é compatível com UTF-8

Descrição

function **sapi_windows_cp_is_utf8**(): bool

Indica se o código de página do processo atual é compatível com UTF-8.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna verdadeiro se o código de página do processo atual for compatível com UTF-8, caso contrário retorna falso.

Veja também

- sapi_windows_cp_get() - Obtém o código de página atual

sapi_windows_cp_set

sapi_windows_cp_set

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_set — Define o código de página do processo

Descrição

function **sapi_windows_cp_set**(int `$codepage`): bool

Define o código de página do processo atual.

Parâmetros

`codepage`

Um identificador de código de página.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- sapi_windows_cp_get() - Obtém o código de página atual

sapi_windows_generate_ctrl_event

sapi_windows_generate_ctrl_event

(PHP 7 >= 7.4.0, PHP 8)

sapi_windows_generate_ctrl_event — Envia um evento CTRL a outro processo

Descrição

function **sapi_windows_generate_ctrl_event**(int `$event`, int `$pid` = 0): bool

Envia um evento CTRL a outro processo no mesmo grupo de processos.

Parâmetros

`event`

O evento `CTRL` a enviar; pode ser `PHP_WINDOWS_EVENT_CTRL_C` ou `PHP_WINDOWS_EVENT_CTRL_BREAK`.

`pid`

O ID do processo ao qual deve-se enviar o evento. Se `0` for informado, o evento é enviado a todos os processos do mesmo grupo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Uso Básico de sapi_windows_generate_ctrl_event()

Este exemplo mostra como passar eventos `CTRL+BREAK` para um processo filho. Neste caso, o processo filho mostra `Ainda estou vivo` a cada segundo, até que o usuário pressione `CTRL+BREAK`, o que faz com que apenas o processo filho seja terminado.

```php
<?php
// envia eventos CTRL+BREAK para o processo filho
sapi_windows_set_ctrl_handler('sapi_windows_generate_ctrl_event');

// cria um processo filho que mostra uma mensagem a cada segundo
$cmd = ['php', '-r', 'while (true) { echo "Ainda estou vivo\n"; sleep(1); }'];
$descspec = array(['pipe', 'r'], ['pipe', 'w'], ['pipe', 'w']);
$options = ['create_process_group' => true];
$proc = proc_open($cmd, $descspec, $pipes, null, null, $options);
while (true) {
    echo fgets($pipes[1]);
}
?>
```

Veja também

- proc_open() - Executa um comando e abre ponteiros de arquivo para entrada/saída
- sapi_windows_set_ctrl_handler() - Define ou remove um manipulador de evento CTRL

sapi_windows_set_ctrl_handler

sapi_windows_set_ctrl_handler

(PHP 7 >= 7.4.0, PHP 8)

sapi_windows_set_ctrl_handler — Define ou remove um manipulador de evento CTRL

Descrição

function **sapi_windows_set_ctrl_handler**(?callable `$handler`, bool `$add` = **`true`**): bool

Define ou remove um manipulador de evento `CTRL`, que permite que os processos CLI do Windows interceptem ou ignorem eventos `CTRL+C` e `CTRL+BREAK`. Observe que em ambientes multi-thread, isto só é possível quando a função é chamada da thread principal.

Parâmetros

`handler`

Uma função de chamada de retorno a ser definida ou removida. Se definida, esta função será chamada sempre que um evento `CTRL +C` ou `CTRL +BREAK` ocorrer. A função deve ter a seguinte assinatura:

function **handler**(int `$event`): void

`event`

O evento `CTRL` que foi recebido; pode ser **`PHP_WINDOWS_EVENT_CTRL_C`** ou **`PHP_WINDOWS_EVENT_CTRL_BREAK`**.

Definir um `handler` como **`null`** faz com que o processo ignore eventos `CTRL +C` , mas não os eventos `CTRL +BREAK` .

`add`

Se for igual a **`true`**, o manipulador é definido. Se for **`false`**, o manipulador é removido.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Uso Básico de **sapi_windows_set_ctrl_handler()**

Este exemplo mostra como interceptar eventos `CTRL`.

```php
<?php
function ctrl_handler(int $event)
{
    switch ($event) {
        case PHP_WINDOWS_EVENT_CTRL_C:
            echo "CTRL+C foi pressionado\n";
```

```
            break;
        case PHP_WINDOWS_EVENT_CTRL_BREAK:
            echo "CTRL+BREAK foi pressionado\n";
            break;
    }
}

sapi_windows_set_ctrl_handler('ctrl_handler');
while (true); // repetição infinita, para que o manipulador seja disparado
?>
```

Veja também

- sapi_windows_generate_ctrl_event() - Envia um evento CTRL a outro processo

sapi_windows_vt100_support

sapi_windows_vt100_support

(PHP 7 >= 7.2.0, PHP 8)

sapi_windows_vt100_support — Obtém ou define suporte VT100 para o fluxo especificado associado a um buffer de saída de um console Windows.

Descrição

function **sapi_windows_vt100_support**(resource `$stream`, ?bool `$enable` = **`null`**): bool

Se `enable` for **`null`**, a função retorna **`true`** se o fluxo `stream` tem códigos de controle VT100 habilitados, **`false`** caso não tenha.

Se `enable` for do tipo bool, a função tentará habilitar ou desabilitar os recursos VT100 do fluxo `stream`. Se o recurso tiver sido habilitado (ou desabilitado) com sucesso, a função retornará **`true`**, caso contrário retornará **`false`**.

Na inicialização, o PHP tenta habilitar o recurso VT100 dos fluxos **`STDOUT`**/**`STDERR`**. A propósito, se esses fluxos forem redirecionados para um arquivo, os recursos VT100 podem não ser habilitados.

Se o suporte a VT100 for habilitado, é possível usar sequências de controle como são conhecidas pelo terminal VT100. Elas permitem a alteração da saída do terminal. No Windows essas sequências são chamadas Sequências de Terminais Virtuais do Console.

> **Aviso**
>
> Esta função usa a opção **`ENABLE_VIRTUAL_TERMINAL_PROCESSING`** implementada na API do Windows 10, por isso o recurso VT100 pode não estar disponível em versões antigas do Windows.

Parâmetros

`stream`

O fluxo no qual a função irá operar.

`enable`

Se for do tipo bool, o recurso VT100 será habilitado (se **`true`**) ou desabilitado (se **`false`**).

Valor Retornado

Se `enable` for **`null`**: retorna **`true`** se o recurso VT100 for habilitado, **`false`** caso contrário.

Se `enable` for do tipo bool: Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`enable` agora pode ser nulo.

Examples

Exemplo #1 Estado padrão de sapi_windows_vt100_support()

Por padrão, STDOUT e STDERR têm o recurso VT100 habilitado.

```
php -r "var_export(sapi_windows_vt100_support(STDOUT));echo ' ';var_export(sapi_windows_vt100_support(STDERR));"
```

O exemplo acima produzirá algo semelhante a:

```
true true
```

A propósito, se um fluxo for redirecionado, o recurso VT100 não será habilitado:

```
php -r "var_export(sapi_windows_vt100_support(STDOUT));echo ' ';var_export(sapi_windows_vt100_support(STDERR));" 2>NUL
```

O exemplo acima produzirá algo semelhante a:

```
true false
```

Exemplo #2 sapi_windows_vt100_support() mudando de estado

Não será possível habilitar o recurso VT100 feature de STDOUT ou STDERR se o fluxo for redirecionado.

```
php -r "var_export(sapi_windows_vt100_support(STDOUT, true));echo ' ';var_export(sapi_windows_vt100_support(STDERR, true));"
2>NUL
```

O exemplo acima produzirá algo semelhante a:

```
true false
```

Exemplo #3 Exemplo de uso do suporte a VT100 habilitado

```php
<?php
$out = fopen('php://stdout','w');
fwrite($out, 'Eu me esqueci de uma leta.');
// Move o cursor dois caracteres para trás
fwrite($out, "\033[2D");
// Insere um espaço vazio, deslocando o texto existente para a direita -> 'Eu me esqueci de uma let a.'
fwrite($out, "\033[1@");
fwrite($out, 'r');
?>
```

O exemplo acima produzirá:

```
Eu me esqueci de uma letra.
```

show_source

show_source

(PHP 4, PHP 5, PHP 7, PHP 8)

show_source — Sinônimo de highlight_file()

Descrição

Esta função é um apelido para: highlight_file().

sleep

sleep

(PHP 4, PHP 5, PHP 7, PHP 8)

sleep — Atrasa a execução

Descrição

function **sleep**(int `$seconds`): int

Atrasa a execução do programa pelo número de segundos informado no parâmetro `seconds`.

> **Nota** :
>
> Para atrasar a execução do programa por uma fração de um segundo, use a função usleep() pois a função **sleep()** espera um número int. Por exemplo `sleep(0.25)` irá pausar a execução do programa por `0` segundo.

Parâmetros

`seconds`

Tempo de parada em segundos (deve ser maior ou igual a `0`).

Valor Retornado

Retorna zero em caso de sucesso.

Se a chamada foi interrompida por um sinal, a função **sleep()** retorna um valor não-zero. No Windows, este valor será sempre `192` (o valor da constante **`WAIT_IO_COMPLETION`** dentro da API do Windows). Em outras plataformas, o valor de retorno será o número de segundos que faltam para terminar a parada.

Erros/Exceções

Se o número de segundos especificado em `seconds` for negativo, esta função irá lançar um ValueError.

Registro de Alterações

Versão	Descrição
8.0.0	A função lança um ValueError para valores negativos de `seconds`; anteriormente, um **`E_WARNING`** era emitido, e a função retornava **`false`**.

Examples

Exemplo #1 Exemplo de **sleep()**

```php
<?php

// horário atual
echo date('h:i:s') . "\n";

// dorme por 10 seconds
sleep(10);

// acorde !
echo date('h:i:s') . "\n";

?>
```

Este exemplo irá mostrar (depois de 10 segundos)

```
05:31:23
05:31:33
```

Veja também

- usleep() - Atrasa a execução em microssegundos
- time_nanosleep() - Atrasa por um número de segundos e nanossegundos
- time_sleep_until() - Faz o script pausar até o horário especificado
- set_time_limit() - Limita o máximo tempo de execução

sys_getloadavg

sys_getloadavg

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

sys_getloadavg — Obtém a média de carga do sistema

Descrição

function **sys_getloadavg**(): array|false

Retorna três amostras representando a carga média do sistema (o número de processos na fila de execução do sistema) nos últimos 1, 5 e 15 minutos, respectivamente. Retorna `false` em caso de falha.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com três amostras (últimos 1, 5 e 15 minutos).

Examples

Exemplo #1 Um exemplo de sys_getloadavg()

```php
<?php
$load = sys_getloadavg();
if ($load[0] > 0.80) {
    header('HTTP/1.1 503 Muito ocupado, tente novamente mais tarde');
    die('Servidor muito ocupado. Por favor tente novamente mais tarde.');
}
?>
```

Notas

> **Nota** :
>
> Esta função não foi implementada em plataformas Windows.

time_nanosleep

time_nanosleep

(PHP 5, PHP 7, PHP 8)

time_nanosleep — Atrasa por um número de segundos e nanossegundos

Descrição

function **time_nanosleep**(int $seconds, int $nanoseconds): array|bool

Atrasa a execução do programa pelo número de segundos informado em `seconds` e de nanossegundos informado em `nanoseconds`.

Parâmetros

`seconds`

Deve ser um inteiro não negativo.

`nanoseconds`

Deve ser um inteiro não negativo menor que um 1 bilhão.

> **Nota** :
>
> No Windows, o sistema pode atrasar mais que o número de nanossegundos informado, dependendo do equipamento.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Se o atraso foi interrompido por um sinal, um array associativo será retornado com os componentes:

- `seconds` - número de segundos remanescentes no atraso
- `nanoseconds` - número de nanossegundos remanescentes no atraso

Examples

Exemplo #1 Exemplo de time_nanosleep()

```php
<?php
// Cuidado! Não vai funcionar como esperado se um array for retornado
if (time_nanosleep(0, 500000000)) {
    echo "Dormi por meio segundo.\n";
}

// Assim é melhor:
if (time_nanosleep(0, 500000000) === true) {
    echo "Dormi por meio segundo.\n";
}

// E assim é o melhor:
$nano = time_nanosleep(2, 100000);

if ($nano === true) {
    echo "Dormi por 2 segundos, 100 microssegundos.\n";
} elseif ($nano === false) {
    echo "Pausa falhou.\n";
} elseif (is_array($nano)) {
    $seconds = $nano['seconds'];
    $nanoseconds = $nano['nanoseconds'];
    echo "Interrompido por um sinal.\n";
    echo "Tempo remanescente: $seconds segundos, $nanoseconds nanossegundos.";
}
?>
```

Veja também

- sleep() - Atrasa a execução
- usleep() - Atrasa a execução em microssegundos
- time_sleep_until() - Faz o script pausar até o horário especificado
- set_time_limit() - Limita o máximo tempo de execução

time_sleep_until

time_sleep_until

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

time_sleep_until — Faz o script pausar até o horário especificado

Descrição

function **time_sleep_until**(float `$timestamp`): bool

Faz o script pausar até o horário especificado em `timestamp`.

Parâmetros

`timestamp`

O timestamp que representa o horário no qual o script deve acordar.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o `timestamp` especificado estiver no passado, esta função irá gerar um `E_WARNING`.

Examples

Exemplo #1 Um exemplo de time_sleep_until()

```php
<?php

//retorna falso e gera um alerta
var_dump(time_sleep_until(time()-1));

// pode ser que funcione somente em computadores mais rápidos, irá pausar por até 0,2 segundo
var_dump(time_sleep_until(microtime(true)+0.2));

?>
```

Notas

> **Nota** :
>
> Todos os sinais serão entregues depois que o script acordar.

Veja também

- sleep() - Atrasa a execução
- usleep() - Atrasa a execução em microssegundos
- time_nanosleep() - Atrasa por um número de segundos e nanossegundos
- set_time_limit() - Limita o máximo tempo de execução

uniqid

uniqid

(PHP 4, PHP 5, PHP 7, PHP 8)

uniqid — Gera um identificador baseado em horário

Descrição

function **uniqid**(string `$prefix` = "", bool `$more_entropy` = `false`): string

Obtém um identificador baseado no horário atual com precisão de microssegundos, prefixado com o parâmetro `prefix` e adicionando opcionalmente um valor gerado aleatoriamente.

> **Cuidado**

> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Aviso**
>
> Esta função não garante a unicidade do valor retornado porque o valor é baseado no horário atual em microssegundos ou no horário atual com uma pequena quantidade de dados aleatórios adicionados se `more_entropy` for **`true`**.

Parâmetros

`prefix`

> Pode ser útil, por exemplo, para gerar identificadores simultaneamente em vários servidores que poderiam gerar o mesmo identificador no mesmo microssegundo. Isso pode acontecer até no mesmo servidor se o relógio do sistema for atrasado, como em um ajuste de NTP.
>
> Com um `prefix` vazio, a string retornada terá 13 caracteres. Se `more_entropy` for **`true`**, a string terá 23 caracteres.

`more_entropy`

> Se informado como **`true`**, **uniqid()** irá adicionar entropia (utilizando um gerador linear congruente) ao final do do valor retornado, o que aumentará as chances que o resultado seja globalmente único.

Valor Retornado

Retorna um identificador baseado em horário, como uma string.

> **Aviso**
>
> Esta função não garante a unicidade do valor retornado.

Examples

Exemplo #1 Exemplo de uniqid()

```php
<?php
/* Um identificador, por exemplo: 4b3403665fea6 */
printf("uniqid(): %s\r\n", uniqid());

/* Também pode prefixar o identificador, que é o mesmo que
 * fazer:
 *
 * $uniqid = $prefix . uniqid();
 * $uniqid = uniqid($prefix);
 */
printf("uniqid('php_'): %s\r\n", uniqid('php_'));

/* Pode-se também ativar o parâmetro more_entropy, o que
 * é exigido em alguns sistemas, como o Cygwin. Isto faz que uniqid()
 * produza um valor como: 4b340550242239.64159797
 */
printf("uniqid('', true): %s\r\n", uniqid('', true));
?>
```

Notas

> **Nota** :
>
> Sob o Cygwin, o parâmetro `more_entropy` deve ser configurado como **`true`** para que essa função funcione.

Veja também

- random_bytes() - Obtém bytes aleatórios criptograficamente seguros

unpack

unpack

(PHP 4, PHP 5, PHP 7, PHP 8)

unpack — Desempacota dados de uma string binária

Descrição

function **unpack**(string `$format`, string `$string`, int `$offset` = 0): array|false

Desempacota uma srting binária em um array de acordo com o formato especificado em `format`.

Os dados desempacotados são armazenados em um array associativo. Para que isto seja feito, os diferentes códigos de formato devem ser nomeados e separados por uma barra /. Se um argumento repetidor estiver presente, cada chave do array terá um número de sequência atrás do nome fornecido.

Form feitas alterações para que esta função ficasse alinhada com a linguagem Perl:

- O código "a" agora retém os bytes NULL ao final da string.
- O código "A" agora remove todos os tipos de espaço ASCII do final da string (espaços, tabulações, novas linhas, retorno de carro e bytes NULL).
- O código "Z" foi adicionado para strings com bytes NULL à esquerda, e remove bytes NULL à direita.

Parâmetros

`format`

Consulte a página da função pack() para uma explicação sobre os códigos de formatos.

`string`

Os dados empacotados.

`offset`

O deslocamento na string a partir de onde os dados serão desempacotados.

Valor Retornado

Retorna um array associativo contendo elementos desempacotados da string binária, ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
7.2.0	Tipos float e double suportam Big Endian e Little Endian.
7.1.0	O parâmetro opcional `offset` foi adicionado.

Examples

Exemplo #1 Exemplo de unpack()

```php
<?php
$binarydata = "\x04\x00\xa0\x00";
$array = unpack("cchars/nint", $binarydata);
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [chars] => 4
```

```
    [int] => 160
)
```

Exemplo #2 Exemplo de unpack() com um repetidor

```php
<?php
$binarydata = "\x04\x00\xa0\x00";
$array = unpack("c2chars/nint", $binarydata);
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [chars1] => 4
    [chars2] => 0
    [int] => 40960
)
```

Notas

> **Cuidado**
>
> Observe que internamente o PHP armazena com sinal os valores inteiros. Se um inteiro grande longo sem sinal for desempacotado, e for do mesmo tamanho que os valores armazenados internamente pelo PHP, o resultado será um número negativo mesmo que tenha sido especificado um desempacotamento sem sinal.

> **Cuidado**
>
> Se um elemento não for nomeado, seão usados índices numéricos iniciando em `1`. Fique ciente que se houver mais de um elemento sem nome, alguns dados serão sobrescritos porque a numeração reiniciará a partir de `1` para cada elemento.
>
> **Exemplo #3 Exemplo de unpack() com chaves sem nome**
>
> ```php
> <?php
> $binarydata = "\x32\x42\x00\xa0";
> $array = unpack("c2/n", $binarydata);
> var_dump($array);
> ?>
> ```
>
> O exemplo acima produzirá:
>
> ```
> array(2) {
>   [1]=>
>   int(160)
>   [2]=>
>   int(66)
> }
> ```
>
> Observe que o primeiro valor especificado por `c` foi sobrescrito pelo primeiro valor de `n` especificado.

Veja também

- pack() - Empacota dados em uma string binária

usleep

usleep

(PHP 4, PHP 5, PHP 7, PHP 8)

usleep — Atrasa a execução em microssegundos

Descrição

function **usleep**(int `$microseconds`): void

Atrasa a execução do programa pelo número especificado de microssegundos.

Parâmetros

`microseconds`

Tempo de parada em microssegundos. Um microssegundo é um milionésimo de segundo.

> **Nota** :
>
> Valores maiores que `1000000` (isto é, pausar por mais que um segundo) pode não ser suportado pelo sistema operacional. Use a função sleep() neste caso.

> **Nota** :
>
> A pausa pode ser aumentada um pouco (isto é, ficar mais longa que o parâmetro `microseconds`) por qualquer atividade do sistema, por tempo gasto no processamento da chamada ou pela granularidade dos cronômetros do sistema.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de usleep()

```php
<?php

// Horário atual
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

// espera por 2 milissegundos
usleep(2000);

// de volta!
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

// espera por 30 milissegundos
usleep(30000);

// de volta novamente!
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

?>
```

O exemplo acima produzirá:

```
11:13:28.005
11:13:28.007
```

```
11:13:28.037
```

Veja também

- sleep() - Atrasa a execução
- time_nanosleep() - Atrasa por um número de segundos e nanossegundos
- time_sleep_until() - Faz o script pausar até o horário especificado
- set_time_limit() - Limita o máximo tempo de execução

Registro de Alterações

As alterações a seguir foram feitas em classes/funções/métodos desta extensão.

Version	Function	Description
8.4.0	exit	exit agora é uma função verdadeira, portanto segue a conversão automática de tipos usual, é afetada pela declaração strict_types, pode ser chamada com argumentos nomeados, e pode ser uma função variável.
	exit	Uma chamada a exit sem parâmetros de dentro de funções de desligamento ou de destrutores de objetos agora redefine o código de saída para 0; anteriormente, o código de saída definido por uma chamada anterior de exit era mantido.
	highlight_string	O tipo de retorno foi alterado de stringbool para stringtrue.
8.3.0	highlight_file	O HTML resultante foi alterado.
	highlight_string	O HTML resultante foi alterado.
8.1.0	define	value agora pode ser um objeto.
8.0.0	constant	Se a constante não estiver definida, constant agora lança uma exceção Error; anteriormente um E_WARNING era gerado e null era retornado.
	define	Passar true para case_insensitive agora emite um E_WARNING. Passar false ainda é permitido.
	ignore_user_abort	enable agora pode ser nulo.
	pack	Esta função não mais retorna false em caso de falha.
	sapi_windows_vt100_support	enable agora pode ser nulo.
	sleep	A função lança um ValueError para valores negativos de seconds; anteriormente, um E_WARNING era emitido, e a função retornava false.
7.3.0	define	case_insensitive foi descontinuado e será removido na versão 8.0.0.
7.2.0	pack	Tipos float e double suportam tanto Big Endian quanto Little Endian.
	unpack	Tipos float e double suportam Big Endian e Little Endian.
7.1.1	pack	Os códigos "e", "E", "g" e "G" foram adicionados para habilitar suporte a ordem de byte para float e double.
7.1.0	unpack	O parâmetro opcional offset foi adicionado.
7.0.15	pack	Os códigos "e", "E", "g" e "G" foram adicionados para habilitar suporte a ordem de byte para float e double.

Mail

Introdução

A função mail() permite enviar correio eletrônico (e-mail).

Instalação/Configuração

Índice

Dependências

Para que as funções da extensão Mail estejam disponíveis, o PHP deve ter acesso ao binário do `sendmail` no sistema durante a compilação. Se outro programa de correio eletrônico for utilizado, como qmail ou postfix, certifique-se de usar os empacotadores de sendmail apropriados que o acompanham. O PHP irá primeiramente procurar pelo sendmail na variável *PATH*, e depois nos seguintes locais: `/usr/bin:/usr/sbin:/usr/etc:/etc:/usr/ucblib:/usr/lib`. É altamente recomendado ter o sendmail disponível em *PATH*. Além disso, o usuário sob o qual o processo PHP é executado deve ter permissão para acessar o binário sendmail.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração da função Mail

Nome	Padrão	Modificável	Registro de Alterações
mail.add_x_header	"0"	`INI_PERDIR`	
mail.mixed_lf_and_crlf	"0"	`INI_SYSTEM`|`INI_PERDIR`	Adicionada no PHP 8.2.4
mail.log	NULL	`INI_SYSTEM`|`INI_PERDIR`	
mail.force_extra_parameters	NULL	`INI_SYSTEM`	
SMTP	"localhost"	`INI_ALL`	
smtp_port	"25"	`INI_ALL`	
sendmail_from	NULL	`INI_ALL`	
sendmail_path	"/usr/sbin/sendmail -t -i"	`INI_SYSTEM`	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`mail.add_x_header` bool

Adiciona `X-PHP-Originating-Script` que incluirá o UID do script seguido pelo nome do arquivo.

`mail.log` string

O caminho para o arquivo de registro que irá armazenar todas as chamadas à função mail(). As entradas de registro incluirão o caminho completo do script, número da linha, endereço de destino `To` e cabeçalhos.

`mail.mixed_lf_and_crlf` bool

Permite reverter o separador de linha para cabeçalhos de e-mail e corpos de mensagem para LF (Alimentação de Linha), imitando o comportamento não compatível do PHP 7. É fornecido como uma medida de compatibilidade para certos Agentes de Transferência de Correio Eletrônico (MTAs) não compatíveis que falham em processar corretamente CRLF (Retorno de Carro + Alimentação de Linha) como um separador de linha em cabeçalhos de e-mail e conteúdo de mensagem.

`mail.force_extra_parameters` string

Força a adição de parâmetros específicos que serão passados como parâmetros extras para o binário sendmail. Estes parâmetros sempre substituirão o valor do quinto parâmetro da função mail().

Além do comportamento padrão para `INI_SYSTEM`, este valor também pode ser definido com `php_value` no *httpd.conf* (mas isto não é recomendado).

`SMTP` string

Usado somente no Windows: nome de servidor ou endereço IP do servidor SMTP que o PHP deve usar para e-mail envio com a função mail().

`smtp_port` int

Usado somente no Windows: Número da porta para se conectar ao servidor definido com a configuração de `SMTP` ao enviar e-mail com a função mail(); a porta padrão é 25.

`sendmail_from` string

Qual endereço de e-mail `"From:"` deve ser usado no e-mail enviado diretamente via SNMP (somente Windows). Esta diretiva também define o cabeçalho `"Return-Path:"`.

`sendmail_path` string

Onde o programa **sendmail** pode ser encontrado, normalmente em */usr/sbin/sendmail* ou */usr/lib/sendmail*. **configure** faz uma tentativa de localizar e definir o padrão, mas se ele falhar, pode ser definido aqui.

Sistema que não usam o **sendmail** devem configurar essa diretiva para a alternativa ao sendmail que o sistema oferece, se houver. Por exemplo, usuários do » Qmail normalmente usam */var/qmail/bin/sendmail* ou */var/qmail/bin/qmail-inject*.

qmail-inject não requer nehuma opção para o processar o e-mail corretamente.

Esta diretiva também funciona no Windows. Se definida, *smtp*, *smtp_port* e *sendmail_from* são ignorados e o comando especificado é executado.

Funções da Mail

Índice

ezmlm_hash

ezmlm_hash

(PHP 4 >= 4.0.2, PHP 5, PHP 7)

ezmlm_hash — Calcula o valor do hash necessário para EZMLM

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.4.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

function **ezmlm_hash**(string `$addr`): int

ezmlm_hash() calcula valor do hash necessário ao se manter listas de e-mail EZMLM num banco de dados MySQL.

Parâmetros

`addr`

O endereço de e-mail para o qual está sendo calculado o hash.

Valor Retornado

O valor do hash de `addr`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função foi removida.
7.4.0	Esta função foi descontinuada.

Examples

Exemplo #1 Calculando o hash e inscrevendo um usuário

```
<?php

$user = "joecool@example.com";
$hash = ezmlm_hash($user);
$query = sprintf("INSERT INTO sample VALUES (%s, '%s')", $hash, $user);
$db->query($query); // usando interface de banco de dados PHPLIB

?>
```

mail

mail

(PHP 4, PHP 5, PHP 7, PHP 8)

mail — Envia e-mail

Descrição

function **mail**(
  string $to,
  string $subject,
  string $message,
  array|string $additional_headers = [],
  string $additional_params = ""
): bool

Envia um e-mail.

Parâmetros

`to`

Destinatário, ou destinatários do e-mail.

O formato desta string precisa estar de acordo com a » RFC 2822. Alguns exemplos:

- user@example.com
- user@example.com, anotheruser@example.com
- User
- User , Another User

`subject`

Assunto do e-mail a ser enviado.

> **Cuidado**
>
> Assunto deve satisfazer a » RFC 2047.

`message`

Mensagem a ser enviada.

Cada linha deve ser separada com um CRLF (\r\n). Linhas não devem ser maiores que 70 caracteres.

> **Cuidado**
>
> (Somente Windows) Quando o PHP está se comunicando com o servidor SMTP diretamente, e um ponto final é encontrado no início de uma linha, ela é removida. Para se defender disto, substitua estas ocorrência por dois pontos seguidos.
>
> ```
> <?php
> $text = str_replace("\n.", "\n..", $text);
> ?>
> ```

`additional_headers` (opcional)

String ou array a ser inserido no final do cabeçalho do e-mail.

Isto é tipicamente usado para adicionar cabeçalhos extras (From, Cc, e Bcc). Múltiplos cabeçalhos extras devem ser separados com um CRLF (\r\n). Se dados externos forem usados para compor este cabeçalho, os dados devem ser sanitizados para que nenhum cabeçalho indesejado possa ser injetado.

Se um array for passado, suas chaves são os nomes dos cabeçalhos e seus valores são os respectivos valores dos cabeçalhos.

> **Nota** :
>
> Ao enviar e-mail, ele *precisa* conter um cabeçalho `From`. Este pode ser definido com o parâmetro `additional_headers`, ou um padrão pode ser definido no *php.ini*.
>
> Se isto não foi feito, ocorrerá uma mensagem de erro similar a `Warning: mail(): "sendmail_from" not set in php.ini or custom "From:" header missing`. O cabeçalho `From` define também o `Return-Path` ao enviar diretamente via SMTP (Windows somente).

> **Nota** :
>
> Se mensagens não estão sendo recebidas, tente usar somente um LF (\n). Alguns agentes Unix de transferência de e-mail (mais notavelmente » qmail) substituem LF por CRLF automaticamente (que leva à duplicação de CR se CRLF for usado). Este deve ser um último recurso, porque não está conforme a » RFC 2822.

`additional_params` (opcional)

O parâmetro `additional_params` pode ser usado para passar um parâmetro adicional para o programa configurado para usa quando enviando e-mail usando a configuração `sendmail_path`. Por exemplo, isto pode ser usado para definir o endereço do envelope remetente quando usando sendmail com a opção do sendmail `-f`.

Este parâmetro é escapado por escapeshellcmd() internamente para prevenir a execução do comando. escapeshellcmd() impede a execução de comandos, mas permite adicionar parâmetros adicionais. Por razões de segurança, é recomendado que o usuário higienize este parâmetro para evitar adicionar parâmetros indesejados ao comando shell.

Uma vez que escapeshellcmd() é aplicada automaticamente, alguns caracteres que são permitidos como endereços de e-mail por RFCs da Internet não podem ser usados. **mail()** não pode permitir tais caracteres, portanto, em programas onde o uso de tais caracteres é necessário, meios alternativos de envio de e-mails (como o uso de um framework ou biblioteca) são recomendados.

O usuário com o qual o servidor web roda deve ser adicionado como um usuário confiável à configuração do sendmail para evitar que um cabeçalho 'X-Warning' seja adicionado à mensagem quando o remetente do envelope (-f) é definido usando este método. Para usuários do sendmail, este arquivo é */etc/mail/trusted-users*.

Valor Retornado

Retorna **`true`** se o e-mail foi aceito com sucesso para entrega, **`false`** caso contrário.

É importante notar que somente pelo fato do e-mail ter sido aceito para entrega, NÃO significa que o e-mail realmente alcancará o destino esperado.

Registro de Alterações

Versão	Descrição
7.2.0	O parâmetro `additional_headers` agora aceita um array.

Examples

Exemplo #1 Enviando e-mail.

Usando **mail()** para enviar um simples e-mail:

```php
<?php
// A mensagem
$message = "Line 1\r\nLine 2\r\nLine 3";

// Se as linhas forem maiores que 70 caracteres, deve-se usar wordwrap()
```

```php
$message = wordwrap($message, 70, "\r\n");

// Envia
mail('caffeinated@example.com', 'Meu Assunto', $message);
?>
```

Exemplo #2 Enviando e-mail com cabeçalhos extras.

A adição de cabeçalhos básicos, informando à MUA os endereços From e Reply-To:

```php
<?php
$to      = 'nobody@example.com';
$subject = 'the subject';
$message = 'hello';
$headers = 'From: webmaster@example.com' . "\r\n" .
    'Reply-To: webmaster@example.com' . "\r\n" .
    'X-Mailer: PHP/' . phpversion();

mail($to, $subject, $message, $headers);
?>
```

Exemplo #3 Enviando e-mail com cabeçalhos extras no formato array

Este exemplo envia o mesmo e-mail que o exemplo imediatamente acima, mas passa os cabeçalhos adicionais como array (disponível a partir do PHP 7.2.0).

```php
<?php
$to      = 'nobody@example.com';
$subject = 'the subject';
$message = 'hello';
$headers = array(
    'From' => 'webmaster@example.com',
    'Reply-To' => 'webmaster@example.com',
    'X-Mailer' => 'PHP/' . phpversion()
);

mail($to, $subject, $message, $headers);
?>
```

Exemplo #4 Enviando e-mail com um parâmetro de linha de comando adicional.

O parâmetro `additional_params` pode ser usado para passar um parâmetro adicional para o programa configurado para usar ao enviar e-mail usando o `sendmail_path`.

```php
<?php
mail('nobody@example.com', 'the subject', 'the message', null,
   '-fwebmaster@example.com');
?>
```

Exemplo #5 Enviando e-mail HTML

É também possível enviar e-mail HTML com **mail()**.

```php
<?php
// Múltiplos recipientes
$to = 'joao@example.com, susana@example.com'; // observe a vírgula

// Assunto
$subject = 'Lembrete de Aniversários para Novembro';

// Message
$message = '
<html>
<head>
  <title>Lembrete de Aniversários para Novembro</title>
</head>
<body>
  <p>Aqui estão os próximos aniversariantes de novembro!</p>
  <table>
    <tr>
```

```
      <th>Pessoa</th><th>Dia</th><th>Mês</th><th>Ano</th>
    </tr>
    <tr>
      <td>João</td><td>10</td><td>Agosto</td><td>1990</td>
    </tr>
    <tr>
      <td>Susana</td><td>17</td><td>Agosto</td><td>2003</td>
    </tr>
  </table>
</body>
</html>
';

// Para enviar e-mail HTML, o cabeçalho Content-type deve ser definido
$headers[] = 'MIME-Version: 1.0';
$headers[] = 'Content-type: text/html; charset=iso-8859-1';

// Cabeçalhos adicionais
$headers[] = 'To: Maria <maria@example.com>, Rafael <rafael@example.com>';
$headers[] = 'From: Lembrete de Aniversário <niver@example.com>';
$headers[] = 'Cc: arquivo_niver@example.com';
$headers[] = 'Bcc: verifica_niver@example.com';

// Envia o e-mail
mail($to, $subject, $message, implode("\r\n", $headers));
?>
```

> **Nota** :
>
> Se a intenção é enviar e-mail HTML ou outro formato complexo, é recomendado usar o pacote PEAR » PEAR::Mail_Mime.

Notas

> **Nota** :
>
> A implentação SMTP (somente Windows) de **mail()** difere bastante da implentação do sendmail. Primeiro, ele não usa um binário local para compor mensagens mas apenas opera com sockets diretos, o que significa que uma `MTA` é necessária monitorando um socket de rede (que pode ser ou o localhost ou uma máquina remota).
>
> Segundo, cabeçalhos customizados como `From:`, `Cc:`, `Bcc:` e `Date:` **não** são interpretados pela `MTA`, mas são analisados pelo PHP.
>
> Desta forma, o parâmetro `to` não deve ser um endereço na forma de "Algo <alguem@example.com>". O comando de e-mail poderia não analisar isto adequadamente enquanto se comunica com o MTA.

> **Nota** :
>
> É válido notar que a função **mail()** não é apropriada para um grande volume de e-mail em um loop. Esta função abre e fecha um SMTP socket para cada e-mail, o que não é muito eficiente.
>
> Para enviar uma grande quantidade de e-mail, veja os pacotes » PEAR::Mail, e » PEAR::Mail_Queue.

> **Nota** :
>
> O seguintes RFCs podem ser úteis: » RFC 1896, » RFC 2045, » RFC 2046, » RFC 2047, » RFC 2048, » RFC 2049 e » RFC 2822.

Veja também

- mb_send_mail() - Envia email codificado
- imap_mail() - Envia uma mensagem de e-mail
- » PEAR::Mail
- » PEAR::Mail_Mime

Biblioteca Cliente URL

Introdução

O PHP suporta libcurl, uma biblioteca criada por Daniel Stenberg, que permite conexão e comunicação com diferentes tipos de servidores usando diferentes tipos de protocolos. libcurl atualmente suporta os protocolos http, https, ftp, gopher, telnet, dict, file e ldap. libcurl também suporta certificados HTTPS, HTTP POST, HTTP PUT, envio via FTP (isto também pode ser feito com a extensão ftp do PHP), envio HTTP por formulário, proxies, cookies, e autenticação com usuário e senha.

Instalação/Configuração

Índice

Dependências

Para utilizar as funções cURL no PHP, é necessário instalar o pacote » libcurl. O PHP requer libcurl versão 7.10.5 ou superior. A partir do PHP 7.3.0, a versão 7.15.5 ou superior é requerida. A partir do PHP 8.0.0, a versão 7.29.0 ou superior é requerida. A partir do PHP 8.4.0, a versão 7.61.0 ou superior é requerida.

Instalação

Para usar o suporte a cURL no PHP, é necessário compilar o PHP com a opção **--with-curl[=DIR]** onde DIR é o diretório que contém os diretórios *lib* e *include*. No diretório *include*, deve haver uma pasta chamada *curl* que deve conter os arquivos *easy.h* e *curl.h*. Deve haver um arquivo chamado *libcurl.a* localizado no diretório *lib*.

> **Nota** : **Nota aos usuários de Win32**
>
> Para ativar este módulo em ambiente Windows, *libeay32.dll* e *ssleay32.dll* (ou a partir do OpenSSL 1.1 *libcrypto-*.dll* e *libssl-*.dll*), precisam estar presentes na variável de ambiente *PATH*. O arquivo *libssh2.dll* também precisa estar em *PATH*.
>
> Não será necessário o arquivo *libcurl.dll* do site do cURL.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração cURL

Nome	Padrão	Modificável	Registro de Alterações
curl.cainfo	NULL	`INI_SYSTEM`	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`curl.cainfo` string

Um valor padrão para a opção `CURLOPT_CAINFO`. Precisa ser um caminho absoluto.

Tipos de Recurso

Antes do PHP 8.0.0, esta extensão definia três tipos de recursos: o identificador `curl`, o identificador `curl_multi` e o identificador `curl_share`.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

Descrições e usos para estas constantes estão documentados nas funções curl_setopt(), curl_multi_setopt() e curl_getinfo().

Constantes	Descrição
`CURLALTSVC_H1` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
`CURLALTSVC_H2` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
`CURLALTSVC_H3` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
`CURLALTSVC_READONLYFILE` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
`CURLAUTH_ANY` (int)	
`CURLAUTH_ANYSAFE` (int)	
`CURLAUTH_AWS_SIGV4` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.75.0.
`CURLAUTH_BASIC` (int)	
`CURLAUTH_BEARER` (int)	Disponível a partir do PHP 7.3.0 e cURL 7.61.0
`CURLAUTH_DIGEST` (int)	
`CURLAUTH_DIGEST_IE` (int)	Usa autenticação HTTP Digest no estilo IE. Disponível a partir do cURL 7.19.3.
`CURLAUTH_GSSAPI` (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.1
`CURLAUTH_GSSNEGOTIATE` (int)	
`CURLAUTH_NEGOTIATE` (int)	Disponível a partir do PHP 7.0.7 e cURL 7.38.0.
`CURLAUTH_NONE` (int)	Disponível a partir do cURL 7.10.6.
`CURLAUTH_NTLM` (int)	
`CURLAUTH_NTLM_WB` (int)	Disponível a partir do PHP 7.0.7 e cURL 7.22.0
`CURLAUTH_ONLY` (int)	Este é um meta-símbolo. A operação binária OR deste valor junto com um valor único de autenticação específica força a libcurl a pesquisar por autenticação irrestrita e se não utilizada, apenas esse único algoritmo de autenticação será aceitável. Disponível a partir do cURL 7.21.3.
`CURLFOLLOW_ALL` (int)	Valor para `CURLOPT_FOLLOWLOCATION` que permite seguir redirecionamentos e mantém um método de requisição personalizado definido com `CURLOPT_CUSTOMREQUEST` em uso para todas as requisições, mesmo após redirecionamentos. Disponível a partir do PHP 8.5.0 e cURL 8.13.0.
`CURLFOLLOW_OBEYCODE` (int)	Valor para `CURLOPT_FOLLOWLOCATION` que permite seguir redirecionamentos, respeitando o código de resposta HTTP: um método de requisição personalizado definido com `CURLOPT_CUSTOMREQUEST` é mantido, exceto que ele é alterado para `GET` nos códigos de status de redirecionamento que o exigem (como 301, 302 e 303). Disponível a partir do PHP 8.5.0 e cURL 8.13.0.
`CURLFOLLOW_FIRSTONLY` (int)	Valor para `CURLOPT_FOLLOWLOCATION` que permite seguir redirecionamentos, mas usa um método de requisição personalizado definido com `CURLOPT_CUSTOMREQUEST` apenas para a

	primeira requisição; as requisições subsequentes seguem o método ditado pelo código de resposta do redirecionamento. Disponível a partir do PHP 8.5.0 e cURL 8.13.0.
CURLFTPAUTH_DEFAULT (int)	
CURLFTPAUTH_SSL (int)	
CURLFTPAUTH_TLS (int)	
CURLFTPMETHOD_DEFAULT (int)	Disponível a partir do PHP 8.2.0 e cURL 7.15.3.
CURLFTPMETHOD_MULTICWD (int)	Faz uma única operação `CWD` para cada parte no URL fornecido. Disponível a partir do cURL 7.15.3.
CURLFTPMETHOD_NOCWD (int)	libcurl não faz nenhuma operação `CWD`. libcurl faz as operações `SIZE`, `RETR`, `STOR` etc. e fornece um caminho completo ao servidor para todos estes comandos. Disponível a partir do cURL 7.15.3.
CURLFTPMETHOD_SINGLECWD (int)	libcurl faz uma operação `CWD` com o caminho completo do diretório de destino e depois opera no arquivo como no caso do CURLFTPMETHOD_MULTICWD. Disponível a partir do cURL 7.15.3.
CURLFTPSSL_ALL (int)	
CURLFTPSSL_CCC_ACTIVE (int)	Inicia o desligamento e espera por uma resposta. Disponível a partir do cURL 7.16.2.
CURLFTPSSL_CCC_NONE (int)	Não tenta usar o CCC (Clear Command Channel - canal de comando em texto puro). Disponível a partir do cURL 7.16.2.
CURLFTPSSL_CCC_PASSIVE (int)	Não inicia o desligamento, mas espera que o servidor o faça. Não envia uma resposta. Disponível a partir do cURL 7.16.1.
CURLFTPSSL_CONTROL (int)	
CURLFTPSSL_NONE (int)	
CURLFTPSSL_TRY (int)	
CURLFTP_CREATE_DIR (int)	Disponível a partir do PHP 7.0.7 e cURL 7.19.3
CURLFTP_CREATE_DIR_NONE (int)	Disponível a partir do PHP 7.0.7 e cURL 7.19.3
CURLFTP_CREATE_DIR_RETRY (int)	Disponível a partir do PHP 7.0.7 e cURL 7.19.3
CURLGSSAPI_DELEGATION_FLAG (int)	Permite delegação incondicional de delegação de credenciais GSSAPI. Disponível a partir do cURL 7.22.0.
CURLGSSAPI_DELEGATION_POLICY_FLAG (int)	Delega somente se a opção `OK-AS-DELEGATE` estiver ativa no tíquete de serviço se este recruso for suportado pela implementação GSS-API e a definicão de `GSS_C_DELEG_POLICY_FLAG` estava disponível no momento da compilação. Disponível a partir do cURL 7.22.0.
CURLHEADER_SEPARATE (int)	Disponível a partir do PHP 7.0.7 e cURL 7.37.0.
CURLHEADER_UNIFIED (int)	Disponível a partir do PHP 7.0.7 e cURL 7.37.0.

CURLHSTS_ENABLE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.74.0
CURLHSTS_READONLYFILE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.74.0
CURLKHMATCH_LAST (int)	Disponível a partir do PHP 8.3.0 e cURL 7.19.6
CURLKHMATCH_MISMATCH (int)	Disponível a partir do PHP 8.3.0 e cURL 7.19.6
CURLKHMATCH_MISSING (int)	Disponível a partir do PHP 8.3.0 e cURL 7.19.6
CURLKHMATCH_OK (int)	Disponível a partir do PHP 8.3.0 e cURL 7.19.6
CURLMIMEOPT_FORMESCAPE (int)	Disponível a partir do PHP 8.3.0 e cURL 7.81.0
CURLMSG_DONE (int)	
CURLPIPE_HTTP1 (int)	Disponível a partir do cURL 7.43.0.
CURLPIPE_MULTIPLEX (int)	Disponível a partir do cURL 7.43.0.
CURLPIPE_NOTHING (int)	Disponível a partir do cURL 7.43.0.
CURLPROXY_HTTP (int)	Disponível a partir do cURL 7.10.
CURLPROXY_HTTPS (int)	Disponível a partir do PHP 7.3.0 e cURL 7.52.0
CURLPROXY_HTTP_1_0 (int)	Disponível a partir do PHP 7.0.7 e cURL 7.19.3
CURLPROXY_SOCKS4 (int)	Disponível a partir do cURL 7.10.
CURLPROXY_SOCKS4A (int)	Disponível a partir do cURL 7.18.0.
CURLPROXY_SOCKS5 (int)	Disponível a partir do cURL 7.10.
CURLPROXY_SOCKS5_HOSTNAME (int)	Disponível a partir do cURL 7.18.0.
CURLPX_BAD_ADDRESS_TYPE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_BAD_VERSION (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_CLOSED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_GSSAPI (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_GSSAPI_PERMSG (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0

CURLPX_GSSAPI_PROTECTION (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_IDENTD (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_IDENTD_DIFFER (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_LONG_HOSTNAME (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_LONG_PASSWD (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_LONG_USER (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_NO_AUTH (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_OK (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_RECV_ADDRESS (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_RECV_AUTH (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_RECV_CONNECT (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_RECV_REQACK (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_ADDRESS_TYPE_NOT_SUPPORTED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_COMMAND_NOT_SUPPORTED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_CONNECTION_REFUSED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_GENERAL_SERVER_FAILURE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_HOST_UNREACHABLE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_NETWORK_UNREACHABLE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_NOT_ALLOWED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_TTL_EXPIRED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REPLY_UNASSIGNED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_REQUEST_FAILED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0

CURLPX_RESOLVE_HOST (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_SEND_AUTH (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_SEND_CONNECT (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_SEND_REQUEST (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_UNKNOWN_FAIL (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_UNKNOWN_MODE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLPX_USER_REJECTED (int)	Disponível a partir do PHP 8.2.0 e cURL 7.73.0
CURLSSH_AUTH_AGENT (int)	Disponível a partir do PHP 7.0.7 e cURL 7.28.0
CURLSSH_AUTH_ANY (int)	
CURLSSH_AUTH_DEFAULT (int)	
CURLSSH_AUTH_GSSAPI (int)	Disponível a partir do PHP 7.3.0 e cURL 7.58.0
CURLSSH_AUTH_HOST (int)	
CURLSSH_AUTH_KEYBOARD (int)	
CURLSSH_AUTH_NONE (int)	
CURLSSH_AUTH_PASSWORD (int)	
CURLSSH_AUTH_PUBLICKEY (int)	
CURLSSLOPT_ALLOW_BEAST (int)	Disponível a partir do cURL 7.25.0
CURLSSLOPT_AUTO_CLIENT_CERT (int)	Disponível a partir do PHP 8.2.0 e cURL 7.77.0
CURLSSLOPT_NATIVE_CA (int)	Disponível a partir do PHP 8.2.0 e cURL 7.71.0
CURLSSLOPT_NO_PARTIALCHAIN (int)	Disponível a partir do PHP 8.2.0 e cURL 7.68.0
CURLSSLOPT_NO_REVOKE (int)	Disponível a partir do PHP 7.0.7 e cURL 7.44.0
CURLSSLOPT_REVOKE_BEST_EFFORT (int)	Disponível a partir do PHP 8.2.0 e cURL 7.70.0
CURLUSESSL_ALL (int)	Requer SSL para todas as comunicações ou falha com **CURLE_USE_SSL_FAILED**. Disponível a partir do cURL 7.17.0.
CURLUSESSL_CONTROL (int)	Requer SSL para conexão de controle ou falha com **CURLE_USE_SSL_FAILED**. Disponível a partir do cURL 7.17.0.
CURLUSESSL_NONE (int)	Não tenta usar SSL. Disponível a partir do cURL 7.17.0.

CURLUSESSL_TRY (int)	Tenta usar SSL, procede normalmente caso não seja possível. Observe que o servidor pode fechar a conexão se a negociação não for bem sucedida. Disponível a partir do cURL 7.17.0.
CURLVERSION_NOW (int)	
CURLWS_RAW_MODE (int)	Disponível a partir do PHP 8.3.0 e cURL 7.86.0
CURL_FNMATCHFUNC_FAIL (int)	Retornado pela função de correspondência a coringas se um erro tiver ocorrido. Disponível a partir do cURL 7.21.0.
CURL_FNMATCHFUNC_MATCH (int)	Retornado pela função de correspondência a coringas se o padrão corresponder à string. Disponível a partir do cURL 7.21.0.
CURL_FNMATCHFUNC_NOMATCH (int)	Retornado pela função de correspondência a coringas se o padrão não corresponder à string. Disponível a partir do cURL 7.21.0.
CURL_HTTP_VERSION_1_0 (int)	
CURL_HTTP_VERSION_1_1 (int)	
CURL_HTTP_VERSION_2 (int)	Disponível a partir do PHP 7.0.7 e cURL 7.43.0
CURL_HTTP_VERSION_2TLS (int)	Disponível a partir do PHP 7.0.7 e cURL 7.47.0
CURL_HTTP_VERSION_2_0 (int)	Disponível a partir do cURL 7.33.0
CURL_HTTP_VERSION_2_PRIOR_KNOWLEDGE (int)	Disponível a partir do PHP 7.0.7 e cURL 7.49.0
CURL_HTTP_VERSION_3 (int)	Disponível a partir do PHP 8.4.0 e cURL 7.66.0.
CURL_HTTP_VERSION_3ONLY (int)	Disponível a partir do PHP 8.4.0 e cURL 7.88.0.
CURL_HTTP_VERSION_NONE (int)	
CURL_IPRESOLVE_V4 (int)	Usa apenas endereços IPv4 ao estabelecer uma conexão ou ao escolher uma a partir do pool de conexões. Disponível a partir do cURL 7.10.8.
CURL_IPRESOLVE_V6 (int)	Usa apenas endereços IPv6 ao estabelecer uma conexão ou ao escolher uma a partir do pool de conexões. Disponível a partir do cURL 7.10.8.
CURL_IPRESOLVE_WHATEVER (int)	Usa endereços de todas as versões de IP permitidas pelo sistema. Disponível a partir do cURL 7.10.8.
CURL_MAX_READ_SIZE (int)	Disponível a partir do PHP 7.3.0 e cURL 7.53.0
CURL_NETRC_IGNORED (int)	
CURL_NETRC_OPTIONAL (int)	
CURL_NETRC_REQUIRED (int)	
CURL_PUSH_DENY (int)	Disponível a partir do PHP 7.1.0 e cURL 7.44.0

CURL_PUSH_OK (int)	Disponível a partir do PHP 7.1.0 e cURL 7.44.0
CURL_READFUNC_PAUSE (int)	Disponível a partir do cURL 7.18.0.
CURL_REDIR_POST_301 (int)	Disponível a partir do PHP 7.0.7 e cURL 7.18.2
CURL_REDIR_POST_302 (int)	Disponível a partir do PHP 7.0.7 e cURL 7.18.2
CURL_REDIR_POST_303 (int)	Disponível a partir do PHP 7.0.7 e cURL 7.25.1
CURL_REDIR_POST_ALL (int)	Disponível a partir do PHP 7.0.7 e cURL 7.18.2
CURL_RTSPREQ_ANNOUNCE (int)	Quando enviado por um cliente, este método altera a descrição da sessão. `ANNOUNCE` atua como um PUT ou POST HTTP da mesma forma que **CURL_RTSPREQ_SET_PARAMETER**. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_DESCRIBE (int)	Usada para obter a descrição baixo nível de um fluxo. A aplicação deve observar que formatos ela entende no cabeçalho `Accept:`. A menos que seja definido manualmente, a libcurl automaticamente adiciona `Accept: application/sdp`. Cabeçalhos de condições temporais são adicionados a requisições DESCRIBE se a opção **CURLOPT_TIMECONDITION** for usada. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_GET_PARAMETER (int)	Recupera um parâmetro do servidor. Por padrão, a libcurl adiciona um cabeçalho `Content-Type: text/parameters` em todas as requisições não vazias a menos que um cabeçalho personalizado seja definido. `GET_PARAMETER` atua da mesma forma que um PUT ou um POST HTTP. Aplicações cujas intenções sejam enviar uma mensagem de pulsação devem usar uma requisição `GET_PARAMETER` vazia. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_OPTIONS (int)	Usada para recuperar os métodos disponíveis do servidor. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_PAUSE (int)	Envia um comando `PAUSE` ao servidor. Use a opção **CURLOPT_RANGE** com um único valor para indicar quando o fluxo deve parar (ex.: npt=25). Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_PLAY (int)	Envia um comando `PLAY` ao servidor. Use a opção **CURLOPT_RANGE** para modificar o momento que o fluxo deve continuar (ex.: npt=10-15). Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_RECEIVE (int)	Defina o tipo de requisição RTSP para este falor para receber dados RTP intercalados. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_RECORD (int)	Usada para informar ao servidor que grave a sessão. Use a opção **CURLOPT_RANGE** para modificar o tempo de gravação. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_SETUP (int)	Usada para inicializar a camada de transporte para a sessão. Disponível a partir do cURL 7.20.0.
CURL_RTSPREQ_SET_PARAMETER (int)	Define um parâmetro no servidor. Disponível a partir do cURL 7.20.0.

CURL_RTSPREQ_TEARDOWN (int)	Termina uma sessão RTSP. O simples fechamento de uma conexão não termina a sessão RTSP já que é válido controlar uma sessão RTSP a partir de diferentes conexões. Disponível a partir do cURL 7.20.0.
CURL_SSLVERSION_DEFAULT (int)	
CURL_SSLVERSION_MAX_DEFAULT (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_MAX_NONE (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_0 (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_1 (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_2 (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_3 (int)	Disponível a partir do PHP 7.3.0 e cURL 7.54.0
CURL_SSLVERSION_SSLv2 (int)	
CURL_SSLVERSION_SSLv3 (int)	
CURL_SSLVERSION_TLSv1 (int)	
CURL_SSLVERSION_TLSv1_0 (int)	
CURL_SSLVERSION_TLSv1_1 (int)	
CURL_SSLVERSION_TLSv1_2 (int)	
CURL_SSLVERSION_TLSv1_3 (int)	Disponível a partir do PHP 7.3.0 e cURL 7.52.0
CURL_TIMECOND_IFMODSINCE (int)	
CURL_TIMECOND_IFUNMODSINCE (int)	
CURL_TIMECOND_LASTMOD (int)	
CURL_TIMECOND_NONE (int)	
CURL_TLSAUTH_SRP (int)	Disponível a partir do cURL 7.21.4.
CURL_VERSION_ALTSVC (int)	Disponível a partir do PHP 7.3.6 e cURL 7.64.1
CURL_VERSION_ASYNCHDNS (int)	Resolução de DNS assíncrona. Disponível a partir do PHP 7.3.0 e cURL 7.10.7
CURL_VERSION_BROTLI (int)	Disponível a partir do PHP 7.3.0 e cURL 7.57.0
CURL_VERSION_CONV (int)	Suporte a conversão de caracteres. Disponível a partir do PHP 7.3.0 e cURL 7.15.4
CURL_VERSION_CURLDEBUG (int)	Rastreamento de memória para depuração suportado. Disponível a partir do PHP 7.3.6 e cURL 7.19.6
CURL_VERSION_DEBUG (int)	Compilado com funcionalidades de depuração. Disponível a partir do PHP 7.3.0 e cURL 7.10.6

CURL_VERSION_GSASL (int)	Disponível a partir do PHP 8.2.0 e cURL 7.76.0
CURL_VERSION_GSSAPI (int)	Compilado com a biblioteca GSS-API. Disponível a partir do PHP 7.3.0 e cURL 7.38.0
CURL_VERSION_GSSNEGOTIATE (int)	Autenticação de negociação GSS é suportada. Disponível a partir do PHP 7.3.0 e cURL 7.10.6 (descontinuado a partir do cURL 7.38.0)
CURL_VERSION_HSTS (int)	Disponível a partir do PHP 8.2.0 e cURL 7.74.0
CURL_VERSION_HTTP2 (int)	Suporte interno ao HTTP2. Disponível a partir do cURL 7.33.0
CURL_VERSION_HTTP3 (int)	Disponível a partir do PHP 8.2.0 e cURL 7.66.0
CURL_VERSION_HTTPS_PROXY (int)	Disponível a partir do PHP 7.3.0 e cURL 7.52.0
CURL_VERSION_IDN (int)	Nomes de Domínio Internacionalizados são suportados. Disponível a partir do PHP 7.3.0 e cURL 7.12.0
CURL_VERSION_IPV6 (int)	Habilitado para IPv6.
CURL_VERSION_KERBEROS4 (int)	Autenticação Kerberos V4 é suportada.
CURL_VERSION_KERBEROS5 (int)	Autenticação Kerberos V5 é suportada. Disponível a partir do PHP 7.0.7 e cURL 7.40.0
CURL_VERSION_LARGEFILE (int)	Suporta arquivos maiores que 2GB. Disponível a partir do cURL 7.33.0
CURL_VERSION_LIBZ (int)	Recursos libz estão presentes.
CURL_VERSION_MULTI_SSL (int)	Disponível a partir do PHP 7.3.0 e cURL 7.56.0
CURL_VERSION_NTLM (int)	Autenticação NTLM é suportada. Disponível a partir do PHP 7.3.0 e cURL 7.10.6
CURL_VERSION_NTLM_WB (int)	Delegação NTLM para winbind helper é suportado. Disponível a partir do PHP 7.3.0 e cURL 7.22.0
CURL_VERSION_PSL (int)	Lista Pública de Sufixos do Mozilla, utilizada para verificação de cookies de domínio. Disponível a partir do PHP 7.3.6 e cURL 7.47.0
CURL_VERSION_SPNEGO (int)	Autenticação SPNEGO é suportada. Disponível a partir do PHP 7.3.0 e cURL 7.10.8
CURL_VERSION_SSL (int)	Opções SSL estão presentes.
CURL_VERSION_SSPI (int)	Compilado com Windows SSPI. Disponível a partir do PHP 7.3.0 e cURL 7.13.2
CURL_VERSION_TLSAUTH_SRP (int)	Autenticação TLS-SRP é suportada. Disponível a partir do PHP 7.3.0 e cURL 7.21.4

CURL_VERSION_UNICODE (int)	Disponível a partir do PHP 8.2.0 e cURL 7.72.0
CURL_VERSION_UNIX_SOCKETS (int)	Sockets de domínio Unix suportados. Disponível a partir do PHP 7.0.7 e cURL 7.40.0
CURL_VERSION_ZSTD (int)	Disponível a partir do PHP 8.2.0 e cURL 7.72.0
CURL_WRITEFUNC_PAUSE (int)	Disponível a partir do cURL 7.18.0.
CURL_PREREQFUNC_OK (int)	Disponível a partir do PHP 8.4.0 e cURL 7.80.0.
CURL_PREREQFUNC_ABORT (int)	Disponível a partir do PHP 8.4.0 e cURL 7.80.0.

curl_setopt()

Constantes	Descrição
CURLOPT_ABSTRACT_UNIX_SOCKET (int)	Habilita o uso de um soquete de domínio Unix abstrato ao invés de estabelecer uma conexão TCP a um servidor e define o caminho para a string informada. Esta opção compatilha da mesma semântica de CURLOPT_UNIX_SOCKET_PATH. Estas duas opções compartilham o mesmo armazenamento e portanto somente uma delas pode ser definida por identificador. Disponível a partir do PHP 7.3.0 e cURL 7.53.0.
CURLOPT_ACCEPT_ENCODING (int)	Define uma string com o conteúdo do cabeçalho `Accept-Encoding:` enviado em uma requisição HTTP. Se definida para `null`, desabilita o envio do cabeçalho `Accept-Encoding:`. O padrão é `null`. Disponível a partir do cURL 7.21.6.
CURLOPT_ACCEPTTIMEOUT_MS (int)	O número máximo de milissegundos de espera para que um servidor se conecte novamente ao cURL quando uma conexão FTP ativa é usada. Esta opção aceita qualquer valor que possa ser convertido em um int válido. O padrão é `60000` milissegundos. Disponível a partir do cURL 7.24.0.
CURLOPT_ADDRESS_SCOPE (int)	O valor do ID do escopo a ser usado ao conectar-se a endereços IPv6. Esta opção aceita qualquer valor que possa ser convertido em um int válido. O padrão é `0`. Disponível a partir do cURL 7.19.0.
CURLOPT_ALTSVC (int)	Passa uma string com o nome do arquivo para o cURL usar como o arquivo de cache Alt-Svc para leitura de conteúdo de cache existente e possivelmente também gravar de volta após uma transferência, a menos que CURLALTSVC_READONLYFILE esteja definido CURLOPT_ALTSVC_CTRL. Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
CURLOPT_ALTSVC_CTRL (int)	Popula a máscara de bits com o conjunto correto de recursos para instruir o cURL sobre como lidar com Alt-Svc para as transferências usando este identificador. cURL somente aceita cabeçalhos Alt-Svc através de HTTPS. Também irá completar uma requisição para uma origem alternativa se essa origem for apropriadamente servida através de HTTPS. Definir qualquer bit irá habilitar o motor Alt-svc. Pode ser uma das constantes CURLALTSVC_*. O padrão é o motor Alt-svc desabilitado. Disponível a partir do PHP 8.2.0 e cURL 7.64.1.
CURLOPT_APPEND (int)	Definir esta opção como `1` fará com que os uploads de FTP sejam anexados ao arquivo remoto em vez de sobrescrevê-lo. O padrão é `0`. Disponível a partir do cURL 7.17.0.
CURLOPT_AUTOREFERER (int)	`true` para definir automaticamente o campo `Referer:` nas requisições onde ele segue um redirecionamento `Location:`. O padrão é `0`. Disponível a partir do cURL 7.1.0.

CURLOPT_AWS_SIGV4 (int)	Fornece autenticação de assinatura AWS V4 em cabeçalho HTTP(S) como uma string. Esta opção substitui qualquer outros tipos de autenticação que tenham sido definidos em **CURLOPT_HTTPAUTH**. Este método não pode ser combinado com outros tipos de autenticação. Disponível a partir do PHP 8.2.0 e cURL 7.75.0.
CURLOPT_BINARYTRANSFER (int)	Esta constante não é mais usada desde o PHP 5.5.0. Descontinuada a partir do PHP 8.4.0.
CURLOPT_BUFFERSIZE (int)	O tamanho do buffer a ser usado para cada leitura. Entretanto, não há garantia de que esta requisição será cumprida. Esta opção aceita qualquer valor que possa ser convertido para um int válido. O padrão é **CURL_MAX_WRITE_SIZE** (atualmente, 16kB). Disponível a partir do cURL 7.10.
CURLOPT_CAINFO (int)	Uma string com o nome de um arquivo contendo um ou mais certificados com o qual o par será verificado. Só faz sentido se usado em combinação com **CURLOPT_SSL_VERIFYPEER**. Pode requerer um caminho absoluto. Disponível a partir do cURL 7.4.2.
CURLOPT_CAINFO_BLOB (int)	Uma string binária com conteúdo codificado em PEM contendo um ou mais certificados com os quais o par será verificado. Esta opção substitui a **CURLOPT_CAINFO**. Disponível a partir do PHP 8.2.0 e cURL 7.77.0.
CURLOPT_CAPATH (int)	Uma string com o diretótio que mantém múltiplos certificados CA. Use esta opção junto com **CURLOPT_SSL_VERIFYPEER**. Disponível a partir do cURL 7.9.8.
CURLOPT_CA_CACHE_TIMEOUT (int)	Define o tempo máximo em segundo que qualquer armazenamento de certificado CA em cache na memória pode ser mantido e reutilizado para novas conexões. Esta opção aceita qualquer valor que possa ser convertido para um int válido. O padrão é `86400` (24 horas). Disponível a partir do PHP 8.3.0 e cURL 7.87.0.
CURLOPT_CERTINFO (int)	**true** para enviar informação de certificação SSL para **STDERR** em transferências seguras. Requer **CURLOPT_VERBOSE** ligada para ter efeito. O padrão é **false**. Disponível a partir do cURL 7.19.1.
CURLOPT_CONNECTTIMEOUT (int)	O número de segundos a esperar durante tentativa de conexão. Use `0` para esperar indefinidamente. Esta opção aceita qualquer valor que possa ser convertido para um int válido. O padrão é `300`. Disponível a partir do cURL 7.7.0.
CURLOPT_CONNECTTIMEOUT_MS (int)	O número de milissegundos a esperar durante tentativa de conexão. Use `0` para esperar indefinidamente. Se o cURL tiver sido compilado para usar resolução de nome padrão do sistema, esta porção da conexão ainda usará a resolução de segundos inteiros para limites de tempo com um tempo mínimo permitido de um segundo. Esta opção aceita qualquer valor que possa ser convertido para um int válido. O padrão é `300000`. Disponível a partir do cURL 7.16.2.
CURLOPT_CONNECT_ONLY (int)	**true** informa à biblioteca para realizar toda a autenticação de proxy requerida e configuração de conexão, mas sem transferência de dados. Esta opção está implementada para HTTP, SMTP e POP3. O padrão é **false**. Disponível a partir do cURL 7.15.2.
CURLOPT_CONNECT_TO (int)	Conecta a um servidor e porta específicos ao invés do servidor e porta da URL. Aceita um array de strings com o formato `HOST:PORT:CONNECT-TO-HOST:CONNECT-TO-PORT`. Disponível a partir do PHP 7.0.7 e cURL 7.49.0.
CURLOPT_COOKIE (int)	Uma string com o conteúdo do cabeçalho `Cookie:` a ser usado na requisição HTTP. Observe que cookies múltiplos são separados por ponto-e-vírgula seguido de um espaço (ex.: `fruit=apple; colour=red`). Disponível a partir do cURL 7.1.0.

CURLOPT_COOKIEFILE (int)	Uma string com o nome do arquivo contendo os dados do cookie. O arquivo de cookie pode ser no formato Netscape ou apenas cabeçalhos puros no estilo HTTP gravados em um arquivo. Se o nome for uma string vazia, nenhum cookie é carregado, porém o gerenciamento de cookies ainda estará habilitado. Disponível a partir do cURL 7.1.0.
CURLOPT_COOKIEJAR (int)	Uma string com o nome do arquivo para gravar todos os cookies internos para quando o destrutor do identificador for chamado. Disponível a partir do cURL 7.9.0. **Aviso** A partir do PHP 8.0.0, curl_close() não tem função e *NÃO* destrói o identificador. Se for necessário gravar cookies antes do identificador ser destruido automaticamente, execute `curl_setopt($ch, CURLOPT_COOKIELIST, "FLUSH");`.
CURLOPT_COOKIELIST (int)	Uma string de cookie (isto é, uma linha única no formato Netscape/Mozilla ou um cabeçalho Set-Cookie no estilo HTTP) adiciona esse cookie único ao armazenamento interno de cookies. `ALL` apaga todos os cookies da memória , `SESS` apaga todos os cookies de sessão mantidos na memória , `FLUSH` grava todos os cookies conhecidos no arquivo especificado por **CURLOPT_COOKIEJAR** , `RELOAD` carrega todos os cookies dos arquivos especificados por **CURLOPT_COOKIEFILE** . Disponível a partir do cURL 7.14.1.
CURLOPT_COOKIESESSION (int)	**true** para marcar esta como uma nova "sessão" de cookie. Irá forçar o cURL a ignorar todos os cookies que estiverem para ser carregados que sejam "cookies de sessão" da sessão anterior. Por padrão, o cURL sempre guarda e carrega todos os cookies, independente se houver cookies de sessão ou não. Cookies de sessão são cookies sem data de expiração que devem existir apenas durante esta "sessão". Disponível a partir do cURL 7.9.7.
CURLOPT_CRLF (int)	**true** para converter novas linhas Unix para novas linhas CRLF nas transferências. Disponível a partir do cURL 7.1.0.
CURLOPT_CRLFILE (int)	Passa uma string nomeando um arquivo com a concatenação da CRL (Certificate Revocation List) (em formato PEM) para utilização na validação do certificado que ocorre durante a troca SSL. Quando cURL é construído para usar GnuTLS, não há como influenciar o uso da CRL passada para ajudar no processo de verificação. Quando cURL é construído com suporte OpenSSL, ambas `X509_V_FLAG_CRL_CHECK` e `X509_V_FLAG_CRL_CHECK_ALL` estarão definidas, exigindo verificação de CRL em todos os elementos da cadeia de certificados se um arquivo CRL for passado. Observe também que **CURLOPT_CRLFILE** implica em **CURLSSLOPT_NO_PARTIALCHAIN** a partir do cURL 7.71.0 devido a um problema do OpenSSL. Disponível a partir do cURL 7.19.0.
CURLOPT_CUSTOMREQUEST (int)	Um método de requisição customizado a ser usado no lugar de `GET` ou `HEAD` ao fazer uma requisição HTTP. Isto é útil para fazer `DELETE` ou outras requisições HTTP mais obscuras. Valores válidos são por exemplo `GET`, `POST`, `CONNECT` entre outros; Não insira aqui uma requisição HTTP inteira. Por exemplo, escrever `GET /index.html HTTP/1.0\r\n\r\n` estaria incorreto. Esta opção aceita uma string ou **null**. Disponível a partir do cURL 7.1.0. **Nota** : Isto não deve ser feito sem primeiro garantir que o servidor suporta o método de requisição.
CURLOPT_DEFAULT_PROTOCOL (int)	Uma string com o protocolo padrão a ser usado se a URL não tiver um nome de esquema. Disponível a partir do PHP 7.0.7 e cURL 7.45.0.
CURLOPT_DIRLISTONLY (int)	Definir esta opção como `1` terá efeitos diferentes com base no protocolo com o qual é usada. URLs baseados em FTP e SFTP listarão apenas os nomes dos arquivos em um diretório. POP3

	listará a mensagem ou mensagens de e-mail no servidor POP3. Para FILE, esta opção não tem efeito, pois os diretórios são sempre listados neste modo. Usar esta opção com `CURLOPT_WILDCARDMATCH` impedirá que este último tenha qualquer efeito. O padrão é `0`. Disponível a partir do cURL 7.17.0.
`CURLOPT_DISALLOW_USERNAME_IN_URL` (int)	`true` para não permitir URLs que incluem um nome de usuário. Nomes de usuário são permitidos por padrão. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
`CURLOPT_DNS_CACHE_TIMEOUT` (int)	O número de segundos para manter as entradas DNS na memória. Esta opção é definida para `120` (2 minutos) por padrão. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.9.3.
`CURLOPT_DNS_INTERFACE` (int)	Define o nome da interface de rede à qual a resolução de DNS deve vincular. Precisa ser um nome de interface (não um endereço). Esta opção aceita uma string ou `null`. Disponível a partir do PHP 7.0.7 e cURL 7.33.0.
`CURLOPT_DNS_LOCAL_IP4` (int)	Define o endereço IPv4 local ao qual a resolução DNS deve vincular. O argumento deve conter um único endereço numérico IPv4. Esta opção aceita uma string ou `null`. Disponível a partir do PHP 7.0.7 e cURL 7.33.0.
`CURLOPT_DNS_LOCAL_IP6` (int)	Define o endereço IPv6 local ao qual a resolução DNS deve vincular. O argumento deve conter um único endereço numérico IPv6. Esta opção aceita uma string ou `null`. Disponível a partir do PHP 7.0.7 e cURL 7.33.0.
`CURLOPT_DNS_SERVERS` (int)	Passa uma string com uma lista separada por vírgulas de servidores DNS a serem usadosem vez do padrão do sistema (ex.: `192.168.1.100,192.168.1.101:8080`). Disponível a partir do cURL 7.24.0.
`CURLOPT_DNS_SHUFFLE_ADDRESSES` (int)	`true` para embaralhar a ordem de todos os endereços de retorno de forma que sejam usados em ordem aleatória, quando um nome é resolvido e mais de um endereço IP é retornado. Isso pode fazer com que IPv4 seja usado antes de IPv6 ou vice-versa. Disponível a partir do PHP 7.3.0 e cURL 7.60.0.
`CURLOPT_DNS_USE_GLOBAL_CACHE` (int)	`true` para usar cache de DNS global. Esta opção não é segura para threads. É condicionalmente habilitada por padrão se o PHP for compilado sem uso de threads (CLI, FCGI, Apache2-Prefork, etc.). Disponível a partir do cURL 7.9.3 e descontinuada a partir do cURL 7.11.1. A partir do PHP 8.4, esta opção não tem mais qualquer efeito.
`CURLOPT_DOH_SSL_VERIFYHOST` (int)	Definida como `2` verifica os campos de nome de certificado SSL do servidor de DNS-sobre-HTTPS, comparando com o nome do host. Disponível a partir do PHP 8.2.0 e cURL 7.76.0.
`CURLOPT_DOH_SSL_VERIFYPEER` (int)	Definida como `1` habilita e como `0` desabilita a verificação da autenticidade do certificado SSL do servidor de DNS-sobre-HTTPS. Disponível a partir do PHP 8.2.0 e cURL 7.76.0.
`CURLOPT_DOH_SSL_VERIFYSTATUS` (int)	Definida como `1` habilita e como `0` desabilita a verificação do status do certificado do servidor de DNS-sobre-HTTPS usando a extensão TLS "Certificate Status Request" (grampeamento OCSP). Disponível a partir do PHP 8.2.0 e cURL 7.76.0.
`CURLOPT_DOH_URL` (int)	Fornece a URL do DNS-sobre-HTTPS. Esta opção aceita uma string ou `null`. Disponível a partir do PHP 8.1.0 e cURL 7.62.0.

CURLOPT_EGDSOCKET (int)	Similar a **CURLOPT_RANDOM_FILE**, exceto por um nome de arquivo para um soquete "Entropy Gathering Daemon". Disponível a partir do cURL 7.7.0 e descontinuada a partir do cURL 7.84.0.
CURLOPT_ENCODING (int)	O conteúdo do cabeçalho `Accept-Encoding:` com uma string. Isto habilita a decodificação da resposta. Codificações suportadas são: `identity`, `deflate`, `gzip`. Se uma string vazia for informada, é enviado um cabeçalho contendo todas os tipos de codificação. Disponível a partir do cURL 7.10 e descontinuada a partir do cURL 7.21.6.
CURLOPT_EXPECT_100_TIMEOUT_MS (int)	O limite de tempo para respostas `Expect: 100-continue` em milissegundos. O padrão é `1000` milissegundos. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do PHP 7.0.7 e cURL 7.36.0.
CURLOPT_FAILONERROR (int)	**true** para falhar prolixamente se o código HTTP retornado for maior ou igual a `400`. O comportamento padrão é retornar a página normalmente, ignorando o código. Disponível a partir do cURL 7.1.0.
CURLOPT_FILE (int)	Aceita um resource identificador de arquivo no qual a transferência deve ser gravada. O padrão é **STDOUT** (a janela do navegador). Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.9.7.
CURLOPT_FILETIME (int)	Defina como **true** para tentar buscar a data de modificação do documento remoto. Este valor pode ser buscado usando-se a opção **CURLINFO_FILETIME** com curl_getinfo(). Disponível a partir do cURL 7.5.0.

CURLOPT_FNMATCH_FUNCTION (int)	Passa um tipo callable que será usado para correspondência com caracteres coringas. A assinatura desta função deve ser:
	
	function callback(resource `$curlHandle`, string `$pattern`, string `$string`): int
	
	`curlHandle`
	O identificador cURL.
	
	`pattern`
	A expressão com coringas.
	
	`string`
	A string na qual a correspodência com coringas será executada.
	
	A função deve retornar **CURL_FNMATCHFUNC_MATCH** se a expressão corresponder à string, **CURL_FNMATCHFUNC_NOMATCH** se não corresponder ou **CURL_FNMATCHFUNC_FAIL** se um erro ocorrer. Disponível a partir do cURL 7.21.0.

CURLOPT_FOLLOWLOCATION (int)	Defina como **true** para seguir qualquer cabeçalho `Location:` que o servidor envia como parte do cabeçalho HTTP. Consulte também **CURLOPT_MAXREDIRS**. Esta constante não está disponível quando open_basedir está habilitada. Disponível a partir do cURL 7.1.0.
CURLOPT_FORBID_REUSE (int)	Defina como **true** para forçar a conexão a ser fechada explicitamente ao terminar o processamento, e não ser enfileirada para reúso. Disponível a partir do cURL 7.7.0.
CURLOPT_FRESH_CONNECT (int)	Defina como **true** para forçar o uso de uma nova conexão ao invés de uma em cache. Disponível a partir do cURL 7.7.0.

CURLOPT_FTPAPPEND (int)	Defina como **true** para anexar ao arquivo remoto ao invés de sobrescrevê-lo. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.16.4.
CURLOPT_FTPASCII (int)	Um apelido para **CURLOPT_TRANSFERTEXT**. Use este último. Disponível a partir do cURL 7.1, descontinuada a partir do cURL 7.11.1 e pela última vez disponível no cURL 7.15.5. Removida a partir do PHP 7.3.0.
CURLOPT_FTPLISTONLY (int)	Defina como **true** para apenas listar os nomes de um diretório FTP. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.16.4.
CURLOPT_FTPPORT (int)	Uma string que será usada para obter o endereço IP para a instrução FTP PORT. A instrução PORT solicita ao servidor remoto que se conecte ao endereço IP especificado. A string pode ser um endereço IP puro, um nome de servidor, um nome de interface de rede (no Unix), ou apenas um hífen - para usar o endereço IP padrão do sistema. Esta opção aceita uma string ou **null**. Disponível a partir do cURL 7.1.0.
CURLOPT_FTPSSLAUTH (int)	Define o método de autenticação FTP sobre SSL (se ativado) para qualquer das constantes **CURLFTPAUTH_***. O padrão é **CURLFTPAUTH_DEFAULT**. Disponível a partir do cURL 7.12.2.
CURLOPT_FTP_ACCOUNT (int)	Passe uma string que será enviada como informação da conta via FTP (usando o comando ACCT) depois que o nome de usuário e a senha forem fornecidos ao servidor. Se definida como **null**, desativa o envio das informações da conta. O padrão é **null**. Disponível a partir do cURL 7.13.0.
CURLOPT_FTP_ALTERNATIVE_TO_USER (int)	Passa uma string que será usada para tentativa de autenticação por FTP se a negociação USER/PASS falhar. Disponível a partir do cURL 7.15.5.
CURLOPT_FTP_CREATE_MISSING_DIRS (int)	Defina como **true** para criar diretórios ausentes quando uma operação FTP encontrar um caminho que não existe atualmente. Disponível a partir do cURL 7.10.7.
CURLOPT_FTP_FILEMETHOD (int)	Informa ao cURL que método deve ser usado para alcançar um arquivo em um servidor FTP(S). Valores possíveis são quaisquer das constantes **CURLFTPMETHOD_***. O padrão é **CURLFTPMETHOD_MULTICWD**. Disponível a partir do cURL 7.15.1.
CURLOPT_FTP_RESPONSE_TIMEOUT (int)	Um tempo limite em segundos em que o cURL aguardará uma resposta de um servidor FTP. Esta opção substitui **CURLOPT_TIMEOUT**. Esta opção aceita qualquer valor que possa ser convertido em um int válido. Este nome de opção foi substituído por **CURLOPT_SERVER_RESPONSE_TIMEOUT**, disponível a partir do PHP 8.4.0. Disponível a partir do cURL 7.10.8 e descontinuado a partir do cURL 7.85.0.
CURLOPT_FTP_SKIP_PASV_IP (int)	Se esta opção for definida como 1, o cURL não usará o endereço IP que o servidor sugere em sua resposta 227 ao comando PASV do cURL, mas usará o endereço IP usado para o conexão. O número da porta recebido da resposta 227 não será ignorado pelo cURL. O padrão é 1 a partir do cURL 7.74.0. e 0 antes disso. Disponível a partir do cURL 7.15.0.
CURLOPT_FTP_SSL (int)	Disponível a partir do cURL 7.11.0 e descontinuada a partir do cURL 7.16.4.
CURLOPT_FTP_SSL_CCC (int)	Esta opção faz com que o cURL use CCC (Clear Command Channel), que desliga a camada SSL/TLS após a autenticação, tornando o restante da comunicação do canal de controle descriptografado. Use uma das constantes **CURLFTPSSL_CCC_***. O padrão é **CURLFTPSSL_CCC_NONE**. Disponível a partir do cURL 7.16.1.

CURLOPT_FTP_USE_EPRT (int)	Defina como **true** para usar EPRT (e LPRT) ao realizar downloads FTP ativos. Use **false** para desabilitar EPRT e LPRT e usar PORT somente. Disponível a partir do cURL 7.10.5.
CURLOPT_FTP_USE_EPSV (int)	Defina como **true** para primeiro tentar um comando EPSV para transferências FTP antes de reverter para PASV. Defina como **false** para desabilitar EPSV. Disponível a partir do cURL 7.9.2.
CURLOPT_FTP_USE_PRET (int)	Definida para 1, envia um comando PRET antes de PASV (e EPSV). Não tem efeito ao usar o modo ativo de transferências FTP. O padrão é 0. Disponível a partir do cURL 7.20.0.
CURLOPT_GSSAPI_DELEGATION (int)	Definida como **CURLGSSAPI_DELEGATION_FLAG**, permite a delegação incondicional de credenciais GSSAPI. Definida como **CURLGSSAPI_DELEGATION_POLICY_FLAG**, delega somente se a opção OK-AS-DELEGATE estiver definida no ticket de serviço. O padrão é **CURLGSSAPI_DELEGATION_NONE**. Disponível a partir do cURL 7.22.0.
CURLOPT_HAPPY_EYEBALLS_TIMEOUT_MS (int)	Tempo para o IPv6 começar à frente no algoritmo "happy eyeballs". Este algoritmo tenta a conexão simutânea a endereços IPv4 e IPv6 para servidores de pilha dupla, preferindo IPv6 primeiro pelo tempo especificado em milissegundos. O padrão é **CURL_HET_DEFAULT**, que atualmente é de 200 milissegundos. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do PHP 7.3.0 e cURL 7.59.0.
CURLOPT_HAPROXYPROTOCOL (int)	Defina como **true** para enviar um cabeçalho de protocolo PROXY HAProxy v1 no início da conexão. A ação padrão é não enviar este cabeçalho. Disponível a partir do PHP 7.3.0 e cURL 7.60.0.
CURLOPT_HEADER (int)	Defina como **true** para incluir o cabeçalho na saída enviada à função de retorno definida por **CURLOPT_WRITEFUNCTION**. Disponível a partir do cURL 7.1.0.

CURLOPT_HEADERFUNCTION (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(resource `$curlHandle`, string `$headerData`): int
	
	`curlHandle`
	
	O identificador cURL.
	
	`headerData`
	
	Os dados de cabeçalho que precisam ser escritos pela função.
	
	A função deve retornar o número de bytes escritos. Disponível a partir do cURL 7.7.2.

CURLOPT_HEADEROPT (int)	Evia cabeçalhos HTTP tanto para prooxy quanto para host, ou separadamente. Valores possíveis são quaisquer das constantes **CURLHEADER_***. O padrão é **CURLHEADER_SEPARATE** a partir do cURL 7.42.1, e **CURLHEADER_UNIFIED** antes dessa versão. Disponível a partir do PHP 7.0.7 e cURL 7.37.0.
CURLOPT_HSTS (int)	Uma string com o nome do arquivo de cache HSTS (HTTP Strict Transport Security). ou **null** para permitir HSTS sem ler ou escrever em qualuqer arquivo e limpar a lista de arquivos dos quais os dados HSTS são lidos. Disponível a partir do PHP 8.2.0 e cURL 7.74.0.
CURLOPT_HSTS_CTRL (int)	Aceita uma máscara de bits de recursos HSTS (HTTP Strict Transport Security) definidos pelas constantes **CURLHSTS_***. Disponível a partir do PHP 8.2.0 e cURL 7.74.0.
CURLOPT_HTTP09_ALLOWED (int)	Informa se respostas HTTP/0.9 são permitidas. O padrão é **false** a partir do cURL 7.66.0; anteriormente o padrão era **true**. Disponível a partir do PHP 7.3.15 e 7.4.3, respectivamente, e cURL 7.64.0.

CURLOPT_HTTP200ALIASES (int)	Um array de respostas HTTP `200` que serão tratadas como respostas válidas e não como erros. Disponível a partir do cURL 7.10.3.
CURLOPT_HTTPAUTH (int)	Uma máscara de bits dos métodos de autenticação HTTP a serem usados. As opções são: **CURLAUTH_BASIC**, **CURLAUTH_DIGEST**, **CURLAUTH_GSSNEGOTIATE**, **CURLAUTH_NTLM**, **CURLAUTH_AWS_SIGV4**, **CURLAUTH_ANY**, **CURLAUTH_ANYSAFE**. Se mais de um método for usado, o cURL irá consultar o servidor para ver que métodos ele suporta e escolherá o melhor. **CURLAUTH_ANY** liga todos os bits. O cURL selecionará automaticamente o que ele considerar o mais seguro. **CURLAUTH_ANYSAFE** liga todos os bits exceto o **CURLAUTH_BASIC**. O cURL selecionará automaticametne o que ele considerar o mais seguro. Disponível a partir do cURL 7.10.6.
CURLOPT_HTTPGET (int)	Defina como **true** para redefinir o método de requisição HTTP para `GET`. Como `GET` é o padrão, isto somente é necessário se o método de requisição tiver sido alterado. Disponível a partir do cURL 7.8.1.
CURLOPT_HTTPHEADER (int)	Um array de campos de cabeçalho HTTP a definir, no formato `array('Content-type: text/plain', 'Content-length: 100')` Disponível a partir do cURL 7.1.0.
CURLOPT_HTTPPROXYTUNNEL (int)	Defina como **true** para encapsular através de um determinado proxy HTTP. Disponível a partir do cURL 7.3.0.
CURLOPT_HTTP_CONTENT_DECODING (int)	**false** para obter o corpo de resposta HTTP bruto. Disponível a partir do cURL 7.16.2.
CURLOPT_HTTP_TRANSFER_DECODING (int)	Se definida para `0`, a decodificação de transferência é desabilitada. Se definida como `1`, a decodificação de transferência é habilitada. O cURL faz decodificação de transferência em partes por padrão, a menos que esta opção esteja definida como `0`. O padrão é `1`. Disponível a partir do cURL 7.16.2.
CURLOPT_HTTP_VERSION (int)	Defina para uma das constantes **CURL_HTTP_VERSION_*** para que o CURL use a versão especificada. Disponível a partir do cURL 7.9.1.
CURLOPT_IGNORE_CONTENT_LENGTH (int)	Se definida como `1`, ignora o cabeçalho `Content-Length` na resposta HTTP e ignore a solicitação ou dependência dele para transferências FTP. O padrão é `0`. Disponível a partir do cURL 7.14.1.
CURLOPT_INFILE (int)	Aceita um resource identificador do arquivo a ser lido pela transferência durante o envio. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.9.7. Use **CURLOPT_READDATA** em seu lugar.
CURLOPT_INFILESIZE (int)	O tamanho esperado do arquivo, em bytes, ao enviá-lo a um servidor remoto. Observe que usar esta opção não impedirá que o cURL envie mais dados, já que o que exatamente será enviado depende de **CURLOPT_READFUNCTION**. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.1.0.
CURLOPT_INFILESIZE_LARGE (int)	O tamanho esperado, em bytes, do arquivo ao fazer o upload de um arquivo para um site remoto. Esta é a variante de 64 bits de **CURLOPT_INFILESIZE**, permitindo que tamanhos maiores que 2 GB sejam especificados. Disponível a partir do PHP 8.5.0.
CURLOPT_INTERFACE (int)	Aceita uma string com o nome da interface de rede de saída a ser usada. Pode ser um nome de interface, um endereço IP ou um nome de servidor. Disponível a partir do cURL 7.1.0.

CURLOPT_IPRESOLVE (int)	Permite que uma aplicação selecione que tipo de endereço IP deve ser usado ao resolver nomes de servidor. Isto é interessante somente ao usar nomes de servidor resolvam endereços usando mais que uma versão de IP. Defina para uma das constantes **CURL_IPRESOLVE_***. O padrão é **CURL_IPRESOLVE_WHATEVER**. Disponível a partir do cURL 7.10.8.
CURLOPT_ISSUERCERT (int)	Se definida como uma string nomeando um arquivo que contém um certificado CA no formato PEM, uma verificação adicional no certificado de mesmo nível é executada para verificar se o emissor é de fato aquele associado ao certificado fornecido pela opção. Para que o resultado da verificação seja considerado uma falha, esta opção deve ser utilizada em combinação com a opção **CURLOPT_SSL_VERIFYPEER**. Disponível a partir do cURL 7.19.0.
CURLOPT_ISSUERCERT_BLOB (int)	Passe uma string com dados binários de um certificado de CA SSL no formato PEM. Se definida, uma verificação adicional do certtificado do par é realizada para verificar se o emissor é o associado com o certificado fornecido pela opção. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_KEEP_SENDING_ON_ERROR (int)	Defina como **true** para continuar enviando o corpo da requisição se o código HTTP retornado for igual ou maior que `300`. A ação padrão seria parar o envio e fechar o fluxo ou a conexão. Adequado para autenticação NTLM manual. A maior parte das aplicações não precisa desta opção. Disponível a partir do PHP 7.3.0 e cURL 7.51.0.
CURLOPT_KEYPASSWD (int)	Aceita uma string com a senha requerida para usar a chave privada **CURLOPT_SSLKEY** ou **CURLOPT_SSH_PRIVATE_KEYFILE**. Definir esta opção para **null** desabilita o uso de uma senha para estas opções. Disponível a partir do cURL 7.17.0.
CURLOPT_KRB4LEVEL (int)	O nível de segurança KRB4 (Kerberos 4). Qualquer dos valores string a seguir é válido (na ordem do menos para o mais potente): `clear`, `safe`, `confidential`, `private`. Se a string não corresponder a uma dessas opções, `private` é usada. Definir esta opção como **null** irá desabilitar a segurança KRB4. Atualmente, a segurança KRB4 funciona somente com transações FTP. Disponível a partir do cURL 7.3.0 e descontinuada a partir do cURL 7.17.0.
CURLOPT_KRBLEVEL (int)	Define o nível de segurança Kerberos para FTP e também habilita o reconhecimento de Kerberos. Isso deve ser definido como uma das seguintes strings: `clear`, `safe`, `confidential`, `private`. Se a string estiver definida, mas não corresponder a nenhuma delas, `private` será usada. Definir esta opção como **null** desativará o suporte Kerberos para FTP. O padrão é **null**. Disponível a partir do cURL 7.16.4 e descontinuado a partir do cURL 8.17.0.
CURLOPT_LOCALPORT (int)	Define o número da porta local do soquete usado para a conexão. Esta opção aceita qualquer valor que possa ser convertido em um int válido. O padrão é `0`. Disponível a partir do cURL 7.15.2.
CURLOPT_LOCALPORTRANGE (int)	O número de tentativas que o cURL faz para encontrar um número de porta local funcional, começando com aquele definido com **CURLOPT_LOCALPORT**. Esta opção aceita qualquer valor que possa ser convertido em um int válido. O padrão é `1`. Disponível a partir do cURL 7.15.2.
CURLOPT_LOGIN_OPTIONS (int)	Pode ser usada para definir opções de login específicas do protocolo, tal como o mecanismos preferível de autenticação via `AUTH=NTLM` ou `AUTH=*`, e deve ser usada em conjunto com a opção **CURLOPT_USERNAME**. Disponível a partir do PHP 7.0.7 e cURL 7.34.0.
CURLOPT_LOW_SPEED_LIMIT (int)	A velocidade de transferência, em bytes por segundo, em que a transferência deve ficar abaixo durante a contagem de **CURLOPT_LOW_SPEED_TIME** segundos antes que o PHP considere que a transferência está muito lenta e aborte. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.1.0.

CURLOPT_LOW_SPEED_TIME (int)	O número de segundos que a velocidade de transferência deve estar abaixo de **CURLOPT_LOW_SPEED_LIMIT** antes que o PHP considere a transferência muito lenta e aborte. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.1.0.
CURLOPT_MAIL_AUTH (int)	Define uma string com o endereço de autenticação (identidade) de uma mensagem enviada que está sendo retransmitida para outro servidor. O endereço não deve ser especificado entre parênteses angulares (<>). Se uma string vazia for usada, um par de parênteses angulares será enviado pelo cURL conforme exigido pela RFC 2554. Disponível a partir do cURL 7.25.0.
CURLOPT_MAIL_FROM (int)	Define uma string com o endereço de e-mail do remetente ao enviar mensagens SMTP. O endereço de e-mail deve ser especificado entre parênteses angulares (<>) ao redor dele, que se não forem especificados serão adicionados automaticamente. Se este parâmetro não for especificado, um endereço vazio será enviado ao servidor SMTP, o que poderá fazer com que o e-mail seja rejeitado. Disponível a partir do cURL 7.20.0.
CURLOPT_MAIL_RCPT (int)	Deve ser definida como um array de strings com os destinatários a serem passados ao servidor em uma solicitação de email SMTP. Cada destinatário deve ser especificado entre parênteses angulares (<>). Se um parêntese angular não for usado como primeiro caractere, o cURL assumirá que um único endereço de e-mail foi fornecido e colocará esse endereço entre parênteses angulares. Disponível a partir do cURL 7.20.0.
CURLOPT_MAIL_RCPT_ALLLOWFAILS (int)	Defina como `1` para permitir que o comando `RCPT TO` falhe para alguns destinatários, o que faz com que o cURL ignore erros para destinatários individuais e proceda com os destinatários aceitos remanescentes. Se todos os destinatários aivarem falhas e esta opção estiver especificada, o cURL aborta a conversação SMTP e retorna o erro recebido do último comando `RCPT TO`. Substituído pela **CURLOPT_MAIL_RCPT_ALLOWFAILS** a partir do cURL 8.2.0. Disponível a partir do PHP 8.2.0 e cURL 7.69.0. Descontinuada a partir do cURL 8.2.0.
CURLOPT_MAXAGE_CONN (int)	O tempo máximo ocioso permitido para que uma conexão existente possa ser considerada para reúso. O tempo máximo padrão é de `118` segundos. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do PHP 8.2.0 e cURL 7.65.0.
CURLOPT_MAXCONNECTS (int)	A quantidade máxima permitida de conexões persistentes. Quando o limite é atingido, a mais antiga no cache é fechada para evitar o aumento do número de conexões abertas. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.7.0.
CURLOPT_MAXFILESIZE (int)	Define o tamanho máximo aceito (em bytes) de um arquivo para recebimento. Se o arquivo solicitado for maior que esse valor, a transferência será abortada e **CURLE_FILESIZE_EXCEEDED** será retornado. Passar `0` desabilita esta opção, e passar um tamanho negativo retorna uma **CURLE_BAD_FUNCTION_ARGUMENT**. Se o tamanho do arquivo não for conhecido antes do início do recebimento, esta opção não terá efeito. Para definir um limite de tamanho acima de `2GB`, **CURLOPT_MAXFILESIZE_LARGE** deve ser usada. A partir do cURL 8.4.0, esta opção também interrompe as transferências em andamento se atingirem esse limite. Esta opção aceita qualquer valor que possa ser convertido em um int válido. O padrão é `0`. Disponível a partir do cURL 7.10.8.
CURLOPT_MAXFILESIZE_LARGE (int)	O tamanho máximo permitido para arquivo recebido. Se o arquivo requisitado for maior que este valor, a transferência não iniciará e **CURLE_FILESIZE_EXCEEDED** será retornada. O tamanho do arquivo nem sempre é conhecido antes da transferência, para esses casos esta opção não tem efeito mesmo que a transferência termine sendo maior que este limite informado. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do PHP 8.2.0 e cURL 7.11.0.

CURLOPT_MAXLIFETIME_CONN (int)	O tempo máximo em segundos, desde a criação da conexão, permitido para que uma conexão existente possa ser considerada para reúso. Se uma conexão no cache for mais antiga que este valor, ela será fechada assim que as transferências em progresso terminem. O padrão é `0` segundo, o que significa que a opção está desabilitada e todas as conexões são elegíveis para reúso. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do PHP 8.2.0 e cURL 7.80.0.
CURLOPT_MAXREDIRS (int)	A quantidade máxima de redirecionamentos HTTP a seguir. Esta opção deve ser usada junto com **CURLOPT_FOLLOWLOCATION**. O valor padrão de `20` está definido para evitar redirecionamentos infinitos. Definir para `-1` permite redirecionamentos infinitos e `0` recusa todos os redirecionamentos. Disponível a partir do cURL 7.5.0.
CURLOPT_MAX_RECV_SPEED_LARGE (int)	Se um recebimento exceder esta velocidade (medida em bytes por segundo) na média cumulativa durante a transferência, ela será pausada para manter a média abaixo ou igual ao valor do parâmetro. O padrão é velocidade ilimitada. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.15.5.
CURLOPT_MAX_SEND_SPEED_LARGE (int)	Se um envio exceder esta velocidade (medida em bytes por segundo) na média cumulativa durante a transferência, ela será pausada para manter a média abaixo ou igual ao valor do parâmetro. O padrão é velocidade ilimitada. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.15.5.
CURLOPT_MIME_OPTIONS (int)	Defina como uma máscara de bits das constantes **CURLMIMEOPT_***. Atualmente, há apenas uma opção disponível: **CURLMIMEOPT_FORMESCAPE**. Disponível a partir do PHP 8.3.0 e cURL 7.81.0.
CURLOPT_MUTE (int)	Defina como **true** para manter silêncio completo no que se refere às funções cURL. Use **CURLOPT_RETURNTRANSFER** em seu lugar. Disponível a partir do cURL 7.1.0, descontinuada a partir do cURL 7.8.0 e pela última vez disponível no cURL 7.15.5. Removida a partir do PHP 7.3.0.
CURLOPT_NETRC (int)	Defina como **true** para ler o arquivo *~/.netrc* buscando usuário e senha para o servidor remoto com o qual a conexão está sendo estabelecida. Disponível a partir do cURL 7.1.0.
CURLOPT_NETRC_FILE (int)	Define uma string contendo o nome completo do caminho para um arquivo `.netrc`. Se esta opção for omitida e **CURLOPT_NETRC** estiver definida, o cURL verificará se há um arquivo `.netrc` no diretório inicial do usuário atual. Disponível a partir do cURL 7.11.0.
CURLOPT_NEW_DIRECTORY_PERMS (int)	Configura o valor das permissões (int) definidas nos diretórios recém-criados no servidor remoto. O valor padrão é `0755`. Os únicos protocolos que podem usar isso são `sftp://`, `scp://` e `file://`. Disponível a partir do cURL 7.16.4.
CURLOPT_NEW_FILE_PERMS (int)	Configura o valor das permissões (como um int) que são definidas em arquivos recém-criados no servidor remoto. O valor padrão é `0644`. Os únicos protocolos que podem usar isso são `sftp://`, `scp://` e `file://`. Disponível a partir do cURL 7.16.4.
CURLOPT_NOBODY (int)	Defina como **true** para excluir o corpo da saída. Para HTTP(S), o cURL faz uma requisição HEAD. Para quase todos os outros protocolos, o cURL não solicita os dados do corpo de nenhuma forma. Alterar esta opção para **false** resultará na inclusão dos dados do corpo na saída. Disponível a partir do cURL 7.1.0.
CURLOPT_NOPROGRESS (int)	Defina como **true** para desabilitar a medição de progresso para transferências cURL. > **Nota** :

	> O PHP define automaticamente esta opção para `true`, ela só deve ser alterada para depuração. Disponível a partir do cURL 7.1.0.
`CURLOPT_NOPROXY` (int)	Define uma string que consiste em uma lista separada por vírgulas de nomes de servidor que não requerem um proxy para serem acessados. Cada nome nesta lista é correspondido como um domínio que contém o nome do servidor ou como o próprio nome do servidor. O único coringa disponível na string é um único caractere `*` que corresponde a todos os servidores, desativando efetivamente o proxy. Definir esta opção como uma string vazia habilita o proxy para todos os nomes de servidor. Desde o cURL 7.86.0, os endereços IP definidos com esta opção podem ser fornecidos usando a notação CIDR. Disponível a partir do cURL 7.19.4.
`CURLOPT_NOSIGNAL` (int)	`true` para ignorar qualquer função cURL que faça com que um sinal seja enviado ao processo PHP. Ligado por padrão em SAPIs multi-thread para que opções de limite de tempo ainda possam ser usadas. Disponível a partir do cURL 7.10.

`CURLOPT_PASSWDFUNCTION` (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(resource `$curlHandle`, string `$passwordPrompt`, int `$maximumPasswordLength`): string
	
	`curlHandle`
	O identificador cURL.
	
	`passwordPrompt`
	Uma string contendo um texto solicitando senha.
	
	`maximumPasswordLength`
	Comprimento máximo da senha.
	
	A função deve retornar uma string contendo a senha. Disponível a partir do cURL 7.4.2, descontinuada a partir do cURL 7.11.1 e pela última vez disponível no cURL 7.15.5. Removida a partir do PHP 7.3.0.

`CURLOPT_PASSWORD` (int)	Aceita uma string com a senha a ser usada na autenticação. Disponível a partir do cURL 7.19.1.
`CURLOPT_PATH_AS_IS` (int)	Defina como `true` para que o cURL não altere caminhos URL antes de encaminhá-los ao servidor. O padrão é `false`, que resolve as expressões `/../` ou `/./` que possam existir na parte do caminho da URL que deveriam ser removidas de acordo com a RFC 3986 seção 5.2.4. Disponível a partir do PHP 7.0.7 e cURL 7.42.0.
`CURLOPT_PINNEDPUBLICKEY` (int)	Define uma string com a chave pública fixada. A string pode ser o nome do arquivo da chave pública fixada no formato PEM ou DER. A string também pode ser qualquer número de hashes sha256 codificadas em base 64 precedidas por `sha256//` e separadas por `;`. Disponível a partir do PHP 7.0.7 e cURL 7.39.0.
`CURLOPT_PIPEWAIT` (int)	Defina como `true` para esperar que uma conexão existente confirme que pode fazer multiplexação e fazer uso dela, antes de criar e utilizar uma nova conexão. Disponível a partir do PHP 7.0.7 e cURL 7.43.0
`CURLOPT_PORT` (int)	Um int com um número alternativo de porta para conexão ao invés da porta especificada na URL ou da porta padrão para o protocolo usado. Disponível a partir do cURL 7.1.0.
`CURLOPT_POST` (int)	Defina como `true` para fazer uma requisição `POST` HTTP. Esta requisição usa o cabeçalho `application/x-www-form-urlencoded`. O padrão é `false`. Disponível a partir do cURL 7.1.0.

CURLOPT_POSTFIELDS (int)	Os dados completos para enviar em uma operação POST HTTP. Este parâmetro pode ser passado como uma string codificada em URL como '`para1=val1&para2=val2&...`' ou como um array com o nome do campo como chave e o dado do campo como valor. Se `value` for um array, o cabeçalho `Content-Type` será definido para `multipart/form-data`. Arquivos podem ser enviados usando CURLFile ou CURLStringFile, onde `value` precisa ser um array. Disponível a partir do cURL 7.1.0.
CURLOPT_POSTQUOTE (int)	Um array de strings de comandos FTP a executar no servidor depois que a requisição FTP tenha sido realizada. Disponível a partir do cURL 7.1.0.
CURLOPT_POSTREDIR (int)	Uma máscara de bits de **CURL_REDIR_POST_301**, **CURL_REDIR_POST_302** e **CURL_REDIR_POST_303** se o método POST HTTP deve ser mantido quando **CURLOPT_FOLLOWLOCATION** estiver definida e um tipo específico de redirecionamento ocorrer. Disponível a partir do cURL 7.19.1.
CURLOPT_PRE_PROXY (int)	Define uma string contendo o nome do servidor ou endereço IP numérico com pontos a ser usado como o pré-proxy ao qual o cURL se conecta antes de se conectar ao proxy HTTP(S) especificado na opção **CURLOPT_PROXY** para a próxima requisição. O pré-proxy somente pode ser um proxy SOCKS e deve ser prefixado com `[esquema]://` para especificar que tipo de socks é utilizado. Um endereço numérico IPv6 precisa ser escrito entre [colchetes]. Definir o pré-proxy para uma string vazia desabilita explicitamente o uso de um pré-proxy. Para especificar o número da porta nesta string, adicione `:[porta]` ao final do nome do servidor. O número da porta do proxy opcionalmente pode ser especificado coma opção separada **CURLOPT_PROXYPORT**. O padrão é usar a porta 1080 para proxies se não for especificada. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PREQUOTE (int)	Define um array de strings de comandos FTP a serem passados ao servidor depois que o tipo de transferência for definido. Esses comandos não são executados quando uma listagem de diretórios é realizada, apenas para transferências de arquivos. Disponível a partir do cURL 7.9.5.
CURLOPT_PRIVATE (int)	Quaisquer dados que devem ser associados com este identificador cURL. Estes dados podem ser recuperados subsequentemente com a opção **CURLINFO_PRIVATE** de curl_getinfo(). cURL não faz nada com estes dados. Ao usar um multi-identificador cURL, estes dados privados são tipicamente uma chave única para identificar um identificador cURL padrão. Disponível a partir do cURL 7.10.3.

CURLOPT_PROGRESSFUNCTION (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(
	  resource `$curlHandle`,
	  int `$bytesToDownload`,
	  int `$bytesDownloaded`,
	  int `$bytesToUpload`,
	  int `$bytesUploaded`
	): int
	
	`curlHandle`
	O identificador cURL.
	
	`bytesToDownload`
	O número total de bytes esperados no recebimento desta transferência.
	
	`bytesDownloaded`
	O número de bytes já recebidos.
	
	`bytesToUpload`
	O número total de bytes esperados no envio desta transferência.

	`bytesUploaded` O número de bytes já enviados. A função deve retornar um int com um valor diferente de zero para abortar a transferência e definir um erro **`CURLE_ABORTED_BY_CALLBACK`**. > **Nota** : > > A função de retorno é chamada somente quando a opção **`CURLOPT_NOPROGRESS`** estiver definida para **`false`**. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.32.0. Use **`CURLOPT_XFERINFOFUNCTION`** em seu lugar.
`CURLOPT_PROTOCOLS` (int)	Uma máscara de bits de valores **`CURLPROTO_*`**. Se usada, esta máscara limita que protocolos a libcurl pode usar na transferência. O padrão é **`CURLPROTO_ALL`**, ou seja, o cURL aceitará todos os protocolos suportados. Consulte também **`CURLOPT_REDIR_PROTOCOLS_STR`**. Disponível a partir do cURL 7.19.4 e descontinuada a partir do cURL 7.85.0.
`CURLOPT_PROTOCOLS_STR` (int)	Defina para uma string com uma lista de nomes de protocolos (esquemas de URL) a serem permitidos na transferência, separados por vírgula, insensíveis a maiúsculas/minúsculas. Defina como `ALL` para habilitar todos os protocolos. Por padrão, o cURL aceita todos os protocolos com os quais foi compilado para suportar. Os protocolos disponíveis são: `DICT`, `FILE`, `FTP`, `FTPS`, `GOPHER`, `GOPHERS`, `HTTP`, `HTTPS`, `IMAP`, `IMAPS`, `LDAP`, `LDAPS`, `MQTT`, `POP3`, `POP3S`, `RTMP`, `RTMPE`, `RTMPS`, `RTMPT`, `RTMPTE`, `RTMPTS`, `RTSP`, `SCP`, `SFTP`, `SMB`, `SMBS`, `SMTP`, `SMTPS`, `TELNET`, `TFTP`, `WS`, `WSS`. Disponível a partir do PHP 8.3.0 e do cURL 7.85.0.
`CURLOPT_PROXY` (int)	Uma string com o proxy HTTP através do qual as requisições serão enviadas. Deve ser o nome do host, o endereço IP numérico com pontos ou um endereço IPv6 numérico escrito entre [colchetes]. Disponível a partir do cURL 7.1.0.
`CURLOPT_PROXYAUTH` (int)	Uma máscara de bits dos método(s) de autenticação (constantes **`CURLAUTH_*`**) para usar na conexão ao proxy. Para autenticação no proxy, somente **`CURLAUTH_BASIC`** e **`CURLAUTH_NTLM`** são suportadas atualmente. O padrão é **`CURLAUTH_BASIC`**. Disponível a partir do cURL 7.10.7.
`CURLOPT_PROXYHEADER` (int)	Um array de strings de cabeçalhos HTTP customizados para passar aos proxies. Disponível a partir do PHP 7.0.7 e cURL 7.37.0.
`CURLOPT_PROXYPASSWORD` (int)	Define uma string com a senha a ser usada para autenticação com o proxy. Disponível a partir do cURL 7.19.1.
`CURLOPT_PROXYPORT` (int)	Um int com o número da porta do proxy para conexão. Este número de porta também pode ser definido em **`CURLOPT_PROXY`**. Definir como zero faz com que o cURL use o número de porta do proxy padrão ou o número de porta especificado na string do URL do proxy. Disponível a partir do cURL 7.1.0.
`CURLOPT_PROXYTYPE` (int)	Define o tipo do proxy, que pode ser uma das constantes **`CURLPROXY_*`**. O padrão é **`CURLPROXY_HTTP`**. Disponível a partir do cURL 7.10.
`CURLOPT_PROXYUSERNAME` (int)	Define uma string com o nome de usuário a ser usado para autenticação com o proxy. Disponível a partir do cURL 7.19.1.
`CURLOPT_PROXYUSERPWD` (int)	Uma string com usuário e senha formatados como `[usuário]:[senha]` para usar na conexão ao proxy. Disponível a partir do cURL 7.1.0.

CURLOPT_PROXY_CAINFO (int)	O caminho para o pacote de Certificados de Autoridade (CA) do proxy. Defina o caminho como uma string com o nome do arquivo que contém um ou mais certificados com os quais o prohy HTTPS será verificado. Esta opção é para conexão a um proxy HTTPS, não a um servidor HTTPS. O padrão é o caminho do sistema onde espera-se que o pacote de certificados CA do cURL esteja armazenado. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_CAINFO_BLOB (int)	Uma string com o nome de um arquivo PEM contendo um ou mais certificados com os quais o proxy HTTPS será verificado. Esta opção é para conexão a um proxy HTTPS, não a um servidor HTTPS. O padrão é o caminho do sistema onde espera-se que o pacote de certificados CA do cURL esteja armazenado. Disponível a partir do PHP 8.2.0 e cURL 7.77.0.
CURLOPT_PROXY_CAPATH (int)	Uma string com o diretório que contém múltiplos certificados CA com os quais o proxy HTTPS será verificado. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_CRLFILE (int)	Defina como uma string com o nome do arquivo com a concatenação de CRL (Lista de Revogação de Certificado) no formato PEM a ser usado na validação de certificado que ocorre durante a troca de informações SSL. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_ISSUERCERT (int)	Uma string com o nome do arquivo do certificado SSL do emissor do proxy. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_PROXY_ISSUERCERT_BLOB (int)	Uma string com o certificado SSL do emissor do proxy. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_PROXY_KEYPASSWD (int)	Define a string a ser usada como a senha necessária para usar a chave privada **CURLOPT_PROXY_SSLKEY**. Nunca é necessária uma senha para carregar um certificado, mas será necessária uma para carregar a chave privada. Esta opção é para conexão a um proxy HTTPS, não a um servidor HTTPS. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_PINNEDPUBLICKEY (int)	Define a chave pública fixada para o proxy HTTPS. A string pode ser o nome do arquivo da chave pública fixada com formato esperado de PEM ou DER. A string também pode ser qualquer número de hashes sha256 codificados em base64 precedidas por `sha256//` e separadas por `;`. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SERVICE_NAME (int)	Uma string com o nome do serviço de autenticação de proxy. Disponível a partir do PHP 7.0.7, cURL 7.43.0 (para proxies HTTP) e cURL 7.49.0 (para proxies SOCKS5).
CURLOPT_PROXY_SSLCERT (int)	Uma string com o nome do arquivo do certificado de cliente usado para conectar-se ao proxy HTTPS. O formato padrão é P12 no Transporte Seguro e PEM em outros mecanismos, e pode ser alterado com **CURLOPT_PROXY_SSLCERTTYPE**. Com NSS ou Transporte Seguro, este também pode ser o apelido do certificado usado para autenticação, conforme nomeado no banco de dados de segurança. Se for pretendido usar um arquivo do diretório atual, deve ser precedido por `./` para evitar confusão com um apelido. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSLCERTTYPE (int)	Uma string com o formato do certificado de cliente usado ao conectar-se a um proxy HTTPS. Os formatos suportados são PEM e DER, exceto com Transporte Seguro. OpenSSL (versões 0.9.3 e posteriores) e Transporte Seguro (no iOS 5 ou posterior, ou OS X 10.7 ou posterior) também suportam P12 para arquivos codificados em PKCS#12. O padrão é PEM. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSLCERT_BLOB (int)	Uma string com o certificado de cliente do proxy SSL. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.

CURLOPT_PROXY_SSLKEY (int)	Uma string com o nome do arquivo da chave privada usada para conexão com o proxy HTTPS. O formato padrão é PEM e pode ser alterado com **CURLOPT_PROXY_SSLKEYTYPE**. (somente iOS e Mac OS X) Esta opção será ignorada se o cURL tiver sido compilado com Transporte Seguro. Disponível se compilado com TLS habilitado. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSLKEYTYPE (int)	Uma string com o formato da chave privada. Os formatos suportados são: PEM, DER, ENG. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSLKEY_BLOB (int)	Uma string com a chave privada para conexão ao proxy HTTPS. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_PROXY_SSLVERSION (int)	Define a versão do TLS do proxy HTTPS para uma das constantes **CURL_SSLVERSION_***. O padrão é **CURL_SSLVERSION_DEFAULT**. **Aviso** É melhor não definir esta opção e deixar o padrão **CURL_SSLVERSION_DEFAULT** que tentará descobrir a versão do protocolo SSL remoto. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSL_CIPHER_LIST (int)	Uma string com a lista de cifras, separadas por dois-pontos (:), a serem usadas para a conexão com o proxy HTTPS. Quando usada com OpenSSL, vírgulas ou espaços também são separadores aceitáveis e !, - e + podem ser usados como operadores. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSL_OPTIONS (int)	Define as opções de comportamento do proxy SSL, que é uma máscara de bits das constantes **CURLSSLOPT_***. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSL_VERIFYHOST (int)	Defina como 2 para verificar os campos de nome do certificado do proxy HTTPS em relação ao nome do proxy. Quando definido como 0 a conexão é bem-sucedida independentemente dos nomes usados no certificado. Use essa capacidade com cuidado! Se definida como 1 no cURL 7.28.0 e versões anteriores, serve como uma opção de depuração. Se definida como 1 no cURL 7.28.1 ao 7.65.3, a constante **CURLE_BAD_FUNCTION_ARGUMENT** é retornada. A partir do cURL 7.66.0, 1 e 2 são tratados como o mesmo valor. O padrão é 2. Em ambientes de produção, o valor desta opção deve ser mantido em 2. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_SSL_VERIFYPEER (int)	Defina como **false** para impedir que o cURL verifique o certificado do par. Certificados alternativos para verificação podem ser especificados com a opção **CURLOPT_CAINFO** ou um diretório de certificado pode ser especificado com a opção **CURLOPT_CAPATH**. Quando definida como **false**, a verificação do certificado será sempre bem-sucedida. **true** é o padrão. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_TLS13_CIPHERS (int)	Uma string com a lista de conjuntos de criptografia a serem usados para a conexão TLS 1.3 com um proxy. Esta opção é usada atualmente apenas quando o cURL é compilado para usar OpenSSL 1.1.1 ou posterior. Se um SSL diferente estiver sendo usado, os conjuntos de criptografia TLS 1.3 podel ser configurados com a opção **CURLOPT_PROXY_SSL_CIPHER_LIST**. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
CURLOPT_PROXY_TLSAUTH_PASSWORD (int)	Uma string com a senha a ser usada para o método de autenticação TLS especificado com a opção **CURLOPT_PROXY_TLSAUTH_TYPE**. Requer que a opção **CURLOPT_PROXY_TLSAUTH_USERNAME** também esteja definida. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLOPT_PROXY_TLSAUTH_TYPE (int)	O método de autenticação TLS usado para a conexão HTTPS. O método suportado é SRP.

	> **Nota** : > A autenticação Secure Remote Password (SRP) para TLS fornece autenticação mútua se ambos os lados tiverem um segredo compartilhado. Para usar TLS-SRP, as opções `CURLOPT_PROXY_TLSAUTH_USERNAME` e `CURLOPT_PROXY_TLSAUTH_PASSWORD` também precisam ser definidas. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
`CURLOPT_PROXY_TLSAUTH_USERNAME` (int)	O usuário a ser usado para o método de autenticação TLS especificado com a opção `CURLOPT_PROXY_TLSAUTH_TYPE`. Requer que a opção `CURLOPT_PROXY_TLSAUTH_PASSWORD` também esteja definida. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
`CURLOPT_PROXY_TRANSFER_MODE` (int)	Quando igual a `1`, define o modo de transferência (binário ou ASCII) para transferências FTP feitas através de um proxy HTTP, anexando `type=a` ou `type=i` ao URL. Sem esta configuração ou definida como `0`, `CURLOPT_TRANSFERTEXT` não tem efeito ao fazer FTP através de um proxy. O padrão é `0`. Disponível a partir do cURL 7.18.0.
`CURLOPT_PUT` (int)	`true` para usar o método PUT HTTP para um arquivo. O arquivo precisa ser definido com `CURLOPT_READDATA` e `CURLOPT_INFILESIZE`. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.12.1.
`CURLOPT_QUICK_EXIT` (int)	Defina como `true` para o cURL ignorar a etapa de limpeza de recursos ao recuperar de um limite de tempo excedido. Isto permite um encerramento rápido do processo cURL às custas de um possível vazamento de recursos associados. Disponível a partir do PHP 8.3.0 e cURL 7.87.0.
`CURLOPT_QUOTE` (int)	Um array de strings de comandos FTP a executar no servidor antes da requisição FTP. Disponível a partir do cURL 7.1.0.
`CURLOPT_RANDOM_FILE` (int)	Uma string com o nome do arquivo a ser usado para semear o gerador de números aleatórios para o SSL. Disponível a partir do cURL 7.7.0 e descontinuada a partir do cURL 7.84.0.
`CURLOPT_RANGE` (int)	Uma string com o(s) intervalo(s) de dados a serem recuperados no formato `X-Y` onde X ou Y são opcionais. Transferências HTTP também suportam vários intervalos, separados por vírgulas no formato `X-Y,N-M`. Defina como `null` para desabilitar a requisição de um intervalo de bytes. Disponível a partir do cURL 7.1.0.
`CURLOPT_READDATA` (int)	Define um ponteiro de arquivo do tipo resource que será usado pela função de leitura de arquivo definida com `CURLOPT_READFUNCTION`. Disponível a partir do cURL 7.9.7.

`CURLOPT_READFUNCTION` (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(resource `$curlHandle`, resource `$streamResource`, int `$maxAmountOfDataToRead`): string
	
	`curlHandle`
	
	O identificador cURL.
	
	`streamResource`
	
	resource de fluxo fornecido ao cURL através da opção `CURLOPT_READDATA`.
	
	`maxAmountOfDataToRead`
	
	A quantidade máxima de dados a serem lidos.

	A função deve retornar uma string com comprimento igual ou menor que a quantidade de dados solicitados, normalmente lendo-a do resource de fluxo passado. Deve retornar uma string vazia para sinalizar EOF. Disponível a partir do cURL 7.1.0.
CURLOPT_REDIR_PROTOCOLS (int)	Máscara de bits de valores **CURLPROTO_*** que limita quais protocolos o cURL pode usar em uma transferência que segue um redirecionamento quando **CURLOPT_FOLLOWLOCATION** está habilitado. Isso permite limitar transferências específicas para usar apenas um subconjunto de protocolos em redirecionamentos. A partir do cURL 7.19.4, por padrão o cURL permitirá todos os protocolos, exceto FILE e SCP. Antes do cURL 7.19.4, o cURL seguiria incondicionalmente todos os protocolos suportados. Veja também **CURLOPT_PROTOCOLS** para valores constantes de protocolo. Disponível a partir do cURL 7.19.4 e descontinuada a partir do cURL 7.85.0.
CURLOPT_REDIR_PROTOCOLS_STR (int)	Defina como uma string com uma lista de nomes de protocolos (esquemas URL), insensíveis a maiúsculas/minúsculas, separados por vírgulas, para permitir o seguimento de um redirecionamento quando **CURLOPT_FOLLOWLOCATION** estiver habilitada. Defina como ALL para habilitar todos os protocolos. A partir do cURL 7.65.2, o padrão é FTP, FTPS, HTTP e HTTPS. Do cURL 7.40.0 até o 7.65.1, o padrão são todos os protocolos exceto FILE, SCP, SMB e SMBS. Antes do cURL 7.40.0, o padrão são todos os protocolos exceto FILE e SCP. Os protocolos disponíveis são: DICT, FILE, FTP, FTPS, GOPHER, GOPHERS, HTTP, HTTPS, IMAP, IMAPS, LDAP, LDAPS, MQTT, POP3, POP3S, RTMP, RTMPE, RTMPS, RTMPT, RTMPTE, RTMPTS, RTSP, SCP, SFTP, SMB, SMBS, SMTP, SMTPS, TELNET, TFTP, WS, WSS. Disponível a partir do PHP 8.3.0 e cURL 7.85.0.
CURLOPT_REFERER (int)	Uma string com o conteúdo do cabeçalho `Referer:` a ser usado em uma requisição HTTP. Disponível a partir do cURL 7.1.0.
CURLOPT_REQUEST_TARGET (int)	Uma string a ser usada na próxima requisição em vez do caminho extraído do URL. Disponível a partir do PHP 7.3.0 e cURL 7.55.0.
CURLOPT_RESOLVE (int)	Forneça um array de strings com endereços personalizados separados por dois-pontos para um par específico de servidor e porta no formato: `array( "example.com:80:127.0.0.1", "example2.com:443:127.0.0.2", )` Disponível a partir do cURL 7.21.3.
CURLOPT_RESUME_FROM (int)	O deslocamento, em bytes, a partir do qual uma transferência deve ser retomada. Esta opção aceita qualquer valor que possa ser convertido para um int válido. Disponível a partir do cURL 7.1.0.
CURLOPT_RETURNTRANSFER (int)	**true** para retornar a transferência como uma string do valor de retorno de curl_exec() em vez de enviá-la diretamente.
CURLOPT_RTSP_CLIENT_CSEQ (int)	Define um int com o número CSEQ a ser emitido para a próxima solicitação RTSP. Útil se o aplicativo estiver retomando uma conexão interrompida anteriormente. O CSEQ é inrementado a partir deste novo número. O padrão é `0`. Disponível a partir do cURL 7.20.0.
CURLOPT_RTSP_REQUEST (int)	Define o tipo de solicitação RTSP a ser feita. Deve ser uma das constantes **CURL_RTSPREQ_***. Disponível a partir do cURL 7.20.0.
CURLOPT_RTSP_SERVER_CSEQ (int)	Define um int com o número CSEQ esperado para a próxima solicitação do servidor RTSP para o cliente. Este recurso (escutar solicitações do servidor) não está implementado. O padrão é `0`. Disponível a partir do cURL 7.20.0.
CURLOPT_RTSP_SESSION_ID (int)	Defina uma string com o valor do ID da sessão RTSP atual para o identificador. Depois que esse valor for definido como qualquer valor não **null**, o cURL retornará **CURLE_RTSP_SESSION_ERROR** se o ID recebido do servidor não corresponder. Se definido como **null**, o cURL define

	automaticamente o ID na primeira vez que o servidor o define em uma resposta. O padrão é `null`. Disponível a partir do cURL 7.20.0.
`CURLOPT_RTSP_STREAM_URI` (int)	Define uma string com o URI do fluxo para operar. Se não for definido, o padrão do cURL é operar em opções genéricas do servidor, passando `*` no lugar do URI do fluxo RTSP. Ao trabalhar com RTSP, `CURLOPT_RTSP_STREAM_URI` indica qual URL enviar ao servidor no cabeçalho da solicitação enquanto `CURLOPT_URL` indica onde a conexão deve ser feita. Disponível a partir do cURL 7.20.0.
`CURLOPT_RTSP_TRANSPORT` (int)	Define o cabeçalho `Transport:` para esta sessão RTSP. Disponível a partir do cURL 7.20.0.
`CURLOPT_SAFE_UPLOAD` (int)	Sempre `true`, o que desativa o suporte ao prefixo `@` para envio de arquivos em `CURLOPT_POSTFIELDS`, o que significa que valores começando com `@` podem ser passados com segurança como campos. CURLFile pode ser usada para envios.
`CURLOPT_SASL_AUTHZID` (int)	A string de identidade de autorização (autzid) para a transferência. Aplicável apenas ao mecanismo de autenticação PLAIN SASL onde é opcional. Quando não especificada, apenas a identidade de autenticação (authcid) especificada pelo nome de usuário será enviada ao servidor, juntamente com a senha. O servidor derivará o authzid do authcid quando não for fornecido, que será usado internamente. Disponível a partir do PHP 8.2.0 e cURL 7.66.0.
`CURLOPT_SASL_IR` (int)	`true` para habilitar o envio da resposta inicial no primeiro pacote. Disponível a partir do PHP 7.0.7 e cURL 7.31.0.
`CURLOPT_SERVICE_NAME` (int)	Uma string com o nome do serviço de autenticação. Disponível a partir do PHP 7.0.7 e cURL 7.43.0.
`CURLOPT_SHARE` (int)	Um resultado de curl_share_init(). Faz com que o identificador cURL use os dados do identificador compartilhado. Disponível a partir do cURL 7.10.
`CURLOPT_SOCKS5_AUTH` (int)	Os métodos de autenticação SOCKS5 a serem utilizados. As opções são: `CURLAUTH_BASIC`, `CURLAUTH_GSSAPI`, `CURLAUTH_NONE`. Quando mais de um método estiver definido, o cURL irá interrogar o servidor para ver que métodos ele suporta, e escolherá o melhor. O padrão é `CURLAUTH_BASIC|CURLAUTH_GSSAPI`. Defina o usuário e senha reais com a opção `CURLOPT_PROXYUSERPWD`. Disponível a partir do PHP 7.3.0 e cURL 7.55.0.
`CURLOPT_SOCKS5_GSSAPI_NEC` (int)	Definida como `1` habilita, e como `0` desabilita, a troca desprotegida da negociação do modo de proteção como parte da negociação GSSAPI. Disponível a partir do cURL 7.19.4.
`CURLOPT_SOCKS5_GSSAPI_SERVICE` (int)	Define uma string contendo o nome do serviço SOCKS5. O padrão é `rcmd` Disponível a partir do cURL 7.19.4 e descontinuada a partir do cURL 7.49.0. Use `CURLOPT_PROXY_SERVICE_NAME` em seu lugar.
`CURLOPT_SSH_AUTH_TYPES` (int)	Uma máscara de bits que consiste de uma ou mais das seguintes constantes: `CURLSSH_AUTH_PUBLICKEY`, `CURLSSH_AUTH_PASSWORD`, `CURLSSH_AUTH_HOST`, `CURLSSH_AUTH_KEYBOARD`, `CURLSSH_AUTH_AGENT`, `CURLSSH_AUTH_ANY`. O padrão é `CURLSSH_AUTH_ANY`. Disponível a partir do cURL 7.16.1.
`CURLOPT_SSH_COMPRESSION` (int)	`true` para habilitar, `false` para desabilitar a compactação SSH interna. Observe que o servidor pode desconsiderar esta requisição. O padrão é `false` Disponível a partir do PHP 7.3.0 e cURL 7.56.0.

CURLOPT_SSH_HOSTKEYFUNCTION (int)	Uma função do tipo callable que será chamada quando a verificação de chave do host SSH for necessária. A função precisa ter a seguinte assinatura:
	
	function callback(
	  resource $curlHandle,
	  int $keyType,
	  string $key,
	  int $keyLength
	): int
	
	`curlHandle`
	O identificador cURL.
	
	`keyType`
	Um dos tipos de chave **CURLKHTYPE_***.
	
	`key`
	A chave a ser verificada.
	
	`keyLength`
	O comprimento da chave em bytes.
	
	Esta função substitui **CURLOPT_SSH_KNOWNHOSTS**. Disponível a partir do PHP 8.3.0 e cURL 7.84.0.

CURLOPT_SSH_HOST_PUBLIC_KEY_MD5 (int)	Uma string contendo 32 dígitos hexadecimais que deve conter a soma de verificação MD5 da chave pública do servidor remoto, e o cURL rejeitará a conexão com o servidor, a menos que as somas correspondam. Esta opção é apenas para transferências SCP e SFTP. Disponível a partir do cURL 7.17.1.
CURLOPT_SSH_HOST_PUBLIC_KEY_SHA256 (int)	Uma string com o hash SHA256 codificado em base64 da chave pública do servidor remoto. A transferência falhará se o hash fornecido não corresponder ao hash fornecido pelo servidor remoto. Disponível a partir do PHP 8.2.0 e cURL 7.80.0.
CURLOPT_SSH_KNOWNHOSTS (int)	Deve ser definida como o nome do arquivo *known_host* a ser usado, que deve usar o formato de arquivo OpenSSH conforme suportado pela libssh2. Disponível a partir do cURL 7.19.6.
CURLOPT_SSH_PRIVATE_KEYFILE (int)	O nome do arquivo com a chave privada. Se não for usado, o padrão do cURL é *$HOME/.ssh/id_dsa* se a variável de ambiente HOME estiver definida, e apenas `id_dsa` no diretório atual se HOME não estiver definida. Se o arquivo estiver protegido por senha, defina a senha com **CURLOPT_KEYPASSWD**. Disponível a partir do cURL 7.16.1.
CURLOPT_SSH_PUBLIC_KEYFILE (int)	O nome do arquivo com a chave pública. Se não for usado, o padrão do cURL é *$HOME/.ssh/id_dsa.pub* se a variável de ambiente HOME estiver definida, e apenas `id_dsa.pub` no diretório atual se HOME não estiver definida. Disponível a partir do cURL 7.16.1.
CURLOPT_SSLCERT (int)	O nome de um arquivo que contém um certificado no formato PEM. Disponível a partir do cURL 7.1.0.
CURLOPT_SSLCERTPASSWD (int)	A senha requerida para usar o certificado **CURLOPT_SSLCERT**. Disponível a partir do cURL 7.1.0 e descontinuada a partir do cURL 7.17.0.
CURLOPT_SSLCERTTYPE (int)	Uma string com o formato do certificado. Os formatos suportados são: `PEM`, `DER`, `ENG`, `P12`. A partir do OpenSSL 0.9.3, o formato `P12` (para arquivos codificados em PKCS#12) está disponível. O padrão é `PEM`. Disponível a partir do cURL 7.9.3.

CURLOPT_SSLCERT_BLOB (int)	Uma string com o certificado de cliente SSL do blob de memória. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_SSLENGINE (int)	Uma string com o identificador do mecanismo de criptografia da chave SSL privada especificada em **CURLOPT_SSLKEY**. Disponível a partir do cURL 7.9.3.
CURLOPT_SSLENGINE_DEFAULT (int)	Uma string com o identificador do mecanismo de criptografia usado para operações de criptografia assimétricas. Disponível a partir do cURL 7.9.3.
CURLOPT_SSLKEY (int)	O nome de um arquivo que contém uma chave SSL privada. Disponível a partir do cURL 7.9.3.
CURLOPT_SSLKEYPASSWD (int)	A senha secreta necessária para usar a chave SSL privada especificada em **CURLOPT_SSLKEY**. > **Nota** : > Como esta opção contém uma senha confidencial, lembre-se de manter seguro o script PHP que a contém. Disponível a partir do cURL 7.9.3 e descontinuada a partir do cURL 7.17.0.
CURLOPT_SSLKEYTYPE (int)	O tipo de chave da chave SSL privada especificada em **CURLOPT_SSLKEY**. Tipos de chave suportados são: PEM, DER, ENG. O padrão é PEM. Disponível a partir do cURL 7.9.3.
CURLOPT_SSLKEY_BLOB (int)	Uma string com a chave privada para certificado de cliente do blob de memória. Disponível a partir do PHP 8.1.0 e cURL 7.71.0.
CURLOPT_SSLVERSION (int)	Uma das constantes **CURL_SSLVERSION_***. > **Aviso** > É melhor não definir esta opção e deixar no padrão, já que definir para **CURL_SSLVERSION_SSLv2** ou **CURL_SSLVERSION_SSLv3** é bastante perigoso, dadas as vulnerabilidades conhecidas no SSLv2 e no SSLv3. O padrão é **CURL_SSLVERSION_DEFAULT**. Disponível a partir do cURL 7.1.0.
CURLOPT_SSL_CIPHER_LIST (int)	Uma string com uma lista de cifras separadas por dois-pontos, a serem usadas para a conexão TLS 1.2 (1.1, 1.0). Disponível a partir do cURL 7.9.
CURLOPT_SSL_EC_CURVES (int)	Uma lista delimitada por dois pontos de algoritmos de curva elíptica. Por exemplo, `X25519:P-521` é uma lista válida de duas curvas elípticas. Esta opção define os algoritmos de troca de chaves do cliente na comunicação inicial do SSL, se o SSL com o qual o cURL foi compilado suportá-los. Disponível a partir do PHP 8.2.0 e cURL 7.73.0.
CURLOPT_SSL_ENABLE_ALPN (int)	**false** para desabilitar o ALPN na comunicação inicial do SSL (se o SSL com o qual o cURL foi compilado suportá-lo), que pode ser usado para negociar HTTP2. Disponível a partir do PHP 7.0.7 e do cURL 7.36.0 e descontinuada a partir do cURL 7.86.0.
CURLOPT_SSL_ENABLE_NPN (int)	**false** para desabilitar o NPN na comunicação inicial do SSL (se o SSL com o qual a libcurl foi compilada suportá-lo), que pode ser usado para negociar HTTP2. Disponível a partir do PHP 7.0.7 e cURL 7.36.0.

CURLOPT_SSL_FALSESTART (int)	**true** para habilitar e **false** para desabilitar o início falso do TLS que é um modo onde um cliente TLS inicia o envio de dados da aplicação antes de verificar a mensagem `Finished` do servidor. Disponível a partir do PHP 7.0.7 e cURL 7.42.0. Descontinuado desde o cURL 8.15.0.
CURLOPT_SSL_OPTIONS (int)	Define as opções de comportamento do SSL, que é uma máscara de bits das constantes **CURLSSLOPT_***. O padrão é nenhum bit definido. Disponível a partir do PHP 7.0.7. e cURL 7.25.0.
CURLOPT_SSL_SESSIONID_CACHE (int)	Definida como `0` desativa, e como `1` ativa o cache de ID de sessão SSL. Por padrão todas as transferências são feitas usando o cache habilitado. Disponível a partir do cURL 7.16.0.
CURLOPT_SSL_VERIFYHOST (int)	`2` para verificar que um campo "Common Name" ou um campo "Subject Alternate Name" no certificado SSL corresponde ao nome de servidor informado. `0` para não verificar os nomes. `1` não deve ser usado. Em ambientes de produção o valor para esta opção deve ser mantido em `2` (valor padrão). Suporte para o valor `1` foi removido no cURL 7.28.1. Disponível a partir do cURL 7.8.1.
CURLOPT_SSL_VERIFYPEER (int)	**false** para impedir o cURL de verificar o certificado do servidor. Certificados alternativos para verificar podem ser especificados com a opção **CURLOPT_CAINFO** ou um diretório de certificados pode ser especificado com a opção **CURLOPT_CAPATH**. **true** é o padrão a partir do cURL 7.10. Pacote padrão de certificados CA instalado a partir do cURL 7.10. Disponível a partir do cURL 7.4.2.
CURLOPT_SSL_VERIFYSTATUS (int)	**true** para habilitar e **false** para desabilitar a verificação do status do certificado. Disponível a partir do PHP 7.0.7 e cURL 7.41.0.
CURLOPT_STDERR (int)	Aceita um identificador de arquivo do tipo resource apontando para uma localização alternativa para enviar os erros no lugar de **STDERR**. Disponível a partir do cURL 7.1.0.
CURLOPT_STREAM_WEIGHT (int)	Define o peso numérico do fluxo (um número entre `1` e `256`). Disponível a partir do PHP 7.0.7 e cURL 7.46.0.
CURLOPT_SUPPRESS_CONNECT_HEADERS (int)	**true** para suprimir cabeçalhos de resposta CONNECT do proxy a partir das funções de retorno de usuário **CURLOPT_HEADERFUNCTION** e **CURLOPT_WRITEFUNCTION**, quando **CURLOPT_HTTPPROXYTUNNEL** é usada e uma requisição CONNECT é realizada. O padrão é **false**. Disponível a partir do PHP 7.3.0 e cURL 7.54.0.
CURLOPT_TCP_FASTOPEN (int)	**true** para habilitar e **false** para desabilitar a extensão "Fast Open" (abertura rápida) do TCP. Disponível a partir do PHP 7.0.7 e cURL 7.49.0.
CURLOPT_TCP_KEEPALIVE (int)	Se definida para `1`, sondas de manutenção de atividade serão enviadas. O atraso e a frequência destas sondas podem ser controlados pelas opções **CURLOPT_TCP_KEEPIDLE** e **CURLOPT_TCP_KEEPINTVL**, desde que o sistema operacional as suporte. Se definida para `0` (padrão), as sondas são desabilitadas. O número máximo de sondas pode ser definido com a opção **CURLOPT_TCP_KEEPCNT**. Disponível a partir do cURL 7.25.0.
CURLOPT_TCP_KEEPIDLE (int)	Define o atraso, em segundos, que o sistema operacional aguardará enquanto a conexão estiver ociosa antes de enviar sondas de manutenção de atividade, se **CURLOPT_TCP_KEEPALIVE** estiver habilitada. Nem todos os sistemas operacionais suportam esta opção. O padrão é `60`. Disponível a partir do cURL 7.25.0.
CURLOPT_TCP_KEEPINTVL (int)	Define o intervalo, em segundos, que o sistema operacional aguardará entre o envio de sondas de manutenção de atividade, se **CURLOPT_TCP_KEEPALIVE** estiver habilitada. Nem todos os sistemas operacionais suportam esta opção. O padrão é `60`. Disponível a partir do cURL 7.25.0.

CURLOPT_TCP_KEEPCNT (int)	Define o número máximo de sondas de verificação de atividade TCP. O padrão é 9. Disponível a partir do PHP 8.4.0 e cURL 8.9.0.
CURLOPT_TCP_NODELAY (int)	**true** para desabilitar o algoritmo Nagle do TCP, que tenta minimizar o número de pacotes pequenos na rede. O padrão é **true**. Disponível a partir do cURL 7.11.2.
CURLOPT_TELNETOPTIONS (int)	Define um array de strings a ser passado às negociações telnet. As variáveis devem estar no formato <opção=valor>. O cURL suporta as opções TTYPE, XDISPLOC e NEW_ENV. Disponível a partir do cURL 7.7.0.
CURLOPT_TFTP_BLKSIZE (int)	Define o tamanho do bloco a ser usado para transmissão de dados TFTP. O intervalo válido é 8-65464 bytes. O padrão de 512 bytes será usado se esta opção não for especificada. O tamanho de bloco especificado só será usado se for suportado pelo servidor remoto. Se o servidor não retornar uma confirmação de opção ou retornar uma confirmação de opção sem tamanho de bloco, o padrão de 512 bytes será usado. Disponível a partir do cURL 7.19.4.
CURLOPT_TFTP_NO_OPTIONS (int)	**true** para não enviar requisições de opções TFTP. O padrão é **false**. Disponível a partir do PHP 7.0.7 e do cURL 7.48.0.
CURLOPT_TIMECONDITION (int)	Define como **CURLOPT_TIMEVALUE** é tratada. Use **CURL_TIMECOND_IFMODSINCE** para retornar a página somente se ela tenha sido modificada desde o momento especificado em **CURLOPT_TIMEVALUE**. Se ela não foi modificada, um cabeçalho 304 Not Modified será retornado assumindo que **CURLOPT_HEADER** seja **true**. Use **CURL_TIMECOND_IFUNMODSINCE** para o efeito reverso. Use **CURL_TIMECOND_NONE** para ignorar **CURLOPT_TIMEVALUE** e sempre retornar a página. **CURL_TIMECOND_NONE** é o padrão. Antes do cURL 7.46.0, o padrão era **CURL_TIMECOND_IFMODSINCE**. Disponível a partir do cURL 7.1.0.
CURLOPT_TIMEOUT (int)	O número máximo de segundos para permitir a execução de funções cURL. O padrão é 0, que significa que as funções nunca expiram durante a transferência. Disponível a partir do cURL 7.1.0.
CURLOPT_TIMEOUT_MS (int)	O número máximo de milissegundos para permitir a execução de funções cURL. Se o cURL for compilado para usar o resolvedor de nome padrão do sistema, essa parte da conexão ainda usará resolução de segundos completos para tempos limite com um mínimo permitido de um segundo. O padrão é 0, que significa que as funções nunca expiram durante a transferência. Disponível a partir do cURL 7.16.2.
CURLOPT_TIMEVALUE (int)	O tempo em segundos desde 1º de janeiro de 1970. O tempo será usado por **CURLOPT_TIMECONDITION**. O padrão é 0. Disponível a partir do cURL 7.1.0.
CURLOPT_TIMEVALUE_LARGE (int)	O tempo em segundos desde 1º de janeiro de 1970. O tempo será usado por **CURLOPT_TIMECONDITION**. O padrão é zero. A diferença entre esta opção e **CURLOPT_TIMEVALUE** é o tipo do argumento. Em sistemas onde o inteiro longo é de somente 32 bits de comprimento, esta opção tem que ser usada para definir datas além do ano 2038. Disponível a partir do PHP 7.3.0 e cURL 7.59.0.
CURLOPT_TLS13_CIPHERS (int)	Uma string com a lista de conjuntos de criptografia a serem usados para a conexão TLS 1.3, separados por dois-pontos. Esta opção é usada atualmente apenas quando o cURL é compilado para usar OpenSSL 1.1.1 ou posterior. Se um SSL diferente estiver sendo usado, os conjuntos de criptografia TLS 1.3 podem ser configurados com a opção **CURLOPT_SSL_CIPHER_LIST**. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
CURLOPT_TLSAUTH_PASSWORD (int)	Define uma senha a ser usada para o método de autenticação TLS especificado com a opção **CURLOPT_TLSAUTH_TYPE**. Requer que a opção **CURLOPT_TLSAUTH_USERNAME** também seja definida. Este

	recurso depende do TLS SRP, que não funciona com o TLS 1.3. Disponível a partir do cURL 7.21.4.
CURLOPT_TLSAUTH_TYPE (int)	Define uma string com o método de autenticação TLS. O método suportado é `SRP` (autenticação TLS "Secure Remote Password"). Disponível a partir do cURL 7.21.4.
CURLOPT_TLSAUTH_USERNAME (int)	Define uma string com o nome de usuário a ser usado para o método de autenticação TLS especificado com a opção **CURLOPT_TLSAUTH_TYPE**. Requer que a opção **CURLOPT_TLSAUTH_PASSWORD** também esteja definida. Este recurso depende do TLS SRP, que não funciona com o TLS 1.3. Disponível a partir do cURL 7.21.4.
CURLOPT_TRANSFER_ENCODING (int)	Definida como `1` ativa, e como `0` desativa a solicitação de `Transfer Encoding` compactada na solicitação HTTP de saída. Se o servidor responder com uma `Transfer Encoding` compactada, o cURL irá descompactá-la automaticamente na recepção. O padrão é `0`. Disponível a partir do cURL 7.21.6.
CURLOPT_TRANSFERTEXT (int)	**true** para usar modo ASCII para transferências FTP. Para LDAP, os dados são recuperados em texto puro ao invés de HTML. Em sistemas Windows, **STDOUT** não será definido para modo binário. O padrão é **false**. Disponível a partir do cURL 7.1.1.
CURLOPT_UNIX_SOCKET_PATH (int)	Permite o uso de soquetes de domínio Unix como terminal de conexão e define o caminho para a string fornecida. Defina como **null** para desabilitar. O padrão é **null**. Disponível a partir do PHP 7.0.7 e cURL 7.40.0.
CURLOPT_UNRESTRICTED_AUTH (int)	**true** para continuar enviando usuário e senha ao seguir localizações (usando **CURLOPT_FOLLOWLOCATION**), mesmo quando o nome do servidor tiver mudado. O padrão é **false**. Disponível a partir do cURL 7.10.4.
CURLOPT_UPKEEP_INTERVAL_MS (int)	Alguns protocolos possuem mecanismos de "manutenção da conexão". Esses mecanismos geralmente enviam algum tráfego nas conexões existentes para mantê-las ativas. Esta opção define o intervalo de manutenção da conexão. Atualmente, o único protocolo com mecanismo de manutenção de conexão é o HTTP/2. Quando o intervalo de manutenção da conexão é excedido, um quadro PING HTTP/2 é enviado na conexão. O padrão é **CURL_UPKEEP_INTERVAL_DEFAULT** que atualmente é `60` segundos. Disponível a partir do PHP 8.2.0 e cURL 7.62.0.
CURLOPT_UPLOAD (int)	**true** para se preparar e realizar um envio. O padrão é **false**. Disponível a partir do cURL 7.1.0.
CURLOPT_UPLOAD_BUFFERSIZE (int)	Tamanho de buffer preferido em bytes para o buffer de upload cURL. O tamanho do buffer de upload por padrão é de 64 kilobytes. O tamanho máximo do buffer que pode ser definido é de 2 megabytes. O tamanho mínimo do buffer que pode ser definido é de 16 kilobytes. Disponível a partir do PHP 8.2.0 e cURL 7.62.0.
CURLOPT_URL (int)	O URL a ser buscado. Isto também pode ser definido ao inicializar uma sessão com curl_init(). Disponível a partir do cURL 7.1.0.
CURLOPT_USE_SSL (int)	Define o nível desejado de SSL/TLS para a transferência ao usar FTP, SMTP, POP3, IMAP etc. Todos esses são protocolos que iniciam em texto simples e são "atualizados" para SSL usando o comando STARTTLS. Deve ser definida como uma das constantes **CURLUSESSL_***. Disponível a partir do cURL 7.17.0.
CURLOPT_USERAGENT (int)	O conteúdo do cabeçalho `User-Agent:` a ser usado em uma requisição HTTP. Disponível a partir do cURL 7.1.0.

CURLOPT_USERNAME (int)	O nome de usuário a ser utilizado na autenticação. Disponível a partir do cURL 7.19.1.
CURLOPT_USERPWD (int)	Um nome de usuário e senha formatados como `[usuário]:[senha]` a serem usados para a conexão. Disponível a partir do cURL 7.1.0.
CURLOPT_VERBOSE (int)	**true** para gerar informações detalhadas. Grava a saída em **STDERR** ou no arquivo especificado usando **CURLOPT_STDERR**. O padrão é **false**. Disponível a partir do cURL 7.1.0.
CURLOPT_WILDCARDMATCH (int)	Se definida como `1`, transfere vários arquivos de acordo com um padrão de nome de arquivo. O padrão pode ser especificado como parte da opção **CURLOPT_URL**, usando um padrão semelhante ao fnmatch (Shell Pattern Matching) na última parte da URL (nome do arquivo). Disponível a partir do cURL 7.21.0.

CURLOPT_WRITEFUNCTION (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(resource `$curlHandle`, string `$data`): int
	
	`curlHandle`
	
	O identificador cURL.
	
	`data`
	
	Os dados a serem escritos.
	
	Os dados devem ser gravados por esta função de retorno e ela deve retornar o número exato de bytes escritos ou a transferência será abortada com erro. Disponível a partir do cURL 7.1.0.

CURLOPT_WRITEHEADER (int)	Aceita um identificador de arquivo do tipo resource no qual a parte do cabeçalho da transferência é gravada. Disponível a partir do cURL 7.1.0.
CURLOPT_WS_OPTIONS (int)	Aceita uma máscara de bits configurando as opções de comportamento do WebSocket. A única opção disponível é **CURLWS_RAW_MODE**. O padrão é `0`. Disponível a partir do PHP 8.3.0 e cURL 7.86.0.

CURLOPT_XFERINFOFUNCTION (int)	Uma função de retorno do tipo callable com a seguinte assinatura:
	
	function callback(
	resource `$curlHandle`,
	int `$bytesToDownload`,
	int `$bytesDownloaded`,
	int `$bytesToUpload`,
	int `$bytesUploaded`
	): int
	
	`curlHandle`
	
	O identificador cURL.
	
	`bytesToDownload`
	
	O número total de bytes esperados para recebimento nesta transferência.
	
	`bytesDownloaded`
	
	O número de bytes recebidos até o momento.
	
	`bytesToUpload`
	
	O número total de bytes esperados para envio nesta transferência.
	
	`bytesUploaded`

	O número de bytes enviados até o momento. Retorne `1` para abortar a transferência e definir um erro `CURLE_ABORTED_BY_CALLBACK`. Disponível a partir do PHP 8.2.0 e cURL 7.32.0.
`CURLOPT_SERVER_RESPONSE_TIMEOUT` (int)	Um tempo limite em segundos que o cURL esperará por uma resposta de um servidor FTP, SFTP, IMAP, SCP, SMTP ou POP3. Esta opção substitui a opção `CURLOPT_FTP_RESPONSE_TIMEOUT` existente que foi descontinuada no cURL 7.85.0. Disponível a partir do PHP 8.4.0.
`CURLOPT_XOAUTH2_BEARER` (int)	Especifica o token de acesso do OAuth 2.0. Defina como `null` para desabilitar. O padrão é `null`. Disponível a partir do PHP 7.0.7 e cURL 7.33.0.

`CURLOPT_PREREQFUNCTION` (int)	Uma função do tipo callable com a seguinte assinatura que é chamada depois que a conexão é estabelecida, mas antes que a carga útil da solicitação (por exemplo, a solicitação GET/POST/DELETE de uma conexão HTTP) seja enviada e pode ser usada para cancelar ou permitir a conexão dependendo do endereço IP de origem e destino e dos números de porta:
	
	function callback(
	CurlHandle `$curlHandle`,
	string `$destination_ip`,
	string `$local_ip`,
	int `$destination_port`,
	int `$local_port`
	): int
	
	`curlHandle`
	
	O identificador cURL.
	
	`destination_ip`
	
	O IP primário do servidor remoto estabelecido com esta conexão. Para FTP, este é o IP da conexão de controle. Os endereços IPv6 são representados sem colchetes.
	
	`local_ip`
	
	O IP de origem para esta conexão. Os endereços IPv6 são representados sem colchetes.
	
	`destination_port`
	
	O número da porta primária no servidor remoto estabelecido com esta conexão. Para FTP, esta é a porta para a conexão de controle. Este pode ser um número de porta TCP ou UDP dependendo do protocolo.
	
	`local_port`
	
	O número da porta de origem para esta conexão. Este pode ser um número de porta TCP ou UDP dependendo do protocolo.
	
	Retorne `CURL_PREREQFUNC_OK` para permitir a solicitação ou `CURL_PREREQFUNC_ABORT` para abortar a transferência. Disponível a partir do PHP 8.4.0 e cURL 7.80.0.

`CURLOPT_DEBUGFUNCTION` (int)	Disponível a partir do PHP 8.4.0. Esta opção requer a opção `CURLOPT_VERBOSE` habilitada. Uma função do tipo callable para substituir a saída detalhada do cURL padrão. Esse retorno de chamada é chamado durante vários estágios da solicitação com informações detalhadas de depuração. O retorno de chamada deve corresponder à seguinte assinatura:
	
	function callback(CurlHandle `$curlHandle`, int `$type`, string `$data`): void
	
	`curlHandle`
	
	O identificador cURL.
	
	`type`
	
	Uma das seguintes constantes indicando o tipo do valor `data`:

Constantes	Descrição
`CURLINFO_TEXT` (int)	Texto informativo.
`CURLINFO_HEADER_IN` (int)	Dados de cabeçalho (ou semelhantes a cabeçalho) recebidos do par.
`CURLINFO_HEADER_OUT` (int)	Dados de cabeçalho (ou semelhantes a cabeçalho) enviados ao par.
`CURLINFO_DATA_IN` (int)	Dados de protocolo não processados recebidos do par. Mesmo que os dados sejam codificados ou compactados, eles não são fornecidos decodificados nem descompactados para esse retorno de chamada.
`CURLINFO_DATA_OUT` (int)	Dados de protocolo enviados ao par.
`CURLINFO_SSL_DATA_IN` (int)	Dados SSL/TLS (binários) recebidos do par.
`CURLINFO_SSL_DATA_OUT` (int)	Dados SSL/TLS (binários) enviados ao par.

`data`

Dados detalhados de depuração do tipo indicado pelo parâmetro `type`.

curl_share_setopt()

Constantes	Descrição
`CURL_LOCK_DATA_CONNECT` (int)	Compartilha/descompartilha o cache de conexão. Disponível a partir do PHP 7.3.0 e cURL 7.10.3.
`CURL_LOCK_DATA_COOKIE` (int)	Compartilha/descompartilha dados de cookies. Disponível a partir do cURL 7.10.3.
`CURL_LOCK_DATA_DNS` (int)	Compartilha/descompartilha cache DNS. Observe que quando vários identificadores cURL são usados, todos os identificadores adicionados ao mesmo identificador múltiplo compartilharão o cache DNS por padrão. Disponível a partir do cURL 7.10.3.
`CURL_LOCK_DATA_PSL` (int)	Compartilha/descompartilha o compartilhamento da Lista Pública de Sufixos. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
`CURL_LOCK_DATA_SSL_SESSION` (int)	Compartilha/descompartilha IDs de sessão SSL, reduzindo o tempo gasto no handshake SSL ao se reconectar ao mesmo servidor. Observe que os IDs de sessão SSL são reutilizados no mesmo identificador por padrão. Disponível a partir do cURL 7.10.3.
`CURLSHOPT_NONE` (int)	Disponível a partir do cURL 7.10.3.
`CURLSHOPT_SHARE` (int)	Especifica um tipo de dados que deve ser compartilhado. Disponível a partir do cURL 7.10.3.
`CURLSHOPT_UNSHARE` (int)	Especifica um tipo de dados que não será mais compartilhado. Disponível a partir do cURL 7.10.3.

curl_getinfo()

Constantes	Descrição
`CURLINFO_APPCONNECT_TIME` (int)	Tempo em segundos que levou desde o início até que a conexão/handshake SSL/SSH com o host remoto fosse concluída.
`CURLINFO_APPCONNECT_TIME_T` (int)	Tempo, em microssegundos, desde o início até que a conexão/handshake SSL/SSH com o host remoto fosse concluída. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
`CURLINFO_CAINFO` (int)	Caminho do certificado CA integrado padrão. Disponível a partir do PHP 8.3.0 e cURL 7.84.0.
`CURLINFO_CAPATH` (int)	String de caminho de CA integrada padrão. Disponível a partir do PHP 8.3.0 e cURL 7.84.0.
`CURLINFO_CERTINFO` (int)	Cadeia de certificados TLS.
`CURLINFO_CONDITION_UNMET` (int)	Informações sobre condicional de tempo não cumprido.
`CURLINFO_CONN_ID` (int)	O ID da última conexão usada pela transferência. O ID da conexão é único entre todas as conexões que usam o mesmo cache de conexão e é útil para diferenciar a reutilização de conexões. Disponível a partir do PHP 8.5.0 e cURL 8.2.0.
`CURLINFO_CONNECT_TIME` (int)	Tempo decorrido para estabelecer a conexão em segundos.
`CURLINFO_CONNECT_TIME_T` (int)	Tempo total gasto, em microssegundos, desde o início até a conexão com o host remoto (ou proxy) ser concluída. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
`CURLINFO_CONTENT_LENGTH_DOWNLOAD` (int)	Comprimento do conteúdo do download, lido do campo "Content-Length:". Descontinuado desde o cURL 7.55.0. Use `CURLINFO_CONTENT_LENGTH_DOWNLOAD_T` em seu lugar.
`CURLINFO_CONTENT_LENGTH_DOWNLOAD_T` (int)	O comprimento do conteúdo do recebimento. Este é o valor lido do campo "Content-Length:" menos 1 se o tamanho não for conhecido. Disponível a partir do PHP 7.3.0 e cURL 7.55.0.
`CURLINFO_CONTENT_LENGTH_UPLOAD` (int)	Tamanho especificado do envio. Descontinuado desde o cURL 7.55.0. Use `CURLINFO_CONTENT_LENGTH_UPLOAD_T` em seu lugar.
`CURLINFO_CONTENT_LENGTH_UPLOAD_T` (int)	O tamanho especificado do envio. -1 se o tamanho não for conhecido. Disponível a partir do PHP 7.3.0 e cURL 7.55.0.
`CURLINFO_CONTENT_TYPE` (int)	`Content-Type` do documento solicitado. NULL indica que o servidor não enviou cabeçalho `Content-Type` válido
`CURLINFO_COOKIELIST` (int)	Todos os cookies conhecidos.
`CURLINFO_EFFECTIVE_METHOD` (int)	Obtém o último método HTTP utilizado.
`CURLINFO_EFFECTIVE_URL` (int)	Último URL efetivo
`CURLINFO_FILETIME` (int)	Horário remoto do documento recuperado, com `CURLOPT_FILETIME` habilitado; se -1 for retornado o horário do documento é desconhecido.

CURLINFO_FILETIME_T (int)	Horário remoto do documento recuperado (como timestamp Unix), uma alternativa a **CURLINFO_FILETIME** para permitir que sistemas com variáveis longas de 32 bits extraiam datas fora do intervalo de timestamp de 32 bits. Disponível a partir do PHP 7.3.0 e cURL 7.59.0.
CURLINFO_FTP_ENTRY_PATH (int)	Caminho de entrada no servidor FTP.
CURLINFO_HEADER_OUT (int)	A string de solicitação enviada. Para que isso funcione, adicione a opção **CURLINFO_HEADER_OUT** ao identificador chamando curl_setopt().
CURLINFO_HEADER_SIZE (int)	Tamanho total de todos os cabeçalhos recebidos.
CURLINFO_HTTPAUTH_AVAIL (int)	Máscara de bits indicando o(s) método(s) de autenticação HTTP disponível(is) de acordo com a resposta anterior.
CURLINFO_HTTPAUTH_USED (int)	Máscara de bits indicando o(s) método(s) de autenticação HTTP realmente usado(s) na solicitação anterior. Disponível a partir do PHP 8.5.0 e cURL 8.12.0.
CURLINFO_HTTP_CODE (int)	O último código de resposta. A partir do cURL 7.10.8, este é um apelido legado de **CURLINFO_RESPONSE_CODE**.
CURLINFO_HTTP_CONNECTCODE (int)	O código de resposta CONNECT.
CURLINFO_HTTP_VERSION (int)	A versão usada na última conexão HTTP. O valor de retorno será uma das constantes **CURL_HTTP_VERSION_*** definidas ou 0 se a versão não puder ser determinada. Disponível a partir do PHP 7.3.0 e cURL 7.50.0.
CURLINFO_LASTONE (int)	O último valor na enumeração `CURLINFO` subjacente em `libcurl`.
CURLINFO_LOCAL_IP (int)	Endereço IP local (fonte) da conexão mais recente.
CURLINFO_LOCAL_PORT (int)	Porta local (fonte) da conexão mais recente.
CURLINFO_NAMELOOKUP_TIME (int)	Tempo em segundos até a resolução do nome ser concluída.
CURLINFO_NAMELOOKUP_TIME_T (int)	Tempo em microssegundos desde o início até a conclusão da resolução do nome. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
CURLINFO_NUM_CONNECTS (int)	Número de conexões que o curl teve que criar para alcançar a transferência anterior.
CURLINFO_OS_ERRNO (int)	Número de erro devido a uma falha de conexão. O número é específico do sistema operacional e do sistema.
CURLINFO_PRETRANSFER_TIME (int)	Tempo em segundos desde o início até pouco antes do início da transferência do arquivo.
CURLINFO_PRETRANSFER_TIME_T (int)	Tempo decorrido desde o início até o início da transferência do arquivo, em microssegundos. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.

`CURLINFO_PRIMARY_IP` (int)	Endereço IP da conexão mais recente.
`CURLINFO_PRIMARY_PORT` (int)	Porta de destino da conexão mais recente.
`CURLINFO_PRIVATE` (int)	Dados privados associados a este identificador cURL, previamente definido com a opção `CURLOPT_PRIVATE` de curl_setopt().
`CURLINFO_PROTOCOL` (int)	O protocolo usado na última conexão HTTP. O valor retornado será exatamente um dos valores `CURLPROTO_*`. Disponível a partir do PHP 7.3.0 e cURL 7.52.0. Descontinuado desde o cURL 7.85.0. Use `CURLINFO_SCHEME` em seu lugar.
`CURLINFO_PROXYAUTH_AVAIL` (int)	Máscara de bits indicando o(s) método(s) de autenticação de proxy disponível(is)is de acordo com a resposta anterior.
`CURLINFO_PROXYAUTH_USED` (int)	Máscara de bits indicando o(s) método(s) de autenticação de proxy realmente usado(s) na solicitação anterior. Disponível a partir do PHP 8.5.0 e cURL 8.12.0.
`CURLINFO_PROXY_ERROR` (int)	O código de erro de proxy detalhado (SOCKS) quando a transferência mais recente retornou um erro `CURLE_PROXY`. O valor retornado será exatamente um dos valores `CURLPX_*`. O código de erro será `CURLPX_OK` se nenhum código de resposta estiver disponível. Disponível a partir do PHP 8.2.0 e cURL 7.73.0.
`CURLINFO_PROXY_SSL_VERIFYRESULT` (int)	O resultado da verificação do certificado solicitado (usando a opção `CURLOPT_PROXY_SSL_VERIFYPEER`). Usado apenas para proxies HTTPS. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
`CURLINFO_QUEUE_TIME_T` (int)	O tempo, em microssegundos, que a transferência ficou retida em uma fila de espera antes de ser iniciada, devido aos limites definidos com `CURLMOPT_MAX_TOTAL_CONNECTIONS` ou opções semelhantes. Disponível a partir do PHP 8.5.0 e cURL 8.6.0.
`CURLINFO_REDIRECT_COUNT` (int)	Número de redirecionamentos, com a opção `CURLOPT_FOLLOWLOCATION` habilitada.
`CURLINFO_REDIRECT_TIME` (int)	Tempo em segundos de todas as etapas de redirecionamento antes do início da transação final, com a opção `CURLOPT_FOLLOWLOCATION` habilitada.
`CURLINFO_REDIRECT_TIME_T` (int)	Tempo total, em microssegundos, necessário para todas as etapas de redirecionamento, incluindo pesquisa de nome, conexão, pré-transferência e transferência, antes do início da transação final. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
`CURLINFO_REDIRECT_URL` (int)	Com a opção `CURLOPT_FOLLOWLOCATION` desabilitada: URL de redirecionamento encontrada na última transação, que deverá ser solicitada manualmente a seguir. Com a opção `CURLOPT_FOLLOWLOCATION` habilitada: fica vazia. A URL de redirecionamento neste caso está disponível em `CURLINFO_EFFECTIVE_URL`.
`CURLINFO_REFERER` (int)	O cabeçalho `Referer`. Disponível a partir do PHP 8.2.0 e cURL 7.76.0.
`CURLINFO_REQUEST_SIZE` (int)	Tamanho total de solicitações emitidas, atualmente apenas para solicitações HTTP.
`CURLINFO_RESPONSE_CODE` (int)	O último código de resposta. Disponível a partir do cURL 7.10.8.

CURLINFO_RETRY_AFTER (int)	As informações do cabeçalho `Retry-After` ou zero se não houver cabeçalho válido. Disponível a partir do PHP 8.2.0 e cURL 7.66.0
CURLINFO_RTSP_CLIENT_CSEQ (int)	Próximo CSeq do cliente RTSP.
CURLINFO_RTSP_CSEQ_RECV (int)	CSeq recebido recentemente.
CURLINFO_RTSP_SERVER_CSEQ (int)	Próximo CSeq do servidor RTSP.
CURLINFO_RTSP_SESSION_ID (int)	ID da sessão RTSP.
CURLINFO_SCHEME (int)	O esquema de URL usado para a conexão mais recente. Disponível a partir do PHP 7.3.0 e cURL 7.52.0.
CURLINFO_SIZE_DOWNLOAD (int)	Número total de bytes recebidos. Descontinuado desde o cURL 7.55.0. Use **CURLINFO_SIZE_DOWNLOAD_T** em seu lugar.
CURLINFO_SIZE_DOWNLOAD_T (int)	Número total de bytes que foram recebidos. O número é apenas para a última transferência e será redefinido novamente a cada nova transferência. Disponível a partir do PHP 7.3.0 e cURL 7.50.0.
CURLINFO_SIZE_UPLOAD (int)	Número total de bytes enviados. Descontinuado desde o cURL 7.55.0. Use **CURLINFO_SIZE_UPLOAD_T** em seu lugar.
CURLINFO_SIZE_UPLOAD_T (int)	Número total de bytes que foram enviados. Disponível a partir do PHP 7.3.0 e cURL 7.50.0.
CURLINFO_SPEED_DOWNLOAD (int)	Velocidade média de recebimento. Descontinuado desde o cURL 7.55.0. Use **CURLINFO_SPEED_DOWNLOAD_T** em seu lugar.
CURLINFO_SPEED_DOWNLOAD_T (int)	A velocidade média de recebimento em bytes/segundo medida para o recebimento completo. Disponível a partir do PHP 7.3.0 e cURL 7.50.0.
CURLINFO_SPEED_UPLOAD (int)	Velocidade média de envio. Descontinuado desde o cURL 7.55.0. Use **CURLINFO_SPEED_UPLOAD_T** em seu lugar.
CURLINFO_SPEED_UPLOAD_T (int)	A velocidade média de envio em bytes/segundo medida para o envio completo. Disponível a partir do PHP 7.3.0 e cURL 7.50.0.
CURLINFO_SSL_ENGINES (int)	Motores criptográficos OpenSSL suportados.
CURLINFO_SSL_VERIFYRESULT (int)	Resultado da verificação da certificação SSL solicitada pela configuração **CURLOPT_SSL_VERIFYPEER**.
CURLINFO_STARTTRANSFER_TIME (int)	Tempo em segundos até que o primeiro byte esteja prestes a ser transferido.
CURLINFO_STARTTRANSFER_TIME_T (int)	Tempo, em microssegundos, desde o início até o recebimento do primeiro byte. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.

CURLINFO_TOTAL_TIME (int)	Tempo total de transação em segundos para a última transferência.
CURLINFO_TOTAL_TIME_T (int)	Tempo total em microssegundos para a transferência anterior, incluindo resolução de nomes, conexão TCP etc. Disponível a partir do PHP 7.3.0 e cURL 7.61.0.
CURLINFO_USED_PROXY (int)	Indica se a transferência anterior utilizou um proxy; retorna `1` se um proxy foi utilizado e `0` caso contrário. Disponível a partir do PHP 8.5.0 e cURL 8.7.0.
CURLINFO_POSTTRANSFER_TIME_T (int)	Tempo decorrido do início até o último byte ser enviado, em microssegundos. Disponível a partir do PHP 8.4.0 e cURL 8.10.0.

curl_multi_setopt()

Constantes	Descrição
CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE (int)	Especifica o limite de comprimento do bloco para pipeline em bytes. Disponível a partir do PHP 7.0.7 e cURL 7.30.0
CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE (int)	Especifica o limite de tamanho para penalidade de pipeline em bytes. Disponível a partir do PHP 7.0.7 e cURL 7.30.0
CURLMOPT_MAXCONNECTS (int)	Especifica a quantidade máxima de conexões abertas simultaneamente que o libcurl pode armazenar em cache. Por padrão, o tamanho será aumentado para caber quatro vezes o número de identificadores adicionados via curl_multi_add_handle(). Quando o cache está cheio, curl fecha a mais antiga do cache para evitar que o número de conexões abertas aumente. Disponível a partir do cURL 7.16.3.
CURLMOPT_MAX_CONCURRENT_STREAMS (int)	Especifica o número máximo de fluxos simultâneos para conexões que cURL deve suportar em conexões usando HTTP/2. Os valores válidos variam de `1` a `2147483647` (`2^31 - 1`). O valor passado aqui seria respeitado com base em outras propriedades de recursos do sistema. O padrão é `100`. Disponível a partir do PHP 8.2.0 e cURL 7.67.0.
CURLMOPT_MAX_HOST_CONNECTIONS (int)	Especifica o número máximo de conexões com um único host. Disponível a partir do PHP 7.0.7 e cURL 7.30.0
CURLMOPT_MAX_PIPELINE_LENGTH (int)	Especifica o número máximo de solicitações em um pipeline. Disponível a partir do PHP 7.0.7 e cURL 7.30.0
CURLMOPT_MAX_TOTAL_CONNECTIONS (int)	Especifica o número máximo de conexões abertas simultaneamente. Disponível a partir do PHP 7.0.7 e cURL 7.30.0
CURLMOPT_PIPELINING (int)	Passe 1 para habilitar ou 0 para desabilitar. Habilitar o pipeline em um identificador múltiplo fará com que ele tente executar o Pipelining HTTP na medida do possível para transferências usando esse identificador. Isso significa que adicionar uma segunda solicitação que possa usar uma conexão já existente "canalizará" a segunda solicitação na mesma conexão. A partir do cURL 7.43.0, o valor é uma máscara de bits e passar 2 tentará multiplexar a nova transferência por meio de uma conexão HTTP/2 existente. A passagem 3 instrui o cURL a solicitar pipeline e multiplexação independentemente um do outro. A partir do cURL 7.62.0, a configuração do bit de pipelining não tem efeito. Em vez de literais inteiros, as constantes CURLPIPE_* também podem ser usadas. Disponível a partir do cURL 7.16.0.
CURLMOPT_PUSHFUNCTION (int)	Passe um callable que será registrado para lidar com pushes do servidor e deverá ter a seguinte assinatura:

	function pushfunction(resource `$parent_ch`, resource `$pushed_ch`, array `$headers`): int
	
	`parent_ch`
	O identificador cURL pai (a solicitação feita pelo cliente).
	
	`pushed_ch`
	Um novo identificador cURL para a solicitação enviada.
	
	`headers`
	Os cabeçalhos da promessa push.
	
	A função push deve retornar **`CURL_PUSH_OK`** se puder lidar com o push, ou **`CURL_PUSH_DENY`** para rejeitá-lo. Disponível a partir do PHP 7.1.0 e cURL 7.44.0

Constantes do protocolo cURL

Constantes	Descrição
`CURLPROTO_ALL` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_DICT` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_FILE` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_FTP` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_FTPS` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_GOPHER` (int)	Disponível a partir do cURL 7.21.2.
`CURLPROTO_HTTP` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_HTTPS` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_IMAP` (int)	Disponível a partir do cURL 7.20.0.
`CURLPROTO_IMAPS` (int)	Disponível a partir do cURL 7.20.0.
`CURLPROTO_LDAP` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_LDAPS` (int)	Disponível a partir do cURL 7.19.4.
`CURLPROTO_MQTT` (int)	Disponível a partir do PHP 8.2.0 e cURL 7.71.0.
`CURLPROTO_POP3` (int)	Disponível a partir do cURL 7.20.0.
`CURLPROTO_POP3S` (int)	Disponível a partir do cURL 7.20.0.
`CURLPROTO_RTMP` (int)	Disponível a partir do cURL 7.21.0.

CURLPROTO_RTMPE (int)	Disponível a partir do cURL 7.21.0.
CURLPROTO_RTMPS (int)	Disponível a partir do cURL 7.21.0.
CURLPROTO_RTMPT (int)	Disponível a partir do cURL 7.21.0.
CURLPROTO_RTMPTE (int)	Disponível a partir do cURL 7.21.0.
CURLPROTO_RTMPTS (int)	Disponível a partir do cURL 7.21.0.
CURLPROTO_RTSP (int)	Disponível a partir do cURL 7.20.0.
CURLPROTO_SCP (int)	Disponível a partir do cURL 7.19.4.
CURLPROTO_SFTP (int)	Disponível a partir do cURL 7.19.4.
CURLPROTO_SMB (int)	Disponível a partir do PHP 7.0.7 e cURL 7.40.0.
CURLPROTO_SMBS (int)	Disponível a partir do PHP 7.0.7 e cURL 7.40.0.
CURLPROTO_SMTP (int)	Disponível a partir do cURL 7.20.0.
CURLPROTO_SMTPS (int)	Disponível a partir do cURL 7.20.0.
CURLPROTO_TELNET (int)	Disponível a partir do cURL 7.19.4.
CURLPROTO_TFTP (int)	Disponível a partir do cURL 7.19.4.

cURL error constants

Constantes	Descrição
CURLE_ABORTED_BY_CALLBACK (int)	Abortado por retorno de chamada. Um retorno de chamada retornou "abort" para libcurl.
CURLE_BAD_CALLING_ORDER (int)	
CURLE_BAD_CONTENT_ENCODING (int)	Codificação de transferência não reconhecida.
CURLE_BAD_DOWNLOAD_RESUME (int)	O download não pôde ser retomado porque o deslocamento especificado estava fora do limite do arquivo.
CURLE_BAD_FUNCTION_ARGUMENT (int)	Uma função foi chamada com um parâmetro incorreto.
CURLE_BAD_PASSWORD_ENTERED (int)	
CURLE_COULDNT_CONNECT (int)	Falha ao conectar ao host ou proxy.
CURLE_COULDNT_RESOLVE_HOST (int)	Não foi possível resolver o host. O host remoto fornecido não foi resolvido.

CURLE_COULDNT_RESOLVE_PROXY (int)	Não foi possível resolver o proxy. O host proxy fornecido não pôde ser resolvido.
CURLE_FAILED_INIT (int)	Falha no código de inicialização antecipada. É provável que seja um erro ou problema interno, ou um problema de recursos em que algo fundamental não pôde ser feito no momento da inicialização.
CURLE_FILESIZE_EXCEEDED (int)	Tamanho máximo do arquivo excedido.
CURLE_FILE_COULDNT_READ_FILE (int)	Um arquivo fornecido com FILE:// não pôde ser aberto. Provavelmente porque o caminho do arquivo não identifica um arquivo existente ou devido à falta de permissões de arquivo apropriadas.
CURLE_FTP_ACCESS_DENIED (int)	
CURLE_FTP_BAD_DOWNLOAD_RESUME (int)	
CURLE_FTP_CANT_GET_HOST (int)	Uma falha interna ao pesquisar o host usado para a nova conexão.
CURLE_FTP_CANT_RECONNECT (int)	
CURLE_FTP_COULDNT_GET_SIZE (int)	
CURLE_FTP_COULDNT_RETR_FILE (int)	Esta foi uma resposta inesperada a um comando 'RETR' ou uma transferência de zero byte concluída.
CURLE_FTP_COULDNT_SET_ASCII (int)	
CURLE_FTP_COULDNT_SET_BINARY (int)	
CURLE_FTP_COULDNT_STOR_FILE (int)	
CURLE_FTP_COULDNT_USE_REST (int)	O comando FTP REST retornou um erro. Isso nunca deveria acontecer se o servidor estiver saudável.
CURLE_FTP_PARTIAL_FILE (int)	
CURLE_FTP_PORT_FAILED (int)	O comando FTP PORT retornou erro. Isso acontece principalmente quando um endereço bom o suficiente não foi especificado para uso da libcurl. Consulte **CURLOPT_FTPPORT**.
CURLE_FTP_QUOTE_ERROR (int)	
CURLE_FTP_SSL_FAILED (int)	
CURLE_FTP_USER_PASSWORD_INCORRECT (int)	
CURLE_FTP_WEIRD_227_FORMAT (int)	Os servidores FTP retornam uma linha 227 como resposta a um comando PASV. Se a libcurl não conseguir analisar essa linha, esse código de retorno será retornado.
CURLE_FTP_WEIRD_PASS_REPLY (int)	Após enviar a senha do FTP ao servidor, a libcurl espera uma resposta adequada. Este código de erro indica que um código inesperado foi retornado.
CURLE_FTP_WEIRD_PASV_REPLY (int)	A libcurl não conseguiu obter um resultado sensato do servidor como resposta a um comando PASV ou EPSV. O servidor está falho.
CURLE_FTP_WEIRD_SERVER_REPLY (int)	O servidor enviou dados que o libcurl não pôde analisar. Este código de erro é conhecido como **CURLE_WEIRD_SERVER_REPLY** a partir do cURL 7.51.0.
CURLE_FTP_WEIRD_USER_REPLY (int)	
CURLE_FTP_WRITE_ERROR (int)	

`CURLE_FUNCTION_NOT_FOUND` (int)	Função não encontrada. Uma função zlib necessária não foi encontrada.
`CURLE_GOT_NOTHING` (int)	Nada foi retornado do servidor e, dadas as circunstâncias, não receber nada é considerado um erro.
`CURLE_HTTP_NOT_FOUND` (int)	
`CURLE_HTTP_PORT_FAILED` (int)	
`CURLE_HTTP_POST_ERROR` (int)	Este é um erro estranho que ocorre principalmente devido a confusão interna.
`CURLE_HTTP_RANGE_ERROR` (int)	
`CURLE_HTTP_RETURNED_ERROR` (int)	Isto é retornado se `CURLOPT_FAILONERROR` estiver definido como `true` e o servidor HTTP retornar um código de erro maior ou igual a 400.
`CURLE_LDAP_CANNOT_BIND` (int)	O LDAP não pode ser vinculado. A operação de ligação LDAP falhou.
`CURLE_LDAP_INVALID_URL` (int)	
`CURLE_LDAP_SEARCH_FAILED` (int)	A pesquisa LDAP falhou.
`CURLE_LIBRARY_NOT_FOUND` (int)	
`CURLE_MALFORMAT_USER` (int)	
`CURLE_OBSOLETE` (int)	
`CURLE_OK` (int)	Tudo bem. Proceda normalmente.
`CURLE_OPERATION_TIMEDOUT` (int)	Tempo limite de operação. O período de tempo limite especificado foi alcançado de acordo com as condições.
`CURLE_OPERATION_TIMEOUTED` (int)	
`CURLE_OUT_OF_MEMORY` (int)	Falha na solicitação de alocação de memória.
`CURLE_PARTIAL_FILE` (int)	A transferência de arquivos foi mais curta ou maior que o esperado. Isso acontece quando o servidor primeiro informa um tamanho de transferência esperado e depois entrega dados que não correspondem ao tamanho fornecido anteriormente.
`CURLE_PROXY` (int)	Erro de estabelecimento de comunicação com o proxy. `CURLINFO_PROXY_ERROR` fornece detalhes extras sobre o problema específico. Disponível a partir do PHP 8.2.0 e cURL 7.73.0.
`CURLE_READ_ERROR` (int)	Ocorreu um problema ao ler um arquivo local ou um erro retornado pelo retorno de chamada de leitura.
`CURLE_RECV_ERROR` (int)	Falha no recebimento de dados da rede.
`CURLE_SEND_ERROR` (int)	Falha ao enviar dados de rede.
`CURLE_SHARE_IN_USE` (int)	
`CURLE_SSH` (int)	Ocorreu um erro não especificado durante a sessão SSH. Disponível a partir do cURL 7.16.1.
`CURLE_SSL_CACERT` (int)	

CURLE_SSL_CACERT_BADFILE (int)	Problema ao ler o certificado SSL CA.
CURLE_SSL_CERTPROBLEM (int)	Problema com o certificado do cliente local.
CURLE_SSL_CIPHER (int)	Não foi possível usar a cifra especificada.
CURLE_SSL_CONNECT_ERROR (int)	Ocorreu um problema em algum lugar na comunicação inicial SSL/TLS. A leitura da mensagem no buffer de erros fornece mais detalhes sobre o problema. Podem ser certificados (formatos de arquivo, caminhos, permissões), senhas e outros.
CURLE_SSL_ENGINE_NOTFOUND (int)	O mecanismo de criptografia especificado não foi encontrado.
CURLE_SSL_ENGINE_SETFAILED (int)	Falha ao definir o mecanismo de criptografia SSL selecionado como padrão.
CURLE_SSL_PEER_CERTIFICATE (int)	
CURLE_SSL_PINNEDPUBKEYNOTMATCH (int)	Falha ao corresponder à chave fixada especificada com CURLOPT_PINNEDPUBLICKEY.
CURLE_TELNET_OPTION_SYNTAX (int)	
CURLE_TOO_MANY_REDIRECTS (int)	Muitos redirecionamentos. Ao seguir redirecionamentos, a libcurl atingiu o valor máximo. O limite pode ser definido com CURLOPT_MAXREDIRS.
CURLE_UNKNOWN_TELNET_OPTION (int)	
CURLE_UNSUPPORTED_PROTOCOL (int)	A URL passada para libcurl usou um protocolo que a libcurl não suporta. O problema pode ser uma opção de tempo de compilação que não foi usada, uma string de protocolo com erro ortográfico ou apenas um protocolo para o qual a libcurl não possui código.
CURLE_URL_MALFORMAT (int)	O URL não foi formatado corretamente.
CURLE_URL_MALFORMAT_USER (int)	
CURLE_WEIRD_SERVER_REPLY (int)	O servidor enviou dados que a libcurl não pôde analisar. Este código de erro era conhecido como CURLE_FTP_WEIRD_SERVER_REPLY antes do cURL 7.51.0. Disponível a partir do PHP 7.3.0 e cURL 7.51.0.
CURLE_WRITE_ERROR (int)	Ocorreu um erro ao gravar dados recebidos em um arquivo local ou um erro foi retornado para a libcurl a partir de um retorno de chamada de gravação.

Constantes de status curl_multi_*

Constantes	Descrição
CURLM_ADDED_ALREADY (int)	Um identificador simples já adicionado a um identificador múltiplo, assim, houve tentativa de ser adicionado por uma segunda vez. Disponível a partir do cURL 7.32.1.
CURLM_BAD_EASY_HANDLE (int)	Um identificador simples não era bom/válido. Isso pode significar que não é um identificador simples, ou possivelmente que o identificador já está em uso por este ou outro identificador múltiplo. Disponível a partir do cURL 7.9.6.
CURLM_BAD_HANDLE (int)	O identificador passado não é um identificador múltiplo válido. Disponível a partir do cURL 7.9.6.

CURLM_CALL_MULTI_PERFORM (int)	A partir do cURL 7.20.0, essa constante não é usada. Antes do cURL 7.20.0, esse status podia ser retornado por curl_multi_exec() quando curl_multi_select() ou uma função semelhante era chamada antes de retornar qualquer outra constante. Disponível a partir do cURL 7.9.6.
CURLM_INTERNAL_ERROR (int)	Erro interno `libcurl`. Disponível a partir do cURL 7.9.6.
CURLM_OK (int)	Sem erros. Disponível a partir do cURL 7.9.6.
CURLM_OUT_OF_MEMORY (int)	Ficou sem memória ao processar vários identificadores. Disponível a partir do cURL 7.9.6.

curl_pause()

Constantes	Descrição
CURLPAUSE_ALL (int)	Pausa o envio e recebimento de dados. Disponível a partir do cURL 7.18.0.
CURLPAUSE_CONT (int)	Retoma o envio e recebimento de dados. Disponível a partir do cURL 7.18.0.
CURLPAUSE_RECV (int)	Pausa o recebimento de dados. Disponível a partir do cURL 7.18.0.
CURLPAUSE_RECV_CONT (int)	Retoma o recebimento de dados. Disponível a partir do cURL 7.18.0.
CURLPAUSE_SEND (int)	Pausa o envio de dados. Disponível a partir do cURL 7.18.0.
CURLPAUSE_SEND_CONT (int)	Retoma o envio de dados. Disponível a partir do cURL 7.18.0.

Examples

Índice

Exemplos básicos do cURL

Uma vez que o PHP tenha sido compilado com suporte ao cURL, pode-se começar a usar as funções cURL. A ideia por trás das funções cURL é inicializar uma sessão cURL usando curl_init(), o que permite configurar as opções para a transferência através de curl_setopt(), e depois executar a sessão com curl_exec().

Aqui está um exemplo simples que usa as funções cURL para enviar uma requisição POST:

Exemplo #1 Usando o módulo cURL do PHP para enviar uma requisição POST

```
<?php
$data = ['foo' => 'bar', 'baz' => 48];
$url = "http://www.example.com/handler.php";

$ch = curl_init($url);

// diz ao CURL para retornar a resposta em vez de enviá-la para stdout
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// define os dados POST, o método correspondente e os cabeçalhos
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query($data));

// envia a requisição e obtém a resposta
$response = curl_exec($ch);

if(curl_error($ch)) {
    // trata o erro, ou apenas
```

```
    throw new RuntimeException(curl_error($ch));
}
```

Outro exemplo que usa as funções cURL para enviar uma requisição POST com JSON:

Exemplo #2 Usando o módulo cURL do PHP para enviar uma requisição POST com JSON

```php
<?php
$post_data = ['foo' => 'bar', 'baz' => 48];
$url = "http://www.example.com/api/endpoint";

$ch = curl_init($url);

// diz ao CURL para retornar a resposta em vez de enviá-la para stdout
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// define os dados POST e o método correspondente
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($post_data));

// define o cabeçalho necessário
curl_setopt($ch, CURLOPT_HTTPHEADER, ['Content-Type: application/json']);

// envia a requisição e obtém a resposta
$response = curl_exec($ch);

if(curl_error($ch)) {
    // trata o erro, ou apenas
    throw new RuntimeException(curl_error($ch));
}
```

Aqui está um outro exemplo que usa as funções cURL para receber o conteúdo da página inicial de exemplo.com.br em um arquivo:

Exemplo #3 Usando o módulo cURL do PHP para receber a página exemplo.com.br

```php
<?php

$ch = curl_init("http://www.exemplo.com.br/");
$fp = fopen("pagina_exemplo.txt", "w");

curl_setopt($ch, CURLOPT_FILE, $fp);
curl_setopt($ch, CURLOPT_HEADER, 0);

curl_exec($ch);
if(curl_error($ch)) {
    fwrite($fp, curl_error($ch));
}
fclose($fp);
```

Funções cURL

Índice

curl_close

curl_close

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_close — Fecha uma sessão cURL

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **curl_close**(CurlHandle `$handle`): void

> **Nota** :
>
> Esta função não tem efeito. Antes do PHP 8.0.0, esta função era usada para fechar o recurso.

Fecha uma sessão cURL e libera todos os recursos. O identificador cURL, `handle`, também é removido.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Esta função foi descontinuada.
8.0.0	Esta função agora é um NOP.
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Iniciando uma nova sessão cURL e acessando uma página web

```php
<?php
// Cria uma nova sessão cURL
$ch = curl_init();

// Configura uma URL e outras opções apropriadas
curl_setopt($ch, CURLOPT_URL, "http://www.exemplo.com.br/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// Executa a sessão
curl_exec($ch);

// Fecha a sessão cURL e libera recursos
curl_close($ch);
?>
```

Veja também

- curl_init() - Inicializa uma sessão cURL
- curl_multi_close() - Remove todos os identificadores cURL de um multi-identificador

curl_copy_handle

curl_copy_handle

(PHP 5, PHP 7, PHP 8)

curl_copy_handle — Copia um identificador cURL com todas as suas opções

Descrição

function **curl_copy_handle**(CurlHandle `$handle`): CurlHandle|false

Copia um identificador cURL mantendo as opções escolhidas.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna um novo identificador cURL, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.
8.0.0	Em caso de sucesso, esta função agora retorna uma instância CurlHandle; anteriormente, retornava um recurso do tipo resource.

Examples

Exemplo #1 Copiando um identificador cURL

```php
<?php
// Cria uma nova sessão cURL
$ch = curl_init();

// Configura uma URL e outras opções apropriadas
curl_setopt($ch, CURLOPT_URL, 'http://www.exemplo.com.br/');
curl_setopt($ch, CURLOPT_HEADER, 0);

// Copia o identificador
$ch2 = curl_copy_handle($ch);

// Executa a sessão (abrindo a página exemplo.com.br)
curl_exec($ch2);
?>
```

curl_errno

curl_errno

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

curl_errno — Retorna o número do último erro

Descrição

function **curl_errno**(CurlHandle `$handle`): int

Retorna o número do erro para a última operação cURL.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna o número do erro ou `0` (zero) caso nenhum erro tenha ocorrido.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_errno()

```php
<?php
// Cria um novo identificador cURL para uma página existente
$ch = curl_init('http://404.php.net/');

// Executa
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

```php
curl_exec($ch);

// Verifica se ocorreu algum erro
if(curl_errno($ch))
{
    echo 'Erro cURL: ' . curl_error($ch);
}
?>
```

Veja também

- curl_error() - Retorna uma string contendo o último erro da sessão atual
- » Códigos de erros cURL

curl_error

curl_error

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

curl_error — Retorna uma string contendo o último erro da sessão atual

Descrição

function **curl_error**(CurlHandle `$handle`): string

Retorna uma mensagem de erro em texto simples para a última operação cURL.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna a mensagem de erro ou `''` (string vazia) se nenhum erro tiver ocorrido.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_error()

```php
<?php
// Inicializa o cURL usando um destino não existente
$ch = curl_init('http://404.php.net/');
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

if(curl_exec($ch) === false)
{
    echo 'Erro Curl: ' . curl_error($ch);
}
else
{
    echo 'Operação concluída sem erros';
}
?>
```

Veja também

- curl_errno() - Retorna o número do último erro
- » Códigos de erros cURL

curl_escape

curl_escape

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_escape — Codifica uma string para o formato URL

Descrição

function **curl_escape**(CurlHandle `$handle`, string `$string`): string|false

Essa função codifica uma string para o formato URL de acordo com » RFC 3986.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`string`

A string a ser codificada.

Valor Retornado

Retorna a string codificada ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_escape()

```php
<?php

// Cria um identificador curl
$ch = curl_init();

// Codifica uma string usada como parâmetro GET
$location = curl_escape($ch, 'Hofbräuhaus / München');
// Resultado: Hofbr%C3%A4uhaus%20%2F%20M%C3%BCnchen

// Cria uma URL com a string codificada
$url = "http://example.com/add_location.php?location={$location}";
// Resultado: http://example.com/add_location.php?location=Hofbr%C3%A4uhaus%20%2F%20M%C3%BCnchen

// Define as opções e envia a requisição HTTP
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_exec($ch);

?>
```

Veja também

- curl_unescape() - Decodifica a parte codificada da URL fornecida
- urlencode() - Codifica uma string no formato URL
- rawurlencode() - Codifica uma URL de acordo com RFC 3986
- » RFC 3986

curl_exec

curl_exec

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_exec — Executa uma sessão cURL

Descrição

function **curl_exec**(CurlHandle `$handle`): string|bool

Executa a sessão cURL fornecida.

Esta função deve ser chamada após inicializar uma sessão cURL e todas as opções para a sessão são definidas.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Em caso de sucesso, esta função descarrega o resultado diretamente em `stdout` e retorna **`true`**, ou **`false`** em caso de falha. No entanto, se a opção **`CURLOPT_RETURNTRANSFER`** estiver definida, ela retornará o resultado em caso de sucesso ou **`false`** em caso de falha.

> **Aviso**
>
> Esta função pode retornar o valor booleano **`false`**, mas também pode retornar um valor não booleano que pode ser avaliado como **`false`**. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

> **Nota** :
>
> Observe que os códigos de retorno que indicam erros (por exemplo, `404 Not found`) não são tratados como falhas. Use curl_getinfo() para verificar esses casos.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Obtendo uma página da web

```php
<?php
// Cria um novo identificador cURL
$ch = curl_init();

// Configura o URL e as opções
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// Obtém os dados
curl_exec($ch);
?>
```

Veja também

- curl_multi_exec() - Executa as subconexões do identificador cURL atual

curl_getinfo

curl_getinfo

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

curl_getinfo — Obtém informação sobre uma transferência específica

Descrição

function **curl_getinfo**(CurlHandle `$handle`, ?int `$option` = **`null`**): mixed

Obtém informação sobre a última transferência.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`option`

Uma das constantes **`CURLINFO_*`**.

Valor Retornado

Se o parâmetro `option` for fornecido, retorna seu valor. Caso contrário, retorna um array associativo com os elementos a seguir (que correspondem a `option`), ou **`false`** em caso de falha:

- "url"
- "content_type"
- "http_code"
- "header_size"
- "request_size"
- "filetime"
- "ssl_verify_result"
- "redirect_count"
- "total_time"
- "namelookup_time"
- "connect_time"
- "pretransfer_time"
- "size_upload"
- "size_download"
- "speed_download"
- "speed_upload"
- "download_content_length"
- "upload_content_length"
- "starttransfer_time"
- "redirect_time"
- "certinfo"
- "primary_ip"
- "primary_port"
- "local_ip"
- "local_port"
- "redirect_url"
- "request_header" (Este somente será definido se a opção **`CURLINFO_HEADER_OUT`** for configurada por uma chamada prévia a curl_setopt())
- "posttransfer_time_us" (Disponível a partir do PHP 8.4.0 e cURL 8.10.0)

Observe que dados privados não são incluídos no array associativo e devem ser obtidos individualmente com a opção `CURLINFO_PRIVATE`.

Registro de Alterações

Versão	Descrição
8.4.0	Introduzida a constante `CURLINFO_POSTTRANSFER_TIME_T` e a chave `posttransfer_time_us` (Curl 8.10.0 ou posterior).
8.3.0	Introduzidas as constantes `CURLINFO_CAINFO` e `CURLINFO_CAPATH`.
8.2.0	Introduzidas as constantes `CURLINFO_PROXY_ERROR`, `CURLINFO_REFERER` e `CURLINFO_RETRY_AFTER`.
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.
8.0.0	`option` agora pode ser nulo; anteriormente, o padrão era `0`.
7.3.0	Introduzidas as opções `CURLINFO_CONTENT_LENGTH_DOWNLOAD_T`, `CURLINFO_CONTENT_LENGTH_UPLOAD_T`, `CURLINFO_HTTP_VERSION`, `CURLINFO_PROTOCOL`, `CURLINFO_PROXY_SSL_VERIFYRESULT`, `CURLINFO_SCHEME`, `CURLINFO_SIZE_DOWNLOAD_T`, `CURLINFO_SIZE_UPLOAD_T`, `CURLINFO_SPEED_DOWNLOAD_T`, `CURLINFO_SPEED_UPLOAD_T`, `CURLINFO_APPCONNECT_TIME_T`, `CURLINFO_CONNECT_TIME_T`, `CURLINFO_FILETIME_T`, `CURLINFO_NAMELOOKUP_TIME_T`, `CURLINFO_PRETRANSFER_TIME_T`, `CURLINFO_REDIRECT_TIME_T`, `CURLINFO_STARTTRANSFER_TIME_T` e `CURLINFO_TOTAL_TIME_T`.

Examples

Exemplo #1 Exemplo de curl_getinfo()

```php
<?php
// Cria um manipulador cURL
$ch = curl_init('http://www.example.com/');

// Executa
curl_exec($ch);

// Verifica se algum erro ocorreu
if (!curl_errno($ch)) {
  $info = curl_getinfo($ch);
  echo 'Demorou ', $info['total_time'], ' segundos para enviar uma requisição a ', $info['url'], "\n";
}
?>
```

Exemplo #2 Exemplo de curl_getinfo() com parâmetro `option`

```php
<?php
// Cria um identificador cURL
$ch = curl_init('http://www.example.com/');

// Executa
curl_exec($ch);

// Verifica o código de estado do HTTP
if (!curl_errno($ch)) {
  switch ($http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE)) {
    case 200:  # OK
      break;
    default:
      echo 'Código HTTP inesperado: ', $http_code, "\n";
  }
}
?>
```

Notas

> **Nota** :
>
> Informações obtidas por esta função são mantidas se o identificador for reutilizado. Isto significa que a menos que uma estatística seja substituída internamente por esta função, a informação anterior é retornada.

curl_init

curl_init

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_init — Inicializa uma sessão cURL

Descrição

function **curl_init**(?string `$url` = `null`): CurlHandle|false

Inicializa uma nova sessão e retorna um identificador cURL.

Parâmetros

`url`

Se fornecido, a opção `CURLOPT_URL` será definida com este valor. Esta opção pode ser definida manualmente usando a função curl_setopt().

> **Nota** :
>
> O protocolo `file` é desativado no cURL se open_basedir estiver definido.

Valor Retornado

Retorna um identificador cURL em caso de sucesso, `false` em caso de erro.

Registro de Alterações

Versão	Descrição
8.0.0	No caso de sucesso, a função retorna uma instância de CurlHandle; anteriormente retornava um resource.
8.0.0	`url` agora pode ser nulo.

Examples

Exemplo #1 Inicializando uma nova sessão cURL e buscando uma página da web

```php
<?php

// Inicializa uma nova sessão cURL
$ch = curl_init();

// Configura o URL e outras opções
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// Executa a chamada e envia ao navegador
curl_exec($ch);

?>
```

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL

curl_multi_add_handle

curl_multi_add_handle

(PHP 5, PHP 7, PHP 8)

curl_multi_add_handle — Adiciona um identificador cURL comum a um multi-identificador cURL

Descrição

function **curl_multi_add_handle**(CurlMultiHandle `$multi_handle`, CurlHandle `$handle`): int

Adiciona o identificador `handle` ao multi-identificador `multi_handle`

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna 0 em caso de sucesso, ou um dos códigos de erro `CURLM_*`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Veja também

- curl_multi_remove_handle() - Remove um identificador de um conjunto de identificadores cURL
- curl_multi_init() - Retorna um multi-identificador cURL
- curl_init() - Inicializa uma sessão cURL

curl_multi_close

curl_multi_close

(PHP 5, PHP 7, PHP 8)

curl_multi_close — Remove todos os identificadores cURL de um multi-identificador

Descrição

function **curl_multi_close**(CurlMultiHandle `$multi_handle`): void

Remove todos os objetos CurlHandle anexados ao CurlMultiHandle, como se curl_multi_remove_handle() tivesse sido chamada para cada um deles.

Antes do PHP 8.0.0, esta função também fechava o recurso do multi-identificador cURL, tornando-o inutilizável.

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL

curl_multi_errno

curl_multi_errno

(PHP 7 >= 7.1.0, PHP 8)

curl_multi_errno — Retorna o último número de erro de curl_multi

Descrição

function **curl_multi_errno**(CurlMultiHandle `$multi_handle`): int

Retorna um número inteiro contendo o último número de erro de multi curl.

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

Valor Retornado

Retorna um número inteiro contendo o último número de erro de multi curl.

Registro de Alterações

Versão	Descrição
8.0.0	A função não retorna mais `false` em caso de falha.
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.

Veja também

- curl_errno() - Retorna o número do último erro

curl_multi_exec

curl_multi_exec

(PHP 5, PHP 7, PHP 8)

curl_multi_exec — Executa as subconexões do identificador cURL atual

Descrição

function **curl_multi_exec**(CurlMultiHandle `$multi_handle`, int `&$still_running`): int

Processa cada um dos identificadores na pilha. Este método pode ser chamado independentemente de um identificador precisar ou não ler ou gravar dados.

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`still_running`

Uma referência a um sinalizador para informar se as operações ainda estão em execução.

Valor Retornado

Um código cURL definido nas Constantes Predefinidas cURL.

> **Nota** :
>
> Esta função apenas retorna erros relacionados a toda a pilha de multi-identificadores. Ainda pode ter ocorrido problemas em transferências individuais mesmo quando esta função retorna `CURLM_OK`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_multi_exec()

Este exemplo criará dois identificadores cURL para uma lista de URLs que serão adicionados a um multi-identificador e executados de forma assíncrona.

```php
<?php

$urls = [
    "https://www.php.net/",
    "https://www.example.com/",
];

$mh = curl_multi_init();
$map = new WeakMap();

foreach ($urls as $url) {
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
    curl_multi_add_handle($mh, $ch);
    $map[$ch] = $url;
}

do {
    $status = curl_multi_exec($mh, $unfinishedHandles);
    if ($status !== CURLM_OK) {
        throw new \Exception(curl_multi_strerror(curl_multi_errno($mh)));
    }

    while (($info = curl_multi_info_read($mh)) !== false) {
        if ($info['msg'] === CURLMSG_DONE) {
            $handle = $info['handle'];
            curl_multi_remove_handle($mh, $handle);
            $url = $map[$handle];

            if ($info['result'] === CURLE_OK) {
                $statusCode = curl_getinfo($handle, CURLINFO_HTTP_CODE);

                echo "A requisição a {$url} terminou com o status HTTP {$statusCode}:", PHP_EOL;
                echo curl_multi_getcontent($handle);
                echo PHP_EOL, PHP_EOL;
            } else {
                echo "A requisição a {$url} falhou com erro: ", PHP_EOL;
                echo curl_strerror($info['result']);
                echo PHP_EOL, PHP_EOL;
            }
        }
    }

    if ($unfinishedHandles) {
        if (($updatedHandles = curl_multi_select($mh)) === -1) {
            throw new \Exception(curl_multi_strerror(curl_multi_errno($mh)));
        }
    }
} while ($unfinishedHandles);

curl_multi_close($mh);

?>
```

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL
- curl_multi_select() - Aguarda até que leitura ou escrita seja possível em qualquer conexão multi-identificadores cURL
- curl_exec() - Executa uma sessão cURL

curl_multi_getcontent

curl_multi_getcontent

(PHP 5, PHP 7, PHP 8)

curl_multi_getcontent — Retorna o conteúdo de um identificador cURL se `CURLOPT_RETURNTRANSFER` estiver definido

Descrição

function **curl_multi_getcontent**(CurlHandle `$handle`): ?string

Se `CURLOPT_RETURNTRANSFER` for uma opção configurada para um identificador específico, esta função retornará o conteúdo do identificador cURL na forma de uma string.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna o conteúdo de um identificador cURL se `CURLOPT_RETURNTRANSFER` estiver definida ou `null` se não estiver.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL

curl_multi_info_read

curl_multi_info_read

(PHP 5, PHP 7, PHP 8)

curl_multi_info_read — Obtém informação sobre as transferências atuais

Descrição

function **curl_multi_info_read**(CurlMultiHandle `$multi_handle`, int `&$queued_messages = null`): array|false

Interroga o multi-identificador se existem mensagens ou informações das transferências individuais. Mensagens podem incluir informações como um código de erro da transferência ou apenas o fato de uma transferência ter sido concluída.

Chamadas repetidas a esta função irão retornar um novo resultado a cada vez, até que um valor `false` seja retornado como um sinal de que não há mais o que receber no momento. O inteiro apontado pelo parâmetro `queued_messages` conterá o número de mensagens remanescentes depois que esta função foi chamada.

> **Aviso**
>
> Os dados apontados pelo recurso retornado não irão sobreviver à chamada a curl_multi_remove_handle().

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`queued_messages`

Número de mensagens que ainda estão na fila

Valor Retornado

Em caso de sucesso, retorna um array associativo para a mensagem, `false` em caso de falha.

Conteúdo do array retornado

Chave:	Valor:
`msg`	A constante `CURLMSG_DONE`. Outros valores de retorno autalmente não estão disponíveis.
`result`	Uma das constantes `CURLE_*`. Se tudo estiver correto, a constante `CURLE_OK` será o resultado.

Chave:	Valor:
`handle`	Identificador cURL.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL

curl_multi_init

curl_multi_init

(PHP 5, PHP 7, PHP 8)

curl_multi_init — Retorna um multi-identificador cURL

Descrição

function **curl_multi_init**(): CurlMultiHandle

Permite o processamento de multi-identificadores cURL de forma assíncrona.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um multi-identificador cURL.

Registro de Alterações

Versão	Descrição
8.0.0	Em caso de sucesso, esta função agora retorna uma instância de CurlMultiHandle; anteriormente, um recurso do tipo resource era retornado.

Veja também

- curl_init() - Inicializa uma sessão cURL
- curl_multi_close() - Remove todos os identificadores cURL de um multi-identificador

curl_multi_remove_handle

curl_multi_remove_handle

(PHP 5, PHP 7, PHP 8)

curl_multi_remove_handle — Remove um identificador de um conjunto de identificadores cURL

Descrição

function **curl_multi_remove_handle**(CurlMultiHandle `$multi_handle`, CurlHandle `$handle`): int

Remove o identificador `handle` do multi-identificador `multi_handle`. Quando o identificador `handle` tiver sido removido, ainda é totalmente correto executar curl_exec() neste identificador. Remover o identificador `handle` enquanto estiver sendo usado irá efetivamente parar a transferência em progresso envolvendo este identificador.

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna 0 em caso de sucesso, ou um dos códigos de erro `CURLM_*`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Veja também

- curl_init() - Inicializa uma sessão cURL
- curl_multi_init() - Retorna um multi-identificador cURL
- curl_multi_add_handle() - Adiciona um identificador cURL comum a um multi-identificador cURL

curl_multi_select

curl_multi_select

(PHP 5, PHP 7, PHP 8)

curl_multi_select — Aguarda até que leitura ou escrita seja possível em qualquer conexão multi-identificadores cURL

Descrição

function **curl_multi_select**(CurlMultiHandle `$multi_handle`, float `$timeout` = 1.0): int

Bloqueia a execução do script até que um identificador incluído no multi-identificador cURL seja capaz de progredir na próxima chamada a curl_multi_exec() ou até que o tempo limite seja alcançado (o que ocorrer primeiro).

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`timeout`

Tempo em segundos a esperar por uma resposta da conexão multi-identificadores cURL ativa.

Valor Retornado

Em caso de sucesso, retorna o número de descritores ativos contidos nos conjuntos de descritores. Pode ser `0` se não houve atividade em nenhum dos descritores. Em caso de falha, esta função retornará `-1` em uma falha de seleção (da chamada de sistema subjacente `select()`).

Erros/Exceções

Lança um ValueError se `timeout` for menor que `0` ou maior que `PHP_INT_MAX`.

Registro de Alterações

Versão	Descrição
8.4.0	Agora lança um ValueError se `timeout` for menor que `0` ou maior que `PHP_INT_MAX`.
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.

Veja também

- curl_multi_init() - Retorna um multi-identificador cURL

curl_multi_setopt

curl_multi_setopt

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_multi_setopt — Define múltiplas opções para cURL

Descrição

function **curl_multi_setopt**(CurlMultiHandle `$multi_handle`, int `$option`, mixed `$value`): bool

Define uma opção no multi-identificador cURL informado.

Parâmetros

`multi_handle`

Um manipulador cURL múltiplo retornado por curl_multi_init().

`option`

Uma das constantes **`CURLMOPT_*`**.

`value`

O valor a ser configurado em `option`. Consulte a descrição das constantes **`CURLMOPT_*`** para detalhes sobre o tipo de valor que cada constante espera.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.2.0	Introduzida a constante **`CURLMOPT_MAX_CONCURRENT_STREAMS`**.
8.0.0	O parâmetro `multi_handle` agora espera uma instância de CurlMultiHandle; anteriormente, um resource era esperado.
7.1.0	Introduzida a constante **`CURLMOPT_PUSHFUNCTION`**.
7.0.7	Introduzidas as constantes **`CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE`**, **`CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE`**, **`CURLMOPT_MAX_HOST_CONNECTIONS`**, **`CURLMOPT_MAX_PIPELINE_LENGTH`** e **`CURLMOPT_MAX_TOTAL_CONNECTIONS`**.

curl_multi_strerror

curl_multi_strerror

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_multi_strerror — Retorna uma string que descreve o código de erro

Descrição

function **curl_multi_strerror**(int `$error_code`): ?string

Retorna um texto descrevendo o código de erro **`CURLM_*`** informado.

Parâmetros

`error_code`

Uma das constantes **`CURLM_*`**.

Valor Retornado

Retorna a descrição do erro para um código de erro válido, caso contrário **`null`**.

Veja também

- curl_strerror() - Retorna uma string que descreve o código de erro informado
- » Códigos de erros do cURL

curl_pause

curl_pause

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_pause — Pausa ou continua uma conexão

Descrição

function **curl_pause**(CurlHandle `$handle`, int `$flags`): int

Pausa ou continua uma sessão cURL. Uma sessão pode ser pausada enquanto uma transferência estiver em progresso, em leitura, escrita, ou ambos, através de chamada a esta função a partir de uma chamade de retorno registrada com curl_setopt().

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`flags`

Uma das constantes **CURLPAUSE_***.

Valor Retornado

Retorna um código de erro (**CURLE_OK** se não houver erro).

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

curl_reset

curl_reset

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_reset — Redefine todas as opções de um identificador de sessão libcurl

Descrição

function **curl_reset**(CurlHandle `$handle`): void

Essa função reinicializa todas as opções configuradas no identificador cURL informado para os valores padrões.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_reset()

```php
<?php
// Cria um identificador curl
$ch = curl_init();

// Configura a opção CURLOPT_USERAGENT
curl_setopt($ch, CURLOPT_USERAGENT, "My test user-agent");

// Redefine todas as opções que foram configuradas anteriormente
```

```php
curl_reset($ch);

// Envia a requisição HTTP
curl_setopt($ch, CURLOPT_URL, 'http://example.com/');
curl_exec($ch); // o user-agent configurado anteriormente não será enviado porque foi redefinido por curl_reset
?>
```

Notas

> **Nota** :
>
> **curl_reset()** também reseta a URL informada no parâmetro curl_init().

Veja também

- curl_setopt() - Define uma opção para uma transferência cURL

curl_setopt

curl_setopt

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_setopt — Define uma opção para uma transferência cURL

Descrição

function **curl_setopt**(CurlHandle `$handle`, int `$option`, mixed `$value`): bool

Define uma opção no identificador de sessão cURL fornecido.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`option`

A opção `CURLOPT_*` a ser definida.

`value`

O valor a ser definido em `option`. Consulte a descrição das constantes `CURLOPT_*` para detalhes sobre o tipo de valor que cada constante espera.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.4.0	`CURLOPT_DNS_USE_GLOBAL_CACHE` não tem mais qualquer efeito, e habilitála em compilações PHP seguras para threads não dispara mais um alerta.
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.
7.3.15, 7.4.3	Introduzido `CURLOPT_HTTP09_ALLOWED`.
7.3.0	Introduzido `CURLOPT_ABSTRACT_UNIX_SOCKET`, `CURLOPT_KEEP_SENDING_ON_ERROR`, `CURLOPT_PRE_PROXY`, `CURLOPT_PROXY_CAINFO`, `CURLOPT_PROXY_CAPATH`, `CURLOPT_PROXY_CRLFILE`, `CURLOPT_PROXY_KEYPASSWD`, `CURLOPT_PROXY_PINNEDPUBLICKEY`, `CURLOPT_PROXY_SSLCERT`, `CURLOPT_PROXY_SSLCERTTYPE`, `CURLOPT_PROXY_SSL_CIPHER_LIST`, `CURLOPT_PROXY_SSLKEY`, `CURLOPT_PROXY_SSLKEYTYPE`, `CURLOPT_PROXY_SSL_OPTIONS`, `CURLOPT_PROXY_SSL_VERIFYHOST`, `CURLOPT_PROXY_SSL_VERIFYPEER`, `CURLOPT_PROXY_SSLVERSION`, `CURLOPT_PROXY_TLSAUTH_PASSWORD`, `CURLOPT_PROXY_TLSAUTH_TYPE`, `CURLOPT_PROXY_TLSAUTH_USERNAME`, `CURLOPT_SOCKS5_AUTH`, `CURLOPT_SUPPRESS_CONNECT_HEADERS`, `CURLOPT_DISALLOW_USERNAME_IN_URL`, `CURLOPT_DNS_SHUFFLE_ADDRESSES`, `CURLOPT_HAPPY_EYEBALLS_TIMEOUT_MS`, `CURLOPT_HAPROXYPROTOCOL`, `CURLOPT_PROXY_TLS13_CIPHERS`, `CURLOPT_SSH_COMPRESSION`, `CURLOPT_TIMEVALUE_LARGE` e `CURLOPT_TLS13_CIPHERS`.

Versão	Descrição
7.0.7	Introduzido CURL_HTTP_VERSION_2, CURL_HTTP_VERSION_2_PRIOR_KNOWLEDGE, CURL_HTTP_VERSION_2TLS, CURL_REDIR_POST_301, CURL_REDIR_POST_302, CURL_REDIR_POST_303, CURL_REDIR_POST_ALL, CURL_VERSION_KERBEROS5, CURL_VERSION_PSL, CURL_VERSION_UNIX_SOCKETS, CURLAUTH_NEGOTIATE, CURLAUTH_NTLM_WB, CURLFTP_CREATE_DIR, CURLFTP_CREATE_DIR_NONE, CURLFTP_CREATE_DIR_RETRY, CURLHEADER_SEPARATE, CURLHEADER_UNIFIED, CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE, CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE, CURLMOPT_MAX_HOST_CONNECTIONS, CURLMOPT_MAX_PIPELINE_LENGTH, CURLMOPT_MAX_TOTAL_CONNECTIONS, CURLOPT_CONNECT_TO, CURLOPT_DEFAULT_PROTOCOL, CURLOPT_DNS_INTERFACE, CURLOPT_DNS_LOCAL_IP4, CURLOPT_DNS_LOCAL_IP6, CURLOPT_EXPECT_100_TIMEOUT_MS, CURLOPT_HEADEROPT, CURLOPT_LOGIN_OPTIONS, CURLOPT_PATH_AS_IS, CURLOPT_PINNEDPUBLICKEY, CURLOPT_PIPEWAIT, CURLOPT_PROXY_SERVICE_NAME, CURLOPT_PROXYHEADER, CURLOPT_SASL_IR, CURLOPT_SERVICE_NAME, CURLOPT_SSL_ENABLE_ALPN, CURLOPT_SSL_ENABLE_NPN, CURLOPT_SSL_FALSESTART, CURLOPT_SSL_VERIFYSTATUS, CURLOPT_STREAM_WEIGHT, CURLOPT_TCP_FASTOPEN, CURLOPT_TFTP_NO_OPTIONS, CURLOPT_UNIX_SOCKET_PATH, CURLOPT_XOAUTH2_BEARER, CURLPROTO_SMB, CURLPROTO_SMBS, CURLPROXY_HTTP_1_0, CURLSSH_AUTH_AGENT e CURLSSLOPT_NO_REVOKE.

Examples

Exemplo #1 Inicializando uma nova sessão cURL e buscando uma página da web

```php
<?php
// cria um novo identificador cURL
$ch = curl_init();

// define URL e outras opções apropriadas
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, false);

// pega URL e passá-lo para o navegador
curl_exec($ch);
?>
```

Notas

> **Nota** :
>
> Passar um array para CURLOPT_POSTFIELDS codificará os dados como *multipart/form-data*, enquanto passar uma string codificada em URL codificará os dados como *application/x-www-form-urlencoded*.

Veja também

- curl_setopt_array() - Define múltiplas opções para uma transferência cURL
- CURLFile
- CURLStringFile

curl_setopt_array

curl_setopt_array

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

curl_setopt_array — Define múltiplas opções para uma transferência cURL

Descrição

function **curl_setopt_array**(CurlHandle `$handle`, array `$options`): bool

Define múltiplas opções para uma sessão cURL. Esta função é útil para definir um alto número de opções cURL sem chamar repetidas vezes a função curl_setopt().

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`options`

Um array especificando que opções serão definidas e seus valores. As chaves devem ser constantes válidas da função curl_setopt() ou suas equivalentes em número inteiro.

Valor Retornado

Retorna `true` se todas as opções forem definidas com sucesso. Se uma opção não puder ser definida com sucesso, `false` é retornado imediatamente, ignorando quaisquer opções posteriores do array `options`.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Inicializando uma nova sessão cURL e recebendo uma página da web

```php
<?php
// cria um novo identificador cURL
$ch = curl_init();

// define URL e outras opções apropriadas
$options = array(CURLOPT_URL => 'http://www.example.com/',
                 CURLOPT_HEADER => false
                );

curl_setopt_array($ch, $options);

// captura a URL e passa ao navegador
curl_exec($ch);
?>
```

Notas

> **Nota** :
>
> Assim como a função curl_setopt(), passar um array em `CURLOPT_POST` irá codificar os dados como *multipart/form-data*, enquanto que passar uma string codificada como URL irá codificar os dados como *application/x-www-form-urlencoded*.

Veja também

- curl_setopt() - Define uma opção para uma transferência cURL

curl_share_close

curl_share_close

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_close — Fecha um identificador de compartilhamento de cURL

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **curl_share_close**(CurlShareHandle `$share_handle`): void

> **Nota** :
>
> Esta função não tem efeito. Antes do PHP 8.0.0, esta função era usada para fechar o recurso.

Fecha um identificador de compartilhamento cURL e libera todos os recursos.

Parâmetros

`share_handle`

Um manipulador cURL de compartilhamento retornado por curl_share_init().

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Esta função foi descontinuada.
8.0.0	Esta função agora é um NOP.
8.0.0	O parâmetro `share_handle` agora espera uma instância de CurlShareHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_share_setopt()

Este exemplo irá criar um identificador de compartilhamento de cURL, adicionar dois identificadores cURL nele, e então executá-los com o compartilhamento de cookies.

```php
<?php
// Cria o identificador de compartilhamento de cURL e configura-o para compartilhar os cookies
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Inicializa o primeiro identificador cURL e atribui o identificador de compartilhamento à ele
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Executa o primeiro identificador cURL
curl_exec($ch1);

// Inicializa o segundo identificador cURL e atribui o identificador de compartilhamento à ele
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Executa o segundo identificador cURL
// Todos os cookies do identificador $ch1 são compartilhados com o identificador $ch2
curl_exec($ch2);

// Fecha o identificador de compartilhamento de cURL
curl_share_close($sh);
?>
```

Veja também

- curl_share_init() - Inicializa um identificador de compartilhamento de cURL

curl_share_errno

curl_share_errno

(PHP 7 >= 7.1.0, PHP 8)

curl_share_errno — Retorna o último número de erro do curl compartilhado

Descrição

function **curl_share_errno**(CurlShareHandle `$share_handle`): int

Retorna um número inteiro contendo o último número de erro de curl compartilhado.

Parâmetros

`share_handle`

Um manipulador cURL de compartilhamento retornado por curl_share_init().

Valor Retornado

Retorna um inteiro contendo o último número de erro de curl compartilhado.

Registro de Alterações

Versão	Descrição
8.0.0	A função não retorna mais `false` em caso de falha.
8.0.0	O parâmetro `share_handle` agora espera uma instância de CurlShareHandle; anteriormente, um resource era esperado.

Veja também

- curl_errno() - Retorna o número do último erro

curl_share_init

curl_share_init

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_init — Inicializa um identificador de compartilhamento de cURL

Descrição

function **curl_share_init**(): CurlShareHandle

Permite compartilhar dados entre identificadores cURL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um identificador de compartilhamento cURL.

Registro de Alterações

Versão	Descrição
8.0.0	Este função agora retorna uma instância de CurlShareHandle; anteriormente, um recurso do tipo resource era retornado.

Examples

Exemplo #1 Exemplo de curl_share_init()

Este exemplo irá criar um identificador de compartilhamento de cURL, adicionar dois identificadores cURL nele, e então executá-los com o compartilhamento de cookies.

```
<?php
// Cria o identificador de compartilhamento de cURL e configura-o para compartilhar os cookies
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Inicializa o primeiro identificador cURL e atribui o identificador de compartilhamento à ele
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Executa o primeiro identificador cURL
curl_exec($ch1);

// Inicializa o segundo identificador cURL e atribui o identificador de compartilhamento à ele
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Executa o segundo identificador cURL
// Todos os cookies do identificador $ch1 são compartilhados com o identificador $ch2
```

```
curl_exec($ch2);
?>
```

Veja também

- curl_share_setopt() - Configura uma opção para um identificador de compartilhamento cURL
- curl_share_init_persistent() - Inicializa um identificador compartilhado cURL persistente

curl_share_init_persistent

curl_share_init_persistent

(PHP 8 >= 8.5.0)

curl_share_init_persistent — Inicializa um identificador compartilhado cURL **persistente**

Descrição

function **curl_share_init_persistent**(array `$share_options`): CurlSharePersistentHandle

Inicializa um identificador compartilhado cURL **persistente** com as opções de compartilhamento informadas. Diferente de curl_share_init(), os identificadores criados por esta função não serão destruídos ao final da requisição PHP. Se um identificador compartilhado persistente com o mesmo conjunto de `share_options` for encontrado, ele será reutilizado.

Parâmetros

`share_options`

Um array não vazio de constantes **`CURL_LOCK_DATA_*`**.

> **Nota** :
>
> **`CURL_LOCK_DATA_COOKIE`** não é permitida e, se utilizada, esta função lançará uma exceção ValueError. Compartilhar cookies entre requisições PHP pode levar a uma mistura inadvertida de cookies sensíveis entre usuários.

Valor Retornado

Retorna um objeto CurlSharePersistentHandle.

Erros/Exceções

- Se `share_options` estiver vazio, esta função lança uma exceção ValueError.
- Se `share_options` contiver um valor que não corresponde a uma constante **`CURL_LOCK_DATA_*`**, esta função lança uma exceção ValueError.
- Se `share_options` contiver **`CURL_LOCK_DATA_COOKIE`**, esta função lança uma exceção ValueError.
- Se `share_options` contiver um valor não inteiro, esta função lança uma exceção TypeError.

Examples

Exemplo #1 Exemplo de **curl_share_init_persistent()**

Este exemplo criará um identificador de compartilhamento cURL persistente e demonstrará conexões compartilhadas entre si. Se isso for executado em um PHP SAPI de longa duração, `$sh` sobreviverá entre as requisições SAPI.

```
<?php

// Cria ou recupera um identificador de compartilhamento cURL persistente definido para compartilhar pesquisas e conexões de
DNS
$sh = curl_share_init_persistent([CURL_LOCK_DATA_DNS, CURL_LOCK_DATA_CONNECT]);

// Inicializa o primeiro identificador cURL e atribui a ele o identificador de compartilhamento
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Executa o primeiro identificador cURL que pode reutilizar a conexão de uma solicitação SAPI anterior
```

```php
curl_exec($ch1);

// Inicializa o segundo identificador cURL e atribui o identificador de compartilhamento a ele
$ch2 = curl_init("http://example.com/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Executa o segundo identificador cURL. Isso reutilizará a conexão de $ch1
curl_exec($ch2);

?>
```

Veja também

- curl_setopt() - Define uma opção para uma transferência cURL
- curl_share_init() - Inicializa um identificador de compartilhamento de cURL

curl_share_setopt

curl_share_setopt

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_setopt — Configura uma opção para um identificador de compartilhamento cURL

Descrição

function **curl_share_setopt**(CurlShareHandle `$share_handle`, int `$option`, mixed `$value`): bool

Configura uma opção no identificador de compartilhamento cURL informado.

Parâmetros

`share_handle`

Um manipulador cURL de compartilhamento retornado por curl_share_init().

`option`

Uma das constantes **`CURLSHOPT_*`**.

`value`

Uma das constantes **`CURL_LOCK_DATA_*`**.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `share_handle` agora espera uma instância de CurlShareHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de **curl_share_setopt()**

Este exemplo irá criar um identificador de compartilhamento de cURL, adicionar dois identificadores cURL nele, e então executá-los com o compartilhamento de cookies.

```php
<?php
// Cria o identificador de compartilhamento de cURL e configura-o para compartilhar os cookies
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Inicializa o primeiro identificador cURL e atribui o identificador de compartilhamento à ele
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Executa o primeiro identificador cURL
curl_exec($ch1);
```

```php
// Inicializa o segundo identificador cURL e atribui o identificador de compartilhamento à ele
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Executa o segundo identificador cURL
// Todos os cookies do identificador $ch1 são compartilhados com o identificador $ch2
curl_exec($ch2);
?>
```

curl_share_strerror

curl_share_strerror

(PHP 7 >= 7.1.0, PHP 8)

curl_share_strerror — Retorna uma string descrevendo o código de erro fornecido

Descrição

function **curl_share_strerror**(int `$error_code`): ?string

Retorna uma mensagem de erro de texto descrevendo o código de erro fornecido.

Parâmetros

`error_code`

Uma das constantes dos » códigos de erro cURL.

Valor Retornado

Retorna a descrição do erro ou `null` para código de erro inválido.

Veja também

- curl_share_errno() - Retorna o último número de erro do curl compartilhado
- curl_strerror() - Retorna uma string que descreve o código de erro informado

curl_strerror

curl_strerror

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_strerror — Retorna uma string que descreve o código de erro informado

Descrição

function **curl_strerror**(int `$error_code`): ?string

Retorna um texto descrevendo o código de erro informado como parâmetro.

Parâmetros

`error_code`

Uma das constantes de » códigos de erros do cURL.

Valor Retornado

Retorna a descrição do erro ou `null` se o código de erro for inválido.

Examples

Exemplo #1 Exemplo de curl_errno()

```php
<?php
// Cria um identificador curl com um protocolo escrito de forma incorreta na URL
$ch = curl_init("htp://example.com/");
```

```php
// Envia a requisição
curl_exec($ch);

// Verifica se houve erros e exibe a mensagem de erro
if($errno = curl_errno($ch)) {
    $error_message = curl_strerror($errno);
    echo "Erro cURL ({$errno}):\n {$error_message}";
}
?>
```

O exemplo acima produzirá:

```
Erro cURL (1):
 Unsupported protocol
```

Veja também

- curl_errno() - Retorna o número do último erro
- curl_error() - Retorna uma string contendo o último erro da sessão atual
- » Códigos de erros do cURL

curl_unescape

curl_unescape

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_unescape — Decodifica a parte codificada da URL fornecida

Descrição

function **curl_unescape**(CurlHandle `$handle`, string `$string`): string|false

Essa função decodifica a parte codificada da URL fornecida.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

`string`

A URL codificada a ser decodificada.

Valor Retornado

Retorna a string decodificada ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	O parâmetro `handle` agora espera uma instância de CurlHandle; anteriormente, um resource era esperado.

Examples

Exemplo #1 Exemplo de curl_escape()

```php
<?php
// Cria um identificador curl
$ch = curl_init('http://example.com/redirect.php');

// Envia a requisição HTTP e segue os redirecionamentos
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, 1);
```

```php
curl_exec($ch);

// Captura a última URL
$effective_url = curl_getinfo($ch, CURLINFO_EFFECTIVE_URL);
// ie. "http://example.com/show_location.php?loc=M%C3%BCnchen"

// Decodifica a URL
$effective_url_decoded = curl_unescape($ch, $effective_url);
// "http://example.com/show_location.php?loc=München"
?>
```

Notas

> **Nota** :
>
> **curl_unescape()** não decodifica os símbolos de mais (+) em espaços. Mas urldecode() faz isso.

Veja também

- curl_escape() - Codifica uma string para o formato URL
- urlencode() - Codifica uma string no formato URL
- urldecode() - Decodifica uma URL codificada
- rawurlencode() - Codifica uma URL de acordo com RFC 3986
- rawurldecode() - Decodifica uma URL

curl_upkeep

curl_upkeep

(PHP 8 >= 8.2.0)

curl_upkeep — Executa qualquer verificação de manutenção de conexão

Descrição

function **curl_upkeep**(CurlHandle `$handle`): bool

Disponível se construído em libcurl >= 7.62.0.

Alguns protocolos possuem mecanismos de "manutenção de conexão". Esses mecanismos geralmente enviam algum tráfego nas conexões existentes para mantê-las vivas; isso pode impedir que as conexões sejam fechadas devido a firewalls excessivamente zelosos, por exemplo.

A manutenção da conexão está atualmente disponível apenas para conexões HTTP/2. Uma pequena quantidade de tráfego geralmente é enviada para manter uma conexão ativa. O HTTP/2 mantém sua conexão enviando um quadro PING HTTP/2.

Parâmetros

`handle`

Um manipulador cURL retornado por curl_init().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de curl_upkeep()

```php
<?php
$url = "https://example.com";

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_HTTP_VERSION,CURL_HTTP_VERSION_2_0);
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
curl_setopt($ch, CURLOPT_UPKEEP_INTERVAL_MS, 200);
```

```php
if (curl_exec($ch)) {
    usleep(300);
    var_dump(curl_upkeep($ch));
}
?>
```

Veja também

- curl_init() - Inicializa uma sessão cURL

curl_version

curl_version

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_version — Obtém informação de versão do cURL

Descrição

function **curl_version**(): array|false

Retorna informação sobre a versão do cURL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array associativo com os seguintes elementos:

Chave	Descrição do Valor
version_number	Número de 24 bits da versão do cURL
version	String com a versão do cURL
ssl_version_number	Número de 24 bits da versão do OpenSSL
ssl_version	String com a versão do OpenSSL
libz_version	String com a versão do zlib
host	Informação sobre o servidor onde o cURL foi compilado
age	
features	Uma máscara de bits das constantes `CURL_VERSION_*`
protocols	Um array de nomes de protocolos suportados pelo cURL
feature_list	Um array associativo de todos os recursos conhecidos do cURL, e se eles são suportados (`true`) ou não (`false`)

Registro de Alterações

Versão	Descrição
8.4.0	Chave `features_list` adicionada.
8.0.0	O parâmetro opcional `age` foi removido.
7.4.0	O parâmetro opcional `age` foi descontinuado; se um valor for passado, será ignorado.

Examples

Exemplo #1 Exemplo de curl_version()

Este exemplo verifica quais características estão disponíveis na compilação do cURL usando a máscara de bits `'features'` retornada por **curl_version()**.

```php
<?php
// Obtém o array de versão do curl
$version = curl_version();

// Estes são os campos de bit que podem ser usados
```

```php
// para verificar características na compilação do curl
$bitfields = Array(
            'CURL_VERSION_IPV6',
            'CURL_VERSION_KERBEROS4',
            'CURL_VERSION_SSL',
            'CURL_VERSION_LIBZ'
            );


foreach($bitfields as $feature)
{
    echo $feature . ($version['features'] & constant($feature) ? ' presente' : ' ausente');
    echo PHP_EOL;
}
?>
```

CurlHandle

(PHP 8)

Introdução

Uma classe totalmente opaca que substitui os recursos `curl` a partir do PHP 8.0.0.

Resumo da classe

final class **CurlHandle** {
}

CurlMultiHandle

(PHP 8)

Introdução

Uma classe totalmente opaca que substitui os recursos `curl_multi` a partir do PHP 8.0.0.

Resumo da classe

final class **CurlMultiHandle** {
}

CurlShareHandle

(PHP 8)

Introdução

Uma classe totalmente opaca que substitui os recursos `curl_share` a partir do PHP 8.0.0.

Resumo da classe

final class **CurlShareHandle** {
}

CurlSharePersistentHandle

(PHP 8 >= 8.5.0)

Introdução

Representa um identificador cURL "compartilhado" persistente.

Resumo da classe

final class **CurlSharePersistentHandle** {
/* Propriedades */
public readonly array *$options*;

}

Propriedades

options

Os caches `CURL_LOCK_DATA_*` compartilhados entre requisições que usam este identificador.

CURLFile

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introdução

Esta classe ou CURLStringFile deve ser utilizada para fazer o envio de arquivos com `CURLOPT_POSTFIELDS`.

A desserialização de instâncias **CURLFile** não é permitida. A partir do PHP 7.4.0, é explicitamente proibida.

Resumo da classe

class **CURLFile** {
/* Propriedades */
public string *$name* = "";
public string *$mime* = "";
public string *$postname* = "";
/* Métodos */
public function __construct(string `$filename`, ?string `$mime_type` = `null`, ?string `$posted_filename` = `null`)
public function getFilename(): string
public function getMimeType(): string
public function getPostFilename(): string
public function setMimeType(string `$mime_type`): void
public function setPostFilename(string `$posted_filename`): void
}

Propriedades

name

Nome do arquivo a ser feito o upload.

mime

MIME type do arquivo (o padrão é `application/octet-stream`).

postname

O nome do arquivo no campo de upload (o valor padrão é a propriedade *name*).

Veja também

- curl_setopt()
- CURLStringFile

Índice

CURLFile::__construct

CURLFile::__construct

curl_file_create

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::__construct -- curl_file_create — Cria um objeto CURLFile

Descrição

Estilo orientado a objetos

public function **CURLFile::__construct**(string `$filename`, ?string `$mime_type = null`, ?string `$posted_filename = null`)

Estilo procedural

function **curl_file_create**(string `$filename`, ?string `$mime_type = null`, ?string `$posted_filename = null`): CURLFile

Cria um objeto CURLFile, usado para o upload de um arquivo **`CURLOPT_POSTFIELDS`**.

Parâmetros

`filename`

Caminho para o arquivo que será enviado.

`mime_type`

O tipo de mídia do arquivo.

`posted_filename`

Nome do arquivo a ser usado nos dados de envio.

Valor Retornado

Retorna um objeto CURLFile.

Registro de Alterações

Versão	Descrição
8.0.0	`mime_type` e `posted_filename` agora podem ser nulos; anteriormente seus valores padrões eram `0`.

Examples

Exemplo #1 Exemplo de CURLFile::__construct()

Estilo orientado a objetos

```php
<?php
/* http://example.com/upload.php:
<?php var_dump($_FILES); ?>
*/

// Cria um manipulador cURL
$ch = curl_init('http://example.com/upload.php');

// Cria um objeto CURLFile
$cfile = new CURLFile('cats.jpg','image/jpeg','test_name');

// Atribui dados ao POST
$data = array('test_file' => $cfile);
curl_setopt($ch, CURLOPT_POST,1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Executa o manipulador
curl_exec($ch);
?>
```

Estilo procedural

```php
<?php
/* http://example.com/upload.php:
<?php var_dump($_FILES); ?>
*/

// Cria um manipulador cURL
$ch = curl_init('http://example.com/upload.php');

// Cria um objeto CURLFile
$cfile = curl_file_create('cats.jpg','image/jpeg','test_name');

// Atribui dados ao POST
```

```
$data = array('test_file' => $cfile);
curl_setopt($ch, CURLOPT_POST,1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Executa o manipulador
curl_exec($ch);
?>
```

O exemplo acima produzirá:

```
array(1) {
  ["test_file"]=>
  array(5) {
    ["name"]=>
    string(9) "test_name"
    ["type"]=>
    string(10) "image/jpeg"
    ["tmp_name"]=>
    string(14) "/tmp/phpPC9Kbx"
    ["error"]=>
    int(0)
    ["size"]=>
    int(46334)
  }
}
```

Exemplo #2 Exemplo de CURLFile::__construct() enviando múltiplos arquivos

Estilo orientado a objetos

```
<?php
$request = curl_init('http://www.example.com/upload.php');
curl_setopt($request, CURLOPT_POST, true);
curl_setopt($request, CURLOPT_SAFE_UPLOAD, true);
curl_setopt($request, CURLOPT_POSTFIELDS, [
    'blob[0]' => new CURLFile(realpath('first-file.jpg'), 'image/jpeg'),
    'blob[1]' => new CURLFile(realpath('second-file.txt'), 'text/plain'),
    'blob[2]' => new CURLFile(realpath('third-file.exe'), 'application/octet-stream'),
]);
curl_setopt($request, CURLOPT_RETURNTRANSFER, true);

echo curl_exec($request);

var_dump(curl_getinfo($request));
```

Estilo procedural

```
<?php
// procedural
$request = curl_init('http://www.example.com/upload.php');
curl_setopt($request, CURLOPT_POST, true);
curl_setopt($request, CURLOPT_SAFE_UPLOAD, true);
curl_setopt($request, CURLOPT_POSTFIELDS, [
    'blob[0]' => curl_file_create(realpath('first-file.jpg'), 'image/jpeg'),
    'blob[1]' => curl_file_create(realpath('second-file.txt'), 'text/plain'),
    'blob[2]' => curl_file_create(realpath('third-file.exe'), 'application/octet-stream'),
]);
curl_setopt($request, CURLOPT_RETURNTRANSFER, true);

echo curl_exec($request);

var_dump(curl_getinfo($request));
```

O exemplo acima produzirá:

```
array(26) {
  ["url"]=>
  string(31) "http://www.example.com/upload.php"
  ["content_type"]=>
  string(24) "text/html; charset=UTF-8"
  ["http_code"]=>
  int(200)
  ["header_size"]=>
  int(198)
  ["request_size"]=>
  int(196)
  ["filetime"]=>
  int(-1)
  ["ssl_verify_result"]=>
  int(0)
  ["redirect_count"]=>
  int(0)
  ["total_time"]=>
  float(0.060062)
  ["namelookup_time"]=>
  float(0.028575)
  ["connect_time"]=>
  float(0.029011)
  ["pretransfer_time"]=>
  float(0.029121)
  ["size_upload"]=>
  float(3230730)
  ["size_download"]=>
  float(811)
  ["speed_download"]=>
  float(13516)
  ["speed_upload"]=>
  float(53845500)
  ["download_content_length"]=>
  float(811)
  ["upload_content_length"]=>
  float(3230730)
  ["starttransfer_time"]=>
  float(0.030355)
  ["redirect_time"]=>
  float(0)
  ["redirect_url"]=>
  string(0) ""
  ["primary_ip"]=>
  string(13) "0.0.0.0"
  ["certinfo"]=>
  array(0) {
  }
  ["primary_port"]=>
  int(80)
  ["local_ip"]=>
  string(12) "0.0.0.0"
  ["local_port"]=>
  int(34856)
}
```

Veja também

- curl_setopt() - Define uma opção para uma transferência cURL

CURLFile::getFilename

CURLFile::getFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getFilename — Obtém o nome do arquivo

Descrição

public function **CURLFile::getFilename**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do arquivo.

CURLFile::getMimeType

CURLFile::getMimeType

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getMimeType — Obtém o tipo de mídia

Descrição

public function **CURLFile::getMimeType**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tipo de MIME.

CURLFile::getPostFilename

CURLFile::getPostFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getPostFilename — Obtém o nome do arquivo para requisição POST

Descrição

public function **CURLFile::getPostFilename**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do arquivo utilizando o POST.

CURLFile::setMimeType

CURLFile::setMimeType

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::setMimeType — Define o tipo de mídia

Descrição

public function **CURLFile::setMimeType**(string `$mime_type`): void

Parâmetros

`mime_type`

Tipo de mídia a ser utilizado nos dados da requisição POST.

Valor Retornado

Nenhum valor é retornado.

CURLFile::setPostFilename

CURLFile::setPostFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::setPostFilename — Define o nome do arquivo para a requisição POST

Descrição

public function **CURLFile::setPostFilename**(string `$posted_filename`): void

Parâmetros

`posted_filename`

Nome do arquivo a ser utilizado nos dados da requisição POST.

Valor Retornado

Nenhum valor é retornado.

CURLStringFile

(PHP 8 >= 8.1.0)

Introdução

CURLStringFile torna possível enviar um arquivo diretamente de uma variável. Isto é semelhante a CURLFile, mas funciona com o conteúdo do arquivo, não com o nome do arquivo. Esta classe ou CURLFile deve ser usada para enviar o conteúdo do arquivo com `CURLOPT_POSTFIELDS`.

Resumo da classe

class **CURLStringFile** {
/* Propriedades */
public string *$data*;
public string *$postname*;
public string *$mime*;
/* Métodos */
public function __construct(string `$data`, string `$postname`, string `$mime` = "application/octet-stream")
}

Propriedades

data

O conteúdo a ser enviado.

postname

O nome do arquivo a ser usado nos dados de envio.

mime

Tipo de mídia do arquivo (o padrão é `application/octet-stream`).

Veja também

- curl_setopt()
- CURLFile

Índice

CURLStringFile::__construct

CURLStringFile::__construct

(PHP 8 >= 8.1.0)

CURLStringFile::__construct — Cria um objeto CURLStringFile

Descrição

public function **CURLStringFile::__construct**(string `$data`, string `$postname`, string `$mime` = "application/octet-stream")

Cria um objeto CURLStringFile, usado para carregar um arquivo com **`CURLOPT_POSTFIELDS`**.

Parâmetros

`data`

O conteúdo a ser carregado.

`postname`

O nome do arquivo a ser usado nos dados de carregamento.

`mime`

Tipo MIME do arquivo (o padrão é `application/octet-stream`).

Examples

Exemplo #1 Exemplo de CURLStringFile::__construct()

```php
<?php
/* http://example.com/upload.php:
<?php
var_dump($_FILES);
var_dump(file_get_contents($_FILES['test_string']['tmp_name']));
?>
*/

// Cria um identificador cURL
$ch = curl_init('http://example.com/upload.php');

// Cria um objeto CURLStringFile
$cstringfile = new CURLStringFile('test upload contents','test.txt','text/plain');

// Atribui dados POST
$data = array('test_string' => $cstringfile);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Executa o manipulador
curl_exec($ch);
?>
```

O exemplo acima produzirá:

```
array(1) {
  ["test_string"]=>
  array(5) {
    ["name"]=>
    string(8) "test.txt"
    ["type"]=>
    string(10) "text/plain"
    ["tmp_name"]=>
    string(14) "/tmp/phpTtaoCz"
    ["error"]=>
    int(0)
    ["size"]=>
    int(20)
  }
```

```
}
string(20) "test upload contents"
```

Veja também

- curl_setopt() - Define uma opção para uma transferência cURL

Estrutura de resumo de mensagens Hash

Introdução

Esta extensão fornece funções que podem ser usadas para processamento direto ou incremental de mensagens de comprimento arbitrário usando uma variedade de algortimos de hash, incluindo a geração de valores HMAC e derivações chave, incluindo HKDF e PBKDF2.

De forma geral, existem três categorias de algoritmos de hash. Uma lista completa de algoritmos pode ser encontrada na documentação da função hash_algos().

- Algoritmos de soma de verificação (como `"crc32b"` ou `"adler32"`): Estes são usados para calcular somas de verificação, úteis em situações em que erros de transmissão precisam ser detectados. São geralmente muito rápidos. Estes algoritmos frequentemente geram valores que facilmente "previsíveis" ou podem ser manipulados para criar colisões, portanto são totalmente inadequados para uso em criptografia.
- Algoritmos não criptográficos (como a família xxHash): Estes são frequentemente usados para calcular valores de hash para tabelas de hash, já que são projetados para gerar uma boa distribuição sobre strings de entrada arbitrárias. Também são geralmente rápidos, mas também inadequados para uso em criptografia.
- Algoritmos criptográficos (como a família SHA-2): Estes são projetados para gerar valores de hash que são representativos de suas entradas mas não são previsíveis e nem sujeitos a colisões. O desempenho geralmente é uma preocupação secundária, mas os equipamentos modernos frequentemente suportam um gerenciamento especial para estes algoritmos que o PHP tenta fazer uso quando disponível.

  O Centro de Recursos de Segurança Computacional do NIST tem uma » explicação dos algoritmos atualmente aprovados pelos padrões de processamento de informação dos Estados Unidos.

  > **Cuidado**
  >
  > Já foi provado que alguns dos primeiros algoritmos criptográficos, tais como `"md4"`, `"md5"` e `"sha1"`, estão sujeiros a ataques de colisão e por isso é recomendado que não sejam mais usados para aplicações de criptografia.

Consulte também as Perguntas Frequentes sobre Hash Seguro de Senhas para informação sobre melhores práticas para o uso de funções de hash ao lidar com senhas.

Instalação/Configuração

Índice

Instalação

A extensão Hash é uma extensão do núcleo do PHP, por isso está sempre habilitada.

Antes do PHP 7.4.0, a extensão Hash era incluída e compilada no PHP por padrão mas podia ser desabilitada explicitamente usando **--disable-hash**.

Antes do PHP 5.1.2, a extensão Hash era instalada como um » módulo PECL.

Tipos de Recurso

Esta extensão define um recurso de Contexto de Hash retornado por hash_init().

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

HASH_HMAC (int)

Sinalizador opcional para hash_init(). Indica que o algoritmo de chaveamento de digestão HMAC deve ser aplicado ao contexto de hash atual.

HashContext

(PHP 7 >= 7.2.0, PHP 8)

Introdução

Resumo da classe

final class **HashContext** {
/* Métodos */
private function __construct()
public function __debugInfo(): array
public function __serialize(): array
public function __unserialize(array `$data`): void
}

Índice

HashContext::__construct

HashContext::__construct

(PHP 7 >= 7.2.0, PHP 8)

HashContext::__construct — Construtor privado para impedir instanciação direta

Descrição

private function **HashContext::__construct**()

Parâmetros

Esta função não possui parâmetros.

HashContext::__debugInfo

HashContext::__debugInfo

(PHP 8 >= 8.4.0)

HashContext::__debugInfo — Retorna informação de depuração sobre o contexto de hash

Descrição

public function **HashContext::__debugInfo**(): array

Este método não foi feito para ser chamado diretamente; ele é invocado por var_dump() e funções relacionadas ao inspecionar uma instância de HashContext instance.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array associativo com informações de depuração. Ele contém uma chave `algo` com o nome do algoritmo de hash em uso pelo contexto.

Examples

Exemplo #1 Exemplo de **HashContext::__debugInfo()** example

```php
<?php
$ctx = hash_init('sha256');
var_dump($ctx);
?>
```

O exemplo acima produzirá:

```
object(HashContext)#1 (1) {
  ["algo"]=>
  string(6) "sha256"
}
```

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- var_dump() - Despeja na saída informações sobre uma variável

HashContext::__serialize

HashContext::__serialize

(PHP 8)

HashContext::__serialize — Serializa o objeto HashContext

Descrição

public function **HashContext::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

HashContext::__unserialize

HashContext::__unserialize

(PHP 8)

HashContext::__unserialize — Desserializa o parâmetro `data` em um objeto HashContext

Descrição

public function **HashContext::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

O valor a ser desserializado.

Valor Retornado

Nenhum valor é retornado.

Hash Funções

Índice

hash

hash

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash — Gera um valor hash (resumo da mensagem)

Descrição

```
function hash(
    string $algo,
    string $data,
    bool $binary = false,
    array $options = []
): string
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_algos().

`data`

Mensagem a ter o hash gerado.

`binary`

Quando definido como `true`, gera dados binários brutos. `false` gera caracteres hexadecimais minúsculos.

`options`

Um array de opções para os vários algoritmos de hash. Atualmente, apenas o parâmetro `"seed"` é suportado pelas variantes MurmurHash.

Valor Retornado

Retorna uma string contendo o resumo da mensagem calculado como caracteres hexadecimais minúsculos a menos que `binary` seja definido como true, caso em que a representação binária bruta do resumo da mensagem é retornada.

Erros/Exceções

Lança uma exceção ValueError se `algo` for desconhecido.

Registro de Alterações

Versão	Descrição
8.1.0	O parâmetro `options` foi adicionado.
8.0.0	Agora lança uma exceção ValueError se `algo` for desconhecido; anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.

Examples

Exemplo #1 Um exemplo de hash()

```php
<?php
echo hash('sha256', 'The quick brown fox jumped over the lazy dog.');
?>
```

O exemplo acima produzirá:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_file() - Gera um valor hash usando o conteúdo de um arquivo fornecido
- hash_hmac() - Gera um valor de hash com chave usando o método HMAC

hash_algos

hash_algos

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_algos — Retorna uma lista de algoritmos de hash registrados

Descrição

function **hash_algos**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array indexado numericamente contendo a lista de algoritmos de hash suportados.

Registro de Alterações

Versão	Descrição
8.1.0	Foi adicionado suporte para algoritmos MurmurHash3 e xxHash.
7.4.0	Foi adicionado suporte para crc32c.
7.1.0	Suporte para sha512/224, sha512/256, sha3-224, sha3-256, sha3-384 e sha3-512 foi adicionado.

Examples

Exemplo #1 Exemplo de **hash_algos()**

A partir do PHP 8.1.0, **hash_algos()** retornará a seguinte lista de nomes de algoritmos.

```php
<?php
print_r(hash_algos());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => md2
    [1] => md4
    [2] => md5
    [3] => sha1
    [4] => sha224
    [5] => sha256
    [6] => sha384
    [7] => sha512/224
    [8] => sha512/256
    [9] => sha512
    [10] => sha3-224
    [11] => sha3-256
    [12] => sha3-384
    [13] => sha3-512
    [14] => ripemd128
    [15] => ripemd160
```

```
    [16] => ripemd256
    [17] => ripemd320
    [18] => whirlpool
    [19] => tiger128,3
    [20] => tiger160,3
    [21] => tiger192,3
    [22] => tiger128,4
    [23] => tiger160,4
    [24] => tiger192,4
    [25] => snefru
    [26] => snefru256
    [27] => gost
    [28] => gost-crypto
    [29] => adler32
    [30] => crc32
    [31] => crc32b
    [32] => crc32c
    [33] => fnv132
    [34] => fnv1a32
    [35] => fnv164
    [36] => fnv1a64
    [37] => joaat
    [38] => murmur3a
    [39] => murmur3c
    [40] => murmur3f
    [41] => xxh32
    [42] => xxh64
    [43] => xxh3
    [44] => xxh128
    [45] => haval128,3
    [46] => haval160,3
    [47] => haval192,3
    [48] => haval224,3
    [49] => haval256,3
    [50] => haval128,4
    [51] => haval160,4
    [52] => haval192,4
    [53] => haval224,4
    [54] => haval256,4
    [55] => haval128,5
    [56] => haval160,5
    [57] => haval192,5
    [58] => haval224,5
    [59] => haval256,5
)
```

Veja também

- hash() - Gera um valor hash (resumo da mensagem)
- hash_hmac_algos() - Retorna uma lista de algoritmos de hash registrados adequados para hash_hmac

hash_copy

hash_copy

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

hash_copy — Copia contexto de hash

Descrição

function **hash_copy**(HashContext `$context`): HashContext

Parâmetros

`context`

Contexto de hash retornado por hash_init().

Valor Retornado

Retorna uma cópia do contexto de hash.

Registro de Alterações

Versão	Descrição
7.2.0	Aceita e retorna HashContext em vez de recurso.

Examples

Exemplo #1 Exemplo de hash_copy()

```php
<?php
$context = hash_init("sha256");
hash_update($context, "The quick brown fox ");

/* copia o contexto para poder continuar a usá-lo */
$copy_context = hash_copy($context);

echo hash_final($context), "\n";

hash_update($copy_context, "jumped over the lazy dog.");
echo hash_final($copy_context), "\n";
?>
```

O exemplo acima produzirá:

```
b29d66e56ed90cce9b0165c43fedec612b60a071974d8be4513e18580d55b5bd
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

hash_equals

hash_equals

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

hash_equals — Comparação de strings segura para ataque de temporização

Descrição

function **hash_equals**(#[\SensitiveParameter] string `$known_string`, #[\SensitiveParameter] string `$user_string`): bool

Verifica se duas strings são iguais sem vazar informações sobre o conteúdo de `known_string` por meio do tempo de execução.

Esta função pode ser usada para mitigar ataques de temporização. Executar uma comparação regular com `===` levará mais ou menos tempo para executar dependendo se os dois valores são diferentes ou não e em qual posição a primeira diferença pode ser encontrada, vazando assim informações sobre o conteúdo do `known_string` secreto.

> **Cuidado**
>
> É importante fornecer a string fornecida pelo usuário como o segundo parâmetro, em vez do primeiro.

Parâmetros

`known_string`

A string conhecida que deve ser mantida em segredo.

`user_string`

A string fornecida pelo usuário para comparação.

Valor Retornado

Retorna `true` quando as duas strings são iguais, `false` caso contrário.

Examples

Exemplo #1 Exemplo de hash_equals()

```php
<?php
$secretKey = '8uRhAeH89naXfFXKGOEj';

// O valor e a assinatura são fornecidos pelo usuário, por exemplo, dentro da URL
// e recuperados usando $_GET.
$value = 'username=rasmuslerdorf';
$signature = '8c35009d3b50caf7f5d2c1e031842e6b7823a1bb781d33c5237cd27b57b5f327';

if (hash_equals(hash_hmac('sha256', $value, $secretKey), $signature)) {
    echo "O valor está assinado corretamente.", PHP_EOL;
} else {
    echo "O valor foi adulterado.", PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
O valor está assinado corretamente.
```

Notas

> **Nota** :
>
> Ambos os argumentos devem ter o mesmo comprimento para serem comparados com sucesso. Quando argumentos de comprimentos diferentes são fornecidos, `false` é retornado imediatamente e o comprimento da string conhecida pode ser vazado em caso de um ataque de temporização.

Veja também

- hash_hmac() - Gera um valor de hash com chave usando o método HMAC

hash_file

hash_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_file — Gera um valor hash usando o conteúdo de um arquivo fornecido

Descrição

```
function hash_file(
    string $algo,
    string $filename,
    bool $binary = false,
    array $options = []
): string|false
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_algos().

`filename`

URL que descreve o local do arquivo a ter o hash gerado; Suporta empacotadores fopen().

`binary`

Quando definido como `true`, gera dados binários brutos. `false` caracteres gera hexadecimais minúsculos.

`options`

Um array de opções para os vários algoritmos de hash. Atualmente, apenas o parâmetro `"seed"` é suportado pelas variantes MurmurHash.

Valor Retornado

Retorna uma string contendo o resumo da mensagem calculado como caracteres hexadecimais minúsculos a menos que `binary` seja definido como true, caso em que a representação binária bruta do resumo da mensagem é retornada, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.1.0	O parâmetro `options` foi adicionado.

Examples

Exemplo #1 Usando **hash_file()**

```php
<?php
/* Cria um arquivo para cálculo de seu hash */
file_put_contents('example.txt', 'The quick brown fox jumped over the lazy dog.');

echo hash_file('sha256', 'example.txt');
?>
```

O exemplo acima produzirá:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_hmac_file() - Gera um valor hash com chave usando o método HMAC e o conteúdo de um arquivo fornecido

hash_final

hash_final

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_final — Finaliza um hash incremental e retorna o resumo resultante

Descrição

function **hash_final**(HashContext `$context`, bool `$binary` = `false`): string

Parâmetros

`context`

Contexto de hash retornado por hash_init().

`binary`

Quando definido como `true`, gera dados binários brutos. `false` gera caracteres hexadecimais minúsculos.

Valor Retornado

Retorna uma string contendo o resumo da mensagem calculado como caracteres hexadecimais minúsculos a menos que `binary` seja definido como true, caso em que a representação binária bruta do resumo da mensagem é retornada.

Registro de Alterações

Versão	Descrição
7.2.0	Aceita HashContext em vez de recurso.

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_update() - Injeta dados em um contexto de hash ativo
- hash_update_stream() - Injeta dados em um contexto de hash ativo a partir de um fluxo
- hash_update_file() - Injeta dados em um contexto de hash ativo a partir de um arquivo

hash_hkdf

hash_hkdf

(PHP 7 >= 7.1.2, PHP 8)

hash_hkdf — Gera uma derivação de chave HKDF de uma entrada de chave fornecida

Descrição

```
function hash_hkdf(
    string $algo,
    #[\SensitiveParameter] string $key,
    int $length = 0,
    string $info = "",
    string $salt = ""
): string
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_hmac_algos().

> **Nota** :
>
> Funções de hash não criptográficas não são permitidas.

`key`

Material de chaveamento de entrada (binário bruto). Não pode estar vazio.

`length`

Comprimento de saída desejado em bytes. Não pode ser maior que 255 vezes o tamanho da função hash escolhida.

Se `length` for `0`, o comprimento de saída será definido como padrão para o tamanho da função hash escolhida.

`info`

String de informação específica da aplicação/contexto.

`salt`

Salt a ser usado na derivação.

Embora opcional, adicionar salt aleatório melhora significativamente a força do HKDF.

Valor Retornado

Retorna uma string contendo uma representação binária bruta da chave derivada (também conhecida como material de chaveamento de saída - OKM).

Erros/Exceções

Lança uma exceção ValueError se `key` estiver vazio, `algo` for desconhecido ou não criptográfico, `length` for menor que `0` ou muito grande (maior que 255 vezes o tamanho da função hash).

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança uma exceção ValueError em caso de erro. Anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.

Examples

O exemplo abaixo produz um par de chaves separadas, adequadas para a criação de uma construção de criptografia seguida de HMAC, usando AES-256 e SHA-256 para criptografia e autenticação, respectivamente.

Exemplo #1 Exemplo de hash_hkdf()

```php
<?php
// Gera uma chave aleatória e adiciona um salt para fortalecê-la durante a derivação.
$inputKey = random_bytes(32);
$salt = random_bytes(16);

// Deriva um par de chaves separadas, usando a mesma entrada criada acima.
$encryptionKey = hash_hkdf('sha256', $inputKey, 32, 'aes-256-encryption', $salt);
$authenticationKey = hash_hkdf('sha256', $inputKey, 32, 'sha-256-authentication', $salt);

var_dump($encryptionKey !== $authenticationKey); // bool(true)
?>
```

Veja também

- hash_pbkdf2() - Gera uma derivação de chave PBKDF2 de uma senha fornecida
- » RFC 5869
- » implementação do usuário

hash_hmac

hash_hmac

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_hmac — Gera um valor de hash com chave usando o método HMAC

Descrição

```
function hash_hmac(
    string $algo,
    string $data,
    #[\SensitiveParameter] string $key,
    bool $binary = false
): string
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_hmac_algos().

> **Nota** :
>
> Funções de hash não criptográficas não são permitidas.

`data`

Mensagem a ter o hash gerado.

`key`

Chave secreta compartilhada usada para gerar a variante HMAC do resumo da mensagem.

`binary`

Quando definido como `true`, gera dados binários brutos. `false` gera caracteres hexadecimais minúsculos.

Valor Retornado

Retorna uma string contendo o resumo da mensagem calculado como caracteres hexadecimais minúsculos a menos que `binary` seja definido como true, caso em que a representação binária bruta do resumo da mensagem é retornada.

Erros/Exceções

Lança uma exceção ValueError se `algo` for desconhecido ou for uma função hash não criptográfica.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança uma exceção ValueError se `algo` for desconhecido ou for uma função hash não criptográfica; anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.
7.2.0	O uso de funções de hash não criptográficas (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) foi desabilitado.

Examples

Exemplo #1 Exemplo de hash_hmac()

```php
<?php
echo hash_hmac('sha256', 'The quick brown fox jumped over the lazy dog.', 'secret');
?>
```

O exemplo acima produzirá:

```
9c5c42422b03f0ee32949920649445e417b2c634050833c5165704b825c2a53b
```

Veja também

- hash_hmac_algos() - Retorna uma lista de algoritmos de hash registrados adequados para hash_hmac
- hash_hmac_file() - Gera um valor hash com chave usando o método HMAC e o conteúdo de um arquivo fornecido
- hash_equals() - Comparação de strings segura para ataque de temporização

hash_hmac_algos

hash_hmac_algos

(PHP 7 >= 7.2.0, PHP 8)

hash_hmac_algos — Retorna uma lista de algoritmos de hash registrados adequados para hash_hmac

Descrição

function **hash_hmac_algos**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array indexado numericamente contendo a lista de algoritmos de hash suportados adequados para hash_hmac().

Examples

Exemplo #1 Exemplo de **hash_hmac_algos()**

```php
<?php
print_r(hash_hmac_algos());
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => md2
    [1] => md4
    [2] => md5
    [3] => sha1
    [4] => sha224
    [5] => sha256
    [6] => sha384
    [7] => sha512/224
    [8] => sha512/256
    [9] => sha512
    [10] => sha3-224
    [11] => sha3-256
    [12] => sha3-384
    [13] => sha3-512
    [14] => ripemd128
    [15] => ripemd160
    [16] => ripemd256
    [17] => ripemd320
    [18] => whirlpool
    [19] => tiger128,3
    [20] => tiger160,3
    [21] => tiger192,3
    [22] => tiger128,4
    [23] => tiger160,4
    [24] => tiger192,4
    [25] => snefru
    [26] => snefru256
    [27] => gost
    [28] => gost-crypto
    [29] => haval128,3
    [30] => haval160,3
    [31] => haval192,3
    [32] => haval224,3
    [33] => haval256,3
    [34] => haval128,4
    [35] => haval160,4
    [36] => haval192,4
    [37] => haval224,4
    [38] => haval256,4
    [39] => haval128,5
    [40] => haval160,5
    [41] => haval192,5
    [42] => haval224,5
    [43] => haval256,5
)
```

Notas

> **Nota** :
>
> Antes do PHP 7.2.0, o único meio de obter uma lista de algoritmos de hash suportados era chamar hash_algos() que também retorna algoritmos de hash que não são adequados para hash_hmac().

Veja também

- hash_hmac() - Gera um valor de hash com chave usando o método HMAC
- hash_algos() - Retorna uma lista de algoritmos de hash registrados

hash_hmac_file

hash_hmac_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_hmac_file — Gera um valor hash com chave usando o método HMAC e o conteúdo de um arquivo fornecido

Descrição

```
function hash_hmac_file(
    string $algo,
    string $filename,
    #[\SensitiveParameter] string $key,
    bool $binary = false
): string|false
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_hmac_algos().

> **Nota** :
>
> Funções de hash não criptográficas não são permitidas.

`filename`

URL descrevendo o local do arquivo a ter o hash gerado; Suporta empacotadores fopen.

`key`

Chave secreta compartilhada usada para gerar a variante HMAC do resumo da mensagem.

`binary`

Quando definido como `true`, gera dados binários brutos. `false` gera caracteres hexadecimais minúsculos.

Valor Retornado

Retorna uma string contendo o resumo da mensagem calculado como caracteres hexadecimais minúsculos a menos que `binary` esteja definido como true, caso em que a representação binária bruta do resumo da mensagem é retornada. Retorna `false` se o arquivo `filename` não puder ser lido.

Erros/Exceções

Lança uma exceção ValueError se `algo` for desconhecido ou for uma função hash não criptográfica.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança uma exceção ValueError se `algo` for desconhecido ou for uma função hash não criptográfica; anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.
7.2.0	O uso de funções de hash não criptográficas (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) foi desabilitado.

Examples

Exemplo #1 Exemplo de **hash_hmac_file()**

```
<?php
/*  Cria um arquivo para cálculo de seu hash */
```

```php
file_put_contents('example.txt', 'The quick brown fox jumped over the lazy dog.');

echo hash_hmac_file('sha256', 'example.txt', 'secret');
?>
```

O exemplo acima produzirá:

```
9c5c42422b03f0ee32949920649445e417b2c634050833c5165704b825c2a53b
```

Veja também

- hash_hmac() - Gera um valor de hash com chave usando o método HMAC
- hash_hmac_algos() - Retorna uma lista de algoritmos de hash registrados adequados para hash_hmac
- hash_init() - Inicializa um contexto de hash incremental
- hash_equals() - Comparação de strings segura para ataque de temporização

hash_init

hash_init

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_init — Inicializa um contexto de hash incremental

Descrição

```
function hash_init(
    string $algo,
    int $flags = 0,
    #[\SensitiveParameter] string $key = "",
    array $options = []
): HashContext
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_algos().

> **Nota** :
>
> Funções de hash não criptográficas não são permitidas se o sinalizador `HASH_HMAC` for especificado.

`flags`

Configurações opcionais para geração de hash, atualmente suporta apenas uma opção: `HASH_HMAC`. Quando especificado, o `key` *precisa* ser especificado.

`key`

Quando `HASH_HMAC` é especificado para `flags`, uma chave secreta compartilhada a ser usada com o método de hash HMAC deve ser fornecida neste parâmetro.

`options`

Um array de opções para os vários algoritmos de hash. Atualmente, apenas o parâmetro `"seed"` é suportado pelas variantes MurmurHash.

Valor Retornado

Retorna um contexto de hash para uso com hash_update(), hash_update_stream(), hash_update_file(), e hash_final().

Erros/Exceções

- Lança uma exceção ValueError se `algo` for desconhecido ou for uma função hash não criptográfica ou se `key` estiver vazio.
- Passar opções de configuração do tipo errado em `options` agora emitirá um erro `E_DEPRECATED` porque elas podem ser interpretadas incorretamente. Isso se tornará um ValueError no futuro.

Registro de Alterações

Versão	Descrição
8.4.0	Passar opções de um tipo errado foi descontinuado.
8.1.0	O parâmetro `options` foi adicionado.
8.0.0	Agora lança uma exceção ValueError se o `algo` for desconhecido ou for uma função hash não criptográfica ou se `key` estiver vazio. Anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.
7.2.0	O uso de funções hash não criptográficas (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) com `HASH_HMAC` foi desabilitado.
7.2.0	Retorna HashContext em vez de recurso.

Examples

Exemplo #1 Exemplo de hash incremental

```php
<?php
$hash = hash('sha256', 'The quick brown fox jumped over the lazy dog.');

$ctx = hash_init('sha256');
hash_update($ctx, 'The quick brown fox ');
hash_update($ctx, 'jumped over the lazy dog.');
$incremental_hash = hash_final($ctx);

echo $incremental_hash, PHP_EOL;
var_dump($hash === $incremental_hash);
?>
```

O exemplo acima produzirá:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
bool(true)
```

Veja também

- hash_algos() - Retorna uma lista de algoritmos de hash registrados
- hash_update() - Injeta dados em um contexto de hash ativo
- hash_update_file() - Injeta dados em um contexto de hash ativo a partir de um arquivo
- hash_update_stream() - Injeta dados em um contexto de hash ativo a partir de um fluxo
- hash_final() - Finaliza um hash incremental e retorna o resumo resultante

hash_pbkdf2

hash_pbkdf2

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

hash_pbkdf2 — Gera uma derivação de chave PBKDF2 de uma senha fornecida

Descrição

function **hash_pbkdf2**(
  string `$algo`,
  #[\SensitiveParameter] string `$password`,

```
    string $salt,
    int $iterations,
    int $length = 0,
    bool $binary = false,
    array $options = []
): string
```

Parâmetros

`algo`

Nome do algoritmo de hash selecionado (por exemplo, `"sha256"`). Para uma lista de algoritmos suportados, veja hash_hmac_algos().

> **Nota** :
>
> Funções de hash não criptográficas não são permitidas.

`password`

A senha a ser usada para a derivação.

`salt`

O salt a ser usado para a derivação. Este valor deve ser gerado aleatoriamente.

`iterations`

O número de iterações internas a serem executadas para a derivação.

`length`

O comprimento da string de saída. Se `binary` for `true` isso corresponde ao comprimento em bytes da chave derivada, se `binary` for `false` isso corresponde ao dobro do comprimento em bytes da chave derivada (pois cada byte da chave é retornado como dois caracteres hexadecimais).

Se `0` for passado, toda a saída do algoritmo fornecido será usada.

`binary`

Quando definido como `true`, gera dados binários brutos. `false` gera caracteres hexadecimais minúsculos.

`options`

Um array de opções para os vários algoritmos de hash. Atualmente, apenas a chave `"seed"` é suportada pelas variantes MurmurHash.

Valor Retornado

Retorna uma string contendo a chave derivada como hexadecimais minúsculos, a menos que `binary` esteja definido como `true`, nesse caso, a representação binária bruta da chave derivada é retornada.

Erros/Exceções

Lança uma exceção ValueError se o algoritmo for desconhecido, o parâmetro `iterations` for menor ou igual a `0`, o parâmetro `length` for menor que `0` ou o parâmetro `salt` for muito longo (maior que `INT_MAX - 4`).

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança uma exceção ValueError em caso de erro. Anteriormente, `false` era retornado e uma mensagem `E_WARNING` era emitida.
7.2.0	O uso de funções de hash não criptográficas (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) foi desabilitado.

Examples

Exemplo #1 Exemplo de hash_pbkdf2(), uso básico

```php
<?php
$password = "password";
$iterations = 600000;

// Gera um salt aleatório criptograficamente seguro usando random_bytes()
$salt = random_bytes(16);

$hash = hash_pbkdf2("sha256", $password, $salt, $iterations, 20);
var_dump($hash);

// para binário bruto, $length precisa ser reduzido pela metade para resultados equivalentes
$hash = hash_pbkdf2("sha256", $password, $salt, $iterations, 10, true);
var_dump(bin2hex($hash));?>
```

O exemplo acima produzirá algo semelhante a:

```
string(20) "120fb6cffcf8b32c43e7"
string(20) "120fb6cffcf8b32c43e7"
```

Notas

> **Cuidado**
>
> O método PBKDF2 pode ser usado para fazer hash de senhas para armazenamento. No entanto, deve ser notado que password_hash() ou crypt() com **CRYPT_BLOWFISH** são mais adequados para armazenamento de senhas.

Veja também

- password_hash() - Cria um hash de senha
- hash_hkdf() - Gera uma derivação de chave HKDF de uma entrada de chave fornecida
- sodium_crypto_pwhash() - Deriva uma chave de uma senha, usando Argon2

hash_update

hash_update

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update — Injeta dados em um contexto de hash ativo

Descrição

function **hash_update**(HashContext `$context`, string `$data`): true

Parâmetros

`context`

Contexto de hash retornado por hash_init().

`data`

Mensagem a ser incluída no resumo do hash.

Valor Retornado

Sempre retorna **true**.

Registro de Alterações

Versão	Descrição
8.4.0	Agora tem um tipo de retorno de true em vez de bool.

Versão	Descrição
7.2.0	Aceita HashContext em vez de recurso.

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_update_file() - Injeta dados em um contexto de hash ativo a partir de um arquivo
- hash_update_stream() - Injeta dados em um contexto de hash ativo a partir de um fluxo
- hash_final() - Finaliza um hash incremental e retorna o resumo resultante

hash_update_file

hash_update_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update_file — Injeta dados em um contexto de hash ativo a partir de um arquivo

Descrição

function **hash_update_file**(HashContext `$context`, string `$filename`, ?resource `$stream_context = null`): bool

Parâmetros

`context`

Contexto de hash retornado por hash_init().

`filename`

URL que descreve o local do arquivo a ser hash; Suporta empacotadores fopen().

`stream_context`

Contexto de fluxo conforme retornado por stream_context_create().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`stream_context` agora pode ser nulo.
7.2.0	Aceita HashContext no lugar de um recurso.

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_update() - Injeta dados em um contexto de hash ativo
- hash_update_stream() - Injeta dados em um contexto de hash ativo a partir de um fluxo
- hash_final() - Finaliza um hash incremental e retorna o resumo resultante
- hash_file() - Gera um valor hash usando o conteúdo de um arquivo fornecido

hash_update_stream

hash_update_stream

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update_stream — Injeta dados em um contexto de hash ativo a partir de um fluxo

Descrição

function **hash_update_stream**(HashContext `$context`, resource `$stream`, int `$length = -1`): int

Parâmetros

`context`

Contexto de hash retornado por hash_init().

`stream`

Identificador de arquivo aberto conforme retornado por qualquer função de criação de fluxo.

`length`

Número máximo de caracteres a serem copiados de `stream` para o contexto de hash.

Valor Retornado

Número real de bytes adicionados ao contexto de hash de `stream`.

Registro de Alterações

Versão	Descrição
7.2.0	Aceita HashContext em vez de recurso.

Examples

Exemplo #1 Exemplo de hash_update_stream()

```php
<?php
$fp = tmpfile();
fwrite($fp, 'jumped over the lazy dog.');
rewind($fp);

$ctx = hash_init('sha256');
hash_update($ctx, 'The quick brown fox ');
hash_update_stream($ctx, $fp);
echo hash_final($ctx);
?>
```

O exemplo acima produzirá:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

Veja também

- hash_init() - Inicializa um contexto de hash incremental
- hash_update() - Injeta dados em um contexto de hash ativo
- hash_final() - Finaliza um hash incremental e retorna o resumo resultante

Geradores de números aleatórios e funções relacionadas à aleatoriedade

Introdução

Constantes predefinidas

As constantes abaixo estão sempre disponíveis como parte do núcleo do PHP.

`MT_RAND_MT19937` (int)

Indica que a implementação correta de » Mt19937 (Mersenne Twister) será usada pelo algoritmo, ao criar um Random\Engine\Mt19937 usando Random\Engine\Mt19937::__construct() ou propagando o Mersenne Twister global com mt_srand().

`MT_RAND_PHP` (int)

Indica que uma implementação incorreta do Mersenne Twister será usada pelo algoritmo, ao criar uma instância Random\Engine\Mt19937 usando Random\Engine\Mt19937::__construct() ou propagando o global Mersenne Twister com mt_srand().

A implementação incorreta está disponível para compatibilidade retroativa com mt_srand() anteriores ao PHP 7.1.0.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

Examples

Exemplo #1 Exemplo de Random

```php
<?php
$r = new \Random\Randomizer();

// Gerando um nome de domínio aleatório
printf(
    "%s.example.com\n",
    $r->getBytesFromString('abcdefghijklmnopqrstuvwxyz0123456789', 16)
);

// Embaralhando um array:
$fruits = [ 'vermelho' => '🍎', 'verde' => '🥝', 'amarelo' => '🍌', 'rosa' => '🍑', 'roxo' => '🍇' ];
echo "Salada: ", implode(', ', $r->shuffleArray($fruits)), "\n";

// Embaralhando chaves de array
$fruits = [ 'vermelho' => '🍎', 'verde' => '🥝', 'amarelo' => '🍌', 'rosa' => '🍑', 'roxo' => '🍇' ];

$keys = $r->pickArrayKeys($fruits, 2);
// Procurando os valores para as chaves escolhidas.
$selection = array_map(
    static fn ($key) => $fruits[$key],
    $keys
);

echo "Valores: ", implode(', ', $selection), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
j87fzv1p0daiwmlo.example.com
Salada: 🥝, 🍇, 🍎, 🍌, 🍑
```

```
Valores: 🍌, 🍊
```

Funções Random

Índice

getrandmax

getrandmax

(PHP 4, PHP 5, PHP 7, PHP 8)

getrandmax — Mostrar o maior valor aleatório possível

Descrição

function **getrandmax**(): int

Retorna o valor máximo que pode ser retornado por uma chamada a rand().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O maior valor aleatório possível retornado por rand()

Veja também

- rand() - Gera um número inteiro aleatório
- srand() - Semeia o gerador de números aleatórios
- mt_getrandmax() - Mostra o maior valor aleatório possível

lcg_value

lcg_value

(PHP 4, PHP 5, PHP 7, PHP 8)

lcg_value — Gerador congruente linear combinado

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.4.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
function **lcg_value**(): float

lcg_value() retorna um número pseudoaleatório no intervalo de (0, 1). A função combina dois CGs com períodos de 2^31 - 85 e 2^31 - 249. O período desta função é igual ao produto de ambos os primos.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**

> Escalar o valor de retorno para um intervalo diferente usando multiplicação ou adição (também chamado de transformação afim) pode resultar em uma tendência no valor resultante, pois os pontos flutuantes não são igualmente densos ao longo da reta numérica. Como nem todos os valores podem ser representados exatamente por um ponto flutuante, o resultado da transformação afim também pode resultar em valores fora do intervalo solicitado.
>
> Use Random\Randomizer::getFloat() para gerar um float aleatório dentro de um intervalo arbitrário. Use Random\Randomizer::getInt() para gerar um número inteiro aleatório dentro de um intervalo arbitrário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um valor flutuante pseudoaleatório entre 0,0 e 1,0, inclusive.

Registro de Alterações

Versão	Descrição
8.4.0	Esta função foi descontinuada.

Veja também

- Random\Randomizer::getFloat() - Obtém um float selecionado uniformemente
- Random\Randomizer::getInt() - Obtém um inteiro selecionado uniformemente
- random_int() - Obtém um número inteiro selecionado uniformemente e criptograficamente seguro

mt_getrandmax

mt_getrandmax

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_getrandmax — Mostra o maior valor aleatório possível

Descrição

function **mt_getrandmax**(): int

Retorna o valor máximo que pode ser retornado por uma chamada a mt_rand().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o valor aleatório máximo retornado por uma chamada para mt_rand() sem argumentos, que é o valor máximo que pode ser usado para seu parâmetro `max` sem que o resultado seja aumentado (e portanto menos aleatório).

Veja também

- mt_rand() - Gera um valor aleatório através do Gerador de Números Aleatórios Mersenne Twister
- mt_srand() - Semeia o Gerador de Números Aleatórios Mersenne Twister
- getrandmax() - Mostrar o maior valor aleatório possível

mt_rand

mt_rand

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_rand — Gera um valor aleatório através do Gerador de Números Aleatórios Mersenne Twister

Descrição

function **mt_rand**(): int
function **mt_rand**(int `$min`, int `$max`): int

Muitos geradores de números aleatórios de libcs mais antigos têm características duvidosas ou desconhecidas e são lentos. A função **mt_rand()** é um substituto imediato para a antiga rand(). Ele usa um gerador de números aleatórios com características conhecidas usando o » Mersenne Twister, que produzirá números aleatórios quatro vezes mais rápido do que o libc rand() médio fornece.

Se chamado sem os argumentos opcionais `min`, `max` **mt_rand()** retorna um valor pseudo-aleatório entre 0 e mt_getrandmax(). Se você quiser um número aleatório entre 5 e 15 (inclusive), por exemplo, use `mt_rand(5, 15)`.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**
>
> Esta função usa a instância global do Mt19937 (“Mersenne Twister”) como a fonte de aleatoriedade a por isso compartilha seu estado com todas as outras funções que usam o Mt19937 global. Usar qualquer uma destas funções avança a sequência para *todas* as outras funções, independentemente do escopo.
>
> Gerar sequências repetíveis ao semear mt_srand() ou srand() com um valor conhecido também gerará saída repetível desta função.
>
> Prefira usar métodos de Random\Randomizer em todo código novo.

Parâmetros

`min`

Valor mais baixo opcional a ser retornado (padrão: 0)

`max`

Valor mais alto opcional a ser retornado (padrão: mt_getrandmax())

Valor Retornado

Um valor inteiro aleatório entre `min` (ou 0) e `max` (ou mt_getrandmax(), inclusive).

Erros/Exceções

- Se `max` for menor que `min`, uma exceção ValueError será lançada.

Registro de Alterações

Versão	Descrição
8.0.0	Uma exceção ValueError será lançada se `max` for menor que `min`; anteriormente, um `E_WARNING` era emitido e a função retornava `false`.
7.2.0	**mt_rand()** recebeu uma correção para um bug de descontinuação de módulo. Isso significa que as sequências geradas com uma semente específica podem ser diferentes do PHP 7.1 em máquinas de 64 bits.
7.1.0	rand() foi transformado em um apelido de **mt_rand()**.
7.1.0	**mt_rand()** foi atualizado para usar a versão fixa e correta do algoritmo Mersenne Twister. Para voltar ao comportamento antigo, use mt_srand() com `MT_RAND_PHP` como segundo parâmetro.

Examples

Exemplo #1 Exemplo de **mt_rand()**

```php
<?php
echo mt_rand(), "\n";
echo mt_rand(), "\n";

echo mt_rand(5, 15), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
1604716014
1478613278
6
```

Notas

> **Aviso**
>
> O intervalo `min` `max` deve estar dentro do intervalo mt_getrandmax(). Ou seja, (`max` - `min`) <= mt_getrandmax() Caso contrário, **mt_rand()** poderá retornar números aleatórios piores do que deveria.

Veja também

- mt_srand() - Semeia o Gerador de Números Aleatórios Mersenne Twister
- mt_getrandmax() - Mostra o maior valor aleatório possível
- random_int() - Obtém um número inteiro selecionado uniformemente e criptograficamente seguro
- random_bytes() - Obtém bytes aleatórios criptograficamente seguros

mt_srand

mt_srand

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_srand — Semeia o Gerador de Números Aleatórios Mersenne Twister

Descrição

function **mt_srand**(?int `$seed` = `null`, int `$mode` = `MT_RAND_MT19937`): void

Semeia o gerador de números aleatórios com `seed` ou com um valor aleatório se nenhum `seed` for fornecido.

> **Nota** :
>
> Não é necessário inicializar o gerador de números aleatórios com srand() ou **mt_srand()**, pois isso é feito automaticamente.

> **Cuidado**
>
> Como o mecanismo Mt19937 (“Mersenne Twister”) aceita apenas um único inteiro de 32 bits como semente, o número de sequências aleatórias possíveis é limitado a apenas 2^{32} (ou seja, 4.294.967.296), embora o Mt19937 possua um gigantesco período de 2^{19937}-1.
>
> Ao depender da geração aleatória implícita ou explícita de sementes, as duplicações aparecerão muito mais cedo. Sementes duplicadas são esperadas com probabilidade de 50% depois de menos de 80.000 sementes geradas aleatoriamente de acordo com o problema do aniversário. Uma probabilidade de 10% de uma semente duplicada acontece depois de gerar aleatoriamente cerca de 30.000 sementes.
>
> Isso torna o Mt19937 inadequado para aplicações em que sequências duplicadas não devem ocorrer com mais do que uma probabilidade desprezível. Se a geração reproduzível de sementes for necessária, ambos os mecanismos Random\Engine\Xoshiro256StarStar e Random\Engine\PcgOneseq128XslRr64 suportam sementes muito maiores que são improváveis de colidir aleatoriamente. Se a reprodutibilidade não for necessária, o mecanismo Random\Engine\Secure fornece aleatoriedade criptograficamente segura.

Parâmetros

`seed`

Preenche o estado com valores gerados com um gerador congruente linear que foi propagado com `seed` interpretado como um número inteiro sem sinal de 32 bits.

Se `seed` for omitido ou `null`, um número inteiro aleatório e sem sinal de 32 bits será usado.

`mode`

Use uma das constantes a seguir para especificar a implementação do algoritmo a ser usado.

- `MT_RAND_MT19937`: A implementação correta do Mt19937, disponível a partir do PHP 7.1.0.
- `MT_RAND_PHP` Usa uma implementação incorreta do Mersenne Twister que foi usada como padrão até o PHP 7.1.0. Este modo está disponível para compatibilidade com versões anteriores.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.3.0	`seed` agora é anulável.
7.1.0	srand() foi transformado em um apelido de **mt_srand()**.
7.1.0	mt_rand() foi atualizado para usar a versão fixa e correta do algoritmo Mersenne Twister. Para voltar ao comportamento antigo, use **mt_srand()** com `MT_RAND_PHP` como segundo parâmetro.

Veja também

- mt_rand() - Gera um valor aleatório através do Gerador de Números Aleatórios Mersenne Twister
- mt_getrandmax() - Mostra o maior valor aleatório possível
- srand() - Semeia o gerador de números aleatórios

rand

rand

(PHP 4, PHP 5, PHP 7, PHP 8)

rand — Gera um número inteiro aleatório

Descrição

function **rand**(): int
function **rand**(int `$min`, int `$max`): int

Se chamado sem os argumentos opcionais `min`, `max` **rand()** retorna um número inteiro pseudo-aleatório entre 0 e getrandmax(). Se a intenção for obter um número aleatório entre 5 e 15 (inclusive), por exemplo, use `rand(5, 15)`.

> **Cuidado**
>
> Esta função não gera valores criptograficamente seguros e *não deve* ser usada para propósitos criptográficos ou fins que exijam que os valores retornados sejam impossíveis de adivinhar.
>
> Se aleatoriedade criptograficamente segura for necessária, a classe Random\Randomizer pode ser usada com o mecanismo Random\Engine\Secure. Para casos de uso simples, as funções random_int() e random_bytes() fornecem uma API conveniente e segura que é garantida pelo CSPRNG do sistema operacional.

> **Cuidado**
>
> Esta função usa a instância global do Mt19937 (“Mersenne Twister”) como a fonte de aleatoriedade a por isso compartilha seu estado com todas as outras funções que usam o Mt19937 global. Usar qualquer uma destas funções avança a sequência para *todas* as outras funções, independentemente do escopo.
>
> Gerar sequências repetíveis ao semear mt_srand() ou srand() com um valor conhecido também gerará saída repetível desta função.

> Prefira usar métodos de Random\Randomizer em todo código novo.

> **Nota** :
>
> Anteriormente ao PHP 7.1.0, getrandmax() resultava em apenas 32767 em algumas plataformas (como o Windows por exemplo). Se for necessário um intervalo maior que 32767, especificar `min` e `max` permitirá a criação de um intervalo maior que esse ou pode ser considerado usar mt_rand() alternativamente.

> **Nota** :
>
> A partir do PHP 7.1.0, **rand()** usa o mesmo gerador de números aleatórios que mt_rand(). Para preservar a compatibilidade com versões anteriores, **rand()** permite que `max` seja menor que `min` em vez de retornar `false` como mt_rand().

Parâmetros

`min`

O menor valor a ser retornado (padrão: 0)

`max`

O valor mais alto a ser retornado (padrão: getrandmax())

Valor Retornado

Um valor pseudoaleatório entre `min` (ou 0) e `max` (ou getrandmax(), inclusive).

Registro de Alterações

Versão	Descrição
7.2.0	**rand()** recebeu uma correção para um bug de descontinuação de módulo. Isso significa que as sequências geradas com uma semente específica podem ser diferentes do PHP 7.1 em máquinas de 64 bits.
7.1.0	**rand()** foi transformado em um apelido de mt_rand().

Examples

Exemplo #1 Exemplo de rand()

```php
<?php
echo rand(), "\n";
echo rand(), "\n";

echo rand(5, 15), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
7771
22264
11
```

Notas

> **Aviso**
>
> O intervalo `min` `max` deve estar dentro do intervalo getrandmax(). Ou seja, abs(`max` - `min`) <= getrandmax(). Caso contrário, **rand()** poderá retornar números aleatórios de baixa qualidade.

Veja também

- srand() - Semeia o gerador de números aleatórios
- getrandmax() - Mostrar o maior valor aleatório possível
- mt_rand() - Gera um valor aleatório através do Gerador de Números Aleatórios Mersenne Twister
- random_int() - Obtém um número inteiro selecionado uniformemente e criptograficamente seguro
- random_bytes() - Obtém bytes aleatórios criptograficamente seguros

random_bytes

random_bytes

(PHP 7, PHP 8)

random_bytes — Obtém bytes aleatórios criptograficamente seguros

Descrição

function **random_bytes**(int `$length`): string

Gera uma string contendo bytes aleatórios selecionados uniformemente com o comprimento solicitado em `length`.

Como os bytes retornados são selecionados de forma completamente aleatória, é provável que a string resultante contenha caracteres não imprimíveis ou sequências UTF-8 inválidas. Pode ser necessário codificá-la antes da transmissão ou exibição.

A aleatoriedade gerada por esta função é adequada para todas as aplicações, incluindo a geração de segredos de longo prazo, como chaves de criptografia.

As fontes de aleatoriedade usadas para esta função são as seguintes:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 se CCRandomGenerateBytes estiver disponível em tempo de compilação): CCRandomGenerateBytes()

  macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*
- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Qualquer combinação de sistema operacional e versão do PHP não mencionada anteriormente: */dev/urandom*
- Se nenhuma das fontes estiver disponível ou todas falharem em gerar aleatoriedade, então uma exceção Random\RandomException será lançada.

> **Nota** :
>
> Embora esta função tenha sido adicionada ao PHP no PHP 7.0, uma » implementação no espaço do usuário está disponível para o PHP 5.2 ao 5.6, inclusive.

Parâmetros

`length`

O comprimento da sequência aleatória que deve ser retornada em bytes; deve ser `1` ou superior.

Valor Retornado

Uma string contendo o número solicitado de bytes aleatórios criptograficamente seguros.

Erros/Exceções

- Se uma fonte apropriada de aleatoriedade não puder ser encontrada, uma exceção Random\RandomException será lançada.
- Se o valor de `length` for menor que `1`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.2.0	No caso de uma falha de CSPRNG, esta função agora lançará uma Random\RandomException. Anteriormente, uma Exception simples era lançada.

Examples

Exemplo #1 Exemplo de random_bytes()

```php
<?php
$bytes = random_bytes(5);
var_dump(bin2hex($bytes));
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(10) "385e33f741"
```

Veja também

- Random\Randomizer::getBytes() - Obtém bytes aleatórios
- random_int() - Obtém um número inteiro selecionado uniformemente e criptograficamente seguro
- bin2hex() - Converte um dado binário em representação hexadecimal
- base64_encode() - Codifica dados com MIME base64

random_int

random_int

(PHP 7, PHP 8)

random_int — Obtém um número inteiro selecionado uniformemente e criptograficamente seguro

Descrição

function **random_int**(int `$min`, int `$max`): int

Gera um número inteiro selecionado uniformemente entre o mínimo e o máximo fornecidos.

A aleatoriedade gerada por esta função é adequada para todas as aplicações, incluindo a geração de segredos de longo prazo, como chaves de criptografia.

As fontes de aleatoriedade usadas para esta função são as seguintes:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 se CCRandomGenerateBytes estiver disponível em tempo de compilação): CCRandomGenerateBytes()

macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*

- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Qualquer combinação de sistema operacional e versão do PHP não mencionada anteriormente: */dev/urandom*
- Se nenhuma das fontes estiver disponível ou todas falharem em gerar aleatoriedade, então uma exceção Random\RandomException será lançada.

> **Nota** :
>
> Embora esta função tenha sido adicionada ao PHP no PHP 7.0, uma » implementação no espaço do usuário está disponível para o PHP 5.2 ao 5.6, inclusive.

Parâmetros

`min`

O menor valor a ser retornado.

`max`

O maior valor a ser retornado.

Valor Retornado

Um número inteiro criptograficamente seguro e selecionado uniformemente do intervalo fechado [`min`, `max`]. Ambos `min` e `max` são valores de retorno possíveis.

Erros/Exceções

- Se uma fonte apropriada de aleatoriedade não puder ser encontrada, uma exceção Random\RandomException será lançada.
- Se `max` é menor que `min`, um ValueError será lançado.

Registro de Alterações

Versão	Descrição
8.2.0	No caso de uma falha de CSPRNG, esta função agora lançará um Random\RandomException. Anteriormente um raso Exception era lançado.

Examples

Exemplo #1 Exemplo de **random_int()**

```php
<?php
var_dump(random_int(100, 999));
var_dump(random_int(-1000, 0));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(248)
int(-898)
```

Veja também

- Random\Randomizer::getInt() - Obtém um inteiro selecionado uniformemente
- random_bytes() - Obtém bytes aleatórios criptograficamente seguros

srand

srand

(PHP 4, PHP 5, PHP 7, PHP 8)

srand — Semeia o gerador de números aleatórios

Descrição

function **srand**(?int `$seed` = `null`, int `$mode` = `MT_RAND_MT19937`): void

Semeia o gerador de números aleatórios com `seed` ou com um valor aleatório se nenhum `seed` for informado.

> **Nota** :
>
> Não é necessário inicializar o gerador de números aleatórios com **srand()** ou mt_srand(), pois isso é feito automaticamente.

> **Cuidado**
>
> Como o mecanismo Mt19937 (“Mersenne Twister”) aceita apenas um único inteiro de 32 bits como semente, o número de sequências aleatórias possíveis é limitado a apenas 2^{32} (ou seja, 4.294.967.296), embora o Mt19937 possua um gigantesco período de 2^{19937}-1.
>
> Ao depender da geração aleatória implícita ou explícita de sementes, as duplicações aparecerão muito mais cedo. Sementes duplicadas são esperadas com probabilidade de 50% depois de menos de 80.000 sementes geradas aleatoriamente de acordo com o problema do aniversário. Uma probabilidade de 10% de uma semente duplicada acontece depois de gerar aleatoriamente cerca de 30.000 sementes.
>
> Isso torna o Mt19937 inadequado para aplicações em que sequências duplicadas não devem ocorrer com mais do que uma probabilidade desprezível. Se a geração reproduzível de sementes for necessária, ambos os mecanismos Random\Engine\Xoshiro256StarStar e Random\Engine\PcgOneseq128XslRr64 suportam sementes muito maiores que são improváveis de colidir aleatoriamente. Se a reprodutibilidade não for necessária, o mecanismo Random\Engine\Secure fornece aleatoriedade criptograficamente segura.

> **Nota** :
>
> A partir do PHP 7.1.0, **srand()** tornou-se um apelido de mt_srand().

Parâmetros

`seed`

Preenche o estado com valores gerados com um gerador congruente linear que foi propagado com `seed` interpretado como um número inteiro sem sinal de 32 bits.

Se `seed` for omitido ou `null`, um número inteiro aleatório e sem sinal de 32 bits será usado.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.3.0	`seed` agora é anulável.
7.1.0	**srand()** foi transformado em um apelido de mt_srand().

Veja também

- rand() - Gera um número inteiro aleatório
- getrandmax() - Mostrar o maior valor aleatório possível

- mt_srand() - Semeia o Gerador de Números Aleatórios Mersenne Twister

Random\Randomizer

(PHP 8 >= 8.2.0)

Introdução

Fornece uma API de alto nível para a aleatoriedade fornecida por um Random\Engine.

Resumo da classe

namespace **Random**;
final class **Randomizer** {
/* Propriedades */
public readonly Random\Engine *$engine*;
/* Métodos */
public function __construct(?Random\Engine `$engine` = **`null`**)
public function getBytes(int `$length`): string
public function getBytesFromString(string `$string`, int `$length`): string
public function getFloat(float `$min`, float `$max`, Random\IntervalBoundary `$boundary` = **`Random\IntervalBoundary::ClosedOpen`**): float
public function getInt(int `$min`, int `$max`): int
public function nextFloat(): float
public function nextInt(): int
public function pickArrayKeys(array `$array`, int `$num`): array
public function __serialize(): array
public function shuffleArray(array `$array`): array
public function shuffleBytes(string `$bytes`): string
public function __unserialize(array `$data`): void
}

Propriedades

engine

A fonte de aleatoriedade de baixo nível para os métodos Random\Randomizer.

Índice

Random\Randomizer::__construct

Random\Randomizer::__construct

(PHP 8 >= 8.2.0)

Random\Randomizer::__construct — Constrói um novo Randomizer

Descrição

public function **Random\Randomizer::__construct**(?Random\Engine `$engine` = **`null`**)

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`engine`

O Random\Engine a ser usado para gerar aleatoriedade.

Se `engine` for omitido ou **`null`**, um novo objeto Random\Engine\Secure será usado.

Examples

Exemplo #1 Exemplo de Random\Randomizer::__construct()

```php
<?php
$r = new \Random\Randomizer();
$r = new \Random\Randomizer(new \Random\Engine\Mt19937());
?>
```

Random\Randomizer::getBytes

Random\Randomizer::getBytes

(PHP 8 >= 8.2.0)

Random\Randomizer::getBytes — Obtém bytes aleatórios

Descrição

public function **Random\Randomizer::getBytes**(int `$length`): string

Gera uma string contendo bytes aleatórios selecionados uniformemente com o comprimento solicitado em `length`.

Como os bytes retornados são selecionados de forma completamente aleatória, a string resultante provavelmente conterá caracteres não imprimíveis ou sequências UTF-8 inválidas. Pode ser necessário codificá-la antes da transmissão ou exibição.

Parâmetros

`length`

O comprimento da string aleatória que deve ser retornada em bytes; deve ser `1` ou maior.

Valor Retornado

Uma string contendo o número solicitado de bytes aleatórios.

Erros/Exceções

- Se o valor de `length` for menor que `1`, um ValueError será gerado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de **Random\Randomizer::getBytes()**

```php
<?php
$r = new \Random\Randomizer();

echo bin2hex($r->getBytes(8)), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
ebdbe93cd56682c2
```

Veja também

- random_bytes() - Obtém bytes aleatórios criptograficamente seguros
- bin2hex() - Converte um dado binário em representação hexadecimal
- base64_encode() - Codifica dados com MIME base64
- Random\Randomizer::getBytesFromString() - Obtém bytes aleatórios de uma string de origem

Random\Randomizer::getBytesFromString

Random\Randomizer::getBytesFromString

(PHP 8 >= 8.3.0)

Random\Randomizer::getBytesFromString — Obtém bytes aleatórios de uma string de origem

Descrição

public function **Random\Randomizer::getBytesFromString**(string `$string`, int `$length`): string

Gera uma string contendo bytes aleatórios selecionados uniformemente a partir da `string` de entrada com o comprimento solicitado em `length`.

A chance de um byte ser selecionado é proporcional à sua participação na `string` de entrada. Se cada byte ocorrer a mesma quantidade de vezes, cada byte terá a mesma probabilidade de ser selecionado.

Parâmetros

`string`

A string da qual os bytes retornados são selecionados.

`length`

O comprimento da string aleatória que deve ser retornada em bytes; deve ser `1` ou maior.

Valor Retornado

Uma string contendo o número solicitado de bytes aleatórios retirados da `string` de entrada.

Erros/Exceções

- Se `string` estiver vazio, um ValueError será gerado.
- Se o valor de `length` for menor que `1`, um ValueError será gerado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::getBytesFromString()

```php
<?php
$randomizer = new \Random\Randomizer();

printf(
    "%s.example.com",
    $randomizer->getBytesFromString('abcdefghijklmnopqrstuvwxyz0123456789', 16)
);
?>
```

O exemplo acima produzirá algo semelhante a:

```
3zsw04eiubcf82jd.example.com
```

Exemplo #2 Gerando um código aleatório para autenticação multifator

```php
<?php
// O mecanismo Secure é o padrão, mas o tornamos explícito porque
// códigos multifatoriais são sensíveis à segurança.
$randomizer = new \Random\Randomizer(new \Random\Engine\Secure());

echo implode('-', str_split($randomizer->getBytesFromString('0123456789', 20), 5));
?>
```

O exemplo acima produzirá algo semelhante a:

```
11551-80418-27047-42075
```

Exemplo #3 Selecionando de uma string com distribuição não uniforme

```php
<?php
$randomizer = new \Random\Randomizer();

echo $randomizer->getBytesFromString('aaaaabcdef', 20);
?>
```

O exemplo acima produzirá algo semelhante a:

```
fddacbeaaeaaacaaaaca
```

Veja também

- Random\Randomizer::getBytes() - Obtém bytes aleatórios

Random\Randomizer::getFloat

Random\Randomizer::getFloat

(PHP 8 >= 8.3.0)

Random\Randomizer::getFloat — Obtém um float selecionado uniformemente

Descrição

public function **Random\Randomizer::getFloat**(float `$min`, float `$max`, Random\IntervalBoundary `$boundary` = **`Random\IntervalBoundary::ClosedOpen`**): float

Retorna um float uniformemente selecionado e equidistribuído de um intervalo solicitado.

Devido à precisão limitada, nem todos os números reais podem ser representados exatamente como um número de ponto flutuante. Se um número não puder ser representado exatamente, ele será arredondado para o valor exato representável mais próximo. Além disso, números flutuantes não são igualmente densos em toda a reta numérica. Como números flutuantes usam um expoente binário, a distância entre dois números flutuantes vizinhos dobra a cada potência de dois. Em outras palavras: há o mesmo número de floats representáveis entre `1.0` e `2.0` como entre `2.0` e `4.0`, `4.0` e `8.0`, `8.0` e `16.0`, e assim por diante.

A amostragem aleatória de um número arbitrário dentro do intervalo solicitado, por exemplo, dividindo dois números inteiros, pode resultar em uma distribuição enviesada por esse motivo. O arredondamento necessário fará com que alguns números flutuantes sejam retornados com mais frequência do que outros, especialmente em torno de potências de dois quando a densidade de números flutuantes muda.

Random\Randomizer::getFloat() implementa um algoritmo que retornará um ponto flutuante selecionado uniformemente do maior conjunto possível de pontos flutuantes exatamente representáveis e equidistribuídos dentro do intervalo solicitado. A distância entre os pontos flutuantes selecionáveis ("tamanho do passo") corresponde à distância entre os pontos flutuantes com a menor densidade, ou seja, a distância entre os pontos flutuantes no limite do intervalo com o maior valor absoluto. Isso significa que nem todos os pontos flutuantes representáveis dentro de um determinado intervalo podem ser retornados se o intervalo cruzar uma ou mais potências de dois. O passo a passo começará a partir do limite do intervalo com o maior valor absoluto para garantir que os passos se alinhem com os pontos flutuantes exatamente representáveis.

Limites de intervalos fechados sempre serão incluídos no conjunto de floats selecionáveis. Portanto, se o tamanho do intervalo não for um múltiplo exato do tamanho do passo e o limite com o menor valor absoluto for um limite fechado, a distância entre esse limite e seu float selecionável mais próximo será menor que o tamanho do passo.

> **Cuidado**
>
> O pós-processamento dos floats retornados provavelmente quebrará a equidistribuição uniforme, porque os floats intermediários dentro de uma operação matemática estão passando por arredondamento implícito. O intervalo solicitado deve corresponder o mais próximo possível ao intervalo desejado e o arredondamento só deve ser realizado como uma operação explícita antes de exibir o número selecionado para um usuário.

Explicação do algoritmo usando valores de exemplo

Para exemplificar o funcionamento do algoritmo, considere uma representação de ponto flutuante que utiliza uma mantissa de 3 bits. Esta representação é capaz de representar 8 valores de ponto flutuante diferentes entre potências de dois consecutivas. Isso significa que entre `1.0` e `2.0` todos os passos de tamanho `0.125` são exatamente representáveis e entre `2.0` e `4.0` todos os passos de tamanho `0.25` são exatamente representáveis. Na realidade, os floats do PHP utilizam uma mantissa de 52 bits e podem representar 2^{52} valores diferentes entre cada potência de dois. Isso significa que

- `1.0`
- `1.125`
- `1.25`
- `1.375`
- `1.5`
- `1.625`
- `1.75`
- `1.875`
- `2.0`
- `2.25`
- `2.5`
- `2.75`
- `3.0`
- `3.25`
- `3.5`
- `3.75`
- `4.0`

são os floats exatamente representáveis entre `1.0` e `4.0`.

Agora, considere que `$randomizer->getFloat(1.625, 2.5, IntervalBoundary::ClosedOpen)` é chamado, ou seja, um float aleatório começando em `1.625` até, mas não incluindo, `2.5` ser solicitado. O algoritmo primeiro determina o tamanho do passo na fronteira com o maior valor absoluto (`2.5`). O tamanho do passo nessa fronteira é `0.25`.

Observe que o tamanho do intervalo solicitado é `0.875`, que não é um múltiplo exato de `0.25`. Se o algoritmo começasse a avançar no limite inferior `1.625`, encontraria `2.125`, que não é exatamente representável e sofreria arredondamento implícito. Portanto, o algoritmo começa a avançar no limite superior `2.5`. Os valores selecionáveis são:

- `2.25`
- `2.0`
- `1.75`
- `1.625`

`2.5` não está incluído, pois o limite superior do intervalo solicitado é um limite aberto. `1.625` está incluído, embora sua distância até o valor mais próximo `1.75` seja `0.125`, que é menor que o tamanho do passo previamente determinado de `0.25`. O motivo para isso é que o intervalo solicitado é fechado no limite inferior (`1.625`) e limites fechados são sempre incluídos.

Por fim, o algoritmo seleciona uniformemente um dos quatro valores selecionáveis aleatoriamente e o retorna.

Por que dividir dois números inteiros não funciona

No exemplo anterior, há oito números de ponto flutuante representáveis entre cada subintervalo delimitado por uma potência de dois. Para exemplificar por que dividir dois inteiros não funcionaria bem para gerar um número flutuante aleatório, considere que há 16 números de ponto flutuante equidistribuídos no intervalo aberto à direita de `0.0` até, mas não incluindo, `1.0`. Metade deles são os oito valores exatamente representáveis entre `0.5` e `1.0`, a outra metade são os valores entre `0.0` e `1.0` cujo passo é de `0.0625`. Eles podem ser facilmente gerados dividindo um número inteiro aleatório entre `0` e `15` por `16` para obter um dos seguintes:

- `0.0`
- `0.0625`

- `0.125`
- `0.1875`
- `0.25`
- `0.3125`
- `0.375`
- `0.4375`
- `0.5`
- `0.5625`
- `0.625`
- `0.6875`
- `0.75`
- `0.8125`
- `0.875`
- `0.9375`

Este float aleatório poderia ser escalado para o intervalo aberto à direita de `1.625` até, mas não incluindo, `2.75`, multiplicando-o pelo tamanho do intervalo (`0.875`) e adicionando o mínimo `1.625`. Essa chamada transformação afim resultaria nos valores:

- `1.625` arredondado para `1.625`
- `1.679` arredondado para `1.625`
- `1.734` arredondado para `1.75`
- `1.789` arredondado para `1.75`
- `1.843` arredondado para `1.875`
- `1.898` arredondado para `1.875`
- `1.953` arredondado para `2.0`
- `2.007` arredondado para `2.0`
- `2.062` arredondado para `2.0`
- `2.117` arredondado para `2.0`
- `2.171` arredondado para `2.25`
- `2.226` arredondado para `2.25`
- `2.281` arredondado para `2.25`
- `2.335` arredondado para `2.25`
- `2.390` arredondado para `2.5`
- `2.445` arredondado para `2.5`

Observe como o limite superior de `2.5` seria retornado, apesar de ser um limite aberto e, portanto, excluído. Observe também como `2.0` e `2.25` têm o dobro de probabilidade de serem retornados em comparação com os outros valores.

Parâmetros

`min`

O limite inferior do intervalo.

`max`

O limite superior do intervalo.

`boundary`

Especifica se os limites do intervalo são valores de retorno possíveis.

Valor Retornado

Um float uniformemente selecionado e equidistribuído do intervalo especificado por `min`, `max` e `boundary`. Se `min` e `max` são valores de retorno possíveis depende do valor de `boundary`.

Erros/Exceções

- Se o valor de `min` não for finito (is_finite()), um ValueError será gerado.
- Se o valor de `max` não for finito (is_finite()), um ValueError será lançado.
- Se o intervalo solicitado não contiver nenhum valor, um ValueError será gerado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::getFloat()

```php
<?php
$randomizer = new \Random\Randomizer();

// Observe que a granularidade da latitude é o dobro da
// granularidade da longitude.
//
// Para a latitude, o valor pode ser -90 e 90.
// Para a longitude, o valor pode ser 180, mas não -180, porque
// -180 e 180 referem-se à mesma longitude.
printf(
    "Lat: %+.6f Lng: %+.6f",
    $randomizer->getFloat(-90, 90, \Random\IntervalBoundary::ClosedClosed),
    $randomizer->getFloat(-180, 180, \Random\IntervalBoundary::OpenClosed),
);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Lat: +69.244304 Lng: -53.548951
```

Notas

> **Nota** :
>
> Este método implementa o algoritmo de seção γ, conforme publicado em » Drawing Random Floating-Point Numbers from an Interval. Frédéric Goualard, ACM Trans. Model. Comput. Simul., 32:3, 2022 para obter as propriedades comportamentais desejadas.

> **Cuidado**
>
> O underflow é intencionalmente deixado sem tratamento no algoritmo de seção γ. Isso pode resultar em valores incorretos sendo retornados para intervalos com limites na faixa subnormal de números de ponto flutuante, ou seja, para limites com um valor absoluto menor que aproximadamente 2^{-1020} (cerca de `8.9e-308`).

Veja também

- Random\Randomizer::nextFloat() - Obtém um float do intervalo aberto à direita [0,0, 1,0)
- Random\Randomizer::getInt() - Obtém um inteiro selecionado uniformemente

Random\Randomizer::getInt

Random\Randomizer::getInt

(PHP 8 >= 8.2.0)

Random\Randomizer::getInt — Obtém um inteiro selecionado uniformemente

Descrição

public function **Random\Randomizer::getInt**(int `$min`, int `$max`): int

Parâmetros

`min`

O menor valor a ser retornado.

`max`

O maior valor a ser retornado.

Valor Retornado

Um inteiro selecionado uniformemente do intervalo fechado [`min`, `max`]. Tanto `min` quanto `max` são valores de retorno possíveis.

Erros/Exceções

- Se `max` for menor que `min`, um ValueError será gerado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de **Random\Randomizer::getInt()**

```php
<?php
$r = new \Random\Randomizer();

// Inteiro aleatório no intervalo:
echo $r->getInt(1, 100), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
42
```

Veja também

- random_int() - Obtém um número inteiro selecionado uniformemente e criptograficamente seguro
- Random\Randomizer::getFloat() - Obtém um float selecionado uniformemente

Random\Randomizer::nextFloat

Random\Randomizer::nextFloat

(PHP 8 >= 8.3.0)

Random\Randomizer::nextFloat — Obtém um float do intervalo aberto à direita [0,0, 1,0)

Descrição

public function **Random\Randomizer::nextFloat**(): float

Retorna um float uniformemente selecionado e equidistribuído do intervalo aberto à direita de `0.0` até, mas não incluindo, `1.0`.

A chance de um float retornado estar dentro de um determinado subintervalo aberto à direita é proporcional ao tamanho do subintervalo. Isso significa que a chance de um float ser *menor que* `0.5` é de 50%, o que é igual à chance de um float ser *pelo menos* `0.5`. Da mesma forma, a chance de um float estar dentro do intervalo aberto à direita de `0.2` até, mas não incluindo, `0.25` é exatamente 5%.

Esta propriedade facilita o uso de **Random\Randomizer::nextFloat()** para gerar um booleano aleatório com uma determinada chance, verificando se o float retornado é *menor que* uma determinada chance.

> **Nota** :
>
> O domínio dos floats retornáveis de **Random\Randomizer::nextFloat()** é idêntico ao de `Randomizer::getFloat(0.0, 1.0, IntervalBoundary::ClosedOpen)`.
>
> A implementação interna de **Random\Randomizer::nextFloat()** é mais eficiente.

> **Cuidado**

> Escalar o valor de retorno para um intervalo diferente usando multiplicação ou adição (a chamada transformação afim) pode resultar em um viés no valor resultante, já que os floats não são igualmente densos na reta numérica. Como nem todos os valores podem ser representados exatamente por um float, o resultado da transformação afim também pode resultar em valores fora do intervalo solicitado devido ao arredondamento implícito. Uma explicação detalhada dos problemas com a transformação afim é fornecida na documentação de Random\Randomizer::getFloat().
>
> Use Random\Randomizer::getFloat() para gerar um float aleatório dentro de um intervalo arbitrário. Use Random\Randomizer::getInt() para gerar um inteiro aleatório dentro de um intervalo arbitrário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um float uniformemente selecionado e equidistribuído do intervalo aberto à direita (`IntervalBoundary::ClosedOpen`) [0,0, 1,0). `0.0` é um possível valor de retorno, `1.0` não é.

Erros/Exceções

- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::nextFloat()

```php
<?php
$r = new \Random\Randomizer();

// O booleano resultante será verdadeiro com a chance fornecida.
$chance = 0.5;

$bool = $r->nextFloat() < $chance;

echo ($bool ? "Você ganhou" : "Você perdeu"), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Você ganhou
```

Exemplo #2 Escala incorreta usando uma transformação afim

```php
<?php
final class MaxEngine implements Random\Engine {
    public function generate(): string {
        return "\xff";
    }
}

$randomizer = new \Random\Randomizer(new MaxEngine);

$min = 3.5;
$max = 4.5;

// NÃO FAÇA ISTO:
//
// Isso retornará 4.5, apesar da amostragem nextFloat() de
// um intervalo aberto à direita, que nunca retornará 1.
printf("Escala errada: %.17g", $randomizer->nextFloat() * ($max - $min) + $min);

// Correto:
```

```php
// $randomizer->getFloat($min, $max, \Random\IntervalBoundary::ClosedOpen);
?>
```

O exemplo acima produzirá:

```
Escala errada: 4.5
```

Veja também

- Random\Randomizer::getFloat() - Obtém um float selecionado uniformemente

Random\Randomizer::nextInt

Random\Randomizer::nextInt

(PHP 8 >= 8.2.0)

Random\Randomizer::nextInt — Obtém um inteiro positivo

Descrição

public function **Random\Randomizer::nextInt**(): int

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um inteiro positivo entre 0 e um valor máximo, dependendo do número de bytes retornado de Random\Engine::generate(). O máximo exato pode ser calculado como $2^{\$engine_bytes * 8 - 1} - 1$.

Erros/Exceções

- Para evitar inconsistências, o PHP de 32 bits lançará Random\RandomException se o tamanho da saída de Random\Engine::generate() exceder 32 bits, pois o inteiro selecionado não pode ser retornado sem perdas. Isso afeta os mecanismos nativos de 64 bits Random\Engine\PcgOneseq128XslRr64 e Random\Engine\Xoshiro256StarStar. Qualquer mecanismo de ambiente de usuário que retorne mais de 4 bytes de aleatoriedade também é afetado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::nextInt()

```php
<?php
$r = new \Random\Randomizer();

// "Próximo" inteiro aleatório:
echo $r->nextInt(), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
8041689838856078718
```

Random\Randomizer::pickArrayKeys

Random\Randomizer::pickArrayKeys

(PHP 8 >= 8.2.0)

Random\Randomizer::pickArrayKeys — Seleciona chaves aleatórias de um array

Descrição

public function **Random\Randomizer::pickArrayKeys**(array `$array`, int `$num`): array

Seleciona uniformemente `num` chaves de array distintas do `array` de entrada.

Cada chave do `array` de entrada tem a mesma probabilidade de ser retornada.

> **Cuidado**
>
> A seleção das chaves do array depende da estrutura interna do `array` de entrada. As chaves do array retornadas podem ser diferentes para dois arrays de entrada iguais e dois Random\Engines com estado idêntico, dependendo de como os arrays de entrada foram criados.

Parâmetros

`array`

O array cujas chaves são selecionadas.

`num`

O número de chaves de array a serem retornadas; deve estar entre `1` e o número de elementos no `array`.

Valor Retornado

Um array contendo `num` chaves de array distintas de `array`.

O array retornado será uma lista (array_is_list()). Será um subconjunto do array retornado por array_keys().

Erros/Exceções

- Se `num` for menor que `1` ou maior que o número de elementos em `array`, um ValueError será gerado.
- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::pickArrayKeys()

```php
<?php
$r = new \Random\Randomizer();

$frutas = [ 'vermelho' => '🍎', 'verde' => '🥝', 'amarelo' => '🍌', 'rosa' => '🍑', 'roxo' => '🍇' ];

// Escolhe 2 chaves de array aleatórias:
echo "Chaves: ", implode(', ', $r->pickArrayKeys($frutas, 2)), "\n";

// Escolhe outras 3:
echo "Chaves: ", implode(', ', $r->pickArrayKeys($frutas, 3)), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Chaves: amarelo, roxo
Chaves: vermelho, verde, amarelo
```

Exemplo #2 Escolhendo valores aleatórios

```php
<?php
$r = new \Random\Randomizer();

$frutas = [ 'vermelho' => '🍎', 'verde' => '🥝', 'amarelo' => '🍌', 'rosa' => '🍑', 'roxo' => '🍇' ];

$chaves = $r->pickArrayKeys($frutas, 2);
// Procura os valores das chaves escolhidas.
$selection = array_map(
    static fn ($chave) => $frutas[$chave],
    $chaves
);

echo "Valores: ", implode(', ', $selection), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Valores: 🍎, 🍇
```

Veja também

- array_keys() - Retorna todas as chaves ou uma parte das chaves de um array

Random\Randomizer::__serialize

Random\Randomizer::__serialize

(PHP 8 >= 8.2.0)

Random\Randomizer::__serialize — Serializa o objeto Randomizer

Descrição

public function **Random\Randomizer::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de Random\Randomizer::__serialize()

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Randomizer::shuffleArray

Random\Randomizer::shuffleArray

(PHP 8 >= 8.2.0)

Random\Randomizer::shuffleArray — Obtém uma permutação de um array

Descrição

public function **Random\Randomizer::shuffleArray**(array `$array`): array

Retorna uma permutação selecionada uniformemente do `array` de entrada.

Cada permutação possível do `array` de entrada tem a mesma probabilidade de ser retornada.

Parâmetros

`array`

O array cujos valores são embaralhados.

O array de entrada não será modificado.

Valor Retornado

Uma permutação dos valores de `array`.

As chaves do `array` de entrada não serão preservadas; o array retornado será uma lista (array_is_list()).

Erros/Exceções

- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de **Random\Randomizer::shuffleArray()**

```php
<?php
$r = new \Random\Randomizer();

$frutas = [ 'vermelho' => '🍅', 'verde' => '🥝', 'amarelo' => '🍌', 'rosa' => '🍑', 'roxo' => '🍇' ];

// Embaralha o array:
echo "Salada: ", implode(', ', $r->shuffleArray($frutas)), "\n";

// Embaralha novamente:
echo "Outra salada: ", implode(', ', $r->shuffleArray($frutas)), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Salada: 🍅, 🥝, 🍇, 🍌, 🍑
Outra salada: 🍑, 🍇, 🥝, 🍅, 🍌
```

Random\Randomizer::shuffleBytes

Random\Randomizer::shuffleBytes

(PHP 8 >= 8.2.0)

Random\Randomizer::shuffleBytes — Obtém uma permutação byte a byte de uma string

Descrição

public function **Random\Randomizer::shuffleBytes**(string `$bytes`): string

Retorna uma permutação uniformemente selecionada dos `bytes` de entrada.

Cada permutação possível dos `bytes` de entrada tem a mesma probabilidade de ser retornada.

Parâmetros

`bytes`

A string cujos bytes são embaralhados.

A string de entrada não será modificada.

Valor Retornado

Uma permutação dos bytes do parâmetro `bytes`.

Erros/Exceções

- Qualquer erro ou exceção Throwable lançados pelo método Random\Engine::generate() do mecanismo `Random\Randomizer::$engine` subjacente.

Examples

Exemplo #1 Exemplo de Random\Randomizer::shuffleBytes()

```php
<?php
$r = new \Random\Randomizer();

// Embaralhar bytes em uma string:
echo "«", $r->shuffleBytes("PHP is great!"), "»\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
« ga rHs!PPiet»
```

Exemplo #2 O embaralhamento byte a byte quebra os caracteres Unicode

```php
<?php
$r = new \Random\Randomizer();

$unicode = "🍎, 🥝, 🍌, 🍑, 🍆";
$shuffled = $r->shuffleBytes( $unicode );

// O embaralhamento byte a byte de caracteres não ASCII os destrói,
// resultando em sequências inválidas (indicadas pelo caractere
// de substituição Unicode) ou até mesmo caracteres totalmente
// diferentes aparecendo na saída.
echo "Original: ", $unicode, "\n";
echo "Embaralhado: «", $shuffled, "»\n";
echo "Bytes embaralhados: ", bin2hex($shuffled), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Original: 🍅, 🥝, 🍌, 🍑, 🍇
Shuffled: «🍌 🍇🍇,🍅🍅🍅🍅🍅🍑,🍌 🍇🍇 🍌, 🍌,🍇🍇»
Shuffled Bytes: 87208e912c8d9fa5f0f0f09f8d9f2cf09f208c9d20f02c209f2c8d8d
```

Random\Randomizer::__unserialize

Random\Randomizer::__unserialize

(PHP 8 >= 8.2.0)

Random\Randomizer::__unserialize — Desserializa o parâmetro `data` em um objeto Randomizer

Descrição

public function **Random\Randomizer::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Randomizer::__unserialize()

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\IntervalBoundary

(PHP 8 >= 8.3.0)

Introdução

A enumeração **Random\IntervalBoundary** especifica se um intervalo inclui os valores dos limites dentro do conjunto de valores que caem no intervalo.

Resumo da enumeração

namespace **Random**;
enum **IntervalBoundary**
{

case ClosedOpen ; // Um intervalo aberto à direita. O limite inferior é incluído no intervalo, o superior não.

case ClosedClosed ; // Um intervalo fechado. Ambos os limites são incluídos no intervalo.

case OpenClosed ; // Um intervalo aberto à esquerda. O limite superior é incluído no intervalo, o inferior não.

case OpenOpen ; // Um intervalo aberto. Nenhum limite é incluído no intervalo.

}

Random\Engine

(PHP 8 >= 8.2.0)

Introdução

Um **Random\Engine** fornece uma fonte de aleatoriedade de baixo nível retornando bytes aleatórios que são consumidos por APIs de alto nível para executar suas operações. A interface **Random\Engine** permite trocar o algoritmo usado para gerar aleatoriedade, porque cada algoritmo faz compensações diferentes para se adequar a casos de uso específicos. Alguns algoritmos são muito rápidos, mas geram aleatoriedade de qualidade inferior, enquanto outros algoritmos são mais lentos, mas geram uma aleatoriedade melhor, até uma aleatoriedade criptograficamente segura, conforme fornecido pelo mecanismo Random\Engine\Secure.

O PHP fornece vários **Random\Engine**s prontos para uso para acomodar diferentes casos de uso. O mecanismo Random\Engine\Secure que é apoiado por um CSPRNG é a escolha padrão segura recomendada, a menos que o aplicativo exija sequências reproduzíveis ou desempenho muito alto.

Resumo da Interface

namespace **Random**;
interface **Engine** {
/* Métodos */
public function generate(): string
}

Índice

Random\Engine::generate

Random\Engine::generate

(PHP 8 >= 8.2.0)

Random\Engine::generate — Gera aleatoriedade

Descrição

public function **Random\Engine::generate**(): string

Retorna aleatoriedade e avança o estado do algoritmo em uma etapa.

A aleatoriedade é representada por uma string binária contendo bytes aleatórios. Esta representação permite interpretar de forma inequívoca os bits aleatórios gerados pelo algoritmo, por exemplo, para acomodar diferentes tamanhos de saída usados por diferentes algoritmos.

Algoritmos que operam nativamente em valores inteiros devem retornar o inteiro na ordem de bytes Little-Endian, por exemplo, aproveitando a função pack() com o código de formato P. A interface de alto nível fornecida pelo Random\Randomizer interpretará os bytes aleatórios retornados como inteiros Little-Endian sem sinal se uma representação numérica for necessária.

É altamente recomendável que cada bit da string retornada seja selecionado de maneira uniforme e independente, pois alguns aplicativos exigem aleatoriedade com base no nível de bit para funcionar corretamente. Por exemplo, geradores congruentes lineares geralmente geram aleatoriedade de qualidade inferior para os bits menos significativos do valor inteiro de retorno e, portanto, não seriam apropriados para aplicações que exigem aleatoriedade em nível de bit.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string não vazia contendo bytes aleatórios.

> **Nota** :
>
> A Random\Randomizer funciona internamente com inteiros sem sinal de 64 bits. Se a string retornada contiver mais de 64 bits (8 bytes) de aleatoriedade, os bytes excedentes serão ignorados. Outros aplicativos podem processar mais de 64 bits ao mesmo tempo.

Erros/Exceções

- Se a geração de aleatoriedade falhar, uma Random\RandomException deverá ser lançada. Qualquer outra Exception lançada durante a geração deve ser capturada e agrupada em uma Random\RandomException.
- Se a string retornada estiver vazia, um Random\BrokenRandomEngineError será lançado pelo Random\Randomizer.
- Se o algoritmo implementado for severamente tendencioso, um Random\BrokenRandomEngineError pode ser lançado pelo Random\Randomizer para evitar loops infinitos se a amostragem de rejeição for necessária para retornar resultados imparciais.

Examples

Exemplo #1 Exemplo de Random\Engine::generate()

```php
<?php
/**
 * Implementa um Gerador Linear Congruente com módulo 65536,
 * multiplicador 61 e incremento 17 retornando um número inteiro de 8 bits.
 *
 * Nota: Este mecanismo é adequado apenas para fins de demonstração.
 * Geradores Congruentes Lineares geralmente geram aleatoriedade
 * de baixa qualidade e esta implementação específica tem
 * um período muito curto de 16 bits que é inadequado para
 * quase todo caso de uso do mundo real.
 */
final class LinearCongruentialGenerator implements \Random\Engine
{
    private int $state;

    public function __construct(?int $seed = null)
    {
        if ($seed === null) {
            $seed = random_int(0, 0xffff);
        }

        $this->state = $seed & 0xffff;
    }

    public function generate(): string
    {
        $this->state = (61 * $this->state + 17) & 0xffff;

        return pack('C', $this->state >> 8);
    }
}

$r = new \Random\Randomizer(
    new LinearCongruentialGenerator(seed: 1)
);

echo "Número da Sorte: ", $r->getInt(0, 99), "\n";
?>
```

O exemplo acima produzirá:

```
Número da Sorte: 4
```

Random\CryptoSafeEngine

(PHP 8 >= 8.2.0)

Introdução

Uma interface de marcador indicando que Random\Engine retorna aleatoriedade criptograficamente segura.

Resumo da Interface

namespace **Random**;
interface **CryptoSafeEngine** extends Random\Engine {
/* Métodos herdados */
public function Random\Engine::generate(): string
}

Random\Engine\Secure

(PHP 8 >= 8.2.0)

Introdução

Gera aleatoriedade criptograficamente segura usando o CSPRNG do sistema operacional.

A aleatoriedade gerada por este Random\Engine é adequada para todas as aplicações, incluindo a geração de segredos de longo prazo, como chaves de criptografia.

O mecanismo **Random\Engine\Secure** é a escolha padrão segura recomendada, a menos que a aplicação exija sequências reproduzíveis ou desempenho muito alto.

Resumo da classe

namespace **Random\Engine**;
final class **Secure** implements Random\CryptoSafeEngine {
/* Métodos */
public function generate(): string
}

Índice

Random\Engine\Secure::generate

Random\Engine\Secure::generate

(PHP 8 >= 8.2.0)

Random\Engine\Secure::generate — Gera aleatoriedade criptograficamente segura

Descrição

public function **Random\Engine\Secure::generate**(): string

Retorna aleatoriedade criptograficamente segura.

As fontes de aleatoriedade usadas para esta função são as seguintes:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 se CCRandomGenerateBytes estiver disponível em tempo de compilação): CCRandomGenerateBytes()

macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*

- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Qualquer combinação de sistema operacional e versão do PHP não mencionada anteriormente: */dev/urandom*
- Se nenhuma das fontes estiver disponível ou todas falharem em gerar aleatoriedade, então uma exceção Random\RandomException será lançada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string contendo `PHP_INT_SIZE` bytes aleatórios seguros criptograficamente.

Erros/Exceções

- Se uma fonte apropriada de aleatoriedade não puder ser encontrada, uma exceção Random\RandomException será lançada.

Random\Engine\Mt19937

(PHP 8 >= 8.2.0)

Introdução

Implementa o algoritmo » Mt19937 ("Mersenne Twister").

Resumo da classe

```
namespace Random\Engine;
final class Mt19937 implements Random\Engine {
/* Métodos */
public function __construct(?int $seed = null, int $mode = MT_RAND_MT19937)
public function __debugInfo(): array
public function generate(): string
public function __serialize(): array
public function __unserialize(array $data): void
}
```

Índice

Random\Engine\Mt19937::__construct

Random\Engine\Mt19937::__construct

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__construct — Constrói um novo mecanismo Mt19937

Descrição

public function **Random\Engine\Mt19937::__construct**(?int $seed = null, int $mode = MT_RAND_MT19937)

> **Cuidado**
>
> Como o mecanismo Mt19937 ("Mersenne Twister") aceita apenas um único inteiro de 32 bits como semente, o número de sequências aleatórias possíveis é limitado a apenas 2^{32} (ou seja, 4.294.967.296), embora o Mt19937 possua um gigantesco período de 2^{19937}-1.
>
> Ao depender da geração aleatória implícita ou explícita de sementes, as duplicações aparecerão muito mais cedo. Sementes duplicadas são esperadas com probabilidade de 50% depois de menos de 80.000 sementes geradas aleatoriamente de acordo com o

problema do aniversário. Uma probabilidade de 10% de uma semente duplicada acontece depois de gerar aleatoriamente cerca de 30.000 sementes.

Isso torna o Mt19937 inadequado para aplicações em que sequências duplicadas não devem ocorrer com mais do que uma probabilidade desprezível. Se a geração reproduzível de sementes for necessária, ambos os mecanismos Random\Engine\Xoshiro256StarStar e Random\Engine\PcgOneseq128XslRr64 suportam sementes muito maiores que são improváveis de colidir aleatoriamente. Se a reprodutibilidade não for necessária, o mecanismo Random\Engine\Secure fornece aleatoriedade criptograficamente segura.

Parâmetros

`seed`

Preenche o estado com valores gerados com um gerador congruencial linear que foi semeado com `seed` interpretado como um inteiro sem sinal de 32 bits.

Se `seed` for omitido ou `null`, um inteiro aleatório de 32 bits sem sinal será usado.

`mode`

Use uma das seguintes constantes para especificar a implementação do algoritmo a ser usado.

- `MT_RAND_MT19937`: A implementação correta do Mt19937.
- `MT_RAND_PHP`: Uma implementação incorreta para compatibilidade com versões anteriores de mt_srand() anterior ao PHP 7.1.0.

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.3.0. O uso deste recurso é fortemente desencorajado.

Examples

Exemplo #1 Exemplo de Random\Engine\Mt19937::__construct()

```php
<?php
// Usa uma semente aleatória de 32 bits.
$e = new \Random\Engine\Mt19937();

$r = new \Random\Randomizer($e);
?>
```

Random\Engine\Mt19937::__debugInfo

Random\Engine\Mt19937::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__debugInfo — Retorna o estado interno do mecanismo

Descrição

public function **Random\Engine\Mt19937::__debugInfo**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de Random\Engine\Mt19937::__debugInfo()

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\Mt19937::generate

Random\Engine\Mt19937::generate

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::generate — Gerar 32 bits de aleatoriedade

Descrição

public function **Random\Engine\Mt19937::generate**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string que representa um inteiro de 32 bits sem sinal em ordem little-endian.

Random\Engine\Mt19937::__serialize

Random\Engine\Mt19937::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__serialize — Serializa o objeto Mt19937

Descrição

public function **Random\Engine\Mt19937::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de Random\Engine\Mt19937::__serialize()

```php
<?php
```

```
/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\Mt19937::__unserialize

Random\Engine\Mt19937::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__unserialize — Desserializa o parâmetro `data` em um objeto Mt19937

Descrição

public function **Random\Engine\Mt19937::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Engine\Mt19937::__unserialize()

```
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\PcgOneseq128XslRr64

(PHP 8 >= 8.2.0)

Introdução

Implementa um » gerador congruente permutado (PCG) com 128 bits de estado, transformações de saída XSL e RR e 64 bits de saída.

Resumo da classe

namespace **Random\Engine**;

final class **PcgOneseq128XslRr64** implements Random\Engine {
/* Métodos */
public function __construct(string|int|null $seed = null)
public function __debugInfo(): array
public function generate(): string
public function jump(int $advance): void
public function __serialize(): array
public function __unserialize(array $data): void
}

Índice

Random\Engine\PcgOneseq128XslRr64::__construct

Random\Engine\PcgOneseq128XslRr64::__construct

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__construct — Constrói um novo mecanismo PCG Oneseq 128 XSL RR 64

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::__construct**(string|int|null $seed = null)

Parâmetros

seed

A forma como o estado interno de 128 bits (16 bytes) consistindo em um inteiro sem sinal de 128 bits é semeado depende do tipo usado como seed.

Tipo	Descrição
null	Preenche o estado com 16 bytes aleatórios gerados usando o CSPRNG.
int	Preenche o estado definindo-o como 0, avançando o mecanismo um passo, adicionando o valor de seed interpretado como um inteiro de 64 bits sem sinal, e avançando o mecanismo outro passo.
string	Preenche o estado interpretando uma string de 16 bytes como um inteiro little-endian sem sinal de 128 bits.

Erros/Exceções

- Se o comprimento de uma string seed não for 16 bytes, um ValueError será gerado.

Examples

Exemplo #1 Exemplo de Random\Engine\PcgOneseq128XslRr64::__construct()

```php
<?php
// Usa uma semente de 128 bits aleatória.
$e = new \Random\Engine\PcgOneseq128XslRr64();

$r = new \Random\Randomizer($e);
?>
```

Exemplo #2 Derivando uma semente de uma string

```php
<?php
$string = "My string seed";

// Hash da string com SHA-256 truncado usando saída binária
// para transformar a $string em uma semente de 128 bits. Usar a mesma
// string resultará na mesma sequência de aleatoriedade.
$e = new \Random\Engine\PcgOneseq128XslRr64(
    substr(hash('sha256', $string, binary: true), 0, 16)
);

echo bin2hex($e->generate()), "\n";
?>
```

O exemplo acima produzirá:

```
8333ef59315b16d8
```

Random\Engine\PcgOneseq128XslRr64::__debugInfo

Random\Engine\PcgOneseq128XslRr64::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__debugInfo — Retorna o estado interno do mecanismo

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::__debugInfo**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de **Random\Engine\PcgOneseq128XslRr64::__debugInfo()**

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\PcgOneseq128XslRr64::generate

Random\Engine\PcgOneseq128XslRr64::generate

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::generate — Gera 64 bits de aleatoriedade

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::generate**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string que representa um inteiro de 64 bits sem sinal em ordem little-endian.

Random\Engine\PcgOneseq128XslRr64::jump

Random\Engine\PcgOneseq128XslRr64::jump

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::jump — Move o mecanismo com eficiência para frente em várias etapas

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::jump**(int `$advance`): void

Avança o estado do algoritmo pelo número de passos dado por `advance`, como se Random\Engine\PcgOneseq128XslRr64::generate() fosse chamado essa quantidade de vezes.

Parâmetros

`advance`

O número de passos para avançar; deve ser `0` ou maior.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

- Se `advance` for menor que `0`, um ValueError será gerado.

Examples

Exemplo #1 Exemplo de Random\Engine\PcgOneseq128XslRr64::jump()

```php
<?php
$a = new \Random\Engine\PcgOneseq128XslRr64(0);
$b = clone $a;

for ($i = 0; $i < 1_000; $i++) {
    $a->generate();
}
$b->jump(1_000);

echo "A: ", bin2hex($a->generate()), "\n";
echo "B: ", bin2hex($b->generate()), "\n";
?>
```

O exemplo acima produzirá:

```
A: e6d0d5813913a424
B: e6d0d5813913a424
```

Exemplo #2 Os métodos randomizadores podem chamar o mecanismo mais de uma vez

```php
<?php
$a = new \Random\Randomizer(new \Random\Engine\PcgOneseq128XslRr64(42659));
$b = new \Random\Randomizer(clone $a->engine);
```

```php
$a->getInt(1, 1572864); // Executa duas chamadas para generate().
$a->getInt(1, 1572864);

$b->engine->jump(2);

// Porque a primeira chamada para ->getInt() chamou ->generate() duas vezes
// os mecanismos não correspondem após executar um ->jump(2).
echo "A: ", bin2hex($a->engine->generate()), "\n";
echo "B: ", bin2hex($b->engine->generate()), "\n";

// Agora o mecanismo B corresponde ao mecanismo A.
echo "B: ", bin2hex($b->engine->generate()), "\n";
?>
```

O exemplo acima produzirá:

```
A: 1e9f3107d56653d0
B: a156c0086dd79d44
B: 1e9f3107d56653d0
```

Random\Engine\PcgOneseq128XslRr64::__serialize

Random\Engine\PcgOneseq128XslRr64::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__serialize — Serializa o objeto PcgOneseq128XslRr64

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de **Random\Engine\PcgOneseq128XslRr64::__serialize()**

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\PcgOneseq128XslRr64::__unserialize

Random\Engine\PcgOneseq128XslRr64::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__unserialize — Desserializa o parâmetro `data` em um objeto PcgOneseq128XslRr64

Descrição

public function **Random\Engine\PcgOneseq128XslRr64::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Engine\PcgOneseq128XslRr64::__unserialize()

```
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\Xoshiro256StarStar

(PHP 8 >= 8.2.0)

Introdução

Implementa o algoritmo » xoshiro256**.

Resumo da classe

namespace **Random\Engine**;
final class **Xoshiro256StarStar** implements Random\Engine {
/* Métodos */
public function __construct(string|int|null `$seed` = **`null`**)
public function __debugInfo(): array
public function generate(): string
public function jump(): void
public function jumpLong(): void
public function __serialize(): array
public function __unserialize(array `$data`): void
}

Índice

Random\Engine\Xoshiro256StarStar::__construct

Random\Engine\Xoshiro256StarStar::__construct

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__construct — Constrói um novo mecanismo xoshiro256**

Descrição

public function **Random\Engine\Xoshiro256StarStar::__construct**(string|int|null `$seed = null`)

Parâmetros

`seed`

A forma como o estado interno de 256 bits (32 bytes) composto por quatro inteiros de 64 bits sem sinal é semeado depende do tipo usado como `seed`.

Type	Descrição
null	Preenche o estado com 32 bytes aleatórios gerados usando o CSPRNG.
int	Preenche o estado com quatro valores consecutivos gerados com o algoritmo SplitMix64 que foi semeado com `seed` interpretado como um inteiro de 64 bits sem sinal.
string	Preenche o estado interpretando uma string de 32 bytes como quatro inteiros little-endian sem sinal de 64 bits.

Erros/Exceções

- Se o comprimento de uma string `seed` não for 32 bytes, um ValueError será gerado.
- Se uma string `seed` consistir em 32 bytes NUL (`"\x00"`), um ValueError será gerado.

Examples

Exemplo #1 Exemplo de Random\Engine\Xoshiro256StarStar::__construct()

```php
<?php
// Usa uma semente de 256 bits aleatória.
$e = new \Random\Engine\Xoshiro256StarStar();

$r = new \Random\Randomizer($e);
?>
```

Exemplo #2 Derivando uma semente de uma string

```php
<?php
$string = "My string seed";

// Faça o hash da string com SHA-256 usando a saída binária para transformar a
// $string em uma semente de 256 bits. Usar a mesma string resultará
// na mesma sequência de aleatoriedade.
$e = new \Random\Engine\Xoshiro256StarStar(
    hash('sha256', $string, binary: true)
);

echo bin2hex($e->generate()), "\n";
?>
```

O exemplo acima produzirá:

```
6e013453678388c2
```

Random\Engine\Xoshiro256StarStar::__debugInfo

Random\Engine\Xoshiro256StarStar::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__debugInfo — Retorna o estado interno do mecanismo

Descrição

public function **Random\Engine\Xoshiro256StarStar::__debugInfo**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de Random\Engine\Xoshiro256StarStar::__debugInfo()

```
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\Xoshiro256StarStar::generate

Random\Engine\Xoshiro256StarStar::generate

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::generate — Gera 64 bits de aleatoriedade

Descrição

public function **Random\Engine\Xoshiro256StarStar::generate**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string que representa um inteiro de 64 bits sem sinal em ordem little-endian.

Random\Engine\Xoshiro256StarStar::jump

Random\Engine\Xoshiro256StarStar::jump

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::jump — Move o mecanismo com eficiência para frente em passos de 2^128

Descrição

public function **Random\Engine\Xoshiro256StarStar::jump**(): void

Avança o estado do algoritmo em 2^{128} passos, como se Random\Engine\Xoshiro256StarStar::generate() fosse chamado 2^{128} vezes.

O objetivo de um salto é facilitar a criação de um novo mecanismo Random\Engine\Xoshiro256StarStar a partir de um mecanismo Random\Engine\Xoshiro256StarStar já existente. O mecanismo inicial atua como um projeto, que pode ser clonado e saltado repetidamente para criar 2^{128} sequências não sobrepostas com 2^{128} valores cada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Engine\Xoshiro256StarStar::jump()

```php
<?php
use Random\Engine\Xoshiro256StarStar;
use Random\Randomizer;

$blueprintRng = new Xoshiro256StarStar(0);

$fibers = [];
for ($i = 0; $i < 8; $i++) {
    $fiberRng = clone $blueprintRng;
    $blueprintRng->jump();

    $fiber = new Fiber(static function () use ($fiberRng, $i): void {
        $randomizer = new Randomizer($fiberRng);

        while (true) {
            Fiber::suspend();

            echo "{$i}: ", $randomizer->getInt(0, 100), "\n";
        }
    });
    $fiber->start();

    $fibers[] = $fiber;
}

// Mesmo que os Fibers sejam executados em ordem aleatória, eles imprimirão o mesmo valor
// todas as vezes, porque cada um tem sua própria instância única do RNG.
$randomizer = new Randomizer();

$fibers = $randomizer->shuffleArray($fibers);
foreach ($fibers as $fiber) {
    $fiber->resume();
}

$fibers = $randomizer->shuffleArray($fibers);
foreach ($fibers as $fiber) {
    $fiber->resume();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
4: 89
3: 10
2: 63
1: 75
6: 41
```

```
5: 56
0: 16
7: 60
7: 34
6: 58
1: 74
4: 63
3: 3
5: 42
2: 45
0: 86
```

Veja também

- Random\Engine\Xoshiro256StarStar::jumpLong() - Avança eficientemente o mecanismo em 2^192 passos

Random\Engine\Xoshiro256StarStar::jumpLong

Random\Engine\Xoshiro256StarStar::jumpLong

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::jumpLong — Avança eficientemente o mecanismo em 2^192 passos

Descrição

public function **Random\Engine\Xoshiro256StarStar::jumpLong**(): void

Avança o estado do algoritmo em 2^{192} passos, como se Random\Engine\Xoshiro256StarStar::generate() fosse chamado 2^{192} vezes.

O objetivo de um salto longo é facilitar a criação de um novo mecanismo Random\Engine\Xoshiro256StarStar a partir de um mecanismo Random\Engine\Xoshiro256StarStar já existente. O mecanismo inicial atua como um processo, que pode ser clonado e saltado repetidamente para criar 2^{64} sequências não sobrepostas com 2^{192} valores cada.

O salto longo pode ser combinado com Random\Engine\Xoshiro256StarStar::jump() para dividir ainda mais cada uma das 2^{64} sequências geradas pelo salto longo, em 2^{64} sequências de 2^{128} valores cada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Engine\Xoshiro256StarStar::jumpLong()

```php
<?php
$blueprintRng = new \Random\Engine\Xoshiro256StarStar(0);

// Cada mecanismo pai terá seu próprio bloco de 2**192 valores.
$parent1 = clone $blueprintRng;
$blueprintRng->jumpLong();

$parent2 = clone $blueprintRng;
$blueprintRng->jumpLong();

// Cada um dos mecanismos filhos terá seu próprio bloco de 2**128 valores
// retirado do bloco de 2**192 valores do mecanismo pai.
$child1a = clone $parent1;
$parent1->jump();
$child1b = clone $parent1;
$parent1->jump();

$child2a = clone $parent2;
```

```php
$parent2->jump();
$child2b = clone $parent2;
$parent2->jump();

echo "Child 1A: ", bin2hex($child1a->generate()), "\n";
echo "Child 1B: ", bin2hex($child1b->generate()), "\n";
echo "Child 2A: ", bin2hex($child2a->generate()), "\n";
echo "Child 2B: ", bin2hex($child2b->generate()), "\n";
?>
```

O exemplo acima produzirá:

```
Child 1A: b4f275cb365fec99
Child 1B: 2cd646c8ed156237
Child 2A: eb3729a722a504e7
Child 2B: d4208dc85bdd6dc3
```

Veja também

- Random\Engine\Xoshiro256StarStar::jump() - Move o mecanismo com eficiência para frente em passos de 2^128

Random\Engine\Xoshiro256StarStar::__serialize

Random\Engine\Xoshiro256StarStar::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__serialize — Serializa o objeto Xoshiro256StarStar

Descrição

public function **Random\Engine\Xoshiro256StarStar::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de **Random\Engine\Xoshiro256StarStar::__serialize()**

```php
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\Engine\Xoshiro256StarStar::__unserialize

Random\Engine\Xoshiro256StarStar::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__unserialize — Desserializa o parâmetro `data` em um objeto Xoshiro256StarStar

Descrição

public function **Random\Engine\Xoshiro256StarStar::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Random\Engine\Xoshiro256StarStar::__unserialize()

```
<?php

/* ... */

?>
```

O exemplo acima produzirá algo semelhante a:

```
...
```

Random\RandomError

(PHP 8 >= 8.2.0)

Introdução

A classe base para Errors que ocorrem durante a geração ou uso de aleatoriedade.

Resumo da classe

namespace **Random**;
class **RandomError** extends Error {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string

final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Random\BrokenRandomEngineError

(PHP 8 >= 8.2.0)

Introdução

Indica que o Random\Engine usado está quebrado, por exemplo, porque é severamente tendencioso.

Resumo da classe

namespace **Random**;
class **BrokenRandomEngineError** extends Random\RandomError {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Random\RandomException

(PHP 8 >= 8.2.0)

Introdução

A classe base para Exceptions que ocorrem durante a geração ou uso de aleatoriedade.

Resumo da classe

namespace **Random**;
class **RandomException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */

```
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Biblioteca Padrão PHP (SPL)

Introdução

A extensão SPL - Standard PHP Library (Biblioteca Padrão PHP) define interfaces e classes destinadas a resolver problemas comuns.

A extensão fornece várias estruturas de dados, exceções genéricas, iteradores classe para trabalhar com sistemas de arquivos usando uma API orientada a objeto, e funções utilitárias.

Interfaces

Índice

SPL fornece um conjunto de interfaces para melhorar iteradores, e um par de interfaces para implementar o Observer Design Pattern.

Veja também

- Interfaces e Classes predefinidas

OuterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Classes que implementam **OuterIterator** podem ser usadas para iterar sobre iteradores.

Resumo da Interface

interface **OuterIterator** extends Iterator {
/* Métodos */
public function getInnerIterator(): ?Iterator
/* Métodos herdados */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}

Índice

OuterIterator::getInnerIterator

OuterIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

OuterIterator::getInnerIterator — Retorna o iterador interno para a entrada atual

Descrição

public function **OuterIterator::getInnerIterator**(): ?Iterator

Retorna o iterador interno para a entrada atual do iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o iterador interno para a entrada atual se existir, ou `null` caso contrário.

RecursiveIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Classes que implementam **RecursiveIterator** podem ser usadas para iterar sobre iteradores de forma recursiva.

Resumo da Interface

interface **RecursiveIterator** extends Iterator {
/* Métodos */
public function getChildren(): ?RecursiveIterator
public function hasChildren(): bool
/* Métodos herdados */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}

Índice

RecursiveIterator::getChildren

RecursiveIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIterator::getChildren — Retorna um iterador para a entrada atual

Descrição

public function **RecursiveIterator::getChildren**(): ?RecursiveIterator

Retorna um iterador para a entrada atual do iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um iterador para a entrada atual se ela existir, ou `null` caso contrário.

Veja também

- RecursiveIterator::hasChildren() - Retorna se um iterador pode ser criado para a entrada atual

RecursiveIterator::hasChildren

RecursiveIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIterator::hasChildren — Retorna se um iterador pode ser criado para a entrada atual

Descrição

public function **RecursiveIterator::hasChildren**(): bool

Retorna se um iterador pode ser criado para a entrada atual. RecursiveIterator::getChildren().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a entrada atual puder ser iterada, caso contrário retorna `false`.

Veja também

- RecursiveIterator::getChildren() - Retorna um iterador para a entrada atual

SeekableIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

O iterador Seekable.

Resumo da Interface

interface **SeekableIterator** extends Iterator {
/* Métodos */
public function seek(int `$offset`): void
/* Métodos herdados */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}

Examples

Exemplo #1 Utilização básica

Este exemplo demonstra a criação de um **SeekableIterator** personalizado, buscando uma posição e lidando com uma posição inválida.

```php
<?php
class MySeekableIterator implements SeekableIterator {

    private $position;

    private $array = array(
        "first element",
        "second element",
        "third element",
        "fourth element"
    );

    /* Método necessário para a interface SeekableIterator */

    public function seek($position) {
      if (!isset($this->array[$position])) {
          throw new OutOfBoundsException("invalid seek position ($position)");
      }

      $this->position = $position;
    }

    /* Methods required for Iterator interface */

    public function rewind() {
        $this->position = 0;
    }

    public function current() {
        return $this->array[$this->position];
    }

    public function key() {
        return $this->position;
    }

    public function next() {
        ++$this->position;
    }

    public function valid() {
```

```
        return isset($this->array[$this->position]);
    }
}

try {

    $it = new MySeekableIterator;
    echo $it->current(), "\n";

    $it->seek(2);
    echo $it->current(), "\n";

    $it->seek(1);
    echo $it->current(), "\n";

    $it->seek(10);

} catch (OutOfBoundsException $e) {
    echo $e->getMessage();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
first element
third element
second element
invalid seek position (10)
```

Índice

SeekableIterator::seek

SeekableIterator::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SeekableIterator::seek — Aponta para uma posição

Descrição

public function **SeekableIterator::seek**(int `$offset`): void

Aponta o iterador para uma posição específica.

Parâmetros

`offset`

A posição para onde apontar.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Implementações devem lançar uma OutOfBoundsException se o `offset` não for acessível.

Examples

Exemplo #1 Exemplo de SeekableIterator::seek()

Aponta para o item na posição 3 no iterador (ArrayIterator implementa SeekableIterator).

```
<?php
$array = array("apple", "banana", "cherry", "damson", "elderberry");
```

```php
$iterator = new ArrayIterator($array);
$iterator->seek(3);
echo $iterator->current();
?>
```

O exemplo acima produzirá algo semelhante a:

```
damson
```

Veja também

- SeekableIterator
- Iterator

SplObserver

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A interface **SplObserver** é usada junto com SplSubject para implementar o padrão de design Observer.

Resumo da Interface

interface **SplObserver** {
/* Métodos */
public function update(SplSubject `$subject`): void
}

Índice

SplObserver::update

SplObserver::update

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObserver::update — Recebe atualização do sujeito

Descrição

public function **SplObserver::update**(SplSubject `$subject`): void

Este método é chamado quando qualquer SplSubject ao qual o observador está anexado chama SplSubject::notify().

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`subject`

O SplSubject notificando o observador de uma atualização.

Valor Retornado

Nenhum valor é retornado.

SplSubject

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A interface **SplSubject** é usada em conjunto com SplObserver para implementar o padrão de design Observer.

Resumo da Interface

interface **SplSubject** {
/* Métodos */
public function attach(SplObserver `$observer`): void
public function detach(SplObserver `$observer`): void
public function notify(): void
}

Índice

SplSubject::attach

SplSubject::attach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::attach — Anexa um SplObserver

Descrição

public function **SplSubject::attach**(SplObserver `$observer`): void

Anexa um SplObserver para que ele possa ser notificado sobre atualizações.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`observer`

O SplObserver para anexar.

Valor Retornado

Nenhum valor é retornado.

SplSubject::detach

SplSubject::detach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::detach — Desanexa um observador

Descrição

public function **SplSubject::detach**(SplObserver `$observer`): void

Desanexa um observador do assunto para não mais notificá-lo sobre atualizações.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`observer`

O SplObserver para desanexar.

Valor Retornado

Nenhum valor é retornado.

SplSubject::notify

SplSubject::notify

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::notify — Notifica um observador

Descrição

public function **SplSubject::notify**(): void

Notifica todos os observadores anexados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Estruturas de dados

Índice

SPL fornece um conjunto de estruturas de dados padrão. Elas são agrupadas aqui de acordo com sua implementação subjacente, que geralmente define seu campo de aplicação geral.

Listas Duplamente Encadeadas

Uma Lista Duplamente Encadeada (DLL) é uma lista de nós vinculados em ambas as direções entre si. As operações de iteração, acesso a ambas as extremidades, adição ou remoção de nós têm um custo de O(1) quando a estrutura subjacente é uma DLL. Portanto, ela fornece uma implementação decente para pilhas e filas.

- SplDoublyLinkedList
    - SplStack
    - SplQueue

Heaps

Heaps são estruturas semelhantes a árvores que seguem a propriedade de heap: cada nó é maior ou igual aos seus filhos, quando comparados usando o método de comparação implementado, que é global para o heap.

- SplHeap
    - SplMaxHeap
    - SplMinHeap
- SplPriorityQueue

Arrays

Arrays são estruturas que armazenam os dados de forma contínua, acessíveis via índices.

> **Nota** :
>
> Não os confunda com o tipo array nativo do PHP. Os arrays PHP são, na verdade, tabelas de hash ordenadas. Entretanto, a SPL fornece a classe ArrayObject para encapsular arrays do PHP em um objeto.

- SplFixedArray

Map

Um mapa é uma estrutura de dados que contém pares chave-valor. Os arrays PHP podem ser vistos como mapas de inteiros/strings para valores. SPL fornece um mapa de objetos para dados. Esse mapa também pode ser usado como um conjunto de objetos.

- SplObjectStorage

SplDoublyLinkedList

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplDoublyLinkedList fornece as principais funcionalidades de uma lista duplamente encadeada.

Resumo da classe

class **SplDoublyLinkedList** implements Iterator, Countable, ArrayAccess, Serializable {
/* Constantes */
public const int *IT_MODE_LIFO*;
public const int *IT_MODE_FIFO*;
public const int *IT_MODE_DELETE*;
public const int *IT_MODE_KEEP*;
/* Métodos */
public function add(int `$index`, mixed `$value`): void
public function bottom(): mixed
public function count(): int
public function current(): mixed
public function getIteratorMode(): int
public function isEmpty(): bool
public function key(): int
public function next(): void
public function offsetExists(int `$index`): bool
public function offsetGet(int `$index`): mixed
public function offsetSet(?int `$index`, mixed `$value`): void
public function offsetUnset(int `$index`): void
public function pop(): mixed
public function prev(): void
public function push(mixed `$value`): void
public function rewind(): void
public function serialize(): string
public function setIteratorMode(int `$mode`): int
public function shift(): mixed
public function top(): mixed
public function unserialize(string `$data`): void
public function unshift(mixed `$value`): void
public function valid(): bool
}

Constantes predefinidas

Direção da Iteração

`SplDoublyLinkedList::IT_MODE_LIFO`

A lista será iterada em uma ordem de último a entrar, primeiro a sair, como uma pilha.

`SplDoublyLinkedList::IT_MODE_FIFO`

A lista será iterada em uma ordem de primeiro a entrar, primeiro a sair, como uma fila.

Comportamento da Iteração

`SplDoublyLinkedList::IT_MODE_DELETE`

A iteração removerá os elementos iterados.

`SplDoublyLinkedList::IT_MODE_KEEP`

A iteração não removerá os elementos iterados.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

SplDoublyLinkedList::add

SplDoublyLinkedList::add

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

SplDoublyLinkedList::add — Adiciona/insere um novo valor no índice especificado

Descrição

public function **SplDoublyLinkedList::add**(int `$index`, mixed `$value`): void

Insere o valor `value` no `index` especificado, reorganizando o valor anterior nesse índice (e todos os valores subsequentes) pela lista.

Parâmetros

`index`

O índice onde o novo valor será inserido.

`value`

O novo valor para o `index`.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança OutOfRangeException quando o `index` está fora dos limites ou quando o `index` não pode ser interpretado como um número inteiro.

SplDoublyLinkedList::bottom

SplDoublyLinkedList::bottom

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::bottom — Espreita o nó desde o início da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::bottom**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do primeiro nó.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplDoublyLinkedList::count

SplDoublyLinkedList::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::count — Conta o número de elementos na lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::count**(): int

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de elementos na lista duplamente encadeada.

SplDoublyLinkedList::current

SplDoublyLinkedList::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::current — Retorna a entrada de array atual

Descrição

public function **SplDoublyLinkedList::current**(): mixed

Obtém o nó atual da lista duplamente encadeada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó atual.

SplDoublyLinkedList::getIteratorMode

SplDoublyLinkedList::getIteratorMode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::getIteratorMode — Retorna o modo de iteração

Descrição

public function **SplDoublyLinkedList::getIteratorMode**(): int

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna os diferentes modos e flags que afetam a iteração.

SplDoublyLinkedList::isEmpty

SplDoublyLinkedList::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::isEmpty — Verifica se a lista duplamente encadeada está vazia

Descrição

public function **SplDoublyLinkedList::isEmpty**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna se a lista duplamente encadeada está vazia.

SplDoublyLinkedList::key

SplDoublyLinkedList::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::key — Retorna o índice do nó atual

Descrição

public function **SplDoublyLinkedList::key**(): int

Esta função retorna o índice do nó atual

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O índice do nó atual.

SplDoublyLinkedList::next

SplDoublyLinkedList::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::next — Move para a próxima entrada

Descrição

public function **SplDoublyLinkedList::next**(): void

Move o iterador para o próximo nó.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplDoublyLinkedList::offsetExists

SplDoublyLinkedList::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetExists — Retorna se o $index solicitado existe

Descrição

public function **SplDoublyLinkedList::offsetExists**(int `$index`): bool

Parâmetros

`index`

O índice sendo verificado.

Valor Retornado

`true` se o `index` solicitado existe, caso contrário, **`false`**

SplDoublyLinkedList::offsetGet

SplDoublyLinkedList::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetGet — Retorna o valor no $index especificado

Descrição

public function **SplDoublyLinkedList::offsetGet**(int `$index`): mixed

Parâmetros

`index`

O índice com o valor.

Valor Retornado

O valor no `index` especificado.

Erros/Exceções

Lança OutOfRangeException quando o `index` está fora dos limites ou quando o `index` não pode ser interpretado como um número inteiro.

SplDoublyLinkedList::offsetSet

SplDoublyLinkedList::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetSet — Define o valor no $index especificado como $value

Descrição

public function **SplDoublyLinkedList::offsetSet**(?int `$index`, mixed `$value`): void

Define o valor no `index` especificado como `value`.

Parâmetros

`index`

O índice sendo definido. Se for `null`, o próximo valor será adicionado após o último item.

`value`

O novo valor para o `index`.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança OutOfRangeException quando o `index` está fora dos limites ou quando o `index` não pode ser interpretado como um número inteiro.

SplDoublyLinkedList::offsetUnset

SplDoublyLinkedList::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetUnset — Remove o valor no $index especificado

Descrição

public function **SplDoublyLinkedList::offsetUnset**(int `$index`): void

Remove o valor no índice especificado.

Parâmetros

`index`

O índice sendo removido.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança OutOfRangeException quando o `index` está fora dos limites ou quando o `index` não pode ser interpretado como um número inteiro.

SplDoublyLinkedList::pop

SplDoublyLinkedList::pop

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::pop — Remove um nó do final da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::pop**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó removido.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplDoublyLinkedList::prev

SplDoublyLinkedList::prev

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::prev — Move para a entrada anterior

Descrição

public function **SplDoublyLinkedList::prev**(): void

Move o iterador para o nó anterior.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplDoublyLinkedList::push

SplDoublyLinkedList::push

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::push — Adiciona um elemento no final da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::push**(mixed `$value`): void

Adiciona o `value` no final da lista duplamente encadeada.

Parâmetros

`value`

O valor a ser adicionado.

Valor Retornado

Nenhum valor é retornado.

SplDoublyLinkedList::rewind

SplDoublyLinkedList::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::rewind — Rebobina o iterador de volta ao início

Descrição

public function **SplDoublyLinkedList::rewind**(): void

Isso rebobina o iterador para o início.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplDoublyLinkedList::serialize

SplDoublyLinkedList::serialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplDoublyLinkedList::serialize — Serializa o armazenamento

Descrição

public function **SplDoublyLinkedList::serialize**(): string

Serializa o armazenamento.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A string serializada.

Veja também

- SplDoublyLinkedList::unserialize() - Deserializa o armazenamento

SplDoublyLinkedList::setIteratorMode

SplDoublyLinkedList::setIteratorMode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::setIteratorMode — Define o modo de iteração

Descrição

public function **SplDoublyLinkedList::setIteratorMode**(int `$mode`): int

Parâmetros

`mode`

Existem dois conjuntos ortogonais de modos que podem ser definidos:

- A direção da iteração (uma ou outra):
  - `SplDoublyLinkedList::IT_MODE_LIFO` (Estilo pilha)
  - `SplDoublyLinkedList::IT_MODE_FIFO` (Estilo fila)
- O comportamento do iterador (um ou outro):
  - `SplDoublyLinkedList::IT_MODE_DELETE` (Os elementos são excluídos pelo iterador)
  - `SplDoublyLinkedList::IT_MODE_KEEP` (Os elementos são percorridos pelo iterador)

O modo padrão é: `SplDoublyLinkedList::IT_MODE_FIFO` | `SplDoublyLinkedList::IT_MODE_KEEP`

> **Aviso**
>
> A direção da iteração não pode ser alterada para as classes SplStack e SplQueue, ela é sempre `SplDoublyLinkedList::IT_MODE_FIFO`. Tentar modificá-la resultará em uma exceção RuntimeException sendo lançada.

Valor Retornado

Retorna os diferentes modos e flags que afetam a iteração.

SplDoublyLinkedList::shift

SplDoublyLinkedList::shift

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::shift — Remove um nó do início da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::shift**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó removido.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplDoublyLinkedList::top

SplDoublyLinkedList::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::top — Observa o nó do final da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::top**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do último nó.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplDoublyLinkedList::unserialize

SplDoublyLinkedList::unserialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplDoublyLinkedList::unserialize — Deserializa o armazenamento

Descrição

public function **SplDoublyLinkedList::unserialize**(string `$data`): void

Deserializa o armazenamento, a partir de SplDoublyLinkedList::serialize().

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

A string serializada.

Valor Retornado

Nenhum valor é retornado.

Veja também

- SplDoublyLinkedList::serialize() - Serializa o armazenamento

SplDoublyLinkedList::unshift

SplDoublyLinkedList::unshift

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::unshift — Acrescenta um elemento ao início da lista duplamente encadeada

Descrição

public function **SplDoublyLinkedList::unshift**(mixed `$value`): void

Acrescenta `value` ao início da lista duplamente encadeada.

Parâmetros

`value`

O valor a ser acrescentado.

Valor Retornado

Nenhum valor é retornado.

SplDoublyLinkedList::valid

SplDoublyLinkedList::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::valid — Verifica se a lista duplamente encadeada contém mais nós

Descrição

public function **SplDoublyLinkedList::valid**(): bool

Verifica se a lista duplamente encadeada contém mais nós.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a lista duplamente encadeada contiver mais nós, `false` caso contrário.

SplStack

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplStack fornece as principais funcionalidades de uma pilha implementada usando uma lista duplamente encadeada definindo o modo do iterador como `SplDoublyLinkedList::IT_MODE_LIFO`.

Resumo da classe

class **SplStack** extends SplDoublyLinkedList {
/* Constantes herdadas */
public const int *SplDoublyLinkedList::IT_MODE_LIFO*;
public const int *SplDoublyLinkedList::IT_MODE_FIFO*;
public const int *SplDoublyLinkedList::IT_MODE_DELETE*;
public const int *SplDoublyLinkedList::IT_MODE_KEEP*;
/* Métodos herdados */
public function SplDoublyLinkedList::add(int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::bottom(): mixed
public function SplDoublyLinkedList::count(): int
public function SplDoublyLinkedList::current(): mixed
public function SplDoublyLinkedList::getIteratorMode(): int
public function SplDoublyLinkedList::isEmpty(): bool
public function SplDoublyLinkedList::key(): int
public function SplDoublyLinkedList::next(): void
public function SplDoublyLinkedList::offsetExists(int `$index`): bool
public function SplDoublyLinkedList::offsetGet(int `$index`): mixed
public function SplDoublyLinkedList::offsetSet(?int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::offsetUnset(int `$index`): void
public function SplDoublyLinkedList::pop(): mixed
public function SplDoublyLinkedList::prev(): void
public function SplDoublyLinkedList::push(mixed `$value`): void
public function SplDoublyLinkedList::rewind(): void
public function SplDoublyLinkedList::serialize(): string
public function SplDoublyLinkedList::setIteratorMode(int `$mode`): int
public function SplDoublyLinkedList::shift(): mixed
public function SplDoublyLinkedList::top(): mixed
public function SplDoublyLinkedList::unserialize(string `$data`): void
public function SplDoublyLinkedList::unshift(mixed `$value`): void
public function SplDoublyLinkedList::valid(): bool
}

Examples

Exemplo #1 Exemplo de **SplStack**

```php
<?php
$q = new SplStack();
$q[] = 1;
$q[] = 2;
$q[] = 3;
foreach ($q as $elem)  {
```

```
 echo $elem."\n";
}
?>
```

O exemplo acima produzirá:

```
3
2
1
```

SplQueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplQueue fornece as principais funcionalidades de uma fila implementada usando uma lista duplamente encadeada, estabelecendo o modo do iterador como `SplDoublyLinkedList::IT_MODE_FIFO`.

Resumo da classe

class **SplQueue** extends SplDoublyLinkedList {
/* Constantes herdadas */
public const int *SplDoublyLinkedList::IT_MODE_LIFO*;
public const int *SplDoublyLinkedList::IT_MODE_FIFO*;
public const int *SplDoublyLinkedList::IT_MODE_DELETE*;
public const int *SplDoublyLinkedList::IT_MODE_KEEP*;
/* Métodos */
public function dequeue(): mixed
public function enqueue(mixed `$value`): void
/* Métodos herdados */
public function SplDoublyLinkedList::add(int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::bottom(): mixed
public function SplDoublyLinkedList::count(): int
public function SplDoublyLinkedList::current(): mixed
public function SplDoublyLinkedList::getIteratorMode(): int
public function SplDoublyLinkedList::isEmpty(): bool
public function SplDoublyLinkedList::key(): int
public function SplDoublyLinkedList::next(): void
public function SplDoublyLinkedList::offsetExists(int `$index`): bool
public function SplDoublyLinkedList::offsetGet(int `$index`): mixed
public function SplDoublyLinkedList::offsetSet(?int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::offsetUnset(int `$index`): void
public function SplDoublyLinkedList::pop(): mixed
public function SplDoublyLinkedList::prev(): void
public function SplDoublyLinkedList::push(mixed `$value`): void
public function SplDoublyLinkedList::rewind(): void
public function SplDoublyLinkedList::serialize(): string
public function SplDoublyLinkedList::setIteratorMode(int `$mode`): int
public function SplDoublyLinkedList::shift(): mixed
public function SplDoublyLinkedList::top(): mixed
public function SplDoublyLinkedList::unserialize(string `$data`): void
public function SplDoublyLinkedList::unshift(mixed `$value`): void
public function SplDoublyLinkedList::valid(): bool
}

Examples

Exemplo #1 Exemplo de SplQueue

```php
<?php
$q = new SplQueue();
$q[] = 1;
$q[] = 2;
$q[] = 3;
foreach ($q as $elem)  {
 echo $elem."\n";
}
?>
```

O exemplo acima produzirá:

```
1
2
3
```

Exemplo #2 Gestão eficiente de tarefas usando SplQueue

```php
<?php
$q = new SplQueue();
$q->setIteratorMode(SplQueue::IT_MODE_DELETE);
// ... adicionar algumas tarefas à fila ...
// processá-las.
foreach ($q as $task) {
    // ... processar a tarefa $task ...
    // adicionar novas tarefas à fila
    $q[] = $newTask;
    // ...
}
?>
```

Índice

SplQueue::dequeue

SplQueue::dequeue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplQueue::dequeue — Remove um nó da fila

Descrição

public function **SplQueue::dequeue**(): mixed

Remove o `value` do topo da fila.

> **Nota** :
>
> **SplQueue::dequeue()** é um sinônimo de SplDoublyLinkedList::shift().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó retirado da fila.

SplQueue::enqueue

SplQueue::enqueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplQueue::enqueue — Adiciona um elemento à fila

Descrição

public function **SplQueue::enqueue**(mixed `$value`): void

Enfileira `value` no final da fila.

> **Nota** :
>
> **SplQueue::enqueue()** é um sinônimo de SplDoublyLinkedList::push().

Parâmetros

`value`

O valor a ser enfileirado.

Valor Retornado

Nenhum valor é retornado.

SplHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplHeap fornece as principais funcionalidades de um Heap.

Resumo da classe

abstract class **SplHeap** implements Iterator, Countable {
/* Métodos */
protected function compare(mixed `$value1`, mixed `$value2`): int
public function count(): int
public function current(): mixed
public function extract(): mixed
public function insert(mixed `$value`): true
public function isCorrupted(): bool
public function isEmpty(): bool
public function key(): int
public function next(): void
public function recoverFromCorruption(): true
public function rewind(): void
public function top(): mixed
public function valid(): bool
}

Índice

SplHeap::compare

SplHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::compare — Compara elementos para colocá-los corretamente no heap enquanto faz o ajuste ascendente

Descrição

protected function **SplHeap::compare**(mixed `$value1`, mixed `$value2`): int

Compara `value1` com `value2`.

> **Aviso**

> Lançar exceções em **SplHeap::compare()** pode corromper o Heap e colocá-lo em um estado bloqueado. Você pode desbloqueá-lo chamando SplHeap::recoverFromCorruption(). No entanto, alguns elementos podem não ser colocados corretamente e, portanto, podem quebrar a propriedade do heap.

Parâmetros

`value1`

O valor do primeiro nó sendo comparado.

`value2`

O valor do segundo nó sendo comparado.

Valor Retornado

Resultado da comparação, inteiro positivo se `value1` for maior que `value2`, 0 se forem iguais, inteiro negativo caso contrário.

> **Nota** :
>
> Ter múltiplos elementos com o mesmo valor em um Heap não é recomendado. Eles acabarão em uma posição relativa arbitrária.

SplHeap::count

SplHeap::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::count — Conta o número de elementos no heap

Descrição

public function **SplHeap::count**(): int

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de elementos no heap.

SplHeap::current

SplHeap::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::current — Retorna o nó atual apontado pelo iterador

Descrição

public function **SplHeap::current**(): mixed

Obtém o nó atual da estrutura de dados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó atual.

SplHeap::extract

SplHeap::extract

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::extract — Extrai um nó do topo do heap e ajusta para cima

Descrição

public function **SplHeap::extract**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó extraído.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplHeap::insert

SplHeap::insert

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::insert — Insere um elemento no heap, ajustando-o para cima

Descrição

public function **SplHeap::insert**(mixed `$value`): true

Insere o `value` no heap.

Parâmetros

`value`

O valor a ser inserido.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.4.0	**SplHeap::insert()** agora tem um tipo de retorno provisório de true.

SplHeap::isCorrupted

SplHeap::isCorrupted

(PHP 7, PHP 8)

SplHeap::isCorrupted — Indica se o heap está em um estado corrompido

Descrição

public function **SplHeap::isCorrupted**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna **`true`** se o heap estiver corrompido, **`false`** caso contrário.

SplHeap::isEmpty

SplHeap::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::isEmpty — Verifica se o heap está vazio

Descrição

public function **SplHeap::isEmpty**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna se o heap está vazio.

SplHeap::key

SplHeap::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::key — Retorna o índice do nó atual

Descrição

public function **SplHeap::key**(): int

Esta função retorna o índice do nó atual

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O índice do nó atual.

SplHeap::next

SplHeap::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::next — Move para o próximo nó

Descrição

public function **SplHeap::next**(): void

Move para o próximo nó. Isso irá excluir o nó superior do heap.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplHeap::recoverFromCorruption

SplHeap::recoverFromCorruption

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::recoverFromCorruption — Recupera do estado corrompido e permite outras ações no heap

Descrição

public function **SplHeap::recoverFromCorruption**(): true

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.4.0	**SplHeap::recoverFromCorruption()** agora tem um tipo de retorno provisório de true.

SplHeap::rewind

SplHeap::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::rewind — Rebobina o iterador de volta ao início (sem ação)

Descrição

public function **SplHeap::rewind**(): void

Isso rebobina o iterador para o início. Isso não faz nada para heaps, pois o iterador é virtual e na verdade nunca se move do topo da heap.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplHeap::top

SplHeap::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::top — Espreita o nó do topo da heap

Descrição

public function **SplHeap::top**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do nó no topo.

Erros/Exceções

Lança RuntimeException quando a estrutura de dados está vazia.

SplHeap::valid

SplHeap::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::valid — Verifica se a heap contém mais nós

Descrição

public function **SplHeap::valid**(): bool

Verifica se a heap contém mais nós.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a heap contiver mais nós, `false` caso contrário.

SplMaxHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplMaxHeap fornece as principais funcionalidades de um heap, mantendo o máximo no topo.

Resumo da classe

class **SplMaxHeap** extends SplHeap {
/* Métodos */
protected function compare(mixed `$value1`, mixed `$value2`): int
/* Métodos herdados */
protected function SplHeap::compare(mixed `$value1`, mixed `$value2`): int
public function SplHeap::count(): int
public function SplHeap::current(): mixed
public function SplHeap::extract(): mixed
public function SplHeap::insert(mixed `$value`): true
public function SplHeap::isCorrupted(): bool
public function SplHeap::isEmpty(): bool
public function SplHeap::key(): int
public function SplHeap::next(): void
public function SplHeap::recoverFromCorruption(): true
public function SplHeap::rewind(): void
public function SplHeap::top(): mixed
public function SplHeap::valid(): bool
}

Índice

SplMaxHeap::compare

SplMaxHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplMaxHeap::compare — Compara elementos para colocá-los corretamente na heap enquanto faz o ajuste ascendente

Descrição

protected function **SplMaxHeap::compare**(mixed `$value1`, mixed `$value2`): int

Compara `value1` com `value2`.

Parâmetros

`value1`

O valor do primeiro nó sendo comparado.

`value2`

O valor do segundo nó sendo comparado.

Valor Retornado

Resultado da comparação, inteiro positivo se `value1` for maior que `value2`, 0 se forem iguais, inteiro negativo caso contrário.

> **Nota** :
>
> Ter múltiplos elementos com o mesmo valor em uma Heap não é recomendado. Eles acabarão em uma posição relativa arbitrária.

SplMinHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplMinHeap fornece as principais funcionalidades de um heap, mantendo o mínimo no topo.

Resumo da classe

class **SplMinHeap** extends SplHeap {
/* Métodos */
protected function compare(mixed `$value1`, mixed `$value2`): int
/* Métodos herdados */
protected function SplHeap::compare(mixed `$value1`, mixed `$value2`): int
public function SplHeap::count(): int
public function SplHeap::current(): mixed
public function SplHeap::extract(): mixed
public function SplHeap::insert(mixed `$value`): true
public function SplHeap::isCorrupted(): bool
public function SplHeap::isEmpty(): bool
public function SplHeap::key(): int
public function SplHeap::next(): void
public function SplHeap::recoverFromCorruption(): true
public function SplHeap::rewind(): void
public function SplHeap::top(): mixed
public function SplHeap::valid(): bool
}

Índice

SplMinHeap::compare

SplMinHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplMinHeap::compare — Comparar elementos para colocá-los corretamente na heap enquanto faz o ajuste ascendente

Descrição

protected function **SplMinHeap::compare**(mixed `$value1`, mixed `$value2`): int

Comparar `value1` com `value2`.

Parâmetros

`value1`

O valor do primeiro nó sendo comparado.

`value2`

O valor do segundo nó sendo comparado.

Valor Retornado

Resultado da comparação, inteiro positivo se `value1` for menor que `value2`, 0 se forem iguais, inteiro negativo caso contrário.

> **Nota** :
>
> Ter múltiplos elementos com o mesmo valor em uma heap não é recomendado. Eles acabarão em uma posição relativa arbitrária.

SplPriorityQueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplPriorityQueue fornece as principais funcionalidades de uma fila priorizada, implementada usando um heap máximo.

> **Nota** :
>
> A ordem dos elementos com prioridade idêntica é *indefinida*. Pode diferir da ordem em que foram inseridos.

Resumo da classe

class **SplPriorityQueue** implements Iterator, Countable {
/* Constantes */
public const int *EXTR_BOTH*;
public const int *EXTR_PRIORITY*;
public const int *EXTR_DATA*;
/* Métodos */
public function compare(mixed `$priority1`, mixed `$priority2`): int
public function count(): int
public function current(): mixed
public function extract(): mixed
public function getExtractFlags(): int
public function insert(mixed `$value`, mixed `$priority`): true
public function isCorrupted(): bool
public function isEmpty(): bool
public function key(): int
public function next(): void
public function recoverFromCorruption(): true
public function rewind(): void
public function setExtractFlags(int `$flags`): int
public function top(): mixed
public function valid(): bool
}

Constantes predefinidas

`SplPriorityQueue::EXTR_BOTH`

`SplPriorityQueue::EXTR_PRIORITY`

`SplPriorityQueue::EXTR_DATA`

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

SplPriorityQueue::compare

SplPriorityQueue::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::compare — Comparar prioridades para colocar elementos corretamente no heap ao subir

Descrição

public function **SplPriorityQueue::compare**(mixed `$priority1`, mixed `$priority2`): int

Compara `priority1` com `priority2`.

Parâmetros

`priority1`

A prioridade do primeiro nó sendo comparado.

`priority2`

A prioridade do segundo nó sendo comparado.

Valor Retornado

Resultado da comparação, inteiro positivo se `priority1` for maior que `priority2`, 0 se forem iguais, inteiro negativo caso contrário.

> **Nota** :
>
> Múltiplos elementos com a mesma prioridade serão desenfileirados em nenhuma ordem específica.

SplPriorityQueue::count

SplPriorityQueue::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::count — Conta o número de elementos na fila

Descrição

public function **SplPriorityQueue::count**(): int

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de elementos na fila.

SplPriorityQueue::current

SplPriorityQueue::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::current — Retorna o nó atual apontado pelo iterador

Descrição

public function **SplPriorityQueue::current**(): mixed

Obtém o nó atual da estrutura de dados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor ou a prioridade (ou ambos) do nó atual, dependendo da opção de extração.

SplPriorityQueue::extract

SplPriorityQueue::extract

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::extract — Extrai um nó do topo do heap e desloca para cima

Descrição

public function **SplPriorityQueue::extract**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor ou a prioridade (ou ambos) do nó extraído, dependendo da opção de extração.

SplPriorityQueue::getExtractFlags

SplPriorityQueue::getExtractFlags

(PHP 7, PHP 8)

SplPriorityQueue::getExtractFlags — Obtém as opções de extração

Descrição

public function **SplPriorityQueue::getExtractFlags**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna as opções de extração.

SplPriorityQueue::insert

SplPriorityQueue::insert

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::insert — Insere um elemento na fila deslocando-o para cima

Descrição

public function **SplPriorityQueue::insert**(mixed `$value`, mixed `$priority`): true

Insere o `value` com a prioridade `priority` na fila.

Parâmetros

`value`

O valor a ser inserido.

`priority`

A prioridade associada.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.4.0	**SplPriorityQueue::insert()** agora tem um tipo de retorno provisório de true.

SplPriorityQueue::isCorrupted

SplPriorityQueue::isCorrupted

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::isCorrupted — Indica se a fila de prioridade está em um estado corrompido

Descrição

public function **SplPriorityQueue::isCorrupted**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a fila de prioridade estiver corrompida, `false` caso contrário.

SplPriorityQueue::isEmpty

SplPriorityQueue::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::isEmpty — Verifica se a fila está vazia

Descrição

public function **SplPriorityQueue::isEmpty**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna se a fila está vazia.

SplPriorityQueue::key

SplPriorityQueue::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::key — Retorna o índice do nó atual

Descrição

public function **SplPriorityQueue::key**(): int

Esta função retorna o índice do nó atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O índice do nó atual.

SplPriorityQueue::next

SplPriorityQueue::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::next — Move para o próximo nó

Descrição

public function **SplPriorityQueue::next**(): void

Extrai o nó superior da fila.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplPriorityQueue::recoverFromCorruption

SplPriorityQueue::recoverFromCorruption

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::recoverFromCorruption — Recupera do estado corrompido e permite outras ações na fila

Descrição

public function **SplPriorityQueue::recoverFromCorruption**(): true

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.4.0	**SplPriorityQueue::recoverFromCorruption()** agora tem um tipo de retorno provisório de true.

SplPriorityQueue::rewind

SplPriorityQueue::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::rewind — Rebobina o iterador de volta ao início (sem operação)

Descrição

public function **SplPriorityQueue::rewind**(): void

Isso rebobina o iterador para o início. Isso não tem efeito para heaps pois o iterador é virtual e na verdade nunca se move do topo do heap.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplPriorityQueue::setExtractFlags

SplPriorityQueue::setExtractFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::setExtractFlags — Define o modo de extração

Descrição

public function **SplPriorityQueue::setExtractFlags**(int `$flags`): int

Parâmetros

`flags`

Define o que é extraído por SplPriorityQueue::current(), SplPriorityQueue::top() e SplPriorityQueue::extract().

- `SplPriorityQueue::EXTR_DATA` (0x00000001): Extrai os dados
- `SplPriorityQueue::EXTR_PRIORITY` (0x00000002): Extrai a prioridade
- `SplPriorityQueue::EXTR_BOTH` (0x00000003): Extrai um array contendo ambos

O modo padrão é `SplPriorityQueue::EXTR_DATA`.

Valor Retornado

Retorna as opções de extração.

SplPriorityQueue::top

SplPriorityQueue::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::top — Observa o nó do topo da fila

Descrição

public function **SplPriorityQueue::top**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor ou prioridade (ou ambos) do nó do topo, dependendo da opção de extração.

SplPriorityQueue::valid

SplPriorityQueue::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::valid — Verifica se a fila contém mais nós

Descrição

public function **SplPriorityQueue::valid**(): bool

Verifica se a fila contém mais nós.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a fila contém mais nós, `false` caso contrário.

SplFixedArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

A classe SplFixedArray fornece as principais funcionalidades de um array. A principal diferença entre um SplFixedArray e um array PHP normal é que o SplFixedArray deve ser redimensionado manualmente e permite apenas inteiros dentro do intervalo como índices. A vantagem é que ele usa menos memória do que um array padrão.

Resumo da classe

class **SplFixedArray** implements IteratorAggregate, ArrayAccess, Countable, JsonSerializable {
/* Métodos */
public function __construct(int `$size` = 0)
public function count(): int
public function current(): mixed
public static function fromArray(array `$array`, bool `$preserveKeys` = `true`): SplFixedArray
public function getIterator(): Iterator
public function getSize(): int
public function jsonSerialize(): array
public function key(): int

public function next(): void
public function offsetExists(int $index): bool
public function offsetGet(int $index): mixed
public function offsetSet(int $index, mixed $value): void
public function offsetUnset(int $index): void
public function rewind(): void
public function __serialize(): array
public function setSize(int $size): true
public function toArray(): array
public function __unserialize(array $data): void
public function valid(): bool
#[\Deprecated]
public function __wakeup(): void
}

Registro de Alterações

Versão	Descrição
8.4.0	Acessos fora dos limites em **SplFixedArray** agora lançam exceções do tipo OutOfBoundsException em vez de RuntimeException. Como OutOfBoundsException é uma classe filha de RuntimeException, nenhuma mudança de comportamento é exibida ao tentar capturar essas exceções.
8.2.0	Os métodos mágicos SplFixedArray::__serialize() e SplFixedArray::__unserialize() foram adicionados à classe **SplFixedArray**.
8.1.0	**SplFixedArray** implementa JsonSerializable agora.
8.0.0	**SplFixedArray** implementa IteratorAggregate agora. Anteriormente, implementava Iterator.

Examples

Exemplo #1 Exemplo de uso de SplFixedArray

```php
<?php
// Inicialize o array com um comprimento fixo
$array = new SplFixedArray(5);

$array[1] = 2;
$array[4] = "foo";

var_dump($array[0]); // NULL
var_dump($array[1]); // int(2)

var_dump($array["4"]); // string(3) "foo"

// Aumenta o tamanho do array para 10
$array->setSize(10);

$array[9] = "asdf";

// Reduz o array para um tamanho de 2
$array->setSize(2);

// As seguintes linhas lançam uma RuntimeException: Índice inválido ou fora do intervalo
try {
    var_dump($array["non-numeric"]);
} catch(RuntimeException $re) {
    echo "RuntimeException: ".$re->getMessage()."\n";
}

try {
    var_dump($array[-1]);
} catch(RuntimeException $re) {
    echo "RuntimeException: ".$re->getMessage()."\n";
}

try {
    var_dump($array[5]);
} catch(RuntimeException $re) {
    echo "RuntimeException: ".$re->getMessage()."\n";
```

```
}
?>
```

O exemplo acima produzirá:

```
NULL
int(2)
string(3) "foo"
RuntimeException: Index invalid or out of range
RuntimeException: Index invalid or out of range
RuntimeException: Index invalid or out of range
```

Índice

SplFixedArray::__construct

SplFixedArray::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::__construct — Constrói um novo array fixo

Descrição

public function **SplFixedArray::__construct**(int `$size` = 0)

Inicializa um array fixo com um número de valores `null` igual ao `size`.

Parâmetros

`size`

O tamanho do array fixo. Isso espera um número entre `0` e `PHP_INT_MAX`.

Erros/Exceções

Lança um ValueError quando `size` é um número inteiro negativo.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança um ValueError se `size` é negativo; anteriormente lançava um InvalidArgumentException.

Examples

Exemplo #1 Exemplo de SplFixedArray::__construct()

```php
<?php
$array = new SplFixedArray(5);

$array[1] = 2;
$array[4] = "foo";

foreach($array as $v) {
  var_dump($v);
}
?>
```

O exemplo acima produzirá:

```
NULL
int(2)
NULL
```

```
NULL
string(3) "foo"
```

SplFixedArray::count

SplFixedArray::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::count — Retorna o tamanho do array

Descrição

public function **SplFixedArray::count**(): int

Retorna o tamanho do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tamanho do array.

Examples

Exemplo #1 Exemplo de SplFixedArray::count()

```php
<?php
$array = new SplFixedArray(5);
echo $array->count() . "\n";
echo count($array) . "\n";
?>
```

O exemplo acima produzirá:

```
5
5
```

Notas

> **Nota** :
>
> Este método é funcionalmente equivalente a SplFixedArray::getSize().

> **Nota** :
>
> A contagem de elementos é sempre igual ao tamanho definido porque todos os valores são inicialmente inicializados com `null`.

Veja também

- SplFixedArray::getSize() - Obtém o tamanho do array

SplFixedArray::current

SplFixedArray::current

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::current — Retorna a entrada atual do array

Descrição

public function **SplFixedArray::current**(): mixed

Obtém o elemento atual do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do elemento atual.

Erros/Exceções

Lança RuntimeException quando o ponteiro interno do array aponta para um índice inválido ou está fora dos limites.

SplFixedArray::fromArray

SplFixedArray::fromArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::fromArray — Importa um array PHP em uma instância SplFixedArray

Descrição

public static function **SplFixedArray::fromArray**(array `$array`, bool `$preserveKeys = true`): SplFixedArray

Importa o array PHP `array` em uma nova instância de SplFixedArray

Parâmetros

`array`

O array a ser importado.

`preserveKeys`

Tenta preservar os índices numéricos usados no array original.

Valor Retornado

Retorna uma instância de SplFixedArray contendo o conteúdo do array.

Examples

Exemplo #1 Exemplo de SplFixedArray::fromArray()

```php
<?php
$fa = SplFixedArray::fromArray(array(1 => 1, 0 => 2, 3 => 3));

var_dump($fa);

$fa = SplFixedArray::fromArray(array(1 => 1, 0 => 2, 3 => 3), false);

var_dump($fa);
?>
```

O exemplo acima produzirá:

```
object(SplFixedArray)#1 (4) {
  [0]=>
  int(2)
  [1]=>
```

```
  int(1)
  [2]=>
  NULL
  [3]=>
  int(3)
}
object(SplFixedArray)#2 (3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

SplFixedArray::getIterator

SplFixedArray::getIterator

(PHP 8)

SplFixedArray::getIterator — Recupera o iterador para percorrer o array

Descrição

public function **SplFixedArray::getIterator**(): Iterator

Retorna o iterador para percorrer o array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma instância de um objeto que implementa Iterator para percorrer o array.

SplFixedArray::getSize

SplFixedArray::getSize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::getSize — Obtém o tamanho do array

Descrição

public function **SplFixedArray::getSize**(): int

Obtém o tamanho do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tamanho do array, como um int.

Examples

Exemplo #1 Exemplo de SplFixedArray::getSize()

```php
<?php
$array = new SplFixedArray(5);
echo $array->getSize()."\n";
$array->setSize(10);
```

```php
echo $array->getSize()."\n";
?>
```

O exemplo acima produzirá:

```
5
10
```

Notas

> **Nota** :
>
> Este método é funcionalmente equivalente a SplFixedArray::count()

Veja também

- SplFixedArray::count() - Retorna o tamanho do array

SplFixedArray::jsonSerialize

SplFixedArray::jsonSerialize

(PHP 8 >= 8.1.0)

SplFixedArray::jsonSerialize — Retorna uma representação que pode ser convertida em JSON

Descrição

public function **SplFixedArray::jsonSerialize**(): array

Serializa o array para um valor que pode ser serializado nativamente pelo json_encode().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna os dados do array que podem ser serializados pelo json_encode(), que é um valor de qualquer tipo que não seja um resource.

SplFixedArray::key

SplFixedArray::key

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::key — Retorna o índice atual do array

Descrição

public function **SplFixedArray::key**(): int

Retorna o índice atual do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O índice atual do array.

SplFixedArray::next

SplFixedArray::next

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::next — Move para a próxima entrada

Descrição

public function **SplFixedArray::next**(): void

Move o iterador para a próxima entrada do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplFixedArray::offsetExists

SplFixedArray::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetExists — Retorna se o índice solicitado existe

Descrição

public function **SplFixedArray::offsetExists**(int `$index`): bool

Verifica se o índice solicitado `index` existe.

Parâmetros

`index`

O índice sendo verificado.

Valor Retornado

`true` se o `index` solicitado existir, caso contrário `false`

SplFixedArray::offsetGet

SplFixedArray::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetGet — Retorna o valor no índice especificado

Descrição

public function **SplFixedArray::offsetGet**(int `$index`): mixed

Retorna o valor no índice `index`.

Parâmetros

`index`

O índice com o valor.

Valor Retornado

O valor no `index` especificado.

Erros/Exceções

Lança RuntimeException quando o `index` está fora do tamanho definido do array ou quando o `index` não pode ser analisado como um número inteiro.

SplFixedArray::offsetSet

SplFixedArray::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetSet — Define um novo valor em um índice especificado

Descrição

public function **SplFixedArray::offsetSet**(int `$index`, mixed `$value`): void

Define o valor no `index` especificado como `value`.

Parâmetros

`index`

O índice a ser definido.

`value`

O novo valor para o `index`.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança RuntimeException quando o `index` está fora do tamanho definido do array ou quando o `index` não pode ser analisado como um número inteiro.

SplFixedArray::offsetUnset

SplFixedArray::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetUnset — Remove o valor no $index especificado

Descrição

public function **SplFixedArray::offsetUnset**(int `$index`): void

Remove o valor no índice especificado.

Parâmetros

`index`

O índice a ser removido.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança RuntimeException quando o `index` está fora do tamanho definido do array ou quando o `index` não pode ser analisado como um número inteiro.

SplFixedArray::rewind

SplFixedArray::rewind

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::rewind — Rebobina o iterador de volta ao início

Descrição

public function **SplFixedArray::rewind**(): void

Rebobina o iterador para o início.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplFixedArray::__serialize

SplFixedArray::__serialize

(PHP 8 >= 8.2.0)

SplFixedArray::__serialize — Serializa o objeto SplFixedArray

Descrição

public function **SplFixedArray::__serialize**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

SplFixedArray::setSize

SplFixedArray::setSize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::setSize — Altera o tamanho de um array

Descrição

public function **SplFixedArray::setSize**(int `$size`): true

Altera o tamanho de um array para o novo tamanho especificado por `size`. Se `size` for menor que o tamanho atual do array, quaisquer valores após o novo tamanho serão descartados. Se `size` for maior que o tamanho atual do array, o array será preenchido com valores `null`.

Parâmetros

`size`

O novo tamanho do array. Isso deve ser um valor entre `0` e `PHP_INT_MAX`.

Valor Retornado

Sempre retorna `true`.

Erros/Exceções

Lança ValueError quando `size` for menor que zero.

Registro de Alterações

Versão	Descrição
8.4.0	**SplFixedArray::setSize()** agora tem um tipo de retorno provisório de true.

Examples

Exemplo #1 Exemplo de SplFixedArray::setSize()

```php
<?php
   $array = new SplFixedArray(5);
   echo $array->getSize()."\n";
   $array->setSize(10);
   echo $array->getSize()."\n";
?>
```

O exemplo acima produzirá:

```
5
10
```

SplFixedArray::toArray

SplFixedArray::toArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::toArray — Retorna um array PHP do array fixo

Descrição

public function **SplFixedArray::toArray**(): array

Retorna um array PHP do array fixo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array PHP, semelhante ao array fixo.

Examples

Exemplo #1 Exemplo de SplFixedArray::toArray()

```php
<?php
$fa = new SplFixedArray(3);
$fa[0] = 0;
$fa[2] = 2;
var_dump($fa->toArray());
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  int(0)
  [1]=>
  NULL
  [2]=>
  int(2)
}
```

SplFixedArray::__unserialize

SplFixedArray::__unserialize

(PHP 8 >= 8.2.0)

SplFixedArray::__unserialize — Desserializa o parâmetro `data` em um objeto SplFixedArray

Descrição

public function **SplFixedArray::__unserialize**(array `$data`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

O valor sendo deserializado.

Valor Retornado

Nenhum valor é retornado.

SplFixedArray::valid

SplFixedArray::valid

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::valid — Verifica se o array contém mais elementos

Descrição

public function **SplFixedArray::valid**(): bool

Verifica se o array contém mais elementos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna **`true`** se o array contém mais elementos, **`false`** caso contrário.

SplFixedArray::__wakeup

SplFixedArray::__wakeup

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

SplFixedArray::__wakeup — Reinicializa o array após ser desserializado

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.4.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **SplFixedArray::__wakeup**(): void

Reinicializa o array após ser desserializado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.4.0	Este método foi descontinuado, use SplFixedArray::__unserialize() em seu lugar.

ArrayObject

(PHP 5, PHP 7, PHP 8)

Introdução

Esta classe permite que objetos funcionem como arrays.

> **Nota** :
>
> Empacotar objetos com esta classe tem uma falha fundamental e portanto seu uso com objetos é desencorajado.

Resumo da classe

```
class ArrayObject implements IteratorAggregate, ArrayAccess, Serializable, Countable {
/* Constantes */
public const int STD_PROP_LIST;
public const int ARRAY_AS_PROPS;
/* Métodos */
public function __construct(array|object $array = [], int $flags = 0, string $iteratorClass = ArrayIterator::class)
public function append(mixed $value): void
public function asort(int $flags = SORT_REGULAR): true
public function count(): int
public function exchangeArray(array|object $array): array
public function getArrayCopy(): array
public function getFlags(): int
public function getIterator(): Iterator
public function getIteratorClass(): string
public function ksort(int $flags = SORT_REGULAR): true
public function natcasesort(): true
public function natsort(): true
public function offsetExists(mixed $key): bool
public function offsetGet(mixed $key): mixed
public function offsetSet(mixed $key, mixed $value): void
public function offsetUnset(mixed $key): void
public function serialize(): string
public function setFlags(int $flags): void
public function setIteratorClass(string $iteratorClass): void
public function uasort(callable $callback): true
public function uksort(callable $callback): true
public function unserialize(string $data): void
}
```

Constantes predefinidas

Flags do ArrayObject

`ArrayObject::STD_PROP_LIST`

As propriedades do objeto têm sua funcionalidade normal quando acessadas como lista (var_dump(), `foreach`, etc.).

`ArrayObject::ARRAY_AS_PROPS`

As entradas podem ser acessadas como propriedades (leitura e escrita). A classe **ArrayObject** usa sua própria lógica para acessar propriedades, portanto, nenhum aviso ou erro é gerado ao tentar ler ou escrever propriedades dinâmicas.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

ArrayObject::append

ArrayObject::append

(PHP 5, PHP 7, PHP 8)

ArrayObject::append — Anexa o valor

Descrição

public function **ArrayObject::append**(mixed `$value`): void

Anexa um novo valor como o último elemento.

> **Nota** :
>
> Este método não pode ser chamado quando o ArrayObject foi construído a partir de um objeto. Use ArrayObject::offsetSet() em vez disso.

Parâmetros

`value`

O valor sendo anexado.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayObject::append()

```
<?php
$arrayobj = new ArrayObject(array('first','second','third'));
$arrayobj->append('fourth');
$arrayobj->append(array('five', 'six'));
var_dump($arrayobj);
?>
```

O exemplo acima produzirá:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(5) {
    [0]=>
    string(5) "first"
    [1]=>
    string(6) "second"
    [2]=>
    string(5) "third"
    [3]=>
    string(6) "fourth"
    [4]=>
    array(2) {
      [0]=>
      string(4) "five"
      [1]=>
      string(3) "six"
```

```
    }
  }
}
```

Veja também

- ArrayObject::offsetSet() - Define o valor no índice especificado para novo valor

ArrayObject::asort

ArrayObject::asort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::asort — Ordenar as entradas por valor

Descrição

public function **ArrayObject::asort**(int `$flags` = `SORT_REGULAR`): true

Ordena as entradas em ordem crescente, de forma que suas chaves mantenham sua correlação com os valores que estão associados a elas.

Isso é usado principalmente ao ordenar arrays associativos onde a ordem real dos elementos é significativa.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()
- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::asort()

```php
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
$fruitArrayObject = new ArrayObject($fruits);
$fruitArrayObject->asort();

foreach ($fruitArrayObject as $key => $val) {
    echo "$key = $val\n";
}
?>
```

O exemplo acima produzirá:

```
c = apple
b = banana
d = lemon
a = orange
```

As frutas foram ordenadas em ordem alfabética, e a chave associada a cada entrada foi mantida.

Veja também

- ArrayObject::ksort() - Ordena as entradas pela chave
- ArrayObject::natsort() - Ordena as entradas usando um algoritmo de "ordem natural"
- ArrayObject::natcasesort() - Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas
- ArrayObject::uasort() - Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves
- ArrayObject::uksort() - Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário
- asort() - Ordena um array em ordem crescente e mantém a associação de índices

ArrayObject::__construct

ArrayObject::__construct

(PHP 5, PHP 7, PHP 8)

ArrayObject::__construct — Constrói um novo objeto de array

Descrição

public function **ArrayObject::__construct**(array|object `$array` = [], int `$flags` = 0, string `$iteratorClass` = **`ArrayIterator::class`**)

Constrói um novo objeto de array (do tipo object).

Parâmetros

`array`

O parâmetro `array` aceita um array ou um Object.

`flags`

Opções para controlar o comportamento do objeto ArrayObject. Consulte ArrayObject::setFlags().

`iteratorClass`

Especifica a classe que será usada para iteração do objeto ArrayObject. A classe deve ser um subtipo da classe ArrayIterator.

Examples

Exemplo #1 Exemplo de ArrayObject::__construct()

```php
<?php

$array = [
    '1' => 'um',
```

```php
    '2' => 'dois',
    '3' => 'três'
];

$arrayobject = new ArrayObject($array);

var_dump($arrayobject);

?>
```

O exemplo acima produzirá:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [1]=>
    string(3) "um"
    [2]=>
    string(3) "dois"
    [3]=>
    string(5) "três"
  }
}
```

Veja também

- ArrayObject::setflags() - Define as opções de comportamento

ArrayObject::count

ArrayObject::count

(PHP 5, PHP 7, PHP 8)

ArrayObject::count — Obtém o número de propriedades públicas no ArrayObject

Descrição

public function **ArrayObject::count**(): int

Obtém o número de propriedades públicas no ArrayObject.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de propriedades públicas no ArrayObject.

> **Nota** :
>
> Quando o ArrayObject é construído a partir de um array, todas as propriedades são públicas.

Examples

Exemplo #1 Exemplo de ArrayObject::count()

```php
<?php
class Example {
    public $public = 'prop:public';
    private $prv   = 'prop:private';
    protected $prt = 'prop:protected';
```

```
}

$arrayobj = new ArrayObject(new Example());
var_dump($arrayobj->count());

$arrayobj = new ArrayObject(array('first','second','third'));
var_dump($arrayobj->count());
?>
```

O exemplo acima produzirá:

```
int(1)
int(3)
```

ArrayObject::exchangeArray

ArrayObject::exchangeArray

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::exchangeArray — Troca o array por outro

Descrição

public function **ArrayObject::exchangeArray**(array|object `$array`): array

Troca o array atual por outro array ou object.

Parâmetros

`array`

O novo array ou object para trocar pelo array atual.

Valor Retornado

Retorna o array antigo.

Examples

Exemplo #1 Exemplo de ArrayObject::exchangeArray()

```
<?php
// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);
// Array de locais na Europa
$locations = array('Amsterdam', 'Paris', 'London');

$fruitsArrayObject = new ArrayObject($fruits);

// Agora troque as frutas por locais
$old = $fruitsArrayObject->exchangeArray($locations);
var_dump($old);
var_dump($fruitsArrayObject);

?>
```

O exemplo acima produzirá:

```
array(4) {
  ["lemons"]=>
  int(1)
  ["oranges"]=>
```

```
  int(4)
  ["bananas"]=>
  int(5)
  ["apples"]=>
  int(10)
}
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [0]=>
    string(9) "Amsterdam"
    [1]=>
    string(5) "Paris"
    [2]=>
    string(6) "London"
  }
}
```

ArrayObject::getArrayCopy

ArrayObject::getArrayCopy

(PHP 5, PHP 7, PHP 8)

ArrayObject::getArrayCopy — Cria uma cópia do ArrayObject

Descrição

public function **ArrayObject::getArrayCopy**(): array

Exporta o ArrayObject para um array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma cópia do array. Quando o ArrayObject se refere a um objeto, um array com as propriedades desse objeto será retornado.

Examples

Exemplo #1 Exemplo de **ArrayObject::getArrayCopy()**

```php
<?php
// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);
$fruitsArrayObject['pears'] = 4;

// cria uma cópia do array
$copy = $fruitsArrayObject->getArrayCopy();
var_dump($copy);

?>
```

O exemplo acima produzirá:

```
array(5) {
  ["lemons"]=>
  int(1)
  ["oranges"]=>
  int(4)
  ["bananas"]=>
```

```
  int(5)
  ["apples"]=>
  int(10)
  ["pears"]=>
  int(4)
}
```

ArrayObject::getFlags

ArrayObject::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::getFlags — Obtém as flags de comportamento

Descrição

public function **ArrayObject::getFlags**(): int

Obtém as flags de comportamento do ArrayObject. Consulte o método ArrayObject::setFlags para obter uma lista das flags disponíveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna as flags de comportamento do ArrayObject.

Examples

Exemplo #1 Exemplo de **ArrayObject::getFlags()**

```php
<?php
// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Obtém as flags atuais
$flags = $fruitsArrayObject->getFlags();
var_dump($flags);

// Define novas flags
$fruitsArrayObject->setFlags(ArrayObject::ARRAY_AS_PROPS);

// Obtém as novas flags
$flags = $fruitsArrayObject->getFlags();
var_dump($flags);
?>
```

O exemplo acima produzirá:

```
int(0)
int(2)
```

Veja também

- ArrayObject::setFlags() - Define as opções de comportamento

ArrayObject::getIterator

ArrayObject::getIterator

(PHP 5, PHP 7, PHP 8)

ArrayObject::getIterator — Retorna o iterador do ArrayObject

Descrição

public function **ArrayObject::getIterator**(): Iterator

Cria um novo Iterator (o padrão é ArrayIterator) a partir de uma instância de ArrayObject.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um iterador de um ArrayObject.

Examples

Exemplo #1 Exemplo de ArrayObject::getIterator()

```php
<?php

$array = [
    '1' => 'um',
    '2' => 'dois',
    '3' => 'três',
];

$arrayobject = new ArrayObject($array);

$iterator = $arrayobject->getIterator();

while ($iterator->valid()) {
    echo $iterator->key() . ' => ' . $iterator->current() . "\n";

    $iterator->next();
}

?>
```

O exemplo acima produzirá:

```
1 => um
2 => dois
3 => três
```

ArrayObject::getIteratorClass

ArrayObject::getIteratorClass

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::getIteratorClass — Obtém o nome da classe do iterador para o ArrayObject

Descrição

public function **ArrayObject::getIteratorClass**(): string

Obtém o nome da classe do iterador de array que é usado por ArrayObject::getIterator().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome da classe do iterador que é usado para iterar sobre este objeto.

Examples

Exemplo #1 Exemplo de ArrayObject::getIteratorClass()

```php
<?php
// ArrayIterator Personalizado  (herda de ArrayIterator)
class MyArrayIterator extends ArrayIterator {
    // implementação personalizada
}

// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Obtém o nome da classe atual
$className = $fruitsArrayObject->getIteratorClass();
var_dump($className);

// Define novo nome de classe
$fruitsArrayObject->setIteratorClass('MyArrayIterator');

// Obtém o novo nome de classe do iterador
$className = $fruitsArrayObject->getIteratorClass();
var_dump($className);
?>
```

O exemplo acima produzirá:

```
string(13) "ArrayIterator"
string(15) "MyArrayIterator"
```

Veja também

- O método ArrayObject::setIteratorClass

ArrayObject::ksort

ArrayObject::ksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::ksort — Ordena as entradas pela chave

Descrição

public function **ArrayObject::ksort**(int `$flags` = `SORT_REGULAR`): true

Ordena as entradas pela chave, mantendo as correlações entre chave e entrada. Isso é útil principalmente para arrays associativos.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- **`SORT_REGULAR`** - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- **`SORT_NUMERIC`** - compara os elementos numericamente
- **`SORT_STRING`** - compara os elementos como strings
- **`SORT_LOCALE_STRING`** - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- **`SORT_NATURAL`** - compara os elementos como strings usando "ordenação natural" como natsort()
- **`SORT_FLAG_CASE`** - pode ser combinada (usando o operador bit a bit OR) com **`SORT_STRING`** ou **`SORT_NATURAL`** para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **`true`**; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::ksort()

```php
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
$fruitArrayObject = new ArrayObject($fruits);
$fruitArrayObject->ksort();

foreach ($fruitArrayObject as $key => $val) {
    echo "$key = $val\n";
}
 ?>
```

O exemplo acima produzirá:

```
a = orange
b = banana
c = apple
d = lemon
```

Veja também

- ArrayObject::asort() - Ordenar as entradas por valor
- ArrayObject::natsort() - Ordena as entradas usando um algoritmo de "ordem natural"
- ArrayObject::natcasesort() - Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas
- ArrayObject::uasort() - Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves
- ArrayObject::uksort() - Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário
- ksort() - Ordena um array pelas chaves em ordem crescente

ArrayObject::natcasesort

ArrayObject::natcasesort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::natcasesort — Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas

Descrição

public function **ArrayObject::natcasesort**(): true

Este método é uma versão insensível a maiúsculas e minúsculas de ArrayObject::natsort.

Este método implementa um algoritmo de classificação que ordena strings alfanuméricas da maneira como um ser humano faria, mantendo associações chave/valor. Isso é descrito como uma "ordem natural".

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::natcasesort()

```php
<?php
$array = array('IMG0.png', 'img12.png', 'img10.png', 'img2.png', 'img1.png', 'IMG3.png');

$arr1 = new ArrayObject($array);
$arr2 = clone $arr1;

$arr1->asort();
echo "Ordenação padrão\n";
var_dump($arr1);

$arr2->natcasesort();
echo "\nOrdenação natural (insensível a maiúsculas/minúsculas)\n";
var_dump($arr2);
?>
```

O exemplo acima produzirá:

```
Ordenação padrão
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(6) {
    [0]=>
    string(8) "IMG0.png"
    [5]=>
    string(8) "IMG3.png"
    [4]=>
    string(8) "img1.png"
    [2]=>
    string(9) "img10.png"
    [1]=>
    string(9) "img12.png"
```

```
    [3]=>
    string(8) "img2.png"
  }
}

Ordenação natural (insensível a maiúsculas/minúsculas)
object(ArrayObject)#2 (1) {
  ["storage":"ArrayObject":private]=>
  array(6) {
    [0]=>
    string(8) "IMG0.png"
    [4]=>
    string(8) "img1.png"
    [3]=>
    string(8) "img2.png"
    [5]=>
    string(8) "IMG3.png"
    [2]=>
    string(9) "img10.png"
    [1]=>
    string(9) "img12.png"
  }
}
```

Para obter mais informações, consulte a página de Comparação de Cadeias de Caracteres em Ordem Natural de Martin Pool: » Natural Order String Comparison.

Veja também

- ArrayObject::asort() - Ordenar as entradas por valor
- ArrayObject::ksort() - Ordena as entradas pela chave
- ArrayObject::natsort() - Ordena as entradas usando um algoritmo de "ordem natural"
- ArrayObject::uasort() - Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves
- ArrayObject::uksort() - Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário
- natcasesort() - Ordena um array utilizando o algoritmo de "ordem natural" sem diferenciar maiúsculas e minúsculas

ArrayObject::natsort

ArrayObject::natsort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::natsort — Ordena as entradas usando um algoritmo de "ordem natural"

Descrição

public function **ArrayObject::natsort**(): true

Este método implementa um algoritmo de ordenação que ordena strings alfanuméricas da maneira como um ser humano faria, mantendo associações chave/valor. Isso é descrito como uma "ordem natural". Um exemplo da diferença entre esse algoritmo e os algoritmos regulares de ordenação de strings de computador (usados no método ArrayObject::asort) pode ser visto no exemplo abaixo.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::natsort()

```php
<?php
$array = array("img12.png", "img10.png", "img2.png", "img1.png");

$arr1 = new ArrayObject($array);
$arr2 = clone $arr1;

$arr1->asort();
echo "Ordenação padrão\n";
var_dump($arr1);

$arr2->natsort();
echo "\nOrdenação natural\n";
var_dump($arr2);
?>
```

O exemplo acima produzirá:

```
Ordenação padrão
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    [3]=>
    string(8) "img1.png"
    [1]=>
    string(9) "img10.png"
    [0]=>
    string(9) "img12.png"
    [2]=>
    string(8) "img2.png"
  }
}

Ordenação natural
object(ArrayObject)#2 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    [3]=>
    string(8) "img1.png"
    [2]=>
    string(8) "img2.png"
    [1]=>
    string(9) "img10.png"
    [0]=>
    string(9) "img12.png"
  }
}
```

Para mais informações, consulte a página de » Comparação de Strings em Ordem Natural de Martin Pool.

Veja também

- ArrayObject::asort() - Ordenar as entradas por valor
- ArrayObject::ksort() - Ordena as entradas pela chave
- ArrayObject::natcasesort() - Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas
- ArrayObject::uasort() - Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves

- ArrayObject::uksort() - Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário
- natsort() - Ordena um array utilizando o algoritmo de "ordem natural"

ArrayObject::offsetExists

ArrayObject::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetExists — Retorna se o índice solicitado existe

Descrição

public function **ArrayObject::offsetExists**(mixed $key): bool

Parâmetros

key

O índice sendo verificado.

Valor Retornado

`true` se o índice solicitado existe, caso contrário `false`

Examples

Exemplo #1 Exemplo de ArrayObject::offsetExists()

```php
<?php
$arrayobj = new ArrayObject(array('zero', 'one', 'example'=>'e.g.'));
var_dump($arrayobj->offsetExists(1));
var_dump($arrayobj->offsetExists('example'));
var_dump($arrayobj->offsetExists('notfound'));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
bool(false)
```

ArrayObject::offsetGet

ArrayObject::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetGet — Retorna o valor no índice especificado

Descrição

public function **ArrayObject::offsetGet**(mixed $key): mixed

Parâmetros

key

O índice com o valor.

Valor Retornado

O valor no índice especificado ou `null`.

Erros/Exceções

Produz uma mensagem de erro `E_NOTICE` quando o índice especificado não existe.

Examples

Exemplo #1 Exemplo de ArrayObject::offsetGet()

```
<?php
$arrayobj = new ArrayObject(array('zero', 7, 'example'=>'e.g.'));
var_dump($arrayobj->offsetGet(1));
var_dump($arrayobj->offsetGet('example'));
var_dump($arrayobj->offsetExists('notfound'));
?>
```

O exemplo acima produzirá:

```
int(7)
string(4) "e.g."
bool(false)
```

ArrayObject::offsetSet

ArrayObject::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetSet — Define o valor no índice especificado para novo valor

Descrição

public function **ArrayObject::offsetSet**(mixed `$key`, mixed `$value`): void

Define o valor no índice especificado para novo valor.

Parâmetros

`key`

O índice sendo definido.

`value`

O novo valor para o `key`.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayObject::offsetSet()

```
<?php
class Example {
    public $property = 'prop:public';
}
$arrayobj = new ArrayObject(new Example());
$arrayobj->offsetSet(4, 'four');
$arrayobj->offsetSet('group', array('g1', 'g2'));
var_dump($arrayobj);

$arrayobj = new ArrayObject(array('zero','one'));
$arrayobj->offsetSet(null, 'last');
var_dump($arrayobj);
?>
```

O exemplo acima produzirá:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  object(Example)#2 (3) {
    ["property"]=>
    string(11) "prop:public"
    ["4"]=>
    string(4) "four"
    ["group"]=>
    array(2) {
      [0]=>
      string(2) "g1"
      [1]=>
      string(2) "g2"
    }
  }
}
object(ArrayObject)#3 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [0]=>
    string(4) "zero"
    [1]=>
    string(3) "one"
    [2]=>
    string(4) "last"
  }
}
```

Veja também

- ArrayObject::append() - Anexa o valor

ArrayObject::offsetUnset

ArrayObject::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetUnset — Remove o valor no índice especificado

Descrição

public function **ArrayObject::offsetUnset**(mixed $key): void

Remove o valor no índice especificado.

Parâmetros

key

O índice sendo removido.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayObject::offsetUnset()

```php
<?php
$arrayobj = new ArrayObject(array(0=>'zero',2=>'two'));
$arrayobj->offsetUnset(2);
```

```php
var_dump($arrayobj);
?>
```

O exemplo acima produzirá:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(1) {
    [0]=>
    string(4) "zero"
  }
}
```

ArrayObject::serialize

ArrayObject::serialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayObject::serialize — Serializa um ArrayObject

Descrição

public function **ArrayObject::serialize**(): string

Serializa um ArrayObject.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A representação serializada do ArrayObject.

Examples

Exemplo #1 Exemplo de ArrayObject::serialize()

```php
<?php
$o = new ArrayObject();

$s1 = serialize($o);
$s2 = $o->serialize();

var_dump($s1);
var_dump($s2);
?>
```

O exemplo acima produzirá:

```
string(45) "C:11:"ArrayObject":21:{x:i:0;a:0:{};m:a:0:{}}"
string(21) "x:i:0;a:0:{};m:a:0:{}"
```

Veja também

- ArrayObject::unserialize() - Desserializa um ArrayObject
- serialize() - Gera uma representação armazenável de um valor
- Serializando objetos

ArrayObject::setFlags

ArrayObject::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::setFlags — Define as opções de comportamento

Descrição

public function **ArrayObject::setFlags**(int `$flags`): void

Define as opções que alteram o comportamento do ArrayObject.

Parâmetros

`flags`

O novo comportamento do ArrayObject. Podem ser máscara de bits ou constantes nomeadas. É fortemente recomendado o uso de constantes nomeadas para garantir compatibilidade com futuras versões.

As opções de comportamento disponíveis estão listadas abaixo. Os significados reais dessas opções são descritos nas constantes predefinidas.

Opções de comportamento do ArrayObject

Valor	Constante
1	ArrayObject::STD_PROP_LIST
2	ArrayObject::ARRAY_AS_PROPS

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayObject::setFlags()

```php
<?php
// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Tenta usar chave de array como propriedade
var_dump($fruitsArrayObject->lemons);
// Define a opção para que as chaves de array possam ser usadas como propriedades do ArrayObject
$fruitsArrayObject->setFlags(ArrayObject::ARRAY_AS_PROPS);
// Tenta novamente
var_dump($fruitsArrayObject->lemons);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Warning: Undefined property: ArrayObject::$lemons in ...
NULL
int(1)
```

ArrayObject::setIteratorClass

ArrayObject::setIteratorClass

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::setIteratorClass — Define a classe do iterador para o ArrayObject

Descrição

public function **ArrayObject::setIteratorClass**(string `$iteratorClass`): void

Define o nome da classe do iterador de array que é usado por ArrayObject::getIterator().

Parâmetros

`iteratorClass`

O nome da classe do iterador de array a ser usado ao iterar sobre este objeto.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayObject::setIteratorClass()

```php
<?php
// ArrayIterator Personalizado (herda de ArrayIterator)
class MyArrayIterator extends ArrayIterator {
    // implementação personalizada
}

// Array de frutas disponíveis
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Define a classe do iterador para a nova
$fruitsArrayObject->setIteratorClass('MyArrayIterator');
var_dump($fruitsArrayObject->getIterator());

?>
```

O exemplo acima produzirá:

```
object(MyArrayIterator)#2 (1) {
  ["storage":"ArrayIterator":private]=>
  object(ArrayObject)#1 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["lemons"]=>
      int(1)
      ["oranges"]=>
      int(4)
      ["bananas"]=>
      int(5)
      ["apples"]=>
      int(10)
    }
  }
}
```

ArrayObject::uasort

ArrayObject::uasort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::uasort — Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves

Descrição

public function **ArrayObject::uasort**(callable `$callback`): true

Esta função ordena as entradas de forma que as chaves mantenham sua correlação com a entrada à qual estão associadas, usando uma função de comparação definida pelo usuário.

Isso é usado principalmente ao ordenar arrays associativos onde a ordem real dos elementos é significativa.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::uasort()

```php
<?php
// Função de comparação
function cmp($a, $b) {
    if ($a == $b) {
        return 0;
    }
    return ($a < $b) ? -1 : 1;
}

// Array a ser ordenado
$array = array('a' => 4, 'b' => 8, 'c' => -1, 'd' => -9, 'e' => 2, 'f' => 5, 'g' => 3, 'h' => -4);
$arrayObject = new ArrayObject($array);
var_dump($arrayObject);

// Ordena e imprime o array resultante
$arrayObject->uasort('cmp');
var_dump($arrayObject);
?>
```

O exemplo acima produzirá:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["a"]=>
    int(4)
    ["b"]=>
    int(8)
    ["c"]=>
    int(-1)
    ["d"]=>
    int(-9)
    ["e"]=>
    int(2)
    ["f"]=>
    int(5)
    ["g"]=>
    int(3)
    ["h"]=>
    int(-4)
  }
}
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["d"]=>
    int(-9)
    ["h"]=>
    int(-4)
    ["c"]=>
    int(-1)
    ["e"]=>
    int(2)
    ["g"]=>
    int(3)
    ["a"]=>
    int(4)
    ["f"]=>
    int(5)
    ["b"]=>
    int(8)
  }
}
```

Veja também

- ArrayObject::asort() - Ordenar as entradas por valor
- ArrayObject::ksort() - Ordena as entradas pela chave
- ArrayObject::natsort() - Ordena as entradas usando um algoritmo de "ordem natural"
- ArrayObject::natcasesort() - Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas
- ArrayObject::uksort() - Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário
- uasort() - Ordena um array com uma função de comparação definida pelo usuário e mantém associações de índices

ArrayObject::uksort

ArrayObject::uksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::uksort — Ordena as entradas pelas chaves usando uma função de comparação definida pelo usuário

Descrição

public function **ArrayObject::uksort**(callable `$callback`): true

Esta função ordena as chaves das entradas usando uma função de comparação fornecida pelo usuário. A correlação chave-entrada será mantida.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Examples

Exemplo #1 Exemplo de ArrayObject::uksort()

```php
<?php
function cmp($a, $b) {
    $a = preg_replace('@^(a|an|the) @', '', $a);
    $b = preg_replace('@^(a|an|the) @', '', $b);
    return strcasecmp($a, $b);
}

$array = array("John" => 1, "the Earth" => 2, "an apple" => 3, "a banana" => 4);
$arrayObject = new ArrayObject($array);
$arrayObject->uksort('cmp');

foreach ($arrayObject as $key => $value) {
    echo "$key: $value\n";
}
?>
```

O exemplo acima produzirá:

```
an apple: 3
a banana: 4
the Earth: 2
John: 1
```

Veja também

- ArrayObject::asort() - Ordenar as entradas por valor
- ArrayObject::ksort() - Ordena as entradas pela chave
- ArrayObject::natsort() - Ordena as entradas usando um algoritmo de "ordem natural"
- ArrayObject::natcasesort() - Ordena um array usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas
- ArrayObject::uasort() - Ordena as entradas com uma função de comparação definida pelo usuário e mantém a associação de chaves
- uksort() - Ordena um array pelas chaves utilizando uma função de comparação definida pelo usuário.

ArrayObject::unserialize

ArrayObject::unserialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayObject::unserialize — Desserializa um ArrayObject

Descrição

public function **ArrayObject::unserialize**(string `$data`): void

Desserializa um ArrayObject serializado.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

O ArrayObject serializado.

Valor Retornado

Nenhum valor é retornado.

Veja também

- ArrayIterator::serialize() - Serializa
- unserialize() - Cria um valor PHP a partir de uma representação armazenada
- Serializando Objetos

SplObjectStorage

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A classe SplObjectStorage fornece um mapa de objetos para dados ou, ao ignorar os dados, um conjunto de objetos. Esse propósito duplo pode ser útil em muitos casos que envolvem a necessidade de identificar objetos de forma única.

Resumo da classe

class **SplObjectStorage** implements Countable, SeekableIterator, Serializable, ArrayAccess {
/* Métodos */
public function addAll(SplObjectStorage `$storage`): int
#[\Deprecated]
public function attach(object `$object`, mixed `$info` = null): void
#[\Deprecated]
public function contains(object `$object`): bool
public function count(int `$mode` = COUNT_NORMAL): int
public function current(): object
#[\Deprecated]
public function detach(object `$object`): void
public function getHash(object `$object`): string
public function getInfo(): mixed
public function key(): int
public function next(): void

public function offsetExists(object $object): bool
public function offsetGet(object $object): mixed
public function offsetSet(object $object, mixed $info = null): void
public function offsetUnset(object $object): void
public function removeAll(SplObjectStorage $storage): int
public function removeAllExcept(SplObjectStorage $storage): int
public function rewind(): void
public function seek(int $offset): void
public function serialize(): string
public function setInfo(mixed $info): void
public function unserialize(string $data): void
public function valid(): bool
}

Examples

Exemplo #1 **SplObjectStorage** como um conjunto

```php
<?php
// Como um conjunto de objetos
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new stdClass;

$s->attach($o1);
$s->attach($o2);

var_dump($s->contains($o1));
var_dump($s->contains($o2));
var_dump($s->contains($o3));

$s->detach($o2);

var_dump($s->contains($o1));
var_dump($s->contains($o2));
var_dump($s->contains($o3));
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
bool(false)
bool(true)
bool(false)
bool(false)
```

Exemplo #2 **SplObjectStorage** como um mapa

```php
<?php
// Como um mapa de objetos para dados
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new stdClass;

$s[$o1] = "data for object 1";
$s[$o2] = array(1,2,3);

if (isset($s[$o2])) {
    var_dump($s[$o2]);
```

```
}
?>
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

Registro de Alterações

Versão	Descrição
8.4.0	Implementa SeekableIterator, anteriormente somente Iterator estava implementado.

Índice

SplObjectStorage::addAll

SplObjectStorage::addAll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::addAll — Adiciona todos os objetos de outro armazenamento

Descrição

public function **SplObjectStorage::addAll**(SplObjectStorage `$storage`): int

Adiciona todos os pares de objetos-dados de um armazenamento diferente no armazenamento atual.

Parâmetros

`storage`

O armazenamento que você deseja importar.

Valor Retornado

O número de objetos no armazenamento.

Examples

Exemplo #1 Exemplo de SplObjectStorage::addAll()

```
<?php
$o = new stdClass;
$a = new SplObjectStorage();
$a[$o] = "hello";

$b = new SplObjectStorage();
$b->addAll($a);
echo $b[$o]."\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
hello
```

Veja também

- SplObjectStorage::removeAll() - Remove objetos contidos em outro armazenamento do armazenamento atual

SplObjectStorage::attach

SplObjectStorage::attach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::attach — Adiciona um objeto no armazenamento

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **SplObjectStorage::attach**(object `$object`, mixed `$info` = null): void

Adiciona um object dentro do armazenamento e, opcionalmente, associa-a a alguns dados.

Este método é um sinônimo de SplObjectStorage::offsetSet().

Parâmetros

`object`

O object a ser adicionado.

`info`

Os dados a serem associados ao object.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Este método foi descontinuado em favor de SplObjectStorage::offsetSet().

Examples

Exemplo #1 Exemplo de SplObjectStorage::attach()

```php
<?php
$o1 = new stdClass;
$o2 = new stdClass;
$s = new SplObjectStorage();
$s->attach($o1); // similar a $s[$o1] = NULL;
$s->attach($o2, "hello"); // similar a $s[$o2] = "hello";

var_dump($s[$o1]);
var_dump($s[$o2]);

?>
```

O exemplo acima produzirá algo semelhante a:

```
NULL
```

```
string(5) "hello"
```

Veja também

- SplObjectStorage::detach() - Remove um object do armazenamento
- SplObjectStorage::offsetSet() - Associa dados a um objeto no armazenamento

SplObjectStorage::contains

SplObjectStorage::contains

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::contains — Verifica se no armazenamento contém um objeto específico

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **SplObjectStorage::contains**(object `$object`): bool

Verifica se no armazenamento contém o object fornecido.

Parâmetros

`object`

O object a ser procurado.

Valor Retornado

Retorna `true` se o object estiver no armazenamento, `false` caso contrário.

Registro de Alterações

Versão	Descrição
8.5.0	Este método foi descontinuado em favor de SplObjectStorage::offsetExists().

Examples

Exemplo #1 Exemplo de SplObjectStorage::contains()

```php
<?php
$o1 = new stdClass;
$o2 = new stdClass;

$s = new SplObjectStorage();

$s[$o1] = "hello";
var_dump($s->contains($o1));
var_dump($s->contains($o2));
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(false)
```

Veja também

- SplObjectStorage::offsetExists() - Verifica se um objeto existe no armazenamento

SplObjectStorage::count

SplObjectStorage::count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::count — Retorna o número de objetos no armazenamento

Descrição

public function **SplObjectStorage::count**(int `$mode` = `COUNT_NORMAL`): int

Conta o número de objetos no armazenamento.

Parâmetros

`mode`

Se o parâmetro opcional `mode` for definido como `COUNT_RECURSIVE` (ou 1), **SplObjectStorage::count()** contará recursivamente o armazenamento.

Valor Retornado

O número de objetos no armazenamento.

Examples

Exemplo #1 Exemplo de SplObjectStorage::count()

```php
<?php
$s = new SplObjectStorage();
$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1);
$s->attach($o2);
$s->attach($o1);
var_dump($s->count());
var_dump(count($s));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(2)
int(2)
```

Veja também

- SplObjectStorage::attach() - Adiciona um objeto no armazenamento
- SplObjectStorage::detach() - Remove um object do armazenamento

SplObjectStorage::current

SplObjectStorage::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::current — Retorna a entrada atual do armazenamento

Descrição

public function **SplObjectStorage::current**(): object

Retorna a entrada atual do armazenamento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O object na posição atual do iterador.

Registro de Alterações

Versão	Descrição
8.1.0	**SplObjectStorage::current()** agora lança uma exceção Error se a posição atual for inválida. Anteriormente, `false` era retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::current()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar a current($s)
    $data   = $s->getInfo();

    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

Veja também

- SplObjectStorage::rewind() - Retrocede o iterador para o primeiro elemento do armazenamento
- SplObjectStorage::key() - Retorna o índice em que o iterador está atualmente
- SplObjectStorage::next() - Move para a próxima entrada
- SplObjectStorage::valid() - Retorna se a entrada atual do iterador é válida
- SplObjectStorage::getInfo() - Retorna os dados associados à entrada atual do iterador

SplObjectStorage::detach

SplObjectStorage::detach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::detach — Remove um object do armazenamento

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **SplObjectStorage::detach**(object `$object`): void

Remove o object do armazenamento.

Parâmetros

`object`

O object a ser removido.

Valor Retornado

Nenhum valor é retornado.

Registro de Alterações

Versão	Descrição
8.5.0	Este método foi descontinuado em favor de SplObjectStorage::offsetUnset().

Examples

Exemplo #1 Exemplo de SplObjectStorage::detach()

```php
<?php
$o = new stdClass;
$s = new SplObjectStorage();
$s->attach($o);
var_dump(count($s));
$s->detach($o);
var_dump(count($s));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(1)
int(0)
```

Veja também

- SplObjectStorage::attach() - Adiciona um objeto no armazenamento
- SplObjectStorage::removeAll() - Remove objetos contidos em outro armazenamento do armazenamento atual

SplObjectStorage::getHash

SplObjectStorage::getHash

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplObjectStorage::getHash — Calcula um identificador único para os objetos contidos

Descrição

public function **SplObjectStorage::getHash**(object `$object`): string

Este método calcula um identificador para os objetos adicionados a um objeto SplObjectStorage.

A implementação em SplObjectStorage retorna o mesmo valor que spl_object_hash().

O objeto de armazenamento nunca conterá mais de um objeto com o mesmo identificador. Portanto, pode ser usado para implementar um conjunto (um armazenamento de valores únicos) onde a qualidade de um objeto ser único é determinada pelo fato do valor retornado por esta função ser único.

Parâmetros

`object`

O objeto cujo identificador será calculado.

Valor Retornado

Uma string com o identificador calculado.

Erros/Exceções

Uma RuntimeException é lançada quando o valor retornado não é uma string.

Examples

Exemplo #1 Exemplo de SplObjectStorage::getHash()

```php
<?php
class OneSpecimenPerClassStorage extends SplObjectStorage {
    public function getHash($o) {
        return get_class($o);
    }
}
class A {}

$s = new OneSpecimenPerClassStorage;
$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new A;

$s[$o1] = 1;
// $o2 é considerado igual a $o1, então o valor é substituído
$s[$o2] = 2;
$s[$o3] = 3;

// esses são considerados iguais aos objetos anteriores
// então eles podem ser usados para acessar os valores armazenados sob eles
$p1 = new stdClass;
$p2 = new A;
echo $s[$p1], "\n";
echo $s[$p2], "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
2
3
```

Veja também

- spl_object_hash() - Retorna o ID de hash para o objeto fornecido

SplObjectStorage::getInfo

SplObjectStorage::getInfo

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::getInfo — Retorna os dados associados à entrada atual do iterador

Descrição

public function **SplObjectStorage::getInfo**(): mixed

Retorna os dados ou informações associados ao objeto apontado pela posição atual do iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Os dados associados à posição atual do iterador.

Examples

Exemplo #1 Exemplo de SplObjectStorage::getInfo()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar a current($s)
    $data   = $s->getInfo();

    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

Veja também

- SplObjectStorage::current() - Retorna a entrada atual do armazenamento
- SplObjectStorage::rewind() - Retrocede o iterador para o primeiro elemento do armazenamento
- SplObjectStorage::key() - Retorna o índice em que o iterador está atualmente
- SplObjectStorage::next() - Move para a próxima entrada
- SplObjectStorage::valid() - Retorna se a entrada atual do iterador é válida
- SplObjectStorage::setInfo() - Define os dados associados à entrada atual do iterador

SplObjectStorage::key

SplObjectStorage::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::key — Retorna o índice em que o iterador está atualmente

Descrição

public function **SplObjectStorage::key**(): int

Retorna o índice em que o iterador está atualmente.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O índice correspondente à posição do iterador.

Examples

Exemplo #1 Exemplo de SplObjectStorage::key()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar a current($s)

    var_dump($index);
    var_dump($object);
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(0)
object(stdClass)#2 (0) {
}
int(1)
object(stdClass)#3 (0) {
}
```

Veja também

- SplObjectStorage::rewind() - Retrocede o iterador para o primeiro elemento do armazenamento
- SplObjectStorage::current() - Retorna a entrada atual do armazenamento
- SplObjectStorage::next() - Move para a próxima entrada
- SplObjectStorage::valid() - Retorna se a entrada atual do iterador é válida

SplObjectStorage::next

SplObjectStorage::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::next — Move para a próxima entrada

Descrição

public function **SplObjectStorage::next**(): void

Move o iterador para o próximo object no armazenamento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::next()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar a current($s)

    var_dump($index);
    var_dump($object);
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(0)
object(stdClass)#2 (0) {
}
int(1)
object(stdClass)#3 (0) {
}
```

Veja também

- SPLObjectStorage::rewind() - Retrocede o iterador para o primeiro elemento do armazenamento

SplObjectStorage::offsetExists

SplObjectStorage::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetExists — Verifica se um objeto existe no armazenamento

Descrição

public function **SplObjectStorage::offsetExists**(object `$object`): bool

Verifica se um object existe no armazenamento.

> **Nota** :
>
> **SplObjectStorage::offsetExists()** é um sinônimo de SplObjectStorage::contains().

Parâmetros

`object`

O object a ser procurado.

Valor Retornado

Retorna **`true`** se o object existe no armazenamento, e **`false`** caso contrário.

Examples

Exemplo #1 Exemplo de SplObjectStorage::offsetExists()

```php
<?php
$s = new SplObjectStorage;
$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1);

var_dump($s->offsetExists($o1)); // Similar a isset($s[$o1])
var_dump($s->offsetExists($o2)); // Similar a isset($s[$o2])
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(false)
```

Veja também

- SplObjectStorage::offsetSet() - Associa dados a um objeto no armazenamento
- SplObjectStorage::offsetGet() - Retorna os dados associados a um object
- SplObjectStorage::offsetUnset() - Remove um objeto do armazenamento

SplObjectStorage::offsetGet

SplObjectStorage::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetGet — Retorna os dados associados a um object

Descrição

public function **SplObjectStorage::offsetGet**(object `$object`): mixed

Retorna os dados associados a um object no armazenamento.

Parâmetros

`object`

O object a ser procurado.

Valor Retornado

Os dados previamente associados ao object no armazenamento.

Erros/Exceções

Lança UnexpectedValueException quando o `object` não pôde ser encontrado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::offsetGet()

```php
<?php
$s = new SplObjectStorage;

$o1 = new stdClass;
$o2 = new stdClass;

$s[$o1] = "hello";
$s->attach($o2);


var_dump($s->offsetGet($o1)); // Similar a $s[$o1]
var_dump($s->offsetGet($o2)); // Similar a $s[$o2]
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(5) "hello"
NULL
```

Veja também

- SplObjectStorage::offsetSet() - Associa dados a um objeto no armazenamento
- SplObjectStorage::offsetExists() - Verifica se um objeto existe no armazenamento
- SplObjectStorage::offsetUnset() - Remove um objeto do armazenamento

SplObjectStorage::offsetSet

SplObjectStorage::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetSet — Associa dados a um objeto no armazenamento

Descrição

public function **SplObjectStorage::offsetSet**(object `$object`, mixed `$info = null`): void

Associa dados a um object no armazenamento.

> **Nota** :
>
> **SplObjectStorage::offsetSet()** é um alias de SplObjectStorage::attach().

Parâmetros

`object`

O object para associar dados.

`info`

Os dados para associar ao object.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::offsetSet()

```php
<?php
$s = new SplObjectStorage;

$o1 = new stdClass;

$s->offsetSet($o1, "hello"); // Similar a $s[$o1] = "hello";

var_dump($s[$o1]);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(5) "hello"
```

Veja também

- SplObjectStorage::attach() - Adiciona um objeto no armazenamento
- SplObjectStorage::offsetGet() - Retorna os dados associados a um object
- SplObjectStorage::offsetExists() - Verifica se um objeto existe no armazenamento
- SplObjectStorage::offsetUnset() - Remove um objeto do armazenamento

SplObjectStorage::offsetUnset

SplObjectStorage::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetUnset — Remove um objeto do armazenamento

Descrição

public function **SplObjectStorage::offsetUnset**(object `$object`): void

Remove um object do armazenamento.

> **Nota** :
>
> **SplObjectStorage::offsetUnset()** é um alias de SplObjectStorage::detach().

Parâmetros

`object`

O object a ser removido.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::offsetUnset()

```php
<?php
$o = new stdClass;
$s = new SplObjectStorage();
$s->attach($o);
var_dump(count($s));
$s->offsetUnset($o); // Similar a unset($s[$o])
var_dump(count($s));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(1)
int(0)
```

Veja também

- SplObjectStorage::offsetGet() - Retorna os dados associados a um object
- SplObjectStorage::offsetSet() - Associa dados a um objeto no armazenamento
- SplObjectStorage::offsetExists() - Verifica se um objeto existe no armazenamento

SplObjectStorage::removeAll

SplObjectStorage::removeAll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::removeAll — Remove objetos contidos em outro armazenamento do armazenamento atual

Descrição

public function **SplObjectStorage::removeAll**(SplObjectStorage `$storage`): int

Remove objetos contidos em outro armazenamento do armazenamento atual.

Parâmetros

`storage`

O armazenamento contendo os elementos a serem removidos.

Valor Retornado

Retorna o número de objetos restantes.

Examples

Exemplo #1 Exemplo de SplObjectStorage::removeAll()

```php
<?php
$o1 = new stdClass;
$o2 = new stdClass;
$a = new SplObjectStorage();
$a[$o1] = "foo";

$b = new SplObjectStorage();
$b[$o1] = "bar";
$b[$o2] = "gee";

var_dump(count($b));
$b->removeAll($a);
var_dump(count($b));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(2)
int(1)
```

Veja também

- SplObjectStorage::addAll() - Adiciona todos os objetos de outro armazenamento

SplObjectStorage::removeAllExcept

SplObjectStorage::removeAllExcept

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

SplObjectStorage::removeAllExcept — Remove todos os objetos, exceto aqueles contidos em outro armazenamento, do armazenamento atual

Descrição

public function **SplObjectStorage::removeAllExcept**(SplObjectStorage `$storage`): int

Remove todos os objetos, exceto aqueles contidos em outro armazenamento, do armazenamento atual.

Parâmetros

`storage`

O armazenamento contendo os elementos a serem retidos no armazenamento atual.

Valor Retornado

Retorna o número de objetos restantes.

Examples

Exemplo #1 Exemplo de SplObjectStorage::removeAllExcept()

```php
<?php
$a = (object) 'a';
$b = (object) 'b';
$c = (object) 'c';

$foo = new SplObjectStorage;
$foo->attach($a);
$foo->attach($b);

$bar = new SplObjectStorage;
$bar->attach($b);
$bar->attach($c);

$foo->removeAllExcept($bar);
var_dump($foo->contains($a));
var_dump($foo->contains($b));
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(false)
bool(true)
```

SplObjectStorage::rewind

SplObjectStorage::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::rewind — Retrocede o iterador para o primeiro elemento do armazenamento

Descrição

public function **SplObjectStorage::rewind**(): void

Retrocede o iterador para o primeiro elemento do armazenamento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::rewind()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar a current($s)
    $data   = $s->getInfo();

    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

Veja também

- SplObjectStorage::next() - Move para a próxima entrada

SplObjectStorage::seek

SplObjectStorage::seek

(PHP 8 >= 8.4.0)

SplObjectStorage::seek — Aponta o iterador para uma posição

Descrição

public function **SplObjectStorage::seek**(int `$offset`): void

Aponta para uma determinada posição no iterador.

Parâmetros

`offset`

A posição a ser apontada.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança uma exceção OutOfBoundsException se `offset` não for apontável.

Examples

Exemplo #1 Exemplo de SplObjectStorage::seek()

Aponta para o item na posição 2 no iterador.

```
<?php
class Test {
    public function __construct(public string $marker) {}
}

$a = new Test("a");
$b = new Test("b");
$c = new Test("c");

$storage = new SplObjectStorage();
$storage[$a] = "primeiro";
$storage[$b] = "segundo";
$storage[$c] = "terceiro";

$storage->seek(2);
var_dump($storage->key());
var_dump($storage->current());
?>
```

O exemplo acima produzirá:

```
int(2)
object(Test)#3 (1) {
  ["marker"]=>
  string(1) "c"
}
```

Veja também

- SeekableIterator

SplObjectStorage::serialize

SplObjectStorage::serialize

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplObjectStorage::serialize — Serializa o armazenamento

Descrição

public function **SplObjectStorage::serialize**(): string

Retorna uma representação em string do armazenamento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string representando o armazenamento.

Examples

Exemplo #1 Exemplo de SplObjectStorage::serialize()

```php
<?php
$s = new SplObjectStorage;
$o = new stdClass;
$s[$o] = "data";

echo $s->serialize()."\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
x:i:1;O:8:"stdClass":0:{},s:4:"data";;m:a:0:{}
```

Veja também

- SplObjectStorage::unserialize() - Desserializa um armazenamento a partir de sua representação de string

SplObjectStorage::setInfo

SplObjectStorage::setInfo

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::setInfo — Define os dados associados à entrada atual do iterador

Descrição

public function **SplObjectStorage::setInfo**(mixed `$info`): void

Associa dados, ou informações, ao objeto atualmente apontado pelo iterador.

Parâmetros

`info`

Os dados a serem associados à entrada atual do iterador.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::setInfo()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $s->setInfo("new");
    $s->next();
}
var_dump($s[$o1]);
var_dump($s[$o2]);
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(3) "new"
string(3) "new"
```

Veja também

- SplObjectStorage::current() - Retorna a entrada atual do armazenamento
- SplObjectStorage::rewind() - Retrocede o iterador para o primeiro elemento do armazenamento
- SplObjectStorage::key() - Retorna o índice em que o iterador está atualmente
- SplObjectStorage::next() - Move para a próxima entrada
- SplObjectStorage::valid() - Retorna se a entrada atual do iterador é válida
- SplObjectStorage::getInfo() - Retorna os dados associados à entrada atual do iterador

SplObjectStorage::unserialize

SplObjectStorage::unserialize

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplObjectStorage::unserialize — Desserializa um armazenamento a partir de sua representação de string

Descrição

public function **SplObjectStorage::unserialize**(string `$data`): void

Desserializa entradas de armazenamento e as associa ao armazenamento atual.

Parâmetros

`data`

A representação serializada de um armazenamento.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplObjectStorage::unserialize()

```php
<?php
$s1 = new SplObjectStorage;
$s2 = new SplObjectStorage;
$o = new stdClass;
```

```php
$s1[$o] = "data";

$s2->unserialize($s1->serialize());

var_dump(count($s2));
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(1)
```

Veja também

- SplObjectStorage::serialize() - Serializa o armazenamento

SplObjectStorage::valid

SplObjectStorage::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::valid — Retorna se a entrada atual do iterador é válida

Descrição

public function **SplObjectStorage::valid**(): bool

Retorna se a entrada atual do iterador é válida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a entrada do iterador for válida, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplObjectStorage::valid()

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    echo $s->key()."\n";
    $s->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
0
1
```

Veja também

- SplObjectStorage::current() - Retorna a entrada atual do armazenamento
- SplObjectStorage::getInfo() - Retorna os dados associados à entrada atual do iterador

Exceções

Índice

SPL fornece um conjunto de exceções padrão.

Árvore de Classes de Exceções SPL

- LogicException (estende Exception)
    - BadFunctionCallException
        - BadMethodCallException
    - DomainException
    - InvalidArgumentException
    - LengthException
    - OutOfRangeException
- RuntimeException (estende Exception)
    - OutOfBoundsException
    - OverflowException
    - RangeException
    - UnderflowException
    - UnexpectedValueException

Veja também

- Exceções predefinidas

BadFunctionCallException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um retorno de chamada se refere a uma função indefinida ou se alguns argumentos estão faltando.

Resumo da classe

class **BadFunctionCallException** extends LogicException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string

final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

BadMethodCallException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um retorno de chamada se refere a um método indefinido ou se alguns argumentos estão faltando.

Resumo da classe

class **BadMethodCallException** extends BadFunctionCallException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DomainException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um valor não aderir a um domínio de dados válido definido.

Resumo da classe

class **DomainException** extends LogicException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable

final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

InvalidArgumentException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um argumento não for do tipo esperado.

Resumo da classe

class **InvalidArgumentException** extends LogicException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

LengthException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um comprimento for inválido.

Resumo da classe

class **LengthException** extends LogicException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int

final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

LogicException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção que representa um erro na lógica do programa. Esse tipo de exceção deve levar diretamente a uma correção no seu código.

Resumo da classe

class **LogicException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OutOfBoundsException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um valor não for uma chave válida. Isso representa erros que não podem ser detectados em tempo de compilação.

Resumo da classe

class **OutOfBoundsException** extends RuntimeException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string

final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OutOfRangeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada quando um índice ilegal é solicitado. Isso representa erros que devem ser detectados em tempo de compilação.

Resumo da classe

class **OutOfRangeException** extends LogicException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OverflowException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada ao adicionar um elemento a um contêiner cheio.

Resumo da classe

class **OverflowException** extends RuntimeException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int

final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

RangeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada para indicar erros de intervalo durante a execução do programa. Normalmente, isso significa que houve um erro aritmético que não seja de subfluxo/sobrefluxo. Esta é a versão em tempo de execução de DomainException.

Resumo da classe

class **RangeException** extends RuntimeException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

RuntimeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se ocorrer um erro que só pode ser encontrado em tempo de execução.

Resumo da classe

class **RuntimeException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int

final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

UnderflowException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada ao realizar uma operação inválida em um contêiner vazio, como remover um elemento.

Resumo da classe

class **UnderflowException** extends RuntimeException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

UnexpectedValueException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Exceção lançada se um valor não corresponder a um conjunto de valores. Tipicamente, isso ocorre quando uma função chama outra função e espera que o valor retornado seja de um determinado tipo ou valor, excluindo erros relacionados à aritmética ou ao buffer.

Resumo da classe

class **UnexpectedValueException** extends RuntimeException {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int

final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Iteradores

Índice

SPL fornece um conjunto de iteradores para percorrer por objetos.

Árvore de Classes de Iteradores SPL

- ArrayIterator
    - RecursiveArrayIterator
- EmptyIterator
- IteratorIterator
    - AppendIterator
    - CachingIterator
        - RecursiveCachingIterator
    - FilterIterator
        - CallbackFilterIterator
            - RecursiveCallbackFilterIterator
        - RecursiveFilterIterator
            - ParentIterator
        - RegexIterator
            - RecursiveRegexIterator
    - InfiniteIterator
    - LimitIterator
    - NoRewindIterator
- MultipleIterator
- RecursiveIteratorIterator
    - RecursiveTreeIterator
- DirectoryIterator (estende SplFileInfo)
    - FilesystemIterator
        - GlobIterator
        - RecursiveDirectoryIterator

AppendIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Um Iterador que itera sobre vários iteradores um após o outro.

Resumo da classe

class **AppendIterator** extends IteratorIterator {
/* Métodos */
public function __construct()
public function append(Iterator `$iterator`): void
public function current(): mixed
public function getArrayIterator(): ArrayIterator
public function getIteratorIndex(): ?int
public function key(): scalar
public function next(): void
public function rewind(): void
public function valid(): bool
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

AppendIterator::append

AppendIterator::append

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::append — Acrescenta um iterador

Descrição

public function **AppendIterator::append**(Iterator `$iterator`): void

Acrescenta um iterador.

Parâmetros

`iterator`

O iterador a ser acrescentado.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de AppendIterator::append()

```php
<?php
$array_a = new ArrayIterator(array('a', 'b', 'c'));
$array_b = new ArrayIterator(array('d', 'e', 'f'));

$iterator = new AppendIterator;
$iterator->append($array_a);
$iterator->append($array_b);

foreach ($iterator as $current) {
    echo $current;
}
?>
```

O exemplo acima produzirá:

```
abcdef
```

Veja também

- AppendIterator::__construct() - Constrói um AppendIterator

AppendIterator::__construct

AppendIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::__construct — Constrói um AppendIterator

Descrição

public function **AppendIterator::__construct**()

Constrói um AppendIterator.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Iterando o AppendIterator com foreach

```php
<?php
$pizzas   = new ArrayIterator(array('Margarita', 'Siciliana', 'Hawaii'));
$toppings = new ArrayIterator(array('Cheese', 'Anchovies', 'Olives', 'Pineapple', 'Ham'));

$appendIterator = new AppendIterator;
$appendIterator->append($pizzas);
$appendIterator->append($toppings);

foreach ($appendIterator as $key => $item) {
    echo $key . ' => ' . $item . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
0 => Margarita
1 => Siciliana
2 => Hawaii
0 => Cheese
1 => Anchovies
2 => Olives
3 => Pineapple
4 => Ham
```

Exemplo #2 Iterando o AppendIterator com a API do AppendIterator

```php
<?php
$pizzas   = new ArrayIterator(array('Margarita', 'Siciliana', 'Hawaii'));
$toppings = new ArrayIterator(array('Cheese', 'Anchovies', 'Olives', 'Pineapple', 'Ham'));

$appendIterator = new AppendIterator;
$appendIterator->append($pizzas);
$appendIterator->append($toppings);
```

```php
while ($appendIterator->valid()) {
    printf(
        '%s => %s => %s%s',
        $appendIterator->getIteratorIndex(),
        $appendIterator->key(),
        $appendIterator->current(),
        PHP_EOL
    );
    $appendIterator->next();
}
?>
```

O exemplo acima produzirá:

```
0 => 0 => Margarita
0 => 1 => Siciliana
0 => 2 => Hawaii
1 => 0 => Cheese
1 => 1 => Anchovies
1 => 2 => Olives
1 => 3 => Pineapple
1 => 4 => Ham
```

Notas

> **Cuidado**
>
> Ao usar iterator_to_array() para copiar os valores do AppendIterator para um array, você deve definir o argumento opcional `use_key` como **`false`**. Quando `use_key` não é **`false`**, quaisquer chaves recorrentes em iteradores internos serão sobrescritas no array retornado. Não há como preservar as chaves originais.

Veja também

- AppendIterator::append() - Acrescenta um iterador

AppendIterator::current

AppendIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::current — Obtém o valor atual

Descrição

public function **AppendIterator::current**(): mixed

Obtém o valor atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor atual se for válido ou **`null`** caso contrário.

Veja também

- Iterator::current() - Retorna o elemento atual
- AppendIterator::key() - Obtém a chave atual
- AppendIterator::valid() - Verifica a validade do elemento atual
- AppendIterator::next() - Move para o próximo elemento

- AppendIterator::rewind() - Rebobina o Iterador

AppendIterator::getArrayIterator

AppendIterator::getArrayIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

AppendIterator::getArrayIterator — Obtém o ArrayIterator

Descrição

public function **AppendIterator::getArrayIterator**(): ArrayIterator

Este método obtém o ArrayIterator que é usado para armazenar os iteradores adicionados com AppendIterator::append().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um ArrayIterator contendo os iteradores anexados.

Veja também

- **AppendIterator::getInnerIterator()**

AppendIterator::getIteratorIndex

AppendIterator::getIteratorIndex

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

AppendIterator::getIteratorIndex — Obtém um índice dos iteradores

Descrição

public function **AppendIterator::getIteratorIndex**(): ?int

Obtém o índice do iterador interno atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o índice baseado em zero, inteiro, do iterador interno atual se existir, ou `null` caso contrário.

Examples

Exemplo #1 Exemplo básico de AppendIterator.getIteratorIndex()

```php
<?php
$array_a = new ArrayIterator(array('a' => 'aardwolf', 'b' => 'bear', 'c' => 'capybara'));
$array_b = new ArrayIterator(array('apple', 'orange', 'lemon'));

$iterator = new AppendIterator;
$iterator->append($array_a);
$iterator->append($array_b);

foreach ($iterator as $key => $current) {
    echo $iterator->getIteratorIndex() . '  ' . $key . ' ' . $current . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
0  a aardwolf
0  b bear
0  c capybara
1  0 apple
1  1 orange
1  2 lemon
```

Veja também

- **AppendIterator::getInnerIterator()**
- AppendIterator::getArrayIterator() - Obtém o ArrayIterator

AppendIterator::key

AppendIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::key — Obtém a chave atual

Descrição

public function **AppendIterator::key**(): scalar

Obtém a chave atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave atual se for válida ou `null` caso contrário.

Examples

Exemplo #1 Exemplo básico de AppendIterator::key()

```php
<?php
$array_a = new ArrayIterator(array('a' => 'aardwolf', 'b' => 'bear', 'c' => 'capybara'));
$array_b = new ArrayIterator(array('apple', 'orange', 'lemon'));

$iterator = new AppendIterator;
$iterator->append($array_a);
$iterator->append($array_b);

// Iteração manual
$iterator->rewind();
while ($iterator->valid()) {
    echo $iterator->key() . ' ' . $iterator->current() . PHP_EOL;
    $iterator->next();
}

echo PHP_EOL;

// Com foreach
foreach ($iterator as $key => $current) {
    echo $key . ' ' . $current . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
a aardwolf
b bear
```

```
c capybara
0 apple
1 orange
2 lemon

a aardwolf
b bear
c capybara
0 apple
1 orange
2 lemon
```

Veja também

- Iterator::key() - Retorna a chave do elemento atual
- AppendIterator::current() - Obtém o valor atual
- AppendIterator::valid() - Verifica a validade do elemento atual
- AppendIterator::next() - Move para o próximo elemento
- AppendIterator::rewind() - Rebobina o Iterador

AppendIterator::next

AppendIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::next — Move para o próximo elemento

Descrição

public function **AppendIterator::next**(): void

Move para o próximo elemento. Se isso significa ir para outro Iterador, então ele rebobina aquele Iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- Iterator::next() - Move para o próximo elemento
- AppendIterator::current() - Obtém o valor atual
- AppendIterator::key() - Obtém a chave atual
- AppendIterator::valid() - Verifica a validade do elemento atual
- AppendIterator::rewind() - Rebobina o Iterador

AppendIterator::rewind

AppendIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::rewind — Rebobina o Iterador

Descrição

public function **AppendIterator::rewind**(): void

Rebobina para o primeiro elemento do primeiro Iterador interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- Iterator::rewind() - Retrocede o Iterator para o primeiro elemento
- AppendIterator::current() - Obtém o valor atual
- AppendIterator::key() - Obtém a chave atual
- AppendIterator::next() - Move para o próximo elemento
- AppendIterator::valid() - Verifica a validade do elemento atual

AppendIterator::valid

AppendIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::valid — Verifica a validade do elemento atual

Descrição

public function **AppendIterator::valid**(): bool

Verifica a validade do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a iteração atual for válida, `false` caso contrário.

Veja também

- AppendIterator::current() - Obtém o valor atual
- AppendIterator::key() - Obtém a chave atual
- AppendIterator::next() - Move para o próximo elemento
- AppendIterator::rewind() - Rebobina o Iterador

ArrayIterator

(PHP 5, PHP 7, PHP 8)

Introdução

Permite a remoção de elementos e a modificação de chaves e valores ao iterar sobre arrays ou objects.

Para iterar o mesmo array mais de uma vez, é recomendado instanciar ArrayObject e usar a instância de **ArrayIterator** criada implicitamente ao usar `foreach` para iterar sobre o array armazenado, ou criar uma chamando o método ArrayObject::getIterator() manualmente.

Resumo da classe

class **ArrayIterator** implements SeekableIterator, ArrayAccess, Serializable, Countable {
/* Constantes */
public const int *STD_PROP_LIST*;
public const int *ARRAY_AS_PROPS*;
/* Métodos */
public function __construct(array|object `$array` = [], int `$flags` = 0)
public function append(mixed `$value`): void
public function asort(int `$flags` = **`SORT_REGULAR`**): true
public function count(): int
public function current(): mixed
public function getArrayCopy(): array
public function getFlags(): int
public function key(): string|int|null

public function ksort(int $flags = SORT_REGULAR): true
public function natcasesort(): true
public function natsort(): true
public function next(): void
public function offsetExists(mixed $key): bool
public function offsetGet(mixed $key): mixed
public function offsetSet(mixed $key, mixed $value): void
public function offsetUnset(mixed $key): void
public function rewind(): void
public function seek(int $offset): void
public function serialize(): string
public function setFlags(int $flags): void
public function uasort(callable $callback): true
public function uksort(callable $callback): true
public function unserialize(string $data): void
public function valid(): bool
}

Constantes predefinidas

ArrayIterator Flags

`ArrayIterator::STD_PROP_LIST`

As propriedades de objeto têm sua funcionalidade normal quando acessados como listas (var_dump(), `foreach` etc.).

`ArrayIterator::ARRAY_AS_PROPS`

Itens podem ser acessados como propriedades (leitura e escrita).

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

ArrayIterator::append

ArrayIterator::append

(PHP 5, PHP 7, PHP 8)

ArrayIterator::append — Anexa um elemento

Descrição

public function **ArrayIterator::append**(mixed $value): void

Anexa o valor como o último elemento.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`value`

O valor a ser anexado.

Valor Retornado

Nenhum valor é retornado.

Notas

> **Nota** :
>
> Este método não pode ser chamado quando o ArrayIterator refere-se a um object.

Veja também

- ArrayIterator::next() - Avança para o próximo elemento

ArrayIterator::asort

ArrayIterator::asort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::asort — Ordena entradas pelos valores

Descrição

public function **ArrayIterator::asort**(int `$flags = SORT_REGULAR`): true

Ordena as entradas pelos seus valores.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()
- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Veja também

- ArrayIterator::ksort() - Ordena entradas por chaves
- ArrayIterator::natcasesort() - Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas
- ArrayIterator::natsort() - Ordena entradas naturalmente
- ArrayIterator::uasort() - Ordena com uma função de comparação definida pelo usuário e manter a associação de índices
- ArrayIterator::uksort() - Ordena por chaves usando uma função de comparação definida pelo usuário
- asort() - Ordena um array em ordem crescente e mantém a associação de índices

ArrayIterator::__construct

ArrayIterator::__construct

(PHP 5, PHP 7, PHP 8)

ArrayIterator::__construct — Constrói um ArrayIterator

Descrição

public function **ArrayIterator::__construct**(array|object `$array` = [], int `$flags` = 0)

Constrói um ArrayIterator object.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`array`

O array ou objeto a ser iterado.

`flags`

Flags para controlar o comportamento do objeto ArrayIterator. Veja ArrayIterator::setFlags().

Veja também

- ArrayIterator::getArrayCopy() - Obtém cópia do array

ArrayIterator::count

ArrayIterator::count

(PHP 5, PHP 7, PHP 8)

ArrayIterator::count — Conta os elementos

Descrição

public function **ArrayIterator::count**(): int

Retorna o número de elementos no array, ou o número de atributos públicos no object.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de elementos ou atributos públicos no array ou object, respectivamente.

Veja também

- ArrayIterator::getFlags() - Retorna as flags de comportamento

ArrayIterator::current

ArrayIterator::current

(PHP 5, PHP 7, PHP 8)

ArrayIterator::current — Retorna a entrada atual do array

Descrição

public function **ArrayIterator::current**(): mixed

Obtém a entrada atual do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A entrada atual do array.

Examples

Exemplo #1 Exemplo de ArrayIterator::current()

```
<?php
$array = array('1' => 'um',
               '2' => 'dois',
               '3' => 'três');

$arrayobject = new ArrayObject($array);

for($iterator = $arrayobject->getIterator();
    $iterator->valid();
    $iterator->next()) {

    echo $iterator->key() . ' => ' . $iterator->current() . "\n";
}
?>
```

O exemplo acima produzirá:

```
1 => um
2 => dois
3 => três
```

ArrayIterator::getArrayCopy

ArrayIterator::getArrayCopy

(PHP 5, PHP 7, PHP 8)

ArrayIterator::getArrayCopy — Obtém cópia do array

Descrição

public function **ArrayIterator::getArrayCopy**(): array

Obtém uma cópia de um array.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma cópia do array, ou um array de propriedades públicas se o ArrayIterator referir-se a um object.

Veja também

- ArrayIterator::valid() - Verifica se o iterador de array contém mais entradas

ArrayIterator::getFlags

ArrayIterator::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayIterator::getFlags — Retorna as flags de comportamento

Descrição

public function **ArrayIterator::getFlags**(): int

Retorna as flags de comportamento do ArrayIterator. Veja o método ArrayIterator::setFlags para saber as flags disponíveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna as flags de comportamento do ArrayIterator.

Veja também

- ArrayIterator::setFlags() - Define as flags que irão alterar o comportamento
- ArrayIterator::valid() - Verifica se o iterador de array contém mais entradas

ArrayIterator::key

ArrayIterator::key

(PHP 5, PHP 7, PHP 8)

ArrayIterator::key — Retorna a chave do elemento atual do array

Descrição

public function **ArrayIterator::key**(): string|int|null

Esta função retorna a chave do elemento atual do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave do atual elemento do array.

Examples

Exemplo #1 Exemplo de ArrayIterator::key()

```php
<?php
$array = array('chave' => 'valor');

$arrayobject = new ArrayObject($array);
$iterator = $arrayobject->getIterator();

echo $iterator->key(); // chave
?>
```

ArrayIterator::ksort

ArrayIterator::ksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::ksort — Ordena entradas por chaves

Descrição

public function **ArrayIterator::ksort**(int `$flags = SORT_REGULAR`): true

Ordena as entradas pelas suas chaves.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`flags`

O segundo parâmetro opcional `flags` pode ser usado para modificar o comportamento da ordenação usando estes valores:

Flags dos tipos de ordenação:

- `SORT_REGULAR` - compara os elementos normalmente; os detalhes são descritos na seção operadores de comparação
- `SORT_NUMERIC` - compara os elementos numericamente
- `SORT_STRING` - compara os elementos como strings
- `SORT_LOCALE_STRING` - compara os elementos como strings, com base na localidade atual. A localidade pode ser alterada usando setlocale()
- `SORT_NATURAL` - compara os elementos como strings usando "ordenação natural" como natsort()
- `SORT_FLAG_CASE` - pode ser combinada (usando o operador bit a bit OR) com `SORT_STRING` ou `SORT_NATURAL` para ordenar strings sem fazer distinção entre maiúsculas e minúsculas

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Veja também

- ArrayIterator::asort() - Ordena entradas pelos valores
- ArrayIterator::natcasesort() - Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas
- ArrayIterator::natsort() - Ordena entradas naturalmente
- ArrayIterator::uasort() - Ordena com uma função de comparação definida pelo usuário e manter a associação de índices
- ArrayIterator::uksort() - Ordena por chaves usando uma função de comparação definida pelo usuário
- ksort() - Ordena um array pelas chaves em ordem crescente

ArrayIterator::natcasesort

ArrayIterator::natcasesort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::natcasesort — Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas

Descrição

public function **ArrayIterator::natcasesort**(): true

Ordena as entradas pelos valores usando um algoritmo de "ordem natural" insensível a maiúsculas e minúsculas.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Veja também

- ArrayIterator::asort() - Ordena entradas pelos valores
- ArrayIterator::ksort() - Ordena entradas por chaves
- ArrayIterator::natsort() - Ordena entradas naturalmente
- ArrayIterator::uasort() - Ordena com uma função de comparação definida pelo usuário e manter a associação de índices
- ArrayIterator::uksort() - Ordena por chaves usando uma função de comparação definida pelo usuário
- natcasesort() - Ordena um array utilizando o algoritmo de "ordem natural" sem diferenciar maiúsculas e minúsculas

ArrayIterator::natsort

ArrayIterator::natsort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::natsort — Ordena entradas naturalmente

Descrição

public function **ArrayIterator::natsort**(): true

Ordena as entradas pelos valores usando um algoritmo de "ordem natural".

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Veja também

- ArrayIterator::asort() - Ordena entradas pelos valores
- ArrayIterator::ksort() - Ordena entradas por chaves
- ArrayIterator::natcasesort() - Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas

- ArrayIterator::uasort() - Ordena com uma função de comparação definida pelo usuário e manter a associação de índices
- ArrayIterator::uksort() - Ordena por chaves usando uma função de comparação definida pelo usuário
- natsort() - Ordena um array utilizando o algoritmo de "ordem natural"

ArrayIterator::next

ArrayIterator::next

(PHP 5, PHP 7, PHP 8)

ArrayIterator::next — Avança para o próximo elemento

Descrição

public function **ArrayIterator::next**(): void

Avança o iterador para o próximo elemento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayIterator::next()

```php
<?php
$arrayobject = new ArrayObject();

$arrayobject[] = 'zero';
$arrayobject[] = 'um';

$iterator = $arrayobject->getIterator();

while($iterator->valid()) {
    echo $iterator->key() . ' => ' . $iterator->current() . "\n";

    $iterator->next();
}
?>
```

O exemplo acima produzirá:

```
0 => zero
1 => um
```

ArrayIterator::offsetExists

ArrayIterator::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetExists — Verificar se o deslocamento existe

Descrição

public function **ArrayIterator::offsetExists**(mixed $key): bool

Verifica se o deslocamento existe.

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O deslocamento sendo verificado.

Valor Retornado

`true` se o deslocamento existir, caso contrário `false`

Veja também

- ArrayIterator::valid() - Verifica se o iterador de array contém mais entradas

ArrayIterator::offsetGet

ArrayIterator::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetGet — Obtém valor para um deslocamento

Descrição

public function **ArrayIterator::offsetGet**(mixed `$key`): mixed

Obtém o valor do deslocamento fornecido.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O deslocamento para obter o valor.

Valor Retornado

O valor no deslocamento `key`.

Veja também

- ArrayIterator::offsetSet() - Define valor para um deslocamento
- ArrayIterator::offsetUnset() - Remove valor para um deslocamento

ArrayIterator::offsetSet

ArrayIterator::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetSet — Define valor para um deslocamento

Descrição

public function **ArrayIterator::offsetSet**(mixed `$key`, mixed `$value`): void

Define um valor para um determinado deslocamento.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O índice para definir.

`value`

O novo valor a ser armazenado no índice.

Valor Retornado

Nenhum valor é retornado.

Veja também

- ArrayIterator::offsetGet() - Obtém valor para um deslocamento
- ArrayIterator::offsetUnset() - Remove valor para um deslocamento

ArrayIterator::offsetUnset

ArrayIterator::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetUnset — Remove valor para um deslocamento

Descrição

public function **ArrayIterator::offsetUnset**(mixed `$key`): void

Remove um valor para um deslocamento.

Se a iteração estiver em andamento e **ArrayIterator::offsetUnset()** for usado para remover o índice atual da iteração, a posição da iteração será avançada para o próximo índice. Como a posição da iteração também é avançada no final de um loop `foreach`, o uso de **ArrayIterator::offsetUnset()** dentro de um loop `foreach` pode resultar em índices sendo pulados.

Parâmetros

`key`

O deslocamento a ser removido.

Valor Retornado

Nenhum valor é retornado.

Veja também

- ArrayIterator::offsetGet() - Obtém valor para um deslocamento
- ArrayIterator::offsetSet() - Define valor para um deslocamento

ArrayIterator::rewind

ArrayIterator::rewind

(PHP 5, PHP 7, PHP 8)

ArrayIterator::rewind — Retrocede o array

Descrição

public function **ArrayIterator::rewind**(): void

Esta função retrocede o iterador para o início do array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de ArrayIterator::rewind()

```php
<?php
$arrayobject = new ArrayObject();

$arrayobject[] = 'zero';
$arrayobject[] = 'um';
$arrayobject[] = 'dois';

$iterator = $arrayobject->getIterator();

$iterator->next();
echo $iterator->key(); // 1

$iterator->rewind(); // retrocedendo ao início
echo $iterator->key(); // 0
?>
```

ArrayIterator::seek

ArrayIterator::seek

(PHP 5, PHP 7, PHP 8)

ArrayIterator::seek — Aponta para uma posição

Descrição

public function **ArrayIterator::seek**(int `$offset`): void

Aponta para um posição informada no iterador.

Parâmetros

`offset`

A posição a ser apontada.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança uma OutOfBoundsException se a posição `offset` não for apontável.

ArrayIterator::serialize

ArrayIterator::serialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayIterator::serialize — Serializa

Descrição

public function **ArrayIterator::serialize**(): string

Serializa.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O ArrayIterator serializado.

Veja também

- ArrayIterator::unserialize() - Desserializa

ArrayIterator::setFlags

ArrayIterator::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayIterator::setFlags — Define as flags que irão alterar o comportamento

Descrição

public function **ArrayIterator::setFlags**(int `$flags`): void

Define as flags que alteram o comportamento do ArrayIterator

Parâmetros

`flags`

O novo comportamento do ArrayIterator. Podem ser utilizadas máscara de bits ou constantes. O uso de constantes é fortemente encorajado para garantir compatibilidade com versões futuras.

As flags disponíveis estão listadas abaixo. O atual significado das flags estão descritas nas constantes predefinidas.

Flags do comportamento do ArrayIterator

value	constant
1	ArrayIterator::STD_PROP_LIST
2	ArrayIterator::ARRAY_AS_PROPS

Valor Retornado

Nenhum valor é retornado.

Veja também

- ArrayIterator::getFlags() - Retorna as flags de comportamento

ArrayIterator::uasort

ArrayIterator::uasort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::uasort — Ordena com uma função de comparação definida pelo usuário e manter a associação de índices

Descrição

public function **ArrayIterator::uasort**(callable `$callback`): true

Este método ordena os elementos de forma que os índices mantenham sua correlação com os valores aos quais estão associados, usando uma função de comparação definida pelo usuário.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é **`true`**; anteriormente, era bool.

Veja também

- ArrayIterator::asort() - Ordena entradas pelos valores
- ArrayIterator::ksort() - Ordena entradas por chaves
- ArrayIterator::natcasesort() - Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas
- ArrayIterator::natsort() - Ordena entradas naturalmente
- ArrayIterator::uksort() - Ordena por chaves usando uma função de comparação definida pelo usuário
- uasort() - Ordena um array com uma função de comparação definida pelo usuário e mantém associações de índices

ArrayIterator::uksort

ArrayIterator::uksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::uksort — Ordena por chaves usando uma função de comparação definida pelo usuário

Descrição

public function **ArrayIterator::uksort**(callable `$callback`): true

Este método ordena os elementos pelas chaves usando uma função de comparação fornecida pelo usuário.

> **Nota** :
>
> Se dois elementos são comparados como iguais, eles mantêm sua ordem original. Antes do PHP 8.0.0, sua ordem relativa no array ordenado era indefinida.

Parâmetros

`callback`

A função de comparação deve retornar um inteiro menor que, igual ou maior que zero se o primeiro argumento for considerado respectivamente menor que, igual ou maior que o segundo.

function callback(mixed `$a`, mixed `$b`): int

> **Cuidado**
>
> Retornar valores *não inteiros* da função de comparação, como float, resultará em uma conversão interna do valor retornado da função callback para int. Portanto, valores como `0.99` e `0.1` serão convertidos para o valor inteiro `0`, o que comparará esses valores como iguais.

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo do retorno agora é `true`; anteriormente, era bool.

Veja também

- ArrayIterator::asort() - Ordena entradas pelos valores
- ArrayIterator::ksort() - Ordena entradas por chaves
- ArrayIterator::natcasesort() - Ordena entradas naturalmente, sem diferenciar maiúsculas de minúsculas
- ArrayIterator::natsort() - Ordena entradas naturalmente
- ArrayIterator::uasort() - Ordena com uma função de comparação definida pelo usuário e manter a associação de índices
- uksort() - Ordena um array pelas chaves utilizando uma função de comparação definida pelo usuário.

ArrayIterator::unserialize

ArrayIterator::unserialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayIterator::unserialize — Desserializa

Descrição

public function **ArrayIterator::unserialize**(string `$data`): void

Desserializa.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`data`

O objeto ArrayIterator serializado a ser desserializado.

Valor Retornado

Nenhum valor é retornado.

Veja também

- ArrayIterator::serialize() - Serializa

ArrayIterator::valid

ArrayIterator::valid

(PHP 5, PHP 7, PHP 8)

ArrayIterator::valid — Verifica se o iterador de array contém mais entradas

Descrição

public function **ArrayIterator::valid**(): bool

Verifica se o array contém mais alguma entrada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o iterador for válido, caso contrário retorna `false`.

Examples

Exemplo #1 Exemplo de ArrayIterator::valid()

```php
<?php
$array = array('1' => 'um');

$arrayobject = new ArrayObject($array);
$iterator = $arrayobject->getIterator();

var_dump($iterator->valid()); // bool(true)

$iterator->next(); // avança para o próximo item

// bool(false) porque existe somente um elemento no array
var_dump($iterator->valid());
?>
```

CachingIterator

(PHP 5, PHP 7, PHP 8)

Introdução

Este objeto suporta iteração em cache sobre outro iterador.

Resumo da classe

class **CachingIterator** extends IteratorIterator implements ArrayAccess, Countable, Stringable {
/* Constantes */
public const int *CALL_TOSTRING*;
public const int *CATCH_GET_CHILD*;
public const int *TOSTRING_USE_KEY*;
public const int *TOSTRING_USE_CURRENT*;
public const int *TOSTRING_USE_INNER*;
public const int *FULL_CACHE*;
/* Métodos */
public function __construct(Iterator `$iterator`, int `$flags` = **`CachingIterator::CALL_TOSTRING`**)
public function count(): int
public function current(): mixed
public function getCache(): array
public function getFlags(): int
public function hasNext(): bool
public function key(): scalar
public function next(): void
public function offsetExists(string `$key`): bool
public function offsetGet(string `$key`): mixed
public function offsetSet(string `$key`, mixed `$value`): void
public function offsetUnset(string `$key`): void
public function rewind(): void
public function setFlags(int `$flags`): void
public function __toString(): string
public function valid(): bool
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Constantes predefinidas

`CachingIterator::CALL_TOSTRING`

Converte cada elemento para string.

`CachingIterator::CATCH_GET_CHILD`

Não lançar exceção ao acessar filhos.

`CachingIterator::TOSTRING_USE_KEY`

Usar key para conversão para string.

`CachingIterator::TOSTRING_USE_CURRENT`

Usar current para conversão para string.

`CachingIterator::TOSTRING_USE_INNER`

Usar inner para conversão para string.

`CachingIterator::FULL_CACHE`

Cache de todos os dados lidos.

Registro de Alterações

Versão	Descrição
8.0.0	**CachingIterator** implementa Stringable agora.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

CachingIterator::__construct

CachingIterator::__construct

(PHP 5, PHP 7, PHP 8)

CachingIterator::__construct — Constrói um novo objeto CachingIterator para o iterador

Descrição

public function **CachingIterator::__construct**(Iterator `$iterator`, int `$flags` = `CachingIterator::CALL_TOSTRING`)

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

Iterador a ser armazenado em cache

`flags`

Máscara de bits de opções.

CachingIterator::count

CachingIterator::count

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

CachingIterator::count — O número de elementos no iterador

Descrição

public function **CachingIterator::count**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Pode retornar o número de elementos no iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de elementos iterados.

CachingIterator::current

CachingIterator::current

(PHP 5, PHP 7, PHP 8)

CachingIterator::current — Retorna o elemento atual

Descrição

public function **CachingIterator::current**(): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Pode retornar o elemento atual na iteração.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Diversos

Veja também

- Iterator::current() - Retorna o elemento atual

CachingIterator::getCache

CachingIterator::getCache

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::getCache — Recupera o conteúdo do cache

Descrição

public function **CachingIterator::getCache**(): array

Recupera o conteúdo do cache.

> **Nota** :
>
> A opção `CachingIterator::FULL_CACHE` deve estar sendo utilizada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array contendo os itens do cache.

Erros/Exceções

Lança uma BadMethodCallException quando a opção `CachingIterator::FULL_CACHE` não está sendo utilizada.

Examples

Exemplo #1 Exemplo de CachingIterator::getCache()

```php
<?php
$iterator = new ArrayIterator(array(1, 2, 3));
$cache    = new CachingIterator($iterator, CachingIterator::FULL_CACHE);

$cache->next();
$cache->next();
var_dump($cache->getCache());

$cache->next();
var_dump($cache->getCache());
?>
```

O exemplo acima produzirá:

```
array(2) {
  [0]=>
  int(1)
  [1]=>
  int(2)
}
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

CachingIterator::getFlags

CachingIterator::getFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::getFlags — Obtém as opções utilizadas

Descrição

public function **CachingIterator::getFlags**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Obtenha a máscara de bits das opções usadas para esta instância de CachingIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Descrição...

CachingIterator::hasNext

CachingIterator::hasNext

(PHP 5, PHP 7, PHP 8)

CachingIterator::hasNext — Verifica se o iterador interno possui um próximo elemento válido

Descrição

public function **CachingIterator::hasNext**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

CachingIterator::key

CachingIterator::key

(PHP 5, PHP 7, PHP 8)

CachingIterator::key — Retorna a chave para o elemento atual

Descrição

public function **CachingIterator::key**(): scalar

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Este método pode retornar uma chave para o elemento atual.

Parâmetros

Esta função não possui parâmetros.

CachingIterator::next

CachingIterator::next

(PHP 5, PHP 7, PHP 8)

CachingIterator::next — Move o iterador para frente

Descrição

public function **CachingIterator::next**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Mover o iterador para frente.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

CachingIterator::offsetExists

CachingIterator::offsetExists

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetExists — O propósito do offsetExists

Descrição

public function **CachingIterator::offsetExists**(string `$key`): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O índice sendo verificado.

Valor Retornado

Retorna `true` se uma entrada referenciada pelo offset existe, `false` caso contrário.

CachingIterator::offsetGet

CachingIterator::offsetGet

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetGet — O propósito de offsetGet

Descrição

public function **CachingIterator::offsetGet**(string `$key`): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

Descrição...

Valor Retornado

Descrição...

CachingIterator::offsetSet

CachingIterator::offsetSet

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetSet — O propósito do offsetSet

Descrição

public function **CachingIterator::offsetSet**(string `$key`, mixed `$value`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O índice do elemento a ser definido.

`value`

O novo valor para o `key`.

Valor Retornado

Nenhum valor é retornado.

CachingIterator::offsetUnset

CachingIterator::offsetUnset

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetUnset — O objetivo do offsetUnset

Descrição

public function **CachingIterator::offsetUnset**(string `$key`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`key`

O índice do elemento a ser removido.

Valor Retornado

Nenhum valor é retornado.

CachingIterator::rewind

CachingIterator::rewind

(PHP 5, PHP 7, PHP 8)

CachingIterator::rewind — Rebobina o iterador

Descrição

public function **CachingIterator::rewind**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Rebobina o iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

CachingIterator::setFlags

CachingIterator::setFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::setFlags — O propósito da função setFlags

Descrição

public function **CachingIterator::setFlags**(int `$flags`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Define as opções para o objeto CachingIterator.

Parâmetros

`flags`

Máscara de bits das opções a serem definidas.

Valor Retornado

Nenhum valor é retornado.

CachingIterator::__toString

CachingIterator::__toString

(PHP 5, PHP 7, PHP 8)

CachingIterator::__toString — Retorna a representação em string do elemento atual

Descrição

public function **CachingIterator::__toString**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Obtém a representação em string do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A representação em string do elemento atual.

CachingIterator::valid

CachingIterator::valid

(PHP 5, PHP 7, PHP 8)

CachingIterator::valid — Verifica se o elemento atual é válido

Descrição

public function **CachingIterator::valid**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Verifica se o elemento atual é válido.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

CallbackFilterIterator

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introdução

Resumo da classe

class **CallbackFilterIterator** extends FilterIterator {
/* Métodos */
public function __construct(Iterator `$iterator`, callable `$callback`)
public function accept(): bool
/* Métodos herdados */
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Examples

O callback deve aceitar até três argumentos: o item atual, a chave atual e o iterador, respectivamente.

Exemplo #1 Argumentos de callback disponíveis

```php
<?php

/**
 * Callback para CallbackFilterIterator
 *
 * @param $current   Valor do item atual
 * @param $key       Chave do item atual
 * @param $iterator  Iterador sendo filtrado
 * @return boolean   TRUE para aceitar o item atual, FALSE caso contrário
 */
function my_callback($current, $key, $iterator) {
    // Seu código de filtragem aqui
}

?>
```

Qualquer callable pode ser usado; como uma string contendo um nome de função, um array para um método ou uma função anônima.

Exemplo #2 Exemplos básicos de callback

```php
<?php

$dir = new FilesystemIterator(__DIR__);

// Filtrar arquivos grandes ( > 100MB)
function is_large_file($current) {
    return $current->isFile() && $current->getSize() > 104857600;
}
$large_files = new CallbackFilterIterator($dir, 'is_large_file');
```

```php
// Filtrar diretórios
$files = new CallbackFilterIterator($dir, function ($current, $key, $iterator) {
    return $current->isDir() && ! $iterator->isDot();
});

?>
```

Índice

CallbackFilterIterator::accept

CallbackFilterIterator::accept

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

CallbackFilterIterator::accept — Chama o callback com o valor atual, a chave atual e o iterador interno como argumentos

Descrição

public function **CallbackFilterIterator::accept**(): bool

Este método chama o callback com o valor atual, a chave atual e o iterador interno.

Espera-se que o callback retorne `true` se o item atual deve ser aceito, ou `false` caso contrário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` para aceitar o item atual, ou `false` caso contrário.

Veja também

- Exemplos de CallbackFilterIterator
- CallbackFilterIterator::__construct() - Cria um iterador filtrado a partir de outro iterador

CallbackFilterIterator::__construct

CallbackFilterIterator::__construct

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

CallbackFilterIterator::__construct — Cria um iterador filtrado a partir de outro iterador

Descrição

public function **CallbackFilterIterator::__construct**(Iterator `$iterator`, callable `$callback`)

Cria um iterador filtrado usando o `callback` para determinar quais itens são aceitos ou rejeitados.

Parâmetros

`iterator`

O iterador a ser filtrado.

`callback`

O callback, que deve retornar `true` para aceitar o item atual ou `false` caso contrário. Veja Exemplos.

Pode ser qualquer valor callable válido.

Veja também

- Exemplos de CallbackFilterIterator
- CallbackFilterIterator::accept() - Chama o callback com o valor atual, a chave atual e o iterador interno como argumentos

DirectoryIterator

(PHP 5, PHP 7, PHP 8)

Introdução

A classe DirectoryIterator fornece uma interface simples para visualização de conteúdo de sistemas de arquivos.

Resumo da classe

```
class DirectoryIterator extends SplFileInfo implements SeekableIterator {
/* Métodos */
public function __construct(string $directory)
public function current(): mixed
public function getBasename(string $suffix = ""): string
public function getExtension(): string
public function getFilename(): string
public function isDot(): bool
public function key(): mixed
public function next(): void
public function rewind(): void
public function seek(int $offset): void
public function __toString(): string
public function valid(): bool
/* Métodos herdados */
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}
```

Índice

DirectoryIterator::__construct

DirectoryIterator::__construct

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::__construct — Constrói um novo iterador de diretório a partir de um caminho

Descrição

public function **DirectoryIterator::__construct**(string `$directory`)

Constrói um novo iterador de diretório a partir de um caminho.

Parâmetros

`directory`

O caminho do diretório a ser percorrido.

Erros/Exceções

Lança um UnexpectedValueException se o `directory` não existir.

Lança um ValueError se o `directory` for uma string vazia.

Registro de Alterações

Versão	Descrição
8.0.0	Agora Lança um ValueError se `directory` for uma string vazia; anteriormente, lançava um RuntimeException.

Examples

Exemplo #1 Um exemplo de DirectoryIterator::__construct()

Este exemplo listará o conteúdo do diretório que contém o script.

```
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if (!$fileinfo->isDot()) {
        var_dump($fileinfo->getFilename());
    }
}
?>
```

Veja também

- SplFileInfo
- Iterator

DirectoryIterator::current

DirectoryIterator::current

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::current — Retorna o item atual do DirectoryIterator

Descrição

public function **DirectoryIterator::current**(): mixed

Obtém o item atual do DirectoryIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O item atual do DirectoryIterator.

Examples

Exemplo #1 Um exemplo de DirectoryIterator::current()

Este exemplo listará o conteúdo do diretório que contém o script.

```php
<?php
$iterator = new DirectoryIterator(__DIR__);
while($iterator->valid()) {
    $file = $iterator->current();
    echo $iterator->key() . " => " . $file->getFilename() . "\n";
    $iterator->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
0 => .
1 => ..
2 => apple.jpg
3 => banana.jpg
4 => index.php
5 => pear.jpg
```

Veja também

- DirectoryIterator::key() - Retorna a chave para o item atual do DirectoryIterator
- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator
- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início
- DirectoryIterator::valid() - Verifica se a posição atual do DirectoryIterator é um arquivo válido
- Iterator::current() - Retorna o elemento atual

DirectoryIterator::getBasename

DirectoryIterator::getBasename

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

DirectoryIterator::getBasename — Retorna o nome do item corrente do DirectoryIterator

Descrição

public function **DirectoryIterator::getBasename**(string `$suffix = ""`): string

Retorna o nome do item corrente do DirectoryIterator.

Parâmetros

`suffix`

Se o nome base terminar no `suffix`, ele é retirado.

Valor Retornado

O nome do item corrente do DirectoryIterator.

Examples

Exemplo #1 Exemplo do DirectoryIterator::getBasename()

Esse exemplo irá listar todo o nome base e o nome base com o sufixo `.jpg` removido dos nomes de arquivo do diretório o qual o script irá rodar.

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if ($fileinfo->isFile()) {
        echo $fileinfo->getBasename() . "\n";
        echo $fileinfo->getBasename('.jpg') . "\n";
    }
```

```
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
apple.jpg
apple
banana.jpg
banana
index.php
index.php
pear.jpg
pear
```

Veja também

- DirectoryIterator::getFilename() - Retorna o nome do arquivo do item atual do DirectoryIterator
- **DirectoryIterator::getPath()**
- **DirectoryIterator::getPathname()**
- basename() - Retorna a última parte do caminho
- pathinfo() - Retorna informações sobre um caminho de arquivo

DirectoryIterator::getExtension

DirectoryIterator::getExtension

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

DirectoryIterator::getExtension — Obtém a extensão do arquivo

Descrição

public function **DirectoryIterator::getExtension**(): string

Recupera a extensão do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string contendo a extensão do arquivo, ou uma string vazia se o arquivo não tiver extensão.

Examples

Exemplo #1 Exemplo de DirectoryIterator::getExtension()

```php
<?php

$directory = new DirectoryIterator(__DIR__);
foreach ($directory as $fileinfo) {
    if ($fileinfo->isFile()) {
        echo $fileinfo->getExtension() . "\n";
    }
}

?>
```

O exemplo acima produzirá algo semelhante a:

```
php
txt
jpg
gz
```

Notas

> **Nota** :
>
> Outra maneira de obter a extensão é utilizar a função pathinfo().
>
> ```php
> <?php
> $extension = pathinfo($fileinfo->getFilename(), PATHINFO_EXTENSION);
> ?>
> ```

Veja também

- DirectoryIterator::getFilename() - Retorna o nome do arquivo do item atual do DirectoryIterator
- DirectoryIterator::getBasename() - Retorna o nome do item corrente do DirectoryIterator
- pathinfo() - Retorna informações sobre um caminho de arquivo

DirectoryIterator::getFilename

DirectoryIterator::getFilename

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::getFilename — Retorna o nome do arquivo do item atual do DirectoryIterator

Descrição

public function **DirectoryIterator::getFilename**(): string

Obtém o nome do arquivo do item atual do DirectoryIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do arquivo do item atual do DirectoryIterator.

Examples

Exemplo #1 Um exemplo de DirectoryIterator::getFilename()

Este exemplo listará o conteúdo do diretório que contém o script.

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    echo $fileinfo->getFilename() . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
.
..
apple.jpg
banana.jpg
```

```
index.php
pear.jpg
```

Veja também

- DirectoryIterator::getBasename() - Retorna o nome do item corrente do DirectoryIterator
- **DirectoryIterator::getPath()**
- **DirectoryIterator::getPathname()**
- pathinfo() - Retorna informações sobre um caminho de arquivo

DirectoryIterator::isDot

DirectoryIterator::isDot

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::isDot — Determina se o item atual do DirectoryIterator é '.' ou '..'

Descrição

public function **DirectoryIterator::isDot**(): bool

Determina se o item atual do DirectoryIterator é um diretório e é `.` ou `..`

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a entrada for `.` ou `..`, caso contrário `false`

Examples

Exemplo #1 Um exemplo de DirectoryIterator::isDot()

Este exemplo listará todos os arquivos, omitindo as entradas `.` e `..`

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
foreach ($iterator as $fileinfo) {
    if (!$fileinfo->isDot()) {
        echo $fileinfo->getFilename() . "\n";
    }
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
apple.jpg
banana.jpg
example.php
pears.jpg
```

Veja também

- **DirectoryIterator::getType()**
- **DirectoryIterator::isDir()**
- **DirectoryIterator::isFile()**
- **DirectoryIterator::isLink()**

DirectoryIterator::key

DirectoryIterator::key

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::key — Retorna a chave para o item atual do DirectoryIterator

Descrição

public function **DirectoryIterator::key**(): mixed

Obtém a chave para o item atual do DirectoryIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave para o item atual do DirectoryIterator como um int.

Registro de Alterações

Versão	Descrição
8.1.0	Quando o iterador não está inicializado, agora é lançado um Error. Anteriormente, o método retornava `false`.

Examples

Exemplo #1 Um exemplo de DirectoryIterator::key()

```
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if (!$fileinfo->isDot()) {
        echo $fileinfo->key() . " => " . $fileinfo->getFilename() . "\n";
    }
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
0 => apple.jpg
1 => banana.jpg
2 => index.php
3 => pear.jpg
```

Veja também

- DirectoryIterator::current() - Retorna o item atual do DirectoryIterator
- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator
- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início
- DirectoryIterator::valid() - Verifica se a posição atual do DirectoryIterator é um arquivo válido
- Iterator::key() - Retorna a chave do elemento atual

DirectoryIterator::next

DirectoryIterator::next

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::next — Mover para o próximo item do DirectoryIterator

Descrição

public function **DirectoryIterator::next**(): void

Move para o próximo item do DirectoryIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de DirectoryIterator::next()

Lista o conteúdo de um diretório usando um loop while.

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
while($iterator->valid()) {
    echo $iterator->getFilename() . "\n";
    $iterator->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
.
..
apple.jpg
banana.jpg
index.php
pear.jpg
```

Veja também

- DirectoryIterator::current() - Retorna o item atual do DirectoryIterator
- DirectoryIterator::key() - Retorna a chave para o item atual do DirectoryIterator
- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início
- DirectoryIterator::valid() - Verifica se a posição atual do DirectoryIterator é um arquivo válido
- Iterator::next() - Move para o próximo elemento

DirectoryIterator::rewind

DirectoryIterator::rewind

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::rewind — Rebobina o DirectoryIterator de volta ao início

Descrição

public function **DirectoryIterator::rewind**(): void

Rebobina o DirectoryIterator de volta ao início.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de DirectoryIterator::rewind()

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));

$iterator->next();
echo $iterator->key(); // 1

$iterator->rewind(); // retocedendo até o início
echo $iterator->key(); // 0
?>
```

Veja também

- DirectoryIterator::current() - Retorna o item atual do DirectoryIterator
- DirectoryIterator::key() - Retorna a chave para o item atual do DirectoryIterator
- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator
- DirectoryIterator::valid() - Verifica se a posição atual do DirectoryIterator é um arquivo válido
- Iterator::rewind() - Retrocede o Iterator para o primeiro elemento

DirectoryIterator::seek

DirectoryIterator::seek

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DirectoryIterator::seek — Aponta para uma determinada posição no item do DirectoryIterator

Descrição

public function **DirectoryIterator::seek**(int `$offset`): void

Aponta para uma determinada posição no item do DirectoryIterator.

Parâmetros

`offset`

Posição para apontar com referência em zero.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de DirectoryIterator::seek()

Aponta para o quarto item no diretório que contém o script. Os dois primeiros geralmente são `.` e `..`

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
$iterator->seek(3);
if ($iterator->valid()) {
    echo $iterator->getFilename();
} else {
    echo 'Sem arquivos na posição 3';
}
?>
```

Veja também

- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início
- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator
- SeekableIterator::seek() - Aponta para uma posição

DirectoryIterator::__toString

DirectoryIterator::__toString

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::__toString — Retorna o nome do arquivo como texto

Descrição

public function **DirectoryIterator::__toString**(): string

Retorna o nome do item corrente do DirectoryIterator como texto.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do item corrente do DirectoryIterator como texto.

Examples

Exemplo #1 Exemplo do DirectoryIterator::__toString()

Este exemplo irá listar o conteúdo do diretório o qual o script está rodando.

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    echo $fileinfo;
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
.
..
maça.jpg
banana.jpg
index.php
pera.jpg
```

Veja também

- DirectoryIterator::getFilename() - Retorna o nome do arquivo do item atual do DirectoryIterator
- O método mágico __toString()

DirectoryIterator::valid

DirectoryIterator::valid

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::valid — Verifica se a posição atual do DirectoryIterator é um arquivo válido

Descrição

public function **DirectoryIterator::valid**(): bool

Verifica se a posição atual do DirectoryIterator é um arquivo válido.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a posição é válida, caso contrário `false`

Examples

Exemplo #1 Um exemplo de DirectoryIterator::valid()

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));

// Loop to end of iterator
while($iterator->valid()) {
    $iterator->next();
}

$iterator->valid(); // FALSE
$iterator->rewind();
$iterator->valid(); // TRUE

?>
```

Veja também

- DirectoryIterator::current() - Retorna o item atual do DirectoryIterator
- DirectoryIterator::key() - Retorna a chave para o item atual do DirectoryIterator
- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator
- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início
- Iterator::valid() - Verifica se a posição atual é válida

EmptyIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A classe EmptyIterator para um iterador vazio.

Resumo da classe

class **EmptyIterator** implements Iterator {
/* Métodos */
public function current(): never
public function key(): never
public function next(): void
public function rewind(): void
public function valid(): false
}

Índice

EmptyIterator::current

EmptyIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::current — O método current()

Descrição

public function **EmptyIterator::current**(): never

Esta função não deve ser chamada. Ela lança uma exceção quando acessada.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Erros/Exceções

Lança uma Exception se for chamada.

Valor Retornado

Nenhum valor é retornado.

EmptyIterator::key

EmptyIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::key — O método key()

Descrição

public function **EmptyIterator::key**(): never

Esta função não deve ser chamada. Ela lança uma exceção ao ser acessada.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Erros/Exceções

Lança uma Exception se for chamada.

Valor Retornado

Nenhum valor é retornado.

EmptyIterator::next

EmptyIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::next — O método next()

Descrição

public function **EmptyIterator::next**(): void

Nenhuma operação, nada a ser feito.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

EmptyIterator::rewind

EmptyIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::rewind — O método rewind()

Descrição

public function **EmptyIterator::rewind**(): void

Nenhuma operação, nada a ser feito.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

EmptyIterator::valid

EmptyIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::valid — Verifica se o elemento atual é válido

Descrição

public function **EmptyIterator::valid**(): false

Verifica se o elemento atual é válido.

> **Nota** :
>
> O EmptyIterator está sempre vazio e nunca terá um valor válido. O valor de retorno é sempre `false`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`false`

FilesystemIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

O iterador do sistema de arquivos

Resumo da classe

class **FilesystemIterator** extends DirectoryIterator {
/* Constantes */
public const int *CURRENT_MODE_MASK*;
public const int *CURRENT_AS_PATHNAME*;
public const int *CURRENT_AS_FILEINFO*;
public const int *CURRENT_AS_SELF*;
public const int *KEY_MODE_MASK*;
public const int *KEY_AS_PATHNAME*;

```
public const int FOLLOW_SYMLINKS;
public const int KEY_AS_FILENAME;
public const int NEW_CURRENT_AND_KEY;
public const int OTHER_MODE_MASK;
public const int SKIP_DOTS;
public const int UNIX_PATHS;
/* Métodos */
public function __construct(string $directory, int $flags = FilesystemIterator::KEY_AS_PATHNAME |
FilesystemIterator::CURRENT_AS_FILEINFO | FilesystemIterator::SKIP_DOTS)
public function current(): string|SplFileInfo|FilesystemIterator
public function getFlags(): int
public function key(): string
public function next(): void
public function rewind(): void
public function setFlags(int $flags): void
/* Métodos herdados */
public function DirectoryIterator::current(): mixed
public function DirectoryIterator::getBasename(string $suffix = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int $offset): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}
```

Constantes predefinidas

`FilesystemIterator::CURRENT_AS_PATHNAME`

Faz com que FilesystemIterator::current() retorne o pathname.

FilesystemIterator::CURRENT_AS_FILEINFO

Faz com que FilesystemIterator::current() retorne uma instância SplFileInfo.

FilesystemIterator::CURRENT_AS_SELF

Faz com que FilesystemIterator::current() retorne $this (o FilesystemIterator).

FilesystemIterator::CURRENT_MODE_MASK

Máscara para FilesystemIterator::current()

FilesystemIterator::KEY_AS_PATHNAME

Faz com que FilesystemIterator::key() retorne o pathname.

FilesystemIterator::KEY_AS_FILENAME

Faz com que FilesystemIterator::key() retorne o filename.

FilesystemIterator::FOLLOW_SYMLINKS

Faz com que RecursiveDirectoryIterator::hasChildren() siga os links simbólicos.

FilesystemIterator::KEY_MODE_MASK

Máscara para FilesystemIterator::key()

FilesystemIterator::NEW_CURRENT_AND_KEY

Mesmo que `FilesystemIterator::KEY_AS_FILENAME | FilesystemIterator::CURRENT_AS_FILEINFO`.

FilesystemIterator::OTHER_MODE_MASK

Máscara usada para FilesystemIterator::getFlags() e FilesystemIterator::setFlags().

FilesystemIterator::SKIP_DOTS

Pula os arquivos de ponto (`.` e `..`).

FilesystemIterator::UNIX_PATHS

Faz com que os caminhos usem barra inclinada no estilo Unix, independentemente do padrão do sistema. Observe que o `path` passado ao construtor não é modificado.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

FilesystemIterator::__construct

FilesystemIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::__construct — Constrói um novo iterador de sistema de arquivos

Descrição

public function **FilesystemIterator::__construct**(string `$directory`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO | FilesystemIterator::SKIP_DOTS)

Constrói um novo iterador de sistema de arquivos a partir do `directory`.

Parâmetros

`directory`

O caminho do item do sistema de arquivos a ser iterado.

`flags`

As opções podem ser fornecidas e afetarão o comportamento de alguns métodos. Uma lista das opções pode ser encontrada em Constantes predefinidas do FilesystemIterator. Elas também podem ser definidas posteriormente com FilesystemIterator::setFlags()

Erros/Exceções

Lança uma UnexpectedValueException se o `directory` não existir.

Lança um ValueError se o `directory` for uma string vazia.

Registro de Alterações

Versão	Descrição
8.2.0	Antes do PHP 8.2.0, `FilesystemIterator::SKIP_DOTS` era sempre definido e não podia ser removido.
8.0.0	Agora lança um ValueError se `directory` for uma string vazia; anteriormente lançava um RuntimeException.

Examples

Exemplo #1 Exemplo de FilesystemIterator::__construct()

```php
<?php
$it = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::CURRENT_AS_FILEINFO);
foreach ($it as $fileinfo) {
    echo $fileinfo->getFilename() . "\n";
}
?>
```

A saída do exemplo acima no PHP 8.2 é semelhante a:

```
.
..
apples.jpg
banana.jpg
example.php
```

A saída do exemplo acima antes do PHP 8.2.0 é semelhante a:

```
apples.jpg
banana.jpg
example.php
```

Veja também

- FilesystemIterator::setFlags() - Define opções de manipulação
- DirectoryIterator::__construct() - Constrói um novo iterador de diretório a partir de um caminho

FilesystemIterator::current

FilesystemIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::current — O arquivo atual

Descrição

public function **FilesystemIterator::current**(): string|SplFileInfo|FilesystemIterator

Obtém informações do arquivo do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome do arquivo, as informações do arquivo ou $this dependendo das opções definidas. Veja as constantes FilesystemIterator.

Examples

Exemplo #1 Exemplo de FilesystemIterator::current()

Este exemplo listará o conteúdo do diretório que contém o script.

```php
<?php
$iterator = new FilesystemIterator(__DIR__, FilesystemIterator::CURRENT_AS_PATHNAME);
foreach ($iterator as $fileinfo) {
    echo $iterator->current() . "\n";
}
?>
```

A saída do exemplo acima no PHP 8.2 é semelhante a:

```
/www/examples/.
/www/examples/..
/www/examples/apple.jpg
/www/examples/banana.jpg
/www/examples/example.php
```

Veja também

- Constantes FilesystemIterator
- DirectoryIterator::current() - Retorna o item atual do DirectoryIterator
- DirectoryIterator::getFileName() - Retorna o nome do arquivo do item atual do DirectoryIterator

FilesystemIterator::getFlags

FilesystemIterator::getFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::getFlags — Obtem as opções de manipulação

Descrição

public function **FilesystemIterator::getFlags**(): int

Obtém as opções de manipulação, conforme definidas em FilesystemIterator::__construct() ou FilesystemIterator::setFlags().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor inteiro das opções definidas.

Veja também

- FilesystemIterator::__construct() - Constrói um novo iterador de sistema de arquivos
- FilesystemIterator::setFlags() - Define opções de manipulação

FilesystemIterator::key

FilesystemIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::key — Recupera a chave para o arquivo atual

Descrição

public function **FilesystemIterator::key**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho ou nome do arquivo dependendo das opções definidas. Veja as constantes FilesystemIterator.

Examples

Exemplo #1 Exemplo de FilesystemIterator::key()

Este exemplo listará o conteúdo do diretório que contém o script.

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_FILENAME);
foreach ($iterator as $fileinfo) {
    echo $iterator->key() . "\n";
}
?>
```

A saída do exemplo acima no PHP 8.2 é semelhante a:

```
.
..
apple.jpg
banana.jpg
example.php
```

Veja também

- Constantes FilesystemIterator
- DirectoryIterator::key() - Retorna a chave para o item atual do DirectoryIterator
- DirectoryIterator::getFilename() - Retorna o nome do arquivo do item atual do DirectoryIterator
- **DirectoryIterator::getPathname()**

FilesystemIterator::next

FilesystemIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::next — Move para o próximo arquivo

Descrição

public function **FilesystemIterator::next**(): void

Mover para o próximo arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de FilesystemIterator::next()

Liste o conteúdo de um diretório usando um loop while.

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__));
while($iterator->valid()) {
    echo $iterator->getFilename() . "\n";
    $iterator->next();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
apple.jpg
banana.jpg
example.php
```

Veja também

- DirectoryIterator::next() - Mover para o próximo item do DirectoryIterator

FilesystemIterator::rewind

FilesystemIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::rewind — Retrocede para o início

Descrição

public function **FilesystemIterator::rewind**(): void

Retrocede o diretório para o início.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de FilesystemIterator::rewind()

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_FILENAME);

echo $iterator->key() . "\n";

$iterator->next();
echo $iterator->key() . "\n";

$iterator->rewind();
echo $iterator->key() . "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
apple.jpg
banana.jpg
apple.jpg
```

Veja também

- DirectoryIterator::rewind() - Rebobina o DirectoryIterator de volta ao início

FilesystemIterator::setFlags

FilesystemIterator::setFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::setFlags — Define opções de manipulação

Descrição

public function **FilesystemIterator::setFlags**(int `$flags`): void

Define opções de manipulação.

Parâmetros

`flags`

As opções de manipulação para definir. Veja as Constantes FilesystemIterator.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de FilesystemIterator::key()

Este exemplo demonstra a diferença entre as opções FilesystemIterator::KEY_AS_PATHNAME e FilesystemIterator::KEY_AS_FILENAME.

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_PATHNAME);
echo "Key as Pathname:\n";
foreach ($iterator as $key => $fileinfo) {
    echo $key . "\n";
}

$iterator->setFlags(FilesystemIterator::KEY_AS_FILENAME);
echo "\nKey as Filename:\n";
foreach ($iterator as $key => $fileinfo) {
    echo $key . "\n";
}
?>
```

A saída do exemplo acima no PHP 8.2 é semelhante a:

```
Key as Pathname:
/www/examples/.
/www/examples/..
/www/examples/apple.jpg
/www/examples/banana.jpg
/www/examples/example.php
```

```
Key as Filename:
.
..
apple.jpg
banana.jpg
example.php
```

Veja também

- FilesystemIterator::__construct() - Constrói um novo iterador de sistema de arquivos
- FilesystemIterator::getFlags() - Obtem as opções de manipulação

FilterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este iterador abstrato filtra os valores indesejados. Esta classe deve ser estendida para implementar filtros de iterador personalizados. O FilterIterator::accept() deve ser implementado na subclasse.

Resumo da classe

abstract class **FilterIterator** extends IteratorIterator {
/* Métodos */
public function __construct(Iterator `$iterator`)
public function accept(): bool
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

FilterIterator::accept

FilterIterator::accept

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::accept — Verifica se o elemento atual do iterador é aceitável

Descrição

public function **FilterIterator::accept**(): bool

Retorna se o elemento atual do iterador é aceitável por meio deste filtro.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o elemento atual for aceitável, caso contrário `false`.

Examples

Exemplo #1 Exemplo de FilterIterator::accept()

```php
<?php
// Este iterador filtra todos os valores com menos de 10 caracteres
class LengthFilterIterator extends FilterIterator {

    public function accept() {
        // Aceita apenas strings com comprimento de 10 ou mais caracteres
        return strlen(parent::current()) >= 10;
    }

}

$arrayIterator = new ArrayIterator(array('test1', 'mais de 10 caracteres'));
$lengthFilter = new LengthFilterIterator($arrayIterator);

foreach ($lengthFilter as $value) {
    echo $value . "\n";
}
?>
```

O exemplo acima produzirá:

```
mais de 10 caracteres
```

FilterIterator::__construct

FilterIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::__construct — Constrói um FilterIterator

Descrição

public function **FilterIterator::__construct**(Iterator `$iterator`)

Constrói um novo FilterIterator, que consiste em um `iterator` passado com filtros aplicados a ele.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O iterador que está sendo filtrado.

Veja também

- LimitIterator::__construct() - Constrói um LimitIterator

FilterIterator::current

FilterIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::current — Retorna o valor do elemento atual

Descrição

public function **FilterIterator::current**(): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Obtém o valor do atual elemento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do elemento atual.

Veja também

- FilterIterator::key() - Retorna a chave do elemento atual
- FilterIterator::next() - Avança o iterador

FilterIterator::key

FilterIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::key — Retorna a chave do elemento atual

Descrição

public function **FilterIterator::key**(): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Obtém a chave atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave atual.

Veja também

- FilterIterator::next() - Avança o iterador
- FilterIterator::current() - Retorna o valor do elemento atual

FilterIterator::next

FilterIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::next — Avança o iterador

Descrição

public function **FilterIterator::next**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Avança o iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- FilterIterator::current() - Retorna o valor do elemento atual
- FilterIterator::key() - Retorna a chave do elemento atual

FilterIterator::rewind

FilterIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::rewind — Recomeça o iterador

Descrição

public function **FilterIterator::rewind**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Recomeça o iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- FilterIterator::current() - Retorna o valor do elemento atual
- FilterIterator::key() - Retorna a chave do elemento atual
- FilterIterator::next() - Avança o iterador

FilterIterator::valid

FilterIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::valid — Verifica se o elemento atual é válido ou não

Descrição

public function **FilterIterator::valid**(): bool

Verifica se o elemento atual é válido.

> **Nota** :
>
> A implementação padrão desta função irá inicialmente retornar `false` até que o iterador interno seja avançado ao primeiro elemento aceito.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o elemento atual é válido, `false` caso contrário

GlobIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Itera através de um sistema de arquivos de forma semelhante ao glob().

Resumo da classe

class **GlobIterator** extends FilesystemIterator implements Countable {
/* Constantes herdadas */
public const int *FilesystemIterator::CURRENT_MODE_MASK*;
public const int *FilesystemIterator::CURRENT_AS_PATHNAME*;
public const int *FilesystemIterator::CURRENT_AS_FILEINFO*;
public const int *FilesystemIterator::CURRENT_AS_SELF*;
public const int *FilesystemIterator::KEY_MODE_MASK*;
public const int *FilesystemIterator::KEY_AS_PATHNAME*;
public const int *FilesystemIterator::FOLLOW_SYMLINKS*;
public const int *FilesystemIterator::KEY_AS_FILENAME*;
public const int *FilesystemIterator::NEW_CURRENT_AND_KEY*;
public const int *FilesystemIterator::OTHER_MODE_MASK*;
public const int *FilesystemIterator::SKIP_DOTS*;
public const int *FilesystemIterator::UNIX_PATHS*;
/* Métodos */
public function __construct(string `$pattern`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME |
FilesystemIterator::CURRENT_AS_FILEINFO)
public function count(): int
/* Métodos herdados */
public function FilesystemIterator::current(): string|SplFileInfo|FilesystemIterator
public function FilesystemIterator::getFlags(): int
public function FilesystemIterator::key(): string
public function FilesystemIterator::next(): void
public function FilesystemIterator::rewind(): void
public function FilesystemIterator::setFlags(int `$flags`): void
public function DirectoryIterator::current(): mixed
public function DirectoryIterator::getBasename(string `$suffix` = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int `$offset`): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string `$suffix` = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string `$class` = `null`): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string `$class` = `null`): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false

public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = **false**, ?resource $context = **null**): SplFileObject
public function SplFileInfo::setFileClass(string $class = **SplFileObject::class**): void
public function SplFileInfo::setInfoClass(string $class = **SplFileInfo::class**): void
public function SplFileInfo::__toString(): string
}

Índice

GlobIterator::__construct

GlobIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

GlobIterator::__construct — Constrói um diretório usando glob

Descrição

public function **GlobIterator::__construct**(string $pattern, int $flags = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO)

Constrói um novo iterador de diretório a partir de uma expressão glob.

Parâmetros

`pattern`

Um padrão glob().

`flags`

Sinalizadores de opção, os sinalizadores podem ser uma máscara de bits das constantes FilesystemIterator.

Erros/Exceções

Lança uma UnexpectedValueException se o diretório definido em `directory` não existir.

Lança um ValueError se o diretório definido em `directory` for uma string vazia.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança um ValueError se `directory` for uma string vazia; anteriormente lançava um RuntimeException.

Examples

Exemplo #1 Exemplo de GlobIterator

```php
<?php
$iterator = new GlobIterator('*.dll', FilesystemIterator::KEY_AS_FILENAME);

if (!$iterator->count()) {
    echo 'Sem correspondência';
} else {
    $n = 0;

    printf("Corresponderam %d item(s)\r\n", $iterator->count());

    foreach ($iterator as $item) {
        printf("[%d] %s\r\n", ++$n, $iterator->key());
```

```
    }
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Corresponderam 2 item(s)
[1] php5ts.dll
[2] php_gd2.dll
```

Veja também

- DirectoryIterator::__construct() - Constrói um novo iterador de diretório a partir de um caminho
- GlobIterator::count() - Obtém o número de diretórios e arquivos
- glob() - Encontra caminhos que combinam com uma expressão

GlobIterator::count

GlobIterator::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

GlobIterator::count — Obtém o número de diretórios e arquivos

Descrição

public function **GlobIterator::count**(): int

Obtém o número de diretórios e arquivos encontrados pela expressão glob.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de diretórios e arquivos retornados, como um int.

Examples

Exemplo #1 Exemplo de **GlobIterator::count()**

```
<?php
$iterator = new GlobIterator('*.xml');

printf("Matched %d item(s)\r\n", $iterator->count());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Matched 8 item(s)
```

Veja também

- GlobIterator::__construct() - Constrói um diretório usando glob
- count() - Conta todos os elementos de um array ou de um objeto Countable
- glob() - Encontra caminhos que combinam com uma expressão

InfiniteIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

InfiniteIterator permite iterar infinitamente sobre um iterador sem ter que rebobiná-lo manualmente ao alcançar o seu final.

Resumo da classe

class **InfiniteIterator** extends IteratorIterator {
/* Métodos */
public function __construct(Iterator `$iterator`)
public function next(): void
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

InfiniteIterator::__construct

InfiniteIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

InfiniteIterator::__construct — Constrói um InfiniteIterator

Descrição

public function **InfiniteIterator::__construct**(Iterator `$iterator`)

Constrói um InfiniteIterator a partir de um Iterator.

Parâmetros

`iterator`

O iterador para iterar infinitamente.

Examples

Exemplo #1 Exemplo de InfiniteIterator::__construct()

```php
<?php
$arrayit  = new ArrayIterator(array('gato','cachorro'));
$infinite = new InfiniteIterator($arrayit);
$limit    = new LimitIterator($infinite, 0, 7);
foreach($limit as $value)
{
    echo "$value\n";
}
?>
```

O exemplo acima produzirá:

```
gato
cachorro
gato
cachorro
gato
```

```
cachorro
gato
```

Veja também

- InfiniteIterator::next() - Movimenta o Iterador Interno para frente ou rebobina

InfiniteIterator::next

InfiniteIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

InfiniteIterator::next — Movimenta o Iterador Interno para frente ou rebobina

Descrição

public function **InfiniteIterator::next**(): void

Movimenta o Iterator interno para o próximo elemento, se houver um, caso contrário, rebobina o Iterator interno de volta ao início.

> **Nota** :
>
> Mesmo um InfiniteIterator irá parar se o seu Iterator interno estiver vazio.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- InfiniteIterator::__construct() - Constrói um InfiniteIterator

IteratorIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este envoltório de iterador permite a conversão de qualquer coisa que seja Traversable em um Iterador. É importante entender que a maioria das classes que não implementam Iterators têm razões, pois provavelmente elas não permitem o conjunto completo de recursos de Iterators. Se for o caso, técnicas devem ser fornecidas para prevenir uso inadequado, caso contrário, espere exceções ou erros fatais.

Resumo da classe

class **IteratorIterator** implements OuterIterator {
/* Métodos */
public function __construct(Traversable `$iterator`, ?string `$class` = `null`)
public function current(): mixed
public function getInnerIterator(): ?Iterator
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
}

Notas

> **Nota** :

> Esta classe permite o acesso aos métodos do iterador interno através do método mágico __call.

Índice

IteratorIterator::__construct

IteratorIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::__construct — Cria um iterador de qualquer coisa que seja percorrível

Descrição

public function **IteratorIterator::__construct**(Traversable `$iterator`, ?string `$class` = `null`)

Cria um iterador de qualquer coisa que seja percorrível.

Parâmetros

`iterator`

O iterador percorrível.

`class`

O nome da classe a ser usada para o iterador interno. Isso permite especificar uma classe de iterador diferente para envolver o iterador fornecido. Por padrão, usará a própria classe `IteratorIterator`.

Veja também

- Traversable

IteratorIterator::current

IteratorIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::current — Obtém o valor atual

Descrição

public function **IteratorIterator::current**(): mixed

Obtém o valor do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do elemento atual.

Veja também

- IteratorIterator::key() - Obtém a chave do elemento atual

IteratorIterator::getInnerIterator

IteratorIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::getInnerIterator — Obtém o iterador interno

Descrição

public function **IteratorIterator::getInnerIterator**(): ?Iterator

Obtém o iterador interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O iterador interno passado para IteratorIterator::__construct(), ou `null` quando não há iterador interno.

Veja também

- Iterator
- OuterIterator

IteratorIterator::key

IteratorIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::key — Obtém a chave do elemento atual

Descrição

public function **IteratorIterator::key**(): mixed

Obtém a chave do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave do elemento atual.

Veja também

- IteratorIterator::current() - Obtém o valor atual

IteratorIterator::next

IteratorIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::next — Avança para o próximo elemento

Descrição

public function **IteratorIterator::next**(): void

Avança para o próximo elemento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- IteratorIterator::rewind() - Rebobina para o primeiro elemento
- IteratorIterator::valid() - Verifica se o elemento atual é válido

IteratorIterator::rewind

IteratorIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::rewind — Rebobina para o primeiro elemento

Descrição

public function **IteratorIterator::rewind**(): void

Rebobina para o primeiro elemento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- IteratorIterator::next() - Avança para o próximo elemento
- IteratorIterator::valid() - Verifica se o elemento atual é válido

IteratorIterator::valid

IteratorIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::valid — Verifica se o elemento atual é válido

Descrição

public function **IteratorIterator::valid**(): bool

Verifica se o elemento atual é válido.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o elemento atual for válido, caso contrário `false`

Veja também

- iterator_count() - Conta os elementos em um iterador
- IteratorIterator::current() - Obtém o valor atual

LimitIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A classe **LimitIterator** permite a iteração sobre um subconjunto limitado de itens em um Iterator.

Resumo da classe

class **LimitIterator** extends IteratorIterator {
/* Métodos */
public function __construct(Iterator `$iterator`, int `$offset` = 0, int `$limit` = -1)
public function current(): mixed
public function getPosition(): int
public function key(): mixed
public function next(): void
public function rewind(): void
public function seek(int `$offset`): int
public function valid(): bool

```
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}
```

Examples

Exemplo #1 Exemplo de uso de LimitIterator

```php
<?php

// Cria um iterador para ser limitado
$fruits = new ArrayIterator(array(
    'apple',
    'banana',
    'cherry',
    'damson',
    'elderberry'
));

// Percorre pelas três primeiras frutas somente
foreach (new LimitIterator($fruits, 0, 3) as $fruit) {
    var_dump($fruit);
}

echo "\n";

// Percorre a partir da terceira fruta até o final
// Observação: a posição inicia a partir de zero para "apple"
foreach (new LimitIterator($fruits, 2) as $fruit) {
    var_dump($fruit);
}

?>
```

O exemplo acima produzirá:

```
string(5) "apple"
string(6) "banana"
string(6) "cherry"

string(6) "cherry"
string(6) "damson"
string(10) "elderberry"
```

Índice

LimitIterator::__construct

LimitIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::__construct — Constrói um LimitIterator

Descrição

public function **LimitIterator::__construct**(Iterator `$iterator`, int `$offset` = 0, int `$limit` = -1)

Constrói um novo LimitIterator a partir de um `iterator` com um `offset` inicial e um `limit` máximo.

Parâmetros

`iterator`

O Iterator a ser limitado.

`offset`

Deslocamento opcional do limite.

`limit`

Contagem opcional do limite.

Erros/Exceções

Lança um ValueError se o `offset` for menor que `0` ou o `limit` for menor que `-1`.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança um ValueError se o `offset` for menor que `0`; anteriormente lançava um RuntimeException.
8.0.0	Agora lança um ValueError se o `limit` for menor que `-1`; anteriormente lançava um RuntimeException.

Examples

Exemplo #1 Exemplo de LimitIterator::__construct()

```php
<?php
$ait = new ArrayIterator(array('a', 'b', 'c', 'd', 'e'));
$lit = new LimitIterator($ait, 1, 3);
foreach ($lit as $value) {
    echo $value . "\n";
}
?>
```

O exemplo acima produzirá:

```
b
c
d
```

Veja também

- Exemplos de LimitIterator

LimitIterator::current

LimitIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::current — Obtém o elemento atual

Descrição

public function **LimitIterator::current**(): mixed

Obtém o elemento atual do Iterator interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o elemento atual ou `null` se não houver nenhum.

Veja também

- LimitIterator::key() - Obtém a chave atual
- LimitIterator::next() - Avança o iterador
- LimitIterator::rewind() - Retorna o iterador para o começo especificado no construtor
- LimitIterator::seek() - Aponta para a posição especificada
- LimitIterator::valid() - Verifica se o elemento atual é válido ou não

LimitIterator::getPosition

LimitIterator::getPosition

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::getPosition — Retorna a posição atual

Descrição

public function **LimitIterator::getPosition**(): int

Obtém a posição atual baseada em zero do Iterator interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A posição atual.

Examples

Exemplo #1 Exemplos de LimitIterator::getPosition()

```php
<?php
$fruits = array(
    'a' => 'apple',
    'b' => 'banana',
    'c' => 'cherry',
    'd' => 'damson',
    'e' => 'elderberry'
);
$array_it = new ArrayIterator($fruits);
$limit_it = new LimitIterator($array_it, 2, 3);
foreach ($limit_it as $item) {
    echo $limit_it->getPosition() . ' ' . $item . "\n";
}
?>
```

O exemplo acima produzirá:

```
2 cherry
3 damson
4 elderberry
```

Veja também

- FilterIterator::key() - Retorna a chave do elemento atual

LimitIterator::key

LimitIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::key — Obtém a chave atual

Descrição

public function **LimitIterator::key**(): mixed

Obtém a chave para o item atual no Iterator interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a chave para o item atual.

Veja também

- LimitIterator::getPosition() - Retorna a posição atual
- LimitIterator::current() - Obtém o elemento atual
- LimitIterator::next() - Avança o iterador
- LimitIterator::rewind() - Retorna o iterador para o começo especificado no construtor
- LimitIterator::seek() - Aponta para a posição especificada
- LimitIterator::valid() - Verifica se o elemento atual é válido ou não

LimitIterator::next

LimitIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::next — Avança o iterador

Descrição

public function **LimitIterator::next**(): void

Avança o iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- LimitIterator::current() - Obtém o elemento atual
- LimitIterator::key() - Obtém a chave atual
- LimitIterator::rewind() - Retorna o iterador para o começo especificado no construtor
- LimitIterator::seek() - Aponta para a posição especificada
- LimitIterator::valid() - Verifica se o elemento atual é válido ou não

LimitIterator::rewind

LimitIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::rewind — Retorna o iterador para o começo especificado no construtor

Descrição

public function **LimitIterator::rewind**(): void

Retrocede o iterador para o offset inicial especificado em LimitIterator::__construct().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- LimitIterator::current() - Obtém o elemento atual
- LimitIterator::key() - Obtém a chave atual
- LimitIterator::next() - Avança o iterador
- LimitIterator::seek() - Aponta para a posição especificada
- LimitIterator::valid() - Verifica se o elemento atual é válido ou não

LimitIterator::seek

LimitIterator::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::seek — Aponta para a posição especificada

Descrição

public function **LimitIterator::seek**(int `$offset`): int

Move o iterador para o deslocamento especificado por `offset`.

Parâmetros

`offset`

A posição a ser procurada.

Valor Retornado

Retorna a posição após o apontamento.

Erros/Exceções

Lança um OutOfBoundsException se a posição estiver fora dos limites especificados em LimitIterator::__construct().

Veja também

- LimitIterator::current() - Obtém o elemento atual
- LimitIterator::key() - Obtém a chave atual
- LimitIterator::rewind() - Retorna o iterador para o começo especificado no construtor
- LimitIterator::next() - Avança o iterador
- LimitIterator::valid() - Verifica se o elemento atual é válido ou não

LimitIterator::valid

LimitIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::valid — Verifica se o elemento atual é válido ou não

Descrição

public function **LimitIterator::valid**(): bool

Verifica se o elemento atual é válido ou não.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- LimitIterator::current() - Obtém o elemento atual
- LimitIterator::key() - Obtém a chave atual
- LimitIterator::rewind() - Retorna o iterador para o começo especificado no construtor
- LimitIterator::next() - Avança o iterador
- LimitIterator::seek() - Aponta para a posição especificada

MultipleIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Um Iterador que itera sequencialmente sobre todos os iteradores anexados

Resumo da classe

```
class MultipleIterator implements Iterator {
/* Constantes */
public const int MIT_NEED_ANY;
public const int MIT_NEED_ALL;
public const int MIT_KEYS_NUMERIC;
public const int MIT_KEYS_ASSOC;
/* Métodos */
public function __construct(int $flags = MultipleIterator::MIT_NEED_ALL | MultipleIterator::MIT_KEYS_NUMERIC)
public function attachIterator(Iterator $iterator, string|int|null $info = null): void
public function containsIterator(Iterator $iterator): bool
public function countIterators(): int
public function current(): array
public function detachIterator(Iterator $iterator): void
public function getFlags(): int
public function key(): array
public function next(): void
public function rewind(): void
public function setFlags(int $flags): void
public function valid(): bool
}
```

Constantes predefinidas

`MultipleIterator::MIT_NEED_ANY`

Não requer que todos os subiteradores sejam válidos na iteração.

`MultipleIterator::MIT_NEED_ALL`

Requer que todos os subiteradores sejam válidos na iteração.

`MultipleIterator::MIT_KEYS_NUMERIC`

As chaves são criadas a partir da posição dos subiteradores.

`MultipleIterator::MIT_KEYS_ASSOC`

As chaves são criadas a partir das informações associadas aos subiteradores.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

MultipleIterator::attachIterator

MultipleIterator::attachIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::attachIterator — Anexa informações do iterador

Descrição

public function **MultipleIterator::attachIterator**(Iterator `$iterator`, string|int|null `$info` = **`null`**): void

Anexa informações do iterador.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O novo iterador a ser anexado.

`info`

As informações associativas para o iterador, que deve ser um int, um string, ou **`null`**.

Valor Retornado

Descrição...

Erros/Exceções

Uma **IllegalValueException** se o parâmetro `iterator` for inválido, ou se `info` já estiver associado a informações.

Veja também

- MultipleIterator::__construct() - Constrói um novo MultipleIterator

MultipleIterator::__construct

MultipleIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::__construct — Constrói um novo MultipleIterator

Descrição

public function **MultipleIterator::__construct**(int `$flags` = MultipleIterator::MIT_NEED_ALL | MultipleIterator::MIT_KEYS_NUMERIC)

Constrói um novo MultipleIterator.

Parâmetros

`flags`

As opções a serem definidas, de acordo com as Constantes de opção.

- **`MultipleIterator::MIT_NEED_ALL`** ou **`MultipleIterator::MIT_NEED_ANY`**
- **`MultipleIterator::MIT_KEYS_NUMERIC`** ou **`MultipleIterator::MIT_KEYS_ASSOC`**

O padrão é **`MultipleIterator::MIT_NEED_ALL`**|**`MultipleIterator::MIT_KEYS_NUMERIC`**.

Examples

Exemplo #1 Iterando um MultipleIterator

```
<?php
$people = new ArrayIterator(array('John', 'Jane', 'Jack', 'Judy'));
```

```php
$roles  = new ArrayIterator(array('Developer', 'Scrum Master', 'Project Owner'));

$team = new MultipleIterator($flags);
$team->attachIterator($people, 'person');
$team->attachIterator($roles, 'role');

foreach ($team as $member) {
    print_r($member);
}
?>
```

Saída com `$flags = MIT_NEED_ALL|MIT_KEYS_NUMERIC`

```
Array
(
    [0] => John
    [1] => Developer
)
Array
(
    [0] => Jane
    [1] => Scrum Master
)
Array
(
    [0] => Jack
    [1] => Project Owner
)
```

Saída com `$flags = MIT_NEED_ANY|MIT_KEYS_NUMERIC`

```
Array
(
    [0] => John
    [1] => Developer
)
Array
(
    [0] => Jane
    [1] => Scrum Master
)
Array
(
    [0] => Jack
    [1] => Project Owner
)
Array
(
    [0] => Judy
    [1] =>
)
```

Saída com `$flags = MIT_NEED_ALL|MIT_KEYS_ASSOC`

```
Array
(
    [person] => John
```

```
    [role] => Developer
)
Array
(
    [person] => Jane
    [role] => Scrum Master
)
Array
(
    [person] => Jack
    [role] => Project Owner
)
```

Saída com `$flags = MIT_NEED_ANY|MIT_KEYS_ASSOC`

```
Array
(
    [person] => John
    [role] => Developer
)
Array
(
    [person] => Jane
    [role] => Scrum Master
)
Array
(
    [person] => Jack
    [role] => Project Owner
)
Array
(
    [person] => Judy
    [role] =>
)
```

Veja também

- Constantes de Opção
- MultipleIterator::valid() - Verifica a validade dos subiteradores

MultipleIterator::containsIterator

MultipleIterator::containsIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::containsIterator — Verifica se um iterador está anexado

Descrição

public function **MultipleIterator::containsIterator**(Iterator `$iterator`): bool

Verifica se um iterador está anexado ou não.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O iterador a ser verificado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- MultipleIterator::valid() - Verifica a validade dos subiteradores

MultipleIterator::countIterators

MultipleIterator::countIterators

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::countIterators — Obtém o número de instâncias de iteradores anexadas

Descrição

public function **MultipleIterator::countIterators**(): int

Obtém o número de instâncias de iteradores anexadas.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de instâncias de iteradores anexadas (como um int).

Veja também

- MultipleIterator::containsIterator() - Verifica se um iterador está anexado

MultipleIterator::current

MultipleIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::current — Obtém as instâncias do iterador registradas

Descrição

public function **MultipleIterator::current**(): array

Obtém o resultado current() das instâncias do iterador registradas.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array contendo os valores atuais de cada iterador anexado.

Erros/Exceções

Uma RuntimeException é lançada se o iterador for inválido (a partir do PHP 8.1.0), ou se o modo `MIT_NEED_ALL` estiver definido e pelo menos um iterador anexado não for válido. Ou uma **IllegalValueException** se uma chave for `null` e `MIT_KEYS_ASSOC` estiver definido.

Registro de Alterações

Versão	Descrição
8.1.0	Uma RuntimeException agora é lançada se **MultipleIterator::current()** for chamado em um iterador inválido. Anteriormente, `false` era retornado.

Veja também

- MultipleIterator::valid() - Verifica a validade dos subiteradores

MultipleIterator::detachIterator

MultipleIterator::detachIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::detachIterator — Desanexa um iterador

Descrição

public function **MultipleIterator::detachIterator**(Iterator `$iterator`): void

Desanexa um iterador.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O iterador a ser desanexado.

Valor Retornado

Nenhum valor é retornado.

Veja também

- MultipleIterator::attachIterator() - Anexa informações do iterador

MultipleIterator::getFlags

MultipleIterator::getFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::getFlags — Obtém informações sobre as opções

Descrição

public function **MultipleIterator::getFlags**(): int

Obtém informações sobre as opções.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Informações sobre as opções, como um int.

Veja também

- Constantes de Opção
- MultipleIterator::__construct() - Constrói um novo MultipleIterator
- MultipleIterator::setFlags() - Define opções

MultipleIterator::key

MultipleIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::key — Obtém as instâncias do iterador registradas

Descrição

public function **MultipleIterator::key**(): array

Obtém o resultado da função key() das instâncias do iterador registradas.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de todas as instâncias do iterador registradas.

Erros/Exceções

Um RuntimeException é lançado se o iterador for inválido (a partir do PHP 8.1.0), ou o modo `MIT_NEED_ALL` estiver definido e pelo menos um iterador anexado não for válido.

Chamando este método de foreach aciona o aviso "Illegal type returned".

Registro de Alterações

Versão	Descrição
8.1.0	Um RuntimeException agora é lançado se **MultipleIterator::key()** for chamado em um iterador inválido. Anteriormente, retornava `false`.

Veja também

- MultipleIterator::current() - Obtém as instâncias do iterador registradas

MultipleIterator::next

MultipleIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::next — Movimenta todos os subiteradores anexados para frente

Descrição

public function **MultipleIterator::next**(): void

Movimenta todos os subiteradores anexados para frente.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- MultipleIterator::rewind() - Rebobina todas as instâncias do iterador anexado

MultipleIterator::rewind

MultipleIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::rewind — Rebobina todas as instâncias do iterador anexado

Descrição

public function **MultipleIterator::rewind**(): void

Rebobina todas as instâncias do iterador anexado.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- MultipleIterator::next() - Movimenta todos os subiteradores anexados para frente

MultipleIterator::setFlags

MultipleIterator::setFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::setFlags — Define opções

Descrição

public function **MultipleIterator::setFlags**(int `$flags`): void

Define opções.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`flags`

As opções a serem definidas, de acordo com Constantes de Opção

Valor Retornado

Nenhum valor é retornado.

Veja também

- Constantes de Opção
- MultipleIterator::__construct() - Constrói um novo MultipleIterator
- MultipleIterator::getFlags() - Obtém informações sobre as opções

MultipleIterator::valid

MultipleIterator::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::valid — Verifica a validade dos subiteradores

Descrição

public function **MultipleIterator::valid**(): bool

Verifica a validade dos subiteradores.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se um ou todos os subiteradores forem válidos, dependendo das opções (flags), caso contrário, retorna `false`

Veja também

- MultipleIterator::__construct() - Constrói um novo MultipleIterator

NoRewindIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este iterador ignora operações de rebobinamento. Isso permite processar um iterador em vários loops foreach parciais.

Resumo da classe

class **NoRewindIterator** extends IteratorIterator {
/* Métodos */
public function __construct(Iterator `$iterator`)
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
/* Métodos herdados */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

NoRewindIterator::__construct

NoRewindIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::__construct — Constrói um NoRewindIterator

Descrição

public function **NoRewindIterator::__construct**(Iterator `$iterator`)

Constrói um NoRewindIterator.

Parâmetros

`iterator`

O iterador sendo usado.

Examples

Exemplo #1 Exemplo de NoRewindIterator::__construct()

O segundo loop não produz saída porque o iterador é usado apenas uma vez, já que não rebobina.

```
<?php
$fruit = array('maçã', 'banana', 'oxicoco');

$arr = new ArrayObject($fruit);
$it  = new NoRewindIterator($arr->getIterator());

echo "Fruta A:\n";
foreach( $it as $item ) {
    echo $item . "\n";
}

echo "Fruta B:\n";
foreach( $it as $item ) {
    echo $item . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Fruta A:
maçã
banana
oxicoco
Fruta B:
```

Veja também

- NoRewindIterator::valid() - Valida o iterador

NoRewindIterator::current

NoRewindIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::current — Obtém o valor atual

Descrição

public function **NoRewindIterator::current**(): mixed

Obtém o valor atual.

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor atual.

Veja também

- NoRewindIterator::key() - Obtém a chave atual

NoRewindIterator::key

NoRewindIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::key — Obtém a chave atual

Descrição

public function **NoRewindIterator::key**(): mixed

Obtém a chave atual.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A chave atual.

Veja também

- NoRewindIterator::next() - Avança para o próximo elemento

NoRewindIterator::next

NoRewindIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::next — Avança para o próximo elemento

Descrição

public function **NoRewindIterator::next**(): void

Avança para o próximo elemento.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Veja também

- NoRewindIterator::rewind() - Impede a operação de rebobinar no iterador interno

NoRewindIterator::rewind

NoRewindIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::rewind — Impede a operação de rebobinar no iterador interno

Descrição

public function **NoRewindIterator::rewind**(): void

Impede a operação de rebobinar no iterador interno.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de NoRewindIterator::rewind()

Este exemplo demonstra que chamando rewind em um objeto NoRewindIterator não tem efeito.

```php
<?php
$fruits = array("limão", "laranja", "maçã", "pera");

$noRewindIterator = new NoRewindIterator(new ArrayIterator($fruits));

echo $noRewindIterator->current() . "\n";
$noRewindIterator->next();
// agora rebobina o iterador (nada deve acontecer)
$noRewindIterator->rewind();
echo $noRewindIterator->current() . "\n";
?>
```

O exemplo acima produzirá:

```
limão
laranja
```

NoRewindIterator::valid

NoRewindIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::valid — Valida o iterador

Descrição

public function **NoRewindIterator::valid**(): bool

Verifica se o iterador é válido.

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- **NoRewindIterator::getInnerIterator()**

ParentIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este FilterIterator estendido permite uma iteração recursiva usando RecursiveIteratorIterator que mostra apenas os elementos que têm filhos.

Resumo da classe

class **ParentIterator** extends RecursiveFilterIterator {
/* Métodos */
public function __construct(RecursiveIterator `$iterator`)
public function accept(): bool
public function getChildren(): ParentIterator
public function hasChildren(): bool
public function next(): void
public function rewind(): void
/* Métodos herdados */
public function RecursiveFilterIterator::getChildren(): ?RecursiveFilterIterator
public function RecursiveFilterIterator::hasChildren(): bool
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

ParentIterator::accept

ParentIterator::accept

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::accept — Determina a aceitabilidade

Descrição

public function **ParentIterator::accept**(): bool

Determina se o elemento atual tem filhos.

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o elemento atual for aceitável, caso contrário, `false`.

Veja também

- ParentIterator::hasChildren() - Verifica se o elemento atual do iterador interno tem ou não filhos
- FilterIterator::accept() - Verifica se o elemento atual do iterador é aceitável

ParentIterator::__construct

ParentIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::__construct — Constrói um ParentIterator

Descrição

public function **ParentIterator::__construct**(RecursiveIterator `$iterator`)

Constrói um ParentIterator em um iterador.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O iterador no qual está sendo construído.

ParentIterator::getChildren

ParentIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::getChildren — Retorna os filhos do iterador interno de um ParentIterator

Descrição

public function **ParentIterator::getChildren**(): ParentIterator

Retorna os filhos do iterador interno contidos em um ParentIterator.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um object.

ParentIterator::hasChildren

ParentIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::hasChildren — Verifica se o elemento atual do iterador interno tem ou não filhos

Descrição

public function **ParentIterator::hasChildren**(): bool

Verifica se o elemento atual do iterador interno tem filhos.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

ParentIterator::next

ParentIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::next — Avança o iterador

Descrição

public function **ParentIterator::next**(): void

Avança o iterador.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

ParentIterator::rewind

ParentIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::rewind — Recomeça o iterador

Descrição

public function **ParentIterator::rewind**(): void

Recomeça o iterador.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveArrayIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este iterador permite desfazer e modificar valores e chaves enquanto itera sobre arrays e objetos, da mesma forma que o ArrayIterator. Além disso, é possível iterar sobre a entrada atual do iterador.

Resumo da classe

class **RecursiveArrayIterator** extends ArrayIterator implements RecursiveIterator {
/* Constantes herdadas */
public const int *ArrayIterator::STD_PROP_LIST*;
public const int *ArrayIterator::ARRAY_AS_PROPS*;
/* Constantes */
public const int *CHILD_ARRAYS_ONLY*;
/* Métodos */
public function getChildren(): ?RecursiveArrayIterator
public function hasChildren(): bool
/* Métodos herdados */
public function ArrayIterator::__construct(array|object `$array` = [], int `$flags` = 0)
public function ArrayIterator::append(mixed `$value`): void
public function ArrayIterator::asort(int `$flags` = **`SORT_REGULAR`**): true
public function ArrayIterator::count(): int
public function ArrayIterator::current(): mixed
public function ArrayIterator::getArrayCopy(): array
public function ArrayIterator::getFlags(): int
public function ArrayIterator::key(): string|int|null
public function ArrayIterator::ksort(int `$flags` = **`SORT_REGULAR`**): true
public function ArrayIterator::natcasesort(): true
public function ArrayIterator::natsort(): true
public function ArrayIterator::next(): void
public function ArrayIterator::offsetExists(mixed `$key`): bool
public function ArrayIterator::offsetGet(mixed `$key`): mixed
public function ArrayIterator::offsetSet(mixed `$key`, mixed `$value`): void
public function ArrayIterator::offsetUnset(mixed `$key`): void
public function ArrayIterator::rewind(): void
public function ArrayIterator::seek(int `$offset`): void
public function ArrayIterator::serialize(): string
public function ArrayIterator::setFlags(int `$flags`): void
public function ArrayIterator::uasort(callable `$callback`): true
public function ArrayIterator::uksort(callable `$callback`): true
public function ArrayIterator::unserialize(string `$data`): void
public function ArrayIterator::valid(): bool
}

Constantes predefinidas

Flags do RecursiveArrayIterator

`RecursiveArrayIterator::CHILD_ARRAYS_ONLY`

Trata apenas arrays (não objetos) como tendo filhos para iteração recursiva.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

RecursiveArrayIterator::getChildren

RecursiveArrayIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveArrayIterator::getChildren — Retorna um iterador para a entrada atual se ela for um array ou um object

Descrição

public function **RecursiveArrayIterator::getChildren**(): ?RecursiveArrayIterator

Retorna um iterador para a entrada atual do iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um iterador para a entrada atual, se ela for um array ou um object; ou `null` em caso de falha.

Erros/Exceções

Uma InvalidArgumentException será lançada se a entrada atual não contiver um array ou um object.

Examples

Exemplo #1 Exemplo de RecursiveArrayIterator::getChildren()

```php
<?php
$fruits = array("a" => "limão", "b" => "laranja", array("a" => "maçã", "p" => "pêra"));

$iterator = new RecursiveArrayIterator($fruits);

while ($iterator->valid()) {

    if ($iterator->hasChildren()) {
        // imprime todos os filhos
        foreach ($iterator->getChildren() as $key => $value) {
            echo $key . ' : ' . $value . "\n";
        }
    } else {
        echo "Sem filhos.\n";
    }

    $iterator->next();
}
?>
```

O exemplo acima produzirá:

```
Sem filhos.
Sem filhos.
a : maçã
p : pêra
```

Veja também

- RecursiveArrayIterator::hasChildren() - Retorna se a entrada atual é um array ou um objeto

RecursiveArrayIterator::hasChildren

RecursiveArrayIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveArrayIterator::hasChildren — Retorna se a entrada atual é um array ou um objeto

Descrição

public function **RecursiveArrayIterator::hasChildren**(): bool

Retorna se a entrada atual é um array ou um object para o qual um iterador pode ser obtido via RecursiveArrayIterator::getChildren().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a entrada atual é um array ou um object, caso contrário, `false` é retornado.

Examples

Exemplo #1 Exemplo de RecursiveArrayIterator::hasChildren()

```php
<?php
$fruits = array("a" => "limão", "b" => "laranja", array("a" => "maçã", "p" => "pêra"));

$iterator = new RecursiveArrayIterator($fruits);

while ($iterator->valid()) {

    // Verifica se há filhos
    if ($iterator->hasChildren()) {
        // imprime todos os filhos
        foreach ($iterator->getChildren() as $key => $value) {
            echo $key . ' : ' . $value . "\n";
        }
    } else {
        echo "Sem filhos.\n";
    }

    $iterator->next();
}
?>
```

O exemplo acima produzirá:

```
Sem filhos.
Sem filhos.
a : maçã
p : pêra
```

Veja também

- RecursiveArrayIterator::getChildren() - Retorna um iterador para a entrada atual se ela for um array ou um object

RecursiveCachingIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

...

Resumo da classe

class **RecursiveCachingIterator** extends CachingIterator implements RecursiveIterator {

```
/* Constantes herdadas */
public const int CachingIterator::CALL_TOSTRING;
public const int CachingIterator::CATCH_GET_CHILD;
public const int CachingIterator::TOSTRING_USE_KEY;
public const int CachingIterator::TOSTRING_USE_CURRENT;
public const int CachingIterator::TOSTRING_USE_INNER;
public const int CachingIterator::FULL_CACHE;
/* Métodos */
public function __construct(Iterator $iterator, int $flags = RecursiveCachingIterator::CALL_TOSTRING)
public function getChildren(): ?RecursiveCachingIterator
public function hasChildren(): bool
/* Métodos herdados */
public function CachingIterator::count(): int
public function CachingIterator::current(): mixed
public function CachingIterator::getCache(): array
public function CachingIterator::getFlags(): int
public function CachingIterator::hasNext(): bool
public function CachingIterator::key(): scalar
public function CachingIterator::next(): void
public function CachingIterator::offsetExists(string $key): bool
public function CachingIterator::offsetGet(string $key): mixed
public function CachingIterator::offsetSet(string $key, mixed $value): void
public function CachingIterator::offsetUnset(string $key): void
public function CachingIterator::rewind(): void
public function CachingIterator::setFlags(int $flags): void
public function CachingIterator::__toString(): string
public function CachingIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}
```

Índice

RecursiveCachingIterator::__construct

RecursiveCachingIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::__construct — Construtor

Descrição

public function **RecursiveCachingIterator::__construct**(Iterator `$iterator`, int `$flags` = **`RecursiveCachingIterator::CALL_TOSTRING`**)

Constrói um novo RecursiveCachingIterator, que consiste em um `iterator` passado.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O iterador sendo usado.

`flags`

As opções. Use **`CALL_TOSTRING`** para chamar **RecursiveCachingIterator::__toString()** para cada elemento (o padrão), e/ou **`CATCH_GET_CHILD`** para capturar exceções ao tentar obter filhos.

Veja também

- CachingIterator::__construct() - Constrói um novo objeto CachingIterator para o iterador

RecursiveCachingIterator::getChildren

RecursiveCachingIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::getChildren — Retorna os filhos do iterador interno como um RecursiveCachingIterator

Descrição

public function **RecursiveCachingIterator::getChildren**(): ?RecursiveCachingIterator

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Os filhos do iterador interno, como um RecursiveCachingIterator; ou `null` se não houver filhos.

RecursiveCachingIterator::hasChildren

RecursiveCachingIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::hasChildren — Verifica se o elemento atual do iterador interno possui filhos

Descrição

public function **RecursiveCachingIterator::hasChildren**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o iterador interno tiver filhos, caso contrário, retorna `false`

RecursiveCallbackFilterIterator

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introdução

Resumo da classe

class **RecursiveCallbackFilterIterator** extends CallbackFilterIterator implements RecursiveIterator {
/* Métodos */
public function __construct(RecursiveIterator `$iterator`, callable `$callback`)
public function getChildren(): RecursiveCallbackFilterIterator
public function hasChildren(): bool
/* Métodos herdados */
public function CallbackFilterIterator::accept(): bool
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed

public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Examples

O callback deve aceitar até três argumentos: o item atual, a chave atual e o iterador, respectivamente.

Exemplo #1 Argumentos disponíveis para o callback

```php
<?php

/**
 * Callback for RecursiveCallbackFilterIterator
 *
 * @param $current   Valor atual do item
 * @param $key       Chave atual do item
 * @param $iterator  Iterador sendo filtrado
 * @return boolean   TRUE para aceitar o item atual, FALSE caso contrário
 */
function my_callback($current, $key, $iterator) {
    // Seu código de filtragem aqui
}

?>
```

Filtrar um iterador recursivo geralmente envolve duas condições. A primeira é que, para permitir a recursão, a função de retorno de chamada deve retornar `true` se o item do iterador atual tiver filhos. O segundo é a condição de filtro normal, como um tamanho de arquivo ou extensão verificação como no exemplo abaixo.

Exemplo #2 Exemplo básico de chamada recursiva

```php
<?php

$dir = new RecursiveDirectoryIterator(__DIR__);

// Filtrar arquivos grandes ( > 100MB)
$files = new RecursiveCallbackFilterIterator($dir, function ($current, $key, $iterator) {
    // Permitir recursão
    if ($iterator->hasChildren()) {
        return TRUE;
    }
    // Verificar se é um arquivo grande
    if ($current->isFile() && $current->getSize() > 104857600) {
        return TRUE;
    }
    return FALSE;
});

foreach (new RecursiveIteratorIterator($files) as $file) {
    echo $file->getPathname() . PHP_EOL;
}

?>
```

Índice

RecursiveCallbackFilterIterator::__construct

RecursiveCallbackFilterIterator::__construct

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::__construct — Cria um RecursiveCallbackFilterIterator a partir de um RecursiveIterator

Descrição

public function **RecursiveCallbackFilterIterator::__construct**(RecursiveIterator `$iterator`, callable `$callback`)

Cria um iterador filtrado a partir de um RecursiveIterator usando o `callback` para determinar quais itens são aceitos ou rejeitados.

Parâmetros

`iterator`

O iterador recursivo a ser filtrado.

`callback`

O callback, que deve retornar **`true`** para aceitar o item atual ou **`false`** caso contrário. Veja Exemplos.

Pode ser qualquer valor callable válido.

Veja também

- Exemplos de RecursiveCallbackFilterIterator
- RecursiveCallbackFilterIterator::getChildren() - Retorna os filhos do iterador interno contidos em um RecursiveCallbackFilterIterator
- RecursiveCallbackFilterIterator::hasChildren() - Verifica se o elemento atual do iterador interno possui filhos

RecursiveCallbackFilterIterator::getChildren

RecursiveCallbackFilterIterator::getChildren

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::getChildren — Retorna os filhos do iterador interno contidos em um RecursiveCallbackFilterIterator

Descrição

public function **RecursiveCallbackFilterIterator::getChildren**(): RecursiveCallbackFilterIterator

Recupera os filhos filtrados do iterador interno.

RecursiveCallbackFilterIterator::hasChildren() deve ser usado para determinar se existem filhos a serem recuperados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um RecursiveCallbackFilterIterator contendo os filhos.

Veja também

- Exemplos de RecursiveCallbackFilterIterator
- RecursiveCallbackFilterIterator::__construct() - Cria um RecursiveCallbackFilterIterator a partir de um RecursiveIterator
- RecursiveCallbackFilterIterator::hasChildren() - Verifica se o elemento atual do iterador interno possui filhos

RecursiveCallbackFilterIterator::hasChildren

RecursiveCallbackFilterIterator::hasChildren

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::hasChildren — Verifica se o elemento atual do iterador interno possui filhos

Descrição

public function **RecursiveCallbackFilterIterator::hasChildren**(): bool

Retorna **`true`** se o elemento atual tiver filhos, **`false`** caso contrário.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o elemento atual tiver filhos, `false` caso contrário.

Examples

Exemplo #1 Uso básico de RecursiveCallbackFilterIterator::hasChildren()

```php
<?php

$dir = new RecursiveDirectoryIterator(__DIR__);

// Itera recursivamente em arquivos XML
$files = new RecursiveCallbackFilterIterator($dir, function ($current, $key, $iterator) {
    // Permite recursão em diretórios
    if ($iterator->hasChildren()) {
        return TRUE;
    }
    // Verifique o arquivo XML
    if (!strcasecmp($current->getExtension(), 'xml')) {
        return TRUE;
    }
    return FALSE;
});

?>
```

Veja também

- Exemplos de RecursiveCallbackFilterIterator
- RecursiveCallbackFilterIterator::__construct() - Cria um RecursiveCallbackFilterIterator a partir de um RecursiveIterator
- RecursiveCallbackFilterIterator::getChildren() - Retorna os filhos do iterador interno contidos em um RecursiveCallbackFilterIterator

RecursiveDirectoryIterator

(PHP 5, PHP 7, PHP 8)

Introdução

O **RecursiveDirectoryIterator** fornece uma interface para iterar de forma recursiva sobre diretórios do sistema de arquivos.

Resumo da classe

class **RecursiveDirectoryIterator** extends FilesystemIterator implements RecursiveIterator {
/* Constantes herdadas */
public const int *FilesystemIterator::CURRENT_MODE_MASK*;
public const int *FilesystemIterator::CURRENT_AS_PATHNAME*;
public const int *FilesystemIterator::CURRENT_AS_FILEINFO*;
public const int *FilesystemIterator::CURRENT_AS_SELF*;
public const int *FilesystemIterator::KEY_MODE_MASK*;
public const int *FilesystemIterator::KEY_AS_PATHNAME*;
public const int *FilesystemIterator::FOLLOW_SYMLINKS*;
public const int *FilesystemIterator::KEY_AS_FILENAME*;
public const int *FilesystemIterator::NEW_CURRENT_AND_KEY*;
public const int *FilesystemIterator::OTHER_MODE_MASK*;
public const int *FilesystemIterator::SKIP_DOTS*;
public const int *FilesystemIterator::UNIX_PATHS*;
/* Métodos */
public function __construct(string `$directory`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO)
public function getChildren(): RecursiveDirectoryIterator
public function getSubPath(): string
public function getSubPathname(): string
public function hasChildren(bool `$allowLinks` = `false`): bool

```
public function key(): string
public function next(): void
public function rewind(): void
/* Métodos herdados */
public function FilesystemIterator::current(): string|SplFileInfo|FilesystemIterator
public function FilesystemIterator::getFlags(): int
public function FilesystemIterator::key(): string
public function FilesystemIterator::next(): void
public function FilesystemIterator::rewind(): void
public function FilesystemIterator::setFlags(int $flags): void
public function DirectoryIterator::current(): mixed
public function DirectoryIterator::getBasename(string $suffix = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int $offset): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}
```

Índice

RecursiveDirectoryIterator::__construct

RecursiveDirectoryIterator::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

RecursiveDirectoryIterator::__construct — Constrói um RecursiveDirectoryIterator

Descrição

public function **RecursiveDirectoryIterator::__construct**(string `$directory`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO)

Constrói um **RecursiveDirectoryIterator()** para o `directory` fornecido.

Parâmetros

`directory`

O caminho do diretório a ser iterado.

`flags`

As opções (flags) podem ser fornecidas e afetarão o comportamento de alguns métodos. Uma lista das opções (flags) pode ser encontrada em Constantes predefinidas de FilesystemIterator. Elas também podem ser definidas posteriormente com FilesystemIterator::setFlags().

Erros/Exceções

Lança uma UnexpectedValueException se o `directory` não existir.

Lança um ValueError se o `directory` for uma string vazia.

Registro de Alterações

Versão	Descrição
8.0.0	Agora lança um ValueError se `directory` for uma string vazia; anteriormente lançava um RuntimeException.

Examples

Exemplo #1 Exemplo de RecursiveDirectoryIterator

```php
<?php

$directory = '/tmp';

$it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

$it->rewind();
while($it->valid()) {

    if (!$it->isDot()) {
        echo 'SubPathName: ' . $it->getSubPathName() . "\n";
        echo 'SubPath:     ' . $it->getSubPath() . "\n";
        echo 'Key:         ' . $it->key() . "\n\n";
    }

    $it->next();
}

?>
```

O exemplo acima produzirá algo semelhante a:

```
SubPathName: fruit/apple.xml
SubPath:     fruit
Key:         /tmp/fruit/apple.xml

SubPathName: stuff.xml
SubPath:
Key:         /tmp/stuff.xml

SubPathName: veggies/carrot.xml
SubPath:     veggies
Key:         /tmp/veggies/carrot.xml
```

Veja também

- FilesystemIterator::__construct() - Constrói um novo iterador de sistema de arquivos
- RecursiveIteratorIterator::__construct() - Constrói um RecursiveIteratorIterator
- Constantes predefinidas de FilesystemIterator

RecursiveDirectoryIterator::getChildren

RecursiveDirectoryIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getChildren — Retorna um iterador para o elemento atual se ele for um diretório

Descrição

public function **RecursiveDirectoryIterator::getChildren**(): RecursiveDirectoryIterator

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome do arquivo, informações do arquivo ou $this dependendo das flags definidas. Veja o FilesystemIterator constantes.

RecursiveDirectoryIterator::getSubPath

RecursiveDirectoryIterator::getSubPath

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getSubPath — Obtém o subcaminho

Descrição

public function **RecursiveDirectoryIterator::getSubPath**(): string

Retorna o subcaminho relativo ao diretório fornecido no construtor.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O subcaminho.

Examples

Exemplo #1 Exemplo de getSubPath()

```
$directory = '/tmp';

      $it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

      foreach ($it as $file) {
          echo 'SubPathName: ' . $it->getSubPathName() . "\n";
          echo 'SubPath:     ' . $it->getSubPath() . "\n\n";
      }
```

O exemplo acima produzirá algo semelhante a:

```
SubPathName: fruit/apple.xml
     SubPath:     fruit

     SubPathName: stuff.xml
     SubPath:

     SubPathName: veggies/carrot.xml
     SubPath:     veggies
```

Veja também

- RecursiveDirectoryIterator::getSubPathName() - Obtém o subcaminho e o nome
- RecursiveDirectoryIterator::key() - Retorna o caminho e o nome do arquivo do elemento atual do diretório

RecursiveDirectoryIterator::getSubPathname

RecursiveDirectoryIterator::getSubPathname

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getSubPathname — Obtém o subcaminho e o nome

Descrição

public function **RecursiveDirectoryIterator::getSubPathname**(): string

Obtém o subcaminho e o nome do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O subcaminho (subdiretório) e o nome do arquivo.

Examples

Exemplo #1 Exemplo de **getSubPathname()**

```
$directory = '/tmp';

      $it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

      foreach ($it as $file) {
          echo 'SubPathName: ' . $it->getSubPathname() . "\n";
          echo 'SubPath:     ' . $it->getSubPath() . "\n\n";
      }
```

O exemplo acima produzirá algo semelhante a:

```
SubPathName: fruit/apple.xml
     SubPath:     fruit

     SubPathName: stuff.xml
     SubPath:

     SubPathName: veggies/carrot.xml
     SubPath:     veggies
```

Veja também

- RecursiveDirectoryIterator::getSubPath() - Obtém o subcaminho
- RecursiveDirectoryIterator::key() - Retorna o caminho e o nome do arquivo do elemento atual do diretório

RecursiveDirectoryIterator::hasChildren

RecursiveDirectoryIterator::hasChildren

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::hasChildren — Retorna se o elemento atual é ou não um diretório que não seja '.' nem '..'

Descrição

public function **RecursiveDirectoryIterator::hasChildren**(bool `$allowLinks = false`): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`allowLinks`

Valor Retornado

Retorna se o atual elemento é um diretório, mas não '.' ou '..'

RecursiveDirectoryIterator::key

RecursiveDirectoryIterator::key

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::key — Retorna o caminho e o nome do arquivo do elemento atual do diretório

Descrição

public function **RecursiveDirectoryIterator::key**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O caminho e nome do arquivo do elemento atual do diretório.

RecursiveDirectoryIterator::next

RecursiveDirectoryIterator::next

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::next — Avança para o próximo elemento

Descrição

public function **RecursiveDirectoryIterator::next**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveDirectoryIterator::rewind

RecursiveDirectoryIterator::rewind

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::rewind — Recomeça a iteração do diretório

Descrição

public function **RecursiveDirectoryIterator::rewind**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveFilterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Este iterador abstrato filtra valores indesejados para um RecursiveIterator. Esta classe deve ser estendida para implementar filtros personalizados. O **RecursiveFilterIterator::accept()** deve ser implementado na subclasse.

Resumo da classe

abstract class **RecursiveFilterIterator** extends FilterIterator implements RecursiveIterator {
/* Métodos */
public function __construct(RecursiveIterator `$iterator`)
public function getChildren(): ?RecursiveFilterIterator
public function hasChildren(): bool
/* Métodos herdados */
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Índice

RecursiveFilterIterator::__construct

RecursiveFilterIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::__construct — Cria um RecursiveFilterIterator a partir de um RecursiveIterator

Descrição

public function **RecursiveFilterIterator::__construct**(RecursiveIterator `$iterator`)

Cria um RecursiveFilterIterator a partir de um RecursiveIterator.

Parâmetros

`iterator`

O RecursiveIterator a ser filtrado.

Examples

Exemplo #1 Exemplo básico de **RecursiveFilterIterator()**

```php
<?php
class TestsOnlyFilter extends RecursiveFilterIterator {
    public function accept() {
        // Aceita o item atual se pudermos percorrer ele
        // ou é um valor começando com "test"
        return $this->hasChildren() || (strpos($this->current(), "test") !== FALSE);
    }
}

$array    = array("test1", array("taste2", "test3", "test4"), "test5");
$iterator = new RecursiveArrayIterator($array);
$filter   = new TestsOnlyFilter($iterator);

foreach(new RecursiveIteratorIterator($filter) as $key => $value)
{
    echo $value . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
test1
test3
test4
test5
```

Exemplo #2 Exemplo de **RecursiveFilterIterator()**

```php
<?php
class StartsWithFilter extends RecursiveFilterIterator {

    protected $word;

    public function __construct(RecursiveIterator $rit, $word) {
        $this->word = $word;
        parent::__construct($rit);
    }

    public function accept() {
        return $this->hasChildren() OR strpos($this->current(), $this->word) === 0;
    }

    public function getChildren() {
        return new self($this->getInnerIterator()->getChildren(), $this->word);
    }
}

$array    = array("test1", array("taste2", "test3", "test4"), "test5");
$iterator = new RecursiveArrayIterator($array);
```

```php
$filter   = new StartsWithFilter($iterator, "test");

foreach(new RecursiveIteratorIterator($filter) as $key => $value)
{
    echo $value . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
test1
test3
test4
test5
```

Veja também

- RecursiveFilterIterator::getChildren() - Retorna os filhos do iterador interno contidos em um RecursiveFilterIterator
- RecursiveFilterIterator::hasChildren() - Verifica se o elemento atual do iterador interno tem filhos
- FilterIterator::accept() - Verifica se o elemento atual do iterador é aceitável

RecursiveFilterIterator::getChildren

RecursiveFilterIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::getChildren — Retorna os filhos do iterador interno contidos em um RecursiveFilterIterator

Descrição

public function **RecursiveFilterIterator::getChildren**(): ?RecursiveFilterIterator

Retorna os filhos do iterador interno contidos em um RecursiveFilterIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um RecursiveFilterIterator contendo os filhos do iterador interno.

Veja também

- RecursiveFilterIterator::hasChildren() - Verifica se o elemento atual do iterador interno tem filhos
- RecursiveIterator::getChildren() - Retorna um iterador para a entrada atual

RecursiveFilterIterator::hasChildren

RecursiveFilterIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::hasChildren — Verifica se o elemento atual do iterador interno tem filhos

Descrição

public function **RecursiveFilterIterator::hasChildren**(): bool

Verifica se o elemento atual do iterador interno tem filhos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o iterador interno tiver filhos, caso contrário `false`

Veja também

- RecursiveFilterIterator::getChildren() - Retorna os filhos do iterador interno contidos em um RecursiveFilterIterator
- RecursiveIterator::hasChildren() - Retorna se um iterador pode ser criado para a entrada atual

RecursiveIteratorIterator

(PHP 5, PHP 7, PHP 8)

Introdução

Pode ser utilizado para percorrer interatores recursivos.

Resumo da classe

class **RecursiveIteratorIterator** implements OuterIterator {
/* Constantes */
public const int *LEAVES_ONLY*;
public const int *SELF_FIRST*;
public const int *CHILD_FIRST*;
public const int *CATCH_GET_CHILD*;
/* Métodos */
public function __construct(Traversable `$iterator`, int `$mode` = `RecursiveIteratorIterator::LEAVES_ONLY`, int `$flags` = 0)
public function beginChildren(): void
public function beginIteration(): void
public function callGetChildren(): ?RecursiveIterator
public function callHasChildren(): bool
public function current(): mixed
public function endChildren(): void
public function endIteration(): void
public function getDepth(): int
public function getInnerIterator(): RecursiveIterator
public function getMaxDepth(): int|false
public function getSubIterator(?int `$level` = `null`): ?RecursiveIterator
public function key(): mixed
public function next(): void
public function nextElement(): void
public function rewind(): void
public function setMaxDepth(int `$maxDepth` = -1): void
public function valid(): bool
}

Constantes predefinidas

`RecursiveIteratorIterator::LEAVES_ONLY`

`RecursiveIteratorIterator::SELF_FIRST`

`RecursiveIteratorIterator::CHILD_FIRST`

`RecursiveIteratorIterator::CATCH_GET_CHILD`

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

RecursiveIteratorIterator::beginChildren

RecursiveIteratorIterator::beginChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::beginChildren — Inicia filhos

Descrição

public function **RecursiveIteratorIterator::beginChildren**(): void

É chamado após chamar **RecursiveIteratorIterator::getChildren()**, e seu associado RecursiveIteratorIterator::rewind().

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::beginIteration

RecursiveIteratorIterator::beginIteration

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::beginIteration — Inicia Iteração

Descrição

public function **RecursiveIteratorIterator::beginIteration**(): void

Chamado quando a iteração começa (após a primeira chamada de RecursiveIteratorIterator::rewind()).

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::callGetChildren

RecursiveIteratorIterator::callGetChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::callGetChildren — Obtém filhos

Descrição

public function **RecursiveIteratorIterator::callGetChildren**(): ?RecursiveIterator

Obtém os filhos do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um **RecursiveIterator()** em caso de sucesso, ou `null` em caso de falha.

RecursiveIteratorIterator::callHasChildren

RecursiveIteratorIterator::callHasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::callHasChildren — Tem filhos

Descrição

public function **RecursiveIteratorIterator::callHasChildren**(): bool

Chamado para cada elemento para testar se ele possui filhos.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna se o elemento possui filhos.

RecursiveIteratorIterator::__construct

RecursiveIteratorIterator::__construct

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

RecursiveIteratorIterator::__construct — Constrói um RecursiveIteratorIterator

Descrição

public function **RecursiveIteratorIterator::__construct**(Traversable `$iterator`, int `$mode` = `RecursiveIteratorIterator::LEAVES_ONLY`, int `$flags` = 0)

Cria um RecursiveIteratorIterator a partir de um RecursiveIterator.

Parâmetros

`iterator`

O iterador a partir do qual está sendo construído. Pode ser um RecursiveIterator ou IteratorAggregate.

`mode`

Modo opcional. Valores possíveis são

- `RecursiveIteratorIterator::LEAVES_ONLY` - O padrão. Lista apenas folhas na iteração.
- `RecursiveIteratorIterator::SELF_FIRST` - Lista folhas e pais na iteração, com os pais vindo primeiro.
- `RecursiveIteratorIterator::CHILD_FIRST` - Lista folhas e pais na iteração, com as folhas vindo primeiro.

`flags`

Opção opcional. Valores possíveis são `RecursiveIteratorIterator::CATCH_GET_CHILD`, que então ignorará exceções lançadas em chamadas para **RecursiveIteratorIterator::getChildren()**.

Examples

Exemplo #1 Iterando um RecursiveIteratorIterator

```
<?php
$array = array(
    array(
        array(
            array(
                'leaf-0-0-0-0',
                'leaf-0-0-0-1'
```

```
            ),
            'leaf-0-0-0'
        ),
        array(
            array(
                'leaf-0-1-0-0',
                'leaf-0-1-0-1'
            ),
            'leaf-0-1-0'
        ),
        'leaf-0-0'
    )
);

$iterator = new RecursiveIteratorIterator(
    new RecursiveArrayIterator($array),
    $mode
);
foreach ($iterator as $key => $leaf) {
    echo "$key => $leaf", PHP_EOL;
}
?>
```

Saída com `$mode = RecursiveIteratorIterator::LEAVES_ONLY`

```
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
0 => leaf-0-0-0
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
0 => leaf-0-1-0
0 => leaf-0-0
```

Saída com `$mode = RecursiveIteratorIterator::SELF_FIRST`

```
0 => Array
0 => Array
0 => Array
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
1 => leaf-0-0-0
1 => Array
0 => Array
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
1 => leaf-0-1-0
2 => leaf-0-0
```

Saída com `$mode = RecursiveIteratorIterator::CHILD_FIRST`

```
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
0 => Array
1 => leaf-0-0-0
0 => Array
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
```

```
0 => Array
1 => leaf-0-1-0
1 => Array
2 => leaf-0-0
0 => Array
```

RecursiveIteratorIterator::current

RecursiveIteratorIterator::current

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::current — Retorna o valor do elemento atual

Descrição

public function **RecursiveIteratorIterator::current**(): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor do elemento atual.

RecursiveIteratorIterator::endChildren

RecursiveIteratorIterator::endChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::endChildren — Encerra filhos

Descrição

public function **RecursiveIteratorIterator::endChildren**(): void

Chamado ao encerrar a recursão de um nível.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::endIteration

RecursiveIteratorIterator::endIteration

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::endIteration — Finaliza Iteração

Descrição

public function **RecursiveIteratorIterator::endIteration**(): void

Chamado quando a iteração termina (quando RecursiveIteratorIterator::valid() retorna pela primeira vez `false`).

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::getDepth

RecursiveIteratorIterator::getDepth

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::getDepth — Retorna a profundidade atual da iteração recursiva

Descrição

public function **RecursiveIteratorIterator::getDepth**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A atual profundidade da iteração recursiva.

RecursiveIteratorIterator::getInnerIterator

RecursiveIteratorIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::getInnerIterator — Obtém iterador interno

Descrição

public function **RecursiveIteratorIterator::getInnerIterator**(): RecursiveIterator

Obtém o subiterador ativo atual.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O subiterador ativo atual.

RecursiveIteratorIterator::getMaxDepth

RecursiveIteratorIterator::getMaxDepth

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::getMaxDepth — Obtém a profundidade máxima

Descrição

public function **RecursiveIteratorIterator::getMaxDepth**(): int|false

Obtém a profundidade máxima permitida.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A profundidade máxima aceita, ou `false` se qualquer profundidade for permitida.

Veja também

- RecursiveIteratorIterator::setMaxDepth() - Define profundidade máxima

RecursiveIteratorIterator::getSubIterator

RecursiveIteratorIterator::getSubIterator

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::getSubIterator — Retorna o sub-iterador ativo atual

Descrição

public function **RecursiveIteratorIterator::getSubIterator**(?int `$level` = `null`): ?RecursiveIterator

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`level`

Valor Retornado

O atual sub-iterador ativo em caso de sucesso; `null` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`level` agora é anulável

RecursiveIteratorIterator::key

RecursiveIteratorIterator::key

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::key — Retorna a chave atual

Descrição

public function **RecursiveIteratorIterator::key**(): mixed

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A atual chave.

RecursiveIteratorIterator::next

RecursiveIteratorIterator::next

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::next — Avança para o próximo elemento

Descrição

public function **RecursiveIteratorIterator::next**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::nextElement

RecursiveIteratorIterator::nextElement

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::nextElement — Próximo elemento

Descrição

public function **RecursiveIteratorIterator::nextElement**(): void

Chamado quando o próximo elemento está disponível.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::rewind

RecursiveIteratorIterator::rewind

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::rewind — Recomeça o iterador no primeiro elemento do primeiro iterador interno

Descrição

public function **RecursiveIteratorIterator::rewind**(): void

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveIteratorIterator::setMaxDepth

RecursiveIteratorIterator::setMaxDepth

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::setMaxDepth — Define profundidade máxima

Descrição

public function **RecursiveIteratorIterator::setMaxDepth**(int `$maxDepth = -1`): void

Define a profundidade máxima permitida.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`maxDepth`

A profundidade máxima permitida. `-1` é usado para qualquer profundidade.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Emite uma Exception se `maxDepth` for menor que `-1`.

RecursiveIteratorIterator::valid

RecursiveIteratorIterator::valid

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::valid — Verifica se a posição atual é válida

Descrição

public function **RecursiveIteratorIterator::valid**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a posição atual for válida, caso contrário `false`

RecursiveRegexIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introdução

Este iterador recursivo pode filtrar outro iterador recursivo por meio de uma expressão regular.

Resumo da classe

```
class RecursiveRegexIterator extends RegexIterator implements RecursiveIterator {
/* Constantes herdadas */
public const int RegexIterator::USE_KEY;
public const int RegexIterator::INVERT_MATCH;
public const int RegexIterator::MATCH;
public const int RegexIterator::GET_MATCH;
public const int RegexIterator::ALL_MATCHES;
public const int RegexIterator::SPLIT;
public const int RegexIterator::REPLACE;
/* Propriedades herdadas */
public ?string $replacement = null;
/* Métodos */
public function __construct(
    RecursiveIterator $iterator,
    string $pattern,
    int $mode = RecursiveRegexIterator::MATCH,
    int $flags = 0,
    int $pregFlags = 0
)
public function getChildren(): RecursiveRegexIterator
public function hasChildren(): bool
/* Métodos herdados */
public function RegexIterator::accept(): bool
public function RegexIterator::getFlags(): int
public function RegexIterator::getMode(): int
public function RegexIterator::getPregFlags(): int
public function RegexIterator::getRegex(): string
public function RegexIterator::setFlags(int $flags): void
public function RegexIterator::setMode(int $mode): void
public function RegexIterator::setPregFlags(int $pregFlags): void
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}
```

Índice

RecursiveRegexIterator::__construct

RecursiveRegexIterator::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::__construct — Cria um novo RecursiveRegexIterator

Descrição

```
public function RecursiveRegexIterator::__construct(
    RecursiveIterator $iterator,
    string $pattern,
```

```
    int $mode = RecursiveRegexIterator::MATCH,
    int $flags = 0,
    int $pregFlags = 0
)
```

Cria um novo iterador de expressão regular.

Parâmetros

`iterator`

O iterador recursivo ao qual este filtro de regex será aplicado.

`pattern`

A expressão regular a ser correspondida.

`mode`

Modo de operação, consulte RegexIterator::setMode() para uma lista de modos.

`flags`

Opções especiais, consulte RegexIterator::setFlags() para uma lista de opções disponíveis.

`pregFlags`

As opções de expressão regular. Essas flags dependem do parâmetro de modo de operação:

RegexIterator preg_flags

modo de operação	opções disponíveis
RecursiveRegexIterator::ALL_MATCHES	Veja preg_match_all().
RecursiveRegexIterator::GET_MATCH	Veja preg_match().
RecursiveRegexIterator::MATCH	Veja preg_match().
RecursiveRegexIterator::REPLACE	nenhum.
RecursiveRegexIterator::SPLIT	Veja preg_split().

Examples

Exemplo #1 Exemplo de RecursiveRegexIterator::__construct()

Cria um novo RegexIterator que filtra todas as strings que começam com 'test'.

```php
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $key1 => $value1) {

    if ($rRegexIterator->hasChildren()) {

        // print all children
        echo "Children: ";
        foreach ($rRegexIterator->getChildren() as $key => $value) {
            echo $value . " ";
        }
        echo "\n";
    } else {
        echo "No children\n";
    }

}
?>
```

O exemplo acima produzirá algo semelhante a:

```
No children
Children: test4 test5
```

Veja também

- preg_match() - Realiza uma correspondência com expressão regular
- preg_match_all() - Realiza uma correspondência global de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_split() - Divide uma string com base em expressão regular

RecursiveRegexIterator::getChildren

RecursiveRegexIterator::getChildren

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::getChildren — Retorna um iterador para a entrada atual

Descrição

public function **RecursiveRegexIterator::getChildren**(): RecursiveRegexIterator

Retorna um iterador para a entrada atual do iterador.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um iterador para a entrada atual, se puder ser iterado pelo iterador interno.

Erros/Exceções

Uma InvalidArgumentException será lançada se a entrada atual não contiver um valor que possa ser iterado pelo iterador interno.

Examples

Exemplo #1 Exemplo de RecursiveRegexIterator::getChildren()

```php
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $key1 => $value1) {

    if ($rRegexIterator->hasChildren()) {

        // imprime todos os filhos
        echo "Children: ";
        foreach ($rRegexIterator->getChildren() as $key => $value) {
            echo $value . " ";
        }
        echo "\n";
    } else {
        echo "No children\n";
    }

}
?>
```

O exemplo acima produzirá:

```
No children
Children: test4 test5
```

Veja também

- RecursiveRegexIterator::hasChildren() - Retorna se um iterador pode ser obtido para a entrada atual

RecursiveRegexIterator::hasChildren

RecursiveRegexIterator::hasChildren

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::hasChildren — Retorna se um iterador pode ser obtido para a entrada atual

Descrição

public function **RecursiveRegexIterator::hasChildren**(): bool

Retorna se um iterador pode ser obtido para a entrada atual. Este iterador pode ser obtido via RecursiveRegexIterator::getChildren().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se um iterador pode ser obtido para a entrada atual, caso contrário, retorna `false`.

Examples

Exemplo #1 Exemplo de RecursiveRegexIterator::hasChildren()

```php
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $value) {
    var_dump($rRegexIterator->hasChildren());
}
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(true)
```

Veja também

- RecursiveRegexIterator::getChildren() - Retorna um iterador para a entrada atual

RecursiveTreeIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Permite iterar sobre um RecursiveIterator para gerar uma árvore gráfica ASCII.

Resumo da classe

```
class RecursiveTreeIterator extends RecursiveIteratorIterator {
/* Constantes herdadas */
public const int RecursiveIteratorIterator::LEAVES_ONLY;
public const int RecursiveIteratorIterator::SELF_FIRST;
public const int RecursiveIteratorIterator::CHILD_FIRST;
public const int RecursiveIteratorIterator::CATCH_GET_CHILD;
/* Constantes */
public const int BYPASS_CURRENT;
public const int BYPASS_KEY;
public const int PREFIX_LEFT;
public const int PREFIX_MID_HAS_NEXT = 1;
public const int PREFIX_MID_LAST = 2;
public const int PREFIX_END_HAS_NEXT = 3;
public const int PREFIX_END_LAST = 4;
public const int PREFIX_RIGHT = 5;
/* Métodos */
public function __construct(
    RecursiveIterator|IteratorAggregate $iterator,
    int $flags = RecursiveTreeIterator::BYPASS_KEY,
    int $cachingIteratorFlags = CachingIterator::CATCH_GET_CHILD,
    int $mode = RecursiveTreeIterator::SELF_FIRST
)
public function beginChildren(): void
public function beginIteration(): RecursiveIterator
public function callGetChildren(): RecursiveIterator
public function callHasChildren(): bool
public function current(): mixed
public function endChildren(): void
public function endIteration(): void
public function getEntry(): string
public function getPostfix(): string
public function getPrefix(): string
public function key(): mixed
public function next(): void
public function nextElement(): void
public function rewind(): void
public function setPostfix(string $postfix): void
public function setPrefixPart(int $part, string $value): void
public function valid(): bool
/* Métodos herdados */
public function RecursiveIteratorIterator::beginChildren(): void
public function RecursiveIteratorIterator::beginIteration(): void
public function RecursiveIteratorIterator::callGetChildren(): ?RecursiveIterator
public function RecursiveIteratorIterator::callHasChildren(): bool
public function RecursiveIteratorIterator::current(): mixed
public function RecursiveIteratorIterator::endChildren(): void
public function RecursiveIteratorIterator::endIteration(): void
public function RecursiveIteratorIterator::getDepth(): int
public function RecursiveIteratorIterator::getInnerIterator(): RecursiveIterator
public function RecursiveIteratorIterator::getMaxDepth(): int|false
public function RecursiveIteratorIterator::getSubIterator(?int $level = null): ?RecursiveIterator
public function RecursiveIteratorIterator::key(): mixed
public function RecursiveIteratorIterator::next(): void
public function RecursiveIteratorIterator::nextElement(): void
public function RecursiveIteratorIterator::rewind(): void
public function RecursiveIteratorIterator::setMaxDepth(int $maxDepth = -1): void
public function RecursiveIteratorIterator::valid(): bool
}
```

Constantes predefinidas

`RecursiveTreeIterator::BYPASS_CURRENT`

`RecursiveTreeIterator::BYPASS_KEY`

`RecursiveTreeIterator::PREFIX_LEFT`

`RecursiveTreeIterator::PREFIX_MID_HAS_NEXT`

`RecursiveTreeIterator::PREFIX_MID_LAST`

`RecursiveTreeIterator::PREFIX_END_HAS_NEXT`

`RecursiveTreeIterator::PREFIX_END_LAST`

`RecursiveTreeIterator::PREFIX_RIGHT`

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

RecursiveTreeIterator::beginChildren

RecursiveTreeIterator::beginChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::beginChildren — Iniciar filhos

Descrição

public function **RecursiveTreeIterator::beginChildren**(): void

Chamado ao descer um nível na recursão.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::beginIteration

RecursiveTreeIterator::beginIteration

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::beginIteration — Inicia iteração

Descrição

public function **RecursiveTreeIterator::beginIteration**(): RecursiveIterator

Chamado quando a iteração começa (após a primeira chamada de RecursiveTreeIterator::rewind()).

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um RecursiveIterator.

RecursiveTreeIterator::callGetChildren

RecursiveTreeIterator::callGetChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::callGetChildren — Obtém filhos

Descrição

public function **RecursiveTreeIterator::callGetChildren**(): RecursiveIterator

Obtém os filhos do elemento atual.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um RecursiveIterator.

RecursiveTreeIterator::callHasChildren

RecursiveTreeIterator::callHasChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::callHasChildren — Tem filhos

Descrição

public function **RecursiveTreeIterator::callHasChildren**(): bool

Chamado para cada elemento para testar se ele tem filhos.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se houver filhos, caso contrário, `false`

RecursiveTreeIterator::__construct

RecursiveTreeIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::__construct — Constrói um RecursiveTreeIterator

Descrição

public function **RecursiveTreeIterator::__construct**(
RecursiveIterator|IteratorAggregate `$iterator`,
int `$flags` = **`RecursiveTreeIterator::BYPASS_KEY`**,
int `$cachingIteratorFlags` = **`CachingIterator::CATCH_GET_CHILD`**,
int `$mode` = **`RecursiveTreeIterator::SELF_FIRST`**
)

Constrói um novo RecursiveTreeIterator a partir do iterador recursivo fornecido.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`iterator`

O RecursiveIterator ou IteratorAggregate a ser iterado.

`flags`

Podem ser fornecidas opções que afetarão o comportamento de alguns métodos. Uma lista das opções pode ser encontrada em Constantes predefinidas do RecursiveTreeIterator.

`caching_it_flags`

Opções para afetar o comportamento do RecursiveCachingIterator usado internamente.

`mode`

Opções para afetar o comportamento do RecursiveIteratorIterator usado internamente.

RecursiveTreeIterator::current

RecursiveTreeIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::current — Obtém o elemento atual

Descrição

public function **RecursiveTreeIterator::current**(): mixed

Obtém o elemento atual prefixado e sufixado.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Obtém o elemento atual prefixado e sufixado.

RecursiveTreeIterator::endChildren

RecursiveTreeIterator::endChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::endChildren — Termina os filhos

Descrição

public function **RecursiveTreeIterator::endChildren**(): void

Chamado quando termina de recursar um nível.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::endIteration

RecursiveTreeIterator::endIteration

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::endIteration — Termina iteração

Descrição

public function **RecursiveTreeIterator::endIteration**(): void

Chamado quando a iteração termina (quando RecursiveTreeIterator::valid() retorna pela primeira vez `false`)

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::getEntry

RecursiveTreeIterator::getEntry

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getEntry — Obtém entrada atual

Descrição

public function **RecursiveTreeIterator::getEntry**(): string

Obtém a parte da árvore construída para o elemento atual.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a parte da árvore construída para o elemento atual.

RecursiveTreeIterator::getPostfix

RecursiveTreeIterator::getPostfix

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getPostfix — Obtém o sufixo

Descrição

public function **RecursiveTreeIterator::getPostfix**(): string

Obtém a string para colocar após o elemento atual.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a string para colocar após o elemento atual.

RecursiveTreeIterator::getPrefix

RecursiveTreeIterator::getPrefix

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getPrefix — Obtém o prefixo

Descrição

public function **RecursiveTreeIterator::getPrefix**(): string

Obtém a string para colocar na frente do elemento atual

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

etorna a string para colocar na frente do elemento atual

RecursiveTreeIterator::key

RecursiveTreeIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::key — Obtém a chave do elemento atual

Descrição

public function **RecursiveTreeIterator::key**(): mixed

Obtém a chave atual com prefixo e sufixo.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a chave atual com prefixo e sufixo.

RecursiveTreeIterator::next

RecursiveTreeIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::next — Move para o próximo elemento

Descrição

public function **RecursiveTreeIterator::next**(): void

Move para o próximo elemento.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::nextElement

RecursiveTreeIterator::nextElement

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::nextElement — Próximo elemento

Descrição

public function **RecursiveTreeIterator::nextElement**(): void

Chamado quando o próximo elemento está disponível.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::rewind

RecursiveTreeIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::rewind — Rebobina o iterador

Descrição

public function **RecursiveTreeIterator::rewind**(): void

Rebobina o iterador para o primeiro elemento do iterador interno do nível superior.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::setPostfix

RecursiveTreeIterator::setPostfix

(PHP 5 >= 5.5.3, PHP 7, PHP 8)

RecursiveTreeIterator::setPostfix — Define sufixo

Descrição

public function **RecursiveTreeIterator::setPostfix**(string `$postfix`): void

Define o sufixo usado em RecursiveTreeIterator::getPostfix().

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`postfix`

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::setPrefixPart

RecursiveTreeIterator::setPrefixPart

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::setPrefixPart — Define uma parte do prefixo

Descrição

public function **RecursiveTreeIterator::setPrefixPart**(int `$part`, string `$value`): void

Define uma parte do prefixo usado na árvore gráfica.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`part`

Uma das constantes RecursiveTreeIterator::PREFIX_*.

`value`

O valor a ser atribuído à parte do prefixo especificada em `part`.

Valor Retornado

Nenhum valor é retornado.

RecursiveTreeIterator::valid

RecursiveTreeIterator::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::valid — Verifica validade

Descrição

public function **RecursiveTreeIterator::valid**(): bool

Verifica se a posição atual é válida.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a posição atual for válida, caso contrário `false`

RegexIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introdução

Este iterador pode ser usado para filtrar outro iterador com base em uma expressão regular.

Resumo da classe

```
class RegexIterator extends FilterIterator {
/* Constantes */
public const int USE_KEY;
public const int INVERT_MATCH;
public const int MATCH;
public const int GET_MATCH;
public const int ALL_MATCHES;
public const int SPLIT;
public const int REPLACE;
/* Propriedades */
public ?string $replacement = null;
/* Métodos */
public function __construct(
    Iterator $iterator,
    string $pattern,
    int $mode = RegexIterator::MATCH,
    int $flags = 0,
    int $pregFlags = 0
)
public function accept(): bool
public function getFlags(): int
public function getMode(): int
public function getPregFlags(): int
public function getRegex(): string
public function setFlags(int $flags): void
public function setMode(int $mode): void
public function setPregFlags(int $pregFlags): void
/* Métodos herdados */
```

public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Constantes predefinidas

Modos de Operação do RegexIterator

`RegexIterator::ALL_MATCHES`

Retorna todas as correspondências para a entrada atual (veja preg_match_all()).

`RegexIterator::GET_MATCH`

Retorna a primeira correspondência para a entrada atual (veja preg_match()).

`RegexIterator::MATCH`

Execute somente a correspondência (filtro) para a entrada atual (veja preg_match()).

`RegexIterator::REPLACE`

Substitui a entrada atual (veja preg_replace(); Não totalmente implementado ainda)

`RegexIterator::SPLIT`

Retorna os valores divididos para a entrada atual (veja preg_split()).

RegexIterator Flags

`RegexIterator::USE_KEY`

Flag especial: Correspondência à chave da entrada em vez do valor da entrada.

`RegexIterator::INVERT_MATCH`

Inverte o valor de retorno de RegexIterator::accept().

Propriedades

replacement

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

RegexIterator::accept

RegexIterator::accept

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::accept — Obtém o status de aceitação

Descrição

public function **RegexIterator::accept**(): bool

Faz corresponder (`string`) **RegexIterator::current()** (ou **RegexIterator::key()** se a opção RegexIterator::USE_KEY estiver definida) contra a expressão regular.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se houver correspondência, `false` caso contrário.

Examples

Exemplo #1 Exemplo de **RegexIterator::accept()**

Este exemplo mostra que apenas os itens que correspondem à expressão regular são aceitos.

```
<?php
$names = new ArrayIterator(array('Ann', 'Bob', 'Charlie', 'David'));
$filter = new RegexIterator($names, '/^[B-D]/');
foreach ($filter as $name) {
    echo $name . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
Bob
Charlie
David
```

Veja também

- Constantes RegexIterator
- RegexIterator::setFlags() - Define as opções

RegexIterator::__construct

RegexIterator::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::__construct — Cria um novo RegexIterator

Descrição

public function **RegexIterator::__construct**(
  Iterator `$iterator`,
  string `$pattern`,
  int `$mode` = **`RegexIterator::MATCH`**,
  int `$flags` = 0,
  int `$pregFlags` = 0
)

Cria um novo RegexIterator que filtra um Iterator usando uma expressão regular.

Parâmetros

`iterator`

O iterador ao qual aplicar este filtro de regex.

`pattern`

A expressão regular para fazer a correspondência.

`mode`

Modo de operação, consulte RegexIterator::setMode() para obter uma lista de modos.

`flags`

Opções especiais, consulte RegexIterator::setFlags() para obter uma lista de opções disponíveis.

`pregFlags`

As opções da expressão regular. Essas opções dependem do parâmetro de modo de operação:

RegexIterator preg_flags

modo de operação	opções disponíveis
RegexIterator::ALL_MATCHES	Veja preg_match_all().
RegexIterator::GET_MATCH	Veja preg_match().
RegexIterator::MATCH	Veja preg_match().
RegexIterator::REPLACE	nenhum.
RegexIterator::SPLIT	Veja preg_split().

Erros/Exceções

Lança uma InvalidArgumentException se o argumento `pattern` for inválido.

Examples

Exemplo #1 Exemplo de RegexIterator::__construct()

Cria um novo RegexIterator que filtra todas as strings que começam com 'test'.

```php
<?php
$arrayIterator = new ArrayIterator(array('test 1', 'another test', 'test 123'));
$regexIterator = new RegexIterator($arrayIterator, '/^test/');

foreach ($regexIterator as $value) {
    echo $value . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
test 1
test 123
```

Veja também

- preg_match() - Realiza uma correspondência com expressão regular
- preg_match_all() - Realiza uma correspondência global de expressão regular
- preg_replace() - Realiza uma pesquisa por uma expressão regular e substitui
- preg_split() - Divide uma string com base em expressão regular

RegexIterator::getFlags

RegexIterator::getFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getFlags — Obtém opções

Descrição

public function **RegexIterator::getFlags**(): int

Retorna as opções, veja RegexIterator::setFlags() para obter uma lista de opções disponíveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna as opções definidas.

Examples

Exemplo #1 Exemplo de RegexIterator::getFlags()

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/');
$regexIterator->setFlags(RegexIterator::USE_KEY);

if ($regexIterator->getFlags() & RegexIterator::USE_KEY) {
    echo 'Filtragem baseada nas chaves dos arrays.';
} else {
    echo 'Filtragem baseada nos valores dos arrays.';
}
?>
```

O exemplo acima produzirá:

```
Filtragem baseada nas chaves dos arrays.
```

Veja também

- RegexIterator::setFlags() - Define as opções

RegexIterator::getMode

RegexIterator::getMode

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getMode — Retorna o modo de operação

Descrição

public function **RegexIterator::getMode**(): int

Retorna o modo de operação, consulte RegexIterator::setMode() para a lista de modos de operação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o modo de operação.

Examples

Exemplo #1 Exemplo de RegexIterator::getMode()

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^[a-z]+/', RegexIterator::GET_MATCH);

$mode = $regexIterator->getMode();
if ($mode & RegexIterator::GET_MATCH) {
    echo 'Obtendo a correspondência para cada item.';
} elseif ($mode & RegexIterator::ALL_MATCHES) {
    echo 'Obtendo todas as correspondências para cada item.';
} elseif ($mode & RegexIterator::MATCH) {
    echo 'Obtendo cada item se ele corresponder.';
} elseif ($mode & RegexIterator::SPLIT) {
    echo 'Obtendo partes separadas de cada um.';
}
?>
```

O exemplo acima produzirá:

```
Obtendo a correspondência para cada item.
```

Veja também

- RegexIterator::setMode() - Define o modo de operação

RegexIterator::getPregFlags

RegexIterator::getPregFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getPregFlags — Retorna as opções da expressão regular

Descrição

public function **RegexIterator::getPregFlags**(): int

Retorna as opções da expressão regular, consulte RegexIterator::__construct() para a lista de opções.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma máscara de bits das opções da expressão regular.

Examples

Exemplo #1 Exemplo de RegexIterator::getPregFlags()

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/\s/', RegexIterator::SPLIT);
$regexIterator->setPregFlags(PREG_SPLIT_NO_EMPTY | PREG_SPLIT_OFFSET_CAPTURE);

if ($regexIterator->getPregFlags() & PREG_SPLIT_NO_EMPTY) {
    echo 'Ignoring empty pieces';
} else {
    echo 'Not ignoring empty pieces';
```

```
}

?>
```

O exemplo acima produzirá:

```
Ignoring empty pieces
```

Veja também

- RegexIterator::setPregFlags() - Define as opções da expressão regular

RegexIterator::getRegex

RegexIterator::getRegex

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RegexIterator::getRegex — Retorna a expressão regular atual

Descrição

public function **RegexIterator::getRegex**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

RegexIterator::setFlags

RegexIterator::setFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setFlags — Define as opções

Descrição

public function **RegexIterator::setFlags**(int `$flags`): void

Define as opções.

Parâmetros

`flags`

As opções a serem definidas, um conjunto de bits de constantes de classe.

As opções disponíveis estão listadas abaixo. Os significados reais dessas bandeiras são descritos no constantes predefinidas.

Opções de RegexIterator

valor	constante
1	RegexIterator::USE_KEY

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de RegexIterator::setFlags()

Cria um novo RegexIterator que filtra todas as entradas cuja chave começa com 'test'.

```php
<?php
$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/');
$regexIterator->setFlags(RegexIterator::USE_KEY);

foreach ($regexIterator as $key => $value) {
    echo $key . ' => ' . $value . "\n";
}
?>
```

O exemplo acima produzirá:

```
teststr2 => another test
```

Veja também

- RegexIterator::getFlags() - Obtém opções

RegexIterator::setMode

RegexIterator::setMode

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setMode — Define o modo de operação

Descrição

public function **RegexIterator::setMode**(int $mode): void

Define o modo de operação.

Parâmetros

`mode`

O modo de operação.

Os modos disponíveis estão listados abaixo. Os significados reais desses modos são descritos no constantes predefinidas.

Modos de RegexIterator

valor	constante
0	RegexIterator::MATCH
1	RegexIterator::GET_MATCH
2	RegexIterator::ALL_MATCHES
3	RegexIterator::SPLIT
4	RegexIterator::REPLACE

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de RegexIterator::setMode()

```php
<?php
$test = array ('str1' => 'test 1', 'test str2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
// Filtra tudo que comece com 'test ' seguido de um ou mais números.
$regexIterator = new RegexIterator($arrayIterator, '/^test (\d+)/');
// Modo de operação: Subtitui o valor atual com a correspondência
$regexIterator->setMode(RegexIterator::GET_MATCH);

foreach ($regexIterator as $key => $value) {
    // imprime os números correspondidos
    echo $key . ' => ' . $value[1] . PHP_EOL;
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
str1 => 1
str3 => 123
```

Veja também

- RegexIterator::getMode() - Retorna o modo de operação

RegexIterator::setPregFlags

RegexIterator::setPregFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setPregFlags — Define as opções da expressão regular

Descrição

public function **RegexIterator::setPregFlags**(int `$pregFlags`): void

Define as opções da expressão regular.

Parâmetros

`pregFlags`

As opções da expressão regular. Consulte RegexIterator::__construct() para uma visão geral das opções disponíveis.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de RegexIterator::setPregFlags()

Cria um novo RegexIterator que filtra todas as entradas em que a chave do array começa com 'test'.

```php
<?php
$test = array ('test 1', 'another test', 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/', RegexIterator::GET_MATCH);

$regexIterator->setPregFlags(PREG_OFFSET_CAPTURE);

foreach ($regexIterator as $key => $value) {
```

```
    var_dump($value);
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(4) "test"
    [1]=>
    int(0)
  }
}
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(4) "test"
    [1]=>
    int(0)
  }
}
```

Veja também

- RegexIterator::getPregFlags() - Retorna as opções da expressão regular

Manipulação de Arquivos

Índice

SPL fornece várias classes para trabalhar com arquivos.

SplFileInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

Introdução

A classe SplFileInfo oferece uma interface orientada a objetos de alto nível para informações sobre um arquivo individual.

Resumo da classe

class **SplFileInfo** implements Stringable {
/* Métodos */
public function __construct(string `$filename`)
public function getATime(): int|false
public function getBasename(string `$suffix` = ""): string
public function getCTime(): int|false
public function getExtension(): string
public function getFileInfo(?string `$class` = **`null`**): SplFileInfo
public function getFilename(): string
public function getGroup(): int|false
public function getInode(): int|false
public function getLinkTarget(): string|false
public function getMTime(): int|false
public function getOwner(): int|false
public function getPath(): string
public function getPathInfo(?string `$class` = **`null`**): ?SplFileInfo
public function getPathname(): string

public function getPerms(): int|false
public function getRealPath(): string|false
public function getSize(): int|false
public function getType(): string|false
public function isDir(): bool
public function isExecutable(): bool
public function isFile(): bool
public function isLink(): bool
public function isReadable(): bool
public function isWritable(): bool
public function openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function setFileClass(string $class = SplFileObject::class): void
public function setInfoClass(string $class = SplFileInfo::class): void
public function __toString(): string
}

Registro de Alterações

Versão	Descrição
8.0.0	**SplFileInfo** implementa Stringable agora.

Índice

SplFileInfo::__construct

SplFileInfo::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::__construct — Constrói um novo objeto SplFileInfo

Descrição

public function **SplFileInfo::__construct**(string $filename)

Cria um novo objeto SplFileInfo para o arquivo especificado. O arquivo não precisa existir ou ser legível.

Parâmetros

`filename`

Caminho para o arquivo.

Examples

Exemplo #1 Exemplo de SplFileInfo::__construct()

```php
<?php
$info = new SplFileInfo('example.php');
if ($info->isFile()) {
    echo $info->getRealPath();
}
?>
```

SplFileInfo::getATime

SplFileInfo::getATime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getATime — Obtém o último horário de acesso do arquivo

Descrição

public function **SplFileInfo::getATime**(): int|false

Obtém o último horário de acesso ao arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o horário em que o arquivo foi acessado pela última vez em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getATime()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last accessed at ' . date('g:i a', $info->getATime());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Last accessed at 1:49 pm
```

Veja também

- fileatime() - Obtém o último horário de acesso do arquivo
- SplFileInfo::getCTime() - Obtém o tempo de mudança do inode
- SplFileInfo::getMTime() - Obtém o tempo da última modificação

SplFileInfo::getBasename

SplFileInfo::getBasename

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getBasename — Obtém o nome base do arquivo

Descrição

public function **SplFileInfo::getBasename**(string `$suffix = ""`): string

Este método retorna o nome base do arquivo, diretório ou link sem informações de caminho.

> **Cuidado**
>
> **SplFileInfo::getBasename()** é sensível ao local, então para visualizar o nome base correto com caminhos de caracteres multibyte, o local correspondente deve ser definido usando a função setlocale().

Parâmetros

`suffix`

Sufixo opcional a ser omitido do nome base retornado.

Valor Retornado

Retorna o nome base sem informações de caminho.

Examples

Exemplo #1 Exemplo de SplFileInfo::getBasename()

```php
<?php
$info = new SplFileInfo('file.txt');
var_dump($info->getBasename());

$info = new SplFileInfo('/path/to/file.txt');
var_dump($info->getBasename());

$info = new SplFileInfo('/path/to/file.txt');
var_dump($info->getBasename('.txt'));
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(8) "file.txt"
string(8) "file.txt"
string(4) "file"
```

Veja também

- SplFileInfo::getFilename() - Obtém o nome do arquivo

SplFileInfo::getCTime

SplFileInfo::getCTime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getCTime — Obtém o tempo de mudança do inode

Descrição

public function **SplFileInfo::getCTime**(): int|false

Retorna o tempo de mudança do inode para o arquivo. O tempo retornado é um timestamp de data/hora Unix.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O tempo da última mudança, em um timestamp de data/hora Unix em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getCTime()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last changed at ' . date('g:i a', $info->getCTime());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Last changed at 1:49 pm
```

Veja também

- filectime() - Obtém o horário de modificação de inode do arquivo
- SplFileInfo::getATime() - Obtém o último horário de acesso do arquivo
- SplFileInfo::getMTime() - Obtém o tempo da última modificação

SplFileInfo::getExtension

SplFileInfo::getExtension

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

SplFileInfo::getExtension — Obtém a extensão do arquivo

Descrição

public function **SplFileInfo::getExtension**(): string

Recupera a extensão do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string contendo a extensão do arquivo, ou uma string vazia se o arquivo não tiver extensão.

Examples

Exemplo #1 Exemplo de SplFileInfo::getExtension()

```php
<?php

$info = new SplFileInfo('foo.txt');
var_dump($info->getExtension());

$info = new SplFileInfo('photo.jpg');
var_dump($info->getExtension());

$info = new SplFileInfo('something.tar.gz');
var_dump($info->getExtension());

?>
```

O exemplo acima produzirá:

```
string(3) "txt"
string(3) "jpg"
string(2) "gz"
```

Notas

> **Nota** :
>
> Outra maneira de obter a extensão é usar a função pathinfo().
>
> ```php
> <?php
> $extension = pathinfo($info->getFilename(), PATHINFO_EXTENSION);
> ?>
> ```

Veja também

- SplFileInfo::getFilename() - Obtém o nome do arquivo
- SplFileInfo::getBasename() - Obtém o nome base do arquivo
- pathinfo() - Retorna informações sobre um caminho de arquivo

SplFileInfo::getFileInfo

SplFileInfo::getFileInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getFileInfo — Obtém um objeto SplFileInfo para o arquivo

Descrição

public function **SplFileInfo::getFileInfo**(?string `$class = null`): SplFileInfo

Este método obtém um objeto SplFileInfo para o arquivo referenciado.

Parâmetros

`class`

Nome de uma classe derivada de SplFileInfo para usar.

Valor Retornado

Um objeto SplFileInfo criado para o arquivo.

Registro de Alterações

Versão	Descrição
8.0.0	`class` é anulável agora.

Veja também

- SplFileInfo::setInfoClass() - Define a classe usada com SplFileInfo::getFileInfo e SplFileInfo::getPathInfo

SplFileInfo::getFilename

SplFileInfo::getFilename

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getFilename — Obtém o nome do arquivo

Descrição

public function **SplFileInfo::getFilename**(): string

Obtém o nome do arquivo sem qualquer informação de caminho.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome do arquivo.

Examples

Exemplo #1 Exemplo de SplFileInfo::getFilename()

```php
<?php
$info = new SplFileInfo('foo.txt');
var_dump($info->getFilename());

$info = new SplFileInfo('/path/to/foo.txt');
var_dump($info->getFilename());
```

```php
$info = new SplFileInfo('http://www.php.net/');
var_dump($info->getFilename());

$info = new SplFileInfo('http://www.php.net/svn.php');
var_dump($info->getFilename());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(7) "foo.txt"
string(7) "foo.txt"
string(11) "www.php.net"
string(7) "svn.php"
```

Veja também

- SplFileInfo::getBasename() - Obtém o nome base do arquivo

SplFileInfo::getGroup

SplFileInfo::getGroup

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getGroup — Obtém o grupo do arquivo

Descrição

public function **SplFileInfo::getGroup**(): int|false

Obtém o grupo do arquivo. O ID do grupo é retornado em formato numérico.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O ID do grupo em formato numérico em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getGroup()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . ' belongs to group id ' . $info->getGroup() . "\n";
print_r(posix_getgrgid($info->getGroup()));
?>
```

O exemplo acima produzirá algo semelhante a:

```
example.jpg belongs to group id 42
Array
(
    [name] => toons
    [passwd] => x
    [members] => Array
```

```
        (
            [0] => tom
            [1] => jerry
        )
    [gid] => 42
)
```

Veja também

- filegroup() - Lê o grupo do arquivo
- posix_getgrgid() - Retorna informação sobre um grupo a partir de um ID de grupo

SplFileInfo::getInode

SplFileInfo::getInode

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getInode — Obtém o inode para o arquivo

Descrição

public function **SplFileInfo::getInode**(): int|false

Obtém o número do inode para o objeto do sistema de arquivos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número do inode para o objeto do sistema de arquivos em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Veja também

- fileinode() - Lê o inode do arquivo

SplFileInfo::getLinkTarget

SplFileInfo::getLinkTarget

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getLinkTarget — Obtém o alvo de um link

Descrição

public function **SplFileInfo::getLinkTarget**(): string|false

Obtém o alvo de um link do sistema de arquivos.

> **Nota** :
>
> O alvo pode não ser o caminho real no sistema de arquivos. Use SplFileInfo::getRealPath() para determinar o verdadeiro caminho no sistema de arquivos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o alvo do link do sistema de arquivos em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getLinkTarget()

```php
<?php
$info = new SplFileInfo('/Users/bbieber/workspace');
if ($info->isLink()) {
    var_dump($info->getLinkTarget());
    var_dump($info->getRealPath());
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(19) "Documents/workspace"
string(34) "/Users/bbieber/Documents/workspace"
```

Veja também

- SplFileInfo::isLink() - Indica se um arquivo é um link
- SplFileInfo::getRealPath() - Obtém o caminho absoluto para o arquivo

SplFileInfo::getMTime

SplFileInfo::getMTime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getMTime — Obtém o tempo da última modificação

Descrição

public function **SplFileInfo::getMTime**(): int|false

Retorna o momento em que o conteúdo do arquivo foi alterado. O tempo retornado é um timestamp de data/hora Unix.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o tempo da última modificação do arquivo, em um timestamp de data/hora Unix em caso de sucesso, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileInfo::getMTime()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last modified at ' . date('g:i a', $info->getMTime());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Last modified at 1:49 pm
```

Veja também

- filemtime() - Obtém o horário de modificação do arquivo
- SplFileInfo::getATime() - Obtém o último horário de acesso do arquivo
- SplFileInfo::getCTime() - Obtém o tempo de mudança do inode

SplFileInfo::getOwner

SplFileInfo::getOwner

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getOwner — Obtém o proprietário do arquivo

Descrição

public function **SplFileInfo::getOwner**(): int|false

Obtém o proprietário do arquivo. O ID do proprietário é retornado em formato numérico.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O ID do proprietário em formato numérico em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Lança RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getOwner()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . ' belongs to owner id ' . $info->getOwner() . "\n";
print_r(posix_getpwuid($info->getOwner()));
?>
```

O exemplo acima produzirá algo semelhante a:

```
example.jpg belongs to user id 501
Array
(
    [name] => tom
    [passwd] => x
    [uid] => 501
    [gid] => 42
    [gecos] => Tom Cat
    [dir] => /home/tom
    [shell] => /bin/bash
)
```

Veja também

- posix_getpwuid() - Retorna informações sobre um usuário por ID de usuário
- SplFileInfo::getGroup() - Obtém o grupo do arquivo

SplFileInfo::getPath

SplFileInfo::getPath

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPath — Obtém o caminho sem o nome do arquivo

Descrição

public function **SplFileInfo::getPath**(): string

Retorna o caminho para o arquivo, omitindo o nome do arquivo e qualquer barra no final.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho para o arquivo.

Examples

Exemplo #1 Exemplo de SplFileInfo::getPath()

```
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->getPath());


$info = new SplFileInfo('/usr/');
var_dump($info->getPath());?>
```

O exemplo acima produzirá algo semelhante a:

```
string(8) "/usr/bin"
string(4) "/usr"
```

Veja também

- SplFileInfo::getRealPath() - Obtém o caminho absoluto para o arquivo
- SplFileInfo::getFilename() - Obtém o nome do arquivo
- SplFileInfo::getPathInfo() - Obtém um objeto SplFileInfo para o caminho

SplFileInfo::getPathInfo

SplFileInfo::getPathInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPathInfo — Obtém um objeto SplFileInfo para o caminho

Descrição

public function **SplFileInfo::getPathInfo**(?string `$class = null`): ?SplFileInfo

Obtém um objeto SplFileInfo para o diretório pai do arquivo atual.

Parâmetros

`class`

Nome de uma classe derivada de SplFileInfo a ser usada, ou ele mesmo se for `null`.

Valor Retornado

Retorna um objeto SplFileInfo para o diretório pai do arquivo em caso de sucesso, ou `null` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`class` é anulável agora.

Examples

Exemplo #1 Exemplo de SplFileInfo::getPathInfo()

```
<?php
$info = new SplFileInfo('/usr/bin/php');
$parent_info = $info->getPathInfo();
var_dump($parent_info->getRealPath());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(8) "/usr/bin"
```

Veja também

- SplFileInfo::setInfoClass() - Define a classe usada com SplFileInfo::getFileInfo e SplFileInfo::getPathInfo

SplFileInfo::getPathname

SplFileInfo::getPathname

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPathname — Obtém o caminho para o arquivo

Descrição

public function **SplFileInfo::getPathname**(): string

Retorna o caminho para o arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O caminho para o arquivo.

Examples

Exemplo #1 Exemplo de SplFileInfo::getPathname()

```
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->getPathname());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(12) "/usr/bin/php"
```

Veja também

- SplFileInfo::getRealPath() - Obtém o caminho absoluto para o arquivo

SplFileInfo::getPerms

SplFileInfo::getPerms

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPerms — Obtém as permissões do arquivo

Descrição

public function **SplFileInfo::getPerms**(): int|false

Obtém as permissões para o arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna as permissões do arquivo em caso de sucesso, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileInfo::getPerms()

```php
<?php
$info = new SplFileInfo('/tmp');
echo substr(sprintf('%o', $info->getPerms()), -4);

$info = new SplFileInfo(__FILE__);
echo substr(sprintf('%o', $info->getPerms()), -4);
?>
```

O exemplo acima produzirá algo semelhante a:

```
1777
0644
```

SplFileInfo::getRealPath

SplFileInfo::getRealPath

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getRealPath — Obtém o caminho absoluto para o arquivo

Descrição

public function **SplFileInfo::getRealPath**(): string|false

Este método expande todos os links simbólicos, resolve referências relativas e retorna o caminho real para o arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho para o arquivo, ou `false` se o arquivo não existir.

Examples

Exemplo #1 Exemplo de SplFileInfo::getRealPath()

```php
<?php
$info = new SplFileInfo('/..//./../../'.__FILE__);
var_dump($info->getRealPath());

$info = new SplFileInfo('/tmp');
var_dump($info->getRealPath());

$info = new SplFileInfo('/I/Do/Not/Exist');
var_dump($info->getRealPath());

$info = new SplFileInfo('php://output');
var_dump($info->getRealPath());

$info = new SplFileInfo("");
var_dump($info->getRealPath());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(28) "/private/tmp/phptempfile.php"
string(12) "/private/tmp"
bool(false)
bool(false)
string(12) "/private/tmp"
```

Veja também

- SplFileInfo::isLink() - Indica se um arquivo é um link
- realpath() - Retorna o path absoluto canonizado

SplFileInfo::getSize

SplFileInfo::getSize

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getSize — Obtém o tamanho do arquivo

Descrição

public function **SplFileInfo::getSize**(): int|false

Retorna o tamanho do arquivo em bytes para o arquivo referenciado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O tamanho do arquivo em bytes em caso de sucesso, ou `false` em caso de falha.

Erros/Exceções

Um RuntimeException será lançado se o arquivo não existir ou ocorrer um erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getSize()

```php
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . " " . $info->getSize();
?>
```

O exemplo acima produzirá algo semelhante a:

```
example.jpg 15385
```

Veja também

- filesize() - Obtém o tamanho do arquivo

SplFileInfo::getType

SplFileInfo::getType

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getType — Obtém o tipo de arquivo

Descrição

public function **SplFileInfo::getType**(): string|false

Retorna o tipo do arquivo referenciado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string representando o tipo da entrada. Pode ser um dos seguintes: `file`, `link`, `dir`, `block`, `fifo`, `char`, `socket` ou `unknown`, ou `false` em caso de falha.

Erros/Exceções

Lança um RuntimeException em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileInfo::getType()

```php
<?php

$info = new SplFileInfo(__FILE__);
echo $info->getType().PHP_EOL;

$info = new SplFileInfo(dirname(__FILE__));
echo $info->getType();

?>
```

O exemplo acima produzirá algo semelhante a:

```
file
dir
```

SplFileInfo::isDir

SplFileInfo::isDir

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isDir — Indica se o arquivo é um diretório

Descrição

public function **SplFileInfo::isDir**(): bool

Este método pode ser usado para determinar se um arquivo é um diretório.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se um diretório, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isDir()

```
<?php
$d = new SplFileInfo(dirname(__FILE__));
var_dump($d->isDir());

$d = new SplFileInfo(__FILE__);
var_dump($d->isDir());
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(false)
```

SplFileInfo::isExecutable

SplFileInfo::isExecutable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isExecutable — Indica se o arquivo é executável

Descrição

public function **SplFileInfo::isExecutable**(): bool

Verifica se o arquivo é executável.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se executável, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isExecutable()

```php
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->isExecutable());

$info = new SplFileInfo('/usr/bin');
var_dump($info->isExecutable());

$info = new SplFileInfo('foo');
var_dump($info->isExecutable());
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(true)
bool(false)
```

SplFileInfo::isFile

SplFileInfo::isFile

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isFile — Indica se o objeto faz referência a um arquivo regular

Descrição

public function **SplFileInfo::isFile**(): bool

Verifica se o arquivo referenciado por este objeto SplFileInfo existe e é um arquivo regular.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o arquivo existe e é um arquivo regular (não é um link), `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isFile()

```php
<?php
$info = new SplFileInfo(__FILE__);
var_dump($info->isFile());

$info = new SplFileInfo(dirname(__FILE__));
var_dump($info->isFile());
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(false)
```

SplFileInfo::isLink

SplFileInfo::isLink

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isLink — Indica se um arquivo é um link

Descrição

public function **SplFileInfo::isLink**(): bool

Use este método para checar se um arquivo referenciado pelo objeto SplFileInfo é um link.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o arquivo é um link, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isLink()

```php
<?php
$info = new SplFileInfo('/caminho/para/ligacao_simbolica');
if ($info->isLink()) {
    echo 'O caminho real é '.$info->getRealPath();
}
?>
```

Veja também

- SplFileInfo::getRealPath() - Obtém o caminho absoluto para o arquivo

SplFileInfo::isReadable

SplFileInfo::isReadable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isReadable — Indica se o arquivo é legível

Descrição

public function **SplFileInfo::isReadable**(): bool

Verifica se o arquivo é legível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se legível, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isReadable()

```php
<?php
$info = new SplFileInfo('readable.jpg');
if ($info->isReadable()) {
    echo $info->getFilename() . ' é legível';
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
```

```
readable.jpg é legível
```

SplFileInfo::isWritable

SplFileInfo::isWritable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isWritable — Indica se a entrada é gravável

Descrição

public function **SplFileInfo::isWritable**(): bool

Verifica se a entrada atual é gravável.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se gravável, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileInfo::isWritable()

```php
<?php
$info = new SplFileInfo('locked.jpg');
if (!$info->isWritable()) {
    echo $info->getFilename() . ' não é gravável';
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
locked.jpg não é gravável
```

SplFileInfo::openFile

SplFileInfo::openFile

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::openFile — Obtém um objeto SplFileObject para o arquivo

Descrição

public function **SplFileInfo::openFile**(string `$mode = "r"`, bool `$useIncludePath = false`, ?resource `$context = null`): SplFileObject

Cria um objeto (object SplFileObject) a partir do arquivo. Isso é útil porque SplFileObject contém métodos adicionais para manipular o arquivo, enquanto SplFileInfo é útil apenas para obter informações, como, por exemplo, se o arquivo é gravável.

Parâmetros

`mode`

O modo de abertura do arquivo. Consulte a documentação de fopen() para descrições dos modos possíveis. O padrão é somente leitura.

`useIncludePath`

Quando definido como **`true`**, o nome do arquivo também é procurado no include_path

`context`

Consulte a seção do manual sobre contexto para uma descrição dos `contextos`.

Valor Retornado

O arquivo aberto como um SplFileObject object.

Erros/Exceções

Uma RuntimeException se o arquivo não puder ser aberto (por exemplo, direitos de acesso insuficientes).

Registro de Alterações

Versão	Descrição
8.0.0	`context` é anulável agora.

Examples

Exemplo #1 Exemplo de SplFileInfo::openFile()

```php
<?php
$fileinfo = new SplFileInfo('/tmp/foo.txt');

if ($fileinfo->isWritable()) {

    $fileobj = $fileinfo->openFile('a');

    $fileobj->fwrite("appended this sample text");
}
?>
```

Veja também

- SplFileObject
- stream_context_create() - Cria um contexto de fluxo
- fopen() - Abre um arquivo ou URL

SplFileInfo::setFileClass

SplFileInfo::setFileClass

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::setFileClass — Define a classe usada com SplFileInfo::openFile()

Descrição

public function **SplFileInfo::setFileClass**(string `$class = SplFileObject::class`): void

Use este método para definir uma classe personalizada que será usada quando SplFileInfo::openFile() for chamado. O nome da classe passado para este método deve ser SplFileObject ou uma classe derivada de SplFileObject.

Parâmetros

`class`

O nome da classe a ser usada quando SplFileInfo::openFile() for chamado.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplFileInfo::setFileClass()

```php
<?php
// Cria a classe estendendo SplFileObject
class MyFoo extends SplFileObject {}

$info = new SplFileInfo(__FILE__);
// Define a classe a ser usada
$info->setFileClass('MyFoo');
var_dump($info->openFile());
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(MyFoo)#2 (0) { }
```

Veja também

- SplFileInfo::openFile() - Obtém um objeto SplFileObject para o arquivo

SplFileInfo::setInfoClass

SplFileInfo::setInfoClass

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::setInfoClass — Define a classe usada com SplFileInfo::getFileInfo() e SplFileInfo::getPathInfo()

Descrição

public function **SplFileInfo::setInfoClass**(string `$class = SplFileInfo::class`): void

Use este método para definir uma classe personalizada que será usada quando SplFileInfo::getFileInfo() e SplFileInfo::getPathInfo() são chamados. O nome da classe passado para este método deve ser SplFileInfo ou uma classe derivada de SplFileInfo.

Parâmetros

`class`

O nome da classe a ser usada quando SplFileInfo::getFileInfo() e SplFileInfo::getPathInfo() são chamados.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplFileInfo::setFileClass()

```php
<?php
// Define uma classe que estende SplFileInfo
class MyFoo extends SplFileInfo {}

$info = new SplFileInfo('foo');
// Define o nome da classe a ser usada
$info->setInfoClass('MyFoo');
var_dump($info->getFileInfo());
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(MyFoo)#2 (0) { }
```

Veja também

- SplFileInfo::getFileInfo() - Obtém um objeto SplFileInfo para o arquivo

SplFileInfo::__toString

SplFileInfo::__toString

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::__toString — Retorna o caminho para o arquivo como uma string

Descrição

public function **SplFileInfo::__toString**(): string

Este método irá retornar o nome do arquivo do arquivo referenciado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o caminho para o arquivo.

Examples

Exemplo #1 Exemplo de SplFileInfo::__toString()

```php
<?php
$info = new SplFileInfo('foo');
var_dump($info->__toString());
echo $info.PHP_EOL;

$info = new SplFileInfo('/usr/bin/php');
var_dump($info->__toString());
echo $info.PHP_EOL;
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(3) "foo"
foo
string(12) "/usr/bin/php"
/usr/bin/php
```

SplFileObject

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

A classe SplFileObject oferece uma interface orientada a objetos para um arquivo.

Resumo da classe

class **SplFileObject** extends SplFileInfo implements RecursiveIterator, SeekableIterator {
/* Constantes */
public const int *DROP_NEW_LINE*;
public const int *READ_AHEAD*;
public const int *SKIP_EMPTY*;

```
public const int READ_CSV;
/* Métodos */
public function __construct(
    string $filename,
    string $mode = "r",
    bool $useIncludePath = false,
    ?resource $context = null
)
public function current(): string|array|false
public function eof(): bool
public function fflush(): bool
public function fgetc(): string|false
public function fgetcsv(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): array|false
public function fgets(): string
public function fgetss(string $allowable_tags = ?): string
public function flock(int $operation, int &$wouldBlock = null): bool
public function fpassthru(): int
public function fputcsv(
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
public function fread(int $length): string|false
public function fscanf(string $format, mixed &...$vars): array|int|null
public function fseek(int $offset, int $whence = SEEK_SET): int
public function fstat(): array
public function ftell(): int|false
public function ftruncate(int $size): bool
public function fwrite(string $data, ?int $length = null): int|false
public function getChildren(): null
public function getCsvControl(): array
public function getFlags(): int
public function getMaxLineLen(): int
public function hasChildren(): false
public function key(): int
public function next(): void
public function rewind(): void
public function seek(int $line): void
public function setCsvControl(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): void
public function setFlags(int $flags): void
public function setMaxLineLen(int $maxLength): void
public function __toString(): string
public function valid(): bool
/* Métodos herdados */
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
```

public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Constantes predefinidas

SplFileObject::DROP_NEW_LINE

Remove quebras de linha no final de uma linha.

SplFileObject::READ_AHEAD

Lê ao rebobinar/avançar.

SplFileObject::SKIP_EMPTY

Pula linhas vazias no arquivo. Isso requer que a flag **READ_AHEAD** esteja ativada para funcionar como esperado.

SplFileObject::READ_CSV

Lê linhas como linhas CSV.

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

SplFileObject::__construct

SplFileObject::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::__construct — Constrói um novo objeto de arquivo

Descrição

public function **SplFileObject::__construct**(
  string $filename,
  string $mode = "r",
  bool $useIncludePath = false,
  ?resource $context = null
)

Constrói um novo objeto de arquivo.

Parâmetros

`filename`

O arquivo para ler.

> **Dica**
>
> Uma URL pode ser usada como um nome de arquivo com esta função se os empacotadores fopen estiverem habilitados. Consulte a função fopen() para mais detalhes sobre como especificar o nome do arquivo. Consulte os Protocolos e

> empacotadores suportados para obter links para informações sobre as capacidades de cada empacotador, notas de uso e informações sobre quaisquer variáveis predefinidas que eles possam fornecer.

`mode`

O modo no qual abrir o arquivo. Consulte fopen() para obter uma lista de modos permitidos.

`useIncludePath`

Se deve procurar no include_path pelo `filename`.

`context`

Um recurso de contexto válido criado com stream_context_create().

Erros/Exceções

Lança uma RuntimeException se o `filename` não puder ser aberto.

Lança uma LogicException se o `filename` for um diretório.

Examples

Exemplo #1 Exemplo de SplFileObject::__construct()

Este exemplo abre o arquivo atual e itera sobre seu conteúdo linha por linha.

```
<?php
$file = new SplFileObject(__FILE__);
foreach ($file as $line_num => $line) {
    echo "Linha $line_num é $line";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Linha 0 é <?php
Linha 1 é $file = new SplFileObject(__FILE__);
Linha 2 é foreach ($file as $line_num => $line) {
Linha 3 é     echo "Linha $line_num é $line";
Linha 4 é }
Linha 5 é ?>
```

Veja também

- SplFileInfo::openFile() - Obtém um objeto SplFileObject para o arquivo
- fopen() - Abre um arquivo ou URL

SplFileObject::current

SplFileObject::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::current — Recupera linha atual do arquivo

Descrição

public function **SplFileObject::current**(): string|array|false

Recupera a linha atual do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Recupera a linha atual do arquivo. Se a flag `SplFileObject::READ_CSV` estiver definida, este método retorna um array contendo a linha atual interpretada como dados CSV. Se o final do arquivo for alcançado, é retornado `false`.

Examples

Exemplo #1 Exemplo de SplFileObject::current()

```php
<?php
$file = new SplFileObject(__FILE__);
foreach ($file as $k => $line) {
   echo ($file->key() + 1) . ': ' . $file->current();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
1: <?php
2: $file = new SplFileObject(__FILE__);
3: foreach ($file as $line) {
4:     echo ($file->key() + 1) . ': ' . $file->current();
5: }
6: ?>
```

Veja também

- SplFileObject::key() - Obtém o número da linha
- SplFileObject::seek() - Aponta para uma linha especificada
- SplFileObject::next() - Lê próxima linha
- SplFileObject::rewind() - Rebobina o arquivo para a primeira linha
- SplFileObject::valid() - Não está no final do arquivo (EOF)

SplFileObject::eof

SplFileObject::eof

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::eof — Chegou ao final do arquivo

Descrição

public function **SplFileObject::eof**(): bool

Determina se o final do arquivo foi alcançado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o arquivo estiver no final (EOF), `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileObject::eof()

```php
<?php
$file = new SplFileObject("fruits.txt");
while ( ! $file->eof()) {
```

```
    echo $file->fgets();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
apple
banana
cherry
date
elderberry
```

Veja também

- SplFileObject::valid() - Não está no final do arquivo (EOF)
- feof() - Testa pelo fim-de-arquivo em um ponteiro de arquivo

SplFileObject::fflush

SplFileObject::fflush

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fflush — Escreve o buffer de saída para o arquivo

Descrição

public function **SplFileObject::fflush**(): bool

Força a escrita de toda a saída em buffer para o arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileObject::fflush()

```
<?php
$file = new SplFileObject('misc.txt', 'r+');
$file->rewind();
$file->fwrite("Foo");
$file->fflush();
$file->ftruncate($file->ftell());
?>
```

Veja também

- SplFileObject::fwrite() - Escreve no arquivo

SplFileObject::fgetc

SplFileObject::fgetc

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgetc — Obtém caractere do arquivo

Descrição

public function **SplFileObject::fgetc**(): string|false

Obtém um caractere do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string contendo um único caractere lido do arquivo ou `false` no EOF.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

Examples

Exemplo #1 Exemplo de SplFileObject::fgetc()

```
<?php
$file = new SplFileObject('file.txt');
while (false !== ($char = $file->fgetc())) {
    echo "$char\n";
}
?>
```

Veja também

- SplFileObject::fgets() - Obtém linha do arquivo

SplFileObject::fgetcsv

SplFileObject::fgetcsv

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgetcsv — Obtém linha do arquivo e interpreta como campos CSV

Descrição

public function **SplFileObject::fgetcsv**(string `$separator = ","`, string `$enclosure = "\""`, string `$escape = "\\"`): array|false

Obtém uma linha do arquivo no formato CSV e retorna um array contendo os campos lidos.

> **Nota** :
>
> As configurações de localidade são consideradas por esta função. Por exemplo, dados em certas codificações de um byte podem ser interpretados incorretamente se `LC_CTYPE` for `en_US.UTF-8`.

Parâmetros

`separator`

O delimitador de campo (apenas um caractere de um byte). Padrão é uma vírgula (`,`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl().

`enclosure`

O caractere de delimitador de campo (apenas um caractere de um byte). Padrão é uma aspa dupla (`"`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl().

`escape`

O caractere de escape (no máximo um caractere de um byte). Padrão é uma barra invertida (`\`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl(). Uma string vazia (`""`) desativa o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado. Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados, ou por uma chamada a SplFileObject::setCsvControl().

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

Valor Retornado

Retorna um array indexado contendo os campos lidos, ou `false` em caso de erro.

> **Nota** :
>
> Uma linha em branco em um arquivo CSV será retornada como um array composto por um único campo `null` a menos que esteja usando **`SplFileObject::SKIP_EMPTY | SplFileObject::DROP_NEW_LINE`**, caso em que as linhas em branco são ignoradas.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
7.4.0	O parâmetro `escape` agora também aceita uma string vazia para desativar o mecanismo de escape proprietário.

Examples

Exemplo #1 Exemplo de SplFileObject::fgetcsv()

```php
<?php
$file = new SplFileObject("data.csv");
while (!$file->eof()) {
    var_dump($file->fgetcsv());
}
?>
```

Exemplo #2 Exemplo de `SplFileObject::READ_CSV`

```php
<?php
$file = new SplFileObject("animals.csv");
$file->setFlags(SplFileObject::READ_CSV);
foreach ($file as $row) {
    list($animal, $class, $legs) = $row;
    printf("A %s is a %s with %d legs\n", $animal, $class, $legs);
}
?>
```

Conteúdo de animals.csv

```
crocodile,reptile,4
dolphin,mammal,0
duck,bird,2
koala,mammal,4
salmon,fish,0
```

O exemplo acima produzirá algo semelhante a:

```
A crocodile is a reptile with 4 legs
A dolphin is a mammal with 0 legs
A duck is a bird with 2 legs
A koala is a mammal with 4 legs
A salmon is a fish with 0 legs
```

Veja também

- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV
- SplFileObject::setFlags() - Define as opções para o SplFileObject
- **`SplFileObject::READ_CSV`**
- SplFileObject::current() - Recupera linha atual do arquivo
- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array

SplFileObject::fgets

SplFileObject::fgets

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgets — Obtém linha do arquivo

Descrição

public function **SplFileObject::fgets**(): string

Obtém uma linha do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string contendo a próxima linha do arquivo.

Erros/Exceções

Lança uma RuntimeException se o arquivo não puder ser lido.

Examples

Exemplo #1 Exemplo de SplFileObject::fgets()

Este exemplo simplesmente imprime o conteúdo de `file.txt` linha por linha.

```php
<?php
$file = new SplFileObject("file.txt");
while (!$file->eof()) {
    echo $file->fgets();
```

```
}
?>
```

Veja também

- fgets() - Lê uma linha de um ponteiro de arquivo
- SplFileObject::fgetss() - Obtém linha do arquivo e remove tags HTML
- SplFileObject::fgetc() - Obtém caractere do arquivo
- SplFileObject::current() - Recupera linha atual do arquivo

SplFileObject::fgetss

SplFileObject::fgetss

(PHP 5 >= 5.1.0, PHP 7)

SplFileObject::fgetss — Obtém linha do arquivo e remove tags HTML

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 7.3.0 e foi *REMOVIDA* a partir do PHP 8.0.0. O uso desta função é fortemente desencorajado.

Descrição

public function **SplFileObject::fgetss**(string `$allowable_tags` = ?): string

Idêntico ao SplFileObject::fgets(), exceto que **SplFileObject::fgetss()** tenta remover quaisquer tags HTML e PHP do texto que lê. A função mantém o estado de análise de chamada para chamada, e como tal não é equivalente a chamar strip_tags() no valor de retorno de SplFileObject::fgets().

Parâmetros

`allowable_tags`

Parâmetro opcional para especificar tags que não devem ser removidas.

Valor Retornado

Retorna uma string contendo a próxima linha do arquivo com código HTML e PHP removido, ou `false` em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileObject::fgetss()

```
<?php
$str = <<<EOD
<html><body>
 <p>Bem-vindo! Hoje é o <?php echo(date('jS')); ?> de <?= date('F'); ?>.</p>
</body></html>
Texto fora do bloco HTML.
EOD;
file_put_contents("sample.php", $str);

$file = new SplFileObject("sample.php");
while (!$file->eof()) {
    echo $file->fgetss();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Bem-vindo! Hoje é o  de .

Texto fora do bloco HTML.
```

Veja também

- fgetss() - Ler uma linha de um ponteiro de arquivo e retira as tags HTML
- SplFileObject::fgets() - Obtém linha do arquivo
- SplFileObject::fgetc() - Obtém caractere do arquivo
- SplFileObject::current() - Recupera linha atual do arquivo
- O filtro string.strip_tags

SplFileObject::flock

SplFileObject::flock

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::flock — Bloqueio de arquivo portável

Descrição

public function **SplFileObject::flock**(int `$operation`, int `&$wouldBlock = null`): bool

Bloqueia ou desbloqueia o arquivo da mesma forma portável que flock().

Parâmetros

`operation`

`operation` é uma das seguintes:

- `LOCK_SH` para adquirir um bloqueio compartilhado (leitor).
- `LOCK_EX` para adquirir um bloqueio exclusivo (escritor).
- `LOCK_UN` para liberar um bloqueio (compartilhado ou exclusivo).

Também é possível adicionar `LOCK_NB` como uma máscara de bits para uma das operações acima, se flock() não deve bloquear durante a tentativa de bloqueio.

`wouldBlock`

Definido como `true` se o bloqueio bloqueasse (condição de errno EWOULDBLOCK).

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileObject::flock()

```php
<?php
$file = new SplFileObject("/tmp/lock.txt", "w");
if ($file->flock(LOCK_EX)) { // faz um bloqueio exclusivo
    $file->ftruncate(0);     // trunca o arquivo
    $file->fwrite("Escrevendo algo aqui\n");
    $file->flock(LOCK_UN);   // libera o bloqueio
} else {
    echo "Não foi possível obter o bloqueio!";
}
?>
```

Veja também

- flock() - Travamento consultivo portável de arquivo

SplFileObject::fpassthru

SplFileObject::fpassthru

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fpassthru — Produz todos os dados restantes em um ponteiro de arquivo

Descrição

public function **SplFileObject::fpassthru**(): int

Lê até o EOF no ponteiro de arquivo fornecido a partir da posição atual e escreve os resultados no buffer de saída.

Você pode precisar chamar SplFileObject::rewind() para redefinir o ponteiro de arquivo para o início do arquivo se você já tiver escrito dados para o arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de caracteres lidos de `handle` e passados para a saída.

Examples

Exemplo #1 Exemplo de SplFileObject::fpassthru()

```php
<?php

// Abre o arquivo em modo binário
$file = new SplFileObject("./img/ok.png", "rb");

// Envia os cabeçalhos corretos
header("Content-Type: image/png");
header("Content-Length: " . $file->getSize());

// Exibe a imagem e encerra o script
$file->fpassthru();
exit;

?>
```

Veja também

- fpassthru() - Imprime todos os dados restantes de um ponteiro de arquivo

SplFileObject::fputcsv

SplFileObject::fputcsv

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplFileObject::fputcsv — Escreve um array de campos como uma linha CSV

Descrição

public function **SplFileObject::fputcsv**(
  array `$fields`,
  string `$separator` = ",",
  string `$enclosure` = "\"",
  string `$escape` = "\\",
  string `$eol` = "\n"
): int|false

Escreve o array de `fields` no arquivo como uma linha CSV.

Parâmetros

`fields`

Um array de valores.

`separator`

O delimitador de campo (apenas um caractere de um byte). Padrão é uma vírgula (`,`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl().

`enclosure`

O caractere de delimitador de campo (apenas um caractere de um byte). Padrão é uma aspa dupla (`"`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl().

`escape`

O caractere de escape (no máximo um caractere de um byte). Padrão é uma barra invertida (`\`) ou o valor definido por uma chamada anterior a SplFileObject::setCsvControl(). Uma string vazia (`""`) desativa o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado. Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados, ou por uma chamada a SplFileObject::setCsvControl().

`eol`

O parâmetro opcional `eol` define uma sequência de fim de linha personalizada.

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

> **Nota** :
>
> Se um caractere de `enclosure` estiver contido em um campo, ele será escapado duplicando-o, a menos que seja imediatamente precedido por um `escape`.

Valor Retornado

Retorna o comprimento da string escrita ou **`false`** em caso de falha.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
8.1.0	O parâmetro opcional `eol` foi adicionado.
7.4.0	O parâmetro `escape` agora também aceita uma string vazia para desativar o mecanismo de escape proprietário.

Examples

Exemplo #1 Exemplo de SplFileObject::fputcsv()

```php
<?php

$list = array (
    array('aaa', 'bbb', 'ccc', 'dddd'),
    array('123', '456', '789'),
    array('"aaa"', '"bbb"')
);

$file = new SplFileObject('file.csv', 'w');

foreach ($list as $fields) {
    $file->fputcsv($fields);
}

?>
```

O exemplo acima escreverá o seguinte em `file.csv`:

```
aaa,bbb,ccc,dddd
123,456,789
"""aaa""","""bbb"""
```

Veja também

- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV
- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array

SplFileObject::fread

SplFileObject::fread

(PHP 5 >= 5.5.11, PHP 7, PHP 8)

SplFileObject::fread — Lê do arquivo

Descrição

public function **SplFileObject::fread**(int `$length`): string|false

Lê o número dado de bytes do arquivo.

Parâmetros

`length`

O número de bytes a serem lidos.

Valor Retornado

Retorna a string lida do arquivo ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileObject::fread()

```php
<?php
// Obtém o conteúdo de um arquivo em uma string
$filename = "/usr/local/something.txt";
$file = new SplFileObject($filename, "r");
```

```php
$contents = $file->fread($file->getSize());
?>
```

Notas

> **Nota** :
>
> Observe que **SplFileObject::fread()** lê a partir da posição atual do ponteiro do arquivo. Use SplFileObject::ftell() para encontrar a posição atual do ponteiro e SplFileObject::rewind() (ou SplFileObject::fseek()) para retroceder a posição do ponteiro.

Veja também

- fread() - Leitura de arquivo segura para binário

SplFileObject::fscanf

SplFileObject::fscanf

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fscanf — Analisa a entrada do arquivo de acordo com um formato

Descrição

public function **SplFileObject::fscanf**(string `$format`, mixed `&...$vars`): array|int|null

Lê uma linha do arquivo e a interpreta de acordo com o `format` especificado.

Qualquer espaço em branco na string `format` corresponde a qualquer espaço em branco na linha do arquivo. Isso significa que até mesmo um tab (`\t`) na string de formato pode corresponder a um único caractere de espaço na sequência de entrada.

Parâmetros

`format`

O formato interpretado para `string`, que é descrito na documentação para sprintf() com as seguintes diferenças:

- A função não reconhece a localidade.
- `F`, `g`, `G` e `b` não são suportados.
- `D` significa número decimal.
- `i` significa inteiro com detecção de base.
- `n` representa o número de caracteres processados até o momento.
- `s` pára de ler em qualquer caractere de espaço em branco.
- `*` em vez de `argnum$` suprime a atribuição desta especificação de conversão.

`vars`

Os valores atribuídos opcionais.

Valor Retornado

Se apenas um parâmetro for passado para este método, os valores analisados serão retornados como uma array. Caso contrário, se parâmetros opcionais forem passados, o método retornará o número de valores atribuídos. Os parâmetros opcionais devem ser passados por referência.

Se houver mais substrings esperadas em `format` do que estiverem disponíveis na linha lida do arquivo, `null` será retornado.

Quando parâmetros opcionais são usados e o fim da linha lida do arquivo é alcançado antes que qualquer valor tenha sido interpretado, `-1` é retornado.

Examples

Exemplo #1 Exemplo de SplFileObject::fscanf()

```php
<?php
$file = new SplFileObject("misc.txt");
while ($userinfo = $file->fscanf("%s %s %s")) {
   list ($name, $profession, $countrycode) = $userinfo;
   // Faça algo com $name $profession $countrycode
```

```
}
?>
```

Conteúdo de users.txt

```
javier   argonaut    pe
hiroshi  sculptor    jp
robert   slacker     us
luigi    florist     it
```

Veja também

- fscanf() - Interpreta a leitura de um arquivo de acordo com um formato
- sscanf() - Interpreta a entrada de uma string de acordo com um formato
- printf() - Envia uma string formatada para a saída
- sprintf() - Retona uma string formatada

SplFileObject::fseek

SplFileObject::fseek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fseek — Aponta para uma posição

Descrição

public function **SplFileObject::fseek**(int `$offset`, int `$whence` = **`SEEK_SET`**): int

Aponta para uma posição no arquivo medida em bytes a partir do início do arquivo, obtida ao adicionar o `offset` à posição especificada por `whence`.

Parâmetros

`offset`

O deslocamento. Um valor negativo pode ser usado para mover-se para trás no arquivo o que é útil quando SEEK_END é usado como o valor de `whence`.

`whence`

Os valores de `whence` são:

- **`SEEK_SET`** - Define a posição igual a `offset` bytes.
- **`SEEK_CUR`** - Define a posição para a localização atual mais `offset`.
- **`SEEK_END`** - Define a posição para o final do arquivo mais `offset`.

Se `whence` não for especificado, é assumido como **`SEEK_SET`**.

Valor Retornado

Retorna 0 se a busca for bem-sucedida, -1 caso contrário. Note que buscar além do EOF não é considerado um erro.

Examples

Exemplo #1 Exemplo de SplFileObject::fseek()

```
<?php
$file = new SplFileObject("somefile.txt");

// Lê a primeira linha
$data = $file->fgets();

// Move de volta para o início do arquivo
// Mesmo como $file->rewind();
$file->fseek(0);
?>
```

Veja também

- fseek() - Aponta para uma posição no arquivo

SplFileObject::fstat

SplFileObject::fstat

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fstat — Obtém informações sobre o arquivo

Descrição

public function **SplFileObject::fstat**(): array

Coleta as estatísticas do arquivo. Comporta-se de forma idêntica a fstat().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com as estatísticas do arquivo; o formato do array é descrito detalhadamente na página do manual do stat().

Examples

Exemplo #1 Exemplo de SplFileObject::fstat()

```php
<?php
$file = new SplFileObject("/etc/passwd");
$stat = $file->fstat();

// Imprime somente a parte associada
print_r(array_slice($stat, 13));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [dev] => 771
    [ino] => 488704
    [mode] => 33188
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 1114
    [atime] => 1061067181
    [mtime] => 1056136526
    [ctime] => 1056136526
    [blksize] => 4096
    [blocks] => 8
)
```

Veja também

- fstat() - Lê informações sobre um arquivo usando um ponteiro de arquivo aberto
- stat() - Obtém informações sobre um arquivo

SplFileObject::ftell

SplFileObject::ftell

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::ftell — Retorna a posição atual do arquivo

Descrição

public function **SplFileObject::ftell**(): int|false

Retorna a posição do ponteiro do arquivo que representa o deslocamento atual no fluxo do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a posição do ponteiro do arquivo como um inteiro, ou `false` em caso de erro.

Examples

Exemplo #1 Exemplo de SplFileObject::ftell()

```php
<?php
$file = new SplFileObject("/etc/passwd");

// Lê a primeira linha
$data = $file->fgets();

// Onde nós estamos?
echo $file->ftell();
?>
```

Veja também

- ftell() - Retorna a posição atual do ponteiro de leitura/gravação do arquivo

SplFileObject::ftruncate

SplFileObject::ftruncate

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::ftruncate — Trunca o arquivo para um determinado comprimento

Descrição

public function **SplFileObject::ftruncate**(int `$size`): bool

Trunca o arquivo para `size` bytes.

Parâmetros

`size`

O tamanho para o qual truncar.

> **Nota** :
>
> Se `size` for maior que o arquivo, ele será estendido com bytes nulos.
>
> Se `size` for menor que o arquivo, os dados extras serão perdidos.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileObject::ftruncate()

```php
<?php
// Cria arquivo contendo "Hello World!"
$file = new SplFileObject("/tmp/ftruncate", "w+");
$file->fwrite("Hello World!");

// Trunca para 5 bytes
$file->ftruncate(5);

// Rebobina e lê os dados
$file->rewind();
echo $file->fgets();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Hello
```

Veja também

- ftruncate() - Trunca um arquivo a um tamanho especificado

SplFileObject::fwrite

SplFileObject::fwrite

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fwrite — Escreve no arquivo

Descrição

public function **SplFileObject::fwrite**(string `$data`, ?int `$length = null`): int|false

Escreve o conteúdo do parâmetro `data` no arquivo.

Parâmetros

`data`

A string a ser escrita no arquivo.

`length`

Se o argumento `length` for do tipo int, a escrita será interrompida após `length` bytes terem sido escritos ou quando o final de `data` for alcançado, o que ocorrer primeiro.

Valor Retornado

Retorna o número de bytes escritos, ou `false` em caso de erro.

Registro de Alterações

Versão	Descrição
8.5.0	`length` agora pode ser nulo.
7.4.0	A função agora retorna `false` em vez de zero em caso de falha.

Examples

Exemplo #1 Exemplo de SplFileObject::fwrite()

```php
<?php
$file = new SplFileObject("fwrite.txt", "w");
$written = $file->fwrite("12345");
```

```php
echo "Wrote $written bytes to file";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Wrote 5 bytes to file
```

Veja também

- fwrite() - Escrita binary-safe em arquivos

SplFileObject::getChildren

SplFileObject::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getChildren — Sem objetivo

Descrição

public function **SplFileObject::getChildren**(): null

Um SplFileObject não possui filhos, então este método retorna `null`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `null`.

Veja também

- RecursiveIterator::getChildren() - Retorna um iterador para a entrada atual

SplFileObject::getCsvControl

SplFileObject::getCsvControl

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

SplFileObject::getCsvControl — Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV

Descrição

public function **SplFileObject::getCsvControl**(): array

Obtém o delimitador, o caractere de contorno e o caractere de escape usados para analisar os campos CSV.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array indexado contendo o delimitador, o caractere de contorno e o caractere de escape.

Registro de Alterações

Versão	Descrição
7.4.0	O caractere de escape agora pode ser uma string vazia.
7.0.10	Adicionado o caractere de escape ao array retornado.

Examples

Exemplo #1 Exemplo de SplFileObject::getCsvControl()

```php
<?php
$file = new SplFileObject("data.txt");
print_r($file->getCsvControl());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => ,
    [1] => "
    [2] => \
)
```

Veja também

- SplFileObject::setCsvControl() - Definir o delimitador, o encapsulador e o caractere de escape para CSV
- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array

SplFileObject::getCurrentLine

SplFileObject::getCurrentLine

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileObject::getCurrentLine — Sinônimo de SplFileObject::fgets()

Descrição

Este método é um apelido para: SplFileObject::fgets().

SplFileObject::getFlags

SplFileObject::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getFlags — Obtém as opções para o SplFileObject

Descrição

public function **SplFileObject::getFlags**(): int

Obtém as opções definidas para uma instância de SplFileObject como um int.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um int representando as opções.

Examples

Exemplo #1 Exemplo de SplFileObject::getFlags()

```php
<?php
$file = new SplFileObject(__FILE__, "r");

if ($file->getFlags() & SplFileObject::SKIP_EMPTY) {
    echo "Skipping empty lines\n";
} else {
    echo "Not skipping empty lines\n";
}

$file->setFlags(SplFileObject::SKIP_EMPTY);

if ($file->getFlags() & SplFileObject::SKIP_EMPTY) {
    echo "Skipping empty lines\n";
} else {
    echo "Not skipping empty lines\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Not skipping empty lines
Skipping empty lines
```

Veja também

- SplFileObject::setFlags() - Define as opções para o SplFileObject

SplFileObject::getMaxLineLen

SplFileObject::getMaxLineLen

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getMaxLineLen — Obtém o comprimento máximo da linha

Descrição

public function **SplFileObject::getMaxLineLen**(): int

Obtém o comprimento máximo da linha definido por SplFileObject::setMaxLineLen().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o comprimento máximo da linha se um tiver sido definido com SplFileObject::setMaxLineLen(), o padrão é `0`.

Examples

Exemplo #1 Exemplo de SplFileObject::getMaxLineLen()

```php
<?php
$file = new SplFileObject("file.txt");
var_dump($file->getMaxLineLen());

$file->setMaxLineLen(20);
var_dump($file->getMaxLineLen());
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(0)
int(20)
```

Veja também

- SplFileObject::setMaxLineLen() - Define o comprimento máximo da linha

SplFileObject::hasChildren

SplFileObject::hasChildren

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileObject::hasChildren — SplFileObject não tem filhos

Descrição

public function **SplFileObject::hasChildren**(): false

Um SplFileObject não tem filhos, então este método sempre retorna `false`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `false`

Veja também

- RecursiveIterator::hasChildren() - Retorna se um iterador pode ser criado para a entrada atual

SplFileObject::key

SplFileObject::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::key — Obtém o número da linha

Descrição

public function **SplFileObject::key**(): int

Obtém o número da linha atual.

> **Nota** :
>
> Este número pode não refletir o número real da linha no arquivo se SplFileObject::setMaxLineLen() for usado para ler comprimentos fixos do arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número da linha atual.

Examples

Exemplo #1 Exemplo de SplFileObject::key()

```php
<?php
$file = new SplFileObject("lipsum.txt");
```

```php
foreach ($file as $line) {
    echo $file->key() . ". " . $line;
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
0. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
1. Duis nec sapien felis, ac sodales nisl.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
```

Exemplo #2 Exemplo de SplFileObject::key() com SplFileObject::setMaxLineLen()

```php
<?php
$file = new SplFileObject("lipsum.txt");
$file->setMaxLineLen(20);
foreach ($file as $line) {
    echo $file->key() . ". " . $line . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
0. Lorem ipsum dolor s
1. it amet, consectetu
2. r adipiscing elit.
3.

4. Duis nec sapien fel
5. is, ac sodales nisl
6. .

7. Lorem ipsum dolor s
8. it amet, consectetu
9. r adipiscing elit.
```

Veja também

- SplFileObject::current() - Recupera linha atual do arquivo
- SplFileObject::seek() - Aponta para uma linha especificada
- SplFileObject::next() - Lê próxima linha
- SplFileObject::rewind() - Rebobina o arquivo para a primeira linha
- SplFileObject::valid() - Não está no final do arquivo (EOF)

SplFileObject::next

SplFileObject::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::next — Lê próxima linha

Descrição

public function **SplFileObject::next**(): void

Avança para a próxima linha no arquivo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplFileObject::next()

```php
<?php
// Lê o arquivo linha por linha
$file = new SplFileObject("misc.txt");
while (!$file->eof()) {
    echo $file->current();
    $file->next();
}
?>
```

Veja também

- SplFileObject::current() - Recupera linha atual do arquivo
- SplFileObject::key() - Obtém o número da linha
- SplFileObject::seek() - Aponta para uma linha especificada
- SplFileObject::rewind() - Rebobina o arquivo para a primeira linha
- SplFileObject::valid() - Não está no final do arquivo (EOF)

SplFileObject::rewind

SplFileObject::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::rewind — Rebobina o arquivo para a primeira linha

Descrição

public function **SplFileObject::rewind**(): void

Rebobina o arquivo de volta para a primeira linha.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança um RuntimeException se não puder ser rebobinado.

Examples

Exemplo #1 Exemplo de SplFileObject::rewind()

```php
<?php
$file = new SplFileObject("misc.txt");

// Loop sobre todo o arquivo
foreach ($file as $line) { }

// Rebobina para a primeira linha
$file->rewind();

// Saída da primeira linha
```

```
echo $file->current();
?>
```

Veja também

- SplFileObject::current() - Recupera linha atual do arquivo
- SplFileObject::key() - Obtém o número da linha
- SplFileObject::seek() - Aponta para uma linha especificada
- SplFileObject::next() - Lê próxima linha
- SplFileObject::valid() - Não está no final do arquivo (EOF)

SplFileObject::seek

SplFileObject::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::seek — Aponta para uma linha especificada

Descrição

public function **SplFileObject::seek**(int `$line`): void

Aponta para a linha especificada no arquivo.

Parâmetros

`line`

O número da linha baseada em zero para apontar.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança uma LogicException se `line` for negativa.

Examples

Exemplo #1 Exemplo de SplFileObject::seek()

Este exemplo imprime a terceira linha do script, que é encontrada na posição 2.

```
<?php
$file = new SplFileObject(__FILE__);
$file->seek(2);
echo $file->current();
?>
```

O exemplo acima produzirá algo semelhante a:

```
$file->seek(2);
```

Veja também

- SplFileObject::current() - Recupera linha atual do arquivo
- SplFileObject::key() - Obtém o número da linha
- SplFileObject::next() - Lê próxima linha
- SplFileObject::rewind() - Rebobina o arquivo para a primeira linha
- SplFileObject::valid() - Não está no final do arquivo (EOF)

SplFileObject::setCsvControl

SplFileObject::setCsvControl

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

SplFileObject::setCsvControl — Definir o delimitador, o encapsulador e o caractere de escape para CSV

Descrição

public function **SplFileObject::setCsvControl**(string `$separator = ","`, string `$enclosure = "\""`, string `$escape = "\\"`): void

Define o delimitador, o caractere de contorno e o caractere de escape para a análise dos campos CSV.

Parâmetros

`separator`

O parâmetro `separator` define o delimitador de campo. Precisa ser um caractere de um byte.

`enclosure`

O parâmetro `enclosure` define o caractere que cerca um campo. Precisa ser um caractere de um byte.

`escape`

O parâmetro `escape` define o caractere de escape. Precisa ser um caractere de um byte ou uma string vazia. Uma string vazia (`""`) desabilita o mecanismo de escape proprietário.

> **Aviso**
>
> No fluxo de entrada, o caractere `enclosure` sempre pode ser escapado por duplicação dentro de uma string delimitada, resultando em um único caractere `enclosure` no resultado interpretado. O caractere `escape` funciona de forma diferente: Se uma sequência de caracteres `escape` e `enclosure` aparecerem no fluxo, ambos os caracteres estarão presentes no valor interpretado. Portanto, para os parâmetros padrão, uma linha CSV como `"a""b","c\"d"` terá os campos interpretados como `a"b` e `c\"d`, respectivamente.

> **Aviso**
>
> A partir do PHP 8.4.0, depender do valor padrão de `escape` foi descontinuado. Ele precisa ser fornecido explicitamente tanto por posição quanto pelo uso de argumentos nomeados.

> **Aviso**
>
> Quando o parâmetro `escape` for definido para algo diferente de uma string vazia (`""`), isso pode resultar em um CSV que não esteja conforme a » RFC 4180 ou que não seja capaz de sobreviver a uma viagem de ida e volta através das funções CSV do PHP. O padrão para `escape` é `"\\"`, portanto é recomendado defini-lo explicitamente para a string vazia. O valor padrão será alterado em uma versão futura do PHP, mas não antes do PHP 9.0.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança uma ValueError se `separator` ou `enclosure` não tiver comprimento de um byte.

Lança uma ValueError se `escape` não tiver comprimento de um byte ou se for uma string vazia.

Registro de Alterações

Versão	Descrição
8.4.0	Depender do valor padrão de `escape` agora está descontinuado.
7.4.0	O parâmetro `escape` agora também aceita uma string vazia para desativar o mecanismo de escape proprietário.

Examples

Exemplo #1 Exemplo de SplFileObject::setCsvControl()

```php
<?php
$file = new SplFileObject("data.csv");
$file->setFlags(SplFileObject::READ_CSV);
$file->setCsvControl('|');
foreach ($file as $row) {
    list ($fruit, $quantity) = $row;
    // Faça algo com os valores
}
?>
```

Conteúdo de data.csv

```
<?php
apples|20
bananas|14
cherries|87
?>
```

Veja também

- SplFileObject::getCsvControl() - Obtém o delimitador, o caractere de fechamento e o caractere de escape para CSV
- SplFileObject::fgetcsv() - Obtém linha do arquivo e interpreta como campos CSV
- SplFileObject::fputcsv() - Escreve um array de campos como uma linha CSV
- fputcsv() - Formata a linha como CSV e a escreve em um ponteiro de arquivo
- fgetcsv() - Lê uma linha do ponteiro de arquivos e a interpreta como campos CSV
- str_getcsv() - Analisa uma string CSV e retorna os dados em um array

SplFileObject::setFlags

SplFileObject::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::setFlags — Define as opções para o SplFileObject

Descrição

public function **SplFileObject::setFlags**(int `$flags`): void

Define as opções a serem usadas pelo SplFileObject.

Parâmetros

`flags`

Máscara de bits das opções a serem definidas. Veja Constantes SplFileObject para as opções disponíveis.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de SplFileObject::setFlags()

```php
<?php
$file = new SplFileObject("data.csv");
$file->setFlags(SplFileObject::READ_CSV);
foreach ($file as $fields) {
    var_dump($fields);
}
?>
```

Veja também

- SplFileObject::getFlags() - Obtém as opções para o SplFileObject

SplFileObject::setMaxLineLen

SplFileObject::setMaxLineLen

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::setMaxLineLen — Define o comprimento máximo da linha

Descrição

public function **SplFileObject::setMaxLineLen**(int `$maxLength`): void

Define o comprimento máximo de uma linha a ser lida.

Parâmetros

`maxLength`

O comprimento máximo de uma linha.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança DomainException quando `maxLength` é menor que zero.

Examples

Exemplo #1 Exemplo de SplFileObject::setMaxLineLen()

```php
<?php
$file = new SplFileObject("lipsum.txt");
$file->setMaxLineLen(20);
foreach ($file as $line) {
    echo $line . "\n";
}
?>
```

Conteúdo de lipsum.txt

```
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Duis nec sapien felis, ac sodales nisl.
Nulla vitae magna vitae purus aliquet consequat.
```

O exemplo acima produzirá algo semelhante a:

```
Lorem ipsum dolor s
it amet, consectetu
r adipiscing elit.

Duis nec sapien fel
is, ac sodales nisl
.

Nulla vitae magna v
itae purus aliquet
consequat.
```

Veja também

- SplFileObject::getMaxLineLen() - Obtém o comprimento máximo da linha

SplFileObject::__toString

SplFileObject::__toString

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::__toString — Retorna a linha atual como uma string

Descrição

public function **SplFileObject::__toString**(): string

Este método retornará a linha atual como uma string.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a linha atual como uma string.

Registro de Alterações

Versão	Descrição
8.1.14, 8.2.1	Alterado de um alias de SplFileObject::fgets() para uma implementação de SplFileObject::current() que retorna uma string CSV quando a opção `SplFileObject::READ_CSV` está definida.
7.2.19, 7.3.6	Alterado de um alias de SplFileObject::current() para um alias de SplFileObject::fgets().

SplFileObject::valid

SplFileObject::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::valid — Não está no final do arquivo (EOF)

Descrição

public function **SplFileObject::valid**(): bool

Verifica se o final do arquivo (EOF) foi alcançado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o final do arquivo (EOF) não foi alcançado, `false` caso contrário.

Examples

Exemplo #1 Exemplo de SplFileObject::valid()

```php
<?php
// Loop sobre um arquivo, linha por linha
$file = new SplFileObject("file.txt");
while ($file->valid()) {
    echo $file->fgets();
}
?>
```

Veja também

- SplFileObject::current() - Recupera linha atual do arquivo
- SplFileObject::key() - Obtém o número da linha
- SplFileObject::seek() - Aponta para uma linha especificada
- SplFileObject::next() - Lê próxima linha
- SplFileObject::rewind() - Rebobina o arquivo para a primeira linha

SplTempFileObject

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

Introdução

A classe SplTempFileObject oferece uma interface orientada a objetos para um arquivo temporário.

Resumo da classe

```
class SplTempFileObject extends SplFileObject {
/* Constantes herdadas */
public const int SplFileObject::DROP_NEW_LINE;
public const int SplFileObject::READ_AHEAD;
public const int SplFileObject::SKIP_EMPTY;
public const int SplFileObject::READ_CSV;
/* Métodos */
public function __construct(int $maxMemory = 2 * 1024 * 1024)
/* Métodos herdados */
public function SplFileObject::current(): string|array|false
public function SplFileObject::eof(): bool
public function SplFileObject::fflush(): bool
public function SplFileObject::fgetc(): string|false
public function SplFileObject::fgetcsv(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): array|false
public function SplFileObject::fgets(): string
public function SplFileObject::fgetss(string $allowable_tags = ?): string
public function SplFileObject::flock(int $operation, int &$wouldBlock = null): bool
public function SplFileObject::fpassthru(): int
public function SplFileObject::fputcsv(
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
public function SplFileObject::fread(int $length): string|false
public function SplFileObject::fscanf(string $format, mixed &...$vars): array|int|null
public function SplFileObject::fseek(int $offset, int $whence = SEEK_SET): int
public function SplFileObject::fstat(): array
public function SplFileObject::ftell(): int|false
public function SplFileObject::ftruncate(int $size): bool
public function SplFileObject::fwrite(string $data, ?int $length = null): int|false
public function SplFileObject::getChildren(): null
public function SplFileObject::getCsvControl(): array
public function SplFileObject::getFlags(): int
public function SplFileObject::getMaxLineLen(): int
public function SplFileObject::hasChildren(): false
public function SplFileObject::key(): int
public function SplFileObject::next(): void
public function SplFileObject::rewind(): void
public function SplFileObject::seek(int $line): void
public function SplFileObject::setCsvControl(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): void
public function SplFileObject::setFlags(int $flags): void
public function SplFileObject::setMaxLineLen(int $maxLength): void
public function SplFileObject::__toString(): string
public function SplFileObject::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
```

public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Índice

SplTempFileObject::__construct

SplTempFileObject::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplTempFileObject::__construct — Constrói um novo objeto de arquivo temporário

Descrição

public function **SplTempFileObject::__construct**(int $maxMemory = 2 * 1024 * 1024)

Constrói um novo objeto de arquivo temporário.

Parâmetros

`maxMemory`

A quantidade máxima de memória (em bytes, o padrão é 2 MB) para o arquivo temporário usar. Se o arquivo temporário exceder esse tamanho, ele será movido para um arquivo no diretório temp do sistema.

Se `maxMemory` for negativo, apenas memória será usada. Se `maxMemory` for zero, nenhuma memória será usada.

Erros/Exceções

Lança uma RuntimeException se ocorrer um erro.

Examples

Exemplo #1 Exemplo de SplTempFileObject()

Este exemplo escreve um arquivo temporário na memória que pode ser escrito e lido.

```
<?php
$temp = new SplTempFileObject();
$temp->fwrite("Esta é a primeira linha.\n");
$temp->fwrite("E esta é a segunda.\n");
echo "Escritos " . $temp->ftell() . " bytes no arquivo temporário.\n\n";

// Retrocede e lê o que foi escrito
$temp->rewind();
foreach ($temp as $line) {
    echo $line;
```

```
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Escritos 47 bytes no arquivo temporário.

Esta é a primeira linha.
E esta é a segunda.
```

Veja também

- SplFileObject
- Fluxos de entrada/saída PHP (para `php://temp` e `php://memory`)

Funções da SPL

Índice

class_implements

class_implements

(PHP 5, PHP 7, PHP 8)

class_implements — Retorna as interfaces que são implementadas pela classe ou interface fornecida

Descrição

function **class_implements**(object|string `$object_or_class`, bool `$autoload = true`): array|false

Esta função retorna um array com os nomes das interfaces que o `object_or_class` fornecido e seus pais implementam.

Parâmetros

`object_or_class`

Um objeto (instância de classe) ou uma string (nome da classe ou interface).

`autoload`

Se deve ou não carregar automaticamente se ainda não estiver carregado.

Valor Retornado

Um array em caso de sucesso, ou `false` quando a classe fornecida não existe.

Examples

Exemplo #1 Exemplo de class_implements()

```
<?php

interface foo { }
class bar implements foo {}

print_r(class_implements(new bar));

// O parâmetro também poder ser especificado como uma string
print_r(class_implements('bar'));

spl_autoload_register();

// use o carregamento automático para carregar a classe 'not_loaded'
```

```php
print_r(class_implements('not_loaded', true));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [interface_of_not_loaded] => interface_of_not_loaded
)
```

Notas

> **Nota** :
>
> Para verificar se um objeto implementa uma interface, use `instanceof` ou a função is_a() em vez disso.

Veja também

- class_parents() - Retorna as classes pai da classe fornecida
- get_declared_interfaces() - Retorna um array com todas as interfaces declaradas
- is_a() - Verifica se um objeto é ou herda de uma classe
- `instanceof`

class_parents

class_parents

(PHP 5, PHP 7, PHP 8)

class_parents — Retorna as classes pai da classe fornecida

Descrição

function **class_parents**(object|string `$object_or_class`, bool `$autoload` = **`true`**): array|false

Esta função retorna um array com o nome das classes pai da `object_or_class` fornecida.

Parâmetros

`object_or_class`

Um objeto (instância de classe) ou uma string (nome da classe).

`autoload`

Se deve carregar automaticamente caso ainda não tenha sido carregado.

Valor Retornado

Um array em caso de sucesso, ou **`false`** quando a classe fornecida não existe.

Examples

Exemplo #1 Exemplo de class_parents()

```php
<?php

class foo { }
class bar extends foo {}

print_r(class_parents(new bar));

// O parâmetro também pode ser especificado como uma string
print_r(class_parents('bar'));

spl_autoload_register();

// use autocarregamento para carregar a classe 'not_loaded'
print_r(class_parents('not_loaded', true));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [parent_of_not_loaded] => parent_of_not_loaded
)
```

Notas

> **Nota** :
>
> Para verificar se um objeto estende uma classe, `instanceof` ou a função is_a() deve ser usada em vez disso.

Veja também

- class_implements() - Retorna as interfaces que são implementadas pela classe ou interface fornecida
- is_a() - Verifica se um objeto é ou herda de uma classe
- `instanceof`

class_uses

class_uses

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

class_uses — Retorna os traits usados pela classe fornecida

Descrição

function **class_uses**(object|string `$object_or_class`, bool `$autoload` = **`true`**): array|false

Esta função retorna um array com os nomes dos traits que a `object_or_class` fornecida usa. No entanto, isso não inclui quaisquer traits usados por uma classe pai.

Parâmetros

`object_or_class`

Um objeto (instância de classe) ou uma string (nome da classe).

`autoload`

Se deve autoload caso ainda não tenha sido carregado.

Valor Retornado

Um array em caso de sucesso, ou `false` quando a classe fornecida não existe.

Examples

Exemplo #1 Exemplo de class_uses()

```php
<?php

trait foo { }
class bar {
  use foo;
}

print_r(class_uses(new bar));

print_r(class_uses('bar'));

spl_autoload_register();

// usando autocarregamento para carregar a classe 'not_loaded'
print_r(class_uses('not_loaded', true));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [trait_of_not_loaded] => trait_of_not_loaded
)
```

Veja também

- class_parents() - Retorna as classes pai da classe fornecida
- get_declared_traits() - Retorna um array de todas as Traits declaradas

iterator_apply

iterator_apply

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_apply — Chama uma função para cada elemento em um iterador

Descrição

function **iterator_apply**(Traversable `$iterator`, callable `$callback`, ?array `$args = null`): int

Chama uma função para cada elemento em um iterador.

Parâmetros

`iterator`

O objeto iterador sobre o qual iterar.

`callback`

A função callback para chamar em cada elemento. Esta função recebe apenas os `args` dados, então é nulária por padrão. Se `count($args) === 3`, por exemplo, a função callback é ternária.

> **Nota** :
>
> A função deve retornar `true` para continuar iterando sobre o `iterator`.

`args`

Um array de argumentos; cada elemento de `args` é passado para a função callback `callback` como um argumento separado.

Valor Retornado

Retorna a contagem de iteração.

Examples

Exemplo #1 Exemplo de iterator_apply()

```php
<?php
function print_caps(Iterator $iterator) {
    echo strtoupper($iterator->current()) . "\n";
    return TRUE;
}

$it = new ArrayIterator(array("Apples", "Bananas", "Cherries"));
iterator_apply($it, "print_caps", array($it));
?>
```

O exemplo acima produzirá:

```
APPLES
BANANAS
CHERRIES
```

Veja também

- array_walk() - Aplica uma determinada função em cada elemento de um array

iterator_count

iterator_count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_count — Conta os elementos em um iterador

Descrição

function **iterator_count**(Traversable|array `$iterator`): int

Conta os elementos em um iterador. **iterator_count()** não garante a manutenção da posição atual do `iterator`.

Parâmetros

`iterator`

O iterador sendo contado.

Valor Retornado

O número de elementos em `iterator`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo de `iterator` foi ampliado de Traversable para Traversable|array.

Examples

Exemplo #1 Exemplo de iterator_count()

```php
<?php
$iterator = new ArrayIterator(array('recipe'=>'pancakes', 'egg', 'milk', 'flour'));
var_dump(iterator_count($iterator));
?>
```

O exemplo acima produzirá:

```
int(4)
```

Exemplo #2 iterator_count() modifica a posição

```php
<?php
$iterator = new ArrayIterator(['one', 'two', 'three']);
var_dump($iterator->current());
var_dump(iterator_count($iterator));
var_dump($iterator->current());
?>
```

O exemplo acima produzirá:

```
string(3) "one"
int(3)
NULL
```

Exemplo #3 iterator_count() em laços `foreach`

```php
<?php
$iterator = new ArrayIterator(['one', 'two', 'three']);
foreach ($iterator as $key => $value) {
    echo "$key: $value (", iterator_count($iterator), ")\n";
}?>
```

O exemplo acima produzirá:

```
0: one (3)
```

iterator_to_array

iterator_to_array

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_to_array — Copia o iterador em um array

Descrição

function **iterator_to_array**(Traversable|array `$iterator`, bool `$preserve_keys = true`): array

Copia os elementos de um iterador em um array.

Parâmetros

`iterator`

O iterador que está sendo copiado.

`preserve_keys`

Se as chaves do iterador serão utilizadas como índices.

Se a chave for um array ou object, um erro warning será gerado. Chaves `null` serão convertidas em uma string vazia, chaves float serão truncadas em sua parte int, chaves resource irão gerar um erro warning e serão convertidas para o seu resource ID, e as chaves bool serão convertidas para inteiros.

> **Nota** :
>
> Se este parâmetro não for definido ou for definido como `true`, as chaves duplicadas serão sobrescritas. O último valor com aquela chave estará no array retornado. Defina esse parâmetro como `false` para receber todos os valores em qualquer caso.

Valor Retornado

Um array contendo os elementos do `iterator`.

Registro de Alterações

Versão	Descrição
8.2.0	O tipo de `iterator` foi ampliado de Traversable para Traversable|array.

Examples

Exemplo #1 Exemplo da iterator_to_array()

```php
<?php
$iterator = new ArrayIterator(array('recipe'=>'pancakes', 'egg', 'milk', 'flour'));
var_dump(iterator_to_array($iterator, true));
var_dump(iterator_to_array($iterator, false));
?>
```

O exemplo acima produzirá:

```
array(4) {
  ["recipe"]=>
  string(8) "pancakes"
  [0]=>
  string(3) "egg"
  [1]=>
  string(4) "milk"
  [2]=>
  string(5) "flour"
```

```
}
array(4) {
  [0]=>
  string(8) "pancakes"
  [1]=>
  string(3) "egg"
  [2]=>
  string(4) "milk"
  [3]=>
  string(5) "flour"
}
```

spl_autoload

spl_autoload

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload — Implementação padrão para __autoload()

Descrição

function **spl_autoload**(string `$class`, ?string `$file_extensions` = **`null`**): void

Esta função destina-se a ser usada como uma implementação padrão para __autoload(). Se nada mais for especificado e spl_autoload_register() for chamado sem quaisquer parâmetros, então **spl_autoload()** será usada para qualquer chamada posterior a __autoload().

Parâmetros

`class`

> O nome da classe sendo instanciada. Ao chamar a função, o nome da classe com o namespace é passado ao parâmetro. O parâmetro `class` não conterá a barra invertida inicial de um identificador completamente qualificado.

`file_extensions`

> Por padrão, a função verifica todos os caminhos de inclusão definidos em include_path procurando por nomes de arquivo construídos pelo nome da classe em minúsculas seguidos pelas extensões de arquivo `.inc` e `.php`.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Lança LogicException quando a classe não é encontrada e não há outros carregadores automáticos registrados.

Registro de Alterações

Versão	Descrição
8.0.0	`file_extensions` é anulável agora.

spl_autoload_call

spl_autoload_call

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_call — Tenta todas as funções __autoload() registradas para carregar a classe solicitada

Descrição

function **spl_autoload_call**(string `$class`): void

Esta função pode ser usada para procurar manualmente por uma classe, interface, trait ou enumeração usando as funções __autoload registradas.

Parâmetros

`class`

O nome da classe sendo procurada.

Valor Retornado

Nenhum valor é retornado.

spl_autoload_extensions

spl_autoload_extensions

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_extensions — Registra e retorna as extensões de arquivo padrão para spl_autoload

Descrição

function **spl_autoload_extensions**(?string `$file_extensions = null`): string

Esta função pode modificar e verificar as extensões de arquivo que a função de fallback built-in __autoload() spl_autoload() estará utilizando.

> **Nota** :
>
> Não deve haver um espaço entre as extensões de arquivo definidas.

Parâmetros

`file_extensions`

Se `null`, simplesmente retorna a lista atual de extensões cada uma separada por vírgula. Para modificar a lista de extensões de arquivo, simplesmente invoque a função com a nova lista de extensões de arquivo a serem usadas em uma única string com cada extensão separada por vírgula.

Valor Retornado

Uma lista delimitada por vírgulas de extensões de arquivo padrão para spl_autoload().

Registro de Alterações

Versão	Descrição
8.0.0	`file_extensions` é anulável agora.

Examples

Exemplo #1 Exemplo de spl_autoload_extensions()

```
<?php
spl_autoload_extensions(".php,.inc");
?>
```

spl_autoload_functions

spl_autoload_functions

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_functions — Retorna todas as funções __autoload() registradas

Descrição

function **spl_autoload_functions**(): array

Obtém todas as funções __autoload() registradas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de todas as funções __autoload() registradas. Se nenhuma função estiver registrada, ou a fila de autoload não estiver ativada, então o valor de retorno será um array vazio.

Registro de Alterações

Versão	Descrição
8.0.0	O valor de retorno foi atualizado para sempre ser um array; anteriormente, esta função retornava `false` se a fila de autoload não estivesse ativada.

spl_autoload_register

spl_autoload_register

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_register — Registra a função dada como implementação __autoload()

Descrição

function **spl_autoload_register**(?callable `$callback` = `null`, bool `$throw` = `true`, bool `$prepend` = `false`): bool

Registrar uma função na fila de __autoload fornecida pelo spl. Se a fila ainda não estiver ativada, será ativada.

Se seu código tiver uma função __autoload() existente, então esta função deve ser registrada explicitamente na fila __autoload. Isso ocorre porque **spl_autoload_register()** substituirá efetivamente o cache do mecanismo para a função __autoload() por spl_autoload() ou spl_autoload_call().

Se houver várias funções de carregamento automático, **spl_autoload_register()** permite isso. Ele cria efetivamente uma fila de funções de carregamento automático e executa cada uma delas na ordem em que são definidas. Em contraste, __autoload() só pode ser definido uma vez.

Parâmetros

`callback`

A função de carregamento automático sendo registrada. Se for `null`, então a implementação padrão de spl_autoload() será registrada.

function callback(string `$class`): void

A `class` não conterá a barra invertida inicial de um identificador totalmente qualificado.

`throw`

Este parâmetro especifica se **spl_autoload_register()** deve lançar exceções quando o `callback` não pode ser registrado.

> **Aviso**
>
> Este parâmetro é ignorado a partir do PHP 8.0.0, e um aviso será emitido se definido como `false`. **spl_autoload_register()** agora sempre lançará um TypeError em argumentos inválidos.

`prepend`

Se `true`, **spl_autoload_register()** irá adicionar o carregador automático na fila de carregamento automático antes de anexá-lo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`callback` é anulável agora.

Examples

Exemplo #1 spl_autoload_register() como substituição para um __autoload() function

```php
<?php

// function __autoload($class) {
//     include 'classes/' . $class . '.class.php';
// }

function my_autoloader($class) {
    include 'classes/' . $class . '.class.php';
}

spl_autoload_register('my_autoloader');

// Ou, usando uma função anônima
spl_autoload_register(function ($class) {
    include 'classes/' . $class . '.class.php';
});

?>
```

Exemplo #2 spl_autoload_register() exemplo onde a classe não é carregada

```php
<?php

namespace Foobar;

class Foo {
    static public function test($class) {
        print '[['. $class .']]';
    }
}

spl_autoload_register(__NAMESPACE__ .'\Foo::test');

new InexistentClass;

?>
```

O exemplo acima produzirá algo semelhante a:

```
[[Foobar\InexistentClass]]
Fatal error: Class 'Foobar\InexistentClass' not found in ...
```

Exemplo #3 O identificador será fornecido sem a barra invertida inicial

```php
<?php

spl_autoload_register(static function ($class) {
    var_dump($class);
});

class_exists('RelativeName');
class_exists('RelativeName\\WithNamespace');
class_exists('\\AbsoluteName');
class_exists('\\AbsoluteName\\WithNamespace');
```

```
?>
```

O exemplo acima produzirá:

```
string(12) "RelativeName"
string(26) "RelativeName\WithNamespace"
string(12) "AbsoluteName"
string(26) "AbsoluteName\WithNamespace"
```

Veja também

- __autoload() - Tenta carregar uma classe indefinida

spl_autoload_unregister

spl_autoload_unregister

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_unregister — Desregistra uma função dada como implementação de __autoload()

Descrição

function **spl_autoload_unregister**(callable `$callback`): bool

Remove uma função da fila de carregamento automático. Se a fila estiver ativada e vazia após a remoção da função fornecida, então ela será desativada.

Quando esta função resulta na desativação da fila, qualquer função __autoload que existia anteriormente não será reativada.

Parâmetros

`callback`

A função de carregamento automático sendo desregistrada.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Registro de Alterações

Versão	Descrição
8.5.0	Passa a função spl_autoload_call() como um argumento de chamada de retorno para desregistrar todos os autocarregadores foi descontinuado. Em vez disso, faça uma iteração através do valor de retorno de spl_autoload_functions() e chame **spl_autoload_unregister()** com cada valor.

spl_classes

spl_classes

(PHP 5, PHP 7, PHP 8)

spl_classes — Retorna as classes SPL disponíveis

Descrição

function **spl_classes**(): array

Esta função retorna um array com as classes SPL atualmente disponíveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array contendo as classes SPL atualmente disponíveis.

Examples

Exemplo #1 Exemplo de spl_classes()

```php
<?php

print_r(spl_classes());

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [ArrayObject] => ArrayObject
    [ArrayIterator] => ArrayIterator
    [CachingIterator] => CachingIterator
    [RecursiveCachingIterator] => RecursiveCachingIterator
    [DirectoryIterator] => DirectoryIterator
    [FilterIterator] => FilterIterator
    [LimitIterator] => LimitIterator
    [ParentIterator] => ParentIterator
    [RecursiveDirectoryIterator] => RecursiveDirectoryIterator
    [RecursiveIterator] => RecursiveIterator
    [RecursiveIteratorIterator] => RecursiveIteratorIterator
    [SeekableIterator] => SeekableIterator
    [SimpleXMLIterator] => SimpleXMLIterator
)
```

spl_object_hash

spl_object_hash

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

spl_object_hash — Retorna o ID de hash para o objeto fornecido

Descrição

function **spl_object_hash**(object `$object`): string

Esta função retorna um identificador de hash único para o objeto. Este ID pode ser usado como uma chave de hash para armazenar objetos ou para identificar um objeto, desde que o objeto não seja destruído. Uma vez que o objeto é destruído, seu hash pode ser reutilizado por outros objetos. Este comportamento é semelhante ao spl_object_id().

Parâmetros

`object`

Qualquer objeto.

Valor Retornado

Uma string que é única para cada objeto atualmente existente e é sempre a mesma para cada objeto.

Examples

Exemplo #1 Exemplo de spl_object_hash()

```php
<?php
$id = spl_object_hash($object);
$storage[$id] = $object;
?>
```

Notas

> **Nota** :
>
> Quando um objeto é destruído, seu hash pode ser reutilizado por outros objetos.

> **Nota** :
>
> Os hashes de objetos devem ser comparados para identidade com `===` e `!==`, porque o hash retornado pode ser uma string numérica. Por exemplo: `000000000000000e6000000000000000`.

Veja também

- spl_object_id() - Retorna o identificador inteiro do objeto fornecido

spl_object_id

spl_object_id

(PHP 7 >= 7.2.0, PHP 8)

spl_object_id — Retorna o identificador inteiro do objeto fornecido

Descrição

function **spl_object_id**(object `$object`): int

Esta função retorna um identificador único para o objeto. O ID do objeto é único durante a vida útil do objeto. Depois que o objeto é destruído, seu ID pode ser reutilizado por outros objetos. Esse comportamento é semelhante ao spl_object_hash().

Parâmetros

`object`

Qualquer objeto.

Valor Retornado

Um identificador inteiro que é único para cada objeto atualmente existente e é sempre o mesmo para cada objeto.

Examples

Exemplo #1 Um exemplo de spl_object_id()

```php
<?php
$id = spl_object_id($object);
$storage[$id] = $object;
?>
```

Notas

> **Nota** :
>
> Quando um objeto é destruído, seu ID pode ser reutilizado por outros objetos.